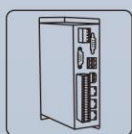
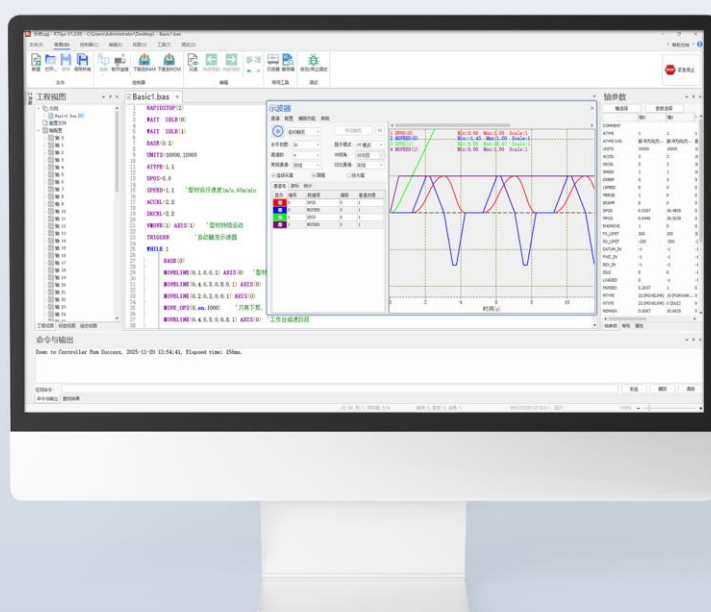
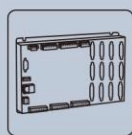


RTBasic 编程手册说明书

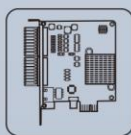
V1.1.5



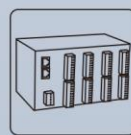
机器视觉运动
控制一体机



运动控制器



运动控制卡



IO扩展模块



人机界面

前言

正运动技术是一家专注于运动控制研究和通用运动控制产品研发的国家级高新技术企业，公司汇聚了来自华为、中兴等公司的优秀人才，在坚持自主创新的同时，积极联合各大高校致力于运动控制基础技术研究，是国内工控领域发展最快的企业之一，也是国内少有、完整掌握运动控制核心、技术和实时工控平台软件技术的企业。

正运动技术的所有产品严格遵循华为的 IPD-CMM 开发流程，具备电信级的稳定性和可靠性，具有良好的软硬件兼容性和扩展性。正运动技术提供强大易用的 RTSys 开发环境，支持 RTBasic、RTPlc 梯形图、RTHmi 组态二次开发，并可混合编程，可实时仿真和在线跟踪 Debug，也支持各种上位机、各种操作系统调用不同编程语言的函数库来开发。

本手册编写的目的是为了更好的服务全国所有的客户，为客户提供更为全面的参考资料，我司致力于对产品的不断优化改进，让客户快速了解正运动的产品，产品手册也会持续更新。

本手册包含正运动 RTSys 软件的使用，指令详解，程序运行逻辑说明，运动缓冲原理，扩展模块，轴的应用，控制器介绍，控制器与其他元件的接线参考示范，可支持多种通信方式。除此之外还提供典型行业的应用例程供编程参考。

相关编程手册

ZMotion PC 函数库编程手册

ZMotion RTPlc 编程手册

ZMotion RTHmi 编程手册

ZMotion RTVision 指令手册

ZMotion RTNC 编程手册

ZMotion 机械手指令说明手册

RTSys 使用手册

以上资料和硬件手册均可从正运动官方网站下载，网址：www.zmotion.com.cn

安全提示

1. 注意事项

控制器集成度高，设计尺寸小巧轻便，易于安装，用户可以有效地利用空间。可以将控制器安装在面板或标准导轨上，并且可以选择水平或垂直安装方式。

作为安装布置系统中各种设备的基本规则，将控制器等低压逻辑型设备与热辐射、高压和电噪声隔离开，远离粉尘、腐蚀性气体、水、油、化学品等场所。

在面板上配置控制器的布局时，由于控制器长时间运行会产生发热现象，应考虑将控制器布置在较凉爽区域，少暴露在高温环境中会延长电子设备的使用寿命，温度过高的环境可能会导致控制器无法正常使用。

安装时还要考虑面板中设备的布线，避免将低压信号线和通信电缆铺设在具有交流动力线和高能量快速开关直流线的槽中。

四周留出足够的空隙以便控制器冷却和接线，控制器可通过自然对流冷却、为保证适当冷却，在设备上方和下方必须留出一定的空隙。

2. 警告

控制器是弱电设备，需要将控制器安装在外壳、控制柜或电控室内等不易触碰的地方，避免非操作人员接触。

机械运行时为了您的人身安全请勿靠近。

请勿对本产品进行分解、修理或改装。

在外部采用控制电路构成紧急停止电路、联锁电路、限制电路等于安全保护相关的电路。

安装或拆卸控制器时应先将控制系统的电源全部断开，避免发生触电或意外设备操作导致不必要的损失。

不遵守这些以上要求可能会导致人员重伤和财产损失，正运动技术公司不承担相应风险与责任。

3. 接线要求

使用螺丝将控制器固定，防止产品跌落或遭受异常冲击导致使用故障。

为保证通讯稳定，控制器通讯电缆应选用带屏蔽层的高性能电缆。

采用 24V 直流电源给控制器供电，另 IO 口需要单独供电，和控制器电源分开。

控制器网络的各个部件为了安全起见，确保控制器和相关设备的所有公共端和接地连接在同一个点接地，该点应该直接连接到系统的大地接地。确定接地点时，应考虑安全接地要求和保护性中断装置的正常运行。

完成所有接线工作后再给控制电路通电，请勿在带电的情况下操作。

所有线路连接应尽可能短，能减少干扰，保证通信质量。

版权说明

本手册版权归深圳市正运动技术有限公司所有，未经正运动公司书面许可，任何人不得翻印、翻译和抄袭本手册中的任何内容。本手册中的信息资料仅供参考。由于改进设计和功能等原因，正运动技术有限公司保留对本资料的最终解释权！内容如有更改，恕不另行通知！如需新版资料，请联系我司相关人员。



调试机器要注意安全！请务必在机器中设计有效的硬件或者机械安全保护装置，并在软件中加入出错处理程序，否则所造成的损失，正运动公司没有义务或责任对此负责。

更新记录

RTBasic 编程手册			
更新日期	版本号	版本（更改）说明	更改人
2025/5/15	V1.1.2	1.新增指令：PORT_485DELAY、SLOT_PERIOD、FLASH_WRITE2、U_WRITE2、ZSMOOTH_MODE、ZSMOOTH_START、ZSMOOTH_END、机械手指令及说明 2.修改回零示意图及指令描述 3.CORNER_MODE 新增 bit4 的语法及说明 4.MTYPE 新增值和指令类型 5.MODBUS，修改 4X 空间示意图并举例 6.错误码表更新	xcx
2025/7/10	V1.1.3	1.新增指令：DRIVE_TORQUEMIN、DISK_CONFIG、REGIST_DELAY、MOVESP 2.AXIS_ADDRESS 增加多槽位描述 3.HW_PSWITCH 新增语法及模式说明 4.MOVESYNC、TIME_TICKUS 适用控制器范围更新 5.轴参数语法补充 6.错误码表更新	xcx
2025/11/20	V1.1.4	1.新增指令：ENCODER_ID、LOCALIO_OFFSET、LOCALAIO_OFFSET、ZMIO_OFFSET、ZMAIO_OFFSET 2.指令语法、描述、例子补充修改 3.错误码表更新	xcx
2026/1/30	V1.1.5	1.新增指令：JERK、ZINDEX_MARK 2.指令新增模式/参数：CAMBOX、ZCUSTOM、ADDAX、AXIS_ZSET、AXISSTATUS、CORNER_MODE、NODE_PDOBUFF 3.指令语法、描述、例子补充修改、优化 4.错误码表更新	xcx

目录

第一章 运动控制产品简介	1
1.1 运动控制产品概述	1
1.2 运动控制产品优点	1
1.3 控制器主要功能描述	2
1.4 控制器应用场景	2
1.5 控制器接口	3
1.6 控制器的使用	4
第二章 RTSys 软件编程	7
2.1 编程软件简介	7
2.2 新建工程	7
2.3 在线命令与输出	11
2.4 示波器的使用	14
示波器面板介绍	14
示波器配置	16
示波器数据导入导出	20
示波器采样方法	20
示波器使用注意事项	20
示波器使用例程	21
2.5 程序调试	25
进入程序调试	25
任务与监控窗口	25
调试工具栏的使用	26
断点调试	27
2.6 视图窗口	28
第三章 Basic 编程基础	29
3.1 编程基础知识	29
程序	29
数据相关	32
3.2 RTSys 的三种编程方式	36
混合编程	36
PLC 与 BASIC 互相调用	36
3.3 寄存器	38
TABLE	38
FLASH	39
VR	40
MODBUS	41
3.4 多任务编程	43
多任务概念	43
多任务状态查看	45
多任务启动与停止	46
任务暂停与恢复	48
Basic 和 PLC 任务相互调用	49
多任务示例	50
3.5 三种中断类型	52
掉电中断	53
外部中断	53
定时器中断	53
3.6 运动缓冲	54
运动缓冲概念	54
运动缓冲区	54

运动缓冲区堵塞	56
运动缓冲中输出	58
第四章 通讯方式	60
4.1 串口通讯	60
串口类型	60
串口连接方法	62
4.2 网口通讯	63
4.3 CAN 总线通讯	66
CAN 接线	66
4.4 U 盘接口	67
4.5 EtherCAT 总线通讯	68
EtherCAT 总线初始化	68
EtherCAT 总线与驱动器通讯	71
EtherCAT 总线连接扩展模块	72
4.6 RTEX 总线通讯	73
第五章 运动控制功能	77
5.1 常见运动模式	77
单轴点动	77
电子凸轮	79
电子齿轮	81
手轮	81
5.2 插补运动	83
插补概念	83
连续插补	86
5.3 前瞻预处理	86
5.4 原点回零	88
5.5 限位相关指令	92
5.6 位置锁存	94
5.7 硬件比较输出	95
5.8 精准输出	96
5.9 振镜控制系统	97
振镜说明	97
振镜应用流程	100
5.10 机械手	105
机械手相关概念	105
正解运动与逆解运动	107
机械手支持的功能	107
机械手应用举例	108
5.11 G 代码	112
第六章 轴相关说明	113
6.1 轴的概念	113
6.2 轴号说明	113
6.3 轴状态	114
6.4 轴速度	115
速度曲线	115
SP 速度	119
6.5 轴映射	120
6.6 轴类型	121
第七章 运动指令	125
7.1 单轴运动指令	125
ADDAX -- 运动叠加	125

CANCEL -- 单轴停止/轴组停止	131
DATUM -- 回零	135
DATUM_OFFSET -- 原点位置偏移	140
VMOVE -- 持续运动	141
FORWARD -- 正向运动	142
REVERSE -- 负向运动	142
MOVEMODIFY -- 修改运动位置	143
7.2 多轴运动指令	145
RAPIDSTOP -- 全部轴停止	145
MOVE -- 直线运动	147
MOVEABS -- 直线运动-绝对	149
MOVEMODIFY2 -- 运动新位置	150
MOVECIRC -- 圆心画弧	152
MOVECIRCABS -- 圆心画弧-绝对	154
MOVECIRC2 -- 三点画弧	155
MOVECIRC2ABS -- 三点画弧-绝对	156
MHELICAL -- 圆心螺旋	157
MHELICALABS -- 圆心螺旋-绝对	159
MHELICAL2 -- 三点螺旋	161
MHELICAL2ABS -- 三点螺旋-绝对	162
MECLIPSEABS -- 椭圆-绝对	164
MECLIPSE -- 椭圆	166
MSPHERICAL -- 空间圆弧	168
MSPHERICALABS -- 空间圆弧-绝对	172
MOVESPIRAL -- 渐开线圆弧	175
MOVESPLINE / MOVESPLINEABS -- 样条插补	177
MOVE_TURNABS -- 旋转台插补	179
MCIRC_TURNABS -- 旋转台圆弧插补	180
MOVESMOOTH -- 倒圆角	182
MOVESP -- SP 运动	183
*SP -- 运动单独速度	184
MOVESCAN -- 振镜运动	186
MPULSCAN -- 振镜运动 2	188
7.3 特殊运动指令	189
MOVE_PAUSE -- 运动暂停	189
MOVE_RESUME -- 运动恢复	191
MOVE_PT -- 单位时间距离	191
MOVE_PTABS -- 单位时间距离-绝对	195
MOVE_PVT -- 单位时间距离（带速度规划）	198
MOVE_PVTABS -- 单位时间距离-绝对（带速度规划）	201
MOVE_PVTPP -- 单位时间距离	203
MOVE_PVTPPABS -- 单位时间距离-绝对	205
MOVE_PTP -- 点到点运动	207
MOVE_PTPABS -- 点到点运动-绝对	210
MOVE_OP -- 缓冲输出	213
MOVE_OP2 -- 缓冲输出 2	217
MOVE_TABLE -- 缓冲 Table	218
MOVE_PARA -- 缓冲参数	219
MOVE_PWM -- 缓冲 PWM	222
MOVE_SYNMOVE -- 缓冲触发其他轴	223
MOVE_ASYNMOVE -- 缓冲触发其他轴 2	224

MOVE_TASK -- 缓冲开启任务	225
MOVE_AOUT -- 缓冲模拟量输出	226
MOVE_DELAY -- 缓冲延时	227
MOVE_WAIT -- 缓冲等待	227
MOVE_CANCEL -- 缓冲停止	228
MOVE_HWPSWITCH2 -- 缓冲硬件比较输出	229
MOVE_HWTIMER -- 缓冲硬件定时	230
MOVE_ADDAX -- 运动叠加	231
MOVELIMIT -- 速度限制	233
7.4 同步运动指令	234
CONNECT -- 同步运动	234
CONNPATH -- 同步运动 2	235
CAM -- 凸轮表运动	236
CAMBOX -- 跟随凸轮表运动	240
MOVELINK -- 自动凸轮	243
MOVESLINK -- 自动凸轮 2	249
MOVELINK_MODIFY -- 同步距离修改	252
MOVESYNC -- 同步运动	256
FLEXLINK -- 激励运动	262
7.5 运动设置指令	265
CLUTCH_RATE -- 连接速度	265
ENCODER_RATIO -- 编码器齿轮比	267
STEP_RATIO -- 电机齿轮比	267
BACKLASH -- 反向间隙补偿	268
PITCHSET -- 螺距补偿	269
PITCH2SET -- 二维螺距补偿	274
PITCH_DIST -- 螺距补偿距离	275
PITCH2_DIST -- 二维螺距补偿距离	275
7.6 机械手指令	276
CONNFRAME -- 机械手逆解	276
CONNREFRAME -- 机械手正解	278
FRAME -- 机械手类型	279
FRAME_STATUS -- 机械手轴状态	279
FRAME_TRANS2 -- 正逆解坐标转换	279
FRAME_ROTATE -- 工件坐标系变换	281
FRAME_ROTATE2 -- 坐标系变换计算	283
WORLD_DPOS -- 世界坐标系	285
MOVER_L/MOVER_LABS -- 关节轴直线插补	286
MOVER_C/MOVER_CABS -- 关节轴平面圆弧	287
MOVER_C3/MOVER_C3ABS -- 关节轴空间圆弧	287
FRAME_CAL -- 参数矫正	288
MOVER2_J/MOVER2_JABS -- 机械手点位运动	289
MOVER2_P/MOVER2_PABS -- 机械手点位运动	290
MOVER2_L/MOVER2_LABS -- 机械手直线运动	290
MOVER2_ARCHP/MOVER2_ARCHPABS -- 机械手点位拱形	293
MOVER2_ARCHPL/MOVER2_ARCHPLABS -- 直线拱形	294
MOVER2_ARCHP3/MOVER2_ARCHP3ABS -- 三维拱形	295
MOVER2_ARC/MOVER2_ARCABS -- 机械手圆弧插补	297
MOVER2_LIMIT -- 选择关节轴速度和加速度约束类型	297
ZSMOOTH -- 机械手运动的平滑过渡系数	298
MOVER2_ZSMOFFSET -- 机械手连续插补平滑的偏移时间	299

FORCE_SPEED -- 机械手运动的速度设置.....	300
MOVER2 点位运动和轨迹运动的速度、加速度设置	301
第八章 程序结构与流程指令	304
8.1 程序符号	304
' -- 注释.....	304
_ -- 换行.....	304
: -- 标号	304
8.2 数据定义指令	304
CONST -- 常量定义.....	304
DIM -- 变量定义	305
LOCAL -- 局部定义.....	306
GLOBAL -- 全局定义.....	306
8.3 数组操作指令	307
DMINS -- 数组链表插入.....	307
DMADD -- 数组批量增加.....	307
DMDEL -- 数组链表删除.....	308
DMCPY -- 数组拷贝.....	308
DMSET -- 数组赋值	309
DMCMP -- 数组比较.....	309
DMSEARCH -- 数组搜索.....	310
SIZEOFARRAY -- 获取数组空间	311
8.4 自定义子函数指令	312
SUB -- 自定义子函数 SUB	312
SUB_PARA -- SUB 传递参数	314
SUB_IFPARA -- SUB 传参判断.....	314
GOSUB/CALL -- SUB 调用	314
GSUB -- 自定义子函数-G 代码格式.....	315
GSUB_PARA -- GSUB 传递参数	316
GSUB_IFPARA -- 判断 GSUB 是否传入参数.....	316
END SUB -- 自定义函数结束.....	317
RETURN -- 函数返回值.....	317
XSUB --自定义 XSUB 子函数	317
RSUB -- 自定义 RSUB 子函数.....	318
8.5 结构体定义指令	318
STRUCTURE -- 结构体定义	318
UNION -- 共用体定义.....	320
8.6 跳转指令	322
GOTO -- 强制跳转.....	322
ON GOSUB -- 条件跳转	322
ON GOTO -- 条件跳转 2.....	322
8.7 条件判断指令	323
IF -- 条件判断结构	323
THEN -- 条件判断结构	324
ENDIF -- 条件判断结构.....	324
ELSEIF -- 条件判断结构.....	324
8.8 循环指令	324
FOR -- for 循环结构	324
TO -- for 循环结构.....	325
STEP -- for 循环结构.....	325
NEXT -- for 循环结构	325
WHILE -- while 循环结构.....	325

WEND -- while 循环结构	326
EXIT -- 退出循环	326
REPEAT -- 条件循环	326
UNTIL -- 条件结构	326
8.9 等待执行指令	327
DELAY -- 延时	327
WAIT UNTIL -- 等待条件满足	327
WAIT IDLE -- 等待轴停止	327
WAIT LOADED -- 等待轴缓冲空	328
8.10 ZINDEX 指针指令	329
ZINDEX_LABEL -- 建立指针索引	329
ZINDEX_CALL -- 访问 SUB 函数	330
ZINDEX_ARRAY -- 访问数组	330
ZINDEX_VAR -- 访问变量	330
ZINDEX_STRUCT -- 访问结构体	331
ZINDEX_MARK -- 指针标号设置	332
第九章 任务相关指令	334
9.1 任务启停指令	334
RUN -- 启动文件任务	334
RUNTASK -- 启动 SUB 任务	334
END -- 结束	335
STOP -- 停止文件任务	335
STOPTASK -- 停止 SUB 任务	335
HALT -- 停止全部任务	336
PAUSE -- 暂停全部任务	336
PAUSETASK -- 暂停指定任务	336
RESUMETASK -- 恢复指定任务	337
9.2 三次文件任务指令	337
FILE3_RUN -- 执行 FILE3 任务	337
FILE3_ONRUN -- FILE3 回调函数	337
FILE3_GOTO -- FILE3 强制跳转	338
FILE3_LINE -- FILE3 行号	338
FILE3_LINES -- 文件总行数	338
FILE3_CHANNEL -- 当前 3 次文件号/通道	339
FILE3_BUFSIZE -- 设置加载缓冲区大小	339
FILE3_SIZE -- 文件大小	339
FILE3_DATETIME -- 文件最后修改日期	340
FILE3_BUFLOAD -- 加载文件	341
FILE3_BUFRUN -- 运行加载文件	341
FILE3_BUF_CLOSE -- 关闭文件	341
FILE3_BUF_SAVE -- 保存/另存文件	342
FILE3_BUF_READ -- 读取行	342
FILE3_BUF_LAYER -- 读取行层次	342
FILE3_BUF_ELEMENT -- 读取行元素总个数	343
FILE3_BUF_ELEMENT_NAME -- 读取行元素名	343
FILE3_BUF_ELEMENT_PARAM -- 读取行元素的参数	343
FILE3_BUF_ELEMENT_PARAM2 -- 读取元素的参数	344
FILE3_BUF_ELEMENT_ALL -- 读取元素的所有参数	344
FILE3_BUF_ELEMENT_ALL2 -- 读取元素的所有参数	345
FILE3_BUF_WRITE -- 写入行	345
FILE3_BUF_INSERT -- 插入行	345

FILE3_BUFDELN -- 删除选中内容	346
FILE3_BUFNOTE -- 注释选中内容	346
FILE3_BUFCOPYN -- 复制 N 行	346
FILE3_BUF CUTN -- 剪切 N 行	347
FILE3_BUF PASTE -- 粘贴到指定位置	347
FILE3_BUF UNDO -- 撤销上一步操作	347
FILE3_BUF REDO -- 恢复上一步操作	347
FILE3_BUF SEEK -- 查找	348
FILE3_BUF REPLACEALL -- 全部替换	348
9.3 任务参数指令	349
BASE_MOVE -- 指定主轴	349
PROC_STATUS -- 任务状态	349
PROC -- 任务编号	350
PROCNUMBER -- 当前任务编号	350
PROC_LINE -- 任务行号	350
PROC_PROGRESS -- 任务指令进度	351
PROC_PRIORITY -- 任务优先级	351
ERROR_LINE -- 任务错误行号	351
RUN_ERROR -- 任务错误码	352
TICKS -- 任务计数周期	352
TIME_TICKUS -- 任务计时	352
第十章 运算符及数学函数指令	353
10.1 算术运算指令	353
+ -- 加法运算	353
- -- 减法运算	353
* -- 乘法运算	354
/ -- 除法运算	354
\ -- 整除运算	354
<< -- 左移位	355
>> -- 右移位	355
MOD -- 求余数	356
ABS -- 绝对值	356
10.2 比较运算指令	357
= -- 比较/赋值运算	357
<> -- 不等于	357
> -- 大于	358
>= -- 大于等于	358
< -- 小于	358
<= -- 小于等于	359
10.3 逻辑运算指令	359
AND -- 按位与	359
OR -- 按位或	360
NOT -- 按位非	360
XOR -- 按位异或	361
EQV -- 按位同或	361
10.4 三角函数指令	362
SIN -- 三角函数正弦	362
ASIN -- 三角函数反正弦	362
COS -- 三角函数余弦	362
ACOS -- 三角函数反余弦	363
TAN -- 三角函数正切	363

ATAN -- 三角函数反正切	363
ATAN2 -- 三角函数反正切 2	363
10.5 指数运算指令	364
EXP -- 指数	364
SQR -- 平方根	364
LN -- 自然对数	364
LOG -- 对数底为 10	364
10.6 数据操作指令	365
SET_BIT -- 按位设置	365
CLEAR_BIT -- 按位置 0	365
READ_BIT -- 按位读取	366
READ_BIT2 -- 按位读取 2	367
FRAC -- 返回小数	367
INT -- 返回整数	367
SGN -- 返回符号	367
IEEE_IN -- 组合浮点数	368
IEEE_OUT -- 提取单字节	368
\$ -- 16 进制	369
10.7 字符串操作指令	369
CHR -- ASCII 码打印	369
HEX -- 16 进制打印	369
STRLEN -- 返回字符串长度	370
TOSTR -- 格式化输出	370
STRCOMP -- 字符串比较	371
STRFIND -- 字符串搜索	371
STRCONV -- 编码转换	371
VAL -- 字符转数值	372
10.8 常数指令	372
PI -- 圆周率	372
TRUE -- 真值	372
FALSE -- 假值	372
ON -- 开启	373
OFF -- 关闭	373
10.9 高级运算指令	373
CRC16 -- CRC 检验计算	373
DTSMOOTH -- table 平滑	373
B_SPLINE -- B 样条平滑	374
TURN_POSMAKE -- 旋转坐标计算	375
ZCUSTOM -- 运动参数计算	375
ZMATH64 -- 64 位计算	380
MODBUS_DOUBLE -- 读取 MODBUS	381
第十一章 轴参数与轴状态指令	382
11.1 轴选择	382
BASE -- 轴选择/轴组选择	382
AXIS -- 临时轴选择	383
11.2 基本参数指令	383
UNITS -- 脉冲当量	383
ATYPE -- 轴类型	384
AXIS_ADDRESS -- 轴地址设置	387
AXIS_ENABLE -- 单轴使能	390
11.3 速度参数指令	390

SPEED -- 运动速度	390
ACCEL -- 加速度	391
DECEL -- 减速度	393
CREEP -- 爬行速度	394
LSPEED -- 起始速度	395
FORCE_SPEED -- SP 速度	396
STARTMOVE_SPEED -- SP 运动开始速度	397
ENDMOVE_SPEED -- SP 运动结束速度	398
FASTDEC -- 快减减速度	400
MSPEED -- 实际反馈速度	401
SPEED_RATIO -- 速度比例	401
SRAMP -- 加减速曲线	402
VP_MODE -- 加减速曲线	403
VP_SPEED -- 当前运动速度	406
INTERP_FACTOR -- 插补速度计算	407
CORNER_ACCEL -- 拐弯加速度	409
JERK -- 加加速度	409
11.4 轴状态查看指令	411
MTYPE -- 当前运动类型	411
NTYPE -- 下条运动类型	412
AXISSTATUS -- 轴状态	413
IDLE -- 运动状态	414
ADDAX_AXIS -- 叠加轴号	414
AXIS_STOPREASON -- 轴停止原因	415
LINK_AXIS -- 连接轴号	415
11.5 运动前瞻指令	415
CORNER_MODE -- 拐角设置	415
DECEL_ANGLE -- 拐角减速开始	420
STOP_ANGLE -- 拐角减速结束	421
FULL_SP_RADIUS -- 限速半径	422
SPLIMIT_RADIUS -- 限速值	423
ZSMOOTH -- 倒角半径	423
MERGE -- 连续插补	424
ZSMOOTH_MODE -- 平滑速度曲线模式	425
ZSMOOTH_START -- 开始平滑模式	425
ZSMOOTH_END -- 退出平滑模式	426
11.6 运动缓冲指令	426
LOADED -- 缓冲空	426
MOVES_BUFFERED -- 当前缓冲数	426
REMAIN_BUFFER -- 剩余缓冲数	426
MOVE_MARK -- 运动标号	427
MOVE_CURMARK -- 当前运动标号	427
LIMIT_BUFFERED -- 运动缓冲限制	428
11.7 位置相关指令	428
DPOS -- 轴指令位置	428
MPOS -- 编码器反馈位置	429
DEFPOS -- 位置偏移	429
OFFPOS -- 偏移位置	430
ENDMOVE -- 当前运动目标位置	430
VECTOR_MOVED -- 当前运动距离	431
REMAIN -- 当前运动剩余距离	432

VECTOR_BUFFERED --缓冲剩余距离	433
VECTOR_BUFFERED2 --目标矢量位置.....	433
ENDMOVE_BUFFER -- 缓冲最终位置.....	434
11.8 原点回零指令	435
DATUM_IN -- 映射原点输入	435
HOMEWAIT -- 回零反找延时	435
11.9 JOG 运动指令	437
FAST_JOG -- 映射点动输入	437
FWD_JOG -- 映射正向 JOG 输入	438
REV_JOG -- 映射负向 JOG 输入	439
JOGSPEED -- JOG 速度	439
FHOLD_IN -- 映射保持输入	440
FHSPEED -- 保持速度	441
11.10 编码器相关指令	442
ENCODER -- 编码器原始值	442
ENCODER_STATUS -- 编码器状态	442
ENCODER_FILTER -- 编码器滤波	442
PP_STEP -- 编码器内部比例	443
ENCODER_BITS -- 编码器绝对值设置	443
ENCODER_ID -- 存储数据字典的编号	444
DRIVE_POSMIN -- 编码器传输原始最小值	444
DRIVE_POSMAX -- 编码器传输原始最大值	444
11.11 锁存相关指令	445
REGIST -- 锁存	445
REG_INPUTS -- 锁存输入映射	449
REGIST_DELAY -- 锁存延时	450
MARK -- 锁存触发	450
MARKB -- 锁存 2 触发	450
MARKC -- 锁存 3 触发	451
MARKD -- 锁存 4 触发	451
OPEN_WIN -- 锁存开始坐标范围	451
CLOSE_WIN -- 锁存结束坐标范围	451
REG_POS -- 锁存位置	452
REG_POSB -- 锁存 2 位置	452
REG_POSC -- 锁存 3 位置	452
REG_POSD -- 锁存 4 位置	453
11.12 限位参数指令	453
FS_LIMIT -- 正向软限位设置	453
RS_LIMIT -- 负向软限位设置	454
FWD_IN -- 映射正限位输入	454
REV_IN -- 映射负限位输入	455
ALM_IN -- 映射报警输入	455
AXISEMG_IN -- 急停信号输入	456
11.13 到位参数指令	457
IN_POS -- 到位标志	457
AXISINP_IN -- 到位信号输入	457
IN_POS_DIST -- 到位距离	458
IN_POS_SPEED -- 到位速度	459
11.14 限幅参数指令	460
REP_OPTION -- 坐标循环模式	460
REP_DIST -- 坐标循环位置	461

FE -- 当前随动误差	461
FE_LIMIT -- 最大随动误差设置	462
FE_RANGE -- 报警时随动误差	462
11.15 高级设置指令	462
INVERT_STEP -- 脉冲模式设置	462
MAX_SPEED -- 脉冲频率限制	464
AXIS_ZSET -- 精准 op 设置	464
AXIS_MODE -- connect 运动保持	466
MOVEOP_DELAY -- 缓冲输出延时	468
MOVEOP_ADIST -- 提前关胶	468
MOVEOP_AHEADMS -- 提前关胶时间	470
DAC -- 总线轴模拟量控制	471
ERRORMASK -- 错误时操作	474
ZSCAN_CORRECT -- 振镜矫正	474
11.16 PID 相关指令	475
D_GAIN -- 微分增益	475
I_GAIN -- 积分增益	475
OV_GAIN -- 速度增益	475
P_GAIN -- 比例增益	476
VFF_GAIN -- 前馈增益	476
AFF_GAIN -- 加速度前馈增益	476
SERVO -- 闭环开关	477
第十二章 输入输出相关指令	480
12.1 输入相关指令	480
IN -- 输入口	480
AIN -- 模拟量输入	480
ZSIMU_IN -- 仿真 IN 输入	481
ZSIMU_AIN -- 仿真模拟量输入	481
ZSIMU_ENCODER -- 仿真编码器输入	481
INVERT_IN -- 反转输入	481
IN_SCAN -- 扫描输入变化	482
IN_EVENT -- 读取输入变化	482
SCAN_EVENT -- 检测变化	483
IN_BUFF -- 读取输入缓冲	483
INFILTER -- 输入口滤波	484
IN_MSFILTER -- 设置输入口滤波	484
LOCALIO_OFFSET -- 本地 IO 偏移	484
12.2 输出相关指令	484
OP -- 输出口	484
AOUT -- 模拟量输出	485
READ_OP -- 读取输出口	486
EXIO_DIR -- 配置 EXIO 接口	486
LOCALAIO_OFFSET -- 本地 AIO 偏移	487
12.3 位置比较输出指令	487
PSWITCH -- 软件位置比较输出	487
HW_PSWITCH -- 硬件位置比较输出	489
HW_PSWITCH2 -- 总线硬件位置比较输出	496
HW_TIMER -- 硬件定时	513
HW_MINTIME -- HW 最小时间间隔	518
HW_PS2AXISNUM -- 设置 PS2 轴号	518
HW_PS2COUNTS -- PS2 比较的点数	520

12.4 PWM 控制指令	521
PWM_FREQ -- PWM 频率	521
PWM_DUTY -- PWM 占空比	521
12.5 蜂鸣器控制指令	522
SPEAKOUT -- 蜂鸣器控制	522
第十三章 通讯相关指令	523
13.1 串口通讯指令	523
SETCOM -- 串口配置	523
ADDRESS -- 控制器站号	525
COM_UNUSED -- 指定串口	525
13.2 CAN 通讯指令	526
CAN -- CAN 通讯	526
CANIO_ADDRESS -- CAN 通讯设置	528
CANIO_ENABLE -- CAN 使能	529
CANIO_STATUS -- CAN 扩展板状态	530
CANIO_INFO -- CAN 扩展板信息	530
13.3 自定义通讯指令	532
GET# -- 读取字符	532
OPEN# -- 打开自定义网口通讯	533
CLOSE# -- 关闭自定义网口通讯	534
PRINT# -- 输出字符串	534
PUTCHAR# -- 输出字符	535
PORT_TARGET -- IP 和端口号配置	537
13.4 打印输出指令	537
PRINT -- 打印信息	537
ERRSWITCH -- 信息输出设置	538
TRACE -- 打印信息 2	539
WARN -- 警告信息	539
ERROR -- 错误信息	540
13.5 通道参数指令	540
PORT -- 通道编号	540
PORT_STATUS -- 通道状态	542
PORT_MODE -- 通道模式	542
FILE_PORT -- 当前通道文件号	543
PROTOCOL -- 通道通讯协议	543
ETH_MODE -- 网口模式设置	544
SEND_AUTOUP -- 主动上报	544
SEND_AUTOUP2 -- 主动上报 2	544
IFAUTOUP_PORT -- 主动上报端口查询	545
PORT_KEEPLIVE -- 超时检测	545
PORT_485DELAY -- 收发数据延迟	545
13.6 MODBUS 通讯指令	546
MODBUS_BIT -- 位寄存器	546
MODBUS_IEEE -- 字寄存器-32 位浮点型	546
MODBUS_LONG -- 字寄存器-32 位整型	547
MODBUS_REG -- 字寄存器-16 位整型	547
MODBUS_STRING -- 字寄存器-字节	548
MODBUSM_DES -- modbus 通讯连接	548
MODBUSM_DES2 -- 控制器间网口通讯	549
MODBUSM_STATE -- modbus 通讯状态	551
MODBUSM_REGSET -- 写对端保持寄存器	552

MODBUSM_REGGET -- 读对端保持寄存器.....	552
MODBUSM_3XGET -- 读对端输入寄存器.....	553
MODBUSM_BITSET -- 写对端线圈.....	554
MODBUSM_BITGET -- 读对端线圈.....	554
MODBUSM_1XGET -- 读对端离散输入.....	554
13.7 控制器互联直接命令指令	555
SEND_RESULT -- 读取 send 结果	555
SEND_CMD -- send 命令	555
SEND_CMDAXIS -- send 命令	555
SEND_ASSIGN -- send 命令	556
SEND_QUERY -- send 命令	556
SEND_QUERYSET -- send 命令	557
13.8 控制器互联文件发送指令	557
SEND_ZAR -- U 盘操作	557
SEND_FLASH -- 数据拷贝	558
SEND_FILE -- 拷贝 U 盘数据	558
SEND_IFLASH -- 拷贝 flash 数据	559
SEND_PERCENT -- 查询指令进度	559
ZAR_CONTROL -- 查看 ZAR 对应控制器类型	559
第十四章 系统相关指令	561
14.1 控制器加密指令	561
APP_PASS -- 密码	561
LOCK -- 锁定控制器	561
UNLOCK -- 解锁控制器	561
14.2 系统时间指令	562
DATE -- 系统日期	562
DATES -- 系统日期	562
DAY -- 系统星期	562
DAYS -- 系统星期	563
RTC_DATE -- 系统日期	563
TIME -- 系统时间	564
TIMES -- 系统时间	564
RTC_TIME -- 系统时间	565
14.3 轴系统参数指令	565
WDOG -- 轴总使能	565
DISABLE_GROUP -- 轴分组	565
ERROR_AXIS -- 报错轴号	566
MOTION_ERROR -- 报错轴列表	566
ERROR_SET -- 报错输出	566
RADIUS_ERRSET -- 圆弧插补检查	567
14.4 IP 参数指令	567
IP_ADDRESS -- IP 地址	567
IP_ADDRESS2 -- IP 地址 2	568
IP_GATEWAY -- IP 网关	568
IP_NETMASK -- IP 掩码	569
IP_IFDHCP -- IP 自动获取	569
IP_IFDHCP2 -- IP 自动获取 2	569
14.5 控制器信息指令	570
VERSION_FPGA -- 系统 FPGA 版本	570
VERSION_BUILD -- 系统固件创建日期	570
VERSION_DATE -- 系统固件版本	570

VERSION -- 系统软件版本	571
ID_HARDWARE -- 控制器硬件型号	571
CONTROL -- 控制器软件型号	572
SYSTEM_ZSET -- 控制器设置	572
LEDOUT -- 控制器指示灯	573
SERIAL_NUMBER -- 控制器唯一 ID	573
SERVO_PERIOD -- 总线通讯周期	574
SYS_ZFEATURE -- 系统规格	575
SYS_IOSET -- 特殊 IO 切换	577
LASER_SET -- 能量并口输出切换	577
ZML_DEFSHIFT -- ZML 设备 shift 时间	577
14.6 日志指令	578
RTLOG_COUNT -- 当前记录日志条数	578
RTLOG_CLEAR -- 清除当前记录的日志	578
RTLOG_ADD -- 添加日志记录错误信息	578
RTLOG_CODE -- 获取日志的错误编号	579
RTLOG_TIMES\$ -- 获取日志的出错时间	579
RTLOG_INFO -- 获取日志的错误信息	579
RTLOG_INFO2 -- 获取日志的错误信息 2	580
?*RTLOG -- 打印当前记录的日志	580
14.7 TABLE 数组指令	581
TABLE -- 系统缺省数组	581
TSIZE -- table 大小	581
TABLESTRING -- 字符串格式打印 table	582
14.8 示波器相关指令	582
TRIGGER -- 触发示波器	582
SCOPE -- 数据采样	583
SCOPE_POS -- 采样点数	583
14.9 VR 相关指令	584
CLEAR -- 清除 VR	584
VR -- 掉电保存	584
VR_INT -- 掉电保存整型	584
VRSTRING -- 掉电保存字符串	585
第十五章 存储相关指令	586
15.1 U 盘相关指令	586
FILE -- U 盘文件操作	586
U_STATE -- U 盘状态	589
U_READ -- 从 U 盘读取	590
U_READDBL -- 从 U 盘读取-double	590
U_READ2 -- 从 U 盘读取 2	591
U_READ2DBL -- 从 U 盘读取 2-double	591
U_READDSB -- DSB 文件读取	592
U_WRITE -- 输出到 U 盘	593
U_WRITEDBL -- 输出到 U 盘-double	594
U_WRITE2 -- 输出到 U 盘 2	594
STICK_READ -- U 盘读取到 table	595
STICK_WRITE -- table 输出到 U 盘	595
STICK_READVR -- U 盘读取到 vr	596
STICK_WRITEVR -- vr 输出到 U 盘	596
DISK_CONFIG -- 示教盒 U 盘功能	597
15.2 FLASH 相关指令	597

FLASH_WRITE -- flash 存储.....	597
FLASH_WRITEDBL -- flash 存储-double.....	598
FLASH_WRITE2 -- flash 存储 2.....	599
FLASH_READ -- flash 读取	599
FLASH_READDBL -- flash 读取-double	600
FLASH_READ2 -- flash 读取 2	600
FLASH_READ2DBL -- flash 读取 2-double	601
FLASHVR -- 拷贝 RAM 的数据	602
FLASH_SECTSIZE -- flash 变量数	603
FLASH_SECTES -- flash 块数.....	603
15.3 文件存储相关指令	603
FILE_ZOPEN -- 打开文件	603
FILE_ZCLOSE -- 关闭文件	604
FILE_ZWRITES -- 文件写入字符串	605
FILE_ZWRITE -- 文件写入字符	605
FILE_ZREAD -- 文件读取字符	606
FILE_ZREADLINE -- 文件行读取.....	606
FILE_ZSEEK -- 文件定位.....	607
FILE_ZSEEKLINE -- 文件行定位.....	607
FILE_ZTELL -- 文件读写位置	608
FILE_ZTELLLINE -- 文件行号	608
第十六章 中断相关指令	609
16.1 三种中断指令	609
INT_ENABLE -- 中断总开关	609
ONPOWEROFF -- 掉电中断 SUB.....	610
INT_ONn -- 外部输入中断 SUB.....	611
INT_OFFn -- 外部输入中断 SUB	611
ONTIMERn -- 定时器中断 SUB.....	612
INT_CYCLE -- 中断周期执行	612
16.2 定时器指令	613
TIMER_IFEND -- 定时器状态.....	613
TIMER_START -- 启动定时器	614
TIMER_COUNT -- 定时器累计时间.....	614
TIMER_STOP -- 停止定时器.....	614
第十七章 总线相关指令	616
17.1 编号释义	616
槽位号	616
设备号	616
驱动器编号	616
17.2 基础指令	616
SLOT_SCAN -- 总线扫描	616
SLOT_START -- 总线开启	617
SLOT_STOP -- 总线停止	618
SLOT_INFO -- 总线信息读取	618
?*SLOT -- 打印总线接口	619
?*ETHERCAT -- 打印 EtherCAT 总线状态.....	619
ZTEST -- EtherCAT 总线信息查询.....	620
?*RTEX -- 打印 Rtex 总线状态	622
?*ZML -- 打印 ZML 信息	623
17.3 SDO 操作指令	623
SDO_WRITE -- 数据字典写入	623

SDO_WRITE_AXIS -- 数据字典写入.....	624
SDO_READ -- 数据字典读取.....	625
SDO_READ_AXIS -- 数据字典读取.....	626
SDO_READ2 -- 数据字典读取至 MODBUS 寄存器.....	627
17.4 设备相关指令.....	627
NODE_COUNT -- 设备个数.....	627
NODE_STATUS -- 设备状态.....	628
NODE_AXIS_COUNT -- 设备电机数.....	628
NODE_IO -- 设备 IO.....	629
NODE_AIO -- 设备模拟量.....	629
NODE_INFO -- 设备信息.....	630
NODE_PROFILE -- PDO 预设置.....	631
NODE_PDOBUFF -- 特殊设备 PDO 设置.....	631
NODE_PDO_WRBUFF -- 偏移修改 PDO.....	632
NODE_PDO_RDBUFF -- 偏移读取 PDO.....	632
NODE_REGWRITE -- ESC 寄存器写入.....	633
NODE_REGREAD -- ESC 寄存器读取.....	633
NODE_PRESET -- 设备预配置.....	634
ZML_INFO -- 设备 XML 查询.....	634
17.5 驱动器相关指令.....	636
DRIVE_MODE -- 驱动器模式.....	636
DRIVE_PROFILE -- 驱动器 PDO 设置.....	636
DRIVE_CW_MODE -- 驱动器设置.....	640
DRIVE_CONTROLWORD -- 驱动器控制字.....	640
DRIVE_STATUS -- 驱动器状态.....	641
DRIVE_IO -- 驱动器 IO.....	642
DRIVE_TORQUE -- 驱动器力矩.....	643
DRIVE_TORQUEMAX -- 驱动器最大力矩限制.....	644
DRIVE_TORQUEMIN -- 驱动器最小力矩限制.....	644
DRIVE_FE -- 驱动器误差.....	644
DRIVE_FE_LIMIT -- 驱动器误差限制.....	645
DRIVE_CLEAR -- 清除报警.....	645
DRIVE_READ -- 参数读取.....	646
DRIVE_WRITE -- 参数写入.....	648
第十八章 ZHD 示教盒参数.....	652
18.1 示教盒相关指令.....	652
LCD_CONNECT -- LCD 编号设置.....	652
IP_CONNECT -- IP 连接.....	652
IP_ADDRESS -- IP 地址.....	652
IP_GATEWAY -- IP 网关.....	652
IP_NETMASK -- IP 掩码.....	653
18.2 控制器相关指令.....	653
LCD_LEDSTATE -- LED 灯状态设置.....	653
LCD_WDOGTIME -- 屏掉线处理时间设置.....	653
第十九章 MotionRT 相关指令.....	655
CARD_INFO -- 读写控制卡信息.....	655
?*CARD -- 打印子卡信息.....	656
REG_CARD -- 锁存选择.....	656
SLOT_SLAVE -- ETHERCAT 冗余配置.....	658
SLOT_PERIOD -- 多 ECAT 总线周期配置.....	658
第二十章 本地后级接口指令.....	659

ZMIO_CONFIG -- 设置/获取模拟量量程和通道状态	659
ZMIO_INFO -- 查询自带 ZMIO 扩展	660
ZMIO_OFFSET -- 自带 ZMIO 扩展 IO 偏移	660
ZMAIO_OFFSET -- 自带 ZMIO 扩展 AIO 偏移	661
第二十一章 CAD 控件及指令	662
21.1 CAD 控件说明	663
21.2 CAD 指令说明 -- 基础指令	664
CAD_CHANNEL -- CAD 通道	664
CAD_LOADFILE -- 从文件中载入图片	665
CAD_LOADTABLE -- 从 TABLE 中载入图形	665
CAD_LOADBIN -- 从 BIN 文件中载入图形	666
CAD_CLOSE -- 关闭 CAD 加载图形	666
CAD_TOCODE -- 导出代码文件 (NC/Z3P)	666
CAD_TABSIZE -- 占用 table 空间	667
CAD_TOTABLE -- 导出 TABLE 数据	667
CAD_TOBIN -- 导出 BIN 文件	668
CAD_TAB2CODE -- TABLE 数据转代码文件	668
CAD_PARA -- 参数设置	668
CAD_DISPLAY -- 显示方式设置	669
CAD_MOUSE -- 鼠标模式设置	669
CAD_AUTOVIEW -- 自适应视图	670
CAD_RANGE -- 获取轨迹范围	670
21.3 CAD 指令说明 -- 自定义控件扩展指令	671
CAD_SETCOLOR -- 设置 CAD 绘制颜色	671
CAD_DRAW -- 绘制 CAD 图形	671
CAD_DRAWTAB -- 从 TABLE 中绘制 CAD 图形	671
CAD_TRANSPOINT -- 转换坐标为控件位置	672
CAD_TRANSPOINT2 -- 转换控件位置为图形坐标	672
CAD_SELECT -- 选中图形	673
21.4 CAD 指令说明 -- 高级指令	673
CAD_MACHTYPE -- 机器类型	673
CAD_OPTIMIZE -- 轨迹优化	674
CAD_EDIT -- 自定义编辑	675
CAD_MOVE -- 轨迹平移	675
CAD_SCALE -- 轨迹缩放	676
CAD_ROTATE -- 轨迹旋转	676
CAD_DELETE -- 轨迹删除	676
CAD_ACTION -- 获取/修改动作数据	676
CAD_ACTIONADD -- 添加动作	677
CAD_ACTIONDEL -- 删除动作数据	677
CAD_UNDO -- 撤销上一步编辑操作	677
CAD_REDO -- 重做上一步撤销动作	678
CAD_SORT -- 手动排序	678
21.5 CAD 参数表	678
通用参数	678
G 代码参数	679
Basic 参数	680
21.6 TABLE 数据格式说明	680
数据格式设计	680
DataType 定义	680
21.7 机器类型工艺	684

通用三轴机床	684
通用二轴机床	684
特殊三轴钻孔机床	684
第二十二章 DT 运动函数	685
MOVEDTSP/MOVEDTABSSP -- DT 直线运动	685
MOVECIRCDTSP/MOVECIRCDTABSSP -- DT 圆弧运动	685
MOVECIRC2DTSP/MOVECIRC2DTABSSP -- DT 三点画圆弧运动	686
MSPHERICALDTSP/MSPHERICALDTABSSP -- DT 空间圆弧运动	687
第二十三章 简易例程	688
23.1 常用操作	688
IO 操作	688
SP 指令连续插补	688
字符串与数据相互转化	689
手轮	689
追剪应用	690
位置比较输出	690
掉电保存	690
机械手应用	690
机械手使用 movesync 指令	690
编码器读取	694
自定义 G 代码	696
23.2 模块通讯	698
CAN 通讯	698
触摸屏通讯	700
自定义网口通讯	703
控制器之间通讯	704
自定义串口通讯和字符串使用	705
23.3 总线初始化	705
EtherCAT 初始化程序	705
Rtex 初始化程序	710
第二十四章 错误与调试	711
24.1 常见问题列表	711
问题排查	711
24.2 解决办法	713
手动运动调试	713
断点调试	714
示波器抓取	715
寄存器查看	716
在线发送命令	716
打印程序信息	716
IO 口快速检测	717
轴参数状态判断	717
附录一 错误码列表	719
附录二 模块扩展	746
ZCAN 扩展模块	746
扩展接线	746
资源映射	747
EtherCAT 扩展模块	751
扩展接线	751
资源映射	752
附录三 触摸屏通讯	754

控制器与触摸屏通讯简介	754
控制器与触摸屏连接	754
连接 ZHD 系列触摸屏	754
连接第三方触摸屏	756
附录四 ETHERCAT 通讯	761
附录五 RTEX 总线.....	765

第一章 运动控制产品简介

1.1 运动控制产品概述

运动控制实现了对机械传动部件的位置、速度、加速度等的实时控制，使其按照预期的轨迹与规定的运动参数完成相应的动作。

控制系统以处理器、检测机构、执行机构为核心，实现逻辑控制、位置控制、轨迹加工控制、机器人运动控制等。其中处理器通常是可编程控制器、单片机、或运动控制器，相当于系统的大脑，主要负责对接收到的信号进行逻辑处理，并给执行机构下发命令，协调系统的正常运转。检测机构通常由各种传感器构成，相当于系统的眼睛，目的是检测系统条件的变化并反馈给控制器，执行机构通常由伺服单元、阀门构成，相当于系统的双手，主要执行控制器下发的命令。

运动控制器是运动控制系统的核心部件，负责产生运动路径的控制指令，用于设备的逻辑控制，将运动参数分配给需要运动的轴，并对被控对象的外部环境变化及时做出响应。

通用运动控制器通常都提供一系列运动规划方法，基于对冲击、加速度和速度等这些可影响动态轨迹精度的量值加以限制，提供对运动控制过程的运动参数的设置和运动相关的指令，使其按预先规定的运动参数和规定的轨迹完成相应的动作。

1.2 运动控制产品优点

正运动技术的运动控制产品包括脉冲型独立式运动控制器、脉冲型网络运动控制卡、总线型独立式运动控制器、总线型 PCI 运动控制卡等，能满足各行各业的运动控制需求，单个控制器可支持 128 轴的运动控制。

运动控制产品支持直线、圆弧、空间圆弧、椭圆、螺旋等多种插补运动，单个插补通道最多支持 16 轴联合插补。支持速度前瞻、电子凸轮、电子齿轮、螺距补偿、固步跟踪、运动叠加、虚拟轴、精准输出、硬件位置锁存、位置比较输出、连续插补、运动暂停等功能。

部分运动控制产品内置了 SCARA、DELTA、六关节等 30 多种机械手运动控制算法，单个控制器可以控制多个机械手，同时支持多个机械手叠加。机械手详细介绍参见“正运动机械手指令说明”文档。

总线型运动控制产品支持 EtherCAT、RTEX 工业以太网运动控制总线，在性能和稳定性方面均处于领先地位，并可支持 EtherCAT 总线、RTEX 总线和脉冲轴三者混合使用。

国内首家推出同时支持 EtherCAT 总线和 RTEX 总线的双总线 PCI 控制卡与双总线运动控制器，EtherCAT 总线周期最快达 100 微秒，同时还支持总线轴硬件位置锁存与位置比较输出。

正运动还为控制产品提供了强大的 RTSys 软件开发环境，操作简单易学。

运动控制器支持以太网、U 盘、CAN 总线、RS485 串口、RS232 串口等通讯接口，通过 CAN 总线或 EtherCAT 总线可以连接正运动的扩展模块，从而扩展输入输出点或脉冲运动轴数（CAN 总线两端需要并接 120 欧姆的电阻）。

控制器具有如下优点：

1. 硬件组成简单，把运动控制器连入 PC 就可组成系统；
2. 除了 RTSys 软件，还支持各种操作系统与编程语言进行上位机软件开发（例如 VC, VB, C#, PYTHON, LABVIEW 等）；
3. 运动控制软件的代码通用性和可移植性较好；
4. 不需要太多培训工作，就可以进行开发，简单易学，支持多人同时开发。

1.3 控制器主要功能描述

控制器的主要功能简介：

项目		描述
任务		以指定条件执行 I/O 刷新和用户程序的功能，支持多任务同时进行，互不干扰，最大任务数在 RTSys 软件“控制器状态”查看
调试		支持断点调试和单步调试，以及任务运行状态查看
中断		支持三种类型的中断（外部中断、定时器中断、掉电中断）
设定监控窗口		监控变量常量、输入输出、轴参数等
编程语言种类		RTSys 编程（BASIC，PLC，HMI），或常用上位机编程语言
在线命令		在线命令栏输入指令参数发送给控制器立即执行
通讯接口	串口	232 串口和 485 串口，支持 MODBUS_RTU 协议和自定义通讯
	网口	通讯速度快，接线方便，支持 MODBUS_TCP 协议和自定义通讯
	U 盘	插入 U 盘，数据交互
	CAN 总线	连接 ZIO/ZMIO-CAN 扩展模块，控制器互联，标准 CAN 设备通讯
	EtherCAT 总线	连接 EtherCAT 驱动器或 EtherCAT 扩展模块
	RTEX 总线	连接 RTEX 驱动器
数据类型	自定义数组	集中相同数据类型的元素，默认浮点型
	自定义变量	默认浮点型
	自定义常量	可以是布尔型，字符串型，时间型，日期型，整型等
	寄存器	自带四类寄存器，TABLE，MODBUS，VR，FLASH
常用运动控制功能	点位运动	JOG 点动
	插补运动	直线、圆弧、空间圆弧、椭圆、螺旋等多种插补，支持连续插补
	电子齿轮	建立主从轴之间的电子齿轮连接
	电子凸轮	分凸轮表运动和自动凸轮两类
	运动叠加	不同轴的运动叠加
	轨迹前瞻	根据前瞻参数，速度或轨迹自行优化
	位置锁存	根据外部信号触发的发生记录轴的位置
	位置比较输出	到达比较点输出 OP 信号，连续比较，快速响应
	精准输出	OP 快速响应

1.4 控制器应用场景

正运动技术的运动控制产品经过众多合作伙伴多年的开发应用，产品广泛应用于 3C 电子半导体、点胶设备、激光加工、印刷包装、特种机床、机器人、舞台娱乐、医疗器械等自动化领域。

电子产品加工行业有贴片机、点胶机、印刷电路板钻孔机、绕线机、焊接机、上下料机械手、紧螺钉机等设备。

纺织服装行业有经编机、染色机、印花机、工业缝纫机、绣花机、切布机、精梳机、捻线机、制鞋机等设备。

印刷包装行业有自动吹瓶机、制袋机、模切机、烫金机、开箱机、装箱机，贴标机、自动颗粒包装机、袋装包装机、报纸印刷机、凹版印刷机等。

哪里有自动化设备，哪里就有运动控制，正运动控制器以其优异的性能、完善的功能为各行各业均能提供优秀的解决方案。

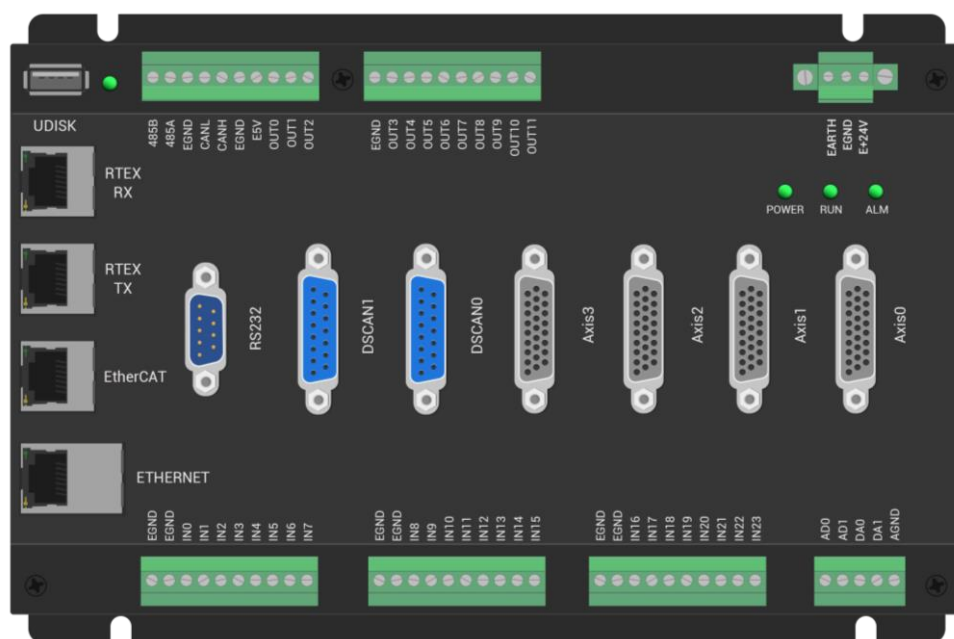
1.5 控制器接口

这里用 ZMC420SCAN 总线型运动控制器为例展开说明。

ZMC420SCAN 总线型运动控制器支持 EtherCAT 总线和 RTEX 总线连接，最多支持 20 个轴的运动控制，支持不同类型轴(脉冲轴、EtherCAT 总线轴、RTEX 总线轴、编码器轴、振镜轴、虚拟轴)的混合插补，支持全功能运动控制，产品详细参数参见硬件手册。

ZMC 运动控制器支持以太网、USB、CAN、485、232 等通讯接口，通过 CAN 总线或 EtherCAT 总线可以连接相应扩展模块，从而扩展输入输出点数或脉冲轴数(CAN 总线接线两端需要并接 120 欧姆的电阻)，扩展方法参见“[模块扩展](#)”。

ZMC420SCAN 控制器外观如下图所示：



接口功能如下表：

标识	接口	个数	说明
RS232	232 串口	1 个	采用 MODBUS_RTU 协议
RS485	485 串口	1 个	采用 MODBUS_RTU 协议
CAN	CAN 总线	1 个	连接 CAN 扩展模块或控制器
ETHERNET	网口	1 个	采用 MODBUS_TCP 协议，通过交换机扩展接口个数
EtherCAT	EtherCAT 总线	1 个	连接 EtherCAT 驱动器或 EtherCAT 扩展模块
RTEX	RTEX 总线	1 个	连接 RTEX 驱动器
UDISK	U 盘	1 个	插入 U 盘设备
E +24V	主电源	1 个	24V 直流电源供电

IN	数字量输入	24 个	NPN 类型，内部 24V 供电
OUT	数字量输出	12 个	NPN 类型，内部 24V 供电
AD	模拟量输入	2 个	分辨率 12 位，量程 0-10V
DA	模拟量输出	2 个	分辨率 12 位，量程 0-10V
DSCAN	振镜轴接口	2 个	接入激光振镜，支持 XY2-100 协议
Axis	脉冲轴接口	4 个	每个接口包含脉冲输出和编码器输入

1.6 控制器的使用

1. 准备工作

软件：安装 RTSys 编程软件或控制器支持上位机的其他编程软件（采用 VC，VB，C#，PYTHON，LABVIEW 等编程）。

设备：控制器、计算机、24V 直流电源、驱动器、步进电机或伺服电机、接线端子、IO 设备、扩展模块等根据需求选择。

连接线：控制器与计算机通讯的连接线，控制器与驱动器轴接口的连接线，IO 接口、电源接口等的连接线。

2. 程序设计

1. 系统架构设计

根据功能需求选择所需元件和连接线，熟悉与该功能相关的控制指令的使用方法，设计系统软件的整体构成，包括变量设计、任务设计、程序功能设计等。

2. 软件设定与编程

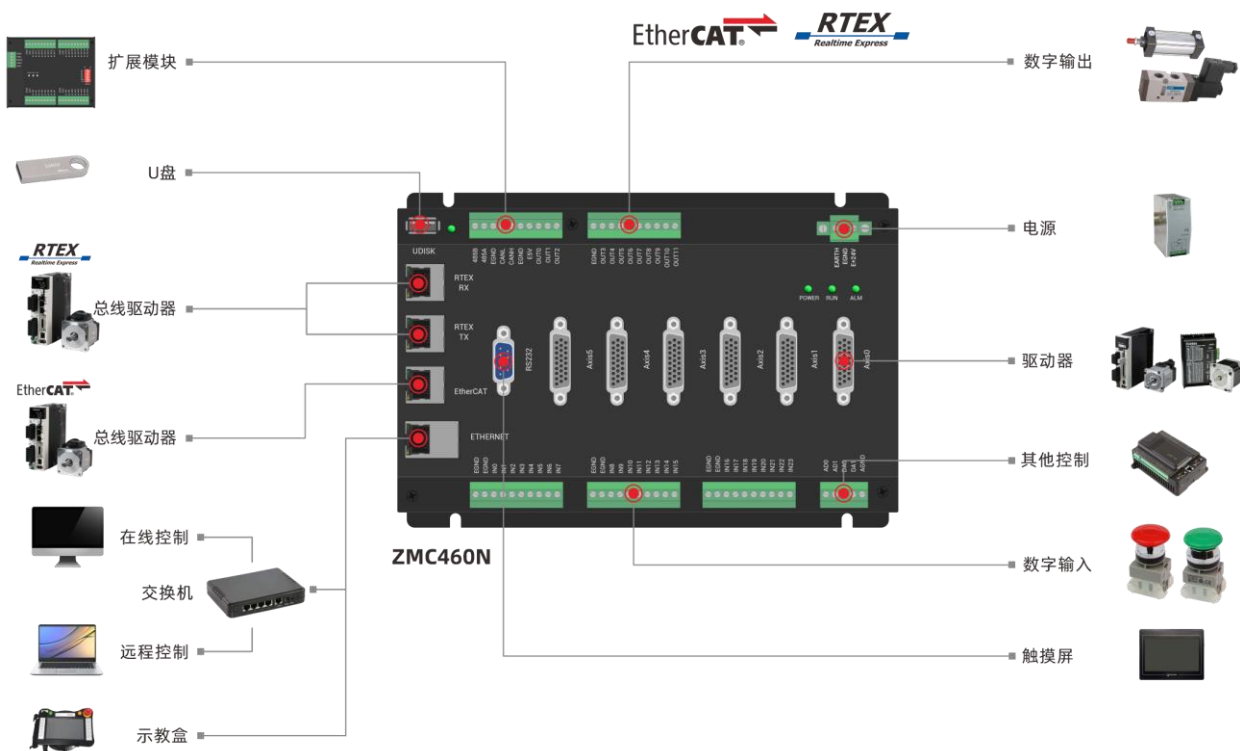
使用 RTSys 软件按步骤 1 的设计编写程序，软件快速使用方法参见本文“[新建工程](#)”小节，或打开 RTSys 软件菜单栏“常用”-“帮助文档”-“RTSys 帮助”查看软件具备的各种功能用法介绍，编写任务和程序模块，进行程序模拟调试。

编程需要设置的参数：BASE 选择轴号、ATYPE 轴类型、UNITS 脉冲当量、SPEED 轴速度、ACCEL 轴加速度，DECEL 轴减速度等基础轴参数，再给轴发送运动指令。

若使用 EtherCAT 总线或 RTEX 总线连接驱动器，编程时需要进行总线初始化操作（参见“[总线初始化](#)”例程）。若需要扩展模块，例如扩展轴或 IO 点数，编程时需要对扩展的轴资源进行轴映射（参见“[轴映射](#)”）；扩展的 IO 资源需要进行 IO 映射，ZCAN 扩展使用扩展板上的拨码开关设置扩展 IO 的编号（参见“[ZCAN 扩展模块](#)”章节），EtherCAT 总线扩展使用 NODE_IO 指令设置扩展的 IO 编号，通过 IO 编号即可访问扩展资源。

3. 安装与接线

安装各种单元，将各单元与控制器采用合适的连接线连接，控制器接线参考如下图。



计算机与控制器接线：可以采用串口通信或网口通讯，串口通信连接控制器的 RS232 串口，网口通讯连接控制器的 EtherNET 网口。

驱动器与控制器接线：驱动器可连接控制器的脉冲轴接口、EtherCAT 总线接口、RTEX 总线接口。驱动器接脉冲口时参见下图，使用总线连接驱动器只需用网线直接插入对应的 EtherCAT 或 RTEX 接口。

脉冲轴与编码器接口(DB26母头)

引脚号	信号
1	EGND
2	ALM(IN24-29)
3	S/ON(OUT12-17)
4	EA-
5	EB-
6	EZ-
7	+5V
8	备用
9	DIR+
10	GND
11	PUL-
12	备用
13	GND
14	+24V
15	备用
16	备用
17	EA+
18	EB+
19	EZ+
20	GND
21	GND
22	DIR-
23	PUL+
24	GND
25	备用
26	备用

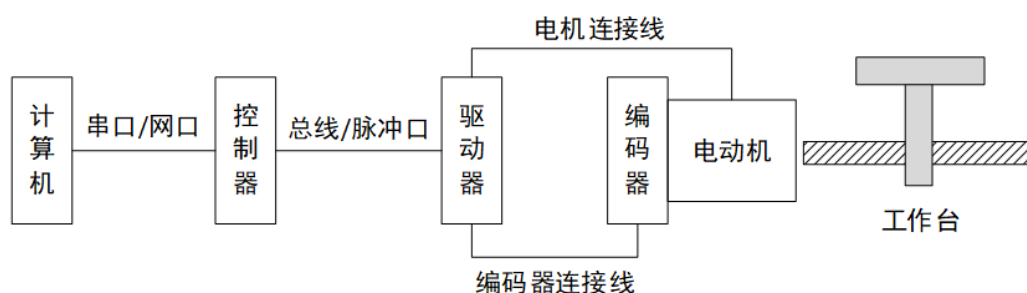
松下A6伺服驱动器

引脚号	信号
36	ALM-
37	ALM+
29	S/ON
22	OA-
49	OB-
24	OZ-
5	SIGN
4	/PULS
41	COM-
7	COM+
21	OA+
48	OB+
23	OZ+
6	/SIGN
3	PULS
13	GND
25	GND

电源接线：将+24V 直流电源正极接控制器电源模块的 24V 接口，负极接 GND 接口，电机接 220V 交流电，IO 设备接在控制器对应的 IO 接口上，部分型号控制器 IO 需要采用 24V 直流 IO 电源单独供电后才可使用。

扩展模块接线：支持扩展 IO 或脉冲轴，可使用 CAN 总线扩展或 EtherCAT 总线扩展。详细方法参见“[模块扩展](#)”章节。

配置参考：



4. 试运行

请确认接线无误后上电，将调试好的程序下载到控制器，开始试运行。使用示波器窗口或其他参数监控窗口确认动作是否符合要求。

第二章 RTSys 软件编程

2.1 编程软件简介

RTSys 是 ZMotion 系列运动控制器的 PC 端程序开发调试与诊断软件，通过它用户能够很容易的对控制器进行程序编辑与配置，快速开发应用程序、实时监控轴运行参数以及对运动控制器正在运行的程序进行实时调试，支持中英双语环境。

RTSys 编程软件支持 RTBasic、RTPlc 梯形图、RTHmi 组态编程，RTBasic 是 ZMotion 运动控制器所使用的 Basic 编程语言，提供所有标准程序语法、变量、数组、条件判断、循环及数学运算。此扩展的 Basic 指令和函数能提供广泛的运动控制功能，例如单轴运动、多轴的同步和插补运动，同时还有对数字和模拟 I/O 的控制。

RTBasic 支持以下功能：

1. 自定义 SUB 过程，可以把一些通用的功能编写为自定义 SUB 过程，方便程序编写和修改。
2. G 代码形式的 SUB 过程，支持 G00、G01、G02、G03、G04、G90、G92 等常用指令。
3. 支持全局的变量（GLOBAL）、数组和 SUB 过程；文件模块变量、数组和 SUB 过程；以及局部变量（LOCAL）。
4. 中断程序（掉电中断、外部中断、定时器中断），例如掉电中断，通过掉电中断时保存数据，可以使得掉电的状态得到恢复。

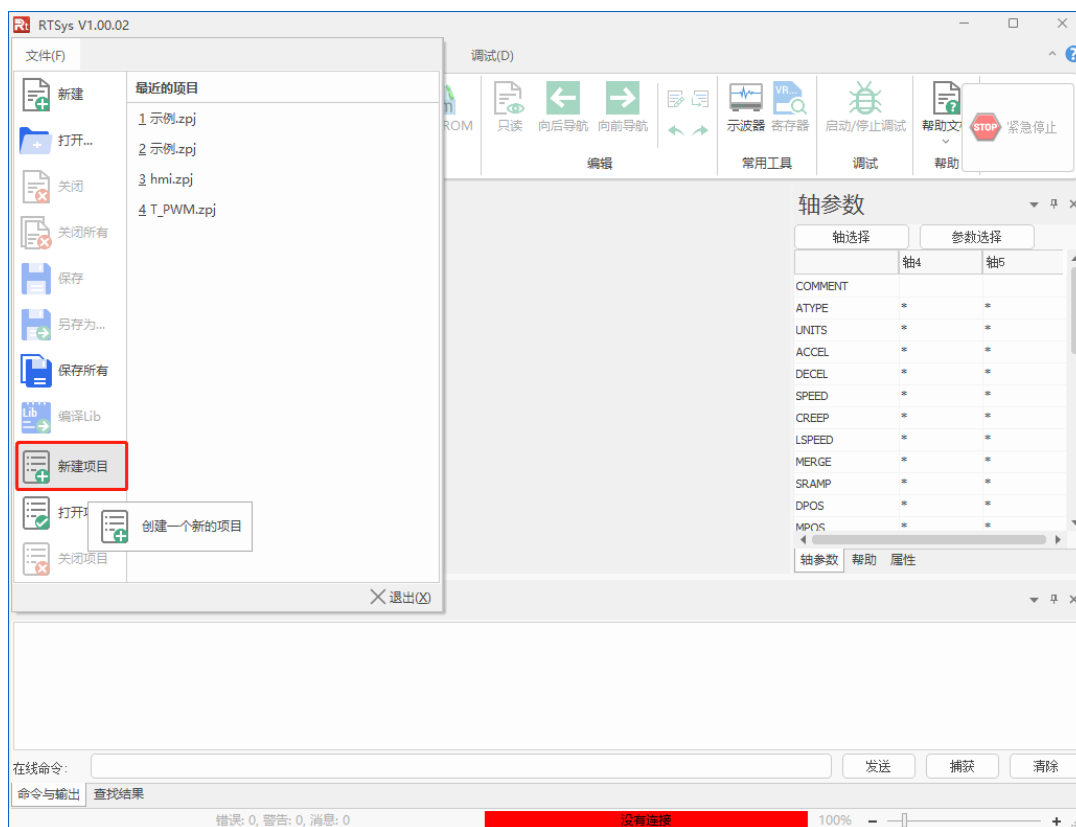
RTBasic 具有实时多任务的特性，多个 RTBasic 程序可以同时构建并多任务实时运行，使得复杂的应用变得简单易行。

通过 PC 在线发送 Basic 命令也可以实现同样的效果，控制器内置的 Basic 程序和 PC 在线 Basic 命令可以同时多任务运行。

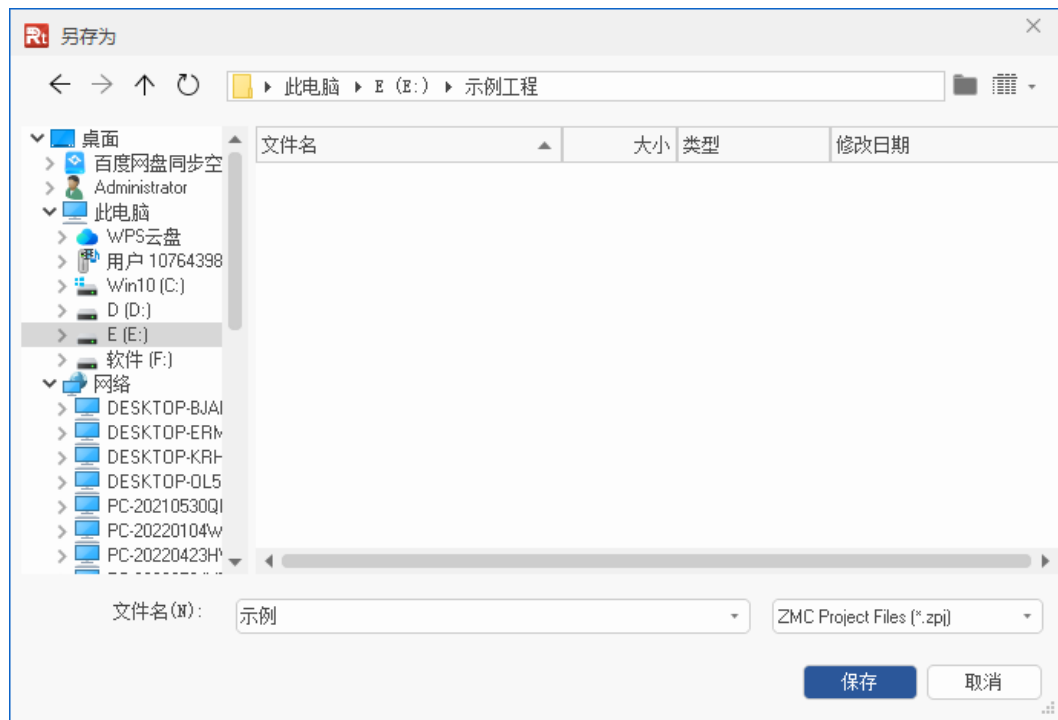
2.2 新建工程

在电脑里新建一个文件夹用来保存即将要建立的工程。打开 RTSys 编程软件，当前说明例程的 RTSys 软件版本为 V1.00.02，更新软件版本请前往正运动官方网站下载，网址：www.zmotion.com.cn。

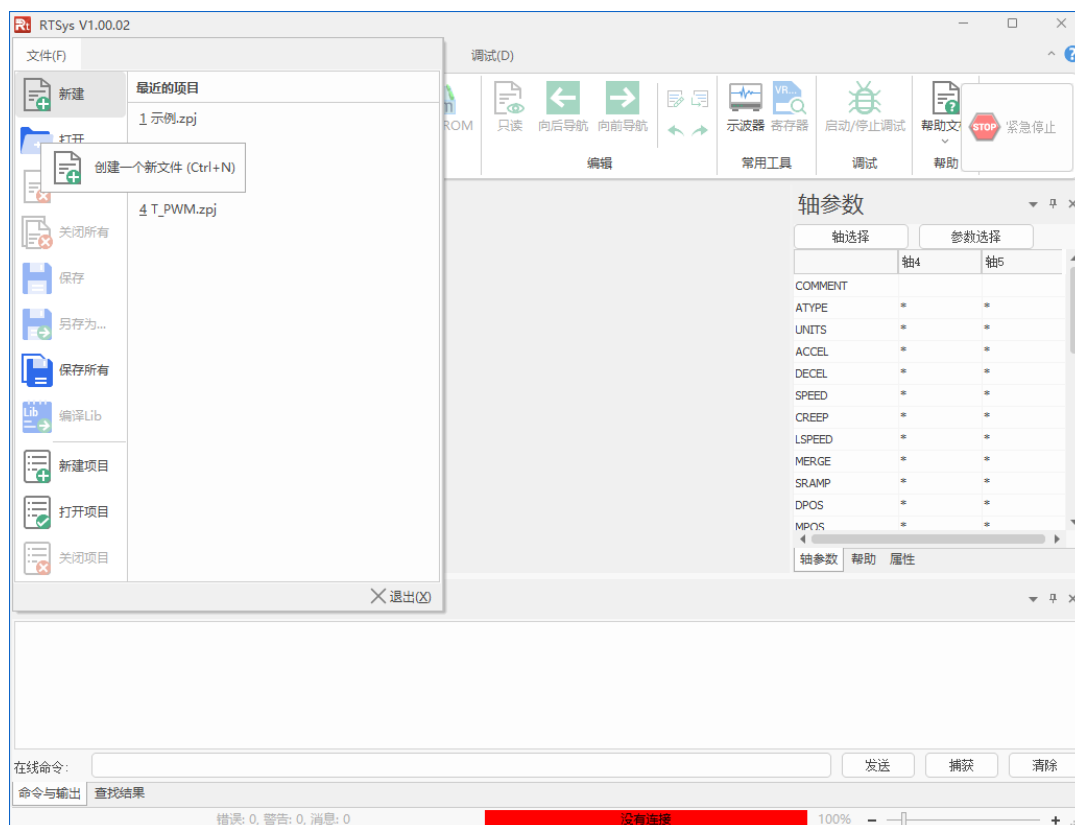
1. 新建项目：菜单栏“文件”→“新建项目”。



2. 点击“新建项目”后弹出“另存为”界面，选择一个文件夹打开，输入文件名后保存项目，后缀为“.zpj”。



3. 新建文件：菜单栏“文件”→“新建”。



点击“新建文件”后，出现下图所示的弹窗，支持 Basic/Plc/Hmi 混合编程，这里选择新建的文件类型为 Basic 后确认。



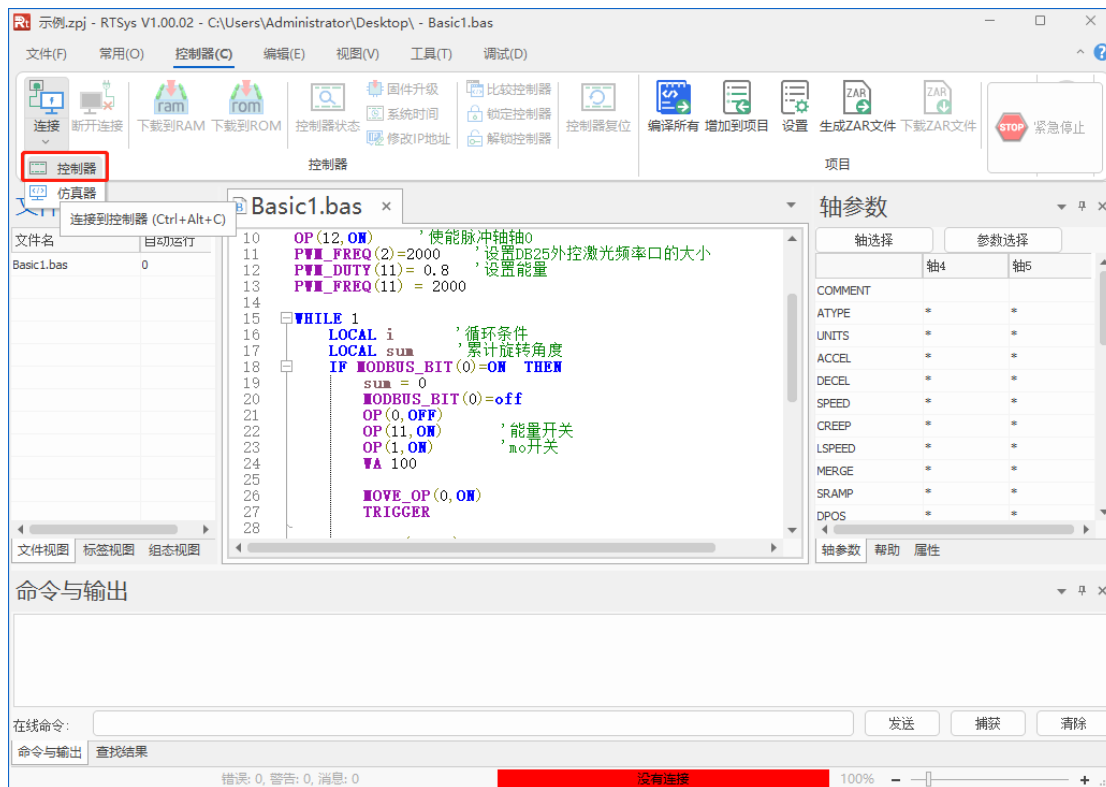
4. 设置文件自动运行：如下图，双击文件右边自动运行的位置，输入任务号“0”。



5. 编辑程序：程序编写完成，点击“保存”按钮保存文件，新建的 Basic 文件会自动保存到项目 zpj 所在的文件夹下。

6. 连接到控制器：在程序输入窗口编辑好程序，点击“控制器”→“连接”→“控制器”。

没有控制器是可选择连接到仿真器仿真运行，点击“控制器”→“连接”→“仿真器”，便可成功连接到仿真器，并弹出仿真器连接成功提示。



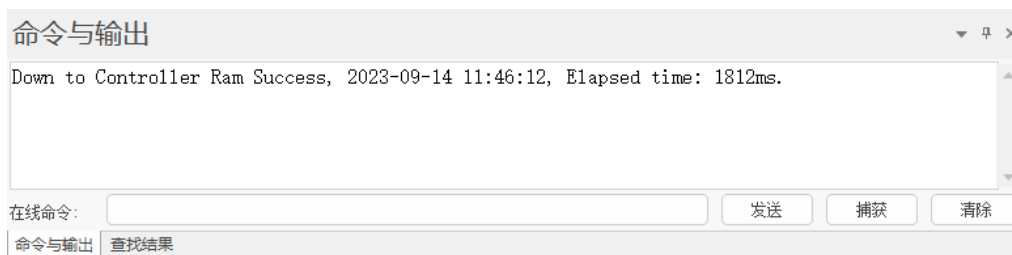
点击“连接”弹出“连接到控制器”窗口，可选择串口连接或网口连接，选择匹配的串口参数或网口 IP 地址后，点击连接即可。连接成功命令与输出窗口打印信息：Connected to Controller:ZMC432 Version:4.64-20170623.



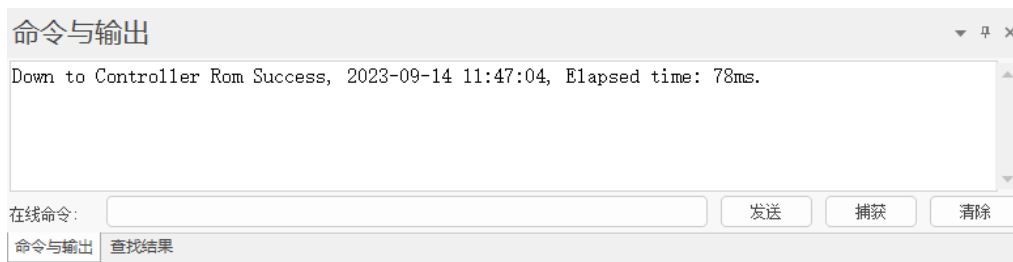
串口连接和网口连接的详细方法参见 RTSys 软件菜单栏“常用”→“帮助文档”→“RTSys 帮助”文档。

7. 下载程序：点击菜单栏“控制器”→“下载到 RAM”或“下载到 ROM”，下载成功命令和输出窗口会有提示，同时程序下载到控制器并自动运行。

成功下载到 RAM:



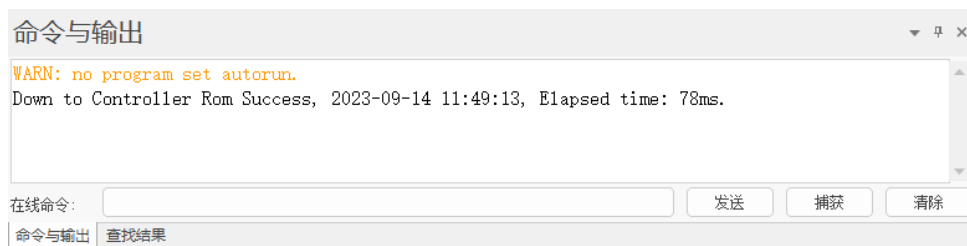
成功下载到 ROM:



RAM 下载掉电后程序不保存，ROM 下载掉电后程序保存。下载到 ROM 的程序下次连接上控制器之后程序会自动按照任务号运行。

注意事项:

1. 打开工程项目时，选择打开项目 **zpj** 文件，若只打开其中的 **Bas** 文件，程序无法下载到控制器。
2. ZMC00x 系列控制器不支持下载到 RAM。
3. 不建立项目的时候，只有 **Bas** 文件无法下载到控制器。
4. 自动运行的数字 0 表示任务编号，以任务 0 运行程序，任务编号不具备优先级。
5. 若整个工程项目内的文件都不设置任务编号，下载到控制器时，系统提示如下信息 **WARN: no program set autorun.**



2.3 在线命令与输出

在线命令与输出窗口可以查询与输出控制器的各种参数、打印程序运行结果、打印程序错误信息，软件开发人员在程序中给出的打印输出函数（由?、PRINT、WARN、ERROR、TRACE 等命令输出）。

注意问号使用英文符号，中文符号输入无效。ERRSWITCH 为 TRACE、WARN、ERROR 指令的控制开关，不同的参数值对应不同的输出效果：

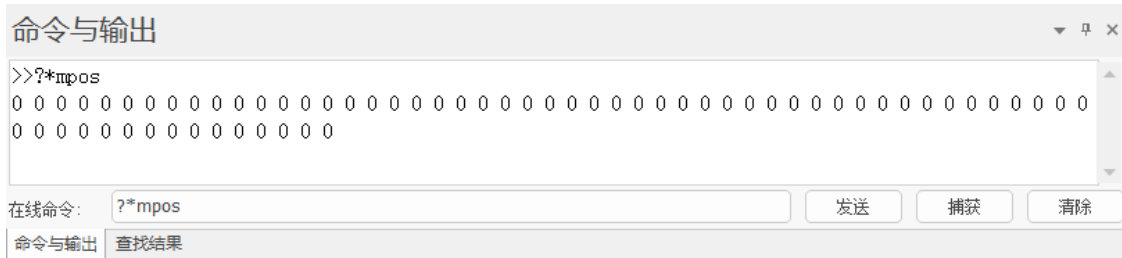
- 0: TRACE、WARN、ERROR 指令全部不输出
- 1: 只输出 ERROR 指令
- 2: 输出 WARN、ERROR 指令
- 3: TRACE、WARN、ERROR 指令全部输出

在线命令与输出窗口如下所示，“>>”表示 RTSys 在线命令输入的指令，在线命令输入“print 1+2”窗口会打印计算结果。

连接了控制器或仿真器就可以使用此功能，不受程序运行状态的限制。



使用在线命令可以查看各种轴的状态。如下图，在线命令输入“?*mpos”窗口会打印出多个轴的测量反馈位置 mpos。



查看系统的状态，常用的打印查看命令有：

?*SET: 打印所有参数值

?*TASK: 打印任务信息

任务正常时只打印任务状态

任务出错时还会打印出错误任务号，具体错误行

?*MAX: 打印所有规格参数

?*FILE: 打印程序文件信息

?*SETCOM: 打印当前串口的配置信息

?*BASE: 打印当前任务的 BASE 列表

?*数组名: 打印数组的所有元素，数组长度不能太长

?*参数名: 打印一个所有轴的单个参数

?*ETHERCAT: 打印 EtherCAT 总线连接设置状态

?*RTEX: 打印 Rtex 总线连接设置状态

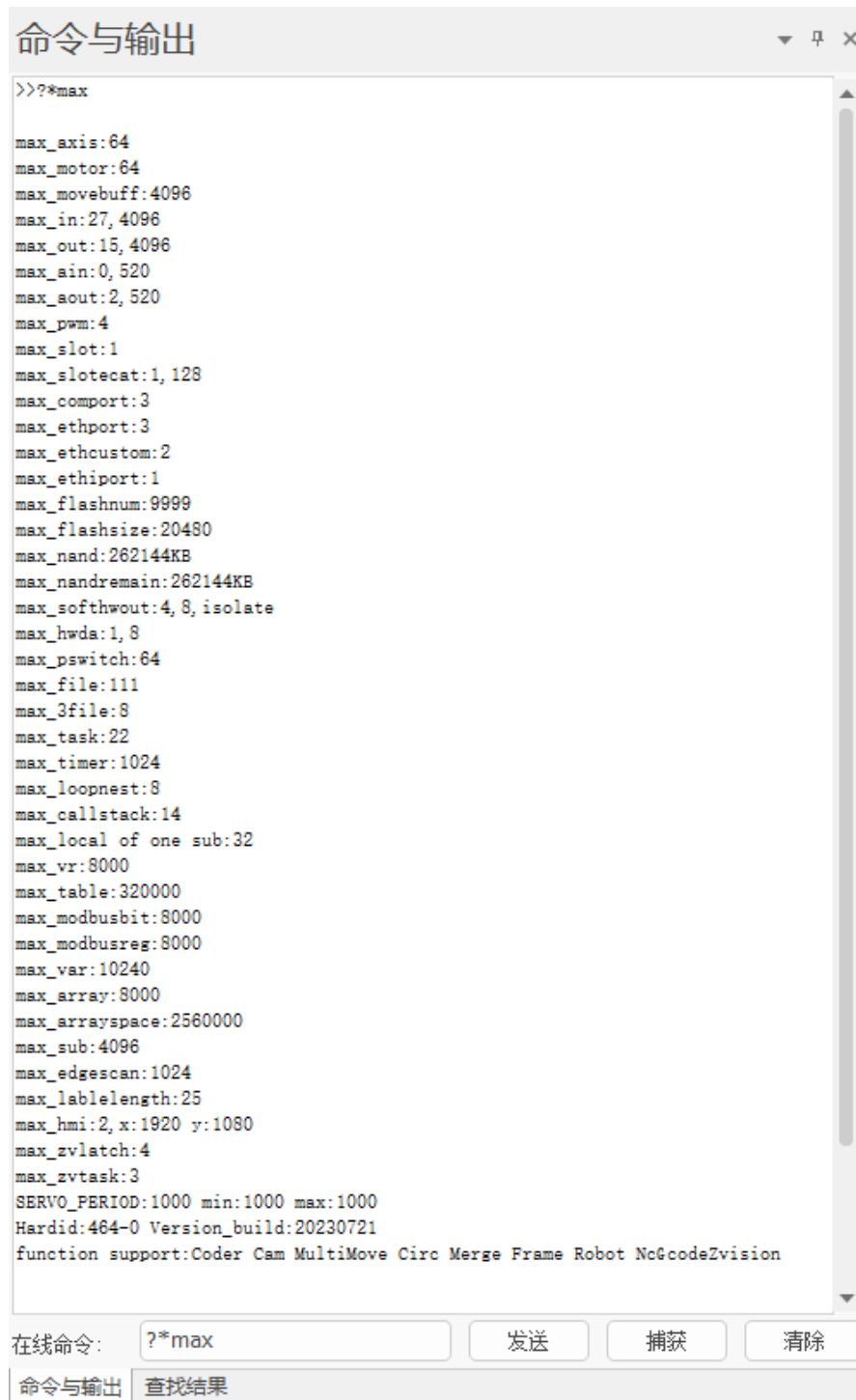
?*FRAME: 打印机械手参数，需要 161022 及以上固件支持

?*SLOT: 打印出控制器槽位口信息（RTEX 口，EtherCAT 口）

?*PORT: 打印所有 PORT 通讯口

?*CYCLEUP: 打印周期上报通道是否使能

连接控制器后，使用?*max 打印控制器所有规格参数结果如下：



修改变量的值。通过“在线命令”可以实现 VR 变量、TABLE 变量、MODBUS 变量、全局变量、系统设置、轴参数、轴状态变量的设置与修改。下图以修改 VR 变量值为例。

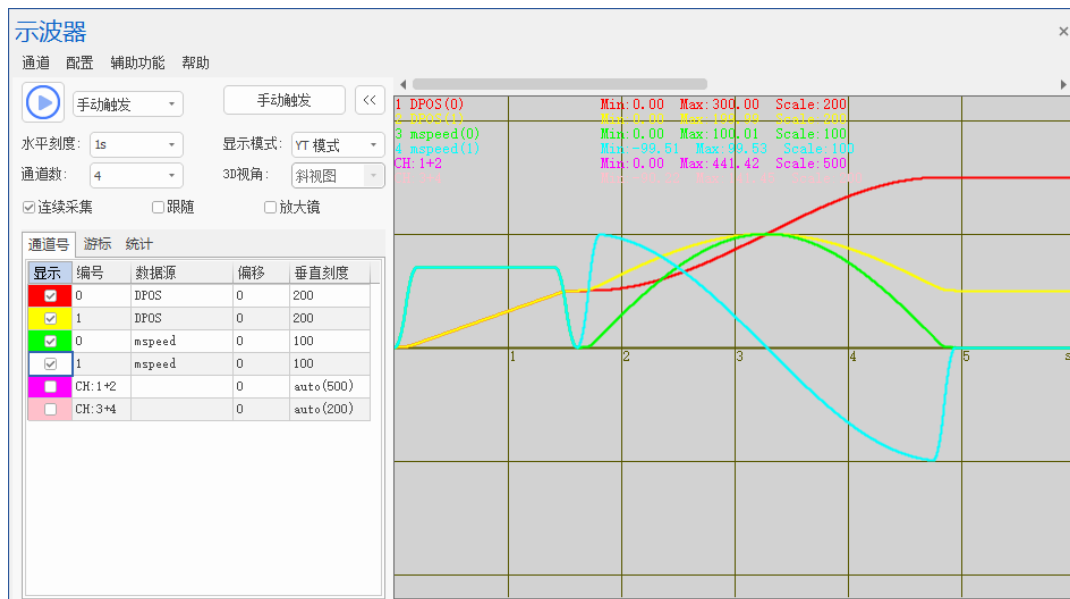


2.4 示波器的使用

示波器面板介绍

示波器属于程序调试与运行中极其重要的一个部分，用于把肉眼看不到的信号转换成图形，便于研究各种信号变化过程。示波器利用控制器内部处理的数据，把数据显示成图形，利用示波器可以显示各种不同的信号，如轴参数、轴状态等。在“工具”→“示波器”中，点击图标，打开示波器窗口。

其中，示波器显示区域中横轴表示时间，时间单位取决于水平刻度、控制器周期和间隔大小，计算公式为：单位时间=水平刻度×间隔×控制器周期（注：单位时间即示波器横轴每一格的时间），例：若水平刻度设置为 1000，间隔设置为 2，控制器周期为 1000us，则每一格的时间等于 2000ms；纵轴则根据数据源类型不同，单位则不同。



在 RTSys 编写好程序后，成功连接到控制器/仿真器后，打开示波器，设置好所需要的数据源及对应编号，选择自动触发/手动触发，点击“”启动按钮，再将程序下载至 RAM/ROM，即可采样。按以上操作选择自动触发时点击启动按钮后立即触发。选择手动触发时，需在点击“”后，点击“手动触发”按钮后下载至 RAM/ROM，或点击“”后直接下载至 RAM/ROM 后等待 Basic 程序触发，才可成功采样（注：等待 Basic 程序触发时，程序中需加入“TRIGGER”指令）。

示波器基础设置按钮功能：

按钮	功能
通道	选择的通道及叠加通道，对比通道不显示。
配置	选择进行示波器参数配置、观察器配置、数据源设计及配置的导入导出。
辅助功能	辅助观察波形，包括搜索波形、对比波形及导入导出波形。
帮助	显示鼠标操作指南界面，提示各个模式下鼠标的快捷操作内容。
	示波器启动开关。启动状态下显示为  ，但不触发示波器采样。
触发模式	下拉中可选择自动触发或手动触发。选择自动触发时手动触发按钮不可用。自动触发在用户点击启动按钮后立即触发；选择手动触发时，需在点击“  ”后，点击“手动触发”按钮后下载至 RAM/ROM，或点击“  ”后直接下载至 RAM/ROM 后等待 Basic 程序触发，才可成功采样（注：等待 Basic 程序触发时，程序中需加入“TRIGGER”指令）。
手动触发	手动触发示波器采样按钮。
<<	按下隐藏通道名称和峰值，只显示通道编号。
水平刻度	横轴一格的刻度。下拉菜单选择，可手动输入数值和单位，默认输入单位为 ms，输入完成后自动转化为 s。将鼠标放置数值框，滚动鼠标可以缩放水平刻度。YT 模式有效，XYZ 模式和 XYZD 模式下变为灵敏度，表示鼠标左键操作的灵敏度。
显示模式	有四种模式可切换，包括 YT 模式、XY 模式、XYZ 模式和 XYZD 模式。通道数设置小于 2 时不可用 XY/XYZ/XYZD 模式；小于 3 时不可用 XYZ/XYZD 模式；小于 4 时不可用 XYZD 模式。
YT 模式	不同数据源随时间变化的曲线，每一个通道显示一条波形。
XY 模式	XY 平面显示两个轴的插补合成轨迹，连续两个同类型通道为一组显示一条波形。
XYZ 模式	XYZ 三维空间显示合成轨迹。依次选择通道作为 X、Y、Z 轴，三个同类型通道为一组显示一条波形(通道类型包括：常规通道、叠加通道、对比常规通道和对比叠加通道)，每种类型最多显示一条波形。
XYZD 模式	XYZD 四通道可视化显示轨迹，其中 XYZ 为三维空间合成轨迹显示，D 为当前显示圆点的直径大小。计算方式为：圆点直径大小=当前 D 数值 ÷ D 基准数值 × D 基准大小。参数修改位于“观察器配置”窗口。依次选择通道作为 X、Y、Z 轴和 D 数值取值通道，四个同类型通道为一组显示一条波形(通道类型包括：常规通道、叠加通道、对比常规通道和对比叠加通道)，每种类型最多显示一条波形。
通道数	设置要采样的常规通道总数，启动状态下不可修改。当设置通道数大于控制器支持通道数时，会弹出提示信息：超过控制器支持的最大通道数。
3D 视角	可选斜视角、正视角、左视角和俯视角，默认斜视角。XYZ 模式和 XYZD 模式有效。
连续采集	不开启连续采集时，到达采样最大采集周期数后便停止采样；开启连续采集之后示波器会持续采样，到达采样最大采集周期数后仍继续采样，忽略设置的采样最大采集周期数，直到按下停止按钮后才会停止采样，采集的数据自动重新覆盖之前的数据。连续采集的所有波形采样数据均能导出。（使用串口时自动取消连续采集功能）
跟随	开启跟随后，横轴自动移动到实时采样处，跟随波形显示。
放大镜	勾选放大镜，鼠标移动到显示区域后自动在鼠标的右下方显示放大视图，放大视图跟随鼠标移动及刷新。放大镜参数修改位于“观察器配置”窗口。YT 模式有效。
显示	选择当前通道曲线是否显示。示波器有四种类型的通道，包含：常规通道 1~8，叠加通道 1~4，对比波形的常规通道 1~8 和对比波形的叠加通道 1~4。
编号	选择需要采集的数据源编号，如：轴号、数字量 IO 编号，模拟量 IO 编号、TABLE 编号、VR 编号、MODBUS 编号等。编号设置范围为 0 到控制器的最大轴数，可手动输入编号。
数据源	选择采集的数据类型。点击鼠标左键可手动输入数据类型，或点击  下拉菜单选择类型参

	数。可以在“数据源设计”窗口设置需要的参数类型。
偏移	波形纵轴偏移量设置，下拉菜单选择偏移量，可手动输入。
垂直刻度	纵轴一格的刻度。选择 auto 时表示自动刻度，示波器停止时可用，刻度值根据当前采集到的波形运动幅度自动变化，使波形能够完整的显示在当前示波器界面上。
注意：若要设置示波器参数，如轴编号、数据源以及示波器“参数配置”窗口，要先停止示波器再设置。	

示波器配置

(一) 参数配置窗口

点击示波器上方菜单栏“配置”按钮，点击“参数配置”，弹出如下所示“参数配置”窗口。

参数配置

基本参数

采集周期(us)

1000

间隔周期数

1

最大采集周期数

10000

自动使用TABLE数组末尾

True

Table位置

240000

导出参数

True

叠加参数

通道数

4

叠加的第一个通道(1)

1

叠加的第二个通道(1)

2

叠加方式(1)

相加

叠加的第一个通道(2)

3

叠加的第二个通道(2)

4

叠加方式(2)

相加

叠加的第一个通道(3)

5

叠加的第二个通道(3)

6

叠加方式(3)

相加

叠加的第一个通道(4)

7

叠加的第二个通道(4)

8

叠加方式(4)

相加

统计参数

显示最大值

True

默认

确定

取消

参数定义如下：

参数	描述
基本参数	
采样周期(us)	示波器两次采样之间的时间间隔。不可修改。
间隔周期数	采样时间间隔，单位为系统周期，与控制器固件版本有关，一般默认 1ms，指令 SERVO_PERIOD 查看。（例：间隔周期数设置为 1，则表示 1 个周期采样一次；间隔周期数设置为 5，则表示 5 个周期采样一次；周期时间取决于控制器固件版本）一般来说，间隔周期数越小，采样数据越准确，单位时间内数据量越大。

最大采集周期数	总共采样的数据次数，数值越大采样范围越大。（即：一个通道采集的数据所需使用的 TABLE 大小）
自动使用 TABLE 数组末尾	抓取数据存放的位置，默认为 True。
TABLE 位置	设置抓取数据存放的位置。一般默认自动使用 TABLE 数据末尾空间，当“自动使用 TABLE 数组末尾”设置为 False 时可以自定义设置，但是设置时注意不要与程序使用的 TABLE 数据区域重合。 查看控制器 TABLE 空间大小有以下三种方法： a.使用 TSIZE 指令读取； b.在“控制器状态”窗口查看； c.在线命令?*max 打印查看。
导出参数	需要导出示波器通道参数信息时选择，勾选后在导出波形时导出示波器参数，包括：基本参数、叠加参数和通道配置参数(编号、数据源、偏移、垂直刻度)。默认 True。
叠加参数	
通道数	选择叠加的通道数量，下拉菜单选择。
叠加的第一/二个通道	用户选择进行叠加的通道号。
叠加方式	设置两条通道之间的叠加方式，下拉菜单选择相加或相减两种叠加方式。
统计参数	
统计参数	设置示波器统计页面显示的参数信息，默认为 True。

(二) 观察器配置窗口

点击示波器上方菜单栏“配置”按钮，点击“观察器配置”，弹出如下所示“观察器配置”窗口，配置完成后，点击“应用”按钮可预览修改后的效果，最后点击“确定”修改成功。



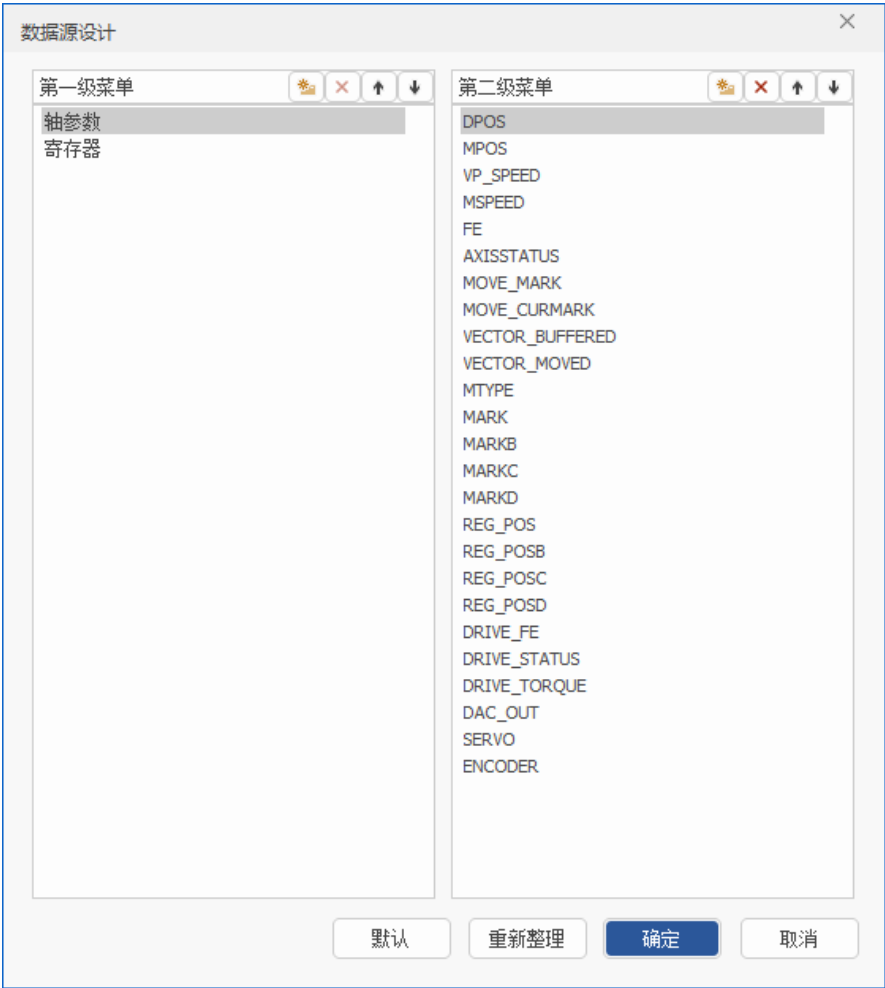
参数定义如下：

参数	作用
背景颜色/网格颜色/ 光标颜色/坐标颜色	设置对应的颜色。
网格线类型	设置波形显示界面网格线类型，可选择实线或虚线。
光标线类型	设置光标线显示类型，可选择实线或虚线。
通道线类型	三种形式可选，包含点、实线、虚线。点显示时示波器按固定周期采集得出一系列采样点的数据，通道可设置参数：点大小；实线和虚线显示时将采样点连成平滑的线段显示，线段更容易发现异常点的数据显示，通道可设置参数：线宽。
字体/字体大小	设置波形显示界面上通道编号、通道名称和峰值的字体和字体大小。
常规通道/叠加通道/ 对比通道/对比叠加 通道	设置对应通道的线宽、点大小和通道颜色。
D 基准数值/ D 基准 大小	用于计算 XYZD 模式下圆点直径大小，直径大小与 D 基准大小/D 基准数值的比值有关，比值越大，圆点直径越大。计算公式为：圆点直径大小=当前 D 数值 ÷ D 基准数值 × D 基准大小。(当前 D 数值为“D 每组取值”的数值)
D 每组点数	每 N 个采集点显示一个圆点。(例如“D 每组点数”设置为 100，即在每 100 个采集点依照“D 每组取值”的取值方式显示一个圆点)

D 每组取值	当前显示圆点尺寸大小在 N 个采集点中的取值方式，可选最大值、最小值和平均值。(例如“D 每组取值”设置为最大值，“D 每组点数”设置为 100，则将每 100 的采集点中的最大值作为计算当前显示圆点直径的大小的依据)
放大镜	设置放大镜放大界面的宽、高和放大倍数。
搜索	设置搜索波形时搜索结果显示的线宽、点大小和通道颜色。

(三) 数据源设计窗口

点击示波器上方菜单栏“配置”按钮，点击“数据源设计”，弹出如下所示“数据源设计”窗口。



按钮功能如下：

按钮	功能
第一/二级菜单栏	数据源选择的菜单栏。当第二级菜单有内容时，则一级菜单的文本为类别，二级菜单的文本为数据源；当二级菜单无内容时，则一级菜单的文本为数据源。
	新增按钮，在一级菜单或二级菜单新增一项内容。
	删除按钮，删除选中一级菜单或二级菜单的指定项。第一级菜单栏的轴参数和寄存器默认不可修改。
	上移/下移按钮，选中菜单栏的一项，用于排序。
重新整理	一级菜单和二级菜单的项按照字符(A~Z)顺序进行排序。

示波器数据导入导出

导入：示波器必须在停止状态下才能导入数据，导入成功能将采样波形复现出来。

导入采样数据方法：点击“导入”，选择导入的数据文件为之前从示波器导出的文件类型后打开即可。

导出：导出参数包括示波器参数设置情况，以及各个通道的数据类型和每个采样点数据。

导出采样数据方法：先在设置里勾选“导出参数”，启动示波器采样，采样完成后点击“导出”，选择文件夹保存示波器数据，导出数据为文本文件。

示波器采样方法

1. 打开工程项目，连接控制器或仿真器，再打开示波器窗口（操作示波器窗口之前需要连接到控制器或仿真器才可以操作）。

2. 在示波器窗口点击“设置”，选择采样通道数，采样深度，采样间隔，采样数据 TABLE 存储位置（一般来说自动使用 TABLE 数组末尾空间即可）和采样类型等，设置完成确认保存当前设置。

3. 再选择采样数据编号和数据源，点击“启动”按钮。

4. 将程序下载到控制器运行，程序里需要包含 TRIGGER 自动触发示波器采样指令，此时示波器开始采样，显示出不同数据源的波形。可调整显示刻度和波形偏移，便于观察不同波形。

若波形精度不高或显示不完整，可点击“停止”按钮后再打开“设置”，调整好采样间隔和采样深度后重新执行上述采样过程。

若需要采样的时间较长，开启“连续采集”功能。

示波器使用注意事项

示波器采样时间计算

例如深度：10000，间隔：5

如果系统周期 $SERVO_PERIOD=1000$ ，也就是 1ms 轨迹规划周期，间隔 5 表示每 5ms 采集一个数据点，一共采集 10000 次数据，采集时间长度为 50s。

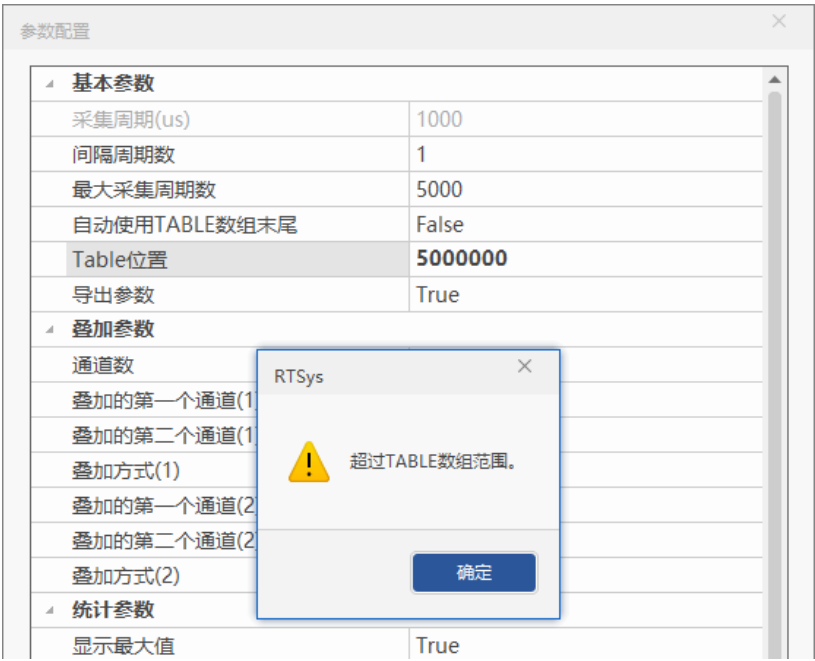
TABLE 数据末尾存储空间计算：

设置抓取数据存放的位置，一般默认自动使用 TABLE 数据末尾空间，此时根据采样数据占用空间大小自动计算起始空间地址。

计算方法：采样数据占用空间大小=通道数*深度

例：若控制器的 TABLE 空间大小为 320000，采样 4 个通道，深度为 30000，每个采样点占用一个 TABLE，所以会占用 $4*30000=120000$ 个 TABLE 位置， $320000-120000=200000$ ，此时 TABLE 的起始位置为 200000。

数据存放的位置也可以自定义配置，若按上面的通道数和深度，起始 TABLE 空间自定义时不能超过 200000，否则无法设置，如下图。



示波器采样数据占用的空间不要与程序使用的 TABLE 数据区域重合。

控制器 TABLE 空间大小可使用 TSIZE 指令读取、在“控制器状态”窗口查看或在线命令?*max 打印查看。

连续采集功能：

不选择连续采集时，到达采样深度后示波器自动停止采样。

在示波器“设置”里勾选连续采集，再开启示波器，示波器触发采样后会持续采样，到达采样深度后仍继续采样，忽略设置的采样深度，直到按下停止才会停止采样。

连续采集的所有波形采样数据均能导出。

示波器使用例程

例一：连续轨迹前瞻应用

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

BASE(0,1)

DPOS=0,0

ATYPE=1,1

UNITS=100,100

SPEED=100,100

ACCEL=1000,1000

DECEL=1000,1000

SRAMP=100,100

MERGE=ON

CORNER_MODE=2

'启动拐角减速

DECEL_ANGLE = 15 * (PI/180)

'设置开始减速角度

STOP_ANGLE = 45 * (PI/180)

'设置结束减速角度

FORCE_SPEED=100

'等比减速时起作用

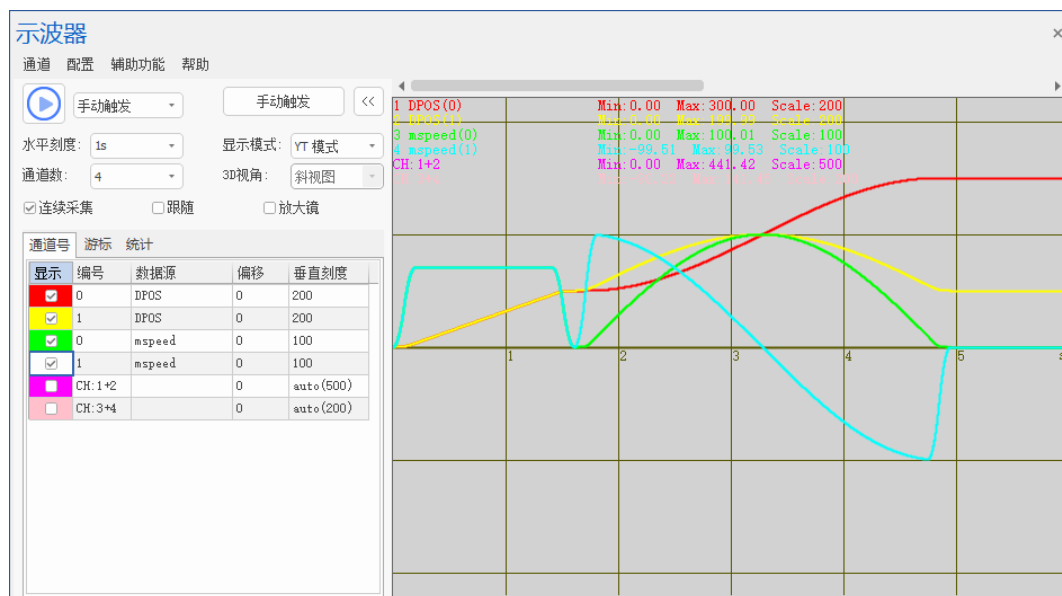
TRIGGER

'自动触发示波器

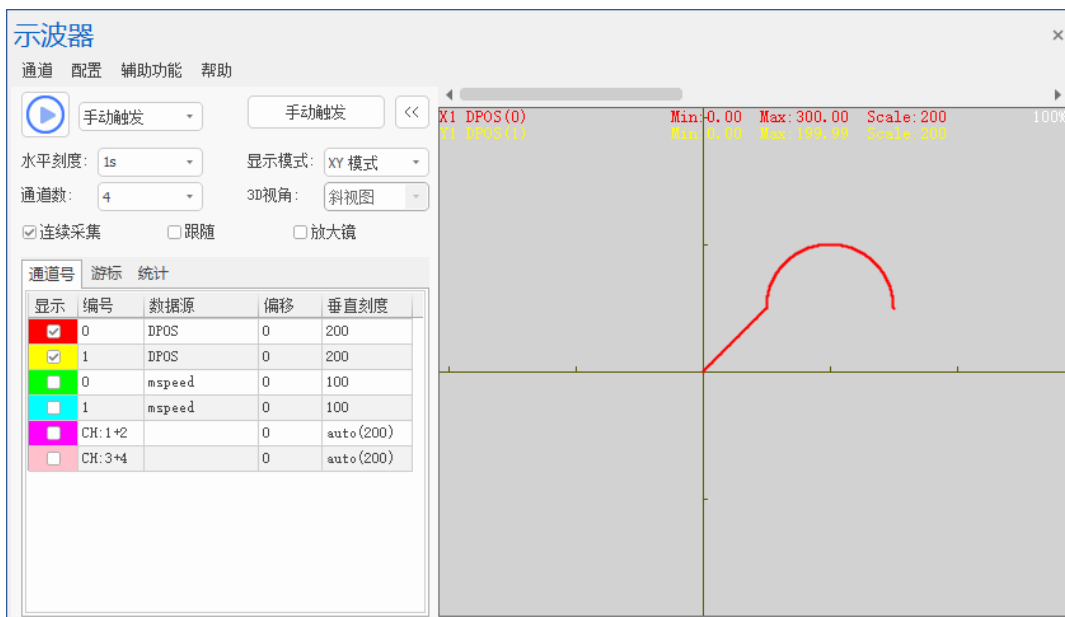
MOVE(100,100)

MOVECIRC(200,0,100,0,1) '半径 100 顺时针画半圆，终点坐标 (300,100)

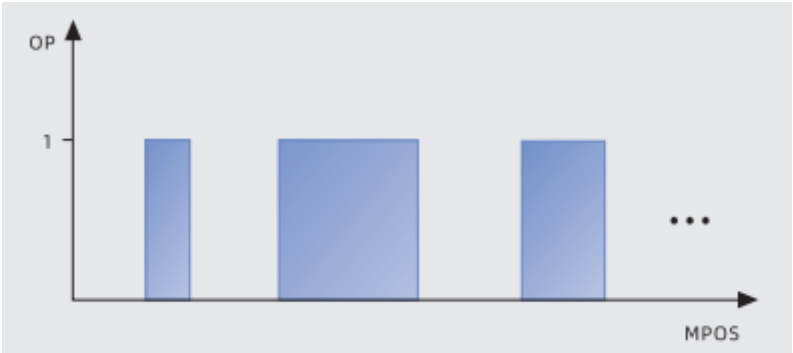
示波器采样轴 0 和轴 1 的速度和位置曲线：



XY 模式下的二轴插补合成轨迹：

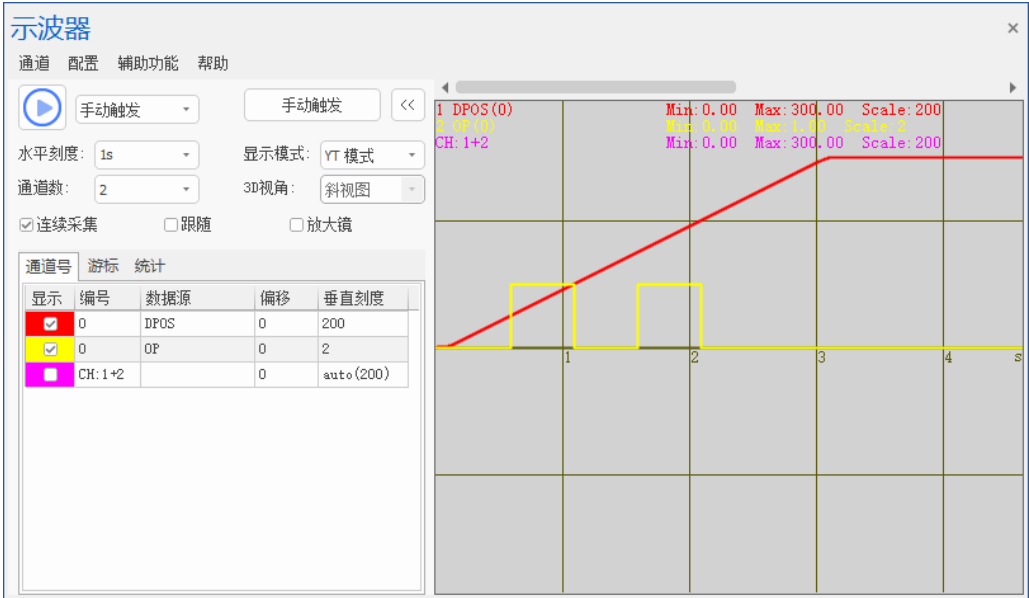


例二：PSO 位置同步输出,到达比较点输出 OP 信号。



```
RAPIDSTOP(2)
WAIT IDLE(0)

BASE(0)
DPOS=0
MPOS=0
ATYPE=1
UNITS=100
SPEED=100
ACCEL=1000
DECEL=1000
OP(0,OFF)
TABLE(0,50,100,150,200)      '比较点坐标
HW_PSWITCH2(2)                '停止并删除没有完成的比较点
HW_PSWITCH2(1, 0, 1, 0, 3,1)  '比较 4 个点，操作输出口 0
TRIGGER                        '自动触发示波器采样
MOVE(300)
```



例三：电子凸轮的应用
RAPIDSTOP(2)

WAIT IDLE(0)

BASE(0) '选择第 0 轴

ATYPE=1 '脉冲方式步进或伺服

DPOS = 0

UNITS = 100 '脉冲当量

SPEED = 200

ACCEL = 2000

DECEL = 2000

'计算 TABLE 的数据

DIM deg, rad, x, stepdeg

stepdeg = 2 '可以通过这个来修改段数，段数越多速度越平稳

FOR deg=0 TO 360 STEP stepdeg

rad = deg * 2 * PI/360 '转换为弧度

X = deg * 25 + 10000 * (1-COS(rad)) '计算每小段位移

TABLE(deg/stepdeg,X) '存储 TABLE

TRACE deg/stepdeg,X

NEXT deg

TRIGGER '触发示波器采样

WHILE 1 '循环运动

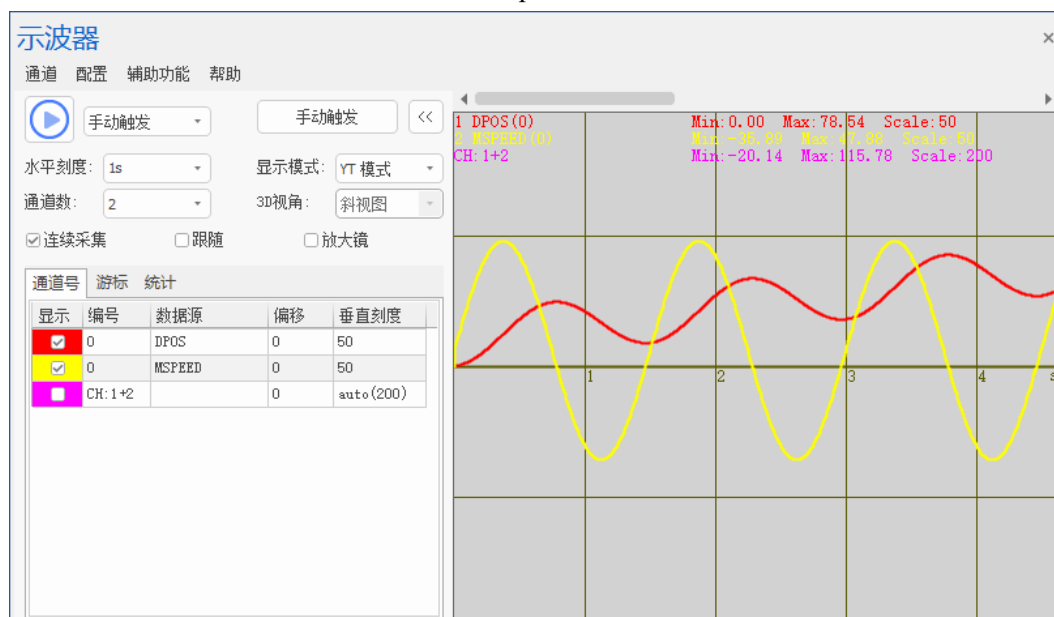
CAM(0, 360/stepdeg, 0.1, 300) '虚拟跟踪总长度 300

WAIT UNTIL IDLE '等待运动停止

WEND

END

运动轨迹：每个凸轮指令运动总时间=distance/speed=300/200=1.5s



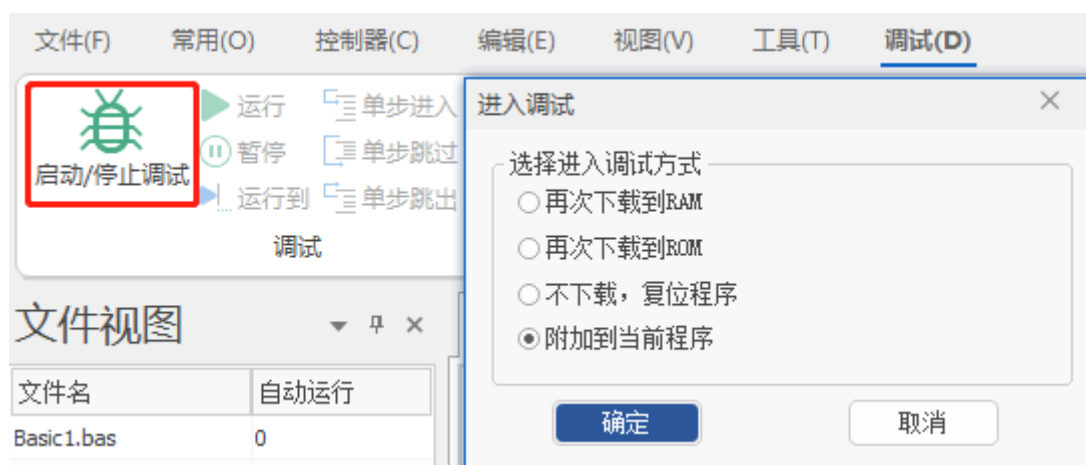
2.5 程序调试

进入程序调试

 **调试机器要注意安全！请务必在机器中设计有效的安全保护装置，并在软件中加入出错处理程序，否则所造成的损失，正运动技术公司没有义务或责任对此负责。**

调试功能可以快速调试程序，查看程序中各任务的运行情况。

RTSys 连接控制器后，从菜单栏选择“调试”→“启动/停止调试”弹出下方窗口。



进入调试有以下四种方式：

再次下载到 RAM：表示程序再次下载到 RAM 运行，RAM 掉电不保存。

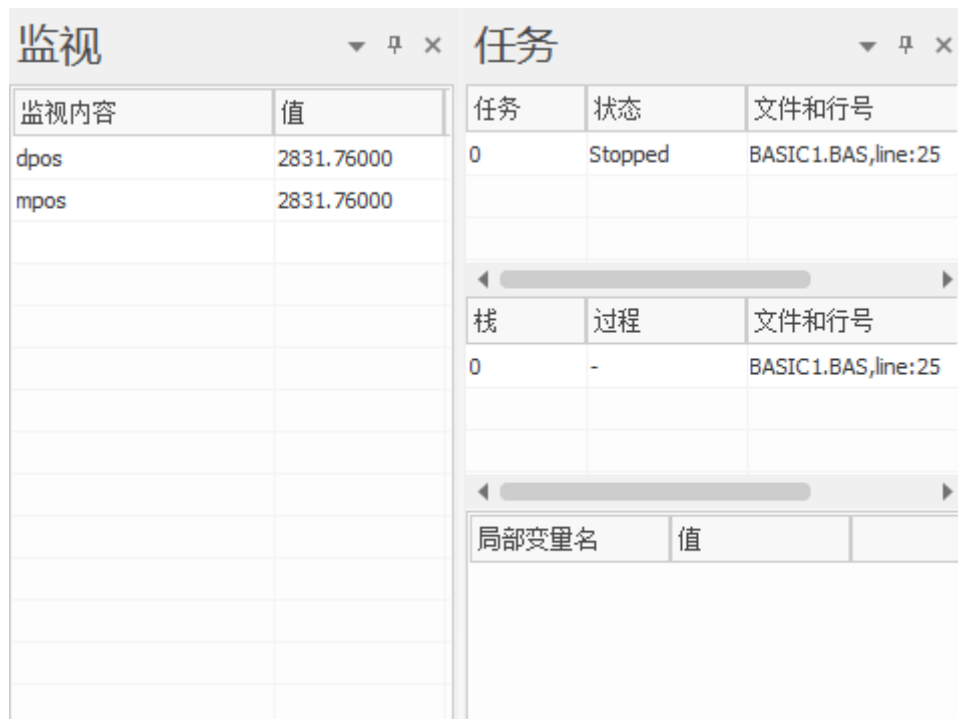
再次下载到 ROM：表示程序再次下载到 ROM 运行，ROM 掉电保存。

不下载，复位程序：表示不下载程序，重新运行之前下载的程序，并打开任务窗口显示目前的运行状态。

附加到当前程序：表示此时程序不下载，仅打开任务窗口显示目前的运行状态。

任务与监控窗口

选择进入调试的方式后，即可打开任务与监视窗口。



任务窗口用于查看任务的运行状态，任务所在的文件和任务运行行号。

可以把全局变量和文件模块变量等有效表达式加入到监视窗口，局部变量不支持，程序运行时自动获取参数值显示出来。也可以在调试状态下，在程序编辑区域选择变量后点击右键“增加到监视”加入到监视内容，或通过双击监视窗口内容名称来修改或增加监视项。

调试工具栏的使用

开启程序调试后，调试工具栏变得有效。



按从上到下、从左到右的顺序：

运行：开始自动运行，遇到断点暂停扫描，再按一下恢复扫描；

暂停：暂停运行；

运行到：运行到光标指定行；

单步进入(F11)：运行到程序里面，按一下向下扫描一行；

单步跳过(F10)：运行到下一条程序；

单步跳出：跳出 SUB 子程序运行；

断点：设置断点，按一下设置，在原位置再按一下取消；

紧急停止：强制停止所有程序运行。

当程序与控制器不一致或是对程序进行再修改后没有及时下载，会导致调试指定的行号产生偏移。

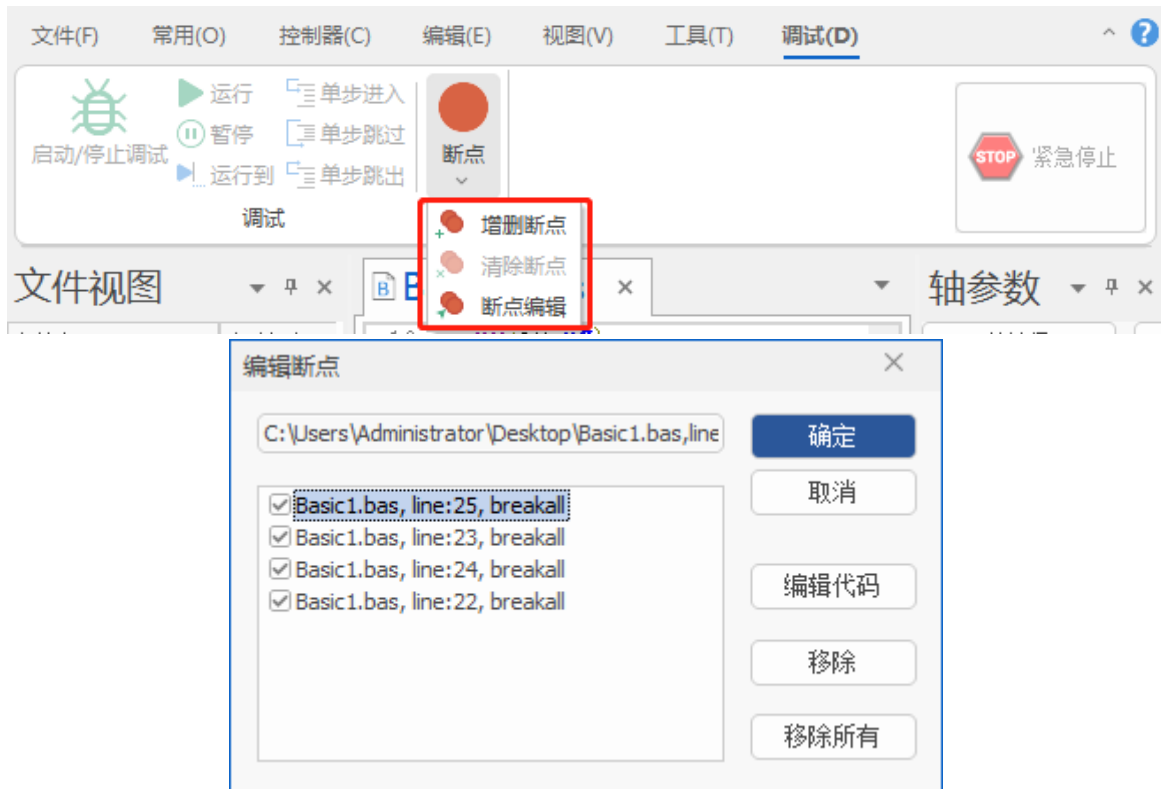
暂停时当前已经提交的运动并不会暂停。

断点调试

可以通过增加断点来捕获和暂停程序的运行。

断点调试可以查看程序运行的具体过程，主要用于判断程序逻辑错误。配合监视内容和轴参数变化情况可以查看程序每执行一步对寄存器、变量、数组等的影响。

断点可通过快捷键 F9 添加、也可以通过菜单栏“调试”→“断点”→“增删断点”，断点可以添加多个，菜单栏“调试”→“断点”→“清除断点”用于一次性清除项目文件中的所有断点。编辑断点窗口可快速移除目标断点或定位到断点处编辑代码。



程序停止在断点处后，就可以进行逐步调试，快捷键 F11，按一次程序向下执行一步。

如下图，调试的光标停在第 25 行，此时第 25 行的语句并未执行，第 24 行的语句已执行，按下 F11 一次才能执行第 25 行。



如果断点是设置在循环中，那么下次循环运行到断点处还是会停止程序。

程序调试完成后，需要清除所有断点再下载程序到控制器运行。否则打印信息提示 Warn

file:"BASIC1.BAS" line:25 task:0, Paused. 断点后的程序暂不扫描。

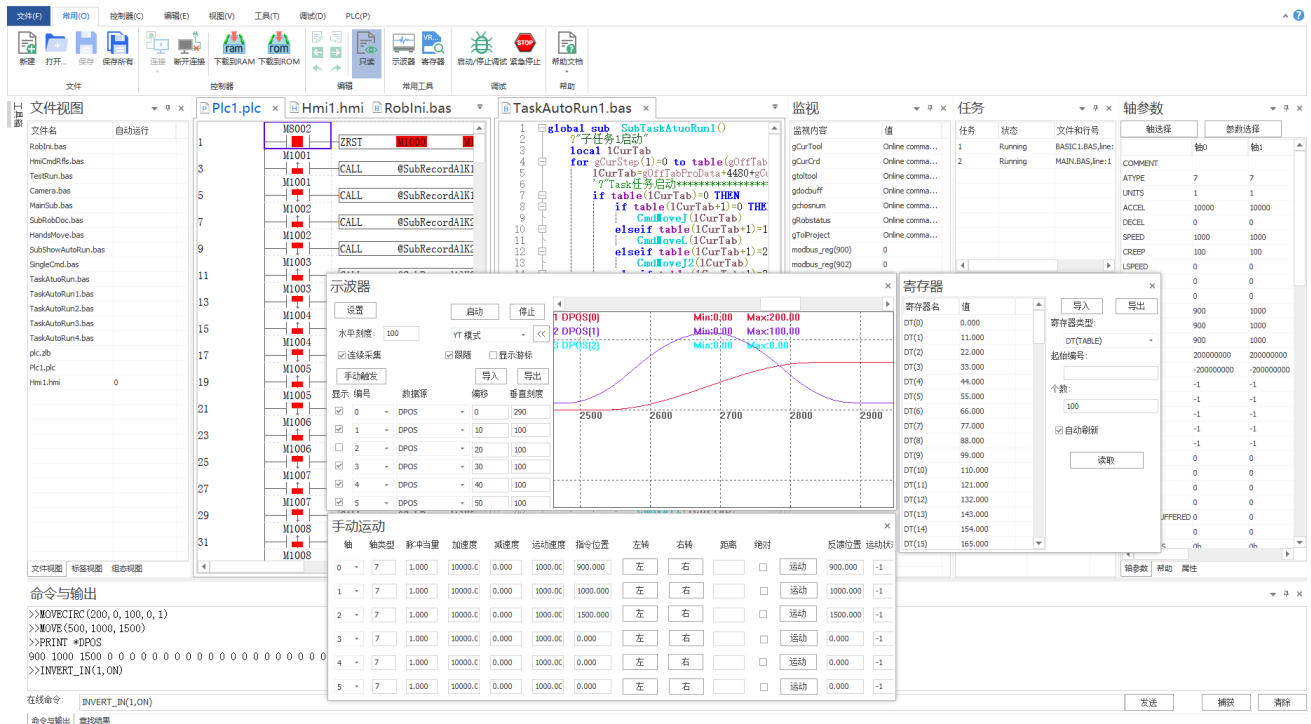
程序在运行途中出现 warn 警告，仍可以继续运行，程序下载后打印 ERROR 错误会停止运行。

2.6 视图窗口

RTSys 软件有多种视图窗口，用户能够很容易的对控制器进行程序编辑与配置，快速开发应用程序、实时监控轴运行参数以及对运动控制器正在运行的程序进行实时调试。

例如，轴参数窗口可以监控运动控制中常见的参数，可读写的轴参数在窗口内双击后直接修改，只读参数不支持修改；输入输出窗口监控 IO 的状态，手动运动窗口快速调试轴的运行状态。

更多视图窗口及其功能描述参见 RTSys 软件帮助菜单，打开“RTSys 使用手册”查看。



第三章 Basic 编程基础

本手册以 Basic 编程语言为例进行详细说明，用 PC 上位机编程的客户，获取更多资料请参考正运动“Zmotion PC 函数库编程手册”。

3.1 编程基础知识

程序

程序是由序列组成的，告诉计算机如何完成一个具体的任务。程序是软件开发人员根据用户需求开发的、用程序设计语言描述的适合计算机执行的指令（语句）序列。

RTBasic 语法不区分大小写，程序中指令的所有标点符号应为英文格式。

一个程序应该包括以下两方面的内容：

1. 对数据的描述。在程序中要指定数据的类型和数据的组织形式，即数据结构（参见：DIM、GLOBAL、CONST 等变量定义语句描述）。
2. 对操作的描述。即操作步骤，也就是算法，结合到运动控制就是运动与动作的工艺。

1. 常见程序结构

为编写算法，我们一般要用到以下几种程序结构描述方式：顺序、选择、循环、延时、等待和子程序调用，子程序调用参见下节。

1. 顺序

在没有条件和循环的情况下，程序总是从上往下运动。当设置自动运行时，文件缺省都是从文件开始顺序往下执行的。

功能块 1

功能块 2

如上，功能块 1 先执行，然后是功能块 2

BASIC 编程下，程序从上往下扫描一次。

PLC 编程下，程序从上往下周期扫描。

2. 选择

根据执行条件的不同，选择不同的语句执行。主要的选择语句有：IF THEN，ON GOTO，ON GOSUB 等。

例程 1：

```
DIM aa
```

```
aa=1
```

```
IF aa=0 THEN
```

```
    语句 1
```

```
ELSEIF aa=1 THEN
```

```
    语句 2
```



```
ELSE
    语句 3
ENDIF
END
```

例程 2:

```
DIM a
a=100
ON a>10 GOTO label1
a=1000
END          '主程序结束

label1:
PRINT a
END          'goto 跳转无法 return 返回。
```

3. 循环

程序重复执行，则称为循环。主要的循环语句有：FOR NEXT，WHILE WEND，REPEAT UNTIL 等。

例程 1:

```
DIM a
a=0
FOR i = 1 TO 10 STEP 1
    a = a+1
    PRINT a
NEXT
END
```

例程 2:

```
DIM a
a=0
WHILE IN(1)=OFF '直到输入 1 有效，退出循环
    a=a+1
    PRINT a
    DELAY(1000)
WEND
END
```

4. 延时

程序遇到 DELAY 延时语句，会停止对应时间后，再继续向下执行。

例程:

```
PRINT 1
DELAY(2000) '延时 2000ms
PRINT 2     '打印 1 延时 2000ms 后打印 2
END
```

5. 等待

程序遇到 WAIT 等待语句时，会停止在此行，直到 WAIT 条件满足，才继续向下执行。

例程

BASE(0,1)

MOVE(100,100)

WAIT IDLE '等待当前插补运动结束

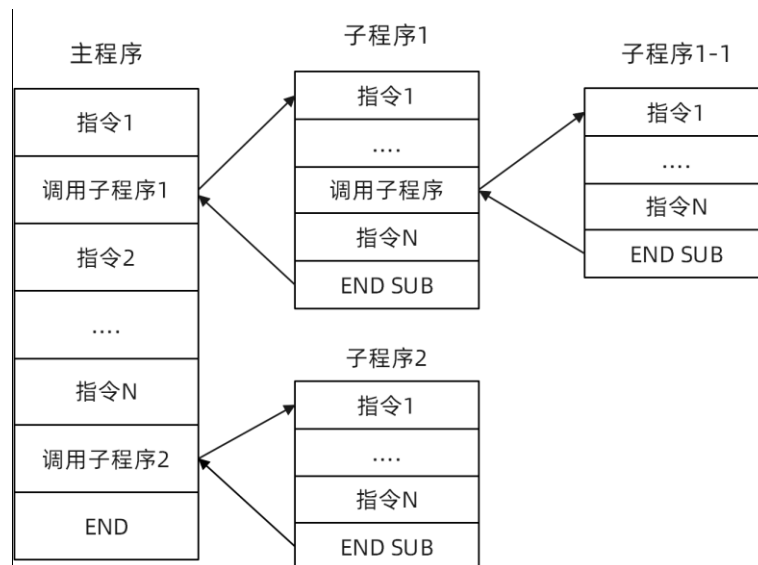
PRINT "运动完成"

程序除了在执行到 WAIT 等待语句，DELAY 延时语句时会阻塞，另外在扫描到运动指令时，如果轴的运动缓冲区满了，此时程序会停在当前运动指令行，直到当前运动完成，缓冲区空出一条，程序就会继续往下执行。缓冲区查看运动缓冲说明。

2. 子程序

编程过程中会经常应用到子程序（Basic 中使用 SUB 指令定义子程序），运用子程序可将编程模块化，各模块之间的关系尽可能简单，在功能上相对独立，相当于简化了主程序，使编程效率更高更易读，能有效地将一个较复杂的程序系统设计任务分解成许多易于控制和处理的子程序和子任务，便于开发和维护。

主程序和子程序的执行逻辑如下图：



SUB 子程序可作为一个子程序开启，运行完 END SUB 返回主程序，也可以使用任务指令 RUNTASK 开启，作为一个任务单独运行，任务开启后与主程序无关，运行完成后子程序任务结束，不返回主程序。

主程序调用子程序嵌套最多 8 层。

子程序分为全局 SUB，文件模块 SUB，全局 SUB 可以应用在所有文件中，文件模块 SUB 只能用于当前文件，子程序还能用于传递参数以及返回参数。

示例：

SUB sub1() '定义过程 SUB1，只能在当前文件中使用

 ?1

 ...

END SUB '自定义 SUB 过程结束

```

GLOBAL SUB g_sub2()      '定义全局过程 g_sub2，可以在任意文件中使用
    ?2
    ...
END SUB                  '自定义 SUB 过程结束

GLOBAL SUB g_sub3(para1,para2)      '定义全局过程 g_sub3，传递两个参数
    ?para1,para2
    ...
    RETURN para1+para2              '函数返回参数相加
END SUB                          '自定义 SUB 过程结束

```

数据相关

1. 数据定义

1. 变量定义

变量是用户可以自定义的参数，变量用于暂时保存与外部设备的通信数据或任务内部处理需要的数据，换言之，它是用于保存带名称和数据类型等属性的数据，无需指定变量与存储器地址之间的分配。

变量定义指令：分为全局变量（GLOBAL）、文件模块变量（DIM）、局部变量（LOCAL）三种。

全局变量（GLOBAL）：可以在项目内的任意文件中使用；

文件模块变量（DIM）：只能在本程序文件内部使用；

局部变量（LOCAL）：主要用在 SUB 中，其他文件无法使用。

变量可以不经定义直接赋值，此时的变量默认为文件模块变量。

示例：

```

GLOBAL g_var2            '定义全局变量 g_var2
DIM VAR1                 '定义文件变量 VAR1

SUB aaa()
    LOCAL v1              '定义局部变量 V1
    v1=100
END SUB

```

2. 常量定义

变量的值因代入该变量的数据而异。与之相对的固定不变的值为常数，常量的值一经定义不能再修改，只可读取。

CONST 定义常量，一次只能定义一个，且定义与赋值必须在一行。常量可定义为全局常量 GLOBAL CONST，全局常量可以在任意文件中使用，不存在 LOCAL CONST 的写法。常数与变量不同，不是保存在存储器中的信息，常见的常量有布尔型、字符串型、时间型、日期型、整型等。

示例：

```

CONST MAX_VALUE = 100000      '定义文件常量
GLOBAL CONST MAX_AXIS=6       '定义全局常量

```

3. 数组定义

数组是指将相同属性的数据集中后对其进行统一定义，并对数据个数进行指定。构成数组的各数据称为“元素”。

数组定义相关指令为 GLOBAL、DIM，不支持 LOCAL 定义。

数组定义时注意数组空间大小的指定，不能使用超出定义范围的空间，否则程序报错数组空间超限。

示例：

DIM array1(15) '定义文件数组，可使用的数组空间编号为 0~14，共 15 个空间

GLOBAL array2(10) '定义全局数组，可使用的数组空间编号为 0~9，共 10 个空间

?*max 查得最大数组空间 max_arrayspace 参数为自定义数组加 TABLE 之和，除去 TABLE 的空间才是自定义数组能够使用的最大空间数，最多可定义的数组个数由 max_array 参数确定。

2. 数据类型

在计算机内部，数据都是以二进制的形式存储和运算的，二进制数据中的位（bit）是计算机存储数据的最小单位。

一个二进制位只能表示 0 或 1 两种状态，要表示更多的信息，就要把多个位组合成一个整体，一般以 8 位二进制组成一个基本单位字节（Byte）。

字节是计算机数据处理的最基本单位，并主要以字节为单位解释信息。一般情况下，一个 ASCII 码占用一个字节，一个汉字国际码占用两个字节。计算机型号不同，其字长是不同的，常用的字长有 8、16、32 和 64 位。

单位转换：1Byte=8bit，1KB=1024B，1MB=1024KB，1GB=1024MB。

常见进制有二进制，八进制，十进制，十六进制。各种运动指令的参数默认为十进制数据。

名称	说明
位（Bit）	位为二进制数值之最基本单位，其状态非 1 即 0
位数（Nibble）	由连续的 4 个位所组成（如 bit3~bit0）一个位数可用以表示十进制数字 0~15 或十六进制 0~F
字节（Byte）	由连续之两个位数所组成（8 位，bit7~bit0）。可表示十进制数字 0~255 或十六进制的 00~FF
字符（Word）	由连续之两个字节所组成（16 位，bit15~bit0）可表示十进制数字 0~65535 或十六进制的 4 个位数值 0000~FFFF
双字符（Double Word）	由连续之两个字符所组成（32 位，bit31~bit0），可表示十进制数字 0~2 ³² -1 或十六进制的 8 个位数值 00000000~FFFFFFFF

数据类型是指对变量表示的值的形式和范围进行特定的规定。声明该变量时，数据类型的大小根据存储器内的数据范围大小而定，存储器内的数据范围越大，可表示的值的范围就越大。

基本数据类型	描述
布尔型	值为 0 或 1 的数据类型
整数型	值为整数的数据类型
实数型	值为实数的数据类型
日期型	以日期格式表示 DD: MM: YYYY
时间型	以时间格式表示 hh: mm: ss
字符串型	值为字符串的数据类型

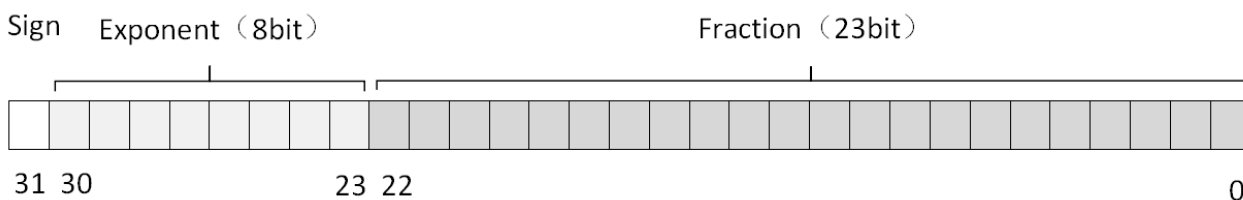
指令的输入或输出变量的数据类型由指令确定。

自定义变量的数据类型属于动态类型，将整数赋值给变量时，变量就是整型；将浮点数赋值给变量，变量就是浮点型。

自定义数组的数据类型分为单精度浮点数和双精度浮点数，参照下文浮点数相关说明。

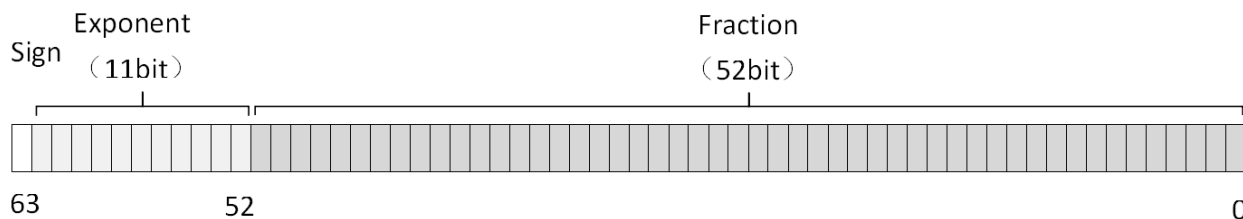
单精度浮点数 32 位，如下图。

数据格式属于单精度浮点数的有：VR、MODBUS_IEEE、TABLE 及自定义数组与变量（ZMC3 系列及之前的控制器）。



双精度浮点数 64 位，如下图。

数据格式属于双精度浮点数的有：TABLE 及自定义数组与变量（ZMC4 系列及之后的控制器）。



常用寄存器数据类型表：

寄存器类型	数据类型	取值范围
MODBUS_BIT	布尔型	0 或 1
MODBUS_REG	16 位整型	-32768 到 32767
MODBUS_LONG	32 位整型	-2147483648 到 2147483647
VR_INT		
MODBUS_IEEE	32 位浮点型	-3.4028235E+38 到 -1.401298E-45
VR		
TABLE ，自定义数组，变量（ZMC3 系列及之前）		
TABLE ，自定义数组，变量（ZMC4 系列及之后）	64 位浮点型	1.7E-308 到 1.7E+308
VRSTRING	字符	一个字符占一个 VR
MODBUS_STRING	字符	一个字符占 8 位

所有数据所需的存储器容量与各数据的总数据大小（容量值）不一致，原因在于，分配至存储器的数据的开头位置自动配置至各数据类型的“校准值（边界值）”倍数位置，各数据类型之间会产生空白。即使数据类型的种类相同，整体占用的数据大小仍会因数据类型的顺序而异。

3. 数据操作

不同类型数据之间的操作要注意数据类型，类型不匹配会产生下列问题：

1. 数据丢失：浮点型向整型转换时会丢失小数部分。

例程：

```
VR(0)=10.314
```

```
MODBUS_REG(0)=0
```

```
MODBUS_REG(0)=VR(0)
```

```
?MODBUS_REG(0)      '结果为 10
```

2. 强制转换：整型存储到浮点型寄存器后会变成浮点型，再使用整型操作数据可能会不正确。

3. 常见使用问题：获取日期时，不要使用单精度浮点型存储，因为日期格式是 8 位的，而单精度浮点数有效位只有 6 位，建议直接使用 32 位整型 MODBUS_LONG 来存储。

部分参数必须用字符串类型常量或变量，各种字符串可以通过“+”合并，对字符串的单个字节的操作需要利用数组来进行。

字符串相关指令列表：

字符串指令	描述
DIM	定义的数组可以直接作为字符串使用，每个元素表示一个字节
“ ”	双引号直接定义字符串常量
CHR	把 ASCII 转成一个字符串常量，此字符串只有一个字节
MODBUS_STRING	标准 MODBUS 协议定义字符串，每个 16 位寄存器存储 2 个字节
VRSTRING	VR 列表作为字符串使用，一个 VR 存储一个字节
+	操作符，两个字符串合并
VAL	数字字符串转换为数值
TOSTR	数值转换为字符串
STRCOMP	字符串比较函数
DMCPY	数组拷贝函数，可以用于拷贝字符串
HEX	返回为十六进制格式，只能用于打印
DATES\$	按 dd: mm: yyyy 格式返回 DATE 设置日期
DAYS\$	返回本日的星期英文名
TIMES\$	按 24 小时的格式 hh: mm: ss 返回当前时间

4. 参数

参数分轴参数、任务参数、系统参数等，参数可读可写（除少数只读参数外）。

在运动前要设置好轴参数（轴类型、脉冲当量、轴速度等），以及相关安全设置（正负硬件限位、正负软件限位、报警信号、急停减速度等）。

参数有自动保存和非自动保存两类。

1. 自动保存的参数修改后自动存储，重新上电不会恢复为缺省值，相关指令有：轴参数指令、[IP_ADDRESS](#)、[APP_PASS](#) 和 [LOCK](#) 等密码指令、[CANIO_ADDRESS](#) 等。

2. 非自动保存的参数上电后恢复为缺省值，要重新修改，比如 [SETCOM](#) 设置串口参数，每次上电后都要再设置一次，所以 [SETCOM](#) 指令要放在程序开头。

5. 掉电存储

控制器具有掉电非易失保护寄存器 VR 和多个扇区存储 FLASH 块。

RTSys 在线命令?FLASH_SECTES 查看 FLASH 扇区数量, ?FLASH_SECTSIZE 查看扇区大小, 可以用于保存掉电的数据。

[ONPOWEROFF](#) 掉电中断可以用编写的程序记录掉电时的位置到 VR, 系统重新上电时, 使用程序将 VR 的数据恢复到当前位置, 因为掉电执行时间非常短, 建议只存几个数据。

使用 SETCOM 指令可以把 VR 与 MODBUS_REG 寄存器匹配起来, 设置指令参数 variable, 详细设置参见 [SETCOM](#) 指令。

例程: 设置 variable=3, 将一个 VR_INT 映射到两个 MODBUS_REG 地址, 换算关系: $VR_INT(num) = MODBUS_REG(num) * 2^{16} + MODBUS_REG(num+1)$

```
SETCOM(38400,8,1,0,0,3)      '配置掉电存储
```

```
VR_INT(0)=0
```

```
MODBUS_REG(0)=1              '低 16 位值为 1
```

```
MODBUS_REG(1)=2              '高 16 位值为 2
```

```
?VR_INT(0)                    '结果: 131073
```

```
END
```

VR 是掉电非易失的, 可以无限次的读写, 数据保存时间为 10 年, 机器设备关键参数建议存储到 FLASH, FLASH 空间更大, 上电时从 FLASH 读出数据写至各变量中。

FLASH 是有写入寿命限制, 不可以无限次的擦写, 经常改写的的数据建议写入 VR。

3.2 RTSys 的三种编程方式

混合编程

RTSys 软件可以支持三种编程方式, 分别为 RTBasic、RTPlc 梯形图和 RTHmi 组态编程, 支持三种语言混合编程, 使用 RTSys 软件编写的程序可以下载到正运动控制器里。

RTBasic、RTPlc 和 RTHmi 之间可以多任务运行, RTBasic 可以多任务号运行, RTPlc 与 RTHmi 均只能一个任务号运行。

例如: 如下图, 同一项目内的两个不同的 BASIC 文件可以设置不同任务号分别运行, 同一项目内的 PLC/HMI 文件都只能有一个任务号。

文件名	自动运行	文件名	自动运行
Basic1.bas	0	Plc1.plc	0
Basic2.bas	1	Plc2.plc	

RTPlc 编程和 RTBasic 编程均具有简单易懂、逻辑结构清晰的特点, 能满足多种编程要求, 目前应用十分广泛。HMI 组态编程适用于正运动 ZHD 系列示教盒, 其他公司的示教盒也能连接到控制器上, 请使用该公司提供的示教盒编程软件。

PLC 与 BASIC 互相调用

PLC 与 BASIC 相关寄存器对应关系:

PLC	BASIC
-----	-------

输入继电器 X	X0~X7	输入口 IN	IN(0)~IN(7)
	X10~X17		IN(8)~IN(15)
	X20~X27		IN(16)~IN(23)
	...		...
	X1770~X1777		IN(1016)~IN(1023)
输出继电器 Y	Y0~Y7	输出口 OP	OP(0)~OP(7)
	Y10~Y17		OP(8)~OP(15)
	Y20~Y27		OP(16)~OP(23)
	...		...
	Y1770~Y1777		OP(1016)~OP(1023)
辅助继电器 M	M0	MODBUS_BIT	MODBUS_BIT(0)
	M1		MODBUS_BIT(1)
	...		...
	M1023		MODBUS_BIT(1023)
特殊继电器 D	D0	MODBUS_REG	MODBUS_REG(0)
	D1		MODBUS_REG(1)
	...		...
	D1023		MODBUS_REG(1023)
浮点寄存器 DT	DT0	TABLE	TABLE(0)
	DT1		TABLE(1)
	...		...
	DT1023		TABLE(1023)
EXE @BASIC 指令		BASIC 指令	

输入继电器 X 对应 [IN](#)，PLC 编程下，X 为 8 进制（X0~X7，X10~X17，...），而控制器的输入口 IN 为 10 进制，编写程序时要特别注意，做进制转换，比如 IN20 口对应 X24，IN8 对应 X10。

输出继电器 Y 对应 [OP](#)，PLC 编程下，Y 为 8 进制（Y0~Y7，Y10~Y17，...），而控制器的输出口 OUT 为 10 进制，编写程序时要特别注意，做进制转换，比如 OUT20 口对应 Y24，OUT8 对应 Y10。

辅助继电器 M 对应 [MODBUS_BIT](#)。

特殊继电器 D 对应 [MODBUS_REG](#)。

浮点寄存器 DT 对应 [TABLE](#)，可以用于与 RTBasic 之间传输数据。

PLC 里使用“EXE @BASIC 指令表达式”可调用 BASIC 指令。

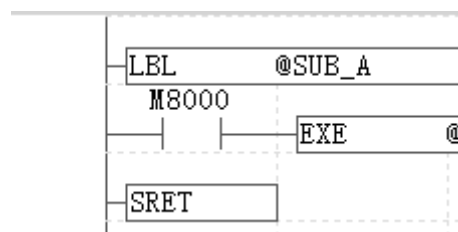
Basic 里可使用语句“RUN “xxx.plc”,任务编号”来启动 PLC 任务。

使用“CALL SUB_FUNC”或“RUNTASK SUB_FUNC”来调用 PLC 子程序 LBL。更多详细内容参见“ZMotion PLC 编程手册”。

```

1  'Basic开启任务调用Plc文件
2  RUN "Plc1.plc",6
3  'Basic开启任务调用Plc子函数
4  RUNTASK 2, SUB_A
5  END

```



3.3 寄存器

控制器寄存器主要有 TABLE、FLASH、VR、MODBUS 寄存器。将 RTSys 软件与控制器连接后，可通过 RTSys 软件“控制器”-“控制器状态”查看该控制器各寄存器的空间大小，也可以通过在线命令和输出窗口输入“?*max”来查看各寄存器的数量，不同的控制器存储空间大小不同。

TABLE

[TABLE](#) 是控制器自带的一个超大数组，数据类型为 32 位浮点型（4 系列及以上为 64 位浮点数），掉电不保存。编写程序时，TABLE 数组不需要再定义，可直接使用，索引下标从 0 开始。

RTBasic 的某些指令可以直接读取 TABLE 内的值作为参数，比如 CAM, CAMBOX, CONNFRAME, CONNREFRAME, MOVE_TURNABS, B_SPLINE, CAN, CRC16, DTSMOOTH, PITCHSET, HW_PSWITCH 等指令。

示波器采样的参数也存储在 TABLE 里。因此在开发应用中要注意多个 TABLE 区域的分配与使用，不要与示波器采样的数据存储区域重合。

1) [TABLE](#) 指令读写数据。

TABLE(0) = 10 'TABLE(0)赋值 10

TABLE(10,100,200,300) '批量赋值, TABLE(10)赋值 100, TABLE(11)赋值 200, TABLE(12)赋值 300

2) [TSIZE](#) 指令可读取 TABLE 空间大小，还可修改 TABLE 空间大小（不能超出 TABLE 最大空间）。

PRINT TSIZE '打印出控制器 TABLE 大小

TSIZE=10000 '设置 TABLE 的大小，不能超过控制器 TABLE 最大 SIZE

3) [TABLESTRING](#) 指令按照字符串格式打印 TABLE 里的数据。

TABLE(100,68,58,92)

PRINT TABLESTRING(100,3) '字符串格式打印数据，转换为 ASCII 码

打印结果: D:\

TABLE 作为参数传递时用法大致相同，以 CAM 凸轮指令为例：

CAM(start point, end point, table multiplier, distance)

start point: 起始点 TABLE 编号，存储第一个点的位置

end point: 结束点 TABLE 编号

table multiplier: 位置乘以这个比例，一般设为脉冲当量值

distance: 参考运动的距离

使用方法示例：

TABLE(10,0,80,75,40,50,20,50,0) 'TABLE 从 10 开始存数据，TABLE(10)赋值 0，TABLE(11)赋值 80

CAM(10,17,100,500) '运动轨迹为 TABLE(10)到 TABLE(17)

查看 TABLE 内数据的方式有 2 种：

第一种：在在线命令行输入?*TABLE(10,8)查询 TABLE(10)开始，依次 8 个数据。



第二种：在寄存器中查看 DT(TABLE)数据，起始编号从 10 开始，个数 10 个。



FLASH

严格来讲，FLASH 不是寄存器，但它与寄存器密切相关，所以放于此章叙述。

FLASH 具有掉电存储功能，读写次数限制为十万次，长期不上电也不会丢失数据。一般用于存放较大的，不需要频繁读写的数据，比如加工的工艺文件。

读与写时要注意保证要操作的变量，数组等名称和次序高度一致，如果不一致会导致数据错乱。

FLASH 使用时是按块编号，块数 [FLASH_SECTES](#) 指令查看，不同的控制器 FLASH 块数与块数据大小都不同，每块数据大小 [FLASH_SECTSIZE](#) 指令查看。

可以在在线命令行查看，如下图。



CAN 通讯设置的参数，IP 地址、APP_PASS、LOCK 密码等系统参数存储到 FLASH。

注意：FLASH 在读取之前先要写入，否则会提示警报 WARN。

FLASH 使用方法：

GLOBAL VAR '变量定义

GLOBAL ARRAY1(200) '数组定义

DIM ARRAY2(100)

'数据存储到 FLASH 块：把 VAR，ARRAY1，ARRAY2 数据依次写入 FLASH 块 1

FLASH_WRITE 1, VAR, ARRAY1, ARRAY2

'FLASH 块数据读取：把 FLASH 块 1 的数据依次读入 VAR，ARRAY1，ARRAY2

FLASH_READ 1, VAR, ARRAY1, ARRAY2 '读取次序与写入次序一致

VR

[VR](#) 寄存器具有掉电存储功能，可无限次读写，但数据空间较小，一般只有 1024 或者更少，最新系列控制器的 VR 空间为 8000，用于保存需要不断修改的数据，例如轴参数、坐标等，数据类型为 32 位浮点型。

可使用 [VR_INT](#) 强制保存为整型，[VRSTRING](#) 强制保存为字符串。VR、VR_INT、VRSTRING 共用一个空间，地址空间是重叠的，VR 和 VR_INT 读写方法相同，VRSTRING 保存 ASCII 码，一个字符占用一个 VR。

VR 的掉电保存原理是控制器内部有铁电存储器，但数据容量较小，所以数据量较大的或需要长久保存的数据最好写到 FLASH 块或导出到 U 盘。

VR 寄存器还可用于 RTEX 控制器传递读写数据，DRIVE_WRITE 参数写入，DRIVE_READ 参数读取，具体使用方法参见第十六章总线相关的 RTEX 总线指令。

使用 [CLEAR](#) 指令清除 VR 内的全部数据，[CLEAR_BIT](#) 指令将 VR 某个位置 0，[READ_BIT](#) 指令读取 VR 寄存器的某个位数据，[SET_BIT](#) 指令将 VR 某个位置 1。

例一：VR 使用方法

VR(0) = 10.58 '赋值

aaa = VR(0) '读取

例二：VR 寄存器数据相互转换

VR(100)=10.12

VR_INT(100)=VR(100) '数据转换

?VR_INT(100) '打印结果：10，浮点数转换成整数，丢失小数位

例三：VRSTRING 存储字符串

VRSTRING(0,4) = "abc" '从 VR(0)开始保存字符串

PRINT VRSTRING(0,4) '打印结果：abc

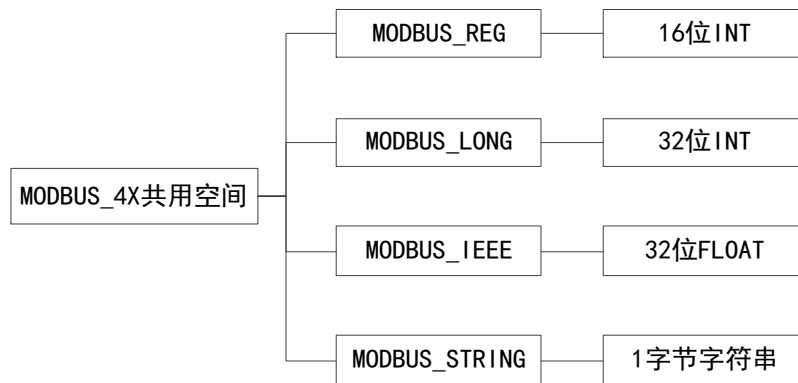


MODBUS

MODBUS 寄存器符合 MODBUS 标准通讯协议，分为位寄存器和字寄存器两类。MODBUS 寄存器的数据掉电不保存。

位寄存器：[MODBUS_BIT](#)，触摸屏一般称为 MODBUS_0X，布尔型。

字寄存器：[MODBUS_REG](#)、[MODBUS_LONG](#)、[MODBUS_IEEE](#)、[MODBUS_STRING](#)，触摸屏一般叫 MODBUS_4X，类型如下图。



控制器中 MODBUS 字寄存器占用同一个变量空间，其中一个 LONG 占用两个 REG 地址，一个 IEEE 也占用两个 REG 地址，使用时要注意错开字寄存器编号地址。

MODBUS_LONG(0) 占用 MODBUS_REG(0) 与 MODBUS_REG(1) 两个 REG 地址。

MODBUS_LONG(1) 占用 MODBUS_REG(1) 与 MODBUS_REG(2) 两个 REG 地址。

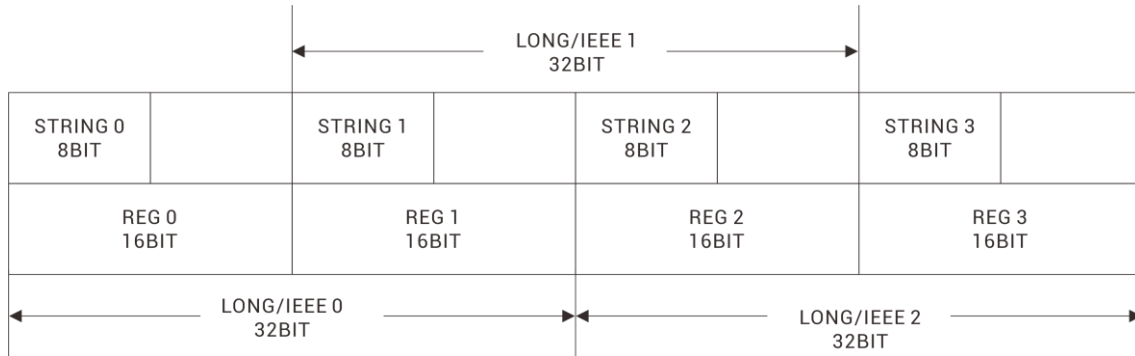
MODBUS_IEEE(0) 占用 MODBUS_REG(0) 与 MODBUS_REG(1) 两个 REG 地址。

MODBUS_IEEE(1) 占用 MODBUS_REG(1) 与 MODBUS_REG(2) 两个 REG 地址。

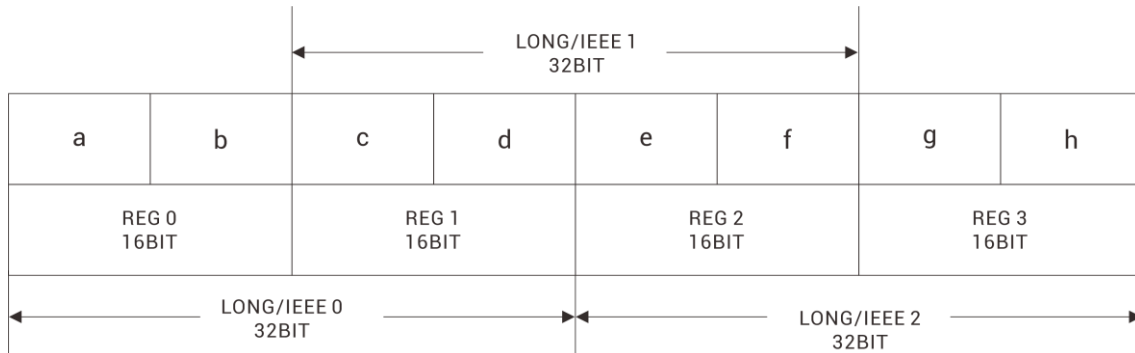
所以要注意 MODBUS_REG, MODBUS_LONG, MODBUS_IEEE 地址在用户应用程序中不能重叠。

计算方式：MODBUS_REG(1) 为高位，MODBUS_REG(0) 为低位， $\text{MODBUS_LONG}(0) = \text{MODBUS_REG}(1) * 2^{16} + \text{MODBUS_REG}(0)$

4X 空间示意图：



例如：MODBUS_STRING(0,20)="abcdefgh"的 4X 空间示意图如下：



例程：

```
MODBUS_REG(0)=0          '初始化置 0
MODBUS_REG(1)=0          '初始化置 0
MODBUS_LONG(0)=70000     'modbus_long 赋值 70000， modbus_reg 范围-32768~32767
?MODBUS_REG(0),MODBUS_REG(1)
'打印出 reg(0)为 4464， reg(1)为 1， long(0)=reg(1)*2^16+reg(0)
```

```
MODBUS_REG(0)=0
'初始化置0
MODBUS_REG(1)=0
'初始化置0
MODBUS_LONG(0)=70000
'modbus_long赋值70000
'modbus_reg范围-32768~32767
?MODBUS_REG(0),MODBUS_REG(1)
'打印出reg(0)为4464， reg(1)为1
'long(0)=reg(1)*2^16+reg(0)
```

监视内容	值
MODBUS_LONG(0)	70000
MODBUS_REG(0)	4464
MODBUS_REG(1)	1

在串口设置(SETCOM 参数)过程中，寄存器选择为 VR 时，此时一个 VR 映射到一个 MODBUS_REG，其中 VR 是 32 位浮点型，MODBUS_REG 是 16 位有符号整数型，从 VR 传递数据给 MODBUS_REG 会丢失小数部分，当 VR 数据超过正负 15 位时，MODBUS_REG 数据会改变；MODBUS_REG 传递数据给 VR 不会有问題，见如下例程，更多信息参见 SETCOM 指令。

例程：

```
VR(0)=0                  '初始化 VR(0)和 MODBUS_REG(0)为 0
MODBUS_REG(0)=0
SETCOM(38400, 8,1,0,0,4,0) '设置 VR 映射到 MODBUS_REG
VR(0)=100.345            '设置 VR(0)=100.345
```

?MODBUS_REG(0) '打印结果为 100, VR 已经映射到 REG, 但是 REG 是整型, 所以小数部分丢失

MODBUS_REG(0)=200 'REG(0)设为 200

?VR(0) '打印结果为 200, REG 变化也会改变 VR

当使用 MODBUS 协议与其他设备通讯时, 就需要将数据放在 MODBUS 寄存器内进行传递, 比如与触摸屏通讯。不进行 MODBUS 通讯时, 亦可将 MODBUS 寄存器作为控制器本地数组使用。

控制器直接从 MODBUS_BIT 地址 10000 开始与输入 IN 口对应, 20000 与输出 OUT 口对应 (注意读取的 IO 是原始的状态, INVERT_IN 反转输入指令不起作用), 30000 与 PLC 编程的 S 寄存器对应。

MODBUS_IEEE 地址 10000 开始对应轴 DPOS 区间, 11000 开始对应轴 MPOS 区间, 12000 开始对应轴 VP_SPEED 区间; MODBUS_REG 的 13000 开始对应模拟量 DA 输出区间, 14000 开始对应模拟量 AD 输入区间。

MODBUS_BIT 地址	意义
0~7999	用户自定义使用
8000~8099	PLC 编程的特殊 M 寄存器
8100~8199	轴 0-99 的 IDLE 标志
8200~8299	轴 0-99 的 BUFFER 剩余标志
10000~14095	对应输入 IN 口
20000~24095	对应输出 OUT 口
30000~34095	对应 PLC 编程的 S 寄存器

MODBUS 字寄存器地址	意义
0~7999	用户自定义使用, 可混用 MODBUS_REG、MODBUS_IEEE、MODBUS_LONG
8000~8099	PLC 编程的特殊 D 寄存器
10000~10198	对应各轴 DPOS, 读写用 MODBUS_IEEE
11000~11198	对应各轴 MPOS, 读写用 MODBUS_IEEE
12000~12198	对应各轴 VPSPEED, 读用 MODBUS_IEEE
13000~13127	模拟量输出 AOUT, 读写用 MODBUS_REG
14000~14255	模拟量输入 AIN, 读用 MODBUS_REG

3.4 多任务编程

多任务概念

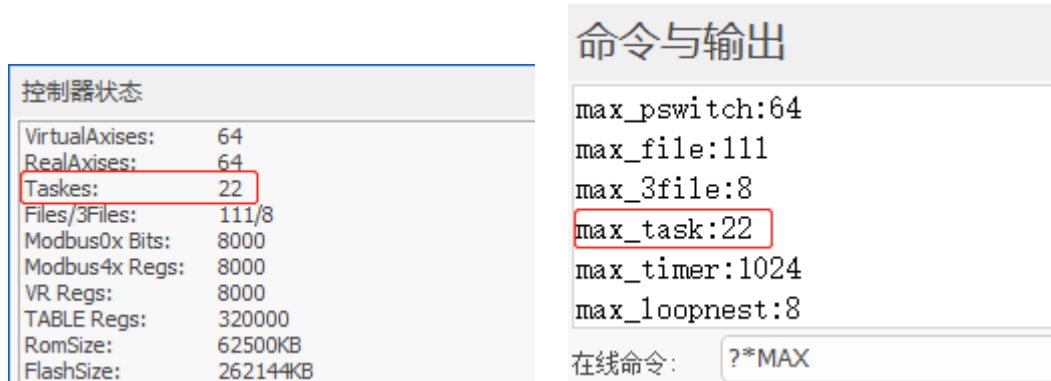
任务是执行 I/O 刷新和用户程序等一系列指令处理的功能, 一个任务是指一个正在运行的程序。

如果多个程序模块能够互不干扰的同时运行, 则称为多任务, 多任务编程在 RTSys 软件上实现。

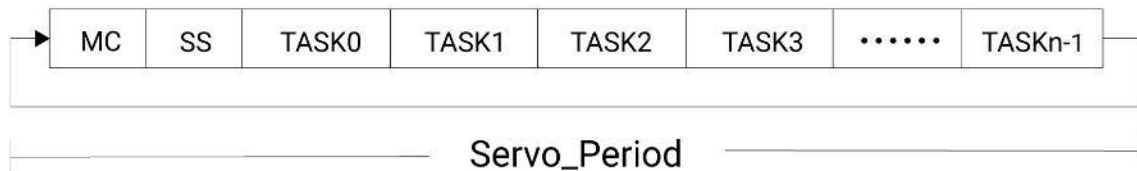
多任务可以将复杂的程序分成几个部分, 分别开任务来同时执行, 每个部分的任务是独立的, 这样就可以使设备的复杂运动过程变得简单明了, 编程更灵活, 没有多任务的场合程序只能顺序执行, 使得程序的执行效率十分低下。

ZMC 运动控制器支持多任务编程，每个任务都有自己唯一的编号，此编号没有优先级意义，只是标识当前程序属于哪一个任务。

不同型号支持的任务数有所不同，支持的具体任务数量，可连接控制器之后，在 RTSys 软件菜单栏“控制器状态”查看或在线命令发送?*max 指令查看，如下图，表示该控制器支持 22 个任务，任务编号范围为 0-21。



运动控制器每个运动控制周期(Servo Period)包含 MC、SS、以及用户多任务程序的运行，如下图所示：



MC: Motion Control、EtherCAT 通讯、中断的实现。Motion Control 包含：单轴运动控制、多轴插补运动、机械手正反解算法；EtherCAT 通讯包含 PDO 通讯与 SDO 通讯。

SS: System Service，包含 RS232 串口通讯，RS485 串口通讯，CAN 通讯，EtherNET 通讯（MODBUS RTU 主从通讯以及 RTSys 相关软件服务）

TASK0、…、TASKn-1：对应于各个任务的运行，第 1 个任务到第 n 个任务，任务编号从 0 开始。

在一个控制周期内，不同的任务根据当前执行的指令的差异，任务占用的时间也会有差异，并不完全相同，任务在默认情形下不存在优先级，可通过 PROC_PRIORITY 指令去设置某个任务的优先级。

Basic 的所有任务只扫描运行一次（除非程序内有死循环才会一直运行）。一个工程项目下 Basic 文件支持同时存在多个自动运行任务。

PLC 主任务循环扫描执行，PLC 子程序任务只运行一次。一个工程项目下 PLC 文件建议只设置一个自动运行任务号。

HMI 程序需要设置自动运行任务号，初始化函数只扫描执行一次，周期函数循环扫描。一个工程项目下 HMI 文件仅支持一个，组态程序要运行只能通过给 HMI 文件设置自动运行任务号。

文件视图		任务		
文件名	自动运行	任务	状态	文件和行号
Basic1.bas	0	0	Stopped	BASIC1.BAS,line:3
Basic2.bas	1	1	Stopped	BASIC2.BAS,line:1
Basic3.bas		2	Stopped	PLC1.PLC,line:3
Plc1.plc	2	3	Stopped	HMI1.HMI,line:1
Hmi1.hmi	3			

控制器同时处理四个任务，如上图，任务 0、1、2、3 之间是并行的，互不干扰，控制器下载程序之后这四个文件任务同时启动，同时还能在文件任务执行的时候，使用任务指令开启 SUB 子程序任务或标记任务，SUB 子程序任务或标记任务一旦开启，便与主程序无关，任务运行停止后可重复触发任务执行。

控制器多任务的优势：

程序模块化：用户可以将程序编写成多个较小的、特定的程序，来实现客户设备指定的功能。

并发性：每个任务可以独立运行，任务开启后，不受其他任务的影响。

简化错误处理：划分多任务运行后错误处理变得简单，只需处理出错的任务。

命令交互：程序处于运行状态时，用户也可以随时进行命令交互，如在线修改运动参数，在线命令栏发送指令等，其他程序不受影响。

多任务状态查看

任务状态有三种，正在运行、停止和暂停，任务状态查看有如下 3 种方式。

1. 任务指令查看

PROC_STATUS：任务状态查看，只读参数。返回值：0-任务停止，1-任务正在运行，3-任务暂停中。

示例：

PRINT PROC_STATUS(0) '打印任务 0 状态
?*PROC_STATUS '打印控制器支持的所有任务的状态

2. 任务窗口查看

在菜单栏“调试”→“启动/停止调试”打开任务窗口，如下图。

任务窗口可以查看已经开启的任务的任务编号、运行状态、当前文件和运行行号，看不到未开启的任务。

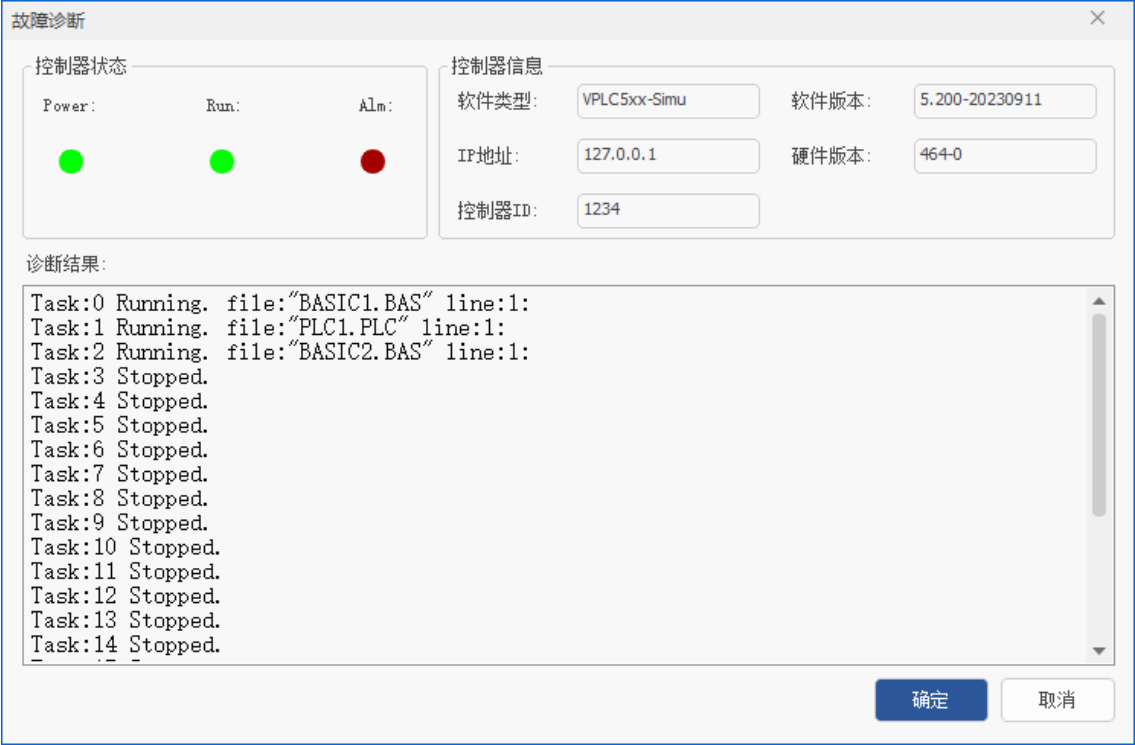
任务		
任务	状态	文件和行号
0	Stopped	BASIC1.BAS,line:6
1	Paused	BASIC1.BAS,line:8
2	Running	BASIC1.BAS,line:8

Basic 的任务在程序扫描完成后，任务变为 Stopped 状态，PLC 主任务由于会循环扫描，所以一直处于

Running 状态。

3. 打开菜单栏“工具”→“故障诊断”窗口。

可查看控制器支持的所有任务编号的状态、所处的文件和运行的行号。
此窗口也可以显示各任务报错的故障信息。



多任务启动与停止

1. 多任务操作指令

多任务主要操作指令如下:

END: 当前任务正常结束

STOP: 停止指定文件运行的任务

STOPTASK: 停止指定任务

HALT: 停止所有任务

RUN: 启动新任务运行一个文件

RUNTASK: 启动新任务运行一个 SUB 或者运行一个带标签的程序

PAUSETASK: 暂停指定任务

RESUMETASK: 恢复指定任务, 恢复后任务从停止处继续往下执行

Basic 和 PLC 的任务操作均是使用以上指令。

控制器状态和?*MAX: 可以查看控制器支持的任务总数与文件总数。

控制器状态	命令与输出
VirtualAxes: 64	max_pswitch:64
RealAxes: 64	max_file:111
Tasks: 22	max_3file:8
Files/3Files: 111/8	max_task:22
Modbus0x Bits: 8000	max_timer:1024
Modbus4x Regs: 8000	max_loopnest:8
VR Regs: 8000	
TABLE Regs: 320000	在线命令: ?*max

2. 多任务启动

任务启动有三种方式，分别是自动运行任务号设置、RUN 指令和 RUNTASK 指令，使用指令开启任务时，程序扫描执行到该指令后再开启任务。

开启任务时注意任务编号的填写，任务不能重复开启。

1) 自动运行任务号：在“文件视图”窗口设置自动运动任务号。控制器上电后首先执行带自动运行任务号的文件，自动运行任务号 Basic 文件可设置多个，PLC 文件和 HMI 文件仅支持一个。自动运行文件为并行运行，上电后同时开启。

2) RUN 指令将文件作为一个任务启动。

示例：RUN "TuXing_001.bas",2 '将 TuXing_001.bas 文件作为任务 2 启动

3) RUNTASK 指令将 SUB 子程序或带标签程序作为一个任务启动。可跨文件开启全局定义的 SUB 子程序，要开启任务的标签程序只能存在本文件内。

示例：RUNTASK 1,task_home '以任务 1 启动 task_home 子程序

3. 多任务停止

停止任务指令有 STOPTASK, STOP, HALT 三种。

任务停止再启动就会从头执行任务。

开启任务时，一般先使用 STOPTASK 停止任务，再 RUNTASK 开启，避免任务出现重复开启报错。

1) STOPTASK 支持停止文件任务、SUB 子程序任务和带标签的任务。

示例：STOPTASK 2 '停止任务 2

2) STOP 指令支持停止 Basic 文件任务，推荐使用 STOPTASK 指令，操作更简单。

3) HALT 指令停止所有任务

示例：HALT '停止项目内的所有任务

快速停止所有任务还可以使用软件菜单栏的“紧急停止”按钮。

示例：项目内有两个任务，下载运行后，任务 0 和任务 1 正在运行中。

文件视图		任务		
文件名	自动运行	任务	状态	文件和行号
Basic1.bas	0	0	Running	BASIC1.BAS,line:169
		1	Running	BASIC1.BAS,line:338

发送在线命令：STOPTASK 0

停止任务 0。



文件视图		任务		
文件名	自动运行	任务	状态	文件和行号
Basic1.bas	0	0	Stopped	BASIC1.BAS,line:81
		1	Running	BASIC1.BAS,line:338

再次启动任务可以再次下载程序。

上程序不能使用 RUN 指令开启自动运行文件任务 0，因为任务 0 中自动开启的任务 1 仍在运行，若使用指令再次开启任务 0，会导致任务 1 重复开启。若停止的任务 1，则可以使用 RUNTASK 指令单独开启任务 1。

任务暂停与恢复

暂停任务使用 PAUSETASK 指令，恢复任务使用 RESUMETASK 指令。恢复后任务从暂停处继续向下执行。暂停的任务支持停止。

1) PAUSETASK：暂停指定任务

示例：PAUSETASK 1 '暂停任务 1

2) RESUMETASK：恢复指定任务

示例：RESUMETASK 1 '继续运行任务 1

示例：项目内有两个任务，下载运行后，任务 0 和任务 1 正在运行中。

文件视图		任务		
文件名	自动运行	任务	状态	文件和行号
Basic1.bas	0	0	Running	BASIC1.BAS,line:169
		1	Running	BASIC1.BAS,line:338

发送在线命令控制任务的暂停或恢复。



在线命令发送：PAUSETASK 0

任务 0 被暂停。

文件视图		任务		
文件名	自动运行	任务	状态	文件和行号
Basic1.bas	0	0	Paused	BASIC1.BAS,line:19
		1	Running	BASIC1.BAS,line:33

在线命令发送：RESUMETASK 0

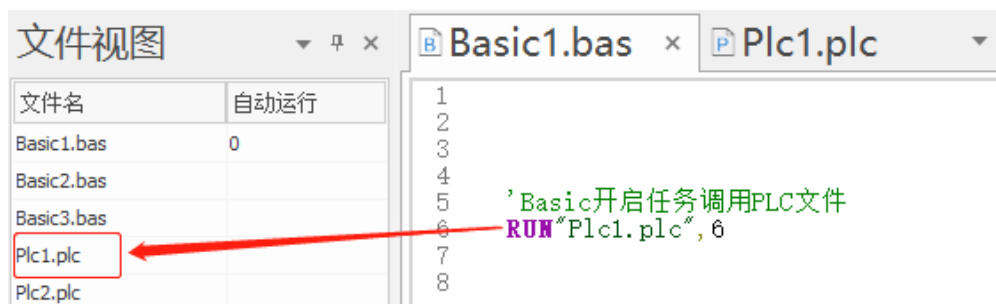
任务 0 恢复运行状态。

文件视图		任务		
文件名	自动运行	任务	状态	文件和行号
Basic1.bas	0	0	Running	BASIC1.BAS,line:169
		1	Running	BASIC1.BAS,line:338

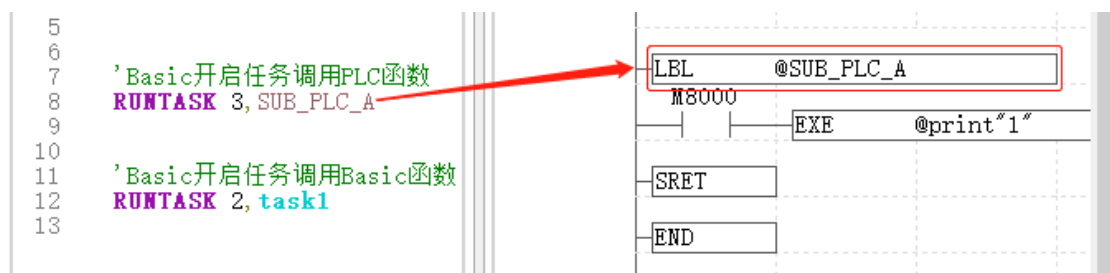
Basic 和 PLC 任务相互调用

1. Basic 调用 PLC 任务

1) Basic 文件内使用 RUN 指令调用 PLC 文件。

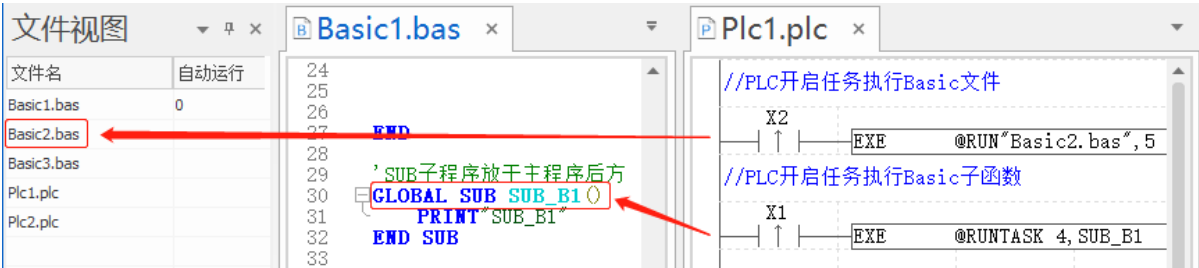


2) Basic 文件内使用 RUNTASK 指令调用 PLC 内 LBL 指令定义的子程序。



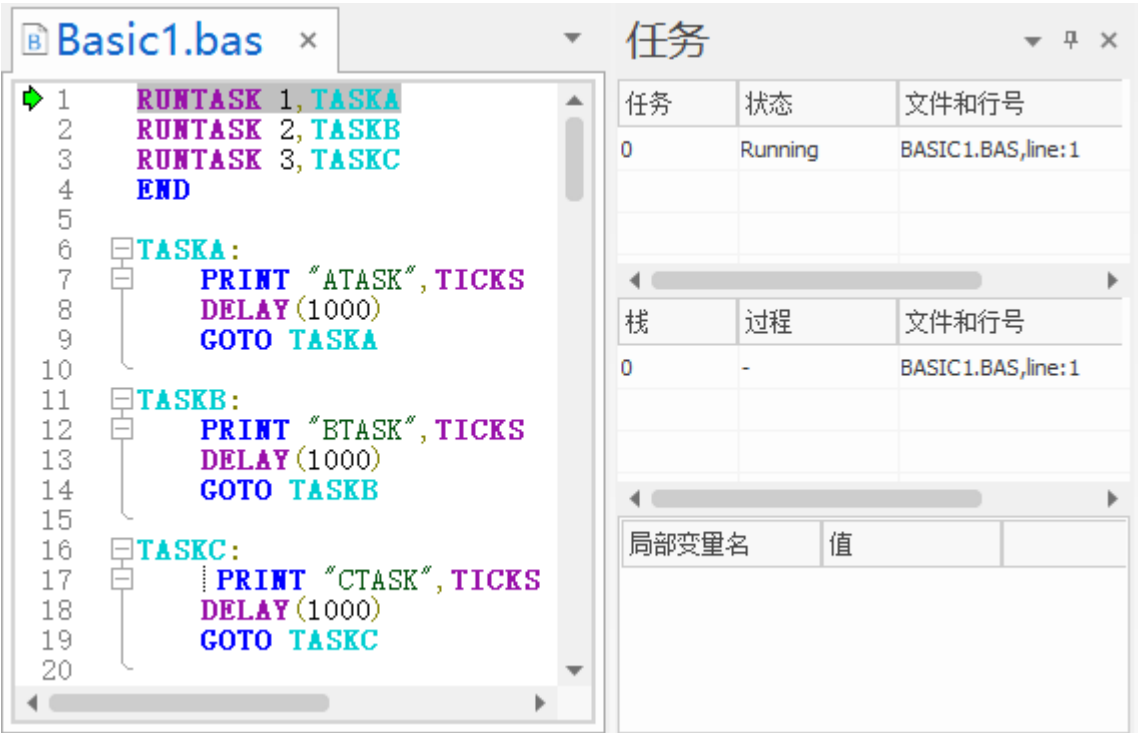
2. PLC 调用 Basic 任务

PLC 内使用 EXE 或 EXEP(脉冲执行)指令，调用 Basic 的任务指令，从而调用 Basic 文件任务或子程序任务。



多任务示例

如下例程，开始单步调试，观察左侧光标的指向，程序扫描到 RUNTASK1 后开启任务 TASKA，开启后继续扫描下一行 RUNTASK2 开启任务 TASKB，RUNTASK3 开启任务 TASKC，遇到 END 主任务 0 停止扫描，TASK、TASKB、TASKC 作为独立的任务分别执行下去。在程序调试窗口可以看到程序执行情况。



上电只有自动运行的任务 0

Basic1.bas

```
1 RUNTASK 1, TASKA
2 RUNTASK 2, TASKB
3 RUNTASK 3, TASKC
4 END
5
6 TASKA:
7   PRINT "ATASK", TICKS
8   DELAY (1000)
9   GOTO TASKA
10
11 TASKB:
12   PRINT "BTASK", TICKS
13   DELAY (1000)
14   GOTO TASKB
15
16 TASKC:
17   PRINT "CTASK", TICKS
18   DELAY (1000)
19   GOTO TASKC
20
```

任务

任务	状态	文件和行号
0	Running	BASIC1.BAS,line:2
1	Running	BASIC1.BAS,line:7

栈

栈	过程	文件和行号
0	-	BASIC1.BAS,line:2

局部变量名

局部变量名	值
-------	---

任务 0 启动任务 1 运行

Basic1.bas

```
1 RUNTASK 1, TASKA
2 RUNTASK 2, TASKB
3 RUNTASK 3, TASKC
4 END
5
6 TASKA:
7   PRINT "ATASK", TICKS
8   DELAY (1000)
9   GOTO TASKA
10
11 TASKB:
12   PRINT "BTASK", TICKS
13   DELAY (1000)
14   GOTO TASKB
15
16 TASKC:
17   PRINT "CTASK",
18   TICKS
19   DELAY (1000)
20   GOTO TASKC
21
```

任务

任务	状态	文件和行号
0	Running	BASIC1.BAS,line:3
1	Running	BASIC1.BAS,line:8
2	Running	BASIC1.BAS,line:12

栈

栈	过程	文件和行号
0	-	BASIC1.BAS,line:3

局部变量名

局部变量名	值
-------	---

任务 0 启动任务 1、任务 2 运行

The screenshot shows the RTBasic IDE with a file named 'Basic1.bas'. The code defines three tasks: TASKA, TASKB, and TASKC, each with a loop of printing ticks, a 1000ms delay, and a GOTO back to the task start. The task monitor window on the right shows task 0 as 'Stopped' and tasks 1, 2, and 3 as 'Running'.

```
1 RUNTASK 1, TASKA
2 RUNTASK 2, TASKB
3 RUNTASK 3, TASKC
4 END
5
6 TASKA:
7   PRINT "ATASK", TICKS
8   DELAY (1000)
9   GOTO TASKA
10
11 TASKB:
12   PRINT "BTASK", TICKS
13   DELAY (1000)
14   GOTO TASKB
15
16 TASKC:
17   PRINT "CTASK", TICKS
18   DELAY (1000)
19   GOTO TASKC
20
21
```

任务	状态	文件和行号
0	Stopped	BASIC1.BAS,line:6
1	Running	BASIC1.BAS,line:8
2	Running	BASIC1.BAS,line:13
3	Running	BASIC1.BAS,line:18

栈	过程	文件和行号
---	----	-------

局部变量名	值
-------	---

任务 0 启动任务 1、任务 2、任务 3 运行

3.5 三种中断类型

RTBasic 中断分为三种，分别为掉电中断、外部中断、定时器中断。

使用中断前必须开启中断总开关，为了避免程序没有初始化好进入中断，控制器上电时中断开关缺省是关闭的。

三类中断运行时，中断程序单独占用一个任务号运行，不存在压栈的情况。

中断使用注意事项：

各中断之间无优先级，支持中断嵌套，多个中断可以同时执行，同一时间处理的中断函数不宜过多。

控制器内部只有一个任务在处理所有的中断信号响应，有一个固定的中断任务号，如果中断处理函数过多，并且中断处理函数的代码太长，会造成所有的中断响应变慢，甚至是中断堵塞，影响其他中断执行。

解决办法：

尽量减少中断的数量，很多应用都可以用循环扫描来处理。

如果有一个中断处理函数特别长的话，调用一个单独的任务来处理中断中的复杂任务，这样就不会堵塞其他的中断响应。

掉电中断

必须是全局的 SUB 函数。控制器只有 1 个掉电中断。掉电中断执行的时间特别有限，只能写少数几条语句，将数据存储在 VR 里。

相关函数：[INT_ENABLE](#)，[ONPOWEROFF](#)。

示例：

```
INT_ENABLE = 1
DPOS(0)=VR(0)           '上电读取保存的数值，恢复坐标
DPOS(1)=VR(1)
DPOS(2)=VR(2)
END                      '主程序结束

GLOBAL SUB ONPOWEROFF () '掉电中断
    VR(0) = DPOS(0)      '保存坐标到 VR
    VR(1) = DPOS(1)
    VR(2) = DPOS(2)
END SUB
```

外部中断

可设置上升沿触发或下降沿触发，必须是全局的 SUB 函数，目前只有中断 IN 口 0-31 才可以使用。必须是支持 PLC 功能的固件才可使用，详情请咨询正运动技术人员。

相关函数：[INT_ONn](#)，[INT_OFFn](#)。

示例：

```
INT_ENABLE=1           '开启中断
END                    '主程序结束

GLOBAL SUB INT_ON0 () '外部上升沿中断程序
    PRINT "输入 IN0 上升沿触发"
END SUB

GLOBAL SUB INT_OFF0 () '外部下降沿中断程序
    PRINT "输入 IN0 下降沿触发"
END SUB
```

定时器中断

达到设定时间后执行的功能，必须是全局的 SUB 函数定时器中断支持同时开启多个，个数由定时器个数决定，定时器个数根据控制器型号，使用?*max 打印查看。

相关函数：[ONTIMERn](#)。

示例：

```
INT_ENABLE=1           '开启中断
TIMER_START(0,100)     '定时器 0 开启，100ms 后执行一次
END                    '主程序结束
```

```

GLOBAL SUB ONTIMER0()      '中断程序
    PRINT "ontimer0 enter"
    'TIMER_START(0,100)    '希望周期执行中断，在 SUB 里再次打开定时器
END SUB

```

3.6 运动缓冲

运动缓冲概念

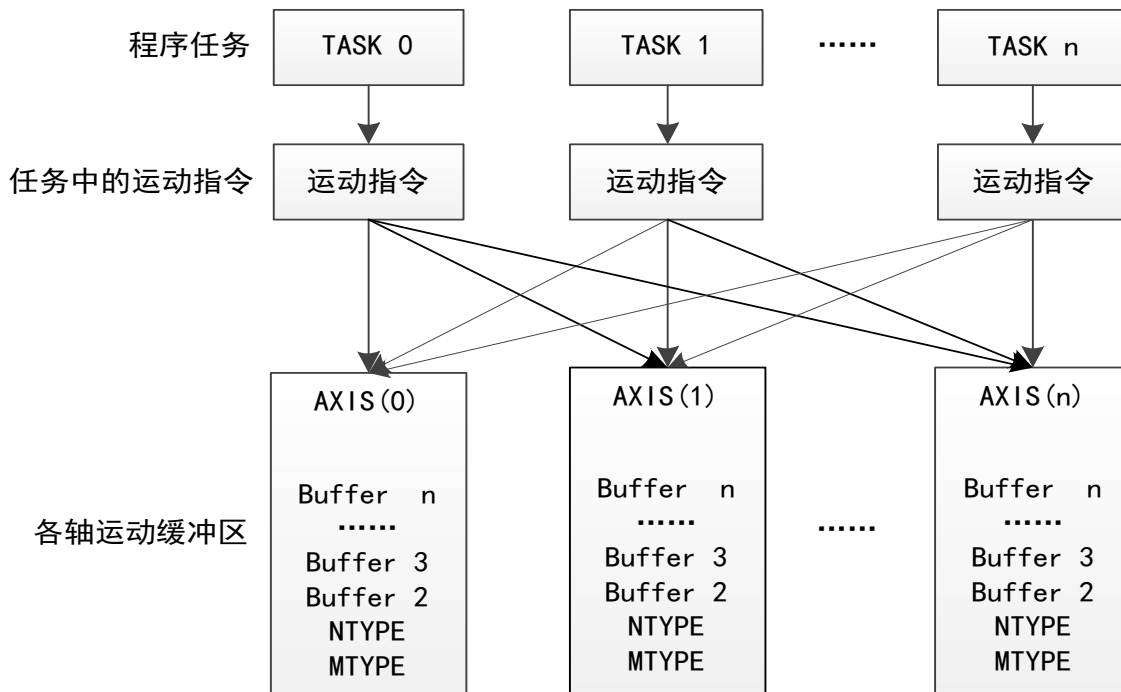
在运行运动指令时，控制器提供了一个缓冲区来保存进入运动缓冲的运动缓冲队列。运动指令存到运动缓冲区，在缓冲区里取出指令再执行，省略了程序扫描的时间，大大提升了实时性，同时也使得程序能正常向下扫描，不会堵塞。

ZMotion 运动控制器具有多级的运动缓冲，当运动缓冲开启的时候，程序在扫描识别到程序任务的第一条运动指令时，将运动指令分配到指定轴的运动缓冲区，电机开始运动，此时程序继续向下扫描到第二条运动指令时，再往运动缓冲区中存，在不断扫描存入运动指令的同时，从运动缓冲区中依次取出运动指令执行。

MTYPE, NTYPE 分别是当前运行的运动指令和第一个缓冲运动指令。

任意一段程序的运动指令都可以进入任意轴的运动缓冲区，由轴号指定。

每个轴的运动缓冲区都是独立的，互不干扰。



运动缓冲区

程序扫描过程中，将扫描到的运动指令存入对应轴的运动缓冲区，在从运动缓冲区中按先进先出的顺序，依次取出运动指令执行，能进入运动缓冲区的指令除了运动指令之外，还包括一系列运动缓冲中输出指

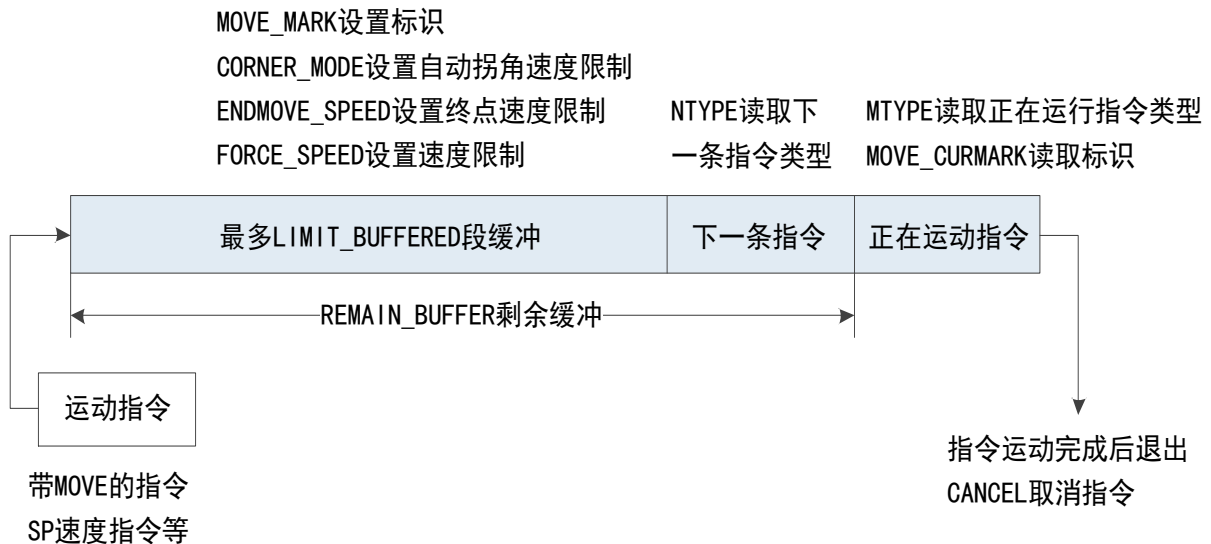
令。

MOVEMODIFY 和 MOVEMODIFY2 属于特例，这两个指令不会进入运动缓冲区。

插补运动缓冲在主轴的运动缓冲区。

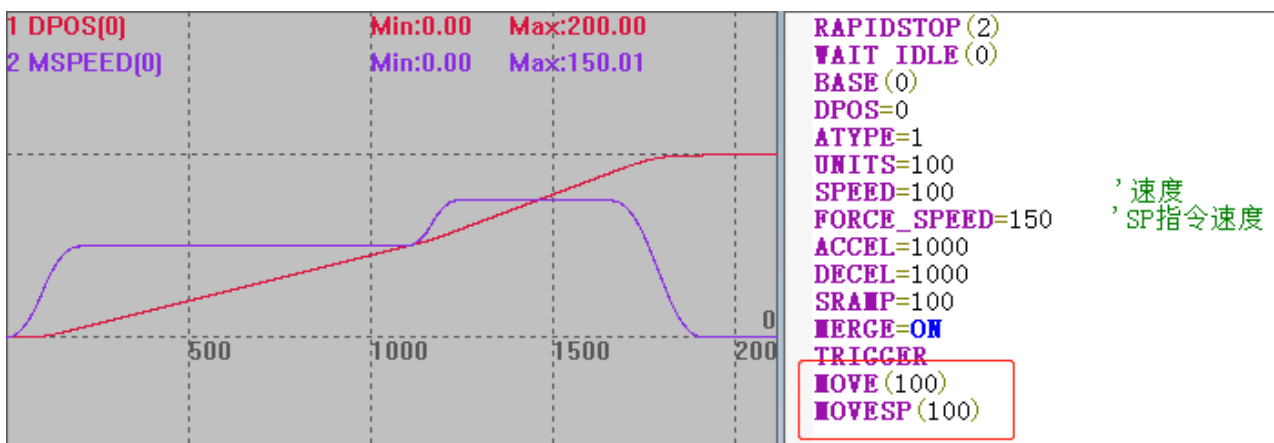
缓冲多条运动指令时，为了判断当前运动执行到哪一条，提供 MOVE_MARK 运动标号和 MOVE_CURMARK 当前运动标号指令。MOVE_MARK 运动标号每扫描一条运动指令+1；MOVE_CURMARK 指令为当前运动的标号，提示当前运动到第几条运动指令，所有运动完成后为-1。

当前运动完成后会自动执行运动缓冲区内的下一条运动。运动指令全部执行完后，运动缓冲区为空，或者使用 CANCEL/RAPIDSTOP 指令清空运动缓冲区。

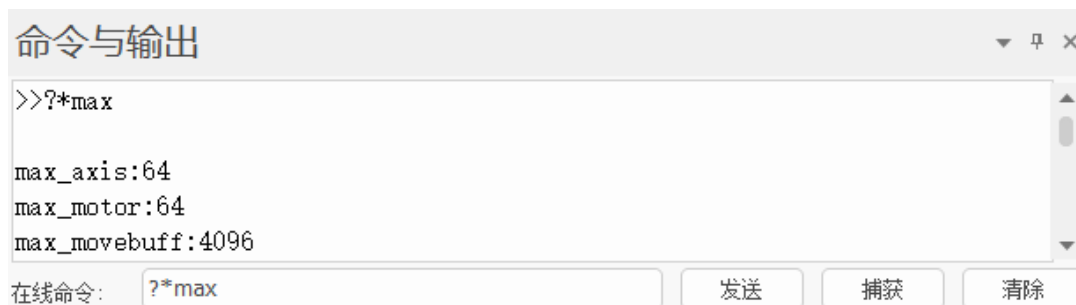


SP 指令又称 SP 运动指令，使用 SP 运动指令时(MOVESP、MOVECIRCSP 等直接在运动指令后方加上 SP)，SP 指令的速度按 SP 速度参数运动，而不是 SPEED 速度，SP 速度包含 FORCE_SPEED、ENDMOVE_SPEED 和 STRATMOVE_SPEED，会随 SP 运动指令写入运动缓存区。

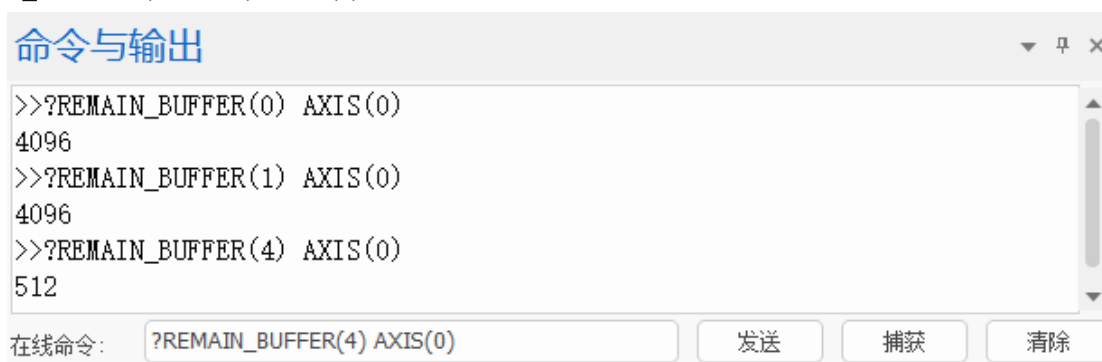
SP 指令与非 SP 指令的运行效果如下，MOVE(100)的速度是 SPEED=100，MOVESP(100)的速度是 FFORCE_SPEED=150。



ZMC4 系列运动控制器每个轴可支持多达 4096 段运动缓冲（不同型号的控制器缓冲个数有区别，具体情况参见控制器硬件手册说明或使用 ?*max 打印查看 max_movebuff 参数），可以手动设置 LIMIT_BUFFERED 运动缓冲限制。



每个轴的运动缓冲都是独立的，互不干扰，且轴的缓冲区大小相同，通过指令 `REMAIN_BUFFER(MTYPE) AXIS(n)` 查看某个轴的剩余可用缓冲区的个数。



不同的运动指令占用的缓冲空间是不同的，越复杂的运动占用的运动缓冲空间越多。例如：ZMC432 控制器，运动缓冲区大小为 4096，缓冲区一次性可缓冲的 MOVE 直线插补指令和 MOVECIRC 圆弧插补指令个数是不同的。

运动缓冲区堵塞

由于每个轴的运动缓冲空间是有限的，当扫描太多运动指令放入运动缓冲区时，多级运动缓冲区全部被塞满，如果程序继续扫描到更多的运动指令，程序也会被堵塞，直到运动指令依次完成并退出，运动缓冲区有了空位，运动指令才会继续进入运动缓冲区。

以 V1.00.02 版本仿真器为例，默认为 4096 个运动缓冲，下图例程中显示该控制器的运动缓冲区最多能存 511 条圆弧插补指令，下载程序后后打印 i 的值为 511，表示当前 FOR 循环并未执行完，程序堵塞了。

Basic1.bas

```

1  RAPIDSTOP(2)
2  WAIT IDLE(0)
3  WAIT IDLE(1)
4
5  BASE(0,1)           ' 选择轴号
6  ATYPE=1,1           ' 脉冲轴类型
7  UNITS=100,100       ' 脉冲当量
8
9  SPEED=100,100       ' 运动速度
10 ACCEL=1000,1000     ' 加速度
11 DECEL=1000,1000     ' 减速度
12 SRAMP=100,100       ' S曲线
13 MERGE=ON            ' 打开连续插补
14 DPOS=0,0
15 TRIGGER              ' 自动触发示波器采样
16 FOR i=0 TO 1000
17     MOVECIRC(200,0,100,0,1)
18     ' 半径100顺时针画半圆，终点坐标(300,100)
19 NEXT
20 END
  
```

轴参数

轴选择	轴0	轴1
MSPEED	58.1100	81.3900
MTYPE	4	4
NTYPE	4	4
REMAIN	248.4000	0
VECTOR_BUFFERED	160776.0500	160776.0500
VP_SPEED	100	76.7500
AXISSTATUS	0h	0h
MOVE_MARK	557	0
MOVE_CURMARK	45	45
AXIS_STOPREASON	0h	0h
MOVES_BUFFERED	511	511

轴参数 帮助 属性

运动缓冲堵塞效果-圆弧插补

Basic1.bas

```

1  RAPIDSTOP(2)
2  WAIT IDLE(0)
3  WAIT IDLE(1)
4
5  BASE(0,1)           ' 选择轴号
6  ATYPE=1,1           ' 脉冲轴类型
7  UNITS=100,100       ' 脉冲当量
8
9  SPEED=100,100       ' 运动速度
10 ACCEL=1000,1000     ' 加速度
11 DECEL=1000,1000     ' 减速度
12 SRAMP=100,100       ' S曲线
13 MERGE=ON            ' 打开连续插补
14 DPOS=0,0
15 TRIGGER              ' 自动触发示波器采样
16 FOR i=0 TO 1000
17     MOVE(100,100)
18 NEXT
19 END
  
```

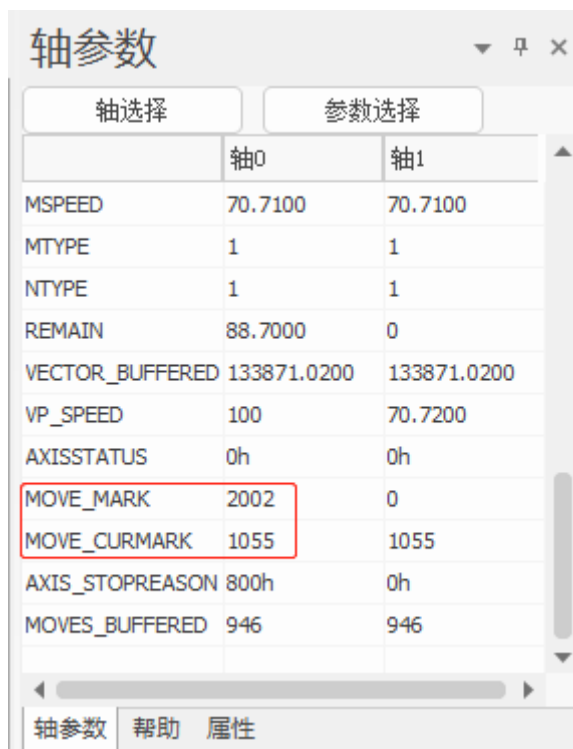
轴参数

轴选择	轴0	轴1
MSPEED	70.7200	70.7200
MTYPE	1	1
NTYPE	1	1
REMAIN	67.7000	0
VECTOR_BUFFERED	140920.9200	140920.9200
VP_SPEED	100	70.7100
AXISSTATUS	0h	0h
MOVE_MARK	2002	0
MOVE_CURMARK	1005	1005
AXIS_STOPREASON	800h	0h
MOVES_BUFFERED	996	996

轴参数 帮助 属性

运动缓冲堵塞效果-直线插补

下图当从运动缓冲区取出部分圆弧运动指令执行之后，缓冲区有了空间，FOR 循环继续执行，并存入运动指令到运动缓冲区。指令执行退出运动缓冲区后，只要运动缓冲区的空间够，新的运动指令一条条往运动缓冲区中存。



	轴0	轴1
MSPEED	70.7100	70.7100
MTYPE	1	1
NTYPE	1	1
REMAIN	88.7000	0
VECTOR_BUFFERED	133871.0200	133871.0200
VP_SPEED	100	70.7200
AXISSTATUS	0h	0h
MOVE_MARK	2002	0
MOVE_CURMARK	1055	1055
AXIS_STOPREASON	800h	0h
MOVES_BUFFERED	946	946

为防止运动缓冲区堵塞导致程序无法继续往下扫描的情况，我们可以在扫描运动指令的时候加入判断处理程序，确认缓冲区有空间了才扫描运动指令。

运动缓冲中输出

运动缓冲中输出指令能进入运动缓冲区，可在运动缓冲中开启 OP 口、延时、输出参数，输出 PWM、开启任务等，详细指令说明参见运动指令章节。

普通输出与运动缓冲中输出的区别：

普通输出指令程序扫描到该行指令便执行输出。

运动缓冲中输出指令在程序扫描之后，将其存入运动缓冲区，运动缓冲区按先进先出的顺序依此取出指令执行，直到取出该输出指令时才会执行输出。

示例：对比 OP 和 MOVE_OP 的输出效果

RAPIDSTOP(2)

WAIT IDLE(0)

BASE(0) '选择轴 0

DPOS=0

UNITS=100 '脉冲当量

SPEED=100 '速度

ACCEL=1000 '加速度

DECEL=1000 '减速度

SRAMP=100 'S 曲线

TRIGGER '触发示波器采样

OP(0,3,\$0) '关闭输出口 0-3

DELAY(1000) '延时

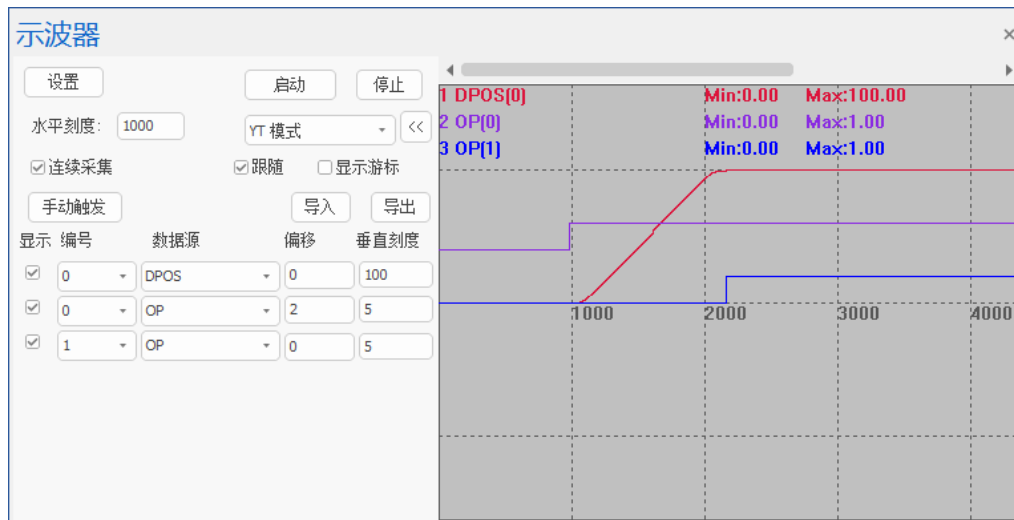
MOVE(100)

MOVE_OP(1,ON) '运动缓冲中输出
OP(0,ON) '普通输出
END

例子运行效果:

延时 1s 之后, 程序扫描到 OP 指令, 输出口 0 立即执行输出。

MOVE_OP 把 IO 操作指令填入运动缓冲区, 所以在运行完 MOVE(100)之后, 输出口 1 才输出。



第四章 通讯方式

4.1 串口通讯

串口类型

控制器包含三类串口，RS232、RS485 和 RS422，其中 RS232 串口所有控制器都包含，绝大部分控制器都包含 RS485 串口，只有少数型号控制器有 RS422 串口。

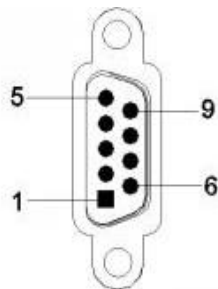
控制器的串口协议均为 MODBUS_RTU，默认做从端，RS232 和 RS485 可通过 SETCOM 指令配置成主端，通讯速率等参数同样也是通过 SETCOM 指令配置，。

控制器串口默认参数：波特率 38400、数据位 8、停止位 1、校验位无，掉电不保存。

1. RS232 串口

控制器的 RS232 接口可以做 MODBUS 主站或从站，支持 1 个主站发送数据，1 个从站接收数据。做主站时，可连接驱动器、变频器、温控仪等，进行数据读出与写入的控制。做从站时，可连接人机界面，用来监控运行状态，常用于连接 PC 或人机界面。

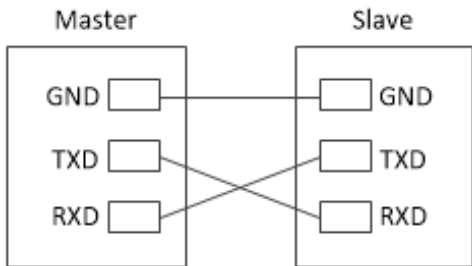
RS232 控制器采用 DB-9 接口，针脚信号说明如下：



针脚号	名称	说明
2	RXD	接收数据引脚
3	TXD	发送数据引脚
5	EGND	电源地
9	E5V	外部电源 5V 输出

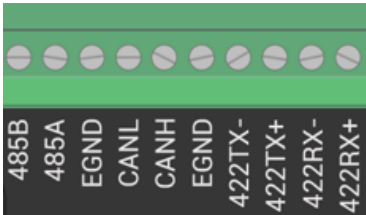
RS232 的标准接线只需要三根线即可，2 根数据信号 TXD 和 RXD，1 根地线 GND，数据信号 TXD 与 RXD 交叉连接，再将 GND 连到一起。

接线参考如下：



2. RS485 串口

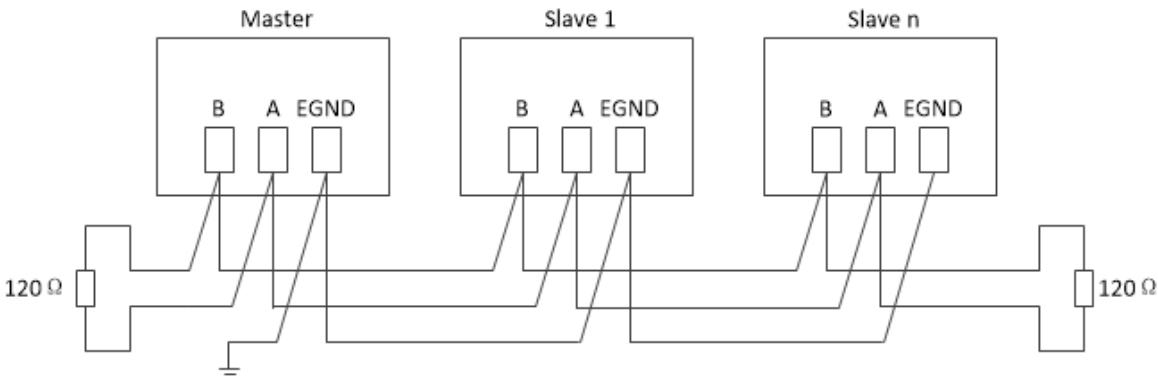
主要提供主/从站的多台通讯设备联机，理论上支持 128 个节点，一主多从。做主站时，可连接驱动器、变频器、温控仪等，进行数据读出与写入的控制；做从站时，能与 PLC 通讯，可连接人机界面，用来监控运行状态。



RS485 接口采用差分传输方式，通过判断 A 与 B 之间的电压差来确定是高电平或低电平。

针脚名称	说明
485B	485-
485A	485+
EGND	电源地

控制器的 RS485 接口采用了简易接线方式，如下图所示，控制器的 485A、485B、GND 地线，分别接第一个从站的 A、B、地线，然后再接第二个从站的 A、B、地线(A 接 A，B 接 B，信号共地)，并且控制器和最后一个从站的 485A 和 485B 要并联 120 欧电阻防止信号反射，线缆需要使用屏蔽双绞线，避免信号干扰，每个节点支线的距离要小于 3m。



3. RS422 串口

仅 ZMC3 系列的部分控制器型号带 RS422 串口。

RS422 数据传输特性与 485 相同。RS422 采用四线制，分别标示为 RX+/RX-（接收信号），RT+/RT-（发送信号），一根信号地线，共 5 根线，四线接口由于采用单独的发送和接收通道，因此不必控制数据方向。

针脚名称	说明
422TX-	发送数据-
422TX+	发送数据+
422RX-	接受数据-
422RX+	接受数据+
EGND	电源地

控制器的 RS422 接口采用了简易接线方式，但相比 RS485 和 RS232，布线成本高，接线容易搞错。控制器的 RS422 接口仅支持接入一个设备。

串口连接方法

串口支持 MODBUS 通讯协议 RTU 模式，常用于连接电脑或触摸屏，通讯时注意串口的参数要匹配，不管哪种串口，除了端口号和接线方法有所不同，默认参数与操作指令都是相同的。

PC 使用串口连接控制的方法如下。

先接好线，在 RTSys 菜单栏点击“控制器”→“连接”，打开如下连接到控制器窗口，会自动列出本计算机上可用的串口号，选择需要连接的串口编号、设置波特率、校验位、停止位之后，点击连接，连接是否成功会在软件输出窗口自动打印出相应信息。

该窗口标题为“连接到控制器”，包含以下配置项：1. 串口选择：下拉菜单显示“1”，波特率选择“38400”，校验位选择“无校验”，停止位选择“0”，右侧有“连接”和“自动连接”按钮。2. IP地址配置：IP地址输入框显示“127.0.0.1”，子网掩码选择“500”，右侧有“连接”和“IP扫描”按钮。3. 本地连接配置：PCI/Local 下拉菜单，右侧有“连接”和“断开连接”按钮。4. 本机IP：输入框显示“192.168.0.55”，右侧有“确定”和“取消”按钮。

控制器串口默认参数：波特率 38400，数据位 8，停止位 1，校验位无，若串口连接失败检查串口号是否正确，修改电脑的通讯端口 COM 的配置，使其与控制器的默认参数一致。

串口参数设置均使用 SETCOM 指令，串口参数是掉电不保存的，控制器重新上电后，SETCOM 参数会还原成默认值，所以请在程序开头写 SETCOM 设置。

串口默认为 MODBUS 从端，可修改 SETCOM 指令的 MODE=14 设置为主端，或 MODE=0 开启串口自定义通讯，即无协议模式，串口自定义通讯模式下使用 GET#指令从自定义串口通道里读取数据，PRITNT #指令从自定义串口通道里输出字符串，PUTCHAR#发指令从自定义串口通道里输出字符（ASCII 码）。

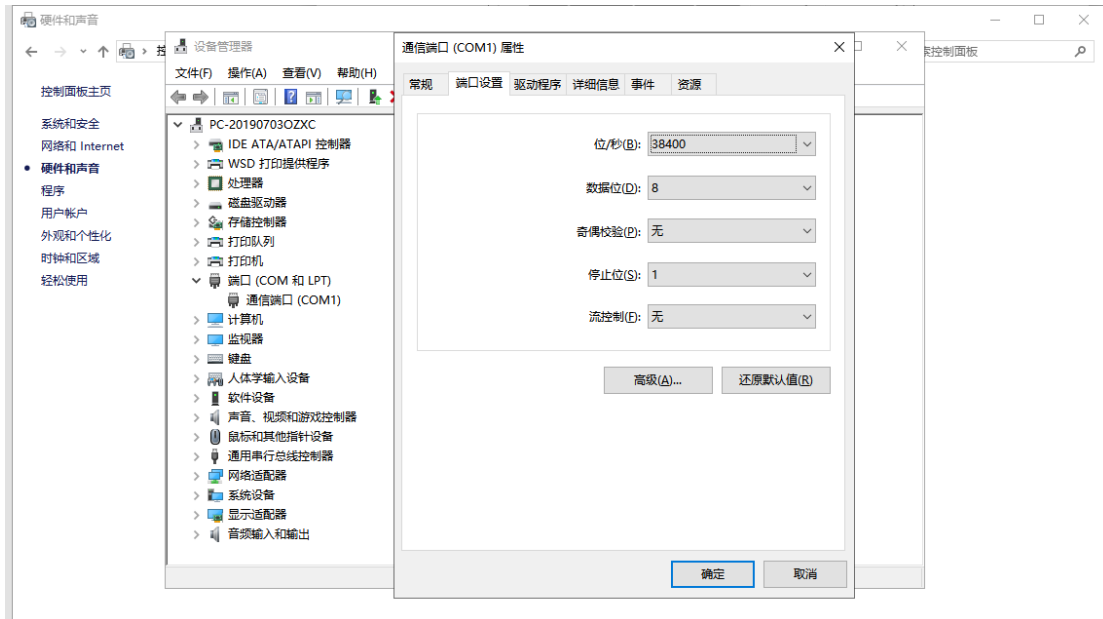
SETCOM 指令 mode 参数配置协议：

Mode 值	描述
0	RAW 数据模式，无协议，此时可以使用 GET#读取；PRITNT #、PUTCHAR#发送
4（缺省）	MODBUS 从端（16 位整数）
14	MODBUS 主端（16 位整数）
15	直接命令执行模式，此时可以直接从串口输入字符串命令(换行符结束)

串口的 MODBUS 通讯方法与串口的自定义通讯方法参见微信公众号“正运动小助手”相关教程。

若连接失败，按下面方法依次排查：

1. 查看串口连接线是否为交叉线。
2. “连接到控制器”里的 COM 口编号、参数是否选择正确。
3. 打开电脑“设备管理器”-“端口”-“通信端口（COM）”-“端口设置”，查看 COM 口设置是否正确，控制器串口默认参数：波特率 38400，数据位 8，停止位 1，校验位无。



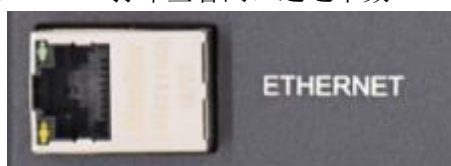
在“端口设置” - “高级”选项中可更改 com 端口号，通过下拉列表选择。



4. 当通过串口连接到控制器时，对应的控制器串口必须配置为 MODBUS 从协议模式（缺省模式），断电重启即可恢复。
5. COM 口是否已被其他程序占用，如串口调试助手等。
6. PC 端是否有足够的串口硬件。
7. 更换串口线/电脑测试。

4.2 网口通讯

控制器网口为 EtherNET 接口，支持 MODBUS_TCP 通讯协议，常用于连接电脑或触摸屏，控制器一般有一个 EtherNET 接口，但是底层至少有两个网口通道，当需要使用网口连接多个设备的时候，可借助交换机，支持网口连接多少个设备使用？*PORT 打印查看网口通道个数。



网线建议使用带屏蔽层的双绞线，保证通讯质量，在 RTSys 菜单栏点击“控制器”→“连接”→“控制器”，打开如下连接到控制窗口，IP 地址在下拉列表中选择，会自动查找当前局域网可用的控制器 IP 地址，选择正确的 IP 之后点击连接。

网口连接是需要保证控制器 IP 地址与电脑的 IP 地址处于同一网段，即四段 IP 地址的前三段相同，最后一段不同，否则会连接失败，连接失败时修改控制器 IP 或者电脑 IP 其中之一即可。

控制器出厂的 IP 是 192.168.0.11，IP 一经修改永久保存。



控制器网口也支持自定义通讯，使用 OPEN#指令打开自定义网口通讯，OPEN#指令支持配置网口通讯主从端，GET#指令从自定义网口通道里读取数据，PRITNT #指令从自定义网口通道里输出字符串，PUTCHAR#发指令从自定义网口通道里输出字符（ASCII 码）。

控制器连接失败可能故障原因及解决办法：

序号	故障描述	解决办法
1	控制器接入电源后 POWER 和 RUN 指示灯不亮	检查电源有无问题 若电源正常检查控制器是否烧坏
2	控制器上电后接入网线，网口指示灯不亮	查看网线两端是否插好 网线本身是否损坏 网线插线槽是否损坏 注意网线是接入 EtherNET 口，而不是 EtherCAT 口
3	连接控制器失败	检查控制器 IP 地址与 PC 是否处于同一 IP 段，需要处于同一网段才可连接使用，详细修改方法参见下文 控制器网口通道全部被占用，建议把不用的暂时关闭后尝试连接 电脑如果卡顿严重，建议尝试多次软件连接 重启控制器再次重复以上连接步骤
4	连接成功，在使用途中偶尔会掉线	网线建议采用金属接头带屏蔽层的网线。在强干扰的情况下，使用水晶头不带屏蔽层的网线会导致通讯不稳定，偶尔发生掉线的情况

修改控制器 IP 地址

先使用串口连接控制器，获取控制器 IP 地址，再修改控制器 IP 地址。

方法一：可以通过菜单栏“控制器”→“修改 IP 地址”窗口直接修改控制器 IP 地址。



方法二：通过 IP_ADDRESS 指令发送在线命令修改。

指令发送修改成功之后自动断开连接，在线命令打印控制器连接错误信息，通过网口连接选择新 IP 地址 192.168.0.23 再次连接控制器，IP 地址修改成功后永久有效。



修改电脑 IP 地址

查看电脑本地 IP 协议版本 4 地址是否为 192.168.0.xxx，前三段与控制器一致，最后一段不能一样，控制器出厂默认 IP 192.168.0.11。如果 IP 地址的第三段不一样，则需要把对应的子网掩码改为 0。

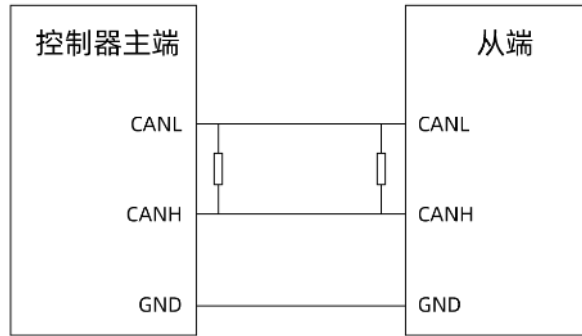
设置好之后，再次打开“连接到控制器”窗口尝试连接到控制器。



4.3 CAN 总线通讯

CAN 接线

控制器的 CAN 总线接口用于接 ZCAN 扩展模块，或连接控制器。CAN 总线连接控制器接线方法与连接 ZCAN 扩展模块相同，不同之处在于板块上没有集成 120 电阻，需要在 CANL 和 CANH 的首尾两端各接一个 120 欧姆电阻，CAN 总线连接扩展模块接线方法参见“[ZCAN 扩展模块](#)”章节。



控制器使用 CAN 总线连接时，此时控制器之间可以使用 CAN 指令通讯，数据通过 TABLE 传递。

控制器缺省做 CAN 通讯的主端，控制器之间的 CAN 通讯需要把一个控制器配置为从端，使用 CANIO_ADDRESS 指令配置主从端，CANIO_ADDRESS=32 配置为主端，CANIO_ADDRESS=其他值配置为从端。

指令语法：CAN(channel, function, tablenum)

channel: CAN 通道，0 表示第一个通道，-1 表示缺省通道

function: 功能号（参见下表，模式 6/7 适用标准帧，模式 16/17 适用扩展帧）

tablenum: 数据存储的 TABLE 位置

值	描述
6	接收，没有数据时，identifier<0
7	发送
16（需要升级固件）	带扩展支持接收，没有数据时，identifier<0
17（需要升级固件）	发送扩展数据，普通数据使用 7 发送

例子：

'发送端：第一个控制器

TABLE(0,1,8,1,2,3,4,5,6,7,8) '发送 cobid=1，8 个字节，依次为 1-8

CAN(0,7,0) '发送数据

'接收端：第二个控制器

CANIO_ADDRESS=1 '设置为 CAN 从端，此参数设置一次即可

CAN(0,6,0) '接收数据

?TABLE(0)

4.4 U 盘接口

正运动大部分运动控制器都带有一个标准 U 盘接口。

没有控制器的场合，可以在 RTSys 根目录新建 udisk 文件夹模拟 U 盘使用，连接到仿真器，调试 U 盘指令。

U 盘接口主要有三方面的用途：

1. 程序升级

通过 U 盘口，下载打包好的 zar 程序包，方便客户更新系统程序。

程序升级之前事先将 zar 程序包下载到 U 盘里面。使用 FILE 指令加载 U 盘文件成功后，zar 程序自动开始运行。

示例：

```
DIM result                '定义变量
IF U_STATE=TRUE THEN      'U 盘插入判断
    result = FILE "find_first",".zar",10 '扫描第一个 zar 格式文件，文件名保存到 VR
    IF result=TRUE THEN    '扫描文件成功判断
        FILE"load_zar",VRSTRING(10,20) '下载扫描到文件名与存储到 VR 里字符相同的 zar 文件
    ENDIF
ENDIF
END
```

2. 加载三次文件

使用 FILE 指令加载 U 盘里保存的三次文件执行。

示例：

```
IF U_STATE=TRUE THEN      '判断 U 盘是否插入
    FILE "FIND_FIRST",".Z3P",800 '查找 Z3P 文件
    ?"文件名: "VRSTRING(800,20),"等待下载"
    FILE "COPY_TO",VRSTRING(800,20),VRSTRING(800,20) '下载 Z3P 文件
    ?"下载 Z3P 文件完成"
ENDIF
```

3. U 盘与寄存器数据交互

U 盘支持读写变量和数组。

FLASH 数据拷贝：多个控制器中 FLASH 存储的数据可以通过 U 盘来相互传递。

VR 寄存器、TABLE 寄存器与 U 盘里的数据互相传递。

读写文件类型为 SD(filename).BIN 或 SD(filename).CSV，不同的指令可操作的文件类型有所区别。

不同型号的控制器 U 盘接口的使用方法都是相同的，将 U 盘插在控制器上的 UDISK 端口即可，控制器上电后有 U 盘插入时，U 盘指示灯亮。

在对 U 盘进行操作之前，先使用 U_STATE 指令判断 U 盘的状态，确保 U 盘能成功通讯，再使用 U 盘相关指令操作。

示例：

```
DIM a,array1(2) '变量、数组定义
a=123
array1(0)=10
array1(1)=20
```



```
IF U_STATE = TRUE THEN '判断 U 盘是否插入
  U_WRITE 0,a,array1 '将变量、数组写入 U 盘的 SD0 文件
  a=456
  array1(0)=11
  U_READ 0,a,array1 '读取 U 盘文件 SD0 数据
  PRINT a,array1(0) '结果: 123, 10
  IF a <> 123 THEN '判断 U 盘读写是否成功
    PRINT "U 盘读取错误"
  ELSE
    PRINT "U 盘成功读取"
  ENDIF
ELSE
  PRINT "U 盘未插入"
ENDIF
END
```

U 盘更多操作参见“[U 盘相关指令](#)”章节。

4.5 EtherCAT 总线通讯

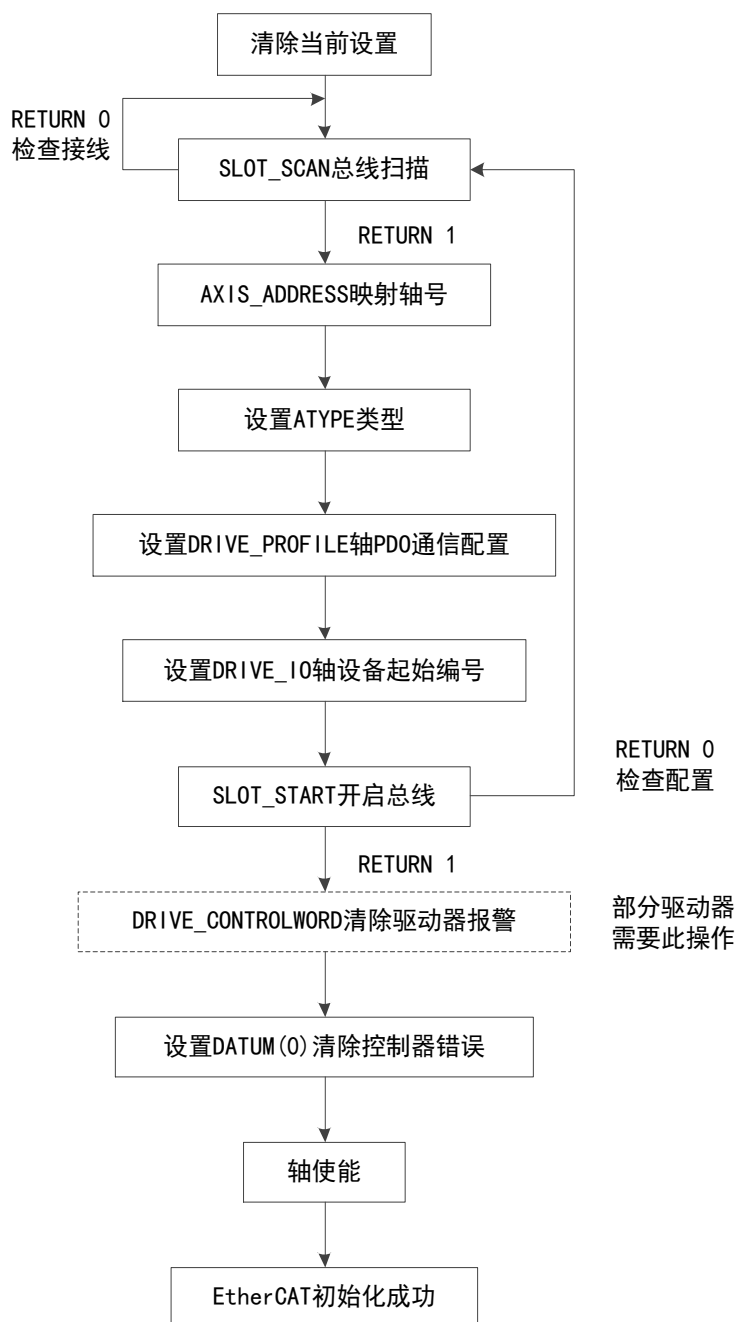
EtherCAT 总线初始化

EtherCAT 总线接口可用于连接 EtherCAT 伺服驱动器和 EtherCAT 扩展模块，无论连接什么模块，EtherCAT 总线都需要编写一段 EtherCAT 总线初始化程序来进行电机和 EtherCAT 扩展模块的使能。使能之后的应用与脉冲电机一致，运动指令都是相同的。

初始化程序一般过程：

1. 使用 SLOT_SCAN 扫描设备，判断 RETURN 是否正确，未连接设备时不会报错。
2. 通过 NODE_INFO/ NODE_AXIS_COUNT 等对设备类型、信息等进行判断。
3. 依次设置 AIXS_ADDRESS, ATYPE, DRIVE_PROFILE, DRIVE_IO 等。
4. SLOT_START 启动设备。
5. 设置每个轴的 AXIS_ENABLE=1，打开轴的使能，设置 WDOG=1，所有轴使能允许。（部分驱动器需要使用 DRIVE_CONTROLWORD 指令清除驱动器报警）
6. 建立连接后主站和从站即可进行周期性数据交换。

EtherCAT 初始化程序一般过程：初始化程序参见[例程](#)



涉及基本概念：

1. NODE 节点编号：根据 EtherCAT 设备接线顺序排列，编号从 0 开始到设备个数减 1。
2. 驱动器编号：根据 EtherCAT 设备接线顺序排列，编号从 0 开始到 EtherCAT 驱动器个数减 1，只在 AXIS_ADDRESS 配置时有作用，即只计算驱动器设备，其他扩展 IO 等设备不计入驱动器编号。
3. 轴号：控制器对连接在控制器上的驱动电机设备设定的编号，编号从 0 开始到连接设备总个数减 1，通过 AXIS_ADDRESS 指令可以将轴号映射到任何一个连接的驱动器设备（脉冲型控制器不支持本地轴号映射）。

设置时注意事项：

1. 设备号按照连接先后顺从编号 0 依次递增编号，需要支持 EtherCAT 总线。
2. 总线控制器上脉冲轴的轴号是固定的。总线也可以调用脉冲轴轴号，ATYPE=65 时总线调用，注意

切换时 DPOS 会发生变化，所以最好不要使轴号冲突。

3. 用 DRIVE_CONTROLWORD(轴号)清除驱动器报警后，需要延时 200ms，再用 DATUM(0)清除控制器报警，不延时可能需要下载两次程序才能清除报警。

4. 如果使用了 DRIVE_CONTROLWORD(轴号)对驱动器操作，那么此驱动器之后的所有驱动器会继承此次设置。所以最好对每个驱动器都设置一次。

5. 运行中连接或断开设备，需要重新扫描，状态才能更新，比如 NODE_COUNT(0)，返回当前连接设备个数，重新扫描后返回的个数才会改变。

6. AXIS_ADDRESS(i)=1，选择的是第一个驱动器，而不是第一个设备。比如连接了两个扩展板后再连接两个驱动器，设备号依次为扩展板 A: 0, 扩展板 B: 1, 驱动器 A: 2, 驱动器 B: 3。此时 AXIS_ADDRESS(i)=1 选择的是驱动器 A，AXIS_ADDRESS(i)=2 选择的是驱动器 B。

按照顺序连续选择，不可先选 2 再选 1，不可选完 1 跳过 2 选择 3。

必须是控制器支持 EtherCAT 才可以使用相关的指令。与 RTEX 总线使用一套程序指令，但是功能有所区别，详细请参见总线指令章节说明。

EtherCAT 相关指令：

指令	含义
SLOT_SCAN	扫描设备
SLOT_START	总线启动
SLOT_STOP	总线停止
ATYPE	轴类型，例如 65 为 EtherCAT 周期位置控制
AXIS_ADDRESS	轴地址配置
WDOG	轴统一使能控制
SERVO_PERIOD	刷新率
AXIS_ENABLE	轴使能
SDO_WRITE	SDO 操作
SDO_READ	SDO 操作
SDO_WRITE_AXIS	驱动器 SDO 操作
SDO_READ_AXIS	驱动器 SDO 操作
?*ETHERCAT	总线信息输出
NODE_COUNT	设备个数
NODE_STATUS	设备状态
NODE_AXIS_COUNT	设备带驱动个数
NODE_IO	设备 IO 编号
NODE_AIO	设备 AIO 编号
NODE_INFO	设备信息读取
NODE_PROFILE	设备 PROFILE 设置，选择 PDO 报文内容
DRIVE_FE	驱动误差
DRIVE_FE_LIMIT	驱动误差设置
DRIVE_STATUS	驱动状态字
DRIVE_TORQUE	驱动器力矩反馈
DRIVE_MODE	驱动器运动模式

DRIVE_PROFILE	驱动器 PROFILE 设置，选择 PDO 报文内容
DRIVE_CONTROLWORD	驱动器控制字
DRIVE_CW_MODE	驱动器控制字操作模式
DRIVE_IO	远程 IO 编号设置
AXISSTATUS	轴状态

EtherCAT 配置与实际连接相符机制：

主站对从站进行初始化后，无论软件中配置从站数量与 EtherCAT 通讯口实际连接是否相符，配置成功的从站都可以通过主站控制。如软件中配置了两个 EtherCAT 从站，实际连接一个，则连接的这一个可以通过运动指令控制，主站和从站建立连接后，即使软件中配置的另一个从站再接入网络，主站也不会和后接入网络的从站建立连接。

EtherCAT 从站掉线与恢复机制：

EtherCAT 从站与主站建立连接后，如果因为通讯线缆拔出等外部原因导致部分 EtherCAT 从站通讯掉线，主站不会与掉线的从站重新建立连接，掉线的从站不能通过运动指令控制，没有掉线的 EtherCAT 从站不受影响。如果因为驱动器错误等内部原因导致的无法与总线通讯，查看该错误能否清除，清除后重新使能即可使用，不能清除需要断电重新执行总线初始化过程。

掉线的从站如果需要重新和主站建立连接，需要拔掉控制器与第一个伺服驱动器之间的 EtherCAT 线缆再插上，或者给控制器重新上电。如果进行上述操作，会影响正常运行的从站，正常运行的从站会和主站重新建立连接，如果有轴处于运行状态，会导致轴立即停止。

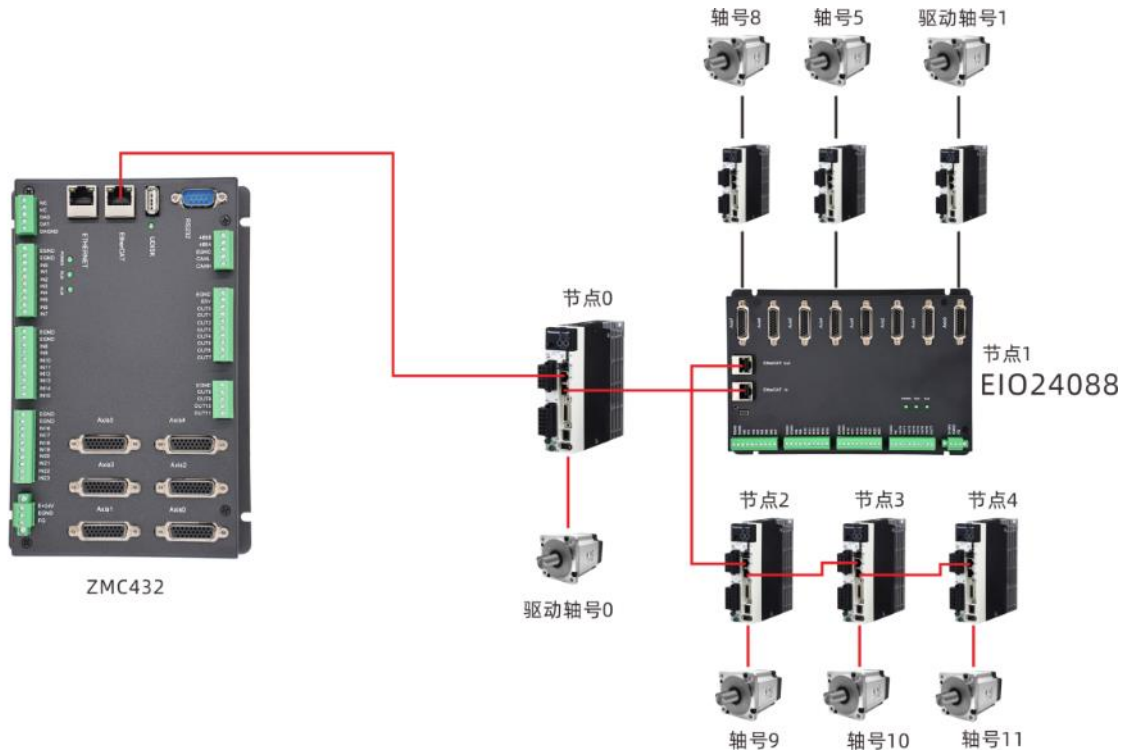
EtherCAT 总线与驱动器通讯

伺服驱动器或与 EtherCAT 扩展模块控制器接线遵循的规则相同，使用一根网线将控制器的 EtherCAT 总线端口与其他设备的 EtherCAT 口相连。

注意伺服驱动器的 EtherCAT 接口有两个，有些驱动器这两个口可以随意接，有些分为 EtherCAT IN 和 EtherCAT OUT，IN 口接上一级设备，OUT 口接下一级设备，二者不能混用，要注意连接顺序。

多轴控制时伺服驱动器的 EtherCAT OUT 口再连接下一级驱动设备的 EtherCAT IN 口，依此类推。

EtherCAT 总线的接线配置如下图：



EtherCAT 总线槽位号默认为 0。

设备号(node)，又叫节点，是指一个槽位上连接的所有设备的编号，从 0 开始，按设备在总线上的连接顺序自动编号。

控制器会自动识别出槽位上的驱动器，编号从 0 开始，按驱动器在总线上的连接顺序自动编号。驱动器编号与设备号不同，只给槽位上的驱动器设备编号，其他设备忽略。

EtherCAT 总线和 RTECH 总线的编号规则相同。

EtherCAT 总线上连接的电机需要编写一段 EtherCAT 总线初始化程序来进行使能。使能之后若 ATYPE=65 位置模式，用法与脉冲电机一致，运动指令都是相同的；若 ATYPE=66 速度模式或 ATYPE=67 力矩模式，可使用 DAC 指令控制轴持续运动，停止将 DAC=0。

EtherCAT 总线连接扩展模块

EtherCAT 扩展模块可扩展 IO 和脉冲轴，接线规则与 EtherCAT 驱动器相同，接线图参考上节，接线时注意扩展模块上的 EtherCAT IN 和 EtherCAT OUT 不能混用。

按要求接线完成，先对 EtherCAT 扩展模块进行初始化，初始化过程中需要对扩展的 IO 和扩展的脉冲轴资源进行映射才能使用，扩展资源映射在总线扫描之后总线开启之前按下述方法操作映射。

EtherCAT 总线上的 IO 映射采用 NODE_IO 指令(数字量)、NODE_AIO 指令(模拟量)设置，轴映射采用 AXIS_ADDRESS 指令映射轴号。

扩展资源映射方法：

1. IO 映射

Slot 槽位号和 node 设备号按照与控制器的连接顺序，从 0 开始自行编号。

NODE_IO 指令设置设备的数字量 IO 起始编号，单个设备的输入输出的起始编号一样。必须总线扫描后才能设置，NODE_AIO 指令使用与 NODE_IO 指令基本相同。

语法：

NODE_IO(slot, node)=iobase

slot: 槽位号, 0-缺省

node: 设备编号, 编号从 0 开始

ioBASE: 映射 IO 起始编号, 设置结果只会是 8 的倍数

NODE_AIO(slot, node[,idir])=aiobase

slot: 槽位号, 0-缺省

node: 设备编号, 编号从 0 开始

idir: AD/DA 选择。0-缺省, 同时设置 AIN、AOUT, 读取时只读 AIN; 3-AIN; 4-AOUT

IO 映射示例:

```
SLOT_SCAN(0)           '扫描总线
IF NODE_COUNT(0)>0 THEN '判断槽位 0 上是否有设备
    NODE_IO(0,0)=32      '设置槽位 0 接口设备 0 的 IO 起始编号为 32
    NODE_AIO(0,1,3)=8    '设置槽位 0 接口设备 1 的 AIN 起始编号为 8
ENDIF
```

2. 轴映射

总线轴需要进行轴映射操作, 采用 AXIS_ADDRESS 指令映射, 操作方法如下:

AXIS_ADDRESS(轴号)=(槽位号<<16)+驱动器编号+1

轴映射写在总线初始化程序中, 扫描总线之后, 开启总线之前。

示例:

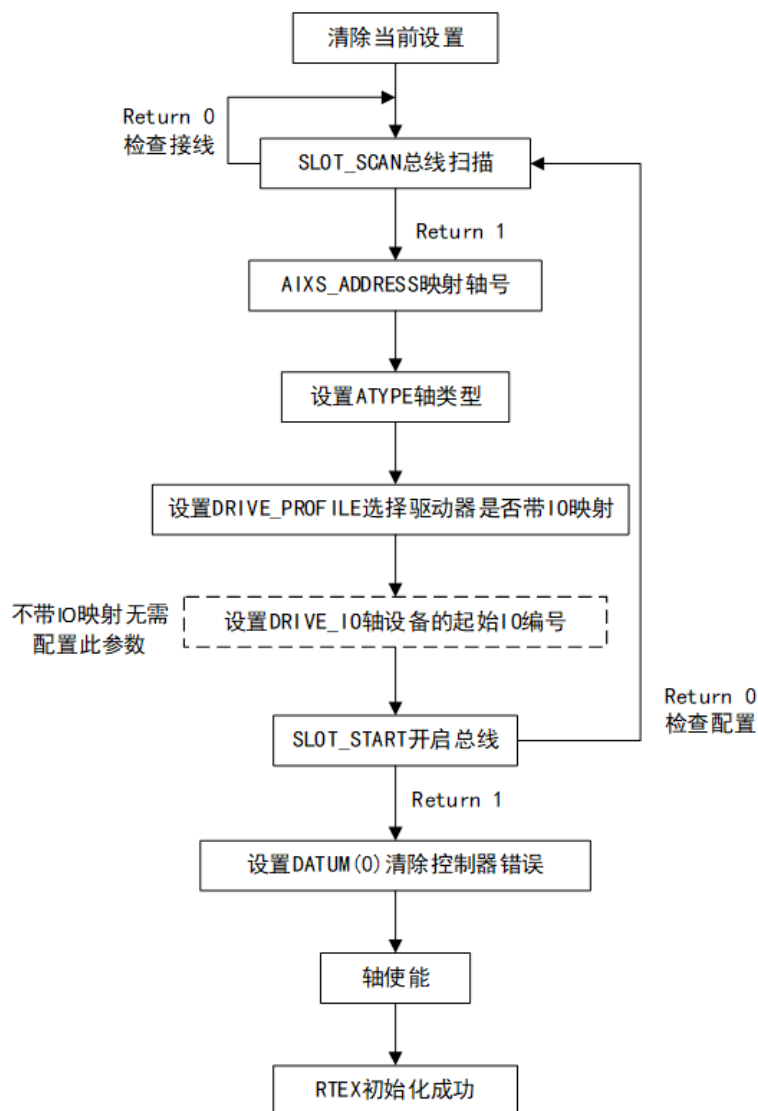
```
AXIS_ADDRESS(0)=(0<<16)+0+1 '第一个 ECAT 驱动器, 驱动器编号 0, 绑定为轴 0
AXIS_ADDRESS(2)=(0<<16)+1+1 '第二个 ECAT 驱动器, 驱动器编号 1, 绑定为轴 2
AXIS_ADDRESS(1)=(0<<16)+2+1 '第三个 ECAT 驱动器, 驱动器编号 2, 绑定为轴 1
ATYPE(0)=65                  '设置为 ECAT 轴类型, 65-位置 66-速度 67-转矩
ATYPE(1)=65
ATYPE(2)=65
```

4.6 RTEX 总线通讯

RTEX 是松下为实现运动控制高速实时性要求独自开发的高端总线技术, 通过简化数据通讯包, 实现高速通信, 速度可达 100Mbps。仅支持连接松下驱动器, 使用前先需要编写一段总线初始化程序使能。

控住器 RTEX 总线通讯有两个接口, 分别为 RX 和 TX, 接线时控制器 RX——驱动器 TX, 控制器 TX——驱动器 RX。

RTEX 的接线方法参见下图:



1. 使用 SLOT_SCAN 扫描设备，判断 RETURN 是否正确，未连接设备时会报错。
2. 通过 NODE_INFO/ NODE_AXIS_COUNT 等对设备类型、信息等进行判断。
3. 依次设置 AIXS_ADDRESS, ATYPE, DRIVE_PROFILE, DRIVE_IO 等。
4. SLOT_START 启动设备。
5. 建立连接后主站和从站即可进行周期性数据交换。

涉及基本概念：

1. NODE 节点编号：根据 RTEX 设备接线顺序排列，编号从 0 开始到设备个数减 1。
2. 驱动器编号：根据 RTEX 设备接线顺序排列，编号从 0 开始到 RTEX 驱动器个数减 1，只在 AIXS_ADDRESS 配置时有作用，即只计算驱动器设备，其他扩展 IO 等设备不计入编号。
3. 轴号：控制器对连接在控制上的驱动设备设定的编号，编号从 0 开始到连接设备总个数减 1，通过 AIXS_ADDRESS 指令可以将轴号映射到任何一个连接的驱动器设备（脉冲型控制器不支持本地轴号映射）。

必须是控制器支持 RTEX 才可以使用相关的指令。与 EtherCAT 总线使用一套程序指令，但是功能有所区别，详细请参见总线指令章节说明。

RTEX 相关指令：

指令	含义
SLOT_SCAN	扫描设备
SLOT_START	总线启动
SLOT_STOP	总线停止
ATYPE	轴类型，例如 50 为 RTEX 周期位置控制
AXIS_ADDRESS	轴地址配置
WDOG	轴统一使能控制
SERVO_PERIOD	刷新率
AXIS_ENABLE	轴使能
?*Rtex	总线信息输出
NODE_COUNT	设备个数
NODE_STATUS	设备状态
NODE_AXIS_COUNT	设备带驱动个数
NODE_IO	设备 IO 编号
NODE_AIO	设备 AIO 编号
NODE_INFO	设备信息读取
DRIVE_STATUS	驱动状态字
DRIVE_TORQUE	驱动器力矩反馈
DRIVE_PROFILE	驱动器 PROFILE 设置，选择 PDO 报文内容
DRIVE_CONTROLWORD	驱动器控制字
DRIVE_CW_MODE	驱动器控制字操作模式
DRIVE_IO	远程 IO 编号设置
DRIVE_CLEAR	驱动器清除报警，没有错误时使用控制器会警告
DRIVE_READ	驱动器读参数
DRIVE_WRITE	驱动器写参数
AXISSTATUS	轴状态

RTEX 从站掉线与恢复机制与 EtherCAT 相同，参见上节 EtherCAT 的说明。

第五章 运动控制功能

5.1 常见运动模式

常见的三种运动模式如下：

1. 点位运动：仅对终点位置有要求，与运动的中间过程即运动轨迹无关。要求定位速度较快，在运动的加速段和减速段，采用不同的加减速控制策略，分为 JOG 点动、MOVE 寸动和 VMOVE 持续运动三类。
2. 连续轨迹运动：该控制又称为插补，系统在高速运动的情况下，既要保证系统加工的轮廓精度，还要保证轴的运动速度不受影响，对小线段加工时，有轨迹前瞻预处理功能，插补运动说明参见下节。
3. 同步运动：是指多个轴之间的运动协调控制，可以是多个轴在运动全程中进行同步，也可以是在运动过程中的局部有速度同步，主要应用在有电子齿轮和电子凸轮功能的系统控制中，产业上有印染、印刷、造纸、轧钢、同步剪切等行业。

单轴点动

相关指令：

指令	说明	用法
VMOVE	持续运动，可选正负方向	VMOVE(运动方向)
CANCEL	单轴停止，四种停止模式	CANCEL(模式)
JOGSPEED	JOG 时的运动速度	JOGSPEED=速度值
FWD_JOG	正向 JOG 输入对应的输入口编号	FWD_JOG=输入编号
REV_JOG	负向 JOG 输入对应的输入口编号	REV_JOG=输入编号
FHSPEED	在 FHOLD_IN 被按下时的保持速度	FHSPEED=速度值
FHOLD_IN	保持输入对应的输入点编号	FHOLD_IN=输入编号
FAST_JOG	快速点动输入编号	FAST_JOG=输入编号

VMOVE 为单轴持续运动指令，当 VMOVE 执行后，除非使用 CANCEL 或 RAPIDSTOP 清除运动缓存，否则会一直运转。

当前面的 VMOVE 运动没有停止时，后方的 VMOVE 指令会自动替换前面的 VMOVE 指令并修改方向，因此无需 CANCEL 前面的 VMOVE 指令。

在 JOG 运动模式下，各轴可以独立设置目标速度、加速度、减速度、速度平滑等运动参数，能够独立运动或停止。

JOG 运动由开关信号控制，可以正向运动和负向运动，通过 FWD_JOG 指令映射正向点动开关、REV_JOG 指令映射负向点动开关，控制器收到信号输入时，对应轴按照 JOGSPEED 速度慢速运动，信号输入中断时轴减速停止，需要持续的 JOG 运动时，保持开关的输入状态即可。

控制器还支持快速点动，FAST_JOG 指令设置快速点动开关，按下快速点动开关轴以 SPEED 运动，没有按下开关时轴以 JOGSPEED 运动。

FHOLD_IN 映射保持输入设置，当有输入信号时，运动轴的速度按照 FHSPEED 的速度参数继续执行当前运动，当 FHOLD_IN 取消输入，轴继续运动，但是运动速度变回 SPEED 速度。

当 REV_JOG 和 FWD_JOG 同时有信号输入时，轴按照 FWD_JOG 正向运行。

MOVE 指令控制轴寸动，设定寸动的距离，给目标轴发送有限个脉冲，可用在单轴或多轴的运动场合，可一次性给多个轴发送脉冲，在点位运动的控制器上只能控制各个轴单独运动，无法联合插补。

单轴点动例一：VMOVE 持续运动

RAPIDSTOP(2)

WAIT IDLE(0)

BASE(0) '选择轴号
 ATYPE=1 '轴类型设置
 UNITS=100 '脉冲当量设置
 SPEED=100 '速度设置
 ACCEL=1000 '加速度设置
 DECEL=1000 '减速度设置
 SRAMP=100 'S 曲线
 DPOS=0 '当前位置清 0
 TRIGGER

WHILE 1 '循环运动
 IF SCAN_EVENT(IN(0))>0 AND IDLE=-1 THEN 'IN(0)按下往左运动
 VMOVE(-1)
 ELSEIF SCAN_EVENT(IN(1))>0 AND IDLE=-1 THEN 'IN(1)有按下往右运动
 VMOVE(1)
 ELSEIF SCAN_EVENT(IN(0))<0 OR SCAN_EVENT(IN(1))<0 THEN
 CANCEL '取消输入时停止运动
 ENDIF
WEND
END

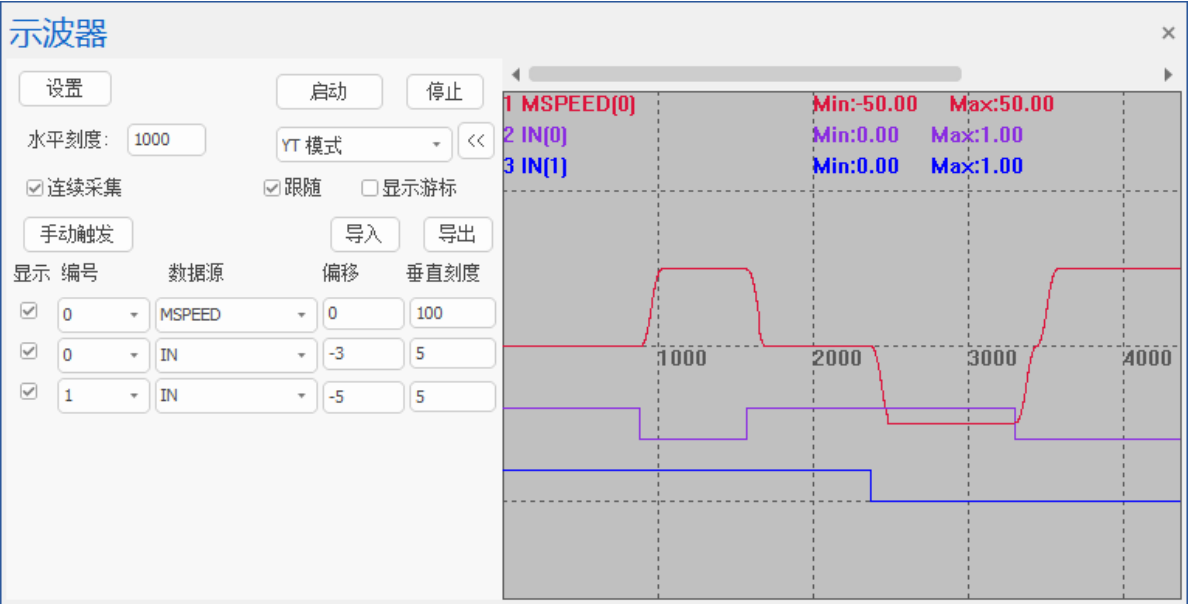
单轴点动例二：JOG 点动

注意映射 JOG 开关后，一定要 INVERT_IN 反转电平，因为 ZMC 系列控制器是 OFF 信号有效，若不反转，则导致信号接入时为 OFF，控制器判断有输入，立即控制轴运动。

BASE(0) '选择轴 0
 ATYPE=1 '脉冲轴类型
 DPOS=0 '坐标清 0
 UNITS=100 '脉冲当量
 SPEED =100 '主轴速度
 ACCEL=1000 '加速度
 DECEL=1000 '减速度
 SRAMP=100 'S 曲线
 TRIGGER '自动触发示波器
 JOGSPEED=50 'JOG 速度 50
 FWD_JOG=0 'TN0 作为正向 JOG 开关
 REV_JOG=1 'TN1 作为负向 JOG 开关

INVERT_IN(0,ON) '输入 0 信号反转
INVERT_IN(1,ON) '输入 1 信号反转

运行效果：
输入 0 口有信号输入时，轴 0 正向运行，速度为 50。
输入 1 口有信号输入时，轴 0 负向运行，速度为 50。
输入 0、1 同时有信号输入时，轴 0 正向运行。

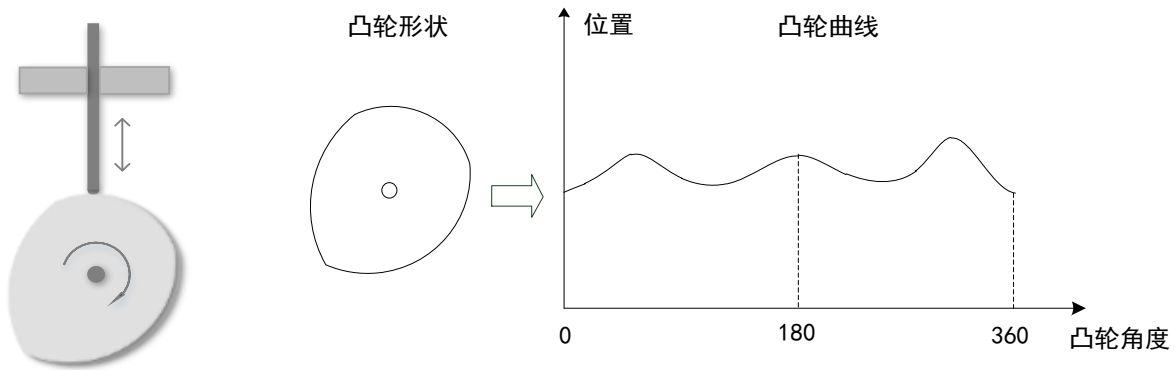


电子凸轮

相关指令：

指令	说明	用法
CAM	凸轮表运动	CAM（凸轮表起始位置，结束位置，比例，运动距离）
CAMBOX	跟随凸轮表运动	参考指令章节
TABLE	保存凸轮表数据	TABLE（数据起始地址，数据区域）
MOVELINK	自动凸轮	定义参考轴和跟随轴，以及同步运动模式
MOVESLINK	自动凸轮 2	定义参考轴和跟随轴，以及同步运动模式

机械凸轮主要由主动件和从动件构成，将旋转运动转换为线性运动，基本结构如下左图。主动件为在金属盘上加工轮廓曲线，一般为匀速旋转运动；从动件一般与凸轮轮廓接触，主动件在运动时，从动件往复运动，运动轨迹由凸轮轮廓决定。



由于机械凸轮机构属于高副机构，故凸轮与从动件之间为点或线接触，不便润滑、易于磨损，有噪声，盘片的加工制造要求较高，维修复杂。因此在很多应用上，使用电子凸轮替代机械凸轮。

电子凸轮指的是用户通过构造凸轮曲线，如上右图，机械凸轮按照凸轮的轮廓可以得出一段转动角度与加工位置运动轨迹，此轨迹为弧线，将该段弧线分解成无数个直线或圆弧轨迹，组合起来得到一串趋近于该弧线的运动轨迹，电子凸轮直接将此段轨迹运动参数装入运动指令，即可控制轴走出目标轨迹，以到达主动件与从动件之间的运动关系，不需要额外的机械结构辅助便可完成凸轮运动，用软件来控制信号，改变程序的相关运动参数就能改变运动曲线，应用灵活性高，工作可靠，操作简单。

正运动控制器为用户提供了多种电子凸轮运动形式：

CAM：凸轮表运动；

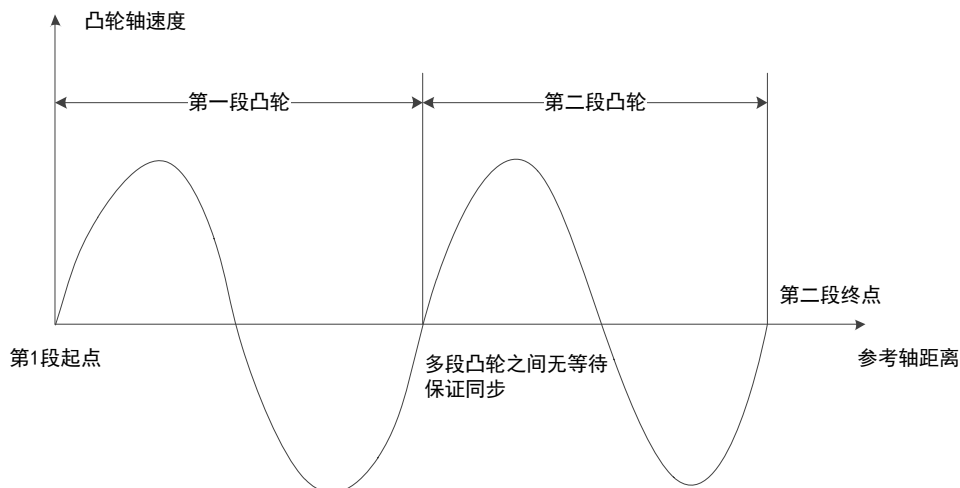
CAMBOX：跟随凸轮表运动；

MOVELINK、MOVESLINK：特定轨迹凸轮运动，又称追剪运动，一般应用于追剪应用；

FLEXLINK：特定轨迹凸轮运动，又称飞剪运动，一般应用于飞剪应用。

凸轮运动过程中不支持运动暂停，凸轮指令执行完成便停止，使用 CANCEL 或 RAPIDSTOP 指令强制取消凸轮运动。

多段凸轮指令被存入运动缓冲区后，下一段凸轮会在前一段凸轮完成后立刻开始，保证同步。



若凸轮指令没有被存入运动缓冲区，多段凸轮间会有等待时间，运动不连续。参见：CAM，CAMBOX。

电子齿轮

相关指令：

指令	说明	用法
CONNECT	连接轴运动	CONNECT（比率，连接轴轴号）
CONNPATH	连接轴运动	CONNPATH（比率，连接轴轴号）
CANCEL	取消连接	可以不带参数直接使用

与凸轮不同，电子齿轮的连接是线性的，电子齿轮功能用于两个轴的连接，将主轴与从轴按照一个常数齿轮比建立连接，不需要物理齿轮，使用指令直接设置电子齿轮的比值，由于是使用软件实现的，故电子齿轮比率可以随时更改，一个主轴能够驱动多个从轴。

电子齿轮功能通过指令 CONNECT、CONNPATH 实现，将一个轴按照一定比例连接到另一个轴上做跟随运动，一条运动指令就能驱动两个电机的运行，通过对这两个电机轴移动量的检测，将位移偏差反馈到控制器并获得同步补偿，这样能使两个轴之间的位移偏差量控制在精度允许范围内。

电子齿轮连接的是脉冲个数，例如主从轴连接比例为 1:5，给主轴发送 1 个脉冲，此时对应给从轴发送 5 个脉冲。

电子齿轮的作用：

1. 脉冲补偿，减少上位机负担（因为目前用的发送脉冲的元件，都有发送脉冲频率的限制）。
2. 匹配电机发出的脉冲数与机械最小移动量，可将指令输入 1 个脉冲对应的工件（或电机）移动量设定为任意值；可实现电机的无极变速，在电机启动和停止时，可以防止失步和过冲现象，这样就能充分发挥电机的潜能。
3. 传递同步运动信息，实现坐标的联动、运动形式之间的变换（旋转-旋转，旋转-直线，直线-直线）、简化控制等。

CONNPATH 与 CONNECT 的相同点：二者的使用语法相同，连接的都是脉冲个数，CONNPATH 连接到单个轴的运动的的效果与 CONNECT 相同。

CONNPATH 与 CONNECT 的区别：CONNECT 连接的是单个轴的目标位置。CONNPATH 是连接的是插补轴的矢量长度，此时需要连接在插补运动的主轴上，连接到插补从轴上无法跟随插补运动。CONNPATH 会跟踪 XY 轴插补的的矢量长度变化，而不是跟踪单独的 X 轴或者 Y 轴。

手轮

相关指令：

指令	说明	用法
CONNECT	连接手轮	CONNECT（比率，连接轴轴号）
CANCEL	取消手轮连接	可以不带参数直接使用

手轮也称为手动脉冲发生器、手脉、手摇脉冲发生器等，属于编码器的一种，用于数控机床、印刷机械等的零位补正和信号分割。当手轮旋转时，编码器产生与手轮运动相对应的信号，选定坐标并对坐标进行定位。

手轮功能是指通过特定的编码器作为手轮脉冲变化输入源，检测编码器脉冲输入变化，使用 CONNECT 指令建立手轮轴与跟随轴的连接，驱动手轮跟随轴运动，该功能主要用于插补运动中辅助运动，指定的编码器可以为端子板上的编码器，也可以是 EtherCAT 总线上的编码器模块。

手轮跟随运动属于位置紧跟型，即手轮脉冲变化 n 个，跟随轴跟随运行 $n \times \text{ratio}$ 个脉冲，速度，加速度按主轴的参数进行规划。

进入手轮运动的条件：跟随手轮的轴处于静止状态，无运动；编码器处于未绑定轴，可用于轴的位置反馈；跟随轴处于使能状态；跟随轴未被设置成手轮模式。

退出手轮模式使用 CANCEL 指令。手轮连接比率可随时切换。

使用流程：设置手轮轴和跟随轴类型，设置各项基本运动参数，使用 CONNECT 指令对手轮和跟随轴按照一定比率进行连接，此时手轮就可以带动跟随轴运动，运动完成 CANCEL 取消手轮连接。

例程：跟随手轮运动

RAPIDSTOP(2)

WAIT IDLE(0)

ERRSWITCH = 3

CONST axishand = 0

BASE(axishand) '选择第 0 轴接手轮

ATYPE=6 '脉冲+方向的手轮，正交输入手轮使用 3

BASE(1) '轴 1 被手轮控制

ATYPE=1 '配置为脉冲轴

DPOS = 0,0

UNITS = 100,100 '脉冲当量，每 mm100 脉冲

SPEED = 200,200

ACCEL = 2000,2000

DECEL = 2000,2000

SRAMP = 20

CLUTCH_RATE = 0 '采用速度加速度来进行限制

DIM poslast

poslast = DPOS

WHILE 1

IF IN(0) = ON AND IN(1) = OFF THEN

CONNECT(1, axishand) '链接到轴 0，倍率 1

ELSEIF IN(1) = ON AND IN(0) = OFF THEN

CONNECT(10, axishand) '链接到轴 0，倍率 10

ELSEIF IN(0) = ON AND IN(1) = ON THEN

CONNECT(50, axishand) '链接到轴 0，倍率 50，对步进，倍率太高会出现丢步或长时间

才能结束

ELSEIF MTYPE = 21 THEN

CANCEL '取消 CONNECT

ENDIF

IF poslast <> DPOS THEN

poslast = DPOS

TRACE DPOS

ENDIF

WEND

END

5.2 插补运动

插补概念

插补是机床数控系统依照一定方法确定刀具运动轨迹的过程，插补是一个实时进行的数据密化的过程，不论是何种插补算法，运算原理基本相同，其作用都是根据给定的信息进行数字计算，不断计算出参与运动的各坐标轴的进给指令，然后分别驱动各自相应的执行部件产生协调运动，以使被控机械部件按理想的路线与速度移动。

插补最常见的两种方式是直线插补和圆弧插补。插补运动至少需要两个轴参与，进行插补运动时，首先需要建立坐标系，将规划轴映射到相应的坐标系中，运动控制器根据坐标映射关系，控制各轴运动，实现要求的运动轨迹。

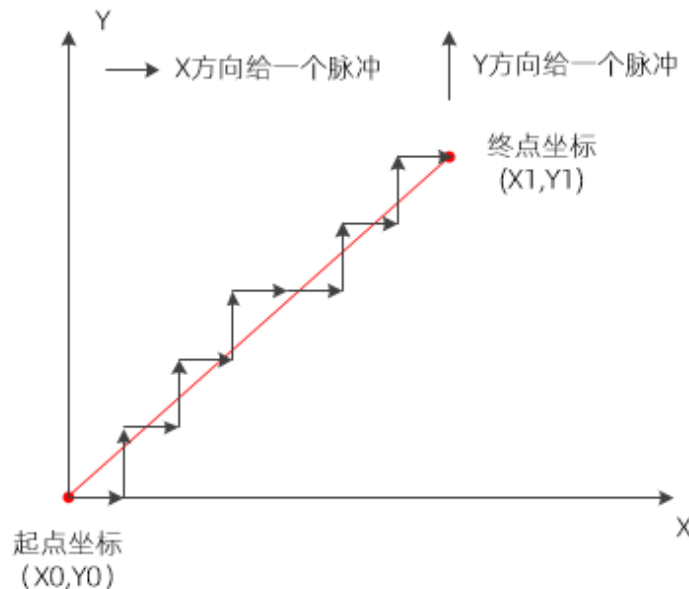
插补运动指令会存入运动缓冲区，再依次从运动缓冲区中取出指令执行，直到插补运动全部执行完。

1. 直线插补原理

直线插补方式中，两点间的插补沿着直线的点群来逼近。首先假设在实际轮廓起始点处沿 X 方向走一小段（给一个脉冲当量轴走一段固定距离），发现终点在实际轮廓的下方，则下一条线段沿 Y 方向走一小段，此时如果线段终点还在实际轮廓下方，则继续沿 Y 方向走一小段，直到在实际轮廓上方以后，再向 X 方向走一小段，依次循环类推，直到到达轮廓终点为止。这样实际轮廓是由一段段的折线拼接而成，虽然是折线，但每一段插补线段在精度允许范围内非常小，那么此段折线还是可以近似看做一条直线段，这就是直线插补。

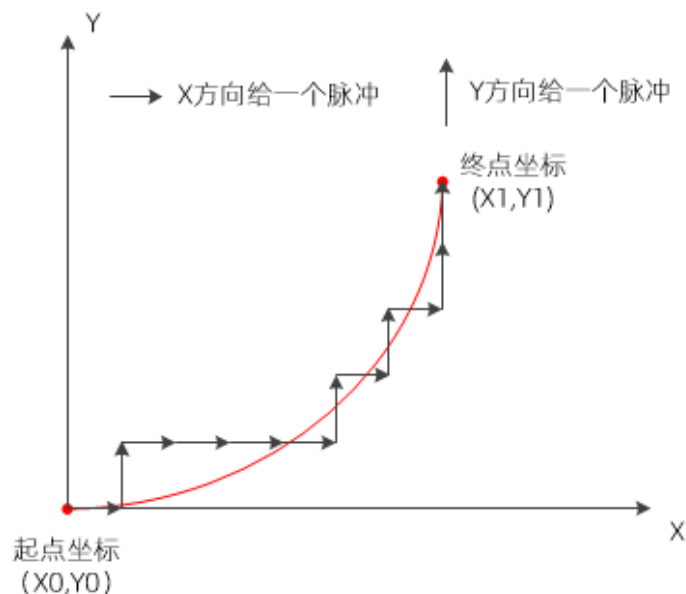
正运动控制器采用硬件插补，插补精度在一个脉冲内，所以轨迹放大依然平滑。

假设轴需要在 XY 平面上从点(X0,Y0)运动到点(X1,Y1)，其直线插补的加工过程如下图所示。



2. 圆弧插补原理

圆弧插补与直线插补类似，给出两端点间的插补数字信息，以一定的算法计算出逼近实际圆弧的点群，控制轴沿这些点运动，加工出圆弧曲线。圆弧插补可以是平面圆弧（至少两个轴），还可以是空间圆弧（至少三个轴）。假设轴需要在 XY 平面第一象限走一段逆圆弧，圆心为起点，其圆弧插补的加工过程如下图所示。



控制器的空间圆弧插补功能是根据当前点和圆弧指令参数设置的终点和中间点（或圆心），由三个点确定圆弧，并实现空间圆弧插补运动，坐标为三维坐标，至少需要三个轴分别沿 X 轴、Y 轴和 Z 轴运动。

3. 运动控制器的插补模式

运动控制器的插补运动模式具有以下功能：

- 1) 可以实现直线插补、圆弧插补、空间圆弧插补、椭圆插补、螺旋插补等；
- 2) 可以在多个坐标系多通道进行多轴插补运动，单通道最多支持 16 轴联合插补；
- 3) 每轴均有运动缓存区，可以实现运动的暂停、恢复等功能，停止插补运动的一个轴，其他轴跟着全部停止；
- 4) 具有缓存区延时和缓存区数字量同步输出的功能；
- 5) 具有预处理功能，控制器自行分析计算目标轨迹，能够实现小线段高速平滑的连续轨迹运动。

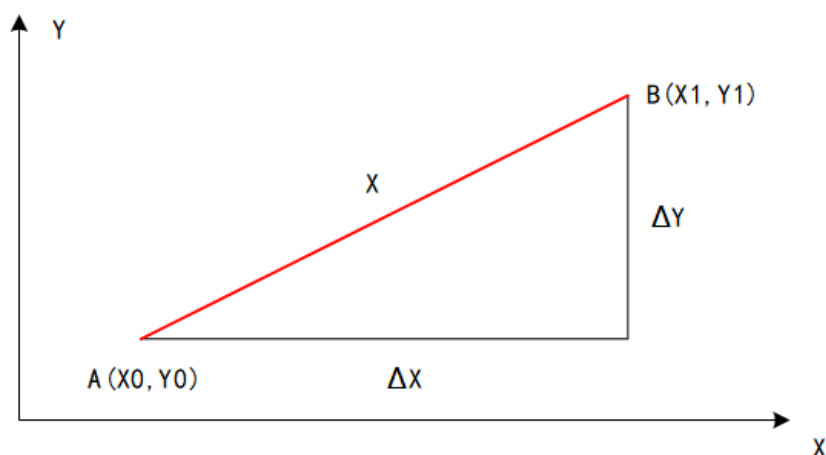
4. 二轴直线插补

轴 0 和轴 1 两轴参与直线插补运动，如下图，2 轴直线插补运动从 A 点运动到 B 点，XY 轴同时启动，并同时到达终点，设置轴 0 的运动距离为 ΔX ，轴 1 的运动距离为 ΔY ，主轴是 BASE 的第一个轴（此时主轴为轴 0），插补主轴运动速度为 S（主轴的设置速度），各个轴的实际速度为主轴的分速度，不等于 S，此时：

插补合成运动距离： $X=[(\Delta X)^2+(\Delta Y)^2]^{\frac{1}{2}}$

轴 0 实际速度： $S_0=S \times \Delta X/X$

轴 1 实际速度： $S_1=S \times \Delta Y/X$



5. 三轴直线插补

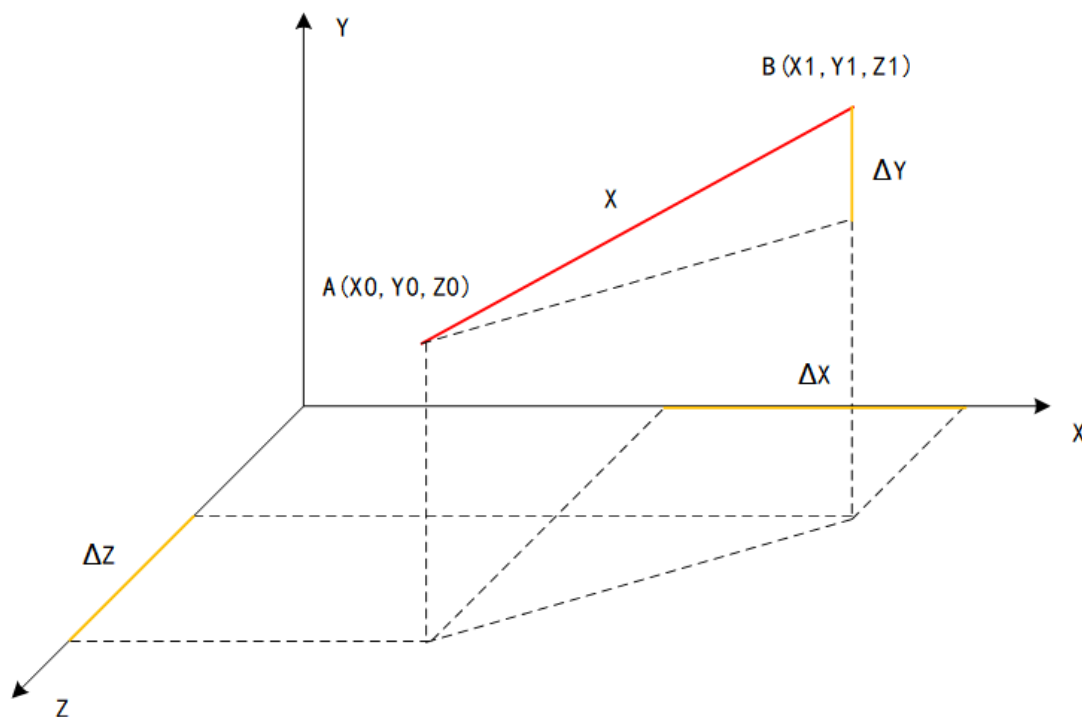
轴 0、轴 1 和轴 2 三轴参与直线插补运动，如下图，3 轴直线插补运动从 A 点运动到 B 点，XYZ 轴同时启动，并同时到达终点，设置轴 0 的运动距离为 ΔX ，轴 1 的运动距离为 ΔY ，轴 2 的运动距离为 ΔZ ，插补主轴轴 0 的运动速度为 S ，各个轴的实际速度为主轴的分速度，不等于 S ，此时：

插补合成运动距离为 $X = [(\Delta X)^2 + (\Delta Y)^2 + (\Delta Z)^2]^{\frac{1}{2}}$

轴 0 实际速度： $S_0 = S \times \Delta X / X$

轴 1 实际速度： $S_1 = S \times \Delta Y / X$

轴 2 实际速度： $S_2 = S \times \Delta Z / X$



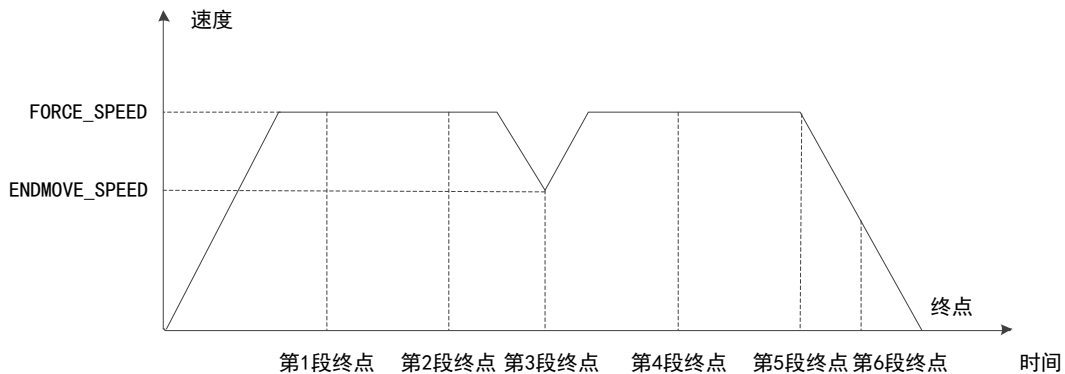
6. 多轴直线插补

多轴直线插补可以理解为轴的多个自由度，是在三维空间里的直线插补。以四轴插补为例，一般是三个轴在 XYZ 平面走直线，另一个轴为旋转轴，按照一定的比例关系做跟随运动。

连续插补

不开启 MERGE 连续插补，上一条插补运动完成后，执行下一条插补时，会先减速停止，再重新加速执行插补运动，实际应用时这种情况会导致加工效率低下，所以需要使连续的插补运动之间不减速，这就是连续插补功能。

若要使插补动作连续，设置 MERGE=ON 以后，相同主轴的插补运动会自动被连续起来，连续两段运动之间不减速，而且 SP 指令可以手动设置运动速度和结束速度，参见：MERGE，SP，CORNER_MODE，ENDMOVE_SPEED，FORCE_SPEED 等指令说明。



5.3 前瞻预处理

前瞻运动相关指令：

指令	说明	用法
CORNER_MODE	拐角模式设置	CORNER_MODE=模式值
MERGE	连续插补	MERGE= ON
DECEL_ANGLE	开始减速的角度	使用时角度单位换算成弧度
STOP_ANGLE	停止减速的角度	使用时角度单位换算成弧度
ZSMOOTH	倒角半径	ZSMOOTH=倒角半径值
FULL_SP_RADIUS	限速半径	曲率限速最大圆弧半径
FORCE_SPEED	强制速度	等比减速以该参数为参考值

在实际加工过程中，为追求加工效率会开启连续插补，运动轨迹的拐角处若不减速，当拐角较大时，会对机台造成较大冲击，影响加工精度。若关闭连续插补，使拐角处减速为 0，虽然保护了机台，但是加工效率受到了较大影响，所以提供了前瞻指令，使在拐角处自动判断是否将拐角速度降到一个合理的值，既不会影响加工精度又能提高加工的速度，这就是轨迹前瞻功能的作用。

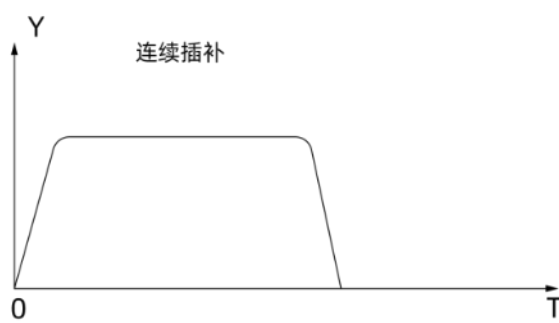
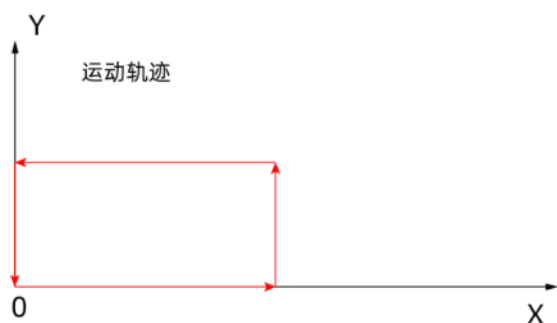
运动控制器的轨迹前瞻可以根据用户的运动路径自动计算出平滑的速度规划，减少机台的冲击，从而提高加工精度。自动分析在运动缓冲区的指令轨迹将会出现的拐点，并依据用户设置的拐角条件，自动计算拐角处的运动速度，也会依据用户设定的最大加速度值计算速度规划，使任何加减速过程中的加减速都不超过 ACCEL 和 DECEL 的值，防止对机械部分产生破坏冲击力。

使用轨迹前瞻和不使用轨迹前瞻的速度规划情况：

假设运动轨迹如下左图，走一个长方形轨迹，分为四段直线插补运动。

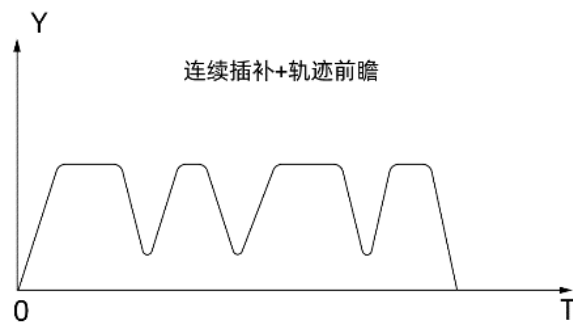
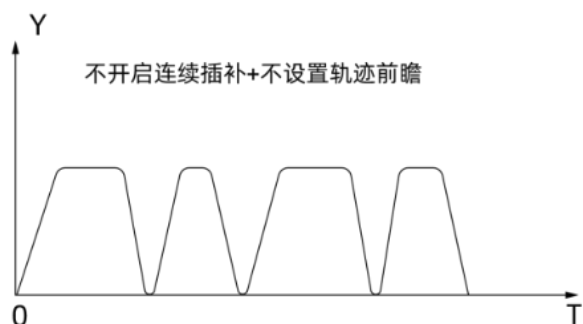
模式一：开启了连续插补后，得出的主轴速度随时间变化的曲线如下右图，主轴的速度是连续的，轨迹

拐角处仍不减速，高速运行时拐角处冲击较大。



模式二：模式一条件下，关闭连续插补，得出的主轴速度随时间变化的曲线如下左图，每走完一段直线后便减速到 0 再开始第二段直线运动，加工效率不高。

模式三：模式一条件下，开启连续插补，并设置了轨迹前瞻参数，得出的主轴速度随时间变化的曲线如下右图，拐角处按照一定的比例减速，加工效率比模式二高。



以上模式若希望速度曲线更柔和，只需通过 **SRAMP** 指令设置速度为 S 曲线。

轨迹前瞻的主要指令 **CORNER_MODE**，用于拐角处的速度规划，常用的三种模式，根据加工的轨迹的实际要求选择不同的模式。

此参数是在插补运动指令调用前生效，一般在参数初始化中设置拐角模式。因为前瞻运动参数会随着运动指令一并存入运动缓冲区，前瞻运动参数可以多次调用，不同模式也可以混合使用，例如 **CONNER_MODE=2+8**，表示同时使用自动拐角减速和曲率限速，根据轨迹段的要求设置合适的前瞻模式，在执行运动指令的时候自动优化轨迹。

注意：**CORNER_MODE** 模式一旦设置参数会存入控制器内，取消需要设置 **CORNER_MODE=0**，否则下次运行时，之前设置的 **CORNER_MODE** 仍会生效。

CORNER_MODE 指令参数说明：

位	值	描述
0	1	预留
1	2	自动拐角减速。 按 ACCEL, DECEL 加减速速度。 此参数是在 MOVE 函数调用前生效。 减速角度根据 DECEL_ANGLE 和 STOP_ANGLE 指令设定。 减速拐角参考速度以 FORCE_SPEED 速度为参考，一定要设置合理的 FORCE_SPEED。
2	4	预留

3	8	自动曲率限速，半径小于设置值时限速，大于限制值时不限速。 此参数在 MOVE 函数调用前修改生效。 限制速度与 FORCE_SPEED 有关。 限制速度= FORCE_SPEED * 实际半径/FULL_SP_RADIUS 限速半径 FULL_SP_RADIUS 设置。
4	16	预留
5	32	自动倒角设置。 此参数在 MOVE 函数调用前修改生效。 此 MOVE 运动自动和前面的 MOVE 运动做倒角处理，倒角半径参考 ZSMOOTH。

CORNER_MODE=2 拐角减速应用场合：不改变运动轨迹，仅在拐角处自动判断是否减速，一般用于改善机台抖动的问题，对轨迹精度有要求，对速度要求没那么快的场合。

CORNER_MODE=8 曲率限速应用场合：不改变运动轨迹，一般应用在圆弧加工，根据圆弧半径计算当前圆弧的限制速度。

CORNER_MODE=32 自动倒角应用场合：改变运动轨迹，不会降低速度，针对轨迹拐角较大的场合，倒角处运动轨迹做自动平滑处理，所以一般应用在对速度要求快，对轨迹精度要求不高的场合。

轨迹前瞻的应用例程参见 [CONNER_MODE](#) 指令。

5.4 原点回零

相关指令：

指令	说明	用法
DATUM	原点回零模式选择	DATUM（回零模式值）
DATUM_IN	映射原点开关	DATUM_IN = 输入编号
FWD_IN	映射正向限位开关	FWD_IN = 输入编号
REV_IN	映射负向限位开关	REV_IN = 输入编号
SPEED	快速找原点速度	设置速度值
CREEP	找原点反向爬行速度	设置速度值
INVERT_IN	输入信号反转	INVERT_IN =（输入编号，ON/OFF）

在高精度自动化设备上都有自己的参考坐标系，工件的运动可以定义为在坐标系上的运动，坐标系的原点即为运动的起始位置，各种加工数据都是以原点为参考点计算的，所以启动控制器执行运动指令之前，设备都要进行回零操作，回到设定的参考坐标系原点，若不进行回零操作，会导致后续运动轨迹错误。

正运动控制器提供了多种回零方式，通过 DATUM 指令设置，不同模式值选择不同的回零方式，各轴按照设置回零的方式自动回零。此指令为单轴回零指令，多轴回零时，需要对每个轴都使用 DATUM 指令。

回零时机台需要接入原点开关（指示原点的位置）和正负限位开关（均为传感器，传感器检测到信号后，表示有输入信号，传给控制器处理）。

单轴找原点时，原点开关通过 DATUM_IN 设置，正负限位开关分别通过 FWD_IN 和 REV_IN 设置。控制器正/负限位信号生效后，会立即停止轴，停止减速度为 FASTDEC。

ZMC 运动控制器为 0 触发有效，输入为 OFF 状态时，表示到达原点/限位，常开类型信号需要采用 INVERT_IN 反转电平。

DATUM 指令支持的回零模式如下表：

值	描述
---	----

0	清除所有轴的错误状态。
1	轴以 CREEP 速度正向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。
2	轴以 CREEP 速度反向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。
3	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关。 找原点阶段碰到正限位开关会直接停止。 爬行阶段碰到负限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS
4	轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关。 找原点阶段碰到负限位开关会直接停止。 爬行阶段碰到正限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。
5	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关，然后再继续以爬行速度反转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。
6	轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关，然后再继续以爬行速度正转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。
8	轴以 SPEED 速度正向运行，直到碰到原点开关。 碰到限位开关会直接停止。
9	轴以 SPEED 速度反向运行，直到碰到原点开关。 碰到限位开关会直接停止。
21	使用 Ethercat 驱动器回零功能，此时 mode2 有效。 设置驱动器回零方式（6098h），缺省 0 表示使用驱动器当前的回零方式。 会使用轴的 SPEED, CREEP, ACCEL, DECEL，乘以 UNITS 后自动设置驱动器的 6099h，609Ah 动作时序： 6060h PDO 切换当前模式→6098h SDO 回零方式→6099h SDO 速度→609Ah SDO 加速度→ PDO 切换控制字启动回零

Z 信号回零必须配置为带 Z 信号 ATYPE。

对于原点在正负限位中间的情况，在各个模式上加 10，表示在回零过程中碰到了限位不取消运动，而是继续反向去找原点，其他条件均与原模式相同。由于原点在正负限位开关之间，因此在回零途中至多遇到一个限位开关。以下回零方式 5~8 均为加 10 模式。

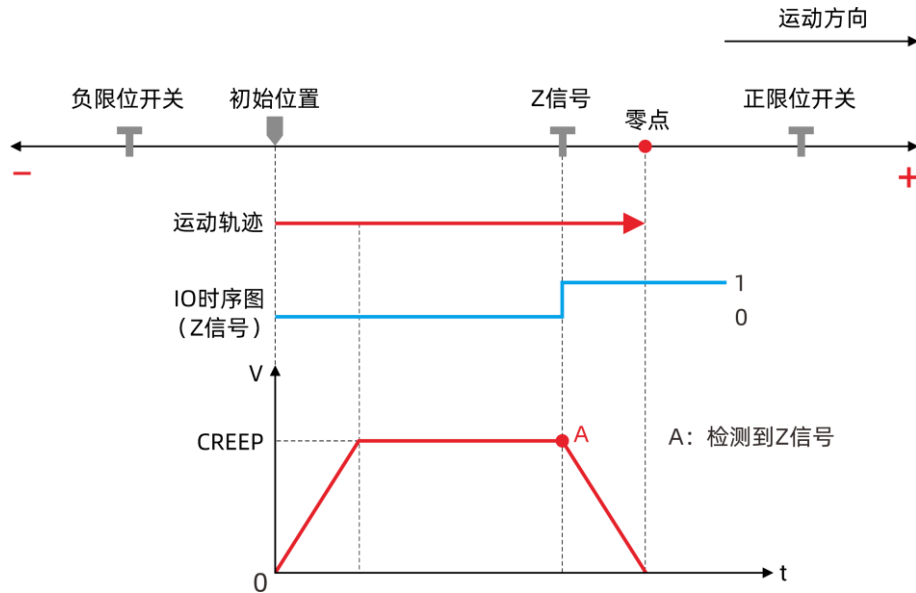
总线控制器使用以上控制器找原点模式完成后，需要手动清零 MPOS。回零模式加 100（模式 100+n 和 110+n 分别对应 n 和 10+n），表示接入编码器后可以自动清零 MPOS(仅限 4 系列，ATYPE=4)

常见回零方式详解：

1. 回零模式 1

如下图，DATUM(1)轴以 CREEP 速度正向运行，检测到 Z 信号后减速停止，停止时所处位置为零点，此时 DPOS 值重置为 0，回零途中若碰到限位立即停止。

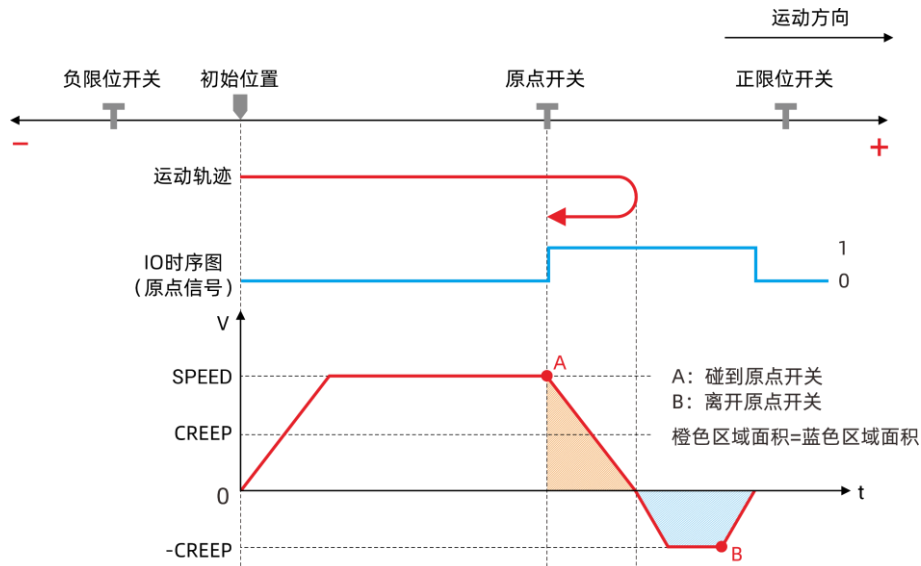
回零模式 2 与模式 1 找原点运动方向相反。



2. 回零模式 3

如下图，DATUM(3)轴先以 SPEED 正向找原点开关，碰到原点开关后开始减速，减速到 0 后再以 CREEP 反向运动，直至离开原点开关。轴停止之后将 DPOS 值重置为 0，当前所处位置为零点，回零途中若碰到限位立即停止。

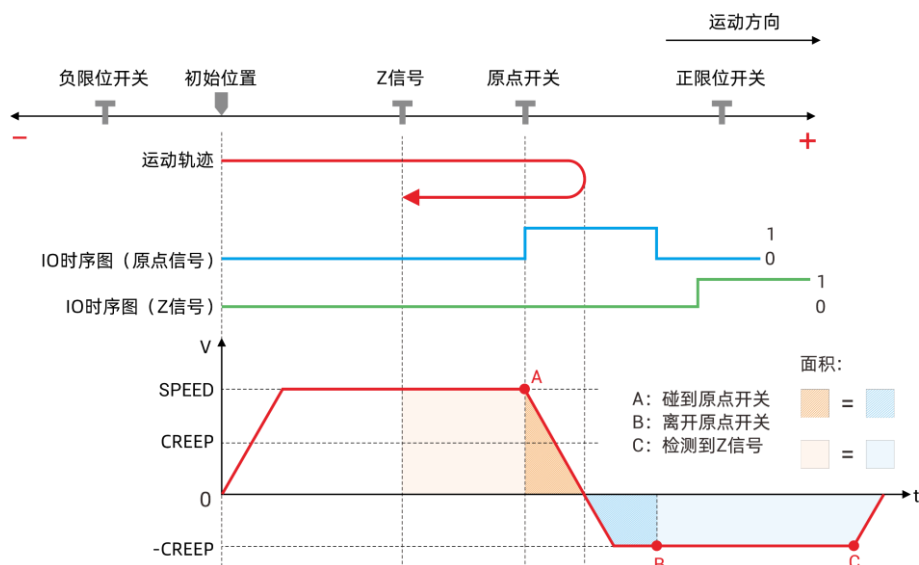
回零模式 4 与模式 3 找原点运动方向相反。



3. 回零模式 5

如下图，DATUM(5)轴先以 SPEED 正向找原点开关，碰到原点开关后开始减速，减速到 0 后再以 CREEP 反向运动，直至检测到 Z 信号减速停止。轴停止之后将 DPOS 值重置为 0，当前所处位置为零点，回零途中若碰到限位立即停止。

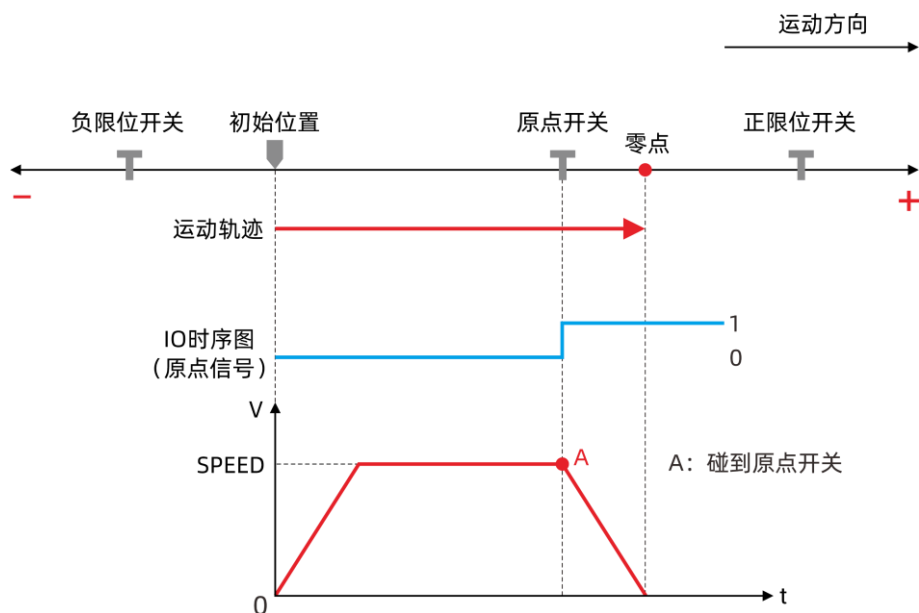
回零模式 6 与模式 5 找原点运动方向相反。



4. 回零模式 8

如下图，DATUM(8)轴以 **SPEED** 速度正向运行，碰到原点开关后减速停止，停止时所处位置为零点，此时 **DPOS** 值重置为 0，回零途中若碰到限位立即停止。

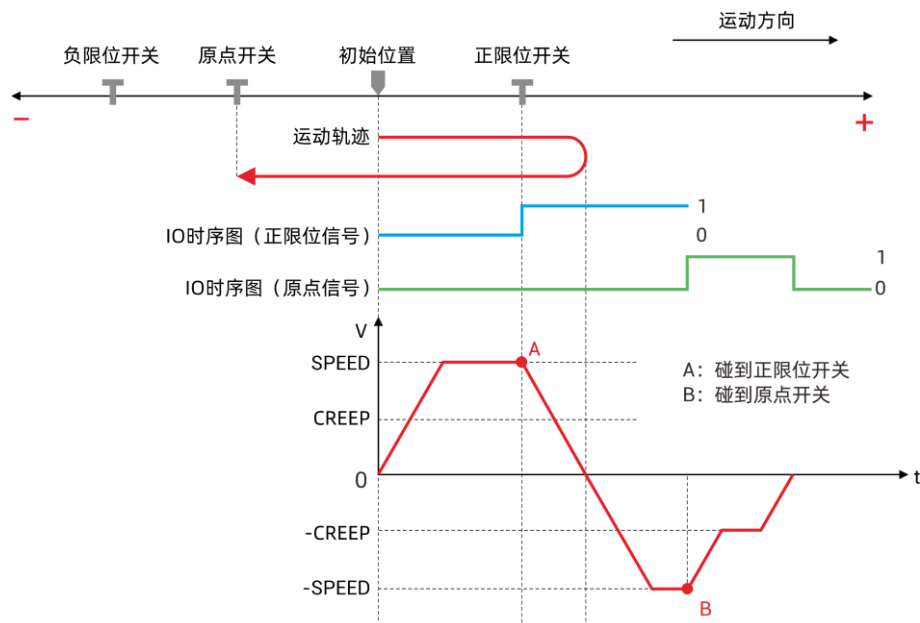
回零模式 9 与模式 8 找原点运动方向相反。



5. 回零模式 13

轴先以 **SPEED** 正向运行，若碰到限位开关，不会报警停止，以 **SPEED** 反向运动，碰到原点开关后减速为 **CREEP** 直至离开原点开关立即停止，回零成功，位置清零。

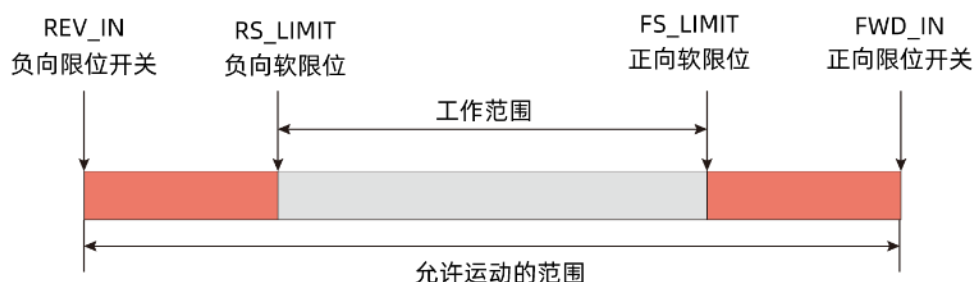
若先碰到原点信号，则与模式 3 相同。



5.5 限位相关指令

指令	说明	用法
FS_LIMIT	正向软限位设置	FS_LIMIT=正限位位置
RS_LIMIT	负向软限位设置	RS_LIMIT=正限位位置
FWD_IN	映射正限位输入	FWD_IN=输入编号
REV_IN	映射负限位输入	REV_IN=输入编号

运动控制器能够通过安装限位开关或者设置软限位来限制各轴的运动范围。硬限位开关和软限位开关用于工艺对象轴的允许运动范围和工作范围。



硬限位开关是限制轴的最大“允许行进范围”的限位开关。硬限位开关是物理开关元件，硬限位开关由指令映射到相应输入开关信号上，根据开关信号是常开还是常闭确定是否要对信号进行翻转，设置完成后，碰到硬限位开关，对应轴立即停止运动，停止减速度为 FASTDEC。

与硬限位开关不同，软限位开关只通过软件程序设定来实现，而无需借助外部的开关元件。软限位开关将限制轴的“工作范围”，由指令直接设置限位位置，轴走到设置位置后立即采用减速度 FASTDEC 停止运动，它们应位于机床限制行进范围的相关硬限位开关的内侧。由于软限位开关的位置较为灵活，因此可根据当前的运行轨迹和具体要求调整轴的工作范围。

工作台碰到限位开关或者规划位置超越软限位时，运动控制器紧急停止工作台的运动。限位触发以后，

轴无法继续运动，此时需要调整轴的位置，使其远离限位位置才能重新开始运动。

轴在碰到限位的时候才会产生停止信号，此时由于减速需要一定的时间，实际轴的位置会越过限位一定距离，假设停止时 SPEED 速度是 v_0 ，快减速 FASTDEC 为 a ，计算公式： $v_t^2 - v_0^2 = 2as$ ，带入下方数据： $0 - 100^2 = 2 * (-1000) * s$ ，得出过冲的距离 $s=5$ ，由此可得，增大 FASTDEC 和减小 SPEED 都能达到减小过冲的目的。

示例：

```

BASE(0)           '选择轴 0
ATYPE=1           '轴类型设置
UNITS=100          '脉冲当量 100
SPEED=100          '速度 100units/s
ACCEL=500          '加速度
DECEL=500          '减速度
FASTDEC=1000       '快减速 100units/s/s
DPOS=0             '坐标清 0
FS_LIMIT=200       '设置正向软限位 200units
MOVE(300)          '运动 300units
WAITIDLE(0)        '等待轴停止
?DPOS(0)           '打印结果：205units
  
```

正/负软限位 FS_LIMIT 和 RS_LIMIT 的值需要位于 -REP_DIST 和 +REP_DIST 之间，软限位参数才起作用，否则正/负向软限位设置无效。取消软限位时，建议不要去修改 REP_DIST 的值，将 FS_LIMIT 和 RS_LIMIT 设置一个较大值即可。

例程：正、负软限位的应用

```

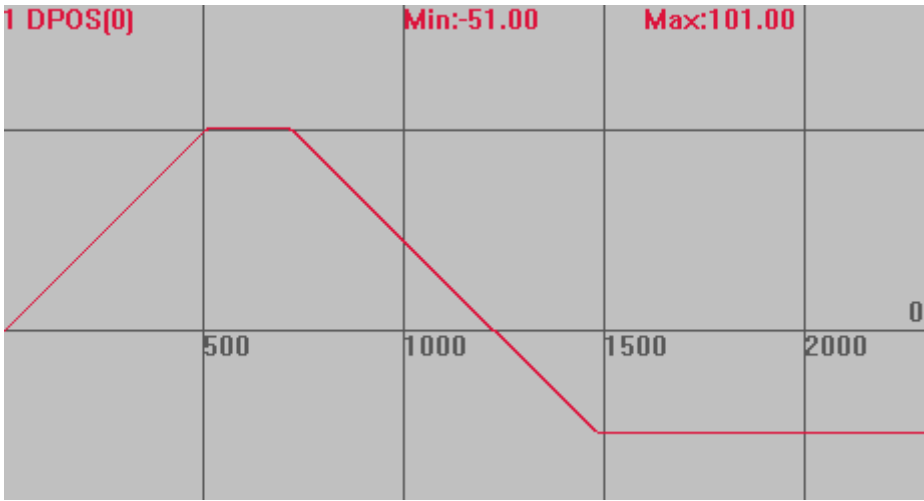
ERRSWITCH = 3
RAPIDSTOP(2)
WAIT IDLE
BASE(0)           '选择 X 轴运动
DPOS = 0
ATYPE=1           '脉冲方式步进或伺服
UNITS = 100       '脉冲当量，每 mm100 脉冲
SPEED = 200
ACCEL = 20000
DECEL = 20000
TRIGGER
'设置软限位
REP_DIST = 100000000 '缺省不用修改这个值
RS_LIMIT = -50       '负向软限位，必须大于 -REP_DIST 才会生效
FS_LIMIT = 100       '正向软限位，必须小于 REP_DIST 才会生效
VMOVE(1)           '正向持续运动
WAIT UNTIL AXISSTATUS AND (512) '判断正向限位是否发生
WAIT IDLE
PRINT "SOFTLIMIT FS", *DPOS
DELAY(200)
VMOVE(-1)          '负向持续运动
WAIT UNTIL AXISSTATUS AND (1024) '判断负向限位是否发生
  
```

```
WAIT IDLE
PRINT "SOFTLIMIT RS", *DPOS
RS_LIMIT = -200000000      '关闭软限位
FS_LIMIT = 200000000
END
```

打印结果:

```
Axis:0 AXISSTATUS:200h,FSOFT
SOFTLIMIT FS    101
Axis:0 AXISSTATUS:400h,RSOFT
SOFTLIMIT RS   -51
```

运动轨迹如下，正向软限位设置的 100，使得轴运动到 100 位置后被迫停止，负向软限位设置的-50，轴负向运动到此位置后无法继续运动。



5.6 位置锁存

相关指令:

指令	说明	用法
REGIST	设置锁存方式	REGIST (方式值)
REG_INPUTS	锁存输入口映射	REG_INPUTS=\$输入口编号
MARK	判断锁存是否触发	WAIT UNTIL MARK
MARKB	判断第二个锁存是否触发	WAIT UNTIL MARKB
MARKC	判断第三个锁存是否触发	WAIT UNTIL MARKC
MARKD	判断第四个锁存是否触发	WAIT UNTIL MARKD
REG_POS	保存锁存的测量反馈位置	打印 REG_POS
REG_POSB	返回锁存 2 的测量反馈位置	打印 REG_POSB
REG_POSC	返回锁存 3 的测量反馈位置	打印 REG_POSC
REG_POSD	返回锁存 4 的测量反馈位置	打印 REG_POSD
OPEN_WIN	锁存触发的开始坐标范围点	OPEN_WIN=POS

CLOSE_WIN	锁存触发的结束坐标范围点	CLOSE_WIN=POS
---------------------------	--------------	---------------

控制器的锁存功能主要用来锁存编码器的位置 MPOS（4 系列及以上控制器最新固件支持虚拟轴、脉冲轴锁存），当锁存信号被触发时，当前位置信息立即被捕获到位置锁存器中，并将前一次锁存的位置坐标清除。读取锁存位置信息时，读取的是最后一次锁存信号触发时锁存的位置信息。不同型号控制器锁存通道口个数以及位置有所差别，参见相应型号控制器的硬件手册。

需要注意的是位置锁存的操作接口是按轴号进行访问的，不同类型的轴锁存参数不同，使用前首先确定轴的类型，支持锁存的轴类型分为以下几种：本地脉冲轴、EtherCAT 轴、RTEX 轴，还需要注意控制器锁存口个数，避免锁存数据溢出导致错误。

脉冲轴类型一般采用 R0，R1，Z 脉冲这三种锁存；总线轴类型一般采用 R2，R3 锁存。

EtherCAT 总线控制器除了支持控制器锁存还支持驱动器锁存，此时使用驱动器 IO 点实现锁存，具体模式查看指令语法。RTEX 只支持控制器锁存。

支持 EtherCAT 驱动器锁存与控制器锁存同时使用时，需要有 4 锁存通道功能。4 个通道指的是 MARK、MARKB、MARKC、MARKD，通过 REG_INPUTS 指定锁存输入口对应的锁存通道。当锁存产生时，轴状态 MARK 会被设置为 ON，同时锁存到的位置会被存储在参数 REG_POS 内。

每个轴的输入信号 R0、R1、Z 信号可以使用锁存功能，R0、R1 输入一般对应到输入口 0 和 1。当使用两个信号锁存时，第二个信号锁存使用 MARKB 和 REG_POSB，MARK 和 REG_POS 需配对使用，即编号一致。

上升沿/下降沿是以控制器内部状态而言，不同类型的输入口可能不一致，需要确认实际锁存的边沿。

锁存功能使用方法：

- 1) 确定当前硬件条件是否满足锁存需求，确定需要锁存位置的轴；
- 2) 设置锁存输入映射口 REG_INPUT，需要输入口支持锁存功能；
- 3) 设置锁存模式 REGIST，等待锁存触发 MARK；
- 4) 锁存完成打印锁存位置信息 REG_POS；
- 5) 可读取锁存位置起始坐标和结束坐标，锁存位置可被其他指令调用。

控制器锁存方式参见 [REGIST](#) 指令描述。

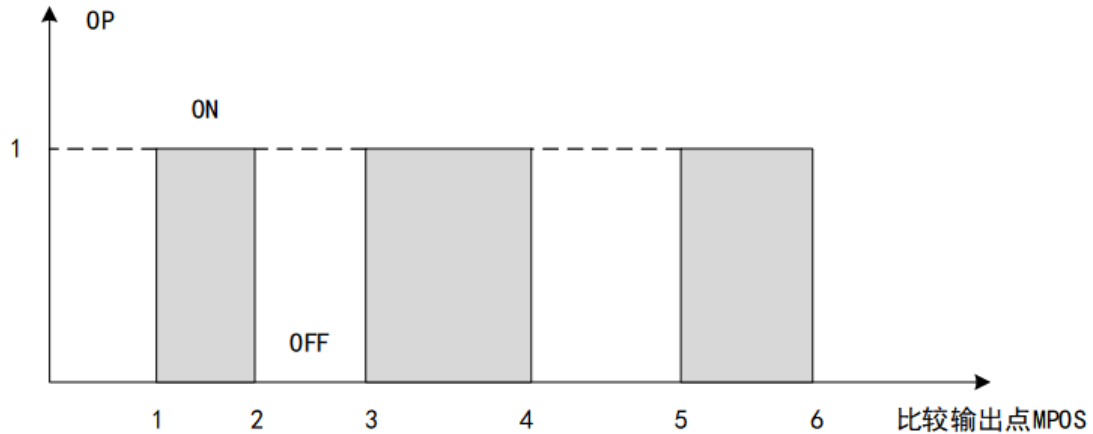
5.7 硬件比较输出

相关指令：

指令	说明	用法
HW_PSWITCH	硬件位置比较输出	设置比较点
HW_PSWITCH2	总线硬件位置比较输出	设置比较点和比较输出口
HW_TIMER	硬件定时输出	周期输出

运动控制器内有位置比较单元，硬件比较输出是通过比较轴是否到达设定位置，来操作输出口动作，一般使用时将编码器位置与设定位置比较，当编码器的位置到达一个设定比较位置时，触发相应输出口电平翻转一次。

如下图所示，到达设置的位置 1，指定输出口电平翻转，到达位置 2 电平再次翻转，到达位置 3 电平再翻转，直到比较完所有的点后，电平维持最后一次翻转后的状态。



硬件比较输出需要 3 系列部分型号和 4 系列及以上控制器支持，需要操作支持此功能的输出口，控制器支持软件比较输出 `PSWITCH` 指令，硬件比较输出 `HW_PSWITCH` 指令（仅支持脉冲轴），`HW_PSWITCH2` 指令（脉冲轴、总线轴均支持）。

对于脉冲轴，`HW_PSWITCH` 和 `HW_PSWITCH2` 的区别是，`HW_PSWITCH` 的轴和输出有一一对应的关系，不需要指定输出轴号；`HW_PSWITCH2` 可以在支持该功能的输出口中指定。`HW_PSWITCH` 指令同一时间可操作多个输出口同时输出。`HW_PSWITCH2` 指令支持的控制器型号更多。

比较编码器的反馈 `MPOS` 位置精度更高，接入了编码器（脉冲轴轴类型 `ATYPE` 为 4 或总线轴类型）便比较编码器反馈位置 `MPOS`，没有接入编码器时（脉冲轴轴类型 `ATYPE` 为 1 或 7）比较目标位置 `DPOS`。

若比较位置为大量连续的等间距输出，可使用 `HW_TIMER` 硬件定时输出，此时需设置起始比较输出位置、间隔距离以及重复周期。

若比较位置为非等间距位置值，使用 `HW_PSWITCH` 和 `HW_PSWITCH2` 指令，`TABLE` 表指定位置进行输出，将需要比较输出的位置数据存储到 `TABLE` 表里，轴走到设定位置后控制输出口开启和关闭，此时需保证 `TABLE` 位置数据在所有比较点完成前不要修改，且 `TABLE` 表中的数据为单调增加的正向距离值或单调减少的负向距离值，否则会出现错误。

比较主轴带编码器输入时，自动使用编码器位置来触发，可以使用 `MOVEOP_DELAY` 参数来调整输出准确时刻。不同的总线驱动器效果可能有差异，也可以通过 `MOVEOP_DELAY` 参数来调整。

5.8 精准输出

相关指令：

指令	说明	用法
<code>MOVE_OP</code>	缓冲中输出	<code>MOVE_OP (编号,状态)</code>
<code>AXIS_ZSET</code>	开启精准输出	按位设置功能
<code>MOVEOP_DELAY</code>	缓冲输出延时	可提前或延时输出

`MOVE_OP` 指令默认为普通输出，普通输出操作需要等待一个控制器周期才可执行，而精准输出操作，可在电机发出一个脉冲内响应，能大大提高工艺的精度，同时可以使用 `MOVEOP_DELAY` 指令调整响应时间（提前或延迟）。

精准输出脉冲输出方式的最小误差 1 个脉冲，总线控制方式的最小误差 1us 以内。

支持硬件比较输出功能的控制器才可使用精准输出功能，二者使用同样的硬件资源。

使用 `AXIS_ZSET` 指令设置是否开启精准输出，开启后再使用 `MOVE_OP` 指令精准输出生效，注意输出通道要选择支持精准输出的通道，也就是支持硬件比较输出的通道，不同型号个数有所区别，一般特殊功能从 IO 编号 0 开始。

精准输出触发方式有两种，目标位置 `DPOS` 触发或编码器反馈 `MPOS` 触发。

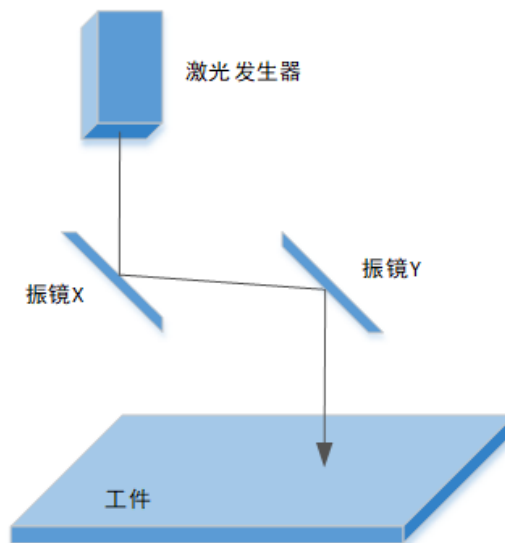
当不带编码器反馈时，精准输出功能自动使用命令位置 `DPOS` 来比较触发，电机总是有一定跟随误差的（跟随误差= $DPOS-MPOS$ ），带编码器反馈时使用编码器反馈 `MPOS` 触发会更准确，是否启动编码器位置同样也是通过 `AXIS_ZSET` 指令配置。根据不同的驱动器差异性效果，也可以使用 `MOVEOP_DELAY` 参数来调整 IO 输出的准确时刻。

5.9 振镜控制系统

振镜说明

1. 振镜工作原理

激光振镜是一种专门用于激光加工领域的特殊的运动器件，它靠两个振镜反射激光，形成 `XY` 平面的运动。激光振镜不同于一般的电机，激光振镜具有非常小的惯量，且在运动的过程中负载非常小，只有两个小的反射镜片 `X` 和 `Y`，分别用不同的电机控制偏转，系统的响应非常快。

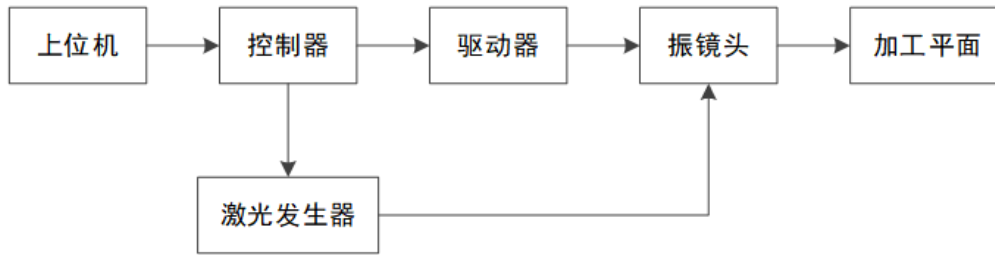


激光振镜运动两种基本的运动：一种为跳转运动，一种为打标运动。

跳转运动的过程中，轴移动到要加工的位置，激光呈关闭状态，不影响轨迹的加工，因此可以以很大的速度运动。打标运动过程中，激光呈开启状态，进行轨迹的加工，因此用户需要根据实际加工要求设置合适的运动的速度。

振镜是一种优良的矢量扫描器件。它是一种特殊的摆动电机（激光振镜），基本原理是通电线圈在磁场中产生力矩，但与旋转电机不同，其转子上通过机械组簧或电子的方法加有复位力矩，大小与转子偏离平衡位置的角度成正比，当线圈通以一定的电流而转子发生偏转到一定的角度时，电磁力矩与回复力矩大小相等，故不能象普通电机一样旋转，只能偏转，偏转角与电流成正比。

2. 振镜控制系统基本结构



振镜系统的由以上几个部分组成一个基础系统，其中振镜头主要元件为 X/Y 两个反射镜片、分别控制 X/Y 镜片旋转的两个电机，根据实际需求还可加入人机操作系统、编码器等。

3. 对控制器的基本要求

因为激光打标机是靠 X/Y 振镜偏转的配合，将激光反射到工作台上，进行精确的雕刻。而振镜的控制是由控制器开环控制的，所以要求必须为线性，即输入信号同偏转角度之间为线性关系。因振镜是快速精密机械，所以要求从一个工作状态到另一个工作状态要求加速度越大越好，这样，打标空等时间就无限小。

振镜运动采用缓冲区运动方式，即用户需要向轴运动缓冲区传递运动及工艺数据，然后启动缓冲区运动，运动控制器则会依次连续执行用户所传递的运动数据，直到所有的运动数据全部运动完成。

在激光振镜运动控制系统中不但有运动的控制，还有激光的控制。如何有效地处理振镜运动和激光开关的配合是一个很重要的问题，只有有效的协调了激光和运动的关系，才能运动出精确的轨迹。

运动控制：打标运动时，激光会按照设定的打标速度沿着给定的打标轨迹运动，在执行打标相关指令时，激光振镜运动控制器会自动开启激光。如果下一条仍是打标指令，激光一直呈开启状态，直到最后一条打标指令结束，或缓存区指令执行完毕，中途在缓冲区若遇到跳转指令，则激光自动关闭，直到遇到打标指令，激光才重新开启。开始运动前为保证打标轨迹正确需调整振镜坐标，同时清空缓冲区。

激光控制：主要包括控制激光的开关控制与发出激光的时长，控制激光的开断使用 OP 指令，激光能量的控制可根据激光器的不同，对应通过模拟量，数字量输出口，以及输出口 PWM 的占空比对应控制能量的大小。

4. 主要应用

主要用于激光打标，包括激光切割、舞台灯光控制、激光打孔等。是一种非接触式、无污染、无磨损的新标记工艺，采用自动化控制，可靠性大大提高。激光打标是利用高能量密度的激光束，随着激光束在材料表面有规律的移动，同时控制激光束的开断，使目标材料表面发生物理或化学变化，激光束就可以在材料表面加工出一个指定的图案。

相比传统标记工艺，激光打标有如下优点：

标记速度快，字迹清晰。

非接触式加工，污染小，无磨损。

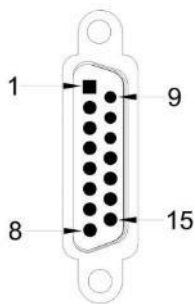
工作方便，防伪能力强。

高速自动运行，生产成本低，运行可靠。

5. ZMC420SCAN 控制器振镜接口信号

ZMC420SCAN 是一款支持激光振镜控制的控制器，自带的每个通用输出口都支持 PWM 功能。

本地轴号 4/5 可以 ATYPE=21 配置为第 1 个振镜，本地轴号 6/7 可以 ATYPE=21 配置为第 2 个振镜，通过 AXIS_ADDRESS 配置可以更改轴号。



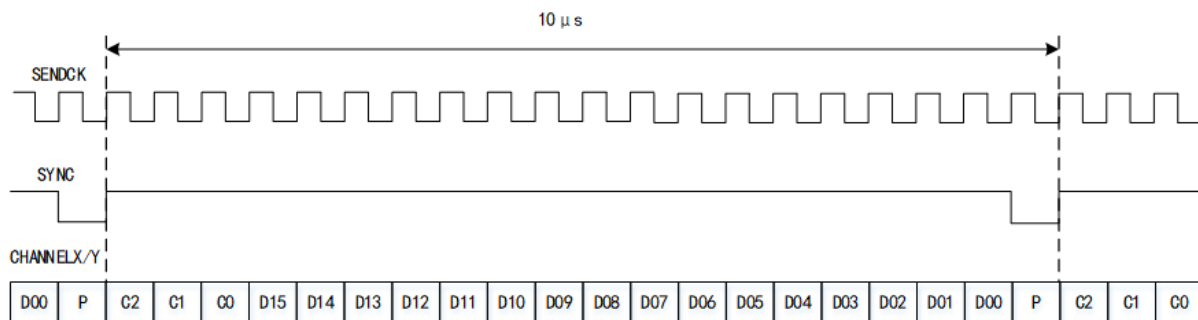
针脚号	信号	说明
1	CLOCK-	时钟信号-
2	SYNC-	同步信号-
3	X channel-	振镜 X 通道信号-
4	Y channel-	振镜 Y 通道信号-
5	NC	保留
6	STATUS-	振镜状态信号-
7	NC	保留
8	GND	数字地
9	CLOCK+	时钟信号+
10	SYNC+	同步信号+
11	X channel+	振镜 X 通道信号+
12	Y channel+	振镜 Y 通道信号+
13	NC	保留
14	STATUS+	振镜状态信号+
15	GND	数字地

6. XY2-100 振镜协议

ZMC420SCAN 支持 XY2-100 振镜协议。

在振镜控制系统中，XY2-100 协议作为数字化激光扫描振镜的接口定义及通信协议被广泛地使用。通讯协议是指双方实体完成通信或服务所必需遵循的规律和约定，XY2-100 协议包括四路信号：SENDCK（时钟信号）、SYNC（同步信号）、CHANNEL X（X 通道数据）、CHANNEL Y（Y 通道数据），这四路信号是一种同步串行传输过程。

数据时序结构如下图所示：



SENDCK 信号是一个频率为 2MHz 的时钟信号，当它从低电平到高电平跳变时，数据位被写入；当它从高电平到低电平跳变时，数据位被反射系统采样。

SYNC 信号用于提供数据转换的同步信息，当它从低电平到高电平时第一位数据被发送；从高电平到低电平时最后一位校验位被发送。

CHANNEL X/Y 是数据信号，它由 20 位组成，其中 C2、C1、C0 是振镜运动方向值，参考值为 001，D15~D0 是数据位，它是 16 位二进制数，用来控制振镜转过的角度大小，最后一位 P 为奇偶校验位，当发送的数据中有偶数个“1”时，对应校验位为“0”，当发送的数据中有奇数个“1”时，对应校验位为“1”。

7. 振镜矫正

振镜一般是需要通过对振镜的矫正来实现，控制振镜移动准确的位置距离，振镜矫正实际上是进行一个理论振镜移动距离与实际振镜移动距离建立一种对应关系，然后在移动的过程中将对应的移动距离与建立的关系相结合，从而达到振镜准确移动位置的目的，达成振镜矫正的效果。

振镜矫正指令如下：

ZSCAN_CORRECT(ixy,imode,imaxline,imaxrow,x1,y1,x2,y2,tableindex)

ixy: 值为 0 或 1，两个振镜选择；0-第一个振镜，1-第二个振镜

imode: 0-关闭矫正功能；1-使用分区矫正

imaxline: 行数，Y 方向的点数为行数

imaxrow: 列数，X 方向的点数为列数

x1,y1,x2,y2: 理论的左下角与右上角的位置

tableindex: 测量的实际坐标开始存储的 table 索引，先 X 再 Y，先第一行（按列数存储），再下一行

振镜矫正点数最多支持 64*64 的矫正点数量，以建立左下角以及右上角的理论坐标，并且通过该理论坐标与写入对应 TABLE 数组中的测量出的实际图形坐标进行对应的处理，对当前连接振镜接口的振镜轴进行矫正，振镜矫正参数是断电不保存的，因此在使用过程中需注意，重新上电后需要重新进行矫正振镜。

振镜是绝对值系统，上电之后控制器就一直处于和电机通信的状态，修改振镜轴 DPOS 会引起振镜的偏移，因此在振镜使用过程中不要随便修改振镜轴的 DPOS 值，如果需要运动，可以通过 MOVEABS 移动到对应的位置。

振镜应用流程

1. 在振镜轴的使用中需先将对应连接的振镜轴 4、5（6、7）设置轴类型为 21。
2. 对对应振镜轴设置轴参数，设置的轴脉冲会影响运动时候的振镜运动距离，因此可将脉冲当量固定为大小，之后通过振镜矫正指令，矫正当前位置的振镜轴运动正确的距离移动。
3. 如果在振镜运动过程中需要对激光的开关光操作，应该选定高速输出口进行控制开关光，并打开对应的精准输出设置，从而达到输出口在达到位置后短时间内进行出光，达成对激光的准确控制。
4. 如果需要振镜轴回零，可以通过 MOVEABS 指令将振镜轴运动到 0 的位置，并且在振镜运动的过程中不能随便进行修改振镜轴的 DPOS 值，否则会造成振镜轴的偏移，对应的振镜电机也会抖动。
5. 振镜轴可以通过轴映射 AXIS_ADDRESS 指令进行轴号调换，通过其他轴号进行操作振镜轴，达到调换轴的目的，除此之外当前振镜轴的方向修改无法通过指令进行修改振镜轴的方向，需要在振镜矫正的部分对需要反向的振镜坐标进行取反操作，之后再重新进行矫正，达到对振镜轴方向修改的目的。
6. 可操作振镜轴与普通电机轴，建立连续插补，建立振镜轴与普通轴之间的联动，实现混合插补。

激光器控制注意事项：

激光器的能量控制有以下控制方式：1.模拟量控制能量。2.数字信号组合控制能量。3.通过 PWM 占空比控制能量的输出。

- 1.模拟量控制能量，模拟量的精度是 10 位的电压大小为 0-10V，数值大小 0-4096 对应控制能量的大小。

2.数字信号组合控制能量，由输出口信号进行组合，能量对应每种组合选择能量。

例：联品激光 mopa 类激光器能量组合如下：

针脚	设置 1	设置 2	设置 3	设置 4	设置 5
针 1	0	0	0	0	1
针 2	0	0	0	0	1
针 3	0	0	0	0	1
针 4	0	0	0	0	1
针 5	0	0	0	1	1
针 6	0	0	1	1	1
针 7	0	1	1	1	1
针 8	1	1	1	1	1
电流	50%	75%	87.5%	93.75%	100%
激光器功率	52%	77%	89%	93%	100%

振镜例程

例一：振镜两轴插补运动

说明：振镜两轴插补实现打标 1 行 5 个 5mm 的小线段圆。

'设置振镜轴轴号，并配置轴类型

BASE(4,5)

ATYPE=21,21

'设置基本参数

UNITS=300,300

SPEED=5000,5000

ACCEL=SPEED*20,SPEED*20

DECEL=SPEED*20,SPEED*20

MOVEABS(0,0)

FORCE_SPEED=5000

'打开连续插补

MERGE=ON

AXIS_ZSET(4)=3 '开启 MOVE_OP 的精准输出功能

'设置频率

PWM_FREQ(2)=2000

WHILE 1

IF MODBUS_BIT(0)=ON THEN

MODBUS_BIT(0)=OFF

OP(0,OFF)

BASE(4,5)

'能量开关

OP(11,ON)

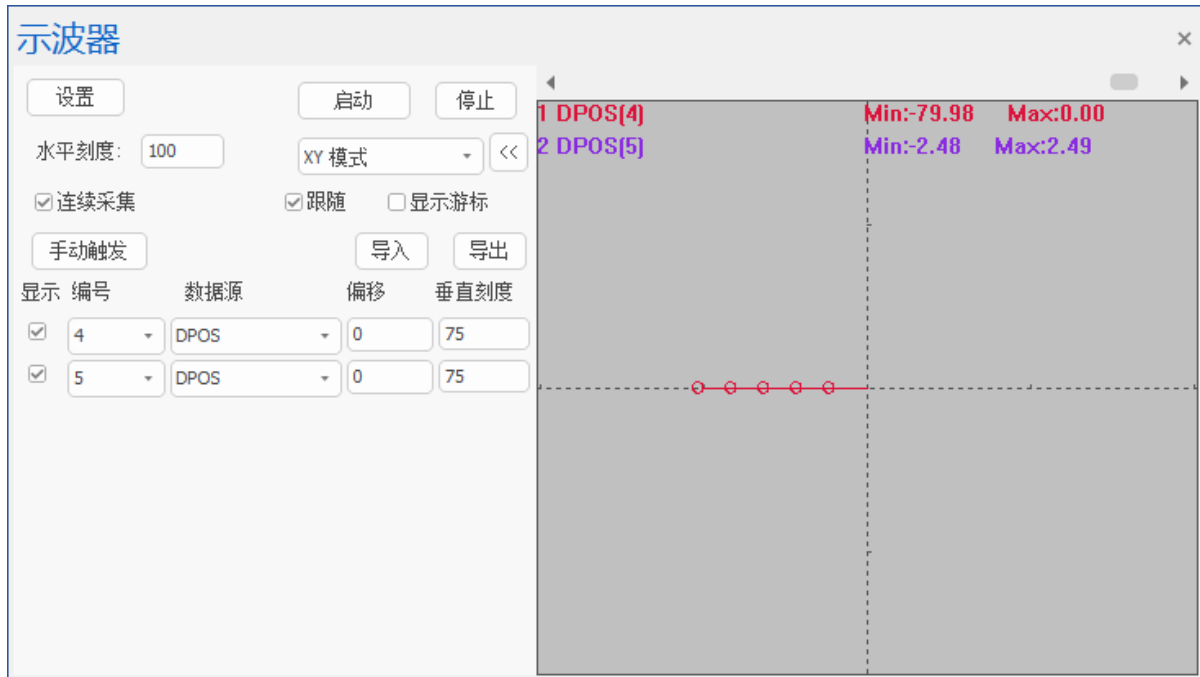
'MO 开关

OP(1,ON)

'打标 5 个小线段圆，轨迹移动数据

```
FOR j = 0 TO 4
    MOVE(-15, 0)
    MOVE_OP(0,ON)
    MOVE(-0.038, 0.434)
    MOVE(-0.113, 0.421)
    MOVE(-0.184, 0.395)
    MOVE(-0.250, 0.357)
    MOVE(-0.308, 0.308)
    MOVE(-0.357, 0.250)
    MOVE(-0.395, 0.184)
    MOVE(-0.421, 0.113)
    MOVE(-0.434, 0.038)
    MOVE(-0.434, -0.038)
    MOVE(-0.421, -0.113)
    MOVE(-0.395, -0.184)
    MOVE(-0.357, -0.250)
    MOVE(-0.308, -0.308)
    MOVE(-0.250, -0.357)
    MOVE(-0.184, -0.395)
    MOVE(-0.113, -0.421)
    MOVE(-0.038, -0.434)
    MOVE(0.038, -0.434)
    MOVE(0.113, -0.421)
    MOVE(0.184, -0.395)
    MOVE(0.250, -0.357)
    MOVE(0.308, -0.308)
    MOVE(0.357, -0.250)
    MOVE(0.395, -0.184)
    MOVE(0.421, -0.113)
    MOVE(0.434, -0.038)
    MOVE(0.434, 0.038)
    MOVE(0.421, 0.113)
    MOVE(0.395, 0.184)
    MOVE(0.357, 0.250)
    MOVE(0.308, 0.308)
    MOVE(0.250, 0.357)
    MOVE(0.184, 0.395)
    MOVE(0.113, 0.421)
    MOVE(0.038, 0.434)
    WAIT IDLE
    MOVE_OP(0,OFF)
NEXT
ENDIF
WEND
```

运动效果图:



例二：振镜轴与普通轴混合插补运动

说明：振镜轴与旋转轴建立插补两轴配合运动，进行打标清洗图形。

清洗长度 58，清洗宽度 30，要清洗的工件放于旋转轴轴 0 上，轴 5 控制激光运动，Y 轴方向往复运动清洗。

```

BASE(0,5)
ATYPE=7,21
UNITS = 10000/360,10000/18
SPEED=1000,5000
ACCEL=SPEED*5, SPEED*5
DECEL=SPEED*5, SPEED*5
MOVEABS(0,0)
MERGE=ON      '打开连续插补
AXIS_ZSET(0)=3  '开启主轴 MOVE_OP 的精准输出功能
OP(12,ON)      '使能脉冲轴轴 0
PWM_FREQ(2)=2000  '设置 DB25 外控激光频率口的大小
PWM_DUTY(11)= 0.8  '设置能量
PWM_FREQ(11) = 2000
  
```

```

WHILE 1
    LOCAL i      '循环条件
    LOCAL sum     '累计旋转角度
    IF MODBUS_BIT(0)=ON THEN
        sum = 0
        MODBUS_BIT(0)=off
    
```

```
OP(0,OFF)
OP(11,ON)      '能量开关
OP(1,ON)       'mo 开关
WA 100

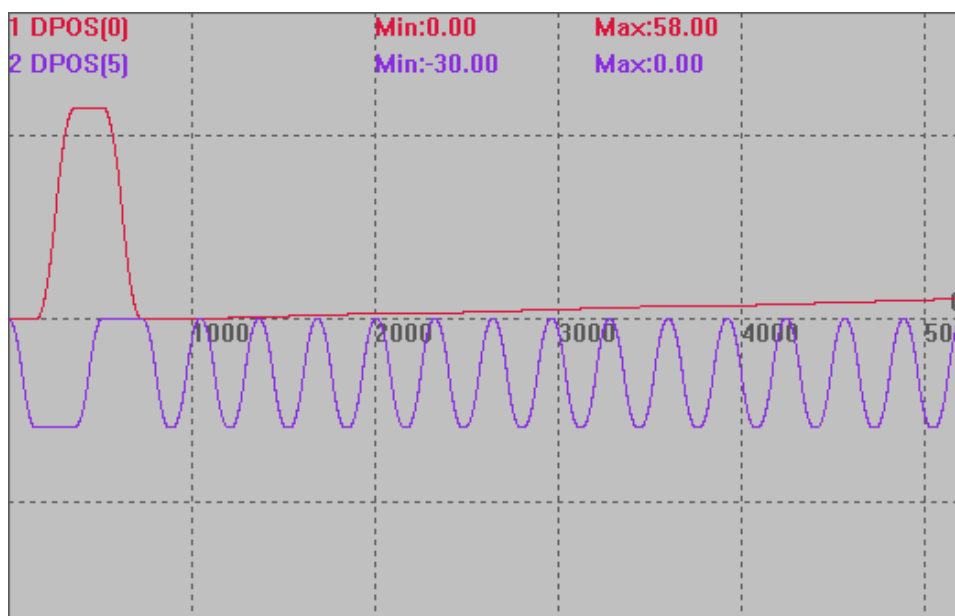
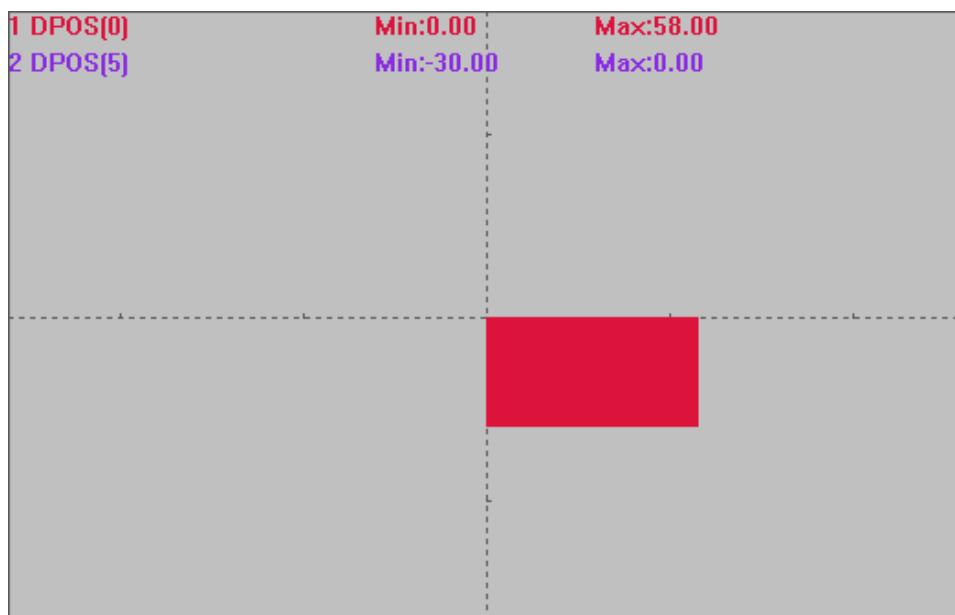
MOVE_OP(0,ON)
TRIGGER

MOVE(0,-30)
WAITIDLE
MOVE(58,0)
WAITIDLE
MOVE(0,30)
WAITIDLE
MOVE(-58,0)
WAITIDLE

'旋转轴旋转一定角度并对旋转轴上面的打标图形进行填充清洗
FOR i = 0 TO 57.6 STEP 0.4      '填充清洗
    sum = sum + 0.4
    MOVE(0,-30)
    MOVE(0.4,0)
    MOVE(0,30)

NEXT
?"圆筒旋转角度", sum
MOVE_OP(0,OFF)
MOVE(-58,0)
ENDIF
WEND
END
```

运动效果图:



5.10 机械手

正运动控制器支持三十多种机械手算法，根据机械手的类型 `frame` 建立机械手连接后使用，能平滑、精准的控制机械手运动，详细使用说明参见“正运动机械手指令手册”。

机械手相关概念

1. 关节轴与虚拟轴

1) 关节轴

关节轴是指实际机械结构中的旋转关节，在程序中一般显示旋转角度。由于电机与旋转关节会存在减速比，所以设置 `units` 时要按照实际关节旋转一圈来设，同时 `table` 中填写结构参数时也要按照旋转关节中心计算，而不是按照电机轴中心计算。

2) 虚拟轴

虚拟轴不是实际存在的，抽象为世界坐标系的 6 个自由度，依次为 X、Y、Z、RX、RY、RZ。可以理解为空间直角坐标系的三个直线轴和绕轴的三个旋转轴，用来确定机械手末端工作点的加工轨迹与坐标。

2. 坐标系

1) 关节坐标系

每个轴相对原点位置的绝对角度，包含机械手所有关节，各关节之间相互独立，坐标单位为角度。操作其中一个关节时不影响其他关节。

2) 直角坐标系

世界坐标系：世界坐标系是被固定在空间上的标准直角坐标系，以机械手的底盘为坐标原点，其位置根据机械手类型确定。虚拟轴操作时就是根据世界坐标系运动，此时各关节会自动解算需要旋转的角度。

用户坐标系：用户对每个作业空间进行定义的直角坐标系，它用于位置寄存器的示教与执行，位置补偿命令的执行等，在没有定义的时候，将由世界坐标系来代替该坐标系。

机械手算法的主要目的是将关节坐标系与直角坐标系建立联系。

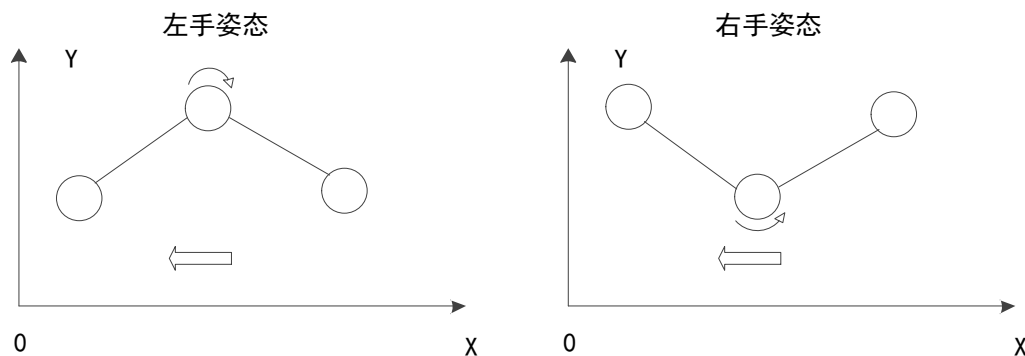
坐标系转换是指在描述同一个空间时，由原来的坐标系转换为另一个坐标系。机械手使用中，常用于确定工件坐标系。

工件坐标系是固定于工件上的笛卡尔坐标系，工件在坐标系相对于世界坐标系存在转换。每个机械手可以拥有若干工件坐标系，用来表示不同的工件，或者表示同一工件在不同的位置。

虚拟轴满足 XYZ 三轴的机械手类型支持此功能。

3. 姿态

机械手姿态在数学上来说，是同一组虚拟轴数值有多组关节轴的解。即机械手在笛卡尔坐标系中运动到某一坐标点，可以有多种运动轨迹，这些运动轨迹就对应着不同姿态，如下图 SCARA 的两种姿态，在 X 方向运动，关节轴可以有两种运动方式。



4. 奇异点

逆解模式下，机械手运动到某一特定位置时，会失去某个自由度，该位置就叫做奇异点，实际使用过程中应避免运动到奇异点。例如当 SCARA 机械手完全伸直时，此时无法在 X 方向平动，需要操作往 X 负方向运动时，结构无法判断使用哪种姿态运动，此时机械手臂无法运动，出现此状况时，先使用正解模式调整关节轴位置，之后再切换到逆解模式使用。

正解运动与逆解运动

机械手建立是通过 [CONNREFRAME](#)（正解）指令和 [CONNFRAME](#)（逆解）指令设置，CONNREFRAME 时虚拟轴 MTYPE（运动类型）值为 34，CONNFRAME 时关节轴 MTYPE 值为 33，可以通过 MTYPE 来查看特定轴是否位于对应的模式。

关节轴和虚拟轴通过 CONNREFRAME 或 CONNFRAME 指令来指明，只要轴数足够，控制器支持多机械手。

程序可以通过运动指令控制关节轴或虚拟轴运动，但同一时刻只能操作虚拟轴或者关节轴，二者无法同时操作。

操作关节轴运动时，虚拟轴需要位于 CONNREFRAME 模式，从而自动指向当前的空间坐标；操作虚拟轴运动时，关节轴需要位于 CONNFRAME 模式，从而自动指向当前的关节轴坐标。

通过 [CANCEL](#) 或 [RAPIDSTOP](#) 指令可以取消机械手模式。

1. 逆解运动

CONNFRAME 对应的运动是逆解运动，此指令作用在关节轴上。此时只能操作虚拟轴，虚拟轴可以在笛卡尔坐标系中做直线、圆弧、空间圆弧等运动，关节轴在 CONNFRAME 的作用下会自动运动到逆解后的位置。

逆解运动模式是指控制器的两种运动模式。机械手保证结束点的位置准确的前提下，会对运动过程的轨迹准确与速度平滑间做个取舍。

逆解运动模式通过设置 [CLUTCH_RATE](#) 连接速度实现，控制器默认 CLUTCH_RATE 值为 1000000。

关节轴的 CLUTCH_RATE	运动模式描述
0	平滑模式：此模式下关节轴使用自己的速度加速度做速度规划，高速时轨迹会有变形。适用于对运动轨迹精度要求不高的场合。
非 0	强制模式：此模式下关节轴完全按照虚拟轴的速度加速度进行规划。此模式可准确回到设定位置，但高速运动时会抖动。

2. 正解运动

CONNREFRAME 对应的运动是正解运动，此指令作用在虚拟轴上。此时只能操作关节轴，关节轴也可以做各种运动，但实际运动的轨迹不是直线圆弧，这种模式一般用于手动调整关节位置或上电点位回零。

关节插补运动是机械手在正解模式下，控制末端点走直线、圆弧等插补运动。

机械手支持的功能

1. 机械手控制

控制机械手末端工作点在世界坐标系下运动。支持多种机械手类型，一个控制器可同时控制多个机械手。机械手有几个电机称为几关节机械手，控制实际机械关节运动的电机轴称为机械手的关节轴，所有的关节轴构成关节坐标系，关节轴在此坐标系中按角度旋转。

2. 坐标系旋转

机械手工作点运动的坐标系，参照世界坐标系进行旋转与偏移。可构建用户坐标系。控制机械手末端工作点在世界坐标系下运动，世界坐标系的坐标轴假想为虚拟轴，按距离单位移动。

3. 机械参数校正

根据机械手关节示教的坐标和特点自动校正当前的机械手参数。

4. 机械手计算

末端工作点坐标与关节轴的坐标之间的计算。

5. 机械手运动仿真

ZRobotView 仿真软件，显示机械手的动作。

6. 控制器仿真

支持离线仿真，在无控制器情况下可以使用。

机械手应用举例

一般来说，生产加工时选择逆解模式，通过给虚拟轴发送坐标位置，控制机械手运动，机械手运动过程中会出现拐角，需要设置拐角减速，防止机台高速拐角时抖动。

编程参考步骤：

1. 参数定义：定义关节长度和各个轴之间的距离，设置各个轴的脉冲当量。
2. 关节轴设置：选择关节轴轴号，设置轴类型、脉冲当量（关节轴脉冲当量需要转换成角度）、速度参数、设置逆解运动模式（CLUTCH_RATE）、拐角减速等。
3. 虚拟轴设置：选择虚拟轴轴号，设置轴类型（ATYPE=0）、脉冲当量。
4. 将机械手参数存储在 TABLE 里。
5. 建立机械手正逆解连接。

六自由度机械手例程：

*****参数定义*****

```

DIM LargeZ      '基座的垂直高度
DIM L1          '1 轴到 2 轴的 X 偏移；转盘中心到大摆臂中心的偏移。
DIM L2          '大摆臂长度
DIM L3          '3 轴中心到 4 轴中心距离
DIM L4          '4 轴到 5 轴的距离。
DIM D5          '5 转一圈，6 转动的圈数，0 表示不关联。
DIM PulesVROneCircle '虚拟姿态轴一圈脉冲数
DIM SmallZ      '末端到 5 轴的垂直距离
DIM SmallX, SmallY '末端到转盘中心的 XY 偏移
DIM InitRx, InitRy, InitRz '初始的姿态，（0，0，0）指向 z 正向

```

*****参数赋值*****

```

LargeZ=50
L1=0
L2=100
L3=0
L4=60
D5=0
SmallZ=10
SmallX=0

```

```

SmallY=0
InitRx=0
InitRy=0
InitRz=0
PulesVROneCircle=360*1000
DIM u_m1      '电机 1 一圈脉冲数
DIM u_m2      '电机 2 一圈脉冲数
DIM u_m3      '电机 3 一圈脉冲数
DIM u_m4      '电机 4 一圈脉冲数
DIM u_m5      '电机 5 一圈脉冲数
DIM u_m6      '电机 6 一圈脉冲数
u_m1=3600
u_m2=3600
u_m3=3600
u_m4=3600
u_m5=3600
u_m6=3600
DIM i_1      '关节 1 传动比
DIM i_2      '关节 2 传动比
DIM i_3      '关节 3 传动比
DIM i_4      '关节 4 传动比
DIM i_5      '关节 5 传动比
DIM i_6      '关节 6 传动比
i_1=1
i_2=1
i_3=1
i_4=1
i_5=1
i_6=1
DIM u_j1      '关节 1 实际一圈脉冲数
DIM u_j2      '关节 2 实际一圈脉冲数
DIM u_j3      '关节 3 实际一圈脉冲数
DIM u_j4      '关节 4 实际一圈脉冲数
DIM u_j5      '关节 5 实际一圈脉冲数
DIM u_j6      '关节 6 实际一圈脉冲数
u_j1=u_m1*i_1
u_j2=u_m2*i_2
u_j3=u_m3*i_3
u_j4=u_m4*i_4
u_j5=u_m5*i_5
u_j6=u_m6*i_6
"关节轴设置"
BASE(0,1,2,3,4,5)      '选择关节轴号 0、1、2、3、4、5
ATYPE=1,1,1,1,1,1      '轴类型设为脉冲轴

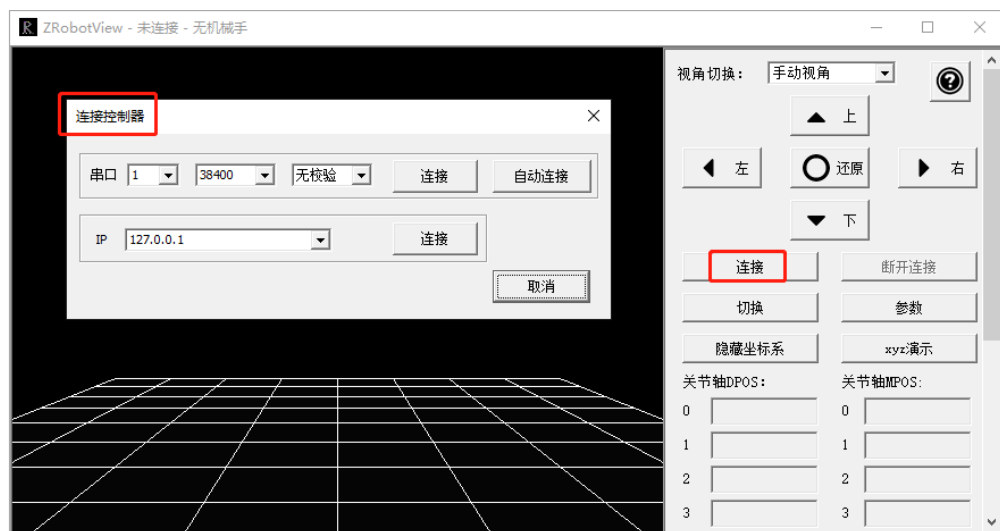
```

```

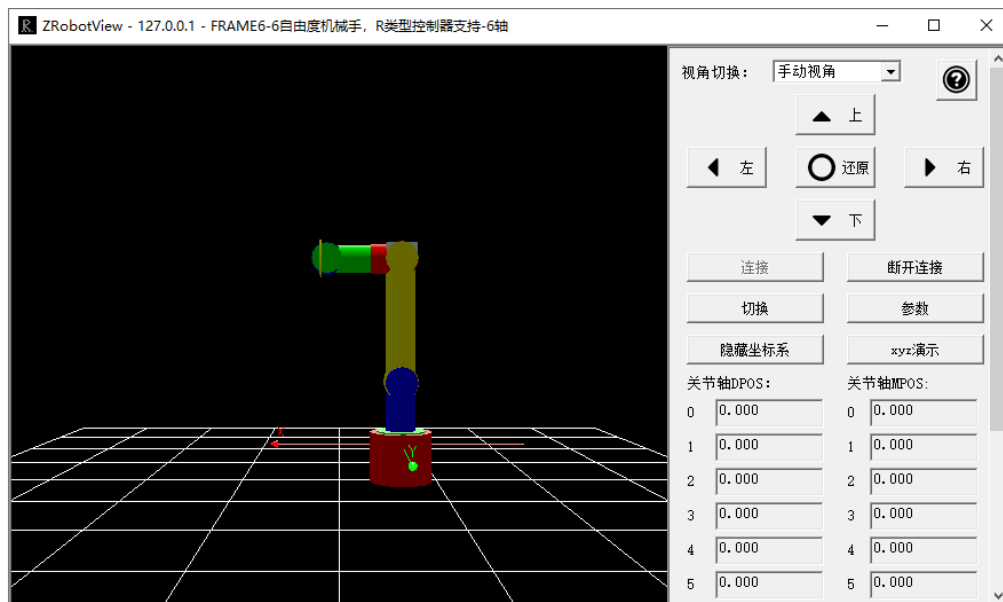
UNITS = u_j1/360,u_j2/360,u_j3/360,u_j4/360,u_j5/360 ,u_j6/360 '把 units 设成每°脉冲数
DPOS=0,0,0,0,0,0 '设置关节轴的位置，此处要根据实际情况来修改
SPEED=100,100,100,100,100,100 '速度参数设置
ACCEL=1000,1000,1000,1000,1000,1000
DECEL=1000,1000,1000,1000,1000,1000
CLUTCH_RATE=0,0,0,0,0,0 '使用关节轴的速度和加速度限制
MERGE=ON '开启连续插补
CORNER_MODE = 2 '启动拐角减速
DECEL_ANGLE = 15 * (PI/180) '开始减速的角度 15 度
STOP_ANGLE = 45 * (PI/180) '降到最低速度的角度 45 度
""""""""虚拟轴设置""""""""
BASE(6,7,8,9,10,11)
ATYPE=0,0,0,0,0,0 '设置为虚拟轴
TABLE(0,LargeZ,L1,L2,L3,L4,D5,u_j1,u_j2,u_j3,u_j4,u_j5,u_j6,PulesVROneCircle,SmallX,SmallY,SmallZ,
InitRx,InitRy,InitRz) '根据手册说明填写参数
UNITS=1000,1000,1000,1000,1000,1000 '运动精度，要提前设置，中途不能变化
""""""""建立机械手连接""""""""
WHILE 1
  IF SCAN_EVENT(IN(0))>0 THEN '输入 0 上升沿触发
    BASE(0,1,2,3,4,5) '选择关节轴号
    CONNFRAME(6,0,6,7,8,9,10,11) '启动逆解连接。
    WAIT LOADED '等待运动加载，此时会自动调整虚拟轴的位置。
    ?"逆解模式"
  ELSEIF SCAN_EVENT(IN(0))<0 THEN '输入 0 下降沿触发
    BASE(6,7,8,9,10,11) '选择虚拟轴号
    CONNREFRAME(6,0,0,1,2,3,4,5) '启动正解连接。
    WAIT LOADED '等待运动加载。
    ?"正解模式"
  ENDIF
WEND
END

```

可启用 ZRobotView 软件仿真，使用方法：将此段程序下载到控制器之后，先建立正解或逆解连接（不建立无法在 ZRobotView 软件上显示机械手），打开 ZRobotView 软件，点击右方“连接”按钮，选择与控制器的连接方式后确认连接（[网口通讯](#)选择与控制器同一 IP 网段，[串口通讯](#)选择与控制器相同串口号、波特率等），此时将会自动建立模拟机械手，仿真运动，还可以使用 RTSys 软件的“手动运动”功能，在该界面通过手动改变轴的坐标模拟机械手运动。



六自由度机械手 ZRobotView 软件仿真图：



5.11 G 代码

ZMC 系列运动控制器作为一个多轴运动控制器，支持标准的计算机数控（Computerized Numerical Control，简称 CNC）功能，实现简易的数控机床控制，同时也可应用于其它一些通过 G 代码进行定位及路径规划的情况。

G 代码（G-code）是最为广泛使用的计算机数控编程语言，主要在计算机辅助制造中用于控制自动机床。

RTBasic 支持 G 代码形式的 SUB 过程，支持标准格式的 G 代码。可根据实际加工需求来自定义 G 代码功能，形成 GSUB 形式来解析 CNC 文件。支持 UG、MasterCam、ArtCAM 等多种 CAD/CAM 软件生成的 NC 加工代码，可应用于雕铣机、精雕机、钻攻中心和加工中心等机床加工场合。

G 代码使用方法参见简易例程章节“[自定义 G 代码](#)”。

第六章 轴相关说明

6.1 轴的概念

在运动控制系统中，将运动控制的对象称为“轴”，运动控制系统中的一个电机控制的运动平台称为一个运动轴，每个运动轴只有一个自由度，可以做直线运动或旋转运动，轴的分类如下：

轴种类	描述
电机轴	使用控制器的脉冲轴接口、EtherCAT 总线或 RTEX 总线接口和驱动器连接，给设备分配轴号，将 1 台电机作为 1 个轴使用。
虚拟轴	运动控制器内的虚拟轴，不使用实际驱动器，或作为同步控制的虚拟主轴和作为机械手算法中的笛卡尔坐标轴使用。
编码器轴	使用控制器本地编码器轴接口，分配为实际编码器输入加以使用。

电机轴：主动运行，电机根据控制器发出的脉冲来运动，发送的脉冲数根据运动参数变化量*UNITS 确定，目标需求位置由 DPOS 参数体现。

编码器轴：被动运行，编码器跟随电机转动，产生脉冲，反馈控制器，控制器接收的脉冲数查看 ENCODER 指令确定，编码器反馈位置由 MPOS 参数体现。

6.2 轴号说明

1. 脉冲轴号

脉冲电机轴：主动运行，根据控制器发出的脉冲来运动，一般分为伺服电机和步进电机。发送的脉冲数根据运动参数变化量*UNITS 确定，目标需求位置由 DPOS 参数体现。

编码器轴：被动运行，跟随电机转动，产生脉冲，反馈控制器，控制器接收的脉冲数查看 ENCODER 指令确定，编码器反馈位置由 MPOS 参数体现。

使用脉冲轴的时候，电机轴号是与其连线的 DB 轴端子接口的编号，印在外壳上，形如 Axis0, Axis1...（没有 DB 接口的请查看相应控制器硬件手册确定轴号）



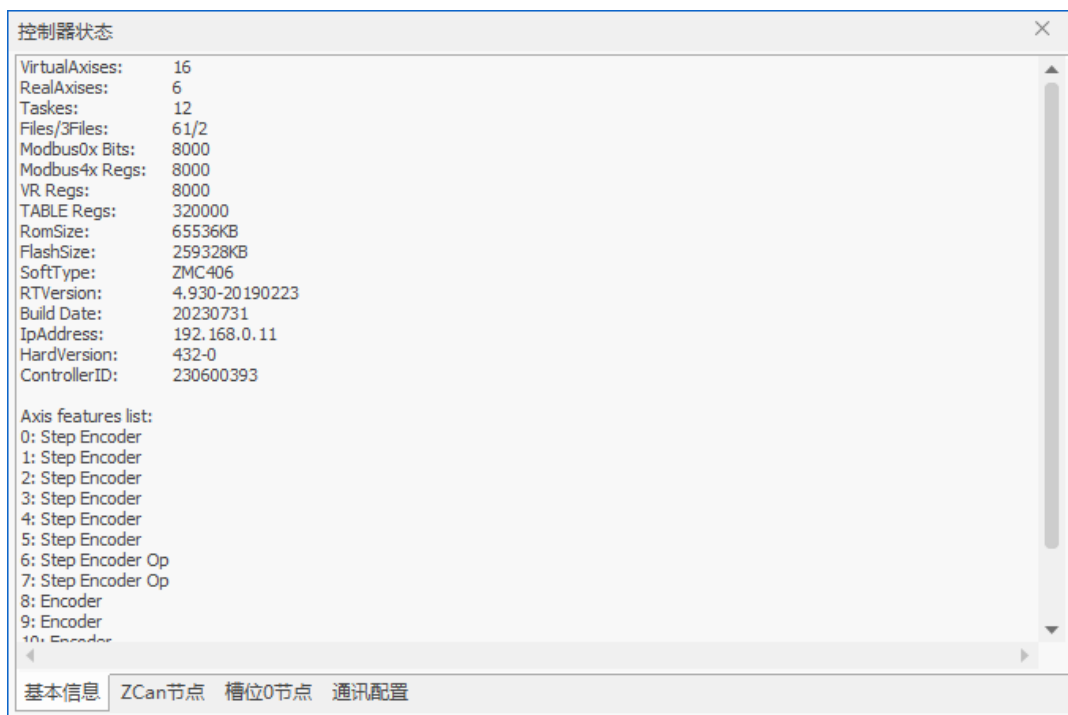
以下方控制器状态为例，每个脉冲轴接口支持接什么类型的轴参见“控制器状态”窗口的 Axis features list 描述，Step 为脉冲输出，Encoder 为编码器反馈。

轴号后备注为“Step Encoder”的就可以配置为既有脉冲输出 Step 又有编码器反馈输入 Encoder，当 ATYPE=4 时满足脉冲输出和编码器反馈在同一个轴号上，此时 DPOS 和 MPOS 都是真实的。当 ATYPE=1 或 7 时，此时只有脉冲输出，接入的编码器反馈在其他轴号上，规则参见下文，DPOS 为真，MPOS 复制 DPOS 的值。

轴号后面只有“Encoder”的就是反馈轴所占的轴号，例如轴 6，反馈轴默认的 ATYPE 都是 3（ATYPE 为 3 时对应正交编码器，可以改为 6，对应脉冲方向型编码器）。

如下图，电机轴号是轴 0，对应的编码器轴号是轴 6，电机轴 1 对应编码器轴 7，依次往下；假设电机脉冲和编码器都连接在 Axis0 接口，那么电机轴号 0，编码器轴号映射到轴 6；假设电机连接在 Axis1 接口，

编码器连接在 Axis2 接口，那么电机轴号 1，编码器轴号 8。



2. 总线轴号

总线轴轴号通过 `AXIS_ADDRESS` 指令来映射总线连接的驱动器设备的轴号；脉冲轴轴号和脉冲控制器轴号一致，电机轴和编码器共用一个轴号。

3. 修改电机运动方向的方法

- 脉冲轴：
- 1) `INVERT_STEP` 指令选择脉冲模式
 - 2) `STEP_RATIO` 指令将分母设为负值
 - 3) 驱动器修改转动方向
- 总线轴：
- 1) `STEP_RATIO` 指令将分母设为负值
 - 2) 驱动器修改转动方向

6.3 轴状态

`AXISSTATUS` 指令查看轴的各种状态。按十进制显示数值，按二进制对应位判断状态，可同时发生多个错误。

轴参数窗口显示的是八进制，使用 `PRINT` 指令打印的值为十进制。

位	说明	打印值	
1	随动误差超限告警	2	2h
2	与远程轴通讯出错	4	4h
3	远程驱动器报错	8	8h
4	正向硬限位	16	10h

5	反向硬限位	32	20h
6	找原点中	64	40h
7	HOLD 速度保持信号输入	128	80h
8	随动误差超限出错	256	100h
9	超过正向软限位	512	200h
10	超过负向软限位	1024	400h
11	CANCEL 执行中	2048	800h
12	脉冲频率超过 MAX_SPEED 限制需要修改降速或修改 MAX_SPEED	4096	1000h
14	机械手指令坐标错误	16384	4000h
18	电源异常	262144	40000h
19	精准输出缓冲溢出	524288	80000h
20	轴速度保护，当轴瞬间速度大于 MAX_SPEED 时发生报警	1048576	100000h
21	运动中触发特殊运动指令失败	2097152	200000h
22	告警信号输入	4194304	400000h
23	轴进入了暂停状态	8388608	800000h

AXIS_STOPREASON 轴历史停止原因锁存，写 0 清除，按位锁存，锁存的是 AXISSTATUS 的信息。

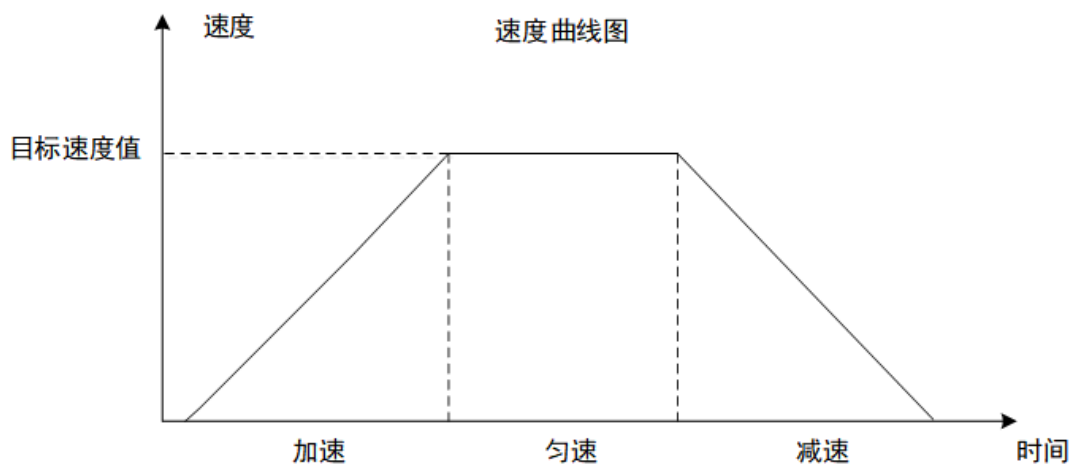
IDLE 指令用于判断加在轴上的运动指令是否完成，运动中返回 0，运动结束返回-1，程序中一般使用 WAIT IDLE(轴号)语句判断轴状态。

MTYPE 指令用于判断轴当前的运动类型，例如 MTYPE 返回值为 1，表示正在进行 MOVE 运动。

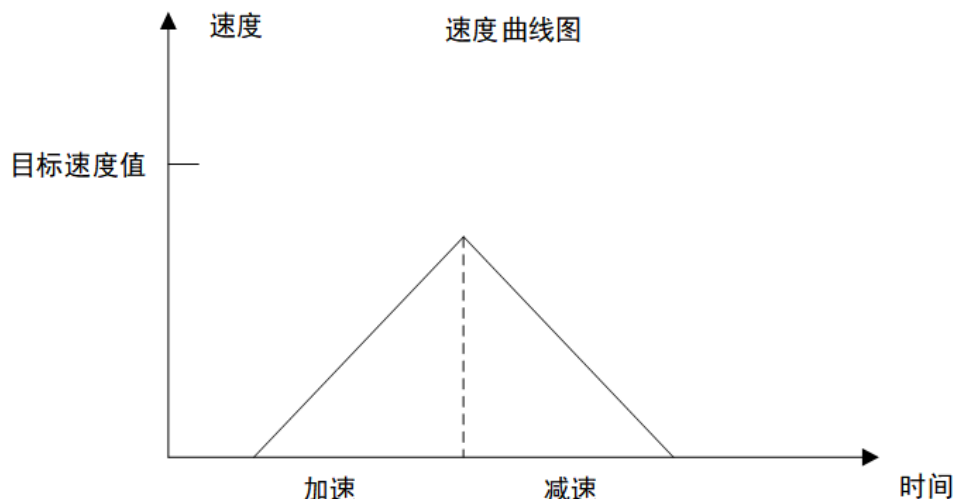
6.4 轴速度

速度曲线

速度曲线一般分为三个阶段：加速阶段、匀速阶段、减速阶段，如下图所示。



当位移较短时，可能不存在匀速阶段，仅有加减速阶段，如下图所示。



常用的速度指令有 SPEED 运动速度，ACCEL 加速度，DECEL 减速度，FASTDEC 快减速等，在轴参数初始化时设置完成，作为运动指令的速度参考。

1. 梯形曲线

不设置 SRAMP（令 SRAMP 等于 0），速度曲线为梯形曲线，在此速度规划方式下，速度曲线按梯形曲线变化。保持速度、加减速等参数值不变。

速度到达设定数值后匀速运动，若只设置加速度，减速度为 0 时，减速度会自动等于加速度的值。一般在运动前就设置好相应的加减速，运动过程中不要修改，运动中调整会导致运动轨迹变化。

例程如下：

```

RAPIDSTOP(2)
WAIT IDLE(0)
BASE(0)
MPOS=0
DPOS=0
UNITS = 100
SPEED = 1000
ACCEL = 10000
DECEL = 10000
SRAMP=0
TRIGGER
MOVE(300)
  
```

此时得到速度曲线如下：此时加减速过程较快，速度变化对机床冲击较大。



2. S 型曲线

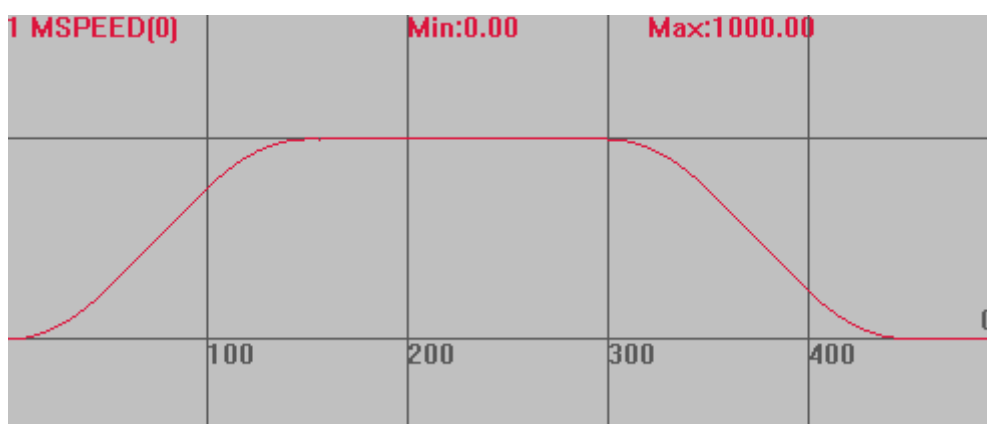
通过设置 SRAMP 的值来设定合适的加减速变化率，使得速度曲线平滑，在机械启停或加减速时减少

抖动。SRAMP 值的范围在 0-250 毫秒之间，设置后加减速过程会变长相应的时间，时间越长速度曲线越平滑。设置时间若超过 250 毫秒，按照 250 毫秒进行平滑。

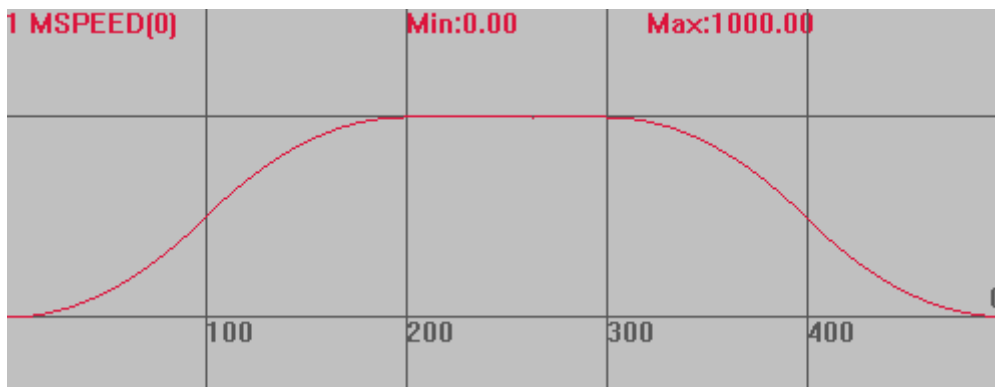
例程：

```
RAPIDSTOP(2)
WAIT IDLE
BASE(0)
DPOS = 0
MPOS = 0
UNITS = 100
SPEED = 1000
ACCEL = 10000
DECEL = 10000
SRAMP=50
TRIGGER
MOVE(300)
```

当 SRAMP=50 时，得到 S 型曲线如下：加减速时更柔和。

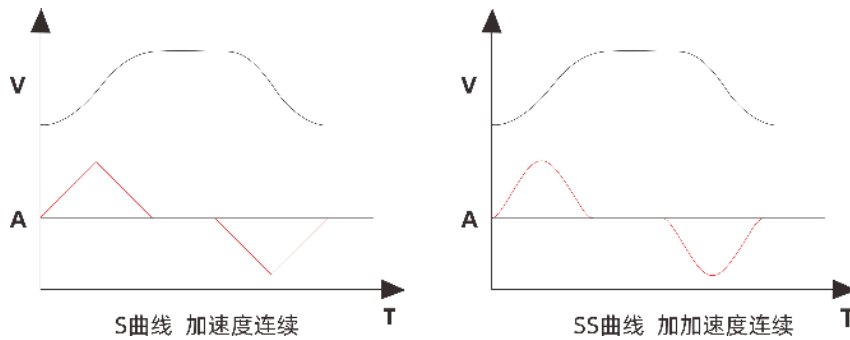


当 SRAMP=100 时，得到 s 型曲线如下：加减速过程变长。



3. SS 型曲线

S 曲线与 SS 曲线都能平滑速度参数，区别参见下图，S 曲线的加加速参数在加减速过程中值是恒定的，SS 曲线使得加加速参数随加减速阶段变化，速度曲线较 S 曲线更平滑，减少轴的抖动。SS 曲线由 VP_MODE 指令配置，指令多种模式可选。



例程：S 曲线和 SS 曲线的对比

BASE(0,1) '选择轴 0, 1

ATYPE=1,1

UNITS=100,100

DPOS=0,0

MPOS=0,0

SPEED=100,100

ACCEL=1000,1000

DECEL=1000,1000

SRAMP=100,100

VP_MODE=7,0 '轴 0 模式 7，轴 1 模式 0

TRIGGER

MOVE(25) AXIS(0)

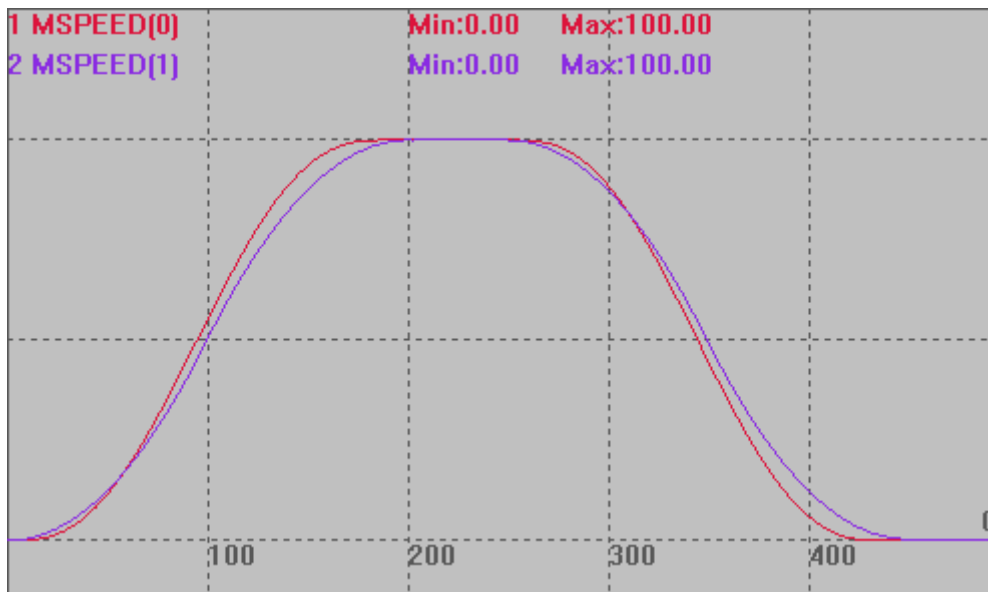
MOVE(25) AXIS(1)

END

速度曲线：模式 7 对加减速度阶段均做了处理。

MSPEED(0)垂直刻度 50，SS 曲线加减速开始和结束阶段更为平滑。

MSPEED(1)垂直刻度 50，S 曲线。



SP 速度

SP 速度应用于插补运动指令带 SP 后缀之后的指令（例如 MOVESP、MOVECICRSP），此时运动速度采用 FORCE_SPEED 参数，而不是 SPEED 参数。

起始速度 STARTMOVE_SPEED：自定义速度的 SP 运动的开始速度。

结束速度 ENDMOVE_SPEED：自定义速度的 SP 运动的结束速度。

强制速度 FORCE_SPEED：自定义速度的 SP 运动的强制速度。

以上三个参数只有使用了带 SP 的运动指令才生效，参数都被带入运动缓冲。

不使用时请将 STARMOVE_SPEED 和 ENDMOVE_SPEED 设置较大值，否则下一段运动指令还会沿用此参数。

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

BASE(0,1) '选择 XY 轴

DPOS = 0,0

MPOS = 0,0

ATYPE=1,1 '脉冲方式步进或伺服

UNITS = 100,100 '脉冲当量

SPEED = 100,100

ACCEL = 200,200

DECEL = 200,200

SRAMP=100,100 'S 曲线

MERGE = ON '启动连续插补

TRIGGER

'第一段

FORCE_SPEED = 50 '第一段速度 50

STARTMOVE_SPEED = 20 '第一段起始的速度 20

ENDMOVE_SPEED = 10 '第一段结束的速度 10

MOVESP(40,40)

'第二段

FORCE_SPEED = 60 '第二段速度 60

STARTMOVE_SPEED = 30 '第二段的起始速度 30

ENDMOVE_SPEED = 40

MOVESP(50,50)

'第三段

FORCE_SPEED = 80 '第三段速度 80

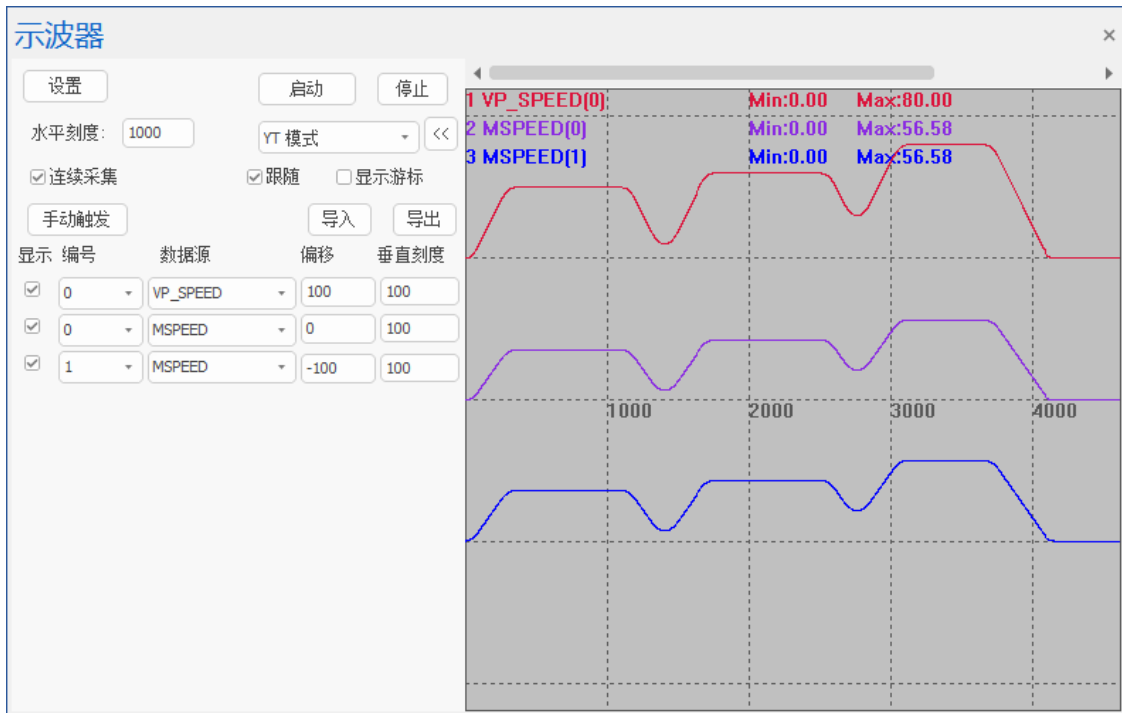
STARTMOVE_SPEED = 30 '第三段的起始速度 30

ENDMOVE_SPEED = 20

MOVESP(60,60)

END

速度变化曲线：从速度为 0 开始运动，第一段的 STARTMOVE_SPEED = 20 不起作用，第一段结束速度 ENDMOVE_SPEED = 10 表示速度降为 10 后第一段运动完成；第二段运动实际从速度 10 开始运动，到 ENDMOVE_SPEED = 40 结束；第三段的起始速度 STARTMOVE_SPEED = 30，小于第二段结束速度 40，第二段完成后速度会降到 30，第三段完成后后面没有运动指令了，所以速度降为 0，ENDMOVE_SPEED 不起作用。



6.5 轴映射

使用控制器的本地脉冲轴时，是不需要轴映射的，采用默认的轴编号即可，参见[轴号说明](#)章节。

使用总线轴和扩展的脉冲轴时，均需要先映射绑定轴号之后才能使用，脉冲轴若要更改默认轴号，支持重新映射配置轴号，映射轴号均采用 [AXIS_ADDRESS](#) 轴映射指令，不同类型的轴映射的语法不同。

轴号可以随意映射，但要在控制器支持的轴数范围内，且映射的轴编号不能重复，一般按顺序依次映射，不容易出错，不同类型的轴通道号排序分别独立，轴号均从 0 开始。

支持本地脉冲轴与 EtherCAT 轴混合插补。映射轴号之后即可调用扩展轴资源。

EtherCAT 的轴号和本地脉冲轴的轴号为各自独立的编码顺序，例如，在某次配置中需要用到两个本地脉冲轴和两个 EtherCAT 轴，配置时轴映射关系如下：

AXIS 0——本地脉冲轴 0；

AXIS 1——本地脉冲轴 1；

AXIS 2——EtherCAT 轴 0；

AXIS 3——EtherCAT 轴 1；

配置之前要将 AXIS 0-3 设为虚拟轴 ATYPE=0，再使用 [AXIS_ADDRESS](#) 指令映射驱动器的轴号，配置完成之后根据轴的特性配置 ATYPE，后续便可对轴 0-3 发送命令。

默认配置文件是按照所接硬件资源的通道总数进行配置。如果硬件资源大于软件资源，默认映射是将所有的软件资源按顺序映射至相应的硬件资源，多余的未映射硬件资源不可控。

注意，一个多轴驱动器上可以连接多个电机，一个电机表示一个轴，每个电机都需要轴号映射，此时相当于该驱动器可以控制多个轴。

6.6 轴类型

轴类型使用 ATYPE 指令根据当前轴的特性去配置，在用户程序初始化时应第一时间完成轴类型的配置，类型不匹配会报错。

所有未分配的轴默认为虚拟轴，ATYPE 的值为 0。

控制器支持的轴类型如下：

ATYPE 类型	描述
0	虚拟轴
1	脉冲方向方式的步进或伺服
2	模拟信号控制方式的伺服
3	正交编码器
4	脉冲方向输出+正交编码器输入
5	脉冲方向输出+脉冲方向编码器输入
6	脉冲方向方式的编码器
7	脉冲方向方式步进或伺服+EZ 信号输入
8	ZCAN 扩展脉冲方向方式步进或伺服
9	ZCAN 扩展正交编码器
10	ZCAN 扩展脉冲方向方式的编码器
20	振镜类型，带振镜状态反馈 振镜连接不上 AXISSTATUS 的 bit2 会置位，ENCODER 返回原始的发送位置，脉冲单位 ZMC408SCAN 支持
21	振镜轴类型，需要控制器支持 缺省系统周期 250us，振镜刷新周期 50us，与固件有关 可以使用普通轴的所有运动控制指令，支持振镜轴与其它轴类型混合插补
22	振镜轴类型，带振镜位置反馈 振镜连接不上 AXISSTATUS 的 bit2 会置位，振镜报警 AXISSTATUS 的 bit3 会置位 MPOS 返回反馈位置，做了反矫正处理，ENCODER 返回原始的反馈位置脉冲单位 ZMC408SCAN 支持
24	远程编码器轴类型 ZHD500X 上手轮使用，部分控制器支持
25	定义一个编码器轴，坐标位置从 MODBUS 或 NODE_PDOBUF 读取 Version_build 230810 后版本支持 BASE(axisnum) AXIS_ADDRESS = (slot << 16) + nodenum ENCODER_ID = (index << 16) + (subindex << 8) + bites ATYPE=25 Slot:

	-1: 从 MODBUS_LONG(nodenum) 里面取编码器位置 0-n: 从 NODE_PDOBUF(slot, nodenum, index, subindex ,type)取编码器位置 ENCODER_ID: 存储数据字典的编号
26	自定义编码器: 使用 C 语言来更新编码器的位置. 支持闭环 version_build 240702 增加.
48	SSI 绝对式编码器
49	BISS 绝对式编码器
50	RTEX 周期位置模式, 需 Rtex 控制器
51	RTEX 周期速度模式, 需 Rtex 控制器
52	RTEX 周期力矩模式, 需 Rtex 控制器 请先关闭驱动器 2 自由度控制模式, 并设置速度限制
65	EtherCAT 周期位置模式, 需支持 EtherCAT
66	EtherCAT 周期速度模式, 需支持 EtherCAT Profile 要设置为 20 或以上
67	EtherCAT 周期力矩模式, 需支持 EtherCAT PROFILE 要设置为 30 或以上
70	EtherCAT 自定义操作, 只读取编码器, 需支持 EtherCAT

1. ATYPE=0 虚拟轴

多轴同步运动时可作为机器的主轴, 从轴都跟随此虚拟主轴。

作为其他轴的叠加轴, 给实际运动的轴叠加一个虚拟轴, 这些虚拟轴可以用 ADDAX 指令 (轴叠加) 进行设置, 然后将各个虚拟轴的运动叠加到实轴。

2. ATYPE=1 或 7 脉冲轴

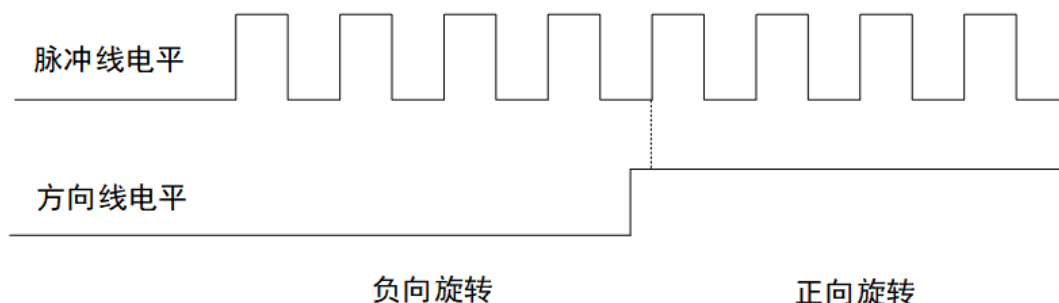
轴的运动由控制器发脉冲控制, 脉冲的方向决定电机的转向, 根据发送脉冲的频率控制轴运动的快慢。

控制器脉冲输出的三种模式: 脉冲+方向、双脉冲、正交脉冲, 采用 INVERT_STEP 指令配置, 默认是脉冲+方向模式。

1) 脉冲方向模式:

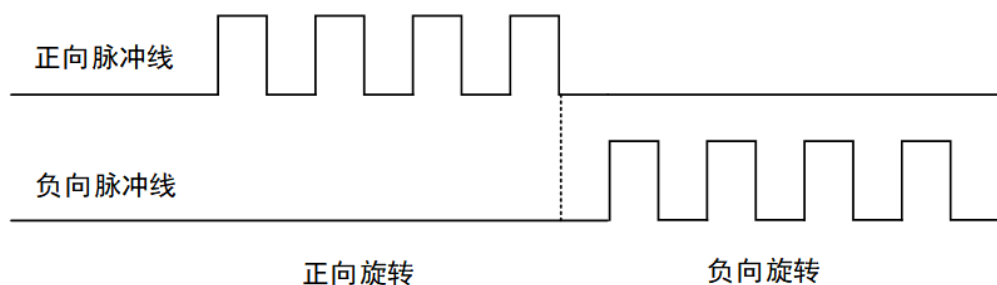
PUL+、PUL-输出指令脉冲串, 脉冲数对应电机运行距离, 而脉冲频率对应电机运行速度。

DIR+、DIR-输出方向信号, 该信号的不同电平对应电机不同的转动方向。此种模式在驱动器中最多。



2) CW/CCW: 双脉冲工作方式

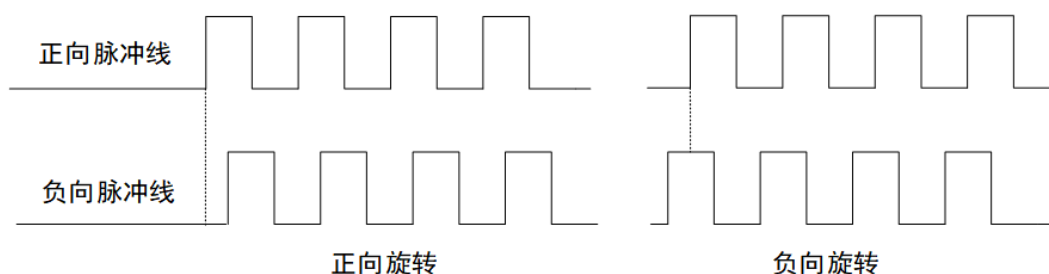
两根线都输出脉冲信号, CW 为正方向脉冲信号, CCW 为反方向脉冲信号, 通常都是差分方式输出, 两信号相位相差角度, 根据相位超前或滞后来决定。



3) AB 相：正交脉冲工作方式

指两个相互独立的相同脉冲信号（都是方波），正方向脉冲信号在负方向脉冲信号之前产生，二者相位相差 90 度，此时为正向旋转；负方向脉冲信号在正方向脉冲信号之前产生，二者相位相差 90 度，此时为负向旋转。

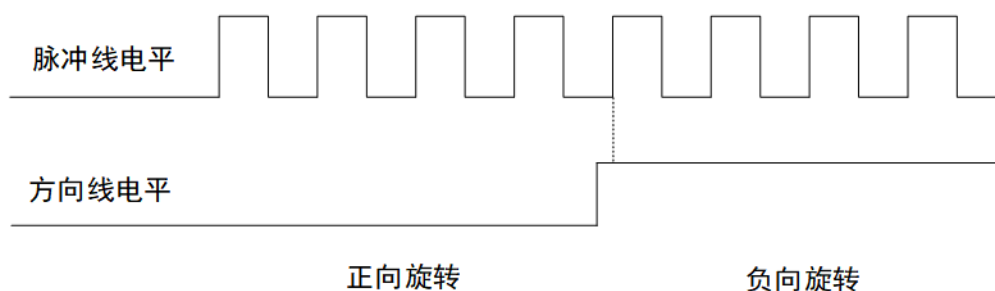
通过两个脉冲之间的相位差来达到计数或编码的作用。



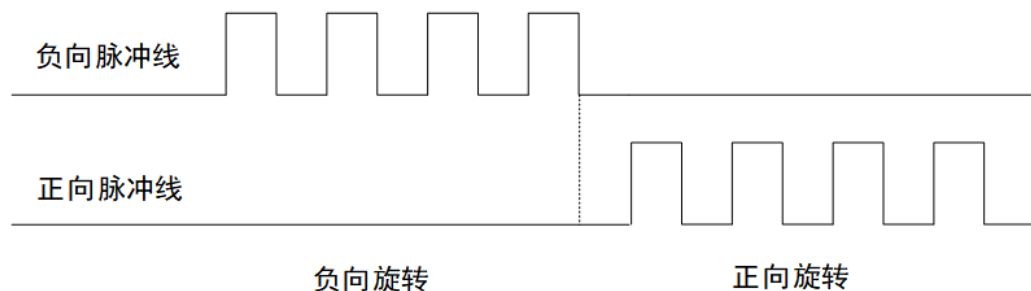
极性对调

若切换脉冲线的正负，即原本正方向脉冲信号变为负方向脉冲信号，负方向脉冲信号变为正方向脉冲信号，此时的运动方向将与上面的情况相反。

1) 脉冲方向模式：



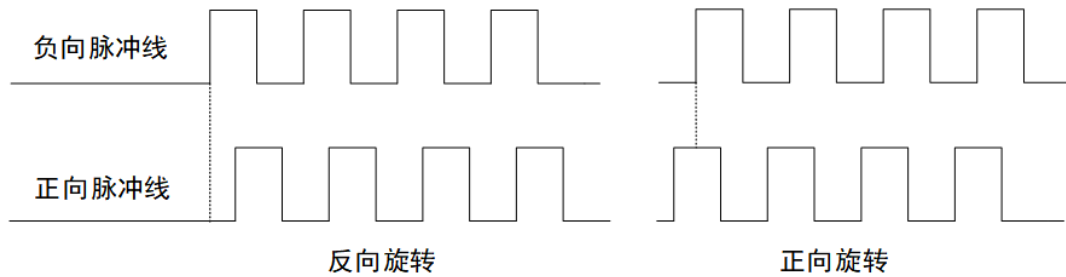
2) CW/CCW：双脉冲工作方式



3) BA 相：正交脉冲工作方式

这种模式下判断旋转方向的标准是观察哪个方向先发出脉冲信号，旋转方向即为负方向

负方向脉冲信号在正方向脉冲信号之前产生，二者相位相差 90 度，此时为负向旋转，正方向脉冲信号在负方向脉冲信号之前产生，二者相位相差 90 度，此时为正向旋转。



以上模式中，脉冲方向模式和双脉冲模式两种极性，共对应 8 种不同的运动状态，AB 相/BA 相模式为部分控制器定制模式。

3. ATYPE=3 或 6 编码器轴

编码器单独占用一个轴号时，根据编码器类型，轴类型可选 3 或 6。

4. ATYPE=4 脉冲轴和编码器轴共用轴号

当前的脉冲轴带编码器反馈时，轴类型设为 4，脉冲输出和编码器输入信号在同一个轴上。

5. ATYPE=8 CAN 扩展轴

使用 CAN 总线扩展轴时，扩展的脉冲轴的轴类型设成 8，扩展轴上接入编码器轴的轴类型设为 9。

6. ATYPE=21 振镜轴号

接入激光振镜设备时，需要将振镜轴类型设为 21，激光振镜轴部分型号支持。

7. ATYPE=50,51,52 RTEX 总线轴号

使用 RTEX 总线驱动器时，轴类型只能在以上三个中选择，其中 ATYPE=50 是位置模式，使用运动指令控制电机运行；ATYPE=51 速度模式，速度模式下使用 DAC 指令设置电机的运行速度，并持续运行；ATYPE=52 力矩模式，力矩模式下使用 DAC 指令设置电机的力矩，并持续运行，速度和力矩模式下不能使用运动指令，故无需设置轴参数，将 DAC=0 停止运行。

速度和力矩模式下若要切换模式，为防止事故，先将 DAC 置 0 后再使用 ATYPE 指令切换。

注意：修改 ATYPE 切换到力矩模式之前，请先将驱动器参数 Pr6.47 的第一位置为 0，关闭 2 自由度控制模式。再通过参数 Pr3.17 设置速度限制。当 Pr3.17(速度限制选择)的设定值是 0 时，通过 Pr3.21 设置速度限制，设定值是 1 时，可以通过 SL_SW 切换转矩控制时的速度限制值采用 Pr3.21 还是 Pr3.22。

8. ATYPE=65,66,67 EtherCAT 总线轴号

使用 EtherCAT 总线驱动器时，轴类型只能在以上三个中选择，其中 ATYPE=65 是位置模式，使用运动指令控制电机运行；ATYPE=66 速度模式，速度模式下使用 DAC 指令设置电机的运行速度，并持续运行，速度单位有两个，脉冲数/S 和 R/MIN，由驱动器确定；ATYPE=52 力矩模式，力矩模式下使用 DAC 指令设置电机的力矩，并持续运行，力矩控制模式下 DAC 值范围 0-1000，对应 0-100%，比如 DAC=10，此时电机力矩为 1%，速度和力矩模式下不能使用运动指令，故无需设置轴参数，将 DAC=0 停止运行。

速度和力矩模式下若要切换模式，为防止事故，先将 DAC 置 0 后再使用 ATYPE 指令切换。

第七章 运动指令

当前运动指令正在执行时，后续调用的运动指令会自动缓冲起来，ZMotion 运动控制器每个轴最多可以支持多达 4096 级运动缓冲（不同型号控制器缓冲个数有区别），当缓冲全部占用时，后续调用的运动指令会阻塞当前任务，直到缓冲有空位，任务才会继续运行。

每个运动指令都带有一个 MOVE_MARK 参数，通过 MOVE_CURMARK 可以知道当前运行到哪一条运动缓冲了。

MOVE 等单轴运动指令采用本轴的 SPEED 等各自单轴的轴参数。

MOVE 等多轴插补运动指令采用 BASE 主轴的 SPEED 等轴参数作为矢量合成速度，但其有相应的 SP 指令，SP 指令可以为每个运动指定多种不同的速度参数，例如：FORCE_SPEED、STARTMOVE_SPEED、ENDMOVE_SPEED，参见相应的*SP 指令。

轴参数 MERGE 用来设置单轴定位或轴组的多轴插补指令中间是否减速到零，MERGE=OFF 时减速到 0，MERGE=ON 时不减速，此时 BASE 主轴的轴参数 CORNER_MODE 会设置多轴插补之间是否自动减速到必要的速度。

ZMotion 运动控制器支持单轴或者轴组的运动暂停与恢复，参见 MOVE_PAUSE，MOVE_RESUME。

ZMotion 运动控制器支持运动叠加，参见 ADDAX。

7.1 单轴运动指令

ADDAX -- 运动叠加

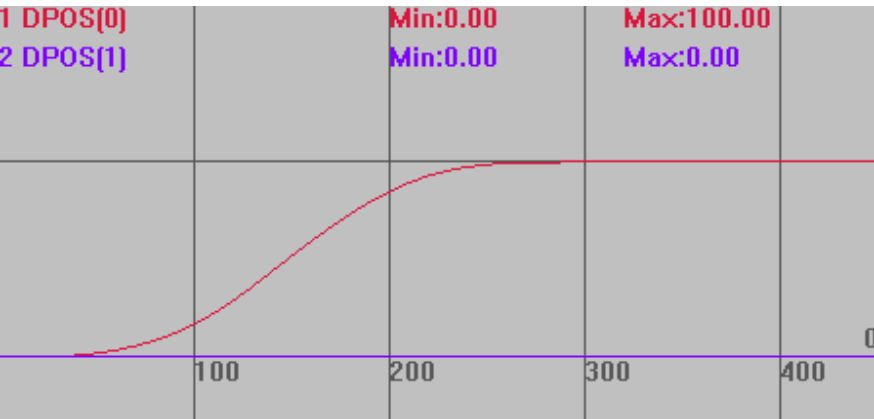
类型	单轴运动指令
描述	运动叠加，把一个轴的运动叠加到另一个轴。 叠加后，被叠加轴不要用绝对运动指令调用运动。可能会导致位置不对。 ADDAX 指令叠加的是脉冲个数，而不是设置的 units 单位。 转换关系：叠加轴运动距离*叠加轴 UNITS/被叠加轴 UNITS=被叠加轴运动距离。 假设轴 A 的 UNITS 是 100，轴 B 的 UNITS 是 50，叠加轴运动 100 把轴 A 的运动叠加到轴 B，此时轴 A 显示运动了 100，轴 B 运动了 $100 \times 100 / 50 = 200$。 把轴 B 的运动叠加到轴 A，此时轴 B 显示运动了 100，轴 A 运动了 $100 \times 50 / 100 = 50$。 轴之间不能相互同时叠加，A 叠加到 B 后，B 不能再叠加到 A。 支持串联叠加，A 运动叠加到 B，B 在叠加到 C。 支持并联叠加，A 运动同时叠加到 B、C。 叠加时速度从被叠加轴开始变化，加减速按照叠加轴加减速及两轴 units 比例确定。 ADDAX 在轴 MTYPE 为 FRAME 或 REFRAME 的时候不起作用。 对于运行速度较慢的控制器，需在轴叠加后延时一个周期，防止出现未完全叠加就开始运动带来的精度误差
语法	叠加：ADDAX(叠加轴号) AXIS(被叠加轴号) 取消叠加：ADDAX(-1) AXIS(被叠加轴号)

	4 系列及以上控制器, 20220728 固件增加叠加选项: BASE(destaxis) ADDAX(srcaxis ,[imode], [para]) destaxis: 叠加的目标轴号 srcaxis: 叠加的源轴轴号 imode: 叠加模式 0: 缺省值, 单轴叠加, 兼容以前的直接脉冲个数叠加 1: 单轴叠加, 支持比例调整 ADDAX(srcaxis , 1, ratio) ratio: 比例值, 支持浮点数, 目标轴距离 = 源轴距离 * ratio. 2: 单轴叠加, 支持齿轮比调整 ADDAX(srcaxis , 2, ratioin, rationout) ratioin: 分子, 整数, 支持负数 rationout: 分母, 正整数. 目标轴距离 = 源轴距离 * ratioin / rationout. 3: 单轴叠加到两个轴, 支持角度调整 BASE(destaxis1, destaxis2) ADDAX(srcaxis , 3, angle) destaxis: 叠加的目标轴 1, 2 angle: 角度, 弧度值, 目标轴 1 距离 = 源轴距离 * cos(angle). 目标轴 2 距离 = 源轴距离 * sin(angle). 注意:取消要分别取消两个轴 ADDAX(-1, 3, 0)或 ADDAX(-1) AXIS(被叠加轴号) 4: 振镜联动叠加, 利用振镜轴来补偿平台轴的偏差, 必须方向和当量一致, 不一致需要齿轮比调整, 或是振镜矫正加比例。 BASE(destaxis, destaxis2) ADDAX(srcaxis , 4, srcaxis2) 利用 srcaxis 来补偿 destaxis, srcaxis2 来补偿 destaxis2. 注意:取消也要两个轴取消 ADDAX(-1 , 4, -1) 或 ADDAX(-1) AXIS(被叠加轴号) 5: 振镜联动叠加, 平台轴反向叠加到振镜轴, 必须方向和当量一致, 不一致需要齿轮比调整, 或是振镜矫正加比例. BASE(destaxis, destaxis2) ADDAX(srcaxis , 5, srcaxis2) srcaxis 反向叠加到 destaxis, srcaxis2 反向叠加到 destaxis2. 注意:取消也要两个轴取消 ADDAX(-1 , 5, -1)或 ADDAX(-1) AXIS(被叠加轴号) 6: 振镜轴叠加, 必须两个轴都是振镜轴, 整个振镜叠加时, 需 XY 分开调用。 BASE(destaxis) '振镜轴 ADDAX(srcaxis,6) '振镜轴 2
适用控制器	通用

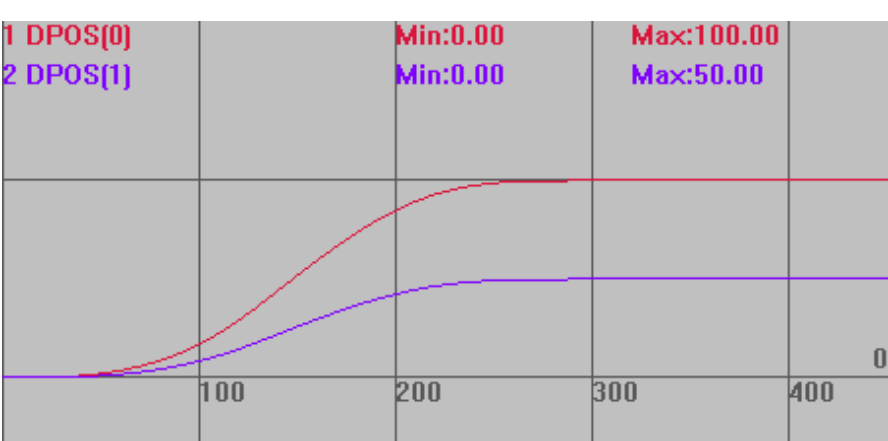
例子

```
例一
BASE(0,1)
ATYPE=1,1
UNITS=100,200      '轴 0 UNITS 设 100，轴 1 UNITS 设 200
SPEED=1000,1000    '速度设 1000
ACCEL=10000,10000  '加速度 10000
DECEL=10000,10000  '减速度 10000
ADDAX(0)  AXIS(1)   '轴 0 的运动叠加到轴 1，按脉冲个数叠加
DELAY(1)           '根据控制器周期设置延时时间
DPOS=0,0           '设置位置为 0,0
TRIGGER            '自动触发示波器
MOVE(100)          '轴 0 运动 100，此时轴 1 运动 100*100/200=50
                  '要考虑到两轴 UNITS 的转换
WAIT IDLE          '等待运行完
ADDAX(-1)  AXIS(1)  '取消叠加
```

不使用叠加指令的运动轨迹（无特殊说明图中示波器曲线均未设置偏移）
DPOS(0)垂直刻度 100
DPOS(1)垂直刻度 100

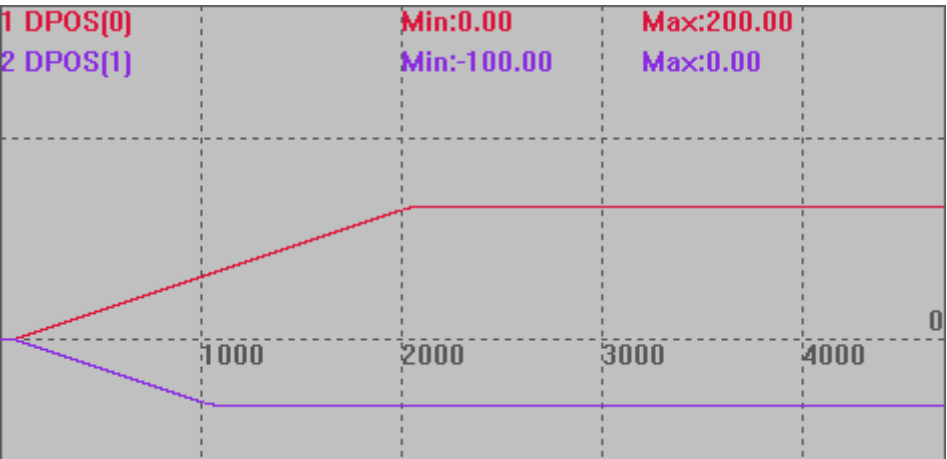


叠加指令使用后的运动轨迹
DPOS(0)垂直刻度 100
DPOS(1)垂直刻度 100

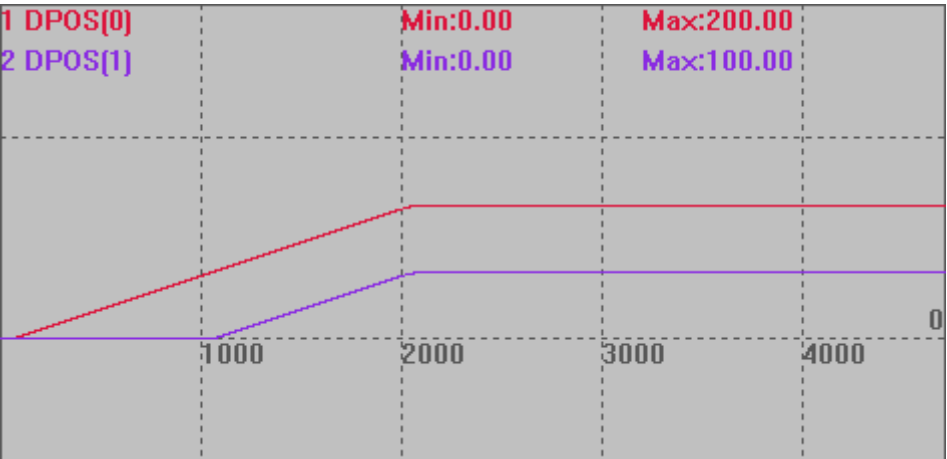


```
例二
RAPIDSTOP(2)
WAIT IDLE
BASE(0,1)
DPOS=0,0
ATYPE=1,1
UNITS=100,100      '脉冲比例为 1:1
SPEED=100,100
ACCEL=1000,1000
DECEL=1000,1000
ADDAX(0)  AXIS(1)   '轴 0 的运动叠加到轴 1
DELAY(1)      '根据控制器周期设置延时时间
TRIGGER
MOVE(200)  AXIS(0)
MOVE(-100)  AXIS(1)
WAIT IDLE      '等待运行完
ADDAX(-1)  AXIS(1)  '取消叠加
```

叠加前:



叠加后: 后续给轴 0 发运动指令, 轴 0 轴 1 一起运动, 仍保持叠加, 直到 ADDAX(-1) AXIS(1)取消叠加。



例三 模式 1

BASE(0,1) '选择轴号

UNITS = 100,100

DPOS =0,0

TRIGGER

BASE(1) '选择被叠加轴

ADDAX(0,1,1.5) AXIS(1) '模式 1 叠加，轴 0 的运动叠加到轴 1，比例 1.5

DELAY(1) '根据控制器周期设置延时时间

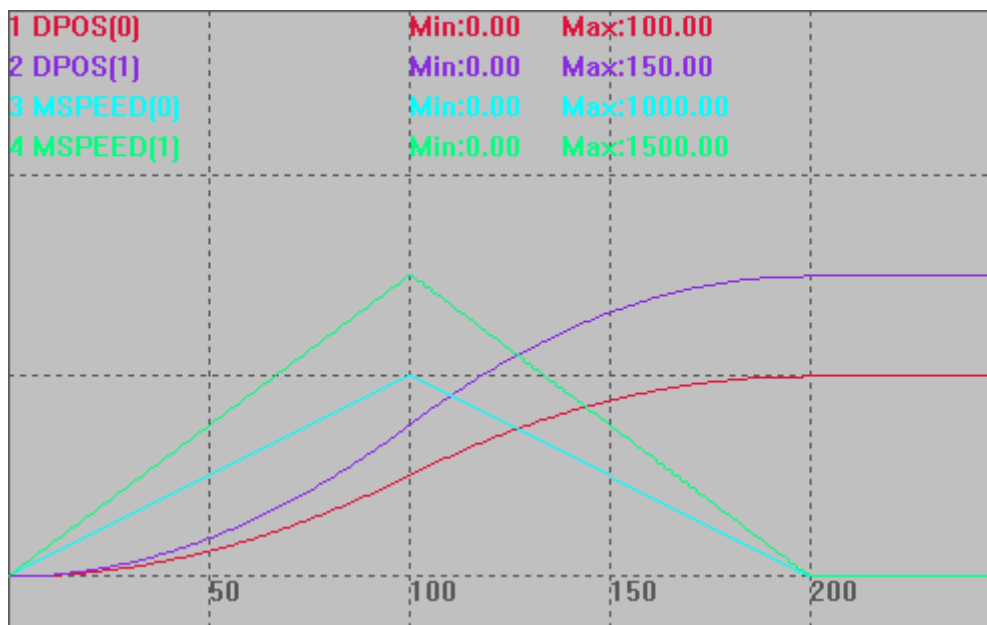
MOVE(100) AXIS(0)

WAIT UNTIL IDLE(0) AND IDLE(1)

?"轴 1 叠加轴号" ADDAX_AXIS(1)

ADDAX(-1) AXIS(1) '取消叠加

脉冲当量相同，轴 1 运动距离为轴 0 的 1.5 倍。



例四 模式 2

BASE(0,1) '选择轴号

UNITS = 100,100

DPOS =0,0

TRIGGER

BASE(1) '选择被叠加轴

ADDAX(0,2,3,5) AXIS(1) '模式 2 叠加，轴 0 的运动叠加到轴 1

DELAY(1) '根据控制器周期设置延时时间

MOVE(100) AXIS(0)

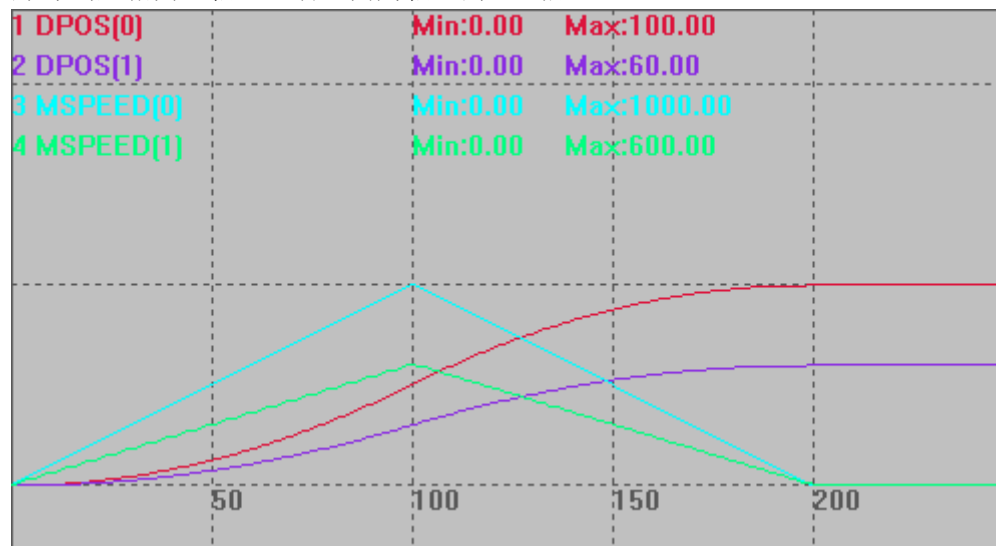
WAIT UNTIL IDLE(0) AND IDLE(1)

?"轴 1 叠加轴号" ADDAX_AXIS(1)

ADDAX(-1) AXIS(1) '取消叠加

END

脉冲当量相同，轴 1 运动距离为轴 0 的 3/5 倍。



例五 模式 3

BASE(0,1,2) '选择轴号

UNITS = 100,100,100

DPOS = 0,0,0

BASE(1,2) '叠加目标轴，轴号 1, 2

TRIGGER

ADDAX(0,3,PI/3) '模式 3 叠加，轴 0 的运动叠加到轴 1 和轴 2

DELAY(1) '根据控制器周期设置延时时间

MOVE(100) AXIS(0)

WAIT UNTIL IDLE(0) AND IDLE(1) AND IDLE(2)

DIM pos1,pos2

pos1=100*cos(PI/3)

pos2=100*sin(PI/3)

?"轴 0 目标位置", 100, "实际位置" ENDMOVE(0)

?"轴 1 目标位置", pos1, "实际位置" ENDMOVE(1)

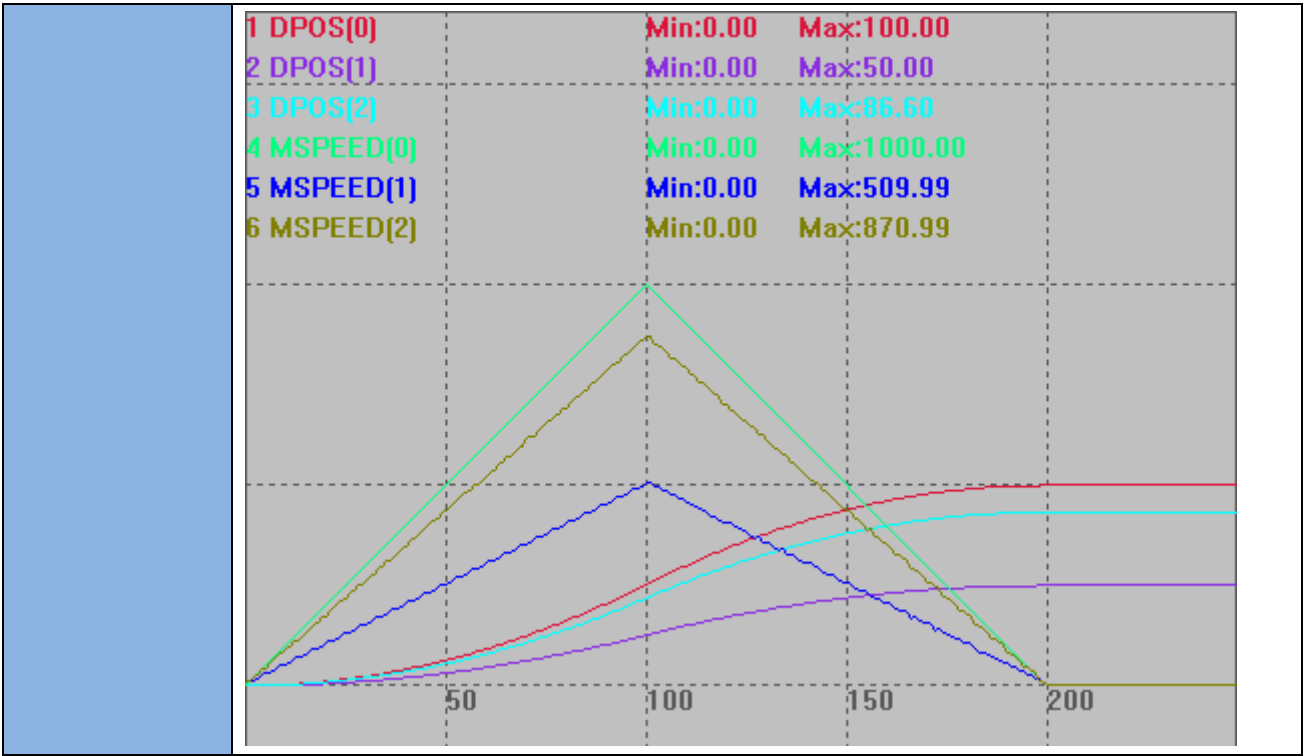
?"轴 2 目标位置", pos2, "实际位置" ENDMOVE(2)

?"轴 1 叠加轴号" ADDAX_AXIS(1)

?"轴 2 叠加轴号" ADDAX_AXIS(2)

ADDAX(-1) AXIS(1) '取消叠加

ADDAX(-1) AXIS(2)

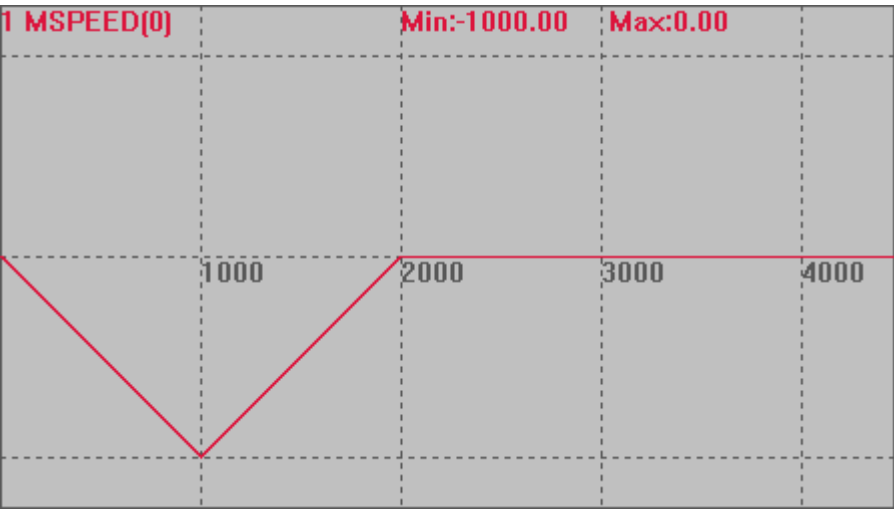


CANCEL -- 单轴停止/轴组停止

类型	单轴运动指令										
描述	BASE 轴减速停止，如果轴参与插补，也停止插补运动。 如果指定轴在 BASE 轴列表中，无论 CANCEL 主轴或者 BASE 轴列表中的轴，都停止轴组的插补运动 模式 2 减速度按 FASTDEC 和 DECEL 中最大的值，一般 FASTDEC 设置大于 DECEL。 CANCEL 后要调用绝对位置运动，必须先 WAIT IDLE 等待停止完成。										
语法	CANCEL(mode) mode: 模式选择 <table><tr><td>0 (缺省)</td><td>取消当前运动</td></tr><tr><td>1</td><td>取消缓冲的运动</td></tr><tr><td>2</td><td>取消当前运动和缓冲运动，停止速度参考快减速 FASTDEC</td></tr><tr><td>3</td><td>立即中断脉冲发送</td></tr><tr><td>4</td><td>取消当前运动和缓冲运动，停止速度参考减速度 DECEL</td></tr></table> CANCEL(4)为 4 系列控制器固件版本 170708 及以上支持。 CANCEL(3)不要对正在插补的从轴使用。	0 (缺省)	取消当前运动	1	取消缓冲的运动	2	取消当前运动和缓冲运动，停止速度参考快减速 FASTDEC	3	立即中断脉冲发送	4	取消当前运动和缓冲运动，停止速度参考减速度 DECEL
0 (缺省)	取消当前运动										
1	取消缓冲的运动										
2	取消当前运动和缓冲运动，停止速度参考快减速 FASTDEC										
3	立即中断脉冲发送										
4	取消当前运动和缓冲运动，停止速度参考减速度 DECEL										
适用控制器	通用										
例子	例一 mode=0 BASE(0) DPOS=0 SRAMP=0 ATYPE=1										

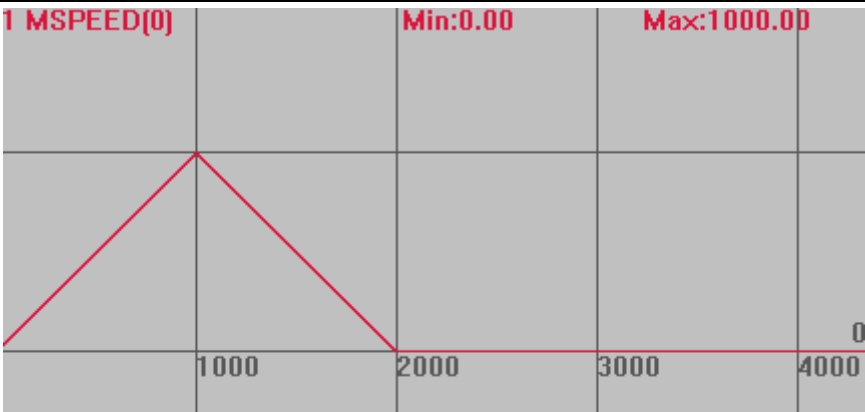
UNITS=100
SPEED=1000
ACCEL=1000
DECEL=1000 '减速度设为 1000
FASTDEC=10000 '快减减速度设为 10000
TRIGGER '自动触发示波器
MOVE(1000) '当前运动
MOVE(-1000) '缓冲运动
CANCEL(0) '此时轴停止

运动轨迹
MSPEED(0)垂直刻度 1000



例二 mode=1
BASE(0)
DPOS=0
SRAMP=0
ATYPE=1
UNITS=100
SPEED=1000
ACCEL=1000
DECEL=1000 '减速度设为 1000
FASTDEC=10000 '快减减速度设为 10000
TRIGGER '自动触发示波器
MOVE(1000) '当前运动
MOVE(-1000) '缓冲运动
CANCEL(1) '此时轴只执行完 MOVE(1000)

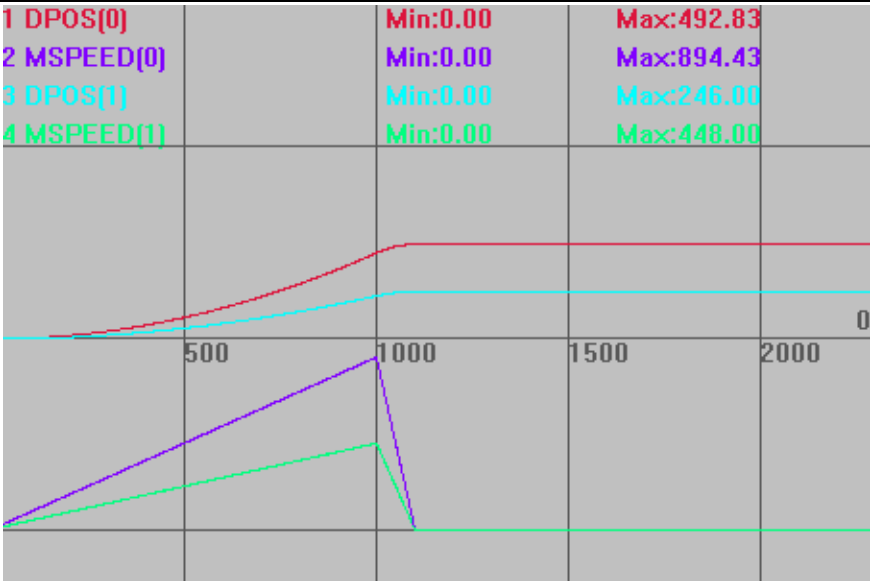
运动轨迹
MSPEED(0)垂直刻度 1000



因为取消的是缓冲，当前运动还是按减速度正常停止。

```
例三 mode=2
BASE(0,1)
DPOS=1,1
ATYPE=1,1
SPEED=1000,1000
ACCEL=1000
DECEL=1000           '减速度设为 1000
FASTDEC=10000        '快减减速度设为 10000
SRAMP=0,0
TRIGGER
MOVE(1000,500)        '插补运动
DELAY(1000)           '延时 1 秒
CANCEL(2) AXIS(1)     '停止轴 1，轴 1 参与了插补，插补运动也停止减速度为 10000
```

运动轨迹和速度曲线
DPOS(0)垂直刻度 1000，无偏移
MSPEED(0)垂直刻度 1000，偏移-1000
DPOS(1)垂直刻度 1000，无偏移
MSPEED(1)垂直刻度 1000，偏移-1000



例四 mode=3

BASE(0)

ATYPE=1

DPOS=0

SPEED=100

ACCEL=1000

DECEL=1000

'减速度设为 1000

FASTDEC=10000

'快减减速度设为 10000

TRIGGER

'自动触发示波器

MOVE(10000)

'当前运动 10000

DELAY(2000)

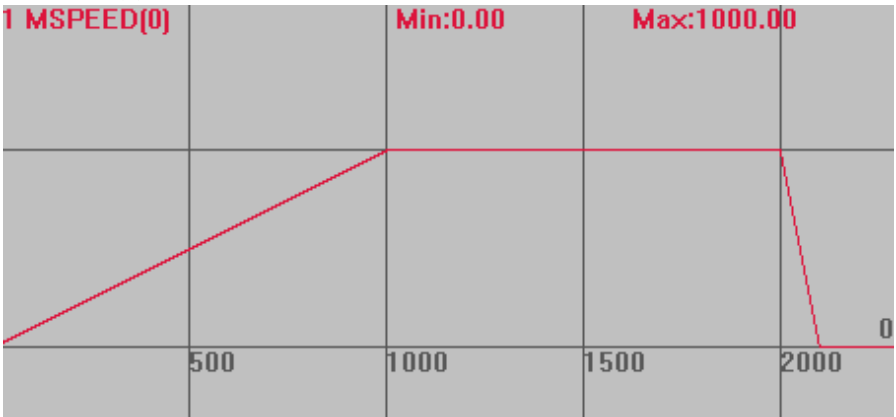
'延时 2 秒

CANCEL(3)

'此时直接切断脉冲发送，轴立即停止

运动轨迹

MSPEED(0)垂直刻度 1000



例五 mode=4

BASE(0)

DPOS=0

	ATYPE=1 UNITS=100 SPEED=1000 ACCEL=10000 DECEL=10000 '减速度设为 10000' FASTDEC=100000 '快减减速度设为 100000' TRIGGER '自动触发示波器' MOVE(1000) '当前运动' MOVE(-2000) '缓冲运动' DELAY(500) CANCEL(4) '急停，减速度 10000'
	运动轨迹和速度曲线 DPOS(0)垂直刻度 500，无偏移 MSPEED(0)垂直刻度 500，无偏移
相关指令	RAPIDSTOP ， DECEL ， FASTDEC

DATUM -- 回零

类型	单轴运动指令
描述	单轴找原点运动。 原点开关通过 DATUM_IN 设置，正负限位开关分别通过 FWD_IN 和 REV_IN 设置。ZMC 控制器为 0 触发有效，输入为 OFF 状态时，表示到达原点/限位，常开类型信号需要采用 INVERT_IN 反转电平。 ECI 控制器为 1 触发有效，输入为 ON 状态时，表示到达原点/限位，常闭类型信号需要采用 INVERT_IN 反转电平。 Z 信号回零必须配置为带 Z 信号 ATYPE（ATYPE=4 或者 7）。

	若配置了 LSPEED，找到原点急停，速度降为 LSPEED 时的位置为原点位置。 多轴回零时，需要每个轴都使用 DATUM 指令。 总线控制器使用控制器找原点模式完成后，需要手动清零 MPOS。 DATUM 指令后不能接绝对指令和 mover 指令。 驱动器回零 DATUM 指令是监控不到的，需要使用驱动器回零里面的驱动器状态返回值																				
语法	DATUM(mode), DATUM(21,mode2) mode: 找原点模式，加 10 表示碰到限位后反找，不会碰到限位停止，例如 13=模式 3+限位反找 10，用于原点在正中间的情况。 ATYPE=4，回零模式加 100（模式 100+n 和 110+n 分别对应 n 和 10+n），表示接入编码器后可以自动清零 MPOS(仅限 4 系列) DATUM(0) AXIS（轴号）清除指定轴的错误状态。 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>清除所有轴的错误状态。</td></tr> <tr> <td>1</td><td>轴以 CREEP 速度正向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。</td></tr> <tr> <td>2</td><td>轴以 CREEP 速度反向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。</td></tr> <tr> <td>3</td><td>轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关。 找原点阶段碰到正限位开关会直接停止。 爬行阶段碰到负限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS</td></tr> <tr> <td>4</td><td>轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关。 找原点阶段碰到负限位开关会直接停止。 爬行阶段碰到正限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。</td></tr> <tr> <td>5</td><td>轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关，然后再继续以爬行速度反转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。</td></tr> <tr> <td>6</td><td>轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关，然后再继续以爬行速度正转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。</td></tr> <tr> <td>8</td><td>轴以 SPEED 速度正向运行，直到碰到原点开关。 碰到限位开关会直接停止。</td></tr> <tr> <td>9</td><td>轴以 SPEED 速度反向运行，直到碰到原点开关。</td></tr> </tbody> </table>	值	描述	0	清除所有轴的错误状态。	1	轴以 CREEP 速度正向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。	2	轴以 CREEP 速度反向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。	3	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关。 找原点阶段碰到正限位开关会直接停止。 爬行阶段碰到负限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS	4	轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关。 找原点阶段碰到负限位开关会直接停止。 爬行阶段碰到正限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。	5	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关，然后再继续以爬行速度反转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。	6	轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关，然后再继续以爬行速度正转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。	8	轴以 SPEED 速度正向运行，直到碰到原点开关。 碰到限位开关会直接停止。	9	轴以 SPEED 速度反向运行，直到碰到原点开关。
值	描述																				
0	清除所有轴的错误状态。																				
1	轴以 CREEP 速度正向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。																				
2	轴以 CREEP 速度反向运行直到 Z 信号出现。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。																				
3	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关。 找原点阶段碰到正限位开关会直接停止。 爬行阶段碰到负限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS																				
4	轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关。 找原点阶段碰到负限位开关会直接停止。 爬行阶段碰到正限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。																				
5	轴以 SPEED 速度正向运行，直到碰到原点开关。然后轴以 CREEP 速度反向运动直到离开原点开关，然后再继续以爬行速度反转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。																				
6	轴以 SPEED 速度反向运行，直到碰到原点开关。然后轴以 CREEP 速度正向运动直到离开原点开关，然后再继续以爬行速度正转直到碰到 Z 信号。 碰到限位开关会直接停止。 DPOS 值重置为 0 同时纠正 MPOS。																				
8	轴以 SPEED 速度正向运行，直到碰到原点开关。 碰到限位开关会直接停止。																				
9	轴以 SPEED 速度反向运行，直到碰到原点开关。																				

	<table><tr><td></td><td>碰到限位开关会直接停止。</td></tr><tr><td>21</td><td> 使用 Ethercat 驱动器回零功能，此时 mode2 有效。 设置驱动器回零方式（6098h），缺省 0 表示使用驱动器当前的回零方式。 会使用轴的 SPEED, CREEP, ACCEL, DECEL，乘以 UNITS 后自动设置驱动器的 6099h，609Ah 动作时序： 6060h PDO 切换当前模式→6098h SDO 回零方式→6099h SDO 速度→609Ah SDO 加速度→ PDO 切换控制字启动回零 </td></tr></table> mode2: mode=21 时有效，缺省 0，非 0 时设置到驱动器回零方式，根据驱动器手册数据字典 6098h 设置值。		碰到限位开关会直接停止。	21	使用 Ethercat 驱动器回零功能，此时 mode2 有效。 设置驱动器回零方式（6098h），缺省 0 表示使用驱动器当前的回零方式。 会使用轴的 SPEED, CREEP, ACCEL, DECEL，乘以 UNITS 后自动设置驱动器的 6099h，609Ah 动作时序： 6060h PDO 切换当前模式→6098h SDO 回零方式→6099h SDO 速度→609Ah SDO 加速度→ PDO 切换控制字启动回零											
	碰到限位开关会直接停止。															
21	使用 Ethercat 驱动器回零功能，此时 mode2 有效。 设置驱动器回零方式（6098h），缺省 0 表示使用驱动器当前的回零方式。 会使用轴的 SPEED, CREEP, ACCEL, DECEL，乘以 UNITS 后自动设置驱动器的 6099h，609Ah 动作时序： 6060h PDO 切换当前模式→6098h SDO 回零方式→6099h SDO 速度→609Ah SDO 加速度→ PDO 切换控制字启动回零															
适用控制器	通用															
例子	例一 直接找原点 BASE(0) DPOS=0 ATYPE=1 SPEED = 100 '找原点速度 CREEP = 10 '反向爬行速度 DATUM_IN=5 '输入 IN5 作为原点开关 INVERT_IN(5,ON) '反转 IN5 电平信号，常开信号进行反转(ZMC 控制器) TRIGGER '自动触发示波器 DATUM(3) '轴 0 先以 100units/s 正向回零，找到原点后以 10units/s 直到离开原点，同时 DPOS 清 0 运动轨迹与速度曲线 DPOS(0)垂直刻度 500 MSPEED(0)垂直刻度 100 IN(5)垂直刻度 10 <table><tr><td>1 DPOS[0]</td><td></td><td>Min:0.00</td><td>Max:459.15</td><td></td></tr><tr><td>2 MSPEED[0]</td><td></td><td>Min:-10.00</td><td>Max:100.00</td><td></td></tr><tr><td>3 IN[5]</td><td></td><td>Min:0.00</td><td>Max:1.00</td><td></td></tr></table> 碰到原点开关之后反向爬行直到离开原点开关，此时 DPOS 清 0，回零运动完成。为	1 DPOS[0]		Min:0.00	Max:459.15		2 MSPEED[0]		Min:-10.00	Max:100.00		3 IN[5]		Min:0.00	Max:1.00	
1 DPOS[0]		Min:0.00	Max:459.15													
2 MSPEED[0]		Min:-10.00	Max:100.00													
3 IN[5]		Min:0.00	Max:1.00													

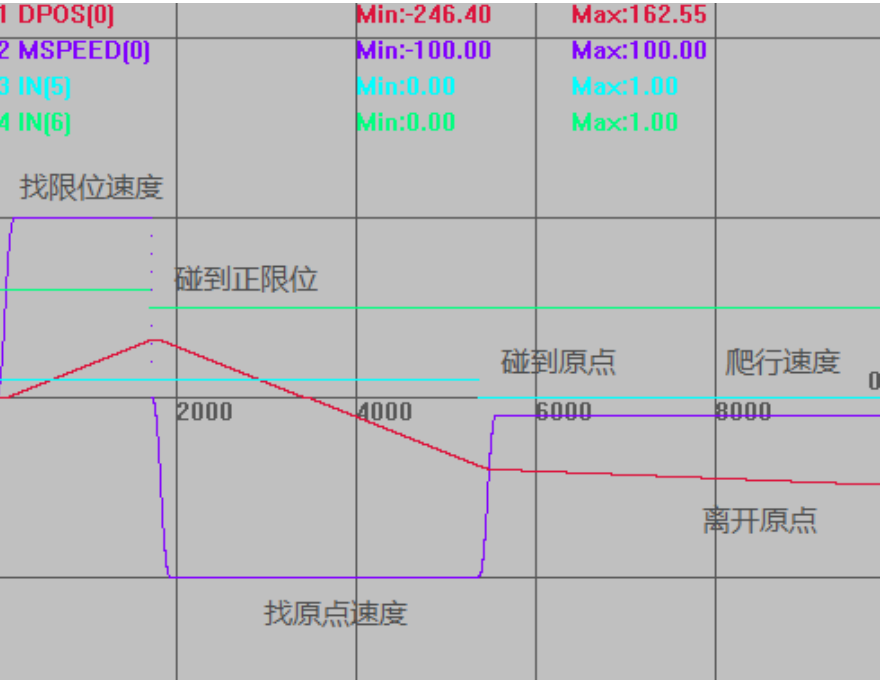
了直观体现，所以爬行过程比较长，实际应用中，爬行过程很短。

例二 遇到限位反找原点

```
BASE(0)
DPOS=0
ATYPE=1
SPEED = 100      '找原点速度
CREEP = 10       '反向爬行速度
DATUM_IN=5       '输入 IN5 作为原点开关
FWD_IN=6         '输入 IN6 作为正限位开关
INVERT_IN(5,ON)  '反转电平信号，一般常闭的信号才进行反转
INVERT_IN(6,ON)
TRIGGER          '自动触发示波器
DATUM(13)      '轴 0 先以 100units/s 正向运动找原点，碰到正限位反向运行，仍以
                  100units/s 找原点，找到原点后以 10units/s 直到离开原点，同时 DPOS
                  清 0
wait until READ_BIT2(6,AXISSTATUS(axisnum))<>1 OR MTYPE(axisnum) <> 12
```

运动轨迹与速度曲线

DPOS(0)垂直刻度 500
MSPEED(0)垂直刻度 100
IN(5)垂直刻度 10
IN(6)垂直刻度 10，偏移 5



例三 EtherCAT 总线回零(松下 A6N 伺服)

先根据 [EtherCAT 初始化例程](#) 正常使能电机
SPEED=100 '找零速度*UNITS 后自动写入 6099

```

CREEP=10                                '爬行速度*UNITS 后自动写入 6099

ACCEL=1000                              '加减速*UNITS 后自动写入 609A
DECEL=1000

DATUM(21,0)                            '按驱动器的回零模式 0 开始回零，此时按驱动器信号判断，而不是按控制器信号判断
WHILE 1
    TABLE(0)=DRIVE_STATUS              '读取状态字判断回零状态
    IF READ_BIT2(10,TABLE(0)) THEN    '根据下图确定
        IF READ_BIT2(12,TABLE(0)) THEN
            ?"回零完成"
        ENDIF
    ENDIF
WEND
END

```

驱动器厂家的回零过程可能不同，具体根据驱动器手册确定。
状态字的 bit13，bit12，bit10 的值组合含义如下表：

bit13	bit12	bit10	描述
0	0	0	原点回零动作中
0	0	1	原点回零动作中断或未开始
0	1	0	原点回零动作完成，但未到达目标位置
0	1	1	原点回零动作正常完成
1	0	0	检出原点回零异常还在动作中
1	0	1	检出原点回零异常，停止状态

例四 Rtex 总线回零(松下 A6N 伺服)

先根据 [Rtex 初始化例程](#) 正常使能电机

```

SPEED=100                              '找零速度
ACCEL=1000                              '加减速
DECEL=1000

DATUM(21,$11)                          '按驱动器当前回零模式开始回零，此时按按驱动器信号判断，而不是按控制器信号判断

```

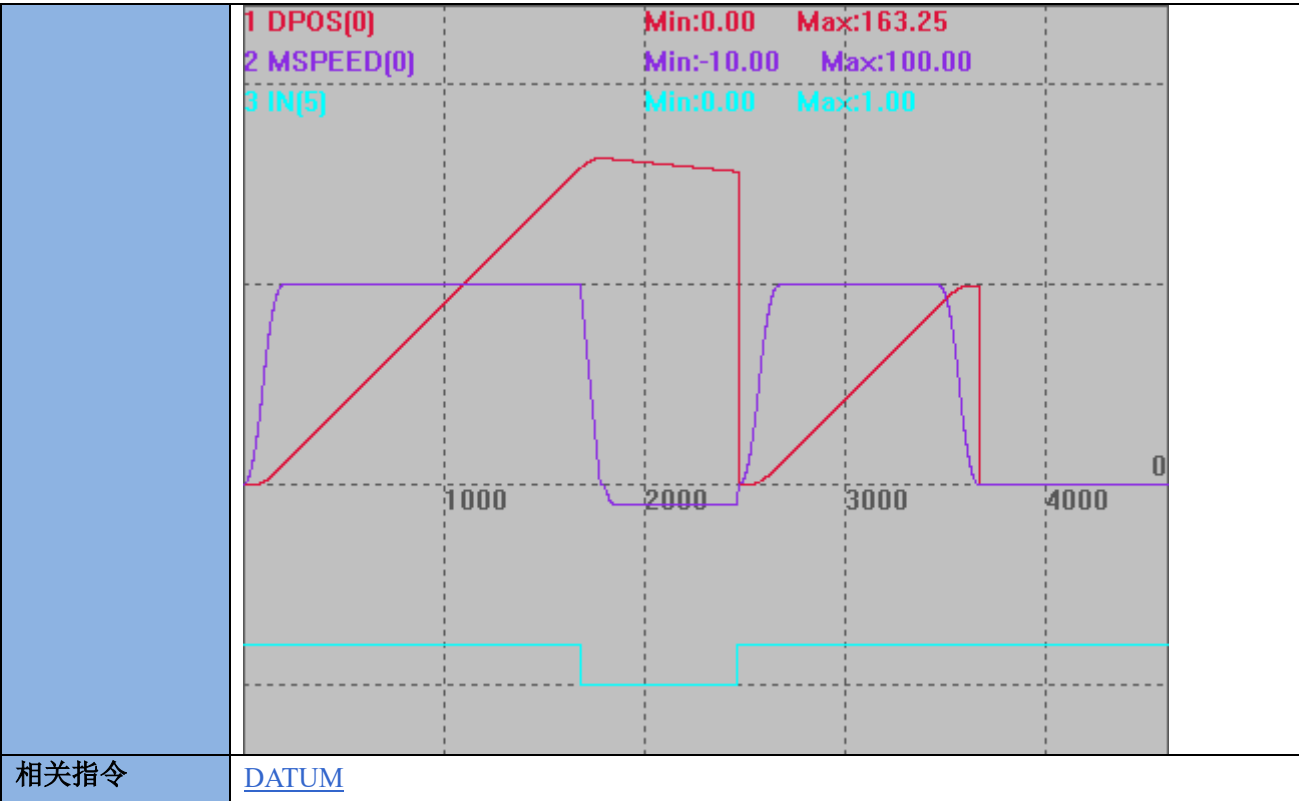
根据驱动器手册确定回零模式

初始化模式	11h	Z 相
	12h	HOME ↑ *2
	13h	HOME ↓ *3
	14h	POT ↑ *2
	15h	POT ↓ *3
	16h	NOT ↑ *2
	17h	NOT ↓ *3
	18h	EXT1 ↑ *2
	19h	EXT1 ↓ *3

		1Ah	EXT2 ↑ *2	
		1Bh	EXT2 ↓ *3	
		1Ch	EXT3 ↑ *2	
		1Dh	EXT3 ↓ *3	
相关指令	DATUM_IN ， DATUM_OFFSET ， INVERT_IN			

DATUM_OFFSET -- 原点位置偏移

类型	单轴运动指令
描述	设置原点的位置偏移。 回原点成功后，轴移动到偏移位置。
语法	DATUM_OFFSET(axis)=distance distance: 偏移距离
适用控制器	4 系列及以上支持
例子	BASE(0) DPOS=0 ATYPE=1 SPEED = 100 '找原点速度 CREEP = 10 '反向爬行速度 DATUM_IN=5 '输入 IN5 作为原点开关 INVERT_IN(5,ON) '反转 IN5 电平信号，常开信号进行反转(ZMC 控制器) TRIGGER '自动触发示波器 DATUM_OFFSET(0)=100 '轴 0 回零后再偏移 DATUM(3) '轴 0 先以 100units/s 正向回零，找到原点后以 10units/s 直到离开原点，同时 DPOS 清 0 运动轨迹与速度曲线：轴最终停止在 DATUM_OFFSET 设置的位置。 DPOS(0)垂直刻度 100 MSPEED(0)垂直刻度 100 IN(5)垂直刻度 5，偏移-5



VMOVE -- 持续运动

类型	单轴运动指令
描述	连续往一个方向运动。 当前面的 VMOVE 运动没有停止时，此 VMOVE 指令会自动替换前面的 VMOVE 指令并修改方向，因此无需 CANCEL 前面的 VMOVE 指令。 VMOVE 指令无法使用多轴矢量模式进行硬件位置比较输出。
语法	VMOVE(dir1) dir1: -1 负向运动, 1 正向运动
适用控制器	通用
例子	BASE(0) DPOS=0 ATYPE=1 SPEED=100 ACCEL=1000 DECEL=1000 '减速度设为 1000 SRAMP=100 VMOVE(-1) '持续负向运动 WAIT UNTIL IN(0)=ON '等待输入 1 有效 VMOVE(1) '持续正向运动 DPOS(0)垂直刻度 500，无偏移 MSPEED(0)垂直刻度 100，无偏移

	IN(0)垂直刻度 10，无偏移			
相关指令	FORWARD , REVERSE			

FORWARD -- 正向运动

类型	单轴运动指令		
描述	BASE 选择轴正向运动 必须 CANCEL 后才能切换 REVERSE 。		
语法	FORWARD [axis(轴号)]		
适用控制器	通用		
例子	例一 BASE(0) FORWARD '轴 0 持续正向运动 WAIT UNTIL IN(1)=ON '等待输入 1 有效 CANCEL(2) 例二 FORWARD AXIS(1) '轴 1 正向运动 WAIT UNTIL IN(1)=ON '等待输入 1 有效 CANCEL(2) AXIS(1)		
相关指令	REVERSE , VMOVE		

REVERSE -- 负向运动

类型	单轴运动指令
描述	BASE 选择轴负向运动。

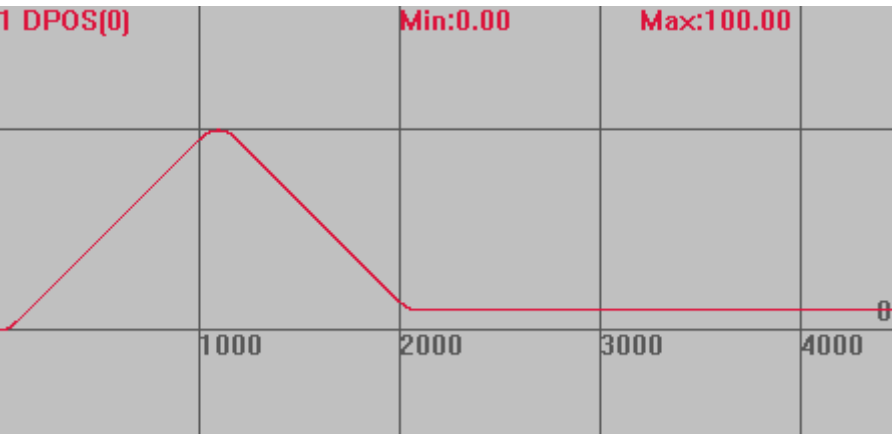
	必须 CANCEL 后才能切换 FORWARD。
语法	REVERSE [axis(轴号)]
适用控制器	通用
例子	例一 BASE(0) REVERSE '轴 0 持续负向运动 WAIT UNTIL IN(1)=ON '等待输入 1 有效 CANCEL(2) 例二 REVERSE AXIS(1) '轴 1 负向运动 WAIT UNTIL IN(1)=ON '等待输入 1 有效 CANCEL(2) AXIS(1)
相关指令	FORWARD , VMOVE

MOVEMODIFY -- 修改运动位置

类型	单轴运动指令
描述	修改上一个运动的目标位置。 前面没有运动时与 MOVEABS 效果一样，但不会进入运动缓冲。见例一 需要 WAIT 指令才能正确使用。见例二 连续插补时使用 MOVEMODIFY 会破坏速度连续性。 MOVEMODIFY 同时对多轴运动时不一定为直线插补。 MOVEMODIFY 和 MOVEMODIFY2 指令在 32 位控制器上使用时不建议设置很大的运动参数，否则可能会导致速度突变的情况；如需设置较大的运动参数，可以设置齿轮比把运动参数降低，或者更换 64 位控制器使用。
语法	MOVEMODIFY (distance) distance: 单个轴的运动距离 目前只支持单轴修改。
适用控制器	通用
例子	例一 BASE(0) ATYPE=1 '脉冲轴类型 UNITS=100 '脉冲当量设置 DPOS=0 SPEED=100 '速度设置 ACCEL=1000 '加速度设置 DECEL=1000 TRIGGER '自动触发示波器 MOVEABS(100) MOVEABS(10) '此时轴会先运动到 100，在往回运动到 10

运动轨迹

DPOS(0)垂直刻度 100



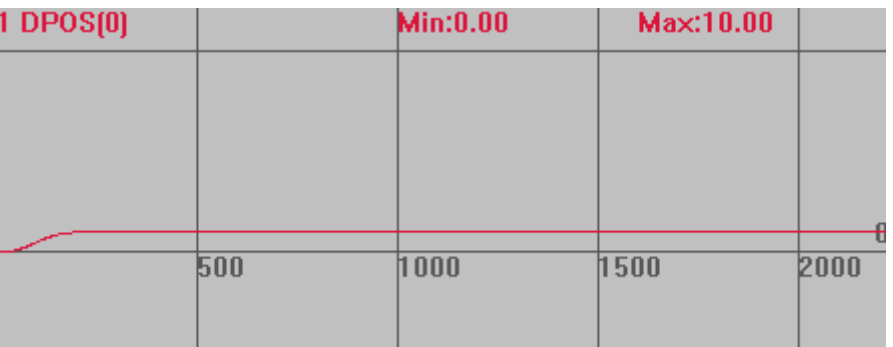
将 MOVEABS 指令变为如下指令：

MOVEMODIFY(100)

MOVEMODIFY(10) '此时轴会直接运动到 10 位置，MOVEMODIFY 不进入运动缓冲

运动轨迹

DPOS(0)垂直刻度 100



例二

BASE (0)

ATYPE=1 '脉冲轴类型
UNITS=100 '脉冲当量设置

DEFPOS(0)
SPEED=100 '速度设置
ACCEL=1000 '加速度设置

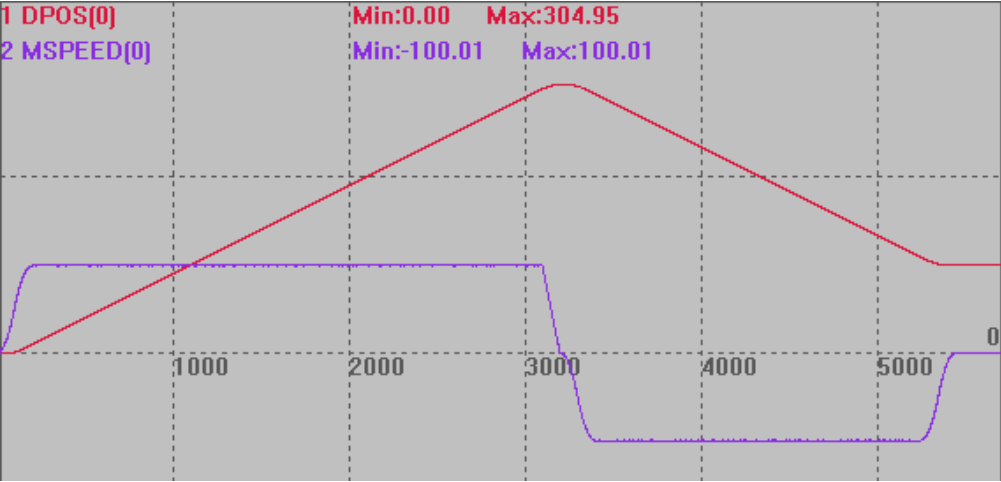
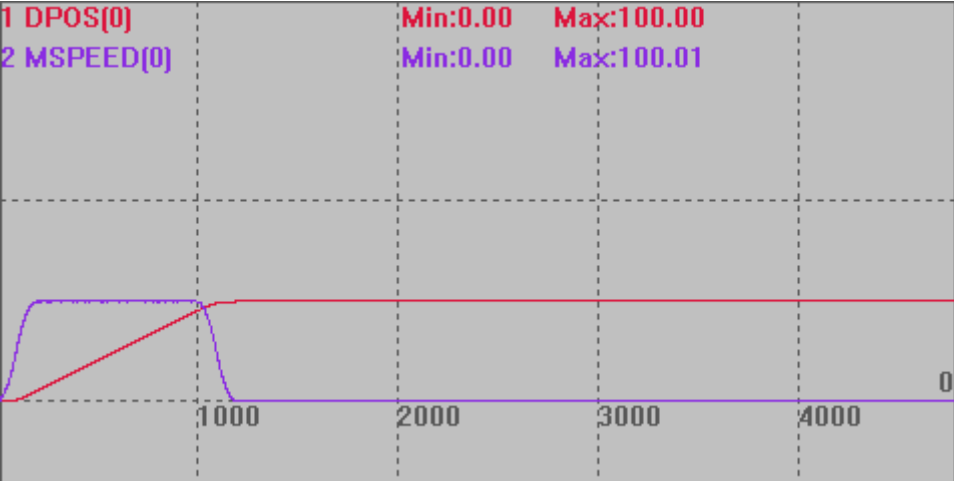
DECEL=1000
SRAMP=100
TRIGGER '自动触发示波器

MOVEABS(500)

WAIT UNTIL DPOS >=300 '等待运动到 300 时，才修改终点位置

MOVEMODIFY (100) '修改目标位置为 100，此时减速停止后再反向运动

使用 WAIT，运动轨迹

	DPOS(0)垂直刻度 200 MSPEED(0)垂直刻度 200 1 DPOS[0] 2 MSPEED[0] Min:0.00 Max:304.95 Min:-100.01 Max:100.01  不使用 WAIT，直接运动到 100，运动轨迹 DPOS(0)垂直刻度 300 1 DPOS[0] 2 MSPEED[0] Min:0.00 Max:100.00 Min:0.00 Max:100.01 
相关指令	MOVEMODIFY2

7.2 多轴运动指令

RAPIDSTOP -- 全部轴停止

类型	多轴运动指令
描述	所有轴立即停止，如果轴参与插补，也停止插补运动。 模式 2 减速度按 FASTDEC 和 DECEL 中最大的值，一般 FASTDEC 设置大于 DECEL。。 RAPIDSTOP 后要调用绝对位置运动，必须先 WAIT IDLE 等待停止完成。 如果下发 RAPIDSTOP 之前有调用运动暂停指令，第一次下发停止运动，第二次下发取消轴暂停状态。
语法	RAPIDSTOP (mode) mode: 模式选择

	<table><tr><td>0（缺省）</td><td>取消当前运动</td></tr><tr><td>1</td><td>取消缓冲的运动</td></tr><tr><td>2</td><td>取消当前运动和缓冲运动，停止速度参考快减速 FASTDEC</td></tr><tr><td>3</td><td>立即中断脉冲发送</td></tr><tr><td>4</td><td>取消当前运动和缓冲运动，停止速度参考减速度 DECEL</td></tr></table> RAPIDSTOP(4)为 4 系列控制器固件版本 170708 及以上支持	0（缺省）	取消当前运动	1	取消缓冲的运动	2	取消当前运动和缓冲运动，停止速度参考快减速 FASTDEC	3	立即中断脉冲发送	4	取消当前运动和缓冲运动，停止速度参考减速度 DECEL														
0（缺省）	取消当前运动																								
1	取消缓冲的运动																								
2	取消当前运动和缓冲运动，停止速度参考快减速 FASTDEC																								
3	立即中断脉冲发送																								
4	取消当前运动和缓冲运动，停止速度参考减速度 DECEL																								
适用控制器	通用																								
例子	例一 BASE(0,1,2) DPOS=1,1,1 ATYPE=1,1,1 UNITS=100,100,100 SPEED=1000 '插补合成运动速度 100 ACCEL=1000 DECEL=1000 '减速度设为 1000 FASTDEC=10000 '快减减速度设为 10000 TRIGGER MOVE(1000,1000,1000) '当前运动 MOVE(-1000,-1000,-1000) '缓冲运动 RAPIDSTOP(1) '此时轴只执行完当前运动 运动轨迹和速度曲线 DPOS(0)垂直刻度 1000，无偏移 MSPEED(0)垂直刻度 1000，偏移-1000 DPOS(1)垂直刻度 1000，偏移 100 MSPEED(1)垂直刻度 1000，偏移-900 DPOS(2)垂直刻度 1000，偏移 200 MSPEED(2)垂直刻度 1000，偏移-800 <table><tr><td>1 DPOS[0]</td><td></td><td>Min:1.00</td><td>Max:1001.00</td></tr><tr><td>2 MSPEED[0]</td><td></td><td>Min:0.00</td><td>Max:577.36</td></tr><tr><td>3 DPOS[1]</td><td></td><td>Min:1.00</td><td>Max:1001.00</td></tr><tr><td>4 MSPEED[1]</td><td></td><td>Min:0.00</td><td>Max:577.36</td></tr><tr><td>5 DPOS[2]</td><td></td><td>Min:1.00</td><td>Max:1001.00</td></tr><tr><td>6 MSPEED[2]</td><td></td><td>Min:0.00</td><td>Max:577.36</td></tr></table>	1 DPOS[0]		Min:1.00	Max:1001.00	2 MSPEED[0]		Min:0.00	Max:577.36	3 DPOS[1]		Min:1.00	Max:1001.00	4 MSPEED[1]		Min:0.00	Max:577.36	5 DPOS[2]		Min:1.00	Max:1001.00	6 MSPEED[2]		Min:0.00	Max:577.36
1 DPOS[0]		Min:1.00	Max:1001.00																						
2 MSPEED[0]		Min:0.00	Max:577.36																						
3 DPOS[1]		Min:1.00	Max:1001.00																						
4 MSPEED[1]		Min:0.00	Max:577.36																						
5 DPOS[2]		Min:1.00	Max:1001.00																						
6 MSPEED[2]		Min:0.00	Max:577.36																						

例二

```

BASE(0,1)
DPOS=0,0
ATYPE=1,1
SPEED=1000
ACCEL=1000
DECEL=1000          '减速度设为 1000
FASTDEC=10000        '快减减速度设为 10000
TRIGGER
MOVE(10000,10000)    '插补运动 10000
DELAY(2000)           '延时 2 秒
RAPIDSTOP (2)         '轴立即停止，减速度 10000
  
```

运动轨迹和速度曲线

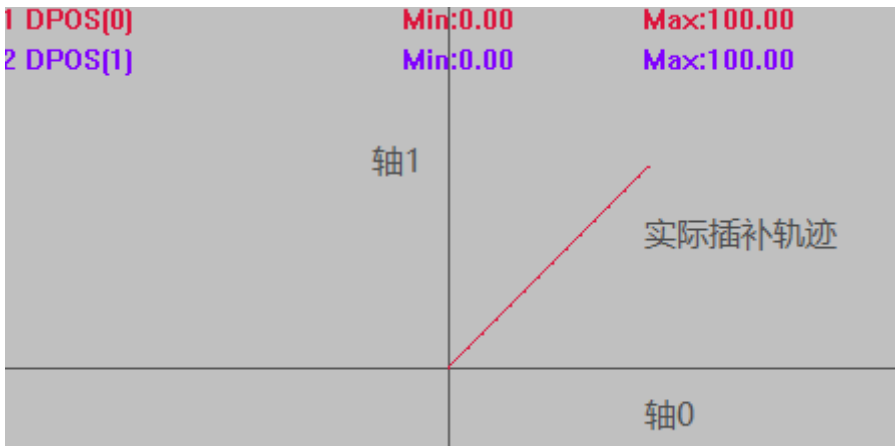
DPOS(0)垂直刻度 1000，无偏移
MSPEED(0)垂直刻度 1000，无偏移
DPOS(1)垂直刻度 1000，偏移 100
MSPEED(1)垂直刻度 1000，偏移 100

Axis	Variable	Min	Max
1	DPOS[0]	Min:0.00	Max:1096.01
2	MSPEED[0]	Min:0.00	Max:707.11
3	DPOS[1]	Min:0.00	Max:1096.01
4	MSPEED[1]	Min:0.00	Max:707.11

MOVE -- 直线运动

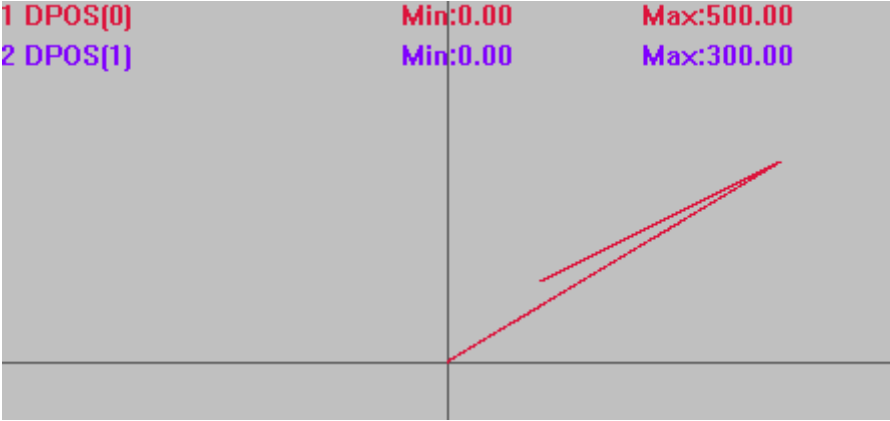
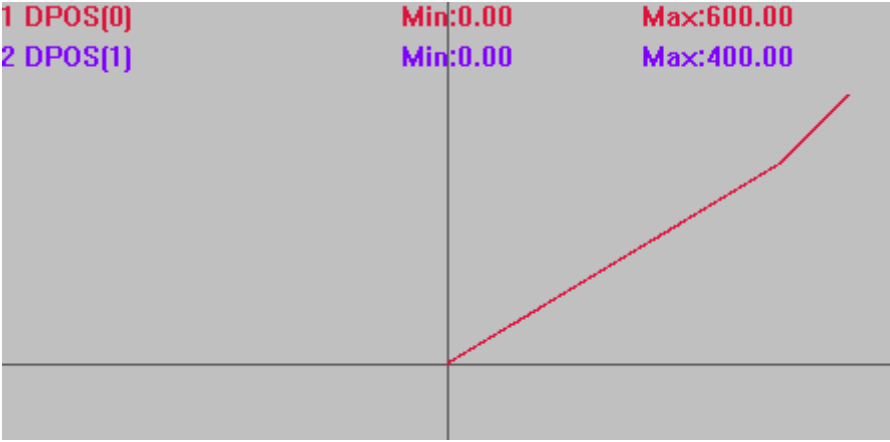
类型	多轴运动指令
描述	直线插补运动，相对运动一段距离。 插补运动时只有主轴速度参数有效，主轴是 BASE 的第一个轴，运动参照这个轴的参数。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。

	插补运动距离 $X=\sqrt{X_0^2 + X_1^2 + X_2^2 + + X_n^2}$ 运动时间 T=X/主轴 SPEED																
语法	MOVE(distance1 [,distance2 [,distance3 [,distance4...]]]) distance1: 第一个轴运动距离 distance2: 下一个轴运动距离																
适用控制器	通用																
例子	例一 BASE(0,1,2) '主轴为轴 0 ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 '脉冲当量设置 SPEED=100,10,1000 '只有主轴速度 100 起作用，作为合成运动的速度 ACCEL=1000,1000,1000 DECEL=1000,1000,1000 DPOS = 0,0,0 TRIGGER '自动触发示波器 MOVE(500,1000,1500) '轴 0， 1， 2 直线插补相对距离 WAIT IDLE '等待运动停止 PRINT *DPOS '打印结果， 500,1000,1500 插补运动各轴实际速度为主轴速度的分速度 MSPEED(0)垂直刻度 100 MSPEED(1)垂直刻度 100 MSPEED(2)垂直刻度 100 VP_SPEED(0)垂直刻度 100 <table><tr><td>1 MSPEED[0]</td><td></td><td>Min:0.00</td><td>Max:26.73</td></tr><tr><td>2 MSPEED[1]</td><td></td><td>Min:0.00</td><td>Max:53.46</td></tr><tr><td>3 MSPEED[2]</td><td></td><td>Min:0.00</td><td>Max:80.18</td></tr><tr><td>4 VP_SPEED[0]</td><td></td><td>Min:0.00</td><td>Max:100.00</td></tr></table> 例二 BASE(0,1) ATYPE=1,1	1 MSPEED[0]		Min:0.00	Max:26.73	2 MSPEED[1]		Min:0.00	Max:53.46	3 MSPEED[2]		Min:0.00	Max:80.18	4 VP_SPEED[0]		Min:0.00	Max:100.00
1 MSPEED[0]		Min:0.00	Max:26.73														
2 MSPEED[1]		Min:0.00	Max:53.46														
3 MSPEED[2]		Min:0.00	Max:80.18														
4 VP_SPEED[0]		Min:0.00	Max:100.00														

	UNITS=100,100 '脉冲当量设置 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 DPOS=0,0 MPOS=0,0 TRIGGER '自动触发示波器 MOVE(100,100) 插补轨迹 DPOS(0)水平刻度 100 DPOS(1)垂直刻度 100 
相关指令	MOVEABS,*SP

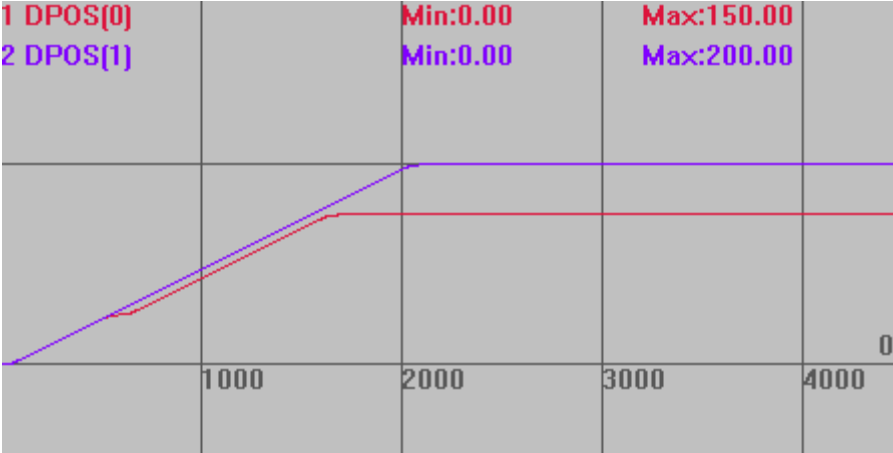
MOVEABS -- 直线运动-绝对

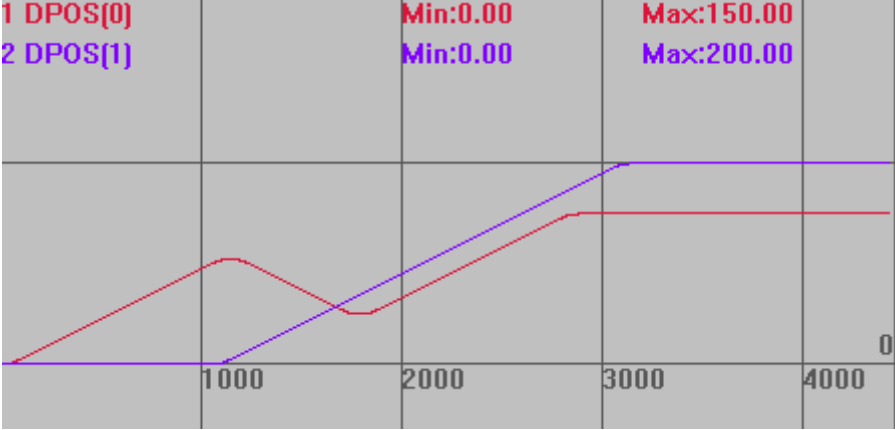
类型	多轴运动指令
描述	直线插补运动，绝对运动到指定坐标。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。
语法	MOVEABS(position1[, position2[, position3[, position4...]]]) position1: 第一个轴运动坐标 position2: 下一个轴运动坐标
适用控制器	通用
例子	BASE(0,1) ATYPE=1,1 UNITS=100,100 DPOS=0,0 MPOS=0,0 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 TRIGGER '自动触发示波器

	MOVEABS(500,300) MOVEABS(100,100) 插补轨迹 DPOS(0)水平刻度 300 DPOS(1)垂直刻度 300 1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:500.00 Max:300.00  如果使用 MOVE 相对运动，其他条件不变，将 MOVEABS 指令改为 MOVE 指令。 插补轨迹 DPOS(0)水平刻度 300 DPOS(1)垂直刻度 300 1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:600.00 Max:400.00 
相关指令	MOVE, *SP

MOVEMODIFY2 -- 运动新位置

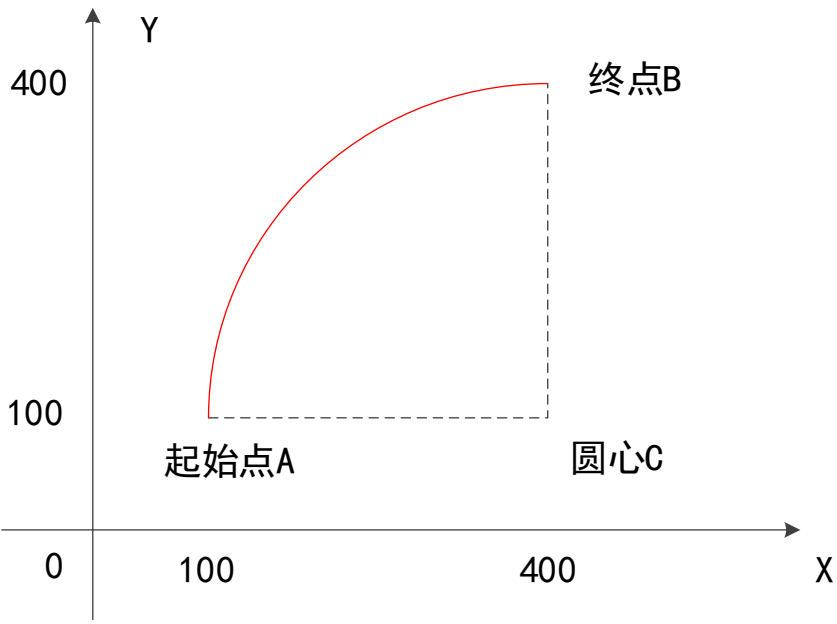
类型	多轴运动指令
描述	强制停止原来的运动，接着按原来的速度与加速度定位到新目标位置。 前面没有运动时与 MOVEABS 效果一样，但各轴的运动是独立的，且不会进入运动缓冲。见例一。 需要 WAIT 指令才能正确使用。见例二。 连续插补时使用 MOVEMODIFY2 会破坏速度连续性。 MOVEMODIFY2 同时对多轴运动时不一定为直线插补。

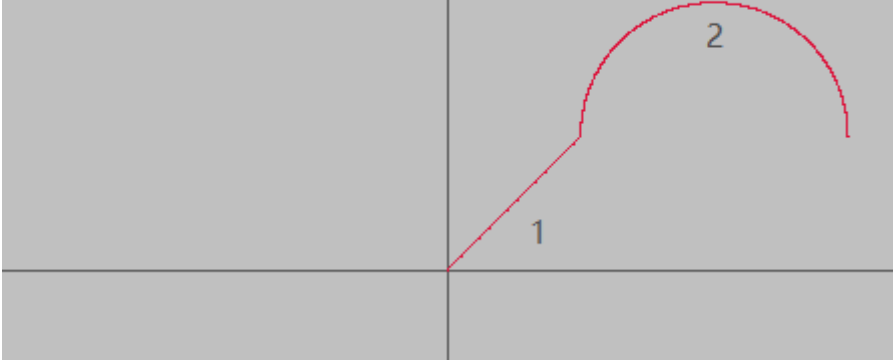
	MOVEMODIFY 和 MOVEMODIFY2 指令在 32 位控制器上使用时不建议设置很大的运动参数，否则可能会导致速度突变的情况；如需设置较大的运动参数，可以设置齿轮比把运动参数降低，或者更换 64 位控制器使用。
语法	MOVEMODIFY2 (abspos1, abspos2,[...]) abspos1: BASE 第一个轴的新目标位置 abspos2: BASE 第二个轴的新目标位置 3X 系列 20161209 以上固件支持. 4 系列 20170509 以上固件支持。
适用控制器	特殊固件
例子	例一 BASE(0,1) ATYPE=1,1 '设为脉冲轴类型 DPOS=0,0 SPEED = 100,100 ACCEL=1000,1000 '加速度设置 DECEL=1000,1000 TRIGGER MOVE(200) AXIS(0) MOVEMODIFY2(50,200) '取消 MOVE(200)，强制定位新的位置(50,200) MOVE(100) AXIS(0) 运动轨迹 DPOS(0)垂直刻度 200 DPOS(1)垂直刻度 200 1 DPOS[0] 2 DPOS[1] Min:0.00 Max:150.00 Min:0.00 Max:200.00  例二 BASE(0,1) ATYPE=1,1 '设为脉冲轴类型 DPOS=0,0 SPEED = 100,100 ACCEL=1000,1000 '加速度设置 DECEL=1000,1000

	TRIGGER MOVE(200) AXIS(0) WAIT UNTIL DPOS(0)>=100 等待轴 0 运行过 100 MOVEMODIFY2(50,200) MOVE(100) AXIS(0) 运动轨迹 竖直刻度同上 1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:150.00 Max:200.00 
相关指令	MOVEMODIFY

MOVECIRC -- 圆心画弧

类型	多轴运动指令
描述	两轴圆弧插补，圆心画弧，相对运动。 BASE 第一轴和第二轴进行圆弧插补，相对移动方式，当终点距离为 0 时为整圆。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 使用时需获得圆心和圆弧终点相对于起始点坐标。 坐标位置请确保正确，否则实际运动轨迹会错误。

	 假设起始点 A 坐标为(100,100)，圆心 C 坐标为(400,100)，终点 B 坐标为(400,400)。圆心 C 相对于起始点 A 的坐标为(300,0)，终点 B 相对于起始点 A 的坐标为(300,300)。
语法	MOVECIRC(end1, end2, centre1, centre2, direction) end1: 终点第一个轴运动坐标，相对于起始点 end2: 终点第二个轴运动坐标，相对于起始点 centre1: 圆心第一个轴运动坐标，相对于起始点 centre2: 圆心第二个轴运动坐标，相对于起始点 direction: 0-逆时针，1-顺时针
适用控制器	通用
例子	BASE(0,1) ATYPE=1,1 '设为脉冲轴类型 UNITS=100,100 DPOS=0,0 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 TRIGGER '自动触发示波器 MOVE(100,100) '先运动 100,100 位置 MOVECIRC(200,0,100,0,1) '半径 100 顺时针画半圆，终点坐标 (300,100) 插补轨迹 DPOS(0)垂直刻度 150 DPOS(1)垂直刻度 150

	1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:300.00 Max:199.99 
	其他条件不变，将运动指令修改，起点与终点相同画整圆。 MOVECIRC(0,0,100,0,0) 半径 100，圆心（100，0）逆时针画圆 插补轨迹 竖直刻度同上 1 DPOS[0] 2 DPOS[1] Min:0.00 Min:-100.00 Max:199.99 Max:100.00 
	相关指令 MOVECIRCABS，MOVECIRC2，*SP

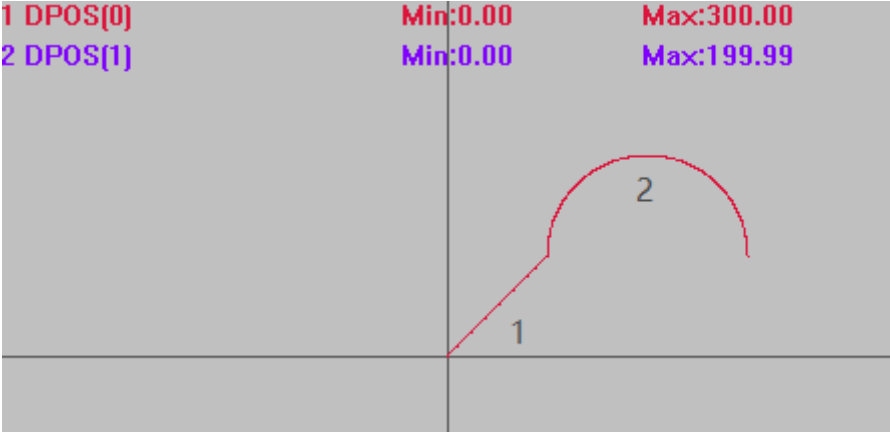
MOVECIRCABS -- 圆心画弧-绝对

类型	多轴运动指令
描述	两轴圆弧插补，圆心画弧，绝对运动。 BASE 第一轴和第二轴进行圆弧插补，绝对移动方式。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 MOVECIRCABS 不支持走整圆，整圆使用 MOVECIRC。
语法	MOVECIRCABS(end1, end2, centre1, centre2, direction) end1： 终点第一个轴运动坐标，绝对位置

	end2: 终点第二个轴运动坐标, 绝对位置 centre1: 圆心第一个轴运动坐标, 绝对位置 centre2: 圆心第二个轴运动坐标, 绝对位置 direction: 0-逆时针, 1-顺时针 坐标位置请确保正确, 否则实际运动轨迹会错误。
适用控制器	通用
例子	BASE(0,1) ATYPE=1,1 '设为脉冲轴类型 UNITS=100,100 DPOS=0,0 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 TRIGGER '自动触发示波器 MOVE(100,100) '先运动 100,100 位置 MOVECIRCABS(200,0,100,0,1) '半径 100 顺时针画 1/4 圆, 终点坐标(200,0) 插补轨迹 DPOS(0)垂直刻度 100 DPOS(1)垂直刻度 100
相关指令	MOVECIRC , MOVECIRC2ABS , *SP

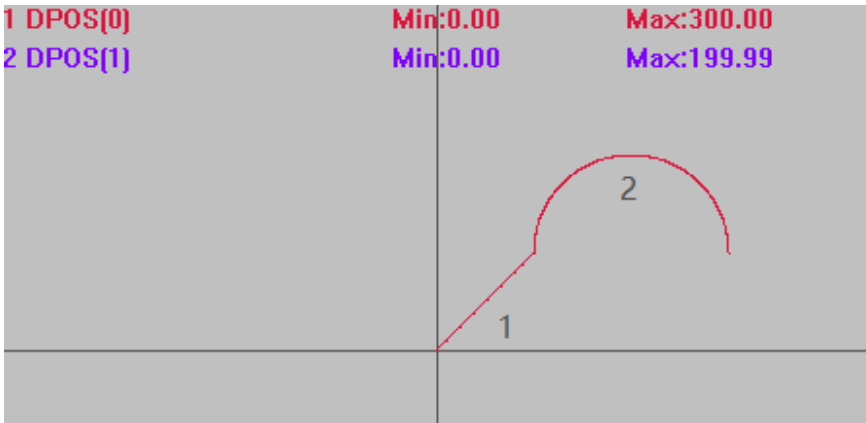
MOVECIRC2 -- 三点画弧

类型	多轴运动指令
描述	两轴圆弧插补, 三点画弧, 相对运动。 BASE 第一轴和第二轴进行圆弧插补, 相对移动方式, 坐标为相对起始点的距离。 自定义速度的连续插补运动可以使用 SP 后缀的指令, 见*SP 描述。 注意: 不能用此指令进行整圆插补运动, 整圆使用 MOVECIRC 相对圆弧, 或连续使用

	两条此类指令。
语法	MOVECIRC2(mid1, mid2, end1, end2) mid1: 中间点第一个轴坐标, 相对起始点距离 mid2: 中间点第二个轴坐标, 相对起始点距离 end1: 结束点第一个轴坐标, 相对起始点距离 end2: 结束点第二个轴坐标, 相对起始点距离 坐标位置请确保正确, 否则实际运动轨迹会错误。
适用控制器	通用
例子	BASE(0,1) ATYPE=1,1 '设为脉冲轴类型 UNITS=100,100 DPOS=0,0 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 TRIGGER '自动触发示波器 MOVE(100,100) '运动 100, 100 位置 MOVECIRC2(100,100,200,0) '三点画半圆, 相对坐标 插补轨迹 DPOS(0)垂直刻度 200 DPOS(1)垂直刻度 200 
相关指令	MOVECIRC2ABS , MOVECIRC , *SP

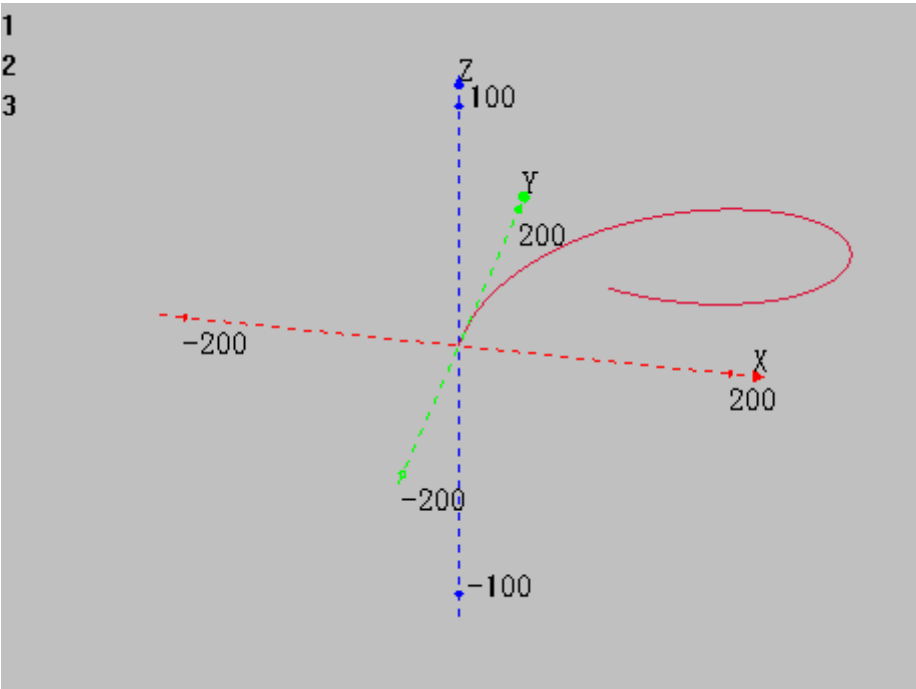
MOVECIRC2ABS -- 三点画弧-绝对

类型	多轴运动指令
描述	圆弧插补, 三点画弧, 绝对运动。 BASE 第一轴和第二轴进行圆弧插补, 绝对移动方式。自定义速度的连续插补运动可以使用 SP 后缀的指令, 见*SP 描述。

	注意：不能用此指令进行整圆插补运动，整圆使用 MOVECIRC 相对圆弧，或连续使用两条此类指令。
语法	MOVECIRC2ABS(mid1, mid2, end1, end2) mid1: 中间点第一个轴坐标，绝对位置 mid2: 中间点第二个轴坐标，绝对位置 end1: 结束点第一个轴坐标，绝对位置 end2: 结束点第二个轴坐标，绝对位置 坐标位置请确保正确，否则实际运动轨迹会错误。
适用控制器	通用
例子	BASE(0,1) ATYPE=1,1 '设为脉冲轴类型 UNITS=100,100 DPOS=0,0 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 TRIGGER '自动触发示波器 MOVE(100,100) '先运动 100,100 位置 MOVECIRC2ABS(200,200,300,100) '三点画半圆，绝对坐标 插补轨迹 DPOS(0)垂直刻度 200 DPOS(1)垂直刻度 200 
相关指令	MOVECIRC2 , MOVECIRCABS , *SP

MHELICAL -- 圆心螺旋

类型	多轴运动指令
描述	螺旋插补，圆心画弧，相对运动。
	BASE 第一轴和第二轴进行圆弧插补，第三轴进行螺旋，相对于起始点。自定义速度的

	连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 可完整螺旋一圈，从 Z 方向看为一整圆。						
语法	MHELICAL(end1,end2,centre1,centre2,direction,distance3,[mode]) end1: 结束点第一个轴运动坐标，相对于起始点 end2: 结束点第二个轴运动坐标，相对于起始点 centre1: 圆心第一个轴运动坐标，相对于起始点 centre2 : 圆心第二个轴运动坐标，相对于起始点 direction: 0-逆时针，1-顺时针 distance3: 第三个轴运动距离 mode: 第三轴的速度计算 <table><tr><td>值</td><td>描述</td></tr><tr><td>0(缺省)</td><td>第三轴参与插补速度计算</td></tr><tr><td>1</td><td>第三轴不参与插补速度计算</td></tr></table> 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0(缺省)	第三轴参与插补速度计算	1	第三轴不参与插补速度计算
值	描述						
0(缺省)	第三轴参与插补速度计算						
1	第三轴不参与插补速度计算						
适用控制器	通用						
例子	BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 TRIGGER MHELICAL(200,-200,200,0,1,100) '原点作为起点，中心(200,0)，终点'(200,-200)，顺时针，Z 轴参与速度计算，运动 100 XYZ 模式下轴 0 轴 1 轴 2 插补轨迹 						

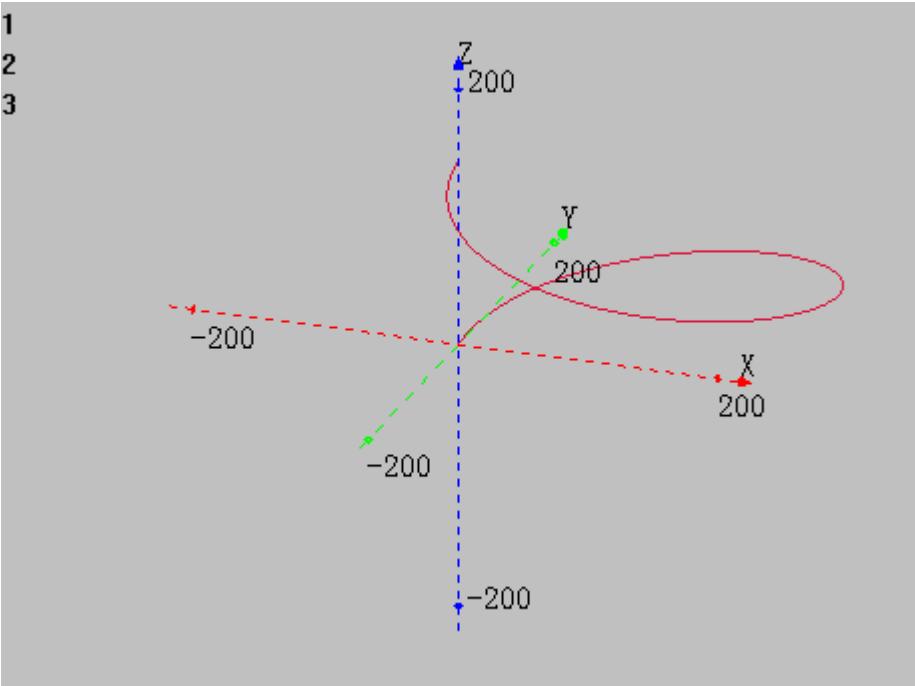
	XY 模式下轴 0 轴 1 插补轨迹 1 DPOS[0] 2 DPOS[1] 3 DPOS[2] Min:0.00 Min:-200.00 Min:0.00 Max:399.99 Max:199.99 Max:100.00
--	---

MHELICALABS -- 圆心螺旋-绝对

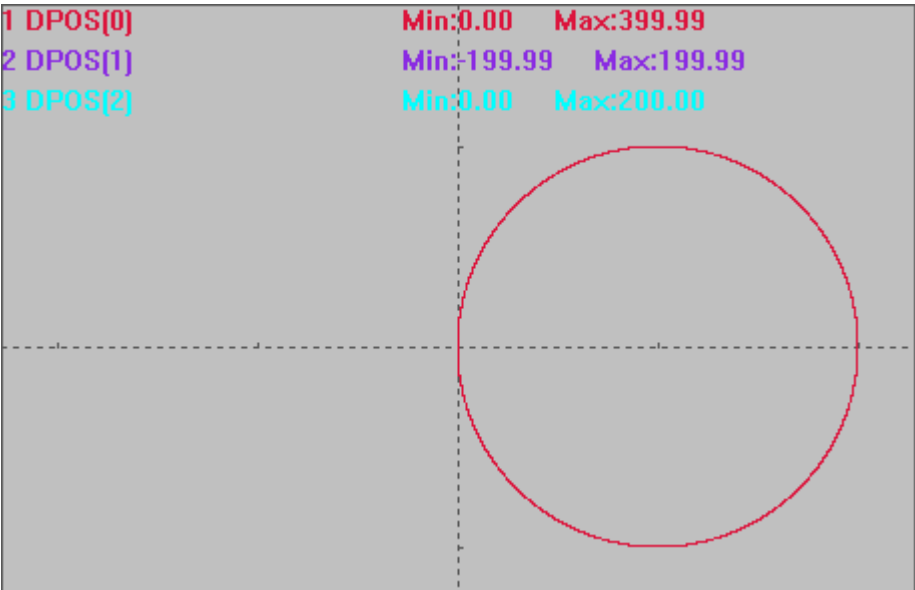
类型	多轴运动指令						
描述	螺旋插补，圆心画弧，绝对运动。 BASE 第一轴和第二轴进行圆弧插补，第三轴进行螺旋，绝对移动方式。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 可完整螺旋一圈，从 Z 方向看为一个整圆。						
语法	MHELICALABS(end1,end2,centre1,centre2,direction,distance3,[mode]) end1: 第一个轴运动坐标 end2: 第二个轴运动坐标 centre1: 第一个轴运动圆心 centre2: 第二个轴运动圆心 direction: 0-逆时针，1-顺时针 distance3: 第三个轴运动坐标 mode: 第三轴的速度计算: <table><tr><td>值</td><td>描述</td></tr><tr><td>0(缺省)</td><td>第三轴参与插补速度计算</td></tr><tr><td>1</td><td>第三轴不参与插补速度计算</td></tr></table> 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0(缺省)	第三轴参与插补速度计算	1	第三轴不参与插补速度计算
值	描述						
0(缺省)	第三轴参与插补速度计算						
1	第三轴不参与插补速度计算						
适用控制器	通用						
例子	BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100						

```
DPOS=0,0,0
SPEED=100,100,100      '主轴速度
ACCEL=1000,1000,1000   '主轴加速度
DECEL=1000,1000,1000
TRIGGER
MHELICALABS(0,0,200,0,1,200)  '起点原点，中心(200,0)，终点（0,0），'顺时针，
                                Z 轴参与速度计算，运动 200
```

XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



XY 模式下轴 0 轴 1 插补轨迹

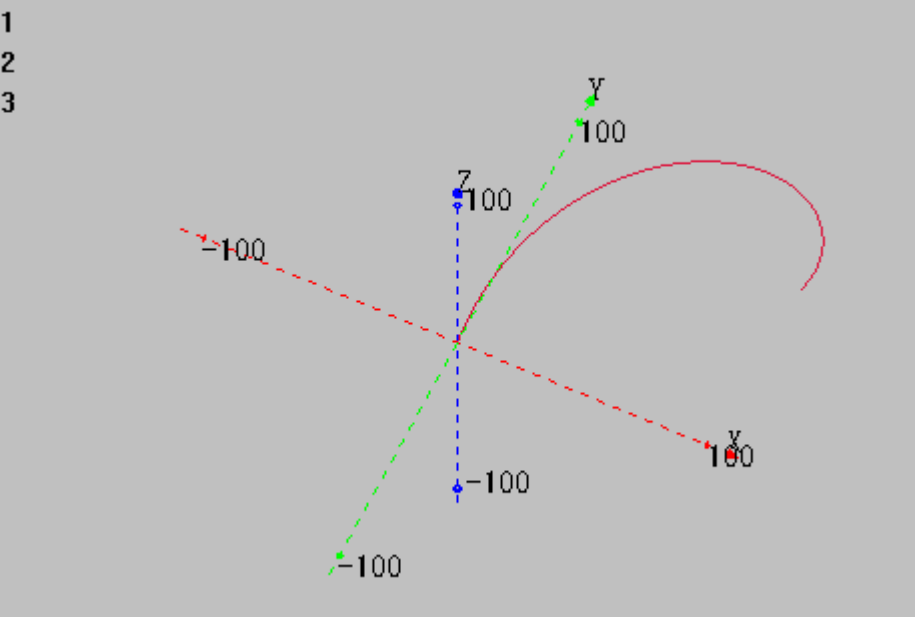
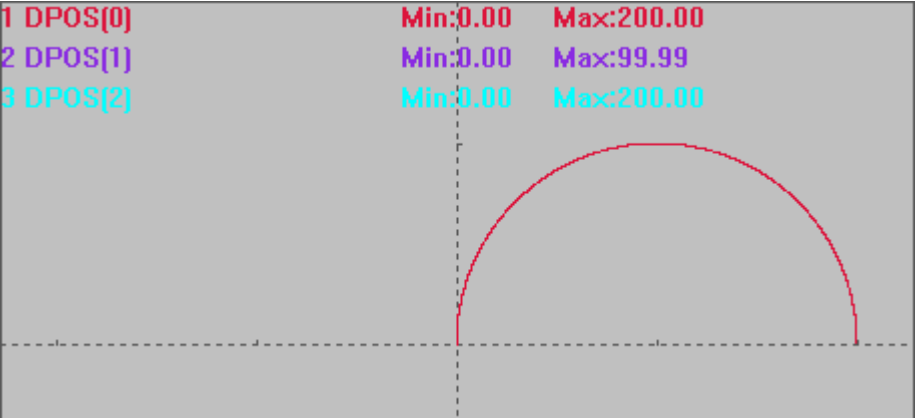


相关指令

[MHELICAL](#), [MHELICAL2ABS](#), [*SP](#)

MHELICAL2 -- 三点螺旋

类型	多轴运动指令						
描述	螺旋插补，三点画弧，相对运动。 BASE 第一轴和第二轴进行圆弧插补，第三轴进行螺旋，相对移动方式。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 无法螺旋一圈，需要螺旋一圈请使用 MHELICAL 或 MHELICALABS。						
语法	MHELICAL2(mid1, mid2, end1, end2, distance3,[mode]) mid1: 中间点第一个轴坐标，相对起始点距离 mid2: 中间点第二个轴坐标，相对起始点距离 end1: 结束点第一个轴坐标，相对起始点距离 end2: 结束点第二个轴坐标，相对起始点距离 distance3: 第三个轴运动距离，相对起始点距离 mode: 第三轴的速度计算 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0(缺省)</td><td>第三轴参与插补速度计算</td></tr> <tr> <td>1</td><td>第三轴不参与插补速度计算</td></tr> </table> 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0(缺省)	第三轴参与插补速度计算	1	第三轴不参与插补速度计算
值	描述						
0(缺省)	第三轴参与插补速度计算						
1	第三轴不参与插补速度计算						
适用控制器	通用						
例子	<pre> BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 TRIGGER MHELICAL2(100,100,200,0,200) '起点原点，中间点(100,100)，终点（200,0），Z 轴参与速度计算，运动 200 </pre> XYZ 模式下轴 0 轴 1 轴 2 插补轨迹						

	1 2 3  XY 模式下轴 0 轴 1 插补轨迹 1 DPOS[0] 2 DPOS[1] 3 DPOS[2] Min:0.00 Min:0.00 Min:0.00 Max:200.00 Max:99.99 Max:200.00 
--	--

MHELICAL2ABS -- 三点螺旋-绝对

类型	多轴运动指令
描述	螺旋插补，三点画弧，绝对运动。 BASE 第一轴和第二轴进行圆弧插补，第三轴进行螺旋，绝对移动方式。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 无法螺旋一圈，需要螺旋一圈请使用 MHELICAL。
语法	MHELICAL2ABS(mid1, mid2, end1, end2, distance3,[mode]) mid1: 中间点第一个轴坐标 mid2: 中间点第二个轴坐标 end1: 结束点第一个轴坐标 end2: 结束点第二个轴坐标 distance3: 第三个轴结束点坐标，勘误：20150306 以前的版本此参数有问题，建

	议使用 MHELICAL2 相对指令 mode: 第三轴的速度计算 <table><tr><th>值</th><th>描述</th></tr><tr><td>0(缺省)</td><td>第三轴参与插补速度计算</td></tr><tr><td>1</td><td>第三轴不参与插补速度计算</td></tr></table> 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0(缺省)	第三轴参与插补速度计算	1	第三轴不参与插补速度计算
值	描述						
0(缺省)	第三轴参与插补速度计算						
1	第三轴不参与插补速度计算						
适用控制器	通用						
例子	BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 MOVE(100,100) '运动到(100,100) TRIGGER MHELICAL2ABS(200,100,200,0,200) '起点(100,100)，中间点(200,100)，结束点(200,0)，Z 轴参与速度计算，运动 200 XYZ 模式下轴 0 轴 1 轴 2 插补轨迹 XY 模式下轴 0 轴 1 插补轨迹						

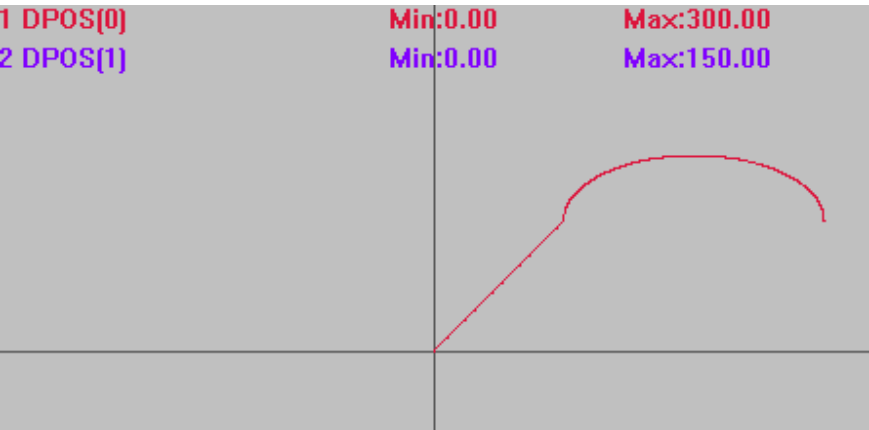
	1 DPOS[0]Min:0.00Max:220.71 2 DPOS[1]Min:0.00Max:120.71 3 DPOS[2]Min:0.00Max:200.00
相关指令	MHELICAL2 , MHELICALABS , *SP

MECLIPSEABS -- 椭圆-绝对

类型	多轴运动指令						
描述	椭圆插补，中心画弧，绝对运动，可选螺旋。 BASE 第一轴和第二轴进行椭圆插补，绝对移动方式，可选第三个轴同步螺旋。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 可画整椭圆(控制器 64 位精度)。起点和终点接近时，正圆轨迹要用相对方式。						
语法	MECLIPSEABS(end1, end2, centre1, centre2, direction, adis, bdis[, end3]) end1: 终点第一个轴运动坐标 end2: 终点第二个轴运动坐标 centre1: 中心第一个轴运动坐标 centre2: 中心第二个轴运动坐标 direction: 0-逆时针，1-顺时针 <table><tr><td>值</td><td>描述</td></tr><tr><td>0</td><td>逆时针</td></tr><tr><td>1</td><td>顺时针</td></tr></table> adis: 第一轴的椭圆半径，半长轴或者半短轴都可 bdis: 第二轴的椭圆半径，半长轴或者半短轴都可，AB 相等时自动为圆弧或螺旋 end3: 第三个轴的运动坐标，需要螺旋时填入 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0	逆时针	1	顺时针
值	描述						
0	逆时针						
1	顺时针						
适用控制器	通用						
例子	例一 不进行螺旋 BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度						

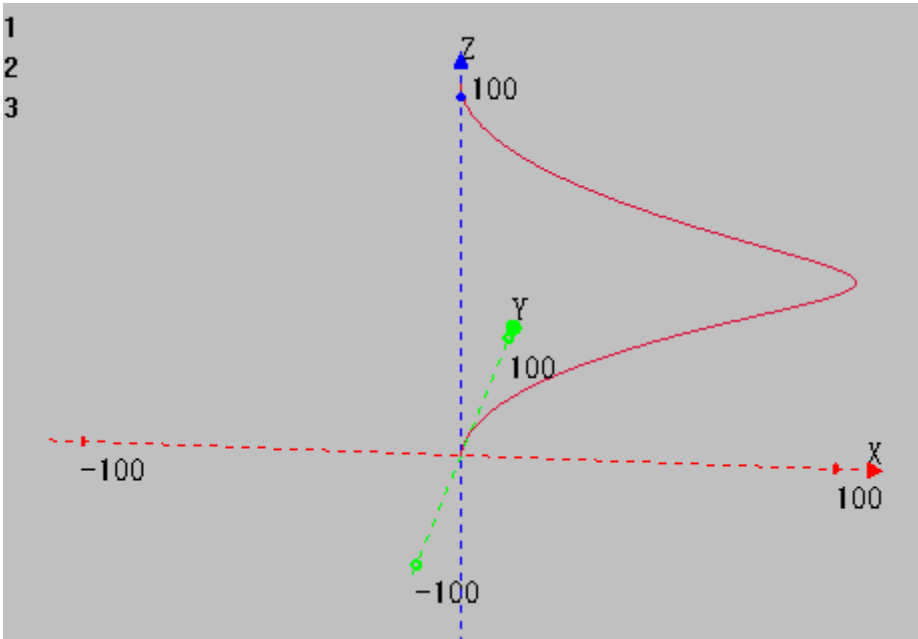
```
DECEL=1000,1000,1000
TRIGGER          '自动触发示波器
MOVE(100,100)
MECLIPSEABS(300,100,200,100,1,100,50)  '中心(200,100), 终点(300,100), 半短轴 50,
      半长轴 100, 画半椭圆圆弧, 不进行螺旋
```

插补轨迹
DPOS(0)垂直刻度 150
DPOS(1)垂直刻度 150

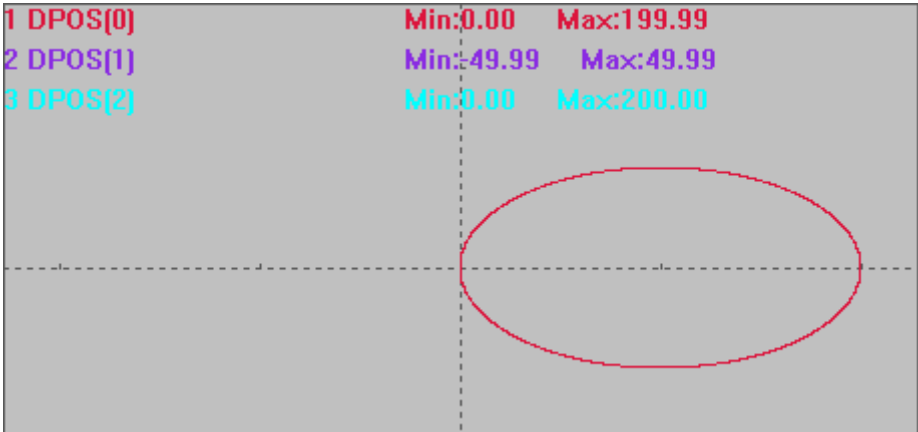


```
例二 进行螺旋
BASE(0,1,2)
ATYPE=1,1,1      '设为脉冲轴类型
UNITS=100,100,100
DPOS=0,0,0
SPEED=100,100,100  '主轴速度
ACCEL=1000,1000,1000  '主轴加速度
DECEL=1000,1000,1000
MECLIPSEABS(0,0,100,0,1,100,50,200)  '中心(100,0), 终点(0,0), 半短轴 50, 半长轴
      100, 画整椭圆, 同时螺旋
```

XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



XY 模式下轴 0 轴 1 插补轨迹

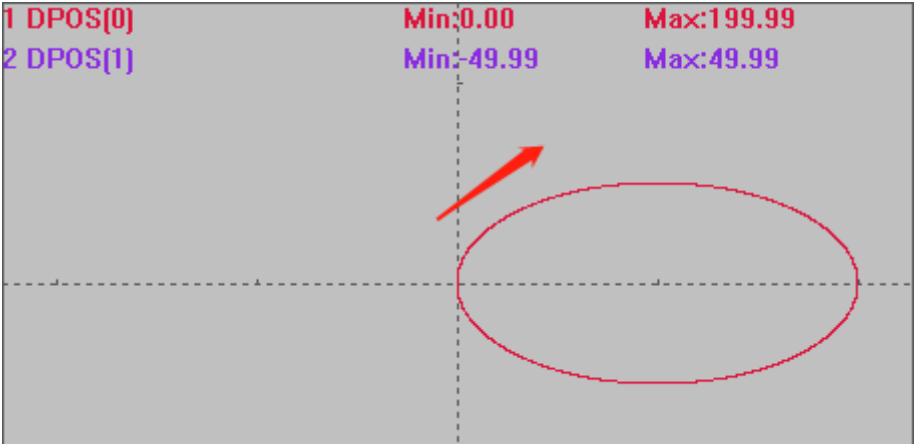


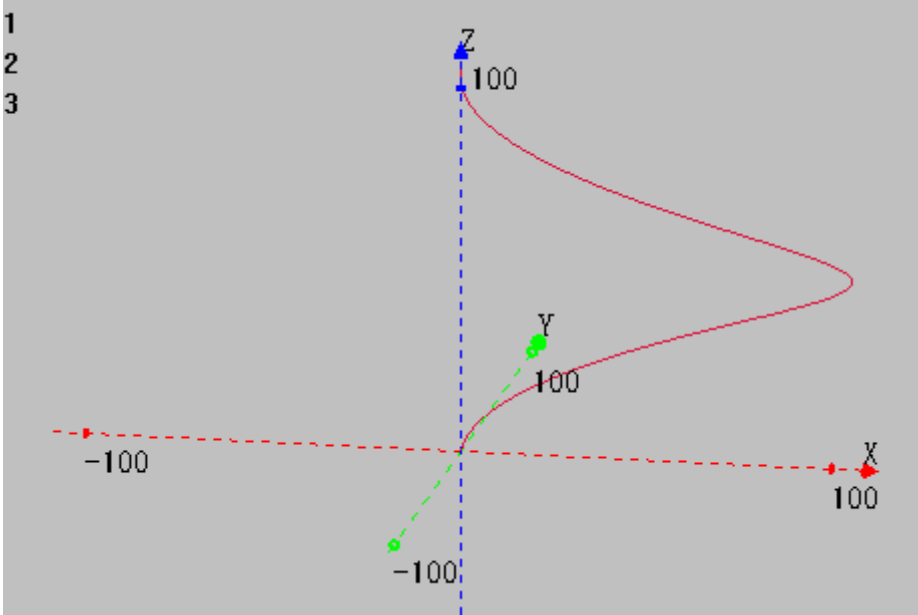
相关指令

[MECLIPSE](#), [*SP](#)

MECLIPSE -- 椭圆

类型	多轴运动指令
描述	椭圆插补，中心画弧，相对运动，可选螺旋。 BASE 第一轴和第二轴进行椭圆插补，相对移动方式，可选第三个轴同步螺旋。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 可画整椭圆。 只能画长轴（短轴）与 X 平行或垂直的椭圆。
语法	MECLIPSE (end1, end2, centre1, centre2, direction, adis, bdis[, end3]) end1: 终点第一个轴运动坐标，相对于起始点 end2: 终点第二个轴运动坐标，相对于起始点 centre1: 中心第一个轴运动坐标，相对于起始点

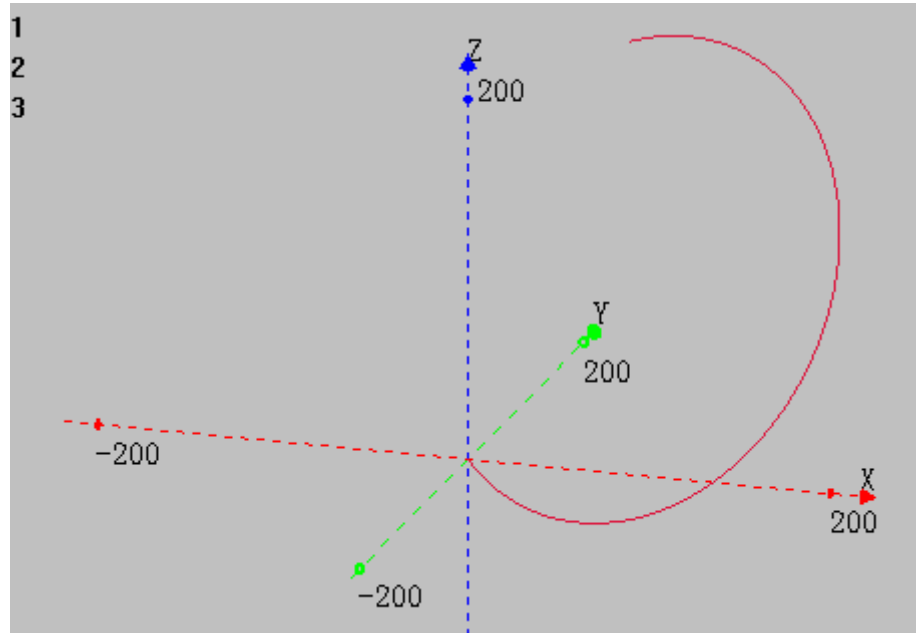
	centre2: 中心第二个轴运动坐标，相对于起始点 direction: 0-逆时针，1-顺时针 <table><tr><th>值</th><th>描述</th></tr><tr><td>0</td><td>逆时针</td></tr><tr><td>1</td><td>顺时针</td></tr></table> adis: 第一轴的椭圆半径，半长轴或者半短轴都可 bdis: 第二轴的椭圆半径，半长轴或者半短轴都可，ab 相等时自动为圆弧或螺旋 end3: 第三个轴的运动距离，需要螺旋时填入 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0	逆时针	1	顺时针
值	描述						
0	逆时针						
1	顺时针						
适用控制器	通用						
例子	例一 不进行螺旋 RAPIDSTOP(2) WAIT IDLE(0) BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 DPOS=0,0,0 TRIGGER '自动触发示波器 MECLIPSE(0,0,100,0,1,100,50) '中心(100,0)，终点(0,0)，半短轴 50，半长轴 100，顺时针画整椭圆，不进行螺旋 插补轨迹 DPOS(0)垂直刻度 100 DPOS(1)垂直刻度 100 1 DPOS[0] 2 DPOS[1] Min:0.00 Min:-49.99 Max:199.99 Max:49.99  例二 进行螺旋 RAPIDSTOP(2) WAIT IDLE(0) BASE(0,1,2)						

	ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 DPOS=0,0,0 TRIGGER '自动触发示波器 MECLIPSE(0,0,100,0,1,100,50,200) '中心(100,0)，终点(0,0)，半短轴 50，半长轴 100， 顺时针画整椭圆圆弧，进行螺旋，第三轴运动距离 200 XYZ 模式下轴 0 轴 1 轴 2 插补轨迹，在例一的基础上合成 Z 轴的运动 
相关指令	MECLIPSEABS ， *SP

MSPHERICAL -- 空间圆弧

类型	多轴运动指令	
描述	空间圆弧插补运动，相对移动方式，可选螺旋。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。	
语法	MSPHERICAL(end1,end2,end3,centre1,centre2,centre3,mode[,distance4][,distance5][,distance6])	
	end1: 第 1 个轴运动距离参数 1	
	end2: 第 2 个轴运动距离参数 1	
	end3: 第 3 个轴运动距离参数 1	
	centre1: 第 1 个轴运动距离参数 2	
	centre2: 第 2 个轴运动距离参数 2	
	centre3: 第 3 个轴运动距离参数 2	
	mode: 指定前面参数的意义	
	值	描述

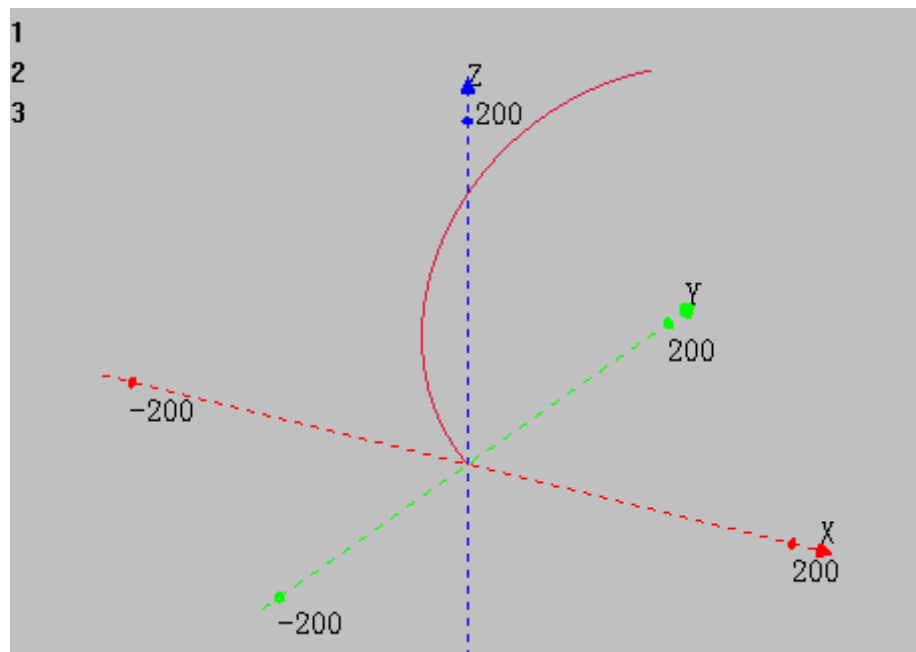
	0	当前点，中间点，终点三点定圆弧 距离参数 1 为终点距离，距离参数 2 为中间点距离
	1	当前点，圆心，终点定圆弧 走最短的圆弧 距离参数 1 为终点距离，距离参数 2 为圆心的距离
	2	当前点，中间点，终点三点定整圆 距离参数 1 为终点距离，距离参数 2 为中间点距离
	3	当前点，圆心，终点定整圆 先走最短的圆弧，再继续走完整圆 距离参数 1 为终点距离，距离参数 2 为圆心的距离
	distane4: 第四轴螺旋的功能，指定第 4 轴的相对距离，此轴不参与速度计算 distane5: 第五轴螺旋的功能，指定第 5 轴的相对距离，此轴不参与速度计算 distane6: 第六轴螺旋的功能，指定第 5 轴的相对距离，此轴不参与速度计算 坐标位置请确保正确，否则实际运动轨迹会错误。	
适用控制器	通用	
例子	BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 TRIGGER 使用圆心为(120,160,150)，半径 250 的圆演示 4 种 mode 的运动轨迹 mode 0: MSPHERICAL(120,160,400,240,320,300,0) '终点(120,160,400)，中间点(240,320,300)，模式 0，三点画弧 XYZ 模式下轴 0 轴 1 轴 2 插补轨迹	



mode 1:

MSPHERICAL(120,160,400,120,160,150,1) '相对位置，终点(120,160,400)，圆心(120,160,150)，模式1，走最短的圆弧

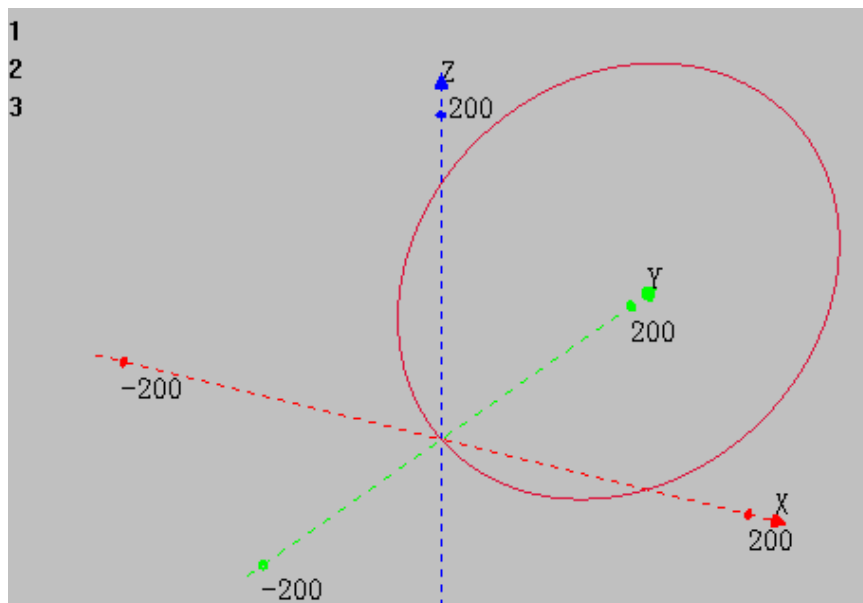
XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



mode 2:

MSPHERICAL(120,160,400,240,320,300,2) 终点(120,160,400)，中间点(240,320,300)，模式2，三点画整圆

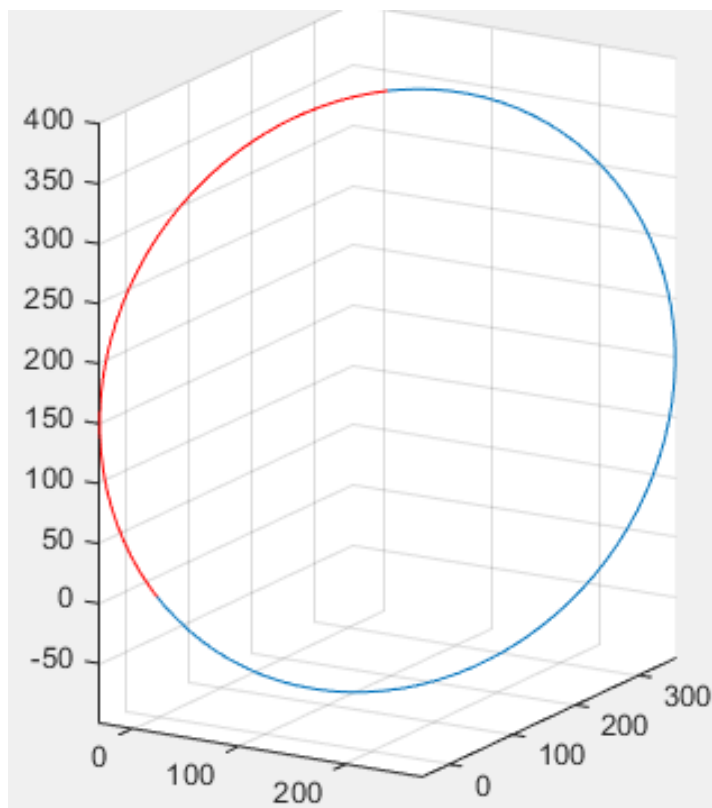
XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



mode 3:

MSPHERICAL(120,160,400,120,160,150,3)'终点(120,160,400)，圆心(120,160,150)，模式3，先走最短的圆弧(红色部分)，再走完整圆

XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



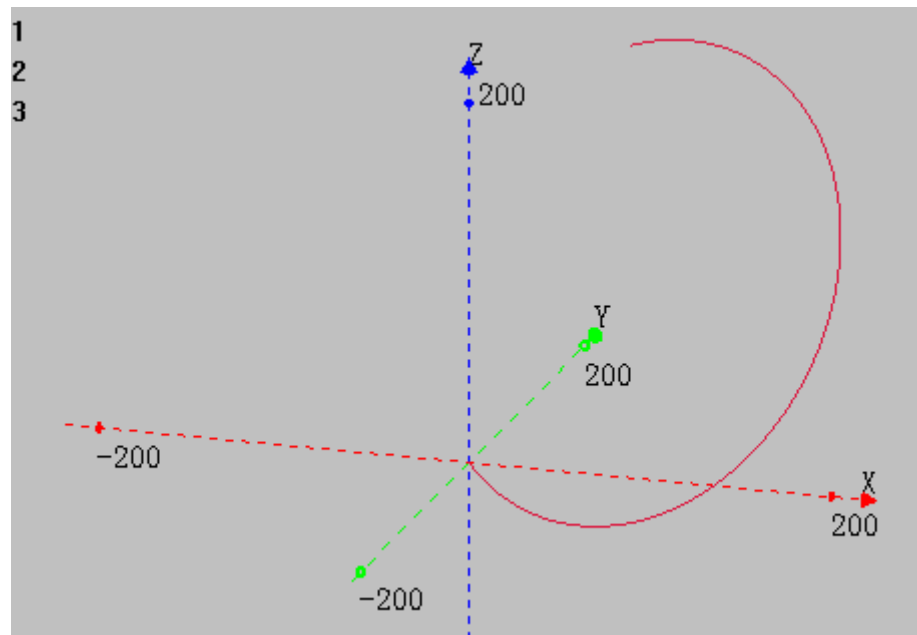
相关指令

[MSPHERICALABS](#)

MSPHERICALABS -- 空间圆弧-绝对

类型	多轴运动指令										
描述	空间圆弧插补运动，绝对移动方式，可选螺旋。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。										
语法	MSPHERICALABS(end1,end2,end3,centre1,centre2,centre3,mode[,distance4][,distance5][,distance6]) end1: 第 1 个轴运动距离参数 1 end2: 第 2 个轴运动距离参数 1 end3: 第 3 个轴运动距离参数 1 centre1: 第 1 个轴运动距离参数 2 centre2: 第 2 个轴运动距离参数 2 centre3: 第 3 个轴运动距离参数 2 mode: 指定前面参数的意义 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>当前点，中间点，终点三点定圆弧 距离参数 1 为终点距离，距离参数 2 为中间点距离</td></tr> <tr> <td>1</td><td>当前点，圆心，终点定圆弧 走最短的圆弧 距离参数 1 为终点距离，距离参数 2 为圆心的距离</td></tr> <tr> <td>2</td><td>当前点，中间点，终点三点定整圆 距离参数 1 为终点距离，距离参数 2 为中间点距离</td></tr> <tr> <td>3</td><td>当前点，圆心，终点定整圆 先走最短的圆弧，再继续走完整圆 距离参数 1 为终点距离，距离参数 2 为圆心的距离</td></tr> </tbody> </table> distane4: 第四轴螺旋的功能，指定第 4 轴的绝对距离，此轴不参与速度计算 distane5: 第五轴螺旋的功能，指定第 5 轴的绝对距离，此轴不参与速度计算 distane6: 第六轴螺旋的功能，指定第 5 轴的绝对距离，此轴不参与速度计算 坐标位置请确保正确，否则实际运动轨迹会错误。	值	描述	0	当前点，中间点，终点三点定圆弧 距离参数 1 为终点距离，距离参数 2 为中间点距离	1	当前点，圆心，终点定圆弧 走最短的圆弧 距离参数 1 为终点距离，距离参数 2 为圆心的距离	2	当前点，中间点，终点三点定整圆 距离参数 1 为终点距离，距离参数 2 为中间点距离	3	当前点，圆心，终点定整圆 先走最短的圆弧，再继续走完整圆 距离参数 1 为终点距离，距离参数 2 为圆心的距离
值	描述										
0	当前点，中间点，终点三点定圆弧 距离参数 1 为终点距离，距离参数 2 为中间点距离										
1	当前点，圆心，终点定圆弧 走最短的圆弧 距离参数 1 为终点距离，距离参数 2 为圆心的距离										
2	当前点，中间点，终点三点定整圆 距离参数 1 为终点距离，距离参数 2 为中间点距离										
3	当前点，圆心，终点定整圆 先走最短的圆弧，再继续走完整圆 距离参数 1 为终点距离，距离参数 2 为圆心的距离										
适用控制器	通用										
例子	BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 TRIGGER 使用圆心为(120,160,150)，半径 250 的圆演示 4 种 mode 的运动轨迹 mode 0: MSPHERICALABS(120,160,400,240,320,300,0) ' 终点 (120,160,400) ， 中 间 点 (240,320,300)，模式 0，三点画弧										

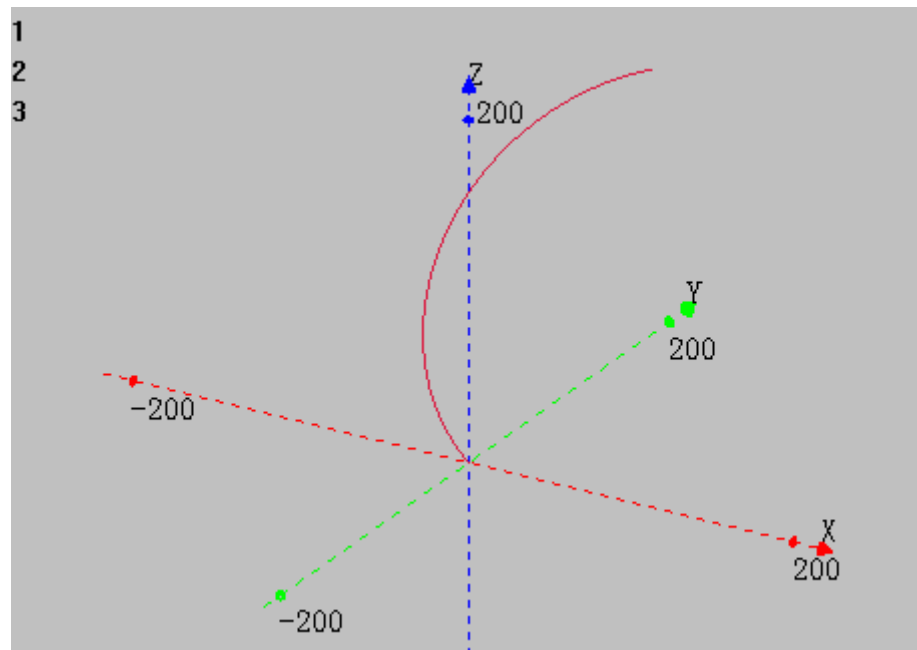
XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



mode 1:

MSPHERICALABS(120,160,400,120,160,150,1) '绝对位置，终点(120,160,400)，圆心(120,160,150)，模式 1，走最短的圆弧

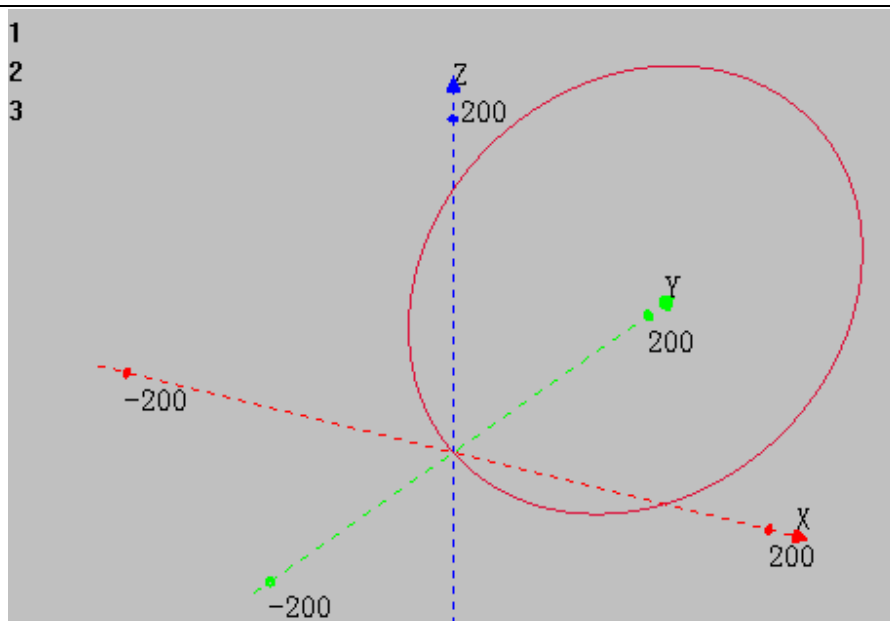
XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



mode 2:

MSPHERICALABS(120,160,400,240,320,300,2) '终点 (120,160,400)，中间点 (240,320,300)，模式 2，三点画整圆

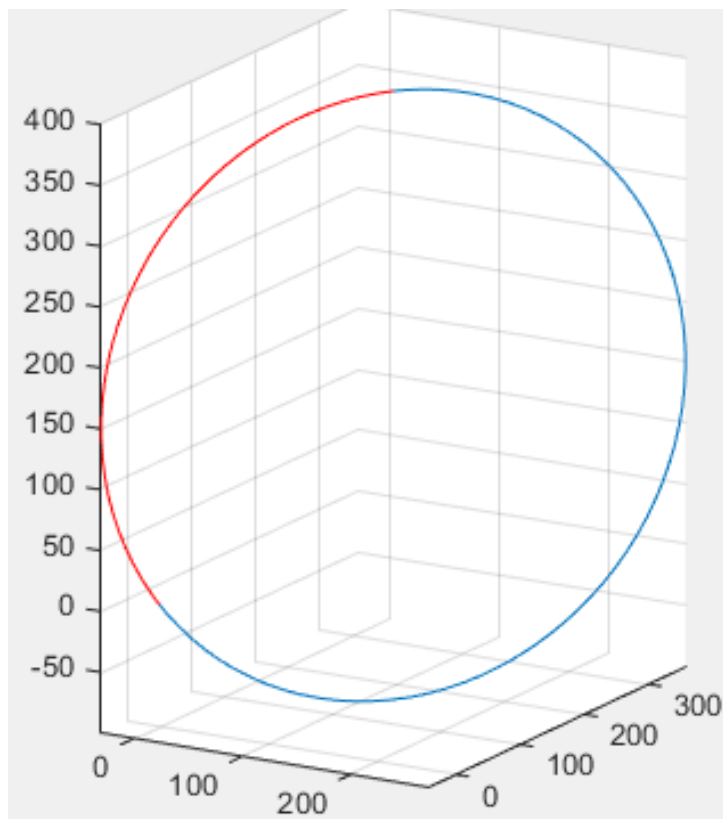
XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



mode 3:

MSPHERICALABS(120,160,400,120,160,150,3) '终点(120,160,400), 圆心(120,160,150),
模式 3, 先走最短的圆弧(红色部分), 再走完整圆

XYZ 模式下轴 0 轴 1 轴 2 插补轨迹

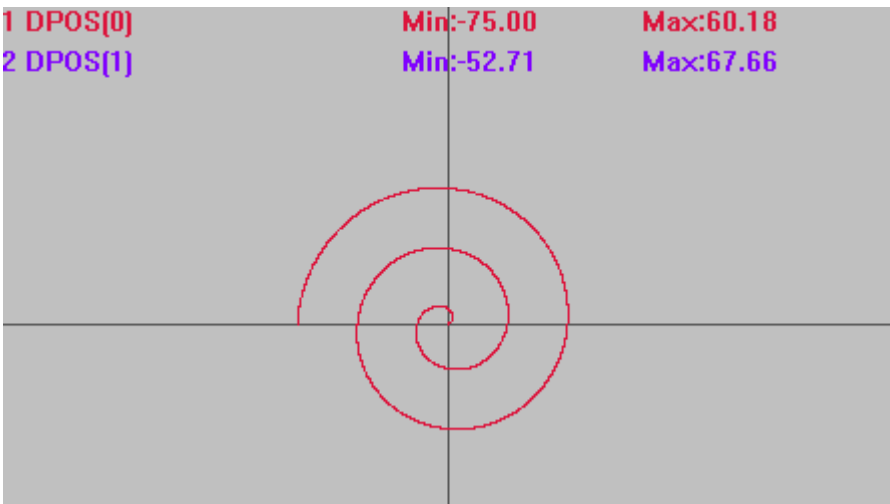


相关指令

[MSPHERICAL](#)

MOVESPIRAL -- 渐开线圆弧

类型	多轴运动指令
描述	渐开线圆弧插补运动，相对移动方式，可选螺旋。 当前点和圆心距离确定起始半径，当起始半径 0 时角度无法确定，直接从 0 角度开始。见例一。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。
语法	MOVESPIRAL(centre1,centre2,circles,pitch[,distance3][,distance4]) centre1: 圆心的第 1 轴相对距离 centre2: 圆心的第 2 轴相对距离 circles: 要旋转的圈数，可以为小数圈，负数表示顺时针，每圈终点位置为起点和圆心连线上的一点，见例二 pitch: 每圈的扩散距离，可以为负 distance3: 第 3 轴螺旋的功能，指定第 3 轴的相对距离，此轴不参与速度计算 distance4: 第 4 轴螺旋的功能，指定第 4 轴的相对距离，此轴不参与速度计算
适用控制器	通用
例子	BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 TRIGGER '自动触发示波器 例一 从起点扩散 MOVESPIRAL(0,0,2.5,30) '此时以起始位置为中心，逆时针旋转 2.5 圈，每圈扩散 30 插补轨迹 DPOS(0)垂直刻度 100 DPOS(1)垂直刻度 100



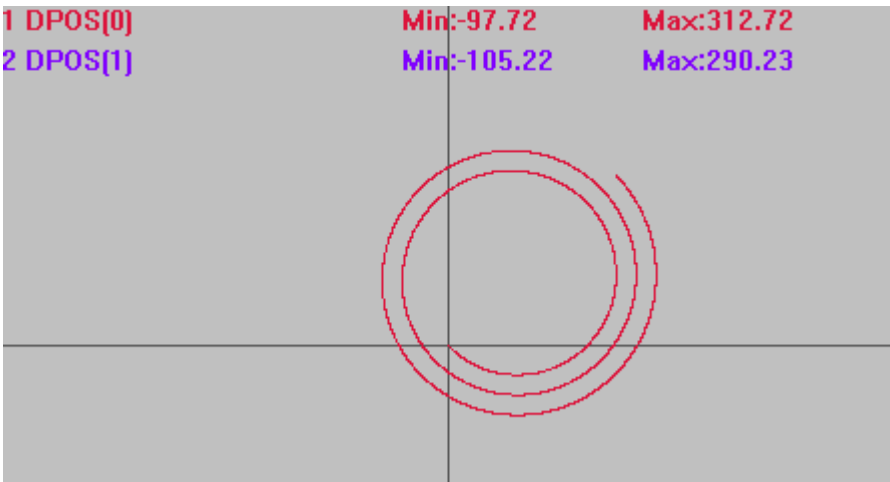
例二 不螺旋

MOVESPIRAL (100,100,2.5,30) '起始半径 100，以(100,100)为圆心，逆时针旋转 2.5 圈，每圈向外扩散 30

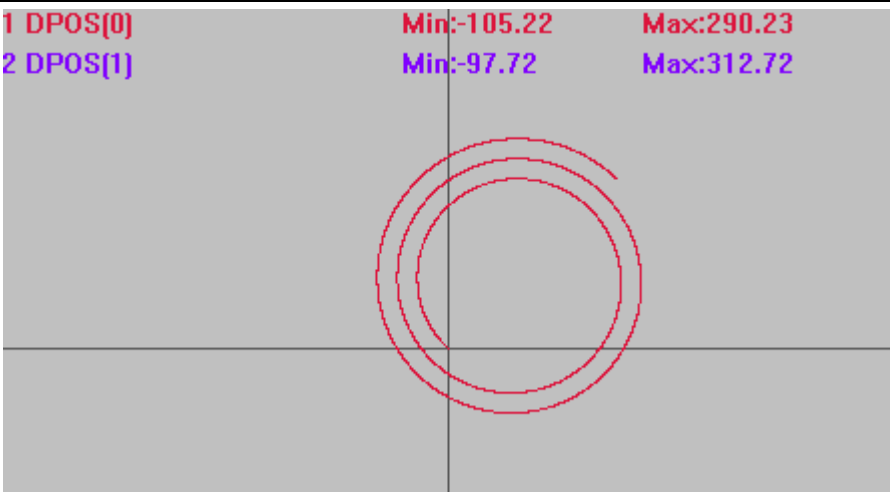
插补轨迹（若轨迹圈数显示不全，将采样间隔适当调大）

DPOS(0)垂直刻度 300

DPOS(1)垂直刻度 300

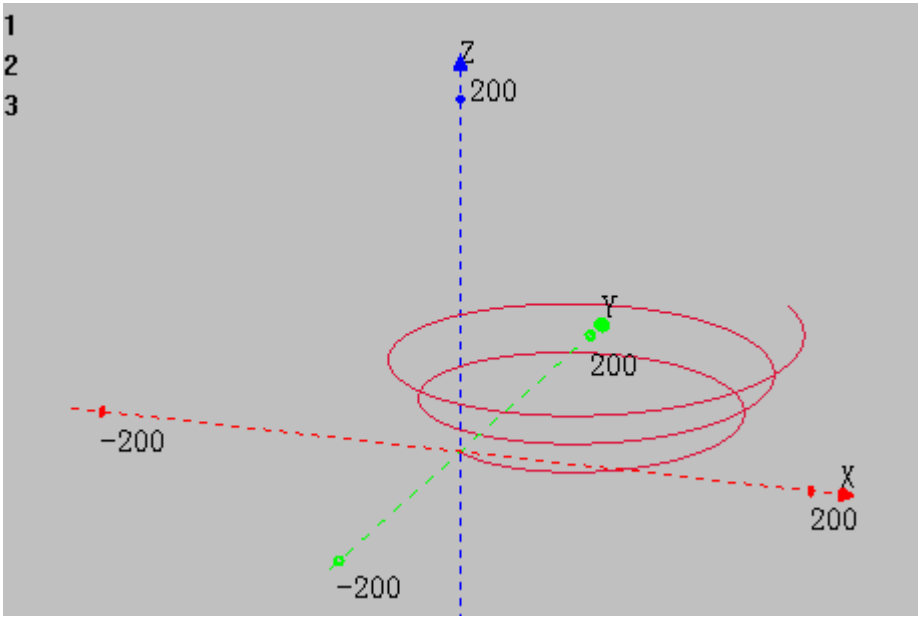


MOVESPIRAL (100,100,-2.5,30) '旋转圈数为负数时(-2.5)，顺时针旋转



例三 螺旋
MOVESPIRAL(100,100,2.5,30,100) '起始半径 100，从原点开始以(100,100)为圆心，逆时针旋转 2.5 圈，每圈向外扩散 30，同时 Z 轴向上运动到 100

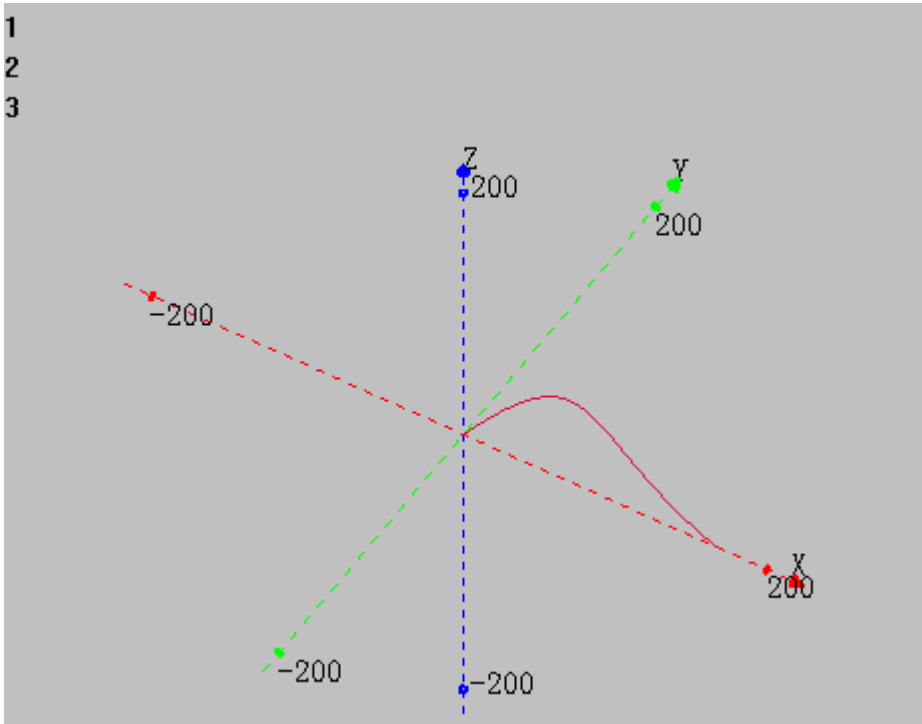
XYZ 模式下轴 0 轴 1 轴 2 插补轨迹



MOVESPLINE / MOVESPLINEABS -- 样条插补

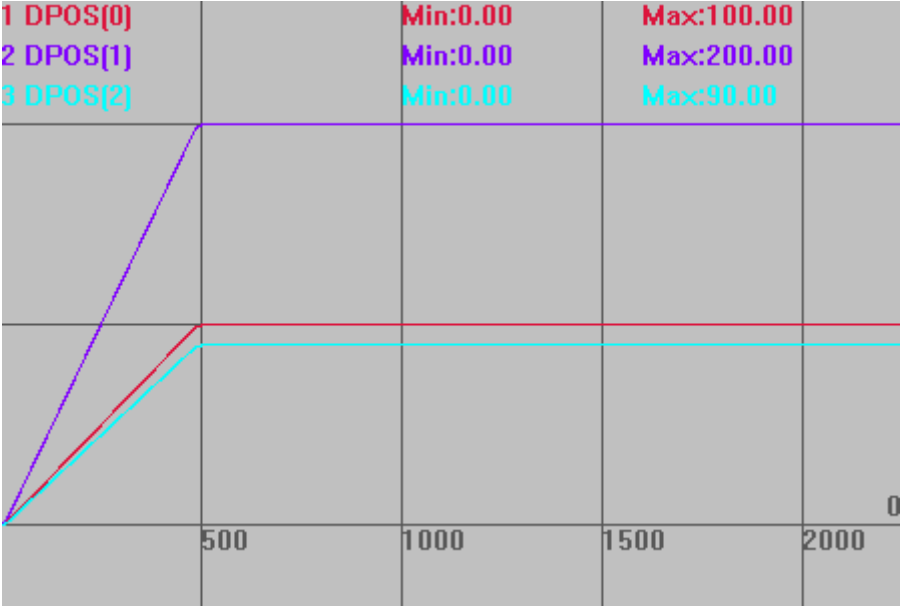
类型	特殊运动指令
描述	样条插补运动，相对/绝对运动。 样条运动的控制点位置提前写入 TABLE 数组。 此运动没有 SP 指令，自定义速度的连续插补运动通过 CORNER_MODE 的 BIT8 来设置。
语法	MOVESPLINE (axes,mode ,dtendcontrol4, dtcontrol2, dtcontrol3) axes: 插补的轴个数

	mode: 模式, 0-使用 3 阶贝塞尔样条 dtendcontrol4: 第 4 个控制点存储 table 索引, 对贝塞尔曲线就是终点 dtcontrol2: 第 2 个控制点存储 table 索引 dtcontrol3: 第 3 个控制点存储 table 索引 对贝塞尔曲线, 第 1 个控制点就是当前位置。
适用控制器	4 系列 170507 以上固件支持, 306x 系列 161208 以上固件支持
例子	例一 BASE(0,1) DPOS=0,0 ATYPE=1,1 '设为脉冲轴类型 SPEED=100,100 '主轴速度 ACCEL=1000,1000 '主轴加速度 DECEL=1000,1000 TRIGGER CORNER_MODE=2 + 256 '设置 SP 运动, 使用 FORCE_SPEED FORCE_SPEED=100 TABLE(0, 100, 100) 'TABLE(0)(1)存储第二个控制点, 相对起点距离 TABLE(10, 200, -100) 'TABLE(10)(11)存储第三个控制点, 相对起点距离 TABLE(20, 300, 0) 'TABLE(20)(21)存储第四个控制点, 终点距离 MOVESPLINE(2, 0, 20, 0, 10) '2 轴相对样条插补 插补轨迹 DPOS(0)垂直刻度 200 DPOS(1)垂直刻度 200 1 DPOS[0] 2 DPOS[1] Min:0.00 Max:300.00 Min:28.86 Max:28.86 例二 BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度

	DECEL=1000 ,1000,1000 TRIGGER TABLE(0, 100, 100,100) 'TABLE(0)(1)存储第二个控制点, 相对起点距离 TABLE(10, 200, -100,200) 'TABLE(10)(11)存储第三个控制点, 相对起点距离 TABLE(20, 300, 0,0) 'TABLE(20)(21)存储第四个控制点, 终点距离 MOVESPLINE(3, 0, 20, 0, 10) '2 轴相对样条插补 XYZ 模式下轴 0 轴 1 轴 2 插补轨迹 
相关指令	CORNER_MODE

MOVE_TURNABS -- 旋转台插补

类型	多轴运动指令
描述	旋转的插补运动，保证在旋转台上是直线运动。 旋转功能是指工作平台在与 XY 平行的平面上旋转，旋转的正向与 XY 的正向要一致（右手法则）。 旋转的参数存储在 TABLE 表里面，依次存储 R 轴号，R 轴一圈脉冲数，X 轴号，Y 轴号，X 圆心，Y 圆心。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 建议直接使用机械手的旋转台算法，详情查看《正运动机械手指令说明》的 frame11/17。
语法	MOVE_TURNABS(tablenum,position1[,position2[,position3[,position4...]]]) tablenum: 存储旋转参数的 table 编号

	position1: 第一个轴运动坐标 position2: 下一个轴运动坐标
适用控制器	通用
例子	BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 '主轴减速度 TABLE(0, 3, 3600, 0,1, 0,0) '设置旋转台的参数 TRIGGER MOVE_TURNABS(0,100,200,90) '直线插补至目标位置 WAIT IDLE '等待运动停止 插补轨迹 DPOS(0)垂直刻度 100 DPOS(1)垂直刻度 100 DPOS(2)垂直刻度 100 
相关指令	MCIRC_TURNABS

MCIRC_TURNABS -- 旋转台圆弧插补

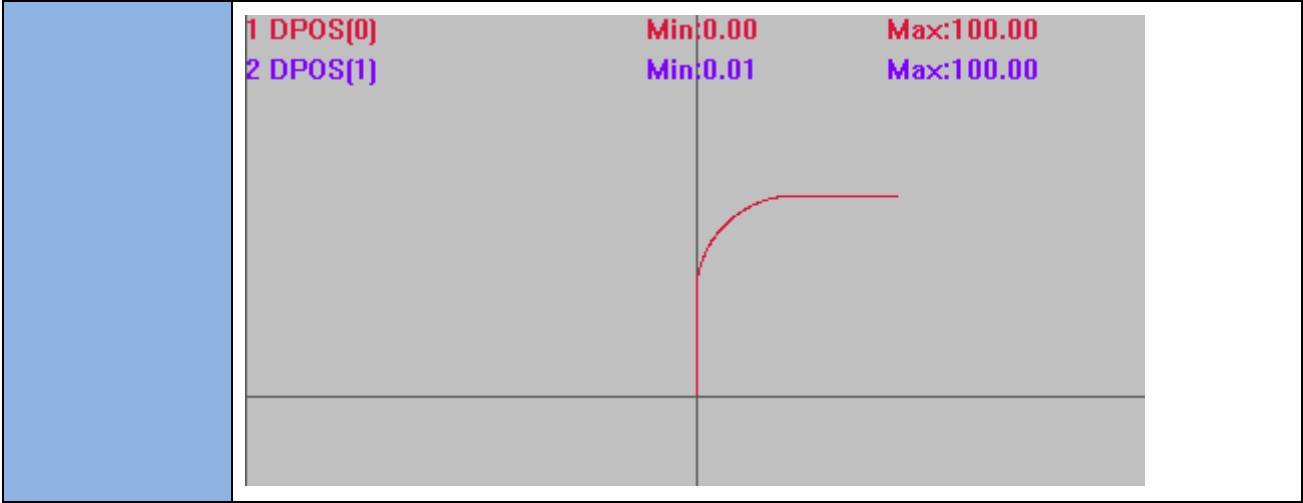
类型	多轴运动指令
描述	旋转的插补运动，保证在旋转台上是圆弧运动。 旋转功能是指工作平台在与 XY 平行的平面上旋转，旋转的正向与 XY 的正向要一致（右手法则）。

	旋转的参数存储在 TABLE 表里面，依次存储 R 轴号，R 轴一圈脉冲数，X 轴号，Y 轴号，X 圆心，Y 圆心。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。															
语法	MCIRC_TURNABS(tablenum, refpos1, refpos2, mode,end1, end2 [, dis3, dis4, dis5]) tablenum: 存储旋转参数的 table 编号 refpos1: 第一个轴参考点，绝对位置 refpos2: 第二个轴参考点，绝对位置 mode: 1-参考点是当前点前面，2-参考点是结束点后面，3-参考点在中间，采用三点定圆的方式 end1: 第一个轴结束点，绝对位置 end2: 第二个轴结束点，绝对位置 dis3: 螺旋轴的结束位置															
适用控制器	通用															
例子	BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 '主轴减速度 TABLE(0, 3, 3600, 0,1, 0,0) '设置旋转台的参数 TRIGGER TURN_POSMAKE(0,100,200,5,10) MCIRC_TURNABS(0,TABLE(10),TABLE(11),3,200,300,10) '圆弧的同时 3 轴旋转 WAIT IDLE 插补轨迹 DPOS(0)垂直刻度 200 DPOS(1)垂直刻度 200 DPOS(2)垂直刻度 200 <table><tr><td>1 DPOS[0]</td><td></td><td>Min:0.00</td><td>Max:200.00</td><td></td></tr><tr><td>2 DPOS[1]</td><td></td><td>Min:0.01</td><td>Max:300.00</td><td></td></tr><tr><td>3 DPOS[2]</td><td></td><td>Min:0.00</td><td>Max:10.00</td><td></td></tr></table>	1 DPOS[0]		Min:0.00	Max:200.00		2 DPOS[1]		Min:0.01	Max:300.00		3 DPOS[2]		Min:0.00	Max:10.00	
1 DPOS[0]		Min:0.00	Max:200.00													
2 DPOS[1]		Min:0.01	Max:300.00													
3 DPOS[2]		Min:0.00	Max:10.00													

相关指令	MOVE_TURNABS , TURN_POSMAKE
------	---

MOVESMOOTH -- 倒圆角

类型	多轴运动指令
描述	空间直线倒圆运动。 根据下一个直线运动的绝对坐标在拐角自动插入圆弧，加入圆弧后会使得运动的终点与直线的终点不一致，拐角过大时不会插入圆弧，当距离不够时会自动减小半径。 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。 此指令为早期开发指令，有轴数限制，建议使用 CORNER_MODE 指令，功能更多。
语法	MOVESMOOTH (end1, end2, end3, next1, next2, next3, radius) end1: 第 1 个轴运动绝对坐标 end2: 第 2 个轴运动绝对坐标 end3: 第 3 个轴运动绝对坐标 next1: 第 1 个轴下一个直线运动绝对坐标 next2: 第 2 个轴下一个直线运动绝对坐标 next3: 第 3 个轴下一个直线运动绝对坐标 radius: 插入圆弧的半径，当过大的时候自动缩小
适用控制器	通用
例子	<pre> BASE(0,1,2) ATYPE=1,1,1 '设为脉冲轴类型 UNITS=100,100,100 DPOS=0,0,0 SPEED=100,100,100 '主轴速度 ACCEL=1000 ,1000,1000 '主轴加速度 DECEL=1000 ,1000,1000 '主轴减速度 TRIGGER '自动触发示波器 MOVESMOOTH (0,100,0,100,100,0,50) '加入圆弧后，实际运动到了'(50,100,0) MOVEABS(100,100,0) 插补轨迹 DPOS(0)垂直刻度 100 DPOS(1)垂直刻度 100 </pre>



MOVESP -- SP 运动

类型	多轴运动指令
描述	直线插补运动，相对运动一段距离，使用 SP 限速 自定义速度的连续插补运动可以使用 SP 后缀的指令，见*SP 描述。
语法	MOVESP(distance1 [,distance2 [,distance3 [,distance4...]]]) distance1: 第一个轴运动距离 distance2: 下一个轴运动距离
适用控制器	通用
例子	BASE(0) DPOS=0 ATYPE=1 UNITS=100 ACCEL=1000 DECEL=1000 SRAMP=100 MERGE=ON '打开连续插补 SPEED=100 '运行速度 100 FORCE_SPEED=80 '限制速度 80 STARTMOVE_SPEED=60 '起始速度 60 ENDMOVE_SPEED=30 '终止速度 30 TRIGGER '自动触发示波器 MOVE(100) '运动 A，不使用 SP 限速 MOVESP(100) '运动 B，使用 SP 限速 FORCE_SPEED=120 '限制速度 120 ENDMOVE_SPEED=30 '终止速度 30 MOVESP(100) '运动 C，使用 SP 限速 速度轨迹 MSPEED(0)垂直刻度 100

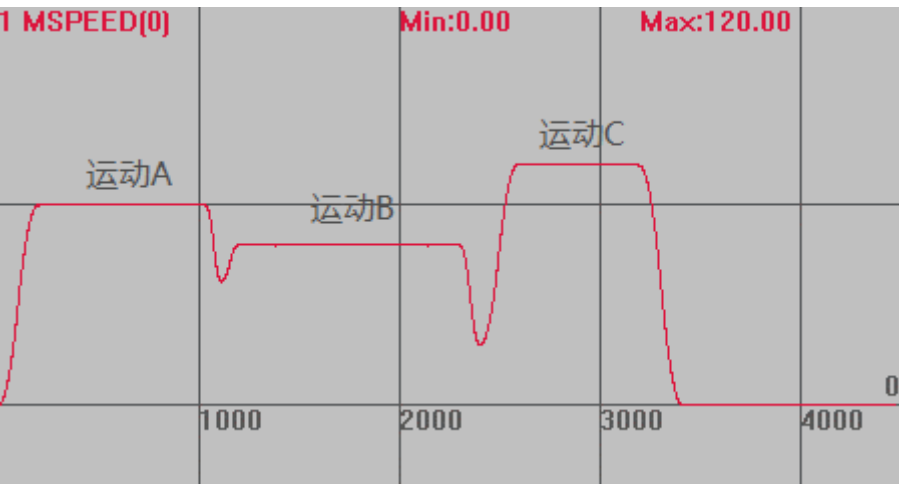
	1 MSPEED[0] Min:0.00 Max:120.00 运动A 运动B 运动C 1000 2000 3000 4000 0 不使用 SP 指令时，运行速度为 SPEED，使用 SP 指令后，运行速度为 FORCE_SPEED。当 STARTMOVE_SPEED 和 ENDMOVE_SPEED 同时设置时，STARTMOVE_SPEED 生效。
相关指令	*SP

*SP -- 运动单独速度

类型	多轴运动指令
描述	用于设置每段运动的运行速度、起始速度和终止速度。 多轴运动指令有对应 SP 运动指令，此时可以使用轴参数 FORCE_SPEED 、STARTMOVE_SPEED 和 ENDMOVE_SPEED 来设置每个运动的速度、起始速度和终止速度，当不需要设置每个运动速度时，不需要使用 SP 指令。
语法	SP 指令有：MOVESP，MOVEABSSP，MOVECIRCSP, MOVECIRCABSSP，MHELICALSP，MHELICALABSSP，MECLIPSESP，MECLIPSEABSSP，MSPHERICALSP。 FORCE_SPEED 、ENDMOVE_SPEED 和 STARTMOVE_SPEED 会随 SP 运动指令写入运动缓存区。
适用控制器	通用
例子	例一 BASE(0) DPOS=0 ATYPE=1 UNITS=100 ACCEL=1000 DECEL=1000 SRAMP=100 MERGE=ON SPEED=100 FORCE_SPEED=80 '打开连续插补 '运行速度 100 '限制速度 80

STARTMOVE_SPEED=60 '起始速度 60
ENDMOVE_SPEED=30 '终止速度 30
TRIGGER '自动触发示波器
MOVE(100) '运动 A，不使用 SP 限速
MOVESP(100) '运动 B，使用 SP 限速
FORCE_SPEED=120 '限制速度 120
ENDMOVE_SPEED=30 '终止速度 30
MOVESP(100) '运动 C，使用 SP 限速

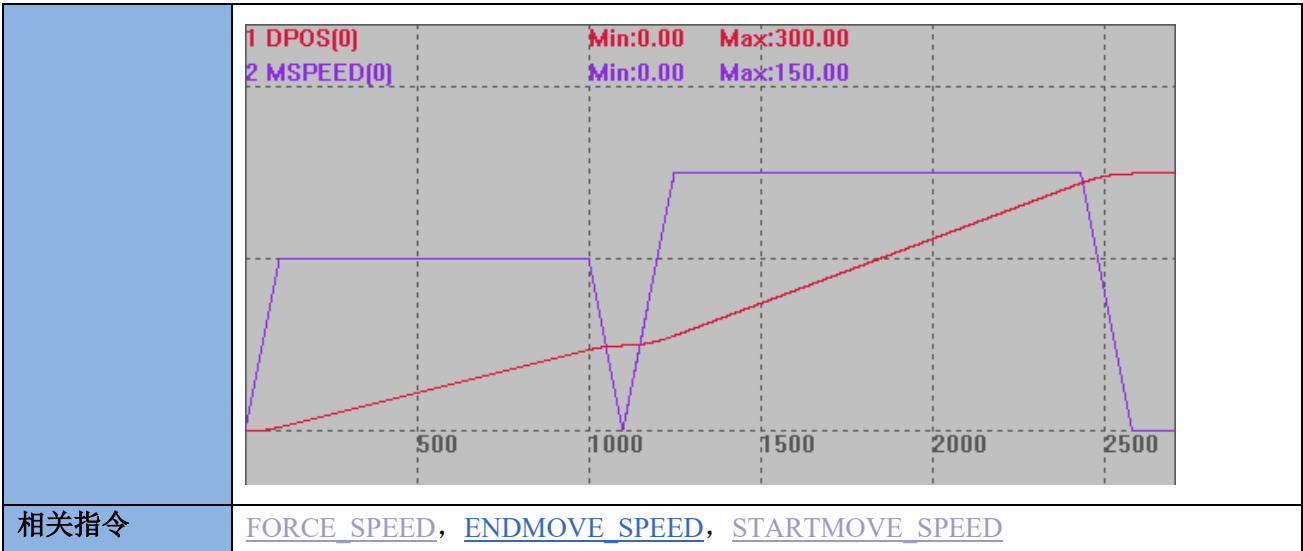
速度轨迹
MSPEED(0)垂直刻度 100



不使用 SP 指令时，运行速度为 SPEED，使用 SP 指令后，运行速度为 FORCE_SPEED。
当 STARTMOVE_SPEED 和 ENDMOVE_SPEED 同时设置时，STARTMOVE_SPEED 生效。

例二
BASE(0)
DPOS=0
ATYPE=1
UNITS=100
ACCEL=1000
DECEL=1000
SPEED=100 '运行速度 100
SRAMP=100 's 曲线
FORCE_SPEED=150 '限制速度 120
TRIGGER
MOVE(100) '运动速度 SPEED
MOVESP(200) '运动速度 FORCE_SPEED

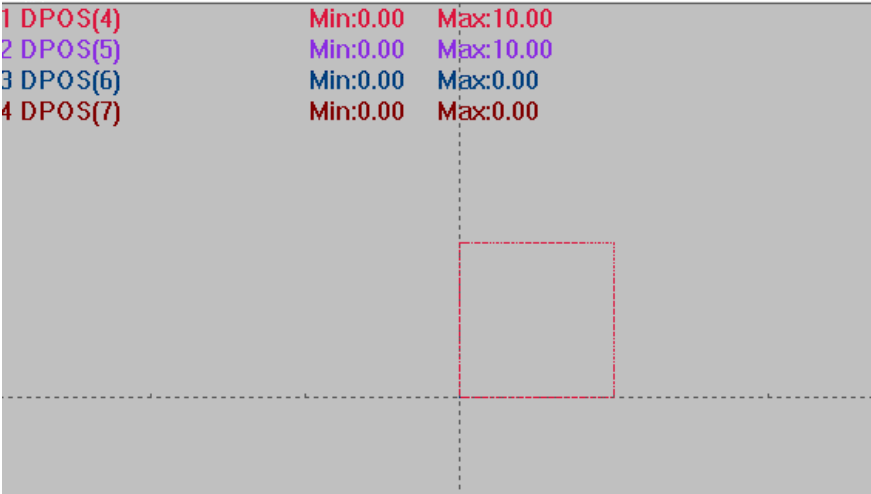
运动轨迹
DPOS(0)垂直刻度 200
MSPEED(0)垂直刻度 100



[FORCE_SPEED](#), [ENDMOVE_SPEED](#), [STARTMOVE_SPEED](#)

MOVESCAN -- 振镜运动

类型	运动指令
描述	不带加减速的运动指令，支持 us 级别的时间控制。 使用 FORCE_SPEED 与矢量距离直接计算出运行时间，例如振镜矢量距离 1，FORCE_SPEED=10000，运动时间为 1/10000，单位 s，即 100us。 支持 MOVESCANABS 绝对运动。 需要振镜控制器固件版本 20180714 以上。 拐角延时、ZSMOOTH 在此运动下意义为最大的拐角延时、实际的延时时间在 DECEL_ANGLE 与 STOP_ANGLE 之间线性分布。 CORNER_MODE 的 bit1 位设置是否使用拐角延时，开启之后 ZSMOOTH 设置要延时的最大时间，微秒单位，运动符合拐角条件延时。 可以与 MOVE_WAIT、MOVE_OP 一起实现 us 级别的时间控制。 非振镜轴也可以使用，但要自己分段控制速度来做加减速。
语法	MOVESCAN(pos1[,pos2] [,pos3]...) pos1: 第一个轴运动距离 pos2: 下一个轴运动距离
适用控制器	振镜控制器
例子	例一 BASE(4,5) AXIS_ZSET=2 '开启精准输出 TRIGGER CORNER_MODE=0 '无拐角延时 MOVE_PAUSE(3) '强制暂停 MOVE_OP(0,1) FORCE_SPEED=10000 MOVESCANABS(0,0) MOVESCANABS(10,0) '振镜运动，时间 10/10000=1000us

	MOVESCANABS(10,10) '振镜运动 MOVESCANABS(0,10) '振镜运动 MOVESCANABS(0,0) 振镜运动 MOVE_DELAY(0.25) '延时 250us MOVE_OP(0,0) '输出 MOVE_RESUME END 振镜轴 XY 模式下合成轨迹如下 DPOS(4)垂直刻度 10 DPOS(5)垂直刻度 10 1 DPOS(4) 2 DPOS(5) 3 DPOS(6) 4 DPOS(7) Min:0.00 Min:0.00 Min:0.00 Min:0.00 Max:10.00 Max:10.00 Max:0.00 Max:0.00 
--	---

例二

BASE(4,5)

AXIS_ZSET=2

CORNER_MODE=2 '拐角延时

ZSMOOTH=100 '拐角最大延时 100u

DECEL_ANGLE = 25 * (PI/180) '设置开始减速拐角，弧度

STOP_ANGLE = 90 * (PI/180) '设置结束减速拐角，弧度

MOVE_PAUSE(3)

MOVE_OP(0,1)

FORCE_SPEED=10000

MOVESCAN(1,0) '运动 100us 的时间

MOVESCAN(0,1) '加 100us 拐角延时，再运动 100us 的时间

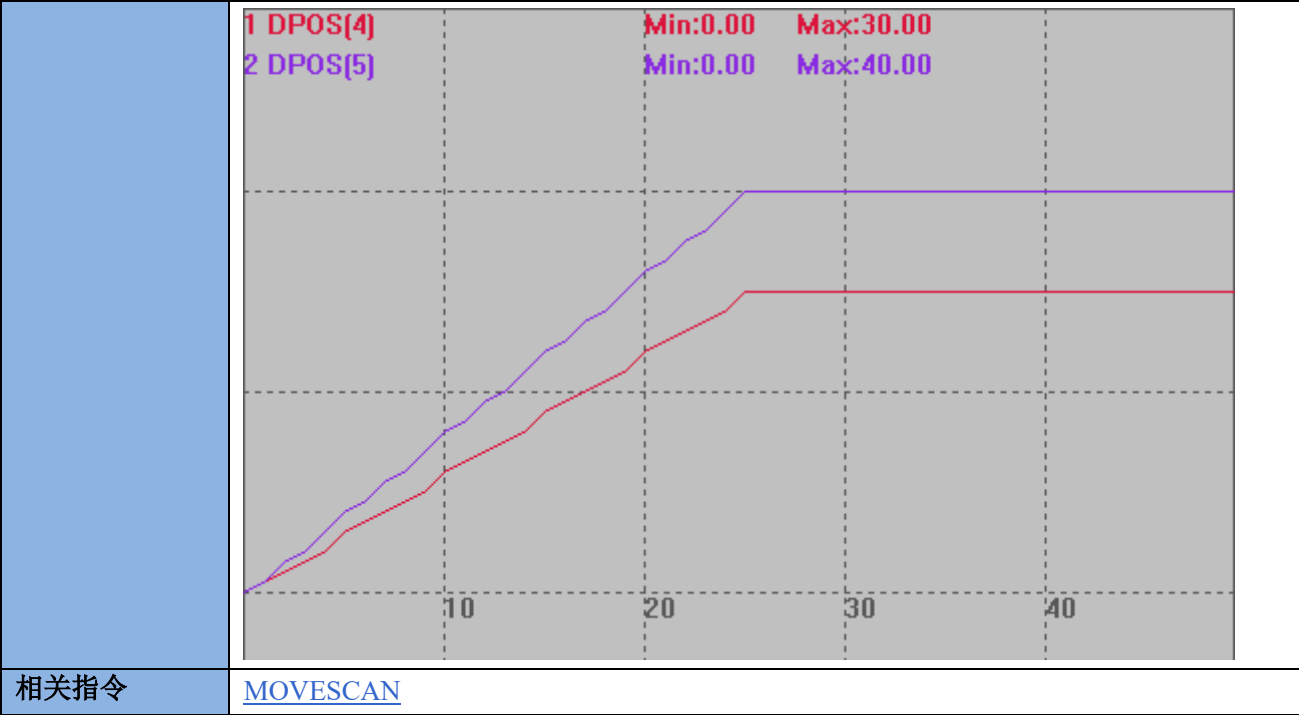
MOVE_DELAY(0.25)

MOVE_OP(0,0) '550us 以后输出

MOVE_RESUME

MPULSCAN -- 振镜运动 2

类型	运动指令
描述	不带加减速的运动指令，单位为脉冲个数。 使用 FORCE_SPEED 与矢量距离直接计算出运行时间，使用 FORCE_SPEED 与矢量距离直接计算出运行时间，例如振镜矢量距离 1，FORCE_SPEED=10000，运动时间为 1/10000，单位 s，即 100us。 支持 MPULSCANABS 绝对运动。 可以与 MOVE_WAIT、MOVE_OP 一起实现 us 级别的时间控制。 非振镜轴类型也可以使用，但要自己分段控制速度来做加减速。 这个指令没有拐角减速，必须增加 MOVE_DELAY 来实现延时减速。 20220225 以上固件支持。
语法	MPULSCAN vectpul, pul1[,pul 2] [,pul 3]... vectpul: 矢量脉冲长度，外面计算出来，减少控制器执行时间 pul1,2,3: 各轴的脉冲距离或长度，直接使脉冲单位，不再进行 UNITS 转换
适用控制器	振镜控制器
例子	<pre> BASE(4,5) ATYPE=21,21 FORCE_SPEED=1000,1000 '振镜运动速度 DPOS=0,0 AXIS_ZSET=2 '开启精准输出 TRIGGER MOVE_OP(0,1) MPULSCANABS 50,30,40 '振镜运动，矢量长度 50 个脉冲，轴 4 运动 30，周五运动 40 MOVE_DELAY(0.2) '延时 200us MOVE_OP(0,0) '输出 END </pre> 振镜轴运动轨迹如下 DPOS(4)垂直刻度 20 DPOS(5)垂直刻度 20



[MOVESCAN](#)

7.3 特殊运动指令

MOVE_PAUSE -- 运动暂停

类型	特殊运动指令										
描述	BASE 轴运动暂停。 只有在单轴或多轴插补运动时有效，多轴联动时一起暂停。 可以通过 <code>AXISSTATUS</code> 来查询是否有暂停。 当轴已经暂停或不在运动中时，调用这个指令会有警告输出，但不影响程序运行。某些运动不支持暂停，如 <code>VMOVE</code>、同步运动指令等。										
语法	<code>MOVE_PAUSE (mode)</code> mode: 暂停方式 <table><tr><td>0(缺省)</td><td>暂停当前运动。</td></tr><tr><td>1</td><td>在当前运动完成后正准备执行下一条运动指令时暂停</td></tr><tr><td>2</td><td>在当前运动完成后正准备执行下一条运动指令时，并且两条指令的 <code>MARK</code> 标识不一样时暂停 这个模式可以用于一个动作由多个指令来实现时，可以在一整个动作完成后暂停</td></tr><tr><td>3</td><td>强制暂停，<code>IDLE</code> 模式下也可以进入暂停状态 20170513 固件版本增加此功能</td></tr><tr><td>4</td><td>指定矢量位置暂停 version_build 241025 以上固件增加</td></tr></table>	0(缺省)	暂停当前运动。	1	在当前运动完成后正准备执行下一条运动指令时暂停	2	在当前运动完成后正准备执行下一条运动指令时，并且两条指令的 <code>MARK</code> 标识不一样时暂停 这个模式可以用于一个动作由多个指令来实现时，可以在一整个动作完成后暂停	3	强制暂停， <code>IDLE</code> 模式下也可以进入暂停状态 20170513 固件版本增加此功能	4	指定矢量位置暂停 version_build 241025 以上固件增加
0(缺省)	暂停当前运动。										
1	在当前运动完成后正准备执行下一条运动指令时暂停										
2	在当前运动完成后正准备执行下一条运动指令时，并且两条指令的 <code>MARK</code> 标识不一样时暂停 这个模式可以用于一个动作由多个指令来实现时，可以在一整个动作完成后暂停										
3	强制暂停， <code>IDLE</code> 模式下也可以进入暂停状态 20170513 固件版本增加此功能										
4	指定矢量位置暂停 version_build 241025 以上固件增加										

	MOVE_PAUSE (mode[, markbits]) markbits: mode=2 时,与 MARK 相与来识别前后 MARK 是否不同类型, 32 位整数. 缺省:-1 MOVE_PAUSE(4, vectorpos) vectorpos: 暂停的矢量位置, 不要小于当前位置 vector_moved, 不要大于最大位置 vector_buffered. ! 必须 merge=on 的状态下使用 ! 分离模式和 MOVER, SPLINE 指令下位置不准确, 不建议使用. ! 不能暂停后立即调用 MOVE_PAUSE(4 来指定下一个暂停位置, 因为 move_resume 会清除暂停.
适用控制器	MOVE_PAUSE (mode)——通用 MOVE_PAUSE (mode[, markbits])——4 系以上, version_build 220711 以上固件增加 MOVE_PAUSE(4, vectorpos)——4 系以上, version_build 241025 以上固件增加
例子	BASE(0) DPOS=0 SPEED=100 例一 mode 0 MOVE(1000) '当前运动 MOVEABS(-100) '缓冲运动 MOVE_PAUSE(0) '模式 0, 暂停当前运动 ?DPOS(0) '打印结果, 0, 此时当前运动只运行了极短时间, 扫描到 MOVE_PAUSE 时直接暂停 例二 mode 1 MOVE(1000) '当前运动 MOVEABS(-100) '缓冲运动 MOVE_PAUSE(1) '模式 1, 先完成当前运动再暂停 WAIT IDLE ?DPOS(0) '此时运行完当前运动, 打印 DPOS 为 1000 例三 mode 2 MOVE_MARK=1 '标号手动设为 1 MOVE(200) '当前运动 MOVE_MARK=1 '标号设为与上一个运动一样 MOVEABS(-100) '缓冲运动 MOVEABS(100) '不设置运动标号, 自动加一 MOVE_PAUSE(2) '模式 2, 先完成当前运动, 然后直到下一条运动指令的标号与当前标号不一致时再暂停 DELAY(3000) '等待暂停后 ?DPOS(0) '打印结果, -100, (速度过慢会导致打印时当前运动还在进行, 导致结果大于-100)

	'此时运行完当前运动，下一条运动的标号被手动设置成相同的，继续执行，直到第 3 条运动标号不一致，暂停 例四 mode4 <pre> WHILE 1 MOVE_PAUSE (4, VECTOR_MOVED +100) WAIT UNTIL AXISSTATUS AND \$800000 ? "PAUSED", VECTOR_MOVED MOVE_RESUME DELAY 1 '注意 AXISSTATUS 要隔一个周期才更新，要延时确保不是上个周期的 AXISSTATUS WEND </pre>
相关指令	MOVE_MARK ， MOVE_RESUME ， AXISSTATUS

MOVE_RESUME -- 运动恢复

类型	特殊运动指令
描述	当 BASE 轴暂停时，从暂停处继续运动。 可以通过 AXISSTATUS 来查询是否有暂停。
语法	MOVE_RESUME
适用控制器	通用
例子	<pre> BASE(0) UNITS=100 DPOS=0 SPEED=100 ACCEL=1000 DECEL=1000 MOVE(100) '当前运动 MOVE(100) '缓冲运动 MOVE_PAUSE(1) '当前运动完成后暂停 WA 2000 '等待当前运动完成 ?DPOS(0) '打印结果，100 DELAY(1000) MOVE_RESUME '继续运行 WAIT IDLE ?DPOS(0) '打印结果，200 </pre>
相关指令	MOVE_PAUSE ， AXISSTATUS

MOVE_PT -- 单位时间距离

类型	特殊运动指令
----	--------

描述	在一段时间内驱动电机运动设置的距离。 一般是 PC 每个周期计算好对应的坐标，然后传给控制器。 支持使用 BASE 指定轴。 运动时的速度=(DIS/TICKS)*1000 units/s。 不要在极短时间运动大距离，脉冲频率会过高，电机堵转，可以分解成小段，重复发送。 MOVE_PT 指令新增多周期速度自动均匀功能。
语法	MOVE_PT (ticks, dis1,dis2...) ticks: 时间的伺服周期数，时间=系统周期*ticks dis1: 运动的距离 SERVO_PERIOD 为 1ms 的控制器 TICKS 为 1ms(不同 SERVO_PERIOD 的 TICKS 时间不一样)
适用控制器	通用
例子	例一 BASE(0) UNITS=100 DPOS = 0 SPEED=100 ACCEL=1000 DECEL=1000 TRIGGER '自动触发示波器 FOR i=0 TO 9 MOVE_PT (4, 10) '在 4tick 内，运动 10units，速度=2500 units/s NEXT WAIT IDLE PRINT *DPOS '打印结果，100 运动曲线 DPOS(0)垂直刻度 100 MSPEED(0)垂直刻度 2000

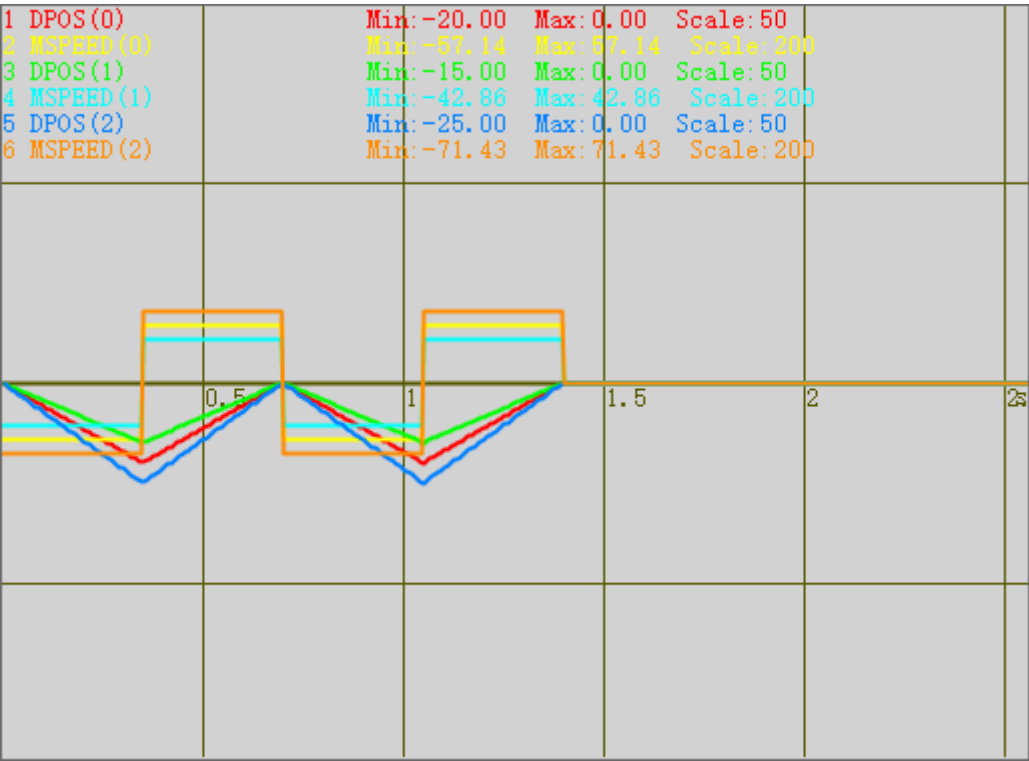
例二

```
BASE(0,1,2)
UNITS = 1000,1000,1000
ATYPE=0,0,0
DPOS = 0,0,0,0
SPEED=10,10,10,10
ACCEL=1000,1000,1000,1000
DECEL=1000,1000,1000,1000
MERGE = ON
TRIGGER
```

```
MOVE_PT(350,-20,-15,-25)'在 350tick 内，轴 0，轴 1，轴 2 运行-20， -15， -25 的距离
MOVE_PT(350,20,15,25)'在 350tick 内，运行 20,15,25 的距离
MOVE_PT(350,-20,-15,-25)
MOVE_PT(350,20,15,25)
WAIT IDLE
```

运动曲线

```
DPOS(0)垂直刻度 50
MSPEED(0)垂直刻度 200
DPOS(1)垂直刻度 50
MSPEED(1)垂直刻度 200
DPOS(2)垂直刻度 50
MSPEED(2)垂直刻度 200
```



例三：

BASE(0)

UNITS = 100

ATYPE=0

DPOS = 0

SPEED=10

ACCEL=100

DECEL=100

DIM timeadd

DIM val1

DIM SPEEDval

timeadd=0

DPOS =100*COS(2*PI*0.2*timeadd+pi)+100

DELAY(1000)

TRIGGER

WHILE TRUE

'周期= $2\pi/|\omega|$

' $x=A*\cos(\omega x+\psi)+C$

'求导得斜率，即速度：

' $v=-A*\omega*\sin(\omega x+\psi)$

val1=100*COS(2*PI*0.2*timeadd+pi)+100

SPEEDval=-100*2*PI*0.2*SIN(2*PI*0.2*timeadd+pi)

? "val1="val1

? "SPEEDval="SPEEDval

MOVE_PVTABS(10,val1,SPEEDval)

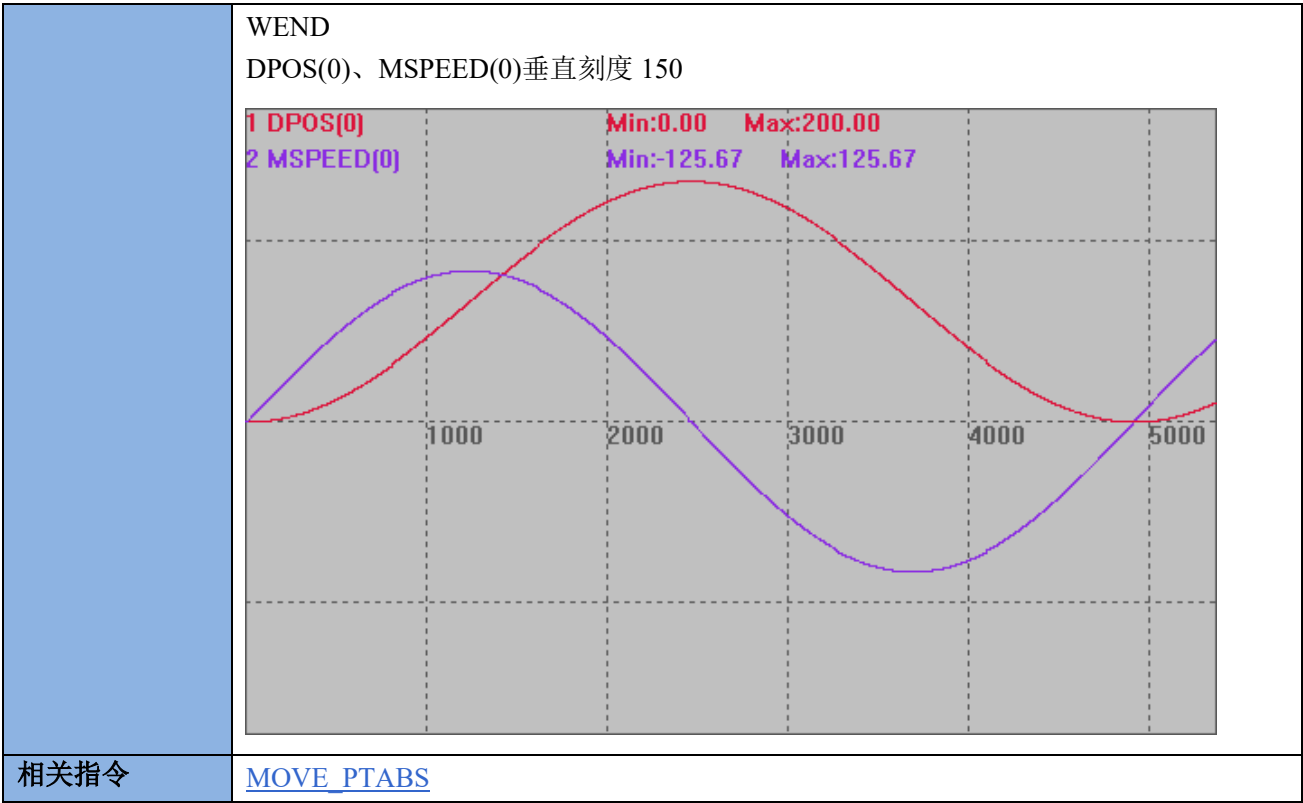
'MOVE_PTABS(10,val1)

timeadd=timeadd+0.01

if timeadd>(2*PI/ABS(2*PI*0.2)*2) THEN

EXIT WHILE

ENDIF



MOVE_PTABS -- 单位时间距离-绝对

类型	特殊运动指令
描述	在一段时间内驱动电机运动到某个位置。 一般是 PC 每个周期计算好对应的坐标，然后传给控制器。 运动时的速度=(DIS/TICKS)*1000 units/s。 不要在极短时间运动大距离，脉冲频率会过高，电机堵转，可以分解成小段，重复发送。
语法	MOVE_PTABS(ticks,dis1,dis2,...) ticks: 时间的伺服周期个数 dis1: 运动的绝对位置
适用控制器	通用
例子	例一 BASE(0,1) DPOS=0,0 MOVE_PTABS (3, 20,20) '3tick 时间内运动到到 20,20 WAIT IDLE PRINT *DPOS '打印结果, 20,20 例二 RAPIDSTOP(2) WAIT IDLE(0)


```

BASE(0)
ATYPE=1
UNITS=100
SPEED=100
ACCEL=1000
DECEL=1000
DPOS = 0

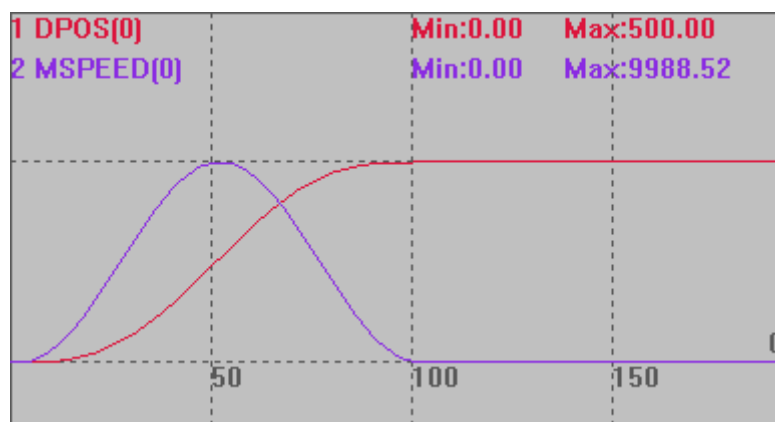
SetSine          '调用函数，产生 SINE 曲线
TRIGGER          '自动触发示波器

FOR i=0 TO 100
    MOVE_PTABS (1, TABLE(i))      '在 1TICK 内，运动 TABLE 距离
NEXT
WAIT IDLE(0)
PRINT DPOS(0)      '打印结果 500
END

GLOBAL SUB SetSine()          '计算小段位移
    LOCAL num_p,scale        '变量定义
    num_p=100
    scale=500
    FOR p=0 TO num_p
        TABLE(p,((-SIN(PI*2*p/num_p)/(PI*2))+p/num_p)*scale) '存储运动参数
    NEXT
END SUB

DPOS(0)竖直刻度 500
MSPEED(0)竖直刻度 10000

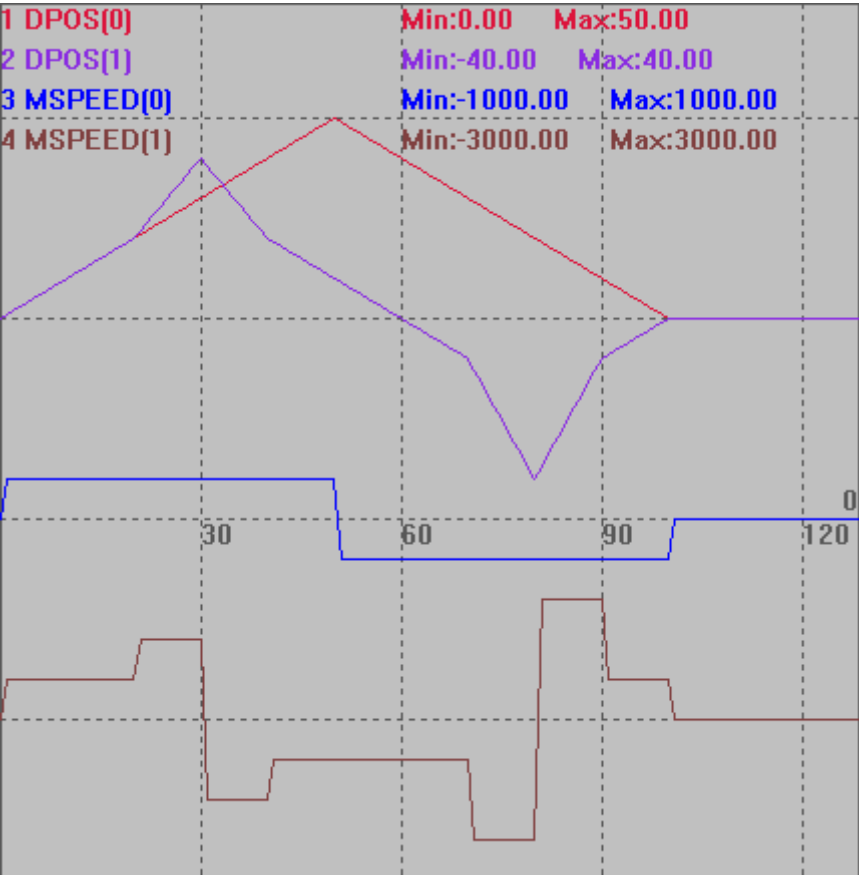
```



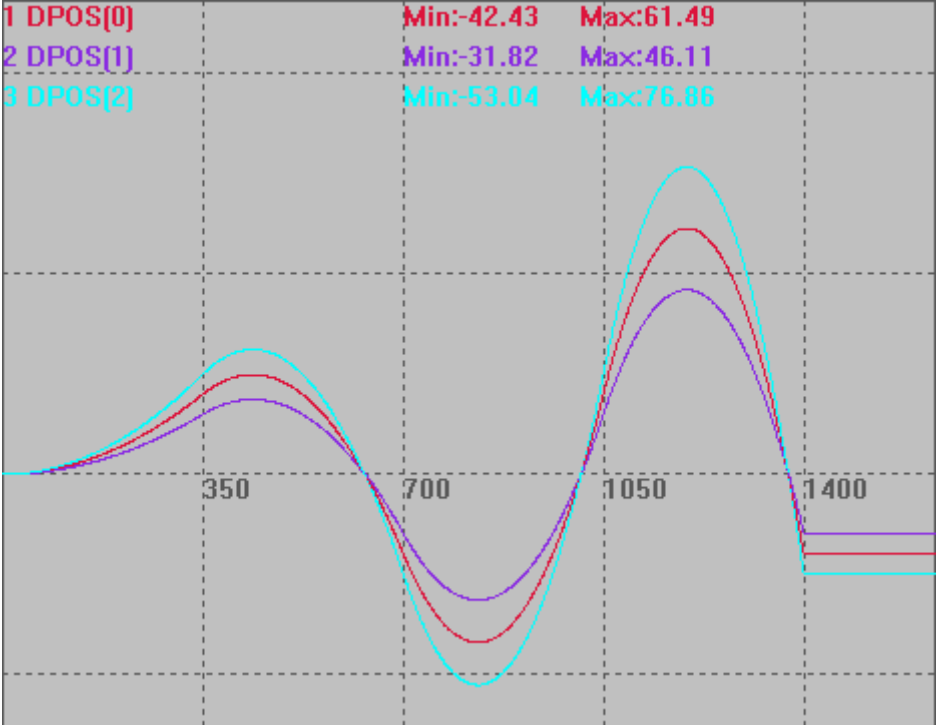
例三
RAPIDSTOP(2)

```
WAIT IDLE(0)
WAIT IDLE(1)
BASE(0,1)
ATYPE=1,1
UNITS=100,100
DPOS=0,0
TRIGGER
MOVE_PTABS (10,10,10)      '10tick 时间内运动到绝对位置(10,10)
MOVE_PTABS (10,20,20)
MOVE_PTABS (10,30,40)
MOVE_PTABS (10,40,20)
MOVE_PTABS (10,50,10)
MOVE_PTABS (10,40,0)
MOVE_PTABS (10,30,-10)
MOVE_PTABS (10,20,-40)
MOVE_PTABS (10,10,-10)
MOVE_PTABS (10,0,0)
END
```

DPOS(0)、DPOS(1) 竖直刻度 50
MSPEED(0)、MSPEED(1) 竖直刻度 5000



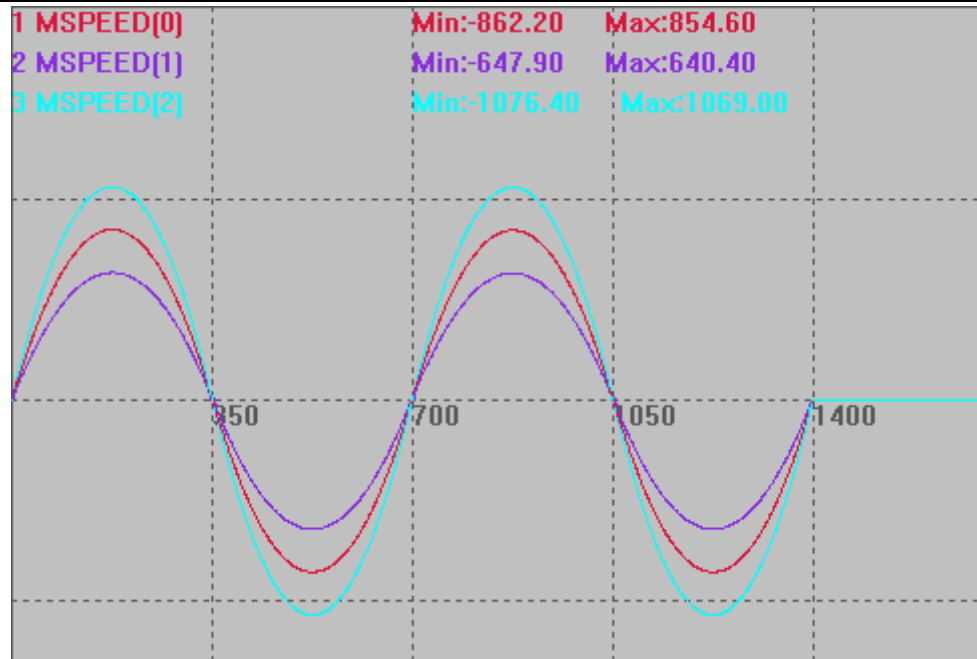
例四

	BASE(0) UNITS = 1000,1000,1000 DPOS = 0,0,0,0 SPEED=10,10,10,10 ACCEL=1000,1000,1000,1000 DECEL=1000,1000,1000,1000 MERGE = ON TRIGGER MOVE_PTABS(350,20,15,25)'在一个周期内，轴 0,1,2 运行到 20,15,25 的位置 MOVE_PTABS(350,-20,-15,-25)'在一个周期内，轴 0,1,2 运行到-20， -15， -25 的位置 MOVE_PTABS(350,20,15,25) MOVE_PTABS(350,-20,-15,-25) WAIT IDLE DPOS(0)、DPOS(1)、DPOS(2)竖直刻度 50 1 DPOS[0] 2 DPOS[1] 3 DPOS[2] Min:-42.43 Min:-31.82 Min:-53.04 Max:61.49 Max:46.11 Max:76.86 
相关指令	MOVE_PT

MOVE_PVT -- 单位时间距离（带速度规划）

类型	特殊运动指令
描述	在一段时间内驱动电机运动设置的距离，带速度规划，可以指定结束速度，小段内速度会自动根据前面的速度与结束速度来自动规划，尽可能连续。 一般是 PC 每个周期计算好对应的坐标，然后传给控制器。 支持使用 BASE 指定轴。 运动时的速度=(DIS/TICKS)*1000 units/s。

	不要在极短时间运动大距离，脉冲频率会过高，电机堵转，可以分解成小段，重复发送。
语法	MOVE_PVT (ticks, dis1, sp1,dis2, sp2 ...) ticks ： 时间的伺服周期数。 dis1: 第一个轴运动的距离 sp1: 第一个轴运动后的结束速度 dis2: 第二个轴的运动距离 sp2: 第二个轴运动后的结束速度 SERVO_PERIOD 为 1000us 的控制器 1 TICKS 为 1ms(不同 SERVO_PERIOD 的 TICKS 时间不一样)
适用控制器	通用
例子	例一 BASE(0,1,2) UNITS = 10,10,10,10 DPOS = 0,0,0,0 SPEED=10,10,10,10 ACCEL=1000,1000,1000,1000 DECEL=1000,1000,1000,1000 MERGE = ON TRIGGER MOVE_PVT(350,200,10,150,10,250,10)'在一个周期内,轴 0,1,2 运行 200,150,250 的距离，结束速度为 10 MOVE_PVT(350,-200,10,-150,10,-250,10)'在一个周期内,轴 0,1,2 运行-200,-150,-250 的距离，结束速度为 10 MOVE_PVT(350,200,10,150,10,250,10) MOVE_PVT(350,-200,10,-150,10,-250,10) DPOS(0)、DPOS(1)、DPOS(2)垂直刻度 200 1 DPOS[0] 2 DPOS[1] 3 DPOS[2] Min:0.00 Min:0.00 Min:0.00 Max:200.00 Max:150.00 Max:250.00 MSPEED(0)、MSPEED (1)、MSPEED (2)垂直刻度 1000



例二：

BASE(0)

UNITS = 100

ATYPE=0

DPOS = 0

SPEED=10

ACCEL=100

DECEL=100

DIM timeadd

DIM val1

DIM SPEEDval

timeadd=0

DPOS = 100 * COS(2 * PI * 0.2 * timeadd + pi) + 100

DELAY(1000)

TRIGGER

WHILE TRUE

'周期 = $2\pi / |\omega|$

' $x = A * \cos(\omega x + \psi) + C$

'求导得斜率，即速度：

' $v = -A * \omega * \sin(\omega x + \psi)$

```
val1=100*COS(2*PI*0.2*timeadd+pi)+100

SPEEDval=-100*2*PI*0.2*SIN(2*PI*0.2*timeadd+pi)

?"val1="val1
?"SPEEDval="SPEEDval

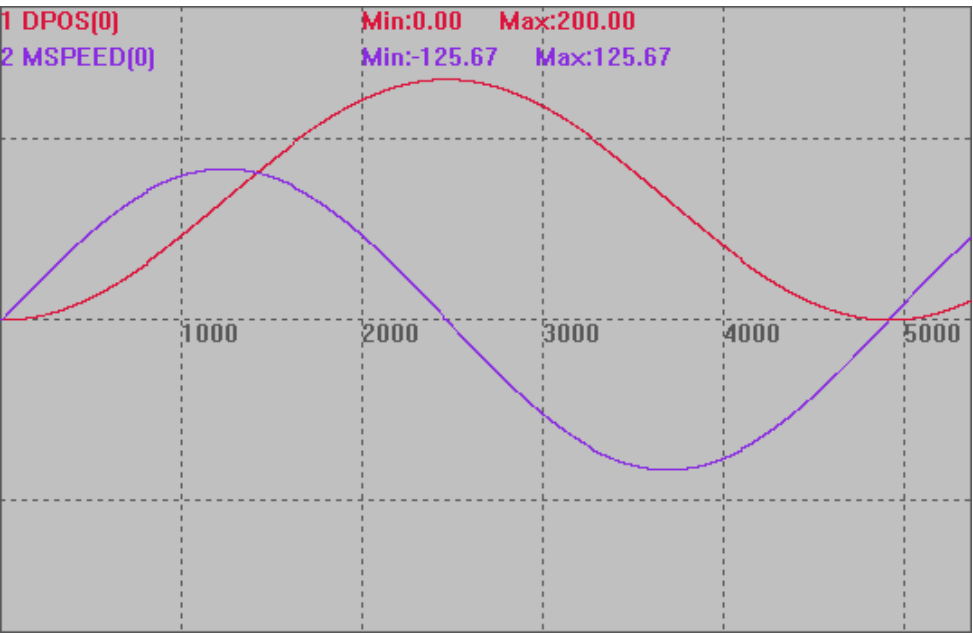
MOVE_PVTABS(10,val1,SPEEDval)

'MOVE_PTABS(10,val1)

timeadd=timeadd+0.01

if timeadd>(2*PI/ABS(2*PI*0.2)*2) THEN
    EXIT WHILE
ENDIF

WEND
DPOS(0)、MSPEED(0)垂直刻度 150
```



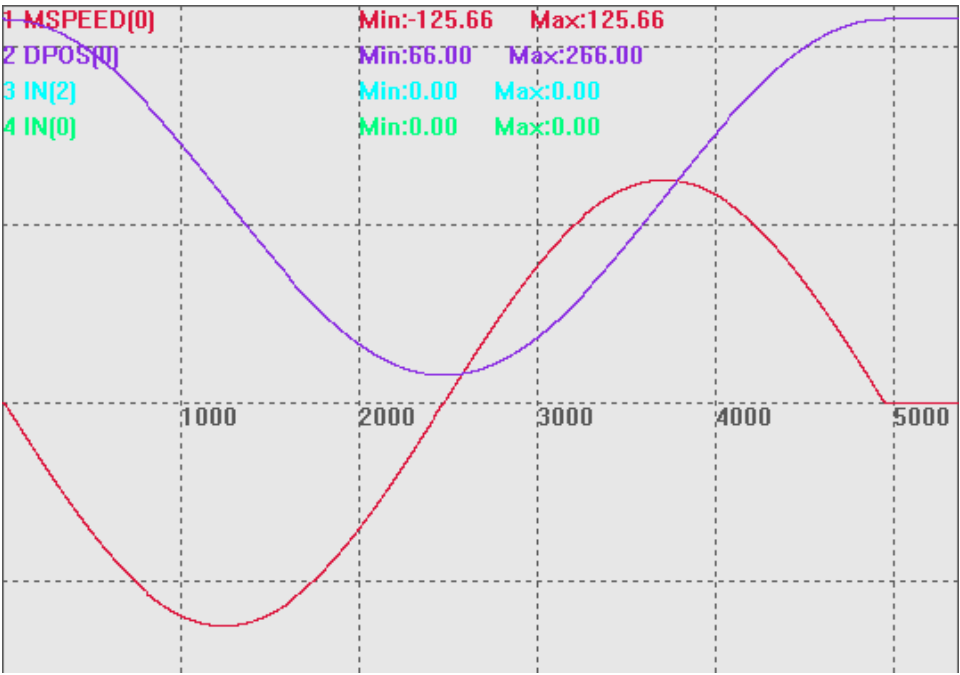
相关指令

[MOVE_PT](#)

MOVE_PVTABS -- 单位时间距离-绝对（带速度规划）

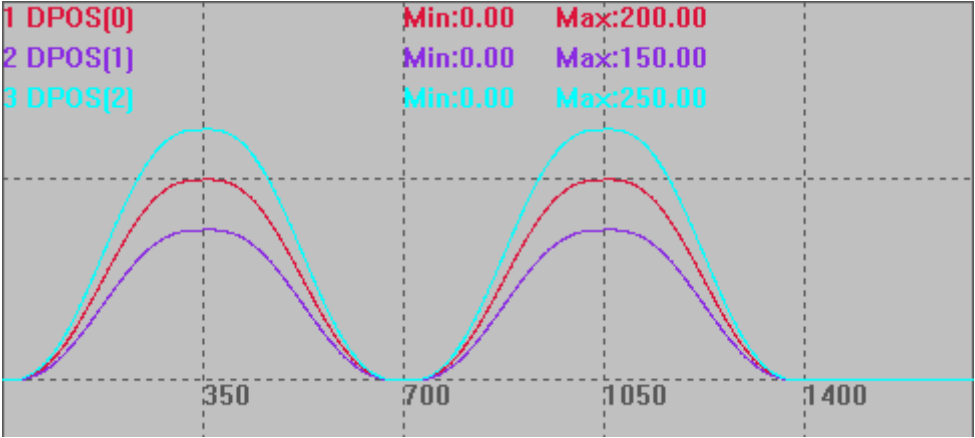
类型	特殊运动指令
描述	在一段时间内驱动电机运动设置的距离，带速度规划，可以指定结束速度，小段内速度会自动根据前面的速度与结束速度来自动规划，尽可能连续。 一般是 PC 每个周期计算好对应的坐标，然后传给控制器。

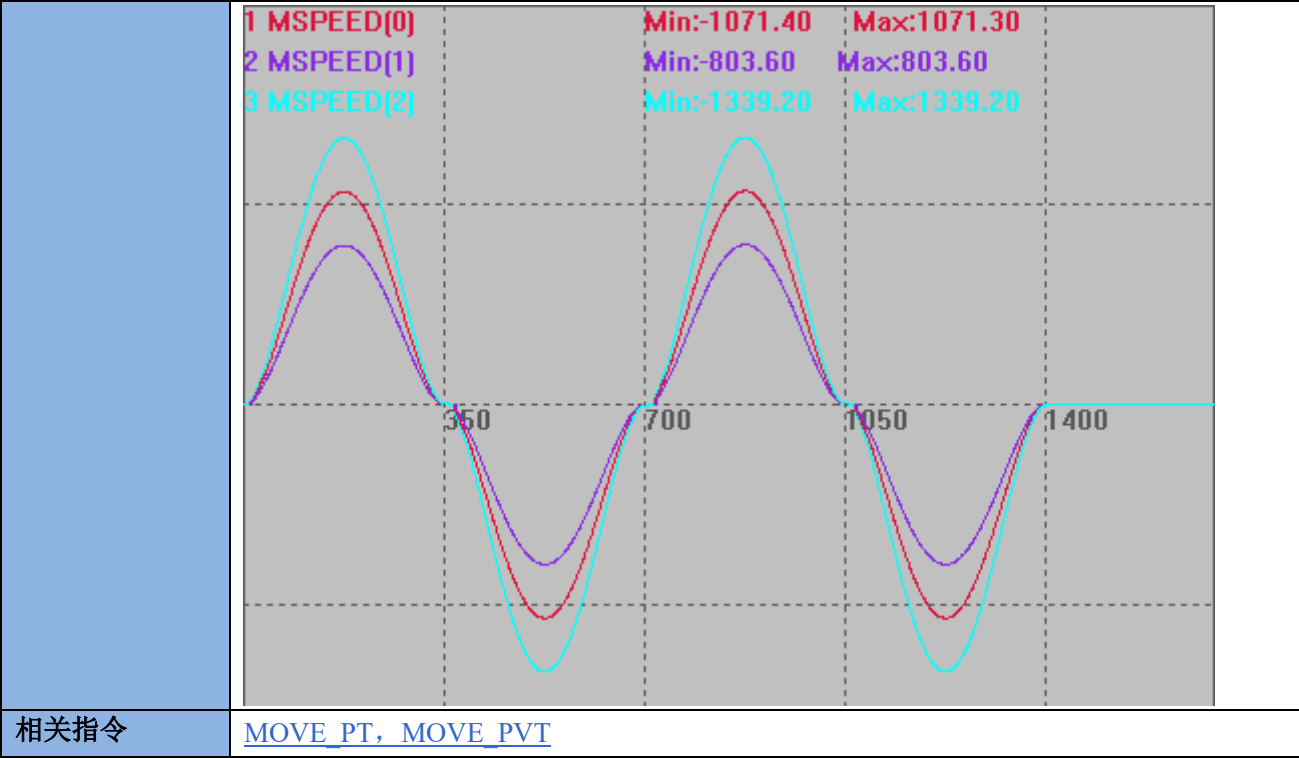
	支持使用 BASE 指定轴。 运动时的速度=(DIS/TICKS)*1000 units/s。 不要在极短时间运动大距离，脉冲频率会过高，电机堵转，可以分解成小段，重复发送。
语法	MOVE_PVTABS (ticks, dis1, sp1,dis2, sp2 ...) ticks : 时间的伺服周期数。 dis1: 第一个轴运动的绝对位置 sp1: 第一个轴运动后的结束速度 dis2: 第二个轴的运动绝对位置 sp2: 第二个轴运动后的结束速度 SERVO_PERIOD 为 1000us 的控制器 1 TICKS 为 1ms(不同 SERVO_PERIOD 的 TICKS 时间不一样)
适用控制器	通用
例子	例一 BASE(0) UNITS = 13107.2 DPOS = 0 SPEED=10 ACCEL=100 DECEL=100 MAX_SPEED=8000000 DIM timeadd DIM val1 DIM SPEEDval timeadd=0 DPOS =100*COS(2*PI*timeadd*0.2)+166 DELAY(1000) TRIGGER WHILE TRUE '周期=$2\pi/ \omega$ 'x=A*COS ($\omega x + \psi$) +C '求导得斜率，即速度： 'v=-A*ω*SIN ($\omega x + \psi$) val1=100*COS(2*PI*timeadd*0.2)+166 SPEEDval=-100*2*PI*0.2*SIN(2*PI*timeadd*0.2)

	<pre>?"val1="val1 ?"SPEEDval="SPEEDval MOVE_PVTABS(10,val1,SPEEDval) 'MOVE_PTABS(10,val1) timeadd=timeadd+0.01 if timeadd>(2*PI/ABS(2*PI*0.2)) THEN EXIT WHILE ENDIF WEND MSPEED(0)无偏移，垂直刻度 100 DPOS(0)偏移-50，垂直刻度 100</pre> 
相关指令	MOVE_PTABS

MOVE_PVTPP -- 单位时间距离

类型	特殊运动指令
描述	同 MOVE_PVT，在一段时间内驱动电机运动设置的距离，带速度规划，可以规划速度保证起始和结束时刻的加速度为 0，可以用来作为指定时间的点位运动，MOVE_PVT 只能用于连续的小段发送，MOVE_PVTPP 可以用于长距离运动。 一般是 PC 每个周期计算好对应的坐标，然后传给控制器。 支持使用 BASE 指定轴。 运动时的速度=(DIS/TICKS)*1000 units/s。 不要在极短时间运动大距离，脉冲频率会过高，电机堵转，可以分解成小段，重复发送。
语法	MOVE_PVTPP (ticks, dis1, sp1,dis2, sp2 ...)

	ticks : 时间的伺服周期数。 dis1: 第一个轴运动的距离 sp1: 第一个轴运动后的结束速度 dis2: 第二个轴运动的距离 sp2: 第二个轴运动后的结束速度 SERVO_PERIOD 为 1000us 的控制器 1 TICKS 为 1ms(不同 SERVO_PERIOD 的 TICKS 时间不一样)
适用控制器	通用
例子	例一 BASE(0,1,2) UNITS = 10,10,10,10 DPOS = 0,0,0,0 SPEED=10,10,10,10 ACCEL=1000,1000,1000,1000 DECEL=1000,1000,1000,1000 MERGE = ON TRIGGER MOVE_PVTPP(350,200,10,150,10,250,10)'在一个周期内，轴 0,1,2 运动 200,150,250 的距离，结束速度为 10. MOVE_PVTPP(350,-200,-10,-150,-10,-250,-10)'在一个周期内，轴 0,1,2 运动-200,-150,-250 的距离，结束速度为 10. MOVE_PVTPP(350,200,10,150,10,250,10) MOVE_PVTPP(350,-200,-10,-150,-10,-250,-10) WAIT IDLE DPOS(0)、DPOS(1)、DPOS(2)垂直刻度 200 1 DPOS[0] 2 DPOS[1] 3 DPOS[2] Min:0.00 Min:0.00 Min:0.00 Max:200.00 Max:150.00 Max:250.00  MSPEED(0)、MSPEED (1)、MSPEED (2)垂直刻度 1000

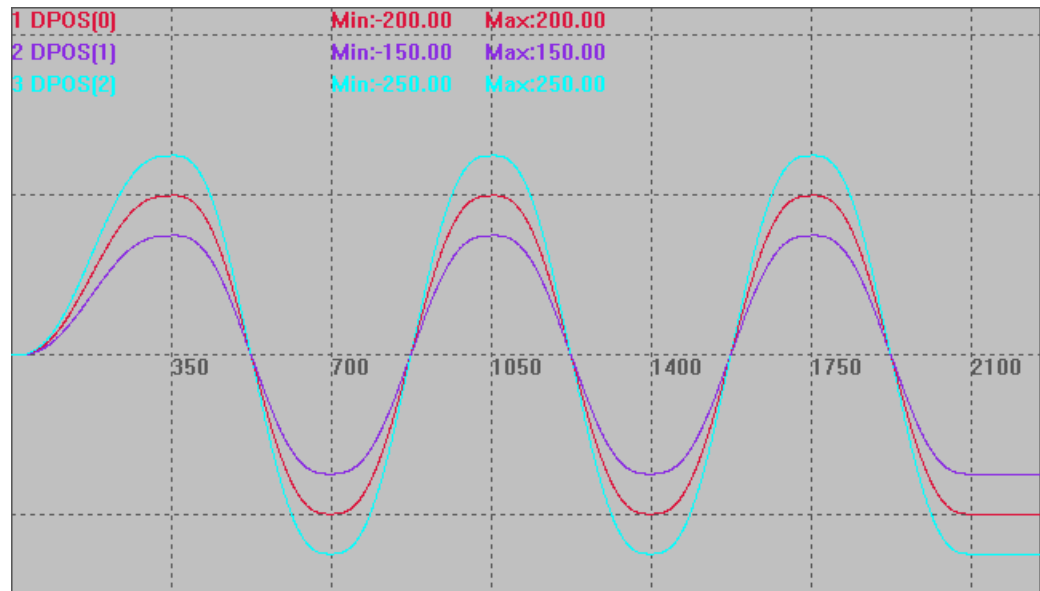


[MOVE_PT](#), [MOVE_PVT](#)

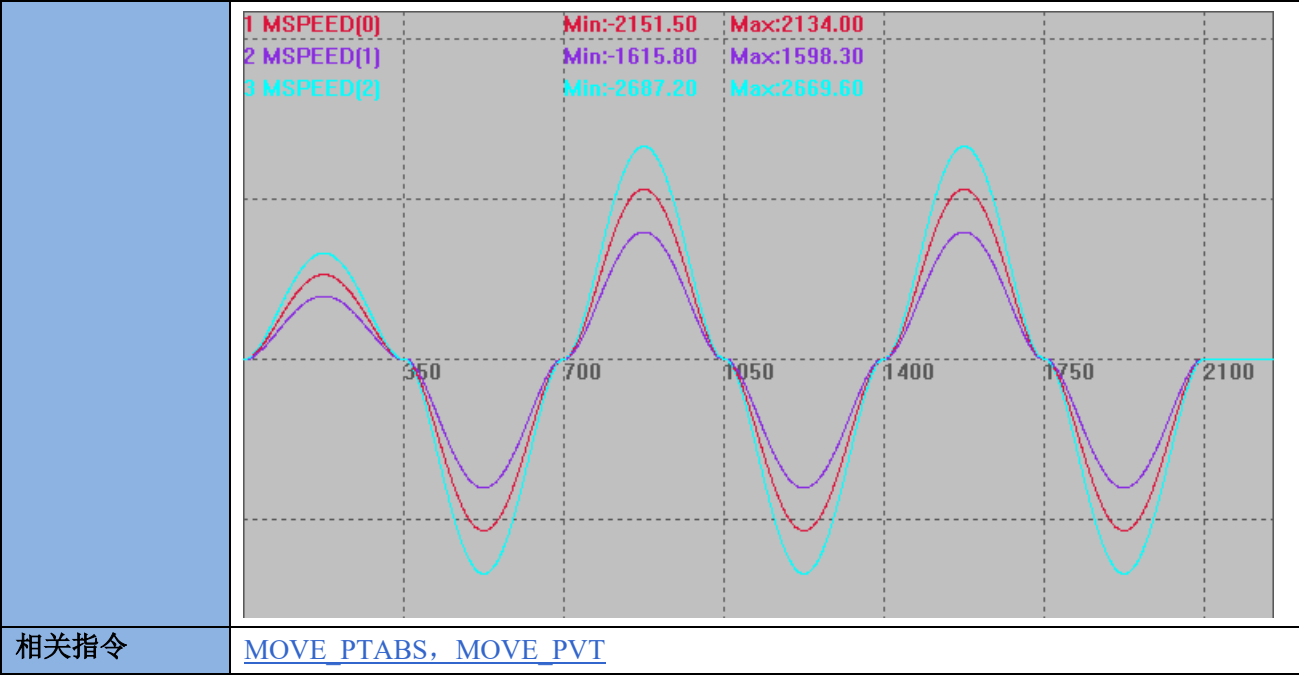
MOVE_PVTPPABS -- 单位时间距离-绝对

类型	特殊运动指令
描述	同 <code>MOVE_PVT</code>，在一段时间内驱动电机运动设置的距离，带速度规划，可以规划速度保证起始和结束时刻的加速度为 0，可以用来作为指定时间的点位运动，<code>MOVE_PVT</code> 只能用于连续的小段发送，<code>MOVE_PVTPP</code> 可以用于长距离运动。 一般是 PC 每个周期计算好对应的坐标，然后传给控制器。 支持使用 <code>BASE</code> 指定轴。 运动时的速度=$(DIS/TICKS)*1000$ units/s。 不要在极短时间运动大距离，脉冲频率会过高，电机堵转，可以分解成小段，重复发送。
语法	<code>MOVE_PVTPPABS (ticks, dis1, sp1,dis2, sp2 ...)</code> ticks : 时间的伺服周期数。 dis1: 第一个轴运动的绝对位置 sp1: 第一个轴运动后的结束速度 dis2: 第二个轴运动的绝对位置 sp2: 第二个轴运动后的结束速度 SERVO_PERIOD 为 1000us 的控制器 1 TICKS 为 1ms(不同 SERVO_PERIOD 的 TICKS 时间不一样)
适用控制器	通用
例子	例一 <code>BASE(0,1,2)</code> <code>UNITS = 10,10,10,10</code>

DPOS = 0,0,0
SPEED=10,10,10,10
ACCEL=1000,1000,1000,1000
DECEL=1000,1000,1000,1000
MERGE = ON
TRIGGER
MOVE_PVTPPABS(350,200,10,150,10,250,10)'在一个周期内，轴 0,1,2 运动到 200,150,250 的位置，结束速度为 10，加速度降为 0
MOVE_PVTPPABS(350,-200,10,-150,10,-250,10)'在一个周期内，轴 0,1,2 运动到-200,-150,-250 的位置，结束速度为 10，加速度降为 0
MOVE_PVTPPABS(350,200,10,150,10,250,10)
MOVE_PVTPPABS(350,-200,10,-150,10,-250,10)
MOVE_PVTPPABS(350,200,10,150,10,250,10)
MOVE_PVTPPABS(350,-200,10,-150,10,-250,10)
WAIT IDLE
DPOS(0)、DPOS(1)、DPOS(2)垂直刻度 200



MSPEED(0)、MSPEED (1)、MSPEED (2)垂直刻度 2000



MOVE_PTP -- 点到点运动

类型	特殊运动指令
描述	直线插补运动指令，用于点位运动。 不支持连续运动中速度前瞻，运动指令使用的各个轴的 SPEED，速度参数会自动带入运动缓冲。 该指令不支持在线命令动态修改，但是可以使用 SPEED_RATIO 指令来调整速度；支持 VP_MODE 设置。
语法	MOVE_PTP(mode,dis1,dis2.....) mode: 模式: BIT0: 1-根据每个轴的速度加速度限制来计算和速度与和加速度。 BIT2: 1-预留。 dis1: 第一个轴运动的距离，单位 uints，支持小数。 dis2: 第二个轴运动的距离，单位 uints，支持小数。
适用控制器	4 系列控制器，version_build 230510 以上版本。
例子	例一： '1 模式单轴与 vp_mode 指令用例 rapidstop(2) trigger speed_ratio = 1 base(0) mpos = 0 dpos = 0 atype = 1 speed = 100 units = 100

```

accel = 1000
decel = 1000
vp_mode = 0
move_ptp(1,200)
wait idle
vp_mode = 6
move_ptp(1,200)
MPOS(0)、MSPEED(0)垂直刻度 200

```



例二：

'1 模式多轴与 speed_ratio 指令用例

```

rapidstop(2)
trigger
speed_ratio = 1
vp_mode = 0,0
base(0,1)
mpos = 0,0
dpos = 0,0
atype = 1,1
speed = 100,200
units = 500,500
accel = 500,500
decel = 500,500
move_ptp(1,150,200)
wait idle
speed_ratio = 0.5
move_ptp(1,150,200)

```

MPOS(0)、MSPEED(0)、MPOS(1)、MSPEED(1)垂直刻度 200



例三：

'0 模式多轴与 speed_ratio 指令用例

rapidstop(2)

trigger

speed_ratio = 1

vp_mode = 0,0

base(0,1)

mpos = 0,0

dpos = 0,0

atype = 1,1

speed = 100,200

units = 500,500

accel = 500,500

decel = 500,500

move_ptp(0,150,200)

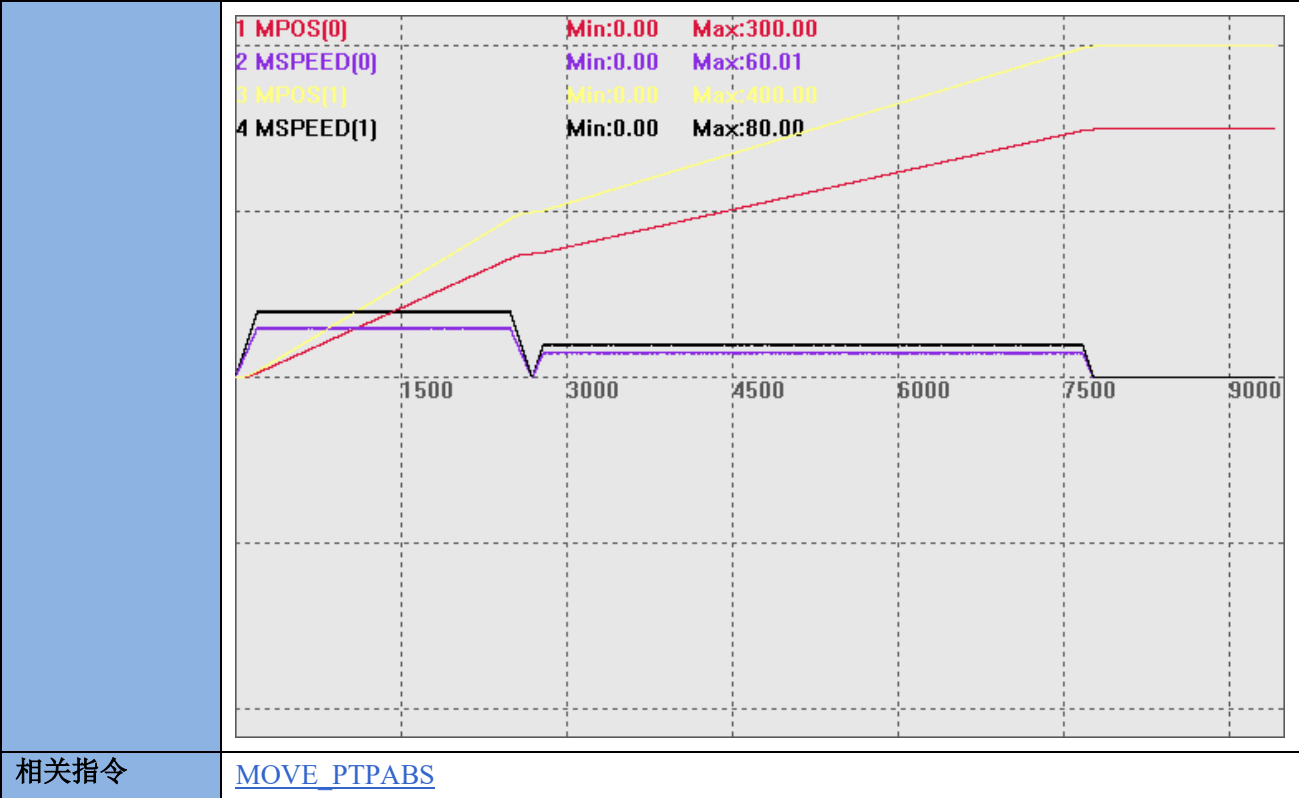
'0 模式下与 move 指令效果一致

wait idle

speed_ratio = 0.5

move_ptp(0,150,200)

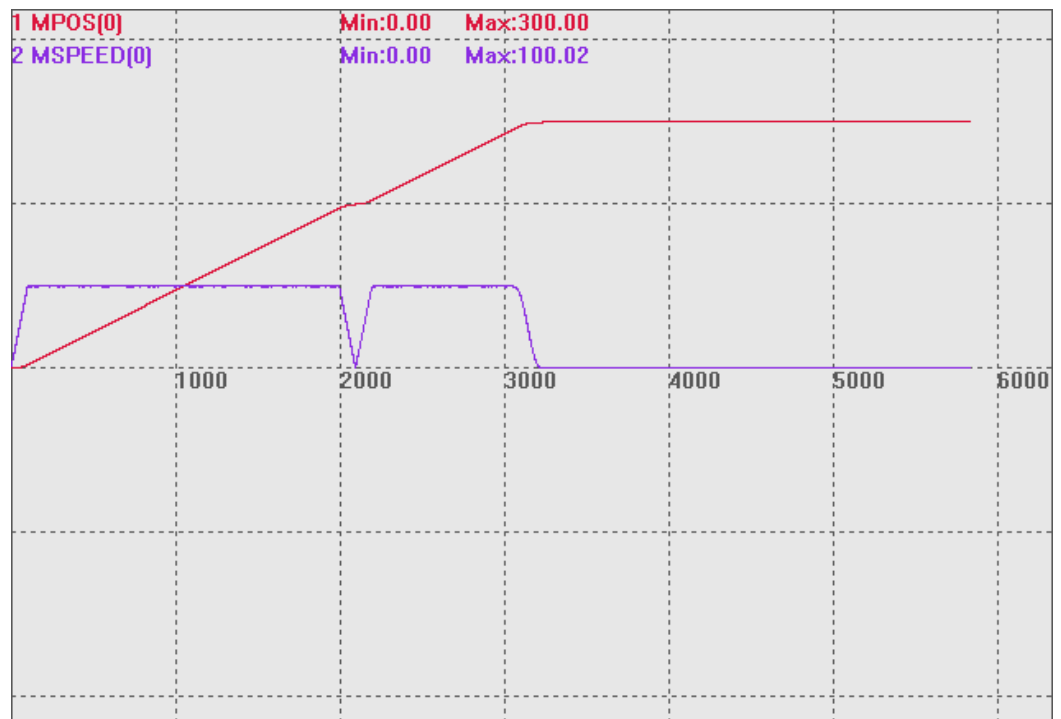
MPOS(0)、MSPEED(0)、MPOS(1)、MSPEED(1)垂直刻度 200



MOVE_PTPABS -- 点到点运动-绝对

类型	特殊运动指令
描述	直线插补运动指令，用于点位运动。 不支持连续运动中速度前瞻，运动指令使用的各个轴的 SPEED，速度参数会自动带入运动缓冲。 该指令不支持在线命令动态修改，但是可以使用 SPEED_RATIO 指令来调整速度；支持 VP_MODE 设置。
语法	MOVE_PTP(mode,dis1,dis2.....) mode: 模式: BIT0: 1-根据每个轴的速度加速度限制来计算和速度与和加速度。 BIT2: 1-预留。 dis1: 第一个轴运动的绝对位置，单位 uints，支持小数。 dis2: 第二个轴运动的绝对位置，单位 uints，支持小数。
适用控制器	4 系列控制器，version_build 230510 以上版本。
例子	例一： '1 模式单轴与 vp_mode 指令用例 rapidstop(2) trigger speed_ratio = 1 base(0) mpos = 0 dpos = 0

```
atype = 1
speed = 100
units = 100
accel = 1000
decel = 1000
vp_mode = 0
move_ptpabs(1,200)
wait idle
vp_mode = 6
move_ptpabs(1,300)
MPOS(0)、MSPEED(0)垂直刻度 200
```



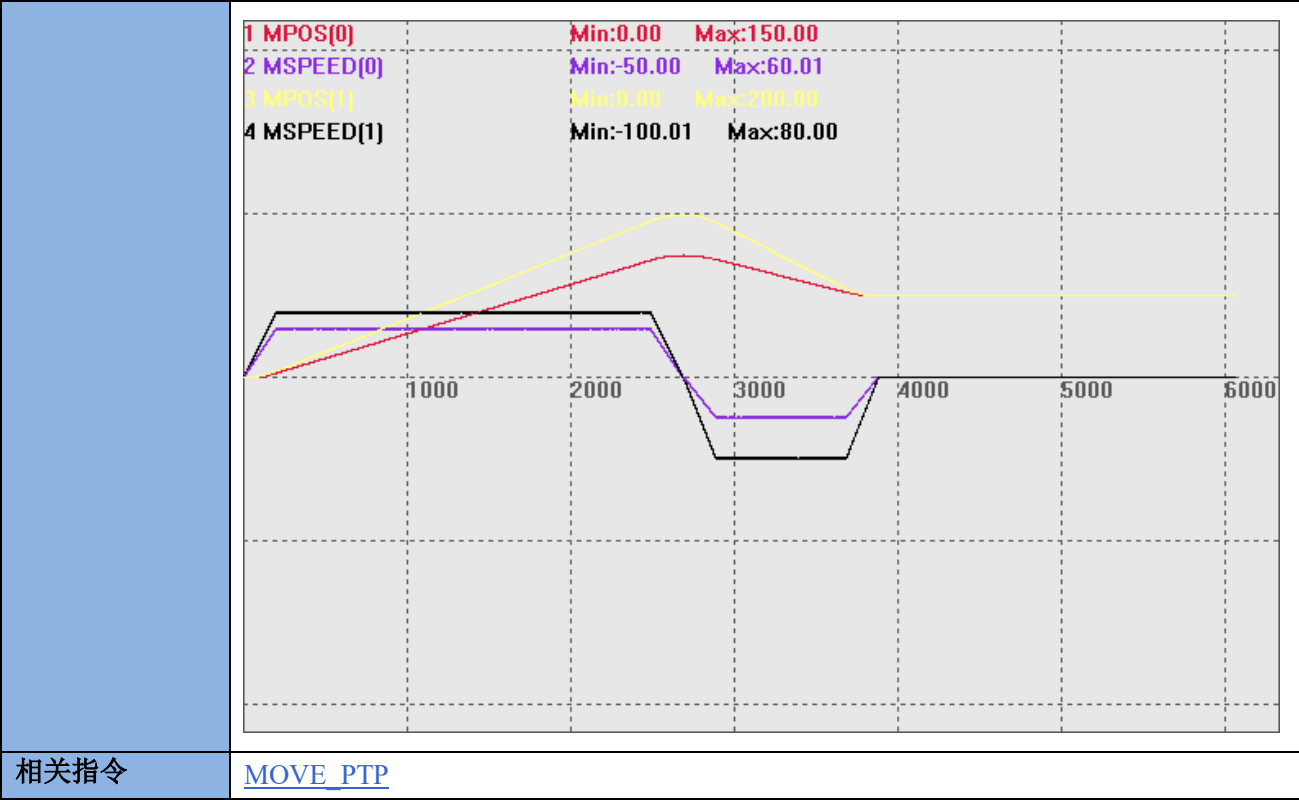
例二：
'1 模式多轴与 speed_ratio 指令用例
rapidstop(2)
trigger
speed_ratio = 1
vp_mode = 0,0
base(0,1)
mpos = 0,0
dpos = 0,0
atype = 1,1
speed = 100,200
units = 500,500
accel = 500,500
decel = 500,500


```
move_ptpabs(1,150,200)
wait idle
speed_ratio = 0.5
move_ptpabs(1,100,100)
MPOS(0)、MSPEED(0)、MPOS(1)、MSPEED(1)垂直刻度 200
```



```
例三：
'0 模式多轴与 speed_ratio 指令用例
rapidstop(2)
trigger
speed_ratio = 1
vp_mode = 0,0
base(0,1)
mpos = 0,0
dpos = 0,0
atype = 1,1
speed = 100,200
units = 500,500
accel = 500,500
decel = 500,500
move_ptpabs(0,150,200)
wait idle
speed_ratio = 0.5
move_ptpabs(1,100,100)
MPOS(0)、MSPEED(0)、MPOS(1)、MSPEED(1)垂直刻度 200
```

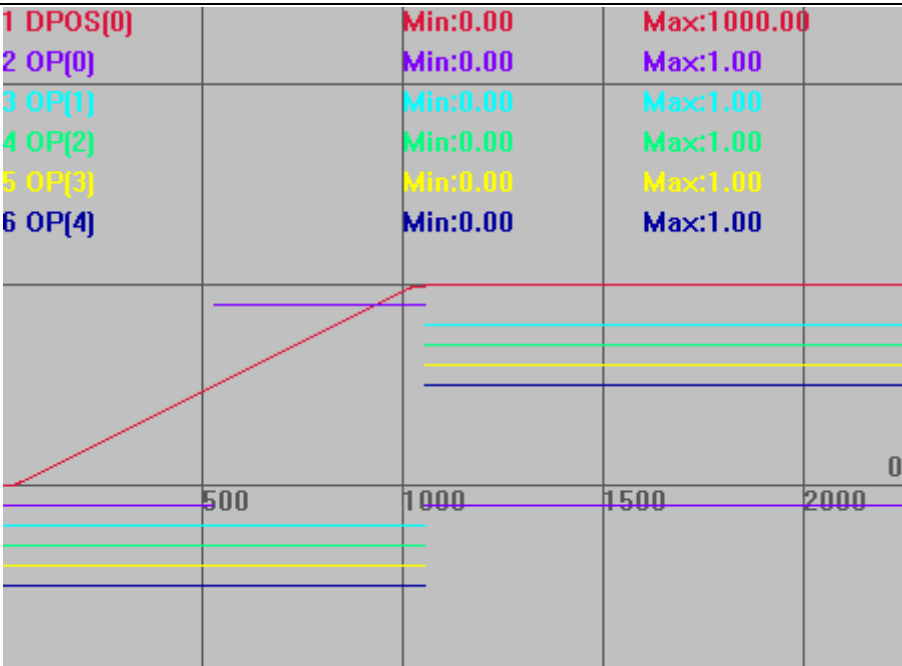
'0 模式下与 moveabs 指令效果一致



MOVE_OP -- 缓冲输出

类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个输出口操作。 这个指令 LOAD 执行时不做任何运动，只操作输出口。语法同 OP 指令。 普通模式，误差在一个扫描周期，所有控制器都可使用。 精准位置输出模式，误差 1 微秒以内，4 系列产品，20170421 以上版本支持。 只有支持硬件比较输出的 OP 口才支持精准输出功能。 每个生效的精准输出 MOVE_OP 需要间隔 1 个周期时间才能继续生效，此间隔内新的 MOVE_OP 自动使用普通方式，超过此间隔，新 MOVE_OP 可继续生效。连续的 MOVE_OP，因为没有间隔时间，只有第一个生效。（某些控制器可以多个精准输出同时触发，具体查看控制器硬件手册说明，例如 ZMC420SCAN 前 8 个输出口支持精准输出，而且每个输出口可以同时使用精准输出） MOVE_OP 的个数不建议超过 10 个。 同时使用多个 OP 口的精准输出功能仅支持 DPOS 位置的比较输出，使用单个 OP 口时才支持 MPOS 位置的比较输出。 在控制器支持 OP 口独立的情况下，不同 OP 口就算多个轴 MOVE_OP 精准输出也不冲突，在 OP 口精准输出功能不独立的情况，同时使用精准功能可能冲突。 MOVE_OP 精准功能基于 BASE 主轴，多个轴插补时，与 BASE 主轴 ATYPE 类型不一样的从轴，无法保证精准位置输出。 增加支持不同 MOVE_OP 指令可设置不同的精准参数，从而支持激光电源的 us 级控

	制，开启精准输出或延时输出等参数指令在 MOVE_OP 调用前设置即可。 4 系列以上控制器，version_build: 20241029 以后固件用 MOVE_OP 不影响平滑。 精准输出判断编码器是否到位，只能找到尽可能靠近的位置，哪个轴影响大就将该轴的细分设大一点。
语法	语法一：MOVE_OP ([ionum],value) ionum: 输出编号，从 0 开始 value: 输出状态，多个输出口操作时按位来指明多个口状态 语法二：MOVE_OP (ionum1, ionum2,value[,mask]) ionum1: 要操作的第一个输出通道 ionum2: 要操作的最后一个输出通道 value: 输出状态，多个输出口操作时按位来指明多个口状态 mask: 按位来设置值，指定哪些 IO 需要操作，不填时从第一个通道到最后一个通道都操作
适用控制器	通用
例子	例一：普通模式使用 BASE(0) UNITS=100 DPOS=0 SPEED=200 ACCEL=1000 DECEL=1000 TRIGGER '自动触发示波器 MOVE(500) MOVE_OP (0,ON) '等待上条运动完成后，OUT0 口输出信号 MOVE(500) MOVE_OP (0,OFF) '等待上条运动完成后，OUT0 口关闭信号 MOVE_OP(1,4,15) 'OUT1-4 口输出信号，15 对应二进制 1111 轨迹曲线，为方便查看，进行了竖直偏移 DPOS(0)垂直刻度 1000 OP(0)垂直刻度 1,偏移-0.1 OP(1)垂直刻度 1,偏移-0.2 OP(2)垂直刻度 1,偏移-0.3 OP(3)垂直刻度 1,偏移-0.4 OP(4)垂直刻度 1,偏移-0.5



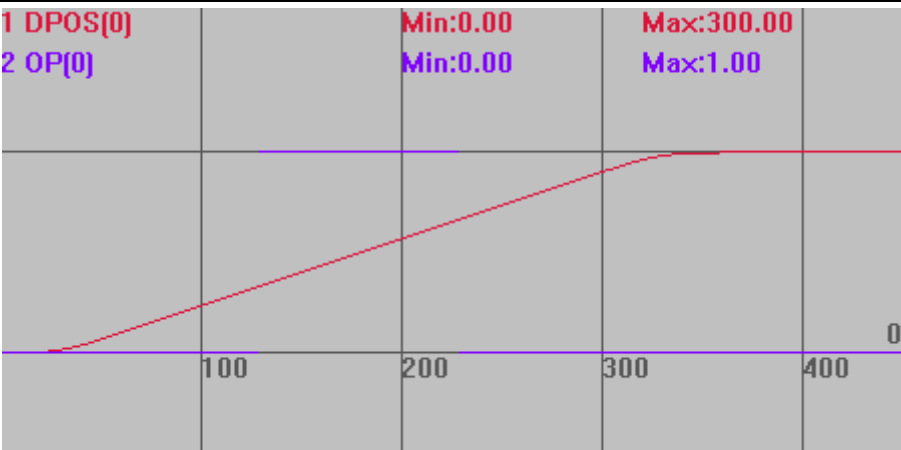
例二：精准位置输出模式

```
BASE(0)
UNITS=100
DPOS=0
SPEED=200
ACCEL=1000
DECEL=1000
TRIGGER          '自动触发示波器
ATYPE=1
MERGE=1
AXIS_ZSET(0)=2   '开启 MOVE_OP 的精准输出功能
MOVE(100)
MOVE_OP(0,1)      '精准生效
MOVE(100)         '超过 2MS 时间, 下个 MOVE_OP 可继续精准生效
MOVE_OP(0,0)      '精准生效
MOVE(100)
```

轨迹曲线

DPOS(0)垂直刻度 300

OP(0)垂直刻度 1



例三：编码器精准位置输出模式

20170505 以上固件版本支持

DIM opnum

AXIS_ZSET = 3+16 'BIT4 支持编码器精确位置

BASE(0)

ATYPE= 4 '轴使用脉冲加编码器类型，必须实际有接编码器线

DPOS=0

MPOS=0

UNITS=1000

SPEED= 1000

ACCEL = 1000

MERGE=1

TRIGGER

opnum = 0

MOVEOP_DELAY = 2 '实际输出时间延迟 2ms

HW_TIMER(0, 10000, 5000, 1, 0, opnum)

OP(opnum,0) '初始化 OP

HW_TIMER(2, 10000, 5000, 1, 0, opnum) '输出变为 on 后, 5000us 后切换为 off

MOVE(200)

MOVE_OP(opnum,1) '输出 on 后靠 HW_TIMER 来延时关闭，延时 5ms 后关闭

MOVE(100)

MOVE_OP(opnum,1)

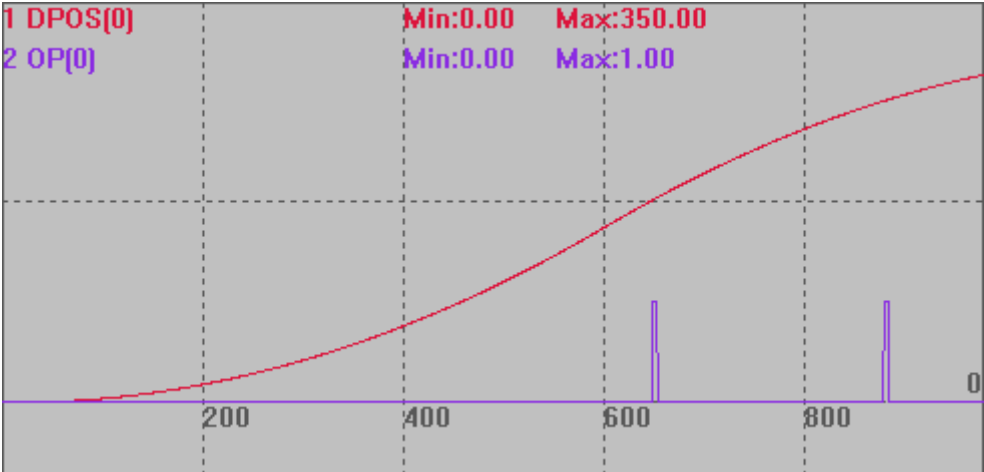
MOVE(50)

END

轨迹曲线

DPOS(0)垂直刻度 200

OP(0)垂直刻度 2

	
相关指令	OP , MOVE_OP2 精准位置输出模式 SYSTEM_ZSET , AXIS_ZSET

MOVE_OP2 -- 缓冲输出 2

类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个输出口操作，指定时间后输出状态翻转。 这个指令 LOAD 执行时不做任何运动，只操作输出口。 单个轴同一时间只支持一个脉冲输出，第二个 MOVE_OP2 指令会自动关闭前面指令的脉冲。
语法	MOVE_OP2(ionum,state,offtimems) ionum: 输出编号，从 0 开始 state: 输出状态 offtimems: 多少 ms 时间后翻转，以产生脉冲输出的效果
适用控制器	通用
例子	BASE(0) UNITS=100 DPOS=0 SPEED=200 ACCEL=1000 DECEL=1000 OP(0,OFF) '关闭 OUT0 口 TRIGGER '自动触发示波器 MOVE(500) MOVE_OP2 (0,ON,1000) '等待上条运动完成，输出口 0 输出一个 1s 的脉冲，脉冲时间不会阻碍下一条运动执行 MOVE(-500) 轨迹曲线 DPOS(0)垂直刻度 1000

	OP(0)垂直刻度 1 1 DPOS[0] 2 OP[0] Min:0.00 Min:0.00 Max:500.00 Max:1.00
相关指令	MOVE_OP , OP

MOVE_TABLE -- 缓冲 Table

类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个 TABLE。 这个指令 LOAD 执行时不做任何运动,只修改 TABLE。此指令的 MTYPE 与 MOVE_OP 一致。 4 系列以上控制器, version_build: 20241029 以后固件用 MOVE_TABLE 不影响平滑。
语法	MOVE_TABLE(tablenum, value) tablenum: table 编号 value: 要修改的值
适用控制器	通用
例子	BASE(0) UNITS=100 DPOS=0 SPEED=200 ACCEL=1000 DECEL=1000 TABLE(0)=0 '先令 TABLE(0)=0 ?TABLE(0) '打印确认 TABLE(0)的值 '打印结果, 0 TRIGGER '自动触发示波器 MOVE(500) MOVE_TABLE(0, 60) '等待运动完成后, TABLE(0)赋值 60 MOVE(500) WAIT IDLE ?TABLE(0) '打印修改后 TABLE(0)的值, 打印结果, 60 轨迹曲线 DPOS(0)垂直刻度 1000

	TABLE(0)垂直刻度 100
相关指令	TABLE

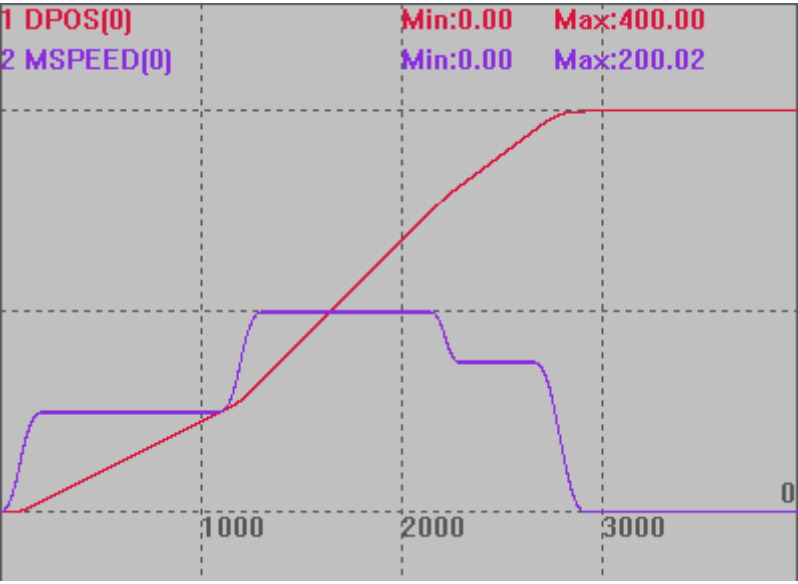
MOVE_PARA -- 缓冲参数

类型	特殊运动指令
描述	BASE 轴运动缓冲修改参数。 这个指令 LOAD 执行时不做任何运动，只修改参数。此指令的 MTYPE 与 MOVE_OP 一致。 注意事项： 1.MOVE_PARA 仅适用于修改全局参数，而不应用于修改那些会被带入运动缓冲区的参数，例如 FORCE_SPEED、CORNER_MODE、INTPFACTOR 和 CORNER_ACC 等。在分离运动模式下，ACCEL 和 SPEED 也会被带入缓冲，因此不能使用 MOVE_PARA 修改。 2.为了确保参数值能够正确地被带入运动缓冲区，必须在 MOVE 指令被填入运动缓冲区之前，通过直接调用参数来进行修改。如果在 MOVE 指令之后尝试修改这些参数，可能会导致参数值的随机性，从而影响运动效果的确定性。
语法	MOVE_PARA(paraname, index, value) paraname: 参数名, 必须是?*set 里面的非只读参数 index: 参数编号 value: 参数值
适用控制器	20170503 以上固件
例子	例一：修改 SPEED <pre>BASE(0) '选择轴 0 ATYPE=1 SPEED=100 PRINT SPEED '打印结果 100 MOVE_PARA(speed ,0,200) '修改轴 0 的 SPEED 参数值为 200 DELAY(1000) PRINT SPEED '打印结果 200</pre> 例二：单轴变速


```
BASE(0)      '选择轴 0
UNITS=1000
ATYPE=1
SPEED=100
ACCEL =1000
DECEL =1000
SRAMP =100
DPOS = 0
MERGE=ON
TRIGGER

MOVE(100)
MOVE_PARA(SPEED,0,200)
MOVE(200)
MOVE_PARA(SPEED,0,150)
MOVE(100)
END
```

垂直刻度均为 200

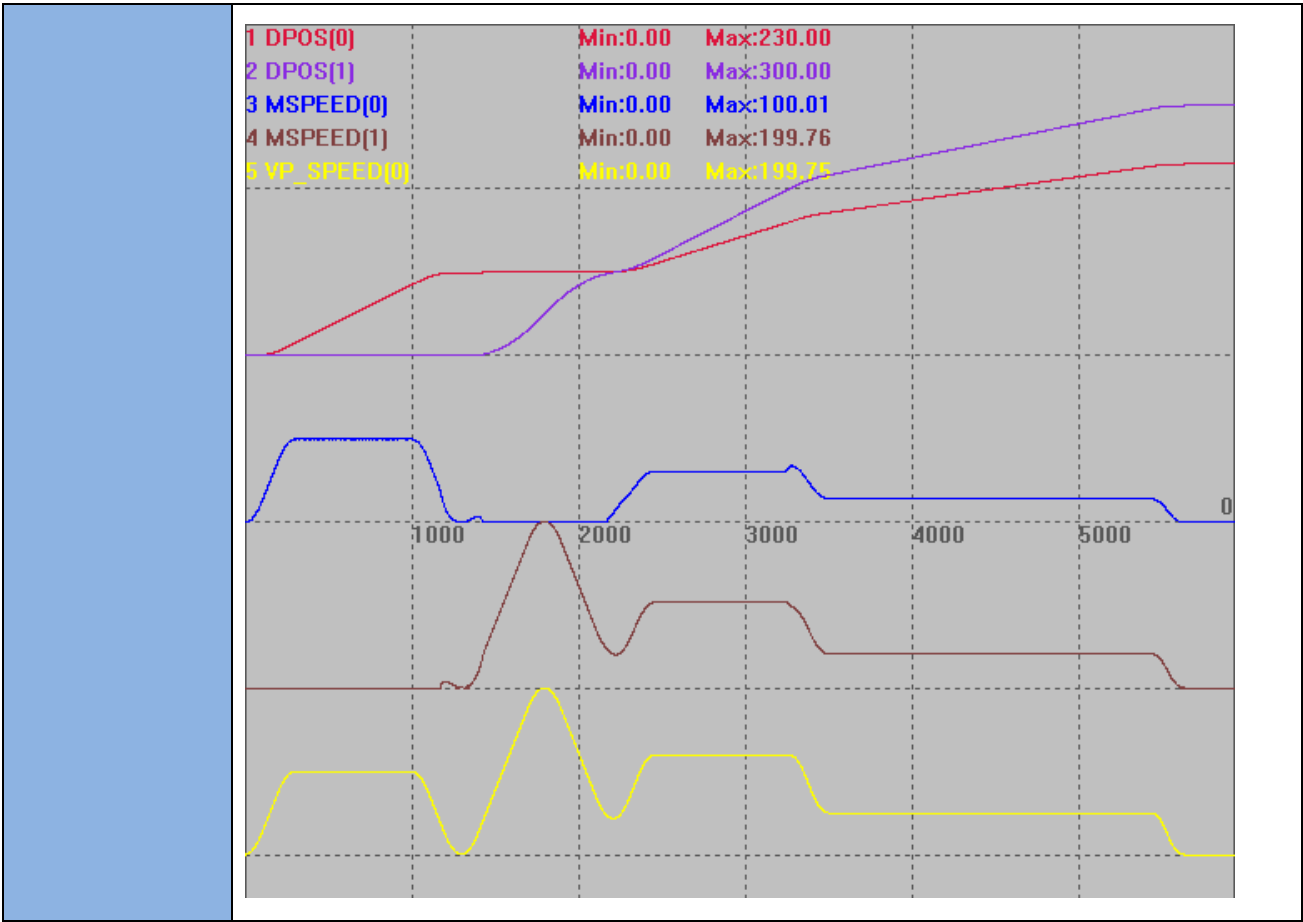


例三：多轴变速

```
RAPIDSTOP(2)
WAIT IDLE(0)
WAIT IDLE(0)

BASE(0,1)
DPOS=0,0
UNITS=100,100
SPEED=100,100      '设置速度
ACCEL=500,500      '设置加速度
```

DECEL=500,500	'设置减速度
SRAMP=100,100	'S 曲线
MERGE=ON	'开启连续插补
CORNER_MODE=2+8+32	'启动拐角减速
DECEL_ANGLE = 15 * (PI/180)	'设置开始减速角度
STOP_ANGLE = 45 * (PI/180)	'设置结束减速角度
ZSMOOTH=2	
FORCE_SPEED=100	'等比减速时起作用
TRIGGER	'自动触发示波器
MOVE(100,0)	
MOVE_PARA(SPEED,0,200)	
MOVE(0,100)	'运动角度大于 45°，完全减速
MOVE_PARA(SPEED,0,120)	
MOVE(60,100)	'运动角度 30.96° 处于 15° ~45°，等比减速
MOVE_PARA(SPEED,0,50)	
MOVE(70,100)	'运动角度 4.03° 小于 15°，不减速
END	
垂直刻度均为 200	



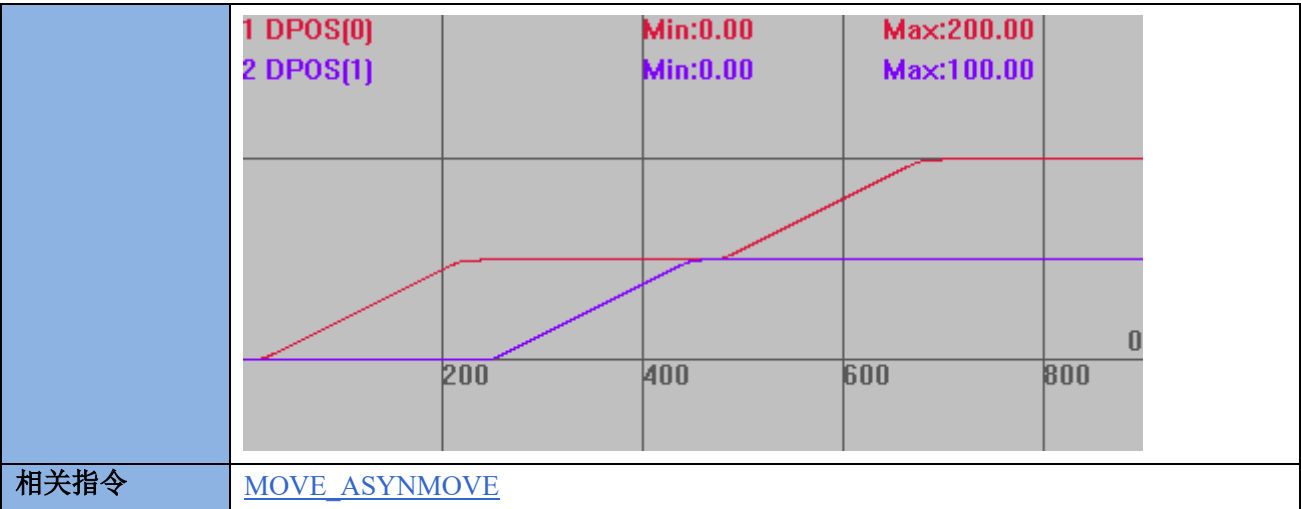
MOVE_PWM -- 缓冲 PWM

类型	特殊运动指令
描述	BASE 轴运动缓冲操作 PWM。 这个指令 LOAD 执行时不做任何运动，只操作 PWM。此指令的 MTYPE 与 MOVE_OP 一致。 PWM 只能通过设置占空比为 0 来关闭，不能通过设置 PWM 频率为 0 实现，PWM 频率一定要在 PWM 开关之前调整。
语法	MOVE_PWM(pwmindex,duty,freq) pwmindex: pwm 编号 duty: 占空比，指有效电平占整个周期的比例；范围 0-1，设置 0 时关闭 pwm；一个周期中先输出有效电平，再输出无效电平 freq: 频率，缺省为 1KHz，硬件最大为 1MHz，软件最大为 2KHz
适用控制器	20170503 以上固件
例程	RAPIDSTOP(2) WAIT IDLE TRIGGER TICKS=0 BASE(0) SPEED = 1000

	<pre> MOVE(10) MOVE_PWM(0, 0.111, 2000) '轴 0 运行到 10 时，操作 PWM0 MOVE_DELAY(111) MOVE_PWM(0, 0.333) MOVE_DELAY(111) MOVE_PWM(0, 0.555, 3000) MOVE(100) WHILE NOT IDLE MOVE_PWM(0, 0, 1000) '关闭 PWM ? -TICKS, PWM_FREQ(0), PWM_DUTY(0) WA 10 WEND </pre>
相关指令	PWM_DUTY , PWM_FREQ

MOVE_SYNMOVE -- 缓冲触发其他轴

类型	特殊运动指令
描述	BASE 轴运动缓冲触发其他轴运动，当前轴等待。 此指令的 MTYPE 与 MOVE_DELAY 一致。
语法	<pre> MOVE_SYNMOVE(axisnum,dis[,ifsp]) </pre> axisnum: 需要同步运动的轴号 dis: 相对运动距离 ifsp: 是否使用 sp 运动，缺省不使用
适用控制器	20170503 以上固件
例程	<pre> RAPIDSTOP(2) WAIT IDLE TRIGGER TICKS=0 BASE(0,1) DPOS=0,0 UNITS=100,100 SPEED = 100,100 ACCEL=1000,1000 DECEL=1000,1000 MOVE(100) MOVE_SYNMOVE(1,100,0) '轴 0 运行到 100 时，轴 1 开始运动 100 MOVE(100) '待轴同步完成，轴 0 再继续 运动轨迹 DPOS(0)垂直刻度 200 DPOS(1)垂直刻度 200 </pre>



MOVE_ASYNCMOVE -- 缓冲触发其他轴 2

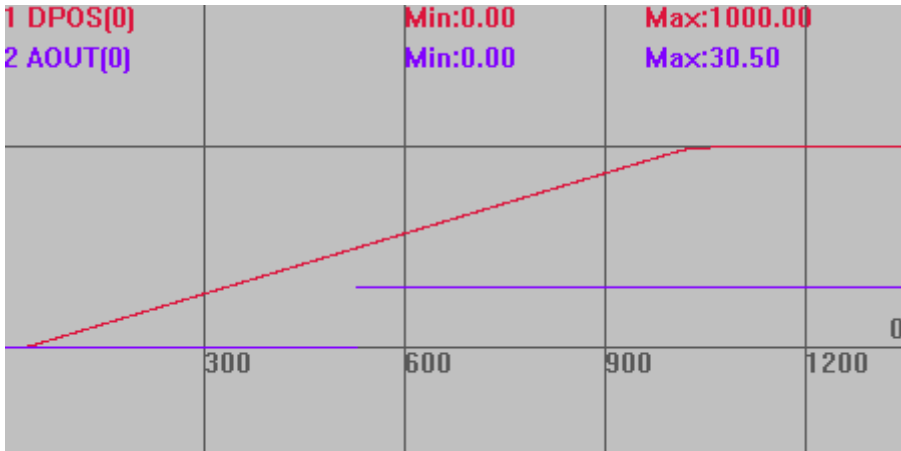
类型	特殊运动指令
描述	BASE 轴运动缓冲触发其他轴，当前轴不等待。 此指令的 MTYPE 与 MOVE_OP 一致。
语法	MOVE_ASYNCMOVE(axisnum,dis[,ifsp]) axisnum: 需要同步运动的轴号 dis: 相对运动距离 ifsp: 是否使用 sp 运动, 缺省不使用
适用控制器	20170503 以上固件
例程	<pre>RAPIDSTOP(2) WAIT IDLE TRIGGER TICKS=0 BASE(0,1) DPOS=0,0 UNITS=100,100 SPEED = 100,100 ACCEL=1000,1000 DECEL=1000,1000 MOVE(100) MOVE_ASYNCMOVE(1,100,0) '轴 0 运行到 100 时，轴 1 开始运动 100 MOVE(100) '轴 0 持续运动 运动轨迹 DPOS(0)垂直刻度 200 DPOS(1)垂直刻度 200</pre>

	1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:200.00 Max:100.00			
相关指令	MOVE_SYNMOVE			

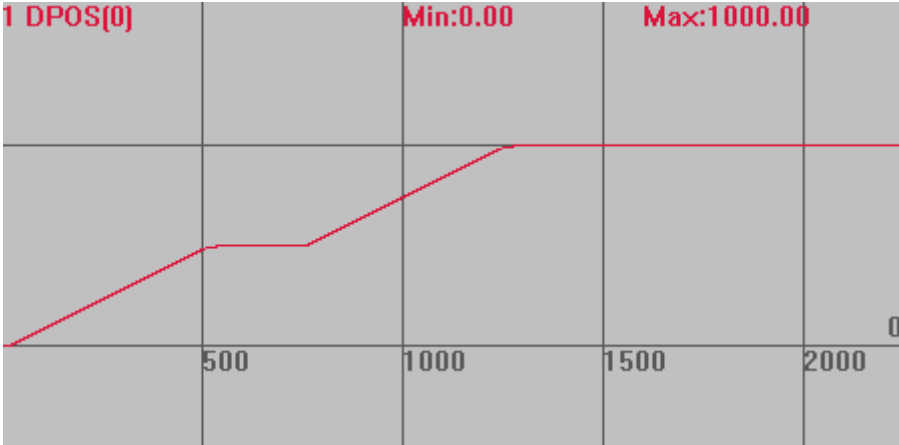
MOVE_TASK -- 缓冲开启任务

类型	特殊运动指令
描述	BASE 轴运动缓冲加入启动 TASK。 这个指令 LOAD 执行时不做任何运动，只启动任务，此指令的 MTYPE 与 MOVE_OP 一致。 当任务已经启动时，会报错，但不影响程序执行。
语法	MOVE_TASK(tasknum, label) tasknum: 任务号 label: 函数名或标号
适用控制器	通用
例子	<pre> BASE(0) DPOS=0 UNITS=100 ACCEL=1000 DECEL=1000 SPEED=100 ACCEL=1000 DECEL=1000 MOVE(100) MOVE_TASK(1, task_move) '第一个运动完成后，将 task_move 作为任务 1 启动 MOVE(100) END TASK_MOVE: PRINT "TASK_MOVE" END </pre>
相关指令	RUNTASK , RUN

MOVE_AOUT -- 缓冲模拟量输出

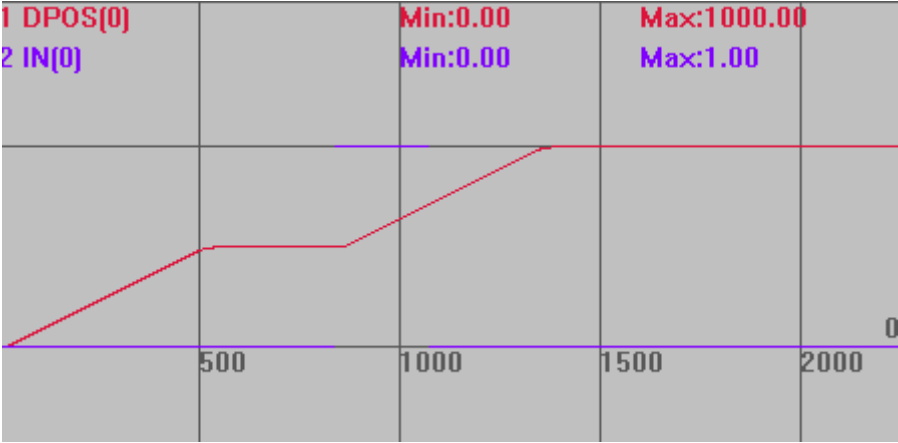
类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个 AOUT 指令。 这个指令 LOAD 执行时不做任何运动，只修改 AOUT 值。此指令的 MTYPE 与 MOVE_OP 一致。 设置激光能量输出时，支持精准输出。
语法	MOVE_AOUT(danum, value) danum: DA 编号 value: 要修改的值
适用控制器	通用，实际只有包含 DA 通道的控制器生效
例子	BASE(0) UNITS=100 DPOS=0 SPEED=200 ACCEL=1000 DECEL=1000 AOUT(0)=0 'DA0 通道赋值 0 ?AOUT(0) '打印确认 TRIGGER '自动触发示波器 MOVE(500) MOVE_AOUT(0, 30.5) '第一个运动完成后,将 DA0 通道赋值 30.5 MOVE(500) WAIT IDLE ?AOUT(0) '打印 DA0, 30.5 运动轨迹 DPOS(0)垂直刻度 1000 AOUT(0)垂直刻度 100 
相关指令	AOUT

MOVE_DELAY -- 缓冲延时

类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个延时。 这个指令 LOAD 执行时不做任何运动，只延时指定时间。 前面的运动指令结束时速度会自动降为 0。
语法	move_delay(timems) 别名：move_wa(timems) timems: 延时，毫秒单位
适用控制器	通用
例子	BASE(0) DPOS=0 ATYPE=1 UNITS=100 SPEED=200 ACCEL=2000 DECEL=2000 TRIGGER '自动触发示波器 MOVE(500) MOVE_DELAY(1000) '两个 MOVE 中间等待 1 秒 MOVE(500) 运动轨迹 DPOS(0)垂直刻度 1000 
相关指令	DELAY

MOVE_WAIT -- 缓冲等待

类型	特殊运动指令
描述	BASE 轴运动缓冲加入一个条件判断。 这个指令 LOAD 执行时不做任何运动，只等待指定的条件满足，前面的运动指令结束时速度会自动降为 0。
语法	MOVE_WAIT(paraname, par anum, eq, value)

	paraname: 选择参数名 (参数可以为: DPOS, MPOS, IN, AIN, VPSPEED, MSPEED, MODBUS_REG, MODBUS_IEEE, MODBUS_BIT, TABLE, VECTOR_BUFFERED, REMAIN) paranum: 参数编号或轴号 eq: 比较模式: 1: 大于等于 ($\geq$) 0: 等于 (不建议使用) -1 小于等于 ($\leq$), 对 IN 等 BIT 类型参数无效 value: 比较值
适用控制器	固件 150802 以上版本, 或 XPLC160405 以上版本支持。
例子	BASE(0) DPOS=0 ATYPE=1 UNITS=100 SPEED=200 ACCEL=2000 DECEL=2000 TRIGGER '自动触发示波器 MOVE(500) MOVE_WAIT(IN, 0, 0, 1) '等待 IN(0)有信号, 才执行下一条运动缓冲 MOVE(500) 运动轨迹 DPOS(0)垂直刻度 1000 IN(0)垂直刻度 1 
相关指令	WAIT UNTIL

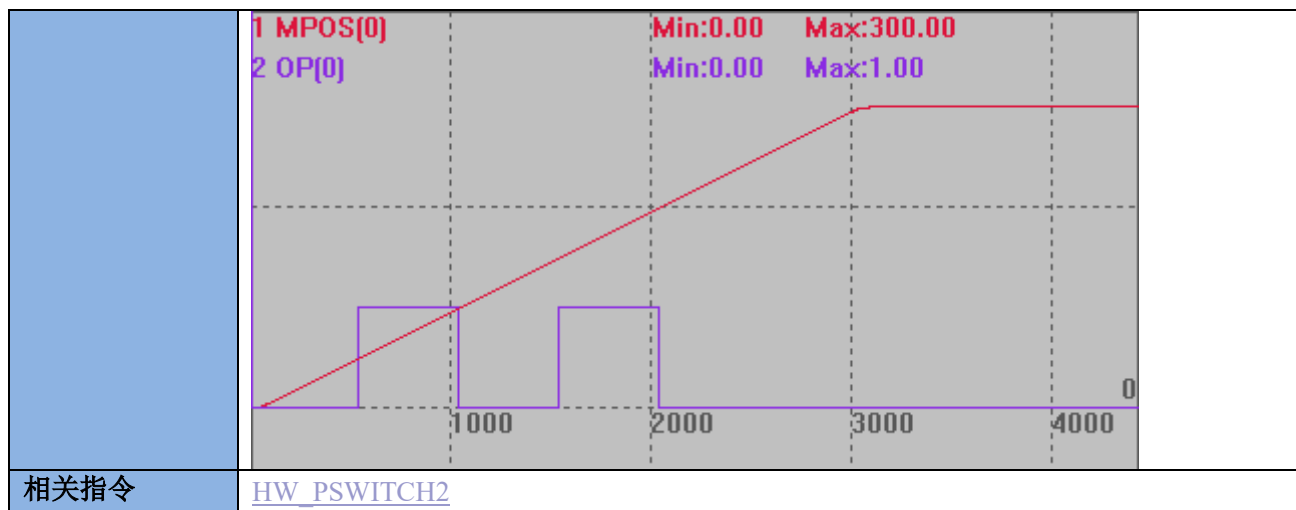
MOVE_CANCEL -- 缓冲停止

类型	特殊运动指令
描述	把 CANCEL 指令写入运动缓冲。
语法	MOVE_CANCEL(iaxis, imode)

	iaxis: 要操作的轴号 imode: 选择 CANCEL 模式, 同 CANCEL 指令
适用控制器	通用
例子	MOVE_CANCEL(1,0) AXIS(0) '轴 0 缓冲里面写入停止轴 1 的指令
相关指令	CANCEL

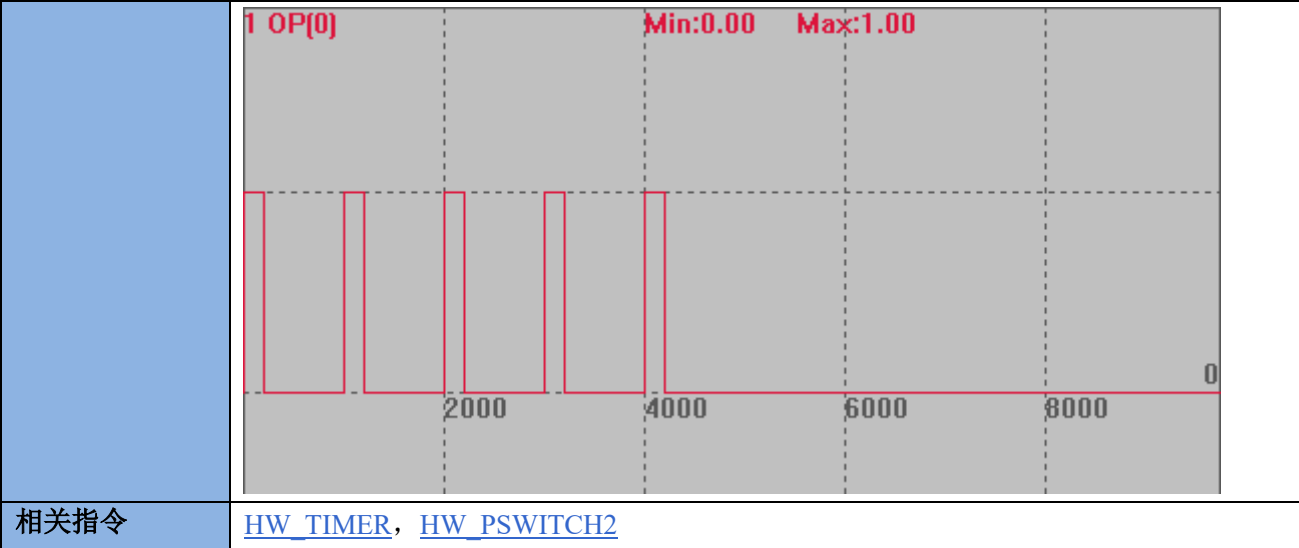
MOVE_HWPSWITCH2 -- 缓冲硬件比较输出

类型	特殊运动指令
描述	把 HW_PSWITCH2 指令写入运动缓冲, 在运动缓冲中执行硬件比较输出。 参数同 HW_PSWITCH2, 支持动态修改 HW_TIMER 的参数, 必须支持 HW 功能的控制器。
语法	MOVE_HWPSWITCH2 (mode, [...]) mode: 启动不同的比较器, 需要填写的参数也有所差异, 参见 HW_PSWITCH2 语法说明与例程。
适用控制器	通用
例子	BASE(0) UNITS=100 SPEED=100 ACCEL=1000 DECEL=1000 DPOS=0 MPOS=0 OP(0,OFF) TABLE(0,50,100,150,200) '比较点坐标设置 MOVE_HWPSWITCH2(2) '停止并删除没有完成的比较点 MOVE_HWPSWITCH2(1, 0, 1, 0, 3,1) '比较 4 个点, 模式 1, 操作输出口 0 TRIGGER '触发示波器 MOVE(300) END 比较输出图 与 HW_PSWITCH2 的输出效果相同, 区别是此例子三条命令进运动缓冲。 MPOS(0)垂直刻度 200 OP(0)垂直刻度 2



MOVE_HWTIMER -- 缓冲硬件定时

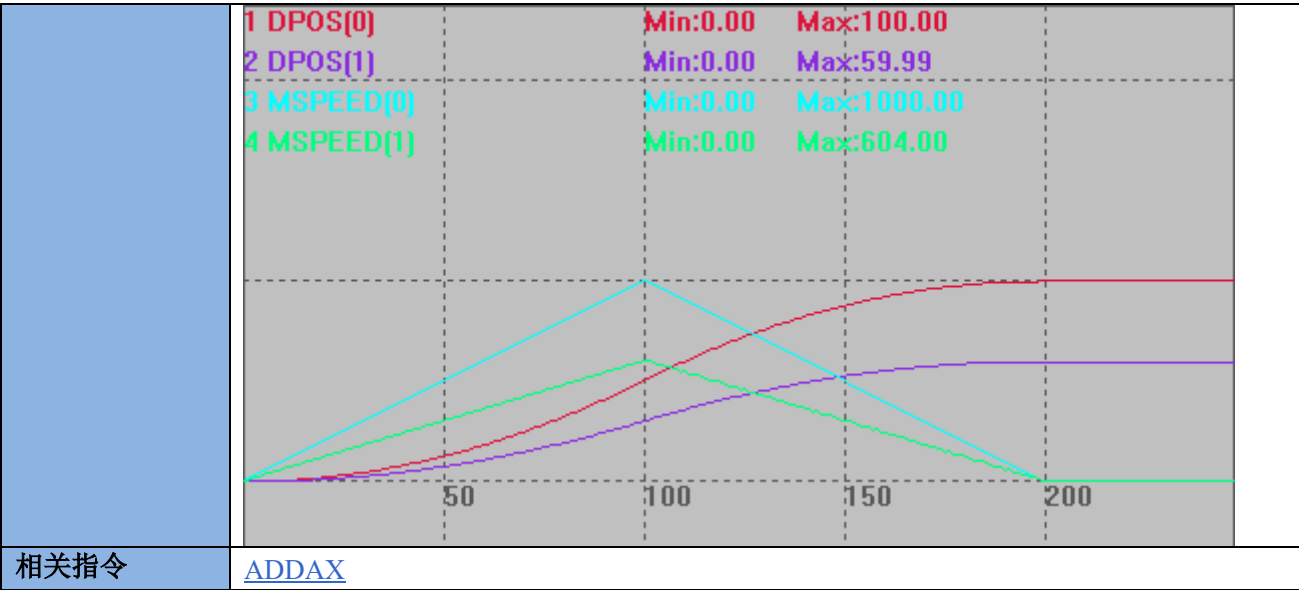
类型	特殊运动指令
描述	把 HW_TIMER 指令写入运动缓冲，在运动缓冲中执行硬件定时。 参数同 HW_TIMER，支持动态修改 HW_TIMER 的参数，必须支持 HW 功能的控制器。 触发 HW_TIMER 必须使用 MOVE_OP 指令，OP 无法触发。 不使用 HW_TIMER 时，调用模式 0 关闭，否则会对后续输出有影响。
语法	MOVE_HWTIMER (mode, [...]) mode: 模式 0/1/2/3/4，不同需要填写的参数也有所差异，参见 HW_TIMER 语法说明与例题。
适用控制器	通用(支持 HW/HW2 功能的控制器)
例子	<pre> BASE(0) ATYPE=1 UNITS=100 SPEED=100 ACCEL=500 DPOS=0 TRIGGER MOVE_OP(0, OFF) MOVE_HWTIMER(2,1000000,200000,5,OFF,0) '输出口 0 变为 on 后，硬件定时器触 发开始计时，500ms 后切换为 off，周期 5 次 MOVE_OP(0, ON) END </pre>



MOVE_ADDAX -- 运动叠加

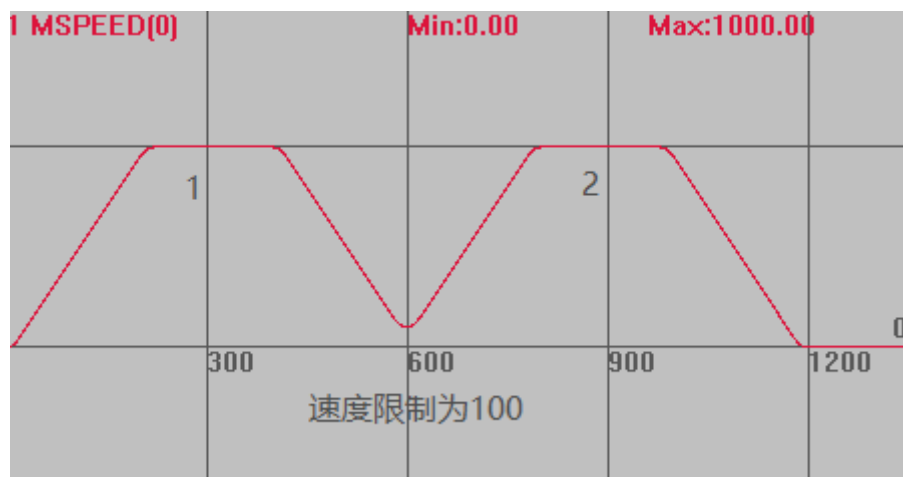
类型	单轴运动指令
描述	运动叠加加入缓冲，把一个轴的运动叠加到另一个轴，支持 BASE_MOVE，这样可以随意调整 destaxis.。 MOVE_ADDAX 指令叠加的是脉冲个数，而不是设置的 units 单位。 转换关系：叠加轴运动距离*叠加轴 UNITS/被叠加轴 UNITS=被叠加轴运动距离。 假设轴 A 的 UNITS 是 100，轴 B 的 UNITS 是 50，叠加轴运动 100 把轴 A 的运动叠加到轴 B，此时轴 A 显示运动了 100，轴 B 运动了 100*100/50=200。 把轴 B 的运动叠加到轴 A，此时轴 B 显示运动了 100，轴 A 运动了 100*50/100=50。 轴之间不能相互同时叠加，A 叠加到 B 后，B 不能再叠加到 A。 支持串联叠加，A 运动叠加到 B，B 在叠加到 C。 支持并联叠加，A 运动同时叠加到 B、C。 叠加时速度从被叠加轴开始变化，加减速按照叠加轴加减速及两轴 units 比例确定。 MOVE_ADDAX 在轴 MTYPE 为 FRAME 或 REFRAME 的时候不起作用。
语法	4 系列及以上控制器，20220728 固件增加叠加选项： MOVE_ADDAX(srcaxis ,[imode], [para]) destaxis: 叠加的目标轴号 srcaxis: 叠加的源轴轴号 imode: 叠加模式 0: 缺省值，单轴叠加，兼容以前的直接脉冲个数叠加 1: 单轴叠加，支持比例调整 MOVE_ADDAX(srcaxis , 1, ratio) ratio: 比例值，支持浮点数，目标轴距离 = 源轴距离 * ratio. 2: 单轴叠加，支持齿轮比调整 MOVE_ADDAX(srcaxis , 2, ratioin, rationout) ratioin: 分子，整数，支持负数

	rationout: 分母, 正整数. 目标轴距离 = 源轴距离 * ratioin / rationout. 3: 单轴叠加到两个轴, 支持角度调整 BASE(destaxis1, destaxis2) MOVE_ADDAX(srcaxis, 3, angle) destaxis: 叠加的目标轴 1, 2 angle: 角度, 弧度值, 目标轴 1 距离 = 源轴距离 * cos(angle). 目标轴 2 距离 = 源轴距离 * sin(angle). 注意: 取消要分别取消两个轴 MOVE_ADDAX(-1, 3, 0) 或 MOVE_ADDAX(-1) AXIS(被叠加轴号) 4: 振镜联动叠加, 利用振镜轴来补偿平台轴的偏差, 必须方向和当量一致, 不一致需要齿轮比调整, 或是振镜矫正加比例. BASE(destaxis, destaxis2) MOVE_ADDAX (srcaxis, 4, srcaxis2) 利用 srcaxis 来补偿 destaxis, srcaxis2 来补偿 destaxis2. 注意: 取消也要两个轴取消. MOVE_ADDAX(-1, 4, -1) 或 MOVE_ADDAX(-1) AXIS(被叠加轴号) 5: 振镜联动叠加, 平台轴反向叠加到振镜轴, 必须方向和当量一致, 不一致需要齿轮比调整, 或是振镜矫正加比例. BASE(destaxis, destaxis2) MOVE_ADDAX (srcaxis, 5, srcaxis2) srcaxis 反向叠加到 destaxis, srcaxis2 反向叠加到 destaxis2. 注意: 取消也要两个轴取消. MOVE_ADDAX(-1, 5, -1) 或 MOVE_ADDAX(-1) AXIS(被叠加轴号) 增加 BASE_MOVE 支持: BASE_MOVE=moveaxis BASE(destaxis) MOVE_ADDAX(srcaxis, [imode], [para]) BASE_MOVE=-1
适用控制器	通用
例子	例一 更多模式说明参见 MOVE_ADDAX 指令 BASE(0,1) '选择轴号 UNITS = 100,100 DPOS =0,0 TRIGGER BASE(1) '选择被叠加轴 MOVE_ADDAX(0,2,3,5) AXIS(1) '模式 2 叠加, 轴 0 的运动叠加到轴 1 MOVE(100) AXIS(0) WAIT UNTIL IDLE(0) AND IDLE(1) ?"轴 1 叠加轴号"ADDAX_AXIS(1) MOVE_ADDAX(-1) AXIS(1) '取消叠加 END 脉冲当量相同, 轴 1 运动距离为轴 0 的 3/5 倍。

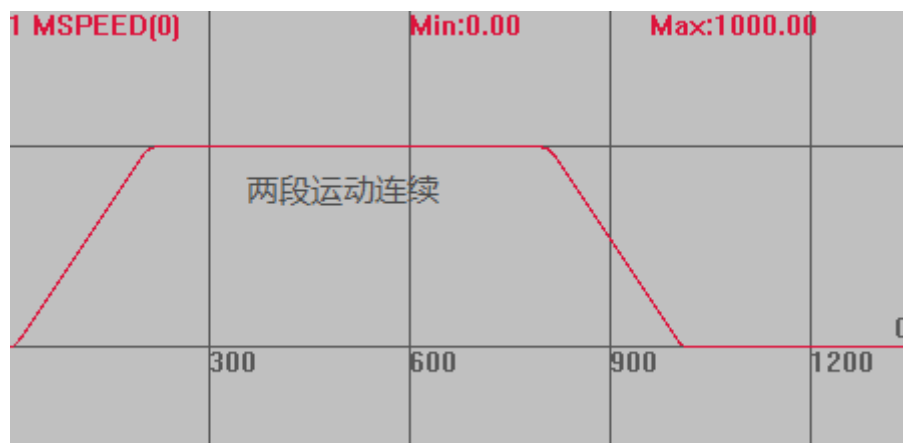


MOVELIMIT -- 速度限制

类型	特殊运动指令
描述	在当前的运动末尾位置增加速度限制，用于强制拐角减速。
语法	MOVELIMIT(limitspeed) limitspeed: 限制到的速度
适用控制器	通用
例子	BASE(0) UNITS=100 DPOS=0 SPEED=1000 '轴速度 ACCEL=1000 '轴加速度 DECEL=1000 SRAMP=100 MERGE=ON '开启连续插补，多段运动间速度连续 TRIGGER '自动触发示波器 MOVE(2000) MOVELIMIT(100) '在两个运动的中间降速到 100 MOVE(2000) 用 MOVELIMIT 限制速度时，插补速度 MSPEED(0)垂直刻度 1000



不限速度时



相关指令	<u>CORNER_MODE</u>
------	------------------------------------

7.4 同步运动指令

CONNECT -- 同步运动

类型	同步运动指令
描述	将当前轴的目标位置与 driving_axis 轴的测量位置通过电子齿轮连接。 连接的是脉冲个数，要考虑不同轴 UNITS 的比例。取消连接时用 CANCEL。 假设连接轴 0 的 UNIST 为 10，被连接轴 1 的 UNITS 为 100。 使用 CONNECT 连接，比例 ratio 为 1，当轴 0 运动 $s1=100$ 时，轴 1 运动 $=s1*UNITS(0)*ratio/UNITS(1)$，此时运动 10。 比率可以通过重复调用指令动态变化。 常用于手轮使用。
语法	CONNECT (ratio, driving_axis) AXIS (连接轴轴号) ratio: 比率，可正可负，注意是脉冲个数的比例 driving_axis: 被连接轴的轴号，手轮时为编码器轴

适用控制器	通用
例子	RAPIDSTOP(2) WAIT IDLE(0) WAIT IDLE(1) BASE(0,1) ATYPE=1,1 UNITS=10,100 DPOS=0,0 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 TRIGGER MOVE(100) AXIS(1) WAIT IDLE(1) CONNECT(1,1) AXIS(0) MOVE(100) AXIS(1) '自动触发示波器' '轴 1 运动 100，此时轴 0 不动' '轴 0 连接到轴 1，比例为 1' '轴 1 运动 100，轴 0 运动 1000' 运动轨迹 DPOS(0)垂直刻度 1000 DPOS(1)垂直刻度 500 1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:1000.10 Max:200.00 0 1000 2000 3000 4000
相关指令	CLUTCH_RATE , CONNPATH

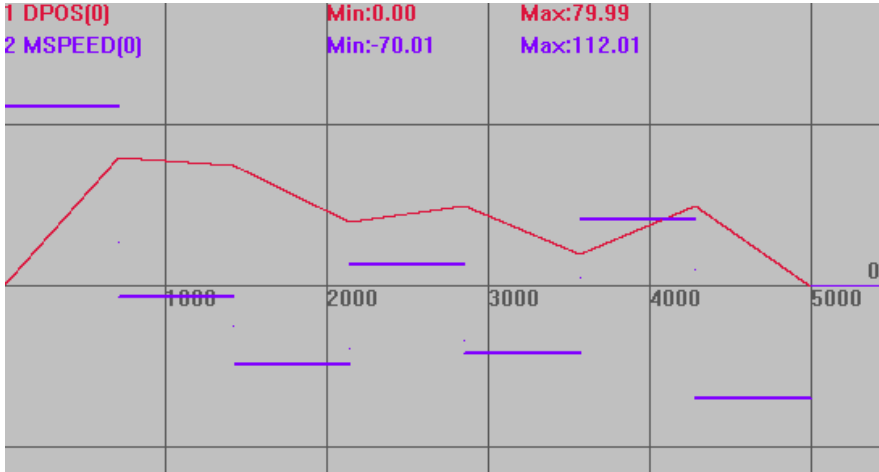
CONNPATH -- 同步运动 2

类型	同步运动指令
描述	将当前轴的目标位置与 driving_axis 轴的插补矢量长度通过电子齿轮连接。 需要连接到插补运动的主轴才能与插补矢量长度建立连接，连接到从轴的效果与 CONNECT 相同。 连接的是脉冲个数，要考虑不同轴 UNITS 的比例。取消连接时用 CANCEL。 比率可以通过重复调用指令动态变化。
语法	CONNPATH (ratio, driving_axis) AXIS (连接轴轴号) Ratio: 比率，可正可负，注意是脉冲个数的比例

	driving_axis: 被连接轴的轴号，手轮时为编码器轴
适用控制器	通用
例子	RAPIDSTOP(2) WAIT IDLE(0) WAIT IDLE(1) BASE(0,1,2) DPOS=0,0,0 ATYPE=1,1,1 UNITS=100,100,100 SPEED=100,100,100 ACCEL=1000,1000,1000 DECEL=1000,1000 ,1000 TRIGGER '自动触发示波器 CONNPATH(1,0) AXIS(2) '轴 2 连接到插补运动的主轴轴 0，比例为 1 MOVE(100,100) '插补运动 运动轨迹 DPOS(0)垂直刻度 100，偏移-50 DPOS(1)垂直刻度 100，偏移-50 DPOS(2)垂直刻度 100，无偏移 1 DPOS[0] 2 DPOS[1] 3 DPOS[2] Min:0.00 Min:0.00 Min:0.00 Max:100.00 Max:100.00 Max:141.41 100020003000 0
相关指令	CLUTCH_RATE ， CONNECT

CAM -- 凸轮表运动

类型	同步运动指令
描述	CAM 指令根据存储在 TABLE 中的数据来决定轴的运动，这些 Table 数据值对应运动轨迹的位置，是相对于运动起始点的绝对位置。

	注：两个或多个 CAM 指令可以同时使用同一段 TABLE 数据区进行操作。 运动的总时间由设置速度和第四个参数决定，运动的实际速度根据 TABLE 轨迹与时间自动匹配。 TBALE 数据需要手动设置，第一个数据为引导点，建议设为 0。 TABLE 数据*table multiplier 这个比例=发出的脉冲数。 请确保指令传递的距离参数*units 是整数个脉冲，否则出现浮点数会导致运动有细微误差。
语法	CAM(start point, end point, table multiplier, distance) start point: 起始点 TABLE 编号，存储第一个点的位置 end point: 结束点 TABLE 编号 table multiplier: 位置乘以这个比例，一般设为脉冲当量值 distance: 参考运动的距离 总时间=distance/轴 speed
适用控制器	通用
例子	例一： RAPIDSTOP(2) WAIT IDLE(0) BASE(0) UNITS=100 DPOS=0 ACCEL=1000 DECEL=1000 SPEED=100 TABLE(10,0,80,75,40,50,20,50,0) 'table 从 10 开始存数据 TRIGGER '自动触发示波器 CAM(10,17,100,500) '运动轨迹为 table10 到 17，运动总时间为 500/100=5s 轨迹与速度 DPOS(0)垂直刻度 100 MSPEED(0)垂直刻度 100 

例二：CAM 在高速高精度运动上的应用

DIM num_p,scale,m,t '变量定义

num_p=100

scale=500

FOR p=0 TO num_p

TABLE(p,((-SIN(PI*2*p/num_p)/(PI*2))+p/num_p)*scale) 'table 存储凸轮表运动参数

NEXT

RAPIDSTOP(2)

WAIT IDLE(0)

BASE(0) '选择轴 0

DEFPOS(0)

SERVO=ON

UNITS=500

SPEED=1000

ACCEL=1000000

DECEL=1000000

TRIGGER

m=10 '代表距离的倍数

t=0.3 '运行时间

SPEED=1000

CAM(0,100,m,SPEED*t)

WAIT IDLE

m=10

t=0.3

SPEED=1000

CAM(0,100,-m,SPEED*t)

WAIT IDLE

m=10

t=0.2

SPEED=500

CAM(0,100,m,SPEED*t)

WAIT IDLE

m=10

t=0.2

SPEED=500

CAM(0,100,-m,SPEED*t)

WAIT IDLE

```

m=20
t=0.3
SPEED=1000
CAM(0,100,m,SPEED*t)
WAIT IDLE

```

```

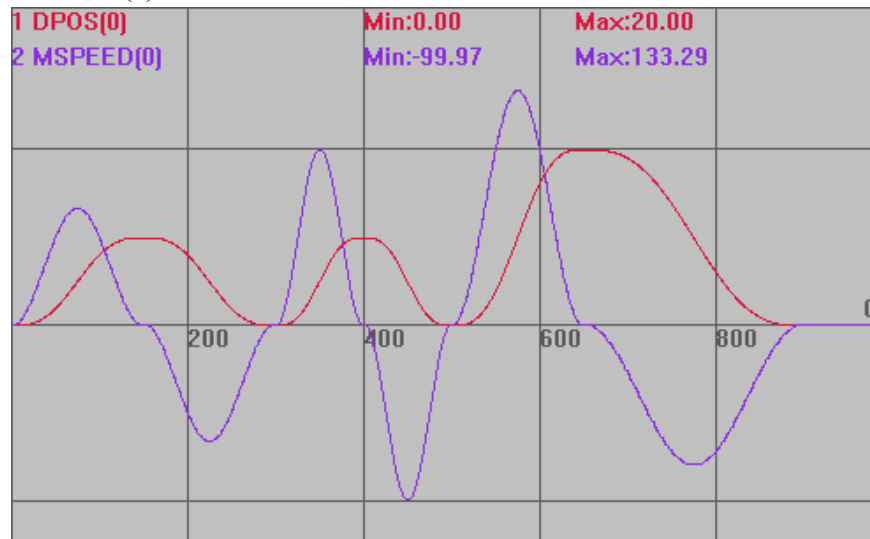
m=20
t=0.5
SPEED=500
CAM(0,100,-m,SPEED*t)
WAIT IDLE

```

```

插补轨迹
DPOS(0)垂直刻度 20
MSPEED(0)垂直刻度 100

```



例三：连续凸轮

```

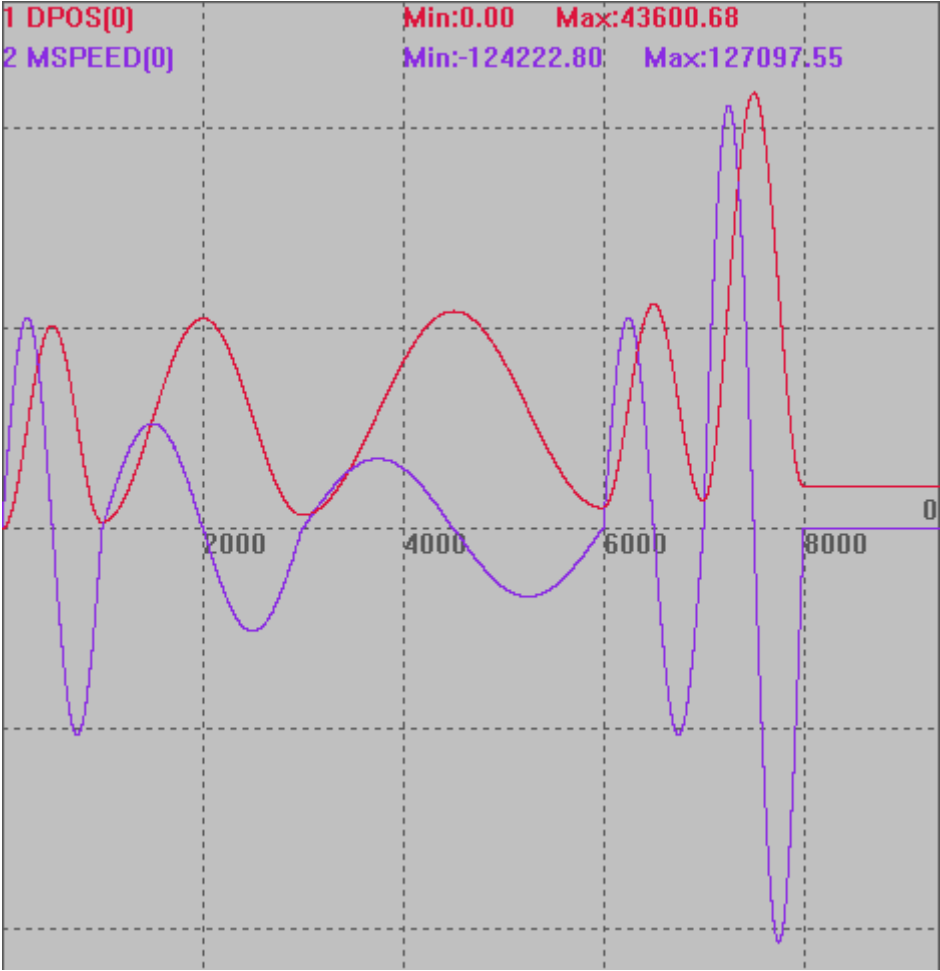
RAPIDSTOP(2)
WAIT IDLE(0)
BASE(0)
UNITS=100
DPOS=0
ACCEL=1000
DECEL=1000
SPEED=100

```

```

DIM rad,x,deg
FOR deg= 0 TO 360 STEP 1      '构造 0-360 度的凸轮数据
    rad = deg * 2 * PI / 360
    x = deg * 2 + 10000 * ( 1 - COS (rad))
    TABLE (deg ,x)

```

	<pre>print deg,x NEXT deg TRIGGER '自动触发示波器 CAM(0,360,100,100) '运动轨迹为 table 0 到 360，运动总时间为 100/100=1s CAM(0,360,100,200) '运动轨迹为 table 0 到 360，运动总时间为 200/100=2s CAM(0,360,100,300) '运动轨迹为 table 0 到 360，运动总时间为 300/100=3s WAIT UNTIL REMAIN_BUFFER(1) > 0 '等待缓冲空间空了再存运动指令 CAM(0,360,100,100) '运动轨迹为 table 0 到 360，运动距离为 100*table 数据/units(0) CAM(0,360,200,100) '运动轨迹为 table 0 到 360，运动总时间 200*table 数据/units(0) DPOS(0)垂直刻度 20000 MSPEED(0)垂直刻度 60000</pre> 
相关指令	CAMBOX

CAMBOX -- 跟随凸轮表运动

类型	同步运动指令
描述	CAMBOX 指令根据存储在 TABLE 中的数据来决定轴的运动，这些 Table 数据值对

	应运动轨迹的位置，是相对于运动起始点的距离。跟随轴的运动根据参考轴的运动来进行。 注：两个或多个 CAMBOX 指令可以同时使用同一段 Table 数据区进行操作。 当前轴运动的总时间由参考轴的运动距离和轴速度确定，速度自动匹配。 Table 数据需要手动设置，第一个数据为引导点，建议设为 0。 Table 数据*table multiplier 这个比例=发出的脉冲数。 请确保指令传递的距离参数*units 是整数个脉冲，否则出现浮点数会导致运动有细微误差。																				
语法	CAMBOX(start_point,end_point,table_multiplier,link_distance,link_axis[,link_options][,link_pos][,link_offpos]) start point: 起始点 TABLE 编号，存储第一个点的位置 end point: 结束点 TABLE 编号 table multiplier: 位置乘以这个比例，一般设为脉冲当量值 link_distance: 参考轴运动的距离 link_axis: 参考轴轴号 link_options: 与参考轴的连接方式，不同的二进制位代表不同的意义 <table border="1"> <thead> <tr> <th>位</th><th>意义</th></tr> </thead> <tbody> <tr> <td>bit0</td><td>当参考轴 MARK 信号事件触发时，当前轴与参考轴开始进行连接运动</td></tr> <tr> <td>bit1</td><td>当参考轴运动到设定的绝对位置时，当前轴与参考轴开始连接运动</td></tr> <tr> <td>bit2</td><td>自动重复连续双向运行。(通过设置 REP_OPTION 指令的 bit1 为 1，可以取消重复)</td></tr> <tr> <td>bit3</td><td>加速度平滑</td></tr> <tr> <td>bit4</td><td>从中间某个位置启动，配合掉电中断实现恢复凸轮</td></tr> <tr> <td>bit5</td><td>只有参考轴的正向运动才连接</td></tr> <tr> <td>bit8</td><td>当参考轴 MARKB 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持</td></tr> <tr> <td>bit9</td><td>当参考轴 MARKC 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持；支持 4 通道锁存的控制器适用</td></tr> <tr> <td>bit10</td><td>当参考轴 MARKD 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持；支持 4 通道锁存的控制器适用</td></tr> </tbody> </table> link_pos: 当 link_options 参数设置为 2 时，该参数表示连接开始启动的绝对位置 link_offpos: 当 link_options 参数 bit4 置为 1 时，该参数表示主轴已经运行完的相对位置	位	意义	bit0	当参考轴 MARK 信号事件触发时，当前轴与参考轴开始进行连接运动	bit1	当参考轴运动到设定的绝对位置时，当前轴与参考轴开始连接运动	bit2	自动重复连续双向运行。(通过设置 REP_OPTION 指令的 bit1 为 1，可以取消重复)	bit3	加速度平滑	bit4	从中间某个位置启动，配合掉电中断实现恢复凸轮	bit5	只有参考轴的正向运动才连接	bit8	当参考轴 MARKB 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持	bit9	当参考轴 MARKC 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持；支持 4 通道锁存的控制器适用	bit10	当参考轴 MARKD 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持；支持 4 通道锁存的控制器适用
位	意义																				
bit0	当参考轴 MARK 信号事件触发时，当前轴与参考轴开始进行连接运动																				
bit1	当参考轴运动到设定的绝对位置时，当前轴与参考轴开始连接运动																				
bit2	自动重复连续双向运行。(通过设置 REP_OPTION 指令的 bit1 为 1，可以取消重复)																				
bit3	加速度平滑																				
bit4	从中间某个位置启动，配合掉电中断实现恢复凸轮																				
bit5	只有参考轴的正向运动才连接																				
bit8	当参考轴 MARKB 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持																				
bit9	当参考轴 MARKC 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持；支持 4 通道锁存的控制器适用																				
bit10	当参考轴 MARKD 信号事件触发时，当前轴与参考轴开始进行连接运动，锁存轴号为参考轴的轴号，需要最新固件支持；支持 4 通道锁存的控制器适用																				
适用控制器	通用																				
例子	ERRSWITCH = 3 RAPIDSTOP(2) WAIT IDLE(0) WAIT IDLE(1) BASE(0,1) '选择轴号 ATYPE=1,1 '脉冲方式步进或伺服																				

```

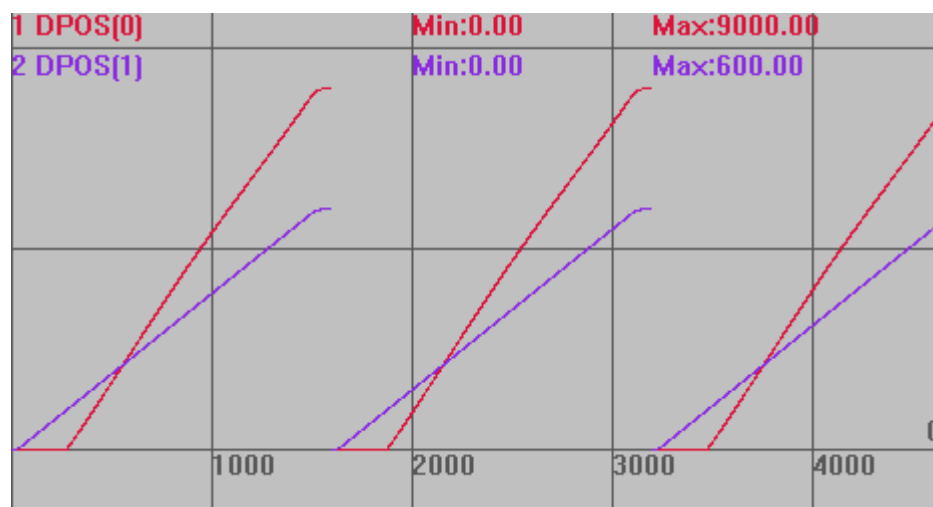
DPOS = 0,0
UNITS = 100,100 '脉冲当量
SPEED = 200,200
ACCEL = 2000,2000
DECEL = 2000,2000
'计算 TABLE 的数据
DIM deg, rad, x, stepdeg
stepdeg = 2 '可以通过这个来修改段数，段数越多速度越平稳
FOR deg=0 TO 360 STEP stepdeg
    rad = deg * 2 * PI/360 '转换为弧度
    x = deg * 25 + 10000 * (1-COS(rad))/100
    TABLE(deg/stepdeg,x) '存储 TABLE
    TRACE deg/stepdeg,x
NEXT deg
TRIGGER '自动触发示波器
WHILE 1 '循环运动
    IF IN(0) = ON THEN '输入 0 有效启动运动
        DPOS = 0,0
        CAMBOX(0,360/stepdeg, 100, 500, 1,2,100) AXIS(0) '参考轴轴 1 运动到 100
        位置时，跟随轴轴 0 启动
        MOVE(600) AXIS(1)
        WAIT UNTIL IDLE AND IDLE(1) '等待运动停止
        DELAY(100) '延时
    ENDIF
WEND
END '停止当前任务

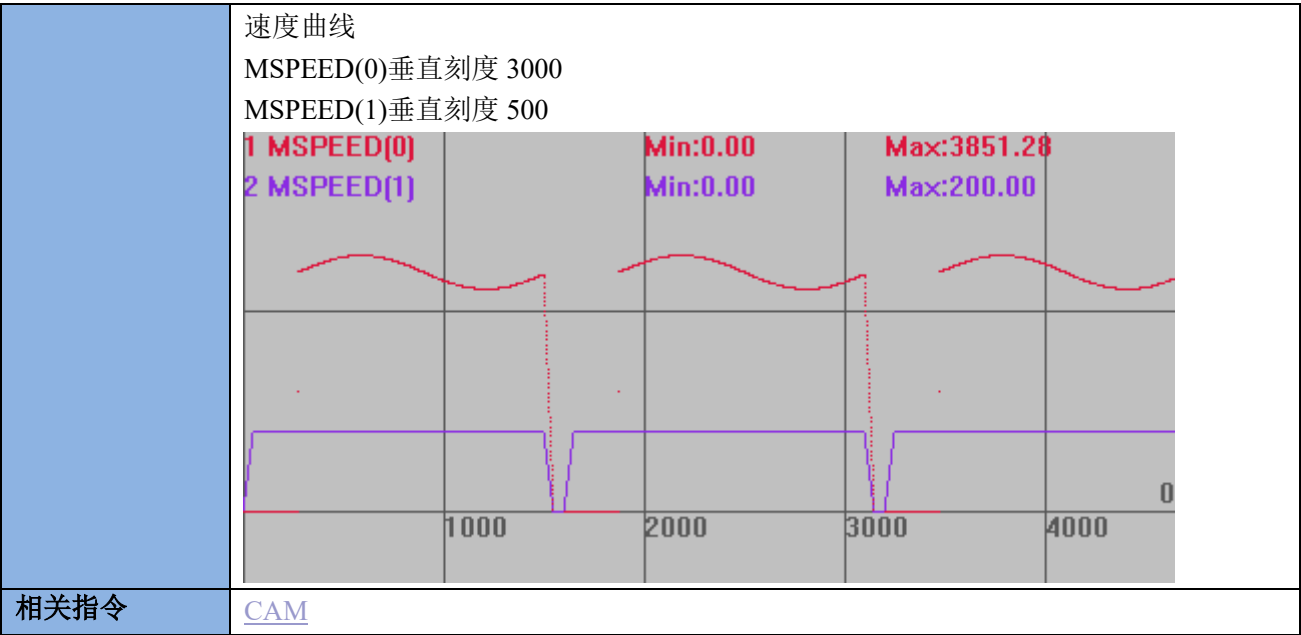
```

运动轨迹

DPOS(0)垂直刻度 5000

DPOS(1)垂直刻度 500

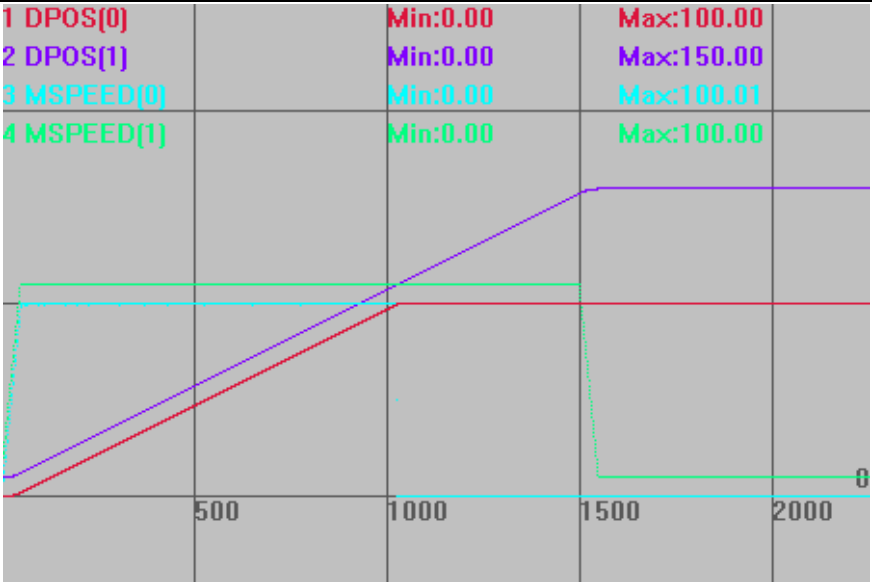




MOVELINK -- 自动凸轮

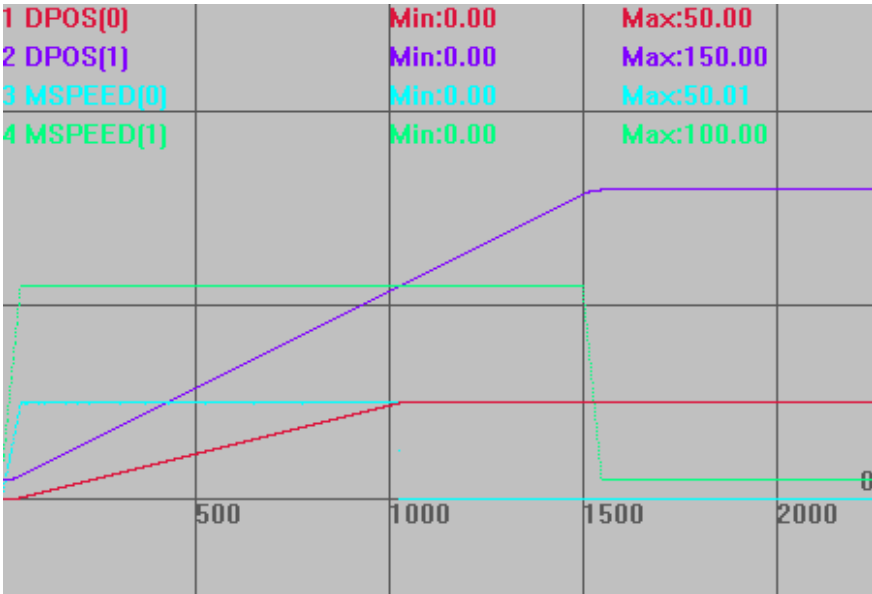
类型	同步运动指令												
描述	此指令用于自定义的凸轮运动，该运动带有可设置的加减速阶段。 被连接轴为参考轴，连接轴为跟随轴。 连接轴的距离分成 3 个阶段应用于参考轴的运动，分别是加速部分、匀速部分和减速部分。 在加速和减速阶段为了与速度匹配，link distance（基本轴运动距离）必须是 distance（跟随轴运动距离）的两倍。 请确保指令传递的距离参数*units 是整数个脉冲，否则出现浮点数会导致运动有细微误差。												
语法	MOVELINK (distance, link dist, link acc, link dec, link axis[,link options] [,link pos][,link offpos]) distance: 从连接开始到结束，跟随轴移动的距离，此参数可正可负，为正数正方向跟随，为负数负方向跟随，采用 units 单位 link dist: 参考轴在连接的整个过程中移动的绝对距离，采用 units 单位 link acc: 在跟随轴加速阶段，参考轴移动的绝对距离，采用 units 单位 link dec: 在跟随轴减速阶段，参考轴移动的绝对距离，采用 units 单位 注：如果参数 3 和参数 4 的和大于第 2 个参数，他们会被自动按比例减小，使其和值与第 2 个参数值相等 link axis: 参考轴的轴号 link options: 连接模式选项，不同的二进制位代表不同的意义 <table><tr><td>模式</td><td>位</td><td>描述</td></tr><tr><td>1</td><td>位 0</td><td>连接精确开始于参考轴上 MARK 事件被触发的时刻</td></tr><tr><td>2</td><td>位 1</td><td>连接开始于参考轴到达一个绝对位置时(见 link pos 参数描述)</td></tr><tr><td>4</td><td>位 2</td><td>当此位被设置时,MOVELINK 会自动重复执行并且可以反向(这</td></tr></table>	模式	位	描述	1	位 0	连接精确开始于参考轴上 MARK 事件被触发的时刻	2	位 1	连接开始于参考轴到达一个绝对位置时(见 link pos 参数描述)	4	位 2	当此位被设置时,MOVELINK 会自动重复执行并且可以反向(这
模式	位	描述											
1	位 0	连接精确开始于参考轴上 MARK 事件被触发的时刻											
2	位 1	连接开始于参考轴到达一个绝对位置时(见 link pos 参数描述)											
4	位 2	当此位被设置时,MOVELINK 会自动重复执行并且可以反向(这											

			个模式可以通过设置轴参数 REP_OPTION 的第 1 位为 1 来清除)
	8	位 3	当设置时,采用 S 曲线加减速。 20170502 以上固件支持
	16	位 4	从中间某个位置启动, 配合掉电中断实现恢复跟随
	32	位 5	只有参考轴为正向运动才连接
	256	位 8	连接精确开始于参考轴上 MARKB 事件被触发的时刻, 需要最新固件支持
link pos: 当 link options 参数设置为 2 时, 该参数表示基本轴在该绝对位置值时, 连接开始 link offpos: 当 link_options 参数 bit4 置为 1 时, 该参数表示主轴已经运行完的相对位置。20170428 以上固件支持			
适用控制器	通用		
例子	例一: RAPIDSTOP(2) WAIT IDLE(0) WAIT IDLE(1) BASE(0,1) '轴 0 为跟随轴, 轴 1 为参考轴 UNITS=100,100 ATYPE=1,1 DPOS=0,0 SPEED=100,100 ACCEL=2000,2000 DECEL=2000,2000 TRIGGER '自动触发示波器 MOVELINK(100,100,0,0,1) AXIS(0) '不设置加减速阶段时, 效果与 CONNECT 相同, 区别在不需要考虑 UNITS 的不同, 且不会有累积误差。此时运动比例 1:1 MOVE(150) AXIS(1) '轴 1 运动 150, 轴 0 跟随轴 1 运动完 100 插补轨迹与速度曲线 DPOS(0)垂直刻度 100, 无偏移 DPOS(1)垂直刻度 100, 偏移 10 MSPEED(0)垂直刻度 100, 无偏移 MSPEED(1)垂直刻度 100, 偏移 10		



MOVELINK(50,100,0,0,1)

竖直刻度同上



例二：追剪应用

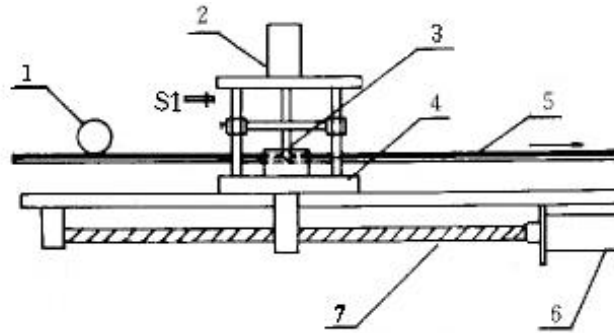


图1 追剪系统原理图

1—测长轮；2—液压缸；3—刀具；4—工作台；
5—型材；6—伺服电机；7—丝杠

型材持续运动，工作台先静止；直到型材持续运动了某段距离，工作台开始加速；待工作台速度与型材一致，然后开关 S1 工作刀具下剪，剪切完后刀具回升；工作台开始减速，然后退回起始点。重复过程，剪切得到设定长度的型材。

假设要切的型材长度为 4m，工作台运行距离 1m，轴 1 为基本轴（型材传送），轴 0 为跟随轴（追剪工作台），OUT0 口控制刀具，飞剪部分程序如下：

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

BASE(0,1)

UNITS=10000,10000

ATYPE=1,1

DPOS=0,0

SPEED=1,1 '型材运行速度 1m/s,60m/min

ACCEL=2,2

DECEL=2,2

VMOVE(1) AXIS(1) '型材持续运动

TRIGGER '自动触发示波器

WHILE 1

BASE(0)

MOVELINK(0,1,0,0,1) AXIS(0) '型材运动 1m 前，工作台静止

MOVELINK(0.4,0.8,0.8,0,1) AXIS(0) '工作台加速阶段

MOVELINK(0.2,0.2,0,0,1) AXIS(0) '速度同步跟随 0.2m

MOVE_OP2(0,on,1000) '刀具下剪，1s 后回升(时间要计算好)

MOVELINK(0.4,0.8,0,0.8,1) AXIS(0) '工作台减速阶段

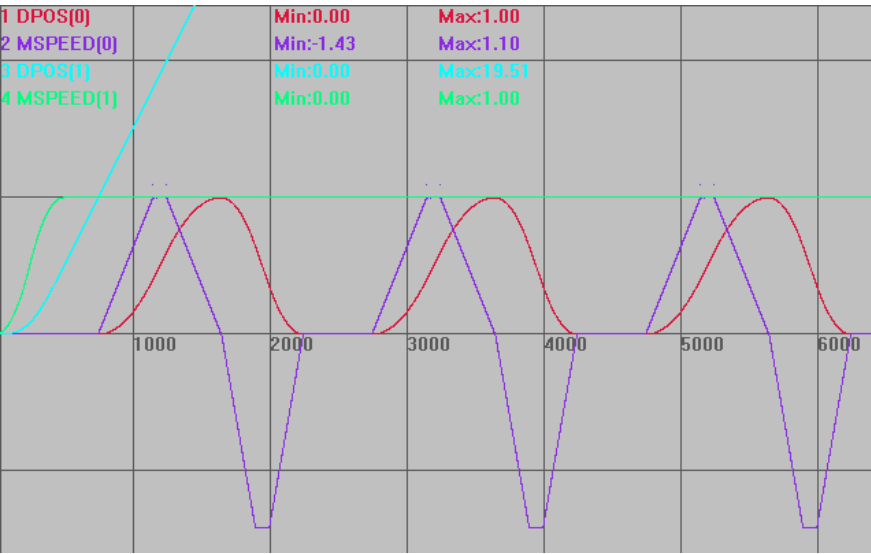
MOVELINK(-1,1.2,0.5,0.5,1) AXIS(0) '工作台回到起始点

WEND

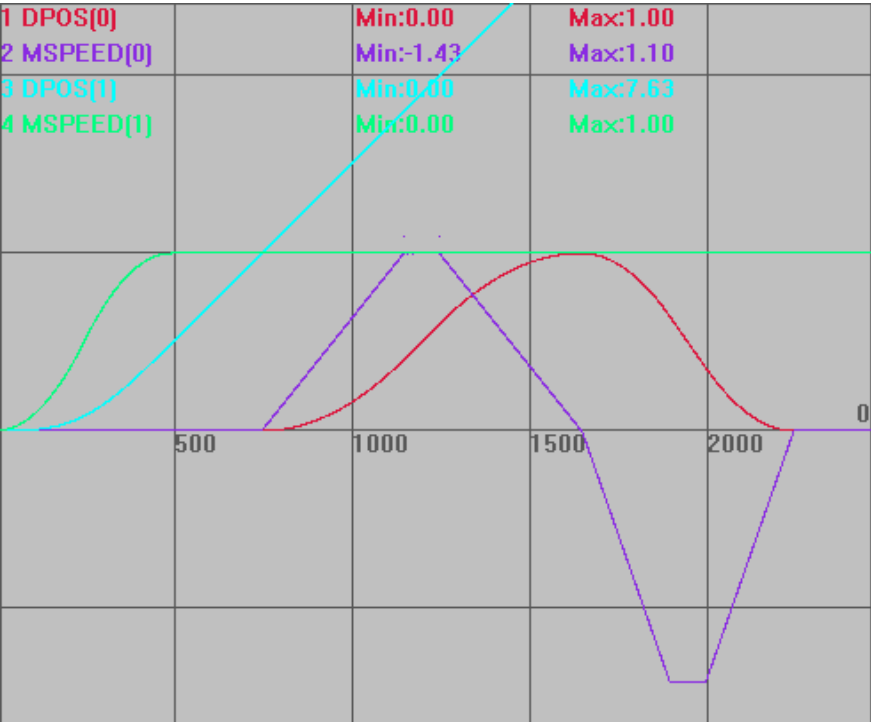
运动轨迹和速度曲线：

DPOS(0)垂直刻度 1，无偏移

MSPEED(0)垂直刻度 1，无偏移
DPOS(1)垂直刻度 1，无偏移
MSPEED(1)垂直刻度 1，无偏移



一个周期内的运行曲线：



工作台(跟随轴)的运动距离：0.4(加速阶段)+0.2(跟随同步)+0.4(减速阶段)=1m 单位，然后-1m 返回运动。

型材(参考轴)的运动距离：1+0.8+0.2+0.8+1.2=4m 单位，全程匀速。

例三：设置 link options bit3=1 时，从轴追剪轴采用 S 曲线加减速

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

```

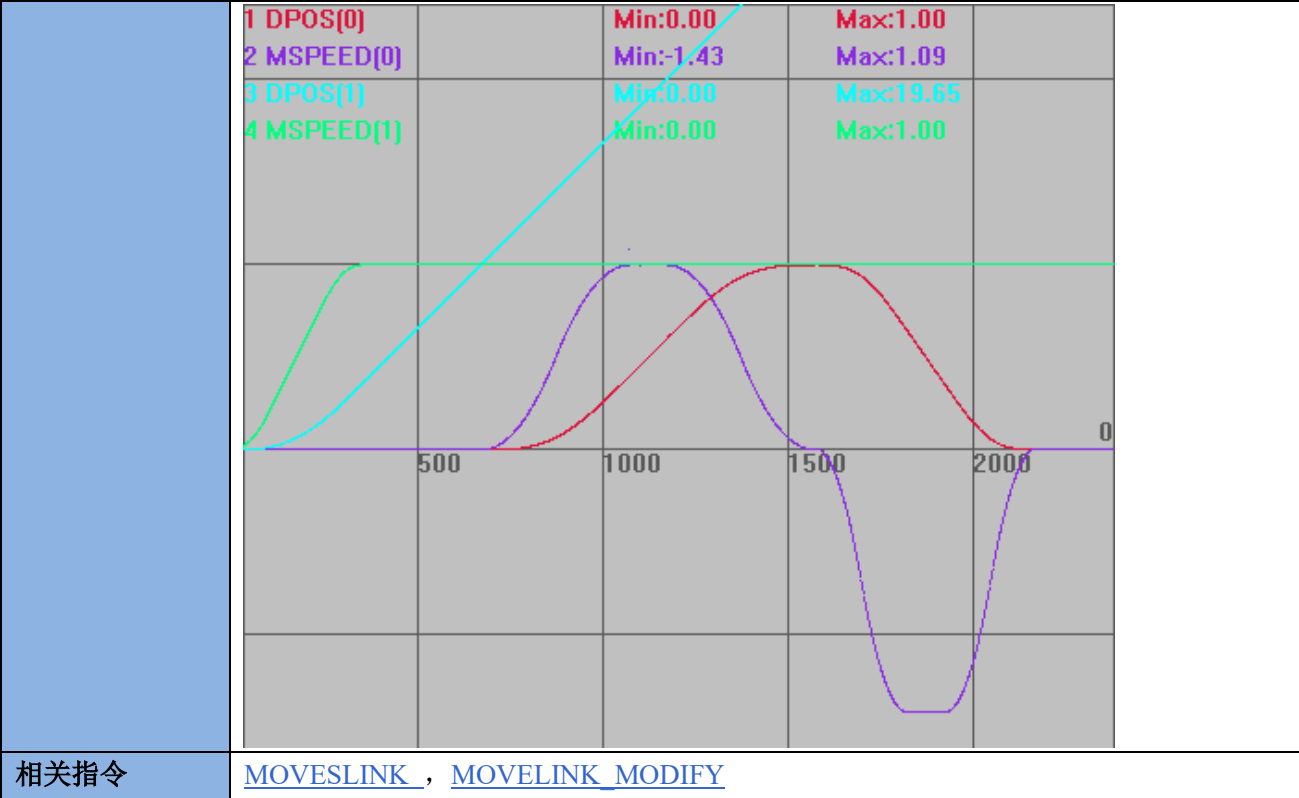
DATUM(0)
BASE(0,1)
UNITS=10000,10000
ATYPE=1,1
DPOS=0,0
SPEED=1,1      "型材运行速度 1m/s,60m/min
ACCEL=2,2
DECEL=2,2
SRAMP=200,200

VMOVE(1) AXIS(1)    '型材持续运动
TRIGGER              '自动触发示波器

WHILE 1
    BASE(0)
    MOVELINK(0,1,0,0,1,8) AXIS(0)  '型材运动 1m 前，工作台静止
    MOVELINK(0.4,0.8,0.8,0,1,8) AXIS(0) '工作台加速阶段
    MOVELINK(0.2,0.2,0,0,1,8) AXIS(0)  '速度同步跟随 0.2m
    MOVE_OP2(0,on,1000)  '刀具下剪，1s 后回升(时间要计算好)
    MOVELINK(0.4,0.8,0,0.8,1,8) AXIS(0) '工作台减速阶段
    MOVELINK(-1,1.2,0.5,0.5,1,8) AXIS(0) '工作台回到起始点
WEND

运动轨迹和速度曲线：
DPOS(0)垂直刻度 1，无偏移
MSPEED(0)垂直刻度 1，无偏移
DPOS(1)垂直刻度 1，无偏移
MSPEED(1)垂直刻度 1，无偏移

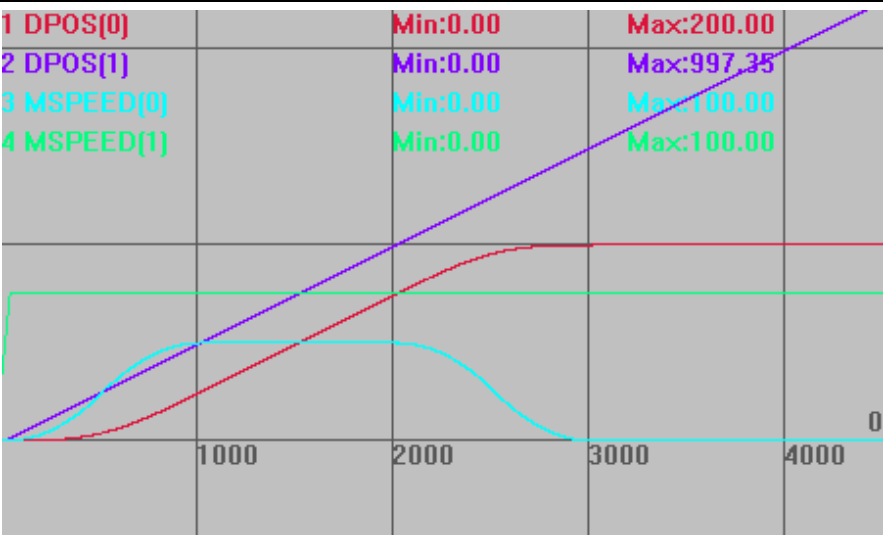
```



MOVESLINK -- 自动凸轮 2

类型	同步运动指令
描述	此指令用于自定义的凸轮运动，该运动自动规划中间曲线，不用计算凸轮表。 被连接轴为参考轴，连接轴为跟随轴。 在加速和减速阶段为了与速度匹配，下一条 MOVESLINK 的 start sp 必须与当前 MOVESLINK 的 end sp 相同。 请确保指令传递的距离参数*units 是整数个脉冲，否则出现浮点数会导致运动有细微误差。
语法	MOVESLINK (distance,link dist,start sp,end sp,link axis[,link options][,link pos][, link offpos]) [, link options] [, link pos] 可选参数不填时，逗号不能省掉，控制器根据参数的位置来判断是什么参数。 distance: 从连接开始到结束，跟随轴移动的距离，此参数可正可负，为正数正方向跟随，为负数负方向跟随，采用 units 单位 link dist: 参考轴在连接的整个过程中移动的绝对距离，采用 units 单位 start sp: 启动时跟随轴和参考轴的速度比例，units/units 单位，负数表示跟随轴负向运动 end sp: 结束时跟随轴和参考轴的速度比例，units/units 单位，负数表示跟随轴负向运动 注：当 start sp = end sp = distance/link dist 时，匀速运动 link axis: 参考轴的轴号 link options: 连接模式选项，不同的二进制位代表不同的意义

	模式	位	描述
	1	位 0	连接精确开始于参考轴上 MARK 事件被触发的时刻
	2	位 1	连接开始于参考轴到达一个绝对位置时(见 link pos 参数描述)
	4	位 2	当此位被设置时, MOVELINK 会自动重复执行并且可以反向(这个模式可以通过设置轴参数 REP_OPTION 的第 1 位为 1 来清除)
	16	位 4	使用 link offpos 从中间来启动, 配合掉电中断实现恢复, 20170428 以上固件支持
	32	位 5	只有参考轴为正向运动才连接
	256	位 8	连接精确开始于参考轴上 MARKB 事件被触发的时刻, 需要最新固件支持
link pos: 当 link options 参数设置为 2 时, 该参数表示参考轴在该绝对位置值时, 连接开始 link offpos: 当 link_options 参数 bit4 置为 1 时, 该参数表示主轴已经运行完的相对位置。20170428 以上固件支持。			
适用控制器	通用		
例子	功能与 MOVELINK 相同, 仅是参数设置区别 例一: RAPIDSTOP(2) WAIT IDLE(0) WAIT IDLE(1) DATUM(0) BASE(0,1) UNITS=100,100 ATYPE=1,1 DPOS=0,0 SPEED=100,100 ACCEL=2000,2000 DECEL=2000,2000 TRIGGER '自动触发示波器 MOVESLINK(50,100,0,1,1) AXIS(0) '轴 0 跟踪轴 1 运动, 从速度 0 到速度一致 MOVESLINK(100,100,1,1,1) AXIS(0) '轴 0 跟踪轴 1 运动, 匀速 100units MOVESLINK(50,100,1,0,1) AXIS(0) '轴 0 跟踪轴 1 运动, 减速到 0 VMOVE(1) AXIS(1) 运动轨迹和速度曲线 DPOS(0)垂直刻度 200, 无偏移 DPOS(1)垂直刻度 200, 无偏移 MSPEED(0)垂直刻度 200, 无偏移 MSPEED(1)垂直刻度 200, 偏移 50		



例二：追剪

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

DATUM(0)

BASE(0,1)

UNITS=10000,10000

ATYPE=1,1

DPOS=0,0

SPEED=1,1

ACCEL=2,2

DECEL=2,2

SRAMP=200,200

TRIGGER '自动触发示波器

VMOVE(1) AXIS(1)

WHILE 1

MOVESLINK(0,1,0,0,1) AXIS(0) '型材运动 1 单位前，工作台静止

MOVESLINK(0.4,0.8,0,1,1) AXIS(0) '工作台加速阶段

MOVESLINK(0.2,0.2,1,1,1) AXIS(0) '速度跟随阶段

MOVESLINK(0.4,0.8,1,0,1) AXIS(0) '工作台减速阶段

MOVESLINK(-1,1.2,0,0,1) AXIS(0) '工作台回到起始点

WEND

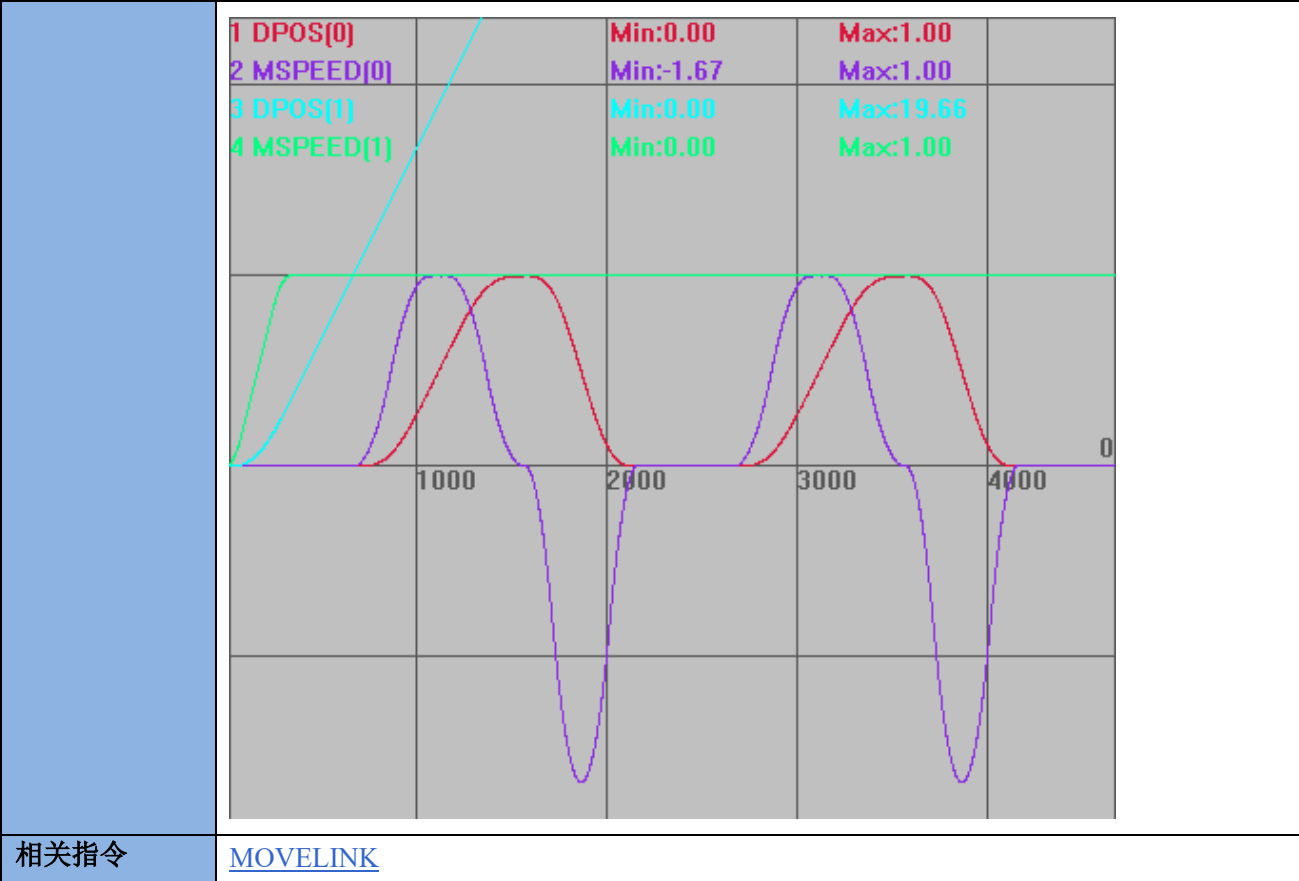
运动轨迹和速度曲线

DPOS(0)垂直刻度 1，无偏移

DPOS(1)垂直刻度 1，无偏移

MSPEED(0)垂直刻度 1，无偏移

MSPEED(1)垂直刻度 1，无偏移



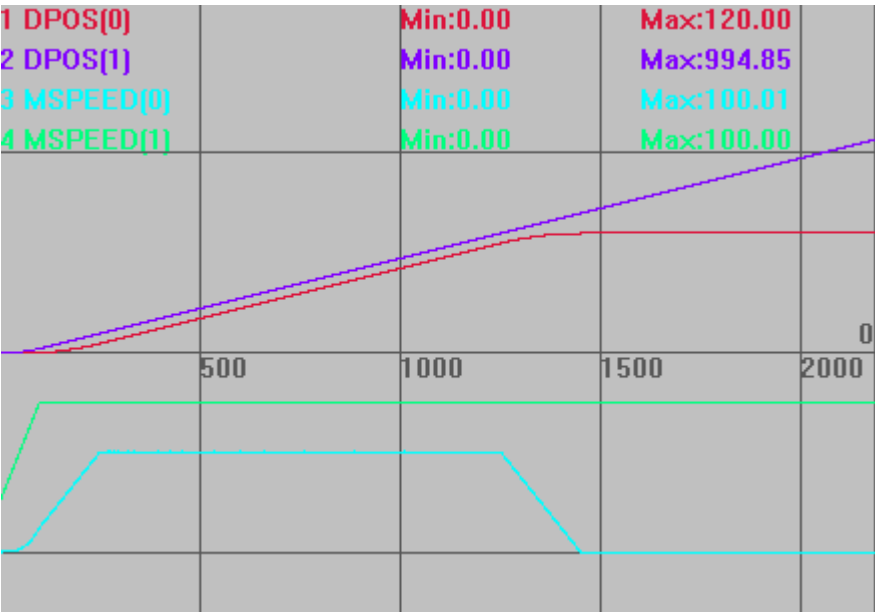
相关指令 [MOVELINK](#)

MOVELINK_MODIFY -- 同步距离修改

类型	轴参数
描述	相对修改 MOVELINK 指令的同步区长度。 带入运动缓冲，只在同步段后设置生效。
语法	VAR1 = MOVELINK_MODIFY, MOVELINK_MODIFY = expression
适用控制器	20160926 以后固件版本支持。
例子	例一： RAPIDSTOP(2) WAIT IDLE(0) WAIT IDLE(1) BASE(0,1) UNITS=100,100 ATYPE=1,1 DPOS=0,0 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 TRIGGER '自动触发示波器 未修改同步距离 MOVELINK(10,20,20,0,1) '工作台加速阶段

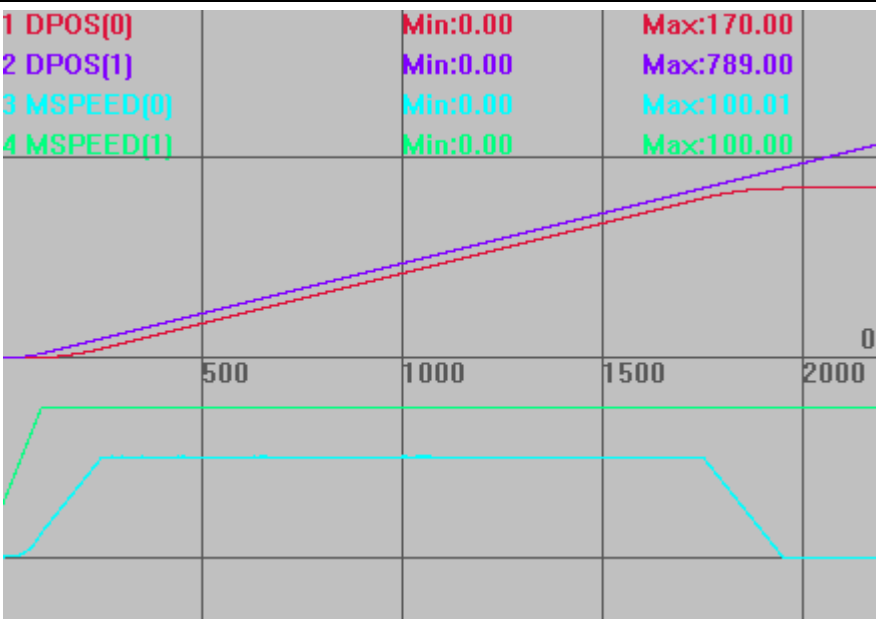
```
MOVELINK(100,100,0,0,1) '同步阶段 100
MOVELINK(10,20,0,20,1)   '减速阶段
VMOVE(1)  AXIS(1)
```

运动轨迹和速度曲线
DPOS(0)垂直刻度 200，无偏移
DPOS(1)垂直刻度 200，无偏移
MSPEED(0)垂直刻度 200，偏移-200
MSPEED(1)垂直刻度 200，偏移-150



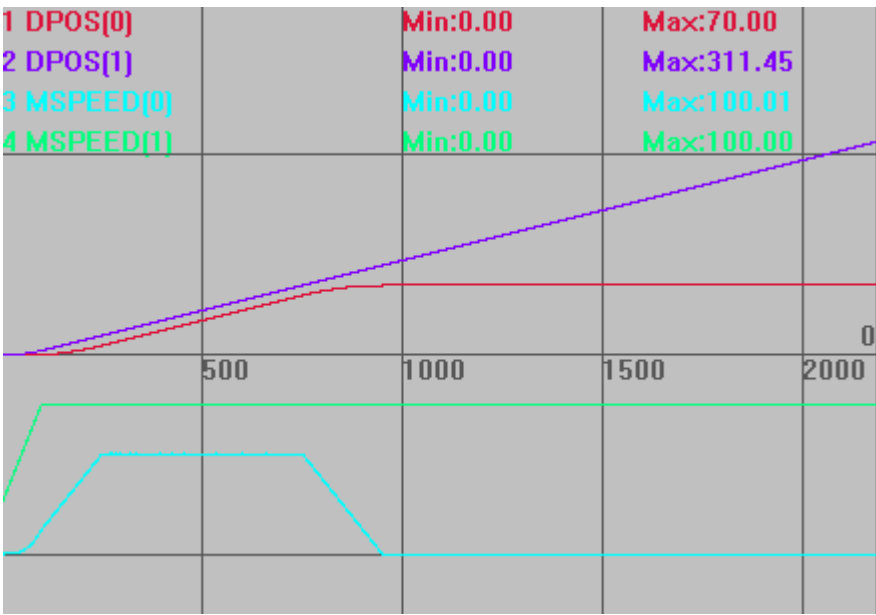
其他条件同上，增加同步距离

```
MOVELINK(10,20,20,0,1)   '工作台加速阶段
MOVELINK(100,100,0,0,1) '同步阶段 100
MOVELINK_MODIFY = 50      '修改同步段为 100+50
MOVELINK(10,20,0,20,1)   '减速阶段
```



其他条件同上，减少同步距离

```
MOVELINK(10,20,20,0,1) '工作台加速阶段
MOVELINK(100,100,0,0,1) '同步阶段 100
MOVELINK_MODIFY = -50 '修改同步段为 100-50
MOVELINK(10,20,0,20,1) '减速阶段
```



注意，只能在同步段后使用此指令，在加减速段使用会报错，无法修改。

```
MOVELINK(10,20,20,0,1) '工作台加速阶段
MOVELINK_MODIFY = 50
MOVELINK(100,100,0,0,1) '同步阶段 100
```

Axis:0 MOVELINK_MODIFY:50.000 failed.

例二：从轴追剪轴采用 S 曲线加减速

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

DATUM(0)

BASE(0,1)

UNITS=10000,10000

ATYPE=0,0

DPOS=0,0

SPEED=1,1 '型材运行速度 1m/s,60m/min

ACCEL=2,2

DECEL=2,2

SRAMP=200,200

STOPTASK 1

RUNTASK 1,Task_FlyShear

DELAY(200)

VMOVE(1) AXIS(1) '型材持续运动

TRIGGER '自动触发示波器

END

Task_FlyShear:

WHILE 1

BASE(0)

'MOVELINK_MODIFY=0 '先清空

MOVELINK(3,4,1,1,1,8) AXIS(0)

WAIT IDLE(0)

BASE(0)

DPOS=0

'MOVELINK_MODIFY=0 '先清空

MOVELINK(3,4,1,1,1,8) AXIS(0)

WAIT UNTIL MPOS(0)>1 '等待跟随轴距离>2

MOVELINK_MODIFY=-1 '把跟随轴距离减少 1

WAIT UNTIL MOVELINK_MODIFY=0 '等待同步偏移完成

WAIT IDLE(0)

BASE(0)

DPOS=0

'MOVELINK_MODIFY=0 '先清空

MOVELINK(3,4,1,1,1,8) AXIS(0)

WAIT UNTIL MPOS(0)>1 '等待跟随轴距离>2

	MOVELINK_MODIFY=1 '把跟随轴距增加 1 WAIT UNTIL MOVELINK_MODIFY=0 '等待同步偏移完成 WEND 运动轨迹和速度曲线 DPOS(0)垂直刻度 1，无偏移 MSPEED(0)垂直刻度 1，无偏移 DPOS(1)垂直刻度 3，无偏移 MSPEED(1)垂直刻度 1，无偏移
相关指令	MOVELINK MOVELINK -- 自动凸轮

MOVESYNC -- 同步运动

类型	运动设置指令				
描述	同步运动，皮带上物体跟随，此运动非插补运动，不保证运动轨迹为直线。 要求皮带轴与 BASE 跟随轴的长度单位是一样的。 当 BASE 轴完成跟随动作后，此指令结束。 MOVESYNC 支持连续使用，会自动保证速度连续，中间可以插入 MOVE_OP 指令，为了避免运动结束时高速跟随中直接停止，最后一条 MOVESYNC 请使用-1 模式。 此指令属于凸轮指令，不支持运动暂停。使用时需要将角度转换为弧度（0~2PI）。 注：多个 movesync 运动中不能插入插补运动或者关节运动，只能通过轴叠加的方式修改运动轨迹，否则会导致速度不连续或者飞车				
语法	MOVESYNC(mode,synctime,syncposition,syncaxis,pos1[,pos2, pos3...]) mode: 模式 <table><tr><th>模式</th><th>描述</th></tr><tr><td>-1</td><td>同步结束模式，运动到指定的绝对位置，此模式运动如果运动缓冲区后面紧接着其它 MOVESYNC 指令，此模式的运动位置不会被执行，此</td></tr></table>	模式	描述	-1	同步结束模式，运动到指定的绝对位置，此模式运动如果运动缓冲区后面紧接着其它 MOVESYNC 指令，此模式的运动位置不会被执行，此
模式	描述				
-1	同步结束模式，运动到指定的绝对位置，此模式运动如果运动缓冲区后面紧接着其它 MOVESYNC 指令，此模式的运动位置不会被执行，此				

			模式下 syncaxis 无效
	-2		强制结束模式，调用时强制停止同一个缓冲区中，此模式运动前的所有 MOVESYNC，并且运动到指定结束位置，此模式运动如果运动缓冲区后面紧接着其它 MOVESYNC 指令，此模式的运动位置不会被执行，此模式下 sync axis 无效 此模式在缓冲期间，只要中间插入了别的缓冲，强制返回失效
	0		BASE 第 1 个轴（x）跟随皮带轴物体
	10		BASE 第 2 个轴（y）跟随皮带轴物体
	20		BASE 第 3 个轴跟随皮带轴物体
mode=0+angle，angle：皮带旋转角度，角度=皮带轴 dpos 正向与 BASE 轴正向的夹角。使用时需要将角度转换为弧度（0~2PI），例如机械手坐标系中，BASE(X 轴)，Mode=PI/4，表示皮带 dpos 增加方向，往坐标系外运动，与 X 轴正向夹角为 45 度；BASE(X 轴)，Mode=PI*5/4，表示皮带 dpos 增加方向，往坐标系外运动，与 X 轴正向夹角为 225 度；同理 Mode=PI/2，皮带在坐标系 y 轴正方向；Mode=PI，皮带在坐标系 x 轴负方向；Mode=(PI*1.75)，皮带在 315 度的方向。如果选择的是 Y 轴与皮带轴的夹角，则考虑皮带 dpos 增加方向与 Y 轴正向夹角即可。 mode=1000+tableindex，几个轴的跟踪比例依次存储 table 里面，这样可以实现三维的皮带跟踪，当皮带有倾斜的时候(固件版本 20180209 增加此功能) synctime：同步时间，ms 单位，本运动在指定时间内完成，完成时 BASE 轴跟上皮带且保持速度一致。0 表示根据运动轴的速度加速度来估计同步时间，可能不准确 syncposition：皮带轴物体被感应到时皮带轴的位置，此指令支持皮带轴坐标循环，但是在指令被调用时确保此参数位置和当前皮带轴位置之间没有发生坐标修改或循环操作，因此此指令调用时不要在坐标循环点附近 syncaxis：皮带轴轴号，-1 表示没有皮带轴，直接运动到 pos1 的位置 pos1：皮带轴物体被感应到时的 BASE 第 1 个轴绝对位置 posn：皮带轴物体被感应到时的 BASE 第 n 个轴绝对位置			
适用控制器	ECI241x、ECI282x、ECI382x、XPLC 全系列、ZMC3 系列以上		
例子	例一： <pre>GLOBAL tmp_val GLOBAL tmp_offPos GLOBAL tmp_AccPos tmp_flag=0 SPEED(0)=800 DPOS(0)=0</pre>		

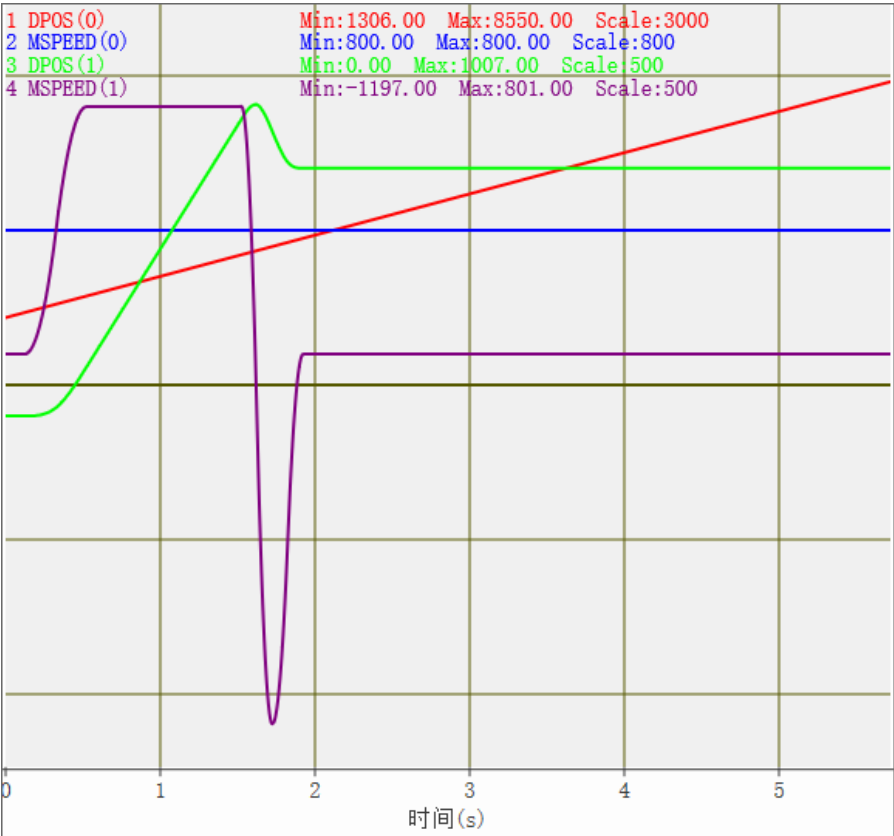
```
DPOS(1)=0
VMOVE(1)AXIS(0)

wait UNTIL tmp_flag=1

tmp_offPos=100
tmp_AccPos=800*0.4/2
tmp_val=Mpos(0)+tmp_offPos+tmp_AccPos
?tmp_val
TRIGGER
base(1)
MOVE_WAIT(mpos,0,1,tmp_val-tmp_AccPos)axis(1)
MOVESYNC(0,400,tmp_val,0,0)AXIS(1)      '加速段
MOVESYNC(0,1000,tmp_val,0,0)AXIS(1)      '匀速段
MOVESYNC(-1,400,tmp_val,-1,800)AXIS(1)    '减速段

End
```

运动轨迹和速度曲线
DPOS(0)垂直刻度 3000，无偏移
DPOS(1)垂直刻度 500，偏移-100
MSPEED(0)垂直刻度 500，无偏移
MSPEED(1)垂直刻度 500，偏移 100



例二：皮带取料

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

BASE(0,1)

DPOS=0,0

UNITS=100,100

ATYPE=1,1

SPEED=100,100

ACCEL=1000,1000

DECEL=1000,1000

TRIGGER

MOVESYNC(0, 0, 100, 1, 120) '同步到皮带物体上

MOVE_OP(1,1) '下降, 如果 Z 轴下降, 可以用 MOVESYNC 运动指令代替

MOVE_OP(0,1) '开吸嘴

MOVESYNC(0, 1000, 100, 1, 120) '继续同步 1s

MOVE_OP(1,0) '上升

MOVESYNC(-1, 0, 0, -1, 400) '走到放料位置 400 处

MOVE_OP(1,1) '下降

MOVE_OP(0,0) '关吸嘴

MOVE_DELAY(2) '延时 2ms, MOVESYNC 连续运动中不能插入此类语句

MOVE_OP(1,0) '上升

MOVEABS(0) '回到原点

VMOVE(1) AXIS(1) '皮带轴运动

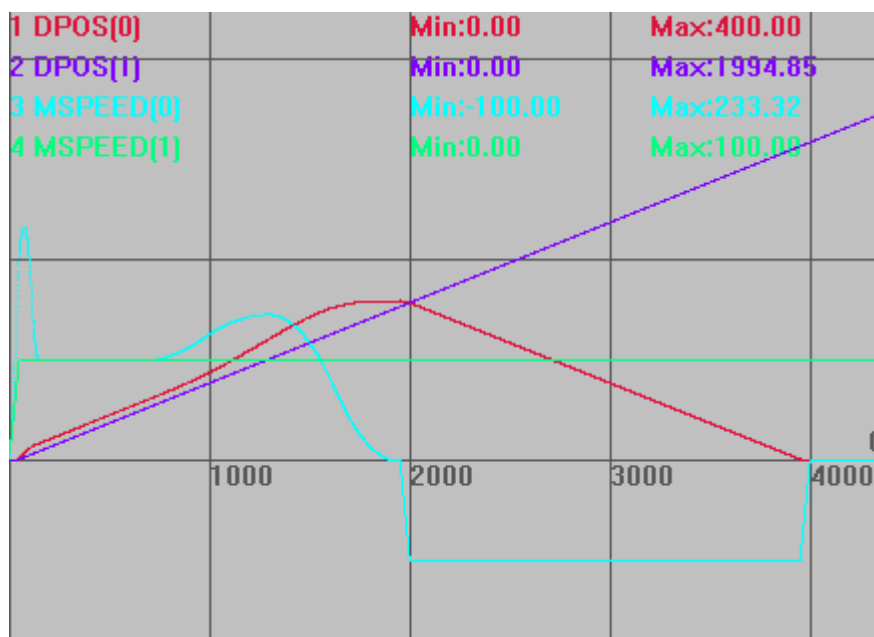
运动轨迹和速度曲线

DPOS(0)垂直刻度 500, 无偏移

DPOS(1)垂直刻度 500, 无偏移

MSPEED(0)垂直刻度 200, 无偏移

MSPEED(1)垂直刻度 200, 无偏移



例三：皮带取料，放另外的皮带上

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

BASE(0,1,2)

DPOS=0,0,0

UNITS=100,100,100

ATYPE=1,1,1

SPEED=1000,100,150 '设置不同速度

ACCEL=1000,1000,1000

DECEL=1000,1000,1000

TRIGGER

MOVESYNC(0, 0, 50, 1, 80) '同步到皮带物体上

MOVE_OP(0,1) '开吸嘴

MOVE_OP(1,1) '下降,如果 Z 轴下降,可以用 movesync 运动指令代替

MOVESYNC(0, 300, 50, 1, 80) '继续同步 2ms

MOVE_OP(1,0) '上升

MOVESYNC(0, 0, 100, 2, 150) '走到第二个皮带上对应位置

MOVE_OP(1,1) '下降

MOVE_OP(0,0) '关吸嘴

MOVESYNC(0,300, 100, 2, 150) '同步 2ms

MOVE_OP(1,0) '上升

MOVESYNC(-1, 0, 0, -1, 0) '走到停止位置

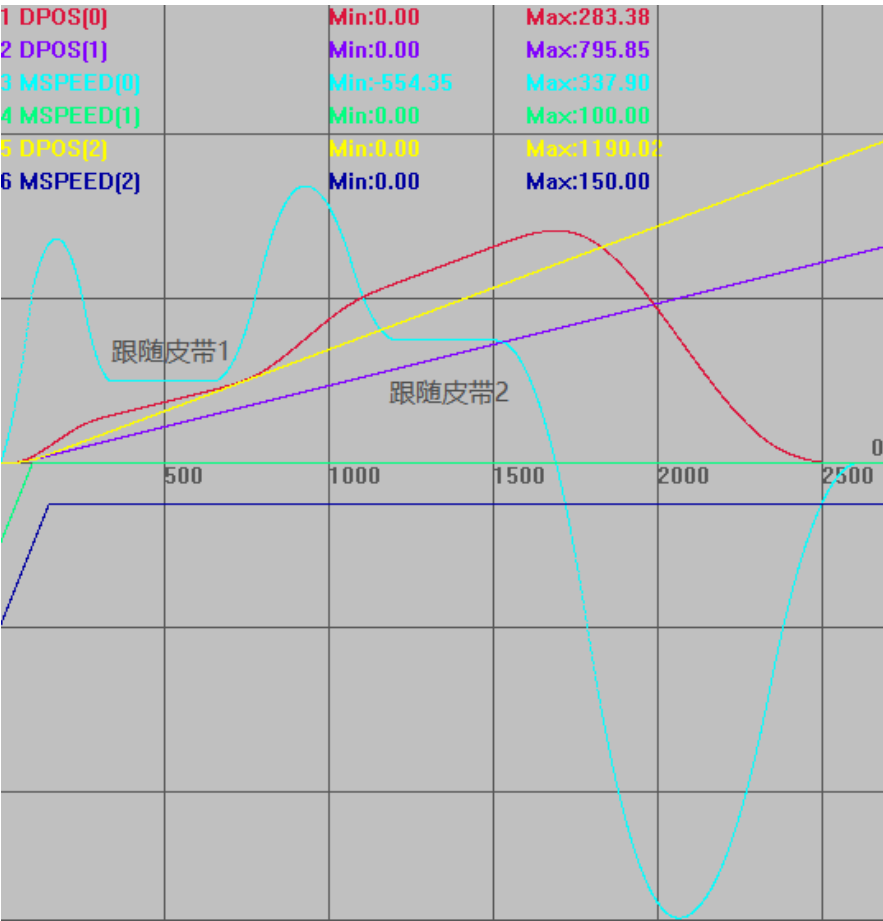
VMOVE(1) AXIS(1) '皮带轴 1 运动

VMOVE(1) AXIS(2) '皮带轴 2 运动

运动轨迹和速度曲线

DPOS(0)垂直刻度 200，无偏移

DPOS(1)垂直刻度 200，无偏移
MSPEED(0)垂直刻度 200，无偏移
MSPEED(1)垂直刻度 200，偏移-200
DPOS(2)垂直刻度 200，无偏移
MSPEED(2)垂直刻度 200，偏移-200



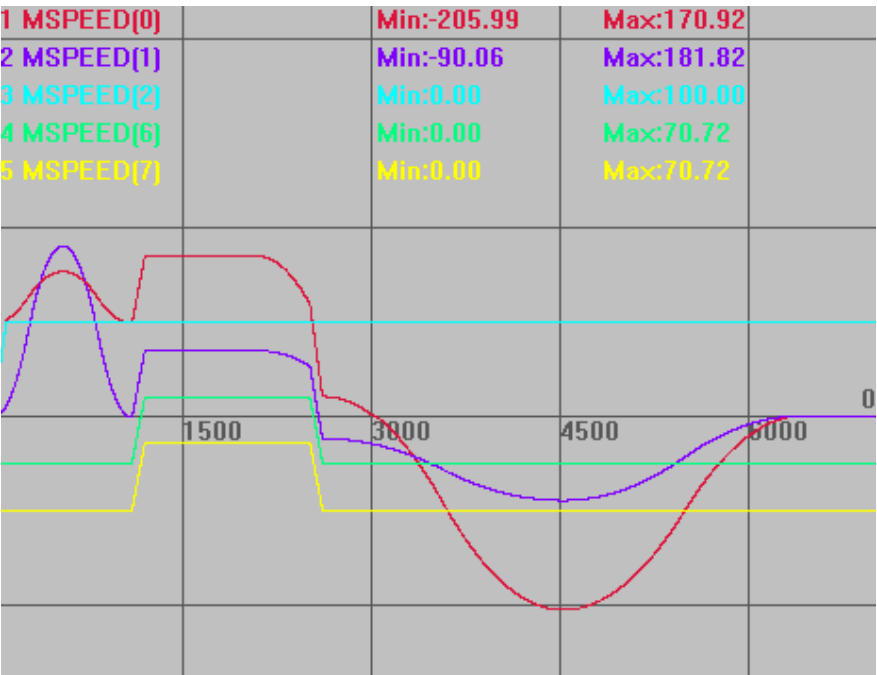
例四：SCARA 机械手 MOVESYNC 同步运动例子见[机械手使用 movesync 指令](#)

例五：皮带物体上雕刻
RAPIDSTOP(2)
WAIT IDLE(0)
WAIT IDLE(1)
BASE(0,1,2,6,7)
UNITS=100,100,100,100,100
DPOS=0,0,0,0,0
SPEED=100,100,100,100,100
ACCEL=1000,1000,1000,1000,1000
DECEL=1000,1000,1000,1000,1000
TRIGGER
ADDAX(6) AXIS(0) '在虚拟轴上雕刻,叠加到实际轴上
ADDAX(7) AXIS(1)

```
BASE(0, 1)
MOVESYNC(0, 0, 50, 2, 80, 100) '同步到皮带物体上
MOVE_TASK(1, task1) '触发叠加轴雕刻
MOVESYNC(0, 1000, 50, 2, 80, 100) '较长雕刻运动时间
MOVESYNC(-1, 0, 0, -1, 0, 0) '走到停止位置
VMOVE(1) AXIS(2) '皮带轴运动
END

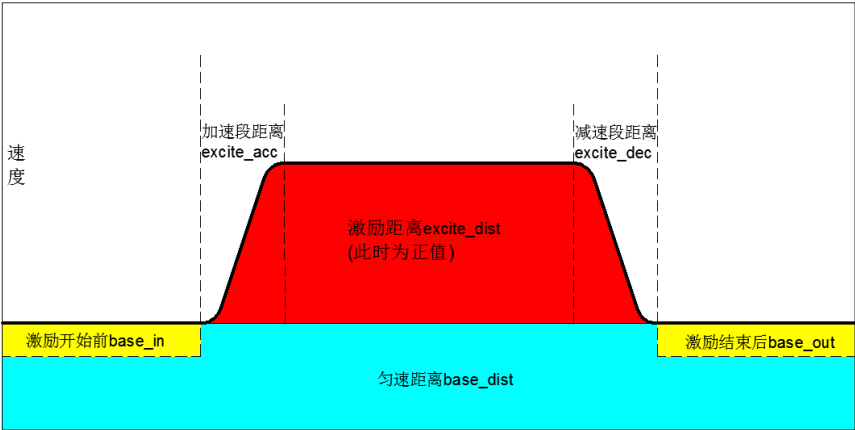
TASK1:
    DELAY (2) '叠加雕刻过程中绝对运动指令位置会不准,延时避开指令调用
    BASE(6, 7)
    MOVE(100,100) '雕刻，双面划线
    WAIT IDLE '等待雕刻结束
    BASE(0, 1)
    MOVESYNC(-2, 0, 0, -1, 0, 0) '雕刻结束强制走到停止位置
```

运动轨迹和速度曲线
MSPEED(0)垂直刻度 200，无偏移
MSPEED(1)垂直刻度 200，无偏移
MSPEED(2)垂直刻度 200，无偏移
MSPEED（6） 竖直刻度 200，偏移-50
MSPEED（7） 竖直刻度 200，偏移-100

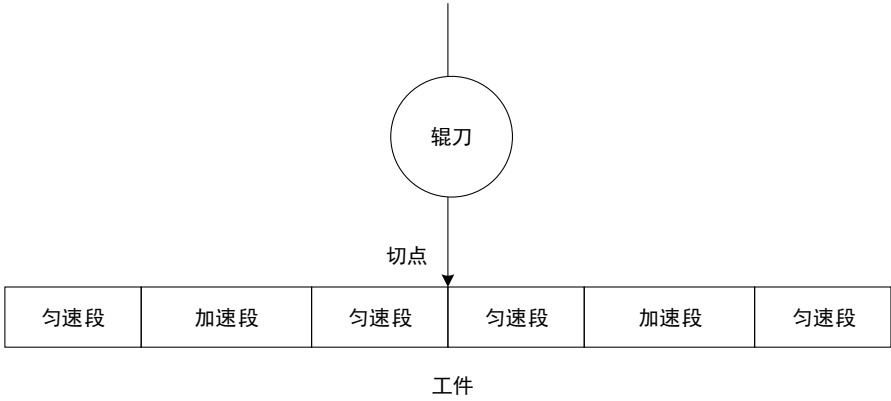


FLEXLINK -- 激励运动

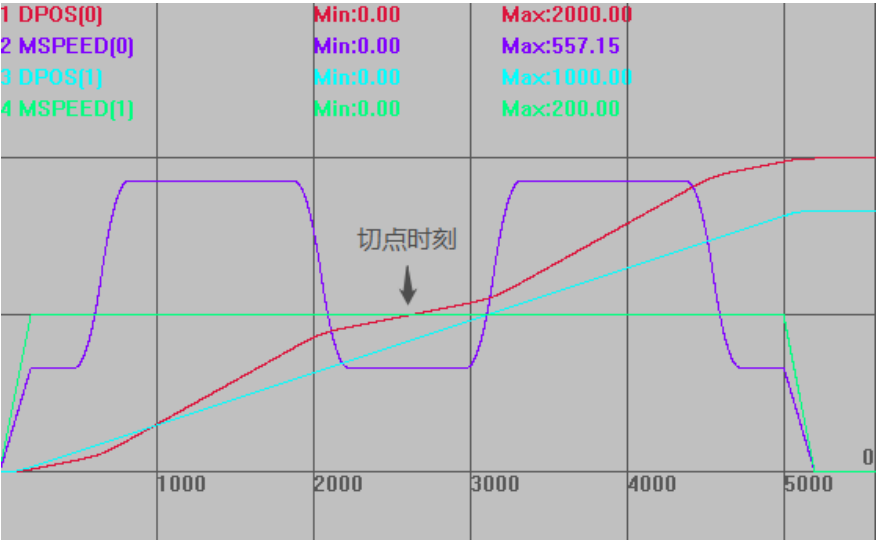
类型	同步运动指令
----	--------

描述	此指令用于轴的激励运动，分为匀速运动和激励运动两部分。  请确保指令传递的距离参数*units 是整数个脉冲，否则出现浮点数会导致运动有细微误差。														
语法	FLEXLINK(base_dist, excite_dist, link_dist, base_in, base_out, excite_acc, excite_dec, link_axis, link_options, [start_pos], [link_offpos]) base_dist: 跟随轴匀速运动距离 excite_dist: 跟随轴激励运动距离，+值表示增加，-值表示减少， 跟随轴运动总距离=base_dist+excite_dist link_dist: 整个指令过程，跟随轴运动完成，主轴移动的距离 base_in: 激励开始前，跟随轴运动距离占 base_dist 的百分比 base_out: 激励完成后，跟随轴剩余运动距离占 base_dist 的百分比（两者相加不要超过 100%，否则 excite_dist 无效） excite_acc: 激励过程中，跟随轴加速阶段运动距离占 excite_dist 的百分比，excite_dist 为负值时，为减速阶段 excite_dec: 激励过程中，跟随轴减速阶段运动距离占 excite_dist 的百分比，excite_dist 为负值时，为加速阶段 (以上 4 个参数在 excite_dist 不为 0 时生效) link_axis: 主轴轴号 link_options: 与参考轴的连接方式，不同的二进制位代表不同的意义 <table><tr><th>位</th><th>描述</th></tr><tr><td>bit0</td><td>当参考轴锁存信号事件 MARK 触发</td></tr><tr><td>bit1</td><td>当参考轴运动到设定的绝对位置时，当前轴与参考轴开始连接运动</td></tr><tr><td>bit2</td><td>自动重复连续双向运行(通过设置 REP_OPTION=1，可以取消重复)</td></tr><tr><td>bit4</td><td>凸轮中间启动</td></tr><tr><td>bit5</td><td>只有参考轴的正向运动才连接</td></tr><tr><td>bit8</td><td>markb 启动</td></tr></table> start_pos: 绝对位置触发 link_offpos: 凸轮中间启动	位	描述	bit0	当参考轴锁存信号事件 MARK 触发	bit1	当参考轴运动到设定的绝对位置时，当前轴与参考轴开始连接运动	bit2	自动重复连续双向运行(通过设置 REP_OPTION=1，可以取消重复)	bit4	凸轮中间启动	bit5	只有参考轴的正向运动才连接	bit8	markb 启动
位	描述														
bit0	当参考轴锁存信号事件 MARK 触发														
bit1	当参考轴运动到设定的绝对位置时，当前轴与参考轴开始连接运动														
bit2	自动重复连续双向运行(通过设置 REP_OPTION=1，可以取消重复)														
bit4	凸轮中间启动														
bit5	只有参考轴的正向运动才连接														
bit8	markb 启动														
适用控制器	4 系列 20170518 以上固件版本支持。 3 系列 20161212 以上固件版本支持。														
例子	轮切应用 假设辊刀切一次周长 500，工件长度 1000，辊刀恒速，移动距离小于工件长度，此时工件需要有加速过程。设置工件前后为匀速段，中间为加速段。														

实际应用中一般是辊刀变速，这里为了演示方便，使用工件变速。

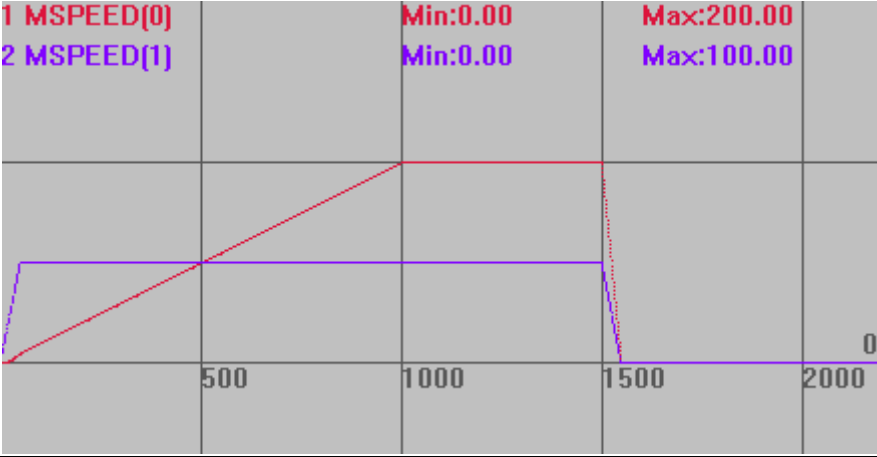


```
RAPIDSTOP(2)
WAIT IDLE(0)
WAIT IDLE(1)
BASE(0,1)
DPOS=0,0
UNITS=100,100
SPEED = 200,200
ACCEL=1000,1000
DECEL=1000,1000
TRIGGER
FLEXLINK(500,500,500,15,15,20,20,1) AXIS(0) '辊刀运行 500 距离，工件运行 1000
距离，在切点时刻保证速度一致，且对应一个周期位置
FLEXLINK(500,500,500,15,15,20,20,1) AXIS(0)
MOVE(1000) AXIS(1)
运动轨迹和速度曲线
DPOS(0)垂直刻度 1000，无偏移
MSPEED(0)垂直刻度 300，无偏移
DPOS(1)垂直刻度 600，无偏移
MSPEED(1)垂直刻度 200，无偏移
```

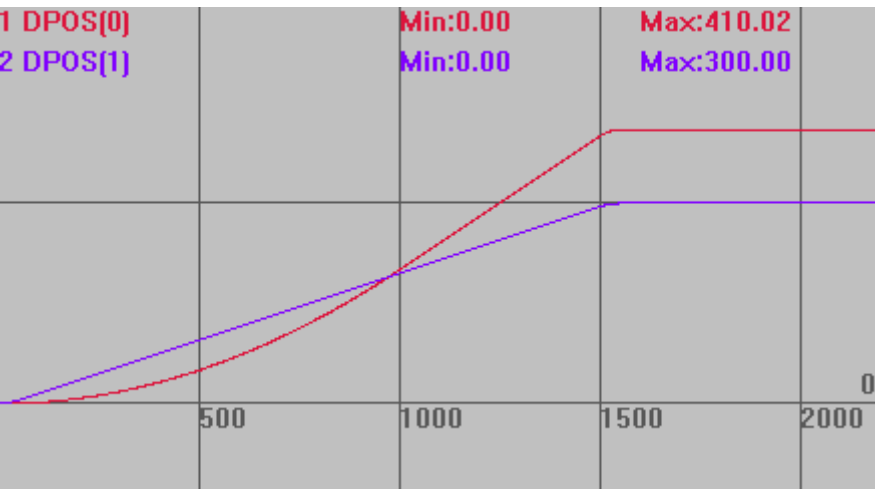


7.5 运动设置指令

CLUTCH_RATE -- 连接速度

类型	轴参数
描述	CONNECT 连接的速度，默认值 1000000。 用于定义连结率从 0 到设置倍率的改变时间，ratio/秒，见例一。 设置值如果不能远大于 CONNECT 连接比例的话，实际连接比例会减小，见例一位移曲线图。 当设置为 0 时，根据跟随轴的速度/加速度参数来跟踪连接（当速度不够高时可能导致要持续运动一段时间才能结束）。
语法	CLUTCH_RATE= value
适用控制器	通用
例子	例一 BASE(0,1) ATYPE=1,1 DPOS=0,0 UNITS=100,100 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 CLUTCH_RATE=1 '设置连接速度为 1ratio/s TRIGGER '自动触发示波器 CONNECT(2,1) AXIS(0) '连接倍率为 2，需要 2 秒建立连接 MOVE(300) AXIS(1) '运动轴 1，轴 0 跟随 速度曲线，连接时间根据连接比例和 CLUTCH_RATE 确定。 MSPEED(0)垂直刻度 200 MSPEED(1)垂直刻度 200 

位移曲线，CLUTCH_RATE 值较小，实际运动比例并没有达到 2:1
DPOS(0)垂直刻度 300
DPOS(1)垂直刻度 300



例二

BASE(0,1)

DPOS=0,0

ATYPE=1,1

UNITS=100,100

SPEED=100,100

ACCEL=500,500

DECEL=500,500

CLUTCH_RATE = 0

'根据跟随轴的速度/加速度来跟踪连接，不一定同步

TRIGGER

'自动触发示波器

CONNECT(2,1) AXIS(0)

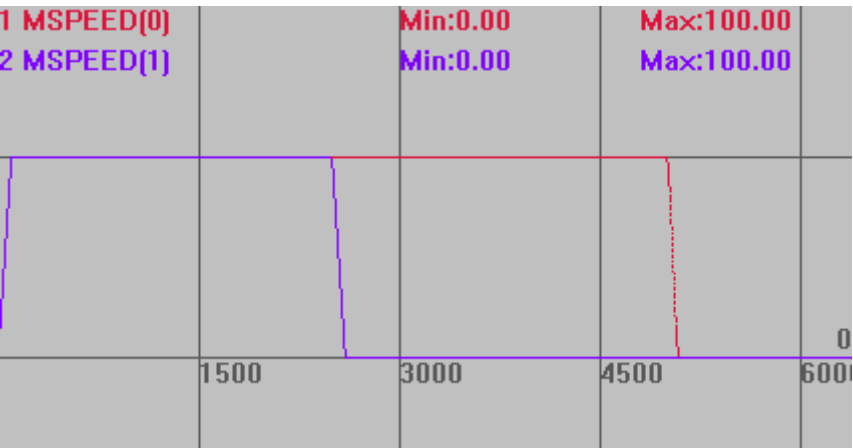
'此时连接时间根据跟随轴速度加速度确定，本例中 0.2s

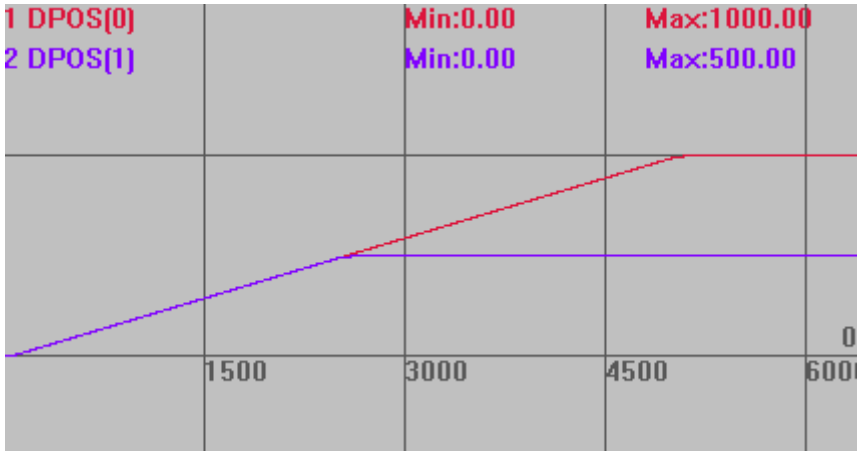
MOVE(500)AXIS(1)

速度曲线

MSPEED(0)垂直刻度 100

MSPEED(1)垂直刻度 100



	位移曲线 DPOS(0)垂直刻度 1000 DPOS(1)垂直刻度 1000 
相关指令	CONNECT

ENCODER_RATIO -- 编码器齿轮比

类型	运动设置指令								
描述	编码器输入齿轮比，缺省(1,1)，设置负值可切换方向 注意：负数的齿轮比，如果出现定位不准，把分子分母负号互换一下								
语法	ENCODER_RATIO(mpos_count, input_count[, mode]) mpos_count: 分子，不要超过 65535 input_count: 分母，不要超过 65535 mode: 模式 <table border="1" data-bbox="443 1303 828 1476"> <thead> <tr> <th>模式</th><th>描述</th></tr> </thead> <tbody> <tr> <td>1</td><td>AB 1X 模式</td></tr> <tr> <td>2</td><td>AB 2X 模式</td></tr> <tr> <td>4</td><td>AB 4X 模式</td></tr> </tbody> </table> 必须先设置 ATYPE 为编码器类型，再调用 mode 设置。 ZMC406 20170502 以后固件支持。	模式	描述	1	AB 1X 模式	2	AB 2X 模式	4	AB 4X 模式
模式	描述								
1	AB 1X 模式								
2	AB 2X 模式								
4	AB 4X 模式								
适用控制器	通用								
例子	ENCODER_RATIO(4,1) '编码器 4 倍输入，等同于 ENCODER_RATIO(1,1,4) ENCODER_RATIO(1,-1) '编码器输入切换方向，等同于 ENCODER_RATIO(-1,1)								
相关指令	PP_STEP ， ENCODER								

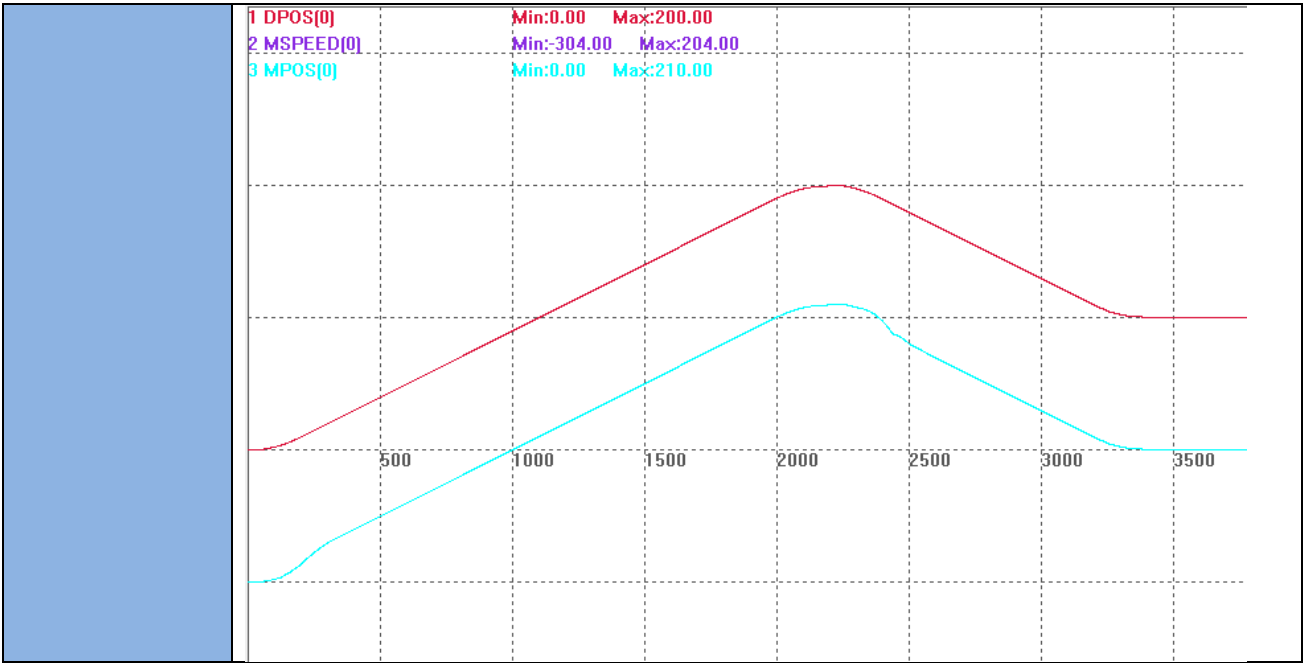
STEP_RATIO -- 电机齿轮比

类型	运动设置指令
描述	设置步进输出齿轮比，缺省(1,1)。范围 -2^{31} 到 $2^{31}-1$ 。

	设置为负值可修改电机方向，但一般不建议。脉冲电机可用 INVERT_STEP 指令；总线电机最好在驱动器上修改。 建议不要随便修改此参数，可以通过修改脉冲当量来达到相同的效果。 注意：负数的齿轮比，如果出现定位不准，把分子分母负号互换一下
语法	STEP_RATIO(output_count, input_count) output_count: 分子，分子分母只有一个能设置负数 input_count: 分母，分子分母只有一个能设置负数
适用控制器	通用
例子	STEP_RATIO(16,1) '16 倍脉冲个数输出，把脉冲当量乘以 16 也可以达到同样的效果

BACKLASH -- 反向间隙补偿

类型	运动设置指令
描述	设置轴的反向间隙，扩展轴无效。
语法	BACKLASH(enable [,dist[,speed,acceleration]]) enable: 使能与否 dist: 距离，units 单位 speed: 反向间隙速度 acceleration: 反向间隙加速度
适用控制器	通用
例子	例一 BACKLASH(0) '关闭反向间隙功能 BACKLASH(1,0.1) '设置反向间隙 0.1mm 例二 RAPIDSTOP(2) WAIT IDLE(0) BASE(0) ATYPE=5 '带编码器反馈 UNITS = 1000 SPEED = 100 ACCEL = SPEED * 10 DECEL = SPEED * 10 SRAMP = 0 DPOS = 0 MPOS = 0 BACKLASH(0) '关闭反向间隙 TRIGGER '应用反向间隙参数 BACKLASH(1, 10, 50, 100) '10units 补偿 MOVE(200) MOVE(-100) '反向时开始补偿 END



PITCHSET -- 螺距补偿

类型	运动设置指令
描述	设置轴的螺距补偿，扩展轴无效。 每点的补偿脉冲个数存储在 TABLE 表里面。
语法	PITCHSET(enable [,startpos,disone,maxpoint,tablenum] [,tablereverse]) enable: 使能与否 startpos: 开始补偿的 mpos 位置，units 单位，startpos 对应的点不存储 disone: 每个点之间的距离，units 单位 maxpoint: 补偿的总点数 tablenum: 螺距补偿表存储的 table 位置，从 startpos 下一个点开始存储，脉冲数单位，存储的都是当前点相对于第一个点的绝对补偿脉冲数 tablereverse: 正向反向不一样的补偿值的情况下使用，螺距补偿表存储的 table 位置 支持动态修改补偿参数； 补偿开始或关闭时，调整 dpos,mpos，而不是产生运动使得 dpos,mpos 正确。 距离参数不能设置负数，否则补偿将完全不起作用。不管电机行走方向如何，pitset 设置的方向永远是正向的。 一维补偿指令没有缓冲区，一次下载一条补偿指令，不能同时下载多条，否则第一条将被取消。 反向时如果补偿表和正向时不一致,可以使用指令中的 tablereverse 参数来设置反向时的补偿表
适用控制器	通用，部分只有点位功能的控制器也支持螺距补偿
例子	单轴螺距补偿： 例一：

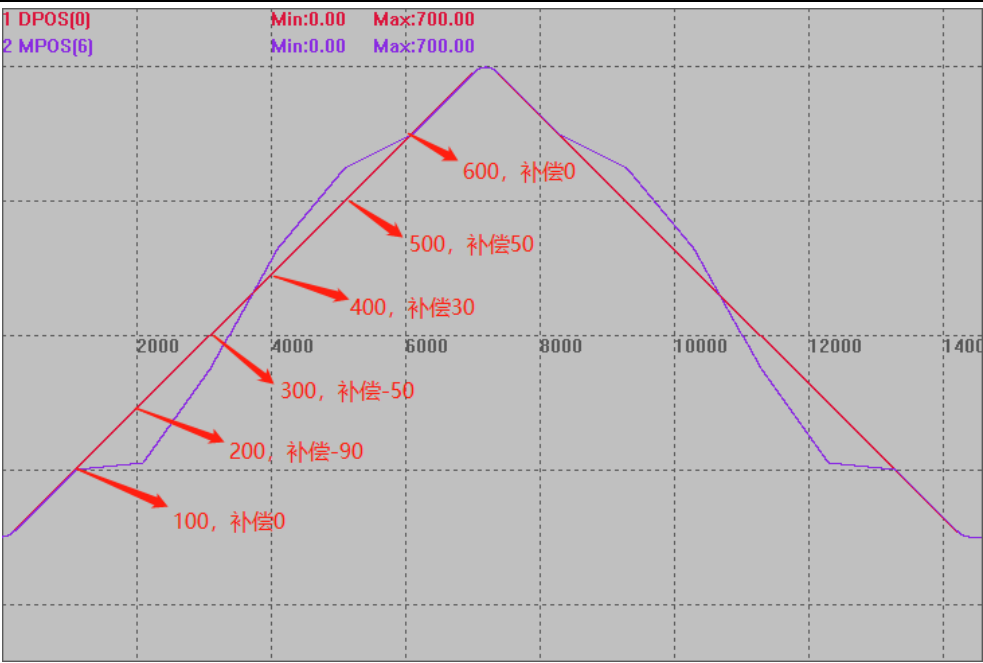
```

ATYPE(6)=6
UNITS(6)=100
DPOS(6)=0

BASE(0)
ATYPE=1
UNITS=100
SPEED=100
ACCEL=500
DECEL=500
TABLE(0,0*UNITS(0),-90*UNITS(0),-50*UNITS(0),30*UNITS(0),50*UNITS(0),0)
'TBALE 存贮螺距补偿值，补偿值是脉冲个数，不是补偿距离值
DPOS=0
MPOS=0
//开始补偿位置（MPOS）      补偿值（距离 100 时的脉冲个数）
//  100                      0
//  200                      -90 个脉冲
//  300                      -50 个脉冲
//  400                      30 个脉冲
//  500                      50 个脉冲
//  600                      0
PITCHSET(1,0,100,6,0)  'MPOS=0 时，开始补偿 6 个点，间隔 100
TRIGGER
MOVE(700)
MOVE(-700)
WAIT IDLE
PITCHSET(0,100,100,6,0)

该波形图，为了更好的显示补偿效果，把轴 0 的脉冲输出，接入到了轴 6 的编码器输入上，方便观察规划位置以及实际位置的区别，以看出补偿效果
DPOS(0)偏移-300，垂直刻度 200
MPOS(6)偏移-300，垂直刻度 200

```



例二：

ATYPE(6)=6
UNITS(6)=100
DPOS(6)=0

BASE(0)
ATYPE=1
UNITS=100
SPEED=100
ACCEL=500
DECEL=500
TABLE(0,0*UNITS(0),-90*UNITS(0),-50*UNITS(0),30*UNITS(0),50*UNITS(0),0)

'TBALE 存储螺距补偿值，补偿值是脉冲个数，不是补偿距离值

DPOS=0

MPOS=0

//开始补偿位置（MPOS） 补偿值（距离 100 时的脉冲个数）

// -100 0

// -200 50 个脉冲

// -300 30 个脉冲

// -400 -50 个脉冲

// -500 -90 个脉冲

// -600 0

PITCHSET(1,-700,100,6,0) 'MPOS=-700 时，开始补偿 6 个点，间隔 100

TRIGGER

MOVE(-700)

MOVE(700)

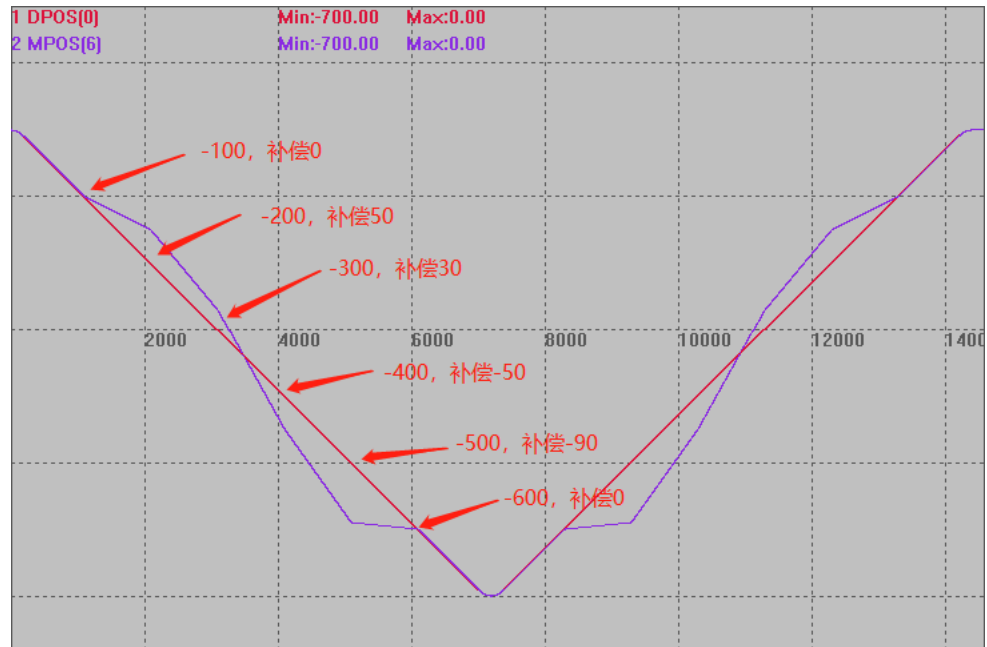
WAIT IDLE

PITCHSET(0,100,100,6,0)

该波形图，为了更好的显示补偿效果，把轴 0 的脉冲输出，接入到了轴 6 的编码器输入上，方便观察规划位置以及实际位置的区别，以看出补偿效果

DPOS(0)偏移 300，垂直刻度 200

MPOS(6)偏移 300，垂直刻度 200



例三：多轴螺距补偿

BASE(6,7)

UNITS=100,100

SPEED=100,100

ACCEL=100,100

DPOS=0,0

MPOS=0,0

BASE(0,1)

ATYPE = 5,5

UNITS=100,100

SPEED=100,100

ACCEL=100,100

DPOS=0,0

MPOS=0,0

TABLE(0,100*UNITS(0),100*UNITS(0),100*UNITS(0)) 'TBALE 存储螺距补偿值，补偿值是脉冲个数，不是补偿距离值

BASE(0)

PITCHSET(0)

PITCHSET(1,50,100,3,0) 'MPOS=50 时，开始补偿 3 个点，间隔 100

```
BASE(1)
PITCHSET(0)
PITCHSET(1,50,100,3,0) 'MPOS=50 时，开始补偿 3 个点，间隔 100

TRIGGER
BASE(0,1)
MOVE(150,150)
wait IDLE
?PITCH_DIST(0)
?PITCH_DIST(1)
MOVE(100,100)
wait IDLE
?PITCH_DIST(0)
?PITCH_DIST(1)
MOVE(100,100)
wait IDLE
?PITCH_DIST(0)
?PITCH_DIST(1)
```

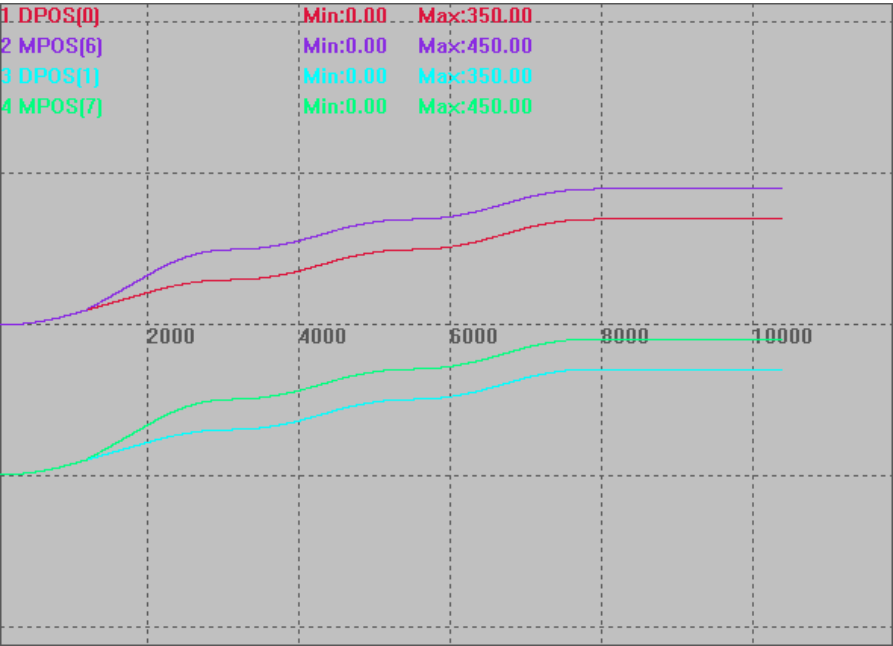
该波形图，为了更好的显示补偿效果，把轴 0/1 的脉冲输出，接入到了轴 6/7 的编码器输入上，方便观察规划位置以及实际位置的区别，以看出补偿效果

DPOS(0)偏移 0，垂直刻度 500

MPOS(6)偏移-0，垂直刻度 500

DPOS(1)偏移-500，垂直刻度 500

MPOS(7)偏移-500，垂直刻度 500

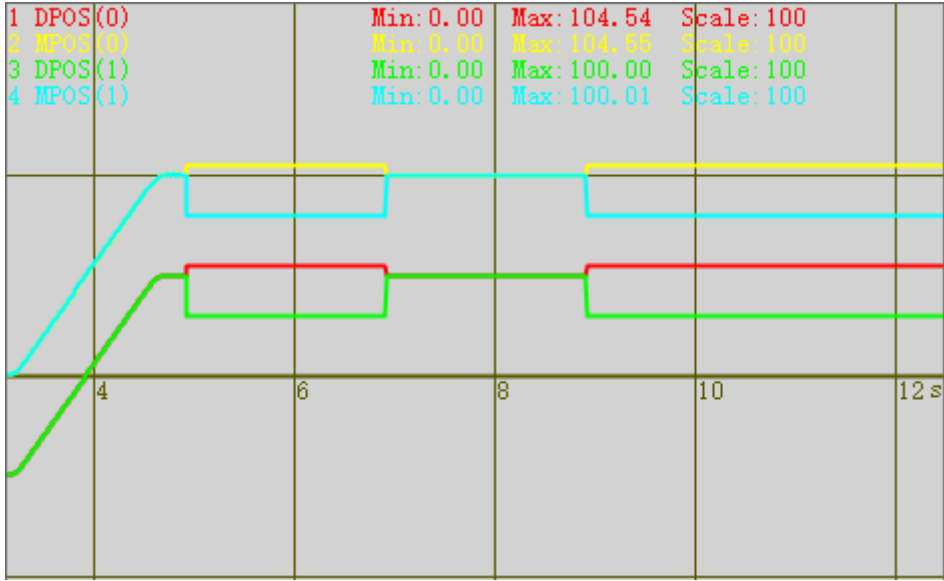


相关指令

[PITCH_DIST](#)

PITCH2SET -- 二维螺距补偿

类型	运动设置指令
描述	设置轴的螺距补偿，扩展轴无效。 每点的补偿脉冲个数存储在 TABLE 表里面。
语法	PITCH2SET(enable,startposx,startposy,disonex,disonex,maxpointx,maxpointy,tableindex) enable: 0- 关闭补偿, 1- XY 补偿, 2- XYX 补偿, 3- XYY 补偿 !注意: 2 模式的 BASE 轴顺序必须 X1, Y, X2; 此时 X2 的补偿值使用 X1 的补偿. !注意: 3 模式的 BASE 轴顺序必须 X, Y1, Y2; 此时 Y2 的补偿值使用 Y1 的补偿. startpos: 开始补偿的 mpos 位置, units 单位, startpos 对应的点不存储 disonex: 每个点之间的距离, units 单位 maxpoint: 补偿的总点数 tablenum: 螺距补偿表存储的 table 位置, 从 startpos 下一个点开始存储, 脉冲数单位, 每个点存储两个数据, 表示 X 方向偏差值与 Y 方向偏差值。先存储第一行(X 方向), 再存储第二行。总共占用 maxpointx*maxpointy*2 个 TABLE 位置 支持动态修改补偿参数; 补偿开始或关闭时, 调整 dpos,mpos, 而不是产生运动使得 dpos, mpos 正确。 !注意: x 轴号要小于 y 轴号, 否则可能轻微不同步. !必须先设置 ATYPE, 然后设置补偿; 修改 ATYPE 时, 先关闭补偿. !设置补偿时, 轴要位于 IDLE 停止状态. !补偿根据 MPOS_ORI 来计算, 带反馈的轴, 必须先保证 MPOS 反馈值正确.
适用控制器	4 系列(VERSION_BUILD > 230511)特殊固件支持, 比较占 CPU, 标准固件不支持
例子	<pre> BASE(0,1) ATYPE=5,5 UNITS=100,100 SPEED=100,100 ACCEL=500,500 DECEL=500,500 DPOS=0,0 MPOS=0,0 TABLE(0,10*UNITS(0),-20*UNITS(0),-20*UNITS(0),-40*UNITS(0),0*UNITS(0),- 0*UNITS(0)) delay(10) trigger PITCH2SET(1,0,0,90,90,2,2,0) MOVE(100,100) WAIT IDLE delay(100) delay(100) PITCH2SET(0) '关闭补偿 delay(2000) PITCH2SET(1,0,0,90,90,2,2,0) '不运动开启补偿, 查看 DPOS 值 </pre>

	delay(2000) PITCH2SET(0) 该波形图，为了更好的显示补偿效果，把轴 0/1 的脉冲输出，接入到了轴 6/7 的编码器输入上，方便观察规划位置以及实际位置的区别，以看出补偿效果 DPOS(0)偏移-50，垂直刻度 100 MPOS(0)偏移 0，垂直刻度 100 DPOS(1)偏移-50，垂直刻度 100 MPOS(1)偏移 0，垂直刻度 100  1 DPOS(0) Min: 0.00 Max: 104.54 Scale: 100 2 MPOS(0) Min: 0.00 Max: 104.66 Scale: 100 3 DPOS(1) Min: 0.00 Max: 100.00 Scale: 100 4 MPOS(1) Min: 0.00 Max: 100.01 Scale: 100
相关指令	PITCH2_DIST

PITCH_DIST -- 螺距补偿距离

类型	轴状态
描述	读取当前轴螺距补偿的距离值，实际的 MPOS 返回值会减去这个值。
语法	VAL=PITCH_DIST(axisnum) axisnum: 轴号
适用控制器	通用
相关指令	PITCHSET

PITCH2_DIST -- 二维螺距补偿距离

类型	轴状态
描述	读取当前轴螺距补偿的距离值，实际的 MPOS 返回值会减去这个值。
语法	BASE(x,y) ‘前两轴为要设置二维螺距的 X,Y 轴号 VAL=PITCH_DIST(axisnum) axisnum: 轴号
适用控制器	通用

相关指令

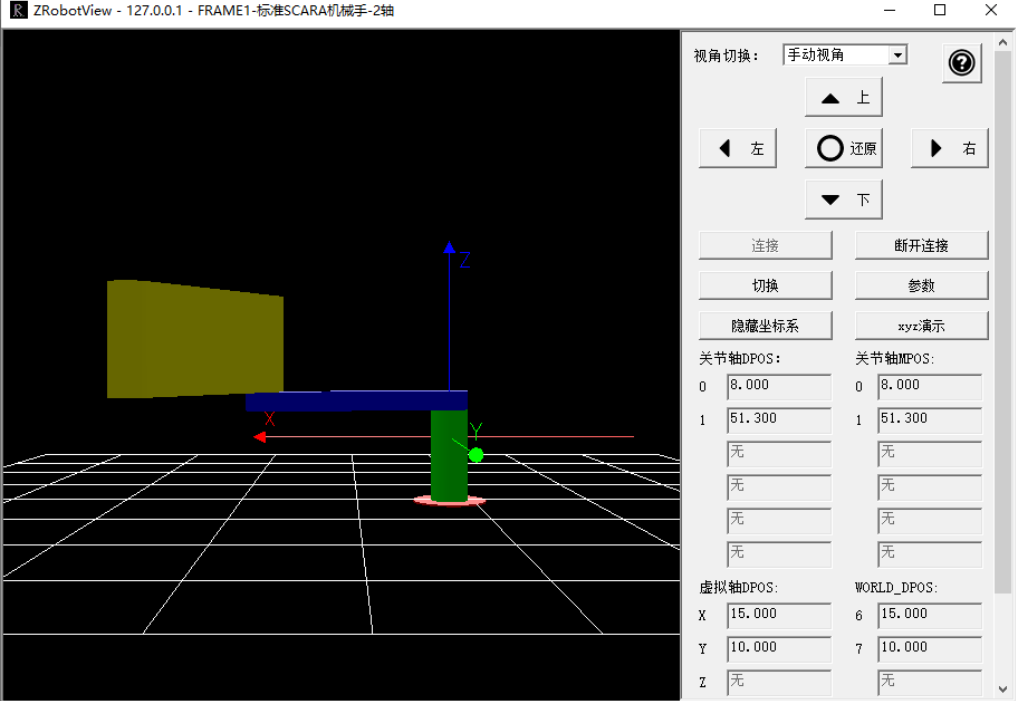
[PITCH2SET](#)

7.6 机械手指令

CONNFRAME -- 机械手逆解

类型	同步运动指令																		
描述	将当前关节坐标系的目标位置与虚拟坐标系的位置关联。 CLUTCH_RATE=0 时关节坐标系的运动速度和加速度受 SPEED 等参数的限制。 ⚠ 当关节轴告警等出错时，此运动会被 CANCEL。 ⚠ 不要在虚拟轴高速运动中 CANCEL 此运动，会引起轴高速运动中停止。 ⚠ 此命令 LOAD 时会自动修改虚拟轴的坐标，使得与关节轴一致，因此调用后需要 WAIT LOADED 后才开始运动。 ⚠ 连接过程中不要修改虚拟轴的坐标，或是调用 DATUM 等可能改坐标的运动，这样会导致关节轴快速运动到新的虚拟位置。 ⚠ CONNFRAME 生效后，关节轴的 MTYPE 为 33，此时无法直接运动关节轴，必须通过运行虚拟轴来间接运动关节轴，当要直接移动关节轴时，先调用 CANCEL 取消 CONNFRAME，再运动关节轴。 ⚠ 当虚拟轴和实际轴都是旋转轴时，例如末端旋转轴，虚拟轴和实际轴的脉冲当量注意要一致。																		
语法	CONNFRAME(frame, tablenum, viraxis0, viraxis1,...) frame: 坐标系类型，1- scara（如需针对特殊的机械手类型定制，请联系厂家） tablenum: 存储转换参数的 TABLE 位置；frame=1 时，依次存放：第一个关节轴长度，第二个关节轴长度，第一个关节轴一圈脉冲数，第二个关节轴一圈脉冲数等，详情参见机械手手册的说明 viraxis0: 虚拟坐标系第一个轴 viraxis1: 虚拟坐标系第二个轴 [...]: 虚拟坐标系第 N 个轴，此运动轴可以是实际轴，具体由机械手类型确定 FRAME 机械结构一览表 详细说明请查看《正运动机械手指令说明》文档。 其他特殊的机械结构如需支持请联系厂家。 <table border="1"> <thead> <tr> <th>frame</th><th>结构类型</th></tr> </thead> <tbody> <tr> <td>1</td><td>标准 SCARA 机械手</td></tr> <tr> <td>101</td><td>SCARA+摆动，虚拟轴定义 4 个</td></tr> <tr> <td>105</td><td>SCARA+摆动，虚拟轴定义 5 个</td></tr> <tr> <td>106</td><td>特殊 SCARA</td></tr> <tr> <td>107</td><td>特殊 SCARA</td></tr> <tr> <td>108</td><td>特殊 5 轴 SCARA</td></tr> <tr> <td>11</td><td>旋转台</td></tr> <tr> <td>17</td><td>双旋转台</td></tr> </tbody> </table>	frame	结构类型	1	标准 SCARA 机械手	101	SCARA+摆动，虚拟轴定义 4 个	105	SCARA+摆动，虚拟轴定义 5 个	106	特殊 SCARA	107	特殊 SCARA	108	特殊 5 轴 SCARA	11	旋转台	17	双旋转台
frame	结构类型																		
1	标准 SCARA 机械手																		
101	SCARA+摆动，虚拟轴定义 4 个																		
105	SCARA+摆动，虚拟轴定义 5 个																		
106	特殊 SCARA																		
107	特殊 SCARA																		
108	特殊 5 轴 SCARA																		
11	旋转台																		
17	双旋转台																		

		18	偏移旋转台	
		19	偏移双旋转台	
		3	码垛机械手	
		103	码垛变形，喷涂机械手	
		5	旋转伸缩机械手	
		15	XY 滑台	
		102	2 轴 delta	
		2	3 轴 delta，R 类型控制器支持	
		12	4 轴 delta，R 类型控制器支持	
		13	3 轴垂直蜘蛛手，R 类型控制器支持	
		25	5 关节机械手	
		6	6 自由度机械手，R 类型控制器支持	
		26	特殊 6 自由度	
		36	特殊 6 自由度	
		100	XYZ+2 轴手腕，虚拟轴定义 3 个	
		104	XYZ+2 轴手腕，虚拟轴定义 5 个	
适用控制器	通用			
例子	DIM L1,L2 L1=10 '第一个关节轴长度 L2=10 '第二个关节轴长度 BASE(0,1) '关节轴号是 0，1 ATYPE=1,1 UNITS=10,10 '脉冲当量，以度为单位 DPOS=0,0 '设置关节轴的位置，此处要根据实际情况来修改 BASE(6,7) '虚拟轴号 6，7 ATYPE=0,0 '设置为虚拟轴 TABLE(0,L1,L2,3600,3600) '参数存储从 TABLE(0)开始存储，两个关节的长度和一圈脉冲数，电机一圈 3600 个脉冲 UNITS=1000,1000 '此脉冲当量要提前设置，中途不能变化 BASE(0,1) CONNFRAME(1,0,6,7) '建立逆解，轴 0/1 的坐标根据轴 6/7 来计算运动关节轴 WAIT LOADED BASE(6,7) MOVEABS(15,10) '虚拟轴发送运动指令 END 连接机械手仿真工具查看运行效果如下图：			

	ZRobotView - 127.0.0.1 - FRAME1-标准SCARA机械手-2轴 
相关指令	CONNREFRAME

CONNREFRAME -- 机械手正解

类型	同步运动指令
描述	将虚拟轴的坐标与关节轴的坐标关联，关节轴运动后，虚拟轴自动走到相应的位置。 此指令为 CONNFRAME 的反运动指令。 ⚠ 当虚拟轴 CONNREFRAME 运动 LOAD 时，关节轴的 CONNFRAME 运动会自动被 CANCEL。 ⚠ 当关节轴的 CONNFRAME 运动 LOAD 时，虚拟轴 CONNREFRAME 运动会自动被 CANCEL。
语法	CONNREFRAME(frame, tablenum, axis0, axis1,[...]) frame: 坐标系类型, 1- scara (如需针对特殊的机械手类型定制, 请联系厂家) tablenum: 存储转换参数的 TABLE 位置; frame=1 时, 依次存放: 第一个关节轴长度, 第二个关节轴长度, 第一个关节轴一圈脉冲数, 第二个关节轴一圈脉冲数等, 详情参见机械手手册的说明 axis0: 关节坐标系第一个轴 axis1: 关节坐标系第二个轴 [...]: 关节坐标系第 N 个轴 BASE 轴与参数里面的轴的位置相反。
适用控制器	通用
例子	DIM L1,L2

	L1=10	'第一个关节轴长度
	L2=10	'第二个关节轴长度
	BASE(0,1)	'假设关节轴号是 0/1
	UNITS=10,10	'脉冲当量 10
	DPOS=0,0	'设置关节轴的位置，此处要根据实际情况来修改
	BASE(6,7)	
	ATYPE=0,0	'设置为虚拟轴
	TABLE(0,L1,L2,3600,3600)	'参数存储从 TABLE(0)开始存储，两个关节的长度和一圈脉冲数，电机一圈 3600 个脉冲
	UNITS=1000,1000	'此脉冲当量要提前设置，中途不能变化
	CONNREFRAME(1,0,0,1)	'轴 6/7 的坐标根据轴 0/1 来计算运动关节轴
	BASE(0,1)	
	MOVEABS(90,0)	'虚拟轴的坐标变为 0,20
相关指令	CONNFRAME	

FRAME -- 机械手类型

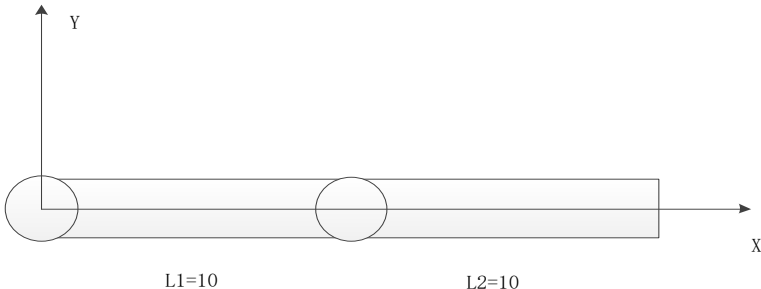
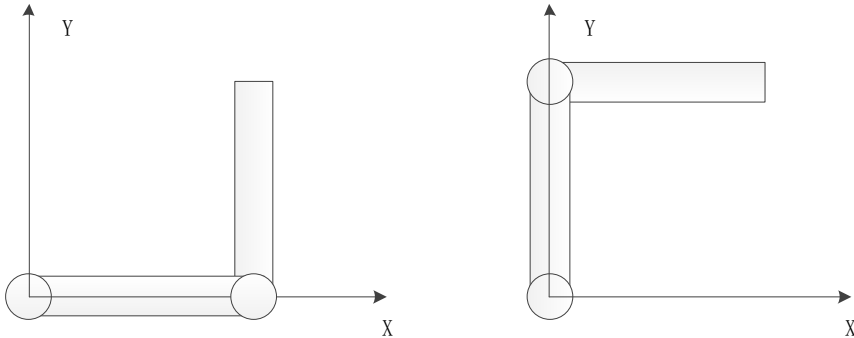
类型	机械手指令
描述	选择机械手类型，请查看正运动机械手手册。
相关指令	CONNFRAME

FRAME_STATUS -- 机械手轴状态

类型	机械手指令
描述	指明当前的机械手姿态。 不是机械手状态时返回-1，FRAME_TRANS2 指令会用到这个姿态。只对 SCARA、类 SCARA 和 6 自由度结构有多种姿态。 SCARA 右手姿态值为 0，左手姿态值为 1。
语法	VAR1=FRAME_STATUS (AXIS)
适用控制器	通用
例子	在线命令输入?FRAME_STATUS，打印出当前姿态。 >>?FRAME_STATUS
相关指令	BASE

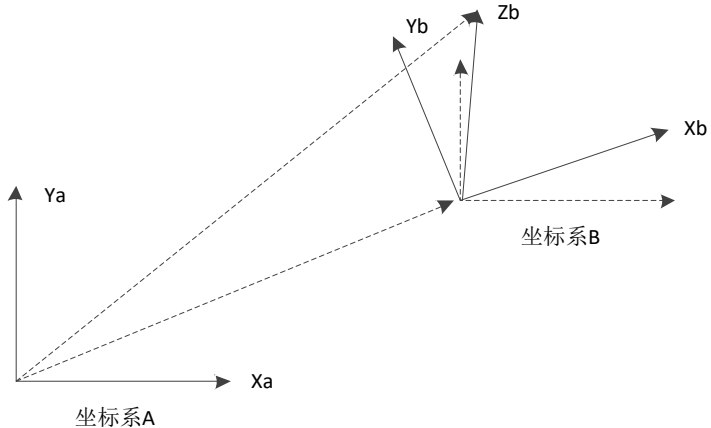
FRAME_TRANS2 -- 正逆解坐标转换

类型	机械手计算指令
描述	坐标转换函数。 使用时必须在正逆解建立的情况下。 使用时必须 base 正确的轴号。逆解时，base 虚拟轴；正解时，base 关节轴。 按照正确的顺序填入相应个数的数据。填入的个数与数据，与?*frame 一致。

语法	FRAME_TRANS2(tablein, tableout, dir) tablein: table 数组中的索引号, 从这个索引开始连续存储数据, 正解时输入关节坐标, 逆解时输入虚拟轴坐标列表, 最后附加姿态 tableout: table 这个索引开始存储数据, 正解时输出虚拟坐标, 最后附加姿态, 逆解时输出关节坐标列表 dir: 模式选择 <table><tr><th>模式</th><th>类型</th><th>描述</th></tr><tr><td>0</td><td>逆解</td><td>虚拟轴到关节轴, 无姿态, 自动使用当前的姿态</td></tr><tr><td>1</td><td>正解</td><td>关节轴到虚拟轴, 无姿态输出</td></tr><tr><td>2</td><td>逆解</td><td>输入虚拟轴坐标, 最后加上姿态</td></tr><tr><td>3</td><td>正解</td><td>输出虚拟轴坐标, 输出最后一个位置填姿态</td></tr></table>	模式	类型	描述	0	逆解	虚拟轴到关节轴, 无姿态, 自动使用当前的姿态	1	正解	关节轴到虚拟轴, 无姿态输出	2	逆解	输入虚拟轴坐标, 最后加上姿态	3	正解	输出虚拟轴坐标, 输出最后一个位置填姿态														
模式	类型	描述																												
0	逆解	虚拟轴到关节轴, 无姿态, 自动使用当前的姿态																												
1	正解	关节轴到虚拟轴, 无姿态输出																												
2	逆解	输入虚拟轴坐标, 最后加上姿态																												
3	正解	输出虚拟轴坐标, 输出最后一个位置填姿态																												
适用控制器	通用																													
例子	以 scara 结构为例, 第一关节轴 L1=10, 第二关节轴 L2=10。table(100)作为输入坐标的存放位置, table(200)作为输出坐标的存放位置。 建立连接后, 关节轴原点坐标 (0,0), 此时虚拟轴坐标为 (20,0), 如下图。  虚拟轴坐标 (10,10) 时, 有两种姿态 关节轴 (0,90) 和关节轴 (90,-90)  坐标转换表 <table><tr><th>状态</th><th>BASE 轴</th><th>输入 X、Y、姿态</th><th>使用指令模式</th><th>输出关节坐标</th></tr><tr><td rowspan="3">逆解</td><td rowspan="3">虚拟轴</td><td>table(100,20,0,0)</td><td rowspan="2">frame_trans2(100,200,0)</td><td>table(200,0,0)</td></tr><tr><td>table(100,10,10,1)</td><td>table(200,0,90)</td></tr><tr><td>table(100,10,10,1)</td><td>frame_trans2(100,200,2)</td><td>table(200,90,-90)</td></tr><tr><th>状态</th><th>BASE 轴</th><th>输入关节坐标</th><th>使用指令模式</th><th>输出 X、Y、姿态</th></tr><tr><td rowspan="3">正解</td><td rowspan="3">关节轴</td><td>table(100,0,0)</td><td rowspan="3">frame_trans2(100,200,1)</td><td>table(200,20,0,0)</td></tr><tr><td>table(100,0,90)</td><td>table(200,10,10,0)</td></tr><tr><td>table(100,90,-90)</td><td>table(200,10,10,0)</td></tr></table>	状态	BASE 轴	输入 X、Y、姿态	使用指令模式	输出关节坐标	逆解	虚拟轴	table(100,20,0,0)	frame_trans2(100,200,0)	table(200,0,0)	table(100,10,10,1)	table(200,0,90)	table(100,10,10,1)	frame_trans2(100,200,2)	table(200,90,-90)	状态	BASE 轴	输入关节坐标	使用指令模式	输出 X、Y、姿态	正解	关节轴	table(100,0,0)	frame_trans2(100,200,1)	table(200,20,0,0)	table(100,0,90)	table(200,10,10,0)	table(100,90,-90)	table(200,10,10,0)
状态	BASE 轴	输入 X、Y、姿态	使用指令模式	输出关节坐标																										
逆解	虚拟轴	table(100,20,0,0)	frame_trans2(100,200,0)	table(200,0,0)																										
		table(100,10,10,1)		table(200,0,90)																										
		table(100,10,10,1)	frame_trans2(100,200,2)	table(200,90,-90)																										
状态	BASE 轴	输入关节坐标	使用指令模式	输出 X、Y、姿态																										
正解	关节轴	table(100,0,0)	frame_trans2(100,200,1)	table(200,20,0,0)																										
		table(100,0,90)		table(200,10,10,0)																										
		table(100,90,-90)		table(200,10,10,0)																										

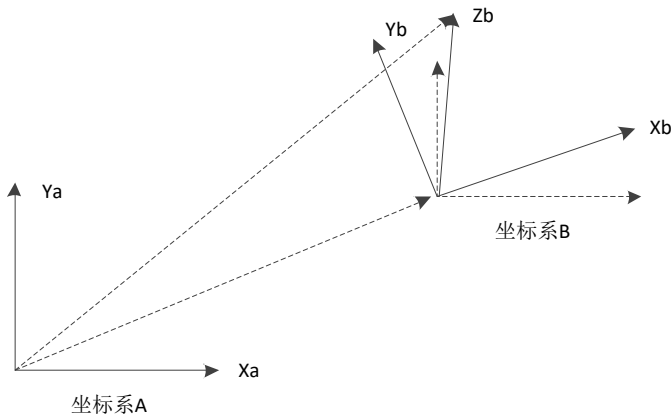
		table(100,90,-90)	frame_trans2(100,200,3)	table(200,10,10,1)
--	--	-------------------	-------------------------	--------------------

FRAME_ROTATE -- 工件坐标系变换

类型	机械手计算指令
描述	用于对工件坐标系进行平移和旋转。 目前只对 FRAME6 同时旋转姿态，其他具有 XYZ 虚拟轴的可以支持 XYZ 三个轴旋转，姿态轴不能旋转。 旋转后，虚拟轴 WORLD_DPOS 表示世界坐标不会改变，虚拟轴 DPOS 表示工件坐标会改变。 使用时需控制器当前存在机械链接。 BASE 轴可以是虚拟轴与关节轴任意一个；BASE 轴无机械手链接时会报 1025 的错误。多种机械手叠加的情况，根据 BASE 轴来识别是哪个机械手模式；如果 BASE (axis_1,axis_2) 中的 axis_1 是模式 1 的机械手轴，axis_2 是模式 2 的机械手轴，那么结果是模式 1 的机械手进行坐标计算，即以 BASE 轴的顺序来计算。
语法	FRAME_ROTATE(X,Y,Z,RX,RY,RZ) X: 坐标系 B 沿 X[^]的平移距离 Y: 坐标系 B 沿 Y[^]的平移距离 Z: 坐标系 B 沿 Z[^]的平移距离 RX: 坐标系 B 沿 X[^]的旋转的弧度 RY: 坐标系 B 沿 Y[^]的旋转的弧度 RZ: 坐标系 B 沿 Z[^]的旋转的弧度  坐标系旋转： 旋转的方法是：x_y_z 固定角坐标系。 首先将做坐标系{B}和一个已知参考坐标系{A}重合。先将{A}绕 Xa 旋转 RX 角，在绕 Ya 旋转 RY 角，最后绕 Za 旋转 RZ 角。
适用控制器	通用
例程	例一 以 FRAME=2, DETLA 为例，对其绕 X 轴旋转 90 度。

	<pre> BASE(0,1,2) RAPIDSTOP ATYPE = 1,1,1 UNITS=3600/360,3600/360,3600/360 DPOS=0,0,0 BASE(6,7,8) ATYPE = 0,0,0 TABLE(0,40,10,32,85,3600,3600,3600, 0, 0, 0) UNITS = 100,100,100 BASE(0,1,2) CONNFRAME(2,0,6,7,8) WAIT LOADED BASE(6,7,8) FRAME_ROTATE(0,0,0,PI/2,0,0) ?"DPOS(7)-WORLD_DPOS(7)=",DPOS(7)-WORLD_DPOS(7) ?"DPOS(8)-WORLD_DPOS(8)=",DPOS(8)-WORLD_DPOS(8) 输出行输出结果： DPOS(7)-WORLD_DPOS(7)=-58.1400 DPOS(8)-WORLD_DPOS(8)=58.1400 例二 以 FRAME=1， SCARA 为例；对其相对于 Z 轴旋转 90 度。 BASE(0,1,2,3) RAPIDSTOP ATYPE = 1,1,1,1 UNITS=3600/360,3600/360,3600/360,1000 DPOS=0,0,0,0 BASE(6,7,8,9) ATYPE = 0,0,0,0 TABLE(0,100,100,3600,3600,3600) UNITS = 100,100,3600/360,1000 BASE(0,1,2,3) CONNFRAME(1,0,6,7,8,9) WAIT LOADED BASE(6,7,8,9) FRAME_ROTATE(0,0,0,0,0,PI/2) ?"DPOS(7)-WORLD_DPOS(7)=",DPOS(7)-WORLD_DPOS(7) ?"DPOS(8)-WORLD_DPOS(8)=",DPOS(8)-WORLD_DPOS(8) 输出行输出结果： DPOS(7)-WORLD_DPOS(7)=-200 DPOS(8)-WORLD_DPOS(8)=0 </pre>
相关指令	FRAME_ROTATE2

FRAME_ROTATE2 -- 坐标系变换计算

类型	机械手计算指令						
描述	手动计算坐标系旋转后的坐标值。 使用时需控制器当前存在机械链接。 base 轴，可以是虚拟轴与关节轴中的任意一个；base 轴无机械手链接，会报 1025 的错误。 多种机械手叠加的情况，根据 base 轴来识别是哪个机械手模式。如果 base（axis_1,axis_2）中的 axis_1 是模式 1 的机械手轴，axis_2 是模式 2 的机械手轴，那么结果是模式 1 的机械手进行坐标计算，即以 base 轴的顺序来计算。						
语法	FRAME_ROTATE2(tablein, tableout, dir[, x,y,z[, rx,ry,rz]]) ret = FRAME_ROTATE2(tablein, tableout, dir[, x,y,z[, rx,ry,rz]]) tablein: 转换前，填写的坐标存储 table 位置 tableout: 转换后，输出的坐标存储 table 位置 dir: 方向选择 <table><tr><td>值</td><td>描述</td></tr><tr><td>0</td><td>DPOS 到 WORLD_DPOS</td></tr><tr><td>1</td><td>WORLD_DPOS 到 DPOS</td></tr></table> X: 坐标系 B 沿 X^的平移距离 Y: 坐标系 B 沿 Y^的平移距离 Z: 坐标系 B 沿 Z^的平移距离 RX: 坐标系 B 沿 X^的旋转的弧度 RY: 坐标系 B 沿 Y^的旋转的弧度 RZ: 坐标系 B 沿 Z^的旋转的弧度 [x,y,z[, rx,ry,rz]]: 不填时使用当前的缺省 rotate 参数 Ret: 返回成功与否；-1-成功，0-未成功 	值	描述	0	DPOS 到 WORLD_DPOS	1	WORLD_DPOS 到 DPOS
值	描述						
0	DPOS 到 WORLD_DPOS						
1	WORLD_DPOS 到 DPOS						
适用控制器	通用						
例程	例一 以 FRAME=2, DETLA 为例；对其绕 X 轴旋转 90 度。 BASE(0,1,2) RAPIDSTOP ATYPE = 1,1,1						


```

UNITS=3600/360,3600/360,3600/360
DPOS=0,0,0
BASE(6,7,8)
ATYPE = 0,0,0
TABLE(0,40,10,32,85,3600,3600,3600, 0, 0, 0 )
UNITS = 100,100,100
BASE(0,1,2)
CONNFRAME(2,0,6,7,8)
WAIT LOADED
FOR i=0 TO 2
TABLE(100+i)=DPOS(i+6)
NEXT
BASE(6,7,8)
FRAME_ROTATE(0,0,0,PI/2,0,0)
BASE(6,7,8)
FRAME_ROTATE(0,0,0,PI/2,0,0)
WAIT LOADED
ret=FRAME_ROTATE2(100,200,1,0,0,0,PI/2,0,0)
IF ret=-1 THEN
    ?"计算值"
    ?"DPOS(6)=",TABLE(200)
    ?"DPOS(7)=",TABLE(201)
    ?"DPOS(8)=",TABLE(202)
    ?"比较值"
    ?"DPOS(6)比较",TABLE(200)-DPOS(6)
    ?"DPOS(7)比较",TABLE(201)-DPOS(7)
    ?"DPOS(8)比较",TABLE(202)-DPOS(8)
ENDIF

```

输出行输出结果:

```

计算值
DPOS(6)=0
DPOS(7)=-58.1400
DPOS(8)=0.0000
比较值
DPOS(6)比较 0
DPOS(7)比较 0
DPOS(8)比较 0.0000

```

例二 以 FRAME=1, SCARA 为例, 对其相对于 Z 轴旋转 90 度。

```

BASE(0,1,2,3)
RAPIDSTOP
ATYPE = 1,1,1,1
UNITS=3600/360,3600/360,3600/360,1000

```

	<pre> DPOS=0,0,0,0 BASE(6,7,8,9) ATYPE = 0,0,0,0 TABLE(0,100,100,3600,3600,3600) UNITS = 100,100,3600/360,1000 BASE(0,1,2,3) CONNFRAME(1,0,6,7,8,9) WAIT LOADED FOR i=0 TO 3 TABLE(100+i)=DPOS(i+6) NEXT BASE(6,7,8,9) FRAME_ROTATE(0,0,0,0,0,PI/2) WAIT LOADED RET=FRAME_ROTATE2(100,200,1,0,0,0,0,0,PI/2) IF RET=-1 THEN ?"计算值" ?"DPOS(6)=",TABLE(200) ?"DPOS(7)=",TABLE(201) ?"DPOS(8)=",TABLE(202) ?"DPOS(9)=",TABLE(203) ?"比较值" ?"DPOS(6)比较",TABLE(200)-DPOS(6) ?"DPOS(7)比较",TABLE(201)-DPOS(7) ?"DPOS(8)比较",TABLE(202)-DPOS(8) ?"DPOS(9)比较",TABLE(203)-DPOS(9) ENDIF 输出行输出结果： 计算值 DPOS(6)=-0.0000 DPOS(7)=-200 DPOS(8)=0 DPOS(9)=0 比较值 DPOS(6)比较 -0.0000 DPOS(7)比较 0 DPOS(8)比较 0 </pre>
相关指令	FRAME_ROTATE

WORLD_DPOS -- 世界坐标系

类型	轴状态
----	-----

描述	虚拟轴参照世界坐标系的坐标值，没有旋转时与 DPOS 相同。
语法	var1=WORLD_DPOS(axis)
适用控制器	通用
例程	在线命令打印 >>?*WORLD_DPOS

MOVER_L/MOVER_LABS -- 关节轴直线插补

类型	运动指令
描述	关节轴直线插补。 机械手关节插补运动，机械手末端直线运动到指定坐标。 此指令正解模式下使用，直接操作关节轴时会有改变姿态的可能，所以要确保起点结束点姿态保持一致，否则会报错。 MOVER 相对指令也要 WAIT IDLE 等待停止完成。
语法	MOVER_L(distance1 [,distance2 [,distance3 [,distance4...]]]) distance1: 第一个轴运动距离 distance2: 下一个轴运动距离
适用控制器	4 系列 20170511 以上固件支持
例程	<pre> BASE(0,1) DPOS=0,0 BASE(6,7) ATYPE = 0,0 '设置为虚拟轴 UNITS=1000,1000 TABLE(0,L1,L2, 100*360, 100*360, 360) CONNREFRAME(1,0,0,1) '第 6/7 轴作为虚拟的 XY 轴，启动连接 WAIT LOADED '关节运动 BASE(0,1) SPEED=400 SRAMP=100 ACCEL=1000 DECEL=1000 MERGE = 1 CORNER_MODE=32 '启动倒角 ZSMOOTH=2 MOVEABS(45,90) '关节运动，此时运动关节角度 MOVER_LABS(90, 0) '末端直线运动 WAIT IDLE '等待运动停止 PRINT *DPOS </pre>
相关指令	MOVER_C ， MOVER_C3

MOVER_C/MOVER_CABS -- 关节轴平面圆弧

类型	运动指令										
描述	关节轴直接走圆弧插补运动。 此指令正解模式下使用。 BASE 虚拟的 XYZ 轴, 否则无法确定 XYZ, 此时的参数也是虚拟轴的距离。 MOVER 相对指令也要 WAIT IDLE 等待停止完成。										
语法	MOVER_C/MOVER_CABS (end1,end2,centre1,centre2,mode,[dis1,...,disn]) end1: 第 1 个轴运动距离参数 1 end2: 第 2 个轴运动距离参数 1 centre1: 第 1 个轴运动距离参数 2 centre2: 第 2 个轴运动距离参数 2 mode: 模式 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>当前点, 中间点, 终点三点定圆弧。 距离参数 1 为终点距离, 距离参数 2 为中间点距离。</td></tr> <tr> <td>1</td><td>当前点, 圆心, 终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离, 距离参数 2 为圆心的距离。</td></tr> <tr> <td>2</td><td>当前点, 中间点, 终点三点定圆。 距离参数 1 为终点距离, 距离参数 2 为中间点距离。</td></tr> <tr> <td>3</td><td>当前点, 圆心, 终点定圆。 先走最短的圆弧, 再继续走完整圆。 距离参数 1 为终点距离, 距离参数 2 为圆心的距离。</td></tr> </tbody> </table> dis1- disn: 螺旋轴的距离	值	描述	0	当前点, 中间点, 终点三点定圆弧。 距离参数 1 为终点距离, 距离参数 2 为中间点距离。	1	当前点, 圆心, 终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离, 距离参数 2 为圆心的距离。	2	当前点, 中间点, 终点三点定圆。 距离参数 1 为终点距离, 距离参数 2 为中间点距离。	3	当前点, 圆心, 终点定圆。 先走最短的圆弧, 再继续走完整圆。 距离参数 1 为终点距离, 距离参数 2 为圆心的距离。
值	描述										
0	当前点, 中间点, 终点三点定圆弧。 距离参数 1 为终点距离, 距离参数 2 为中间点距离。										
1	当前点, 圆心, 终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离, 距离参数 2 为圆心的距离。										
2	当前点, 中间点, 终点三点定圆。 距离参数 1 为终点距离, 距离参数 2 为中间点距离。										
3	当前点, 圆心, 终点定圆。 先走最短的圆弧, 再继续走完整圆。 距离参数 1 为终点距离, 距离参数 2 为圆心的距离。										
适用控制器	4 系列 20170511 以上固件支持										
例程	<pre> L1 = 500 L2 = 500 TABLE(0,L1,L2, 100*360, 100*360, 360) '参数存储在 TABLE0 开始的位置, 电机一圈 360 个脉冲 BASE(6,7) CONNREFRAME(1,0,0,1) '第 6/7 轴作为虚拟的 XY 轴, 启动连接 WAIT LOADED '等待运动加载 BASE(6,7) 'REFRAME 直接 MOVER 运动虚拟轴, 会自动转换到关节轴上 MOVER_LABS(500) MOVER_C(500,0, 250,250, 0) </pre>										
相关指令	MOVER_L , MOVER_C3										

MOVER_C3/MOVER_C3ABS -- 关节轴空间圆弧

类型	运动指令
描述	关节轴直接走空间圆弧插补运动。

	此指令正解模式下使用。 BASE 虚拟的 XYZ 轴，否则无法确定 XYZ，此时的参数也是虚拟轴的距离。 MOVER 相对指令也要 WAIT IDLE 等待停止完成。										
语法	MOVER_C3 (endx, endy, endz, midx, midy, midz, mode[, dis1][,dis2][,dis3]) end1: 第 1 个轴运动距离参数 1 end2: 第 2 个轴运动距离参数 1 end3: 第 3 个轴运动距离参数 1 centre1: 第 1 个轴运动距离参数 2 centre2: 第 2 个轴运动距离参数 2 centre3: 第 3 个轴运动距离参数 2 mode: 模式 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>当前点，中间点，终点三点定圆弧。 距离参数 1 为终点距离，距离参数 2 为中间点距离。</td></tr> <tr> <td>1</td><td>当前点，圆心，终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。</td></tr> <tr> <td>2</td><td>当前点，中间点，终点三点定圆。 距离参数 1 为终点距离，距离参数 2 为中间点距离。</td></tr> <tr> <td>3</td><td>当前点，圆心，终点定圆。 先走最短的圆弧，再继续走完整圆。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。</td></tr> </tbody> </table> dis1- disn: 螺旋轴的距离	值	描述	0	当前点，中间点，终点三点定圆弧。 距离参数 1 为终点距离，距离参数 2 为中间点距离。	1	当前点，圆心，终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。	2	当前点，中间点，终点三点定圆。 距离参数 1 为终点距离，距离参数 2 为中间点距离。	3	当前点，圆心，终点定圆。 先走最短的圆弧，再继续走完整圆。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。
值	描述										
0	当前点，中间点，终点三点定圆弧。 距离参数 1 为终点距离，距离参数 2 为中间点距离。										
1	当前点，圆心，终点定圆弧。 走最短的圆弧。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。										
2	当前点，中间点，终点三点定圆。 距离参数 1 为终点距离，距离参数 2 为中间点距离。										
3	当前点，圆心，终点定圆。 先走最短的圆弧，再继续走完整圆。 距离参数 1 为终点距离，距离参数 2 为圆心的距离。										
适用控制器	4 系列 20170511 以上固件支持										
例程	<pre> L1 = 500 L2 = 500 TABLE(0,L1,L2, 100*360, 100*360, 360) '参数存储在 TABLE0 开始的位置，电机 一圈 360 个脉冲 BASE(6,7) CONNREFRAME(1,0,0,1) '第 6/7 轴作为虚拟的 XY 轴，启动连接 WAIT LOADED '等待运动加载 BASE(6,7,8) 'CONNREFRAME 直接 MOVER 运动虚拟轴，会自动转换到关节轴上 MOVER_LABS(400) MOVER_C3ABS(200,0,0,600,400,0, 0) </pre>										
相关指令	MOVER_L ， MOVER_C										

FRAME_CAL -- 参数矫正

类型	机械手计算指令
描述	根据机械手关节示教的坐标和特点自动校正当前的机械手参数。 Tablein 中存储的关节轴坐标获取，用当前的零点位置与机械手参数建立机械手链接前

	提下，控制机械手末端点运动到校正点，取校正点的关节轴坐标。 校正当前零点位置与理论零点的偏差，计算出在理论零点的关节轴坐标。 校正机械手参数的值（校正部分参数），计算出理论的机械参数的值。 FRAME_CAL 只做计算，指令返回值为-1 计算成功，返回 0 计算失败。 FRAME_CAL 的 BASE 轴必须是在 FRAME 状态的轴。
语法	FRAME_CAL(tablein,space,groups,tableaux, zeroout, [tableout2]) tablein: 关节坐标存储的 table 起始编号，每个点的关节坐标按顺序存储，多个点之间间隔 space space: 每两个点之间间隔的 table 元素 groups: 点数 tableaux: 辅助参数的 table 编号，部分 frame 需要 zeroout: 计算出来理论零点时关节轴的绝对坐标的 table 编号 tableout2: 计算出来的机械手参数存储的位置，不填时直接存储原参数的位置
适用控制器	通用
例程	查看机械手指令说明手册章节

MOVER2_J/MOVER2_JABS -- 机械手点位运动

类型	机械手运动指令
描述	机械手末端通过点到点（PTP）关节插补联动运行到各关节的目标点。 输入是各关节轴值。该机械手指令对 base 设置的关节轴有效。输入的是各关节轴角度值，单位：度 MOVER2_JABS——表示绝对位置方式下点到点运动 MOVER2_J——表示增量位置方式下点到点运动 速度设置具体可见： 该说明书“点位运动和轨迹运动的速度、加速度设置”
语法	MOVER2_J/MOVER2_JABS(End_Joint1,End_Joint2,End_Joint3,End_Joint4,...) End_Joint1: 机械手第一个关节轴的目标值；单位：度 End_Joint2: 机械手下一个关节轴的目标值；单位：度
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(6,7,8,9,10,11) '选择虚拟轴号 CONNREFRAME(6,0,0,1,2,3,4,5) '六自由度机械手模型，第 0~5 轴作为关节轴号?"正解模式" BASE(0,1,2,3,4,5) '选择 012345 关节轴号 FORCE_SPEED=0.9 '设置下面执行的 MOVER2_JABS 点位运动速度为 0.9，即速度设置为满速的 90% MOVER2_JABS(60,60,30,20,0,90) '六轴机械手 PTP 运行到关节轴目标点，输入的目标点为各关节轴的角度值(60,60,30,20,0,90)。
相关指令	MOVER2_P/MOVER2_PABS

MOVER2_P/MOVER2_PABS -- 机械手点位运动

类型	机械手运动指令
描述	机械手末端通过点到点（PTP）关节插补联动运行到指定的目标点位姿。该指令与 MOVER2_J 功能一样，只是输入的是目标点位姿。 该机械手指令对 base 设置的虚拟轴有效。
语法	MOVER2_P/MOVER2_PABS(Fstatus,End_Pos1,End_Pos2,End_Pos3,End4,...) Fstatus: 若 6 自由度机械手，表示确定唯一解的类型值，取值范围 0~7；若是 SCARA，则是左右手系，0 表示右手系，1 表示左手系。-1 表示当前的姿态。 End_Pos1: 机械手虚拟轴的第 1 轴目标值（如 X 轴）；单位：mm End_Pos2: 机械手虚拟轴的第 2 轴目标值（如 Y 轴）；单位：mm End_Pos3: 机械手虚拟轴的第 3 轴目标值（如 Z 轴）；单位：mm End4: 若是六轴机械手，表示目标点姿态的欧拉角 RX 角；若是 SCARA，表示直线目标点姿态的 RX 轴旋转角。单位：度 End5: 若是六自由度机械手，需填写该值，表示直线目标点姿态的欧拉角 RY 角；而 SCARA 不需填写。单位：度 End6: 若是六自由度机械手，需填写该值，表示直线目标点姿态的欧拉角 RZ 角；而 SCARA 不需填写。单位：度 若后面 End7、End8、End9 等继续添加的参数，则为外扩轴的目标值，量纲单位根据用户设置的情况，可设置为旋转轴（单位：度）或移动轴（单位：mm）。
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(6,7,8,9,10,11) '选择虚拟轴号 CONNREFRAME(6,0,0,1,2,3,4,5) '六自由度机械手模型，第 0~5 轴作为关节轴号?"正解模式" BASE(6,7,8,9,10,11) FORCE_SPEED=0.8 '设置下面执行的 MOVER2_PABS 点位运动速度为 0.8，即速度设置为满速的 80% MOVER2_PABS(3,374,0,630,0,90,0) '表示机械手走 PTP 关节插补运行到目标坐标值 (374,0,630)，目标姿态值(0,90,0)，处于 Fstatus=3 的唯一解区域。 MOVER2_PABS(-1,189,-160,600,0,90,0); '表示机械手走直线路径运行到目标坐标值 (189,-160,600)，目标姿态值(0,90,0)不变，依然处于 Fstatus=3 的唯一解区域。
相关指令	MOVER2_J/MOVER2_JABS

MOVER2_L/MOVER2_LABS -- 机械手直线运动

类型	机械手运动指令
描述	机械手末端直线运动到指定的目标点位姿。该机械手指令对 base 设置的虚拟轴有效。 速度设置具体可见：该说明书“点位运动和轨迹运动的速度、加速度设置”

	MOVER2_LABS——表示绝对位置方式下走直线 MOVER2_L——表示增量位置方式下走直线
语法	MOVER2_L/MOVER2_LABS(Fstatus,End_Pos1,End_Pos2,End_Pos3,End4,End5...) Fstatus: 若是 6 自由度机械手, 表示确定唯一解的类型值, 取值范围 0~7; 若是 SCARA, 则是左右手系, 0 表示右手系, 1 表示左手系。-1 表示当前的姿态。 End_Pos1: 机械手虚拟轴的第 1 轴目标值 (如 X 轴); 单位: mm End_Pos2: 机械手虚拟轴的第 2 轴目标值 (如 Y 轴); 单位: mm End_Pos3: 机械手虚拟轴的第 3 轴目标值 (如 Z 轴); 单位: mm End4: 若是六轴机械手, 表示目标点姿态的欧拉角 RX 角; 若是 SCARA, 表示直线目标点姿态的 Rx 轴旋转角。单位: 度 End5: 若是六自由度机械手, 需填写该值, 表示直线目标点姿态的欧拉角 RY 角; 而 SCARA 不需填写。单位: 度 End6: 若是六自由度机械手, 需填写该值, 表示直线目标点姿态的欧拉角 RZ 角; 而 SCARA 不需填写。单位: 度 若后面 End7、End8、End9 等继续添加的参数, 则为外扩轴的目标值, 量纲单位根据用户设置的情况, 可设置为旋转轴 (单位: 度) 或移动轴 (单位: mm)。 备注: 1) MOVER2_L 为相对位姿的增量编程方式。对于 frame102 的两轴并联机械手, 只有二维坐标值, 则只设置 End_Pos1 和 End_Pos2。 2) 直线轨迹的线速度及加速度设置可参考如下示例。
适用控制器	4 系列 20170511 以上固件支持
例程	<pre> "*****"电机、机械手参数定义 dim R1 '上间距半径 dim r2 '下间距半径 dim L1 '上杆长度 dim L2 '下杆长度 R1 = 40 delta 机械手结构相关参数 r2 = 27 L1 = 120 L2 = 230 dim u_m1 '电机 1 一圈脉冲数 dim u_m2 '电机 2 一圈脉冲数 dim u_m3 '电机 3 一圈脉冲数 dim u_m4 '电机 4 一圈脉冲数 u_m1=480000 u_m2=480000 u_m3=480000 u_m4=640000 dim i_1 '关节 1 传动比 dim i_2 '关节 1 传动比 dim i_3 '关节 1 传动比 dim i_4 '关节 1 传动比 </pre>


```

i_1=31
i_2=31
i_3=31
i_4=1

dim u_j1      '关节 1 转一圈的电机脉冲数
dim u_j2      '关节 2 转一圈的电机脉冲数
dim u_j3      '关节 3 转一圈的电机脉冲数
dim u_j4      '末端旋转一圈的电机脉冲数
u_j1=u_m1*i_1
u_j2=u_m2*i_2
u_j3=u_m3*i_3
u_j4=u_m4*i_4

"*****"关节轴设置"*****"
base(0,1,2,3)      '定义实际轴
'atype = 1,1,1,1    '设置为脉冲轴
units=u_j1/360,u_j2/360,u_j3/360,u_j4/360  'units 设为每°的脉冲
FS_LIMIT = 90,90, 150,720    '机械手各关节轴正负限值
RS_LIMIT = -90,-90, -150,-720
dpos=0,0,0,0      '设置关节轴的位置，此处要根据实际情况来修改
mpos=0,0,0,0
SPEED=500,500,500,1500      '设置各关节轴的最大速度
ACCEL=10000,10000,10000,30000  '设置各关节轴的最大加速度和减速度
DECEL=10000,10000,10000,30000

"*****"虚拟轴设置"*****"
base(6,7,8,9)
atype = 0,0,0,0      '设置为虚拟轴
TABLE(0,R1,r2,L1,L2,u_j1,u_j2,u_j3, 0, 0, 0, u_j4 )
UNITS = 1000,1000,1000,u_j4/360      '此脉冲当量要提前设置，中途不能变化
FS_LIMIT = 20000000,20000000, 20000000,20000000
RS_LIMIT = -20000000,-20000000, -20000000,-20000000
accel=30000,30000,30000,30000
decel=30000,30000,30000,30000
FORCE_SPEED(6) = 3000  '直线轨迹线速度设置，6 为虚拟主轴号；只有虚拟主轴号的
                        线速度 3000 起作用，虚拟轴号 7、8 的速度不起作用。
FORCE_SPEED(9) = 1500  '末端旋转轴 Rx 旋转速度设置；9 为 delta 旋转轴轴号

base(6,7,8,9)      '选择 delta 机械手虚拟轴号
CONNREFRAME(14,0,0,1,2,3)      '第 0/1/2/3 轴为关节轴，启动正解连接
wait loaded
wa(10)
?"正解模式"

```

	SYSTEM_ZSET=0'看分轴的速度 trigger 'delta 工艺点位 base(6,7,8,9) mover2_labs(-1,116.56,8.92117,-211,89.66428) wait idle(0) BASE(6,7,8,9)'选择虚拟轴号 CONNREFRAME(14,0,0,1,2,3)'启动正解连接。 WAIT LOADED'等待运动加载，此时会自动调整虚拟轴的位置。 ?"正解模式" BASE(6,7,9,8) SPEED_RATIO = 1.0 '速度比例值为 1.0 ZSMOOTH=100 '平滑系数设置为 100 MOVER2_LABS(1,300,200,-50,0) '表示 SCARA 机械手从当前位姿走直线路径运行到目标点(300,200,-50,0)，手系为左手系。 MOVER2_LABS(-1,400,300,-100,0); '表示 SCARA 机械手走直线路径运行到目标点(400,300,-100,0)，手系依然为左手系。
相关指令	

MOVER2_ARCHP/MOVER2_ARCHPABS -- 机械手点位拱形

类型	机械手运动指令（主要针对 scara 机械手）
描述	点位拱形运动（也称蛙跳指令），机械手末端拱形方式蛙跳一定高度再下降到目标点。如下图所示，点位拱形是由三段 PTP 形成的运动指令。该机械手指令对 base 设置的虚拟轴有效。 速度设置具体可见：该说明书“点位运动和轨迹运动的速度、加速度设置”
语法	MOVER2_ARCHP(H_Pos,H1,H2,Fstatus,End_Pos1,End_Pos2,End_Pos3,End4,...) H_Pos: 虚拟轴第 3 轴蛙跳的最大高度坐标值，即第 3 轴的绝对位置（如 Z 轴）；单位 mm H1: 虚拟轴第 3 轴从起始点的提升高度；单位：mm H2: 虚拟轴第 3 轴到目标点的下降高度；单位：mm Fstatus: 若是 SCARA，则是左右手系，0 表示右手系，1 表示左手系；若是六轴，表示确定唯一解的类型值，取值范围 0~7。-1 表示当前的姿态。 End_Pos1: 机械手虚拟轴的第 1 轴目标值（如 X 轴）；单位：mm End_Pos2: 机械手虚拟轴的第 2 轴目标值（如 Y 轴）；单位：mm End_Pos3: 机械手虚拟轴的第 3 轴目标值（如 Z 轴）；单位：mm End4: 若是六轴机械手，表示目标点姿态的欧拉角 RX 角；若是 SCARA，表示直线目标点姿态的 Rx 轴旋转角。单位：度

	End5: 若是六自由度机械手, 需填写该值, 表示直线目标点姿态的欧拉角 RY 角; 而 SCARA 不需填写。单位: 度 End6: 若是六自由度机械手, 需填写该值, 表示直线目标点姿态的欧拉角 RZ 角; 而 SCARA 不需填写。单位: 度 若后面 End7、End8、End9 等继续添加的参数, 则为外扩轴的目标值, 量纲单位根据用户设置的情况, 可设置为旋转轴 (单位: 度) 或移动轴 (单位: mm)。 备注: 1) 拱形运动两边的提升高度是由 H1 和 H2 确定的, 与 ZSMOOTH 无关。拱形运动倒角的曲线形态会随着各关节最大加速度设置的不同, 会有些变化。 2) 对于 frame102 的两轴并联机械手, 只有 2 维坐标值, 只设置 End_Pos1 和 End_Pos2, H1 和 H2 设置为 0, H_Pos 和 P1、P2 高度一致。
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(6,7,9,8) '选择虚拟轴号 CONNREFRAME(1,0,0,1,3,2) '启动正解连接; 形成 SCARA 机械手正解模型 WAIT LOADED '等待运动加载, 此时会自动调整虚拟轴的位置。 ?"正解模式" BASE(6,7,8,9) MOVER2_PABS(0, 300, -100, -50,0) '右手系下 PTP 运行到目标点 300,-100,-50 MOVER2_ARCHPABS(-25, 10, 20, 0, 300, -400, -50, 0) '右手系下, 拱形运行的 Z 轴方向最高位置限制在-25mm, 从起点 Z 轴提升高度 10mm, Z 轴下降高度 20 到目标点位置 300,-400,-50。
相关指令	MOVER2_ARCHPL、MOVER2_ARCHPLABS

MOVER2_ARCHPL/MOVER2_ARCHPLABS -- 直线拱形

类型	机械手运动指令 (主要针对 delta、码垛机械手, 非 scara)
描述	直线拱形运动 (也称蛙跳指令); 机械手末端拱形方式蛙跳一定高度再下降到目标点。如下图所示, 直线拱形是由三段直线轨迹形成的运动指令。该机械手指令对 base 设置的虚拟轴有效。
语法	MOVER2_ARCHPL(H_Pos,H1,H2,Fstatus,End_Pos1,End_Pos2,End_Pos3,End4,...) H_Pos: 虚拟轴第 3 轴蛙跳的最大高度坐标值, 即第 3 轴的绝对位置 (如 Z 轴); 单位 mm H1: 虚拟轴第 3 轴从起始点的提升高度; 单位: mm H2: 虚拟轴第 3 轴到目标点的下降高度; 单位: mm

	Fstatus: 若是 SCARA, 则是左右手系, 0 表示右手系, 1 表示左手系; 若是六轴, 表示确定唯一解的类型值, 取值范围 0~7。-1 表示当前的姿态。 End_Pos1: 机械手虚拟轴的第 1 轴目标值 (如 X 轴); 单位: mm End_Pos2: 机械手虚拟轴的第 2 轴目标值 (如 Y 轴); 单位: mm End_Pos3: 机械手虚拟轴的第 3 轴目标值 (如 Z 轴); 单位: mm End4: 若是六轴机械手, 表示目标点姿态的欧拉角 RX 角; 若是 SCARA, 表示直线目标点姿态的 Rx 轴旋转角。单位: 度 End5: 若是六自由度机械手, 需填写该值, 表示直线目标点姿态的欧拉角 RY 角; 而 SCARA 不需填写。单位: 度 End6: 若是六自由度机械手, 需填写该值, 表示直线目标点姿态的欧拉角 RZ 角; 而 SCARA 不需填写。单位: 度 若后面 End7、End8、End9 等继续添加的参数, 则为外扩轴的目标值, 量纲单位根据用户设置的情况, 可设置为旋转轴 (单位: 度) 或移动轴 (单位: mm)。 备注: 1) 拱形运动两边的提升高度是由 H1 和 H2 确定的, 与 ZSMOOTH 无关。拱形运动倒角的曲线形态会随着各关节最大加速度设置的不同, 会有些变化。 2) 对于 frame102 的两轴并联机械手, 只有 2 维坐标值, 只设置 End_Pos1 和 End_Pos2, H1 和 H2 设置为 0, H_Pos 和 P1、P2 高度一致。
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(6,7,8,9) '选择虚拟轴号 CONNREFRAME(1,0,0,1,2,3) '启动正解连接。 WAIT LOADED '等待运动加载, 此时会自动调整虚拟轴的位置。 ?"正解模式" BASE(6,7,8,9) MOVER2_PABS(0, 300, -100, -50, 0) '右手系下 PTP 运行到目标点 300,-100,-50 MOVER2_ARCHPLABS(-25, 10, 20, 0, 300, -400, -50, 0) '右手系下, 拱形运行的 Z 轴方向最高位置限制在-25mm, 从起点 Z 轴提升高度 10mm, Z 轴下降高度 20 到目标点位置 300,-400,-50。
相关指令	MOVER2_ARCHP、MOVER2_ARCHP3

MOVER2_ARCHP3/MOVER2_ARCHP3ABS -- 三维拱形

类型	机械手运动指令 (主要针对 6 自由度、码垛机械手, 非 scara)
描述	三维拱形运动 (也称蛙跳指令); 机械手末端拱形方式蛙跳一定高度再下降到目标点。

	如下图所示，直线拱形是由一段直线+PTP+一段直线轨迹形成的运动指令。该机械手指令对 base 设置的虚拟轴有效。
语法	MOVER2_ARCHP3(H_Pos,H1,H2,Fstatus,End_Pos1,End_Pos2,End_Pos3,End4,...) H_Pos: 虚拟轴第 3 轴蛙跳的最大高度坐标值，即第 3 轴的绝对位置（如 Z 轴）；单位 mm H1: 虚拟轴第 3 轴从起始点的提升高度；单位：mm H2: 虚拟轴第 3 轴到目标点的下降高度；单位：mm Fstatus: 若是 SCARA，则是左右手系，0 表示右手系，1 表示左手系；若是六轴，表示确定唯一解的类型值，取值范围 0~7。-1 表示当前的姿态。 End_Pos1: 机械手虚拟轴的第 1 轴目标值（如 X 轴）；单位：mm End_Pos2: 机械手虚拟轴的第 2 轴目标值（如 Y 轴）；单位：mm End_Pos3: 机械手虚拟轴的第 3 轴目标值（如 Z 轴）；单位：mm End4: 若是六轴机械手，表示目标点姿态的欧拉角 RX 角；若是 SCARA，表示直线目标点姿态的 Rx 轴旋转角。单位：度 End5: 若是六自由度机械手，需填写该值，表示直线目标点姿态的欧拉角 RY 角；而 SCARA 不需填写。单位：度 End6: 若是六自由度机械手，需填写该值，表示直线目标点姿态的欧拉角 RZ 角；而 SCARA 不需填写。单位：度 若后面 End7、End8、End9 等继续添加的参数，则为外扩轴的目标值，量纲单位根据用户设置的情况，可设置为旋转轴（单位：度）或移动轴（单位：mm）。 备注: 1) 拱形运动两边的提升高度是由 H1 和 H2 确定的，与 ZSMOOTH 无关。拱形运动倒角的曲线形态会随着各关节最大加速度设置的不同，会有些变化。 2) 对于 frame102 的两轴并联机械手，只有 2 维坐标值，只设置 End_Pos1 和 End_Pos2，H1 和 H2 设置为 0，H_Pos 和 P1、P2 高度一致。
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(6,7,8,9) '选择虚拟轴号 CONNREFRAME(1,0,0,1,2,3) '启动正解连接。 WAIT LOADED '等待运动加载，此时会自动调整虚拟轴的位置。 ?"正解模式" BASE(6,7,8,9) MOVER2_PABS(0, 300, -100, -50,0) ‘右手系下 PTP 运行到目标点 300,-100,-50 MOVER2_ARCHP3ABS(-25, 10, 20, 0, 300, -400, -50, 0) ‘右手系下，拱形运行的 Z 轴方向最高位置限制在-25mm，从起点 Z 轴提升高度 10mm，Z 轴下降高度 20 到目标点位置 300,-400,-50。

相关指令

MOVER2_ARCHP、MOVER2_ARCHPL

MOVER2_ARC/MOVER2_ARCABS -- 机械手圆弧插补

类型	机械手运动指令
描述	空间圆弧运动，即也包含了平面圆弧；输入参数是圆弧的中间点和终点。 该机械手指令对 base 设置的虚拟轴有效。
语法	MOVER2_ARC(Fstatus, midx, midy, midz, endx, endy, endz, enda, endb, endc) Fstatus: 若是 SCARA，则是左右手系，0 表示右手系，1 表示左手系；若是六轴，表示确定唯一解的类型值，取值范围 0~7。-1 表示当前的姿态。 midx: 圆弧中间点的 X 轴绝对位置值；单位：mm midy: 圆弧中间点的 Y 轴绝对位置值；单位：mm midz: 圆弧中间点的 Z 轴绝对位置值；单位：mm endx: 圆弧目标点的 X 轴绝对位置值；单位：mm endy: 圆弧目标点的 Y 轴绝对位置值；单位：mm endz: 圆弧目标点的 Z 轴绝对位置值；单位：mm enda: 若是六自由度机械手，表示圆弧目标点姿态的欧拉角 RX 角；若是 SCARA，表示圆弧目标点姿态的 Rx 轴旋转角。单位：度 endb: 若是六自由度机械手，需填写该值，表示圆弧目标点姿态的欧拉角 RY 角；而 SCARA 不需填写。单位：度 endc: 若是六自由度机械手，需填写该值，表示圆弧目标点姿态的欧拉角 RZ 角；而 SCARA 不需填写。单位：度 若后面 End7、End8、End9 等继续添加的参数，则为外扩轴的目标值，量纲单位根据用户设置的情况，可设置为旋转轴（单位：度）或移动轴（单位：mm）。
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(6,7,8,9) '选择虚拟轴号 CONNREFRAME(1,0,0,1,2,3) '启动正解连接。 WAIT LOADED '等待运动加载，此时会自动调整虚拟轴的位置。 ?"正解模式" BASE(6,7,8,9) MOVER2_ARC(0, 50, 50, -10, 100, 0, -10, 0) ‘右手系下，机械手经中间点(50,50,-10)圆弧插补运行到目标点(100,0,-10)。
相关指令	

MOVER2_LIMIT -- 选择关节轴速度和加速度约束类型

类型	机械手关节轴约束相关的指令
描述	机械手走直线或圆弧轨迹运行时，受各关节轴最大速度和加速度约束的程度。设置新的 mode 值后，后面执行的 basic 指令会一直生效。默认值为 0
语法	MOVER2_LIMIT(mainAxis)=mode

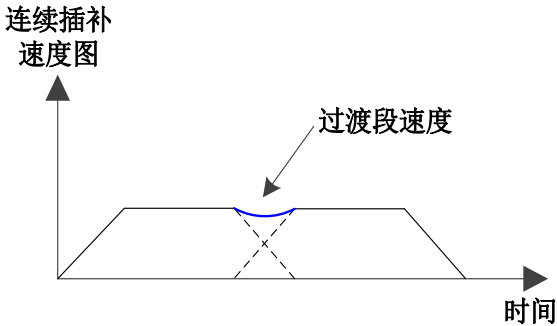
	mainAxis: 每个机械手自身的关节主轴号；不能输入机械手其它轴号 mode: 模式 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>默认值。低速下走轨迹不受关节轴约束，见如下备注</td></tr> <tr> <td>1</td><td>轨迹运行高速低速下整个都受各关节轴的最大速度和最大加速度约束</td></tr> <tr> <td></td><td></td></tr> </tbody> </table> 备注：mode 默认值为 0，表示机械手走轨迹线速度 FORCE_SPEED 不超过 1000mm/s，或者速度比例 SPEED_RATIO 不超过 50%，则不受关节轴的最大速度和加速度约束。	值	描述	0	默认值。低速下走轨迹不受关节轴约束，见如下备注	1	轨迹运行高速低速下整个都受各关节轴的最大速度和最大加速度约束		
值	描述								
0	默认值。低速下走轨迹不受关节轴约束，见如下备注								
1	轨迹运行高速低速下整个都受各关节轴的最大速度和最大加速度约束								
适用控制器	通用								
例程	BASE(6,7,8,9) '选择虚拟轴号 CONNREFRAME(1,10001,0,1,2,3) '启动正解连接。关节主轴号为 0 WAIT LOADED '等待运动加载，此时会自动调整虚拟轴的位置。 ?"正解模式" MOVER2_LIMIT(0)=1 ‘机械手走轨迹整个速度都受关节轴约束 BASE(6,7,8,9) FORCE_SPEED(6)=500 ‘机械手走直线的线速度为 500mm/s MOVER2_LABS(-1,400,300,-100,0) ‘走直线路径运行到目标点(400,300,-100,0)								

ZSMOOTH -- 机械手运动的平滑过渡系数

类型	轴参数
描述	base 指令轴设置机械手时，该指令含义如下： 设置机械手路径规划的平滑系数等级；取值越大，运动轨迹之间的平滑倒角越大。当用于 MOVER2 指令时，ZSMOOTH 的含义与用于 moveabs、MOVER 指令有区别。 若 base 关节轴，则设置关节主轴的 ZSMOOTH 值； 若 base 虚拟轴，则设置虚拟主轴的 ZSMOOTH 值（即虚拟轴的第 1 个轴号）。
语法	VAR1 = ZSMOOTH, ZSMOOTH = smoothdistance 取值范围：0~100。当取值 0，不进行路径平滑；当取值 100，路径平滑倒角越大，机器人运行效率和轨迹光顺度很高。ZSMOOTH 指令默认值为 0
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(6,7,8,9) '选择虚拟轴号 CONNREFRAME(1,0,0,1,2,3) '启动正解连接。 BASE(6,7,8,9) ZSMOOTH=100.0 ‘表示取 ZSMOOTH(6) 的值为 100；将机器人路径规划的平滑系数设置为 100 MOVER2_LABS(0,300,300) ‘末端运动该直线轨迹与下一段轨迹进行 100%平滑过渡处理 MOVER2_LABS(0,350,400) ZSMOOTH=20.0 ‘即 ZSMOOTH(6) 的值为 20

	MOVER2_LABS(0,240,360) '该直线轨迹与下一段轨迹进行 20%平滑处理 MOVER2_LABS(0,-350,-400) BASE(0,1,2,3) ZSMOOTH=50.0 '即 ZSMOOTH(0)的值 50 MOVER2_JABS(10,20,-50)
相关指令	

MOVER2_ZSMOFFSET -- 机械手连续插补平滑的偏移时间

类型	机械手平滑过渡相关指令
描述	连续插补平滑的时间偏移指令。作用是为了提高运动指令之间过渡的合成速度，如下所示，以提高节拍效率。原理是当上一段运动指令开始减速时，下一段运动指令开始加速，设置的插补偏移时间越大，减速则越少或不减速。 
语法	MOVER2_ZSMOFFSET(value) value: 连续运动过渡的平滑时间偏移值；单位：ms 1) 设置为-1，表示内部计算出最优值 2) 设置单位：ms，取值范围：0~1000ms 比如：MOVER2_ZSMOFFSET(60) '设置平滑偏移时间 60ms 3) 设置为 0，表示不进行时间偏移，但原有设置的 ZSMOOTH 值依然生效
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(6,7,8,9) '选择虚拟轴号 CONNREFRAME(1,0,0,1,2,3) '启动正解连接；SCARA 机械手 WAIT LOADED '等待运动加载，此时会自动调整虚拟轴的位置。 ?"正解模式" base(6,7,8,9) zsmooth = 100 '设置平滑过渡程度值 100 MOVER2_PABS(-1,2201.2, 4.1,0,17.5) mover2_zsmoffset(-1) '内部通过计算最优值来进行平滑时间偏移；表示上一条 MOVER2_PABS 与下一条 MOVER2_PABS 进行插补时间偏移。 MOVER2_PABS(-1,1137.2,941,200,68.9) mover2_zsmoffset(0) '表示上一条 MOVER2_PABS 与下一条 MOVER2_LABS 不进行插补时间偏移。

	MOVER2_LABS(-1,71.9,1876.3,-323.9,120.4) wait idle(0)
相关指令	

FORCE_SPEED -- 机械手运动的速度设置

类型	轴参数
描述	此处只对 MOVER2 点位运动单独设置每条运动指令的速度值，具体含义如下： 对于点位运动，通过 base 主轴号下 FORCE_SPEED 速度指令修改点位运动（PTP）多轴插补的运行速度。取值范围 0~1.0。当取值为 1 时，则点位运动是 100%满速运行。 对于 MOVER2_JABS 点位运动，因 base 关节轴，则取关节主轴号作为多轴插补运行速度； 对于 MOVER2_PABS 点位运动，因 base 虚拟轴，则取虚拟主轴号作为多轴插补运行速度（即虚拟轴的第 1 个轴号）。 该指令用于修改 MOVER2_JABS、MOVER2_PABS、MOVER2_ARCHP 等点位运动（PTP）运动类型的速度；当设置之后，机械手各关节轴的速度和加速度值都会柔性地变化。 具体可见：该说明书“点位运动和轨迹运动的速度、加速度设置”章节
语法	MOVER2_JABS、MOVER2_PABS 等点位运动（PTP）指令的速度设置，取值范围 0~1.0。
适用控制器	4 系列 20170511 以上固件支持
例程	BASE(0,1,2,3,4,5) '选择 012345 关节轴号 CONNFRAME(6,0,6,7,8,9,10,11) '六自由度机械手模型，第 6~11 轴作为虚拟的 XYZABC 轴 ?"正解模式" BASE(0,1,2,3,4,5) FORCE_SPEED=0.5 '设置 MOVER_JABS 点位运动的速度为 0.5,即将 MOVER_JABS 速度设置为满速的 50% MOVER_JABS(60,60,30,20,0,90) BASE(6,7,8,9,10,11) FORCE_SPEED=0.2 '设置 MOVER2_PABS 点位运动的速度为 0.2，即将该 MOVER_PABS 速度设置为满速的 20% MOVER2_PABS(-1,374,0,630,0,90,0) 'MOVER_PABS 将 20%速度运行 FORCE_SPEED=0.4 '将 MOVER_PABS 速度设置为满速的 40% MOVER2_PABS(-1,374,0,630,0,90,0) 'MOVER_PABS 将 40%速度运行
相关指令	SPEED_RATIO

MOVER2 点位运动和轨迹运动的速度、加速度设置

1. 设置速度和加速度

上述需求里会涉及到用户个性化设置点位运动（PTP 运动）、轨迹运动指令（如直线、圆弧）的速度和加速度。机械手的每个关节轴最大速度 SPEED 对应电机的最大转速，每个关节轴最大加速度 ACCEL 对应电机的额定扭矩等特征，**SPEED 和 ACCEL 设置后一般是不修改的**，因它们是每个关节轴的固有属性。所以，每个关节轴最大速度 SPEED 值都需要设置，若某个轴 SPEED 设置为 0 则会报 1008 错误。

若修改每个运动指令的加速度及减速度，不同运动类型的速度和加减速度值修改方式，如下表格所示：

	点位运动 (MOVER2_P、MOVER2_J、点位拱形 ARCHP)	轨迹运动 (MOVER2_L、MOVER2_ARC、直线拱形 ARCHPL)	轨迹运动末端的 旋转轴 RX 或姿态速度 (直线、圆弧、直线拱形 ARCHPL)
用户速度设置	通过 base 关节主轴号的 FORCE_SPEED 速度指令修改点位运行速度，取值范围 0~1.0，设置后速度及加速度都会柔性变化。设置为 1，则表示 100%满速运行。 举例，已知 0、1、2、3、4、5 和 6 轴为六自由度机械手的关节轴，其中 0 为关节主轴号，设置 $\text{FORCE_SPEED}(0)=0.6$，表示用户自定义的当前点位运动速度为满速的 60%。	轨迹运动指令的线速度通过已有 FORCE_SPEED 指令修改，单位：mm/s BASE(6,7,9,8) '选择虚拟轴号 CONNREFRAME(1,0,57,58,60,59) '启动正解连接。 BASE(6,7,9,8) MOVER2_LABS(0,364, 0, 0, -90) FORCE_SPEED = 1000 MOVER2_LABS(0,524.8, 160.8, 0, -10) FORCE_SPEED = 1500 MOVER2_LABS(0,524.8, -160.8, 0, -10)	通过已有 FORCE_SPEED 指令修改机械手的姿态偏摆速度，举例，已知 7、8、9、10、11 和 12 轴为六自由度机械手的虚轴，10 即为 6 关节机械手的姿态第 1 轴（即 RX 轴），则 FORCE_SPEED(10)速度是设置末端姿态的速度。再举例，scara 机械手 CONNREFRAME(1,30060,57,58,60,59)，60 为 RX 旋转轴，则 FORCE_SPEED(60)速度为设置的末端姿态速度。单位：度/s
用户加减速度设置	如下 basic 指令写法： $\text{FORCE_SPEED}(0)=0.4$ ‘表示设置 MOVER_JABS 点位运动的速度为满速的 40% MOVER_JABS (60, 60, 30, 20, 0, 90)	通过已有 ACCEL 和 DECEL 指令修改轨迹运行的加速度和减速度值。 举例，设置 7、8、9、10、11 和 12 轴为六自由度机械手的虚轴，则 ACCEL(7)的速度即为用户定义的轨迹线加速度值，DECEL(7)的速度即为用户定义的轨迹线减速度值。单位：mm/s²	通过已有 ACCEL 和 DECEL 指令修改姿态偏摆速度，举例，已知 7、8、9、10、11 和 12 轴为六自由度机械手的虚轴，则 ACCEL(10)的加速度为用户定义的末端姿态加速度，DECEL(10)即为用户定义的末端姿态减速度值。单位：度/s²

对于点位运动，通过修改关节主轴的 FORCE_SPEED 或 SPEED_RATIO 值来进行修改速度。对于轨迹运动（如直线或圆弧），通过设置虚拟主轴的 FORCE_SPEED 来确定走轨迹的线速度值。另外，机械手走

轨迹运动的末端旋转轴 RX 或末端姿态（如 6 关节轴机械手）的速度值，通过虚拟旋转轴的 FORCE_SPEED 进行单独设置。

对于轨迹运动（如直线或圆弧），若用户不想设定虚拟轴的轨迹线速度和线加速度，则进行如下设置：关节主轴的 CORNER_MODE = 1<<8，或者虚拟主轴的 FORCE_SPEED 设置为 0，或者虚拟主轴的加速度 accel 设置为 0。

对于轨迹运动，默认在中低速下，即 FORCE_SPEED 线速度不超过 1000mm/s，或者速度比例值 SPEED_RATIO 不超过 50%的时候，不受关节轴最大速度和加速度约束。若考虑包含中低速的整个速度都受关节轴约束，则通过 MOVER2_LIMIT 指令来设置。

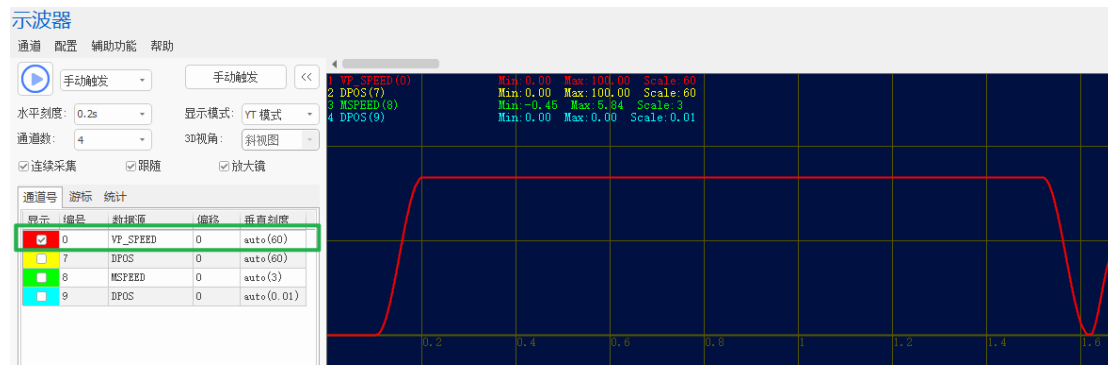
2. 修改速度比例 SPEED_RATIO

若通过 SPEED_RATIO 速度比例指令进行速度修改，所有 MOVER2 运动指令都是设置关节主轴的 SPEED_RATIO 值即可。例如，若关节主轴号为 0，需将运动速度整体变为满速的 90%，只需设置 SPEED_RATIO(0)=0.9。若运动前瞻缓存里分别修改每个运动指令的速度比例 SPEED_RATIO，则通过 MOVE_PARA 指令方式调用 SPEED_RATIO，写法可参考如下示例：

```
BASE(6,7,9,8) '选择虚拟轴号
CONNREFRAME(1,30060,57,58,60,59) '正解连接。SCARA 机械手，关节主轴号为 57
WAIT LOADED '等待运动加载完成
?"正解模式"
WA(200)
BASE(6,7,8,9)
MOVE_PARA(SPEED_RATIO,57,0.5) '将如下点位运动 MOVER2_PABS 速度比例值修改为 0.5
MOVER2_PABS(-1,300,450,-200)
MOVE_PARA(SPEED_RATIO,57,0.8) '将如下直线运动 MOVER2_LABS 速度比例值修改为 0.8
MOVER2_LABS(-1,300,450,-200)
WAIT IDLE(57)
```

3. 速度曲线显示设置

1) 默认 SYSTEM_ZSET 值为 1，速度曲线显示插补速度；例如，关节主轴号为 0，执行运动指令 MOVER2_LABS，则示波器上 VP_SPEED(0)显示的是直线轨迹的合成线速度值。



2) 若 SYSTEM_ZSET 设置为 0, 速度曲线显示单轴的速度。例如, 关节主轴号为 0, SYSTEM_ZSET=0, 执行运动指令 MOVER2_LABS, 则示波器上 VP_SPEED(0)显示的是机械手 57 号关节轴的单轴速度值。

第八章 程序结构与流程指令

8.1 程序符号

' -- 注释

类型	特殊字符
描述	后面全部是注释，直到下一行开始。
适用控制器	通用

_ -- 换行

类型	特殊字符
描述	后面换行继续。 在条件判断语句、内存存储、打印输出时尽量不要使用。
适用控制器	通用

: -- 标号

类型	语法结构
描述	用户过程标号，标号可以作为无参数的 SUB 过程来使用。
语法	Label 标号名称，不能与现有的关键词冲突。
适用控制器	通用
例子	<pre>GOTO label1 END '主程序结束 label1: '加：定义标号 END</pre>

8.2 数据定义指令

CONST -- 常量定义

类型	语法指令
描述	定义符号表示的常数，从而避免直接用数值表示。
语法	CONST CVARNAME = value

	CVARNAME: 常量名 value: 常量值
适用控制器	通用
例子	例一 CONST MAX_VALUE = 100000 '定义常量 TABLE(0)=MAX_VALUE 'table(0)赋值 10000 例二 GLOBAL CONST MAX_AXIS=6 '定义总轴数
相关指令	DIM

DIM -- 变量定义

类型	语法指令
描述	定义文件模块变量，数组。 当变量不定义就赋值时，会自动定义为文件模块变量。 文件模块变量只能在本程序文件内部使用。 数组可以作为字符串使用，一个元素表示一个字节。
语法	DIM varname, arrayname(space) varname: 变量名称 arrayname: 数组名称 space: 数组长度 5 系列以上控制器，22 年 2 月以后固件支持： 1. 变量定义初始化： DIM varname = 1 2. 数组定义初始化： DIM arrayname(size) = {1,2,3} DIM arrayname(size) = "string" 3. 结构定义初始化： DIM strname(size) as structname = {.item1 = 1, .item2 = {1,2,3} } 初始化赋值也可以在其他赋值语句中使用，如 GLOBAL。
适用控制器	通用
例子	DIM ARRAY1(100) '定义数组 ARRAY1 DIM VAR1 '定义变量 VAR1 VAR1 = 100 '赋值语句会自动定义文件模块变量 ARRAY1 = "asdf" ARRAY1(0,100,200,300) '对数组进行连续赋值 'ARRAY1(0)=100, ARRAY1(1)=200, ARRAY1(2)=300
相关指令	CONST , LOCAL , GLOBAL

LOCAL -- 局部定义

类型	语法指令
描述	定义局部变量，局部数组。 局部变量主要用于 SUB 中。 同一个 SUB 的局部变量有个数限制，SUB 过程的输入参数自动转化为局部变量。 不同任务的同一个 SUB 过程调用生成不同的局部变量，同一个任务的 SUB 过程递归调用也生成不同的局部变量。
语法	<pre>SUB subA() LOCAL localname 'localname 局部变量名 ENDSUB</pre>
适用控制器	通用
例子	<pre>SUB aaa() LOCAL v1 '定义局部变量 V1 v1=100 END SUB</pre>
相关指令	DIM , GLOBAL

GLOBAL -- 全局定义

类型	语法指令
描述	定义全局变量，数组；定义全局 SUB 过程。 全局定义的量可以在整个项目的任意程序文件中使用。
语法	语法 1: GLOBAL VAR1 语法 2: GLOBAL SUB SUB1() 语法 3: GLOBAL CONST CVARNAME = value 参数： VAR1: 变量名称 SUB1: 过程名称 CVARNAME: 常量名称 value: 常量值 5 系列以上控制器，22 年 2 月以后固件支持： 4. 变量定义初始化： <pre>GLOBAL varname = 1</pre> 5. 数组定义初始化： <pre>GLOBAL arrayname(size) = {1,2,3} GLOBAL arrayname(size) = "string"</pre> 6. 结构定义初始化： <pre>GLOBAL strname(size) as structname = {.item1 = 1, .item2 = {1,2,3} }</pre> 初始化赋值也可以在其他赋值语句中使用，如 DIM。
适用控制器	通用

例子	GLOBAL SUB g_sub2() '定义全局过程 g_sub2，可以在任意文件中使用 GLOBAL CONST g_convar = 100 '定义全局使用的常量 GLOBAL g_var2 '定义全局变量 g_var2
相关指令	DIM , LOCAL

8.3 数组操作指令

DMINS -- 数组链表插入

类型	语法指令
描述	数组的链表操作，插入后当前元素以及后面的所有元素往后移动位置。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	语法 1: DMINS arrayname(pos, size) 语法 2: DMINS aarrayname(pos) [,size] arrayname: 数组名称 pos: 数组索引 size: 要修改的个数， 语法 1 需注意指令在长度参数大于索引值时会报错
适用控制器	通用
例子	<pre> DIM aa(6) '定义数组 aa FOR i=0 TO 4 '赋值 0,1,2,3,4 aa(i)=i NEXT ?*aa '打印数组所有元素 DMINS aa(0) '插入元素 0，原有的所有元素都向后移动一个位置 aa(0) = 10 '为插入元素赋值 ?*aa '打印插入后的数组所有元素 </pre>
相关指令	DMDEL , DMCPY

DMADD -- 数组批量增加

类型	语法指令
描述	对数组元素值进行批量增加。 不要一次修改超过 500 元素。
语法	DMADD arrayname (pos, size , data) arrayname: 数组名 pos: 起始的索引 size: 要修改的个数，注意加上 pos 不要超过数组大小 data: 要增加的值
适用控制器	通用

例子	<pre> DIM aaa(20) '定义一个空间为 20 的数组 ?*aaa '打印，全为 0 DMADD aaa(10,5,2) '从元素 10 开始，修改 5 个元素值+2 ?*aaa '打印，其中 10、11、12、13、14 为 2，其余为 0 DMADD aaa(10,5,2) '从元素 10 开始，修改 5 个元素值+2 ?*aaa '打印，其中 10、11、12、13、14 为 4，其余为 0 </pre>
相关指令	DMINS , DMCPY

DMDEL -- 数组链表删除

类型	语法指令
描述	数组的链表操作；删除数组元素，删除后当前元素后面的所有元素向前移动。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	<pre> 语法 1: DMDEL arrayname(pos, size) 语法 2: DMDEL aarrayname(pos) [,size] arrayname: 数组名称 pos: 数组索引 size: 要修改的个数 </pre>
适用控制器	通用
例子	<pre> DIM aa(6) '定义数组 aa FOR i=0 TO 4 '赋值 0,1,2,3,4 aa(i)=i NEXT ?*aa '打印数组所有元素 DMDEL aa(0) '删除数组 a 的第 1 个元素 ?*aa '打印删除后的数组所有元素 </pre>
相关指令	DMINS , DMCPY

DMCPY -- 数组拷贝

类型	语法指令
描述	数组拷贝，从数组 Src 拷贝到数组 Des。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	<pre> DMCPY arraydes(startpos),arraysrc(startpos)[,size] arrayname: 数组名称 startpos: 数组起始索引 size: 拷贝的个数，超过最大值会自动缩减 </pre>
适用控制器	通用
例子	<pre> GLOBAL aa(6),bb(6) '定义数组 aa, bb FOR i=0 TO 4 '赋值 aa 0,1,2,3,4 </pre>

	<pre> aa(i)=i NEXT ?*aa '打印数组所有元素 ?*bb DMCPY aa(0), bb(0),6 '把数组 bb 的值赋值给数组 aa ?*aa '打印数组复制后所有元素 ?*bb </pre>
相关指令	DMINS , DMDEL

DMSET -- 数组赋值

类型	语法指令
描述	数组区域赋值。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	DMSET arrayname(pos, size, data) pos: 起始索引 size: 长度 data: 设置的数值
适用控制器	通用
例子	<pre> DMSET TABLE(0,10,2) '数组区域赋值 FOR i=0 TO 9 PRINT "TABLE",i, TABLE(i) '打印数组 NEXT DMSET TABLE(0,10,3) '数组区域赋值 FOR i=0 TO 9 PRINT "TABLE",i, TABLE(i) '打印数组 NEXT </pre>
相关指令	DMINS , DMDEL

DMCMP -- 数组比较

类型	语法指令
描述	数组比较，数组内的元素的值逐个比较后返回结果。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	value = DMCMP(arr1, arr2, size) arr1: 比较数组 arr2: 比较数组 size: 比较元素个数，不可超出比较数组的长度 比较得出返回值： arr1 > arr2, value=1; arr1=arr2, value=0, 比较范围内，元素值均相同；

	arr1 < arr2, value = -1;
适用控制器	通用
例子	<pre> DIM value,i DIM arr3(5), arr5(6) FOR i = 0 TO 4 arr3(i) = i*10 NEXT FOR i = 0 TO 5 arr5(i) = i*100+1 NEXT value = DMCMP(arr3,arr5,5) ?value IF value = -1 THEN ?"小于" ELSEIF value = 1 Then ?"大于" ELSE ?"等于" END IF </pre>
相关指令	DMINS , DMDEL

DMSEARCH -- 数组搜索

类型	语法指令
描述	根据元素的值，搜索该元素在数组中的位置，返回找到的第一个数组索引，搜索不到返回-1。 谨慎对超长数组进行操作，特别是数组 TABLE。
语法	<pre>Pos = DMSEARCH(array, startpos, offset, maxtimes, value)</pre> array 数组名 startpos: 搜索起始位置 offset: 每次搜索跳过的间隔 maxtimes: 最多判断的次数 value: 搜索的数值 返回: Pos: 数组的索引, -1 没有找到
适用控制器	通用
例子	<pre> DIM return, value DIM arr1(10) FOR i = 0 TO 9 arr1(i) = i </pre>

	<pre> NEXT value = DMSEARCH(arr1,0,1,10,3) return = DMSEARCH(arr1,0,1,10,20) IF value = 3 AND return = -1 THEN ?"查找成功" ELSE ?"查找失败" END IF </pre>
相关指令	DMINS , DMDEL

SIZEOFARRAY -- 获取数组空间

类型	语法指令
描述	获取占用的数组空间。
语法	VAR=SIZEOFARRAY(数组名) 返回数组个数，不支持变量 VAR=SIZEOFARRAY(结构名) 返回结构占用空间 VAR=SIZEOFARRAY(结构变量名) 返回结构变量/数组占用空间 5 系列控制器 20180327 以上固件支持。 4 系列控制器 fast 版本 20190107 以上固件支持。
适用控制器	通用
例子	<pre> 例一 返回数组个数 GLOBAL aa(12),bb '定义数组 aa, bb FOR i=0 TO 4 '赋值 aa 0,1,2,3,4 aa(i)=i NEXT ?*aa '打印数组所有元素 ?SIZEOFARRAY(aa) '打印结果: 12 例二 返回结构体变量/数组个数 '声明结构体 AA GLOBAL Structure ClassAA DIM AA_val1 '成员变量 DIM AA_array(10) '成员数组 END Structure '创建结构体变量 GLOBAL Class1 AS ClassAA Class1.AA_val1=123 ?Class1.AA_val1 '打印结果: 123 class1.AA_array="abc" </pre>

	<pre>?class1.AA_array '打印结果: abc ?sizeofarray(class1) '打印结果: 11 ?sizeofarray(classAA) '打印结果: 11 ?sizeofarray(class1.AA_array) '打印结果: 10 ?sizeofarray(Class1.AA_val1) '打印结果: 1</pre>
相关指令	DMINS , DMDEL

8.4 自定义子函数指令

SUB -- 自定义子函数 SUB

类型	语法指令
描述	用户自定义 SUB 过程，可以前面增加 GLOBAL 描述以定义全局使用的 SUB 过程。
语法	<pre>SUB label([para1] [,para2]...)</pre> ... <pre>END SUB</pre> 参数： - label: 过程名称，不能与现有的关键词冲突 - para1: 过程调用时传入的参数，自动作为 LOCAL 局部变量 - para2: 过程调用时传入的参数，自动作为 LOCAL 局部变量 5 系列以上控制器/带 Linux 控制器支持，22 年 2 月以后固件新增： <pre>SUB subname(BYREF paraname[(dimsize)] [AS structname])</pre> - subname: sub 名称 - dimsize: 数组长度，必须常数定义 - structname: 结构类型的名称，支持 BYREF 方式传递 ZVOBJ BYREF 表示引用，此时调用方使用对应类型的变量或数组等填入。 缺省 BYVAL 表示拷贝方式传递，暂时 BYVAL 不支持数组、结构等。 BYREF 的定义的数组，不能使用 {} 进行多个元素赋值，不支持原来的数组多个元素赋值方式 BYREF 传递的数据，不能再使用 ZINDEX 索引功能，原则上都不推荐都 LOCAL 数据取 ZINDEX
适用控制器	通用
例子	例一 <pre>SUB sub1() '定义过程 SUB1，只能在当前文件中使用 ?1 ...</pre>

	<pre> END SUB GLOBAL SUB g_sub2() '定义全局过程 g_sub2，可以在任意文件中使用 ?2 ... END SUB GLOBAL SUB g_sub3(para1,para2) '定义全局过程 g_sub3，传递两个参数 ?Para1,para2 ... RETURN para1+para2 '函数返回参数相加 END SUB 例二 5 系列以上控制器使用 STRUCTURE POS DIM a DIM b(11) END STRUCTURE DIM var1 DIM arr2(11) DIM arr3(300) DIM str3(2) as pos SUB2(var1, arr2, str3) '方式 1 SUB2(var1, arr2(100, 200), str3) '方式 2: 通过 arr2(100, 200) 取数组中间的部分 SUB SUB2(byref var1, byref arr2(100), byref arr3(2) as pos) ?SUB_IFPARA(0) ?var1 ?arr2(1) ?arr3(1).a END SUB SUB SUB3(byref var1, byref obj1 as ZVOBJ) '支持 BYREF 方式传递 ZVOBJ ?SUB_IFPARA(0) ?var1 ?arr2(1) ?arr3(1).a END SUB </pre>
相关指令	SUB PARA , SUB_IFPARA

SUB_PARA -- SUB 传递参数

类型	语法指令
描述	选择 SUB 的传入参数。
语法	SUB_PARA(address) address: 第几个传递参数, 从 0 开始
适用控制器	通用
例子	<pre> SUB AAA(NUM1,NUM2,NUM3) ?SUB_PARA(0) '调用 AAA 时, 打印传递的第一个 num1 值 ?SUB_PARA(1) '打印传递的第二个 num2 值 ?SUB_PARA(2) '打印传递的第三个 num3 值 END SUB </pre>
相关指令	SUB , SUB_IFPARA

SUB_IFPARA -- SUB 传参判断

类型	语法指令
描述	判断 SUB 是否传入参数。
语法	SUB_IFPARA(address) -1 传入, 0 未传入 address: 第几个传递参数, 从 0 开始
适用控制器	通用
例子	<pre> AAA(0,100) '传入 num1,num2 AAA(,100) '只传入 num2 END SUB AAA(NUM1,NUM2) IF SUB_IFPARA(0) THEN '调用 AAA 时, 判断 num1 是否传入 ?1 '传入打印 1 ELSE ?0 ENDIF END SUB </pre>
相关指令	SUB , SUB_PARA

GOSUB/CALL -- SUB 调用

类型	程序结构
描述	SUB 过程调用, 只能调用本文件的 SUB 过程或全局 SUB 过程。 直接调用 SUB 过程时, 可以省掉 GOSUB 语句。 SUB 过程没有参数传递时, 可以省掉括号()。 GOSUB 后当前的内容会压栈, 不能在调用的 SUB 程序中访问当前的局部变量,

	RETURN 返回时出栈。
语法	GOSUB/CALL label label: SUB 过程名
适用控制器	通用
例子	<pre> '主程序 main: GOSUB sub1() sub2(1,2) '传入 1 给 para1, 2 给 para2 CALL sub3 END '定义的 SUB SUB sub1() a=100 PRINT "sub1" RETURN SUB sub2(para1,para2) a=200 PRINT "sub2",para1,para2 RETURN GLOBAL SUB sub3() '可以在另外一个程序文件中 a=300 PRINT "sub3" RETURN </pre>

GSUB -- 自定义子函数-G 代码格式

类型	语法指令
描述	用户自定义 GSUB 过程。 可以前面增加 GLOBAL 描述以定义全局使用的 GSUB 过程，GSUB 过程调用的时候采用 G 代码语法，不要加括号。
语法	<pre> GSUB label([char1] [,char2]...) ... END SUB </pre> 参数： - Label: 过程名称，不能与现有的关键词冲突 - char1: 过程调用时传入的字母参数，自动作为 LOCAL 局部变量 - char2: 过程调用时传入的字母参数，自动作为 LOCAL 局部变量 字母参数只能为单字符
适用控制器	通用

例子	G01 X100 Y100 Z100 U100 '调用 G01 END '主程序结束 GLOBAL GSUB G01(X, Y, Z, U) '定义 GSUB 过程 G01 ... END SUB
相关指令	GSUB_PARA , GSUB_IFPARA

GSUB_PARA -- GSUB 传递参数

类型	语法指令
描述	选择 GSUB 的传入参数。
语法	GSUB_PARA(char) char: GSUB 定义时传入的字母参数
适用控制器	通用
例子	GSUB AAA(X,Y,Z) ?GSUB_PARA(X) '调用 AAA 时, 打印传递的 X 值 ?GSUB_PARA(Y) '打印传递的 Y 值 ?GSUB_PARA(Z) '打印传递的 Z 值 END SUB
相关指令	GSUB , GSUB_IFPARA

GSUB_IFPARA -- 判断 GSUB 是否传入参数

类型	语法指令
描述	判断 SUB 是否传入参数。
语法	GSUB_IFPARA(char) char: GSUB 定义时传入的字母参数 值: -1-传入, 0-未传入
适用控制器	通用
例子	AAA X0 Y100 '传入 X,Y AAA X0 '只传入 X END GSUB AAA(X,Y) IF GSUB_IFPARA(Y) THEN '调用 AAA 时, 判断 Y 是否传入 ?1 '传入打印 1 ELSE ?0 ENDIF END SUB
相关指令	GSUB -- 自定义子函数-G 代码格式 , GSUB_PARA

END SUB -- 自定义函数结束

类型	程序结构
描述	用户自定义 SUB 过程结束，参见 SUB。
适用控制器	通用

RETURN -- 函数返回值

类型	程序结构
描述	用户 SUB 过程返回或返回值。 返回值缺省 0，在外面可以通过 RETURN 来读取上个 SUB 调用的返回值。 不同任务的返回值不同。
语法	RETURN
适用控制器	通用
例子	<pre>CALL sub1 ?RETURN '结果为 111 END '主程序结束 SUB sub1() RETURN 111 '返回 111 END SUB</pre>

XSUB --自定义 XSUB 子函数

类型	程序结构
描述	用户自定义 XSUB 过程，用于传递参数到子函数。 与 RSUB 的区别是调用需要加括号。
语法	语法同 SUB 调用时，可以使用 paraname=value 的语法
适用控制器	5 系及以上
例子	例一 <pre>subX(ARR2=a2, VAR1 =a1, ARR3=pos4) XSUB subX(byref var1, byref arr2(100), byref arr3(2) as pos) ?SUB_IFPARA(0) ?var1 ?arr2(1) ?arr3(1).a END SUB</pre> 例二

	<pre> TR VAR1 = 5,VAR2 = 1 dim a a = 10 TX(var1 = a ,arr = "Zmotion") RSUB TR(VAR1,VAR2) ?SUB_IFPARA(0) ?var1 ?var2 END SUB XSUB TX(byref var1,byref arr(100)) ?var1 ?arr end sub </pre>
相关指令	SUB , RSUB

RSUB -- 自定义 RSUB 子函数

类型	程序结构
描述	用户自定义 RSUB 过程，用于传递参数到子函数 与 XSUB 的区别是调用时无需加括号。
语法	语法同 SUB 调用时，可以使用 paraname=value 的语法
适用控制器	5 系及以上
例子	<pre> subR ARR2=a2, VAR1 =a1, ARR3=pos4 RSUB subR(byref var1, byref arr2(100), byref arr3(2) as pos) ?SUB_IFPARA(0) ?var1 ?arr2(1) ?arr3(1).a END SUB </pre>
相关指令	SUB , XSUB

8.5 结构体定义指令

STRUCTURE -- 结构体定义

类型	语法指令
描述	结构体定义。
语法	Structure 结构名称

	<pre> Dim 成员 1 名称 [As 数据类型 1] ... Dim 成员 n 名称[(数组长度)] [As 数据类型 1] End Structure 数据类型只支持结构体，每个元素都同数组元素，占用一个数组元素空间。 结构体不能递归。 结构变量定义： DIM 变量名 AS 结构名 DIM 结构数组名[(数组长度)] AS 结构名 GLOBAL 变量名 AS 结构名 GLOBAL 结构数组名[(数组长度)] AS 结构名 预留功能： LOCAL 变量名 AS 结构名 支持使用 FLASH_WRITE, FLASH_READ 指令读写结构体定义的变量和数组。 FLASH_WRITE id, 结构变量 FLASH_WRITE id, 结构数组 FLASH_WRITE id, 结构数组(index) FLASH_WRITE id, 结构数组(index).item FLASH_WRITE id, 结构数组(index).item 数组(index) FLASH_READ 同上 支持使用数组操作指令操作结构体数组。 DMINS 结构数组(index) [,numes] DMINS 结构数组(index).item 数组(index) [,numes] DMDEL 同上 DMCPY 结构数组 1(index1), 结构数组 2(index2) [,size] DMSET 只支持对最后一层的数组进行操作，不能对结构数组赋值。 DMSET 结构变量.item 数组(index, size, data) DMADD 同上 </pre>
适用控制器	5 系列控制器 20180327 以上固件支持。 4 系列控制器 fast 版本 20190107 以上固件支持。
例子	<pre> '声明结构体 AA GLOBAL Structure ClassAA DIM AA_val1 '成员变量 DIM AA_array(10) '成员数组 END Structure '声明结构体 BB GLOBAL Structure ClassBB DIM BB_val1 AS ClassAA '成员变量为结构体 </pre>

	<pre> END Structure '创建结构体变量 GLOBAL Class1 AS ClassAA GLOBAL Class2 AS ClassBB Class1.AA_val1=123 ?Class1.AA_val1 class1.AA_array="abc" ?class1.AA_array Class2.BB_val1.AA_val1=567 ?Class2.BB_val1.AA_val1 Class2.BB_val1.AA_array="zxc" ?Class2.BB_val1.AA_array AA_val1=8 FLASH_WRITE 0,AA_val1 AA_val1=123 FLASH_READ 0,AA_val1 ?AA_val1 END </pre>
相关指令	DIM , GLOBAL , UNION

UNION -- 共用体定义

类型	语法指令
描述	共用体定义。 5 系列控制器 20180327 以上固件支持。 4 系列控制器 fast 版本 20190107 以上固件支持。
语法	<pre> UNION 结构名称 Dim 成员 1 名称 [As 数据类型 1] Dim 成员 n 名称[(数组长度)] [As 数据类型 1] END UNION 结构变量定义: DIM 变量名 AS 结构名 DIM 结构数组名[(数组长度)] AS 结构名 </pre>

	GLOBAL 变量名 AS 结构名 GLOBAL 结构数组名[(数组长度)] AS 结构名 预留功能: LOCAL 变量名 AS 结构名 支持使用 FLASH_WRITE, FLASH_READ 指令读写结构体定义的变量和数组。 FLASH_WRITE id, 结构变量 FLASH_WRITE id, 结构数组 FLASH_WRITE id, 结构数组(index) FLASH_WRITE id, 结构数组(index).item FLASH_WRITE id, 结构数组(index).item 数组(index) FLASH_READ 同上 支持使用数组操作指令操作结构体数组。 DMINS 结构数组(index) [,numes] DMINS 结构数组(index).item 数组(index) [,numes] DMDEL 同上 DMCPY 结构数组 1(index1), 结构数组 2(index2) [,size] DMSET 只支持对最后一层的数组进行操作, 不能对结构数组赋值。 DMSET 结构变量.item 数组(index, size, data) DMADD 同上
适用控制器	5 系列控制器 20180327 以上固件支持。 4 系列控制器 fast 版本 20190107 以上固件支持。
例子	<pre>'声明共用体 AA GLOBAL UNION ClassAA DIM AA_val1 '成员变量 DIM BB_val1 '成员变量 DIM AA_array(4) '成员数组 END UNION '创建共用体变量 GLOBAL Class1 AS ClassAA Class1.AA_val1=123 ?Class1.AA_val1 '结果: 123 Class1.BB_val1=456 ?Class1.BB_val1 '结果: 456 ?Class1.AA_val1 '结果: 456 END</pre>
相关指令	DIM , GLOBAL , STRUCTURE

8.6 跳转指令

GOTO -- 强制跳转

类型	程序结构
描述	强制跳转，与 GOSUB 的区别是 GOTO 不会压栈。
语法	GOTO label
适用控制器	通用
例子	<pre> a=100 GOTO label1 '强制跳转至 label1 a=1000 END '主程序结束 label1: PRINT a '结果是 a=100 END 'label1 结束 </pre>

ON GOSUB -- 条件跳转

类型	程序结构
描述	当 expression 条件为真时，调用过程 label 。
语法	<pre> ON expression GOSUB label expression: 判断条件 label: 跳转 sub、label 名称 </pre>
适用控制器	通用
例子	<pre> a=100 ON a>10 GOSUB label1 'a>10 时调用 label 过程 a=1000 PRINT a END '主程序结束 label1: PRINT a RETURN 'SUB 过程要返回 </pre>

ON GOTO -- 条件跳转 2

类型	程序结构
描述	条件跳转，当 expression 条件为真时跳转，不会压栈。
语法	<pre> ON expression GOTO label expression: 判断条件 </pre>

	label: 跳转 sub、label 名称
适用控制器	通用
例子	<pre> a=100 on a>10 goto label1 a=1000 END '主程序结束 label1: PRINT a END 'goto 跳转无法 return 返回 </pre>

8.7 条件判断指令

IF -- 条件判断结构

类型	程序结构
描述	条件判断，同标准 BASIC 结构。
语法	<pre> IF <condition1> THEN commands ELSEIF <condition2> THEN commands ELSE commands ENDIF </pre> 参数： <pre> condition1: 条件 condition2: 条件 </pre>
适用控制器	通用
例子	<pre> 例一 DIM a '定义变量 a=12 '赋值 IF a>11 THEN '条件判断 TRACE "the val a is bigger then 11" ELSEIF a<11 THEN TRACE "the val a is less then 11" ENDIF 例二 IF IN(0) THEN OUT(0,ON) '当单行时可以用不用 endif </pre>
相关指令	THEN ， ENDIF

THEN -- 条件判断结构

类型	程序结构
描述	参见：IF
适用控制器	通用

ENDIF -- 条件判断结构

类型	程序结构
描述	参见：IF
适用控制器	通用

ELSEIF -- 条件判断结构

类型	程序结构
描述	参见：IF
适用控制器	通用

8.8 循环指令

FOR -- for 循环结构

类型	程序结构
描述	循环语句，采用标准 BASIC 语法。
语法	FOR variable=start TO end [STEP increment] commands NEXT variable 参数： variable: 变量名称 start: 起始循环值 end: 结束循环值 increment: 循环步进增量，可选 多任务时，请不要使用（非 local 时）相同的 variable 循环变量，否则会相互干扰。
适用控制器	通用
例子	例一： <pre> LOCAL a FOR a=1 TO 100 '1 循环至 100 PRINT a '打印 a </pre>

	NEXT 例二： DIM i FOR i = 0 TO 50 STEP 2 '1 循环至 50，间隔为 2 TABLE(i) = i ?TABLE(i) NEXT
相关指令	TO , STEP , NEXT

TO -- for 循环结构

类型	程序结构
描述	参见： FOR
适用控制器	通用

STEP -- for 循环结构

类型	程序结构
描述	参见： FOR
适用控制器	通用

NEXT -- for 循环结构

类型	程序结构
描述	参见： FOR
适用控制器	通用

WHILE -- while 循环结构

类型	程序结构
描述	条件满足时执行循环。
语法	WHILE condition ... WEND
适用控制器	通用
例子	a=0 WHILE IN(4)=OFF '直到输入 4 有效，退出循环 a=a+1 PRINT a DELAY(1000)

	WEND
--	------

WEND -- while 循环结构

类型	程序结构
描述	参见: WHILE
适用控制器	通用

EXIT -- 退出循环

类型	程序结构
描述	循环退出语句。多层嵌套需要一层一层退出循环
语法	EXIT FOR, EXIT WHILE
适用控制器	通用
例子	<pre> DIM a FOR a=1 TO 100 '1 循环至 100 PRINT a '打印 a IF a> 20 THEN ?"次数大于 20 次了, 准备退出 For 循环了" EXIT FOR '终止当前循环, 跳出循环体 ENDIF NEXT </pre>

REPEAT -- 条件循环

类型	程序结构
描述	循环语句。 循环执行 commands, condition 为真时退出循环。
语法	REPEAT commands UNTIL condition
适用控制器	通用
例子	<pre> a=0 REPEAT '循环执行下面语句 PRINT a a=a+1 DELAY(1000) UNTIL IN(4)=ON '直到输入 4 有效 </pre>

UNTIL -- 条件结构

类型	程序结构
描述	参见: REPEAT 、 WAIT

适用控制器	通用
-------	----

8.9 等待执行指令

DELAY -- 延时

类型	语法指令
描述	延时 delay time ，单位毫秒。 别名：wa
语法	DELAY(delay time) delay time: 毫秒数
适用控制器	通用
例子	DELAY(100) '延时 100ms

WAIT UNTIL -- 等待条件满足

类型	程序结构
描述	等待直到条件满足。
语法	WAIT UNTIL condition1 [and condition2 or condition3 ...] 可以通过逻辑运算操作多个条件。
适用控制器	通用
例子	例一 WAIT UNTIL DPOS(0) > 0 '等待轴 0 位置超过 0 例二 与 TICKS 一起使用 TICKS=2000 'ticks 设为 2000 WAIT UNTIL TICKS<0 '等待 2s ?"执行下一步" 例三 与逻辑条件一起使用 WAIT UNTIL IDLE(0)=-1 AND IDLE(1)=-1 AND IDLE(2)=-1 '等待轴 0、1、2 都停止

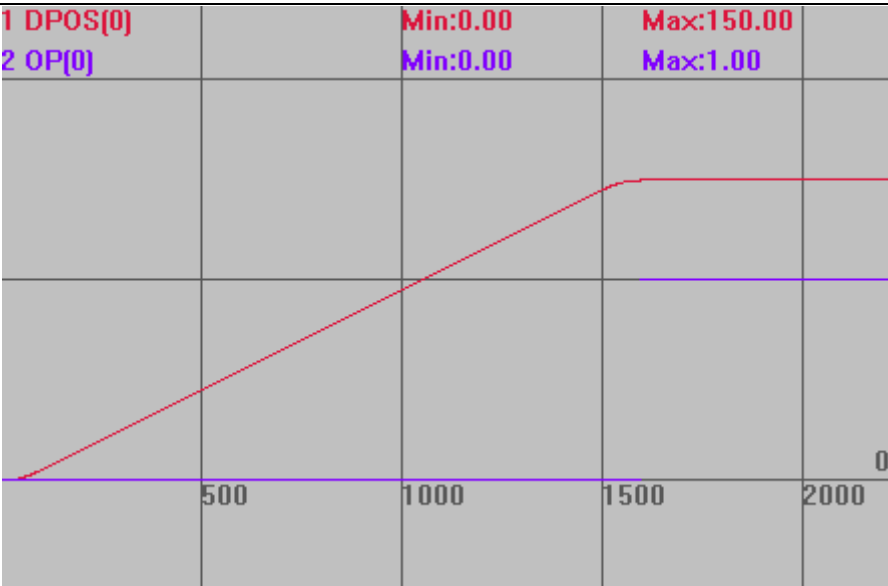
WAIT IDLE -- 等待轴停止

类型	语法指令
描述	等待 BASE 轴运动完成， BASE 轴运动未完成时，不执行后面的程序。 等效于 WAIT UNTIL IDLE。IDLE 作为轴参数，支持轴参数的语法。 注：控制器运动发送完成不代表伺服执行完成
语法	WAIT IDLE

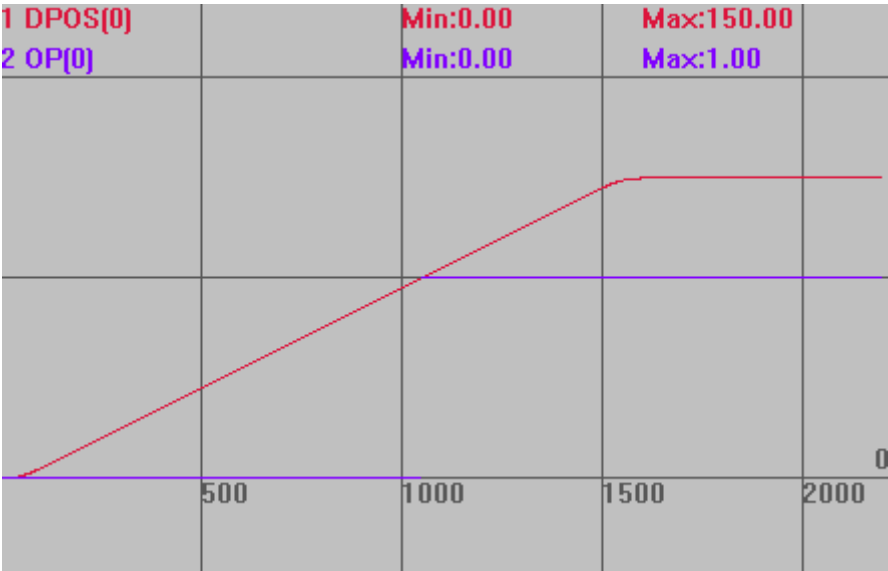
适用控制器	通用
例子	例一 <pre>BASE(0,1) MOVE(100,100) WAIT IDLE '等待当前插补运动结束</pre> 例二 <pre>BASE(0,1) MOVE(100,100) BASE(2,3) MOVE(200,200) WAIT UNTIL IDLE(0) AND IDLE(1) AND IDLE(2) AND IDLE(3) '等待轴 0, 1, 2, 3 停止 ?"运动完成"</pre>

WAIT LOADED -- 等待轴缓冲空

类型	语法指令
描述	等待 BASE 轴运动缓冲空，此命令会阻塞不执行后面的程序。 缓冲区最后一个运动可以正确执行，同时程序向下扫描。 等效于 WAIT UNTIL LOADED，LOADED 作为轴参数，支持轴参数的语法。
语法	WAIT LOADED
适用控制器	通用
例子	与 WAIT IDLE 的区别 <pre>BASE(0) ATYPE=1 UNITS=100 DPOS=0 SPEED=100 ACCEL=1000 MERGE=1 TRIGGER MOVE(100) '当前运动 MOVE(50) '缓冲运动，此时缓冲区只有这一条运动 '当本条运动执行时，缓冲区就已经清空 WAIT IDLE '使用 wait idle 时，要等到所有运动完成才能往下执行 OP(0,ON) '打开 op0</pre> 运动轨迹 <pre>DPOS(0)垂直刻度 100 OP(0)垂直刻度 1</pre>



WAIT LOADED '使用 wait loaded 时，运动缓冲空即可往下执行
OP(0,ON)



8.10 ZINDEX 指针指令

ZINDEX_LABEL -- 建立指针索引

类型	语法指令
描述	建立索引指针，便于后续调用指针内容。
语法	Pointer = zindex_label(subname) subname: 数组或 SUB 名称
适用控制器	通用
例子	DIM arr1(100) '定义数组

	arr1(0,1) '对数组 0 地址赋值 1 Pointer = ZINDEX_LABEL(arr1) '建立索引指针 PRINT ZINDEX_ARRAY(Pointer) (0) '访问数组，打印数组第一位数据，结果 1
相关指令	ZINDEX_CALL , ZINDEX_ARRAY , ZINDEX_VAR

ZINDEX_CALL -- 访问 SUB 函数

类型	语法指令
描述	通过索引指针来调用 SUB 函数。
语法	ZINDEX_CALL(zidnex) (subpara, ...) zidnex: 通过 ZINDEX_LABEL 生成的索引指针 subpara: sub 的参数调用
适用控制器	通用
例子	Pointer = ZINDEX_LABEL(sub1) '建立索引指针 ZINDEX_CALL(Pointer) (2) '调用函数 SUB sub1(a) PRINT a END SUB
相关指令	ZINDEX_LABEL

ZINDEX_ARRAY -- 访问数组

类型	语法指令
描述	通过索引指针来访问数组。
语法	var = ZINDEX_ARRAY (pointer)(index) pointer: 通过 ZINDEX_LABEL 生成的指针索引 index: 数组索引
适用控制器	通用
例子	DIM arr1(100) '定义数组 arr1(0,1) '对数组 0 地址赋值 1 Pointer = ZINDEX_LABEL(arr1) '建立索引指针 PRINT ZINDEX_ARRAY(Pointer) (0) '访问数组，打印数组第一位数据，结果 1
相关指令	ZINDEX_LABEL

ZINDEX_VAR -- 访问变量

类型	语法指令
描述	通过索引指针来访问变量。
语法	ZINDEX_VAR(zidnex) zidnex: 通过 ZINDEX_LABEL 生成的索引指针

	<pre> zindex= ZINDEX_LABEL(varname) ZINDEX_VAR(zindex)=value VAR2 = ZINDEX_VAR(zindex) </pre>
适用控制器	通用
例子	<pre> global gTestVar global VarAdd1 VarAdd1=ZINDEX_LABEL(gTestVar) ZINDEX_VAR(VarAdd1)=10 ?ZINDEX_VAR(VarAdd1) </pre>
相关指令	ZINDEX_LABEL

ZINDEX_STRUCT -- 访问结构体

类型	语法指令
描述	获取结构变量的指针后，通过指针来访问结构变量或数组。
语法	<pre> GLOBAL structarrname(num) As structname zindex = ZINDEX_LABEL(structarrname) ZINDEX_STRUCT(structname,zindex)(arrindex).item = var var = ZINDEX_STRUCT(structname,zindex)(arrindex).item </pre> structarrname: 生成的结构体数组变量 num: 生成的结构体数组变量元素个数 zindex: 通过 ZINDEX_LABEL 生成的索引指针 arrindex: 结构体数组下标 structname: 结构体名称 item: 结构体成员
适用控制器	结构体指针功能只有特殊固件版本支持： 5 系列控制器 20180327 以上固件支持。 4 系列控制器 fast 版本 20190107 以上固件支持。
例子	<pre> 例一 GLOBAL Structure ClassAA '结构体声明 DIM AA_val1 '成员变量 DIM AA_array(10) '成员数组 END Structure GLOBAL Class1 AS ClassAA '结构体全局变量定义 GLOBAL gStructureAdd Class1.AA_array(0,1,2,3) '结构体数组赋值 ?Class1.AA_array(0) '结果： 1 gStructureAdd = ZINDEX_LABEL(Class1) '建立结构体索引指针 </pre>

	<pre>?ZINDEX_STRUCT(ClassAA,gStructureAdd).AA_array(0) '结果: 1 ZINDEX_STRUCT(ClassAA,gStructureAdd).AA_array(0)= 10 ?ZINDEX_STRUCT(ClassAA,gStructureAdd).AA_array(0) '结果: 10 END</pre> 例二 <pre>GLOBAL STRUCTURE stru_node '定义结构体 DIM m_data DIM m_Left DIM m_right DIM m_Temp END STRUCTURE DIM root GLOBAL g_node(100) AS stru_node root = ZINDEX_LABEL(g_node) '创建结构体数组指针 ZINDEX_STRUCT(stru_node,root)(99).m_data = 11 ?ZINDEX_STRUCT(stru_node,root)(99).m_data '结果: 11 END</pre>
相关指令	ZINDEX_LABEL

ZINDEX_MARK -- 指针标号设置

类型	语法指令
描述	给 label 设置标号，从而可以把 label 的数组索引记录下来。
语法	<pre>ZINDEX_MARK(labelname) = mark varmark = ZINDEX_MARK(labelname)</pre> labelname: 如果不同文件都有定义，注意不同文件里面调用的是不同的结果 mark: 标号
适用控制器	支持 ZV 功能或者 5 系列以上的控制器
例子	<pre>dim var1 dim arr1(10),MarkArr(10) dim ArrIndex,VarIndex arr1(0,1,2,3,4,5,6,7,8,9,10) VarIndex = zindex_label(var1) '获取变量指针、数组指针 ArrIndex = zindex_label(arr1) zindex_mark("VarIndex") = 0 '设置变量指针、数组指针标号 zindex_mark("ArrIndex") = 1</pre>

	<pre>markarr(zindex_mark("VarIndex")) = VarIndex '通过获取的标号将指针存储至指针数 组指定数组下标处 markarr(zindex_mark("ArrIndex")) = ArrIndex zindex_var(MarkArr(zindex_mark("VarIndex")))) = 777 '通过标号访问指针数组对应位 置变量、数组 ?zindex_var(MarkArr(0)) ?var1 ?zindex_array(MarkArr(zindex_mark("ArrIndex")))(9) ?zindex_array(MarkArr(1))(9) end</pre>
相关指令	ZINDEX_LABEL

第九章 任务相关指令

RTBasic 支持实时多任务运行，一个文件上也可以同时运行多个任务。通过 RUN 可以从文件第一行开始运行任务，通过 RUNTASK 可以随意指定 SUB 过程开始运行。

9.1 任务启停指令

RUN -- 启动文件任务

类型	任务指令
描述	新建一个任务来执行控制器上的一个文件。 重复启动同一任务会报错。 多次使用 RUN 指令不带任务号参数时，会让同一个文件对应多个任务，建议使用 RUNTASK 指令开启任务。 多任务操作指令有： END：当前任务正常结束 STOP：停止指定文件 STOPTASK：停止指定任务 HALT：停止所有任务 RUN：启动文件执行 RUNTASK：启动任务在一个 SUB 上执行
语法	RUN "filename"[, tasknum] filename：程序文件名，bas 文件可不加扩展名 tasknum：任务号，缺省查找第一个有效的
适用控制器	通用
例子	RUN "aaa", 1 '启动任务 1 运行 aaa.bas 文件
相关指令	RUNTASK

RUNTASK -- 启动 SUB 任务

类型	任务指令
描述	把一个 SUB 过程或是标号作为一个新的任务执行。 重复启动同一任务会报错。
语法	RUNTASK tasknum, label tasknum：任务号 label：自定义 SUB 过程（可带参数）或标号
适用控制器	通用
例子	RUNTASK 1, taska '启动任务 1 来跟踪打印位置 MOVE(1000,100) MOVE(1000,100)

	<pre> END taska: '循环打印位置 WHILE 1 PRINT *mpos DELAY(1000) WEND END </pre>
相关指令	RUN

END -- 结束

类型	任务指令
描述	当前任务结束。 当一个文件中有主程序和 SUB 过程时，一定要在主程序结束后写 END，如果不写 END，程序执行完主程序后，就会依次再执行 SUB 子程序，直到子程序的 END SUB 才停止。
适用控制器	通用
相关指令	RUN , RUNTASK

STOP -- 停止文件任务

类型	任务指令
描述	程序强制停止，操作文件。 再启动任务前都要停止任务。 使用 STOP 指令不带任务号时，一次只会停掉一个任务，而不是此文件对应的所有任务，当一个文件内有多个任务时，建议用 STOPTASK 指令。
语法	<pre>STOP "program name", [tasknum]</pre> program name: 程序文件名，bas 文件可不用带扩展名 tasknum: 任务号，当程序文件有启动多个任务时，缺省任务号最小的任务
适用控制器	通用
例子	<pre> RUN "aaa", 1 '执行 aaa.bas STOP "aaa", 1 '停止任务 1 </pre>
相关指令	STOPTASK , HALT

STOPTASK -- 停止 SUB 任务

类型	任务指令
描述	任务强制停止，操作 SUB 和标记。 再启动任务前都要停止任务。
语法	STOPTASK [tasknum]

	tasknum: 任务号, 缺省当前任务
适用控制器	通用
例子	STOPTASK 2 '停止任务 2
相关指令	STOP , HALT

HALT -- 停止全部任务

类型	任务指令
描述	停止全部任务。 此指令仅限 PC 软件调用, BASIC 程序中若使用此指令, 会导致整个程序停止, 控制器无法运行。
语法	HALT
适用控制器	通用
例子	HALT '停止所有任务
相关指令	STOP , STOPTASK

PAUSE -- 暂停全部任务

类型	任务指令
描述	暂停全部任务。 使用断点生效后也会进入暂停状态。 此指令仅限 PC 软件调用, BASIC 程序中若使用此指令, 会导致整个程序停止, 控制器无法运行。 暂停任务再恢复后, 任务继续往下执行。
语法	PAUSE
适用控制器	通用
例子	PAUSE '暂停所有任务
相关指令	PAUSETASK

PAUSETASK -- 暂停指定任务

类型	任务指令
描述	暂停某一个任务。 暂停任务再恢复后, 任务继续往下执行。
语法	PAUSETASK tasknum tasknum: 任务号, 缺省当前任务
适用控制器	通用
例子	PAUSETASK 1 '暂停任务 1
相关指令	RESUMETASK

RESUMETASK -- 恢复指定任务

类型	任务指令
描述	恢复某一个任务。 暂停任务再恢复后，任务继续往下执行。
语法	RESUMETASK tasknum tasknum: 任务号, 缺省当前任务
适用控制器	通用
例子	PAUSETASK 1 '暂停任务 1 RESUMETASK 1 '继续运行任务 1
相关指令	PAUSETASK

9.2 三次文件任务指令

FILE3_RUN -- 执行 FILE3 任务

类型	任务指令
描述	3 次程序文件的启动命令。 3 次程序文件是一种超大文件，可以动态加载，使用 basic 语法。不支持条件判断、跳转等操作。可以通过指令下载，也可以通过工具软件 zfile3view 进行浏览、上传和下载操作。
语法	FILE3_RUN "filename", tasknum filename: 3 次程序的文件名，必须事先下载到控制器里面 tasknum: 任务号, 缺省查找第一个有效的
适用控制器	必须带大容量存储的控制器和 2015 以上的固件版本支持。
例子	FILE3_RUN "aaa.z3p", 1 '在任务 1 运行 3 次文件 aaa.z3p
相关指令	FILE3_ONRUN

FILE3_ONRUN --FILE3 回调函数

类型	回调函数
描述	3 次文件启动时会自动调用。
语法	GLOBAL FILE3_ONRUN: 标号 GLOBAL SUB FILE3_ONRUN() 自定义 SUB 过程（不能带参数） 回调函数属于 3 次文件任务。
适用控制器	通用
例子	FILE3_RUN "aaa.z3p", 1 '在任务 1 运行 3 次文件 aaa.z3p END

	<pre> GLOBAL SUB FILE3_ONRUN() '3 次任务启动时自动调用 IF 1= PROCNUMBER THEN BASE(0,1,2) '选择 3 次文件的运行轴列表 SPEED=1000 ACCEL=10000 ELSE BASE(4,5,6) ENDIF END SUB </pre>
相关指令	FILE3_RUN

FILE3_GOTO -- FILE3 强制跳转

类型	任务函数
描述	对三次任务有效，强制跳转到指定的行号开始运行。
语法	<pre>FILE3_GOTO(linenum)</pre> linenum: 要跳转的行号，从 1 开始编号
适用控制器	通用
相关指令	FILE3_LINE , FILE3_RUN

FILE3_LINE -- FILE3 行号

类型	任务函数
描述	返回当前 3 次文件运行的行号，无论是否三次文件因为 SUB 调用进入 BASIC 文件，总是返回 3 次文件的行号。
语法	<pre>VALUE=FILE3_LINE([taskid])</pre> taskid: 三次文件的任务号，不填返回函数调用当前任务的
适用控制器	通用
相关指令	FILE3_RUN , FILE3_GOTO

FILE3_LINES -- 文件总行数

类型	FILE3 文件编辑
描述	获取文件总行数 只有加载后的文件才能正确获取行数，否则返回-1
语法	<pre>var = FILE3_LINES()</pre> 返回总行数
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	?FILE3_LINES ' 打印当前加载文件总行数

FILE3_CHANNEL -- 当前 3 次文件号/通道

类型	FILE3 文件编辑
描述	获取/设置当前任务目标 3 次文件号/通道号
语法	<pre>FILE3_CHANNEL[(taskid)] = 3filenum</pre> 设置任务指定目标 3 次文件号，缺省为当前任务 <pre>3filenum = FILE3_CHANNEL[(taskid)]</pre> 获取任务指定目标 3 次文件号，缺省为当前任务
适用控制器	必须带大容量存储的控制器和 2015 以上的固件版本支持。 带 linux 系统或者 4 系以上的控制器
例子	<pre>FILE3_CHANNEL = 1</pre> <pre>FILE3_CHANNEL(6) = 1</pre> <pre>?FILE3_CHANNEL</pre> <pre>?FILE3_CHANNEL(6)</pre>

FILE3_BUFSIZE -- 设置加载缓冲区大小

类型	FILE3 文件编辑
描述	设置、获取 3 次文件加载缓冲区大小
语法	<pre>FILE3_BUFSIZE(num) = size</pre> num: 3 次文件编号，从 0 开始 size: 加载缓冲区大小（字节） <pre>var = FILE3_BUFSIZE(num)</pre> 返回缓冲区大小
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	<pre>FILE3_BUFSIZE(0) = 512000</pre> ' 设置第一个加载区大小为 512kb <pre>FILE3_BUFSIZE(1) = 512000</pre> ' 设置第二个加载区大小为 512kb

FILE3_SIZE -- 文件大小

类型	FILE3 文件编辑
描述	获取文件长度 未指定文件时，且当前未加载文件时，返回-1
语法	<pre>var = FILE3_SIZE([filename])</pre> filename: 指定获取文件，缺省为当前加载文件 返回总长度
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_DATETIME -- 文件最后修改日期

类型	FILE3 文件编辑
描述	获取文件最后修改日期 未指定文件时，且当前未加载文件时，返回空
语法	<pre>var = FILE3_DATETIME(style[,filename])</pre> style: 时间格式，” %Y/%m/%d %H:%M:%S” => 2021/08/14/ 11:03 filename: 指定获取文件，缺省为当前加载文件 返回时间字符串 时间格式： %a 星期几的简写 %A 星期几的全称 %b 月份的简写 %B 月份的全称 %c 标准的日期的时间串 %C 年份的前两位数字 %d 十进制表示的每月的第几天 %D 月/天/年 %e 在两字符域中，十进制表示的每月的第几天 %F 年-月-日 %g 年份的后两位数字，使用基于周的年 %G 年份，使用基于周的年 %h 简写的月份名 %H 24 小时制的小时 %I 12 小时制的小时 %j 十进制表示的每年的第几天 %m 十进制表示的月份 %M 十时制表示的分钟数 %n 新行符 %p 本地的 AM 或 PM 的等价显示 %r 12 小时的时间 %R 显示小时和分钟：hh:mm %S 十进制的秒数 %t 水平制表符 %T 显示时分秒：hh:mm:ss %u 每周的第几天，星期一为第一天（值从 0 到 6，星期一对应 0） %U 每年的第几周，把星期日做为第一天（值从 0 到 53） %V 每年的第几周，使用基于周的年 %w 十进制表示的星期几（值从 0 到 6，星期天为 0） %W 每年的第几周，把星期一做为第一天（值从 0 到 53） %x 标准的日期串 %X 标准的时间串

	%y 不带世纪的十进制年份（值从 0 到 99） %Y 带世纪部分的十制年份 %z, %Z 时区名称，如果不能得到时区名称则返回原字符。 %% 百分号
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	?FILE3_DATETIME(“=>%Y/%m/%d %H:%M:%S”, “daobu.nc”) =>2021/08/14/ 11:03

FILE3_BUFLOAD -- 加载文件

类型	FILE3 文件编辑
描述	加载指定 3 次文件到内存中 在编辑、显示或其他操作 3 次文件前，必须先加载 3 次文件 通过 FILE3_BUFSIZE 设置缓冲区大小，不支持编辑超过 FILE3_BUFSIZE 大小的 3 次文件
语法	FILE3_BUFLOAD(mode, filename) mode: 加载模式 0-仅加载已存在的文件， 1-新建空白文件再加载 2-清空文件再加载（若文件不存在，等同模式 1） 3-加载已存在文件，文件不存在则新建空白文件 filename: 3 次文件名
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	FILE3_BUFLOAD(0, “O0001.nc”)

FILE3_BUFRUN -- 运行加载文件

类型	FILE3 文件编辑
描述	任务运行当前加载的 3 次文件
语法	FILE3_BUFRUN(taskid [,shartline [,ifpause [,file3num]]]) taskid: 指定 3 次文件运行的任务号 startline: 从指定行开始运行，缺省从第 1 行开始 ifpause: 启动后是否立即暂停，缺省 0 不暂停 file3num: 指定启动 3 次文件号，缺省为当前任务指定的三次文件号
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	FILE3_BUFLOAD (0, “O0001.nc”) FILE3_BUFRUN(1) ’ 指定任务 1 运行

FILE3_BUF_CLOSE -- 关闭文件

类型	FILE3 文件编辑
----	------------

描述	卸载/关闭当前加载到内存中的 3 次文件 关闭文件不会自动保存，请提前保存文件
语法	FILE3_BUF_CLOSE ()
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	FILE3_BUF_LOAD(0, "O0001.nc") FILE3_BUF_CLOSE()

FILE3_BUF_SAVE -- 保存/另存文件

类型	FILE3 文件编辑
描述	保存 当前加载到内存的 3 次文件内容 到 flash 中 只保存加载部分，在加载后面的文件内容时，前面缓冲的也会自动保存
语法	FILE3_BUF_SAVE([filename]) filename: 文件名，缺省表示不另存，反之另存为
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	FILE3_BUF_SAVE() ' 保存当前加载文件 FILE3_BUF_SAVE("O0001.nc") ' 当前加载文件另存为 O0001.nc

FILE3_BUF_READ -- 读取行

类型	FILE3 文件编辑
描述	读取指定行内容（行号），返回行字符串 运行文件时也可使用该指令
语法	string = FILE3_BUF_READ(linenum) linenum: 行号，0-为当前运行行/编辑行 string: 返回字符串 FILE3_BUF_READ(linenum, tabfirst) linenum: 行号 tabfirst: 存储行字符串的起始 table 位置
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	dim string(256) string = FILE3_BUF_READ (1) trace string ' 打印第 1 行代码 FILE3_BUF_READ (1, 1000) trace tablestring(1000, 128) ' 打印第 1 行代码，最长 128 个字符

FILE3_BUF_LAYER -- 读取行层次

类型	FILE3 文件编辑
----	------------

描述	获取行层次 行首以空格/tab 符作为表示，4 空格=1 个 tab，行首无空格时表示层次 0，一个 tab 符表示层次 1，以此类推
语法	Var layer = FILE3_BUFLAYER(linenum) linenum: 行号 返回 layer: 层次，返回-1 表示出现不规范的层次标识
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFELENUM -- 读取行元素总个数

类型	代码编辑
描述	获取函数元素个数
语法	Var sum = FILE3_BUFELENUM(linenum) linenum: 行号 返回 Sum: 元素个数，负数表示该行被注释
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFELENAME -- 读取行元素名

类型	代码编辑
描述	获取函数元素名
语法	Var string = FILE3_BUFELENAME(linenum,elenum) linenum: 行号 Elenum: 第几个元素 返回 string: 元素名
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFELEPARA -- 读取行元素的参数

类型	代码编辑
描述	读取行元素的参数
语法	FILE3_BUFELEPARA(linenum,elenum,paranum,paratype,table) linenum: 行号 Elenum: 第几个元素 Paranum: 第几个参数 Paratype: 参数类型 0 数值

	1 名称 2 字符串 3 字符串数值 Table: 储存函数返回值 参数类型=3 时, 第 1 个=数值个数, 第 2 个=第 1 个数值, 类推
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFELEPARA2 -- 读取元素的参数

类型	代码编辑
描述	读取指定元素(字符串)的参数
语法	FILE3_BUFELEPARA2(string,paranum,paratype,table) string: 字符串元素, 如" xxxx(a,b,c)" Paranum: 第几个参数 Paratype: 参数类型 0 数值 1 名称 2 字符串 3 字符串数值 Table: 储存函数返回值 参数类型=3 时, 第 1 个=数值个数, 第 2 个=第 1 个数值, 类推
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFELEPALL -- 读取元素的所有参数

类型	代码编辑
描述	读取指定元素(字符串)的所有参数
语法	FILE3_BUFELEPALL(linenum, elenum, paras, tabletype, tableval) linenum: 行号 elenum: 元素编号 paras: 参数个数 tabletype: 存储参数类型的 table 地址, 顺序存储 0 数值 1 名称 2 字符串 3 字符串数值 tableval: 储存参数值的 table 地址, 每个参数间隔 40 个 table
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFELEPALL2 -- 读取元素的所有参数

类型	代码编辑
描述	读取指定元素(字符串)的所有参数
语法	FILE3_BUFELEPALL(string, paras, tabletype, tableval) string: 字符串元素, 如" xxxx(a,b,c)" paras: 参数个数 tabletype: 存储参数类型的 table 地址, 顺序存储 0 数值 1 名称 2 字符串 3 字符串数值 tableval: 储存参数值的 table 地址, 每个参数间隔 40 个 table
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFWRITE -- 写入行

类型	FILE3 文件编辑
描述	向指定行（行号）写入内容 原先的行数据将会被覆盖掉
语法	FILE3_BUFWRITE(linenum, string) linenum: 行号 string: 写入行字符串
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	<pre>dim string(128) string = "G1 X100 Y100 F100" FILE3_BUFWRITE (1, string) ' 向第 1 行写入数据</pre>

FILE3_BUFINSERT -- 插入行

类型	FILE3 文件编辑
描述	向指定行插入内容（从上方位置插入） 如向第 1 行插入内容, 原来的第 1 行变第 2 行, 插入内容在放入第 1 行
语法	FILE3_BUFINSERT(linenum, string) linenum: 行号 string: 插入行字符串
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	<pre>dim string(128) string = "G1 X100 Y100 F100" FILE3_BUFINSERT(1, string) ' 往第 1 行插入 string</pre>

FILE3_BUFDELN -- 删除选中内容

类型	FILE3 文件编辑
描述	从指定行开始删除 N 行内容
语法	FILE3_BUFDELN(mode, linenum, delnum) mode: 删除模式, 0-完全删除, 1-保留空行 (1 行), 仅留下换行符 linenum: 删除起始行号 delnum: 删除的行数
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFNOTE -- 注释选中内容

类型	FILE3 文件编辑
描述	注释选中的代码内容 (以行为单位) 在每行前面添加注释符 “'” 或 “;”, 不同编辑文件注释符不同, 如 z3p 文件注释符为 “'”, NC 文件注释符为 “;” 对于不支持 NC 文件的控制器, 若加载 NC 文件并进行注释, 默认使用单引号进行注释 删除注释时, 若选中行行首无注释符, 该行不会进行操作
语法	FILE3_BUFNOTE(mode, notelinenum, lines) mode: 0-取消注释, 1-注释 其他值保留扩展, 指定注释符号 (ACSSI 码), 正数表示添加注释, 负数表示删除注释 notelinenum: 注释起始行号 lines: 注释总行数
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFCOPYN -- 复制 N 行

类型	FILE3 文件编辑
描述	从指定行复制 N 行代码到 粘贴板 , 可以通过 FILE3_BUFPASTE 指令进行粘贴到指定位置
语法	FILE3_BUFCOPYN(copylinenum, lines) copylinenum: 复制起始行号 lines: 复制行数
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	FILE3_BUFCOPYN(10, 5) ' 从第 10 行复制 5 行 (10-14 行内容) FILE3_BUFPASTE(15) ' 粘贴到第 15 行 (15-19 行)

FILE3_BUFCUTN -- 剪切 N 行

类型	FILE3 文件编辑
描述	从指定行剪切 N 行代码到 粘贴板 ，可以通过 FILE3_BUFPASTE 指令进行粘贴到指定位置， 被剪切的原内容将被删除
语法	FILE3_BUFCUTN(cutlinenum, lines) cutlinenum: 剪切起始行号 lines: 剪切行数
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	FILE3_BUFCUTN(10, 5) ' 从第 10 行剪切 5 行 (10-14 行内容被删除) FILE3_BUFPASTE(1) ' 粘贴到第 1 行 (1-5 行)

FILE3_BUFPASTE -- 粘贴到指定位置

类型	FILE3 文件编辑
描述	将 粘贴板 的内容复制到指定行，若粘贴板内容为空，则粘贴无效
语法	FILE3_BUFPASTE(pastelinenum) pastelinenum: 粘贴起始行号
适用控制器	带 linux 系统或者 4 系以上的控制器
例子	FILE3_BUFCUTN(10, 5) ' 从第 10 行剪切 5 行 (10-14 行内容被删除) FILE3_BUFPASTE(1) ' 粘贴到第 1 行 (1-5 行)

FILE3_BUFUNDO -- 撤销上一步操作

类型	FILE3 文件编辑
描述	撤销上一步操作 FILE3 编辑指令可支持最大 8 步的撤销恢复功能，超过 8 步的将永久不能撤销恢复，当上一步操作为撤销操作时，才能进行恢复操作
语法	FILE3_BUFUNDO() linenum = FILE3_BUFUNDO() 返回撤销操作行号，操作失败返回 0
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFREDO -- 恢复上一步操作

类型	FILE3 文件编辑
描述	恢复上一步操作 FILE3 编辑指令可支持最大 8 步的撤销恢复功能，超过 8 步的将永久不能撤销恢复，当上一步操作为撤销操作时，才能进行恢复操作
语法	FILE3_BUFREDO()

	linenum = FILE3_BUFREDO() 返回重做操作行号，操作失败返回 0
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFSEEK -- 查找

类型	FILE3 文件编辑
描述	从指定行开始查找指定内容（返回第一个）
语法	line = FILE3_BUFSEEK(mode, startlinenum, string) mode: 查找方式: bit0（是否反向查找），bit1（是否大小写敏感），bit2（是否全字匹配） startlinenum: 查找起始行号， string: 待查找的字符串 line: 返回查找的行号，0 为无查找结果 FILE3_BUFSEEK(tablefirst, mode, startrow, startcol, string) tablefirst: 返回查找值 第 1 个 - 目标所在行号，0 为无结果 第 2 个 - 目标所在位置（列） mode: 查找方式: bit0（是否反向查找），bit1（是否大小写敏感），bit2（是否全字匹配） startrow: 查找起始行号 startrow: 查找行起始列 string: 待查找的字符串
适用控制器	带 linux 系统或者 4 系以上的控制器

FILE3_BUFREPLACEALL -- 全部替换

类型	FILE3 文件编辑
描述	将指定字符串 oldstring 全部 替换成字符串 newstring 单个替换功能可在 basic 上通过查找和读写行实现
语法	FILE3_BUFREPLACEALL (tablefirst, mode, oldstring, newstring) tablefirst: table 索引 第 1 个: 替换个数, -1=替换进行中, >=0 为替换个数 mode: 查找方式: bit0（不使用），bit1（是否大小写敏感），bit2（是否全字匹配） oldstring: 查找的字符串 newstring: 替换的字符串
适用控制器	带 linux 系统或者 4 系以上的控制器

9.3 任务参数指令

BASE_MOVE -- 指定主轴

类型	任务参数
描述	用于强制指定插补运动函数的 BASE 主轴，此参数不修改实际的运动。 缺省值为-1，此时不生效。 20160326 以上固件版本支持。 每个任务都有独立的 BASE_MOVE 参数。 MOVE、MOVEABS、MOVECIRC、MOVE_OP、MOVE_TASK 等插补类型函数有效， 凸轮点位等单轴函数无效。
语法	VAR1 = BASE_MOVE, BASE_MOVE = value
适用控制器	特殊固件支持
例子	BASE_MOVE=2 '强制本任务的插补运动使用轴 2 为主轴，速度参数也使用轴 2 的 MERGE(2)=ON '轴 2 启动连续 SPEED(2)=100 ACCEL(2)=1000 BASE(0,1) MOVE(100,100) '轴 0/1 插补，轴 2 也强制作为主轴参与，但运动距离为 0 MOVE_OP(1,1) BASE(1) MOVE(100) '1 轴运动 100，也使用轴 2 为主轴 BASE_MOVE=-1 '取消本任务的强制主轴

PROC_STATUS -- 任务状态

类型	任务状态
描述	当前任务的状态。 0 任务停止 1 任务正在运行 3 任务暂停中
语法	VAR1 = PROC_STATUS(tasknum) tasknum: 任务号
适用控制器	通用
例子	PRINT PROC_STATUS(0) '打印任务 0 状态 在线命令输入 >>PRINT PROC_STATUS(0) 输出: 1
相关指令	PROC

PROC -- 任务编号

类型	任务修正辅助指令
描述	当访问任务参数，任务状态时可以指定其他的任务。
语法	PROC(tasknum) tasknum: 任务号 可以省略，参考 AXIS
适用控制器	通用
例子	例一 完整写法 PRINT PROC_STATUS PROC(1) '打印任务 1 运行状态 例二 简写 PRINT PROC_STATUS(1) '打印任务 1 运行状态
相关指令	PROCNUMBER

PROCNUMBER -- 当前任务编号

类型	任务特殊状态，系统状态
描述	当前任务的任务号。 当程序不知道当前任务号的时候，通过此指令来访问。 这个状态不能通过 PROC 来修正。
语法	VAR1 = PROCNUMBER
适用控制器	通用
例子	PRINT PROCNUMBER '打印当前任务号
相关指令	PROC

PROC_LINE -- 任务行号

类型	任务状态
描述	任务的当前行号，只能获取其他任务的。
语法	VAR1 = PROC_LINE(tasknum) tasknum: 任务号
适用控制器	通用
例子	PRINT PROC_LINE(0) '打印任务 0 运行到那一行 在线命令输入： >>PRINT PROC_LINE 输出：100
相关指令	PROC

PROC_PROGRESS -- 任务指令进度

类型	任务状态
描述	任务指令进度，FILE 指令使用，范围 0-100。 每个任务都有。 FILE 指令 LOAD_ZAR 时，可以通过 HMI 界面显示进度。 注意：FILE 指令执行时，只有 HMI 任务能同时扫描到，FILE 指令不要直接由 HMI 任务来驱动。
语法	VAR1 = PROC_LINE(tasknum) tasknum: 任务号
适用控制器	通用
相关指令	PROC

PROC_PRIORITY -- 任务优先级

类型	任务状态
描述	任务优先级，范围 1-10，10 最高，默认值为 1，建议只改一个任务。 每个任务都有。 需要使用支持此功能的固件，设置不生效时建议升级固件。
语法	命令语法：PROC_PRIORITY(tasknum)=value 读取语法：VAR1 = PROC_PRIORITY(tasknum) tasknum: 任务号
适用控制器	通用
例子	PROC_PRIORITY(5)=3 '任务 5 优先级为 3 ?PROC_PRIORITY(5) '查看任务 5 优先级
相关指令	PROC

ERROR_LINE -- 任务错误行号

类型	任务状态
描述	当前任务的错误行号。 一般只用在出错后，在线命令查看。
语法	VAR1 = ERROR_LINE(tasknum) tasknum: 任务号
适用控制器	通用
例子	在线命令输入 >>>?ERROR_LINE(1) '打印任务 1 错误行
相关指令	PROC ， ERROR_SET

RUN_ERROR -- 任务错误码

类型	任务状态
描述	任务中最先出现的错误编号。
语法	VAR1 = RUN_ERROR(tasknum) tasknum: 任务号
适用控制器	通用
例子	?* RUN_ERROR(0) '打印任务 0 最先出现的错误号 2043
相关指令	ERROR_LINE

TICKS -- 任务计数周期

类型	任务参数
描述	当前任务的计数周期数，每个周期减一，单位是毫秒。 每个任务都有独立的 TICKS 参数，ZMC00x 系列，ZMC1xx 系列的周期为 1 毫秒。 系统刷新周期修改不影响本 TICKS 的计时。
语法	VAR1 = TICKS, TICKS = value
适用控制器	通用
例子	TICKS = 1000 WAIT UNTIL TICKS < 0 '等待 TICKS<0 后程序往下执行 MOVE(100)
相关指令	TIME_TICKUS

TIME_TICKUS -- 任务计时

类型	任务参数
描述	当前任务的执行时间，单位是微秒，每个 μs 加 1。 每个任务都有独立的 TIME_TICKUS 参数，32 位整数。 系统刷新周期修改不影响本 TIME_TICKUS 的计时。
语法	VAR1 = TIME_TICKUS, TIME_TICKUS = value
适用控制器	部分 3 系列及以上控制器（?*set 打印参数看是否支持）
例子	TIME_TICKUS=0 DELAY(1) '延时 1 毫秒 ?TIME_TICKUS '打印结果：1000，单位微秒
相关指令	TICKS

第十章 运算符及数学函数指令

RTBasic 支持标准 BASIC 的所有运算符，也采用标准 BASIC 的优先级。

优先级顺序：算术运算符 > 比较运算符 > 逻辑运算符。相同优先级的运算符采用从左到右的顺序计算。

算术运算符		比较运算符		逻辑运算符	
描述	符号	描述	符号	描述	符号
求幂	^	等于	=	按位非	Not
负号	-	不等于	<>	按位与	And
乘	*	小于	<	按位或	Or 或
除	/	大于	>	按位异或	Xor
整除	\	小于等于	<=	按位同或	Eqv
求余	Mod 或 %	大于等于	>=		
加	+				
减	-				
左移位	<<				
右移位	>>				

10.1 算术运算指令

+ -- 加法运算

类型	运算符
描述	将两个表达式相加。
语法	expression1 + expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>PRINT 1+2 输出: 3

- -- 减法运算

类型	运算符
描述	将表达式 1 减去表达式 2。
语法	expression1 - expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用

例子	在线命令输入 >>PRINT 2-(2-1) 输出：1
----	-----------------------------------

* -- 乘法运算

类型	运算符
描述	将表达式 1 与表达式 2 相乘。
语法	expression1 * expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>PRINT 10*(1+2) 输出：30

/ -- 除法运算

类型	运算符
描述	将表达式 1 除以表达式 2。
语法	expression1 / expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>PRINT 10/3 输出：3.3333

\ -- 整除运算

类型	运算符
描述	整数除法。
语法	expression1 \ expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>PRINT 10 \ (1+2) 输出：3

<< -- 左移位

类型	运算符
描述	左移位。
语法	expression1 << expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式 运算优先级低于其他四则运算符，共同使用时需要加括号(), 见例三。
适用控制器	通用
例子	例一 直接操作数值 在线命令输入 >>PRINT 8<<1 '二进制左移一位 输出: 16 在线命令输入 >>PRINT 8<<2 '二进制左移两位 输出: 32 例二 操作变量、寄存器 DIM bb bb=8 MODBUS_REG(0)=8 PRINT bb<<1,bb<<2 PRINT MODBUS_REG(0)<<1,MODBUS_REG(0)<<2 例三 优先级比较 >>PRINT 8<<1+1 '此时二进制左移 2 位 输出: 32 >>PRINT (8<<1)+1 '此时二进制左移 1 位 输出: 17

>> -- 右移位

类型	运算符
描述	右移位。
语法	expression1 >> expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式 运算优先级低于其他四则运算符，共同使用时需要加括号(), 见例三。

适用控制器	通用
例子	例一 直接操作数值 在线命令输入 >>PRINT 8>>1 '二进制右移一位 输出：4 在线命令输入 >>PRINT 8>>2 '二进制右移两位 输出：2 例二 操作变量、寄存器 DIM bb bb=8 MODBUS_REG(0)=8 PRINT bb>>1,bb>>2 PRINT MODBUS_REG(0)>>1,MODBUS_REG(0)>>2 例三 优先级比较 >>PRINT 8>>1+1 '此时二进制右移 2 位 输出：2 >>PRINT (8>>1)+1 '此时二进制右移 1 位 输出：5

MOD -- 求余数

类型	运算符
描述	求余数。
语法	expression1 MOD expression2 expression1: 任意有效的表达式，取整数部分。 expression2: 任意有效的表达式，取整数部分。
适用控制器	通用
例子	在线命令输入 >>PRINT 10 MOD (1+2) 输出：1

ABS -- 绝对值

类型	数学函数
描述	求绝对值。
语法	ABS(expression) expression: 任意有效的表达式
适用控制器	通用

例子	PRINT ABS(-11) '结果是 11
----	------------------------

10.2 比较运算指令

= -- 比较/赋值运算

类型	运算符
描述	比较运算符： 如果表达式 1 等于表达式 2，返回 TRUE，否则返回 FALSE 赋值运算符： 将表达式 2 赋值给前面的变量或参数等。
语法	expression1 = expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	例一 ON IN(0)=ON GOTO label1 '如果输入通道 0 为 ON，程序跳转到标识“label1:”首行开始执行 label1: PRINT 12 '打印 12 例二 DIM aaa aaa = 100 '变量 aaa 赋值为 100 PRINT aaa

<> -- 不等于

类型	运算符
描述	如果表达式 1 不等于表达式 2，返回 TRUE，否则返回 FALSE。
语法	expression1 <> expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	ON MODBUS_BIT(0)<>0 GOTO label1 '如果 MODBUS 位寄存器 0 非零值，程序跳转到标识"label1:"开始执行 label1: PRINT 11 '打印 11

> -- 大于

类型	运算符
描述	如果表达式 1 大于表达式 2，返回 TRUE，否则返回 FALSE。
语法	expression1 > expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	WAIT UNTIL MPOS>100 该条语句将使程序循环等待，直到测量反馈位置大于 100 为止。 例一 DIM q '定义变量 q= 2>1 '2 大于 1，所以返回 true PRINT q '打印返回值 例二 DIM a '定义变量 a=0 '变量赋值 REPEAT '循环执行 a=a+1 '加一 ?a '打印 DELAY(200) '延时 UNTIL a>10 '条件判断，直到 a 里面的值大于 10 时，停止循环执行

>= -- 大于等于

类型	运算符
描述	如果表达式 1 大于或等于表达式 2，返回 TRUE，否则返回 FALSE。
语法	expression1 >= expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式
适用控制器	通用
例子	DIM a '定义变量 a= 1>=3 '1 小于 3，所以返回 FALSE PRINT a '打印返回值

< -- 小于

类型	运算符
描述	如果表达式 1 小于表达式 2，返回 TRUE，否则返回 FALSE。

语法	$\text{expression1} < \text{expression2}$ expression1 : 任意有效的表达式 expression2 : 任意有效的表达式
适用控制器	通用
例子	$\text{VAR1}=1<0$ 因为 1 大于 0, 所以 VAR1 的值将等于 FALSE(0)。

<= -- 小于等于

类型	运算符
描述	如果表达式 1 小于或等于表达式 2, 返回 TRUE, 否则返回 FALSE。
语法	$\text{expression1} \leq \text{expression2}$ expression1 : 任意有效的表达式 expression2 : 任意有效的表达式
适用控制器	通用
例子	$\text{VAR1}=1\leq 1$ VAR1 的值将等于 TRUE (-1)。

10.3 逻辑运算指令

AND -- 按位与

类型	运算符		
描述	按位与操作符，只操作整数部分。		
	AND		结果
	0	0	0
	0	1	0
	1	0	0
	1	1	1
语法	expression1 AND expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式		
	expression1 和 expression2 的二进制形式的每一个位上的二进制数字进行按位与(AND)运算之后的结果		
适用控制器	通用		
例子	在线命令输入 >>PRINT 1 AND 2 输出: 0		
	具体操作过程		

	1 对应二进制 01 2 对应二进制 10 与计算后 对应二进制 00 输出十进制 0
--	--

OR -- 按位或

类型	运算符					
描述	按位或操作符，只操作整数部分。					
				OR		结果
				0	0	0
				0	1	1
				1	0	1
	1	1	1			
语法	expression1 OR expression2 expression1：任意有效的表达式 expression2：任意有效的表达式 expression1 和 expression2 的二进制形式的每一个位上的二进制数字进行按位或（OR）运算之后的结果					
适用控制器	通用					
例子	在线命令输入 >>PRINT 1 OR 2 输出：3 具体操作过程 1 对应二进制 01 2 对应二进制 10 或计算后 对应二进制 11 输出十进制 3					

NOT -- 按位非

类型	运算符						
描述	按位非操作符，只操作整数部分，小心对 ON 等整数 NOT。 <table border="1"> <thead> <tr> <th>NOT</th><th>结果</th></tr> </thead> <tbody> <tr> <td>0</td><td>-1</td></tr> <tr> <td>1</td><td>-2</td></tr> </tbody> </table>	NOT	结果	0	-1	1	-2
NOT	结果						
0	-1						
1	-2						
语法	NOT expression1 expression1: 任意有效的表达式						
适用控制器	通用						
例子	在线命令输入 >>PRINT NOT 1						

	输出: -2 具体操作过程 1 对应二进制 ... 0000 0001 非计算后 对应二进制 ... 1111 1110 输出十进制 -2
--	---

XOR -- 按位异或

类型	运算符		
描述	逻辑异或操作符，按位异或，只操作整数部分。		
	XOR		结果
	0	0	0
	0	1	1
	1	0	1
	1	1	0
语法	expression1 XOR expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式		
适用控制器	通用		
例子	在线命令输入 >>PRINT 1 XOR 1 输出：0 具体操作过程 1 对应二进制 01 异或计算后 对应二进制 00 输出十进制 0		

EQV -- 按位同或

类型	运算符		
描述	按位同或操作符，只操作整数部分。		
	EQV		结果
	0	0	1
	0	1	0
	1	0	0
	1	1	1
语法	expression1 EQV expression2 expression1: 任意有效的表达式 expression2: 任意有效的表达式		
适用控制器	通用		
例子	在线命令输入		

	<pre>>>PRINT 2 EQV 1</pre> 输出: -4 具体操作过程 2 对应二进制 ... 0000 0010 1 对应二进制 ... 0000 0001 同或计算后 对应二进制 ... 1111 1100 输出十进制 -4
--	---

10.4 三角函数指令

SIN -- 三角函数正弦

类型	数学函数
描述	正弦三角函数，输入参数为弧度单位。
语法	SIN(expression) expression: 任意有效的表达式
适用控制器	通用
例子	PRINT SIN(PI/6) '结果是 0.5000

ASIN -- 三角函数反正弦

类型	数学函数
描述	反正弦三角函数，返回值为弧度单位。
语法	ASIN(expression) expression: 任意有效的表达式
适用控制器	通用
例子	PRINT ASIN(0.5) '结果是 0.52360

COS -- 三角函数余弦

类型	数学函数
描述	余弦三角函数，输入参数为弧度单位。
语法	COS(expression) expression: 任意有效的表达式
适用控制器	通用
例子	PRINT COS(PI/3) '结果是 0.5000

ACOS -- 三角函数反余弦

类型	数学函数
描述	反余弦三角函数，返回值为弧度单位。
语法	ACOS(expression) expression: 任意有效的表达式
适用控制器	通用
例子	PRINT ACOS(0.5) '结果是 1.04720=PI/3

TAN -- 三角函数正切

类型	数学函数
描述	求正切三角函数，输入参数为弧度单位。
语法	TAN(expression) expression: 任意有效的表达式
适用控制器	通用
例子	PRINT TAN(PI/3) '结果是 1.732

ATAN -- 三角函数反正切

类型	数学函数
描述	求反正切三角函数，返回值为弧度单位。
语法	ATAN(expression) expression: 任意有效的表达式
适用控制器	通用
例子	PRINT ATAN(1) '结果是 0.7854 = (45/180)*PI

ATAN2 -- 三角函数反正切 2

类型	数学函数
描述	反正切三角函数，返回值为弧度单位。
语法	ATAN2(y, x) y: y 坐标 x: x 坐标
适用控制器	通用
例子	PRINT ATAN2(1,0) '结果是 1.5708

10.5 指数运算指令

EXP -- 指数

类型	数学函数
描述	指数函数。
语法	EXP([base,] expvalue) base: 底数, 缺省为 e expvalue: 指数
适用控制器	通用
例子	例一 PRINT EXP(2,4) '结果是 16 (2*2*2*2) 例二 PRINT EXP(1) '结果是 2.7183

SQR -- 平方根

类型	数学函数
描述	平方根函数。
语法	SQR(expression) expression: 任意有效的表达式
适用控制器	通用
例子	a= SQR(4) PRINT a '结果是 2

LN -- 自然对数

类型	数学函数
描述	自然对数函数。
语法	LN(expression) expression: 任意有效的表达式
适用控制器	通用
例子	a= LN(1) PRINT a '结果是 0

LOG -- 对数底为 10

类型	数学函数
描述	对数, 底数为 10。

语法	LOG(expression) expression: 任意有效的表达式
适用控制器	通用
例子	a=LOG(100) PRINT a '结果是 2

10.6 数据操作指令

SET_BIT -- 按位设置

类型	数学指令或函数
描述	位操作，只对整数，修改对应位为 1。 分为命令语法和函数语法。 对 VR 寄存器可设置的位范围是 0-24。
语法	命令语法: SET_BIT(bit#, vr#) 直接操作 VR 寄存器 bit#: 位编号: 0-24 vr#: 要操作的 VR 变量编号, 整数部分 命令语法使用时没有返回值, 直接操作, 修改操作对象的值。 函数语法: ret = SET_BIT(bit#,int) ret: 操作结果 bit#: 位编号: 0-24 int: 要操作的表达式, 取整数部分 函数语法使用时返回操作后的结果, 操作对象的值不变。
适用控制器	通用
例子	例一 命令语法 VR(23)=0.333 SET_BIT(0,23) 'VR(23)的第 0 位将置为 1, 并清除小数部分 ?VR(23) '结果为 1 例二 函数语法 DIM a,b a=0.333 b=0 b=SET_BIT (0,a) '设置 a 的第 0 位的, 结果赋值到 b, 并清除小数 PRINT a,b '打印结果 0.333,1, a 没有改变, b 为 1
相关指令	CLEAR_BIT , READ_BIT , READ_BIT2

CLEAR_BIT -- 按位置 0

类型	数学指令或函数
----	---------

描述	位操作，只对整数，修改对应位为 0。 分为命令语法和函数语法。 对 VR 寄存器可设置的位范围是 0-24。
语法	命令语法: <code>CLEAR_BIT(bit#,vr#)</code> 直接操作 VR 寄存器 bit#: 位编号: 0-24 vr#: 要操作的 VR 变量编号, 整数部分 命令语法使用时没有返回值, 直接操作, 修改操作对象的值。 函数语法: <code>ret = CLEAR_BIT(bit#,int)</code> ret: 操作结果 bit#: 位编号: 0-24 int: 要操作的表达式, 取整数部分 函数语法使用时返回操作后的结果, 操作对象的值不变。
适用控制器	通用
例子	例一 命令语法 <code>VR(23)=3.333</code> <code>CLEAR_BIT(0,23)</code> 'VR(23)的第 0 位将被清除(设置为 0) <code>?VR(23)</code> '打印结果 2, 小数被去除 例二 函数语法 <code>DIM a,b</code> <code>a=3.333</code> <code>b=0</code> <code>b=CLEAR_BIT(0,a)</code> '返回清除 a 的第 0 位和小数后的结果给 b <code>PRINT a,b</code> '结果为 3.333,2, a 不变,b 为 2
相关指令	SET_BIT , READ_BIT , READ_BIT2

READ_BIT -- 按位读取

类型	数学函数
描述	位操作，只对整数，读取对应位状态。 只能操作 VR 寄存器，非 VR 参考 READ_BIT2 。 对 VR 寄存器可设置的位范围是 0-24。
语法	<code>ret = READ_BIT(bit#, vr#)</code> ret: 读取结果: 1 或 0 bit#: 位编号: 0-24 vr#: 要操作的 VR 变量编号
适用控制器	通用
例子	<code>VR(23)=3.333</code> <code>PRINT READ_BIT(0,23)</code> '读取 VR(23)的第 0 位, 结果为 1
相关指令	SET_BIT , CLEAR_BIT , READ_BIT2

READ_BIT2 -- 按位读取 2

类型	数学函数
描述	位操作，只对整数，读取对应位状态。
语法	<pre>ret = READ_BIT2(bit#, int)</pre> ret: 读取结果: 1 或 0 bit#: 位编号: 0-31 int: 要操作的表达式，取整数部分
适用控制器	通用，20130813 以后的固件版本提供了支持
例子	<pre>DIM a,b b=1.64 a=READ_BIT2(0,b) '读取 b 的第 0 位,赋值给 a PRINT a '输出 a 值，结果为 1</pre>
相关指令	SET_BIT ， CLEAR_BIT -- 按位置 0 ， READ_BIT

FRAC -- 返回小数

类型	数学函数
描述	返回小数部分，总是大于 0 的部分。
语法	<pre>FRAC(expression)</pre> expression: 要操作的数
适用控制器	通用
例子	<pre>a=FRAC(1.235) PRINT a '结果是 0.235</pre>

INT -- 返回整数

类型	数学函数
描述	返回整数部分。
语法	<pre>INT(expression)</pre> expression: 任意有效的表达式
适用控制器	通用
例子	<pre>a=INT(1.235) PRINT a '结果是 1 ?INT(-1.1) '打印结果，-2，因为小数部分总处理为正数。</pre>

SGN -- 返回符号

类型	数学函数
描述	返回符号。

	1 大于 0 0 等于 0 -1 小于 0
语法	SGN(expression) expression: 任意有效的表达式
适用控制器	通用
例子	a=SGN(-1.235) PRINT a '结果是-1

IEEE_IN -- 组合浮点数

类型	数学函数
描述	把四个字节组合成一个单精度浮点数。
语法	IEEE_IN(byte0,byte1,byte2,byte3) byte0 - byte3: 四个字节
适用控制器	通用
例子	VAR = IEEE_IN(VR(10),VR(11),VR(12),VR(13)) '将 VR(10)~VR(13)这四个数据合成一个单精度浮点数

IEEE_OUT -- 提取单字节

类型	数学函数
描述	从一个单精度浮点数里面取一个字节。
语法	byte_n = IEEE_OUT(var, n) var: 单精度浮点数 N: 0-3, 取第几个字节
适用控制器	通用
例子	例一 VAR = IEEE_OUT(VR(1),2) '提取 VR(1)的第二个字节 例二 GLOBAL VAR0,VAR1,VAR2,VAR3 VAR0=0 VAR1=0 VAR2=0 VAR3=0 VR(1)=123.456 VAR0 = IEEE_OUT(VR(1),0) VAR1 = IEEE_OUT(VR(1),1) VAR2 = IEEE_OUT(VR(1),2) VAR3 = IEEE_OUT(VR(1),3) VR(2)=0 VR(2)=IEEE_IN(VAR0,VAR1,VAR2,VAR3)

	运行结果:												
	 The screenshot shows a window titled '监视' (Monitor) with a table containing the following data: <table border="1"> <thead> <tr> <th>监视内容</th> <th>值</th> </tr> </thead> <tbody> <tr> <td>var0</td> <td>66</td> </tr> <tr> <td>var1</td> <td>246</td> </tr> <tr> <td>var2</td> <td>233</td> </tr> <tr> <td>var3</td> <td>121</td> </tr> <tr> <td>vr(2)</td> <td>123.4560</td> </tr> </tbody> </table>	监视内容	值	var0	66	var1	246	var2	233	var3	121	vr(2)	123.4560
监视内容	值												
var0	66												
var1	246												
var2	233												
var3	121												
vr(2)	123.4560												

\$ -- 16 进制

类型	特殊字符
描述	表示紧接着的数据是 16 进制格式。
语法	\$hexnum
适用控制器	通用
例子	在线命令输入 >>PRINT \$F 输出: 15

10.7 字符串操作指令

CHR -- ASCII 码打印

类型	字符串函数
描述	返回 ASCII 打印, 用于 PRINT, 或者是放在赋值等号的右边
语法	CHR(expression) expression: 任意有效的表达式
适用控制器	通用
例子	在线命令输入 >>PRINT CHR(66) 输出: B

HEX -- 16 进制打印

类型	字符串函数
描述	返回十六进制格式, 只用于 PRINT。
语法	HEX(expression)

	expression: 任意有效的表达式, 只取整数部分
适用控制器	通用
例子	在线命令输入 >>PRINT HEX(15) 输出: f '十六进制

STRLEN -- 返回字符串长度

类型	字符串函数
描述	返回字符串长度。
语法	len=STRLEN(str) str: 字符串
适用控制器	通用
例子	DIM str_a(20) str_a="len123" ?STRLEN(str_a) 打印结果: 6

TOSTR -- 格式化输出

类型	字符串函数
描述	格式化输出函数, 变量转换成字符串。
语法	TOSTR(VAR1, [N],[DOT]) VAR1: 任意有效的表达式 N: 输出数据总位数, 包括小数点和符号位。当 N 设置负数时, 表示右对齐 DOT: 输出的小数个数, 当 N 太小没有小数位置时, 不输出小数 输出的是字符串类型。只有第一个参数默认打印到小数点后四位。
适用控制器	通用
例子	例一 在线命令输入 >> PRINT TOSTR(2-100,6,2) 输出: -98.00 例二 DIM aa(20) aa="asd13"+TOSTR(354) ?aa '打印结果 asd13354.0000

STRCOMP -- 字符串比较

类型	字符串函数
描述	字符串比较函数，根据两个字符串的情况返回>0、=0、<0。 比较长度在 500 字节内，否则返回值出错误。
语法	STRCOMP(str1, str2) str1: 字符串 1 str2: 字符串 2
适用控制器	通用
例子	DIM AAA(10) AAA = "abc" 在线命令输入 >>PRINT STRCOMP(AAA, "abc") 输出: 0

STRFIND -- 字符串搜索

类型	字符串函数
描述	字符串搜索函数。
语法	STRFIND(str1, str2 [, firstindex]) str1: 待搜索的字符串 str2: 搜索模板字符串 firstindex: 从哪个位置开始搜索，缺省 0 返回: >= 0, 返还查找到的 index; < 0, 没有找到
适用控制器	通用
例子	DIM AAA(10),BBB(3) AAA="AD23GF41" BBB="23G" ?STRFIND(AAA,BBB) '打印搜索到的索引位置, 2

STRCONV -- 编码转换

类型	字符串函数
描述	不同编码的字符串转换。 只支持不带 0 的编码，UTF16 不支持。 支持的编码: CP936、UTF-7、UTF-8、GB2312 等。
语法	string 2= STRCONV("srccodename", "descodename", "string1")
适用控制器	带 linux 控制器和 7 系适用
例子	DIM arrstring(100) arrstring = STRCONV("gb2312","utf8", "文件夹") ?STRCONV("utf8", "gb2312", arrstring) '输出结果: 文件夹

VAL -- 字符转数值

类型	字符串函数
描述	字符串转换为数值。 只能转换数字字符，遇到字母、符号字符停止。
语法	VAL(str1) str1: 字符串
适用控制器	通用
例子	例一 VAR1 = VAL("123") ?VAR1 '打印结果, 123 例二 VAR2 = VAL("123QWE23") ?VAR2 '打印结果, 123

10.8 常数指令

PI -- 圆周率

类型	常数
值	3.14159
适用控制器	通用

TRUE -- 真值

类型	常数
值	-1
适用控制器	通用

FALSE -- 假值

类型	常数
值	0
适用控制器	通用

ON -- 开启

类型	常数
值	1
适用控制器	通用

OFF -- 关闭

类型	常数
值	0
适用控制器	通用

10.9 高级运算指令

CRC16 -- CRC 检验计算

类型	数学函数
描述	CRC16 CCITT 计算。
语法	CRC16(arrayname, index, size[, initial] [, poly]) arrayname: 数据存储所在的数组, 一个字节占一个位置 index: 数据存储所在的数组起始索引 size: 计算字节数 initial: CRC 计算初始值, 缺省\$FFFF poly: 多项式, 暂时只支持 modbus 的\$A001 和 CCITT 的\$1021, 缺省\$A001
适用控制器	通用
例子	TABLE(0, \$FE, \$48, \$06, \$00, \$6D, \$00, \$00, \$00) '8 个数据存储存储在 TABLE CRCVALUE = CRC16(TABLE, 0, 8) '计算 CRC, 结果\$1A0D TABLE(8) = CRCVALUE\256 '计算的 CRC 加在数据的后面, 大端模式 TABLE(9) = CRCVALUE AND \$FF

DTSMOOTH -- table 平滑

类型	数学函数
描述	对 TABLE 存储的点坐标进行平滑调整。
语法	DTSMOOTH (axis, dtfirst, space, points, imode, referradius) axis: 轴数 dtfirst: 第一个点的 TABLE 索引 space: 两个点之间的索引间隔(就是一个点存储占用的空间) points: 总点个数

	imode: 0- 绝对方式, 对曲率半径小于参考值的进行调整 referradius: 参考曲率半径, 可以根据 $\text{半径} = (\text{速度平方}) / \text{拐弯加速度}$ 来计算参考
适用控制器	3X 系列 20161206 以上固件支持 4 系列 20170508 以上固件支持
例子	TABLE(0, 0, 0) TABLE(5, 99, 0) TABLE(10, 100, 0) TABLE(15, 100, 1) TABLE(20, 101, 1) TABLE(25, 200, 1) DTSMOOTH (2, 0, 5, 6, 0, 5) ?*TABLE(0, 2) ?*TABLE(5, 2) ?*TABLE(10, 2) ?*TABLE(15, 2) ?*TABLE(20, 2) ?*TABLE(25, 2)

B_SPLINE -- B 样条平滑

类型	数学函数
描述	将 TABLE 中的数据进行 B 样条平滑。
语法	B_SPLINE(type, data_start, points, data_out, ratio) type: 类型, 目前只支持 1-B 样条 data_start: 图形数据在 TABLE 中的起始位置 points: 图形数据的个数 data_out: 平滑后的图形数据在 TABLE 中起始位置 ratio: B_SPLINE 函数的平滑比率, 平滑后的个数为 $\text{points} * \text{ratio}$ 新增加样条控制点的自动计算功能, 此功能配合 MOVESPLINE 样条曲线运动使用, 4 系列以上产品支持, 4 系列控制器固件版本 20170621 B_SPLINE(type, axes, dtstartpos, dtendpos, dtlastpos, dtnexpos, dtoutcontrol1, dtoutcontrol2) type: 1 兼容原来的功能 11 为连续线段的第一条线段计算样条拟合的控制点. 12 为连续线段的中间线段计算样条拟合的控制点. 13 为连续线段的最后一条线段计算样条拟合的控制点. axes: 参与样条插补的轴数 dtstartpos: 线段起点坐标位于的 table 数组索引, 多个轴连续存储不同的 table, 下同 dtendpos: 线段终点坐标位于的 table 数组索引 dtlastpos: 线段起点的前面点的坐标索引, 用于计算参考, 第一条线段此参数无用

	dtnexpos: 线段终点的后面点的坐标索引, 用于计算参考, 最后一条线段此参数无用 dtoutcontrol1: 输出样条的控制点数据, 贝塞尔的第 1 个控制点(起点也作为控制点除外) dtoutcontrol2: 输出样条的控制点数据, 贝塞尔的第 2 个控制点 起点、dtoutcontrol1、dtoutcontrol2、终点一起构成贝塞尔的 4 个控制点。
适用控制器	通用
例子	例一 type1 B_SPLINE(1,0,10,100,10) '平滑一个 10 点的图形数据, 源图形点在 table 的位置为从 0 到 9, 平滑为 100 点的数据, 并且平滑后的数据从 table 地址 100 开始存放 例二 新增模式 TABLE(0,0,0,0,100,100,100,200,100) 'XY 两轴连续 4 点的坐标数据 B_SPLINE(11, 2, 0, 2, -1, 4, 100,200) '第 1 条线段 ?TABLE(100),TABLE(101),TABLE(200),TABLE(201) B_SPLINE(12, 2, 2, 4, 0, 6, 100,200) '第 2 条线段 ?TABLE(100),TABLE(101),TABLE(200),TABLE(201) B_SPLINE(13, 2, 4, 6, 2, 6, 100,200) '第 3 条线段 ?TABLE(100),TABLE(101),TABLE(200),TABLE(201)

TURN_POSMAKE -- 旋转坐标计算

类型	数学函数
描述	旋转功能的计算函数, 计算旋转台上点(X,Y)旋转 R 度后点的绝对坐标, 旋转的正向与 XY 的正向要一致(右手法则)。
语法	TURN_POSMAKE(tablenum, posx, posy, disR, tableout) tablenum: 旋转功能参数存储 table 编号 posx: X 方向的坐标 posy: Y 方向的坐标 disR: 旋转轴的相对偏移 tableout: 存储计算后的坐标
适用控制器	通用
相关指令	MCIRC_TURNABS

ZCUSTOM -- 运动参数计算

类型	数学函数
描述	计算各种运动指令中的参数, 详细功能请看下方语法功能描述。
语法	功能 2: 计算空间圆弧或空间直线上一定距离后的点的位置。 Table 表参数按三点画圆模式填写其他两个点参数。 填写相对坐标, 返回也是相对的位置。 语法: ZCUSTOM(2,tableend,tablemid,tableout,mode,vectdis) tableend: 存储圆弧终点的 table 索引 (3 维坐标)

tablemid: 存储圆弧中间点的 table 索引, 与当前点一起构成圆弧的 3 个点 (3 维坐标)

tableout: 输出计算数据的 table 索引

mode: 模式

值	描述
1	相对起点的空间圆弧弧长
2	相对终点的空间圆弧弧长
3	相对起点的空间直线距离, 此时 tablemid 不起作用
4	相对终点的空间直线距离, 此时 tablemid 不起作用
5	相对计算空间圆弧的圆心, 此时 vectdis 无效。此时 tableout 依次输出 xyz 圆心, 弧度范围, 弧长

vectdis: 要计算的点相对 mode 的距离, 负数表示往前。圆弧模式时, 正数表示顺时针, 负数表示逆时针

功能 6: 计算空间圆弧的起点和终点的切线角度方向, 弧度单位。

填写相对坐标, 返回也是相对的位置。

语法: ZCUSTOM(6,tableend,tablemid,tableout)

tableend: 存储圆弧终点的 table 索引

tablemid: 存储圆弧中间点的 table 索引, 与当前点一起构成圆弧的 3 个点

tableout: 输出计算数据的 table 索引, 依次输出:起点 xy 方向, 起点 Z 方向, 终点 xy 方向, 终点 Z 方向, 角度弧度单位

功能 7: 输入速度比例, 计算 MOVESLINK 的从轴位置, 主轴位置。

语法: ZCUSTOM(7,distance,link dist,start sp,end sp,速度比例,tableout)

distance: 从连接开始到结束, 跟随轴移动的距离, 采用 units 单位

link dist: 参考轴在连接的整个过程中移动的绝对距离, 采用 units 单位

star sp: 启动时跟随轴和参考轴的速度比例, units/units 单位, 负数表示跟随轴负向运动

end sp: 结束时跟随轴和参考轴的速度比例, units/units 单位, 负数表示跟随轴负向运动

速度比例: 需要计算的点的速度比例, 与参数里面的起始结束点比例意义相同

tableout: 正向找的从轴距离, 正向找的主轴距离, 从反向找的从轴距离, 反向找的主轴距离, 占用 4 个 TABLE (一段曲线, 速度比例可能会有多个解)

功能 8: MOVESLINK 输入从轴位置, 反算主轴的位置。

语法: ZCUSTOM(8,distance,link dist,start sp,end sp,distancemoved,tableout)

distance: 从连接开始到结束, 跟随轴移动的距离

link dist: 从连接开始到结束, 主轴移动的绝对距离

start sp: 起始脉冲速度比例

end sp: 结束脉冲速度比例

distancemoved: 从轴已经运动距离

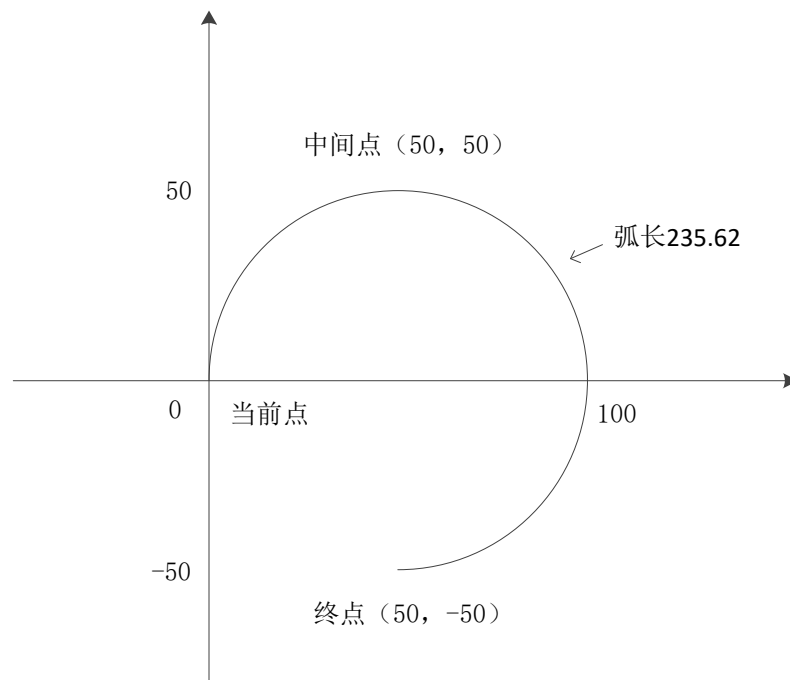
tableout: 输出 table 索引, 输出对应主轴的位置, 如多个解返回第一个

功能 9: FLEXLINK 输入从轴位置, 反算主轴的位置。

	语法：ZCUSTOM (9, base_dist, excite_dist, link_dist, base_in, base_out, excite_acc, excite_dec, distancemoved, tableout) base_dist: 跟随轴匀速运动距离 excite_dist: 跟随轴激励运动距离, 这个与 base_dist 相反的时候无法反算 link_dist: 整个指令过程, 跟随轴运动完成, 主轴移动的距离 base_in: 激励开始前, 跟随轴运动距离占 base_dist 的百分比 base_out: 激励完成后, 跟随轴剩余运动距离占 base_dist 的百分比, 两者相加不要超过 100% excite_acc: 激励过程中, 跟随轴加速阶段运动距离占 excite_dist 的百分比, excite_dist 为负值时, 为减速阶段 excite_dec: 激励过程中, 跟随轴减速阶段运动距离占 excite_dist 的百分比, excite_dist 为负值时, 为加速阶段 distancemoved: 从轴已经运动脉冲数 tableout: 输出 Table 索引, 输出对应主轴的位置, 如多个解返回第一个 功能 10: 从工件坐标上的三点在原来坐标系上的坐标来计算 FRAME_ROTATE 旋转转换的参数, 每个点要存储 xyz 的三个方向的坐标。 语法: ZCUSTOM (10, dtzero, dtx, dty, dtout) dtzero: 工件零点在原来坐标系上的位置 dtx: 工件坐标系 X 轴上的点在原来坐标系上的位置 dty: 工件坐标系 Y 轴上的点在原来坐标系上的位置 dtout: 输出 TABLE 索引, 分别存储: X, Y, Z, RX, RY, RZ 功能 12: 根据圆弧的终点和半径来计算圆心。 语法: ZCUSTOM (12, xpos, ypos, radius, anticlock, dtout) xpos: X 方向的相对坐标 ypos: Y 方向的相对坐标 radius: 半径, 负数表示圆弧扇形角度大于 180 度, 正数表示小于 180 度 anticlock: 0—顺时针, 1—逆时针 dtout: 输出 xy 圆心相对距离, 占用两个位置 注意: dtzero, dtx, dty, dtout 是 table 索引, 只用写索引不需要 table 也写入, table 存的是三个方向的 x,y,z, 用于机械手算法时, 要填入的是虚拟轴的坐标 功能 13: MOVESLINK 输入主轴轴位置, 反算主轴的位置。 语法: ZCUSTOM (13, distance, link dist, start sp, end sp, distancemoved,tableout) distance: 从轴周期脉冲数 link dist: 主轴周期脉冲数 start sp: 起始脉冲速度比例 end sp: 结束脉冲速度比例 distancemoved: 从轴已经运动脉冲数. tableout: 输出 Table 索引, 输出对应从轴的位置. 注意: 不带轴号, 要求 sp 是脉冲/脉冲单位; Distance、Linkdist 是脉冲个数。不考虑 units。
适用控制器	通用

例子

例一 功能 2 使用



TABLE(0,50,-50,0) '设置终点坐标, 相对位置
TABLE(3,50,50,0) '设置中间点坐标, 相对位置

'模式 1, 相对起点

ZCUSTOM(2,0,3,10,1,78.54) '相对起点位置顺时针弧长为 78.54 的圆上点

?TABLE(10),TABLE(11),TABLE(12) '输出相对坐标为 0,0,0

ZCUSTOM(2,0,3,10,1,-78.54) '相对起点位置逆时针弧长为 78.54 的圆上点

?TABLE(10),TABLE(11),TABLE(12) '输出相对坐标为 50,50,0

'模式 2, 相对终点

ZCUSTOM(2,0,3,10,2,78.54) '相对终点位置顺时针弧长为 78.54 的圆上点

?TABLE(10),TABLE(11),TABLE(12) '输出坐标为 50,-50,0

ZCUSTOM(2,0,3,10,2,-78.54) '相对终点位置逆时针弧长为 78.54 的圆上点

?TABLE(10),TABLE(11),TABLE(12) '输出坐标为 100,0,0

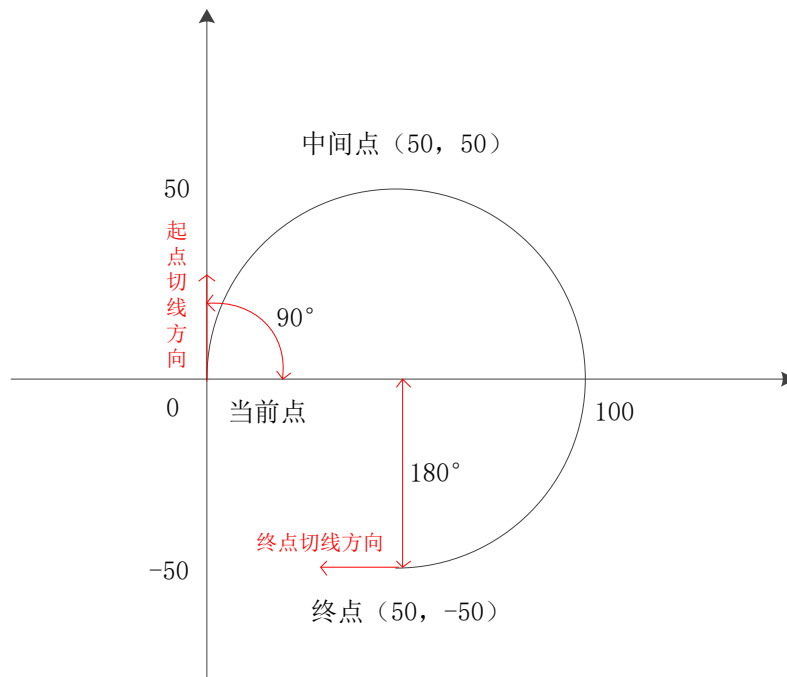
'模式 5, 计算此时三点画圆的圆心、弧度、弧长

ZCUSTOM(2,0,3,10,5,0) '此时距离参数不起作用

?TABLE(10),TABLE(11),TABLE(12) '输出坐标为 50,0,0

?TABLE(13),TABLE(14) '输出弧度 4.712, 弧长 235.62

例二 功能 6, 计算圆弧起点和终点的切线弧度



```
TABLE(0,50,-50,0)    '设置终点坐标, 相对位置
TABLE(3,50,50,0)     '设置中间点坐标, 相对位置
ZCUSTOM(6,0,3,10)    '计算当前点和终点切线的弧度
?TABLE(10),TABLE(11) '输出当前点 1.571,0 (1.571=90*PI/180)
?TABLE(12),TABLE(13) '输出终点-3.142,0 (-3.142=-180*PI/180)
```

例三 功能 8, 反算 MOVESLINK 主轴位置

```
BASE(0,1)
```

```
DPOS=0,0
```

```
UNITS=100,100
```

```
SPEED=100,100
```

```
ACCEL=1000,1000
```

```
TRIGGER
```

```
MOVESLINK(50,100,0,1,1) '建立 MOVESLINK 连接
```

```
MOVEABS(100) AXIS(1)
```

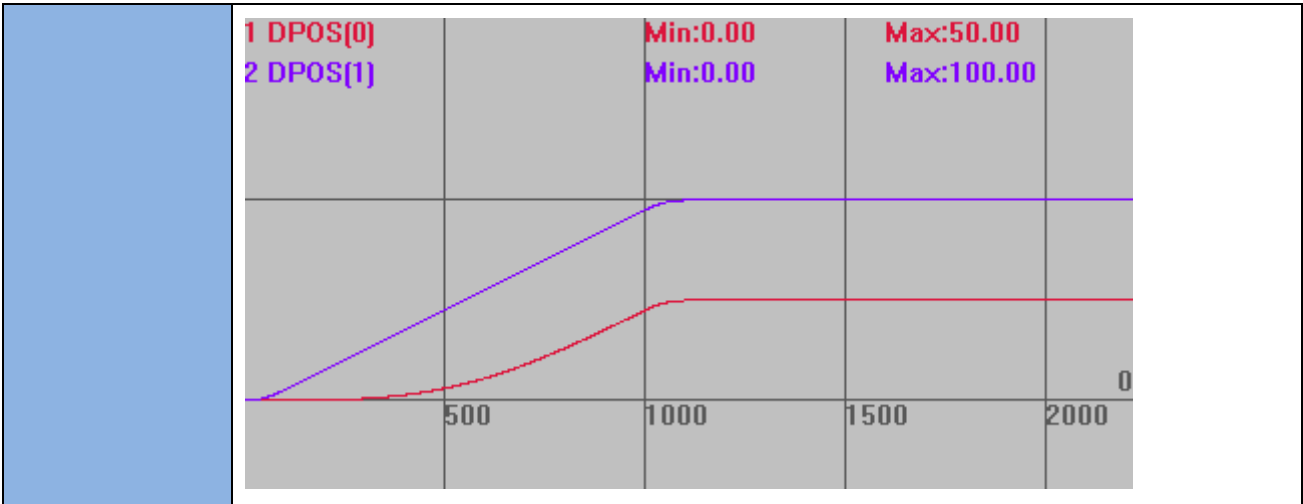
```
ZCUSTOM(8,50,100,0,1,25,10) '根据建立连接的参数, 计算从轴运动 25 时主轴的位置
```

```
?TABLE(10) '输出主轴位置 73.801
```

运动轨迹

```
DPOS(0)垂直刻度 100
```

```
DPOS(1)垂直刻度 100
```

ZMATH64 -- 64 位计算

类型	64 位计算指令																																																							
描述	对存储 D 寄存器 (MODBUS 寄存器) 的 64 位数进行计算。 一个 64 位有符号整数占用 4 个寄存器(小端模式)。 只操作 MODBUS 寄存器，不处理 VR 映射等。 4 系列控制器 20170629 固件以上版本支持。																																																							
语法	ZMATH64(opmode, dindex1, dindex2) opmode: 操作编号 dindex1、dindex2: MODBUS 寄存器编号 <table><tr><th>操 作 编 号</th><th>执行操作</th><th>说明</th></tr><tr><td>1</td><td>64 位整数加法</td><td>D64(dindex1) += D64(dindex2)</td></tr><tr><td>2</td><td>64 位整数减法</td><td>D64(dindex1) -= D64(dindex2)</td></tr><tr><td>3</td><td>64 位整数乘法</td><td>D64(dindex1) *= D64(dindex2)</td></tr><tr><td>4</td><td>64 位整数除法</td><td>D64(dindex1) /= D64(dindex2)</td></tr><tr><td>5</td><td>64 位整数余</td><td>D64(dindex1) %= D64(dindex2)</td></tr><tr><td></td><td></td><td></td></tr><tr><td>11</td><td>64 位整数读取</td><td>D64(dindex1) = D32(dindex2)</td></tr><tr><td>12</td><td>64 位整数转换</td><td>D32(dindex1) = D64(dindex2)</td></tr><tr><td>13</td><td>64 位整数读取</td><td>D64(dindex1) = D32IEEE(dindex2)</td></tr><tr><td>14</td><td>64 位整数转换</td><td>D32IEEE(dindex1) = D64(dindex2)</td></tr><tr><td></td><td></td><td></td></tr><tr><td>15</td><td>赋值</td><td>D64(dindex1) = double64(dindex2)</td></tr><tr><td>16</td><td>赋值</td><td>double64(dindex1) = D64(dindex2)</td></tr><tr><td>17</td><td>赋值</td><td>double64 (dindex1) = D32IEEE (dindex2)</td></tr><tr><td>18</td><td>赋值</td><td>D32IEEE (dindex1) = double64 (dindex2)</td></tr><tr><td></td><td></td><td></td></tr><tr><td>21</td><td>double 加法</td><td>double64 (dindex1) += double64 (dindex2)</td></tr></table>		操 作 编 号	执行操作	说明	1	64 位整数加法	D64(dindex1) += D64(dindex2)	2	64 位整数减法	D64(dindex1) -= D64(dindex2)	3	64 位整数乘法	D64(dindex1) *= D64(dindex2)	4	64 位整数除法	D64(dindex1) /= D64(dindex2)	5	64 位整数余	D64(dindex1) %= D64(dindex2)				11	64 位整数读取	D64(dindex1) = D32(dindex2)	12	64 位整数转换	D32(dindex1) = D64(dindex2)	13	64 位整数读取	D64(dindex1) = D32IEEE(dindex2)	14	64 位整数转换	D32IEEE(dindex1) = D64(dindex2)				15	赋值	D64(dindex1) = double64(dindex2)	16	赋值	double64(dindex1) = D64(dindex2)	17	赋值	double64 (dindex1) = D32IEEE (dindex2)	18	赋值	D32IEEE (dindex1) = double64 (dindex2)				21	double 加法	double64 (dindex1) += double64 (dindex2)
操 作 编 号	执行操作	说明																																																						
1	64 位整数加法	D64(dindex1) += D64(dindex2)																																																						
2	64 位整数减法	D64(dindex1) -= D64(dindex2)																																																						
3	64 位整数乘法	D64(dindex1) *= D64(dindex2)																																																						
4	64 位整数除法	D64(dindex1) /= D64(dindex2)																																																						
5	64 位整数余	D64(dindex1) %= D64(dindex2)																																																						
11	64 位整数读取	D64(dindex1) = D32(dindex2)																																																						
12	64 位整数转换	D32(dindex1) = D64(dindex2)																																																						
13	64 位整数读取	D64(dindex1) = D32IEEE(dindex2)																																																						
14	64 位整数转换	D32IEEE(dindex1) = D64(dindex2)																																																						
15	赋值	D64(dindex1) = double64(dindex2)																																																						
16	赋值	double64(dindex1) = D64(dindex2)																																																						
17	赋值	double64 (dindex1) = D32IEEE (dindex2)																																																						
18	赋值	D32IEEE (dindex1) = double64 (dindex2)																																																						
21	double 加法	double64 (dindex1) += double64 (dindex2)																																																						

	22	double 减法	double64 (dindex1) -= double64 (dindex2)
	23	double 乘法	double64 (dindex1) *= double64 (dindex2)
	24	double 除法	double64 (dindex1) /= double64 (dindex2)
	25	double 余, 求小数	double64 (dindex1) %= double64 (dindex2)
	D32IEEE 表示浮点数存储, 同 MODBUS_IEEE。 D64 表示 64 位有符号整数存储, 可以通过两个 MODBUS_LONG 来读取高 32 和低 32 位。		
适用控制器	通用		
例子	MODBUS_LONG(0)=100 MODBUS_LONG(8)=20 ZMATH64 (1,8,0) '64 位整数加法计算, 两个数相加后存储在 MODBUS_LONG(8)起始地址 ?MODBUS_LONG(0) '打印结果, 100 ?MODBUS_LONG(8) '打印结果, 120		
相关指令	<u>MODBUS_IEEE</u> , <u>MODBUS_LONG</u> , <u>MODBUS_REG</u>		

MODBUS_DOUBLE -- 读取 MODBUS

类型	64 位指令
描述	从 MODBUS 读取 double 数据, 可以赋值给其它变量数组。 3 系列和 3 系列以下等数组为 float 的不支持这个指令。
语法	MODBUS_DOUBLE(index) Index: modbus 寄存器编号
适用控制器	通用
例子	MODBUS_LONG(0)=100 MODBUS_LONG(8)=200 ZMATH64 (16, 0, 8) '64 位赋值 ? MODBUS_DOUBLE (0) '打印结果, 200 ?MODBUS_LONG(0) '打印结果, 0 ?MODBUS_LONG(8) '打印结果, 200
相关指令	<u>ZMATH64</u>

第十一章 轴参数与轴状态指令

轴参数修改语法: SPEED = value,针对 BASE 轴。也可以通过 SPEED AXIS(axisnum) = value 来强制取特定轴, 其中 axis 可以省掉, 变为 SPEED(axisnum)=value。

可以通过 SPEED=value1,value2...来同时设置多个 BASE 轴的参数。

轴参数可写可读, 读取时 axis 也可以省掉: VAR1 = SPEED(axisnum)。

轴状态一般是只读的, 值会随时内部变动, MPOS,DPOS 等可以直接修改的除外。

 当进行插补运动时, 主轴的参数作为插补的参数, BASE 多个轴时, 第一个轴作为主轴。

11.1 轴选择

BASE -- 轴选择/轴组选择

类型	轴参数
描述	选择要设置参数、参与运动的轴。 缺省值依次为: 0, 1, 2... 程序中在下一条 BASE 指令被执行前, 都以上一条 BASE 指令选择轴。 每个任务拥有各自独立的轴列表, 会记住任务中 BASE 选择的轴或者轴组, 用于不同机台的控制。 当插补运动的时候, 第一个轴的运动参数作为插补参数。见例一。 如果在 BASE 指令中没有列出所有的轴, BASE 指令自动将剩余轴顺序排列在后面。见例二。
语法	BASE(axis<,second axis><,third axis>...) axis: 第一个轴 second axis: 下一个轴 ... 参数最多为控制器支持的轴数, 参照对应控制器硬件手册。
适用控制器	通用
例子	例一 BASE(0,1,2,3) SPEED=100,10,20,30 MOVE(100,100,100,100) '轴列表选择为: 0,1,2,3 '此时轴 0,1,2,3 设置了对应速度, 但是插补运动时, 只主轴轴 0 的速度 100 起作用 '轴 0,1,2,3 联合插补运动, 合成运动的速度为 100, 各轴速度为分速度 例二 BASE(1) '轴列表选择为: 1

	MOVE(100,100,100)	'轴 1,2,3 做插补运动
	例三	
	BASE(0,2,5)	'轴列表选择为: 0,2,5
	MOVE(100,100,100)	'轴 0,2,5 做插补运动

AXIS -- 临时轴选择

类型	辅助指令
描述	临时修改一个运动指令或轴参数到一个指定轴上去执行。 对轴参数，AXIS 可以省掉。
语法	AXIS(expression) expression: 临时修正的新轴号，执行完后，轴选择仍以 BASE 指令为准
适用控制器	通用
例子	例一 BASE(0) MOVE(1000) AXIS(1) '此时强制轴 1 运动 1000units MOVE(100) '轴 0 运动 100units 例二 BASE(1) UNITS AXIS(0)=100 '强制指定轴 0 UNITS 设为 100，可简写为 UNITS(0)=100 UNITS(2)=200 '强制指定轴 2 UNITS 设为 200 UNITS=10 '轴 1 UNITS 设为 10
相关指令	BASE

11.2 基本参数指令

UNITS -- 脉冲当量

类型	轴参数
描述	脉冲当量，指定每单位发送的脉冲数，支持 5 位小数精度。 脉冲当量一般设为电机走 1mm 或 1°的所需脉冲数。 控制器以 UNITS 作为基本单位，修改后坐标显示会随 UNITS 改变比例变化。 比如：UNITS=10 对应 DPOS=3000，MPOS=3000 修改 UNITS=100 后 对应 DPOS=300，MPOS=300
语法	可读：VAR1 = UNITS 或 VR1=UNITS(轴号) 可写：UNITS= expression 或 UNITS(轴号)= expression
适用控制器	通用
例子	如何设置

	假设电机 U=3600 脉冲转一圈，丝杠一圈螺距 P=2mm 电机转 1° 对应的 UNITS: UNITS=U/360=3600/360=10, 此时 MOVE(1), 电机转 1° 工作台走 1mm 对应的 UNTIS: UNITS=U/P=3600/2=1800, 此时 MOVE(1), 工作台走 1mm 机台存在减速比时，要把减速比算上，假设减速比 i=2:1 UNITS=U*i/P=3600*2/2=3600 按轴号依次设置: BASE(0,1,2) '选择轴 0, 轴 1, 轴 2 UNITS=10,100,1000 '轴 0 设为 10, 轴 1 设为 100, 轴 2 设为 1000 与 BASE 联合使用时，数据多出 BASE 选择的轴数，依次往后设置轴。 BASE(0,1,2) '选择轴 0, 轴 1, 轴 2 UNITS=10,100,1000,30 '轴 0 设为 10, 轴 1 设为 100, 轴 2 设为 1000, 轴 3 设为 30 数据少于 BASE 选择的轴数，就只设置对应的轴: BASE(0,1,2) '选择轴 0, 轴 1, 轴 2 UNITS=10,100 '轴 0 设为 10, 轴 1 设为 100, 轴 2 不设置 设定指定轴: UNTIS(2)=100 '直接设置轴 2, 与 BASE 选择轴无关
--	--

ATYPE -- 轴类型

类型	轴参数												
描述	轴功能类型设置，只能设置为轴具备的特性（轴特性可查询硬件手册或使用 RTSys 软件连接上控制器以后查看控制器状态）。 最好是在程序初始化的时候就设置好 ATYPE。 ZCAN 扩展轴要先设置 AXIS_ADDRESS，并在设置后延迟 2 个 tick 再调用运动指令，受总线带宽限制，ZCAN 扩展轴不要设置超过 2 个。 对部分产品型号带有独立的编码器，可以使用相应虚拟轴来做编码器轴使用，例如 ZMC206 的电机轴为 0-5 轴，编码器可以通过轴 6-11 来控制，详细可通过 RTSys 软件连接上控制器以后查看控制器状态。												
语法	可读：VAR1 = ATYPE 或 VR1=ATYPE(轴号) 可写：ATYPE= expression 或 ATYPE(轴号)= expression <table><tr><th>ATYPE 类型</th><th>描述</th></tr><tr><td>0</td><td>虚拟轴</td></tr><tr><td>1</td><td>脉冲方向方式的步进或伺服</td></tr><tr><td>2</td><td>模拟信号控制方式的伺服</td></tr><tr><td>3</td><td>正交编码器</td></tr><tr><td>4</td><td>脉冲方向输出+正交编码器输入</td></tr></table>	ATYPE 类型	描述	0	虚拟轴	1	脉冲方向方式的步进或伺服	2	模拟信号控制方式的伺服	3	正交编码器	4	脉冲方向输出+正交编码器输入
ATYPE 类型	描述												
0	虚拟轴												
1	脉冲方向方式的步进或伺服												
2	模拟信号控制方式的伺服												
3	正交编码器												
4	脉冲方向输出+正交编码器输入												

	5	脉冲方向输出+脉冲方向编码器输入
	6	脉冲方向方式的编码器
	7	脉冲方向方式步进或伺服+EZ 信号输入
	8	ZCAN 扩展脉冲方向方式步进或伺服
	9	ZCAN 扩展正交编码器
	10	ZCAN 扩展脉冲方向方式的编码器
	20	振镜类型，带振镜状态反馈 振镜连接不上 AXISSTATUS 的 bit2 会置位，ENCODER 返回原始的发送位置，脉冲单位 ZMC408SCAN 支持
	21	振镜轴类型，需要控制器支持 缺省系统周期 250us，振镜刷新周期 50us，与固件有关 可以使用普通轴的所有运动控制指令，支持振镜轴与其它轴类型混合插补
	22	振镜轴类型，带振镜位置反馈 振镜连接不上 AXISSTATUS 的 bit2 会置位，振镜报警 AXISSTATUS 的 bit3 会置位 MPOS 返回反馈位置，做了反矫正处理，ENCODER 返回原始的反馈位置脉冲单位 ZMC408SCAN 支持
	24	远程编码器轴类型 ZHD500X 上手轮使用，部分控制器支持
	25	定义一个编码器轴，坐标位置从 MODBUS 或 NODE_PDOBUF 读取 Version_build 230810 后版本支持 BASE(axisnum) AXIS_ADDRESS = (slot <<16) + nodenum ENCODER_ID=(index<<16)+(subindex<<8)+ bites ATYPE=25 Slot: -1: 从 MODBUS_LONG(nodenum) 里面取编码器位置 0-n: 从 NODE_PDOBUF(slot, nodenum, index, subindex ,type) 取编码器位置 bites: 数据类型的位数，比如数据 32 位，则写 32 ENCODER_ID: 存储数据字典的编号
	26	自定义编码器: 使用 C 语言来更新编码器的位置. 支持闭环 version_build 240702 增加.
	48	SSI 绝对式编码器
	49	BISS 绝对式编码器
	50	RTEX 周期位置模式，需 Rtex 控制器
	51	RTEX 周期速度模式，需 Rtex 控制器
	52	RTEX 周期力矩模式，需 Rtex 控制器 请先关闭驱动器 2 自由度控制模式，并设置速度限制

	65	EtherCAT 周期位置模式，需支持 EtherCAT
	66	EtherCAT 周期速度模式，需支持 EtherCAT Profile 要设置为 20 或以上
	67	EtherCAT 周期力矩模式，需支持 EtherCAT PROFILE 要设置为 30 或以上
	70	EtherCAT 自定义操作，只读取编码器，需支持 EtherCAT
电机模式 INVERT_STEP 指令设置，默认脉冲方向		
适用控制器	通用	
例子	例一 脉冲类型 BASE(0,1) ATYPE = 1,1 '轴 0,1 设为脉冲控制类型 UNITS=100,100 '脉冲当量设为 100 SPEED=100,100 '速度 100 units/s ACCEL=1000 ,1000 '加速度 1000 units/s/s DECEL=1000 '减速度 1000 units/s/s MOVE(100,100) '直线插补 例二 EtherCAT 总线控制 SLOT_SCAN(0) '总线扫描 BASE(0) AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=65 '轴类型 65，位置控制 SLOT_START(0) '开启总线 AXIS_ENABLE=1 '单轴使能 WDOG=1 '所有轴使能 UNITS=100 '脉冲当量设为 100 SPEED=100 '速度 100 units/s ACCEL=1000 '加速度 1000 units/s/s DECEL=1000 '减速度 1000 units/s/s MOVE(5000) 例三 Rtex 力矩模式 SLOT_SCAN(0) '总线扫描 BASE(0) AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=52 '轴类型 52，Rtex 力矩控制 DRIVE_WRITE(6*256+47,0) '关闭 2 自由度控制 DRIVE_WRITE(3*256+17,0) '选择参数 3.21 作为速度限制 DRIVE_WRITE(3*256+21,2000) '最大速度限制为 2000r/min SLOT_START(0) '开启总线 AXIS_ENABLE=1 '单轴使能 WDOG=1 '所有轴使能 DAC=100 '此时 DAC 发送值控制，具体查看 DAC 指令	

	例四 振镜轴 BASE(4,5) UNTIS=1,1 ATYPE=21,21 '设置为振镜轴 例五 远程编码器轴 BASE(axisnum) AXIS_ADDRESS = lcd 编号 ATYPE=24 例六：定义一个编码器轴 BASE(0) AXIS_ADDRESS = (0<<16) + 0 ENCODER_ID= \$60640020 ATYPE=25 例七：自定义编码器 增加的 C 函数接口： // 读取 DAC_OUT, 实际的轴 DAC 输出值 double motionrt_getaxisdacout(uint32 iaxis); // 虚拟自定义编码器更新 encoder uint32 motionrt_updatecoder(uint32 iaxis, int32 icoder); // 虚拟自定义编码器更新 encoderdot, float -1 到 1 之间 // 一般不用调用 uint32 motionrt_updatecoderdot(uint32 iaxis, float fdot); BASE(0) ENCODER_SERVO= 2 '配置闭环,不使用 AOUT 输出,motionrt_getaxisdacout 读取输出. ATYPE=26 ‘闭环处理参见:<<C 语言支持>> datum(0) FE_LIMIT = 10000 FE_RANGE =10000 axis_enable=1 servo=1
相关指令	AXIS_ADDRESS , INVERT_STEP

AXIS_ADDRESS -- 轴地址设置

类型	轴参数
描述	扩展轴时的轴地址配置。 1.ZCAN 扩展轴时，扩展板上带 8 位拨码开关（V1.3 以上硬件版本）。 受总线带宽限制，ZCAN 扩展轴不要设置超过 2 个。 必须先设置 AXIS_ADDRESS，再设置 ZCAN 扩展轴 ATYPE 类型，修改后必须重新设

置 **ATYPE**。见例一。

位 1-4	CAN 地址拨码，组合值 0-15
位 5-6	CAN 速度拨码，不同组合值速度不同
位 7	特殊功能预留
位 8	120 欧姆电阻拨码，拨 ON 时电阻接通

规则： $\text{AXIS_ADDRESS}(\text{轴号}) = (32 * 0) + \text{ID}$ '扩展板的本地轴接口 0

$\text{AXIS_ADDRESS}(\text{轴号}) = (32 * 1) + \text{ID}$ '扩展板的本地轴接口 1

2.总线驱动器映射轴号，将连接的驱动器按编号一一映射。

驱动器编号根据接线顺序排列，编号从 0 开始到 EtherCAT 驱动器个数减 1。

驱动器编号与设备号不同，设备号包括 ECAT 接口所有连接的设备，驱动器编号只算连接的驱动器。

必须先设置 **AXIS_ADDRESS**，再设置 ECAT 轴 **ATYPE** 类型，修改后必须重新设置 **ATYPE**。见例二。

位 0-15	驱动器编号加 1，0-自动指定
位 16-31	SLOT 编号（多槽位时） 多槽位的时候每个槽位地址号相差 65535，例如槽位 0 地址是 0-65535，槽位 1 地址是 65536-131071

规则： $\text{AXIS_ADDRESS}(\text{轴号}) = (\text{槽位号} \ll 16) + \text{驱动器编号} + 1$

3.本地脉冲轴号重映射，4 系列控制器支持本地脉冲或编码器轴号重新映射，固件 160608 以上版本支持。

重新映射时注意先把原本地脉冲轴设置为虚拟轴。修改后必须重新设置 **ATYPE**。见例四。

位 0-15	映射的本地脉冲轴号
位 16-31	高 16 位全设为 1（等同于十进制下高 16 位 = -1）

规则：**BASE**(重映射的轴号)

ATYPE=0，设置轴类型为 0，低版本不设置会报错

BASE(要修改的本地脉冲轴号)

ATYPE=0，设置轴类型为 0

$\text{AXIS_ADDRESS}(\text{重映射的轴号}) = (-1 \ll 16) + \text{要修改的本地脉冲轴号}$

BASE(重映射的轴号)

ATYPE=1/7

4.MotionRT 控制卡子卡上的脉冲轴、编码器轴映射。

映射时必须先设置 **AXIS_ADDRESS**，再设置 **ATYPE** 类型，修改后必须重新设置 **ATYPE**。

位 0-15	子卡上相对轴号加 1
位 16-31	子卡 CARD 编号

规则：

BASE(重映射的轴号)

ATYPE=0，设置轴类型为 0，低版本不设置会报错

BASE(要修改的本地脉冲轴号)

	ATYPE=0, 设置轴类型为 0 AXIS_ADDRESS(重映射的轴号)=(子卡号<<16)+子卡上相对物理轴号+1 BASE(重映射的轴号) ATYPE=X (重新设置所需的轴类型) 5.取消轴映射: AXIS_ADDRESS=0。 BASE(重映射的轴号) ATYPE(重映射的轴号)=0 AXIS_ADDRESS=0
语法	VAR1 = AXIS_ADDRESS, AXIS_ADDRESS = expression
适用控制器	通用
例子	例一 ZCAN 扩展轴 AXIS_ADDRESS (6)=2+(32*1) '轴 6 映射到 ZCAN 扩展模块 ID2 的轴 1 ATYPE(6)=8 'ZCAN 扩展轴类型 脉冲方向方式步进或伺服 UNITS(6)=100 '脉冲当量 100 SPEED(6)=100 '速度 100units/s ACCEL(6)=1000 '加速度 1000units/s/s MOVE(100) AXIS(6) '扩展轴运动 100units 例二 EtherCAT 手动映射轴号 AXIS_ADDRESS (0)=0+1 '第一个 Ecat 驱动器, 编号 0, 绑定为轴 0 AXIS_ADDRESS (2)=1+1 '第二个 Ecat 驱动器, 编号 1, 绑定为轴 2 AXIS_ADDRESS (1)=2+1 '第三个 Ecat 驱动器, 编号 2, 绑定为轴 1 ATYPE(0)=65 '设置为 Ecat 类型 ATYPE(1)=65 ATYPE(2)=65 例三 EtherCAT 自动映射轴号 AXIS_ADDRESS (0)=0 '自动指定 slot0 的驱动器, 按照接线顺序, 从轴 0 开始依次映射轴号(不建议使用, 建议例二) ATYPE(0)=65 '轴 0 配置为 ECAT 模式 例四 EtherCAT 控制器修改脉冲轴号 '修改前, 操作轴 0, 对应控制器上的轴 0 接口 BASE(16) '重映射的轴号 ATYPE(16)=0 BASE(0) '要修改的本地脉冲轴号 ATYPE(0)=0 '先将本地脉冲轴 0 设为虚拟轴 AXIS_ADDRESS (16)= (-1<<16)+0 '绑定本地脉冲轴 0, 高 16 位=-1 ATYPE(16)=1 '轴 16 配置为脉冲轴, 使用本地脉冲轴 0, 此时, 操作轴 16, 对应控制器轴 0 接口 例五 振镜轴号重映射 ATYPE(4)=0 ATYPE(5)=0

	BASE(X) '希望映射的轴号 AXIS_ADDRESS = (-1<<16)+4 '重映射第一个振镜轴 ATYPE = 21
	BASE(Y) '希望映射的轴号 AXIS_ADDRESS = (-1<<16)+5 '重映射第二个振镜轴 ATYPE = 21
	例六 MotionRT 控制卡子卡上的轴重映射 '将轴 0 重映射为轴 16 BASE(16) '重映射的轴号 ATYPE=0 BASE(0) '要修改的原轴号 ATYPE=0 '设置轴类型为 0 AXIS_ADDRESS(16)=(0<<16)+ 0 + 1 BASE(16) ATYPE = 1 '轴 16 配置为脉冲轴
	相关指令 ATYPE

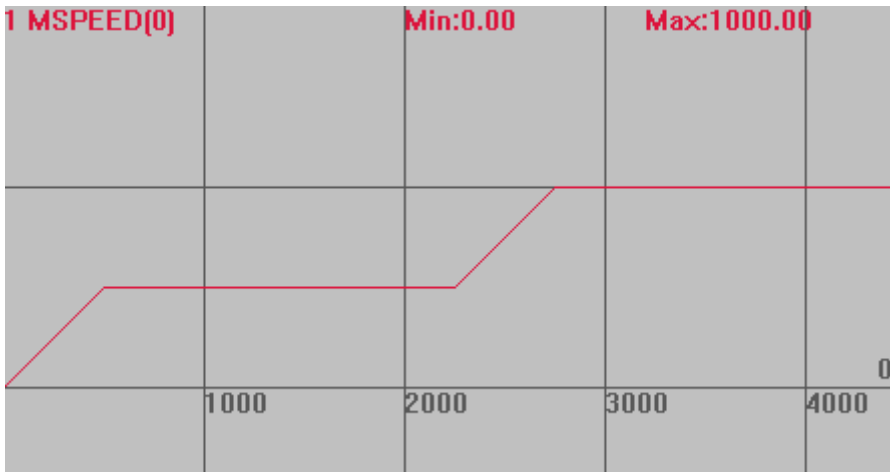
AXIS_ENABLE -- 单轴使能

类型	轴参数
描述	各轴使能设置。 EtherCAT 总线轴需设置，还必须设置 WDOG=1 总使能。
语法	AXIS_ENABLE = 1/0 1 使能，0 关闭
适用控制器	通用
例子	AXIS_ENABLE(0) = 1 '打开轴 0 使能
相关指令	WDOG

11.3 速度参数指令

SPEED -- 运动速度

类型	轴参数
描述	轴速度，单位为 units/s。 若脉冲当量设为 1mm 脉冲数，SPEED 速度设 100，则每秒移动 100mm。 当多轴运动时，作为插补运动的速度。 SPEED 修改后，立刻生效，可以实现动态变速，但是改变瞬间会抖。希望平滑变速请使用 SPEED_RATIO 指令。 当 SP 指令的 FORCE_SPEED 大于 SPEED 时，SPEED 速度也会生效(140716 以后版本 SPEED 不起作用)。

语法	可读: VAR1 = SPEED 或 VAR1 = SPEED(轴号) 可写: SPEED = expression 或 SPEED(轴号) = expression
适用控制器	通用
例子	BASE(0) UNITS=100 '脉冲当量 SPEED =500 '设置 0 轴速度 500units/s ACCEL=1000 '加速度 1000units/s/s DPOS=0 '坐标清 0 TRIGGER '自动触发示波器 VMOVE(1) '持续运动 WAIT UNTIL DPOS(0)>1000 '等待轴 0 运行到 1000 SPEED=1000 '速度改为 1000 速度曲线 MSPEED(0)垂直刻度 1000 
相关指令	FORCE_SPEED , SPEED_RATIO

ACCEL -- 加速度

类型	轴参数
描述	轴加速度，单位为 units/s/s。 SPEED 速度设 100，加速度设 1000，加速到 100 时间为 100/1000 秒。 当多轴运动时，插补运动的加速度为主轴加速度。 建议运动前设置好加速度和减速度，运动中不要修改，运动中调整会导致速度曲线变化。
语法	可读: VAR1 = ACCEL 或 VAR1 = ACCEL(轴号) 可写: ACCEL = expression 或 ACCEL(轴号) = expression
适用控制器	通用
例子	例一 BASE(1,2,3,4) 'BASE 选择轴 ACCEL=100, 100, 100, 100 '设置轴 1, 2, 3, 4 的加速度

ACCEL(2)=200 '设置轴 2 的加速度

例二

BASE(0)

UNITS=100 '脉冲当量

DPOS=0 '坐标清 0

ACCEL=2000

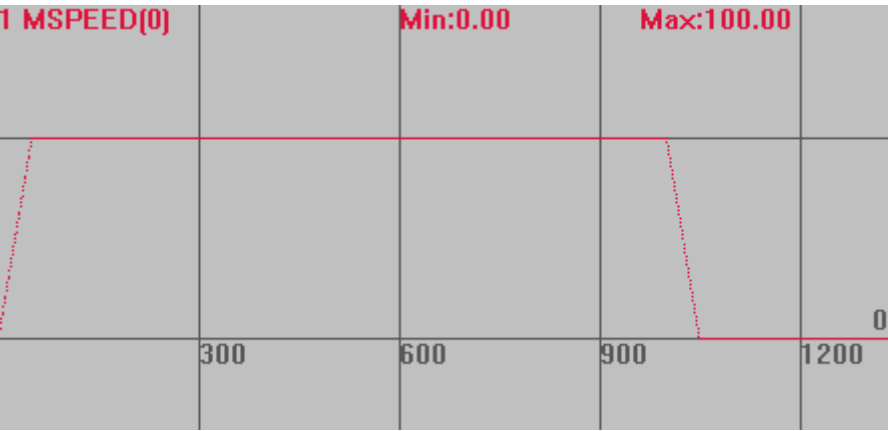
DECEL=2000

SPEED=100

MOVE(100)

加速过程的速度曲线

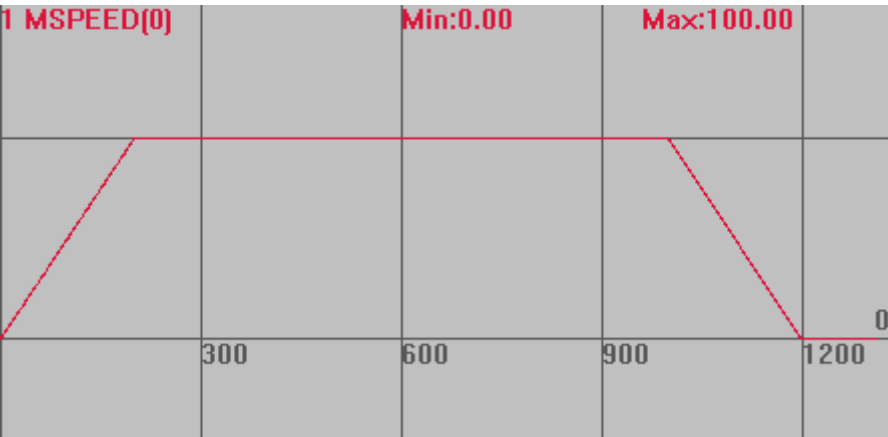
MSPEED(0)垂直刻度 100



变更加减速之后的速度曲线

ACCEL=500

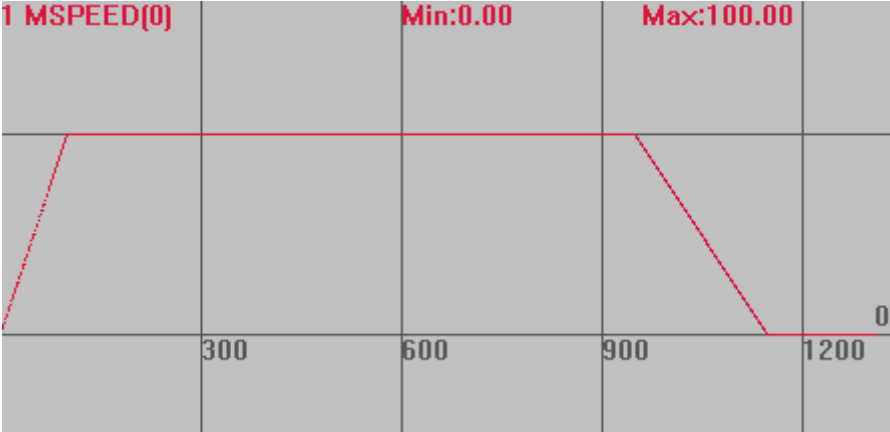
DECEL=500

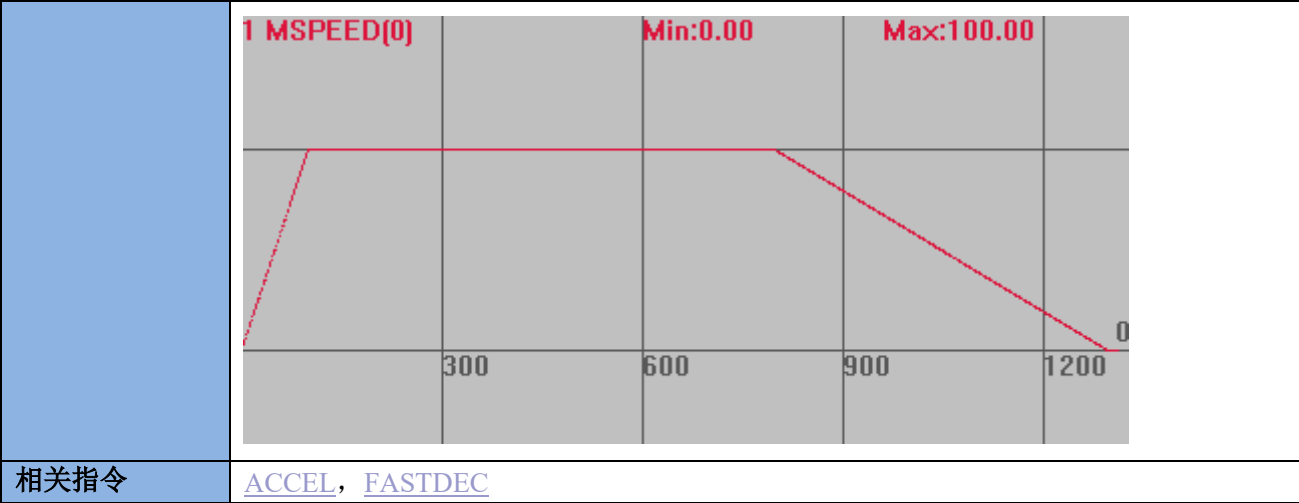


相关指令

[DECEL](#), [SPEED](#)

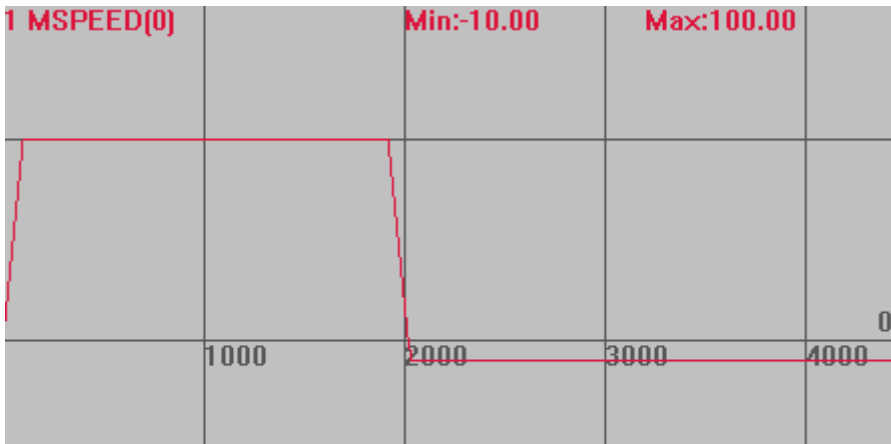
DECEL -- 减速度

类型	轴参数
描述	轴减速度，单位为 units/s/s。 当多轴运动时，作为插补运动的减速度。 当设置为 0 时，自动等于加速度 ACCEL 值，进行对称的加减速。 建议运动前设置好加速度和减速度，运动中不要修改，运动中调整会导致速度曲线变化。
语法	可读：VAR1 = DECEL 或 VAR1 = DECEL(轴号) 可写：DECEL = expression 或 DECEL(轴号) = expression
适用控制器	通用
例子	例一 BASE(1,2,3,4) DECEL =100, 100, 100, 100 '设置轴 1，2，3，4 的减速度 DECEL (0)=200 '设置轴 0 的减速度 PRINT DECEL (0) 例二 BASE(0) '选择轴 0 UNITS=100 '脉冲当量 DPOS=0 '坐标清 0 SPEED=100 '设置速度为 100 units/s ACCEL=1000 DECEL=500 '减速度为 500units/s/s TRIGGER '自动触发示波器 MOVE(200) '移动 200units 速度曲线 MSPEED(0)垂直刻度 100  The graph displays the velocity profile for MSPEED(0). The vertical axis represents velocity in units/s, with a scale factor of 100. The horizontal axis represents time in units. The profile starts at 0, accelerates linearly to a maximum velocity of 100 units/s at time 300. It then maintains this constant velocity until time 900. Finally, it decelerates linearly back to 0 units/s at time 1200. The deceleration phase is labeled with a red '0' at the end of the line. The graph is titled '1 MSPEED(0)' and includes a legend with 'Min:0.00' and 'Max:100.00'. DECEL=200 时的速度曲线



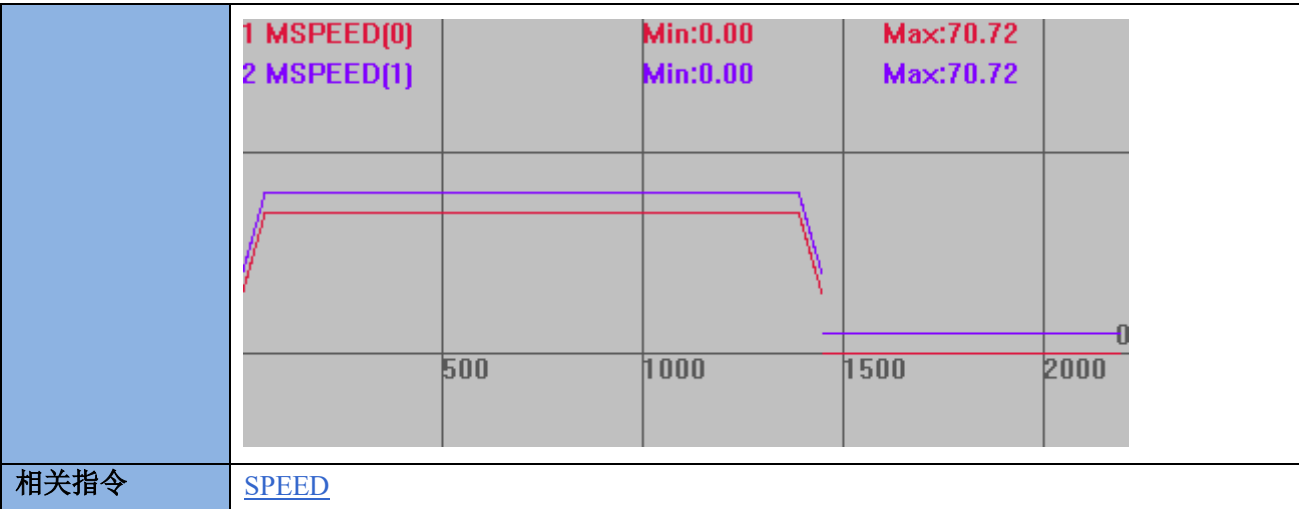
CREEP -- 爬行速度

类型	轴参数
描述	轴回零时爬行速度，用于原点搜寻，单位为 units/s。
语法	可读：VAR1 = CREEP 或 VAR1 = CREEP(轴号) 可写：CREEP = expression 或 CREEP(轴号) = expression
适用控制器	通用
例子	BASE(0) DPOS=0 UNITS=100 ACCEL=1000 DECEL=1000 SPEED = 100 '运行速度设置为 100units/s CREEP = 10 '爬行速度设置为 10units/s DATUM_IN=0 '设置 IN0 为轴零原点 INVERT_IN(0,ON) '反转电平 TRIGGER '自动触发示波器 DATUM(3) '先按速度 100 反找回零，遇原点后按速度 10 反向离开原点 速度曲线 MSPEED(0)垂直刻度 100

	
相关指令	DATUM

LSPEED -- 起始速度

类型	轴参数
描述	轴起始速度，同时用于停止速度，缺省 0，单位为 units/s。 当多轴运动时，作为插补运动的起始速度。 当需要追求效率时，可以考虑设置起始速度。 建议在大多数应用中，LSPEED 的值保持为 0，否则可能带来机器严重抖动。
语法	可读：VAR1 = LSPEED 或 VAR1 = LSPEED(轴号) 可写：LSPEED = expression 或 LSPEED(轴号) = expression
适用控制器	通用
例子	例一 BASE(0,1) '选择轴 0 为主轴 DPOS=0,0 UNITS=100,100 '脉冲当量 100 SPEED=100,100 '主轴速度 100units/s ACCEL=1000,1000 DECEL=1000,1000 LSPEED=40 '起始速度 40units/s TRIGGER '自动触发示波器 MOVE(100,100) '各轴运动距离 速度曲线 MSPEED(0)垂直刻度 100，无偏移 MSPEED(1)垂直刻度 100，偏移 10



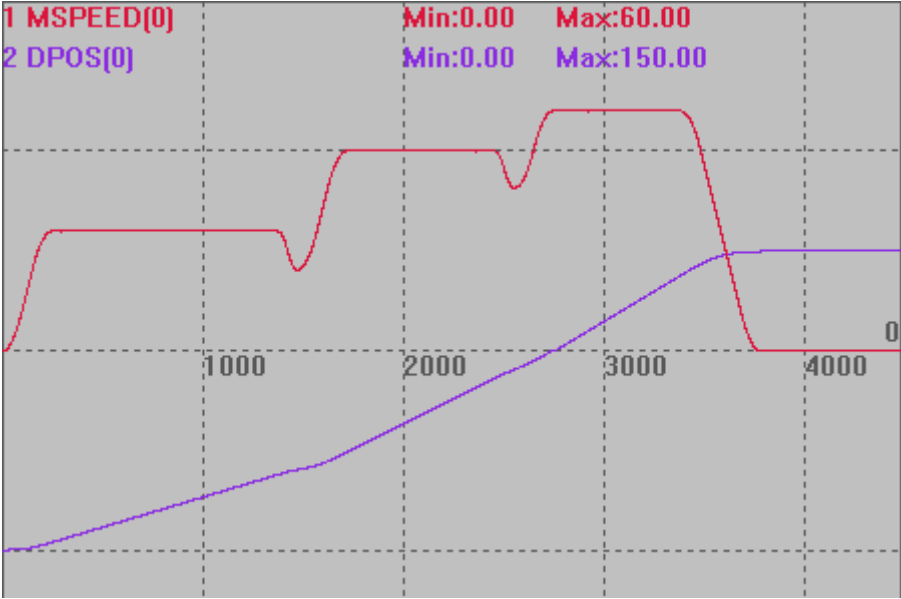
FORCE_SPEED -- SP 速度

类型	轴参数
描述	自定义速度的 SP 运动的强制速度，单位为 units/s 这个参数被带入运动缓冲。 当 FORCE_SPEED 大于 SPEED 时，SPEED 同时也会生效限制住最高速度(140716 以后版本 SPEED 不起作用)。 如果要求进入本段的时候 FORCE_SPEED 已经降为对应的速度，请设置 STARTMOVE_SPEED。
语法	可读：VAR1 = FORCE_SPEED 或 VAR1 = FORCE_SPEED(轴号) 可写：FORCE_SPEED = expression 或 FORCE_SPEED(轴号) = expression
适用控制器	通用
例子	BASE(0) DPOS=0 ATYPE=1 UNITS=100 '脉冲当量 100 ACCEL=500 DECEL=500 SPEED = 100 '速度 100units/s FORCE_SPEED=150 '自定义速度 150units/s SRAMP=100 'S 曲线 MERGE=ON TRIGGER '自动触发示波器 MOVE(100) '不带 SP 使用 SPEED 速度 MOVESP(100) '速度为 150 FORCE_SPEED=200 MOVESP(100) '速度为 200 FORCE_SPEED=50 MOVESP(100) '速度为 50

	END 速度曲线： MSPEED(0)垂直刻度 100 
相关指令	*SP_SPEED -- 运动速度

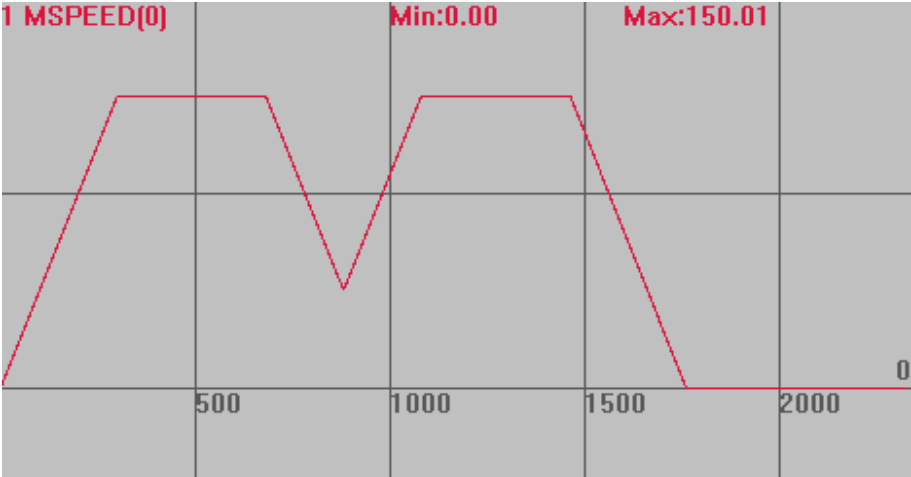
STARTMOVE_SPEED -- SP 运动开始速度

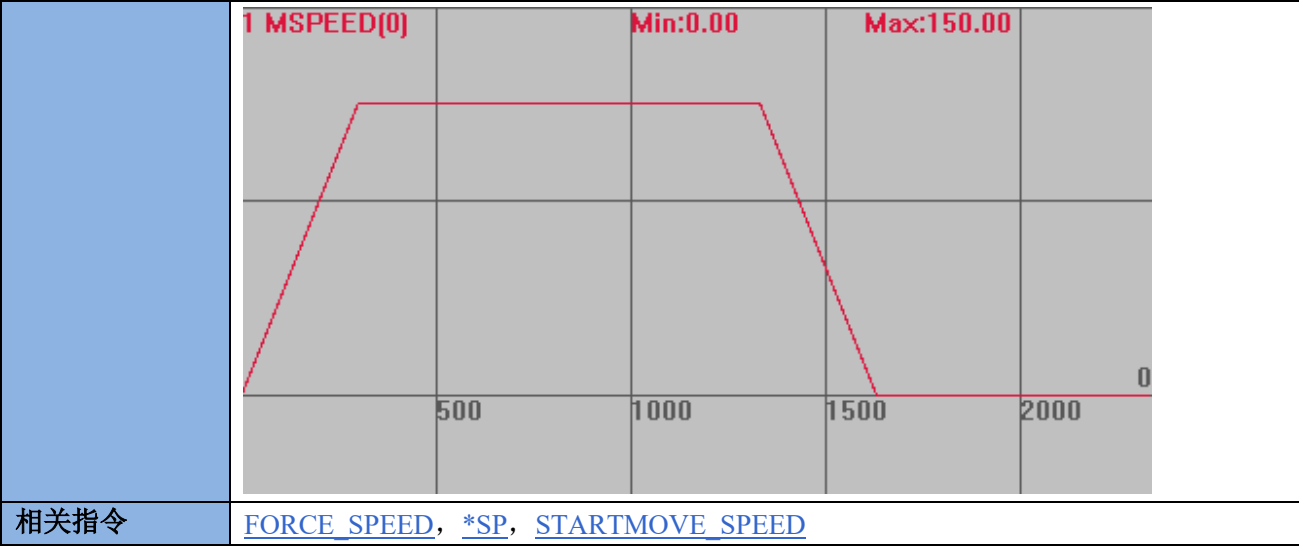
类型	轴参数
描述	自定义速度的 SP 运动的开始速度，这个参数被带入运动缓冲。 只有使用了带 SP 的运动指令才生效。 不使用时请设置较大值。控制器默认值 1000。
语法	可读：VAR1 = STARTMOVE_SPEED 或 VAR1 = STARTMOVE_SPEED(轴号) 可写：STARTMOVE_SPEED = expression 或 STARTMOVE_SPEED(轴号) = expression
适用控制器	通用
例子	RAPIDSTOP(2) WAIT IDLE(0) BASE(0) '选择 XY 轴 DPOS = 0 MPOS = 0 ATYPE=1 '脉冲方式步进或伺服 UNITS = 100 '脉冲当量 SPEED = 100 ACCEL = 200 DECEL = 200 SRAMP=100 'S 曲线 MERGE = ON '启动连续插补 TRIGGER '第一段

	FORCE_SPEED = 30 STARTMOVE_SPEED = 1000 ENDMOVE_SPEED = 1000 MOVESP(40) '第一段速度 30' '不设置，默认值 1000' '不设置，默认值 1000' '第二段' FORCE_SPEED = 50 STARTMOVE_SPEED = 20 ENDMOVE_SPEED = 40 MOVESP(50) '第二段速度 50' '第二段的起始速度 20' '结束速度 40' '第三段' FORCE_SPEED = 60 STARTMOVE_SPEED = 1000 ENDMOVE_SPEED = 1000 MOVESP(60) END '第三段速度 60' '不设置，默认值 1000' '不设置，默认值 1000' 速度和位置曲线 MSPEED(0)垂直刻度 50 DPOS(0)垂直刻度 100，偏移-100 
--	--

ENDMOVE_SPEED -- SP 运动结束速度

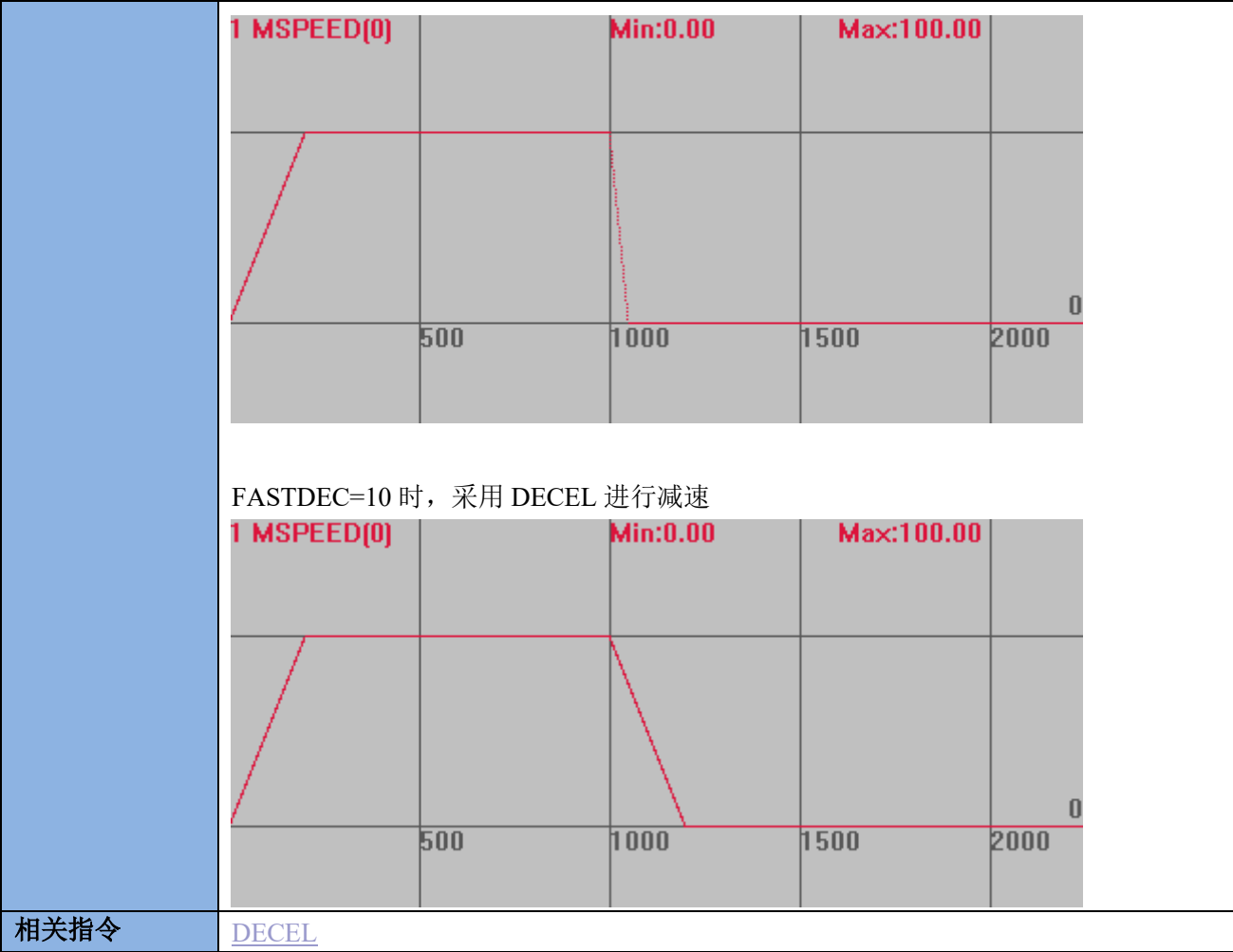
类型	轴参数
描述	自定义速度的 SP 运动的结束速度，这个参数被带入运动缓冲。 只有使用了带 SP 的运动指令才生效。

	不使用时请设置较大值。控制器默认值 1000。
语法	可读：VAR1 = ENDMOVE_SPEED 或 VAR1 = ENDMOVE_SPEED(轴号) 可写：ENDMOVE_SPEED = expression 或 ENDMOVE_SPEED(轴号) = expression
适用控制器	通用
例子	BASE(0) DPOS=0 UNITS=100 MERGE=1 '开启连续插补 SPEED=100 ACCEL=500 DECEL=500 FORCE_SPEED=150 '限制速度为 150units/s ENDMOVE_SPEED=50 '强制结束速度为 50units/s TRIGGER '自动触发示波器 MOVESP(100) MOVESP(100) 速度曲线 MSPEED(0)垂直刻度 100  不使用时 ENDMOVE_SPEED 限制速度时



FASTDEC -- 快减减速度

类型	轴参数	
描述	快减减速度，单位为 units/s/s。 在 CANCEL，RAPIDSTOP，达到限位或异常停止时自动采用。 当设置值为 0 或小于 DECEL 值时自动为 DECEL 值。	
语法	可读：VAR1 = FASTDEC 或 VAR1 = FASTDEC(轴号) 可写：FASTDEC = expression 或 FASTDEC(轴号) = expression	
适用控制器	通用	
例子	BASE(0) '选择轴 0 DPOS=0 UNITS=100 SPEED=100 '速度设为 100 ACCEL=500 DECEL=500 '减速度 500 FASTDEC=2000 '快减减速度 2000 TRIGGER '自动触发示波器 VMOVE(1) '持续正转 DELAY (1000) '等待 1s CANCEL(2) '停止 减速曲线 MSPEED(0)垂直刻度 100	



MSPEED -- 实际反馈速度

类型	轴状态
描述	轴的测量反馈位置速度，单位是 units/s。
语法	VAR1 = MSPEED 或 VAR1 = MSPEED(轴号)
适用控制器	通用
相关指令	UNITS ， VP_SPEED

SPEED_RATIO -- 速度比例

类型	轴参数
描述	轴速度比例 轴实际运动速度为 SPEED*SPEED_RATIO。 用于运动中根据加减速的平滑速度改变。
语法	SPEED_RATIO(轴号) = value value: 比例

	不指定轴号时，默认使用 BASE 指令指定的轴号，插补运动可作用于所有轴，否则只作用于 BASE 的第一个轴。 在线命令不带轴号时，默认对轴 0 起作用。
适用控制器	最新固件支持
例子	<pre> RAPIDSTOP(2) WAIT IDLE SPEED_RATIO = 1 TRIGGER BASE(0) '选择轴 0 DPOS=0 UNITS=100 SPEED = 100 ACCEL = 1000 DECEL = 1000 MERGE = ON SRAMP = 50 MOVE(100) DELAY(500) '等待 0.5s SPEED_RATIO = 0.5 '速度降为 50 WAIT UNTIL VP_SPEED < 80 DELAY(100) '等待 0.1s SPEED_RATIO = 0.3 '速度降为 30 END </pre> 速度曲线 VP_SPEED(0)垂直刻度 100
相关指令	FORCE_SPEED ， SPEED

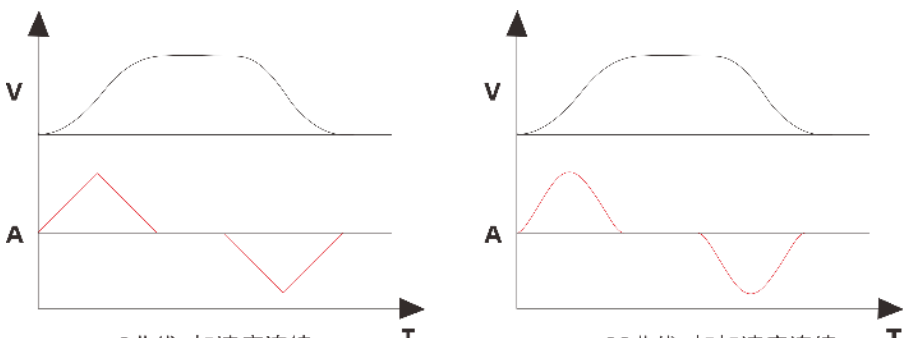
SRAMP -- 加減速曲线

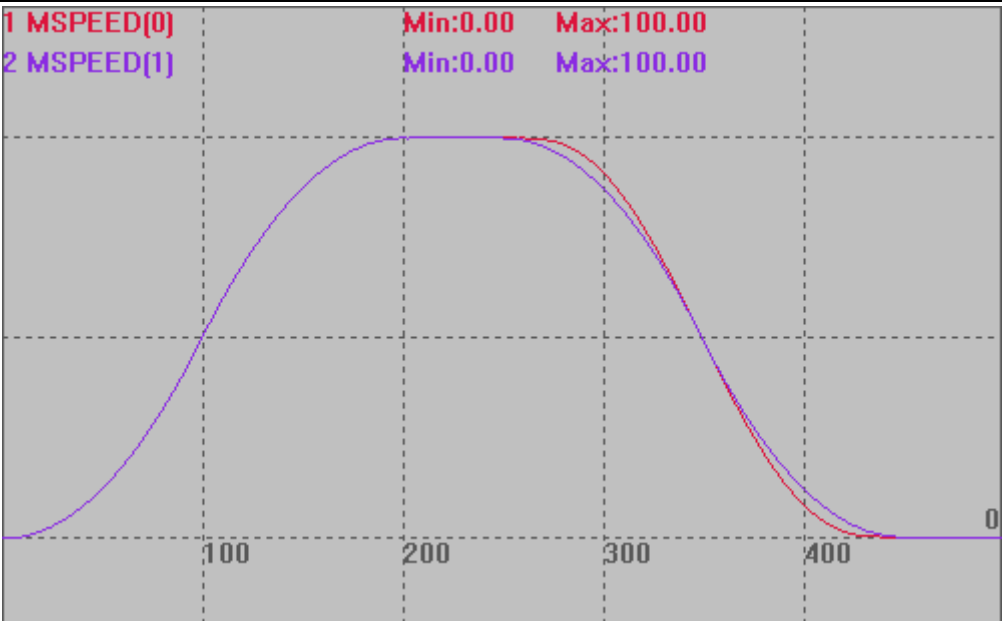
类型	轴参数
描述	加减速过程 S 曲线设置。

语法	VAR1 = SRAMP, SRAMP = smoothms smoothms: 0-250 毫秒，设置后加减速过程会延长相应的时间	
适用控制器	通用	
例子	BASE(0) '选择轴 0 DPOS=0 UNITS=100 '脉冲当量 100 SPEED=100 '速度 1000units/s ACCEL=1000 '加速度 1000units/s/s DECEL=1000 '加速度 1000units/s/s SRAMP=100 'S 曲线时间 100ms TRIGGER '自动触发示波器 MOVE(100) '运动 100units 速度曲线 MSPEED(0)垂直刻度 100  SRAMP=0 时 	
相关指令	ACCEL ， DECEL	

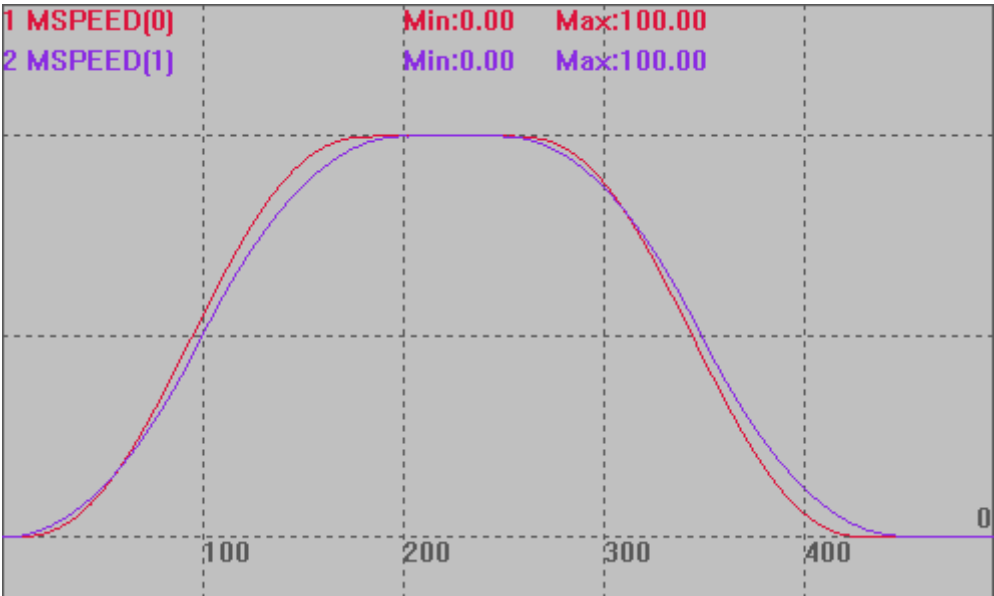
VP_MODE -- 加减速曲线

类型	轴参数
描述	加减速曲线类型选择。

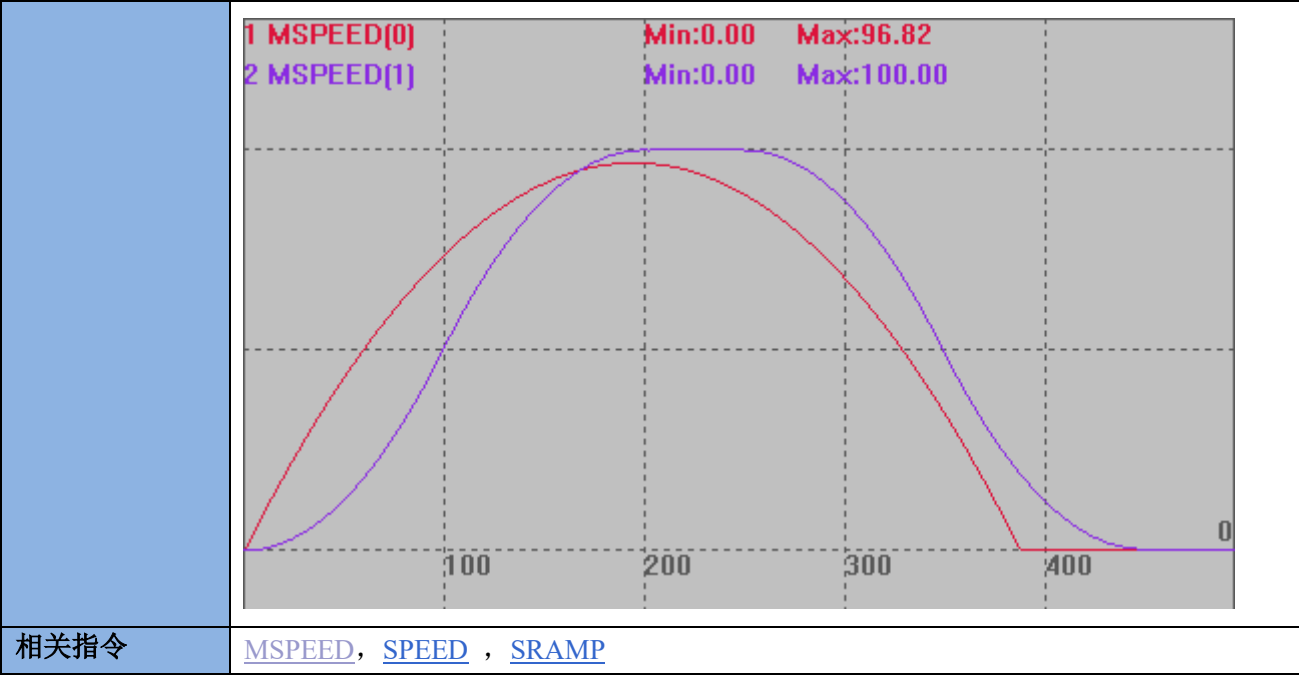
	0 - 缺省值, 使用 <code>sramp</code> 来设置 S 曲线; 4 - 起步时最大加速度, 达到最高速时加速度渐变为 0; 6 - 新增类型 SS 曲线, 加加速度连续的曲线类型, SS 模式比 T 形减速会增加 87%的减速时间。此模式加速时使用 0 模式, 减速时才生效, 方便连续小线段插补; 7 - 新增类型 SS 曲线, 加加速度连续的曲线类型。动态修改轴参数或连续插补可能导致加加速度无法连续, 此时会切换到 0 模式, 因此建议 <code>SRAMP</code> 也设置合适值。 <code>VP_MODE</code> 与 <code>SRAMP</code> 都能平滑速度参数, 区别参见下图: 
语法	<code>VAR1 = VP_MODE</code> 或 <code>VP_MODE(axis)=mode</code> mode: 模式选择
适用控制器	通用
例子	例一 模式 6 <pre>BASE(0,1) '选择轴 0, 1 ATYPE=1,1 UNITS=100,100 DPOS=0,0 MPOS=0,0 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 SRAMP=100,100 VP_MODE=6,0 '轴 0 模式 6, 轴 1 模式 0 TRIGGER MOVE(25) AXIS(0) MOVE(25) AXIS(1)</pre> 速度曲线: 模式 6 对加速度阶段不做处理, 只针对减速度阶段。 <code>MSPEED(0)</code>垂直刻度 50 <code>MSPEED(1)</code>垂直刻度 50



例二 模式 7
VP_MODE=7,0 '轴 0 模式 7，轴 1 模式 0
其他条件按例一，速度曲线如下，模式 7 对加减速度阶段均做了处理。

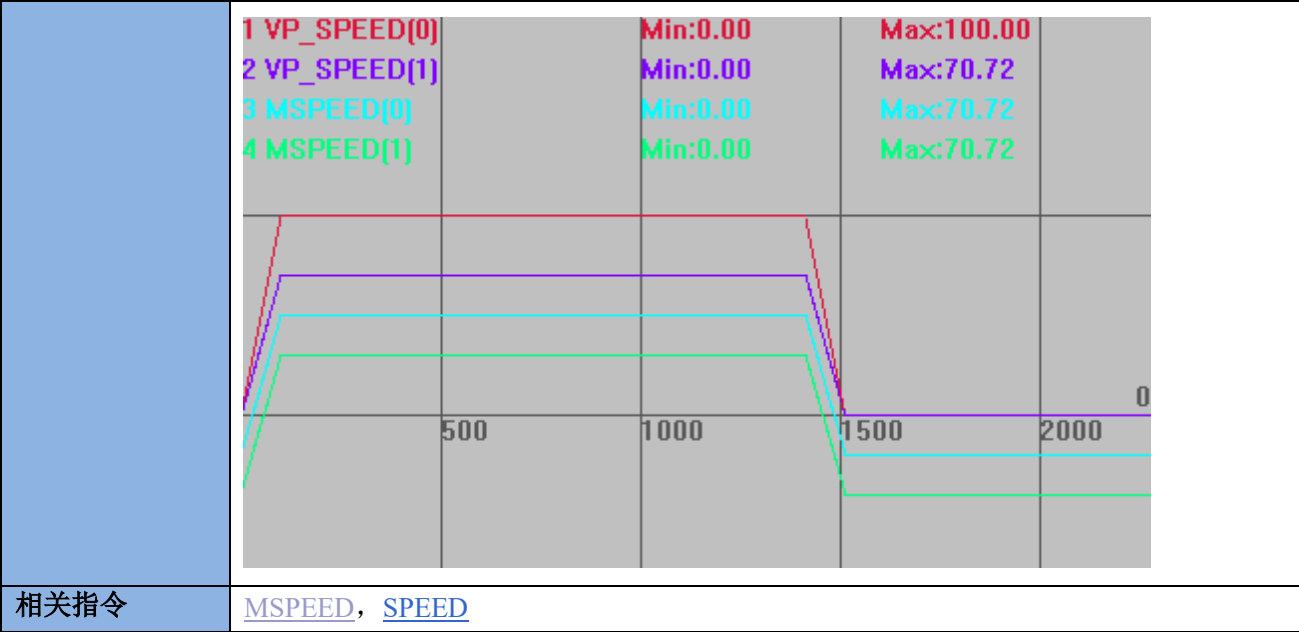


例三 模式 4
VP_MODE=4,0 '轴 0 模式 4，轴 1 模式 0
其他条件按例一，速度曲线如下，起步时最大加速度，达到最高速时加速度渐变为 0。



VP_SPEED -- 当前运动速度

类型	轴状态
描述	返回轴当前运动的速度，单位为 units/s。 当多轴运动时，主轴返回的是插补运动的速度，不是主轴的分速度。 非主轴返回的是相应的分速度，与 MSPEED 效果一致。 VP_SPEED 在默认情况下是为显示多轴合成速度设计的，是没有负值的，除非把 SYSTEM_ZSET 的 bit0 的值设置为 0，就可以用来显示单轴的命令速度，可正可负。
语法	VAR1 = VP_SPEED
适用控制器	通用
例子	BASE(0,1) DPOS=0,0 UNITS=100,100 SPEED =100,100 ACCEL=1000,1000 DECEL=1000,1000 TRIGGER MOVE(100,100) '坐标清 0 '脉冲当量 '主轴速度 100units/s '加速度 1000units/s/s '减速度 1000units/s/s '自动触发示波器 '两轴各运动 100units 速度曲线 主轴 VP_SPEED 是插补合成运动的速度。 非主轴 VP_SPEED 是轴当前分速度，与 MSPEED 一致。 VP_SPEED(0)垂直刻度 100，无偏移 VP_SPEED(1)垂直刻度 100，无偏移 MSPEED(0)垂直刻度 100，偏移-20 MSPEED(1)垂直刻度 100，偏移-40



INTERP_FACTOR -- 插补速度计算

类型	轴参数
描述	插补时轴是否参与速度计算，缺省参与计算（1）。 此参数只对直线的任意轴和螺旋的第三个轴起作用。 运动后请及时还原，否则会导致后续运动不正确。 插补速度计算为 0 的从轴不参与主轴的矢量计算。 部分轴不参与速度计算时，先按参与计算的轴的插补计算出参与轴的分速度和运动总时间，再把不参与计算的轴的运动距离依次除以总时间，得到不参与计算的各轴的相应速度。见例二。 不能把插补的实际运动的所有轴都设置 0，会导致实际速度无穷大。
语法	INTERP_FACTOR = 0/1 0-不参与计算，1-参与计算
适用控制器	通用
例子	例一：全部参与速度计算 BASE(0,1,2) '选择轴 0 为主轴 DPOS=0,0,0 ATYPE=1,1,1 UNITS=100,100,100 '脉冲当量 100 SPEED=100,100,100 '主轴速度 100units/s ACCEL=1000,1000,1000 DECEL=1000,1000,1000 INTERP_FACTOR=1,1,1 '3 轴都参与速度计算 TRIGGER '自动触发示波器 MOVE(100,200,300) '各轴分运动距离

此时根据合成运动速度 100 可以计算出各轴分速度，如下图。

VP_SPEED(0)垂直刻度 100

MSPEED(0)垂直刻度 100

MSPEED(1)垂直刻度 100

MSPEED(2)垂直刻度 100



例二：部分轴不参与速度计算

INTERP_FACTOR=0,1,1 '轴 0 不参与速度计算

此时先计算出仅有 2、3 轴插补时的分速度、运动总时间，然后再以轴 0 的运动距离除以总时间，得到轴 0 的速度，如下图。

垂直刻度同上



例三：只有一个轴参与计算

INVERT_FACTOR=0,1,0 '只有轴 1 进行运动计算

此时轴 1 的速度就是主轴速度 100，然后计算出运动总时间。其他轴依次用运动距离除以总时间，得到各轴速度，如下图。

垂直刻度同上

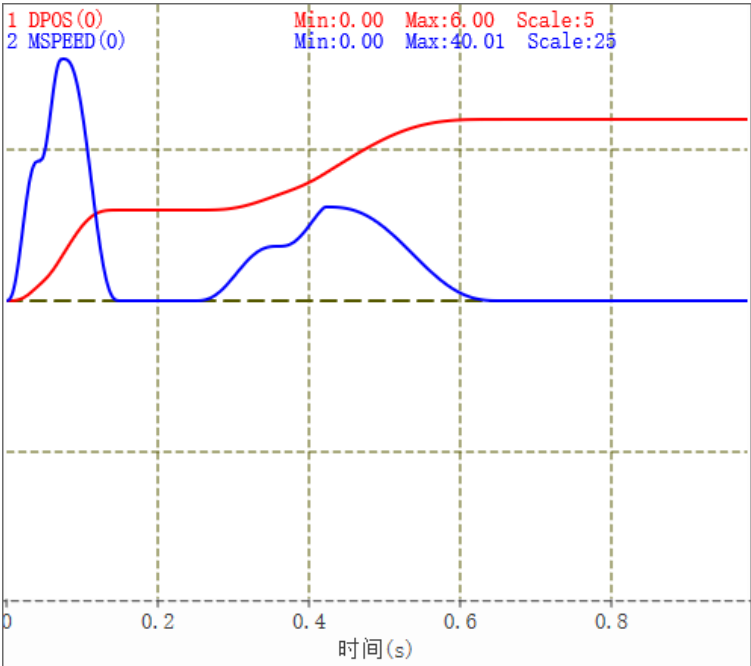
	1 MSPEED[0]		Min:0.00	Max:50.00	
	2 MSPEED[1]		Min:0.00	Max:100.00	
	3 MSPEED[2]		Min:0.00	Max:150.00	
相关指令	BASE_MOVE				

CORNER_ACCEL -- 拐弯加速度

类型	轴参数
描述	拐弯加速度，单位为 units/s/s。 用来设置曲率降速，缺省 0，不生效，设置后替换 FULL_SP_RADIUS 当 CORNER_MODE 设置分离模式时，每个轴设置的拐弯加速度都能生效。 建议与 ZSMOOTH_MODE 一起使用来平滑曲线和速度。 机器实际允许的拐弯加速度参考每个轴的加速度极限值来设置。 必须启用小圆限速，拐弯加速度才生效
语法	可读：VAR1 = CORNER_ACCEL(轴号) 可写 CORNER_ACCEL(轴号) = expression
适用控制器	4 系列控制器 fast 固件, 230926 以后版本支持.
例子	SPEED = 500,500,500,2000,313 ACCEL = 8000,5000,5000,4000,4200 CORNER_ACCEL = 5000,2000,3000,3000,3000
相关指令	ACCEL ， SPEED

JERK -- 加加速度

类型	轴参数
描述	轴加加速度，单位为 units/s/s/s。 缺省 0，不生效。设置值后替换 SRAMP。SRAMP 相当于 ACCEL*1000/JERK。 设置 JERK 后，VP_MODE=7 模式会自动计算降低加减速速度，对小距离运动震动有改善。
语法	可读：VAR1 = JERK 或 VAR1 = JERK(轴号) 可写：JERK = expression 或 JERK(轴号) = expression

适用控制器	4 系列控制器 fast 固件
例子	BASE(0,1) ATYPE = 1,1 DPOS = 0,0 MPOS = 0,0 UNITS = 1000,1000 SPEED = 100,100 ACCEL = 1000,1000 DECEL = 1000,1000 SRAMP = 100,100 MERGE = 1 VP_MODE=7 TRIGGER JERK = 0,0 MOVE(1) MOVE(1) MOVE(1) WAIT IDLE(0) JERK = 1000,1000 WA(100) MOVE(1) MOVE(1) MOVE(1) 曲线图如下： MSPEED(0)垂直刻度 25 DPOS(0)垂直刻度 5 

相关指令

[VP_MODE](#)

11.4 轴状态查看指令

MTYPE -- 当前运动类型

类型	轴状态	
描述	当前正在进行的运动指令类型。 当插补联动时，对从轴总是返回主轴的运动指令类型。	
语法	VAR1 = MTYPE	
	MTYPE	运动指令类型
	0	IDLE (没有运动)
	1	MOVE
	2	MOVEABS
	3	MHELICAL
	4	MOVECIRC
	5	MOVEMODIFY
	6	MOVESP
	7	MOVEABSSP
	8	MHELICALSP
	9	MOVECIRCSP
	10	FORWARD, VMOVE(1)
	11	REVERSE, VMOVE(-1)
	12	DATUMING
	13	CAM
	14	FWD_JOG
	15	REV_JOG
	16	MOVESYNC
	20	CAMBOX
	21	CONNECT
	22	MOVELINK
	23	CONNPATH
	25	MOVESLINK
	26	MOVESPIRAL
	27	MECLIPSE, MECLIPSEABS, MECLIPSESP, MECLIPSEABSSP
	28	MOVE_OP/MOVE_OP2, MOVE_TABLE, MOVE_TASK MOVE_PARA, MOVE_PWM, MOVE_ASYNMOVE, MOVE_AOUT
	29	MOVE_DELAY, MOVE_WAIT, MOVE_SYNMOVE
	31	MSPHERICAL, MSPHERICALSP

	32	MOVE_PT
	33	CONNFRAME
	34	CONNREFRAME
	35	MOVER_L/MOVER_LABS
	36	MOVER_C/MOVER_CABS
	37	MOVER_C3/MOVER_C3ABS
	47	ZSMOOTH_MODE, ZSMOOTH_START, ZSMOOTH_END
	50	MOVER2_JABS
	51	MOVER2_PABS
	52	MOVER2_LABS
	53	MOVER2_ARCABS
	54	MOVER2_ARCHPABS
	55	MOVER2_WAIT
适用控制器	通用	
例子	<pre> WHILE 1 '循环判断 IF MTYPE=0 THEN ?"没有运动" ELSEIF MTYPE=1 THEN ?"直线插补" ELSEIF MTYPE=4 THEN ?"圆弧插补" ENDIF WEND </pre>	
相关指令	NTYPE , REMAIN_BUFFER	

NTYPE -- 下条运动类型

类型	轴状态
描述	当前正在进行的运动指令后面的第一条指令类型。 当插补联动时，对从轴总是返回主轴的运动指令类型。
语法	VAR1 = NTYPE
适用控制器	通用
例子	<pre> WHILE 1 '循环判断 IF NTYPE=0 THEN ?"下一条结束运动" ELSEIF NTYPE=1 THEN ?"下一条直线插补" ELSEIF NTYPE=4 THEN ?"下一条圆弧插补" ... ENDIF WEND </pre>

相关指令

[MTYPE](#)

AXISSTATUS -- 轴状态

类型	轴状态				
描述	查看轴的各种状态。 按十进制显示数值，按二进制对应位判断状态。				
语法	VAR1 = AXISSTATUS				
	位	说明	打印值		
	1	随动误差超限告警	2	2h	
	2	与远程轴通讯出错	4	4h	
	3	远程驱动器报错	8	8h	
	4	正向硬限位	16	10h	
	5	负向硬限位	32	20h	
	6	找原点中	64	40h	
	7	HOLD 速度保持信号输入	128	80h	
	8	随动误差超限出错	256	100h	
	9	超过正向软限位	512	200h	
	10	超过负向软限位	1024	400h	
	11	CANCEL 执行中	2048	800h	
	12	脉冲频率超过 MAX_SPEED 限制.需要修改降速或修改 MAX_SPEED	4096	1000h	
	14	机械手指令坐标错误	16384	4000h	
	18	电源异常	262144	40000h	
	19	精准输出缓冲溢出	524288	80000h	
	20	情况 1：轴速度保护，当轴瞬间速度大于 MAX_SPEED 时发生报警 情况 2：被叠加轴限位或 ALM 报警，设置在主轴上，此时如果主轴运动中，会触发停止	1048576	100000h	
	21	运动中触发特殊运动指令失败	2097152	200000h	
	22	告警信号输入	4194304	400000h	
	23	轴进入了暂停状态	8388608	800000h	
	适用控制器	通用			
	例子	例一：直接读取位（推荐编程时使用） 碰到了正限位时 VAR = READ_BIT2(4,AXISSTATUS(0)) '读取轴 0 是否正限位 PRINT VAR '打印结果，1，发生了正向硬限位			
例二：返回数值判断（推荐直接查看时使用） ?AXISSTATUS(1) '查看轴 1 的状态，结果，48 '48=32+16，同时处于正负限位中，一般是没有反转正负限位开关电平					

	例三：总线通讯错误 在正确使能电机后： 将通讯接线断开，AXISSTATUS 会显示 4，与远程轴通讯出错。 将编码器线断开，对应轴 AXISSTATUS 会显示 8，远程驱动器报错。
相关指令	AXIS_STOPPREASON

IDLE -- 运动状态

类型	轴状态
描述	轴运动状态判断，只能判断是运动中还是停止。 运动中，返回 0，运动结束，返回-1。 当轴关联为机械手，CONNFRAME 逆解时，关节轴一直返回 0；CONNREFRAME 正解时，虚拟轴一直返回 0。
语法	VAR1 = IDLE
适用控制器	通用
例子	例一 <pre>IF IDLE(0) THEN '如果轴 0 停止 BASE(1) MOVE(100) ENDIF</pre> 例二 <pre>BASE(0,1) MOVE(100,100) BASE(2,3) MOVE(200,200) WAIT UNTIL IDLE(0) AND IDLE(1) AND IDLE (2) AND IDLE(3) '等待轴 0，1，2，3 停止</pre>
相关指令	LOADED ， WAIT IDLE

ADDAX_AXIS -- 叠加轴号

类型	轴状态
描述	ADDAX 指令所叠加轴的轴号，-1 表示没有叠加。
语法	VAR1 = ADDAX_AXIS
适用控制器	通用
例子	<pre>ADDAX(0) AXIS(1) '轴 0 的运动叠加到轴 1 ?ADDAX_AXIS(1) '打印出轴 1 叠加的轴，结果，0 ADDAX(-1) AXIS(1) '取消叠加</pre>
相关指令	ADDAX

AXIS_STOPREASON -- 轴停止原因

类型	轴状态
描述	轴历史停止原因锁存。
语法	写 0 清除，按位锁存，意义与 AXISSTATUS 相同。 20150731 以上版本支持。
适用控制器	通用
例子	IF AXIS_STOPREASON AND (512+1024) THEN PRINT "axis el stoped" ENDIF
相关指令	AXISSTATUS

LINK_AXIS -- 连接轴号

类型	轴状态
描述	返回当前连接运动的参考轴号，无连接时返回-1。
语法	VAR1 = LINKAX
适用控制器	通用
例子	CONNECT(2,1)AXIS(0) '轴 0 连接到轴 1 上 ?LINK_AXIS(0) '打印轴 0 的参考轴号，结果，1
相关指令	CAMBOX , MOVELINK , CONNECT

11.5 运动前瞻指令

CORNER_MODE -- 拐角设置

类型	轴参数		
描述	拐角减速等模式设置。		
语法	CORNER_MODE(轴号) = mode		
	mode: 不同的位代表不同的意义，位可以同时使用。		
	位	值	描述
	0	1	预留
	1	2	自动拐角减速 按 ACCEL,DECEL 加减速度 此参数是在 MOVE 函数调用前生效 减速角度定义查看 DECEL_ANGLE 和 STOP_ANGLE 指令 减速拐角参考速度以 FORCE_SPEED 速度为参考，一定要设置合理的 FORCE_SPEED

	2	4	预留
	3	8	自动曲率限速，半径小于设置值时限速，大于限制值时不限速 此参数在 MOVE 函数调用前修改生效 限制速度按 FORCE_SPEED 限速= FORCE_SPEED * 实际半径/FULL_SP_RADIUS 限速半径 FULL_SP_RADIUS 设置 多维小线段也生效
	4	16	超过停止角度，不倒角
	5	32	自动倒角设置 此参数在 MOVE 函数调用前修改生效 此 MOVE 运动自动和前面的 MOVE 运动做倒角处理，倒角半径参考 ZSMOOTH 此倒角针对插补的所有轴，20150701 以后固件支持
	6	64	多轴插补分离速度模式，只支持直线插补 按每个轴的速度加速度分开规划，整体按规划时间最长的运行 SP 模式同时使用 FORCE SPEED 对矢量速度进行限制 4 系列以上 23 年以后固件支持。
	7	128	MOVE 机械手虚拟轴时，同时使用关节轴的速度加速度来限制合成速度加速度 与 BIT6 同时使用生效, 4 系列以上 version_build: 240521 以后支持。 只支持 MOVE 直线指令，圆弧等不支持
	8	256	MOVER 指令强制使用 SP 模式
	9	512	预留
	10	1024	最高速度限制，如果轴速度超过 MAX_SPEED, 降低合速度 只支持直线和螺旋轴.
适用控制器	通用		
例子	为了方便说明每个位的功能，例程都是单独位设置，实际应用时可以多位同时使用。 比如 CONNER_MODE=2+8，打开自动拐角减速和曲率限速 例一：拐角减速 开启拐角减速后，若设置 SP 运动指令的 STARTMOVE_SPEED(SP 指令起始速度)或 ENDMOVE_SPEED(SP 指令结束速度)参数，拐角减速失效，拐角处的速度按以上两个 SP 速度指令设置的的大小运动。 <pre>BASE(0,1) DPOS=0,0 UNITS=100,100 ACCEL=500, 500 '设置加速度 DECEL=500,500 '设置减速度 SPEED=100,100 '设置速度 MERGE=ON '开启连续插补 CORNER_MODE=2 '启动拐角减速 DECAL_ANGLE = 15 * (PI/180) '设置开始减速角度 STOP_ANGLE = 45 * (PI/180) '设置结束减速角度 FORCE_SPEED=100 '等比减速时起作用</pre>		

TRIGGER'自动触发示波器

MOVE(100,0)

MOVE(0,100)'运动角度大于 45°，完全减速

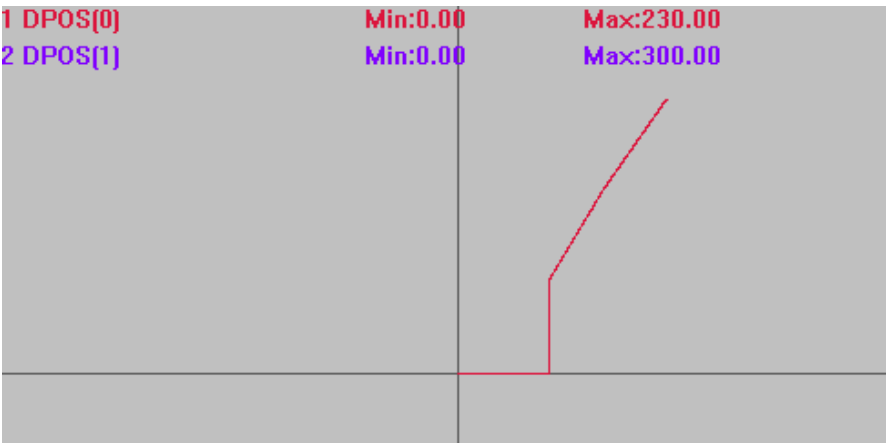
MOVE(60,100)'运动角度 30.96° 处于 15° ~45°，等比减速

MOVE(70,100)'运动角度 4.03° 小于 15°，不减速

轨迹曲线

DPOS(0)垂直刻度 200

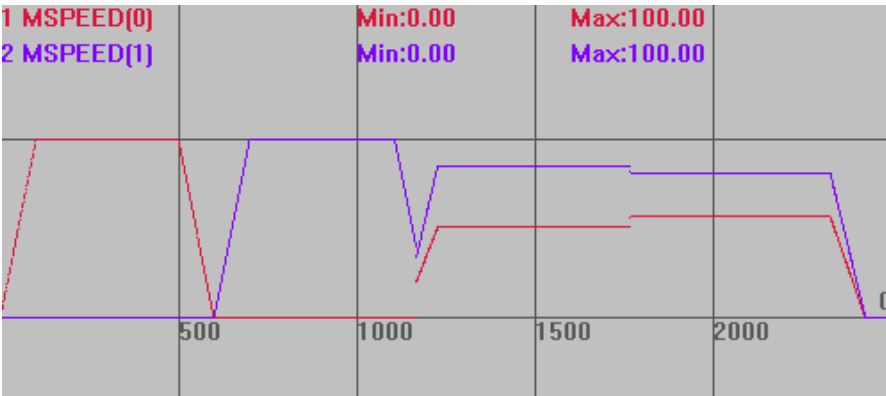
DPOS(1)垂直刻度 200



速度曲线

MSPEED(0)垂直刻度 100

MSPEED(1)垂直刻度 100



使用仿真器查看时曲线可能会有误差，最好连控制器查看。

例二：曲率限速

BASE(0,1)

DPOS=0,0

UNITS=100,100

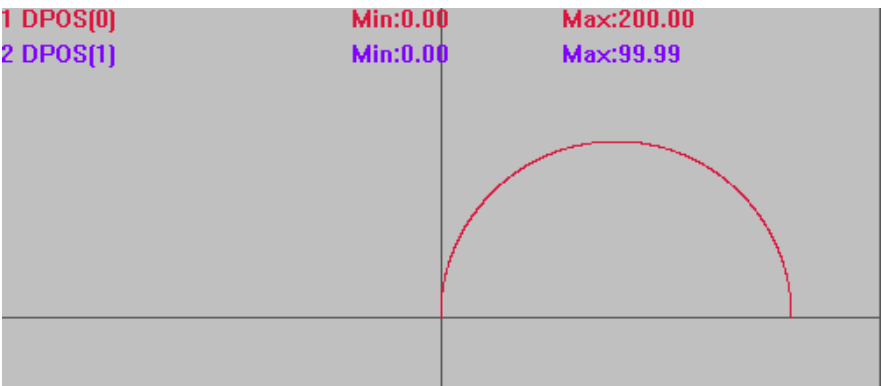
ACCEL=500,500'设置加速度

DECEL=500,500'设置减速度

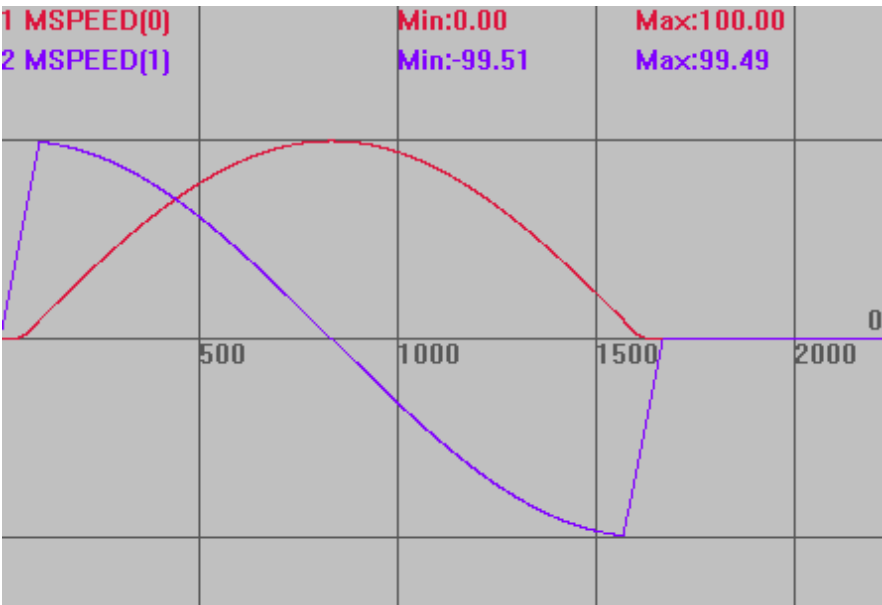
SPEED=100,500'运行速度

CORNER_MODE=8 '启动曲率限速
FORCE_SPEED=120 '曲率限速
FULL_SP_RADIUS=60 '限速半径 60
TRIGGER '自动触发示波器
MOVECIRC(200,0,100,0,1) '半径大于限制值时，不限制速度，按 SPEED 运行
此时速度为 100

轨迹曲线
DPOS(0)垂直刻度 100
DPOS(1)垂直刻度 100



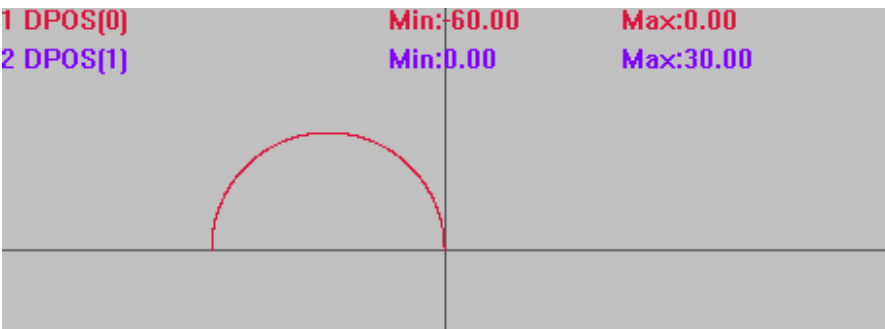
速度曲线
MSPEED(0)垂直刻度 100
MSPEED(1)垂直刻度 100



'半径小于限制值时，限速=FORCE_SPEED * 实际半径/FULL_SP_RADIUS
MOVECIRC(-60,0,-30,0,0) '此时速度 60=120*30/60

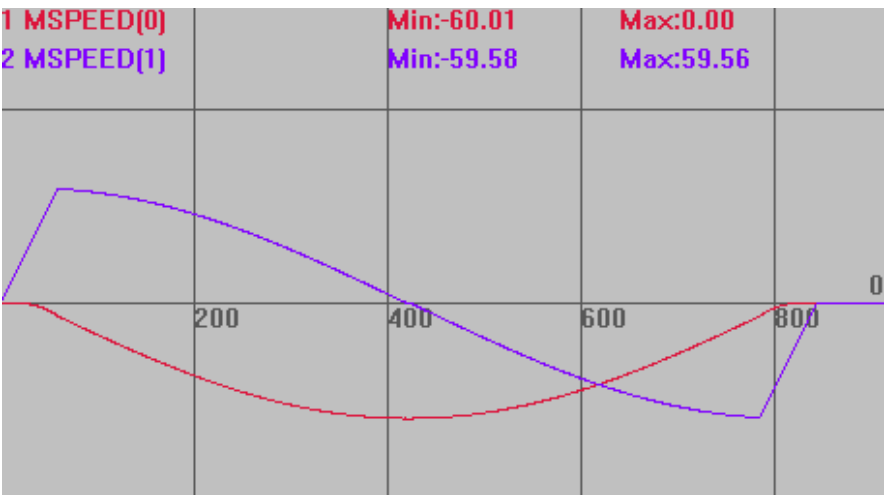
轨迹曲线

DPOS(0)垂直刻度 100
DPOS(1)垂直刻度 100



速度曲线

MSPEED(0)垂直刻度 100
MSPEED(1)垂直刻度 100



例三：倒角

倒角是可以用于直线、圆弧、螺旋等运动的，例程为了直观显示，只做了直线拐角
BASE(0,1)

DPOS=0,0

ACCEL=500,500 '设置加速度

DECEL=500,500 '设置减速度

SPEED=100,100 '运行速度

CORNER_MODE=32 '启动倒角

ZSMOOTH = 10 '倒角参考半径

TRIGGER '自动触发示波器

MOVE(100,0)

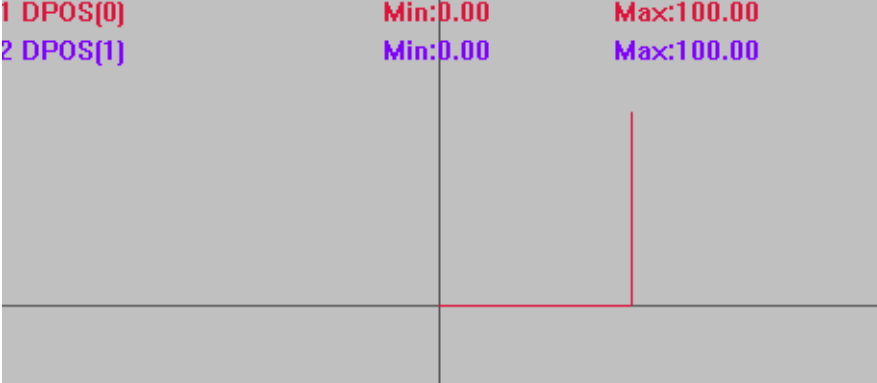
MOVE(0,100) '上面两条直线间自动倒角

插补轨迹

使用倒角

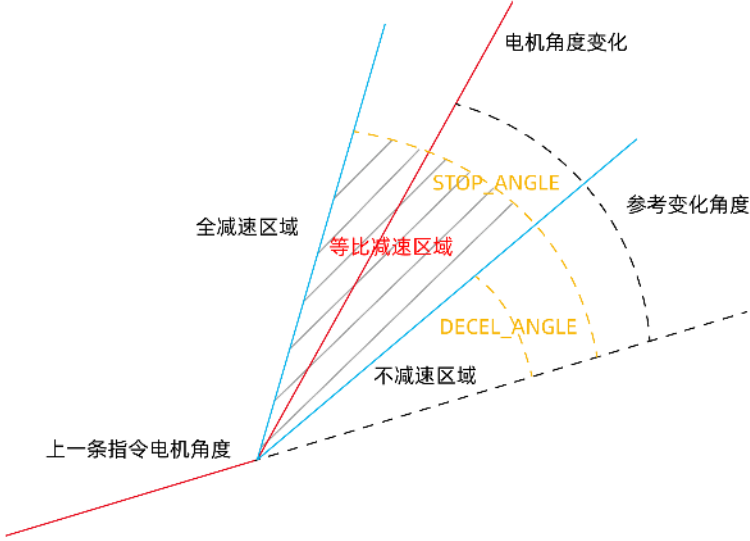
DPOS(0)垂直刻度 100

DPOS(1)垂直刻度 100

	1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:100.00 Max:100.00 
	不使用倒角 DPOS(0)垂直刻度 100 DPOS(1)垂直刻度 100
	1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:100.00 Max:100.00 
相关指令	MERGE , STOP_ANGLE , DECEL_ANGLE , FULL_SP_RADIUS , ZSMOOTH

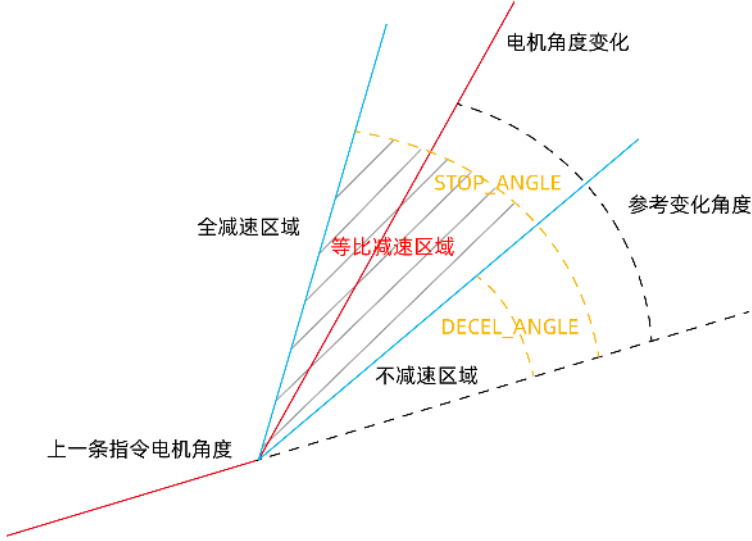
DECEL_ANGLE -- 拐角减速开始

类型	轴参数
描述	开始减速的角度，单位是弧度。 减速拐角参考速度以 FORCE_SPEED 速度为参考，一定要设置合理的 FORCE_SPEED。 角度转换弧度公式：角度值*(PI/180) 减速角度是指电机的参考角度相对上一条运动的变化值。如下图。 此角度值不是实际轨迹的角度，是换算到电机变换的角度，此角度值仅为参考。

	 下一插补运动轨迹处于下方时取绝对值。 直线与圆弧相连时按照圆弧的切线方向计算角度。 与 STOP_ANGLE 一起使用，当实际运动的角度处于 DECEL_ANGLE（上限）与 STOP_ANGLE（下限）时才会减速。
语法	VAR1 = DECEL_ANGLE, DECEL_ANGLE = expression
适用控制器	通用
例子	参考 CORNER_MODE 例程一 CORNER_MODE=2 DECEL_ANGLE = 25 * (PI/180) '设置开始减速拐角，弧度 STOP_ANGLE = 45 * (PI/180) '设置结束减速拐角，弧度 FORCE_SPEED=SPEED '一定要设置 FORCE_SPEED
相关指令	STOP_ANGLE ， CORNER_MODE -- 拐角设置

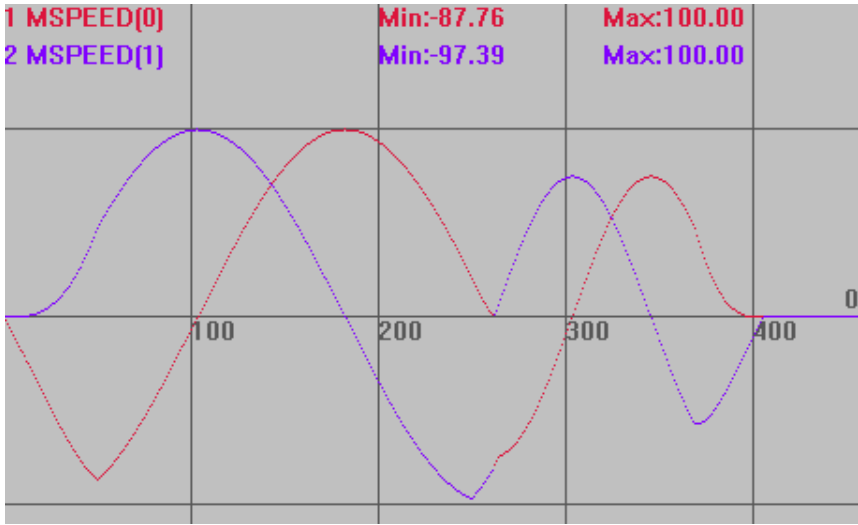
STOP_ANGLE -- 拐角减速结束

类型	轴参数
描述	停止减速到的角度，单位是弧度。 减速拐角参考速度以 FORCE_SPEED 速度为参考，一定要设置合理的 FORCE_SPEED。 角度转换弧度公式：角度值*(PI/180) 减速角度是指电机的参考角度相对上一条运动的变化值。如下图。 此角度值不是实际轨迹的角度，是换算到电机变换的角度，此角度值仅为参考。

	 电机角度变化 参考变化角度 全减速区域 等比减速区域 STOP_ANGLE DECEL_ANGLE 不减速区域 上一条指令电机角度 下一插补运动轨迹处于下方时取绝对值。 直线与圆弧相连时按照圆弧的切线方向计算角度。 与 DECEL_ANGLE 一起使用，当实际运动的角度处于 DECEL_ANGLE（上限）与 STOP_ANGLE（下限）时才会减速。
语法	VAR1 = STOP_ANGLE, STOP_ANGLE= expression
适用控制器	通用
例子	参考 CORNER_MODE 例程一 CORNER_MODE=2 DECEL_ANGLE = 25 * (PI/180) STOP_ANGLE = 90 * (PI/180) FORCE_SPEED=SPEED '一定要设置 FORCE_SPEED
相关指令	DECEL_ANGLE , CORNER_MODE

FULL_SP_RADIUS -- 限速半径

类型	轴参数
描述	曲率限速的最大圆弧半径，单位是 units。 当半径大于 FULL_SP_RADIUS 时，速度是用户程序指定的速度值，当半径小于 FULL_SP_RADIUS 时，控制器会按比例减小速度。 $VP_SPEED = FORCE_SPEED * radius / FULL_SP_RADIUS$。 设置自动倒角时为倒角的参考半径。
语法	VAR1 = FULL_SP_RADIUS, FULL_SP_RADIUS = expression
适用控制器	通用
例子	BASE(0,1) '选择轴 0 为主轴 DPOS=0,0 '坐标清 0 UNITS=100,100 ATYPE=1,1 '轴类型设置 SPEED=100,100 '主轴速度 100units/s

	ACCEL=1000,1000 '加速度 1000units/s/s DECEL=1000,1000 '加速度 1000units/s/s FORCE_SPEED=150 '自定义速度 150units/s CORNER_MODE=8 '开启曲率限速 FULL_SP_RADIUS=8 '限速半径 8 TRIGGER '自动触发示波器 MOVECIRC(10,10,0,10,1) '圆弧半径 10，不限速，按 speed=100 运行 MOVECIRC(4,4,0,4,1) '圆弧半径 4，限速，速度按 $4/8*150=75$ 运行 速度曲线 MSPEED(0)垂直刻度 100 MSPEED(1)垂直刻度 100
	
相关指令	CORNER_MODE CORNER_MODE -- 拐角设置, FORCE_SPEED , SPLIMIT_RADIUS

SPLIMIT_RADIUS -- 限速值

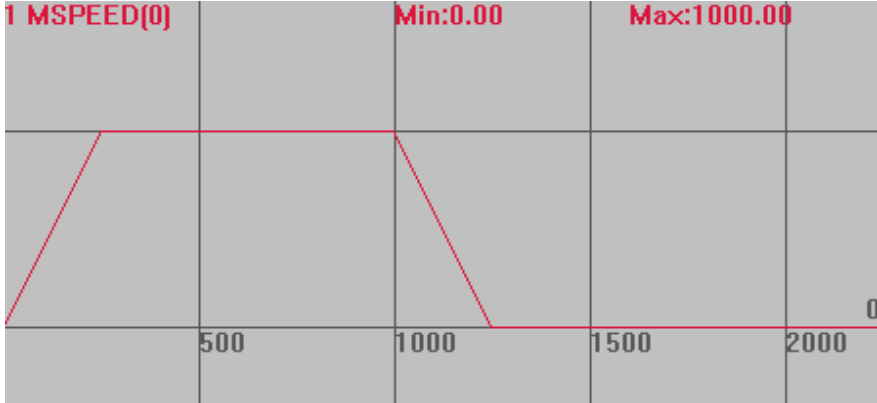
类型	轴参数
描述	曲率限速的最小速度，单位是 units/s。 当小于 LSPEED 时，按 LSPEED 值。
语法	VAR1 = SPLIMIT_RADIUS, SPLIMIT_RADIUS = expression
适用控制器	4 系列控制器 170518 以上固件版本
相关指令	CORNER_MODE , FULL_SP_RADIUS

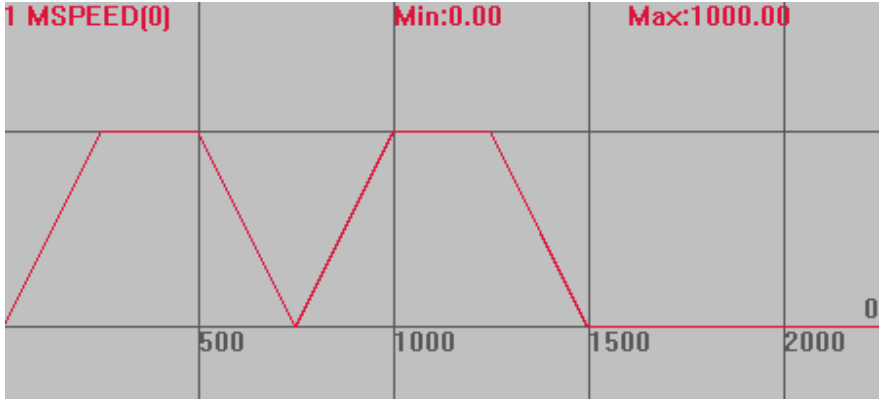
ZSMOOTH -- 倒角半径

类型	轴参数
描述	参考倒角半径， CORNER_MODE 使用。 根据拐角角度自动计算实际拐角半径大小。超过路径时限制为 50%。 90° 时拐角半径为设置值。

语法	VAR1 = ZSMOOTH, ZSMOOTH = smoothdistance
适用控制器	通用
例子	参考 CORNER_MODE 例三
相关指令	CORNER_MODE

MERGE -- 连续插补

类型	轴参数
描述	前后缓冲的运动连接到一起而不减速，用于连续插补。 当 MERGE 设置为 ON 时，多段插补间仍减速，可能原因如下： 1. 可能 MERGE 并没有设置成功，可以打印查看。 2. 控制器是点位运动型号，不支持连续插补，可联系厂家。 3. 设置了 CORNER_MODE 拐角减速，打印确认。 4. 使用了带 SP 的运动指令，并设置了 ENDMOVE_SPEED, STARTMOVE_SPEED, 此时速度由这两条指令确定。 5. 多条插补间切换了主轴，主轴速度参数改变了。 6. 多条插补间加入了 MOVE_DELAY 延时指令，即使延时写 0，也会导致减速。
语法	MERGE = ON/OFF
适用控制器	通用
例子	BASE(0,1) '选择轴 0 DPOS=0,0 UNITS=100,100 SPEED=1000,1000 '速度 1000units ACCEL=1000,1000 DECEL=1000,1000 MERGE=ON '打开连续插补 TRIGGER '自动触发示波器 MOVE(1000,1000) '运动 100units MOVE(0,0) 速度曲线 MSPEED(0)垂直刻度 100 

	MERGE=OFF 时 MSPEED(0)垂直刻度 100 
相关指令	*SP , CORNER_MODE

ZSMOOTH_MODE -- 平滑速度曲线模式

类型	轴参数
描述	多个小段合一的方式平滑速度曲线。 注意：最长平滑 2 倍倒角距离。平顺会导致多个小运动段变成一个运动段，MTYPE 与 MARK 会有差异，结束运动必须调用结束设置，否则可能导致最后几个小运动段位于等待状态。 夹角超过减速角度的线段之间，不平滑
语法	ZSMOOTH_MODE=mode mode: 0-不使用平滑; 1-缺省模式
适用控制器	4 系列以上控制器，230908 以后固件支持
例子	ZSMOOTH_MODE=1
相关指令	ZSMOOTH_START 、 ZSMOOTH_END

ZSMOOTH_START -- 开始平滑模式

类型	轴参数
描述	开始平滑模式。
语法	ZSMOOTH_START([mode]) mode: 平滑模式，缺省 1 等同：ZSMOOTH_MODE = 1
适用控制器	4 系列以上控制器，230908 以后固件支持
例子	ZSMOOTH_START()
相关指令	ZSMOOTH_MODE 、 ZSMOOTH_END

ZSMOOTH_END -- 退出平滑模式

类型	轴参数
描述	退出平滑模式。
语法	ZSMOOTH_END([mode]) mode: 平滑模式, 缺省 0 等同: ZSMOOTH_MODE = 0
适用控制器	4 系列以上控制器, 230908 以后固件支持
例子	ZSMOOTH_END()
相关指令	ZSMOOTH_MODE 、 ZSMOOTH_START

11.6 运动缓冲指令

LOADED -- 缓冲空

类型	轴状态
描述	除了当前运动外, 没有缓冲的指令了, 此时返回 TRUE。 此指令无法判断轴是否停止, 判断轴停止使用 IDLE。
语法	VAR1 = LOADED
适用控制器	通用
相关指令	IDLE , WAIT LOADED

MOVES_BUFFERED -- 当前缓冲数

类型	轴状态
描述	返回当前被缓冲起来的运动指令个数。 不要用总缓冲空间数减去 MOVES_BUFFERED 这个参数来判断是否有剩余的缓冲空间, 特殊的运动指令可能会占用多个缓冲空间, 采用 REMAIN_BUFFER 状态函数来判断更准确。
语法	VAR1 = MOVES_BUFFERED
适用控制器	通用
例子	PRINT MOVES_BUFFERED '打印结果, 0
相关指令	LOADED , LIMIT_BUFFERED , REMAIN_BUFFER

REMAIN_BUFFER -- 剩余缓冲数

类型	特殊轴状态
描述	返回可以剩余使用的缓冲个数。 此状态比较特殊, 本身可以带参数, 因此修正轴号时 AXIS 不能省掉。

	当返回 0 时表示当前轴的缓冲空间满，此时如果继续调用当前轴的运动指令会阻塞任务直到缓冲有空位。
语法	VAR1 = REMAIN_BUFFER ([mtype]) AXIS(axisnum) MTYPE 缺省为最复杂的运动，如空间圆弧。
适用控制器	通用
例子	<pre> DIM movetime '定义变量 movetime = 0 WHILE movetime < 100 '条件循环 IF REMAIN_BUFFER(1) > 0 THEN '如果有剩余缓冲，调用直线运动指令 MOVE(10) movetime = movetime + 1 ENDIF WEND '调用了 100 次 move(10) </pre>
相关指令	LOADED , LIMIT_BUFFERED

MOVE_MARK -- 运动标号

类型	轴参数
描述	下一条要调用的运动指令的 MARK 标号，这个标号会和运动指令一起写入运动缓冲。 每调用一条运动指令，MOVE_MARK 会自动加一。 如果要强制指定 MOVE_MARK，需要每次运动前都设定一次。 通过 MOVE_PAUSE 指令可以在 MARK 不同的边界处暂停。
语法	VAR1 = MOVE_MARK, MOVE_MARK = expression
适用控制器	通用
例子	<pre> MOVE_MARK =1 '设置为标号 1 MOVE(100) MOVE_MARK =1 '设置为标号 1 MOVE(100) MOVE_MARK =2 '设置为标号 2 MOVE(200) MOVE_PAUSE (2) 'MOVE(200)前暂停 </pre>
相关指令	MOVE_CURMARK

MOVE_CURMARK -- 当前运动标号

类型	轴状态
描述	返回当前轴正在运动指令的 MOVE_MARK 标号。
语法	VAR1 = MOVE_CURMARK
适用控制器	通用
例子	<pre> MOVE_MARK =1 MOVE(100) </pre>

	MOVE_MARK =2 MOVE(200) MOVE_MARK =3 MOVE(300) WAIT UNTIL MOVE_CURMARK = 2 '等待 MOVE(200)开始执行的时候，开输出 口 1 OP(1,ON)
相关指令	MOVE_MARK

LIMIT_BUFFERED -- 运动缓冲限制

类型	系统参数
描述	限定运动缓冲个数，不能超过控制器的最大值。
语法	LIMIT_BUFFERED = value, VAR1 = LIMIT_BUFFERED
适用控制器	通用
例子	在线命令打印： <pre>>>?LIMIT_BUFFERED 4096</pre> 修改运动缓冲个数限制值： <pre>>>LIMIT_BUFFERED=2000 >>?LIMIT_BUFFERED 2000</pre>
相关指令	REMAIN_BUFFER , MOVES_BUFFERED

11.7 位置相关指令

DPOS -- 轴指令位置

类型	轴状态
描述	轴的虚拟坐标位置，或称需求位置。 写 DPOS 会自动转换为 OFFPOS 偏移，不会移动电机。 以 UNITS 作为单位。
语法	VAR1 = DPOS, DPOS=expression
适用控制器	通用
例子	DPOS(0) = 0 '轴 0 坐标偏移至 0 ?*DPOS '命令行查看当前 DPOS，打印结果，000000000000
相关指令	MPOS , ENDMOVE , OFFPOS , DEFPOS

MPOS -- 编码器反馈位置

类型	轴状态
描述	轴的测量反馈位置，单位是 units 。 写 MPOS 会自动转换为 OFFPOS 偏移。
语法	VAR1 = MPOS, MPOS=expression
适用控制器	通用
例子	MPOS(0) = 50 'MPOS 偏移至 50 ?*MPOS '打印结果, 50 0 0 0 0 0 0 0 0 0 0
相关指令	DPOS , ENDMOVE

DEFPOS -- 位置偏移

类型	设置轴坐标指令
描述	设置当前轴位置为一个新的绝对位置值，不会对已运行/进入缓冲区的运动产生影响。
语法	DEFPOS(pos1 [,pos2[, pos3[, pos4.....]]]) pos1: 绝对位置，采用 unit 定义单位 pos2: 下一个轴绝对位置，采用 unit 定义单位
适用控制器	通用
例子	例一 <pre> BASE(0,1) '选择轴 0，轴 1 ATYPE=1,1 UNITS=100,100 '脉冲当量设为 100 DPOS=0,0 '把 DPOS 清 0 MOVE(100,100) '轴 0 和轴 1 运动 100 WAIT IDLE ?DPOS(0),DPOS(1) '此时 DPOS 都为 100 DEFPOS(0,10) '设置当前位置 ?DPOS(0),DPOS(1) '此时 DPOS 为 0，10 </pre> 例二 与 OFFPOS 相对改变不同，DEFPOS 是改变为绝对位置 <pre> BASE(0,1) '选择轴 0，轴 1 DPOS=100,100 '设置当前位置为 100,100 ?DPOS(0),DPOS(1) '打印确认，当前位置为 100,100 DEFPOS(10,20) '设置当前位置为 10,20 ?DPOS(0),DPOS(1) '当前位置为 10,20 DEFPOS(10,20) '多次调用 DEFPOS DEFPOS(10,20) ?DPOS(0),DPOS(1) '此时当前位置仍为 10,20 OFFPOS=10,20 '多次调用 OFFPOS OFFPOS=10,20 ?DPOS(0),DPOS(1) '此时当前位置变为 30,60 (10+10+10,20+20+20) </pre>

相关指令	DPOS OFFPOS -- 偏移位置
------	---

OFFPOS -- 偏移位置

类型	轴参数
描述	相对偏移修改所有的坐标，不会对已运行/进入缓冲区的运动产生影响。 当修改完成后，OFFPOS 还原为 0。
语法	VAR1 = OFFPOS, OFFPOS = expression
适用控制器	通用
例子	例一 相对偏移位置 <pre> BASE(0) MOVEABS(1000) WAIT IDLE OFFPOS = -1000 '坐标偏移 1000 PRINT DPOS(0) '打印结果 0 </pre> 例二 不改变在运行运动 <pre> BASE(0) MOVEABS(1000) '运动到绝对位置 1000 OFFPOS =500 '位置偏移 500 WAIT IDLE PRINT DPOS(0) '打印当前位置 1500，此时电机仍运动 1000 </pre> 例三 与 DEFPOS 改变为绝对位置不同，OFFPOS 是相对改变 <pre> BASE(0,1) '选择轴 0，轴 1 DPOS=100,100 '设置当前位置为 100,100 ?DPOS(0),DPOS(1) '打印确认 DEFPOS(10,20) '设置当前位置为 10,20 ?DPOS(0),DPOS(1) '当前位置为 10,20 DEFPOS(10,20) '多次调用 DEFPOS DEFPOS(10,20) ?DPOS(0),DPOS(1) '此时当前位置仍为 10,20 OFFPOS=10,20 '多次调用 OFFPOS OFFPOS=10,20 ?DPOS(0),DPOS(1) '此时当前位置变为 30,60（10+10+10,20+20+20） </pre>
相关指令	DPOS , DEFPOS

ENDMOVE -- 当前运动目标位置

类型	轴状态
描述	轴当前运动的最终目标绝对位置。

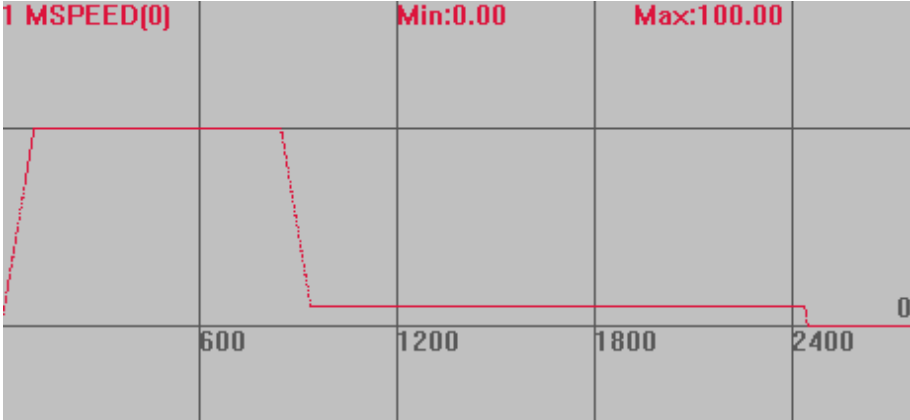
	对 VMOVE, DATUM 等非固定距离的运动, ENDMOVE 并不准确, 或时刻在变化。
语法	VAR1 = ENDMOVE
适用控制器	通用
例子	<pre> BASE(0) SPEED = 10 '速度 10units/s DPOS = 0 '坐标清 0 MOVE(100) '运动 100units WAIT IDLE PRINT ENDMOVE(0) '运行到该条语句时当前运动 move(100)完成 '打印结果, 100 MOVE(200) WAIT IDLE PRINT ENDMOVE(0) '运行到该条语句时当前运动 move(200)完成, 打印结果 300 </pre>
相关指令	DPOS , MPOS , ENDMOVE_BUFFER

VECTOR_MOVED -- 当前运动距离

类型	轴状态
描述	返回轴当前运动的距离, units 单位。 对多轴插补是矢量距离, 使用之前最好手动清零。 此指令只对运动指令生效, 对 ADDAX、CONNECT 等叠加指令的从轴无效。
语法	VAR1 = VECTOR_MOVED VECTOR_MOVED=0
适用控制器	通用
例子	<pre> 例一 单轴 VECTOR_MOVED=0 '手动清 0 MOVE(100) WAIT IDLE ? VECTOR_MOVED '打印出轴 0 运动距离, 结果, 100 例二 多轴 BASE(0,1) DPOS=0,0 VECTOR_MOVED=0 '轴 0 轴组合成矢量运动距离手动清 0 BASE(2,3) DPOS=0,0 VECTOR_MOVED=0 '轴 2 轴组合成矢量运动距离手动清 0 BASE(0,1) MOVE(-300,-400) WAIT IDLE(0) ? VECTOR_MOVED '打印出轴 0 轴组合成矢量运动距离, 结果, 500 MOVE(300,400) WAIT IDLE(0) </pre>

	? VECTOR_MOVED '打印出轴 0 轴组合成矢量运动距离，结果，1000 BASE(2,3) MOVE(30,-40) WAIT IDLE(2) ? VECTOR_MOVED '打印出轴 2 轴组合成矢量运动距离，结果，50 MOVE(30,40) WAIT IDLE(2) ? VECTOR_MOVED '打印出轴 2 轴组合成矢量运动距离，结果，100
相关指令	ENDMOVE

REMAIN -- 当前运动剩余距离

类型	轴状态
描述	返回轴当前运动还未完成的距离，units 单位。
语法	VAR1 = REMAIN
适用控制器	通用
例子	BASE(0) '选择轴 0 DPOS=0 UNITS=100 SPEED=100 '速度 100units/s ACCEL=1000 '加速度 1000units/s DECEL=1000 TRIGGER '自动触发示波器 MOVE(100) '运动 100units WAIT UNTIL REMAIN<20 '等待剩余距离小于 20 SPEED=10 '修改速度 速度曲线 MSPEED(0)垂直刻度 100 
相关指令	VECTOR_BUFFERED

VECTOR_BUFFERED --缓冲剩余距离

类型	轴状态
描述	返回轴当前运动和缓冲运动还未完成的距离，units 单位。 对多轴插补是矢量距离。
语法	VAR1 = VECTOR_BUFFERED
适用控制器	通用
例子	BASE(0) '选择轴 0 UNITS=100 '脉冲当量 100 SPEED=100 '速度 100units/s ACCEL=1000 '加速度 1000units/s/s MOVE(100) '当前运动 100units MOVE(300) '缓冲运动 300units MOVE(-1000) '缓冲运动-1000units ?VECTOR_BUFFERED '返回剩余运动距离，结果，1400
相关指令	REMAIN

VECTOR_BUFFERED2 --目标矢量位置

类型	轴状态
描述	用于读取当前插补指令调用后的目标矢量位置，units 单位。 VECTOR_BUFFERED2=VECTOR_BUFFERED+VECTOR_MOVED 可以读取用作 HW 比较输出。
语法	VAR1 = VECTOR_BUFFERED2
适用控制器	通用
例子	BASE(0,1) '选择轴 0,1 UNITS=100,100 SPEED=100,100 ACCEL=1000,1000 DECEL=1000,1000 MERGE=0,0 VECTOR_BUFFERED2=0,0 DPOS=0,0 MOVE(100,100) '三段插补运动 MOVE(200,200) MOVE(-100,-100) ?"开始运动" ?VECTOR_BUFFERED '返回剩余运动距离，结果 565.68 ?VECTOR_MOVED '当前运动距离，结果 0 ?VECTOR_BUFFERED2 '返回需求目标矢量距离，结果 565.68

	DELAY(1000)	'延时 1s
	?"运动中"	
	?VECTOR_BUFFERED	'剩余运动距离，结果 470.63
	?VECTOR_MOVED	'当前运动距离，结果 95.05
	?VECTOR_BUFFERED2	'返回需求目标矢量距离，结果 565.68
	WAIT IDLE	'等待轴停止
	?"运动结束"	
	?VECTOR_BUFFERED	'剩余运动距离，结果 0
	?VECTOR_MOVED	'当前运动距离，结果 565.68
	?VECTOR_BUFFERED2	'插补指令的需求目标矢量距离，结果 565.68
相关指令	VECTOR_BUFFERED	

ENDMOVE_BUFFER -- 缓冲最终位置

类型	轴状态
描述	轴当前和缓冲的所有运动的最终目标位置。 可用来进行绝对和相对位置的转换，见例二。 对 VMOVE，DATUM 等非固定距离的运动，ENDMOVE_BUFFER 并不准确，或时刻在变化。 使用了循环坐标指令 REP_OPTION 后，ENDMOVE_BUFFER 根据 REP_OPTION 模式按 REP_DIST 设置值依次递减，即最小精度是 REP_DIST（模式 1）或 2 倍 REP_DIST（模式 0），见例三。
语法	VAR1 = ENDMOVE_BUFFER
适用控制器	通用
例子	例一 <pre> BASE(0) SPEED = 10 DPOS = 0 MOVE(100) MOVE(200) PRINT ENDMOVE_BUFFER(0) '直接打印出最终运动完后的绝对坐标 '打印结果，300 </pre> 例二 相对绝对转换 使用 ENDMOVE_BUFFER 与相对运动指令可实现绝对运动的功能，常用于 MSPHERICAL 等只有相对模式的指令。 <pre> BASE(0) UNITS=100 SPEED=100 ACCEL=1000 DPOS=0 </pre>

	<pre> WHILE 1 MOVE(100- ENDMOVE_BUFFER(0)) '运动到 100 位置，不会继续运动 WEND 例三 循环坐标时的返回值 BASE(0) UNITS=100 SPEED=100 ACCEL=1000 DPOS=0 TRIGGER MOVE(1000) REP_DIST=100 '设置坐标循环范围 REP_OPTION=1 '0~100 循环 WHILE 1 ?ENDMOVE_BUFFER(0)'此时打印出 1000,900,800...,100,0, 打印的最小精度是 100 WEND </pre>
相关指令	DPOS , MPOS , ENDMOVE

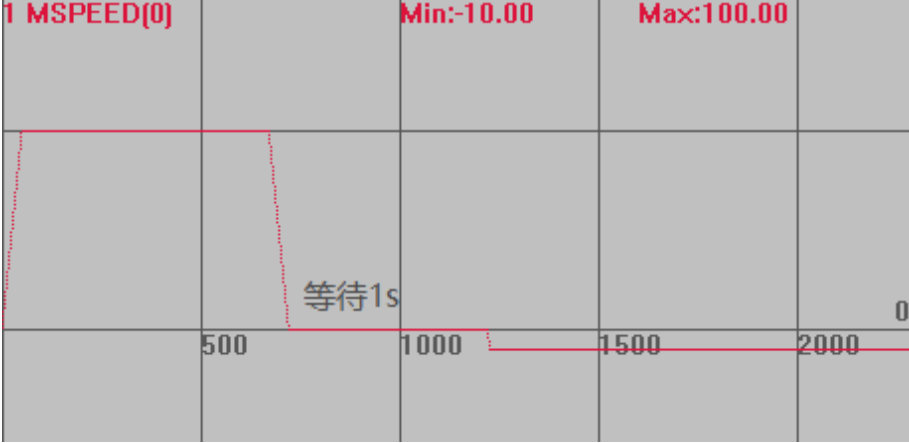
11.8 原点回零指令

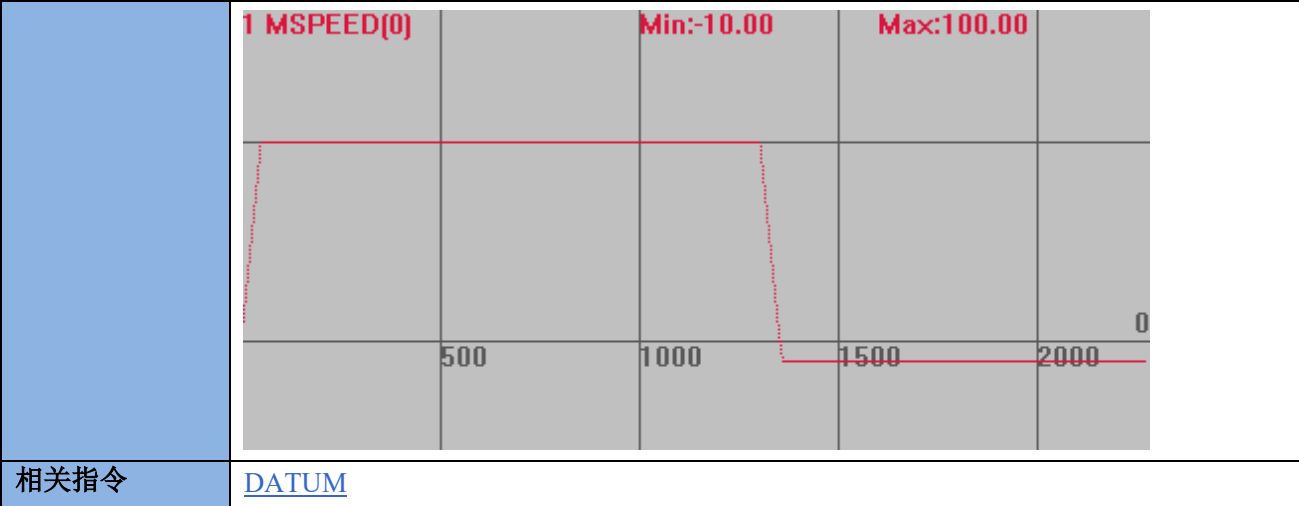
DATUM_IN -- 映射原点输入

类型	轴参数
描述	通用输入口设置为原点开关信号，-1 无效。 设置了原点开关后，ZMC 控制器输入 OFF 时认为有信号输入，要相反效果可以用 INVERT_IN 反转电平。(ECI 系列控制器除外)。
语法	<pre> VAR1 = DATUM_IN, DATUM_IN = expression VAR1 = DATUM_IN(axisnum), DATUM_IN (axisnum)= expression </pre>
适用控制器	通用
例子	<pre> BASE(0,1,2,3) DATUM_IN =6,7,8,9 '将轴 0, 1, 2, 3 原点输入对应到输入口 6, 7, 8, 9 INVERT_IN(6,ON) '把原点信号反转 INVERT_IN(7,ON) INVERT_IN(8,ON) INVERT_IN(9,ON) </pre>
相关指令	DATUM , FWD_IN , REV_IN , INVERT_IN

HOMEWAIT -- 回零反找延时

类型	轴参数
----	-----

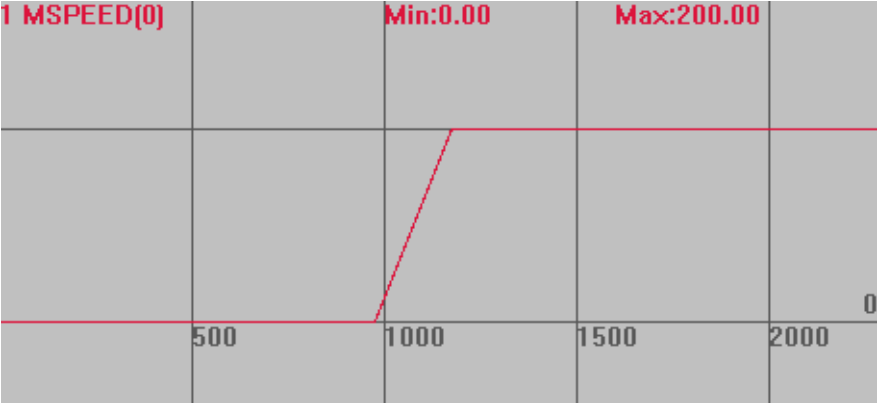
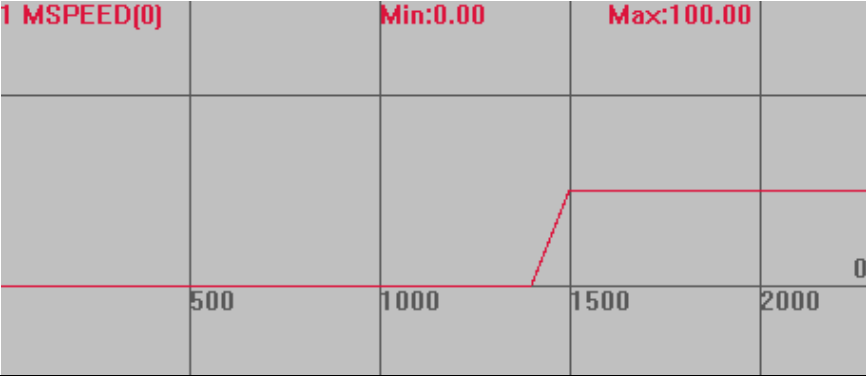
描述	此参数设置等待的时间，单位是毫秒。 对脉冲方式的伺服驱动器，回原点运动时，当反找时要等待一定时间。 控制器默认值为 2ms。		
语法	VAR1 = HOMEWAIT, HOMEWAIT = expression		
适用控制器	通用		
例子	BASE(0) '选择轴 0 DPOS=0 '坐标清 0 UNITS=100 ATYPE=1 SPEED=100 '找原点速度 100units/s ACCEL=1000,1000 '加速度 1000units/s/s DECEL=1000,1000 '加速度 1000units/s/s CREEP=10 '爬行速度 10units/s DATUM_IN=0 '原点信号设为输入 IN0 INVERT_IN(0,ON) '反转信号 HOMEWAIT=1000 '设置反找等待 1s TRIGGER '自动触发示波器 DATUM(3) '正向找原点 速度曲线 碰到 IN0 口时，会先等待 1s，然后在反向爬行 MSPEED(0)垂直刻度 100  HOMEWAIT=2 时，几乎不停止 MSPEED(0)垂直刻度 100		



11.9 JOG 运动指令

FAST_JOG -- 映射点动输入

类型	轴参数		
描述	快速点动的输入的编号，-1 为无效。 如果设置快速点动输入，速度由 SPEED 参数给出。如果没有输入设置，速度由 JOGSPEED 参数给出。见例程。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平(ECI 系列控制器除外)。		
语法	VAR1 = FAST_JOG, FAST_JOG= expression VAR1 = FAST_JOG(axisnum), FAST_JOG (axisnum)= expression		
适用控制器	通用		
例子	BASE(0) '选择轴 0 DPOS=0 '坐标清 0 UNITS=100 ATYPE=1 SPEED=100 '设置速度为 100 units/s ACCEL=500 '加速度为 500units/s/s JOGSPEED=200 '点动速度设为 200units/s FAST_JOG(0)=0 '轴 0 的快速输入设为 IN0 口 FWD_JOG(0)=1 '正向点动开关设为 IN1 口 INVERT_IN(0,ON) '反转电平 INVERT_IN(1,ON) TRIGGER '自动触发示波器 速度曲线 IN0 无输入时，按下 IN1 并保持，轴速度为 JOGSPEED=200		

	MSPEED(0)垂直刻度 200 1 MSPEED(0) Min:0.00 Max:200.00 
	IN0 有输入时，按下 IN1，轴速度为 SPEED=100 MSPEED(0)垂直刻度 200 1 MSPEED(0) Min:0.00 Max:100.00 
相关指令	REV_JOG ， FWD_JOG ， SPEED ， JOGSPEED

FWD_JOG -- 映射正向 JOG 输入

类型	轴参数
描述	正向 JOG 输入对应的输入口编号，-1 无效。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平(ECI 系列控制器除外)。 当有信号输入时，对应轴按照 JOGSPEED 速度正向运动。
语法	VAR1 = FWD_JOG, FWD_JOG= expression VAR1 = FWD_JOG(axisnum), FWD_JOG(axisnum)= expression
适用控制器	通用
例子	BASE(0) '选择轴 0 DPOS=0 '坐标清 0 UNITS=100 ATYPE=1 SPEED=100 '速度 100 ACCEL=500 '加速度为 500units/s/s DECEL=500

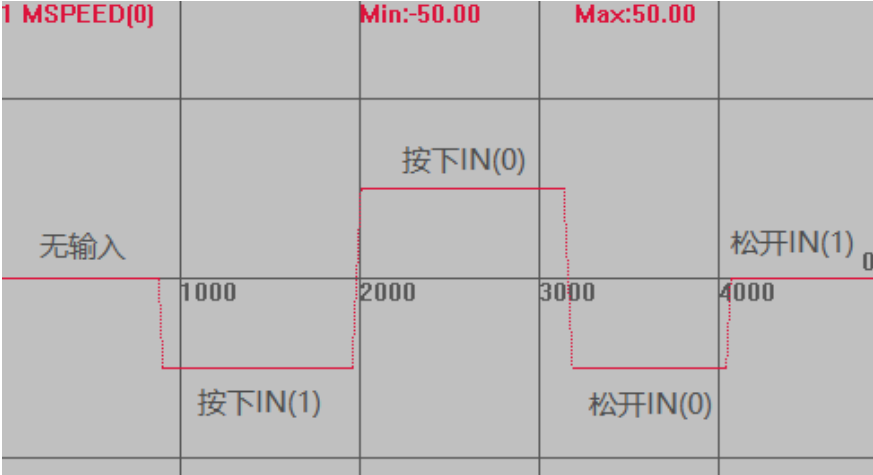
	JOGSPEED=50 'JOG 速度 50 FWD_JOG=0 '输入 IN0 作为正向 JOG 开关 INVERT_IN(0,ON) '反转信号 TRIGGER '自动触发示波器 输入 0 口有信号输入时，轴 0 正向运行，速度为 50。 输入 0 口取消信号输入时，轴 0 停止。 速度曲线 MSPEED(0)垂直刻度 100
相关指令	REV_JOG , JOGSPEED , FAST_JOG

REV_JOG -- 映射负向 JOG 输入

类型	轴参数
描述	负向 JOG 输入对应的输入口编号，-1 无效。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平(ECI 系列控制器除外)。 当有信号输入时，对应轴按照 JOGSPEED 速度负向运动。 当 REV_JOG 和 FWD_JOG 同时有信号输入时，轴按照 FWD_JOG 运行。
语法	VAR1 = REV_JOG, REV_JOG= expression VAR1 = REV_JOG(axisnum), REV_JOG(axisnum)= expression
适用控制器	通用
例子	参考 FWD_JOG 例子
相关指令	FWD_JOG , JOGSPEED , FAST_JOG

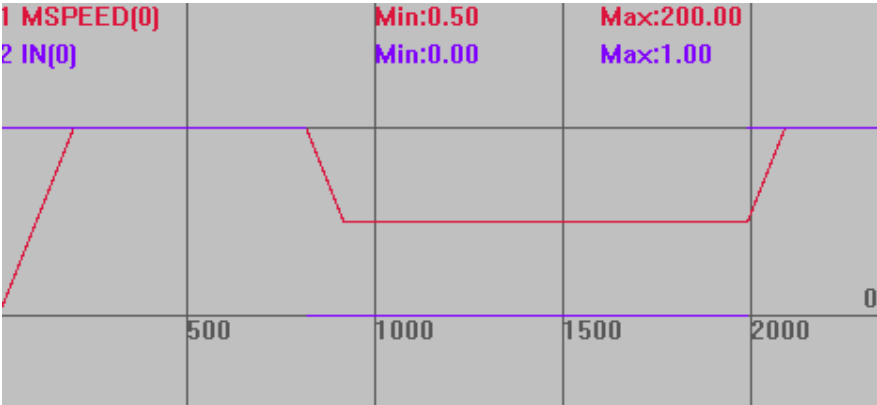
JOGSPEED -- JOG 速度

类型	轴参数
描述	JOG 时的速度，单位为 units/s。 当 REV_JOG/FWD_JOG 被设置，对应输入点按下时，并保持当前输入状态，电机将以

	JOGSPEED 慢速运动，输入点松开运动停止。
语法	JOGSPEED= value, VAR1=JOGSPEED
适用控制器	通用
例子	例一 BASE(0) '选择轴 0 DPOS=0 '坐标清 0 UNITS=100 '脉冲当量 SPEED =100 '主轴速度 100units/s ACCEL=1000 '加速度 1000units/s/s DECEL=1000 '减速度 1000units/s/s TRIGGER '自动触发示波器 JOGSPEED=50 'JOG 速度 50 FWD_JOG=0 '输入 IN0 作为正向 JOG 开关 REV_JOG=1 '输入 IN1 作为负向 JOG 开关 INVERT_IN(0,ON) '反转信号 INVERT_IN(1,ON) 输入 0 口有信号输入时，轴 0 正向运行，速度为 50。 输入 1 口有信号输入时，轴 0 负向运行，速度为 50。 同时有信号输入时，轴 0 正向运行。 速度曲线 MSPEED(0)垂直刻度 100 
相关指令	REV_JOG , FWD_JOG , FAST_JOG

FHOLD_IN -- 映射保持输入

类型	轴参数
描述	保持输入对应的输入点编号，-1 无效。 如果有输入信号，运动轴的速度由 FHSPEED 参数设置。当前运动没有被取消。当取

	消输入，过程中的运动速度返回程序速度。见例程。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平(ECI 系列控制器除外)。 此参数只适用速度控制方式(带 SP 后缀指令)。如果运动不是速度控制（CAMBOX，CONNECT，MOVELINK），不会起作用。
语法	VAR1 = FHOLD_IN, FHOLD_IN = expression VAR1 = FHOLD_IN(axisnum), FHOLD_IN (axisnum)= expression
适用控制器	通用
例子	BASE(0) '选择轴 0 DPOS=0 '坐标清 0 UNITS=100 ATYPE=1 ACCEL=500 '加速度为 500units/s/s DECEL=500 FORCE_SPEED=200 '设置速度为 200 units/s FHSPEED=200 '保持速度设为 100units/s FHOLD_IN(0)=0 '轴 0 的保持输入设为 IN0 口 INVERT_IN(0,ON) '反转电平 TRIGGER '自动触发示波器 MOVESP(10000) '运动 当 IN0 无输入时，轴以 FORCE_SPEED=200 的速度运动 当 IN0 有输入时，轴以 FHSPEED=100 的速度运动，取消输入后，速度变回 200 速度曲线 MSPEED(0)垂直刻度 200 
相关指令	FHSPEED

FHSPEED -- 保持速度

类型	轴参数
描述	轴保持速度，在 FHOLD_IN 被按下保持时的速度，单位为 units/s。 对应的输入处于保持状态时才能一直以此速度运动。

语法	VAR1 = FHSPEED, FHSPEED = expression
适用控制器	通用
例子	见 FHOLD_IN 例程
相关指令	FHOLD_IN , SPEED

11.10 编码器相关指令

ENCODER -- 编码器原始值

类型	轴状态
描述	编码器硬件寄存器原始值。 内部参数，只有配置为需要使用编码器的 ATYPE 时才可以读取。 驱动器有多圈绝对值编码器时，读取的就是多圈值。
语法	VAR1 = ENCODER(轴号)
适用控制器	通用
例子	?*ENCODER '打印各轴编码器值，打印结果，000000000000
相关指令	MPOS , ENCODER_RATIO

ENCODER_STATUS -- 编码器状态

类型	轴状态		
描述	编码器的 EA EB EZ 的状态。		
语法	VAR1 = ENCODER_STATUS		
	位	值	描述
	0	1	EA 状态
	1	2	EB 状态
	2	4	EZ 状态
适用控制器	通用		
例子	?*ENCODER_STATUS '打印全部轴的编码器状态		
相关指令	ATYPE , MPOS		

ENCODER_FILTER -- 编码器滤波

类型	轴参数
描述	内置编码器轴滤波设置，用于使皮带编码器的速度均匀，缺省值为 1，设置范围值=1/周期数，范围为 0.001~1。 缺省 1 -不滤波。 0.5 -2 个周期的滤波。 0.25 -4 个周期的滤波。

	5 系控制器支持，4 系列固件 170706 以上固件版本支持。
语法	ENCODER_FILTER = VALUE
适用控制器	通用
相关指令	ENCODER_RATIO

PP_STEP -- 编码器内部比例

类型	轴参数
描述	编码器输入内部会乘以这个比例。 这个参数效果与 ENCODER_RATIO 叠加，缺省 1。
语法	PP_STEP = VALUE
适用控制器	通用
相关指令	ENCODER_RATIO

ENCODER_BITS -- 编码器绝对值设置

类型	轴参数			
描述	设置 SSI/BISS 编码器绝对值。			
语法	ENCODER_BITS = VALUE			
	编码器类型	位	值	功能描述
	SSI/BISS	Bit0-5	0-32	编码器通讯总位数
		Bit6	64	是否是格雷码
		Bit8-10	256*(0<n<15)	无效的位数，BISS 一般=8
		Bit16-18	65536*(0<n<7)	频率分频调整，缺省 0- 2MHz
ATYPE = 48 ‘SSI 绝对式编码器				
ATYPE = 49 ‘BISS 绝对式编码器				
在使用该指令之前必须要进行轴映射				
适用控制器	带 Ssi 的固件 ZMC432-V2 和 ZMC406-V2 的控制器			
例子	SSI 举例：			
	BASE(n)			
	AXIS_ADDRESS=(-1<<16)+4 '映射第 4 个物理轴位置			
	ENCODER_BITS = 26 '26 位绝对值			
	ATYPE=48			
BISS 举例：				
BASE(n)				
AXIS_ADDRESS=(-1<<16)+5 '映射第 5 个物理轴位置				
ENCODER_BITS = 26 '26 位绝对值，自动带 8 个状态位				
ATYPE=49				

ENCODER_ID -- 存储数据字典的编号

类型	轴参数
描述	配置自定义编码器轴的数据源。
语法	ENCODER_ID=(index<<16)+(subindex<<8)+ bites bites: 数据类型的位数, 比如数据 32 位, 则写 32
适用控制器	通用
例子	定义一个编码器轴: BASE(0) AXIS_ADDRESS = (0<<16) + nodenum ENCODER_ID= \$60640020 ATYPE=25

DRIVE_POSMIN -- 编码器传输原始最小值

类型	轴参数
描述	设置编码器的传输原始值范围最小值
语法	DRIVE_POSMIN = VALUE 修改 ATYPE 前设置. 如果需要 MPOS 也做坐标循环, 修改 REP_OPTION. 范围: 32 位整数。0 到 $2^{32}-1$ DRIVE_POSMAX = $2^{31}-1$ DRIVE_POSMIN = -2^{31}
适用控制器	通用
例子	BASE(0) AXIS_ADDRESS = (0<<16) + nodenum ENCODER_ID= \$60640020 DRIVE_POSMIN = 0 DRIVE_POSMAX= $2^{18}-1$ ‘PDOBUFF 的值在这个范围内. ATYPE=25
相关指令	DRIVE_POSMAX

DRIVE_POSMAX -- 编码器传输原始最大值

类型	轴参数
描述	设置编码器的传输原始值范围最大值
语法	DRIVE_POSMAX = VALUE 修改 ATYPE 前设置. 如果需要 MPOS 也做坐标循环, 修改 REP_OPTION. 范围: 32 位整数。0 到 $2^{32}-1$ DRIVE_POSMAX = $2^{31}-1$

	DRIVE_POSMIN = -2 ³¹
适用控制器	通用
例子	同 DRIVE_POSMIN 例子
相关指令	DRIVE_POSMIN

11.11 锁存相关指令

REGIST -- 锁存

类型	位置锁存指令										
描述	REGIST 指令用来锁存轴的测量反馈位置。 支持编码器轴、总线轴锁存，不同型号的控制器支持锁存的轴类型不同，4 系列及以上控制器最新固件支持虚拟轴、脉冲轴锁存。 EtherCAT 支持驱动器锁存，此时使用驱动器 IO 点实现锁存，需要配置 DRIVE_PROFILE 指令，具体模式查看指令语法。 Rtex 只支持控制器锁存。 4 系列及以上控制器支持 4 锁存通道。 4 个通道指 MARK、MARKB、MARKC、MARKD，通过 REG_INPUTS 指定锁存通道对应的高速输入口，默认对应输入口 0、1、2、3，详情请查看控制器的用户手册中数字量输入章节。 支持 EtherCAT 驱动器锁存与控制器锁存同时使用，需要有 4 锁存通道功能。总线驱动器探针锁存使用 MARK，MARKB 通道，控制器本地 IN 锁存使用 MARKC、MARKD 通道。 锁存模式对应的上升沿和下降沿根据 NPN 还是 PNP 决定 当锁存产生时，轴状态 MARK 会被设置为 ON，同时锁存到的位置会被存储在参数 REG_POS 内。 每个轴的锁存通道相互独立										
语法	语法一 REGIST(mode) mode: 锁存方式 上升下降沿是基于控制器为高电平有效时对应的上升下降沿。如果设置成上升沿触发，则锁存会在外部输入口由导通状态进入截止状态的一瞬间触发；如果设置成下降沿触发，则锁存会在外部输入口由截止状态进入导通状态的一瞬间触发；具体要设置成上升沿还是下降沿触发，要根据外部输入口信号跳变的实际需求。 脉冲轴类型一般采用 R0，R1，Z 脉冲这三种锁存；总线轴本地 IN 锁存使用 R2，R3。 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>1</td><td>当 Z 脉冲上升沿时的绝对位置送到 REG_POS</td></tr> <tr> <td>2</td><td>当 Z 脉冲下降沿时的绝对位置送到 REG_POS</td></tr> <tr> <td>3</td><td>当输入信号 R0 上升沿的绝对位置送到 REG_POS</td></tr> <tr> <td>4</td><td>当输入信号 R0 下降沿的绝对位置送到 REG_POS</td></tr> </tbody> </table>	值	描述	1	当 Z 脉冲上升沿时的绝对位置送到 REG_POS	2	当 Z 脉冲下降沿时的绝对位置送到 REG_POS	3	当输入信号 R0 上升沿的绝对位置送到 REG_POS	4	当输入信号 R0 下降沿的绝对位置送到 REG_POS
值	描述										
1	当 Z 脉冲上升沿时的绝对位置送到 REG_POS										
2	当 Z 脉冲下降沿时的绝对位置送到 REG_POS										
3	当输入信号 R0 上升沿的绝对位置送到 REG_POS										
4	当输入信号 R0 下降沿的绝对位置送到 REG_POS										

6	输入信号 R0 上升沿时的绝对位置送到 REG_POS, Z 信号上升沿时的绝对位置送到 REG_POSB
7	输入信号 R0 上升沿时的绝对位置送到 REG_POS, Z 信号下降沿时的绝对位置送到 REG_POSB
8	输入信号 R0 下降沿时的绝对位置送到 REG_POS, Z 信号上升沿时的绝对位置送到 REG_POSB
9	输入信号 R0 下降沿时的绝对位置送到 REG_POS, Z 信号下降沿时的绝对位置送到 REG_POSB
10	输入信号 R0 上升沿时的绝对位置送到 REG_POS, 输入信号 R1 上升沿时的绝对位置送到 REG_POSB
11	输入信号 R0 上升沿时的绝对位置送到 REG_POS, 输入信号 R1 下降沿时的绝对位置送到 REG_POSB
12	输入信号 R0 下降沿时的绝对位置送到 REG_POS, 输入信号 R1 上升沿时的绝对位置送到 REG_POSB
13	输入信号 R0 下降沿时的绝对位置送到 REG_POS, 输入信号 R1 下降沿时的绝对位置送到 REG_POSB
14	输入信号 R1 上升沿时的绝对位置送到 REG_POSB(14 以后 150804 以后版本支持, 每个锁存通道独立, 支持 4 通道锁存)
15	输入信号 R1 下降沿时的绝对位置送到 REG_POSB
16	Z 信号上升沿时的绝对位置送到 REG_POSB
17	Z 信号下降沿时的绝对位置送到 REG_POSB
18	输入信号 R2 上升沿时的绝对位置送到 REG_POSC
19	输入信号 R2 下降沿时的绝对位置送到 REG_POSC
20	输入信号 R3 上升沿时的绝对位置送到 REG_POSD
21	输入信号 R3 下降沿时的绝对位置送到 REG_POSD

语法二

REGIST(100+mode, tableindex, numes)

mode: 锁存方式

tableindex: 连续锁存的内容存储的 table 位置, 第一个 table 元素存储锁存的个数, 后面存储锁存的坐标, 最多保存个数= numes-1, 溢出时循环写入

numes: 占用的 table 个数

通过把模式加 100 来支持连续锁存, 锁存结果存储到 TABLE 里面。
分别对两个通道进行连续锁存, 可以实现上下边沿的连续锁存。

ECI: 20150829 以上固件支持。

4 系列控制器: 20170523 以上固件支持。

100+mode: 只能使用单一通道的 mode, 加 100 表示使用连续锁存

值	描述
1	当 Z 脉冲上升沿时的绝对位置送到 REG_POS
2	当 Z 脉冲下降沿时的绝对位置送到 REG_POS
3	当输入信号 R0 上升沿的绝对位置送到 REG_POS
4	当输入信号 R0 下降沿的绝对位置送到 REG_POS

	14	输入信号 R1 上升沿时的绝对位置送到 REG_POSB
	15	输入信号 R1 下降沿时的绝对位置送到 REG_POSB
	16	Z 信号上升沿时的绝对位置送到 REG_POSB
	17	Z 信号下降沿时的绝对位置送到 REG_POSB
	18	输入信号 R2 上升沿时的绝对位置送到 REG_POSC
	19	输入信号 R2 下降沿时的绝对位置送到 REG_POSC
	20	输入信号 R3 上升沿时的绝对位置送到 REG_POSD
	21	输入信号 R3 下降沿时的绝对位置送到 REG_POSD
	23	当输入信号 R0 上升沿的绝对位置送到 REG_POSB
	24	当输入信号 R0 下降沿的绝对位置送到 REG_POSB
	33	当输入信号 R0 上升沿的绝对位置送到 REG_POS,下一次切换下降沿, 轮流切换。
	34	当输入信号 R0 下降沿的绝对位置送到 REG_POS, 下一次切换上升沿, 轮流切换。
	35	当输入信号 R1 上升沿的绝对位置送到 REG_POSB,下一次切换下降沿, 轮流切换。下一次切换下降沿, 轮流切换。
	36	当输入信号 R1 下降沿的绝对位置送到 REG_POSB, 下一次切换上升沿, 轮流切换。
适用控制器	有锁存 IN 口。	
例子	以下例程基于 ZMC432 控制器测试。 例一 锁存脉冲轴 0 的输入信号 R0 上跳沿时的位置, 并打印。 <pre> BASE(0) REG_INPUTS=0 '将 R0-R3 都对应输入口 0 ATYPE=1 '脉冲轴 REGIST(3) '选择 R0 锁存模式 WAIT UNTIL MARK '等待锁存触发 PRINT REG_POS '打印锁存位置 </pre> 例二 锁存编码器轴 0 的输入信号 R1 上跳沿时的位置, 并打印。 <pre> BASE(0) REG_INPUTS=0 '将 R0-R3 都对应输入口 0 ATYPE=3 '编码器轴 REGIST(14) '选择 R1 锁存模式 WAIT UNTIL MARKB '等待锁存触发 PRINT REG_POSB '打印锁存位置 </pre> 例三 锁存总线轴 0 的输入信号 R2/R3 边沿时的位置, 并打印。 <pre> BASE(0) REG_INPUTS = \$1000 '映射锁存通道 R3-R0 对应到输入口 1,0,0,0 REGIST(imode) IF imode = 18 OR imode = 19 THEN '通道 R2 锁存 WAIT UNTIL MARKC '探针 1 ?"模式",Imode,"锁存位置 REG_POSC",REG_POSC </pre>	

```

ELSEIF imode = 20 OR imode = 21 THEN                                '通道 R3 锁存
    WAIT UNTIL MARKD                                              '等待锁存触发
    ?"模式",Imode,"锁存位置 REG_POSD",REG_POSD
ENDIF

```

例四 PC 交互位置锁存，一般用于运动抓拍，通过锁存的位置得知抓拍时的实际位置。

```

GLOBAL g_start
GLOBAL g_posx, g_posy
WHILE 1
    WAIT UNTIL g_start=1 '等待 PC 发出启动
    REGIST(4) AXIS(0) '输入 0 锁存，24V 变 0V 的时刻
    REGIST(4) AXIS(1)
    WAIT UNTIL MARK(0) AND MARK(1)
    g_start=0
    g_posx=REG_POS(0)
    g_posy=REG_POS(1)
    PRINT g_posx, g_posy
WEND

```

例五 100+mode 连续锁存

```

DIM num
num=1
BASE(6)
ATYPE=6
REGIST(100+4,5,100) '自动循环，不需要再写入到 while 循环中，table(5)保存锁
存次数，table(6)-table(105)存储每次锁存的数据超过 99 次后，table(5)清 0，重新从
table(6)记录数据
WHILE 1
    local test
    test=table(5)
    WAIT UNTIL table(5)
    ?reg_pos, TABLE(num), TABLE() '打印
    IF num=100 THEN
        num=1
    ELSE
        num=num+1
    ENDIF
    WA 1 '延时 1ms，防抖
WEND

```

例六 总线驱动器锁存，需要配置 DRIVE_PROFILE 支持带探针的模式

```

BASE(iaxis) '选择需要锁存位置的轴号
REGIST(imode) '锁存模式
IF imode = 3 OR imode = 4 THEN
    WAIT UNTIL MARK '探针 1

```

	<pre> ?"模式",imode,"锁存位置 REG_POS",REG_POS DELAY(100) ELSEIF imode = 14 OR imode = 15 THEN WAIT UNTIL MARKB '等待锁存触发 ?"模式",imode,"锁存位置 REG_POSB",REG_POSB DELAY(100) ELSEIF imode = 11 THEN WAIT UNTIL MARK OR MARKB '等待锁存触发 IF MARK THEN ?"模式",Imode,"锁存位置 REG_POS",REG_POS WAIT UNTIL MARKB ?"锁存位置 REG_POSB",REG_POSB ELSEIF MARKB THEN ?"模式",Imode,"锁存位置 REG_POSB",REG_POSB WAIT UNTIL MARK ?"锁存位置 REG_POSB",REG_POS ENDIF DELAY(100) ENDIF </pre>
相关指令	MARK , MARKB , REG_POS , REG_POSB

REG_INPUTS -- 锁存输入映射

类型	轴参数		
描述	设置 R0-R3 输入锁存的输入口映射，每 4bit 对应一个锁存口。		
语法	VAR1 = REG_INPUTS		
	bit	锁存口	输入口范围(例如：ZMC306E)
	bit0-3	R0	0-3
	bit4-7	R1	0-3
	bit8-11	R2	0-3
	bit12-15	R3	0-3
	输入口范围：不同的控制器可以使用的锁存口个数不同。		
适用控制器	3 系列部分 4 系列及以上支持锁存功能，最新固件		
例子	BASE(6) ATYPE=3 REG_INPUTS=\$3210 'R0-R3 分别对应输入口 0, 1, 2, 3 REG_INPUTS=\$1111 'R0-R3 都对输入口 1		
相关指令	REGIST		

REGIST_DELAY -- 锁存延时

类型	轴参数
描述	使用本地输入口锁存总线驱动器的坐标时，总线驱动器的实际位置可能有延时，即伺服周期参数值与默认值不同时，必须调整 REGIST_DELAY 。
语法	regist_delay=timems 别名: regist_wa=timems timems: 延时，毫秒单位
适用控制器	有锁存 IN 口
例子	regist_delay=-1.01 '当 SERVO_PERIOD 为 1ms 时，补偿一个伺服周期加上 10 微秒的传感器输入延迟。 regist_delay=-0.515 '当 SERVO_PERIOD 为 0.5ms 时，补偿一个伺服周期加上 15 微秒的传感器输入延迟。
相关指令	DELAY

MARK -- 锁存触发

类型	轴状态
描述	返回锁存事件是否产生。 当 REGIST 指令执行时，MARK 变真，返回-1。当执行完成时，MARK 变成假，返回 0。
语法	VAR1 = MARK
适用控制器	通用
例子	BASE(0) '选择轴 0 MOVE(100) '运动 100units REGIST(3) '上升沿 R0 WAIT UNTIL MARK '等待触发
相关指令	REG_POS , REGIST

MARKB -- 锁存 2 触发

类型	轴状态
描述	返回对应第二个锁存通道的锁存事件是否产生。 当 REGIST 指令执行时，MARKB 变真，返回-1。当执行完成时，MARKB 变成假，返回 0。
语法	VAR1 = MARKB
适用控制器	通用
例子	BASE(0) '选择轴 0 MOVE(100) '运动 100units REGIST(14) '上升沿 R1 WAIT UNTIL MARKB '等待触发
相关指令	REG_POSB , REGIST

MARKC -- 锁存 3 触发

类型	轴状态
描述	返回对应第三个锁存通道的锁存事件是否产生。 当 REGIST 指令执行时，MARCK 变真，返回-1。当执行完成时，MARKC 变成假，返回 0。
语法	VAR1 = MARKC
适用控制器	通用
例子	BASE(0) '选择轴 0 MOVE(100) '运动 100units REGIST(18) '上升沿 R2 WAIT UNTIL MARKC '等待触发
相关指令	REG_POSC , REGIST

MARKD -- 锁存 4 触发

类型	轴状态
描述	返回对应第四个锁存通道的锁存事件是否产生。 当 REGIST 指令执行时，MARKD 变真，返回-1。当执行完成时，MARKD 变成假，返回 0。
语法	VAR1 = MARKD
适用控制器	通用
例子	BASE(0) '选择轴 0 MOVE(100) '运动 100units REGIST(20) '上升沿 R3 WAIT UNTIL MARKD '等待触发
相关指令	REG_POSD , REGIST

OPEN_WIN -- 锁存开始坐标范围

类型	轴参数
描述	锁存触发的开始坐标范围点。 预留
语法	OPEN_WIN = pos
相关指令	REGIST , CLOSE_WIN

CLOSE_WIN -- 锁存结束坐标范围

类型	轴参数
----	-----

描述	锁存触发的结束坐标范围点。 预留
语法	CLOSE_WIN = pos
相关指令	REGIST , OPEN_WIN

REG_POS -- 锁存位置

类型	轴状态
描述	保存锁存的测量反馈位置(MPOS), units 单位。 锁存位置为编码器反馈位置, 锁存位置要及时读取出来保存, 当编码器坐标继续变化后, REG_POS 会变无效。锁存位置存在范围限制, 当编码器旋转超过该范围, REG_POS 会根据运动方向增加或减少对应范围值, 范围根据不同控制器分为 2^{16} 、 2^{32} 。
语法	VAR1 = REG_POS(轴号)
适用控制器	通用
例子	MOVE(100) '运动 100units REGIST(3) '上升沿 R0 WAIT UNTIL MARK '等待锁存发生 PRINT REG_POS '打印出锁存位置
相关指令	REGIST , MARK

REG_POSB -- 锁存 2 位置

类型	轴状态
描述	返回锁存寄存器 2 的测量反馈位置(MPOS), units 单位。 锁存位置为编码器反馈位置, 锁存位置要及时读取出来保存, 当编码器坐标继续变化后, REG_POSB 会变无效。锁存位置存在范围限制, 当编码器旋转超过该范围, REG_POSB 会根据运动方向增加或减少对应范围值, 范围根据不同控制器分为 2^{16} 、 2^{32} 。
语法	VAR1 = REG_POSB(轴号)
适用控制器	通用
例子	MOVE(100) '运动 100units REGIST(16) '上升沿 EZ 信号触发 WAIT UNTIL MARKB '等待第二个锁存触发 PRINT REG_POSB '打印触发时位置
相关指令	REGIST , MARKB

REG_POSC -- 锁存 3 位置

类型	轴状态
描述	返回锁存寄存器 3 的测量反馈位置(MPOS), units 单位。 锁存位置为编码器反馈位置, 锁存位置要及时读取出来保存, 当编码器坐标继续变化后, REG_POSC 会变无效。锁存位置存在范围限制, 当编码器旋转超过该范围,

	REG_POSC 会根据运动方向增加或减少对应范围值，范围根据不同控制器分为 2^{16} 、 2^{32} 。
语法	VAR1 = REG_POSC(轴号)
适用控制器	通用
例子	MOVE(100) '运动 100units REGIST(18) '上升沿信号触发 WAIT UNTIL MARKC '等待第二个锁存触发 PRINT REG_POSC '打印触发时位置
相关指令	REGIST , MARKC

REG_POSD -- 锁存 4 位置

类型	轴状态
描述	返回锁存寄存器 4 的测量反馈位置(MPOS)，units 单位。 锁存位置为编码器反馈位置，锁存位置要及时读取出来保存，当编码器坐标继续变化后，REG_POSD 会变无效。锁存位置存在范围限制，当编码器旋转超过该范围，REG_POSD 会根据运动方向增加或减少对应范围值，范围根据不同控制器分为 2^{16} 、 2^{32} 。
语法	VAR1 = REG_POSD(轴号)
适用控制器	通用
例子	MOVE(100) '运动 100units REGIST(20) '上升沿信号触发 WAIT UNTIL MARKD '等待第二个锁存触发 PRINT REG_POSD '打印触发时位置
相关指令	REGIST , MARKD

11.12 限位参数指令

FS_LIMIT -- 正向软限位设置

类型	轴参数
描述	正向软限位位置，单位是 units。 当 FS_LIMIT 大于 REP_DIST 时，参数不起作用，正向软限位被禁止。 取消软限位时，建议不要去修改 REP_DIST 的值，将 FS_LIMIT 设置一个较大值即可。 FS_LIMIT 的值默认为 200000000。 软限位无法作为 DATUM 回零时的限位信号参考。
语法	VAR1 =FS_LIMIT, FS_LIMIT = expression VAR1 =FS_LIMIT(axisnum), FS_LIMIT(axisnum) = expression
适用控制器	通用
例子	BASE(0) '选择轴 0

	ATYPE=1 '轴类型设置 UNITS=100 '脉冲当量 100 DPOS=0 '坐标清 0 SPEED=100 '速度 100units/s ACCEL=1000 '加速度 1000units/s/s FS_LIMIT=200 '设置正向软限位 200units MOVE(300) '运动 300units 此时轴运动到 200 位置时会停止，并报错 Axis:0 AXISSTATUS:200h,FSOFT 继续操作轴时只能向负向运行。 取消设置时，只需要把值设大些。 FS_LIMIT=2000000 '取消正向软限位设置
相关指令	RS_LIMIT , FWD_IN , REV_IN

RS_LIMIT -- 负向软限位设置

类型	轴参数
描述	负向软限位位置，单位是 units 。 当 RS_LIMIT 的绝对值大于 REP_DIST 时，参数不起作用，负向软限位被禁止。 取消软限位时，建议不要去修改 REP_DIST 的值，将 RS_LIMIT 设置一个较大值即可。 RS_LIMIT 的值默认为-200000000。 软限位无法作为 DATUM 回零时的限位信号参考。
语法	VAR1 =RS_LIMIT, RS_LIMIT = expression VAR1 =RS_LIMIT(axisnum), RS_LIMIT(axisnum) = expression
适用控制器	通用
例子	RS_LIMIT = -50 '设置负向软限位 50units RS_LIMIT= - 2000000 '取消负向软限位设置
相关指令	FS_LIMIT , FWD_IN , REV_IN

FWD_IN -- 映射正限位输入

类型	轴参数
描述	正向硬件限位开关对应的输入点编号，-1 无效。 控制器限位信号生效后，会立即停止轴，停止减速度为 FASTDEC。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平(ECI 系列控制器相反)。
语法	VAR1 = FWD_IN, FWD_IN = expression VAR1 = FWD_IN(axisnum), FWD_IN(axisnum) = expression
适用控制器	通用
例子	BASE(0,1,2,3) '选择轴 0,1,2,3 FWD_IN=6,7,8,9 '分别设置正向限位开关

	INVERT_IN(6,ON) '反转信号 INVERT_IN(7,ON) INVERT_IN(8,ON) INVERT_IN(9,ON) 当开关输入信号时，对应轴的轴状态会报警 Axis:0 AXISSTATUS:10h,FWD。 此时只可以负向运动。
相关指令	REV_IN ， FS_LIMIT ， FASTDEC

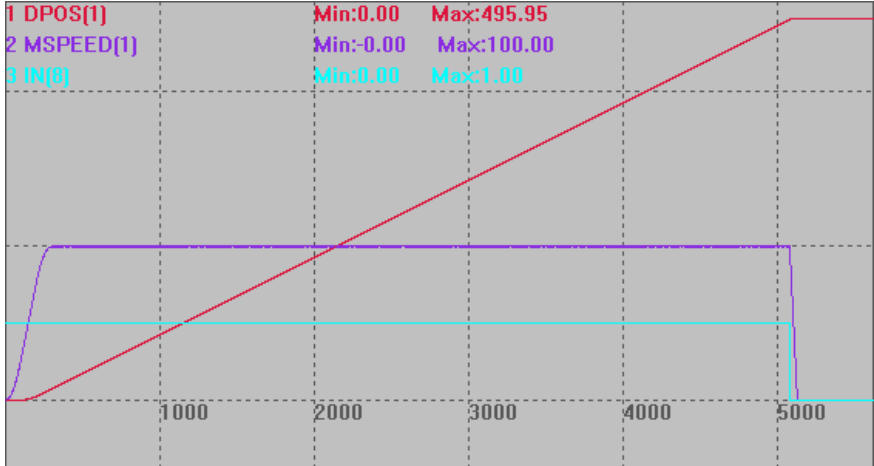
REV_IN -- 映射负限位输入

类型	轴参数
描述	负向硬件限位开关对应的输入点编号，-1 无效。 控制器限位信号生效后，会立即停止轴，停止减速度为 FASTDEC。 输入 OFF 时，认为有信号输入，要相反效果可以用 INVERT_IN 反转电平(ECI 系列控制器相反)。
语法	VAR1 = REV_IN, REV_IN = expression VAR1 = REV_IN(axisnum), REV_IN (axisnum)= expression
适用控制器	通用
例子	参考 FWD_IN 例子
相关指令	FWD_IN ， RS_LIMIT ， FASTDEC

ALM_IN -- 映射报警输入

类型	轴参数
描述	驱动器告警对应的输入口编号，-1 无效。 控制器报警信号生效后，会立即停止轴，停止减速度为 FASTDEC。 设置了告警输入口后，ZMC 控制器缺省为 OFF 有效，常开信号用 INVERT_IN 反转电平；ECI 控制器缺省为 ON 有效，常闭信号用 INVERT_IN 反转电平。
语法	VAR1 = ALM_IN, ALM_IN = expression VAR1 = ALM_IN(axisnum), ALM_IN(axisnum)= expression
适用控制器	通用
例子	BASE(0,1) ALM_IN = 10,11 '将轴 0 告警信号定义到输入口 10，轴 1 定义到 11 INVERT_IN(10,ON) '反转电平开启 INVERT_IN(11,ON)
相关指令	DATUM_IN ， FWD_IN ， REV_IN ， INVERT_IN ， FASTDEC

AXISEMG_IN -- 急停信号输入

类型	轴参数
描述	轴急停信号输入口的配置，缺省-1，不使用急停信号。 控制器急停信号生效后，会立即停止轴，停止减速度为 FASTDEC。 设置了急停输入口后，ZMC 控制器缺省为 OFF 有效，常开信号用 INVERT_IN 反转电平；ECI 控制器缺省为 ON 有效，常闭信号用 INVERT_IN 反转电平。
语法	VAR1 = AXISEMG_IN, AXISEMG_IN = expression VAR1 = AXISEMG_IN (axisnum), AXISEMG_IN(axisnum)= expression
适用控制器	通用，4 系列及以上最新固件。
例子	BASE(1) DPOS=0 UNITS=1000 SPEED=100 '速度设为 100 ACCEL=500 DECEL=500 '减速度 500 FASTDEC=2000 '快减减速度 2000 TRIGGER '自动触发示波器 ATYPE(1)=1 AXISEMG_IN(1)= 8 '输入口 8 设置为轴 1 的急停信号 INVERT_IN(8,ON) '信号反转 MOVE(1000) AXIS(1) 急停信号生效，轴 1 急停。 
相关指令	ALM_IN , FWD_IN , REV_IN , INVERT_IN , FASTDEC

11.13 到位参数指令

IN_POS -- 到位标志

类型	轴参数
描述	读取轴是否到位，轴停止后，返回值-1 到位，返回值 0 未到位。 必须配置好 IN_POS_DIST 与 IN_POS_SPEED，或是配置 AXISINP_IN。 不使用到位功能时，直接使用 IDLE 来判断运动结束。
语法	VAR1 = AXISEMG_IN, AXISEMG_IN = expression VAR1 = AXISEMG_IN (axisnum), AXISEMG_IN(axisnum)= expression
适用控制器	通用，4 系列及以上最新固件。
例子	<pre> BASE(1) DPOS=0 UNITS=1000 SPEED=100 '速度设为 100 ACCEL=500 DECEL=500 '减速度 500 FASTDEC=2000 '快减减速度 2000 AXISINP_IN(1) = 0 '输入口 0 设置为轴 1 的到位信号 INVERT_IN(0,ON) '信号反转 MOVE(1000) AXIS(1) ?IN_POS(1) '打印值：0，未到位 WAIT UNTIL IDLE(1) '等待轴 1 停止 ?IN_POS(1) '到位信号触发，打印值：-1，到位 </pre>
相关指令	IN_POS_DIST ， IN_POS_SPEED ， AXISINP_IN

AXISINP_IN -- 到位信号输入

类型	轴参数
描述	轴到位信号输入口的配置，缺省-1，不使用到位信号。 控制器到位信号生效后，并且停止轴之后，到位标志生效。 设置了到位输入口后，ZMC 控制器缺省为 OFF 有效，常开信号用 INVERT_IN 反转电平；ECI 控制器缺省为 ON 有效，常闭信号用 INVERT_IN 反转电平。
语法	VAR1 = AXISINP_IN, AXISINP_IN = expression VAR1 = AXISINP_IN (axisnum), AXISINP_IN (axisnum)= expression
适用控制器	通用，4 系列及以上最新固件。
例子	<pre> BASE(0) '选择轴 0 ATYPE=1 UNITS=100 '脉冲当量 100 DPOS=0 '坐标清 0 </pre>

	SPEED=100 '速度 100units/s ACCEL=1000 '加速度 1000units/s/s DECEL=1000 '加速度 1000units/s/s AXISINP_IN(0) = 0 '输入口 0 设置为轴 0 的到位信号 INVERT_IN(0,ON) MOVE(10000) AXIS(0) TRIGGER ?IN_POS(0) WAIT UNTIL IDLE(0) '停止后到位信号变化 ?IN_POS(0)
相关指令	IN_POS_DIST , IN_POS , IN_POS_SPEED , INVERT_IN

IN_POS_DIST -- 到位距离

类型	轴参数
描述	到位距离设置，FE 小于此距离，并且 MSPEED 小于 IN_POS_SPEED，认为到位了。 参数范围从 0 开始；此参数和到位信号一起使用时，到位标志由到位信号控制。
语法	VAR1 = IN_POS_DIST, IN_POS_DIST = expression VAR1 = IN_POS_DIST (axisnum), IN_POS_DIST (axisnum)= expression
适用控制器	通用，4 系列及以上最新固件。
例子	<pre> BASE(0) ATYPE=4 '带编码器反馈 UNITS=1000 SPEED=100 '速度设为 100 ACCEL=1000 DECEL=1000 '减速度 500 FASTDEC=10000 '快减速度 2000 DPOS=0 AXISINP_IN(0) = -1 '取消到位信号 IN_POS_DIST = 0.5 '到位距离 IN_POS_SPEED =0.5 '到位速度 MOVE(100) DELAY(100) ?FE ?MSPEED ?IN_POS '打印到位标志 WAIT UNTIL IDLE(0) DELAY(100) ?"-----" ?FE ?MSPEED </pre>

	?IN_POS
相关指令	IN_POS_SPEED , IN_POS , AXISINP_IN

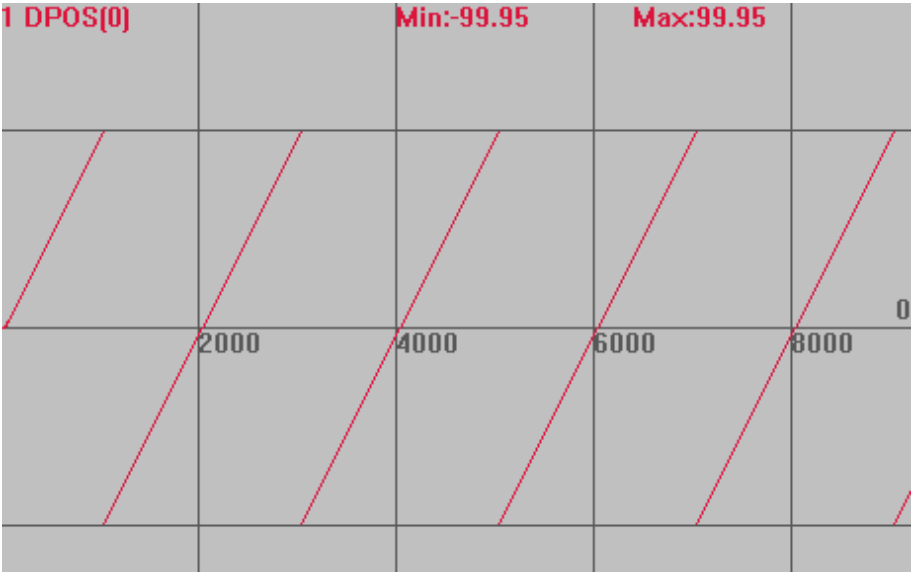
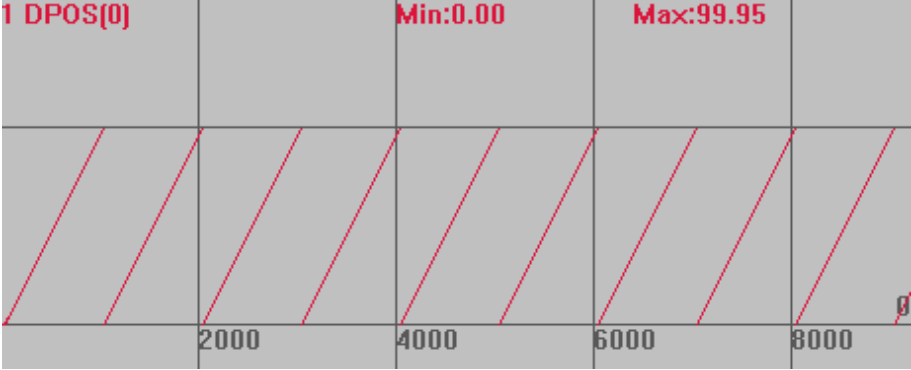
IN_POS_SPEED -- 到位速度

类型	轴参数
描述	到位速度设置，MSPEED 小于此速度，并且 FE 小于 IN_POS_DIST，认为到位了。 参数范围从 0 开始；此参数和到位信号一起使用时，到位标志由到位信号控制。
语法	VAR1 = IN_POS_SPEED, IN_POS_SPEED = expression VAR1 = IN_POS_SPEED (axisnum), IN_POS_SPEED (axisnum)= expression
适用控制器	通用，4 系列及以上最新固件。
例子	<pre> BASE(0) ATYPE=4 '带编码器反馈 UNITS=1000 SPEED=100 '速度设为 100 ACCEL=1000 DECEL=1000 '减速度 500 FASTDEC=10000 '快减减速度 2000 DPOS=0 AXISINP_IN(0) = -1 '取消到位信号 IN_POS_DIST = 0.5 '到位距离 IN_POS_SPEED =0.5 MOVE(100) DELAY(100) ?FE ?MSPEED ?IN_POS '打印到位标志 WAIT UNTIL IDLE(0) DELAY(100) ?"-----" ?FE ?MSPEED ?IN_POS </pre>
相关指令	IN_POS_DIST , IN_POS , AXISINP_IN

11.14 限幅参数指令

REP_OPTION -- 坐标循环模式

类型	轴参数		
描述	坐标重复设置。		
	可用于限制凸轮主轴的坐标循环范围，来实现多条凸轮曲线连续。 使用绝对运动模式时，如果目标位置处于坐标循环范围内，可以正确运动；如果处于坐标循环范围外，运动不正确。 相对运动不受影响。 REP_DIST 超过参数支持的最大数值，就会把 REP_OPTION 的 bit4 置 1。		
语法	VAR1 = REP_OPTION, REP_OPTION = opt		
	opt: 按位来表示不同的意义		
	位	值	描述
	0	1	0-循环范围 - REP_DIST 到 +REP_DIST 1-循环范围为 0 到 +REP_DIST
	1	2	写 1 禁止 CAMBOX 和 MOVELINK 的重复运动，禁止生效后还原为 0
	2	4	预留
	4	16	1-不使用 REP_DIST, 0-使用 REP_DIST
适用控制器	通用		
例子	BASE(0)	'选择轴 0	
	ATYPE=1		
	UNITS=100	'脉冲当量 100	
	DPOS=0	'坐标清 0	
	SPEED=100	'速度 100units/s	
	ACCEL=1000	'加速度 1000units/s/s	
	DECEL=1000	'加速度 1000units/s/s	
	REP_DIST=100	'设置坐标循环范围	
	REP_OPTION=0	'设置循环模式	
	TRIGGER	'自动触发示波器	
	VMOVE(1)	'持续运动	
		坐标曲线	
	DPOS(0)垂直刻度 100		

					
	REP_OPTION=1 时 MSPEED(0)垂直刻度 100				
					
相关指令	CAMBOX , MOVELINK , REP_DIST				

REP_DIST -- 坐标循环位置

类型	轴参数
描述	根据 REP_OPTION 设置来自动循环轴 DPOS 和 MPOS 坐标。
语法	VAR1 = REP_DIST, REP_DIST = expression
适用控制器	通用
例子	参考 REP_OPTION
相关指令	REP_OPTION

FE -- 当前随动误差

类型	轴轴参数
描述	随动误差，该值等于 DPOS-MPOS。
语法	可读：VAR1 = FE(轴号)

	可写: WAIT UNTIL ABS(FE(轴号)) <= var WAIT UNTIL ABS(FE(轴号)) <= var 指令中 FE 包含正负, 所以用 ABS 取绝对值, 一般都在 WAIT IDLE(轴号)后面使用
适用控制器	通用
相关指令	MPOS , DPOS

FE_LIMIT -- 最大随动误差设置

类型	轴参数
描述	最大允许的随动误差值, 缺省: 3。 当超过跟随误差, 产生实时误差, 使能继电器 (WDOG) 清零, 阻止其他发动机运行。这个限制通常用于防护缺省状态, 例如机械被锁住, 丢失编码器反馈, 等等。超过时告警, 通常 AXISSTATUS 报 2h/100h/102h。
语法	VAR1 = FE_LIMIT, FE_LIMIT = expression 适用: FE_LIMIT AXIS(axis)方式设置指定的轴
例子	参见 SERVO
相关指令	FE , SERVO , P_GAIN , D_GAIN , I_GAIN , OV_GAIN , VFF_GAIN , FE_RANGE

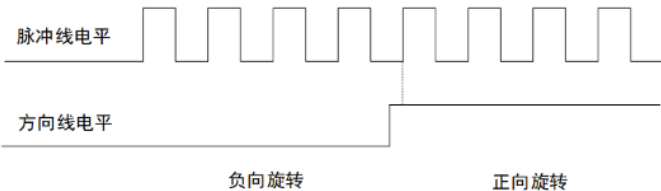
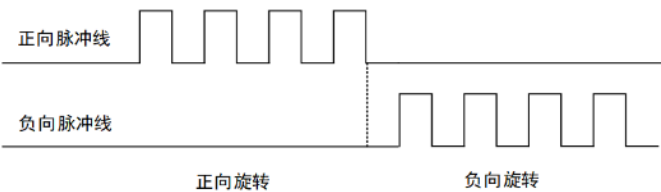
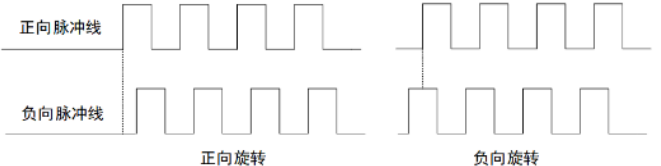
FE_RANGE -- 报警时随动误差

类型	轴参数
描述	报警时的随动误差值。
语法	VAR1 = FE_RANGE, FE_RANGE = expression 适用: FE_RANGE AXIS(axis)方式设置指定的轴
例子	参见 SERVO
相关指令	FE , SERVO , P_GAIN , D_GAIN , I_GAIN , OV_GAIN , VFF_GAIN , FE_LIMIT

11.15 高级设置指令

INVERT_STEP -- 脉冲模式设置

类型	轴参数
描述	伺服/步进脉冲输出模式设置。 有脉冲方向、双脉冲、正交脉冲三种模式, 控制器默认为脉冲方向控制 (模式 0)。正交脉冲目前只有部分控制器支持。 反馈位置 (MPOS) 信息牵涉到很多复杂功能, 例如 MOVE_OP 精准模式, 所以控制器暂不支持修改反馈位置的方向, 需要修改时可以修改驱动相关参数, 例如三菱为 PA14。

语法	INVERT_STEP = mode				
	mode: 模式选择, 缺省 0, 低 8 位 (位 0-位 7) 表示的模式值如下:				
	模 式 值	说明	参考示意图 (正逻辑模式)		
	0-3	脉冲方向模式 脉冲线+方向线			
	4-7	双脉冲方式(或称 CW/CCW) 正向脉冲线+负向脉冲线			
	8-9	AB 输出, 正交脉冲 (部分控制器定制)			
	各个模式对应的电平如下: 若极性对调, 参考运动方向与原来相反。				
	模式值	描述	松下设置参考		三菱设置参考
			Pr0.06	Pr0.07	PA13
	0	脉冲/方向 (脉冲正逻辑) (正向)	0	3	××01h
1	脉冲/方向 (脉冲负逻辑) (正向)	/	/	××11h	
2	脉冲/方向 (脉冲正逻辑) (负向)	1	3	××01h	
3	脉冲/方向 (脉冲负逻辑) (负向)	/	/	××11h	
4	双脉冲 (方向负逻辑) (正向)	/	/	××10h	
5	双脉冲 (方向负逻辑) (负向)	/	/	××10h	
6	双脉冲 (方向正逻辑) (正向)	1	1	××00h(默认)	
7	双脉冲 (方向正逻辑) (负向)	0(默认)	1(默认)	××00h(默认)	
高 8 位 (位 8-位 15) 表示方向变化保护时间, 单位微秒: 0-255					
常用模式为 0、2、6、7。					
如果模式设定不正确, 步进马达可能会在换向时丢失一步的位置, 当不确定步进马达的设置时, 可以设置 100 微秒左右的保护时间。					
适用控制器	通用				
例子	设置为脉冲方向模式: INVERT_STEP = 256*100+0 '设置 100 微秒的保护时间, 模式为 0 设置为双脉冲模式: INVERT_STEP = 256*100+6 '设置 100 微秒的保护时间, 模式为 6 查询脉冲模式设置:				

	在线命令栏输入如下指令查询。 ?INVERT_STEP(0) '打印轴 0 的脉冲模式设置值 ?*INVERT_STEP '打印全部轴的脉冲模式设置值
相关指令	STEP_RATIO

MAX_SPEED -- 脉冲频率限制

类型	轴参数
描述	脉冲输出的最高频率限制。 一旦发现超过此设置值会强制，并且设置 AXISSTATUS。 对编码器轴，设置值低于 500K 时会启用编码器滤波，设置值高于 1M 时会取消编码器滤波设置。默认值为 1000000（老固件的默认脉冲频率为 500000）。 使用直线电机速度较快时，一般容易脉冲频率超限，可把数值适当设置大点。
语法	MAX_SPEED = value
适用控制器	通用
例子	MAX_SPEED AXIS(n)=4000000 '设置轴 n 脉冲速度限制 4000000 BASE(6) ATYPE=3 MAX_SPEED =500000 '启动编码器滤波
相关指令	AXISSTATUS

AXIS_ZSET -- 精准 op 设置

类型	轴参数
描述	对轴启用 MOVEOP 精准输出功能，作用在轴组的主轴上。 SYSTEM_ZSET 修改的同时会修改当前 BASE 轴的 AXIS_ZSET，以兼容旧的程序，一般不建议使用 SYSTEM_ZSET 指令。 设置参数： bit1: 1-使用 MOVE_OP 精确输出功能，0- MOVE_OP 原来的方式 bit4: 1-对带编码器功能的轴，使用编码器位置的 MOVE_OP 精准方式 多个编码器轴插补时，使用 BASE 运动主轴的设置 bit5: 1-CANCEL(2)/RAPIDSTOP(2)急停减速度为 DECEL，0-急停减速度为 FASTDEC bit6: 设置在被叠加轴上，是否使能此功能。1-使能、0-不使能
语法	可读：VALUE=AXIS_ZSET 可写：AXIS_ZSET=VALUE
适用控制器	20170517 以上固件版本
例程	例一 开启精准输出 BASE(0) ATYPE=1 DPOS=0

```

SPEED=100
ACCEL=1000
DECEL=1000
AXIS_ZSET(0)=2  '开启 MOVE_OP 的精准输出功能
MOVE(100)
MOVE_OP(0,1)    '精准生效，选择输出通道 0

```

例二 开启多个编码器精准输出口

一般 4 系列有 4 个通道可以用于精准输出，部分型号有 8 个，假设设备上有 3 个点胶工位都要精准输出。

```

BASE(0,1,2)      '选择轴 0 为主轴
AXIS_ZSET(0)=19  '开启主轴 MOVE_OP 的编码器精准输出功能
.....
BASE(3,4,5)
AXIS_ZSET(3)=19
.....
BASE(6,7,8)
AXIS_ZSET(6)=19
.....

```

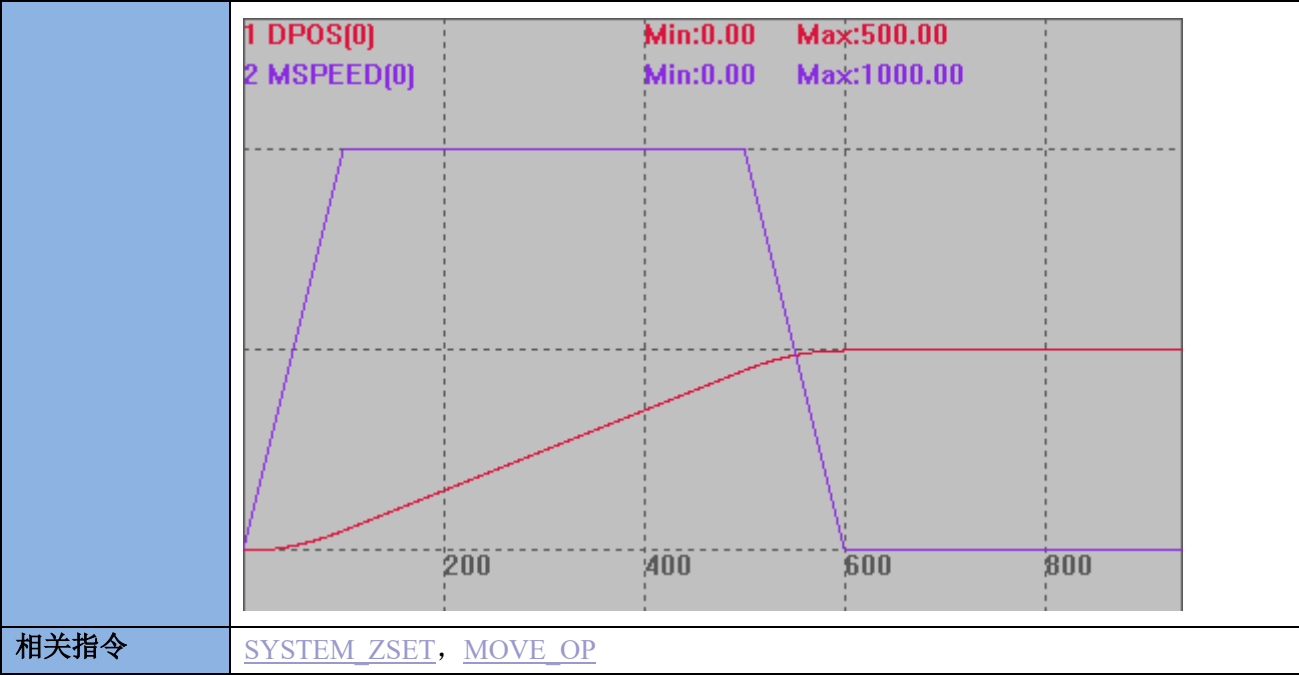
例二 急停减速度选择

```

BASE(0)
DPOS=0
ATYPE=1
UNITS=100
SPEED=1000
ACCEL=10000
DECEL=10000      '减速度设为 1000
FASTDEC=100000   '快减减速度设为 100000
AXIS_ZSET=32     '减速度选择
TRIGGER          '自动触发示波器
MOVE(1000)       '当前运动
MOVE(-2000)      '缓冲运动
DELAY(500)
CANCEL(2)        '急停

```

如下图，不设置 AXIS_ZSET 的减速度应为 100000，设置后减速度为 10000



AXIS_MODE -- connect 运动保持

类型	轴参数
描述	设置 BIT1=1，防止限位和软限位导致 CONNECT 运动退出。 BIT1=0，当从轴碰到限位时，主从轴的 CONNECT 连接关系断开，消除限位报警后，此时操作主轴，从轴不再跟随； BIT1=1，碰到限位后仍然保持连接关系，消除报警后，从轴仍然保持关联。 BIT5 =0，缺省设置，主轴带编码器时优先跟踪 MPOS。 BIT5=1，设置在齿轮或凸轮运动的主轴上，强制指定跟踪主轴的 DPOS，涉及的指令：CAMBOX、CONNECT、MOVELINK、MOVESLINK、MOVESYNC、HW_PSWITCH2。 当速度超过目标速度，加速度超过目标加速度 2 倍时，跟踪滞后很小。4 系列以上，version_build:240226 增加。 注意：硬限位在运动过程中触发且限位方向与运动方向一致会导致同步断开 固件版本 20170616 及以上支持此功能。
语法	VAR1 = AXIS_MODE, AXIS_MODE = expression
例程	例一：不设置 AXIS_MODE 参数 RAPIDSTOP(2) WAIT IDLE BASE(0,1) ATYPE=1,1 UNITS=100,100 DPOS=0,0 SPEED=100,100

```

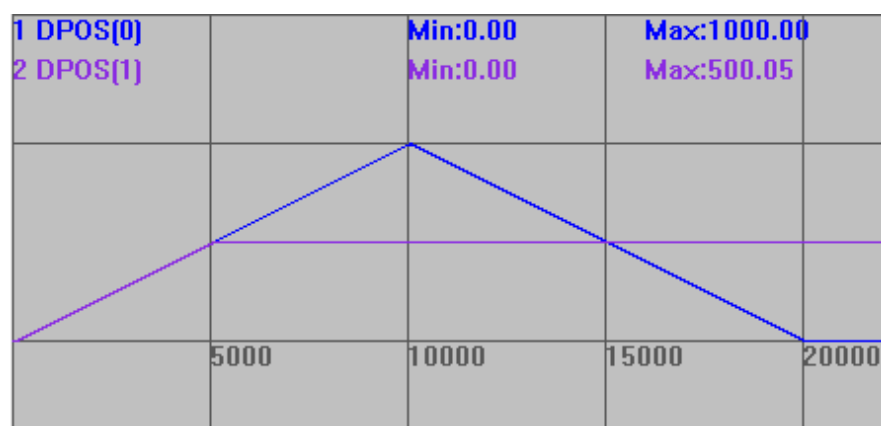
ACCEL=1000,1000
DECEL=1000,1000
AXIS_MODE=0,0          '设置参数
FS_LIMIT=1000,500
TRIGGER                 '自动触发示波器
CONNECT(1,0) AXIS(1)    '轴 1 连接到轴 0，比例为 1
MOVE(1000) AXIS(0)
MOVE(-1000) AXIS(0)

```

运动轨迹如下：

DPOS(0)垂直刻度 1000，无偏移

DPOS(1)垂直刻度 1000，无偏移



轴 1 碰到限位后停止，和轴 0 的 connect 连接断开，后续轴 0 的运动与轴 1 无关。

例二：设置 AXIS_MODE 参数

```

RAPIDSTOP(2)
WAIT IDLE
BASE(0,1)
ATYPE=1,1
UNITS=100,100
DPOS=0,0
SPEED=100,100
ACCEL=1000,1000
DECEL=1000,1000
AXIS_MODE=0,2
FS_LIMIT=1000,500
TRIGGER                 '自动触发示波器
CONNECT(1,0) AXIS(1)    '轴 1 连接到轴 0，比例为 1
MOVE(1000) AXIS(0)
MOVE(-1000) AXIS(0)

```

运动轨迹如下：

DPOS(0)垂直刻度 1000，无偏移

DPOS(1)垂直刻度 1000，无偏移

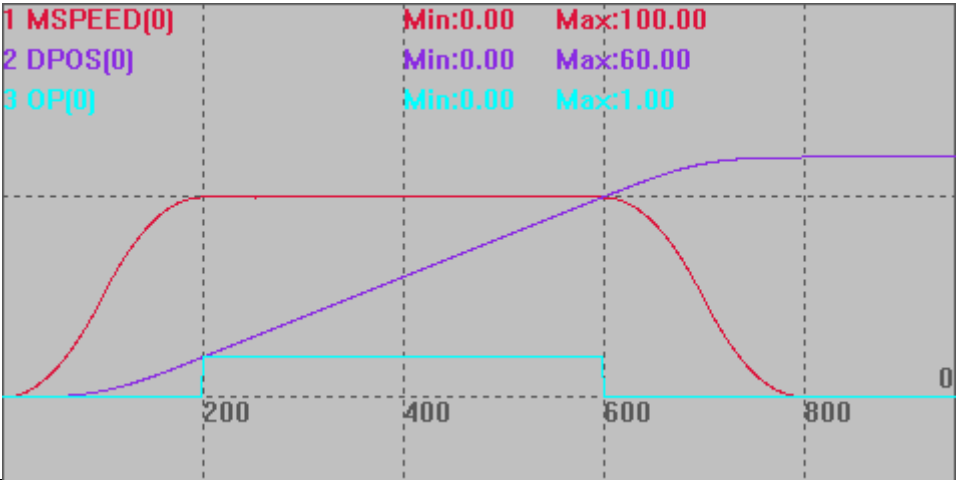
	1 DPOS[0] 2 DPOS[1] Min:0.00 Min:-499.95 Max:1000.00 Max:500.05			
	轴 1 碰到限位后停止，但和轴 0 的 connect 并未断开，运动到限位位置内，跟随轴 0 运动。			
适用控制器	通用			
相关指令	CONNECT ， FS_LIMIT ， RS_LIMIT			

MOVEOP_DELAY -- 缓冲输出延时

类型	轴参数
描述	BASE 轴缓冲信号延时输出。 设置在 BASE 主轴上，当 MOVE_OP 精准功能使用时，可以调整实际触发 OP 操作的时间，毫秒(ms)单位，支持小数，实际最长延时时间 100ms。 设置为负数，可以提前打开 OP，一般步进可以提前 2ms，伺服可以提前 20ms，点胶起胶可以使用。 验证功能推荐使用锁存验证，指令的效果受到轴 FE 的影响，若要忽略该影响，验证延后功能，可以把 ATYPE 设置成 0 或者 1 去测试；提前功能是通过利用伺服滞后实现的，所以提前输出功能的验证时必须带有编码器反馈的实际驱动器上去验证（总线轴也可以）。 moveop_delay 不会手动清零，不清零的话会和 moveop_adist 和新的 moveop_aheadms 共同作用
语法	MOVEOP_DELAY=timems timems: 毫秒数
适用控制器	20170505 以上固件版本
例程	MOVEOP_DELAY = 2 '实际输出时间延迟 2ms
相关指令	MOVE_OP ， HW_PSWITCH ， HW_PSWITCH2

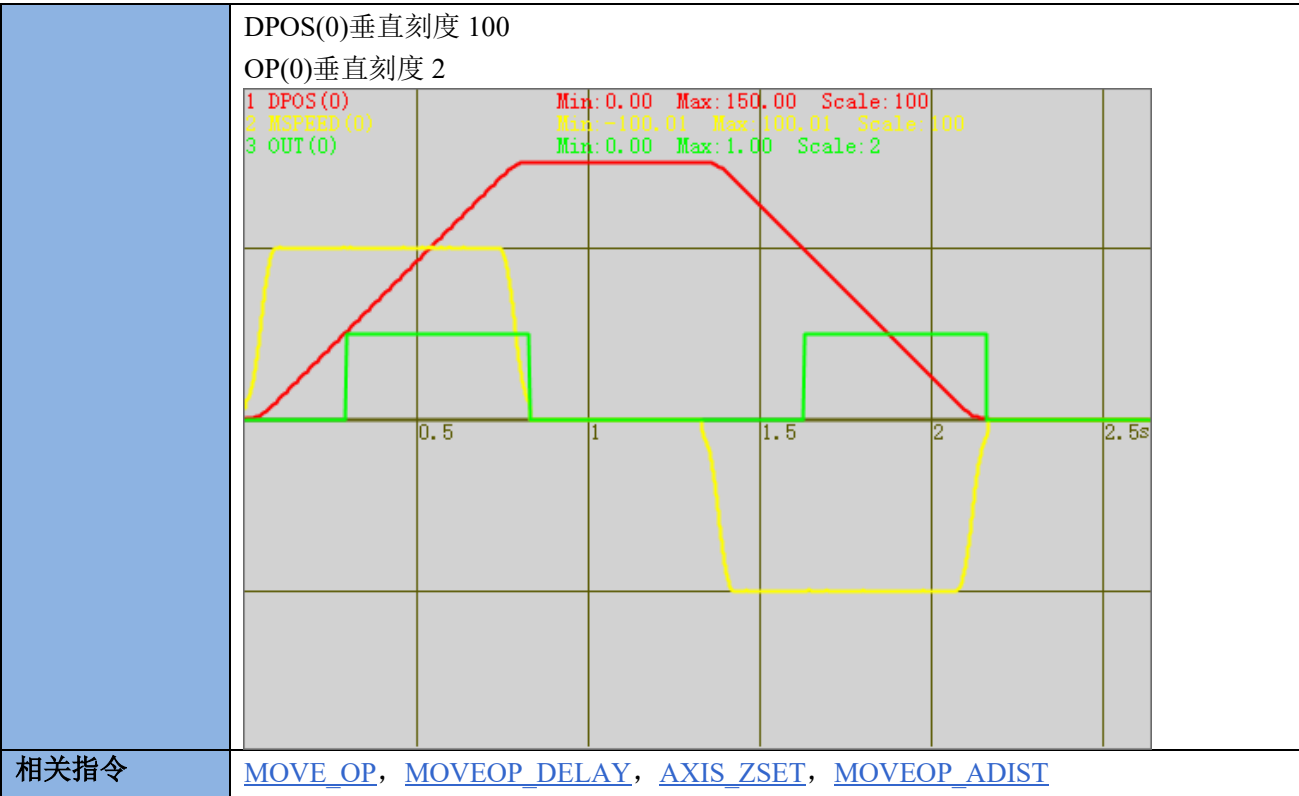
MOVEOP_ADIST -- 提前关胶

类型	轴参数
描述	MOVE_OP 提前一定距离输出设置，缺省 0 不生效，设置正数会比缺省输出位置提前指定矢量距离，设置负数则延后指定矢量距离。

	MOVEOP_ADIST 指令不需要打开精准输出就能生效。 只能对 MOVE_OP 单个 OP 操作有效，对每个轴来说，操作的动作是顺序的，第一个改变位置的 MOVE_OP 操作没有完成，后面的不会操作 MOVEOP_ADIST 带入缓冲后会自动清零，避免影响其它 MOVE_OP。 对支持 HW 功能的控制器，可以由 AXIS_ZSET 来启动 HW 精准输出，同 MOVE_OP 精准，此指令只支持 MOVE 插补类与 MOVESCAN 运动，凸轮、MOVE_PT 等不支持，支持 MOVE 指令跨小线段提前输出。
语法	MOVEOP_ADIST=distance distance: 提前/延后关胶距离
适用控制器	4 系列控制器，fast 固件 190201 以后固件支持
例子	BASE(0) ATYPE=1 UNITS=100 DPOS=0 MPOS=0 SPEED=100 ACCEL=1000 DECEL=1000 MERGE=1 SRAMP=100 TRIGGER MOVEOP_ADIST = -10 '延后 10 个单位距离开胶 MOVE_OP(0,1) MOVE(5) MOVE(50) '实际的开胶,关胶在这个运动 MOVE(5) MOVEOP_ADIST = 10 '提前 10 个单位距离关胶 MOVE_OP(0,0) END 运动曲线 MSPEED(0)垂直刻度 100 DPOS(0)垂直刻度 50 OP(0)垂直刻度 5 
相关指令	MOVE_OP , MOVEOP_DELAY , AXIS_ZSET

MOVEOP_AHEADMS -- 提前关胶时间

类型	轴参数
描述	MOVE_OP 提前一定毫秒单位时间输出, 缺省 0 不生效, 设置正数会比缺省输出位置提前, 设置负数则延后. !这个效果同 MOVEOP_ADIST, MOVE_OP 后自动清零, 不建议使用延后. !输出点时刻匀速运动, 或正在减速时效果更好, 否则可能不准. !提前时间长度不建议超过减速时间. !如果延后, 只支持匀速运动, 建议使用 MOVEOP_DELAY, 效果更好. !不建议与 MOVEOP_ADIST 同时设置.
语法	MOVEOP_AHEADMS = timesms timesms: 时间,单位 ms
适用控制器	4 系列以上控制器, version_build: 20241125 以后固件支持
例子	BASE(0) ATYPE=1 UNITS=100 DPOS=0 MPOS=0 SPEED=100 ACCEL=1000 DECEL=1000 MERGE=1 AXIS_ZSET(0) = 19 SRAMP=100 DELAY(500) TRIGGER MOVE(50) MOVEOP_AHEADMS = -10 MOVE_OP(0,1) MOVE(50) MOVE(50) MOVEOP_AHEADMS = 10 '单位 ms MOVE_OP(0,0) MOVE_DELAY(1000) MOVE(-50) MOVEOP_AHEADMS = -10 MOVE_OP(0,1) MOVE(-50) MOVE(-50) MOVEOP_AHEADMS = 10 '单位 ms MOVE_OP(0,0) MOVE_DELAY(1000) 运动曲线 MSPEED(0)垂直刻度 100



DAC -- 总线轴模拟量控制

类型	轴参数
描述	伺服轴 DA 直接控制，速度或力矩模式下支持。 单位为 DA 模块的刻度，12 位或 16 位。 速度控制时要看驱动器具体单位。 力矩控制时单位为千分之一，等于 1000 时表示 100%力矩。
语法	VAR1 = DAC, DAC = expression
使用控制器	带 EtherCAT 接口或 Rtex 接口，2017 以后固件支持
例程	例一 Rtex 速度控制 请先使用 Rtex 初始化例程，并将例程中的 ATYPE 设置为 51。 然后可在 RTSys 在线命令直接发送 dac 指令，如下图 命令与输出 >>dac=10 在线命令: dac=10 发送 捕获 清除 命令与输出 查找结果 此时电机以 10r/min 的速度旋转，发送 dac= - 10 会反向旋转。 也可以在程序中发送 dac 指令。 速度单位根据驱动器手册确认，如下：

指令速度

[大小]: 带符号 32bit

[单位]: 通过 Pr7.25 (RTEX 速度单位) 设定

Pr7.25	单位
0	[r/min]
1	[指令单位/s]

[设定单位]: 负向最大过速度等级~正向最大过速度等级

r/min 单位下设定时, 在内部演算时换算到指令单位/s, 换算后的值限制在以下范围: -80000001h~7FFFFFFFh

命令与输出

>>drive_read(7*256+25)
0

在线命令: drive_read(7*256+25) 发送 捕获 清除

此时单位为【r/min】

例二 Rtex 力矩控制

请先将驱动器参数 pr6.47 的第一位置为 0, 关闭 2 自由度控制模式; 参数 pr3.17 设置速度限制, 如下图。(参照松下 Rtex 手册)

Pr3.17(速度限制选择)的设定值是 1 时, 可以通过 SL_SW 切换转矩控制时的速度限制值。

分类	3	3	3											
No.	17	21	22											
属性	B	B	B											
参数名称	速度限制选择	速度限制值 1	速度限制值 2											
设定范围	0~1	0~20000	0~20000											
单位	/	r/min	r/min											
功能	<table border="1"><tr><td rowspan="2">值</td><td colspan="2">SL_SW</td></tr><tr><td>0</td><td>1</td></tr><tr><td>0</td><td colspan="2">Pr3.21</td></tr><tr><td>1</td><td>Pr3.21</td><td>Pr3.22</td></tr></table> 设定转矩控制时的速度限制值的选择方式	值	SL_SW		0	1	0	Pr3.21		1	Pr3.21	Pr3.22	设定转矩控制时的速度限制；转矩控制中在不超过速度限制值设定的速度内进行控制，另外，内部值被 Pr5.13(过速度等级)、Pr6.15（第二过速度等级）、Pr9.10（最大过速度等级）的最小设定速度进行限制	Pr3.17(速度限制选择)=1 时，设定 SL_SW 为 1 时的速度限制，另外，内部值被 Pr5.13（过速度等级）、Pr6.15（第二过速度等级）、Pr9.10（最大过速度等级）的最小设定速度进行限制
值	SL_SW													
	0	1												
0	Pr3.21													
1	Pr3.21	Pr3.22												

再使用 [Rtex 初始化例程](#), 并将例程中的 ATYPE 设置为 52。
然后可在 RTSys 在线命令直接发送 dac 指令, 此时电机开始转动。

命令与输出

>>dac=30

在线命令: dac=30 发送 捕获 清除

当前驱动器为 0.03 的力矩，如果 dac 发送过小时，电机无法克服摩擦力，不能转动。

也可以在程序中发送 dac 指令。

力矩控制时 dac 的单位为千分之一的力矩，dac=1000 是表示 100%

[大小]: 带符号 32bit

[单位]: 0.1%

[设定范围]: 负向电机最大转矩~正向电机最大转矩

最大转矩限制[%]= $100 \times \text{Pr9.07} / (\text{Pr9.06} \times 2\frac{1}{2})$

例三 EtherCAT 速度控制

FOR I=0 TO 1 '初次使用将所有轴设为普通脉冲类型

ATYPE(I)=1

NEXT

SLOT_SCAN(0) '总线扫描开始

IF NODE_COUNT(0,0)>0 THEN

AXIS_ADDRESS(0)=1 '将第一个驱动器映射到轴 0

ATYPE(0)=66 '66 为速度控制模式

DRIVE_PROFILE(0)=20 '速度控制要设置为 20

DELAY (200)

SLOT_START(0) '总线扫描成功，启动总线

DRIVE_CONTROLWORD(0)=128 '清除驱动器错误

DELAY (2)

DRIVE_CONTROLWORD(0)=6

DELAY (2)

DRIVE_CONTROLWORD(0)=15

DELAY (2)

DELAY(20)

DATUM(0) '清除控制器错误

BASE(0)

AXIS_ENABLE=1 '映射轴使能打开

WDOG=1 '轴使能

DAC=10000 '电机以 10000 脉冲每秒的速度转动

ENDIF

例四 EtherCAT 力矩控制

将例三中的 ATYPE 改为 67，DRIVE_PROFILE 改为 30 即可。

此时 dac 发送范围 0~1000,1000 表示 100%力矩，反向选择发送负值即可。

相关指令

[SERVO](#)

ERRORMASK -- 错误时操作

类型	轴参数
描述	和 AXISSTATUS 做与运算来决定哪些错误需要关闭 WDOG 。
语法	VAR1 = ERRORMASK, ERRORMASK = expression
适用控制器	通用
例程	<pre> BASE(0) WDOG=1 '开使能 ?AXISSTATUS '打印 16, 发生正向硬限位报警 ERRORMASK=16 DELAY(10) '运算延时 ?WDOG '打印 0, 使能被关闭 </pre>
相关指令	AXISSTATUS , WDOG

ZSCAN_CORRECT -- 振镜矫正

类型	轴参数
描述	矫正振镜轴参数。
语法	ZSCAN_CORRECT(axy,imode,imaxline,imaxrow,x1,y1,x2,y2,tableindex) axy: 值为 0 或 1, 两个振镜选择; 0-第一个振镜, 1-第二个振镜 imode: 0-关闭矫正功能; 1-使用分区矫正, table 输入测量得到的实际位置; 2-使用分区矫正, table 输入需要运行到的脉冲位置, 210701 增加此功能 imaxline: 行数, Y 方向的点数为行数, 数据越多精度越高 imaxrow: 列数, X 方向的点数为列数 x1,y1: 理论的左下角的位置 x2,y2: 理论的右上角的位置 tableindex: 测量的实际坐标开始存储的 table 索引, 先 X 再 Y, 先第一行(按列数存储), 再下一行, 注意此 XY 是实际的物理轴, 第一个物理轴为 X, 第二个为 Y, 与轴号映射的虚拟轴轴号无关, 填入坐标为振镜实际的脉冲位置(支持小数)(XY2 协议坐标范围是-32768-32767)
适用控制器	带振镜轴控制器
例子	<pre> TABLE(0,-40.6,-41.2) TABLE(2, 0,-41) TABLE(4, 41,-42) TABLE(6, -41,0) TABLE(8, 0,0) TABLE(10, 41.2,0) TABLE(12, -40.4,41.2) TABLE(14, 0,41.2) TABLE(16, 41,42.4) FOR i=0 TO 17 TABLE(i) = TABLE(i)*500 '输入的都是脉冲坐标 NEXT </pre>

	ZSCAN_CORRECT(0,1,3,3,-20000,-20000,20000,20000,0)
--	--

11.16 PID 相关指令

D_GAIN -- 微分增益

类型	轴参数
描述	微分增益，非模拟量伺服不支持。 D_GAIN 包括轴的微分增益。微分输出与跟随误差的变化量成正比，缺省：0。 给系统叠加微分增益会产生平滑响应，允许更大的比例增益，太高会产生震动。 注意：为了避免不稳定，伺服增益应该在 SERVO=OFF 时改变。
语法	VAR1 = D_GAIN, VAR1 = D_GAIN 适用：D_GAIN AXIS(axis)方式设置指定的轴
例子	参见 SERVO
相关指令	SERVO , P_GAIN , I_GAIN , OV_GAIN , VFF_GAIN , FE_LIMIT , FE_RANGE

I_GAIN -- 积分增益

类型	轴参数
描述	积分增益，非模拟量伺服不支持。 积分输出通过计算跟随误差的总和。缺省：0。 叠加积分增益到伺服系统减少静止或运动时的位置误差。可以减少超调，振动。 因此适用系统工作在常速和慢加速的场合。 注意：为了避免不稳定，伺服增益应该在 SERVO=OFF 时改变。
语法	VAR1 = I_GAIN, I_GAIN = expression 适用：I_GAIN AXIS(axis)方式设置指定的轴
例子	参见 SERVO
相关指令	SERVO , P_GAIN , D_GAIN , OV_GAIN , VFF_GAIN , FE_LIMIT , FE_RANGE

OV_GAIN -- 速度增益

类型	轴参数
描述	速度增益，非模拟量伺服不支持。 输出速度通过测量位置的改变与 OV_GAIN 参数的乘积得到。缺省：0。 在系统中增加输出速度增益与增加机械的阻尼等价。它能平滑输出并提高比例增益，但会产生大的跟随误差。输出增益太高会产生振动和大的跟随误差。 注意：为了避免不稳定，伺服增益应该在 SERVO=OFF 时改变。
语法	VAR1 = OV_GAIN, OV_GAIN = expression 适用：OV_GAIN AXIS(axis)方式设置指定的轴
例子	参见 SERVO

相关指令	SERVO , P_GAIN , D_GAIN , I_GAIN , VFF_GAIN , FE_LIMIT , FE_RANGE
------	---

P_GAIN -- 比例增益

类型	轴参数
描述	比例增益，非模拟量伺服不支持。 比例输出是跟随误差和 P_GAIN 参数的乘积。缺省：0 比例增益设置伺服响应的刚性，值太高会产生振动。值太低会产生大的跟随误差。 注意：为了避免不稳定，伺服增益应该在 SERVO=OFF 时改变。
语法	VAR1 = P_GAIN, P_GAIN = expression 适用：P_GAIN AXIS(axis)方式设置指定的轴
例子	参见 SERVO
相关指令	SERVO , OV_GAIN , D_GAIN , I_GAIN , VFF_GAIN , FE_LIMIT , FE_RANGE

VFF_GAIN -- 前馈增益

类型	轴参数
描述	速度反馈的前馈增益，非总线伺服不支持。 速度前馈是目标位置的变化与 VFF_GAIN 参数值的乘积，缺省：0。 $60B1\ pdo = \text{轴脉冲速度} * VFF_GAIN$ 闭环轴 PID 计算：轴脉冲速度 * VFF_GAIN 作为速度前馈 叠加速度前馈增益减少运动中的系统跟随误差，增加速度的输出比例。 注意：为了避免不稳定，伺服增益应该在 SERVO=OFF 时改变。
语法	VAR1 = VFF_GAIN, VFF_GAIN = expression 适用：VFF_GAIN AXIS(axis)方式设置指定的轴
例子	参见 SERVO
相关指令	SERVO , P_GAIN , D_GAIN , I_GAIN , OV_GAIN , FE_LIMIT , FE_RANGE , AFF_GAIN

AFF_GAIN -- 加速度前馈增益

类型	轴参数
描述	加速度反馈的前馈增益，非总线伺服不支持。 $60B2\ pdo = (\text{轴每个周期脉冲速度变化}) * AFF_GAIN$ 。
语法	VAR1 = AFF_GAIN, AFF_GAIN = expression 适用：AFF_GAIN AXIS(axis)方式设置指定的轴
例子	参见 SERVO
相关指令	SERVO , P_GAIN , D_GAIN , I_GAIN , OV_GAIN , FE_LIMIT , FE_RANGE , VFF_GAIN

SERVO -- 闭环开关

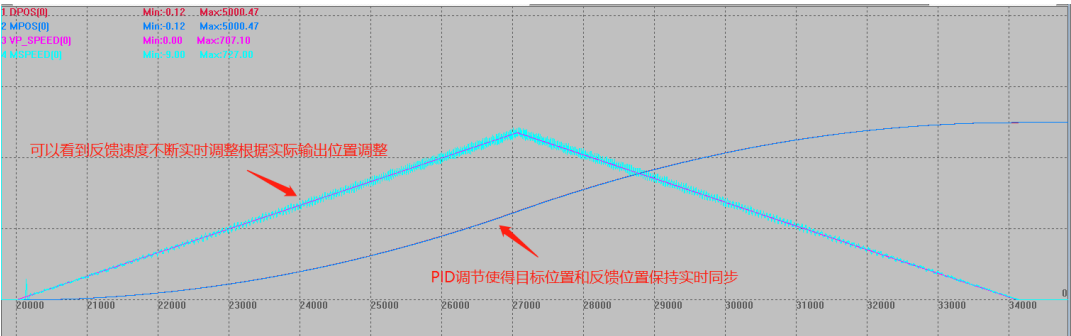
类型	轴参数
描述	闭环开关设置。 值范围：ON/OFF，缺省为 OFF。 跟 P_GAIN, D_GAIN, I_GAIN, OV_GAIN, VFF_GAIN 指令配套为闭环控制系统使用，需要使用模拟量伺服。 注意： 1.ECAT 总线 ATYPE 为 66, 67 模式适用。 2.PID 只能根据实际情况手动调整至最佳，建议带上负载。 3.PID 调节参考口诀： 参数整定找最佳，从小到大顺序查； 先是比例后积分，最后再把微分加； 曲线振荡很频繁，比例度盘要放大； 曲线漂浮绕大弯，比例度盘往小扳； 曲线偏离回复慢，积分时间往下降； 曲线波动周期长，积分时间再加长； 曲线振荡频率快，先把微分降下来； 动差大来波动慢。微分时间应加长； 理想曲线两个波，前高后低四比一； 一看二调多分析，调节质量不会低； 若要反应增快，增大 P 减小 I； 若要反应减慢，减小 P 增大 I； 如果比例太大，会引起系统震荡； 如果积分太大，会引起系统迟钝。
语法	VAR1 = SERVO, SERVO = ON/OFF
适用控制器	支持 4 系列及以上
例子	RAPIDSTOP(2) WAITIDLE TRIGGER '触发示波器 BASE(0) UNITS = 1000 ACCEL = 100 DECEL = 100 SPEED = 1000 CREEP = 100 LSPEED = 0 MERGE = 0 SRAMP = 0 DPOS = 0 MPOS = 0 FE_LIMIT = 10 '修改最大允许随动误差限制范围，超过时告警。

```
FE_RANGE = 10      '设置告警时的随动误差误差值
P_GAIN AXIS(0) = 100 '比例增益，可以使用指定轴的方式
D_GAIN = 5         '微分增益
I_GAIN = 1         '积分增益
OV_GAIN = 0        '速度增益
VFF_GAIN = 0       '前馈增益
AFF_GAIN = 0       '加速度前馈增益
SERVO = ON         '闭环控制打开

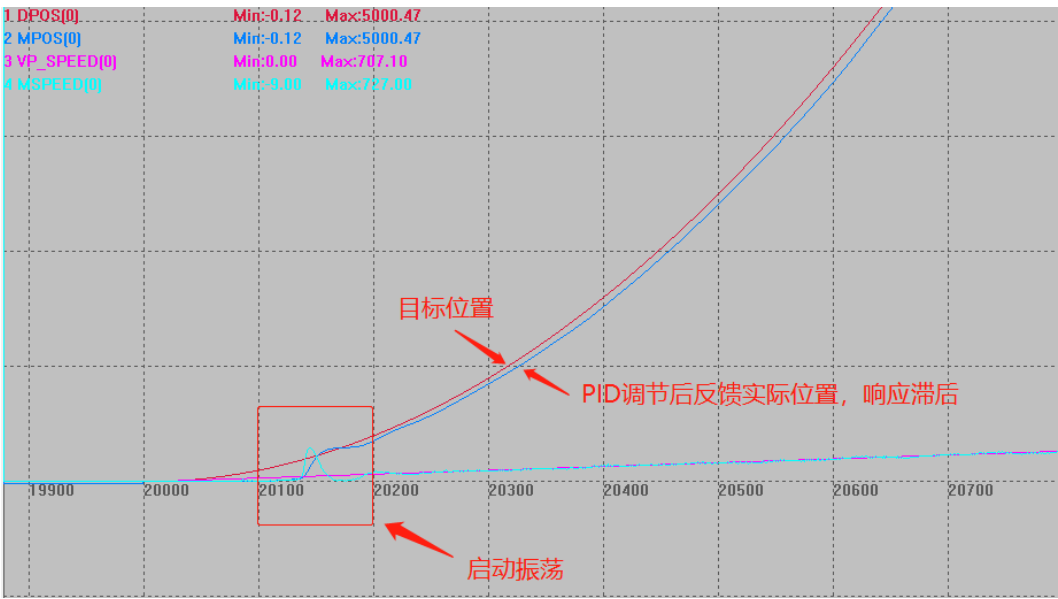
BASE(0)
MOVE(5000)
WAITIDLE
SERVO AXIS(0) = OFF '关闭闭环控制，可以指定轴关闭，建议每次使用后关闭

DELAY(3000)        '3 秒后停止测试
DPOS(0) = 0
MPOS(0) = 0
END
```

整体曲线如下，可以看出 DPOS 和 MPOS 在闭环控制下，PID 调节二者实时同步。



局部曲线如下：



相关指令

[P_GAIN](#), [D_GAIN](#), [I_GAIN](#), [OV_GAIN](#), [VFF_GAIN](#), [FE_LIMIT](#), [FE_RANGE](#)

第十二章 输入输出相关指令

12.1 输入相关指令

IN -- 输入口

类型	输入输出函数
描述	读取输入，没有参数时返回 0-31 的状态。 读取的是 INVERT_IN 翻转以后的状态。 ZIO 扩展板的 IO 通道号与拨码有关，起始值为（16+拨码组合值*16），EIO 总线扩展 IO 使用 NODE_IO 指令，只能设置为 8 的倍数，详细查看硬件手册。 注意 IO 映射编号要大于控制器自身最大的 IO 编号，不能与控制器的编号重合。
语法	IN([channel1],[channel2]) channel1: 要读取的起始输入通道 channel2: 要读取的结束输入通道，没有结束通道时，返回单个通道的输入状态
适用控制器	通用
例子	a=IN(1) '读取通道 1 的输入
相关指令	OP , INVERT_IN

AIN -- 模拟量输入

类型	输入输出函数
描述	读取模拟输入，返回 AD 转换模块的刻度值。 12 位刻度值范围 0~4095 对应 0~10V 电压。 16 位刻度值范围 0~65535 对应 0~10V 电压。 ZAI0 扩展板的 AD 通道号与拨码有关，起始值为（8+拨码组合值*8），ZMIO 总线扩展 AD 使用 NODE_AIO 指令只能设置为 8 的倍数，详细查看硬件手册。 注意 AIO 映射编号要大于控制器自身最大的 AIO 编号，不能与控制器的编号重合。
语法	VAR=AIN(channel) channel: 模拟输入通道 0-127
适用控制器	通用
例子	a=AIN(1) '读取通道 1 的 AD a=AIN(1)*10/4095 '通道 1 的电压值 a=((AIN(1)-32767)*10/65535)*2 '-10V 到 10V 范围时通道 1 的电压值
相关指令	AOUT

ZSIMU_IN -- 仿真 IN 输入

类型	仿真器专用指令
描述	模拟输入 IN 口的输入。
语法	ZSIMU_IN([ionum], value) ionum: 输入编号, 从 0 开始, 没有这个参数时输出 0-31 value: 输入状态
适用控制器	通用
例子	ZSIMU_IN(0,1) '仿真输入 0 有效
相关指令	IN

ZSIMU_AIN -- 仿真模拟量输入

类型	仿真器专用指令
描述	模拟模拟量输入口的输入。
语法	ZSIMU_AIN(ionum, value)
适用控制器	通用
例子	ZSIMU_AIN(0,1024) '仿真通道 0
相关指令	AIN

ZSIMU_ENCODER -- 仿真编码器输入

类型	仿真器专用指令
描述	模拟编码器输入口的输入。 RTSys 仿真器只支持纯编码器轴, 脉冲+编码器轴缺省仿真 MPOS 跟随 DPOS ZDevelop 仿真器支持纯编码器轴, 脉冲+编码器轴, 均可通过指令 zsimu_encoder 修改 encoder
语法	ZSIMU_ENCODER(axis num, value) axis num: 轴编号, 从 0 开始 value: ENCODER 的仿真值
适用控制器	通用
例子	ZSIMU_ENCODER(0,1024) 'ENCODER = 1024
相关指令	ENCODER

INVERT_IN -- 反转输入

类型	特殊指令
描述	反转输入状态, 可以读取判断是否有反转。
语法	INVERT_IN(channel, state), VAR1= INVERT_IN(channel) channel: 输入通道

	state: ON/OFF
适用控制器	通用
例子	INVERT_IN(1,ON) 'ZMC 系列控制器输入 OFF 时认为有信号输入(ECI 系列控制器与之相反) FWD_IN(0)=1 '输入 IN1 作为轴 0 的正向限位信号
相关指令	IN

IN_SCAN -- 扫描输入变化

类型	输入输出函数
描述	扫描输入变动，返回值 1(TRUE)-变动，0(FALSE)-没有变动。 此函数必须固定不断的扫描，返回的是两次扫描之间的变动，可以通过 IN_EVENT 读取变动的具体情况，使用的是 INVERT_IN 翻转以后的状态。 固件版本 20140214 以后的才提供这个支持，扫描范围有宽度限制。 00x 系列控制器只能在单个任务中使用。
语法	VAR1=IN_SCAN([channel1][,channel2]) channel1: 要读取的起始输入通道 channel2: 要读取的结束输入通道，没有结束通道时，扫描单个输入
适用控制器	通用
例子	<pre> WHILE 1 IF IN_SCAN(0,23) THEN '扫描 IN0-23 口电平变化 IF IN_EVENT(0) > 0 THEN 'IN0 上升沿触发 PRINT "IN0 UP", IN_BUFF(0) ELSEIF IN_EVENT(0) < 0 THEN 'IN0 下降沿触发 PRINT "IN0 DOWN", IN_BUFF(0) ENDIF ENDIF WEND </pre>
相关指令	IN_EVENT , SCAN_EVENT , IN_BUFF

IN_EVENT -- 读取输入变化

类型	输入输出函数
描述	读取输入的变化情况。 1-上升沿，-1-下降沿，0-没有变化。 此函数必须与 IN_SCAN 配合使用。
语法	VAR1 = IN_EVENT(IONUM)
适用控制器	通用
例子	参见 IN_SCAN
相关指令	IN_SCAN , SCAN_EVENT

SCAN_EVENT -- 检测变化

类型	输入输出函数
描述	检测表达式的内容变化。 OFF-ON 返回 1, ON-OFF 返回-1, 不变返回 0。 不要在循环内或者多任务调用同一个 SUB 内的 SCAN_EVENT。 150810 之后固件版本支持, 之前版本用 IN_EVENT 和 IN_SCAN。
语法	<pre>ret = SCAN_EVENT (expression)</pre> expression: 任意有效的表达式, 结果会转成 BOOL 类型
适用控制器	通用
例子	例一 输入信号扫描 <pre>WHILE 1 IF SCAN_EVENT(IN(0)) > 0 THEN 'IN0 上升沿触发 PRINT "IN0 ON" ELSEIF SCAN_EVENT(IN(0))<0 THEN 'IN0 下降沿触发 PRINT "IN0 OFF" ENDIF WEND</pre> 例二 寄存器、变量扫描 <pre>WHILE 1 IF SCAN_EVENT(TABLE(0)) > 0 THEN 'TABLE0 上升沿触发 PRINT "TABLE0 ON" ELSEIF SCAN_EVENT(TABLE(0))<0 THEN 'TABLE0 下降沿触发 PRINT "TABLE0 OFF" ENDIF WEND</pre> 在线命令操作 table(0), 打印相关结果
相关指令	IN_SCAN , IN_EVENT

IN_BUFF -- 读取输入缓冲

类型	输入输出函数
描述	读取 IN_SCAN 缓冲的当前输入, 没有参数时返回 0-31 的状态。 读取的是 INVERT_IN 翻转以后的状态。
语法	<pre>IN_BUFF([channel1],[channel2])</pre> channel1: 要读取的起始输入通道, 必须是 IN_SCAN 的输入范围 channel2: 要读取的结束输入通道, 没有结束通道时, 返回单个通道的输入状态
适用控制器	通用
例子	参见 IN_SCAN
相关指令	IN_SCAN

INFILTER -- 输入口滤波

类型	系统参数
描述	本地输入口的滤波参数。滤波时间，ms 单位。 值越大，滤波时间越长，2-9，缺省 2。
语法	VAR1 = INFILTER, INFILTER= expression
适用控制器	通用
例子	INFILTER= 5 ‘在干扰严重场合加大滤波时间

IN_MSFILTER – 设置输入口滤波

类型	特殊指令
描述	单个输入口设置滤波器。
语法	IN_MSFILTER(channel,timems), VAR1=IN_MSFILTER(channel) channel: 输入通道 timems: 滤波时间，ms 单位，精度只能到系统周期，最多 200 个周期，缺省值 0。
适用控制器	20230808 增加，5 系以上
例子	IN_MSFILTER(0,5) ‘将 IN0 设为滤波器，滤波时间 5ms

LOCALIO_OFFSET -- 本地 IO 偏移

类型	XPLC300 专用指令
描述	此功能用偏移 XPLC300 系列控制器本地的 IO 地址。
语法	LOCALIO_OFFSET=value value: IO 起始地址，缺省 0 注：设置的 IO 起始地址只能是 8 的倍数。
适用控制器	XPLC300
例子	LOCALIO_OFFSET=8 ‘本地 IO 地址偏移到 8
相关指令	LOCALAIO_OFFSET

12.2 输出相关指令

OP -- 输出口

类型	输入输出指令和函数
描述	输出或读取输出状态。 当在表达式中使用时，自动为函数语法。

	ZIO 扩展板的 IO 通道号与拨码有关，起始值为（16+拨码组合值*16），EIO 总线扩展 IO 使用 NODE_IO 指令，只能设置为 8 的倍数，详细查看硬件手册。 注意 IO 映射编号要大于控制器自身最大的 IO 编号，不能与控制器的编号重合。 最多可操作 32 个输出口。
语法	OP([ionum],value) 或 OP(ionum1, ionum2,value[,mask]) 或 OP([firstnum],[finalnum]) ionum: 输出编号，从 0 开始 value: 输出状态，多个输出口操作时按位来指明多个口状态 ionum1: 要操作的第一个输出通道 ionum2: 要操作的最后一个输出通道 mask: 用位来指定哪些 IO 需要操作，不填时第一个通道到最后一个通道都操作 firstnum: 输出编号，从 0 开始 finalnum: 输出编号，从 0 开始，没有这个参数时，读取单个输出口状态
适用控制器	通用
例子	例一 单个操作 '翻转输出口 0 IF OP (0) = ON THEN OP (0, OFF) ELSE OP (0, ON) ENDIF 例二 区域操作 OP(0,7,\$FF) 'bit0-bit7 全开 DELAY(1000) OP(0,7,0) OP(8,15,\$FF) 'bit8-bit15 全开 DELAY(1000) OP(8,15,0) OP(0,15,\$FFFF) 'bit0-bit15 全开 DELAY(1000) OP(0,15,0) OP(0,31,\$FFFFFFFF) 'bit0-bit31 全开 DELAY(1000) OP(0,31,0)
相关指令	READ_OP , MOVE_OP

AOUT -- 模拟量输出

类型	输入输出指令或函数
----	-----------

描述	模拟通道输出。 12 位刻度值范围 0~4095 对应 0~10V 电压。 16 位刻度值范围 0~65535 对应 0~10V 电压。 AOUT(2) 对应并口值 0-255，用于设置激光器的功率，408SCAN 和 504SCAN 适用。
语法	AOUT(channel) = value channel: 模拟输出通道 0-63 扩展板 DA 通道号与拨码有关，起始值为（4 + 拨码组合值*4），详细查看硬件手册。
适用控制器	通用
例子	AOUT(1) = 0 '关闭输出 DA 通道 1 AOUT(1) = 4095 'DA1 口输出 10V 电压
相关指令	AIN

READ_OP -- 读取输出口

类型	输入输出函数
描述	读取输出状态。 同 OP，多个输出口操作时按位来指明多个口状态。
语法	READ_OP ([firstnum],[finalnum]) firstnum: 起始输出编号，从 0 开始 finalnum: 结束输出编号，从 0 开始，没有这个参数时，读取单个输出口的状态
适用控制器	通用
例子	'翻转输出口 0 IF READ_OP (0) = ON THEN OP (0, OFF) ELSE OP (0, ON) ENDIF
相关指令	OP

EXIO_DIR – 配置 EXIO 接口

类型	输入输出函数
描述	按位指定 EXIO 扩展接口的输入输出，需配合定制转接板使用。 使用 Fiber 转接板 IO 配置指令 EXIO_DIR(0, \$8FFFF)、YAG 转接板 IO 配置指令 EXIO_DIR(0, \$FCBFE)、SPI 转接板 IO 配置指令 EXIO_DIR(0, \$FFFFA)。
语法	命令语法: Exio_Dir(isel, idirbit) 函数语法: Exio_Dir(isel) isel: Exio 选择，目前固定 0 idirbit: 按位指定输入输出，1-输出，0-输入(缺省)

适用控制器	ZMC408SCAN
例子	EXIO_DIR(0,\$8FFFF) 'Fiber 转接板
相关指令	OP

LOCALAIO_OFFSET -- 本地 AIO 偏移

类型	XPLC300 专用指令
描述	此功能用偏移 XPLC300 系列控制器本地的 AIO 地址。 注：XPLC300 系列控制器无模拟量，无偏移配置效果。
语法	LOCALAIO_OFFSET=value value: AIO 起始地址，缺省 0
适用控制器	XPLC300
例子	LOCALAIO_OFFSET=1 '本地 AIO 地址偏移到 1
相关指令	LOCALIO_OFFSET

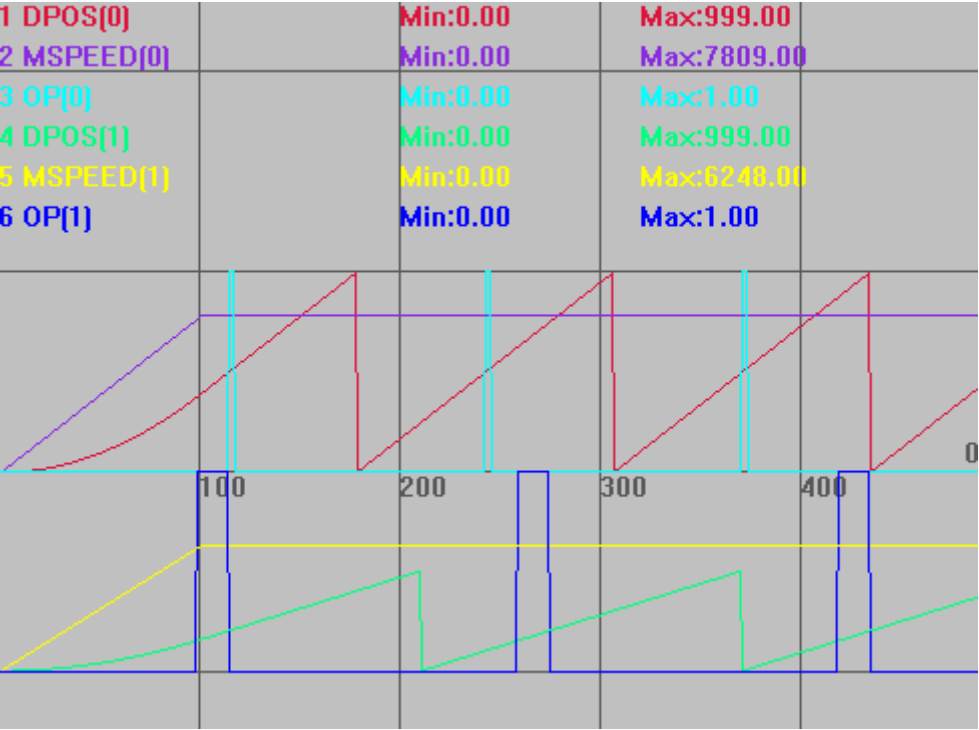
12.3 位置比较输出指令

PSWITCH -- 软件位置比较输出

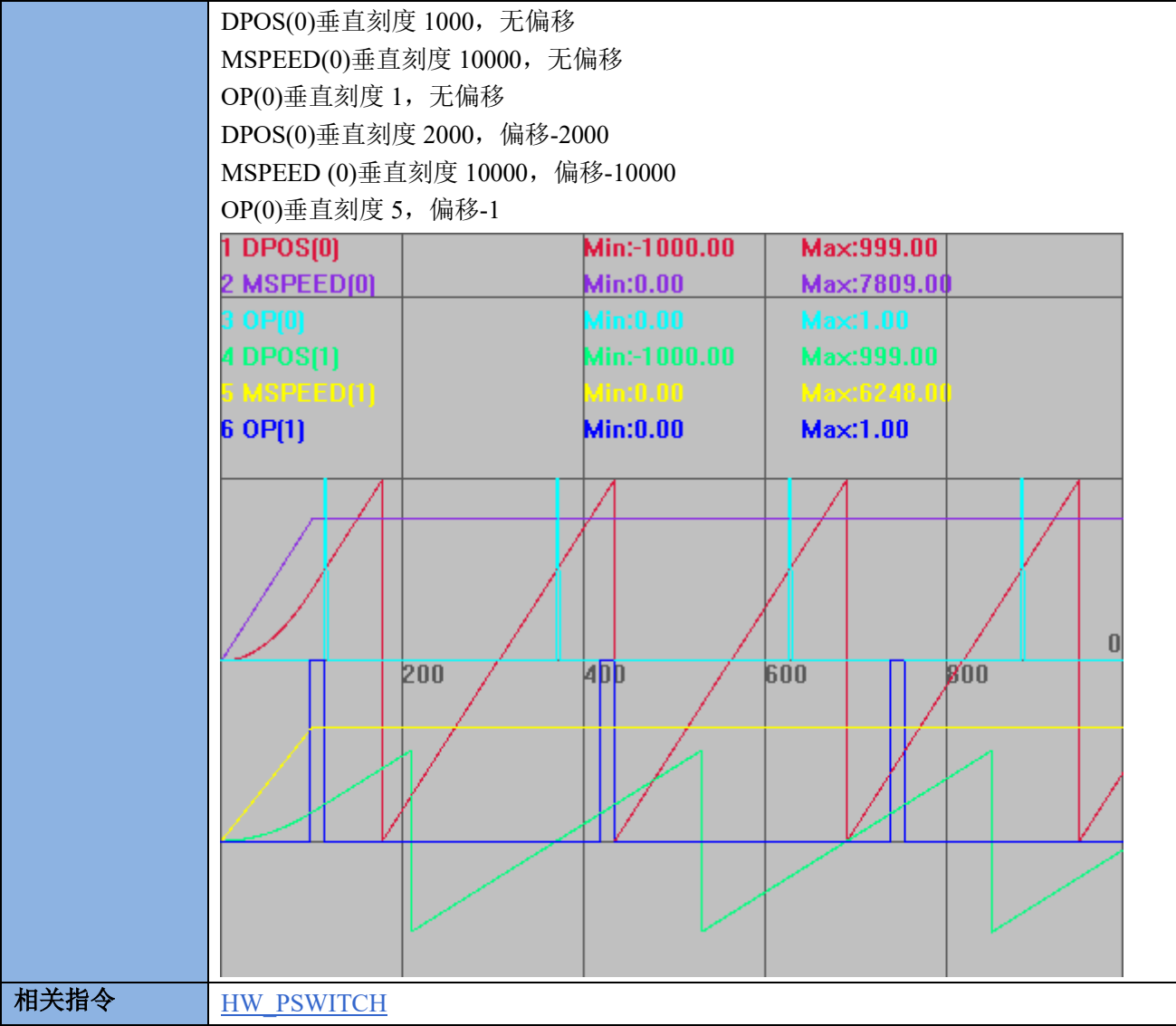
类型	输入输出指令
描述	根据位置比较来操作输出口。 如果多个 PSWITCH 操作同一个输出口，需要编号顺序排在一起。 使用脉冲型电机时只有 ATYPE 为 4 时才是比较反馈位置(MPOS)，默认出厂的 ATYPE 为 1 或 7 比较的是命令位置(DPOS)。
语法	PSWITCH(num,enable,[,axis,op num,op state,set pos,reset pos]) num: 比较器的编号，ZMC1xx 系列有 16 个比较器，编号：0-15 enable: 操作比较器的使能 - ON 启动 / OFF 取消 axis: 指定要获取位置的轴号 op num: 操作的 IO 编号 op state: 输出的状态，1 表示在下面位置范围内输出为 ON，0 表示在下面位置范围内输出为 OFF set pos: 设定产生输出的起始位置，采用 units 单位 reset pos: 设定输出复位的位置，采用 units 单位 不同型号控制器支持的比较器个数不同，使用?*max 指令打印查看 max_pswitch 参数确认个数。
适用控制器	通用
例子	RAPIDSTOP(2) WAIT IDLE DELAY(1000)

```
ERRSWITCH = 3
BASE(0,1)      '选择轴号
ATYPE=1,1      '脉冲方式步进或伺服
DPOS = 0,0
UNITS = 1,1     '脉冲当量
SPEED = 10000,10000
ACCEL=SPEED(0)*10,SPEED(1)*10
DECEL=SPEED(0)*10,SPEED(1)*10
REP_OPTION=1,1  '设置坐标循环范围为 0 到+ REP_DIST
REP_DIST=1000,1000
TRIGGER
MOVE(10000,8000)
PSWITCH(0,ON,0,0,ON,500,520)
PSWITCH(1,ON,1,1,ON,300,400)
END
```

DPOS(0)垂直刻度 1000，无偏移
MSPEED(0)垂直刻度 10000，无偏移
OP(0)垂直刻度 1，无偏移
DPOS(0)垂直刻度 2000，偏移-2000
MSPEED (0)垂直刻度 10000，偏移-10000
OP(0)垂直刻度 5，偏移-1



以上例程，仅修改如下指令，得出波形如下图。
REP_OPTION=0,0 '设置坐标循环范围为- REP_DIST 到+ REP_DIST



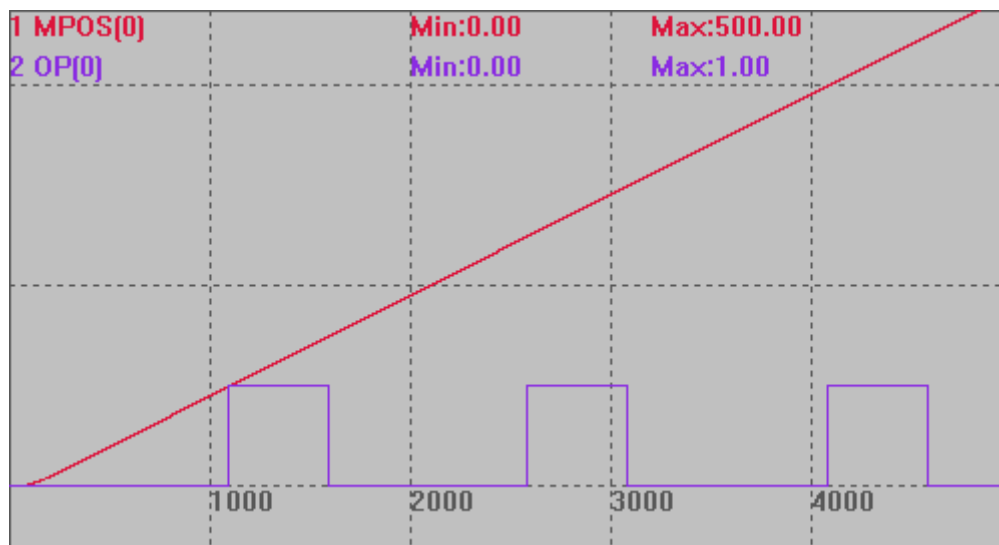
相关指令 [HW_PSWITCH](#)

HW_PSWITCH -- 硬件位置比较输出

类型	轴指令
描述	硬件比较输出，不同的轴对应不同的输出口。 缺省轴 0-5 分别对应输出口 0 1 2 3 0 1。总共 4 个比较输出口。 可以连续调用两个 HW_PSWITCH 指令，通过函数方式可以获取能调用的指令个数。 每个比较点触发都会使得当前输出口电平翻转。 HW 缓冲数 1024 个，可以连续调用 1024 个 HW 指令。 HW 指令调用后,不受后面的坐标修改功能的影响，HW 指令 TABLE 存储的坐标必须在调用时是正确的，因此尽量手动修改坐标,要规避 HW 指令与坐标循环自动修改的随机冲突。 因为自动循环修改坐标不受程序控制，无法确定是在 HW 的前面还是后面，这样 TABLE 里面的坐标就无法确定。

	此指令仅支持脉冲轴硬件位置比较输出，总线轴请使用 HW_PSWITCH2 指令。 使用脉冲型电机时只有 ATYPE 为 4 时才是比较反馈位置(MPOS)，默认出厂的 ATYPE 为 1 或 7 比较的是命令位置(DPOS)。 240111 ZMC432 增加：HW 比较加速 HW_PSWITCH 比较输出可以提到每个周期 32 次，设置 4K(250u)周期时，可以支持 128kHz 的比较速度, 8k(125u) 下支持 256kHz 的比较速度。 ZMC412, PCIE464 可以合并支持
语法	HW_PSWITCH(mode, direction, reserve, tablestart, tableend) Buff = HW_PSWITCH([axisnum]) mode: 1-启动比较器, 2- 停止并删除没完成的比较点 direction: 0-坐标负向, 1- 坐标正向, 2-不判断方向 reserve: 预留 tablestart: 第一个比较点坐标所在 TABLE 编号 tableend: 最后一个比较点坐标所在 TABLE 编号 HW_PSWITCH 没有比较完所有点的话,一定要设置 mode 值为 2,通过 HW_PSWITCH(2) 指令停止并删除没有完成的比较点, 否则后面此输出通道会工作不正常。 矢量比较: (ZMC432 特殊固件支持) !矢量比较固定使用输出口 0, 支持前 6 个物理编码器轴。 !必须轴 ATYPE 为带本地编码器。MPOS 响应要够快, 否则建议使用 HW_PSWITCH2。 !状态每次翻转, 必须保证开始 OP 的状态正确。 !3D 矢量比较模式与单轴模式不要混用, 可能冲突。 !矢量坐标与 VECTOR_MOVED 比较, 开始运动前建议清零。 !比较轴坐标不要使用坐标循环, 或是中途修改坐标 矢量比较语法: mode=5: HW_PSWITCH(5, vectstart, repes, cycledis, ondis) Vectstart: 比较点矢量坐标 Repes: 重复周期, 一个周期内比较两次, 先输出有效状态,再输出无效状态。 Cycledis: 周期距离, 每隔这个距离输出 Opstate, Ondis 后还原为无效状态。 Ondis: 输出有效状态的距离, (Cycledis- Ondis) 为无效状态距离 mode=6: HW_PSWITCH(6, vectstart, repes, cycledis) Vectstart: 比较点矢量坐标 Repes: 重复周期, 一个周期只比较一次。 Cycledis: 周期距离, 每隔这个距离输出一次。 mode=7: HW_PSWITCH(7, tablestart, tableend) tablestart: 第一个比较点矢量坐标所在 TABLE 编号

	tableend: 最后一个比较点矢量坐标所在 TABLE 编号 mode=25: HW_PSWITCH(25, axes, num, tablestart) axes: 比较的轴数 num: 比较的点数. tablestart: 第一个比较点 MPOS 坐标所在 TABLE 编号, 每个点占用 axes 个 TABLE, 依次存储每个轴 MPOS. ! 这个方式直接输入要比较的每个点的坐标, 自动根据前后点坐标关系来确定比较的方向。对前后有方向变化的运动, 尽量使用多个 HW_PSWITCH 指令写入。
适用控制器	4 系列及以上产品, 4 系列固件 20170704 及以上 ZMC420SCAN 不支持 HW_PSWITCH 矢量比较只有 ZMC432 特殊固件支持
例子	以下例程测试环境: ZMC432, 固件: 20170709 (仿真器无法运行此指令)。 BASE(0) '选择轴 0 默认输出 OP(0) ATYPE=4 '采用编码器位置作比较输出参考点, 无编码器改成脉冲轴类型 UNITS=100 SPEED=100 ACCEL=500 MPOS=0 OP(0, OFF) TABLE(0, 100, 150, 250, 300, 400, 450) 'OP0 在 MPOS100-150 间打开, 150-250 间关闭, 250-300 间打开, 300-400 间关闭, 400-450 间打开, 450 后关闭 HW_PSWITCH(2) '停止并删除没有完成的比较点 HW_PSWITCH(1, 1, 0, 0, 5) '启动比较输出 TRIGGER MOVEABS(500) 比较输出图 MPOS(0)垂直刻度 200 OP(0)垂直刻度 2



矢量比较例子：（ZMC432，特殊固件支持）

模式 5:

BASE(0)

ATYPE = 1

DPOS= 0

MPOS= 0

SPEED = 1000

ACCEL= 1000

DECEL= 1000

VECTOR_MOVED = 0

HW_PSWITCH(2)

HW_PSWITCH(5,100,20,40,20)

TRIGGER

MOVEABS(1000)

WAIT UNTIL IDLE(0)

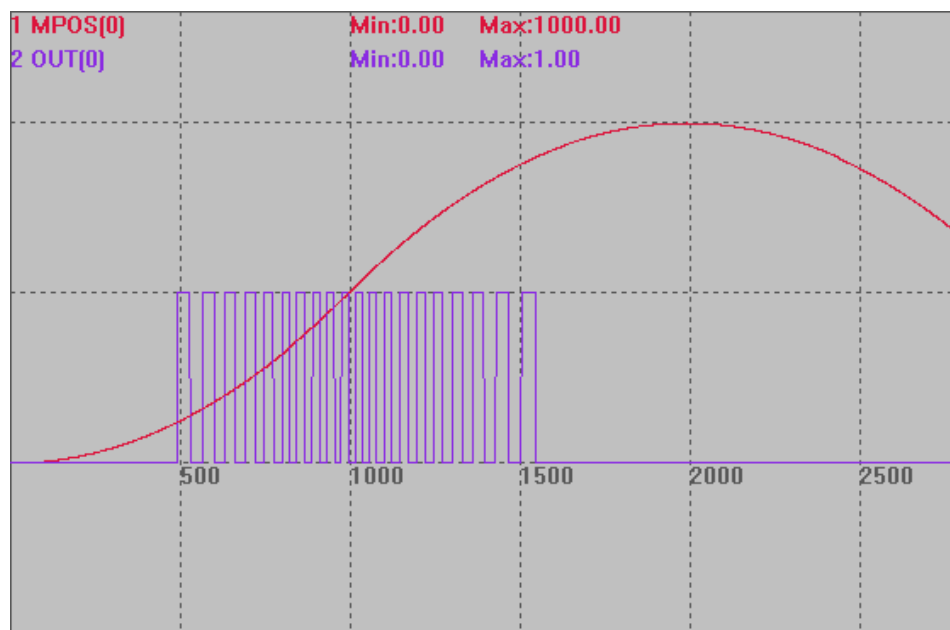
MOVEABS(0)

WAIT UNTIL IDLE(0)

比较输出图

MPOS(0)垂直刻度 500

OP(0)垂直刻度 1

**模式 6:**

BASE(0)

ATYPE= 1

DPOS= 0

MPOS= 0

SPEED= 1000

ACCEL= 1000

DECEL= 1000

VECTOR_MOVED= 0

HW_PSWITCH(2)**HW_PSWITCH(6, 100, 10, 100)**

TRIGGER

MOVEABS(1000)

WAIT UNTIL IDLE(0)

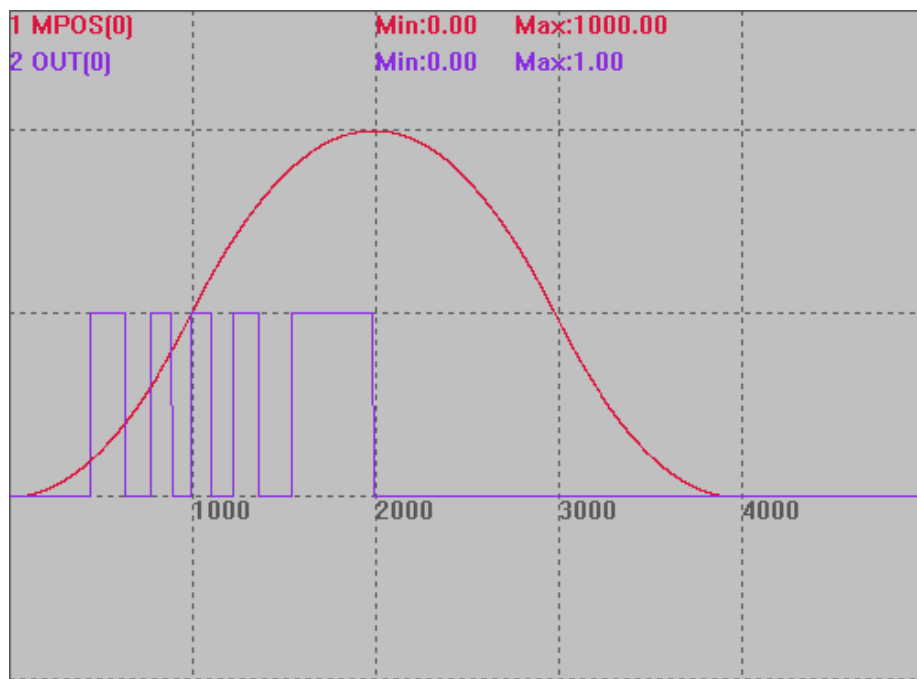
MOVEABS(0)

WAIT UNTIL IDLE(0)

比较输出图

MPOS(0)垂直刻度 500

OP(0)垂直刻度 1

**模式 7:**

BASE(0)

ATYPE = 1

DPOS = 0

MPOS = 0

SPEED = 1000

ACCEL = 1000

DECEL = 1000

VECTOR_MOVED = 0

TABLE(100,50,100,150,200,250,300,350,400,450,500,550,600,650,700,750,800,850,900,950)

HW_PSWITCH(2)**HW_PSWITCH(7, 100,117)**

TRIGGER

MOVEABS(1000)

WAIT UNTIL IDLE(0)

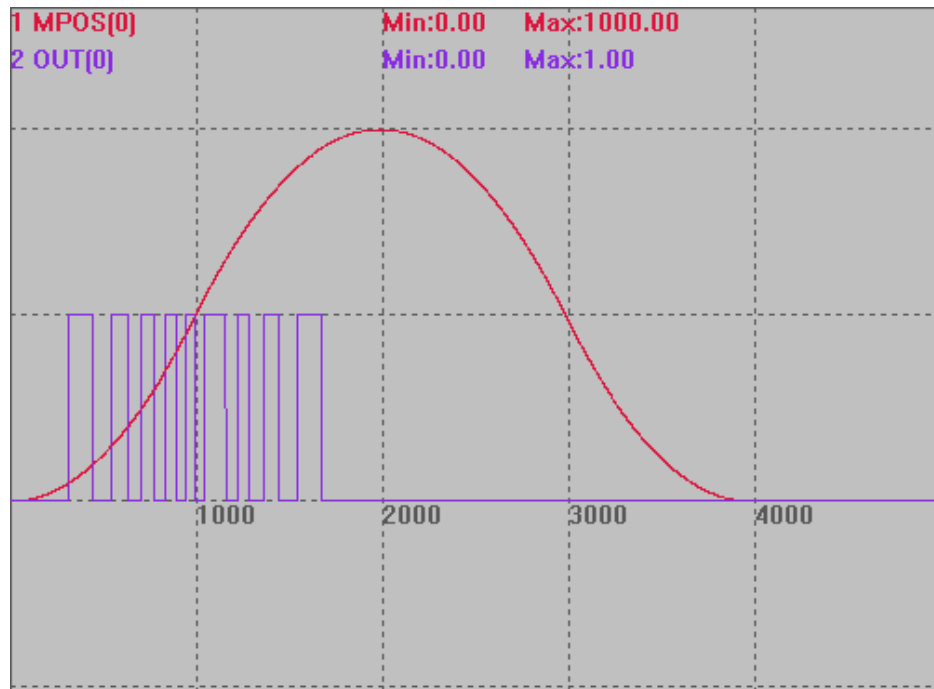
MOVEABS(0)

WAIT UNTIL IDLE(0)

比较输出图

MPOS(0)垂直刻度 500

OP(0)垂直刻度 1

**模式 25:**

BASE(0,1)

ATYPE = 5,5

DPOS= 0,0

MPOS= 0,0

SPEED = 100,100

ACCEL= 1000,1000

DECEL= 1000,1000

VECTOR_MOVED = 0

TRIGGER

for i=0 to 22

 table(i*2) = i

 table(i*2 + 1) = i

 table(100+i*2) = (22-i)

 table(100+i*2 + 1) = (22-i)

next

HW_TIMER(2, 2000, 1000, 1, 0, 0)

HW_PSWITCH(25, 2, 22, 0)

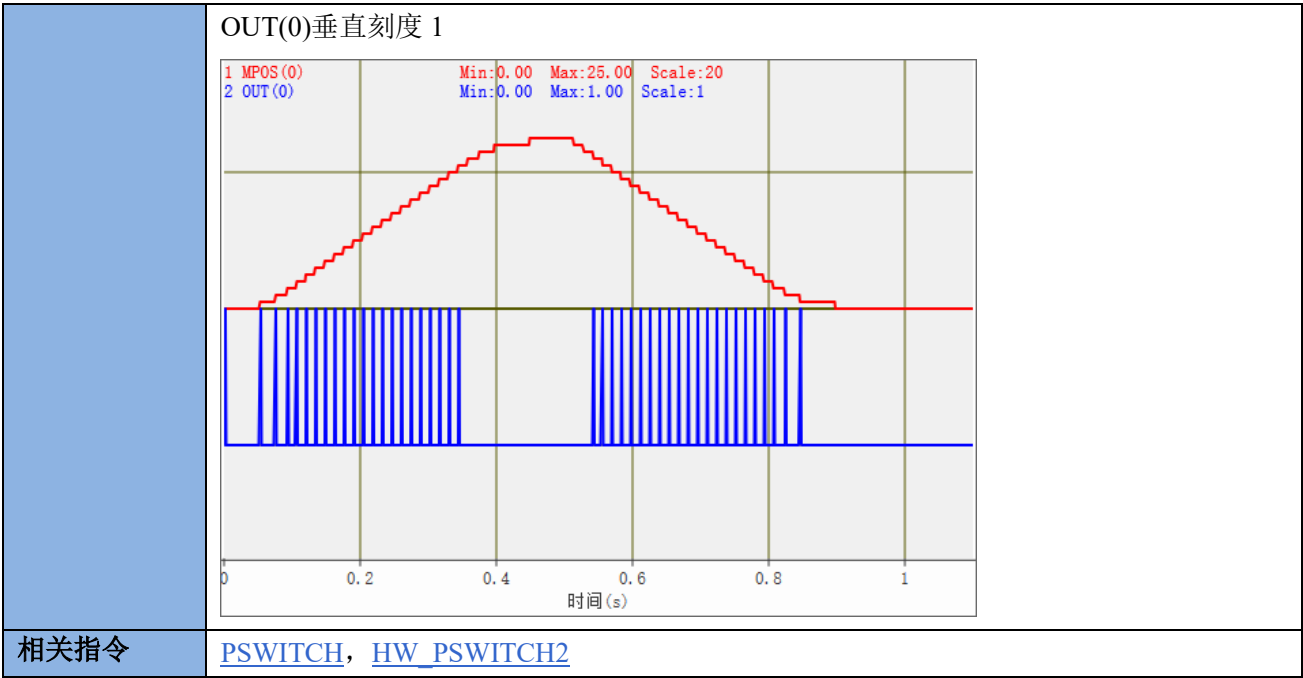
HW_PSWITCH(25, 2, 22, 100)

MOVE(25, 25)

MOVE(-25, -25)

比较输出图

MPOS(0)垂直刻度 20



HW_PSWITCH2 -- 总线硬件位置比较输出

类型	轴指令
描述	总线硬件位置比较输出，也支持脉冲轴，必须使用特定的输出口。 4 系列产品部分有 4 个比较输出口，可以选择不同的比较输出口，一般为 OUT0/1/2/3 口。 比较主轴带编码器输入时，自动使用编码器位置来触发，可以使用 MOVEOP_DELAY 参数来调整输出准确时刻。 不同的总线驱动器效果可能有差异，也可以通过 MOVEOP_DELAY 参数来调整。 HW_PSWITCH2 与 MOVE_OP 精准使用同样的硬件资源，不建议在同一个通道同时使用，可以在不同的通道同时使用。 每个系统周期内只能比较一次，系统周期通过 SERVO_PERIOD 查询。 TABLE 位置数据在所有比较点完成前不要修改。 脉冲轴和总线轴均支持此指令。 总线轴比较反馈位置(MPOS) 使用脉冲型电机时只有 ATYPE 为 4 时才是比较反馈位置(MPOS)，默认出厂的 ATYPE 为 1 或 7 比较的是命令位置(DPOS)。
语法	命令语法：HW_PSWITCH2(mode, [...]) 函数语法：Buff = HW_PSWITCH2([axisnum]) Mode=1：单轴比较 HW_PSWITCH2(1,opnum,opstate,tablestart,tableend[,direction]) mode: 1-启动比较器 opnum: 对应的输出口

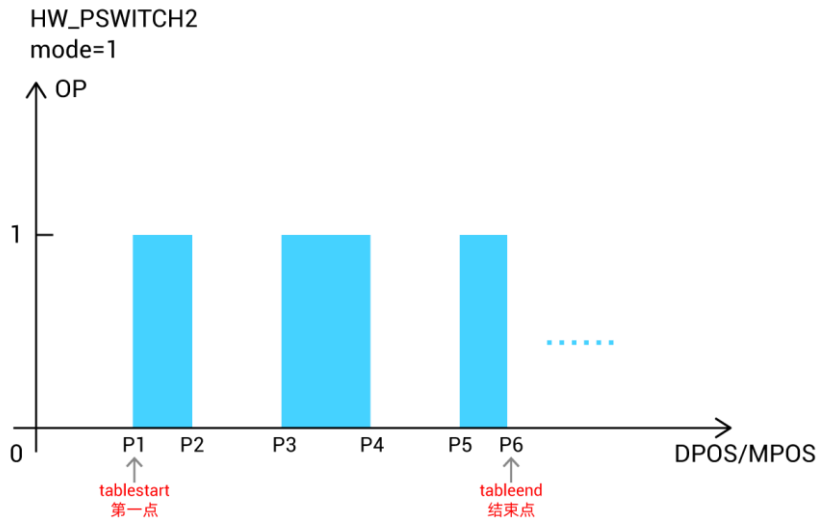
opstate: 第一个比较点的输出状态

tablestart: 第一个比较点绝对坐标所在 TABLE 编号

tableend: 最后一个比较点绝对坐标所在 TABLE 编号

direction: 第一个点判断方向, 0 坐标负向, 1 坐标正向, -1 不使用方向

说明: 比较点写在 TABLE 中, 每到达一个比较位置 OP 反转一次。



Mode=2: 清除比较点

HW_PSWITCH2(2)

mode: 2-停止并删除没完成的比较点

说明: HW_PSWITCH2 没有比较完所有点的话, 一定要设置 mode 值为 2, 通过 HW_PSWITCH2(2)指令停止并删除没有完成的比较点, 否则后面此输出通道会工作不正常。

使用矢量距离比较时, 与 VECTOR_MOVED 进行比较, 建议连续运动前设置 VECTOR_MOVED 初始值。

Mode=3: 矢量比较方式

HW_PSWITCH2(3, opnum, opstate, tablestart, tableend)

mode: 3-启动比较器

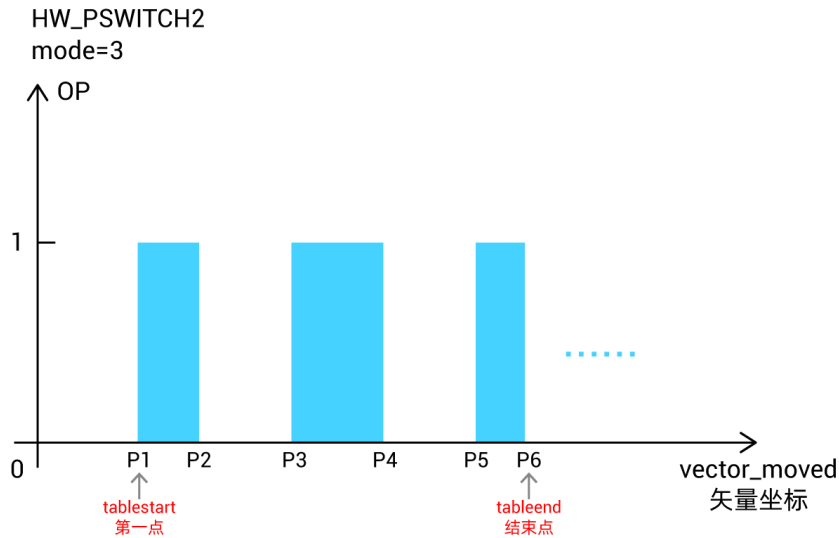
opnum: 对应的输出口

opstate: 第一个比较点的输出状态

tablestart: 第一个比较点 VECTOR_MOVED 坐标所在 TABLE 编号

tableend: 最后一个比较点 VECTOR_MOVED 坐标所在 TABLE 编号

说明: 比较点写在 TABLE 中, 每到达一个比较矢量位置 OP 反转一次。



Mode=4: 矢量比较方式，单个比较点

HW_PSWITCH2(4, opnum, opstate, vectstart)

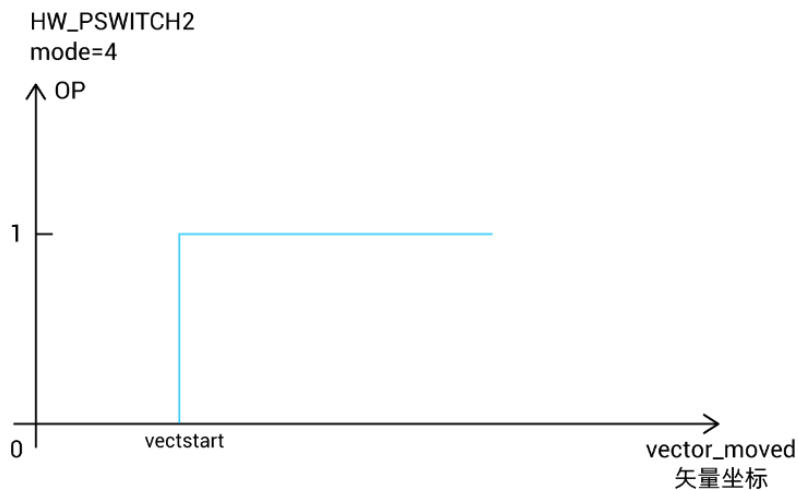
mode: 4-启动比较器

opnum: 对应的输出口

opstate: 第一个比较点的输出状态

vectstart: 比较点 VECTOR_MOVED 当前运动距离

说明：到达指令设置的一个比较矢量位置 OP 反转，比较结束。



Mode=5: 矢量比较方式，周期脉冲模式

HW_PSWITCH2(5, opnum, opstate, vectstart, repes, cycledis, ondis)

mode: 5-启动比较器

opnum: 对应的输出口

opstate: 第一个比较点的输出状态，认为是有效状态，反之认为无效状态

vectstart: 比较点 VECTOR_MOVED 当前运动距离

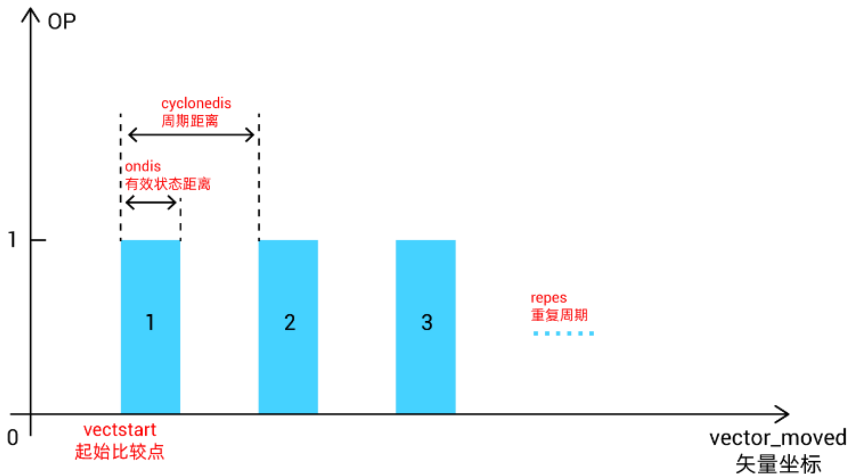
repes: 重复周期，一个周期内比较两次，先输出有效状态，再输出无效状态

cycledis: 周期距离，每隔这个距离输出 opstate, ondis 后还原为无效状态

ondis: 输出有效状态的距离, (cyclendis- ondis)为无效状态距离

说明: 此模式无需 TABLE, 坐标均参考矢量坐标, 从 vectstart 的位置开始比较, 每隔 cyclendis 距离触发一次比较, 重复比较的周期为 repes, 每次触发比较信号后, 保持 ondis 距离后关闭信号, 等待下一周期的触发。

HW_PSWITCH2
mode=5



Mode=6: 矢量比较方式, 周期模式, 与 HW_TIMER 一起使用

HW_PSWITCH2(6, opnum, opstate, vectstart, repes, cyclendis)

mode: 6-启动比较器

opnum: 对应的输出口

opstate: 第一个比较点的输出状态

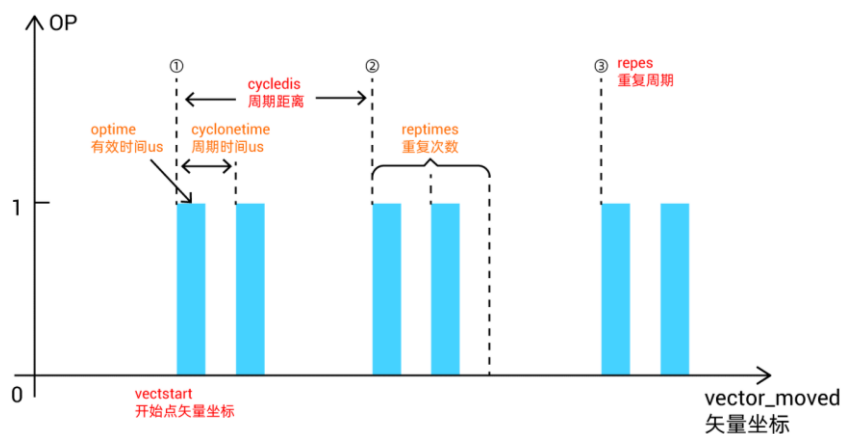
vectstart: 比较点 VECTOR_MOVED 当前运动距离

repes: 重复周期, 一个周期只比较一次

cyclendis: 周期距离, 每隔这个距离输出一次

说明: 此模式无需 TABLE, 坐标均参考矢量坐标, 从 vectstart 的位置开始比较, 每隔 cyclendis 距离触发一次比较, 重复比较的周期为 repes, 每次触发比较信号后, 保持信号的脉冲宽度由 HW_TIMER 指令设置, HW_TIMER 可以控制到达一个触发点控制 OP 反转多次, HW_TIMER 周期走完等待下一周期的触发。

HW_PSWITCH2
mode=6



Mode=7: 与 HW_TIMER 一起使用

HW_PSWITCH2(7, opnum, opstate, tablestart, tableend [, optimeus, optimes, cyctimeus])

mode: 7-启动比较器, opstate 不翻转, 方便与 HW_TIMER 配合使用.

opnum: 对应的输出口

opstate: 第一个比较点的输出状态

tablestart: 第一个比较点 VECTOR_MOVED 坐标所在 TABLE 编号

tableend: 最后一个比较点 VECTOR_MOVED 坐标所在 TABLE 编号

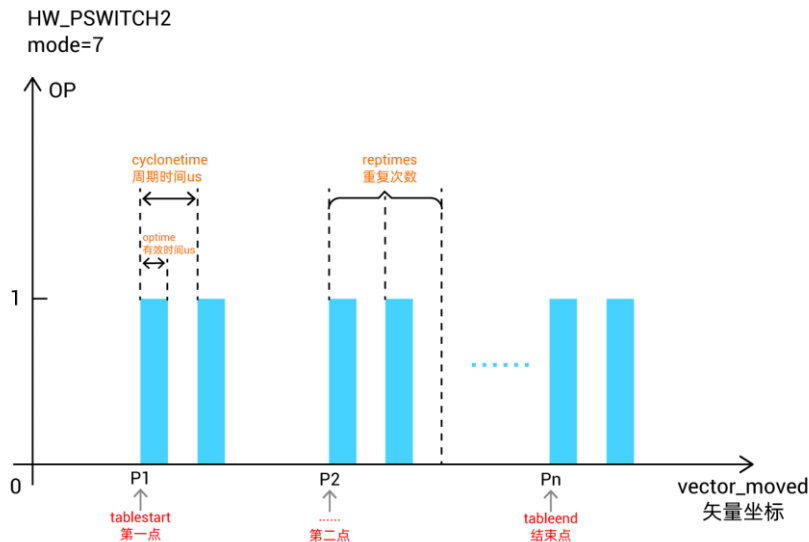
[与 hwtimer 并用时, 可以动态调整 hwtimer 参数]

optimeus: 动态调整 HW_TIMER 的有效时间

optimes: HW_TIMER 的重复周期次数

cyctimeus: 动态调整 HW_TIMER 的脉冲周期时间

说明: 比较点写在 TABLE 中, 坐标均参考矢量坐标, 每到达一个 TABLE 比较矢量位置触发 OP, 此时 OP 的脉冲宽度和每次触发的比较次数由 HW_TIMER 控制; 到达下一个 TABLE 位置, OP 再次触发。

**Mode=8: 单轴比较, 同 Mode1, 但是不翻转信号**

HW_PSWITCH2 模式 1、8 可以和 HW_TIMER 搭配使用, 模式 8 适配一些需要每个位置触发一定时间的单轴飞拍应用。

指令组合适用范围:

HW_PSWITCH2-MODE1 与 HW_TIMER 搭配适用需要间隔一个位置触发一次的情况。

HW_PSWITCH2-MODE8 与 HW_TIMER 搭配适用需要每一个位置触发一次的情况。

脉冲轴和总线轴虚拟轴均适用

以下为 2D, 3D 比较模式

4 系列 170706 以后固件版本支持, 运行轨迹必须顺序通过 fifo 的点。

2D 比较: 25, 26, 每 2 个 table 存储一个点

3D 比较: 35, 36, 每 3 个 table 存储一个点

多个点比较, 每次输出翻转状态。

HW_PSWITCH2(25, opnum, opstate, maxerr, num, tablepos)

HW_PSWITCH2(26, opnum, opstate, maxerr, num, tablepos, [ophwtimeus, ophwtimes, hwcycetimeus])

HW_PSWITCH2(35, opnum, opstate, maxerr, num, tablepos)

HW_PSWITCH2(36, opnum, opstate, maxerr, num, tablepos, [ophwtimeus, ophwtimes, hwcycetimeus])

mode: 25, 26, 35, 36 多维的比较模式

opnum: 对应的输出口

opstate: 第一个比较点的输出状态

maxerr: 比较位置每个轴左右的脉冲个数偏差, 进入偏差范围后开始比较

num: table 里面存储的比较点个数

tablepos: 第一个比较点坐标所在 table 编号

[与 hwtimer 并用时, 可以动态调整 hwtimer 参数]

ophwtimeus: 脉冲时间

ophwtimes: 脉冲个数

hwcycetimeus: 脉冲周期

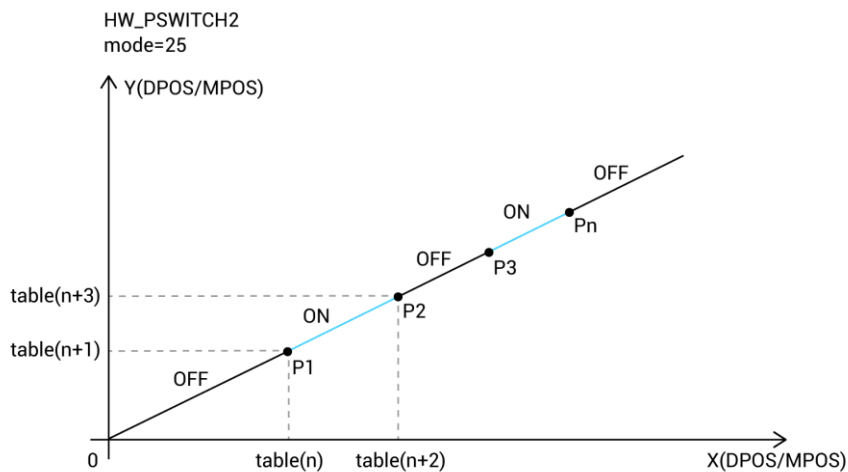
注意: 此模式下 maxerr 偏差参数不能写 0。

Mode=25: 2D 比较

HW_PSWITCH2(25, opnum, opstate, maxerr, num, tablepos)

说明: 比较点写在 TABLE 中, 两个连续的 TABLE 数据组成一个 2D 坐标, 每到达一个比较位置 OP 反转一次。

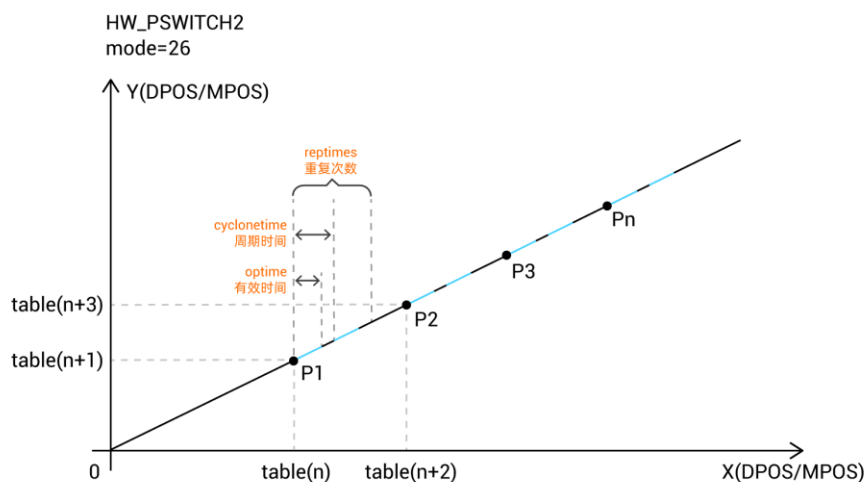
示意中蓝色段表示 OP 开启, 各类常用插补运动均支持比较, 比较点坐标一定的要准确, 否则会影响后面点的比较。



Mode=26: 2D 比较, 与 HW_TIMER 复用

HW_PSWITCH2(26, opnum, opstate, maxerr, num, tablepos, [ophwtimeus, ophwtimes, hwcycetimeus])

说明: 比较点写在 TABLE 中, 两个连续的 TABLE 数据组成一个 2D 坐标, 每到达一个比较位置触发 OP, 每个比较点 OP 反转的次数和反转周期由 HW_TIMER 设置; 到达下一个 TABLE 位置, OP 再次触发。类似模式 7 和模式 36。

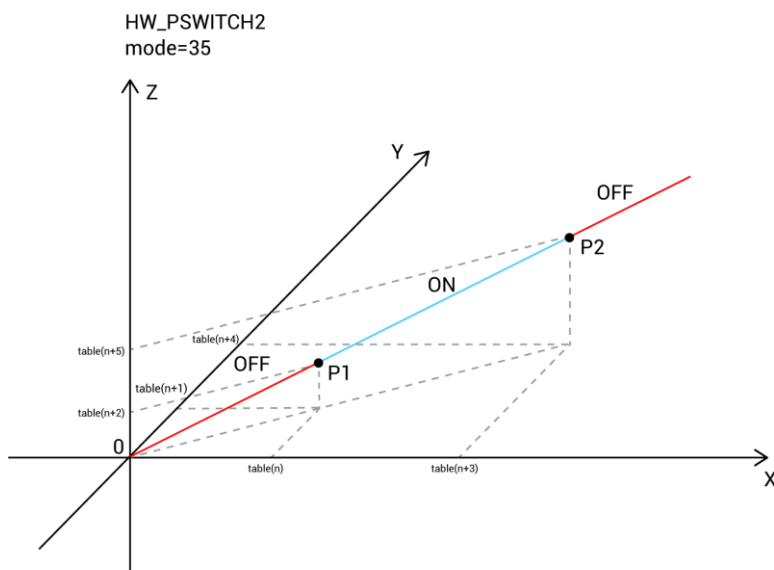


Mode=35: 3D 比较

HW_PSWITCH2(35, opnum, opstate, maxerr, num, tablepos)

说明：比较点写在 TABLE 中，三个连续的 TABLE 数据组成一个 3D 坐标，每到达一个比较位置 OP 反转一次。

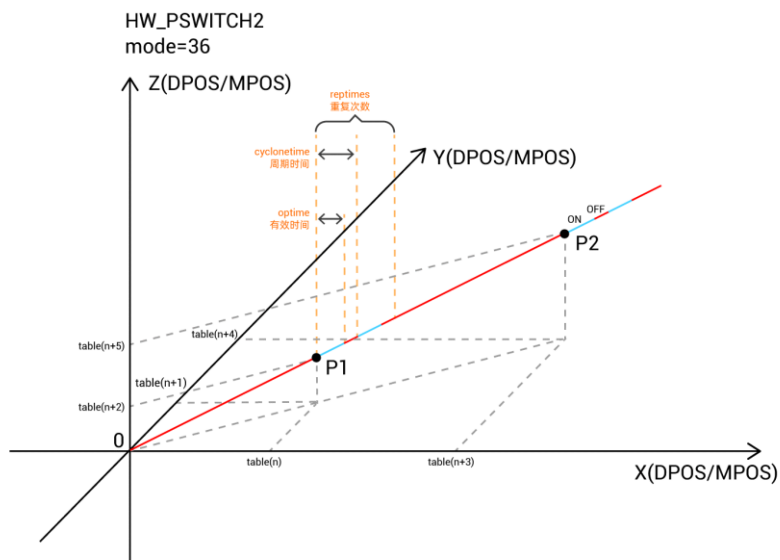
示意中蓝色段表示 OP 开启，各类常用插补运动均支持比较，比较点坐标一定的要准确，否则会影响后面点的比较。



Mode=36: 3D 比较，与 HW_TIMER 复用

HW_PSWITCH2(36, opnum, opstate, maxerr, num, tablepos, [ophwtimeus, ophwtimes, hwcycytimeus])

说明：比较点写在 TABLE 中，三个连续的 TABLE 数据组成一个 3D 坐标，每到达一个比较位置触发 OP，每个比较点 OP 反转的次数和反转周期由 HW_TIMER 设置；到达下一个 TABLE 位置，OP 再次触发。类似模式 26 和模式 7。



适用控制器

3 系列部分、4 系列及以上产品，4 系列固件 20170704 及以上

例子

以下例程测试环境：ZMC432，固件版本 20190128

例一 mode=1, 单轴比较

BASE(0)

DPOS=0

MPOS=0

OP(0,OFF)

TABLE(0,50,100,150,200) '比较点坐标设置

HW_PSWITCH2(2) '停止并删除没有完成的比较点'

HW_PSWITCH2(1, 0, 1, 0, 3,1) '比较 4 个点，操作输出口 0

TRIGGER 触发示波器

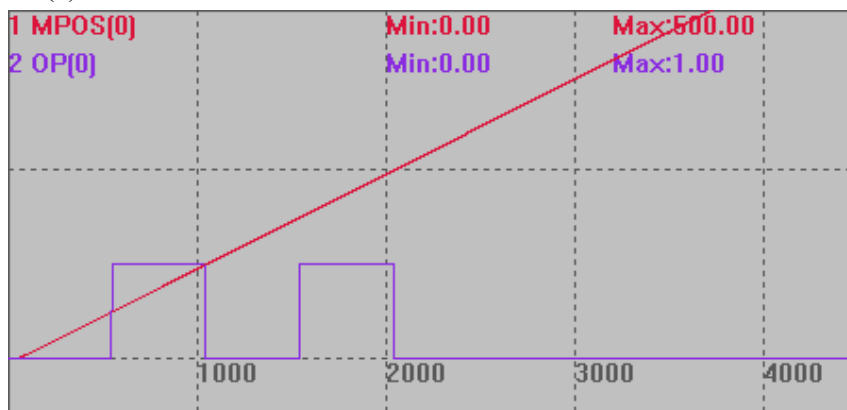
MOVE(500)

比较输出图

运动到 50 时，打开 OUT0；位置 100 时，关闭 OUT0；位置 150 时，打开 OUT0；位置 200 时，关闭 OUT0。

MPOS(0)垂直刻度 200

OP(0)垂直刻度 2



例二 mode=3, 矢量单轴比较模式

BASE(0)

DPOS=0

MPOS=0

OP(0,OFF)

TABLE(0,50,100,150,200) '比较点坐标设置

VECTOR_MOVED=120 '设置矢量起始位置, 会影响比较点

HW_PSWITCH2(2) '停止并删除没有完成的比较点

HW_PSWITCH2(3, 0, 1, 0, 3) '比较 4 个点, 操作输出口 0

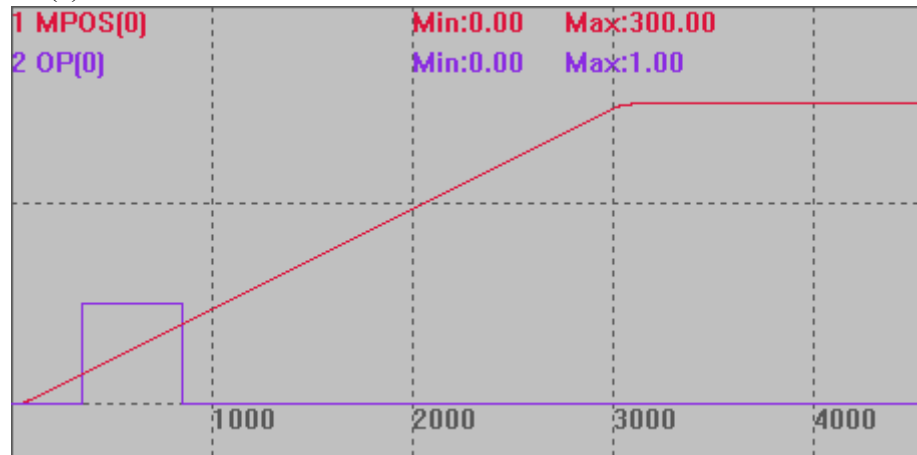
TRIGGER '触发示波器

MOVE(300)

比较输出图

MPOS(0)垂直刻度 200

OP(0)垂直刻度 2



可以看出信号操作直接从例一的位置 120 处开始, 只比较了后面两个点, 矢量比较模式就是将 MPOS(当前位置)+VECTOR_MOVED(之前的位移)与设置位置进行比较。

例三 mode=3, 多轴矢量比较模式, XY 加工模式的 PSO 应用

RAPIDSTOP(2)

WAIT IDLE(0)

WAIT IDLE(1)

BASE(0,1)

ATYPE=1,1

SPEED=100,100

ACCEL=1000,1000

DECEL=1000,1000

SRAMP=100,100

DPOS=0,0

MPOS=0,0

OP(0,OFF)

TABLE(0,50,100,150,200) '比较点坐标设置

```

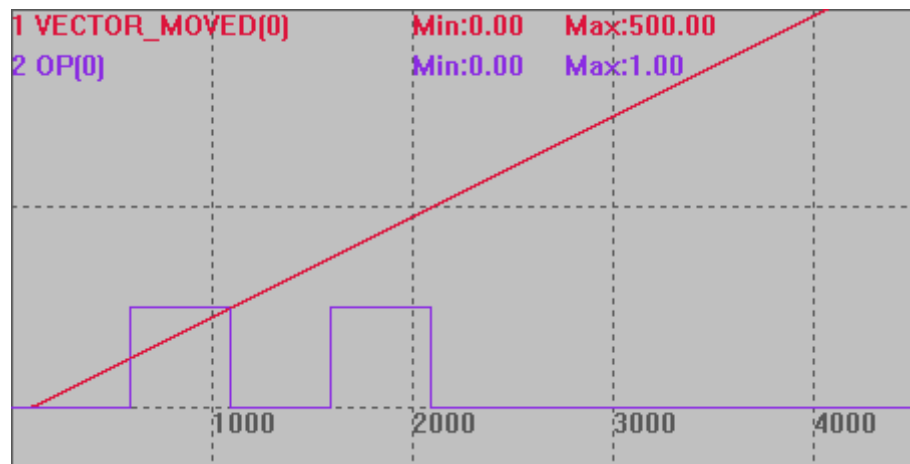
VECTOR_MOVED=0      '设置矢量起始位置
HW_PSWITCH2(2)      '停止并删除没有完成的比较点
HW_PSWITCH2(3, 0, 1, 0, 3)  '比较 4 个点，操作输出口 0
TRIGGER              '触发示波器
MOVE(300,400)
END

```

比较输出图：按 XY 轴合成插补的矢量位置比较

VECTOR_MOVED(0)垂直刻度 200

OP(0)垂直刻度 2



例四 mode=4，单点矢量比较

```
BASE(0)
```

```
DPOS=0
```

```
MPOS=0
```

```
OP(0,OFF)
```

```
VECTOR_MOVED(0) = 0      '设置当前的矢量位置
```

```
HW_PSWITCH2(2)          '停止并删除没有完成的比较点
```

```
HW_PSWITCH2(4, 0, 1, 100)  '启动比较输出，模式 4，输出口 0，第一个比较点
                             输出 ON，从矢量位置 100 开始比较，仅比较 1 次就结束
```

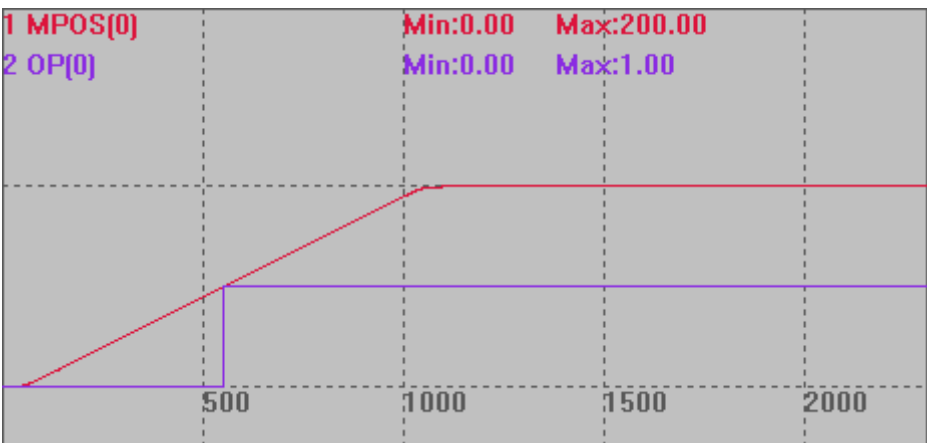
```
TRIGGER                  '触发示波器抓图
```

```
MOVEABS(200)
```

比较输出图

MPOS(0)垂直刻度 100

OP(0)垂直刻度 2



例五 mode=5，周期矢量比较模式，距离复位

BASE(0)

DPOS=0

MPOS=0

OP(0,OFF)

VECTOR_MOVED=0 '设置矢量起始位置为 0，方便观察

HW_PSWITCH2(2) '停止并删除没有完成的比较点

HW_PSWITCH2(5,0,1,100,3,150,50) '位置 100 开始比较，比较 3 次，周期距离 150，输出有效距离 50

TRIGGER '触发示波器

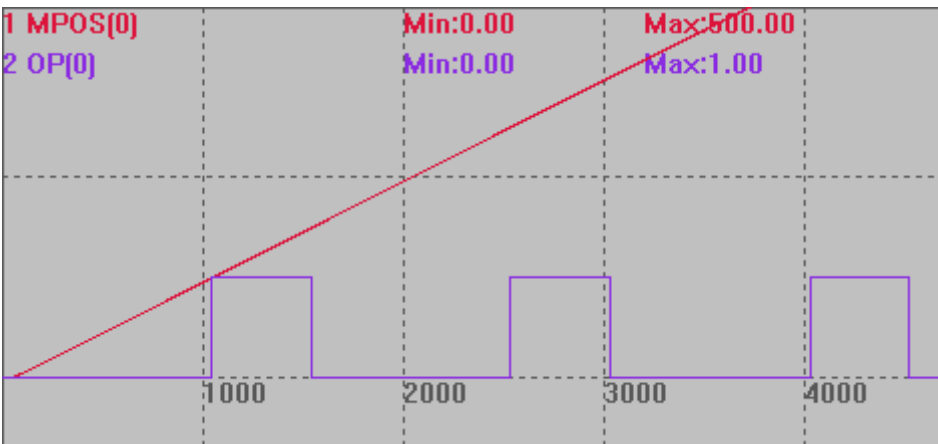
MOVE(500)

比较输出图

位置 100 开始比较，比较 3 次，一个和周期距离 150，前 50 输出有效状态，后 100 输出无效状态。

MPOS(0)垂直刻度 200

OP(0)垂直刻度 2



例六 mode=6，单轴周期矢量模式，时间复位

BASE(0)

DPOS=0

MPOS=0

OP(0,OFF)

VECTOR_MOVED=0 '设置矢量起始位置为 0，方便观察

HW_PSWITCH2(2) '停止并删除没有完成的比较点

HW_PSWITCH2(6,0,1,100,4,100) '位置 100 开始比较，比较 4 次周期距离 100，输出有效时间由 HW_TIMER 指令确定

HW_TIMER(2,100000,50000,1,off,0) '输出变为 on 后 50ms 变为 off

TRIGGER

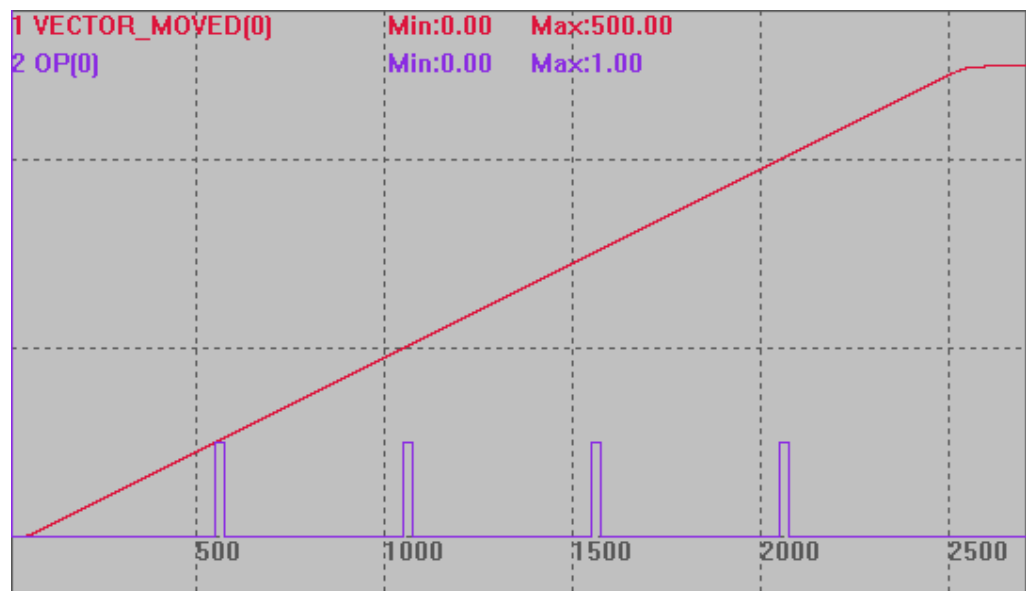
MOVE(500)

比较输出图

位置 100 时开始比较，比较 4 次，输出有效信号后 50ms 反转。

VECTOR_MOVED(0)垂直刻度 200

OP(0)垂直刻度 2



例七 mode=6，多轴周期矢量模式，时间复位

BASE(0,1)

DPOS=0,0

MPOS=0,0

OP(0,OFF)

TABLE(0,50,100,150,200) '比较点坐标设置

VECTOR_MOVED=0 '设置矢量起始位置为 0，方便观察

HW_PSWITCH2(2) '停止并删除没有完成的比较点

HW_PSWITCH2(6,0,1,100,4,100) '位置 100 开始比较，比较 4 次周期距离 100，输出有效时间由 HW_TIMER 指令确定

HW_TIMER(2,100000,50000,1,off,0) '输出变为 on 后 50ms 变为 off

TRIGGER

MOVE(300,400)

比较输出图

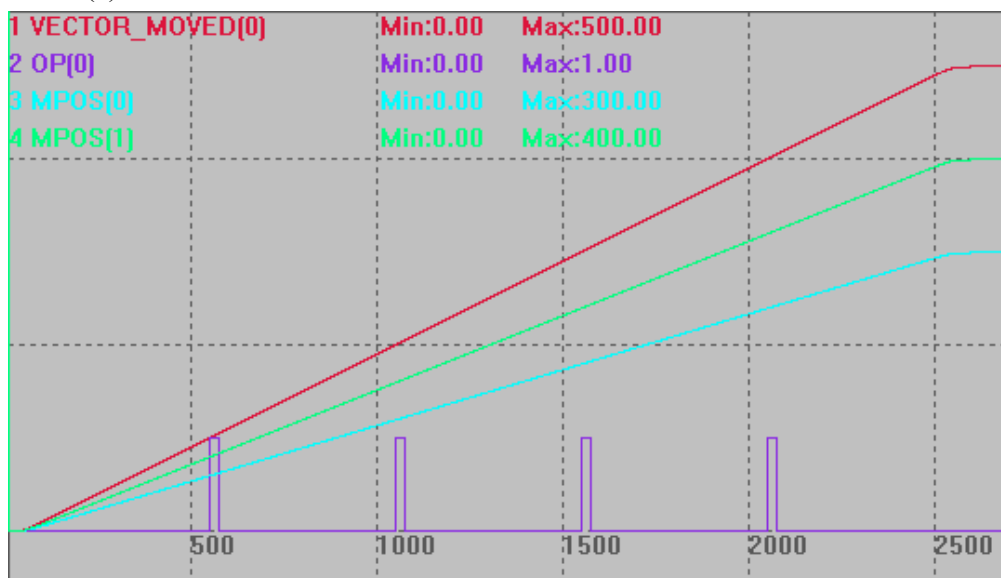
坐标均为两轴的合成矢量位置, 位置 100 时开始比较, 比较 4 次, 输出有效信号后 50ms 反转。

VECTOR_MOVED(0) 垂直刻度 200

OP(0)垂直刻度 2

MPOS(0)垂直刻度 200

MPOS(1)垂直刻度 200



例八 mode=7

BASE(0)

DPOS=0

MPOS=0

OP(0,OFF)

TABLE(0,50,100,150,200,250,300,350,400,450,500,550,600,650,700) '比较点坐标设置

VECTOR_MOVED=0 '设置矢量起始位置为 0, 方便观察

HW_PSWITCH2(2) '停止并删除没有完成的比较点

HW_TIMER(0, 1000000, 500000, 2, OFF, 0) '模式 0 停止硬件定时器

HW_TIMER(2,100000,50000,1,off,0) '输出变为 on 后, 硬件定时器触发开始计时, 50ms 后切换为 off

HW_PSWITCH2(7,0,1,0,13) '位置从 table(0)开始比较, 比较 14 个点, 操作输出口 0

TRIGGER '触发示波器

MOVE(1000)

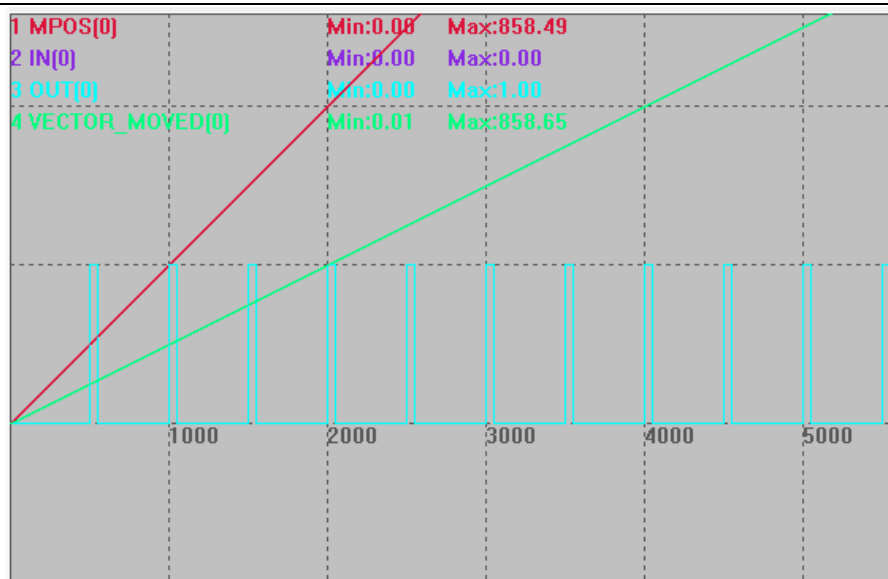
比较输出图

坐标均为两轴的合成矢量位置, 位置从 table(0)开始比较, 比较 14 个点, 输出有效信号后 50ms 反转。

VECTOR_MOVED(0)垂直刻度 200

OUT(0)垂直刻度 1

MPOS(0)垂直刻度 100



例九 mode=7

BASE(0)

DPOS=0

MPOS=0

OP(0,OFF)

TABLE(0,50,100,150,200,250,300,350,400,450,500,550,600,650,700) '比较点坐标设置

VECTOR_MOVED=0 '设置矢量起始位置为 0，方便观察

HW_PSWITCH2(2) '停止并删除没有完成的比较点

HW_TIMER(0, 1000000, 500000, 2, OFF, 0) '模式 0 停止硬件定时器

HW_TIMER(2,100000,50000,1,off,0) '输出变为 on 后,硬件定时器触发开始计时,50ms 后切换为 off

HW_PSWITCH2(7,0,1,0,13,100000,2,200000)'比较 14 个点，操作输出口 0，动态调整硬件定时器有效时间为 100ms 和周期时间 200ms、重复次数为 2

TRIGGER '触发示波器

MOVE(1000)

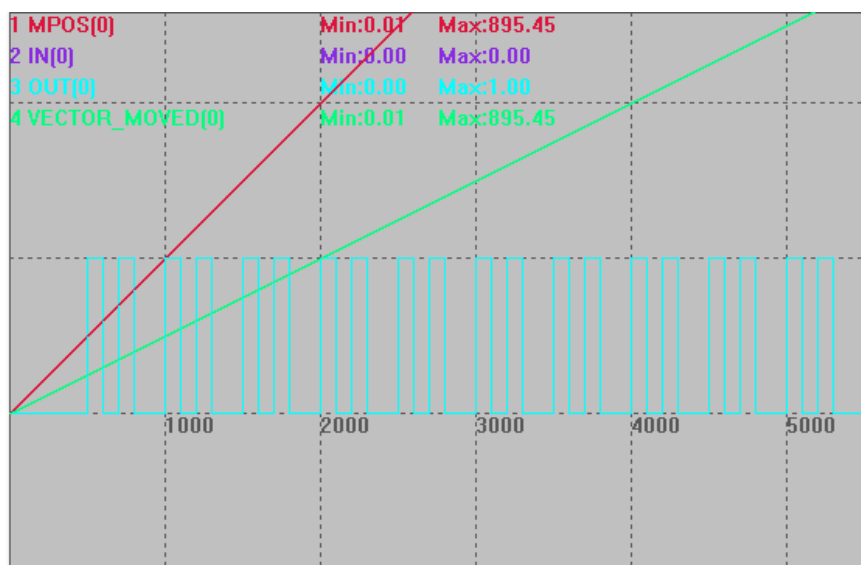
比较输出图

坐标均为两轴的合成矢量位置，位置从 table(0)开始比较，比较 14 个点，输出有效信号后 50ms 反转。

VECTOR_MOVED(0)垂直刻度 200

OUT(0)垂直刻度 1

MPOS(0)垂直刻度 100



例十 mode=25，二维硬件位置比较输出

BASE(0,1) '选择 XY 轴

UNITS=1000,1000

SPEED=100,100

ACCEL=10000,10000

DECEL=10000,10000

'将当前位置设置为 0 点

DPOS=0,0

MPOS=0,0

'提前写入位置比较点 XY 坐标到 table 10~49 中

TABLE(10, 10,0,12,0, 20,0,22,0, 30,0,32,0, 40,0,42,0, 50,0,52,0, 52,10,52,12, 52,20,52,22,
52,30,52,32, 52,40,52,42, 52,50,52,52)

GLOBAL pointNum '比较点数

pointNum=20

GLOBAL startX,startY '起点

startX=0

startY=0

GLOBAL midX,midY '中点

midX=52

midY=0

GLOBAL endX,endY '终点

endX=52

endY=52

```

WHILE 1
    IF TABLE(0)=1 THEN '比较脉冲位置,非精准输出
        WAIT IDLE
        ?"比较脉冲位置,非精准输出"
        '设置参数
        SYSTEM_ZSET=1
        AXIS_ZSET=1,1

    ELSEIF TABLE(0)=2 THEN '比较脉冲位置,精准输出
        WAIT IDLE
        ?"比较脉冲位置,精准输出"
        '设置参数
        SYSTEM_ZSET=3
        AXIS_ZSET=3,3

    ELSEIF TABLE(0)=3 THEN '比较编码器位置,非精准输出
        WAIT IDLE
        ?"比较编码器位置,非精准输出"
        '设置参数
        SYSTEM_ZSET=17
        AXIS_ZSET=17,17

    ELSEIF TABLE(0)=4 THEN '比较编码器位置,精准输出
        WAIT IDLE
        ?"比较编码器位置,精准输出"
        '设置参数
        SYSTEM_ZSET=19
        AXIS_ZSET=19,19
    ENDIF

    IF TABLE(0)<>0 AND IDLE(0)=-1 THEN
        TABLE(0)=0
        ?"启动"

        HW_PSWITCH2(2) '先清除所有的比较
        HW_PSWITCH2(25,0,1,10,pointNum,10) '写入比较, 操作输出口 0

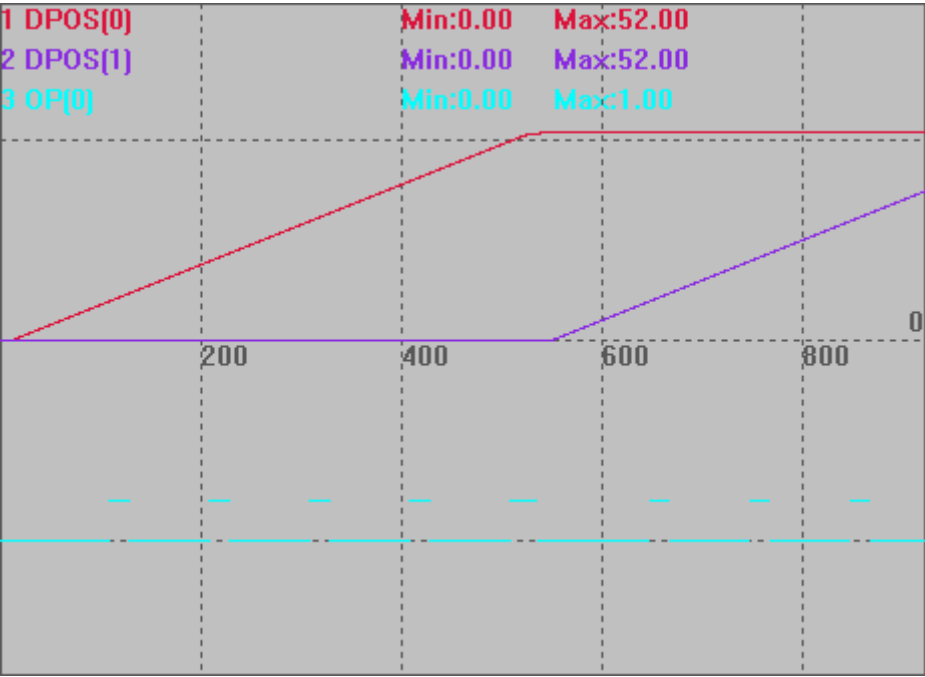
        WA 10
        TRIGGER '启动示波器

        MOVEABS(startX,startY) '开始运动
        MOVEABS(midX,midY)
        MOVEABS(endX,endY)
    ENDIF

```

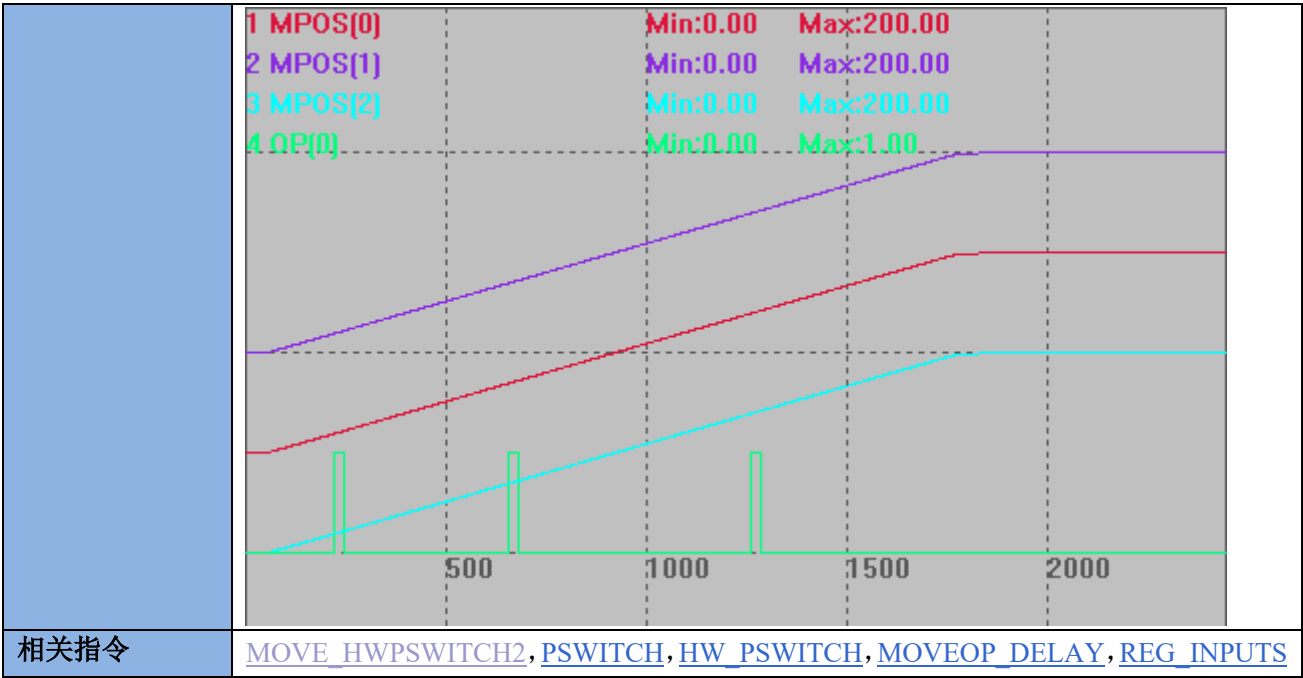
WEND
END

比较输出图
DPOS(0)垂直刻度 50
DPOS(1)垂直刻度 50
OP(0)垂直刻度 5



例十一 mode=35，三维硬件位置比较输出
BASE(0,1,2)
DPOS=0,0,0 '将当前位置设置为 0
MPOS=0,0,0
OP(0,OFF)
TABLE(0, 20,20,20, 40,40,40, 70,70,70, 100,100,100, 140,140,140, 180,180,180)
HW_PSWITCH2(2) '停止并删除没有完成的比较点
HW_PSWITCH2(35, 0, 1, 10, 6, 0) '启动比较输出，模式 35，输出口 0，第一个比较点输出 ON，脉冲偏差 10，table 地址 0-18，6 个坐标
TRIGGER '触发示波器抓图
MOVEABS(200,200,200) '走直线

比较输出图
MPOS(0)垂直刻度 200
MPOS(1)垂直刻度 200
MPOS(2)垂直刻度 200
OP(0)垂直刻度 2



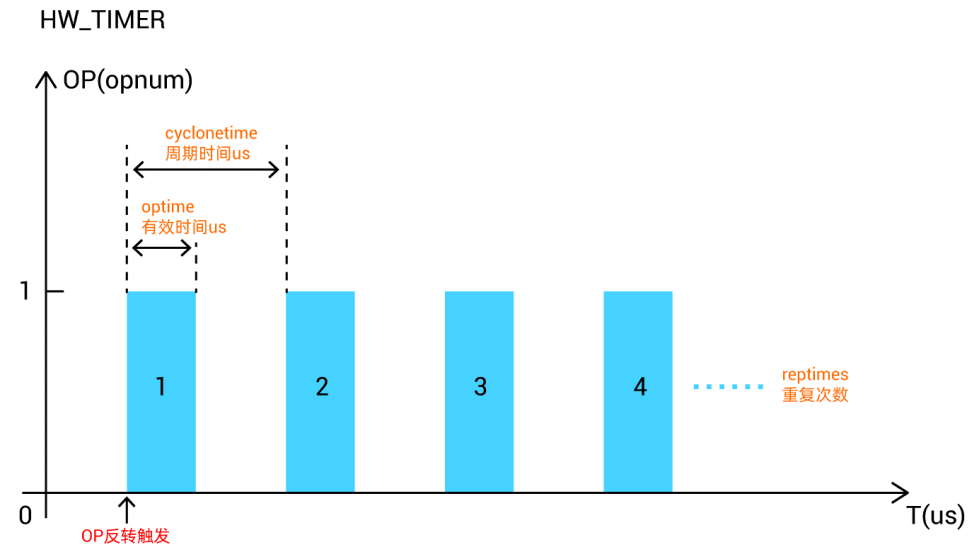
HW_TIMER -- 硬件定时

类型	特殊命令
描述	硬件定时器，用于硬件比较输出后一段时间后还原电平。 建议不超过 100ms 注意事项： HW 非独立的控制器只有一个 HW_TIMER，每次调用会强制停止之前的调用。 HW 独立的控制器，每路 HWOP 有一个 HW_TIMER。 ZMC420SCAN 每个输出出口的 HW_TIMER 功能独立。 HW_TIMER 不使用时需要使用模式 0 停止，否则将持续开启，可能影响后续的比较输出。 先用模式 2 开启硬件定时器，才能使用其他的模式修改。 振镜控制器 20170709 以上固件版本增加功能： OP 和 MOVE_OP 操作会关闭正在进行的 HW_TIMER 脉冲，这样可以使用 HW_TIMER 来实现类似 PWM 的功能，OP 输出打开脉冲输出，下一个 OP 输出关闭脉冲输出，当使用 MOVE_OP 精准输出时，可以实现精准的 PWM 输出无限脉冲功能。 使用?*HW_TIMER 可以看到还有多少脉冲剩余。
语法	HW_TIMER(mode, cyclonetime, optime, reptimes, opstate, opnum) done = HW_TIMER_DONE mode: 0-停止硬件定时器，1-动态修改参数（不修改启动设置），2-启动（启动后不可重复开启），3--停止硬件定时器（类似 0，针对振镜控制器），当前脉冲完成后不再输出，但后面的 OP 仍会继续触发脉冲 cyclonetime: 周期时间，us 单位 optime: 有效时间，us 单位

reptimes: 重复次数, 启动模式, reptimes =0 时, 软关闭 HW_TIMER, 原来的脉冲没有完成的, 会继续输出完成; -1 时无限输出, 除非主动关闭

opstate: 输出缺省状态, 输出口变为非此状态后开始计时 (输出口初始状态 OFF。一般此参数设为 OFF, 将输出口变为 ON 状态后开始计时)

opnum: 输出口编号, 必须能硬件比较输出的口



振镜控制器 20170710 以上版本增加 mode 3, 类似 0, 当前脉冲完成后不再输出, 但后面的 OP 仍会继续触发脉冲。而模式 0 停止之后无法 OP 继续触发, 必须重新启动。

振镜控制器 20170710 以上版本增加 mode 1, 1 支持再次发送之后动态修改定时的参数, 模式 1 不可用于开启硬件定时器, 在模式 2 没有比较完之前执行模式 1 才会生效; 模式 2 不可重复开启。

ZMC420SCAN 221017 增加 mode4, 脉冲 duty 变化功能, 配合模式 1/2 使用。

HW_TIMER(mode4, [trigremaintime,][changetimes,][changeonetick], reserve, opnum)

trigremaintime: 剩余多少个脉冲数时开始调整脉冲的宽度, 注意第一个脉冲不变化, 0-表示不使用

changetimes: 变化脉冲个数

changeonetick: 每个脉冲的宽度变化, us 单位, 可以为负数

reserve: 预留, 设为 0 即可

opnum: 输出口编号, 必须能硬件比较输出的口

应用此模式脉冲宽度要合理设置, 脉冲宽度累加后超出周期时, 后续无法正常输出, 模式 4 不使用时将第 2-4 的参数设为 0, 否则下次仍然生效。

适用控制器

3 系列部分、4 系列及以上产品, 4 系列固件 20170704 及以上

例子

以下例程测试环境: ZMC420SCAN, 固件: 221017 (仿真器无法运行此指令)。为了直观显示波形, 周期设置的较大。

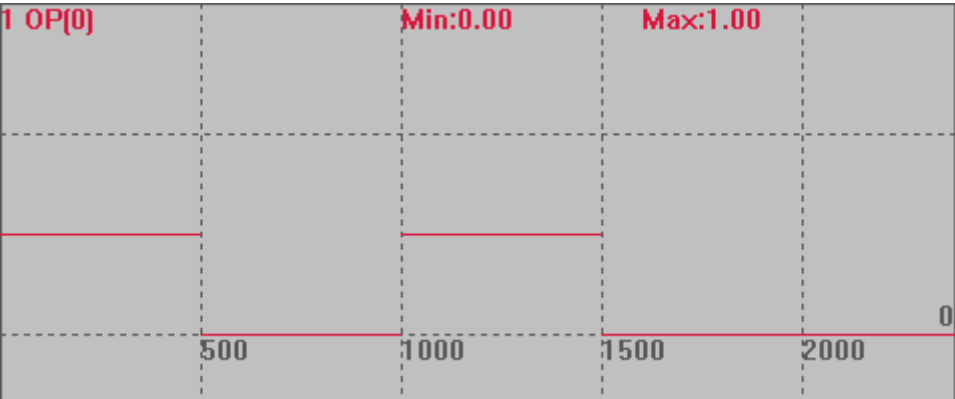
例一: 模式 2, 周期输出脉冲

RAPIDSTOP(2)

WAIT IDLE(0)

```
BASE(0)
ATYPE=1
UNITS=100
SPEED=100
ACCEL=500
DPOS=0
HW_TIMER(0, 1000000, 500000, 2, OFF, 0) '模式 0 停止硬件定时器
TRIGGER
OP(0, OFF)
HW_TIMER(2, 1000000, 500000, 2, OFF, 0) '输出口 0 变为 on 后，硬件定时器触发开始计时，500ms 后切换为 off
OP(0, ON)
```

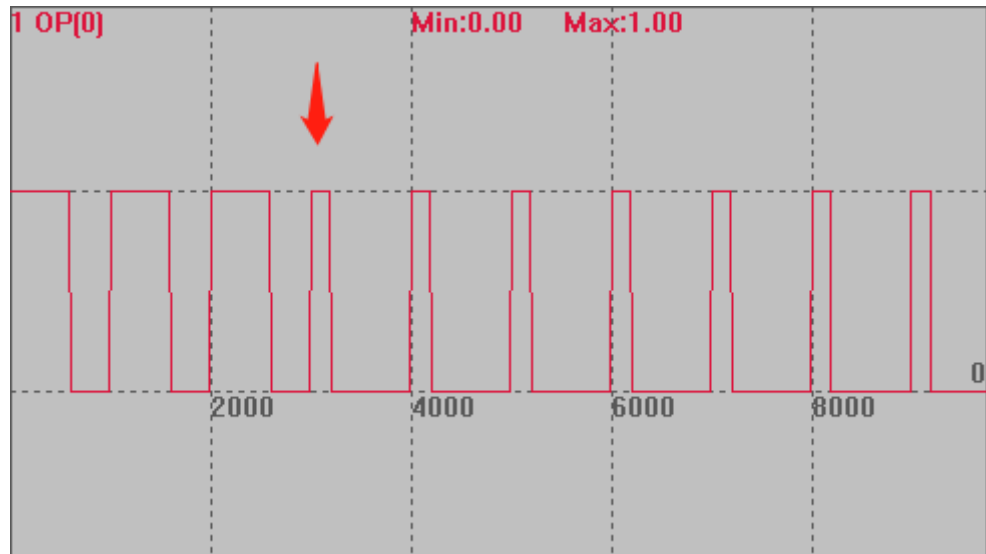
运行效果：以 1000ms 为周期，输出两个周期，每个周期前 500ms 开启，后半周期关闭。



例二：模式 1，周期输出脉冲；模式 2 开启之后，再使用模式 1 动态修改

```
BASE(0)
ATYPE=1
UNITS=100
SPEED=100
ACCEL=500
DPOS=0
OP(0, OFF)
TRIGGER
HW_TIMER(2,1000000,600000,10,OFF,0) '输出口 0 变为 on 后，硬件定时器触发开始计时，600ms 后切换为 off
OP(0, ON)
DELAY(2500) '2.5s 之后模式 1 生效，修改定时时间，时间不能太大，模式 2 若比较完成，模式 1 将无法生效。
HW_TIMER(1, 1000000, 200000, 5, OFF, 0) '动态修改
```

模式 1 可修改上一条的定时时间（上一条可以是模式 1 或模式 2），此例若改成模式 2，再次扫描到模式 2 时 OP 不动作。



例二 模式 3，停止输出，之后仍能 OP 触发

BASE(0)

ATYPE=1

UNITS=100

SPEED=100

ACCEL=500

DPOS=0

OP(0, OFF)

TRIGGER

HW_TIMER(2,1000000,400000,10,OFF,0) 输出口 0 变为 on 后，硬件定时器触发开始计时，400ms 后切换为 off

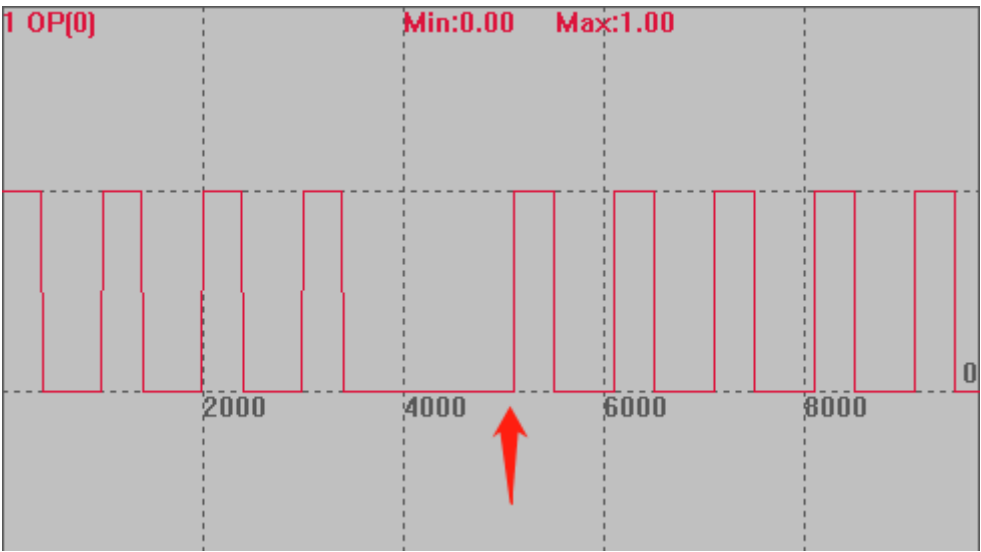
OP(0, ON)

DELAY(3000) '延时 3s 后关闭输出

HW_TIMER(3, 1000000, 400000, 10, OFF, 0)

OP 打开，模式 2 正常输出脉冲，3s 后模式 3 生效，停止输出，再次开启 OP 触发 HW，仍能继续输出 10 次。（箭头处为 OP 再次触发时刻）

此次若使用模式 0，停止后再次开启 OP 无法触发 HW，需重新扫描 HW 命令。



例三

BASE(0)

ATYPE=1

UNITS=100

SPEED=100

ACCEL=500

DPOS=0

OP(0, OFF)

TRIGGER

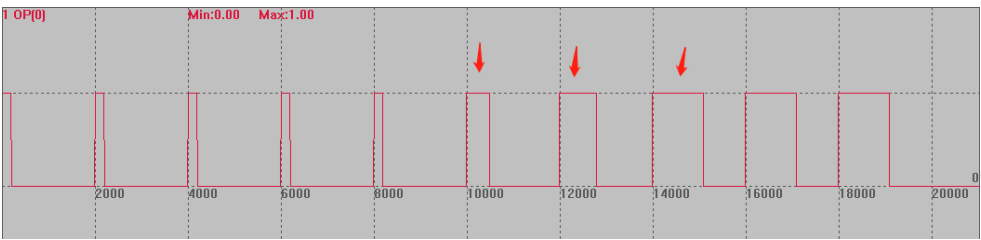
HW_TIMER(2,2000000,200000,10,OFF,0) '输出口 0 变为 on 后，硬件定时器触发开始计时，2s 周期，打开 200ms 后切换为 off

OP(0, ON)

HW_TIMER(4,6,3,300000,OFF,0) '模式 4 用于控制部分脉冲的宽度。

'HW_TIMER(4,0,0,0,OFF,0) '比较完成，关闭模式 4

如下图：模式 2 的作用下产生了 10 个脉冲，模式 4 控制倒数第六个脉冲之后开始生效（因为第一个不变化，故倒数第 6 个无变化，从倒数第 5 个开始生效），连续控制 3 个脉冲宽度，每个脉冲依次累加 300ms 后再固定宽度输出。



相关指令

[HW_PSWITCH2](#)

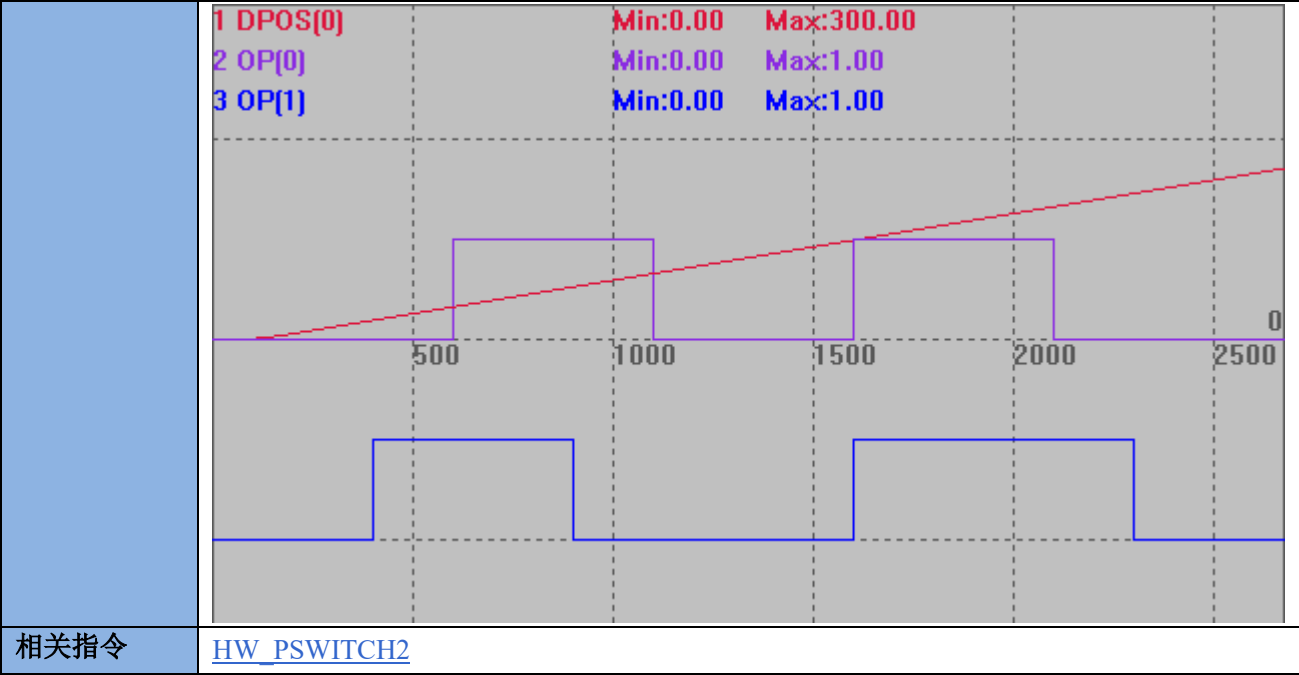
HW_MINTIME -- HW 最小时间间隔

类型	系统参数
描述	设置 HW 精准输出的最小时间间隔(毫秒单位)。 HW_MINTIME 设置大于 HW_TIMER 的脉宽，小于最小间隔。 XPCle1032H, RT0925后的版本PSO模式6触发时机说明 设: 硬件位置比较反馈实际位置间隔时间为: Ts 触发周期时间为: Tcyc 触发有效周期时间为: Top 两次触发最小间隔时间为: Tmin 指令时间参数优先级: HW_MINTIME>HW_TIME HW_WITCHPS2>HW_TIME 触发周期Tcyc (500us) 有效周期Top 250us 两次触发最小间隔Tmin 300us 实际位置反馈间隔时间Ts 600us 1 2 3 4 5 OP MSPEED 抖动 MPOS 两次触发最小间隔Tmin 300us触发点 目标位置比较输出点 实际位置反馈间隔时间Ts 600us触发点 TIME
语法	HW_MINTIME(opnum)=expression opnum: 输出口编号，必须能硬件比较输出的口
适用控制器	特殊固件支持
例子	HW_MINTIME(1)=1 ‘输出口口 1 前后两次 HW 输出间隔至少 1ms.
相关指令	HW_PSWITCH2 、 HW_TIMER

HW_PS2AXISNUM -- 设置 PS2 轴号

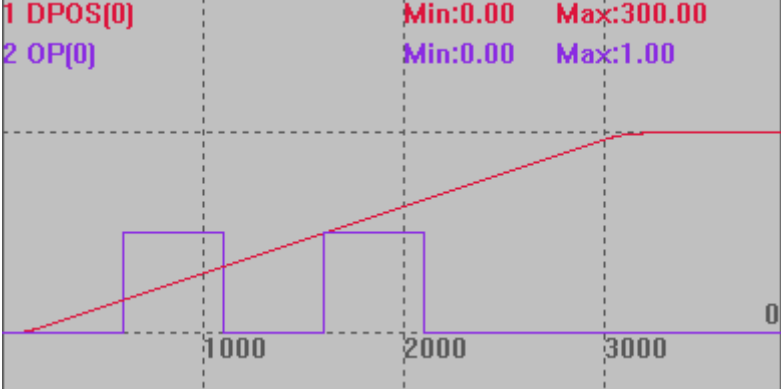
类型	轴参数
描述	设置 HW_PSWITCH2 指令实际操作轴号，缺省-1 表示不修改。 用于把没有使用的轴的 HW_PSWITCH2 缓冲重复利用起来，指向当前运动的主轴，可以

	对当前的主轴同时做多个比较。
语法	HW_PS2AXISNUM(axisnum1)=axisnum2 axisnum1: 缓冲轴号 axisnum2: 实际操作的轴号
适用控制器	4 系列 170705 以后固件版本支持
例子	以下例程测试环境：ZMC432，固件：20170709（仿真器无法运行此指令）。 RAPIDSTOP(2) WAIT IDLE(0) WAIT IDLE(1) BASE(0,1) ATYPE=1,1 UNITS=100,100 SPEED=100,100 ACCEL=500,500 DPOS=0,0 TRIGGER OP(0, OFF) OP(1, OFF) HW_PS2AXISNUM(1)=0 '使用轴 1 的缓冲区，比较轴 0 的位置 VECTOR_MOVED =0 TABLE(0,50,100,150,200) '第一个比较点坐标设置 TABLE(10,30,80,150,220) '第一个比较点坐标设置 HW_PSWITCH2(1, 0, 1, 0, 3,1) '第 1 个比较 HW_PSWITCH2(1, 1, 1, 10, 13,1) AXIS(1) '第 2 个比较，使用 1 轴的比较缓冲，实际比较 0 轴的位置 MOVE(300) WAITIDLE(0) HW_PS2AXISNUM(1)=-1 '取消 END



HW_PS2COUNTS -- PS2 比较的点数

类型	轴状态
描述	HW_PS2COUNTS 指令实际已经比较的点数，HW_PS2COUNTS(2)时清 0。
语法	VAL= HW_PS2COUNTS (axisnum) axisnum: 轴号
适用控制器	4 系列 170706 以后固件版本支持
例子	以下例程测试环境：ZMC432，固件：20170711（仿真器无法运行此指令）。 RAPIDSTOP(2) WAIT IDLE(0) BASE(0) ATYPE=1 UNITS=100 SPEED=100 ACCEL=500 DPOS=0 MPOS=0 OP(0,OFF) TABLE(0,50,100,150,200) '比较点坐标设置 HW_PS2COUNTS(2) '停止并删除没有完成的比较点 HW_PS2COUNTS(1, 0, 1, 0, 3,1) '比较 4 个点，操作输出口 0 TRIGGER '触发示波器 MOVE(300)

	<pre>?HW_PS2COUNTS '打印 0，还没到比较点 WAIT IDLE(0) ?HW_PS2COUNTS '打印 4，比较了 4 个点 END</pre> 
相关指令	HW_PSWITCH2

12.4 PWM 控制指令

PWM_FREQ -- PWM 频率

类型	PWM 控制函数
描述	PWM 频率设置或读取。 PWM 只能通过设置占空比为 0 来关闭，不能通过设置 PWM 频率为 0 实现，不要将频率设为 0，PWM 频率一定要在 PWM 开关之前调整。
语法	PWM_FREQ (index, freq) 或 PWM_FREQ (index)=freq index: 编号，从 0 开始 freq: 频率，硬件 PWM 1M，软件 PWM 2k
适用控制器	支持 PWM 功能的控制器才支持此函数。
例子	PWM_FREQ (0)=1000 '1K 频率 ?PWM_FREQ (0)
相关指令	PWM_DUTY , MOVE_PWM

PWM_DUTY -- PWM 占空比

类型	PWM 控制函数
描述	PWM 占空比设置或读取。 PWM 只能通过设置占空比为 0 来关闭，不能通过设置 PWM 频率为 0 实现，PWM 频率一定要在 PWM 开关之前调整。 占空比指有效电平占整个周期的比例。 一个周期中先输出有效电平，再输出无效电平。

	占空比为0.5  减小占空比  PWM 的实际输出受输出口的控制，必须输出口打开，PWM 才能输出，否则输出被屏蔽掉。可以先开 PWM 功能，然后再开输出口，这样实现激光电源的首脉冲抑制功能。
语法	PWM_DUTY(index, duty) PWM_DUTY(index)=duty index: 编号，从 0 开始 duty: 占空比，0-1，当设置 0 的时候，PWM 关闭
适用控制器	支持 PWM 功能的控制器才支持此函数。
例子	PWM_DUTY(0)=0.5 ?PWM_DUTY(0) 打印结果 0.5
相关指令	PWM_FREQ ， MOVE_PWM

12.5 蜂鸣器控制指令

SPEAKOUT -- 蜂鸣器控制

类型	蜂鸣器控制函数
描述	控制蜂鸣器发声
语法	SPEAKOUT(timems, [freq], [duty]) timems: 蜂鸣器发声的持续时间，单位 ms freq: 频率 duty: 占空比 频率和占空比设置一般都无效
适用控制器	通用
例子	SPEAKOUT(5000) '蜂鸣器响 5s

第十三章 通讯相关指令

13.1 串口通讯指令

SETCOM -- 串口配置

类型	系统指令																								
描述	串口的配置。 控制器重新上电后，SETCOM 参数会还原成默认值，所以请在程序开头写 SETCOM 设置。 ZMC00x 系列不支持 MODBUS 主端。 RS485 切换站号一般都是加要延时。																								
语法	SETCOM (baudrate,databits,stopbits,parity,port[,mode] [,variable] [,timeout]) baudrate: 串口波特率 9600 19200 4800 115200 38400(缺省) 57600 databits: 数据位数 8 stopbits: 停止位, 只能设置 0/1/2 parity: 是否校验: <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>无校验</td></tr> <tr> <td>1</td><td>奇校验</td></tr> <tr> <td>2</td><td>偶校验</td></tr> </table> port: 串口 PORT 编号 0-1, 参见 PORT 描述, 不同的控制器不一样 mode: 协议 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0</td><td>RAW 数据模式, 无协议, 此时可以使用 GET#, PRINT #</td></tr> <tr> <td>4 (缺省)</td><td>MODBUS 从端 (16 位整数)</td></tr> <tr> <td>14</td><td>MODBUS 主端 (16 位整数)</td></tr> <tr> <td>15</td><td>直接命令执行模式, 此时可以直接从串口输入字符串命令(换行符结束)</td></tr> </table> variable: 寄存器选择, 0-VR, 1-TABLE, 2-系统 MODBUS 寄存器 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0</td><td>VR, 此时一个 VR 映射到一个 MODBUS_REG VR 是 32 位浮点型, REG 是 16 位整数型, 从 VR 传递数据给 REG 会丢失小数部分, 当 VR 数据超过正负 15 位时, REG 数据会改变 REG 传递数据给 VR 不会有问題</td></tr> <tr> <td>1</td><td>TABLE, 此时一个 TABLE 数据映射到一个 MODBUS_REG (不推荐) TABLE 是 32 位浮点型, REG 是 16 位整数型, 从 TABLE 传递数据给 REG 会丢失小数部分, 当 TABLE 数据超过正负 15 位时,</td></tr> </table>	值	描述	0 (缺省)	无校验	1	奇校验	2	偶校验	值	描述	0	RAW 数据模式, 无协议, 此时可以使用 GET#, PRINT #	4 (缺省)	MODBUS 从端 (16 位整数)	14	MODBUS 主端 (16 位整数)	15	直接命令执行模式, 此时可以直接从串口输入字符串命令(换行符结束)	值	描述	0	VR, 此时一个 VR 映射到一个 MODBUS_REG VR 是 32 位浮点型, REG 是 16 位整数型, 从 VR 传递数据给 REG 会丢失小数部分, 当 VR 数据超过正负 15 位时, REG 数据会改变 REG 传递数据给 VR 不会有问題	1	TABLE, 此时一个 TABLE 数据映射到一个 MODBUS_REG (不推荐) TABLE 是 32 位浮点型, REG 是 16 位整数型, 从 TABLE 传递数据给 REG 会丢失小数部分, 当 TABLE 数据超过正负 15 位时,
值	描述																								
0 (缺省)	无校验																								
1	奇校验																								
2	偶校验																								
值	描述																								
0	RAW 数据模式, 无协议, 此时可以使用 GET#, PRINT #																								
4 (缺省)	MODBUS 从端 (16 位整数)																								
14	MODBUS 主端 (16 位整数)																								
15	直接命令执行模式, 此时可以直接从串口输入字符串命令(换行符结束)																								
值	描述																								
0	VR, 此时一个 VR 映射到一个 MODBUS_REG VR 是 32 位浮点型, REG 是 16 位整数型, 从 VR 传递数据给 REG 会丢失小数部分, 当 VR 数据超过正负 15 位时, REG 数据会改变 REG 传递数据给 VR 不会有问題																								
1	TABLE, 此时一个 TABLE 数据映射到一个 MODBUS_REG (不推荐) TABLE 是 32 位浮点型, REG 是 16 位整数型, 从 TABLE 传递数据给 REG 会丢失小数部分, 当 TABLE 数据超过正负 15 位时,																								

	<table><tr><td></td><td>REG 数据会改变 REG 传递数据给 TABLE 不会有问题</td></tr><tr><td>2（缺省）</td><td>系统 MODBUS 寄存器，此时 VR 与 MODBUS 寄存器是两片独立区间</td></tr><tr><td>3</td><td>VR_INT 模式，此时一个 VR_INT 映射到两个 MODBUS_REG</td></tr></table> timeout: MODBUS 从端时此参数也生效，意义为帧间最长延时时间(毫秒 ms)，旧固件缺省值 800ms，部分触摸屏在等待时间设置较低时，会导致故障不能恢复，开放后，针对不同的触摸屏可以设置这个值。4 系列控制器固件版本 20190203 增加，通过?*SETCOM 打印查看，如果有这个参数打印即为支持设置。 ⚠ variable 参数是全局的设置，所有的端口共一个。 ⚠ 当设置为 VR 或 TABLE 时，没有用到的输出口与输入口会连在一起。 ⚠ 当设置为 VR 或 TABLE 时，通用输出口会映射到 MODBUS_BIT(0)的位置，输入口会映射到 MODBUS_BIT(1000)的位置，因此此时不要使用 MODBUS_BIT 作为触摸屏的按钮，也不要使用 PLC 中的 M 寄存器。		REG 数据会改变 REG 传递数据给 TABLE 不会有问题	2（缺省）	系统 MODBUS 寄存器，此时 VR 与 MODBUS 寄存器是两片独立区间	3	VR_INT 模式，此时一个 VR_INT 映射到两个 MODBUS_REG
	REG 数据会改变 REG 传递数据给 TABLE 不会有问题						
2（缺省）	系统 MODBUS 寄存器，此时 VR 与 MODBUS 寄存器是两片独立区间						
3	VR_INT 模式，此时一个 VR_INT 映射到两个 MODBUS_REG						
适用控制器	通用						
例子	例一 mode0 RAW 模式 <pre>DIM char1 '定义变量 SETCOM(38400,8,1,0,0,0) '配置串口为 RAW 模式 WHILE 1 GET#0, char1 '发送给通道 0 的字符保存到 char1 PRINT char1 '按 ASCII 码打印通道 0 接受的字符 PUTCHAR#0, char1 '将接收的字符再发送回去 WEND</pre> 松下 A6 伺服编码器读取参照第十三章简易例程 例二 MODBUS 通讯设置 <pre>SETCOM(38400,8,1,0,0,4,2) '设置串口 0 为 modbus 从端，波特率 38400 SETCOM(38400,8,1,0,1,14,2,1000) '设置串口 1 为 modbus 主端，波特率 38400</pre> 具体例程参照 MODBUSM_DES 指令例程 例三 直接字符命令方式 <pre>setcom(38400, 8,1,0,0,15) '设置串口 0 为直接字符串命令方式</pre> 此时在串口调试助手或其他设备，直接发送相关指令可直接操作控制器。 数据发送 1. DCD 2. RXD 清除 清除 发送 发送前 UNITS=10000 发送后 UNITS=100						

	例四 寄存器模式 0	
	VR(0)=0	'初始化 VR(0)和 REG(0)为 0
	MODBUS_REG(0)=0	
	SETCOM(38400, 8,1,0,0,4,0)	'设置 VR 映射到 MODBUS_REG
	VR(0)=100.345	'设置 VR(0)=100.345
	?MODBUS_REG(0)	'打印结果为 100, VR 已经映射到 REG, 但是 REG 是整型, 所以小数部分丢失
	MODBUS_REG(0)=200	'REG(0)设为 200
	?VR(0)	'打印结果为 200, REG 变化也会改变 VR
	例五 寄存器模式 2	
	VR(0)=0	'初始化 VR(0)和 REG(0)为 0
	MODBUS_REG(0)=0	
	SETCOM(38400,8,1,0,0,4,2)	'设置 VR 与 MODBUS_REG 独立
	VR(0)=100.345	设置 VR(0)=100.345
	?MODBUS_REG(0)	'打印结果为 0, VR 不影响 REG
	MODBUS_REG(0)=200	'REG(0)设为 200
	?VR(0)	'打印结果为 100.345, REG 也不影响 VR
相关指令	ADDRESS , PROTOCOL , MODBUSM_DES , PORT	

ADDRESS -- 控制器站号

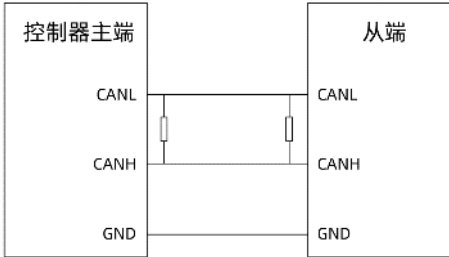
类型	系统参数	
描述	控制器的所有串口的 MODBUS 协议站号 1- 255, 缺省=1。	
语法	ADDRESS = value	
适用控制器	通用	
例子	PRINT ADDRESS	'打印出协议站号
	打印结果: 1	
相关指令	SETCOM , PORT , PROTOCOL	

COM_UNUSED -- 指定串口

类型	系统参数	
描述	按位指定串口是否被 RTBasic 使用。	
	bit0: 串口 1, 值设置 1 表示 RTBasic 不使用	
	bit1: 串口 2, 值设置 1 表示 RTBasic 不使用	
语法	COM_UNUSED = value	
适用控制器	5 系列控制器以上	
例子	COM_UNUSED=1	'RTBasic 不使用 RS232 串口
相关指令	SETCOM , PORT , PROTOCOL	

13.2 CAN 通讯指令

CAN -- CAN 通讯

类型	系统指令										
描述	直接通过 CAN 总线收发数据。 可以实现多个控制器通过 CAN 来通讯，但是同一个 CAN 网络上只能有一个主端 (CANIO_ADDRESS=32)。 较新的固件版本才支持这个功能，如不支持请联系厂家。 接线如下图所示：  CANL - CANL CANH - CANH CANL 和 CANH 的首尾两端分别连接 1 个 120 欧电阻用于阻抗匹配。控制器主端 CANL 和 CANH 接口并联 1 个 120 欧电阻；连接带拨码开关的扩展模块时，将拨码第八位拨为 ON 即表示接入 120 欧电阻，无需额外接电阻。										
语法	CAN(channel, function, tablenum) channel: CAN 通道，0 表示第一个通道，-1 表示缺省通道 function: 功能号 <table><tr><th>值</th><th>描述</th></tr><tr><td>6</td><td>接收，没有数据时，identifier<0</td></tr><tr><td>7</td><td>发送</td></tr><tr><td>16 需要升级固件</td><td>带扩展支持接收，没有数据时，identifier<0</td></tr><tr><td>17 需要升级固件</td><td>发送扩展数据，普通数据使用 7 发送</td></tr></table> tablenum: 数据存储的 TABLE 位置 6、7 模式时依次存储： identifier: Can 通讯对象(Cob-Id)，11 位，前 4 位是功能码，后 7 位是节点 ID，ZCAN 使用的高位数据预留，建议 0-511。接收时如果小于 0 则表明没有收到数据。bit11 表示是否远程帧。 bytes: 数据区域字节数，最多为 8。 data: 数据区域，字节（0-FF）。	值	描述	6	接收，没有数据时，identifier<0	7	发送	16 需要升级固件	带扩展支持接收，没有数据时，identifier<0	17 需要升级固件	发送扩展数据，普通数据使用 7 发送
值	描述										
6	接收，没有数据时，identifier<0										
7	发送										
16 需要升级固件	带扩展支持接收，没有数据时，identifier<0										
17 需要升级固件	发送扩展数据，普通数据使用 7 发送										

	16、17 模式时依次存储： identifier: Can 通讯对象(Cob-Id), 11 位, 前 4 位是功能码, 后 7 位是节点 ID, ZCAN 使用的高位数据预留, 建议 0-511。接收时如果小于 0 则表明没有收到数据。bit11 表示是否远程帧。 identifier extend: 扩展 id, 增加高 11 位与低 18 位, 共同组成 CAN 的 29 位 ID, 不存在填-1。 bytes: 数据区域字节数, 最多为 8。 data: 数据区域, 字节 (0-FF) 。 29 位扩展帧 ID 从右最后一位开始拆分为低 18 位和高 11 位, 高位不足 11 位自动补 0, 例子如下: 扩展帧 ID 值: 1753001 对应二进制为: 1 0111 0101 0011 0000 0000 0001 对应 29 位二进制: 0 0001 0111 0101 0011 0000 0000 0001 高 11 位为: 0 0001 0111 01, 对应 10 进制为 93 低 18 位为: 01 0011 0000 0000 0001, 对应 10 进制为 77825 所以使用 CAN17 模式时, identifier=93, Identifier extend=77825
适用控制器	通用
例子	例一 '发送端: TABLE(0,1,8,1,2,3,4,5,6,7,8) '发送 cobid=1, 8 个字节, 依次为 1-8 CAN(0,7,0) '发送 '接收端: CANIO_ADDRESS=1 '设置为非主控, 此参数设置一次即可 CAN(0,6,0) '接收 ?TABLE(0) 例二 '发送端 TABLE(0,1,10,8,1,2,3,4,5,6,7,8) '发送 cobid=1, 扩展 id10, 8 个字节, 依次为 1-8 CAN(0,17,0) '发送 '接收端: CANIO_ADDRESS=1 '设置为非主控, 此参数设置一次即可 CAN(0,16,0) '接收 ?TABLE(0)

	<pre>例三 CAN17 模式防阻塞，独立任务启动 当未连接通讯设备是 CAN17 会自动跳过，不会发生阻塞等待 CANIO_ADDRESS = 32 '主端 GLOBAL FLAG_SEND_NUM '发送次数 FLAG_SEND_NUM = 1000 WHILE 1 ?FLAG_SEND_NUM IF FLAG_SEND_NUM > 0 THEN TABLE(0,32,8,8,1,2,3,4,5,6,7,8) STOPTASK 2 RUNTASK 2,CAN_SEND WA 10 '稍微增加延时，确保后续发送进行时不会打断前面的发送 ENDIF IF FLAG_SEND_NUM <= 0 THEN ?"发送完毕" EXIT WHILE ENDIF WEND END GLOBAL SUB CAN_SEND() FLAG_SEND_NUM = FLAG_SEND_NUM - 1 CAN(0,17,0) ?"发送成功" END SUB</pre>
相关指令	CANIO_ADDRESS , CANIO_STATUS , CANIO_ENABLE

CANIO_ADDRESS -- CAN 通讯设置

类型	系统参数	
描述	控制器上 CAN 的 CANID 和速度的设置。	
	扩展板的 CAN 速度通过拨码开关设置。	
	自动存储到 FLASH，重启后生效。	
	低 8 位（位 0-7）表示 CANID，范围 0-32，缺省 32，32 表示为主控制器。	
	高 8 位（位 8-15）表示 CAN 速度。	
	高 8 位值	CAN 速度
	0	500 kbps（缺省值）
	1	250kbps

	<table border="1"> <tr> <td>2</td><td>125kbps</td></tr> <tr> <td>3</td><td>1 Mbps</td></tr> </table> 一个网络中，不要配置多个主控制器。 CAN 地址和速度设置必须重启后生效。	2	125kbps	3	1 Mbps
2	125kbps				
3	1 Mbps				
语法	CANIO_ADDRESS = value				
适用控制器	通用				
例子	CANIO_ADDRESS = 32 '设置主端，CAN 波特率 500kbps CANIO_ADDRESS = 32+ 256 '设置主端，CAN 波特率为 250kbps CANIO_ADDRESS = 32+ 512 '设置主端，CAN 波特率为 125kbps CANIO_ADDRESS = 32+ 768 '设置主端，CAN 波特率为 1Mbps CANIO_ADDRESS = 1 '设置 CANID 为 1，此时为从端，500 kbps，不能连接 IO 板 CANIO_ADDRESS = 1 +256 '设置 CANID 为 1，此时为从端，250 kbps，作为 ZCAN 从站使用 CANIO_ADDRESS = 1 +512 '设置 CANID 为 1，此时为从端，125 kbps，作为 ZCAN 从站使用 CANIO_ADDRESS = 1 +768 '设置 CANID 为 1，此时为从端，1 Mbps，作为 ZCAN 从站使用 CANIO_ADDRESS = 3 '设置 CANID 为 3，此时为从端，500 kbps，作为 ZCAN 从站使用 CANIO_ADDRESS = 8 +256 '设置 CAN ID 为 8，此时为从端，250 kbps，作为 ZCAN 从站使用 CANIO_ADDRESS = 16 +512 '设置 CAN ID 为 16，此时为从端，125 kbps，作为 ZCAN 从站使用 CANIO_ADDRESS = 31 +768 '设置 CAN ID 为 31，此时为从端，1Mbps，作为 ZCAN 从站使用 具体使用参考第十七章的 CAN 通讯例子				
相关指令	CANIO_ENABLE ， CAN ， CANIO_STATUS				

CANIO_ENABLE -- CAN 使能

类型	系统参数
描述	使能或禁止内部 CAN 主端功能。 当 CANIO_ADDRESS 设置 32 时，缺省是使能的。
语法	CANIO_ENABLE = ON/OFF
适用控制器	通用
例子	CANIO_ADDRESS = 32 '设置主端，CAN 波特率 500kbps CANIO_ENABLE = ON '打开 CAN 主端功能 CANIO_ENABLE = OFF '关闭 CAN 主端功能
相关指令	CANIO_ADDRESS

CANIO_STATUS -- CAN 扩展板状态

类型	系统状态
描述	获取当前的 IO 板的状态，返回 ON/OFF。 20140325 以上的固件版本支持
语法	CANIO_STATUS(cardnum) cardnum：该扩展模块的拨码 ID
适用控制器	通用
例子	例一 ?*CANIO_STATUS '输出所有 IO 板的状态 打印结果如下图：表示接入了三个扩展模块，拨码 ID 的组合值分别为 1，3，4。  <pre>>>?*CANIO_STATUS 0 1 0 1 1 0</pre> 在线命令： ?*CANIO_STATUS 发送 捕获 清除 <input checked="" type="button" value="命令与输出"/> 查找结果 例二 IF CANIO_STATUS(1) =0 THEN '判断 IO 板的连接状态 PRINT "扩展模块 1 没有连接好" ENDIF
相关指令	CAN , CANIO_ENABLE , CANIO_INFO CANIO_ADDRESS -- CAN 通讯设置

CANIO INFO -- CAN 扩展板信息

类型	系统状态																		
描述	读取当前的 IO 板的信息，返回参数值。 4 系列控制器 170715 以上固件版本支持。																		
语法	CANIO_INFO(canid, isel [, moduleid]) canid: 扩展模块的拨码 ID isel: 读取的参数 <table><tr><td>0</td><td>正运动</td></tr><tr><td>1</td><td>DEVICE，设备编号</td></tr><tr><td>2</td><td></td></tr><tr><td>3</td><td></td></tr><tr><td>4</td><td></td></tr><tr><td>IO 的个数</td><td>描述</td></tr><tr><td>10</td><td>IN 个数，整体的</td></tr><tr><td>11</td><td>OP 个数</td></tr><tr><td>12</td><td>AIN 个数</td></tr></table>	0	正运动	1	DEVICE，设备编号	2		3		4		IO 的个数	描述	10	IN 个数，整体的	11	OP 个数	12	AIN 个数
0	正运动																		
1	DEVICE，设备编号																		
2																			
3																			
4																			
IO 的个数	描述																		
10	IN 个数，整体的																		
11	OP 个数																		
12	AIN 个数																		

	13	AOUT 个数																													
	增加																														
	16	模块数																													
	17	子模块的类型号，必须后面带 moduleid																													
	预留																														
	20	子模块输入																													
	21	子模块输出																													
	22	子模块 AIN																													
	23	子模块 AOUT																													
	例如：耦合器连接的第一个扩展子模块，地址为 0；第二个扩展子模块，地址为 1；依此类推。																														
【ZMIO300-CAN/ZMIO310-CAN 修改扩展子模块模拟量 AD/DA 量程方法】：																															
CANIO_INFO(canid,17, moduleid)=量程类型																															
canid: 扩展模块的拨码 ID																															
moduleid: ZMIO 子模块的信息，扩展子模块地址按接入耦合器的顺序，从 0 依次编号。部分 isel 参数选择需要填这个参数。																															
量程类型：																															
<table><tr><td>类型编号</td><td>模块类型</td><td>类型编号</td><td>模块类型</td><td>对应量程</td></tr><tr><td>2</td><td rowspan="6">（输入模块）4AD 模块</td><td>10</td><td rowspan="6">（输出模块）4DA 模块</td><td>0~10V</td></tr><tr><td>3</td><td>11</td><td>-10~10V</td></tr><tr><td>4</td><td>12</td><td>4~20mA</td></tr><tr><td>5</td><td>13</td><td>0~20mA</td></tr><tr><td>6</td><td>14</td><td>0~5V</td></tr><tr><td>7</td><td>15</td><td>-5~5V</td></tr><tr><td>20</td><td>（输入模块）8AD 模块</td><td>21</td><td>（输出模块）8DA 模块</td><td>0~10V</td></tr></table>		类型编号	模块类型	类型编号	模块类型	对应量程	2	（输入模块）4AD 模块	10	（输出模块）4DA 模块	0~10V	3	11	-10~10V	4	12	4~20mA	5	13	0~20mA	6	14	0~5V	7	15	-5~5V	20	（输入模块）8AD 模块	21	（输出模块）8DA 模块	0~10V
类型编号	模块类型	类型编号	模块类型	对应量程																											
2	（输入模块）4AD 模块	10	（输出模块）4DA 模块	0~10V																											
3		11		-10~10V																											
4		12		4~20mA																											
5		13		0~20mA																											
6		14		0~5V																											
7		15		-5~5V																											
20	（输入模块）8AD 模块	21	（输出模块）8DA 模块	0~10V																											
<table><tr><td>类型编号</td><td>模块类型</td><td>对应量程</td></tr><tr><td>22</td><td>（输入+输出模块）8AD8DA 模块</td><td>0~10V</td></tr></table>		类型编号	模块类型	对应量程	22	（输入+输出模块）8AD8DA 模块	0~10V																								
类型编号	模块类型	对应量程																													
22	（输入+输出模块）8AD8DA 模块	0~10V																													
适用控制器	通用																														
例子	例一 ?CANIO_INFO(1,1) '打印 ID 是 1 的扩展模块的设备编号 例二 修改及查询第一个、第二个扩展子模块 AD/DA 的量程 CANIO_INFO(0,17, 0)=3 '修改查询 AD 量程为-10~10V ?CANIO_INFO(0,17, 0) '查询量程 CANIO INFO(0,17, 1)=11 '修改 DA 量程为-10~10V																														

	?CANIO_INFO(0,17, 1) '查询量程
相关指令	CAN , CANIO_ENABLE , CANIO_STATUS CANIO_ADDRESS -- CAN 通讯设置

13.3 自定义通讯指令

GET# -- 读取字符

类型	系统指令
描述	从 RAW 方式通讯或自定义网口通讯的通道里面读取一个字节，存入一个变量。
语法	语法 1: GET#PORT, VARIABLE 语法 2: GET#PORT, ARRAY[(startindex)] [,maxchares] 语法 3: charesget = GET#PORT, VARIABLE 语法 4: charesget = GET#PORT, ARRAY[(startindex)] [,maxchares] port: 通道号 variable: 存放的变量名 startindex: 存放数组的起始地址 maxchares: 存放的最多数量 语法 1、2 没有读取到会阻塞，这个函数一般在多任务里面进行调用。 语法 3、4 会返回读取到的字节数。 20150522 版本以前只支持语法 1。 UDP 接收必须使用语法 4，采用数组来接收，数组长度不要比一次的 UDP 包长度小。 UDP 每次都是读取一整个包，如果数组长度不够，多余的会丢弃掉。 UDP_SERVER 模式时，每收到一个包，PORT_TARGET 自动变为包的发送方，因此可以同时接收多个从端的数据。
适用控制器	通用
例子	例一 <pre> DIM VAR1 SETCOM(38400,8,1,0,0,0) '开启 RAW 方式 GET#0, VAR1 '从通道 0 读取数据 PRINT VAR1 '打印读取数据 </pre> 例二 <pre> DIM ARRAY1(101) SETCOM(38400,8,1,0,0,0) '开启 RAW 方式 CHARES = GET#0, ARRAY1, 100 '从通道 0 最多读取 100 个字符 If CHARES > 0 THEN ARRAY1(CHARES) = 0 '设置结束 0 PRINT ARRAY1 '字符串打印出来 </pre>

	ENDIF
相关指令	PORT , SETCOM , PRINT#

OPEN# -- 打开自定义网口通讯

类型	系统指令
描述	开启自定义的以太网通讯。 新固件版本提供此功能。
语法	OPEN#PORT, "mode", portnum [, ipaddress] port: 通讯通道, 参见 PORT 描述, 选择自定义网络通道 mode: 讯主从, “TCP_CLIENT” -客户端, “TCP_SERVER” - 服务器端, “UDP_CLIENT” -UDP 客户端, “UDP_SERVER” - UDP 服务器端, “TCP_EXCUTE” —使用直接 TCP 控制 portnum: TCP 或 UDP 端口号, 主端为本地端口号, 从端为对方端口号 ipaddress: 对方 IP 地址, 字符串, 从端的时候要提供 TCP_EXCUTE,表示使用直接 TCP 控制, TCP 发 BASIC 命令过来, 必须带\n 结束字符。 OPEN#port, "TCP_EXCUTE",portnum.进行服务端的监听设置。此远程端口号不能设置为 502, 其他端口号只要保证二边一致即可。 UDP_SERVER 必须先接收对方的数据, 才能发送回数据(除非用 PORT_TARGET 先强制指定对方)。至少 3s 通讯一次, 不然通讯容易自动断开。 UDP_CLIENT 本地端口号随机, 必须先发送给对方, 对方才能知道端口号, 此模式时不是指定对方的包会丢弃掉。 UDP 自定义通讯需要 4 系列控制器 20170628 以上固件版本; XPLC 系列控制器 20170702 以上固件版本。 ?*open 可打印所有 OPEN 的端口信息。
适用控制器	通用
例子	例一 OPEN#11, "TCP_SERVER", 10 '主端设置 OPEN#10, "TCP_CLIENT", 10, "192.168.1.112" '从端设置 例二 OPEN#10, "UDP_SERVER", 1000 'UDP 主端设置 OPEN#11, "UDP_CLIENT", 60000, "192.168.0.120" 'UDP 从端设置 例三 OPEN#3, "TCP_EXCUTE", 500 '直接 TCP 控制, 直接 TCP 客户端发送在线命令的字符串控制, 控制器侧是 TCP 的服务器 详细例程参考自定义网口通讯

相关指令	PORT , PORT_TARGET , SETCOM , PRINT#
------	--

CLOSE# -- 关闭自定义网口通讯

类型	系统指令
描述	关闭自定义的以太网通讯。 新固件版本提供此功能。
语法	<code>CLOSE# ecustomnum</code> ecustomnum: 自定义网口通道号 <code>CLOSE#PORT</code> port: 通讯通道, 关闭 OPEN 的通道
适用控制器	通用
例子	<code>CLOSE# 10</code> '关闭通道 10
相关指令	OPEN# , PORT , PORT_TARGET , SETCOM , PRINT#

PRINT# -- 输出字符串

类型	系统指令
描述	从 RAW 方式通讯或自定义网口通讯的通道口里面输出字符串，碰到 0 结束。 每次调用都是一个 UDP 包发送。 PRINT#直接发送为字符串(""符号自动去除)，不需要 ASCII 转码处理，一次只能发一个数据，如下所示： 命令与输出 <pre>>>print#0,123</pre> 在线命令: <code>print#0,123</code> 串口调试助手 串口设置 串口号 COM4 #US1 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:27:18.123] 命令与输出 <pre>>>print#0,"123"</pre> 在线命令: <code>print#0,"123"</code> 串口调试助手 串口设置 串口号 COM4 #US1 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:23:24.123] 命令与输出 <pre>>>print#0,"asd"</pre> 在线命令: <code>print#0,"asd"</code> 串口调试助手 串口设置 串口号 COM4 #US1 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:28:19.asd]

	命令与输出 >>print#0, 49, 50, 51 Online command fail of error:2029:Label name is invalid. 在线命令: print#0,49,50,51 发送 串口调试助手 串口设置 串口号 COM4 #US1 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:29:40. 49] 命令与输出 >>aa(0, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57) >>print#0, aa 在线命令: print#0,aa 串口调试助手 串口设置 串口号 COM4 #US1 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:44:57. 0123456789] PRINT#发送数组为 ASCII 码，遇 0 停止，如下所示： 命令与输出 >>aa(0, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57) >>print#0, aa 在线命令: print#0,aa 串口调试助手 串口设置 串口号 COM4 #US1 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:44:57. 0123456789] 命令与输出 >>aa(0, 48, 49, 50, 0, 51, 52, 53) >>print#0, aa 在线命令: print#0,aa 串口调试助手 串口设置 串口号 COM4 #US1 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:49:23 012]
语法	PRINT#PORT, "字符串" port: 通道号
适用控制器	通用
例子	DIM VAR(10) '定义数组 VAR = "AAAA" '数组赋值 SETCOM(38400,8,1,0,0,0)'开启 RAW 方式 PRINT#0,VAR '从通道 0 输出数组数据
相关指令	PUTCHAR# , GET# , PORT , SETCOM

PUTCHAR# -- 输出字符

类型	系统指令
描述	从 RAW 方式通讯或自定义网口通讯的通道口输出字符，数组可以代表多个字符。每次调用都是一个 UDP 包发送。 PUTCHAR#发送数据为二进制数值，多个数据逗号隔开，不能直接发送字符串，如下所示：

	命令与输出 >>putchar#0, 123 在线命令: putchar#0,123 串口调试助手 串口设置 串口号 COM4 #USI 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:56:16 1] 命令与输出 >>putchar#0, 49, 50, 51 在线命令: putchar#0,49,50,51 串口调试助手 串口设置 串口号 COM4 #USI 波特率 38400 校验位 NONE 数据日志 [2023-09-27 14:57:18. 123 命令与输出 Online command error, signal:" can't in expression. Online command fail of error:2068:Meet sign can't in expression. 在线命令: <input 123"="" type="text" value="putchar#0,"/> 串口调试助手 串口设置 串口号 COM4 #USI 波特率 38400 校验位 NONE 数据日志 PUTCHAR#发送数组为二进制数值，遇 0 输出为 0，会将整个数组完全发送，所以要确保数组数据正确，如下所示： 命令与输出 >>aa(0, 48, 49, 50, 51, 52, 53) >>putchar#0, aa 在线命令: putchar#0,aa 串口调试助手 串口设置 串口号 COM4 #USI 波特率 38400 校验位 NONE 数据日志 [2023-09-27 15:10:10. 0123456789 命令与输出 >>aa(0, 48, 49, 50, 0, 51, 52, 53) >>putchar#0, aa 在线命令: putchar#0,aa 串口调试助手 串口设置 串口号 COM4 #USI 波特率 38400 校验位 NONE 数据日志 [2023-09-27 15:11:03 012 345789
语法	PUTCHAR#PORT, 字符 port: 通道号 PUTCHAR#PORT, ARRAY(INDEX, NUMES) port: 通道号 index: 开始输出的位置 numes: 输出的字节个数, 二进制方式
适用控制器	通用
例子	DIM VAR1, ARRAY1(10) VAR1 = \$FE ARRAY1 = "ABCDEFGHJI" SETCOM(38400,8,1,0,0,0) PUTCHAR#0, VAR1 PUTCHAR#0, ARRAY1 '定义数组 '数组 1 赋值 '数组 2 赋值 '开启 RAW 方式 '从通道 0 输出数组 1 数据 '从通道 0 输出数组 2 数据
相关指令	PORT , SETCOM , GET# , PRINT#

PORT_TARGET -- IP 和端口号配置

类型	字符串指令
描述	配置通讯对方的 IP 地址和端口号。 4 系列控制器 20170628 以上固件版本，XPLC 系列控制器 20170702 以上固件版本支持。
语法	命令语法： PORT_TARGET(port)="ipaddress:portnum"或 PORT_TARGET(port)="ipaddress" port: 通道号 ipaddress: 对方 IP 地址 portnum: 端口号 返回语法：VAR=PORT_TARGET(port) 返回 IP 地址字符串。
适用控制器	通用
例子	PORT_TARGET="192.168.0.12:502" PORT_TARGET(port)="192.168.0.12" ?PORT_TARGET(port) DIM IPSTRING(100) IPSTRING = PORT_TARGET(port) ?IPSTRING
相关指令	OPEN# ， CANIO_ADDRESS

13.4 打印输出指令

PRINT -- 打印信息

类型	打印输出函数
描述	打印信息到 PC 端 RTSys 软件的输出窗口。 别名：? 通过*参数名可以打印参数值；*特殊字符串可以打印一些特殊的信息。
语法	PRINT expression, "string" 或 ? expression, "string" Expression: 有效表达式 *SET: 打印所有参数值 *TASK: 打印任务信息 任务正常时只打印任务状态 任务出错时还会打印出错误任务号，具体错误行 *MAX: 打印所有规格参数 *FILE: 打印程序文件信息 *SETCOM: 打印当前串口的配置信息 *BASE: 打印当前任务的 BASE 列表（140123 以后版本支持） *数组名: 打印数组的所有元素，数组长度不能太长

	*参数名：打印一个所有轴的单个参数 ?*ETHERCAT：打印 EtherCAT 总线连接设置状态 ?*RTEX：打印 Rtex 总线连接设置状态 ?*FRAME：打印机械手参数，需要 161022 及以上固件支持 ?*SLOT：打印出控制器槽位口信息（RTEX 口，EtherCAT 口） ?*PORT：打印所有 PORT 通讯口 ?*OPEN：打印所有 OPEN 的端口信息
适用控制器	通用
例子	例一 在线命令输入 >>PRINT 1+2 输出：3 例二 在线命令输入 >>PRINT *task '打印所有任务状态 Task:0 Running. file:"hmi.bas" line:280: Task:1 Stopped. Task:2 Stopped. Task:3 Stopped. Task:4 Stopped. Task:5 Stopped. Task:6 Stopped. 例三 在线命令输入 >> ?*mpos 输出：21872.400 0 0 0 0 0 0 0 0 0 0
相关指令	TRACE

ERRSWITCH -- 信息输出设置

类型	系统参数												
描述	调试输出开关，控制 TRACE WARN ERROR 调试输出语句是否真的输出信息。												
语法	ERRSWITCH=switch switch：调试输出的开关 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>TRACE WARN ERROR 指令全部不输出</td></tr> <tr> <td>1</td><td>只输出 ERROR 指令</td></tr> <tr> <td>2</td><td>输出 WARN ERROR 指令</td></tr> <tr> <td>3</td><td>TRACE WARN ERROR 指令全部输出</td></tr> <tr> <td>4</td><td>TRACE WARN ERROR 指令全部输出，以及运动指令监控</td></tr> </tbody> </table> ERRSWITCH=4 适用于 5 系列控制器	值	描述	0	TRACE WARN ERROR 指令全部不输出	1	只输出 ERROR 指令	2	输出 WARN ERROR 指令	3	TRACE WARN ERROR 指令全部输出	4	TRACE WARN ERROR 指令全部输出，以及运动指令监控
值	描述												
0	TRACE WARN ERROR 指令全部不输出												
1	只输出 ERROR 指令												
2	输出 WARN ERROR 指令												
3	TRACE WARN ERROR 指令全部输出												
4	TRACE WARN ERROR 指令全部输出，以及运动指令监控												

适用控制器	通用
例子	例一 ERRSWITCH = 3 '打印全输出，此时 WARN（报警）、ERROR（错误）、TRACE 信息都会输出 例二 ERRSWITCH = 0 '不输出，控制器错误信息和报警信息都不输出 TRACE DPOS(0) '此时无法用 trace 打印出轴 0 的 dpos ?DPOS(0) '可以用 print 打印，结果，0 例三 ERRSWITCH = 4 '输出所有信息，以及运动指令监控 >>MOVE(111) MOVE(111)AXIS(0)TASK(23) '打印当前操作的轴及运动的任务
相关指令	TRACE ， WARN ， ERROR

TRACE -- 打印信息 2

类型	打印输出函数
描述	打印信息到 PC 端 RTSYS 软件的输出窗口。 受 ERRSWITCH 开关的控制。
语法	同 PRINT
适用控制器	通用
例子	ERRSWITCH = 3 'trace 打印全输出 DPOS(0)=0 TRACE "DPOS(0) =" DPOS(0) 打印结果 0
相关指令	ERRSWITCH ， PRINT ， WARN ， ERROR

WARN -- 警告信息

类型	打印输出函数
描述	程序自动打印报警信息到 PC 端 RTSYS 软件的输出窗口。 也可手动打印信息。 受 ERRSWITCH 开关的控制。
语法	程序报警时自动打印。 手动打印同 PRINT。
适用控制器	通用
例子	参考 TRACE 例子
相关指令	ERRSWITCH ， PRINT ， TRACE ， ERROR

ERROR -- 错误信息

类型	打印输出函数
描述	程序自动打印错误信息到 PC 端 RTSYS 软件的输出窗口。 也可手动打印信息。 受 ERRSWITCH 开关的控制。
语法	程序错误时自动打印。 手动打印同 PRINT。
适用控制器	通用
例子	参考 TRACE 例子
相关指令	ERRSWITCH , PRINT , TRACE , WARN

13.5 通道参数指令

PORT -- 通道编号

类型	通道修正辅助指令																												
描述	当访问通道参数时可以指定其他的通道。 其他指令需要选择通道时查看。																												
语法	PORT (portnum) portnum: 通道号 不同型号的控制器支持的通道数不同。在线命令?*port 查看，见例程。 ZMC00x 系列 <table border="1"> <tr> <th>通道号</th><th>协议</th></tr> <tr> <td>0</td><td>串口 A</td></tr> <tr> <td>1</td><td>串口 B</td></tr> </table> ZMC1-2xx 系列，支持同时两个以太网连接，端口号为 502，支持触摸屏的 MODBUS-TCP 协议连接。 <table border="1"> <tr> <th>通道号</th><th>协议</th></tr> <tr> <td>0</td><td>RS232 串口</td></tr> <tr> <td>1</td><td>RS485 串口</td></tr> <tr> <td>2</td><td>以太网接口，连接 1</td></tr> <tr> <td>3</td><td>以太网接口，连接 2</td></tr> <tr> <td>10</td><td>自定义网络通讯通道 1</td></tr> <tr> <td>11</td><td>自定义网络通讯通道 2</td></tr> </table> ZMC3xx 系列：带 3 个串口。 <table border="1"> <tr> <th>通道号</th><th>协议</th></tr> <tr> <td>0</td><td>RS232 串口</td></tr> <tr> <td>1</td><td>RS485 串口</td></tr> <tr> <td>2</td><td>RS422 串口</td></tr> </table>	通道号	协议	0	串口 A	1	串口 B	通道号	协议	0	RS232 串口	1	RS485 串口	2	以太网接口，连接 1	3	以太网接口，连接 2	10	自定义网络通讯通道 1	11	自定义网络通讯通道 2	通道号	协议	0	RS232 串口	1	RS485 串口	2	RS422 串口
通道号	协议																												
0	串口 A																												
1	串口 B																												
通道号	协议																												
0	RS232 串口																												
1	RS485 串口																												
2	以太网接口，连接 1																												
3	以太网接口，连接 2																												
10	自定义网络通讯通道 1																												
11	自定义网络通讯通道 2																												
通道号	协议																												
0	RS232 串口																												
1	RS485 串口																												
2	RS422 串口																												

	<table><tr><td>3</td><td>以太网接口，连接 1</td></tr><tr><td>4</td><td>以太网接口，连接 2</td></tr><tr><td>10</td><td>自定义网络通讯通道 1</td></tr><tr><td>11</td><td>自定义网络通讯通道 2</td></tr></table>	3	以太网接口，连接 1	4	以太网接口，连接 2	10	自定义网络通讯通道 1	11	自定义网络通讯通道 2								
	3	以太网接口，连接 1															
	4	以太网接口，连接 2															
	10	自定义网络通讯通道 1															
	11	自定义网络通讯通道 2															
	ZMC4xx 系列：带 2 个串口。																
	<table><tr><td>通道号</td><td>协议</td></tr><tr><td>0</td><td>RS232 串口</td></tr><tr><td>1</td><td>RS485 串口</td></tr><tr><td>2</td><td>以太网接口，连接 1</td></tr><tr><td>3</td><td>以太网接口，连接 2</td></tr><tr><td>10</td><td>自定义网络通讯通道 1</td></tr><tr><td>11</td><td>自定义网络通讯通道 2</td></tr><tr><td>20</td><td>网络控制器互联通道，不支持 PORT_STATUS 查询。</td></tr></table>	通道号	协议	0	RS232 串口	1	RS485 串口	2	以太网接口，连接 1	3	以太网接口，连接 2	10	自定义网络通讯通道 1	11	自定义网络通讯通道 2	20	网络控制器互联通道，不支持 PORT_STATUS 查询。
	通道号	协议															
	0	RS232 串口															
	1	RS485 串口															
	2	以太网接口，连接 1															
	3	以太网接口，连接 2															
	10	自定义网络通讯通道 1															
	11	自定义网络通讯通道 2															
	20	网络控制器互联通道，不支持 PORT_STATUS 查询。															
	ECI 系列：只有一个串口。																
	<table><tr><td>通道号</td><td>协议</td></tr><tr><td>0</td><td>RS232 串口</td></tr><tr><td>1</td><td>以太网接口，连接 1</td></tr><tr><td>2</td><td>以太网接口，连接 2</td></tr></table>	通道号	协议	0	RS232 串口	1	以太网接口，连接 1	2	以太网接口，连接 2								
	通道号	协议															
0	RS232 串口																
1	以太网接口，连接 1																
2	以太网接口，连接 2																
适用控制器	通用																
例程	查看控制器通道 >>?*PORT Port:0-COM. Port:1-COM. Port:2-ETH. Port:3-ETH. Port:4-ETH. Port:5-ETH. Port:6-ETH. Port:7-ETH. Port:10-ECUSTOM. Port:11-ECUSTOM. Port:12-ECUSTOM. Port:13-ECUSTOM. Port:14-ECUSTOM. Port:15-ECUSTOM. Port:20-CONNECT. 其中 COM 为串口通道，ETH 为网口通讯通道，ECUSTOM 为自定义网口通讯通道，CONNECT 为控制器互联通道。																
相关指令	FILE_PORT ， PORT_STATUS ， PROTOCOL																

PORT_STATUS -- 通道状态

类型	通道参数，只读						
描述	返回当前通道的状态，串口总是返回 1。 MODBUS_TCP 链接上以后，PORT_STATUS 会变为 1。 当对应链接已经作为从端使用时，控制器自动搜索没有使用的 PORT 作为 MODBUS_TCP 主端链接，此时程序无法确定实际使用的是哪个 PORT。 新固件版本提供此功能。						
语法	VAR1 = PORT_STATUS(port) port: 通道号 <table border="1"> <tr> <th>值</th><th>协议</th></tr> <tr> <td>0</td><td>没有连接</td></tr> <tr> <td>1</td><td>有连接使用</td></tr> </table>	值	协议	0	没有连接	1	有连接使用
值	协议						
0	没有连接						
1	有连接使用						
适用控制器	通用						
例子	?PORT_STATUS(0) '返回 232 串口通道状态，结果，1						
相关指令	PORT , SETCOM , ADDRESS						

PORT_MODE -- 通道模式

类型	通道参数
描述	PORT 模式设置，可以动态添加 PORT 类型。
语法	命令语法: PORT_MODE(portnum, mode [,"comname"]) portnum: PORT 编号 mode: 需要设置的模式，必须是空余的 PORT(ZPORT_MODE_NOTUSED)，建议程序最前面设置，支持设置的类型： ZPORT_MODE_COMUSB = 2, // USB 扩展的串口 ZPORT_MODE_ETH = 10, // 标准网络连接 ZPORT_MODE_ETHCUSTOM = 11, // 网络自定义通讯 可以通过?*PORT 或 PORT_MODE 函数语法来查看当前的 mode 设置 comname: 串口时，指明串口名称，Linux 下一般为"/dev/ttySn"或"/dev/ttyUSBn"，通过 linux 命令 ls /dev 可以查看串口列表，windows 下名称为"COMn" 函数语法: mode= PORT_MODE(portnum) portnum: PORT 编号 返回值: ZPORT_MODE_NOTEXIST = -1, // 超过最大 PORT 号 ZPORT_MODE_NOTUSED = 0, // 没有使用，可以用来设置 ZPORT_MODE_COM = 1, // 硬件固定的串口 ZPORT_MODE_COMUSB = 2, // USB 扩展的串口 ZPORT_MODE_LOCAL = 9, // LOCAL 直接通道 ZPORT_MODE_ETH = 10, // 标准网络连接

	ZPORT_MODE_ETHCUSTOM = 11, // 网络自定义通讯 ZPORT_MODE_ETHICONNET = 12, // 互联通道
适用控制器	5 系列控制器，20200302 以后的版本支持
例子	设置自定义网口： PORT_mode(22,11) 设置标准网口 PORT： PORT_mode(22,10) 设置 USB 串口： PORT_mode(22,2,"/dev/ttyUSB0")
相关指令	PORT

FILE_PORT -- 当前通道文件号

类型	通道参数
描述	返回或设置当前通道的缺省文件号。 设定后可以通过在线命令来访问对应文件的文件模块变量或数组。 RTSys 集成开发环境会自动使用此参数，不要在使用 RTSys 时修改此参数。
语法	VAR1 = FILE_PORT, FILE_PORT = filenum
适用控制器	通用
例子	>>FILE_PORT = 0 '当前连接的通道的 FILE_PORT 参数
相关指令	PORT

PROTOCOL -- 通道通讯协议

类型	通道参数，只读										
描述	返回当前通道的通讯协议。										
语法	VAR1 = PROTOCOL(port) port: 通道号 返回值 <table border="1"> <thead> <tr> <th>值</th><th>协议</th></tr> </thead> <tbody> <tr> <td>0</td><td>RAW 数据格式，无协议</td></tr> <tr> <td>3</td><td>MODBUS 协议，控制器为从端（缺省）</td></tr> <tr> <td>14</td><td>MODBUS 协议，控制器为主端</td></tr> <tr> <td>15</td><td>直接命令执行方式</td></tr> </tbody> </table>	值	协议	0	RAW 数据格式，无协议	3	MODBUS 协议，控制器为从端（缺省）	14	MODBUS 协议，控制器为主端	15	直接命令执行方式
值	协议										
0	RAW 数据格式，无协议										
3	MODBUS 协议，控制器为从端（缺省）										
14	MODBUS 协议，控制器为主端										
15	直接命令执行方式										
适用控制器	通用										
相关指令	PORT , SETCOM , ADDRESS										

ETH_MODE -- 网口模式设置

类型	通道参数
描述	网口模式设置，数组类型，每个网口独立设置。
语法	ETH_MODE(isel) = imode isel: 0- 第一个网口, 1-第二个网口(EtherCAT) imode: 0 - 非 eth 模式，传统模式，兼容性好，抗干扰稍差 1 - eth 模式，抗干扰性好，多个链接时需要配合 PC 上防掉线 DLL，标准 DLL 只能链接一个 (有极少数触摸屏或 EtherCAT 驱动器不兼容此模式) 2 - 自动模式(新固件缺省值)，如果有检测到网络没有连上，会自动在 0/1 模式中切换 20170720 以上带 ETH 的固件版本支持。
适用控制器	通用
相关指令	PORT

SEND_AUTOUP -- 主动上报

类型	通道参数
描述	设置网口主动上报的内容。
语法	SEND_AUTOUP(port, typecode, string) port: 支持主动上报的 PORT 编号, -1 自动选择支持主动上报的以太网链接, PC 端调用 ZMC_SetAutoUpCallBack 以后对应链接自动支持主动上报 typecode: 类型码 string: 字符串, 有效的字符串表达式都可以
适用控制器	通用
例子	SEND_AUTOUP(-1,100,"111211")
相关指令	PORT

SEND_AUTOUP2 -- 主动上报 2

类型	通道参数
描述	设置网口主动上报的内容，发送 TABLE 的内容。
语法	SEND_AUTOUP2(port, typecode, tableindex, length) port: 支持主动上报的 PORT 编号, -1 自动选择支持主动上报的以太网链接, PC 端调用 ZMC_SetAutoUpCallBack 以后对应链接自动支持主动上报 typecode: 类型码 tableindex: table 表位置的起始编号 length: 发送的二进制字节个数，一个 table 元素取一个字节
适用控制器	通用

例子	TABLE="1234" SEND_AUTOUP2(-1,200, 0 ,4)
相关指令	PORT

IFAUTOUP_PORT -- 主动上报端口查询

类型	通道参数
描述	查看支持主动上报的 PORT 口。 查询的是?*port 打印的 eth 类型的 PORT 口，0-不支持，1-支持。建议带上索引打印。
语法	VALUE=IFAUTOUP_PORT
适用控制器	通用
例子	查看所有网口通道 ? IFAUTOUP_PORT(3)
相关指令	PORT

PORT_KEEPLIVE – 超时检测

类型	字符串指令
描述	PORT 超时断开检测，检测通讯断开的的时间，超过时间限制后通讯彻底断开，不再自动重连。适用于自定义网口通讯。 4 系列 230315 以上固件版本
语法	PORT_KEEPLIVE(iportnum) = value
适用控制器	通用
例子	OPEN#11,"TCP_SERVER",60000,"192.168.0.11" ?PORT_KEEPLIVE(11) '打印自定义网口通道 11 的超时时间 PORT_KEEPLIVE(11)= 5000 '参考时间设置，完全掉线之后，经过 5000ms 时间后 通讯将不自动恢复 ?PORT_KEEPLIVE(11)
相关指令	PORT

PORT_485DELAY -- 收发数据延迟

类型	PORT 参数
描述	设置 485 端口时，接收数据到发送数据的延迟时间，单位 us。 对方部分设备可能不能快速切换方向，导致控制器太快发送无法接收。
语法	PORT_485DELAY(iportnum) = value
适用控制器	通用
相关指令	PORT

13.6 MODBUS 通讯指令

MODBUS_BIT -- 位寄存器

类型	MODBUS 位寄存器																
描述	修改或者读取 BIT 寄存器，布尔型，触摸屏一般叫 0x 寄存器。 另：特殊的 modbus 0x 寄存器，通过这些寄存器可以直接从触摸屏读取和设置 IO，注意读取的 IO 是原始的状态，INVERT_IN 不起作用。 <table border="1"> <thead> <tr> <th>MODBUS_BIT 地址</th><th>意义</th></tr> </thead> <tbody> <tr> <td>0~7999</td><td>用户自定义使用</td></tr> <tr> <td>8000~8099</td><td>PLC 编程的特殊 M 寄存器</td></tr> <tr> <td>8100~8199</td><td>轴 0-99 的 IDLE 标志</td></tr> <tr> <td>8200~8299</td><td>轴 0-99 的 BUFFER 剩余标志</td></tr> <tr> <td>10000~14095</td><td>对应输入 IN 口</td></tr> <tr> <td>20000~24095</td><td>对应输出 OUT 口</td></tr> <tr> <td>30000~34095</td><td>对应 PLC 编程的 S 寄存器</td></tr> </tbody> </table>	MODBUS_BIT 地址	意义	0~7999	用户自定义使用	8000~8099	PLC 编程的特殊 M 寄存器	8100~8199	轴 0-99 的 IDLE 标志	8200~8299	轴 0-99 的 BUFFER 剩余标志	10000~14095	对应输入 IN 口	20000~24095	对应输出 OUT 口	30000~34095	对应 PLC 编程的 S 寄存器
MODBUS_BIT 地址	意义																
0~7999	用户自定义使用																
8000~8099	PLC 编程的特殊 M 寄存器																
8100~8199	轴 0-99 的 IDLE 标志																
8200~8299	轴 0-99 的 BUFFER 剩余标志																
10000~14095	对应输入 IN 口																
20000~24095	对应输出 OUT 口																
30000~34095	对应 PLC 编程的 S 寄存器																
语法	MODBUS_BIT(first,[last]) = value first: 位寄存器编号，编号从 0 开始 last: 位寄存器编号，编号从 0 开始																
适用控制器	通用																
例子	DIM VAR MODBUS_BIT(100) = 1 'bit100 赋值 1 VAR = MODBUS_BIT(100) '把 bit100 的值赋值到变量 VAR																

MODBUS_IEEE -- 字寄存器-32 位浮点型

类型	MODBUS 字寄存器								
描述	修改或者读取字寄存器，32 位浮点型。触摸屏一般叫 4x 寄存器。 ZMOTION 运动控制器专门具备 MODBUS 字寄存器，会占用两个寄存器的地址。字寄存器间的关系查看“MODBUS 寄存器”。 控制器缺省使用系统 MODBUS 字寄存器，要修改时请设置 SETCOM 的第 7 个参数。 另：特殊的 MODBUS 4x 寄存器，通过这些寄存器可以直接从触摸屏读取一些状态。 <table border="1"> <thead> <tr> <th>寄存器（zero based）</th><th>意义</th></tr> </thead> <tbody> <tr> <td>10000-</td><td>DPOS 读取，每个轴占两个寄存器</td></tr> <tr> <td>11000-</td><td>MPOS 读取，每个轴占两个寄存器</td></tr> <tr> <td>12000-</td><td>VP_SPEED 读取，每个轴占两个寄存器</td></tr> </tbody> </table>	寄存器（zero based）	意义	10000-	DPOS 读取，每个轴占两个寄存器	11000-	MPOS 读取，每个轴占两个寄存器	12000-	VP_SPEED 读取，每个轴占两个寄存器
寄存器（zero based）	意义								
10000-	DPOS 读取，每个轴占两个寄存器								
11000-	MPOS 读取，每个轴占两个寄存器								
12000-	VP_SPEED 读取，每个轴占两个寄存器								
语法	MODBUS_IEEE (regnum) = value regnum: 寄存器编号，编号从 0 开始								

适用控制器	通用
例子	DIM VAR MODBUS_IEEE (100) = 100.10 'ieee100 赋值 100.10 VAR = MODBUS_IEEE (100) 'ieee100 赋值给变量 VAR
相关指令	MODBUS_REG , MODBUS_LONG , MODBUS_STRING

MODBUS_LONG -- 字寄存器-32 位整型

类型	MODBUS 字寄存器
描述	修改或者读取字寄存器，32 位整型。触摸屏一般叫 4x 寄存器。 ZMOTION 运动控制器专门具备 MODBUS 字寄存器，会占用两个寄存器的地址。 字寄存器间的关系查看“ MODBUS 寄存器 ”。 控制器缺省使用系统 MODBUS 字寄存器，要修改时请设置 SETCOM 的第 7 个参数。
语法	MODBUS_LONG (regnum) = value regnum: 寄存器编号，编号从 0 开始
适用控制器	通用
例子	DIM VAR MODBUS_LONG (100) = 100 'long100 赋值 100 VAR = MODBUS_LONG (100) 'long100 赋值给 VAR
相关指令	MODBUS_REG , MODBUS_IEEE , MODBUS_STRING

MODBUS_REG -- 字寄存器-16 位整型

类型	MODBUS 字寄存器		
描述	修改或者读取字寄存器，16 位整型，触摸屏一般叫 4x 寄存器。		
	ZMOTION 运动控制器专门具备 MODBUS 字寄存器。		
	不同型号控制器的字寄存器个数有区别。		
	字寄存器间的关系查看“ MODBUS 寄存器 ”。		
	控制器缺省使用系统 MODBUS 字寄存器，要修改时请设置 SETCOM 的第 7 个参数。		
描述	另：特殊的 modbus 4x 寄存器，通过这些寄存器可以直接从触摸屏读取一些状态。		
	寄存器（zero based）	类型	意义
	13000-	16	DA 输出
	14000-	16	AD 读取
语法	MODBUS_REG(regnum) = value regnum: 寄存器编号，编号从 0 开始		
适用控制器	通用		
例子	DIM VAR		

	MODBUS_REG(100) = 100 'reg100 赋值 100 VAR = MODBUS_REG(100) 'reg100 赋给变量 VAR
相关指令	MODBUS_IEEE , MODBUS_LONG , MODBUS_STRING

MODBUS_STRING -- 字寄存器-字节

类型	MODBUS 字寄存器，字符串函数
描述	读取 MODBUS 寄存器区域的字符串，按字节来读取。触摸屏一般叫 4x 寄存器。字寄存器间的关系查看“ MODBUS 寄存器 ”。
语法	MODBUS_STRING(index, chares) index: 起始的 MODBUS 寄存器编号，编号从 0 开始。不同型号控制器的字寄存器个数有区别 chares: 读取的字符总数
适用控制器	通用
例子	DIM ARR(8) MODBUS_STRING (0, 8) = "abc" 'string0 开始保存字符串，共 8 字节 print MODBUS_STRING(0, 8) '打印保存的字符串，打印结果，abc ARR = MODBUS_STRING(0,8) '字符换赋值给数组
相关指令	MODBUS_REG , MODBUS_LONG , MODBUS_IEEE

MODBUSM_DES -- modbus 通讯连接

类型	通讯指令
描述	设置或读取 MODBUS 主端的对方。 通讯等待时只会阻塞当前这个任务，不影响其他任务。 使用 485 串口与多个设备通讯时，建立通讯连接可加入等待或延时，等上一个设备连接成功再连接下一个设备，防止出现通讯失败的情况。 注意：主站不要在同一时刻读写多个 MODBUS 从站，特别是多任务的情况下，注意错开操作。
语法	MODBUSM_DES (address[,port],[timer],[resendset]) ADDRESS1 = MODBUSM_DES([port]) address: 对端的 modbus 协议站号 port: 当前 modbus 主通讯的 port 号 timer: 消息超时时间设置，缺省 1000ms。 resendset: 超时消息重发设置, 0-不重发, 1-SEND 指令重发, 2-SEND 与 MODBUSM 指令都重发。

MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。

SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过 SEND 指令来修改。

适用控制器	通用
例子	例一 多主端-多从端通讯 <pre> SETCOM(38400,8,1,0,0,14,2,1000) '设置串口 0 为 modbus 主端，通讯等待 1s SETCOM(38400,8,1,0,1,14,2,1000) '设置串口 1 为 modbus 主端，通讯等待 1s WHILE 1 IF IN(0)=1 THEN 'IN0 高电平时使用串口 0 IF IN(1)=1 THEN MODBUSM_DES(1,0) 'IN1 高电平时与从端站号 1 通讯 MODBUSM_REGSET(0,10,0) '本地寄存器复制到对端 MODBUSM_REGGET(20,10,20) '对端寄存器复制到本地 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 ELSE MODBUSM_DES(2,0) 'IN1 低电平时与从端站号 2 通讯 MODBUSM_REGSET(30,10,30) '本地寄存器复制到对端 MODBUSM_REGGET(40,10,40) '对端寄存器复制到本地 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 ENDIF ?"通道 0 状态=", MODBUSM_STATE '打印通讯状态 ELSE 'IN0 低电平时使用串口 1 IF IN(1)=1 THEN MODBUSM_DES(1,1) 'IN1 高电平时与从端站号 1 通讯 MODBUSM_REGSET(50,10,50) '本地寄存器复制到对端 MODBUSM_REGGET(60,10,60) '对端寄存器复制到本地 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 ELSE MODBUSM_DES(2,1) 'IN1 低电平时与从端站号 2 通讯 MODBUSM_REGSET(70,10,70) '本地寄存器复制到对端 MODBUSM_REGGET(80,10,80) '对端寄存器复制到本地 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 ENDIF ?"通道 1 状态=", MODBUSM_STATE '打印通讯状态 ENDIF WEND </pre>
相关指令	SETCOM , PROTOCOL , PORT , MODBUSM_DES2

MODBUSM_DES2 -- 控制器间网口通讯

类型	通讯指令
描述	控制器间网口通讯，也可作为 MODBUS_TCP 主端通讯。 ?*PORT 可以显示出当前可用的网络链接通道。

	>>?*port Port:0-COM. Port:1-COM. Port:2-ETH. Port:3-ETH. Port:4-ETH. Port:5-ETH. Port:6-ETH. Port:7-ETH. Port:8-ETH. Port:9-ETH. Port:10-ECUSTOM. Port:11-ECUSTOM. Port:12-ECUSTOM. Port:13-ECUSTOM. Port:14-ECUSTOM. Port:15-ECUSTOM. Port:20-CONNECT. 作为 MODBUS_TCP 主端时： 选择一个 ETH 模式的 PORT 口作为 MODBUS_TCP 链接通道(不建议用第一个和最后一个)。 当选择的通道已经作为从端被占用时，控制器自动选择一个没有占用的 ETH 模式的 PORT 口作为 MODBUS_TCP 主端链接。 作为控制器互联使用时： 选择 CONNECT 模式的 PORT 口作为控制器互联通道。 注意：主站不要在同一时刻读写多个 MODBUS 从站，特别是多任务的情况下，注意错开操作。
语法	MODBUSM_DES2 (id,port,"desipaddress",[timer],[resendset], [destport502]) id: 对方控制器的 MODBUS 从端 ID，缺省 1 port: 支持两种模式，?*PORT 确认通道号及模式 ETH 时，作为 MODBUS_TCP 主端通道 CONNECT 时，做为控制器互联通道 desipaddress: 字符串， 对方控制器的 IP 地址 timer: 消息超时时间设置，缺省 1000ms resendset: 超时消息重发设置，0-不重发，1-SEND 指令超时重发，2-SEND 与 MODBUSM 指令都超时重发 destport502: 从站端口号，缺省 502 MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。 SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过 SEND 指令来修改。
适用控制器	4 系列产品, 20170117 及以上固件版本支持
例子	例一 MODBUS 主端通讯连接建立 MODBUS_TCP 主端通讯不用考虑丢包。 MODBUSM_DES2(1,4,"192.168.0.12") '按站号和 IP 与从端通讯使用控制器通道 4，?*port 确认 WHILE 1 LASTTICK=TICKS FOR i =0 TO 9999 MODBUS_REG(0) = i

	<pre> MODBUSM_REGSET(0,10,0) '设置对端寄存器 MODBUS_REG(0) = 99 MODBUSM_REGGET(0,10,0) '读取对端寄存器 IF MODBUS_REG(0) <> I THEN ?"REG(0)=" MODBUS_REG(0),"STATE=" MODBUSM_STATE '打印在第几次通讯时错误 ENDIF NEXT ?LASTTICK-TICKS '打印通讯时间 WEND END 例二 控制器间通讯连接建立 网络环境不好时，互连通讯有很小的丢包可能。 MODBUSM_DES2(\$fe,20,"192.168.0.25",10) '控制器从端站号为 fe，控制器主端通 道号为 20，?*port 确认设置 10ms 超时时间 MODBUSM_REGSET(0,10,0) '本地寄存器复制到对端 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 IF MODBUSM_STATE<>0 THEN MODBUSM_REGSET(0,10,0) '出错重发一次 MODBUSM_REGGET(20,10,20) '对端寄存器复制到本地 ENDIF WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束 IF MODBUSM_STATE<>0 THEN MODBUSM_REGGET(20,10,20) '出错重发一次 ENDIF END </pre>
相关指令	ADDRESS , PORT

MODBUSM_STATE -- modbus 通讯状态

类型	通讯状态	
描述	MODBUS 主通讯状。	
	值	描述
	0	状态正常
	1	等待应答中
	2	等待超时
	3	应答错误
语法	VAR1 = MODBUSM_STATE 当前 modbus 主端通讯状态	
适用控制器	通用	
例子	SETCOM(38400,8,2,0,1,14,2,1000) '485 口为 modbus 主端，消息超时时间'1s MODBUSM DES(1,1) '与对端站号 1 通讯	

	<pre> MODBUSM_REGGET(0,10,0) '获取对端寄存器值 WAIT UNTIL MODBUSM_STATE <> 1 '等待消息结束，最多等待 1s，为消息超时时间，获取消息或超时后变为相应值 ?MODBUSM_STATE '打印通讯结果 IF MODBUSM_STATE=0 THEN ?"通讯正常" ELSEIF MODBUSM_STATE=2 THEN ?"通讯超时" ELSEIF MODBUSM_STATE=3 THEN ?"通讯错误" ENDIF </pre>
相关指令	PROTOCOL , PORT , SETCOM

MODBUSM_REGSET -- 写对端保持寄存器

类型	通讯指令
描述	把本地 MODBUS 保存寄存器设置到对端。 对应标准协议功能码 06 或 16，写保持寄存器。
语法	<pre>MODBUSM_REGSET (startreg, num, local_reg)</pre> startreg: 对端的寄存器起始编号，从 0 开始 num: 寄存器个数 local_reg: 从本地系统 MODBUS 寄存器中取值，起始编号
适用控制器	通用
例子	参考 MODBUSM_REGGET 例一
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_REGGET -- 读对端保持寄存器

类型	通讯指令
描述	把对端的 MODBUS 保存寄存器复制到本地。 对应标准协议功能码 03，读保持寄存器。
语法	<pre>MODBUSM_REGGET (startreg, num, local_reg)</pre> startreg: 对端的寄存器起始编号，从 0 开始 num: 寄存器个数 local_reg: 本地系统 MODBUS 寄存器起始编号
适用控制器	通用
例子	<pre> 台达绝对值编码器读取 GLOBAL DIM flag_abs '编码器读取正确标志 flag_abs = 0 GLOBAL DIM total_pul '读到个总脉冲个数 SETCOM(38400,8,2,0,1,14) '设置 485 口为 MODBUS 主端，波特率 38400 MODBUSM_DES(1,1) '设置 485 端口，对方站号 1 P3-00 </pre>

	<pre> MODBUS_LONG(300) = 2 '用 300, 301 传输数据 MODBUSM_REGSET(98,2,300) '设置 P0-49 = 2, 更新参数 MODBUSM_REGGET(98,2,300) TICKS = 1000 WHILE (MODBUS_LONG(300) AND TICKS > 0) '等到 P0-49 变成 0 或 1 秒超时, 即 更新完成或者不成功 MODBUSM_REGGET(98,2,300) WEND IF TICKS < 0 THEN PRINT "伺服更新不成功" flag_abs = 1 RETURN ENDIF MODBUSM_REGGET(100,6,310) IF MODBUS_LONG(310) = 0 THEN '编码器正常 flag_abs = 0 total_pul = MODBUS_LONG(314) ELSE PRINT "编码器出错" flag_abs = 2 ENDIF IF flag_abs = 0 THEN '正确 dpos(0) = -total_pul/units(0) '测试出来的脉冲个数值是反的, 取反还原坐标 ENDIF END </pre>
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_3XGET -- 读对端输入寄存器

类型	通讯指令
描述	把对端的 MODBUS 输入寄存器复制到本地。 对应标准协议功能码 04, 读输入寄存器。
语法	<pre>MODBUSM_3XGET (startreg, num, local_reg)</pre> startreg: 对端的寄存器起始编号, 从 0 开始 num: 寄存器个数 local_reg: 从本地系统 MODBUS 寄存器中取值, 起始编号
适用控制器	通用
例子	MODBUSM_3XGET(0,10,0) '把对端输入寄存器 0-9 复制到通讯本地寄存器 0-9
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_BITSET -- 写对端线圈

类型	通讯指令
描述	把本地 MODBUS 位寄存器设置到对端。 对应标准协议功能码 05 或 15，写线圈。
语法	MODBUSM_BITSET (startreg, num, local_reg) startreg: 对端的寄存器起始编号，从 0 开始 num: 寄存器个数 local_reg: 从本地系统 MODBUS 寄存器中取值，起始编号
适用控制器	通用
例子	MODBUSM_BITSET (0,10,0) '把本地位寄存器 0-9 复制到通讯对端的寄存器 0-9
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_BITGET -- 读对端线圈

类型	通讯指令
描述	把对端 MODBUS 位寄存器复制到本地。 对应标准协议功能码 01，读线圈。
语法	MODBUSM_BITGET (startreg, num, local_reg) startreg: 对端的寄存器起始编号，从 0 开始 num: 寄存器个数 local_reg: 本地系统 MODBUS 寄存器起始编号
适用控制器	通用
例子	MODBUSM_BITGET (0,10,0) '把通讯对端的位寄存器 0-9 复制到本地位寄存器 0-9
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

MODBUSM_1XGET -- 读对端离散输入

类型	通讯指令
描述	把对端 MODBUS 位寄存器复制到本地。 对应标准协议功能码 02，读离散输入。
语法	MODBUSM_1XGET (startreg, num, local_reg) startreg: 对端的寄存器起始编号，从 0 开始 num: 寄存器个数 local_reg: 本地系统 MODBUS 寄存器起始编号
适用控制器	通用
例子	MODBUSM_1XGET (0,10,0) '把通讯对端的位寄存器 0-9 复制到本地位寄存器 0-9
相关指令	ADDRESS , PROTOCOL , PORT , SETCOM

13.7 控制器互联直接命令指令

SEND_RESULT -- 读取 send 结果

类型	通讯指令
描述	读取 send 指令的结果。 返回值：0-成功，其它为错误，包括从控制器返回的错误码。 MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过 SEND 指令来修改。
语法	VAL=SEND_RESULT
适用控制器	4 系列以上控制器支持，支持版本号 20170618
相关指令	SEND_CMD

SEND_CMD -- send 命令

类型	通讯指令
描述	主控制器给从控制器发送 ZMC_DIRECTCOMMAND 命令，结果通过 SEND_RESULT 查看。 发送 BASIC 内容：cmdstring (参数列表) MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过 SEND 指令来修改。
语法	SEND_CMD(cmdstring,可选参数列表) cmdstring: 命令字符串 可选参数列表：个数可变，不带参数的时候，不添加括号
适用控制器	4 系列以上控制器支持，支持版本号 20170618
例子	SEND_CMD("MOVE",DIS1,DIS2,DIS3) SEND_CMD("MOVEABS",DIS1)
相关指令	SEND_RESULT

SEND_CMDAXIS -- send 命令

类型	通讯指令
描述	主控制器给从控制器发送 ZMC_DIRECTCOMMAND 命令，结果通过 SEND_RESULT 查看。 发送 BASIC 内容：cmdstring (参数列表), AXIS(iaxis)

	MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过 SEND 指令来修改。
语法	SEND_CMDAXIS (cmdstring,iaxis,可选参数列表) cmdstring: 命令字符串 iaxis: 轴号 可选参数列表: 个数可变，不带参数的时候，不添加括号
适用控制器	4 系列以上控制器支持，支持版本号 20170618
例子	SEND_CMDAXIS("MOVE",IAXIS,DIS1)
相关指令	SEND_RESULT , SEND_CMD

SEND_ASSIGN -- send 命令

类型	通讯指令
描述	主控制器给从控制器发送 ZMC_DIRECTCOMMAND 命令，结果通过 SEND_RESULT 查看。 发送 BASIC 内容: cmdstring (参数列表)=value MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过 SEND 指令来修改。
语法	SEND_ASSIGN (cmdstring,value,可选参数列表) cmdstring: 命令字符串 value: 赋值的内容 可选参数列表: 个数可变，不带参数的时候，不添加括号
适用控制器	4 系列以上控制器支持，支持版本号 20170618
例子	SEND_ASSIGN("DPOS",0,0) '生成 DPOS(0)=0 SEND_ASSIGN("DPOS(1)",0) '生成 DPOS(1)=0
相关指令	SEND_RESULT

SEND_QUERY -- send 命令

类型	通讯指令
描述	主控制器给从控制器发送 ZMC_DIRECTCOMMAND 命令，结果通过 SEND_RESULT 查看。 发送 BASIC 内容: cmdstring(参数列表)。 不带参数的时候，不添加括号。 接收的内容根据 SEND_QUERYSET 的设置填到 TABLE 里面。 MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过

	SEND 指令来修改。
语法	SEND_QUERY (cmdstring,可选参数列表) cmdstring: 命令字符串 可选参数列表: 个数可变, 不带参数的时候, 不添加括号
适用控制器	4 系列以上控制器支持, 支持版本号 20170618
例子	SEND_QUERYSET(0,1) SEND_QUERY("?dpos",0) 'table(0)保存控制器 DPOS(0)的内容 SEND_QUERY("?REMAIN_BUFFER(1)AXIS(0)",0) '发送字符串内容("?REMAIN_BUFFER(1) AXIS(0)") SEND_QUERYSET(0,2) SEND_QUERY("?dpos(0),dpos(1)") '从控制器会返回两个数据
相关指令	SEND_RESULT , SEND_QUERYSET

SEND_QUERYSET -- send 命令

类型	通讯指令
描述	主控制器给从控制器发送 ZMC_DIRECTCOMMAND 命令, 结果通过 SEND_RESULT 查看。 发送 BASIC 内容: cmdstring (参数列表) AXIS(iaxis) MODBUSM 指令超时重发要注意, 从控制器可能收到 2 次消息, 对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响, 重要标志性变量可以通过 SEND 指令来修改。
语法	SEND_QUERYSET (dtindex, dtnumes) dtindex: 接收内容存储的 TABLE 编号 dtnumes: 接收最多存储的 TABLE 个数
适用控制器	4 系列以上控制器支持, 支持版本号 20170618
例子	SEND_QUERYSET (0,1) SEND_QUERYSET (0,1)
相关指令	SEND_RESULT , SEND_QUERY

13.8 控制器互联文件发送指令

SEND_ZAR -- U 盘操作

类型	通讯指令
描述	通过主控制器的 U 盘升级从控制器的程序, 结果通过 MODBUSM_STATE 查看。 MODBUSM 指令超时重发要注意, 从控制器可能收到 2 次消息, 对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响, 重要标志性变量可以通过

	SEND 指令来修改。
语法	SEND_ZAR("ufilename") ufilename: U 盘文件名, 支持字符串的数组和其它字符串类型
适用控制器	4 系列以上控制器支持, 支持版本号 20170618
例子	SEND_ZAR("1.ZAR")
相关指令	MODBUSM_STATE , SEND_PERCENT

SEND_FLASH --数据拷贝

类型	通讯指令
描述	主控制器的 U 盘和从控制器的 FLASH 相互拷贝, 结果通过 MODBUSM_STATE 查看。 MODBUSM 指令超时重发要注意, 从控制器可能收到 2 次消息, 对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响, 重要标志性变量可以通过 SEND 指令来修改。
语法	SEND_FLASH (dir,uid,flashid) dir: 1-U 盘拷贝到控制器 FLASH, 0-控制器 FLASH 拷贝到 U 盘 uid: U 盘的文件编号, 规则同 U_WRITE flashid: 控制器的 FLASH 编号
适用控制器	4 系列以上控制器支持, 支持版本号 20170618
例子	SEND_FLASH (1,1,1)
相关指令	MODBUSM_STATE , SEND_PERCENT

SEND_FILE -- 拷贝 U 盘数据

类型	通讯指令
描述	主控制器的 U 盘和从控制器的文件相互拷贝, 只支持 BIN 文件和 Z3P 文件, 不支持文件功能的控制器会返回失败。 MODBUSM 指令超时重发要注意, 从控制器可能收到 2 次消息, 对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响, 重要标志性变量可以通过 SEND 指令来修改。
语法	SEND_FILE (dir, "ufile", "contrfile") dir: 1-U 盘拷贝到控制器 FLASH, 0-控制器 FLASH 拷贝到 U 盘 ufile: U 盘的文件名 contrfile: 控制器的文件名
适用控制器	4 系列以上控制器支持, 支持版本号 20170618
例子	SEND_FILE(1,"1.bin","1.bin")
相关指令	MODBUSM_STATE , SEND_PERCENT

SEND_IFLASH -- 拷贝 flash 数据

类型	通讯指令
描述	主控制器的 FLASH 和从控制器的 FLASH 相互拷贝，结果通过 MODBUSM_STATE 查看。 MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过 SEND 指令来修改。
语法	SEND_IFLASH (dir,id,flashid) dir: 1-主控制器 FLASH 拷贝到控制器 FLASH, 0-控制器 FLASH 拷贝到主控制器 FLASH id: 主控制器的 FLASH 编号 flashid: 控制器的 FLASH 编号
适用控制器	4 系列以上控制器支持，支持版本号 20170618
例子	SEND_FLASH (1,1,1)
相关指令	MODBUSM_STATE , SEND_PERCENT

SEND_PERCENT -- 查询指令进度

类型	通讯指令
描述	SEND 等需要长时间完成的指令返回百分比，可以用来显示进度条，范围 0-100。 MODBUSM 指令超时重发要注意，从控制器可能收到 2 次消息，对寄存器的扫描也可能出现 2 次。SEND 指令超时重发对从控制器没有影响，重要标志性变量可以通过 SEND 指令来修改。
语法	percent = SEND_PERCENT ()
适用控制器	4 系列以上控制器支持，支持版本号 20170618
相关指令	SEND_ZAR , SEND_FLASH , SEND_FILE

ZAR_CONTROL -- 查看 ZAR 对应控制器类型

类型	U 盘函数
描述	查看主控制器 U 盘的 ZAR 程序文件的对应控制器类型，此类型在 RTSys 中设置，避免主控制器和从控制器的文件混淆。
语法	id = ZAR_CONTROL ("ufilename") ufilename: U 盘文件名, ZAR 文件
适用控制器	4 系列以上控制器支持，支持版本号 20170618
例子	<pre>id = ZAR_CONTROL("1.ZAR") IF(id/3000=3) Then 'ZHD 系列示教盒 PRINT "zhd program"</pre>

	ENDIF
相关指令	SEND_ZAR , CONTROL

第十四章 系统相关指令

所有的日期时间参数或指令，LOCK 后不能修改。

14.1 控制器加密指令

APP_PASS -- 密码

类型	系统指令
描述	控制器应用密码。 下载 ZAR 包时可以选择校验这个密码，校验错误将不能下载。 LOCK 后不能修改 APP_PASS。 APP_PASS 采用不可逆算法加密，一旦忘记，将无法获知。
语法	APP_PASS(pass) pass: 字母或数字，“_”等少数特殊符号，总共不要超过 16 个字符，不能设置为变量或表达式，否则变量名等会被认为是密码。
适用控制器	通用
例子	APP_PASS(Zmotion)
相关指令	LOCK

LOCK -- 锁定控制器

类型	系统指令
描述	锁定控制器，不让对控制器操作。 上位机的 zpj 程序仍可以修改，无法下载到控制器，但生成的 zar 文件仍可下载。 会阻止对控制器的枚举操作，比如打印所有数组等，但可以打印具体数组的值。 LOCK 密码采用不可逆算法加密，一旦忘记，将不可获知，pass 不能设置变量或表达式，否则变量名等会被认为是密码。
语法	LOCK (pass) pass: 字母或数字，“_”等少数特殊符号，总共不要超过 16 个字符
适用控制器	通用
例子	LOCK(passwd) '锁定控制器，密码为 passwd
相关指令	UNLOCK

UNLOCK -- 解锁控制器

类型	系统指令
----	------

描述	解锁控制器。 LOCK 密码采用不可逆算法加密，一旦忘记，将不可获知。
语法	UNLOCK (pass) pass: LOCK 时的密码
适用控制器	通用
相关指令	LOCK

14.2 系统时间指令

DATE -- 系统日期

类型	系统参数
描述	设置系统日期，掉电保存，或是返回 2000 年 1 月 1 日以后的天数。 仿真器无法修改日期。
语法	DATE= DD:MM:YYYY 或 DD:MM:YY
适用控制器	通用
例子	DATE=27:2:13 在线命令输入 >>PRINT DATE 输出：4806 即 2013:2:27-2000:1:1 = 4806
相关指令	DATE\$, RTC_DATE

DATES\$ -- 系统日期

类型	字符串函数
描述	字符串函数，按 DD:MM:YYYY 格式返回 DATE 设置日期。
语法	DATES\$
适用控制器	通用
例子	DATE=27:2:13 在线命令输入 >>PRINT DATES\$ 输出：27:02:2013
相关指令	DATE , RTC_DATE

DAY -- 系统星期

类型	系统参数
描述	设置系统时钟的星期，0-6，0 表示星期天，掉电保存。 仿真器无法修改日期。

	DAY 不随 DATE 和 RTC_DATE 改变而改变，两者独立。
语法	VAR=DAY, DAY=expression
适用控制器	通用
例子	例一 DAY = 3 例二 在线命令输入 >>PRINT DAY 输出：3
相关指令	DAYS

DAYS -- 系统星期

类型	字符串函数
描述	字符串函数，返回 DAY 设置星期。 DAYS不随 DATE 和 RTC_DATE 改变而改变，两者独立。
语法	DAYS
适用控制器	通用
例子	在线命令输入 >>PRINT DAYS 输出：Wednesday
相关指令	DAY

RTC_DATE --系统日期

类型	系统参数
描述	设置或者获取系统日期，掉电保存，从 2000 年 1 月 1 日起。 注意与 DATE 指令的表示格式是不一样的，返回值类型为整数。 仿真器无法修改日期。
语法	RTC_DATE = YYYYMMDD 或 YYMMDD VAR = RTC_DATE[(days)] 带 days 获取指定天数前后的日期，自动闰年之类处理。20170524 以上固件版本增加。
适用控制器	通用
例子	RTC_DATE = 20130227 在线命令输入 >>PRINT RTC_DATE 输出：20130227 DIM gRTC_Date, CurDate, curYear, curMonth, curDay

	<pre> ?RTC_DATE gRTC_Date = RTC_DATE CurDate=gRTC_Date mod 20000000 ?CurDate curYear=int(CurDate/10000) ?curYear curMonth=int((CurDate-curYear*10000)/100) ?curMonth curDay=CurDate-curYear*10000-curMonth*100 ?curDay </pre>
相关指令	DATE , DATES

TIME -- 系统时间

类型	系统参数
描述	设置系统时钟的时间，返回 0 点以后的秒数。 仿真器无法修改日期。
语法	TIME = hh:mm:ss
适用控制器	通用
例子	例一 TIME= 11:14:40 例二 在线命令输入 >>PRINT TIME 输出：40541
相关指令	TIMES , RTC_TIME

TIMES -- 系统时间

类型	字符串函数
描述	字符串函数，按 24 小时的格式 hh:mm:ss 返回当前时间。
语法	TIMES
适用控制器	通用
例子	在线命令输入 >>PRINT TIMES 输出：11:29:46
相关指令	TIME , RTC_TIME

RTC_TIME -- 系统时间

类型	系统参数
描述	设置或者获取系统时间。 注意与 TIME 指令的表示方式是不一样的。
语法	RTC_TIME = hhmmss
适用控制器	通用
例子	在线命令输入 >>RTC_TIME = 113706 >>PRINT RTC_TIME 输出: 113706
相关指令	TIME , TIMES

14.3 轴系统参数指令

WDOG -- 轴总使能

类型	系统参数
描述	控制所有轴的使能。 使用 EtherCAT 总线时，必须使 WDOG=1。
语法	WDOG=0/1
适用控制器	通用
例子	WDOG=1 '所有轴使能打开
相关指令	AXIS_ENABLE

DISABLE_GROUP -- 轴分组

类型	系统指令
描述	一停多，把几个轴设置为一组，驱动器告警后会关闭组内的所有使能(ECAT 产品支持)，脉冲轴设置无意义。 一般用于多工位分组。
语法	DISABLE_GROUP(AXIS1, AXIS2, ...)
适用控制器	通用
例子	DISABLE_GROUP(-1) '取消所有组设置，此时报警关闭 WDOG DISABLE_GROUP(0,5,1) '轴 0,5,1 设置一组 DISABLE_GROUP(4,2) '轴 4,2 设置一组 当轴 0/5/1 出现问题时，轴 0、轴 5 和轴 1 都会掉使能，而轴 4 和轴 2 不会，同样当轴 4/2 出现问题时，轴 4 和轴 2 会掉使能，轴 0、轴 5 和轴 1 不会
相关指令	WDOG , AXIS_ENABLE

ERROR_AXIS -- 报错轴号

类型	系统状态
描述	出现错误的第一个总线轴轴号，-1 没有。
语法	Var=ERROR_AXIS
适用控制器	通用（总线轴控制器）
例子	?ERROR_AXIS '打印第一个出错轴号，打印结果，-1，当前没有轴错误
相关指令	MOTION_ERROR

MOTION_ERROR -- 报错轴列表

类型	系统状态
描述	出现错误的总线轴列表。 每个位表示一个轴号。位 0-n 表示轴 0-n。
语法	Var=MOTION_ERROR
适用控制器	总线轴控制器
例子	PRINT MOTION_ERROR '打印结果，0，未出错
相关指令	ERROR_AXIS

ERROR_SET -- 报错输出

类型	系统指令
描述	当 BASIC 程序运行出错的时候，输出口自动打开，并且把错误信息写到对应的 MODBUS 寄存器，输出口状态还原时 BASIC 程序会自动重新运行。 寄存器长度至少 32 个字节。
语法	ERROR_SET(输出口,modbus 寄存器地址[,errsubname]) errsubname: 设置一个 SUB 过程进行临时处理，语法错误停止时此函数自动被调用，此过程不要使用 WAIT 等可能阻塞的指令，而且必须很精简。此过程里面再出现语法错误不会再被处理。
适用控制器	通用
例子	ERROR_SET (1,200,error_deal) MOV(30) '拼写错误，此时运行错误，输出口 1 打开，并在寄存器记录错误信息 'Modbus_string(200,32) ="sample_move.bas,6,e2043" END SUB error_deal() '报错时调用函数 ?"进入错误处理 sub" '打印信息 '以下编写需要的功能程序

	END SUB 可以在线命令栏输入?MODBUS_STRING(200,32)查看错误信息: MODBUS_STRING(200,32)="sample_move.bas,6,e2043" sample_move.bas: 表示文件名 6: 表示当前出现错误的行号 e2043: 表示错误码
--	--

RADIUS_ERRSET -- 圆弧插补检查

类型	系统参数
描述	圆弧插补圆心方向半径检查设置。
语法	RADIUS_ERRSET=mode mode : 0- 缺省值, 不检查 1- 起始点结束点半径不一样(相差超过 2 个脉冲), 自动纠正. 2- 不一致, 返回 1006 错误 2022 年 1 月 11 日添加支持.
适用控制器	通用
例子	RADIUS_ERRSET=2 '圆弧指令坐标不对报错 1006, 圆弧指令不运行 MOVECIRC(200,0,98,0,1) '画圆弧 RADIUS_ERRSET=1 '圆弧指令坐标不对自动纠正, 提示 Center error, radius:98.000000 radiusend:102.000000 diff. ?RADIUS_ERRSET '设置查询
相关指令	MOVECIRC

14.4 IP 参数指令

IP_ADDRESS -- IP 地址

类型	系统参数, 自动存储到 FLASH
描述	控制器 IP 地址。 只有带以太网接口的控制器支持, 读取时以 32 位整数返回, 见例程。 修改立刻生效, 使用网口连接时, 修改后网口会断开, 需重连。 单网卡连接控制器, 网卡与控制器处于同一网段即可。 多网卡连接控制器, 网卡之间要为不同网段, 控制器连接哪个网卡就设置为对应网段。 多卡时, 修改 IP 地址后需要重启。
语法	IP_ADDRESS = dot.dot.dot.dot
适用控制器	通用

例子	在线命令输入 <pre>>>IP_ADDRESS=192.168.0.26 >>PRINT IP_ADDRESS</pre> 输出：436250816 具体转换过程如下： 四段转换为二进制 <pre>192 -- 1100 0000 168 -- 1010 1000 0 -- 0000 0000 26 -- 0001 1010</pre> 二进制重新组合 <pre>26 0 168 192 0001 1010 0000 0000 1010 1000 1100 0000</pre> 转化为十进制 <pre>436250816</pre>
相关指令	IP_GATEWAY ， IP_NETMASK

IP_ADDRESS2 -- IP 地址 2

类型	系统参数，自动存储到 FLASH
描述	5 系列控制器第二个网口的 IP 地址。 5 系列控制器支持，出厂默认第二个网口 IP 为 192.168.1.11，读取时以 32 位整数返回。修改立刻生效，使用网口连接时，修改后网口会断开，需重连。 单网卡连接控制器，网卡与控制器处于同一网段即可。 多网卡连接控制器，网卡之间要为不同网段，控制器连接哪个网卡就设置为对应网段。 多卡时，修改 IP 地址后需要重启。
语法	IP_ADDRESS2 = dot.dot.dot.dot
适用控制器	5 系列控制器以上
例子	在线命令输入 <pre>>>IP_ADDRESS=192.168.1.26</pre>
相关指令	IP_ADDRESS

IP_GATEWAY -- IP 网关

类型	系统参数，自动存储到 FLASH
描述	控制器 IP 网关。 只有带以太网接口的控制器支持，读取时以 32 位整数返回。
语法	IP_GATEWAY = dot.dot.dot.dot

适用控制器	通用
例子	在线命令输入 >>IP_GATEWAY =192.168.0.1 >>PRINT IP_GATEWAY 输出：16820416 具体转换过程参考 IP_ADDRESS 例程
相关指令	IP_NETMASK ， IP_ADDRESS

IP_NETMASK -- IP 掩码

类型	系统参数，自动存储到 FLASH
描述	控制器 IP 网络掩码。 只有带以太网接口的控制器支持，读取时以 32 位整数返回。
语法	IP_NETMASK = dot.dot.dot.dot
适用控制器	通用
例子	在线命令输入 >>IP_NETMASK =255.255.252.0 >>PRINT IP_NETMASK 输出：16580607 具体转换过程参考 IP_ADDRESS 例程
相关指令	IP_GATEWAY ， IP_ADDRESS

IP_IFDHCP -- IP 自动获取

类型	系统参数
描述	是否使用 DHCP 设置，缺省是不使用。 自动获取 IP，设置了这个后 RTSys 的固定 IP 没用，只能自动获取，此参数修改后要重启。
语法	IP_IFDHCP = 1 或 0 1-自动获取 IP；0-使用固定 IP
适用控制器	通用
例子	在线命令输入 >>IP_IFDHCP =1
相关指令	IP_ADDRESS

IP_IFDHCP2 -- IP 自动获取 2

类型	系统参数
描述	5 系列控制器的第二个网口是否使用 DHCP 设置，缺省是不使用。 自动获取 IP，设置了这个后 RTSys 的固定 IP 没用，只能自动获取，此参数修改后要重启。

语法	IP_IFDHCP2 = 1 或 0 1-自动获取 IP； 0-使用固定 IP
适用控制器	5 系列控制器
例子	在线命令输入 >>IP_IFDHCP2 =1
相关指令	IP_ADDRESS

14.5 控制器信息指令

VERSION_FPGA – 系统 FPGA 版本

类型	系统状态
描述	系统 FPGA 版本号。
语法	VAR1 = VERSION_FPGA
适用控制器	通用
例子	?*VERSION_FPGA '打印出 FPGA 版本 结果:240104 <pre>>>?*VERSION_FPGA 240104</pre> 在线命令: ?*VERSION_FPGA “控制器状态” - “SoftVersion” 查看

VERSION_BUILD -- 系统固件创建日期

类型	系统状态
描述	系统固件创建的日期。
语法	VAR1 = VERSION_BUILD
适用控制器	通用
例子	?*VERSION_BUILD '打印出固件创建的日期 结果:20240111 <pre>>>?*VERSION_BUILD 20240111</pre> 在线命令: ?*VERSION_BUILD “控制器状态” - “SoftVersion” 查看

VERSION_DATE -- 系统固件版本

类型	系统状态
----	------

描述	系统固件版本号。
语法	VAR1 = VERSION_DATE
适用控制器	通用
例子	?*VERSION_DATE '打印出固件版本 结果:20190305 <pre>>>?*VERSION_DATE 20190305</pre> 在线命令: ?*VERSION_DATE “控制器状态” - “SoftVersion” 查看

VERSION -- 系统软件版本

类型	系统状态
描述	系统软件版本号。
语法	VAR1 = VERSION
适用控制器	通用
例子	?*VERSION '打印出软件版本 结果:4.9300 <pre>>>?*VERSION 4.9300</pre> 在线命令: ?*VERSION “控制器状态” - “SoftVersion” 查看

ID_HARDWARE -- 控制器硬件型号

类型	系统只读参数
描述	返回控制器硬件型号。
语法	Val=ID_HARDWARE
适用控制器	通用
例子	在线命令输入 <pre>>>PRINT ID_HARDWARE '打印控制器型号 输出: 432 >>?*ID_HARDWARE 432</pre> 在线命令: ?*ID_HARDWARE “控制器状态” - “HardVersion” 查看
相关指令	CONTROL

CONTROL -- 控制器软件型号

类型	系统只读参数
描述	返回控制器软件型号。
语法	Val=CONTROL
适用控制器	通用
例子	在线命令输入 <pre>>>PRINT CONTROL</pre> 打印控制器型号 输出: 464 命令与输出 <pre>>>PRINT CONTROL 412</pre> 在线命令: PRINT CONTROL “控制器状态” - “SoftType” 查看 控制器状态 <pre>VirtualAxes: 32 RealAxes: 12 Tasks: 22 Files/3Files: 61/2 Modbus0x Bits: 8000 Modbus4x Regs: 8000 VR Regs: 8000 TABLE Regs: 320000 RomSize: 65536KB FlashSize: 259328KB SoftType: ZMC412 RTVersion: 4.930-20190212 Build Date: 20221103 IpAddress: 192.168.0.38 HardVersion: 432-0 ControllerID: 230601102</pre>
相关指令	ID_HARDWARE

SYSTEM_ZSET -- 控制器设置

类型	系统参数
描述	控制器设置。 设置参数: bit0: 1-VP_SPEED 缺省使用插补速度, 0-VP_SPEED 使用单轴的速度 bit1: 1-使用 MOVE_OP 精确输出功能, 0-MOVE_OP 普通方式 bit4: 1-对带编码器功能的轴, 使用编码器位置的 MOVE_OP 精准方式 bit7: 1-总线时钟优化, 0-脉冲轴时钟优化

	SYSTEM_ZSET 一旦开启，所有支持精准输出功能的输出口都变为精准模式，部分控制器型号在一个控制器周期内只能操作一个精准输出口，新版本固件不建议使用此指令，直接采用 AXIS_ZSET 指令对主轴开启精准输出。 20170505 以上固件版本支持这个位设置。 使用前要求 MPOS 必须跟随 DPOS，编码器精准功能受驱动器的响应影响，输出时刻速度越平稳，效果越好。 总线时钟优化可在总线启动后，通过?*ETHERCAT 命令来查看是否有使用 <pre>>>?*ethercat Slot:0 contain 1 nodes.</pre> dc:ecat-sensitive.Lostcount:0-0
语法	可读: value=SYSTEM_ZSET 可写: SYSTEM_ZSET=value
适用控制器	通用
例子	SYSTEM_ZSET=1 '设置 bit0 为 1
相关指令	MOVE_OP , AXIS_ZSET_MOVE_OP -- 缓冲输出

LEDOUT -- 控制器指示灯

类型	系统指令
描述	操作控制器指示灯。 电源指示灯无法操作。
语法	LEDOUT (num,state) num: 指示灯编号; 1-RUN 指示灯, 2-ALM 指示灯 state: 状态; 0-关闭, 1-打开
适用控制器	通用
例子	LEDOUT(1,0) '关闭运行灯 LEDOUT(2,0) '关闭报警灯

SERIAL_NUMBER -- 控制器唯一 ID

类型	系统只读参数
描述	返回控制器唯一 ID。 是一个唯一的序列号，生成 ZAR 包的时候也可以和这个 ID 绑定，这样这个 ZAR 只能为这个控制器所用。
语法	Var=SERIAL_NUMBER
适用控制器	通用
例子	例一 PRINT SERIAL_NUMBER '打印控制器 ID 打印结果:

	191201941
	例二 控制器的 9 位 ID 保存到 VR，由于 4 系列以下控制器 VR 为单精度浮点型，只有 8 位有效数值，可采用两个 VR 空间保存。
	?SERIAL_NUMBER
	GLOBAL giA,giB
	giA = SERIAL_NUMBER MOD 100000 '取余数
	VR(0)=giA
	giB = SERIAL_NUMBER \ 100000 '整除
	VR(1)=giB
	PRINT giA,VR(0)
	PRINT giB,VR(1)
	打印结果：
	191201941
	1941 1941
	1912 1912

SERVO_PERIOD -- 总线通讯周期

类型	系统参数
描述	总线伺服通讯周期。 缺省 1000 微秒，修改需要固件支持。 4 系列控制器标准固件版本 20170713、4 系列控制器 fast 固件版本 20190106、5 系列控制固件版本 20180307 增加支持用户自行修改周期的功能，必须在控制器支持的周期范围内修改，修改后要重启才生效。 Rtex 驱动使用时请按以下设置 控制器周期 500us 时，将驱动器的 P7.20 设置为 3，P7.21 设置为 1。 控制器周期 1000us 时，将驱动器的 P7.20 设置为 6，P7.21 设置为 1。 查看 Rtex 驱动器设置请看例子。
语法	value=SERVO_PERIOD
适用控制器	通用
例子	在线命令打印控制器通讯周期和 Rtex 驱动器设置 >>PRINT SERVO_PERIOD '打印伺服更新周期，结果 1000 >>DRIVE_READ(7*256+20)AXIS(0) '打印 Rtex 轴 0 驱动器周期设置 >>DRIVE_READ(7*256+21)AXIS(0) '打印 Rtex 轴 0 驱动器周期比值设置 部分固件修改周期： SERVO_PERIOD=500 SERVO_PERIOD=1000
相关指令	SERVO

SYS_ZFEATURE -- 系统规格

类型	系统参数
描述	获取系统最大规格。 ECI150830 以上固件支持，4 系列 170530 以上固件支持。
语法	num = SYS_ZFEATURE (code) num: 返回值 code: 获取数据类型 0-最大虚拟轴数 1-支持电机个数 2-本体输入个数 3-本体输出个数 4-本体模拟输入 AIN 5-本体模拟输出 AOUT 6-PWM 个数 7-BASIC 任务数，不包含中断任务 8-总线槽位数 9-3 次文件个数 10-串口链接数 11-网络链接数 12-网络自定义链接数 0-不支持 13-网络互联主端数，0-不支持，（从端总是支持） 14-FLASH 块数 15-FLASH 块大小 16-VR 个数 17-MODBUS_BIT 个数 18-MODBUS_REG 个数 19-定时器个数 20-数组空间 21-虚拟最大输入个数，对应 PLC 的 X 寄存器个数 22-虚拟最大输出个数，对应 PLC 的 Y 寄存器个数 23-虚拟最大 AIN 个数 24-虚拟最大 AOUT 个数 25-PLC 计数器 26-PLC S 寄存器 27-PLC V 寄存器 28-PLC Z 寄存器 29-PLC L 寄存器 30-HMI 个数，（包括网络 HMI 与本体 HMI） 31-本体自带 HMI 个数 32-最大显存 33-是否支持 ZINDEX 功能 34-RTLOG 的最大个数. 35-支持的子卡数量. (早期版本不支持, 7 系缺省最大 16 个)

36-当前有带的子卡数量.(包括虚拟子卡)
37-当前有带的子卡数量.(不含虚拟子卡)

39-PORT 最大规格,(230814 增加).
40-ZV 最多锁存个数
41-ZV 建议最多任务个数
42-ZV 锁存最大字节空间

50-是否支持 NC 功能
51-是否支持 CANOPEN
52-固件是否带机械手支持.
53-ECAT 总线槽位个数
54-RTEX 总线槽位个数
55-XY2 槽位个数
56-是否支持 MODBUSM 功能
57-是否支持 MODBUSM 的 SEND 指令
58-是否支持 U 盘
59-网络编码器(示教盒手轮)个数
60-最快周期
61-最慢周期
62-ZAR 程序空间(kbytes 单位)
63-Nandflash 空间(kbytes 单位)
64-Nandflash 剩余空间(kbytes 单位)

66-平台编译器类型
67-脉冲的位数.
68-TABLE 的位数.
69-平台指针长度
70-是否支持 C 语言
71-是否支持防碰撞功能
72-是否支持 DDZ 多动子功能.

74-内置驱动器最大个数
75-内置驱动实际个数
76-驱动器参数最大规格
77-示波器最大通道数

99-控制器功能规格, 按 BIT, 2411 支持
 bit2, 凸轮支持
 bit3, 直线插补支持
 bit4, 圆弧插补支持
 bit5, 连续插补支持
 bit10, 基本机械手支持
 bit11, 复杂机械手支持.

	bit12, 多动子支持 bit13, CNC 支持 bit14, 视觉支持 101-是否集成 ZMIO 102-ZMIO 最大输入个数 103-ZMIO 最大输出个数 104-ZMIO 最大 AD 个数 105-ZMIO 最大 DA 个数
适用控制器	通用
例子	?SYS_ZFEATURE (0) '打印最大轴数 ?SYS_ZFEATURE (17) '打印 MODBUS_BIT 个数
相关指令	/

SYS_IOSET -- 特殊 IO 切换

类型	系统指令
描述	特殊 IO 的常开常闭切换。
语法	SYS_IOSET=value 0: 缺省方式, 兼容以前, 自动根据控制器类型来选择, 原点限位等特殊输入 1: ZMC 方式, 原点等特殊输入缺省常闭方式 2: ECI 方式; 原点等特殊输入缺省常开方式
适用控制器	通用
例子	SYS_IOSET=1

LASER_SET – 能量并口输出切换

类型	系统指令
描述	置能量并口是否使用 AOUT 指令输出或 OP 指令输出。
语法	LASER_SET(isel, value) isel: 1-设置能量并口是否使用 AOUT 指令输出 value: 1-使能 AOUT 输出
适用控制器	504SCAN
例子	LASER_SET(1, 1) '使用 AOUT 设置激光能量, 此时对应的原来 OP 指令无效

ZML_DEFSHIFT -- ZML 设备 shift 时间

类型	系统参数
描述	ZML 所有设备的缺省 shift 时间, ns 单位, 修改自动保存 FLASH. 修改后必须重启生效。 version_build 20240719 增加此功能。

语法	value=ZML_DEFSHIFT
适用控制器	通用
例子	在线命令打印 >>PRINT ZML_DEFSHIFT

14.6 日志指令

实时错误日志功能，通过实时 FIFO 来存储常见的错误信息，会循环产生的错误一般不写日志。

4 系列以上控制器 220907 以后的固件增加，早期控制器由于内存有限不能增加此功能。

RTLOG_COUNT -- 当前记录日志条数

类型	日志指令
描述	获取当前记录的日志条数，日志的最大能写入的条数可通过 SYS_ZFEATURE(34)获取。
语法	VAL = RTLOG_COUNT()
适用控制器	4 系列以上控制器 220907 以后的固件增加
例子	? RTLOG_COUNT
相关指令	RTLOG_CLEAR

RTLOG_CLEAR -- 清除当前记录的日志

类型	日志指令
描述	清除当前记录的日志。
语法	RTLOG_CLEAR ([number]) number: 要清除的条数，从最早写入的日志开始清除起，缺省=-1，表示全部清除
适用控制器	4 系列以上控制器 220907 以后的固件增加
例子	RTLOG_CLEAR (10) '清除最早的 10 条日志 RTLOG_CLEAR (-1) '日志全部清除
相关指令	RTLOG_COUNT

RTLOG_ADD – 添加日志记录错误信息

类型	日志指令
描述	应用层可以使用系统日志来记录错误信息。
语法	RTLOG_ADD (code, "string") code: 错误编号 string:字符串，错误信息
适用控制器	4 系列以上控制器 220907 以后的固件增加
例子	RTLOG_ADD(1,"错误信息")

相关指令	RTLOG_CLEAR
------	-----------------------------

RTLOG_CODE -- 获取日志的错误编号

类型	日志指令
描述	获取日志的错误编号。
语法	VAL = RTLOG_CODE([index]) index: 日志选择, 缺省=0 表示最近发生的日志, index=-1 表示最早发生的日志
适用控制器	4 系列以上控制器 220907 以后的固件增加
例子	<pre>RTLOG_CLEAR (-1) '清空 FOR i = 1 TO 20 RTLOG_ADD(i,"123") '写入日志 WA(10) NEXT ?RTLOG_CODE(0) '获取记录的日志编号 打印结果: 20</pre>
相关指令	RTLOG_CLEAR

RTLOG_TIMES\$ -- 获取日志的出错时间

类型	字符串函数
描述	获取日志的出错时间。
语法	String = RTLOG_TIMES\$ ([index]) index: 日志选择, 缺省=0 表示最近发生的日志, index=-1 表示最早发生的日志
适用控制器	4 系列以上控制器 220907 以后的固件增加
例子	<pre>RTLOG_CLEAR (-1) FOR i = 1 TO 20 RTLOG_ADD(i,"123") WA(10) NEXT ?RTLOG_TIMES\$(0) '获取最近发生的日志时间 打印结果: 2022/10/09 16:53:11</pre>
相关指令	RTLOG_CODE

RTLOG_INFO -- 获取日志的错误信息

类型	字符串函数
描述	获取日志的错误信息。
语法	String = RTLOG_INFO ([index]) index: 日志选择, 缺省=0 表示最近发生的日志, index=-1 表示最早发生的日志

适用控制器	4 系列以上控制器 220907 以后的固件增加
例子	<pre>RTLOG_CLEAR (-1) FOR i = 1 TO 20 RTLOG_ADD(i,"123") WA(10) NEXT ?RTLOG_INFO(0) '获取最近发生的日志信息 打印结果：123</pre>
相关指令	RTLOG_INFO2

RTLOG_INFO2 -- 获取日志的错误信息 2

类型	字符串函数
描述	获取日志的错误信息，带错误编号与出错时间。
语法	String = RTLOG_INFO ([index]) index: 日志选择，缺省=0 表示最近发生的日志，index=-1 表示最早发生的日志
适用控制器	4 系列以上控制器 220907 以后的固件增加
例子	<pre>RTLOG_CLEAR (-1) FOR i = 1 TO 20 RTLOG_ADD(i,"123") WA(10) NEXT ?RTLOG_INFO(0) '获取最近发生的日志信息 打印结果：err20,2022/10/09 16:53:11,123</pre>
相关指令	RTLOG_INFO

?*RTLOG -- 打印当前记录的日志

类型	日志指令
描述	快速打印最近发生的 10 条日志。
语法	?*RTLOG
适用控制器	4 系列以上控制器 220907 以后的固件增加
例子	<pre>RTLOG_CLEAR (-1) FOR i = 1 TO 20 RTLOG_ADD(i,"123") WA(10) NEXT ?*RTLOG 打印结果： RTLOG_COUNT:20 err20,2022/10/09 17:02:48,123</pre>

	err19,2022/10/09 17:02:48,123 err18,2022/10/09 17:02:48,123 err17,2022/10/09 17:02:48,123 err16,2022/10/09 17:02:48,123 err15,2022/10/09 17:02:48,123 err14,2022/10/09 17:02:48,123 err13,2022/10/09 17:02:48,123 err12,2022/10/09 17:02:48,123 err11,2022/10/09 17:02:48,123
相关指令	RTLOG_COUNT

14.7 TABLE 数组指令

TABLE -- 系统缺省数组

类型	系统数组
描述	系统缺省建立的全局数组，所有程序都可以访问。 数据录波缓冲区，凸轮数据表，螺距补偿表，机械手参数，等等都是用 table 来存储。
语法	TABLE(index) = value , VAR1 = TABLE(index), TABLE(index [, value1..]) Index: 索引号
适用控制器	通用
例子	TABLE(0) = 10 'table(0)赋值 10 TABLE(10,100,200,300) 'table(10)赋值 100, table(11)赋值 200, table(12)赋值 300
相关指令	TSIZE

TSIZE -- table 大小

类型	系统参数
描述	TABLE 的所有元素个数，可以修改 TABLE 空间大小。 请在程序开头第一个修改，在其它数组定义前，最好放第一行代码。 不能超过 TABLE 最大空间。
语法	Var=TSIZE TSIZE=Value
适用控制器	通用
例子	读取： PRINT TSIZE '打印出控制器 table 大小 设置： TSIZE=10000 '设置 table 的大小，不能超过控制器 table 最大 size
相关指令	TABLE

TABLESTRING -- 字符串格式打印 table

类型	字符串函数
描述	按照字符串格式打印 table 里的数据。 数据自动转换，打印出来的数据为 ASCII 码。
语法	TABLESTRING(index, length) index: 打印数据起始地址 length: 要打印的数据长度
适用控制器	通用
例子	例一 <pre>TABLESTRING(0,5)= "abc" '从 tablestring(0)开始保存字符串 PRINT TABLESTRING(0,5) '打印保存的字符串 '打印结果: abc</pre> 例二 <pre>TABLE(100,68,58,92) PRINT TABLESTRING(100,3) '字符串格式打印数据，转换为 ASCII 码 '打印结果: D:\</pre> 在 RTSys 软件的“工具”-“寄存器”可以查看 TABLE 寄存器的每个位置的数据。 

14.8 示波器相关指令

TRIGGER -- 触发示波器

类型	系统指令
描述	开始执行示波器的数据采样。 150723 以后版本支持，与 RTSys 的示波器功能合用。 示波器开启连续采集之后，不要在 Basic 里调用 TRIGGER 指令，建议手动触发采集。
语法	TRIGGER
适用控制器	通用
例子	例一：

	TRIGGER '示波器开始抓取下面运动的每时刻速度，存储在示波器、功能设置的 TABLE 区间 MOVE(10000) 例二： TRIGGER (1) '示波器触发连续采集
相关指令	SCOPE

SCOPE -- 数据采样

类型	系统指令
描述	数据采样，保存到 TABLE，最多同时采样 8 类数据。 使用 TRIGGER 开始自动采样，采样时间=采样周期*采样个数。 AIN 采集和 SCOPE 不能同时用。
语法	SCOPE(enable[, period]) SCOPE(enable, period, table_start, table_stop, p0 [,p1 [,p2 [,p3 [,p4 [,p5 [,p6 [,p7]]]]]])) enable: 使能与否 period: 系统周期数，一般为 1ms，可用 SERVO_PERIOD 查看 table_start: 采样数据存储在 TABLE 的起始位置 table_stop: TABLE 结束位置，减去起始位置为采样个数 p0~p7: 采样数据类型，等分存储在 TABLE 范围
适用控制器	通用
例子	BASE(0) ATYPE=1 UNITS=100 DPOS=0 SPEED=100 ACCEL=1000 SCOPE(ON,10,0,1000,DPOS(0),MSPEED(0)) '每隔 10ms 抓取 dpos 和 mspeed 存储在 'TABLE 0~999, 0~499 存 dpos, 500~999 存 mspeed 共采样 1000/2*10=5s TRIGGER '开始抓取 MOVE(10000)
相关指令	TRIGGER , SCOPE_POS

SCOPE_POS -- 采样点数

类型	系统指令
描述	只读，返回当前 SCOPE 采样数据的点数
语法	VAR = SCOPE_POS
适用控制器	通用
例子	BASE(0) ATYPE=1 UNITS=100 DPOS=0 SPEED=100 ACCEL=1000

	SCOPE(ON,10,0,1000,DPOS(0),MSPEED(0)) '采样设置 TRIGGER '开始抓取 MOVE(10000) WHILE 1 ?SCOPE_POS '返回当前已采样保存的点数 WEND
相关指令	SCOPE

14.9 VR 相关指令

CLEAR -- 清除 VR

类型	系统指令
描述	清除所有 VR 内容。
语法	CLEAR()
适用控制器	通用
例子	CLEAR() '清除 VR 全部数据
相关指令	VR

VR -- 掉电保存

类型	系统指令
描述	掉电保存寄存器组，32 位浮点型。 注意不同型号控制器的数量有区别。 保存浮点数，与 VR_INT 和 VRSTRING 共用一个空间。
语法	VR(index) = value , VAR1 = VR(index)
适用控制器	通用，ECI 系列不支持掉电保存
例子	VR(0) = 10.58 aaa = VR(0) ?aaa 打印结果 10.5800
相关指令	VR_INT ， VRSTRING

VR_INT -- 掉电保存整型

类型	系统指令
描述	掉电保存寄存器组，32 位整型。 注意不同型号控制器的数量有区别。 只能保存整型，与 VR 和 VRSTRIG 共用一个空间。
语法	VR_INT(index) = value , VAR1 = VR_INT(index)
适用控制器	通用

例子	<pre>VR_INT(0) = 10.58 aaa = VR_INT(0) ?aaa</pre> 打印结果 10，仅保留整数部分
相关指令	VR , VRSTRING

VRSTRING -- 掉电保存字符串

类型	字符串函数
描述	用 VR 来存储字符串。 保存 ASCII 码，与 VR 和 VR_INT 共用一个空间，一个字符占用一个 VR。
语法	<pre>VRSTRING (index[, chares])</pre> index: 起始的 VR 编号，编号从 0 开始 chares: 读取的字符总数
适用控制器	通用
例子	在线命令输入 <pre>>> VRSTRING (0, 8) = "abc" '保存字符串 >>PRINT VRSTRING (0, 8)</pre> 输出: abc
相关指令	VR , VR_INT

第十五章 存储相关指令

ZMotion 运动控制器都拥有内部 FLASH 存储器，部分型号控制器具备外部存储器接口，可以接 U 盘或 SD 卡，参见具体控制器的硬件手册。

U 盘或 SD 卡要格式化为 FAT 格式。

15.1 U 盘相关指令

FILE -- U 盘文件操作

类型	文件指令																																					
描述	加载、搜索控制器或 U 盘文件。 根据对应字符串选择功能。 U 盘读取文件系统支持 fat32 和 fat16，不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。 VPLC5 系列是 Linux 系统，读取文件名 filename 是区分大小写的，名称必须都大写。																																					
语法	value = FILE "function" ,... <table><tr><td>"LOAD_ZAR"</td><td colspan="2"> FILE "LOAD_ZAR","filename" 加载 U 盘里面的 ZAR 升级文件。 filename: 程序文件名 升级失败返回 0，同时会 WARN 输出失败原因。 升级成功后会自动启动 ZAR 文件，因此返回值 TRUE 没有意义。  升级后原来的调试状态的断点会被清除掉。 </td></tr><tr><td>"LOAD_TCF"</td><td colspan="2"> FILE "LOAD_TCF","filename",tableindex,maxsize 专有文件 TCF 文件读取。 filename: 文件名 tableindex: 存储起始 TABLE 索引 maxsize: 存储 TABLE 总索引个数 <table><tr><td>TABLE0</td><td>标志位</td><td></td></tr><tr><td>TABLE1</td><td>总点数</td><td></td></tr><tr><td>TABLE2</td><td>胶头编号</td><td></td></tr><tr><td>TABLE100</td><td>点类型</td><td>第一个点</td></tr><tr><td>TABLE101</td><td>X 坐标</td><td></td></tr><tr><td>TABLE102</td><td>Y 坐标</td><td></td></tr><tr><td>TABLE103</td><td>Z 坐标</td><td></td></tr><tr><td>TABLE104</td><td>预留</td><td></td></tr><tr><td>TABLE105</td><td>点类型</td><td>第二个点</td></tr><tr><td>TABLE106</td><td>X 坐标</td><td></td></tr></table></td></tr></table>		"LOAD_ZAR"	FILE "LOAD_ZAR","filename" 加载 U 盘里面的 ZAR 升级文件。 filename: 程序文件名 升级失败返回 0，同时会 WARN 输出失败原因。 升级成功后会自动启动 ZAR 文件，因此返回值 TRUE 没有意义。  升级后原来的调试状态的断点会被清除掉。		"LOAD_TCF"	FILE "LOAD_TCF","filename",tableindex,maxsize 专有文件 TCF 文件读取。 filename: 文件名 tableindex: 存储起始 TABLE 索引 maxsize: 存储 TABLE 总索引个数 <table><tr><td>TABLE0</td><td>标志位</td><td></td></tr><tr><td>TABLE1</td><td>总点数</td><td></td></tr><tr><td>TABLE2</td><td>胶头编号</td><td></td></tr><tr><td>TABLE100</td><td>点类型</td><td>第一个点</td></tr><tr><td>TABLE101</td><td>X 坐标</td><td></td></tr><tr><td>TABLE102</td><td>Y 坐标</td><td></td></tr><tr><td>TABLE103</td><td>Z 坐标</td><td></td></tr><tr><td>TABLE104</td><td>预留</td><td></td></tr><tr><td>TABLE105</td><td>点类型</td><td>第二个点</td></tr><tr><td>TABLE106</td><td>X 坐标</td><td></td></tr></table>		TABLE0	标志位		TABLE1	总点数		TABLE2	胶头编号		TABLE100	点类型	第一个点	TABLE101	X 坐标		TABLE102	Y 坐标		TABLE103	Z 坐标		TABLE104	预留		TABLE105	点类型	第二个点	TABLE106	X 坐标	
"LOAD_ZAR"	FILE "LOAD_ZAR","filename" 加载 U 盘里面的 ZAR 升级文件。 filename: 程序文件名 升级失败返回 0，同时会 WARN 输出失败原因。 升级成功后会自动启动 ZAR 文件，因此返回值 TRUE 没有意义。  升级后原来的调试状态的断点会被清除掉。																																					
"LOAD_TCF"	FILE "LOAD_TCF","filename",tableindex,maxsize 专有文件 TCF 文件读取。 filename: 文件名 tableindex: 存储起始 TABLE 索引 maxsize: 存储 TABLE 总索引个数 <table><tr><td>TABLE0</td><td>标志位</td><td></td></tr><tr><td>TABLE1</td><td>总点数</td><td></td></tr><tr><td>TABLE2</td><td>胶头编号</td><td></td></tr><tr><td>TABLE100</td><td>点类型</td><td>第一个点</td></tr><tr><td>TABLE101</td><td>X 坐标</td><td></td></tr><tr><td>TABLE102</td><td>Y 坐标</td><td></td></tr><tr><td>TABLE103</td><td>Z 坐标</td><td></td></tr><tr><td>TABLE104</td><td>预留</td><td></td></tr><tr><td>TABLE105</td><td>点类型</td><td>第二个点</td></tr><tr><td>TABLE106</td><td>X 坐标</td><td></td></tr></table>		TABLE0	标志位		TABLE1	总点数		TABLE2	胶头编号		TABLE100	点类型	第一个点	TABLE101	X 坐标		TABLE102	Y 坐标		TABLE103	Z 坐标		TABLE104	预留		TABLE105	点类型	第二个点	TABLE106	X 坐标							
TABLE0	标志位																																					
TABLE1	总点数																																					
TABLE2	胶头编号																																					
TABLE100	点类型	第一个点																																				
TABLE101	X 坐标																																					
TABLE102	Y 坐标																																					
TABLE103	Z 坐标																																					
TABLE104	预留																																					
TABLE105	点类型	第二个点																																				
TABLE106	X 坐标																																					

		TABLE107	Y 坐标		
		TABLE108	Z 坐标		
		TABLE109	预留		
	"LOAD_BYTE"	FILE "LOAD_BYTE", "filename", tableindex, maxsize, offset 字节方式加载文件。 filename: 文件名, 读取 csv 文件时, 数据需要放到第一列 tableindex: 存储起始 TABLE 索引 maxsize: 存储 TABLE 总索引个数 offset: 文件开始读取的字节偏移			
		TABLE0	总字节数		
		TABLE1	读取到的第一个字节		
		TABLE2	第二个字节		
		TABLEn	第 n 个字节		
	"FIND_FIRST"	FILE "FIND_FIRST", type, modbus [,dir] 搜索 U 盘文件。增加可选参数, dir, 字符串参数。 type: 1-文件/2-文件夹/ ".extend" 文件后缀名 modbus: modbus_string(modbus) 存储查找的结果, 优先推荐 modbus, vr 也可以用 dir: 指定搜索路径, 字符串表达式格式, 不指定缺省搜索 U 盘根目录。			
	"FIND_NEXT"	FILE "FIND_NEXT", modbus 搜索下一个 U 盘文件。 modbus: modbus_string(modbus) 存储查找的结果, 优先推荐 modbus, vr 也可以用			
	"FLASH_FIRST"	FILE "FLASH_FIRST", type, modbus 搜索 FLASH 文件, 只支持 BIN 和 Z3P 文件。 type: 1-文件/2-文件夹/ ".extend" 文件后缀名 modbus: modbus_string(modbus) 存储查找的结果, 优先推荐 modbus, vr 也可以用			
	"FLASH_NEXT"	FILE "FLASH_NEXT", modbus 搜索下一个 FLASH 文件。 modbus: modbus_string(modbus) 存储查找的结果, 优先推荐 modbus, vr 也可以用			
	"FLASH_DEL"	FILE "FLASH_DEL", "fileordir" 删除指定文件。 file: 文件名全称, 带扩展名。 支持文件夹的删除, 可以带盘符 A、C C 盘, 表示 FLASH 目录, 不带盘符缺省认为是 FLASH 目录 A 盘, 表示 U 盘			
	"DELETE"	FILE "DELETE", "filename" 删除 U 盘指定文件。4 系以下控制器只支持 Z3P 和 BIN 类型文件, 固件版本 20240719 以后, 4 系以下 NAND flash 类型的控制器增加 CSV 类型			

		filename: 文件名全称, 带扩展名。
	"COPY_FROM"	FILE "COPY_FROM", "FLASH 文件名"[, "U 盘文件名"] FLASH 文件拷贝到 U 盘, 只支持 BIN 和 Z3P 文件。 FLASH 文件名规则: SD 块号.BIN 如 SD0.BIN 表示 FLASH 块号 0 SD1.BIN 表示 FLASH 块号 1
	"COPY_TO"	FILE "COPY_TO", "U 盘文件名"[, "FLASH 文件名"] U 盘文件拷贝到 FLASH, 只支持 BIN 和 Z3P 文件。
	"FLASH_COPY"	FILE "FLASH_COPY" "src","des" 支持文件夹的拷贝 可以带盘符 A、C C 盘, 表示 FLASH 目录 A 盘, 表示 U 盘
	"MAKE_DIR"	FILE "MAKE_DIR" "路径"
	"PATHD"	FILE "PATHD" "dir" 5 系, 7 系, 230909 增加; 增加 D 盘符, 可以手动调整映射路径; 从而支持特殊路径的操作。
function: 功能选择		
适用控制器	通用, 使用 U 盘功能时要求控制器带 U 盘接口	
例子	例一 下载 zar 升级程序 <pre> DIM result '定义变量 IF U_STATE=TRUE THEN 'U 盘插入判断 result = FILE "find_first",".zar",10 '扫描第一个 zar 格式文件, 文件名保存到 VR IF result=TRUE THEN '扫描成功判断 FILE"load_zar",VRSTRING(10,20) '下载 VR 保存的文件名对应的 zar 文件 ENDIF ENDIF END </pre> 例二 查找 zar 升级程序 <pre> FILE "find_next",10 '查找下一个 zar 文件存储结果到 vrstring(10) </pre> 例三 FLASH 与 U 盘数据互相拷贝 <pre> DIM a,aa(8) a=10 FOR i=0 TO 7 aa(i)=i NEXT WHILE 1 IF SCAN_EVENT(IN(0))> 0 THEN FLASH_WRITE 1,a aa FILE"copy_from","sd1.bin" '将 flash 块 1 的数据复制到 U 盘的 sd1 文件 PRINT "flash 块的数据复制到 U 盘" ENDIF END </pre>	

```
ELSEIF  SCAN_EVENT(IN(1))> 0 THEN
    FILE"copy_to","sd1.bin" '读取 sd1 的数据写入 flash 块 1
    PRINT "U 盘数据写入 flash"
    FLASH_READ 1,a,aa
    PRINT *aa
ENDIF
WEND
END
```

例四 读取/删除 U 盘文件

FILE "LOAD_BYTE","00.txt",200,10,0 '读取 U 盘中 00.txt 文本文件的数据保存到 table(200)开始的 10 个地址中，偏移量为 0，从第一个字符开始读取

FILE "DELETE", "sd0.bin" '删除 U 盘上名称为 sd0.bin 的文件

00.txt 文件内容：ZMOTION

读取结果：第一个位置存储字符个数，后面的为依次存储字符数据。

寄存器名	值	
DT(200)	7.000	
DT(201)	90.000	
DT(202)	77.000	
DT(203)	79.000	
DT(204)	84.000	
DT(205)	73.000	
DT(206)	79.000	
DT(207)	78.000	
DT(208)	0.000	
DT(209)	0.000	

例五

FILE "FIND_FIRST", 1, 0, "C:\DIR1" '搜索 U 盘的 DIR1 目录

U_STATE -- U 盘状态

类型	系统状态函数
描述	检查 U 盘是否插入。 有插入返回 TRUE，否则返回 FALSE。 不具备外部存储接口的控制器不支持此命令。 U 盘插入时，如果控制器没有程序运行，会自动加载 AUTORUN.ZAR，出厂没下载任何程序才会生效，4 系列产品 20170423 以上版本支持。如果有程序运行，不会自动加载，此时需要程序里面自行加载。 U 盘里面的文件不要放太多。

	U 盘读取文件系统支持 fat32 和 fat16，不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。
语法	Val=U_STATE
适用控制器	带 U 盘接口
例子	?U_STATE '打印 U 盘状态 IF U_STATE = TRUE THEN 'U 盘已插入 U_READ 1, VAR, ARRAY1, ARRAY2(1) 'U 盘数据读取 ENDIF
相关指令	U_READ , U_WRITE

U_READ -- 从 U 盘读取

类型	存储指令
描述	从外部存储器读取数据到变量或数组里面。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 ieee 浮点数顺序存储，一个变量或一个数组元素占用一个浮点数，可以通过 PC 事先做好文件，然后用 U_READ 指令读取。 U 盘读取文件系统支持 fat32 和 fat16，不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。
语法	U_READ sect_num, [,varname] [,arrayname] [,arrayname(a)][, arrayname(a,length)] sect_num: 文件编号，对应到 SD【filenum】.BIN varname: 变量名 arrayname: 数组名，可以为 TABLE, VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	带 U 盘接口
例子	IF U_STATE = TRUE THEN 'U 盘已插入 U_READ 1, VAR, TABLE(0), ARRAY2(1) '读取 U 盘文件 SD1 数据 ENDIF
相关指令	U_WRITE , U_READ2 , U_STATE

U_READDBL -- 从 U 盘读取-double

类型	存储指令
描述	从外部存储器读取数据到变量或数组里面。 使用方法同 U_READ，区别是 U_READ 读取类型是 float，32 位，U_READDBL 读取的数据类型是 double，64 位。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 ieee 浮点数顺序存储，一个变量或一个数组元素占用一个浮点数，可以通过 PC 事先做好文件，然后用 U_READ 指令读取。

	U 盘读取文件系统支持 fat32 和 fat16，不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。
语法	U_READDBL sect_num, [,varname] [,arrayname] [,arrayname(a)][, arrayname(a,length)] sect_num: 文件编号，对应到 SD【filenum】.BIN varname: 变量名 arrayname: 数组名，可以为 TABLE，VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	带 U 盘接口的 4 系列及以上控制器，20190128 及以上固件支持
例子	IF U_STATE = TRUE THEN 'U 盘已插入 U_READDBL 1, VAR, TABLE(0), ARRAY2(1) '读取 U 盘文件 SD1 数据 ENDIF
相关指令	U_WRITE , U_READ2 , U_STATE

U_READ2 -- 从 U 盘读取 2

类型	存储指令
描述	从外部存储器读取数据到变量或数组里面，支持设定文件读取的起始位置。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 ieee 浮点数顺序存储，一个变量或一个数组元素占用一个浮点数，可以通过 PC 事先做好文件，然后用 U_READ 指令读取。 U 盘读取文件系统支持 fat32 和 fat16，不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。
语法	U_READ2 sect_num, start_num[, varname][, arrayname][, arrayname(a)][, arrayname(a,length)] sect_num: 文件编号，对应到 SD【filenum】.BIN start_num: 文件内读取的起始位置 varname: 变量名 arrayname: 数组名，可以为 TABLE，VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	带 U 盘接口
例子	IF U_STATE = TRUE THEN 'U 盘已插入 U_READ2 1, 10, VAR, TABLE(0), ARRAY2(1) '读取 U 盘文件 SD1 数据从 SD1 位置 10 开始 ENDIF
相关指令	U_READ , U_WRITE , U_STATE

U_READ2DBL -- 从 U 盘读取 2-double

类型	存储指令
----	------

描述	从外部存储器读取数据到变量或数组里面，支持设定文件读取的起始位置。 使用方法同 U_READ2，区别是 U_READ2 读取类型是 float，32 位，U_READ2DBL 读取的数据类型是 double，64 位。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 ieee 浮点数顺序存储，一个变量或一个数组元素占用一个浮点数，可以通过 PC 事先做好文件，然后用 U_READ 指令读取。 U 盘读取文件系统支持 fat32 和 fat16，不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。
语法	<pre>U_READ2DBL sect_num, star_num [,varname] [,arrayname] [,arrayname(a)] [,arrayname(a,length)]</pre> sect_num: 文件编号，对应到 SD【filenum】.BIN start_num: 文件内读取的起始位置 varname: 变量名 arrayname: 数组名，可以为 TABLE，VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	带 U 盘接口的 4 系列及以上控制器，20190128 及以上固件支持
例子	<pre>IF U_STATE = TRUE THEN 'U 盘已插入 U_READ2DBL 1,10,VAR, TABLE(0), ARRAY2(1) '读取 U 盘文件 SD1 数据 从 SD1 位置 10 开始 ENDIF</pre>
相关指令	U_READ ， U_WRITE ， U_STATE

U_READDSB -- DSB 文件读取

类型	存储指令																					
描述	DSB 文件读取。 不具备外部存储接口的控制器不支持此命令。 U 盘读取文件系统支持 fat32 和 fat16，不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。																					
语法	U_READDSB "filename", tableindex, maxsize [, flag] filename: 文件名 tableindex: 存储起始 TABLE 索引 maxsize: 存储 TABLE 总索引个数 flag: 预留 <table><tr><td>编号</td><td>内容</td><td></td></tr><tr><td>0</td><td>标志位: 标明一行的变量数(暂无用)</td><td></td></tr><tr><td>1</td><td>针迹数:</td><td></td></tr><tr><td>2--29</td><td>文件名:28 个字符</td><td></td></tr><tr><td>30</td><td>换色次数:</td><td></td></tr><tr><td>31</td><td>+X:</td><td></td></tr><tr><td>32</td><td>-X:</td><td></td></tr></table>	编号	内容		0	标志位: 标明一行的变量数(暂无用)		1	针迹数:		2--29	文件名:28 个字符		30	换色次数:		31	+X:		32	-X:	
编号	内容																					
0	标志位: 标明一行的变量数(暂无用)																					
1	针迹数:																					
2--29	文件名:28 个字符																					
30	换色次数:																					
31	+X:																					
32	-X:																					

	33	+Y:	
	34	-Y:	
	35	AX: +	
	36	AY: -	
	37	MX: +	
	38	MY: +	
	39	PD:	
	40	L_limt	左边界坐标
	41	R_limt	右边界坐标
	42	U_limt	上边界坐标
	43	D_limt	下边界坐标
	99	已读到总行数	
	100	行类型	第一个点
	101	参数 1: X 相对位移	
	102	参数 2: Y 相对位移	
	103	参数 3: 相对起针点 X 坐标	
	104	参数 4: 相对起针点 Y 坐标	
	105	行类型	第二个点
	106	参数 1: X 相对位移	
	107	参数 2: Y 相对位移	
适用控制器	带 U 盘接口		
相关指令	U_READ , U_WRITE , U_STATE		

U_WRITE -- 输出到 U 盘

类型	存储指令
描述	存储变量或者数组,数组的单个或部分元素到外部存储器里面。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 IEEE 浮点数顺序存储, 一个变量或一个数组元素占用一个浮点数, 可以通过 PC 事先做好文件, 然后用 U_READ 指令读取。 4 系列以下建议使用 USB2.0 的 U 盘, 4 系列或以上支持 USB3.0。
语法	<code>U_WRITE sect_num, [,varname] [,arrayname] [,arrayname(a)] [,arrayname(a,length)]</code> sect_num: 文件编号, 对应到 SD【filenum】.BIN varname: 变量名 arrayname: 数组名, 可以为 TABLE, VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	带 U 盘接口
例子	<pre>IF U_STATE = TRUE THEN 'U 盘已插入 U_WRITE 0, TABLE(0, 10) 'TBALE0-10 的数据写入 U 盘文件 SD0 ENDIF</pre>
相关指令	U_READ , U_STATE

U_WRITEDBL -- 输出到 U 盘-double

类型	存储指令
描述	存储变量或者数组,数组的单个或部分元素到外部存储器里面。 使用方法同 U_WRITE, 区别是 U_WRITE 输出类型是 float, 32 位, U_WRITEDBL 输出的数据类型是 double, 64 位。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 IEEE 浮点数顺序存储, 一个变量或一个数组元素占用一个浮点数, 可以通过 PC 事先做好文件, 然后用 U_READDBL 指令读取。 4 系列以下建议使用 USB2.0 的 U 盘, 4 系列或以上支持 USB3.0。
语法	<pre>U_WRITEDBL sect_num, [,varname] [,arrayname] [,arrayname(a)] [,arrayname(a,length)]</pre> sect_num: 文件编号, 对应到 SD【filenum】.BIN varname: 变量名 arrayname: 数组名, 可以为 TABLE, VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	带 U 盘接口的 4 系列及以上控制器, 20190128 及以上固件支持
例子	<pre>IF U_STATE = TRUE THEN 'U 盘已插入 U_WRITEDBL 0, TABLE(0, 10) 'TBAL0-10 的数据写入 U 盘文件 SD0 ENDIF</pre>
相关指令	U_READ , U_STATE

U_WRITE2 -- 输出到 U 盘 2

类型	存储指令
描述	存储变量或者数组,数组的单个或部分元素到外部存储器里面。 不具备外部存储接口的控制器不支持此命令。 文件格式为 32 位 IEEE 浮点数顺序存储, 一个变量或一个数组元素占用一个浮点数, 可以通过 PC 事先做好文件, 然后用 U_READ 指令读取。 4 系列以下建议使用 USB2.0 的 U 盘, 4 系列或以上支持 USB3.0。
语法	<pre>U_WRITE2 sect_num, start_num [,varname] [,arrayname] [,arrayname(a)] [,arrayname(a,length)]</pre> sect_num: 文件编号, 对应到 SD【filenum】.BIN start_num: 文件内读取的起始位置 varname: 变量名 arrayname: 数组名, 可以为 TABLE, VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	带 U 盘接口
例子	<pre>IF U_STATE = TRUE THEN 'U 盘已插入 U_WRITE2 0, TABLE(0, 10) 'TBAL0-10 的数据写入 U 盘文件 SD0</pre>

	ENDIF
相关指令	U_READ , U_STATE

STICK_READ -- U 盘读取到 table

类型	存储指令						
描述	拷贝外部存储器的数据到 TABLE 里面。 value=TRUE 表示操作成功，否则操作失败。 建议使用 U_READ。 不具备外部存储接口的控制器不支持此命令。 U 盘读取文件系统支持 fat32 和 fat16，不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。						
语法	value = STICK_READ (filenum, table_start [,format]) filenum: 文件号，对应到 SD【filenum】 table_start: 开始操作的起始 TABLE 号 format: 写入文件的格式 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>float 格式存储, .BIN</td></tr> <tr> <td>1</td><td>文本格式存储, .CSV</td></tr> </table>	值	描述	0 (缺省)	float 格式存储, .BIN	1	文本格式存储, .CSV
值	描述						
0 (缺省)	float 格式存储, .BIN						
1	文本格式存储, .CSV						
适用控制器	带 U 盘接口						
例子	STICK_READ (0,10,0)'拷贝外部存储器名称为 SD0 的 bin 文件数据到TABLE, 从TABLE(10)开始保存						
相关指令	U_READ , STICK_READVR						

STICK_WRITE -- table 输出到 U 盘

类型	存储指令						
描述	拷贝 TABLE 的数据到外部存储器里面。 value=TRUE 表示操作成功，否则操作失败。 建议使用 U_WRITE。 不具备外部存储接口的控制器不支持此命令。 4 系列以下建议使用 USB2.0 的 U 盘，4 系列或以上支持 USB3.0。						
语法	value = STICK_WRITE(filenum, table_start [,length [,format]]) filenum: 文件号，对应到 SD【filenum】，最大 9999 table_start: 开始操作的起始 TABLE 号 length: 要操作的 TABLE 元素个数，缺省 128 format: 写入文件的格式 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>0 (缺省)</td><td>float 格式存储, .BIN</td></tr> <tr> <td>1</td><td>文本格式存储, .CSV</td></tr> </table>	值	描述	0 (缺省)	float 格式存储, .BIN	1	文本格式存储, .CSV
值	描述						
0 (缺省)	float 格式存储, .BIN						
1	文本格式存储, .CSV						

适用控制器	带 U 盘接口
例子	STICK_WRITE (0,0,128,0) '拷贝 table 前 128 个元素以 float 格式到外部存储器, 产生 SD0.BIN 文件
相关指令	U_WRITE , STICK_WRITEVR

STICK_READVR -- U 盘读取到 vr

类型	存储指令						
描述	拷贝外部存储器的数据到 VR 里面。 value=TRUE 表示操作成功, 否则操作失败。 建议使用 U_READ。 不具备外部存储接口的控制器不支持此命令。 U 盘读取文件系统支持 fat32 和 fat16, 不支持 ntfs。 4 系列以下建议使用 USB2.0 的 U 盘, 4 系列或以上支持 USB3.0。						
语法	value = STICK_READVR (filenum, vr_start [,format]) filenum: 文件号, 对应到 SD【filenum】 vr_start: 开始操作的起始 VR 号 format: 文件的格式 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0 (缺省)</td><td>float 格式存储</td></tr> <tr> <td>1</td><td>文本格式存储</td></tr> </tbody> </table>	值	描述	0 (缺省)	float 格式存储	1	文本格式存储
值	描述						
0 (缺省)	float 格式存储						
1	文本格式存储						
适用控制器	带 U 盘接口						
例子	STICK_READVR (0,20,0) '拷贝外部存储器名为 SD0 的 bin 文件数据到 VR, 从 VR(20) 开始存储。						
相关指令	U_READ , STICK_READ						

STICK_WRITEVR -- vr 输出到 U 盘

类型	存储指令
描述	拷贝 VR 的数据到外部存储器里面。 value=TRUE 表示操作成功, 否则操作失败。 建议使用 U_WRITE。 不具备外部存储接口的控制器不支持此命令。 4 系列以下建议使用 USB2.0 的 U 盘, 4 系列或以上支持 USB3.0。

语法	value = STICK_READVR (filenum, vr_start [,format]) filenum: 文件号, 对应到 SD【filenum】 vr_start: 开始操作的起始 VR 号 length: 要操作的元素个数, 缺省 128 format: 文件的格式 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0 (缺省)</td><td>float 格式存储, .BIN</td></tr> <tr> <td>1</td><td>文本格式存储, .CSV</td></tr> </tbody> </table>	值	描述	0 (缺省)	float 格式存储, .BIN	1	文本格式存储, .CSV
值	描述						
0 (缺省)	float 格式存储, .BIN						
1	文本格式存储, .CSV						
适用控制器	带 U 盘接口						
例子	STICK_WRITEVR (0,0,128,0) 拷贝 VR 前 128 个元素以 float 格式存储到外部存储器, 生成 SD0.BIN 文件						
相关指令	U_WRITE , STICK_WRITE						

DISK_CONFIG -- 示教盒 U 盘功能

类型	操作指令
描述	在控制器端操作示教盒 U 盘。 ! 切换磁盘设置时, 会自动关闭原来磁盘上打开的所有文件。
语法	DISK_CONFIG disknum, "NONE" 关闭磁盘 DISK_CONFIG disknum, "DEFAULT" 还原缺省配置 DISK_CONFIG disknum, "LCD", iconnectport 映射为当前 LCD 上的磁盘, 必须 ZHD 的固件支持。 Disknum: 本控制器磁盘编号 0-A 盘, 1-B 盘 LCD: HMI 编号 Iconnectport: 使用互联 PORT 来通讯, 建议 21, 不与传统互联通道 20 冲突。?*PORT 可以查看 PORT 列表 ?*DISK 可查看当前的磁盘信息
适用控制器	5 系控制器, version_build 250518 以后固件, 其他 linux 控制器陆续支持

15.2 FLASH 相关指令

FLASH_WRITE -- flash 存储

类型	存储指令
描述	存储变量或者数组, 数组的单个或部分元素到内部 FLASH 里面, 掉电保存。 内部 FLASH 采用顺序存储的方式, 读取的顺序必须与存储时的顺序一致。 内部 FLASH 有存储次数限制, 不要随意循环操作。 注意在运动中不要操作 FLASH, 对运动执行会有影响。
语法	FLASH_WRITE sect_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a,length)] sect_num: flash 块编号, 不同类型不一样

	varname: 变量名 arrayname: 数组名, 可以为 TABLE, VR, MODBUS a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	通用
例子	例一 FLASH_WRITE 1, VAR, ARRAY1, ARRAY2(1) '把 VAR,ARRAY1,ARRAY2(1)的数据依次写入 flash 块 1 例二 TABLE(1)=123.456 FLASH_WRITE 1, TABLE(1) TABLE(1)=200 FLASH_READ 1, TABLE(1) ?TABLE(1) '打印结果 123.45600 例三 FLASH 存储是 float 精度, 对 32 位整数精度的数据, 要使用 2 个 MODBUS_REG 来实现 MODBUS_LONG 的存储 MODBUS_LONG(1)=123456 '使用 MODBUS_REG(1)和 MODBUS_REG(2)存储 FLASH_WRITE 1, MODBUS_REG(1,2) '从 MODBUS_REG(1)开始取两个元素 写入 FLASH 块, 等价于 FLASH_WRITE 1, MODBUS_REG(1), MODBUS_REG(2) MODBUS_LONG(1)=100 FLASH_READ 1, MODBUS_REG(1,2) ?MODBUS_REG(1) '打印结果-7616 ?MODBUS_REG(2) '打印结果 1 ?MODBUS_LONG(1) '打印结果 123456
相关指令	FLASHVR , FLASH_READ

FLASH_WRITEDBL -- flash 存储-double

类型	存储指令
描述	存储变量或者数组, 数组的单个或部分元素到内部 FLASH 里面, 掉电保存。 使用方法同 FLASH_WRITE, 区别是 FLASH_WRITE 存储类型是 float, 32 位, FLASH_WRITEDBL 存储的数据类型是 double, 64 位。 内部 FLASH 采用顺序存储的方式, 读取的顺序必须与存储时的顺序一致。 内部 FLASH 有存储次数限制, 不要随意循环操作。 注意在运动中不要操作 FLASH, 对运动执行会有影响。
语法	FLASH_WRITEDBL sect_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a,length)] sect_num: flash 块编号, 不同类型不一样 varname: 变量名 arrayname: 数组名, 可以为 TABLE, VR, MODBUS

	a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	4 系列及以上控制器，20190128 及以上固件支持
例子	FLASH_WRITEDBL 1,table(0,4) '从 table(0)-table(4)，共 5 个元素 写入 FLASH 块 1
相关指令	FLASHVR ， FLASH_READ

FLASH_WRITE2 -- flash 存储 2

类型	存储指令
描述	存储变量或者数组，数组的单个或部分元素到内部 FLASH 里面，掉电保存。 内部 FLASH 采用顺序存储的方式，读取的顺序必须与存储时的顺序一致。 内部 FLASH 有存储次数限制，不要随意循环操作。 注意在运动中不要操作 FLASH，对运动执行会有影响。 ！注意 1：数据必须先完整写入一遍。 ！注意 2：起始位置可以使用 SIZEOFARRAY 来计算。
语法	FLASH_WRITE2 sect_num start_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a,length)] sect_num: flash 块编号，不同类型不一样 start_num: 文件内读取的起始位置 varname: 变量名 arrayname: 数组名，可以为 TABLE，VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	通用
例子	FLASH_WRITE2 1, TABLE(0,100) '写入 flash FLASH_WRITE2 1, 10, TABLE(10,10)' 中间部分数据写入 flash
相关指令	FLASHVR ， FLASH_READ

FLASH_READ -- flash 读取

类型	存储指令
描述	从内部 FLASH 读取数据到变量，或数组里面。 内部 FLASH 采用顺序存储的方式，读取的顺序必须与存储时的顺序一致。 读取未被写入过的 Flash 块时，会提示 Warn file:"BASIC1.BAS" line:5 task:0, File:C:\SD10.BIN open error, not load.，不影响使用。 注意在运动中不要操作 FLASH，对运动执行会有影响。 Linux 控制器区分 bin 文件后缀名大小写，使用该指令需确保后缀名为大写形式，如 SD1.BIN，否则读取会报错。ZMC 系列控制器则无大小写区分。

语法	FLASH_READ sect_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a,length)] sect_num: flash 块编号, 不同类型不一样 varname: 变量名 arrayname: 数组名, 可以为 TABLE, VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	通用
例子	FLASH_READ 1, VAR, ARRAY1, ARRAY2(1) '读取 FLASH 块 1 的数据, 依次保存到变量 VAR, 数组 ARRAY1, ARRAY2(1)
相关指令	FLASH_READDBL , FLASHVR , FLASH_WRITE

FLASH_READDBL -- flash 读取-double

类型	存储指令
描述	从内部 FLASH 读取数据到变量, 或数组里面。 使用方法同 FLASH_READ, 区别是 FLASH_READ 存储类型是 float, 32 位, FLASH_READDBL 存储的数据类型是 double, 64 位。 内部 FLASH 采用顺序存储的方式, 读取的顺序必须与存储时的顺序一致。 读取未被写入过的 Flash 块时, 会提示 Warn file:"BASIC1.BAS" line:5 task:0, File:C:\SD10.BIN open error, not load., 不影响使用。 注意在运动中不要操作 FLASH, 对运动执行会有影响。
语法	FLASH_READDBL sect_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a,length)] sect_num: flash 块编号, 不同类型不一样 varname: 变量名 arrayname: 数组名, 可以为 TABLE, VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	4 系列及以上控制器, 20190128 及以上固件支持
例子	FLASH_READDBL 1, table(6,5) '读 FLASH 块 1 数据, 位置 6 开始, 读取 5 个, 放入 table(6)-table(10), 共 5 个元素
相关指令	FLASH_READ , FLASHVR , FLASH_WRITE

FLASH_READ2 -- flash 读取 2

类型	存储指令
描述	从内部 FLASH 指定位置读取数据到变量, 或数组里面。 内部 FLASH 采用顺序存储的方式, 读取的顺序必须与存储时的顺序一致。 读取未被写入过的 Flash 块时, 会提示 Warn file:"BASIC1.BAS" line:5 task:0, File:C:\SD10.BIN open error, not load., 不影响使用。

	注意在运动中不要操作 FLASH ，对运动执行会有影响。
语法	FLASH_READ2 sect_num start_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a,length)] sect_num: flash 块编号，不同类型不一样 start_num: 文件内读取的起始位置 varname: 变量名 arrayname: 数组名，可以为 TABLE, VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	通用
例子	<pre> FOR i=0 TO 10 TABLE(i)=120+i NEXT FLASH_WRITE 1, TABLE(0,10) '数据写入 flash FOR i=0 TO 11 TABLE(i)=0 NEXT FLASH_READ2 1,2, TABLE(4,5) '从 table(2)开始依次读出数据，读 5 个数据依次放到 table(4)-table(8) FOR i=0 TO 11 ?"TABLE",i,TABLE(i) NEXT END </pre>
相关指令	FLASH_READ , FLASH_WRITE

FLASH_READ2DBL -- flash 读取 2-double

类型	存储指令
描述	从内部 FLASH 指定位置读取数据到变量，或数组里面。 使用方法同 FLASH_READ2DBL，区别是 FLASH_READ 存储类型是 float，32 位，FLASH_READ2DBL 存储的数据类型是 double，64 位。 内部 FLASH 采用顺序存储的方式，读取的顺序必须与存储时的顺序一致。 读取未被写入过的 Flash 块时，会提示 Warn file:"BASIC1.BAS" line:5 task:0, File:C:\SD10.BIN open error, not load.，不影响使用。 注意在运动中不要操作 FLASH，对运动执行会有影响。
语法	FLASH_READ2DBL sect_num start_num [, varname] [, arrayname] [, arrayname(a)] [, arrayname(a,length)] sect_num: flash 块编号，不同类型不一样

	start_num: 文件内读取的起始位置 varname: 变量名 arrayname: 数组名, 可以为 TABLE, VR a: 操作的数组索引 length: 操作的数组元素个数
适用控制器	4 系列及以上控制器, 20190128 及以上固件支持
例子	<pre>FOR i=0 TO 10 TABLE(i)=120+i NEXT FLASH_WRITEDBL 1, TABLE(0,10) '数据写入 flash FOR i=0 TO 11 TABLE(i)=0 NEXT FLASH_READ2 DBL 1,2, TABLE(4,5) '从 table(2)开始依次读出数据, 读 5 个数据依次放到 table(4)-table(8) FOR i=0 TO 11 ?"TABLE",i, TABLE(i) NEXT END</pre>
相关指令	FLASH_READDBL , FLASH_READ2 , FLASH_WRITE

FLASHVR -- 拷贝 RAM 的数据

类型	存储指令						
描述	拷贝 RAM 的数据到 FLASH 里面。 ZMC00x (除了 005)、ECI 全系列控制器把 TABLE 数据存储到了 FLASH 的最后一个块, 用 FLASH_WRITE, FLASH_READ 指令也可以操作到这个块, 要避免冲突。其他系列控制器把 TABLE 数据存储到另一个独立的存储区域, 无法操作。注意在运动中不要操作 FLASH, 对运动执行会有影响。						
语法	FLASHVR (function) function: 功能选择 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>-1</td><td>整个 TABLE 都存储到 FLASH 并且上电时自动读取到 TABLE</td></tr> <tr> <td>-2</td><td>取消 FLASH 上电时读取到 TABLE</td></tr> </tbody> </table>	值	描述	-1	整个 TABLE 都存储到 FLASH 并且上电时自动读取到 TABLE	-2	取消 FLASH 上电时读取到 TABLE
值	描述						
-1	整个 TABLE 都存储到 FLASH 并且上电时自动读取到 TABLE						
-2	取消 FLASH 上电时读取到 TABLE						
适用控制器	通用						
例子	FLASHVR (-1) 'table 存到 flash 块, 重新上电后再从 flash 读回 table						
相关指令	FLASH_WRITE						

FLASH_SECTSIZE -- flash 变量数

类型	系统状态函数
描述	读取内部 FLASH 一个块可以存储的变量个数。 不同控制器个数不同。
语法	value = FLASH_SECTSIZE
适用控制器	通用
例子	? FLASH_SECTSIZE 打印结果 20480 'ZMC1xx 系列 1024 'ZMC00x 系列 ...
相关指令	FLASH_SECTES

FLASH_SECTES -- flash 块数

类型	系统状态函数
描述	读取控制器内部 FLASH 的总块数。 不同控制器块数不同。 对 ZMC00x 系列控制器，FLASHVR 会使用用最后一个块，要避免冲突。
语法	value = FLASH_SECTES
适用控制器	通用
例子	?FLASH_SECTES '打印出 flash 块数 打印结果 128 'ZMC005
相关指令	FLASH_SECTSIZE

15.3 文件存储相关指令

FILE_ZOPEN -- 打开文件

类型	存储指令
描述	根据 mode 参数指定的模式，打开一个文件，返回文件句柄。
语法	函数语法：handle = FILE_ZOPEN (path, mode) path: 文件路径，字符串，flash 盘符定义为 c，U 盘盘符定义为 a，如 c:\filename.txt 表示 flash 中的 filename.txt 文件。缺省为 flash 中的文件。 5 系列和 7 系列用户想自定义盘符可以通过“FILE "PATHD" "dir"”指令来映射 mode: 打开模式，字符串“w”表示以写的方式打开（以全部删除再写的方式）， “r”表示以读的方式打开，“rw”表示可读可写。

	注意：“w”模式会删掉文件原来写入的数据。
适用控制器	4 系列产品 20170601 以上版本支持
例子	例一 <pre>FILE "copy_to","test.z3p","test1.z3p" 'U 盘文件拷贝到 FLASH f1 = FILE_ZOPEN("c:\test1.z3p","r") '打开 FLASH 中的 test1.z3p 文件 IF f1 >= 0 THEN ?"以只读方式打开 FLASH 存在文件" ELSE ?"error 打开失败" END IF FILE_ZCLOSE(f1)</pre> 例二 <pre>f1 = FILE_ZOPEN("c:\test.z3p","rw") num = FILE_ZWRITES(f1,"0123456789") FILE_ZSEEK(f1,0,0) FILE_ZREAD(f1,30,10) IF num = 10 AND TABLE(31) = 49 AND TABLE(39) = 57 THEN ?"指令读写 FLASH 文件成功" ELSE ?"error 指令读写 FLASH 文件失败" END IF FILE_ZCLOSE(f1)</pre>
相关指令	FILE_ZCLOSE

FILE_ZCLOSE -- 关闭文件

类型	存储指令
描述	关闭文件句柄 handle 指示的文件，如果句柄无效则输出警告。
语法	命令语法：FILE_ZCLOSE (handle) handle: 函数 FILE_ZOPEN 返回的文件句柄
适用控制器	4 系列产品 20170601 以上版本支持
例子	<pre>FILE "copy_to","test.z3p","test1.z3p" 'U 盘文件拷贝到 FLASH f1 = FILE_ZOPEN("c:\test1.z3p","r") '打开 FLASH 中的 test1.z3p 文件 IF f1 >= 0 THEN ?"以只读方式打开 FLASH 存在文件" ELSE ?"error 打开失败" END IF FILE_ZCLOSE(f1)</pre>
相关指令	FILE_ZOPEN

FILE_ZWRITES -- 文件写入字符串

类型	存储指令
描述	向文件句柄 handle 指示的文件中写入字符串 string ，函数返回写入的字符数量。
语法	命令语法：FILE_ZWRITES (handle, string) 函数语法：num = FILE_ZWRITES (handle, string) handle : 函数 FILE_ZOPEN 返回的文件句柄 string : 要写入的普通字符串
适用控制器	4 系列产品 20170601 以上版本支持
例子	<pre>FILE "copy_to","test.z3p","test.z3p" FOR i = 0 TO 9 TABLE(30+i) = 0 NEXT f1 = FILE_ZOPEN("c:\test.z3p","rw") num = FILE_ZWRITES(f1,"0123456789") FILE_ZSEEK(f1,0,0) FILE_ZREAD(f1,30,10) IF num = 10 AND TABLE(31) = 49 AND TABLE(39) = 57 THEN ?"指令读写 FLASH 文件成功" ELSE ?"error 指令读写 FLASH 文件失败" END IF FILE_ZCLOSE(f1)</pre>
相关指令	FILE_ZOPEN ， FILE_ZCLOSE ， FILE_ZSEEK ， FILE_ZREAD

FILE_ZWRITE -- 文件写入字符

类型	存储指令
描述	向文件句柄 handle 指示的文件中写入 count 个字符，字符位于 table 中索引 tableindex 开始的位置，函数返回实际写入的数量。字符写入数量有最大限制，超出则按照最大限制写入。
语法	命令语法：FILE_ZWRITE (handle, tableindex, count) 函数语法：num = FILE_ZWRITE (handle, tableindex, count) handle : 函数 FILE_ZOPEN 返回的文件句柄 tableindex : table 的索引，要写入的字符存放在 tableindex 开始的 table 中， table 每个位置存放一个字符 count : 要写入的字符数量
适用控制器	4 系列产品 20170601 以上版本支持
例子	<pre>FILE "copy_to","test.z3p","test.z3p" FILE "copy_from","test.z3p","test1.z3p" f1 = FILE_ZOPEN("a:\test1.z3p","rw")</pre>

	FILE_ZWRITE(fl,100,512)
相关指令	FILE_ZOPEN , FILE_ZCLOSE

FILE_ZREAD -- 文件读取字符

类型	存储指令
描述	从文件句柄 handle 指示的文件中读取 maxchars 个字符，读取字符放到 table 中索引 tableindex 开始的位置。 如果读取到文件结束符，则提前终止，函数返回读取的字符数。读取的数量有最大限制，超出则按照最大限制读取。
语法	命令语法：FILE_ZREAD (handle, tableindex, maxchars) 函数语法：num = FILE_ZREAD (handle, tableindex, maxchars) handle: 函数 FILE_ZOPEN 返回的文件句柄 tableindex: table 的索引，读取的字符存放到 tableindex 开始的 table 中，table 每个位置存放一个字符 maxchars: 预定读取的最大字符数量，如果读取遇到文件结束，则数量可能不足，可通过返回值获取读取数量
适用控制器	4 系列产品 20170601 以上版本支持
例子	参见 FILE_ZWRITES 例程
相关指令	FILE_ZOPEN , FILE_ZCLOSE , FILE_ZSEEK , FILE_ZWRITES

FILE_ZREADLINE -- 文件行读取

类型	存储指令
描述	从文件句柄 handle 指示的文件中读取一行到 table 的 tableindex 位置，最多读取 maxchars 个字符。 超出部分将不会读出，文件的读取位置将保持在第一个未读字符处。读取从文件当前的读取位置开始，读取结果不包含结尾的换行符，但文件读取位置将跳过换行符，函数返回读取的字符数量。读取的数量有最大限制，超出则按照最大限制读取。
语法	命令语法：FILE_ZREADLINE (handle, tableindex, maxchars) 函数语法：num = FILE_ZREADLINE (handle, tableindex, maxchars) handle: 函数 FILE_ZOPEN 返回的文件句柄 tableindex: table 的索引，读取的字符存放到 tableindex 开始的 table 中，table 每个位置存放一个字符 maxchars: table 最大能够存放的字符数量，如果读取行的字符数量超出 maxchars，多余的部分将不会读出，文件的读取位置将保持在第一个未读字符处
适用控制器	4 系列产品 20170601 以上版本支持
例子	<pre>DIM handle, num handle = FILE_ZOPEN("test.txt", "r") FILE_ZSEEKLINE(handle, 0, 2) '最后一行行首 num = FILE_ZREADLINE(handle, 0, 1000) '读完后定位到行尾，行号不变 num = FILE_ZREADLINE(handle, 0, 1000) '已到行尾，返回 0</pre>

	<pre>?num, table(0) '0 0 ?FILE_ZTELLLINE(handle) '以上读取行号不变 FILE_ZSEEKLINE(handle, -1, 2) '倒数第二行行首 num = FILE_ZREADLINE(handle, 0, 1000) '读取倒数第二行，定位到倒数第一行行首 FILE_ZSEEKLINE(handle, -1, 1) '上一行行首，此处为倒数第二行行首 FILE_ZSEEKLINE(handle, 17, 0) '定位到行号 17（0 开始），即第 18 行行首 FILE_ZSEEKLINE(handle, 0, 1) '移动到本行行首 num = FILE_ZREADLINE(handle, 0, 1000) FILE_ZCLOSE(handle)</pre>
相关指令	FILE_ZSEEKLINE , FILE_ZTELLLINE

FILE_ZSEEK -- 文件定位

类型	存储指令
描述	文件定位，将文件的读写位置移动到 pos 和 mode 指定的位置处。
语法	命令语法：FILE_ZSEEK (handle, pos, mode) handle: 函数 FILE_ZOPEN 返回的文件句柄 pos: 要定位的位置或偏移，根据 mode 的不同，可以取负数 mode: 执行定位功能时的基准位置，可取值 0、1、2。 mode 详细说明： 0: 代表文件头为基准，pos 指示需要定位的位置，执行后文件读写位置将定位到 pos 处，此种情况下 pos 取负值将作为 0 处理即定位到文件开头； 1: 代表当前位置为基准，pos 指示定位的偏移，执行后文件读写位置将相对当前位置移动 pos，正值表示向文件尾移动，负值表示向文件头移动； 2: 代表文件结尾为基准，pos 指示定位的偏移，执行后文件读写位置将相对文件尾向文件头部移动-pos，此种情况 pos 有效取值要小于等于 0，取正值将作为 0 处理，即定位到文件结尾。
适用控制器	4 系列产品 20170601 以上版本支持
例子	参见 FILE_ZWRITES 例程
相关指令	FILE_ZOPEN , FILE_ZCLOSE , FILE_ZREAD , FILE_ZWRITES

FILE_ZSEEKLINE -- 文件行定位

类型	存储指令
描述	文件行定位，将文件的读写位置移动到 line 和 mode 指定的行的行首，一次定位能够处理的字符数量有限，定位的偏移量过大或长行数量较多可能导致定位未完成，可通过 FILE_ZTELLLINE 获取当前行位置进行判断，必要时可通过多次执行来定位。
语法	命令语法：FILE_ZSEEKLINE (handle, line, mode) handle: 函数 FILE_ZOPEN 返回的文件句柄 line: 要定位行的行号或偏移，根据 mode 的不同，可以取负数 mode: 执行定位功能时的基准位置，可取值 0、1、2。

	mode 详细说明: 0: 代表文件头为基准, line 指示需要定位的行号, 执行后文件定位到 line 指示行的行首, 0 表示定位到第一行行首; 1: 代表当前行作为基准, line 指示定位的行偏移, 0 表示定位到当前行的行首, 负数表示向文件头方向进行定位; 2: 代表文件结尾为基准, line 指示定位的行偏移, 0 表示定位到最后一行的行首, 负数则依次向文件头方向进行定位。
适用控制器	4 系列产品 20170601 以上版本支持
例子	参见 FILE_ZREADLINE 例程
相关指令	FILE_ZREADLINE , FILE_ZTELLLINE

FILE_ZTELL -- 文件读写位置

类型	存储指令
描述	返回 handle 文件的当前读写位置, 从 0 开始。
语法	函数语法: <code>pos = FILE_ZTELL (handle)</code> handle : 函数 FILE_ZOPEN 返回的文件句柄
适用控制器	4 系列产品 20170601 以上版本支持
例子	<pre> handle = FILE_ZOPEN("test_w.txt", "r") '使用生成的文件 FILE_ZSEEK(handle, -1, 2) num = FILE_ZREAD(handle, 100, 5) ? num, TABLE(100), TABLE(101) '1 51 0 ?FILE_ZTELL(handle) '等于文件大小 FILE_ZCLOSE(handle) </pre>
相关指令	FILE_ZOPEN , FILE_ZCLOSE

FILE_ZTELLLINE -- 文件行号

类型	存储指令
描述	返回 handle 文件的当前行号, 行号从 0 开始, 只有行操作情况下才能获取行号, 非行操作将会导致行号无效, 返回值为-1。
语法	函数语法: <code>pos = FILE_ZTELLLINE (handle)</code> handle : 函数 FILE_ZOPEN 返回的文件句柄
适用控制器	4 系列产品 20170601 以上版本支持
例子	参见 FILE_ZREADLINE 例程
相关指令	FILE_ZREADLINE , FILE_ZSEEKLINE

第十六章 中断相关指令

16.1 三种中断指令

INT_ENABLE -- 中断总开关

类型	系统参数														
描述	中断总开关。 为了避免程序没有初始化好进入中断，中断开关缺省是关闭的。 控制器内部只有一个任务在处理所有的中断信号响应，有一个固定的中断任务号，如果中断处理函数过多，并且中断处理函数的代码太长，会造成所有的中断响应变慢，甚至是中断堵塞，影响其他中断执行。 解决办法： (1)尽量减少中断的数量，很多应用都可以用循环扫描来处理。 (2)如果有一个中断处理函数特别长的话，就调用一个单独的任务来处理中断中的复杂任务，这样就不会堵塞其他的中断响应。														
语法	INT_ENABLE = switch <table><tr><td>值</td><td>描述</td></tr><tr><td>0（缺省）</td><td>关闭</td></tr><tr><td>1</td><td>打开</td></tr></table>	值	描述	0（缺省）	关闭	1	打开								
值	描述														
0（缺省）	关闭														
1	打开														
适用控制器	通用														
例子	错误示范，中断堵塞 如下图，定时器中断 0 开启，IN(0)为 0，导致中断函数堵塞在第 9 行，由程序无打印结果可得此时定时器中断 1 无法运行。 <pre>1 INT_ENABLE=1 '开启中断 2 TIMER_START(0,1000) '定时器0开启，1000ms后执行一次 3 TIMER_START(1,1100) '定时器1开启，1000ms后执行一次 4 END 5 6 GLOBAL SUB ONTIMER0()'中断处理函数 7 DELAY 1000'假设大量的堵塞性代码 8 WAIT UNTIL IN(0)<>0 9 ?"第一个中断" 10 ENDSUB 11 12 GLOBAL SUB ONTIMER1()'中断处理函数 13 ?"第一个中断" 14 ENDSUB</pre> 任务 <table><tr><th>任务</th><th>状态</th><th>文件和行号</th></tr><tr><td>0</td><td>Stopped</td><td>BASIC1.BAS,line:6</td></tr><tr><td>22</td><td>Running</td><td>BASIC1.BAS,line:8</td></tr></table> 中断程序 <table><tr><th>栈</th><th>过程</th><th>文件和行号</th></tr></table><table><tr><th>局部变量名</th><th>值</th></tr></table> 正确示范 若中断需要处理大量代码，在中断内创建一个任务来处理，如下图任务 3，执行下面程	任务	状态	文件和行号	0	Stopped	BASIC1.BAS,line:6	22	Running	BASIC1.BAS,line:8	栈	过程	文件和行号	局部变量名	值
任务	状态	文件和行号													
0	Stopped	BASIC1.BAS,line:6													
22	Running	BASIC1.BAS,line:8													
栈	过程	文件和行号													
局部变量名	值														

序，打印出“第二个中断”，定时器中断 0 堵塞，定时器中断 1 的运行不受影响。

```
1 INT_ENABLE=1 '开启中断
2 TIMER_START(0,1000)
  '定时器0开启,1000ms后执行一次
3 TIMER_START(1,1100)
  '定时器1开启,1000ms后执行一次
4 END
5
6 GLOBAL SUB ONTIMER0() '中断处理函数
7 '创建一个新任务来处理自己的复杂任务
8 '就不会堵塞其他中断的响应速度
9
10 RUNTASK 3,MyIntHandler()
11 END SUB
12
13 GLOBAL SUB MyIntHandler()
14 DELAY 1000 '假设大量的堵塞性代码
15 WAIT UNTIL IN(0) <> 0
16 '?"第一个中断"
17 END SUB
18
19 GLOBAL SUB ONTIMER1() '中断处理函数
20 '?"第二个中断"
21 END SUB
```

任务	状态	文件和行号
0	Stopped	BASIC1.BAS,line:6
3	Running	BASIC1.BAS,line:15
22	Stopped	BASIC1.BAS,line:21

栈	过程	文件和行号
---	----	-------

局部变量名	值
-------	---

对应代码如下：

```
INT_ENABLE=1          '开启中断
TIMER_START(0,1000)    '定时器 0 开启，1000ms 后执行一次
TIMER_START(1,1100)    '定时器 1 开启，1100ms 后执行一次
END

GLOBAL SUB ONTIMER0() '中断处理函数
'创建一个新任务来处理自己的复杂任务，就不会堵塞其他中断的响应速度
RUNTASK 3, MyIntHandler()
END SUB

GLOBAL SUB MyIntHandler()
  DELAY 1000 '假设大量的堵塞性代码
  WAIT UNTIL IN(0) <> 0
  '?"第一个中断"
END SUB

GLOBAL SUB ONTIMER1() '中断处理函数
  '?"第二个中断"
END SUB
```

相关指令

[ONPOWEROFF](#)，[ONTIMERn](#)

ONPOWEROFF -- 掉电中断 SUB

类型	中断
描述	掉电中断程序入口，必须是全局的 SUB 过程。 控制器只有 1 个掉电中断。 掉电中断执行的时间特别有限，只能写少数几条语句，不能写 FLASH。
语法	GLOBAL ONPOWEROFF() ... END SUB
适用控制器	通用
例子	INT_ENABLE = 1

	<pre> dpos(0)=vr(0) '上电读取保存的数值，恢复坐标 dpos(1)=vr(1) dpos(2)=vr(2) END GLOBAL SUB ONPOWEROFF () vr(0) = dpos(0) '保存坐标 vr(1) = dpos(1) vr(2) = dpos(2) END SUB </pre>
相关指令	INT_ENABLE

INT_ONn -- 外部输入中断 SUB

类型	中断
描述	外部输入中断程序入口，上升沿触发，必须是全局的 SUB 过程。 必须支持 PLC 功能的固件才可使用。 中断 IN 口 0~31，4 系列 IN 口 0~63
语法	<pre> GLOBAL SUB INT_ONn() n 是 IN 编号 ... END SUB </pre>
适用控制器	支持 PLC 功能的固件
例子	<pre> INT_ENABLE=1 '开启中断 END GLOBAL SUB INT_ON0 () '中断程序 PRINT "输入 IN0 上升沿触发" END SUB </pre>
相关指令	INT_OFFn , INT_ENABLE

INT_OFFn -- 外部输入中断 SUB

类型	中断
描述	外部输入中断程序入口，下降沿触发，必须是全局的 SUB 过程。 必须支持 PLC 功能的固件才可使用。 中断 IN 口 0~31，4 系列 IN 口 0~63
语法	<pre> GLOBAL SUB INT_OFFn() n 是 IN 编号 ... END SUB </pre>
适用控制器	支持 PLC 功能的固件
例子	<pre> INT_ENABLE=1 '开启中断 END GLOBAL SUB INT_OFF0 () '中断程序 PRINT "输入 IN0 下降沿触发" </pre>

	END SUB
相关指令	INT_ONn , INT_ENABLE

ONTIMERn -- 定时器中断 SUB

类型	中断
描述	定时器中断程序入口，必须是全局的 SUB 过程。 不同的控制器型号定时器的个数不同，通过 RTSys 软件连接控制器，在线命令发送 “?*max” 查看 max_timer。 <pre>max_task:22 max_timer:1024 max_loopnest:8 max_callstack:10</pre> 在线命令: ?*max
语法	<pre>GLOBAL SUB ONTIMERn() 'n 是定时器编号 ... END SUB</pre>
适用控制器	通用
例子	<pre>INT_ENABLE=1 '开启中断 TIMER_START(0,100) '定时器 0 开启，100ms 后执行一次 END GLOBAL SUB ONTIMER0() '中断程序 PRINT "ontimer0 enter" 'TIMER_START(0,100) '希望周期执行的话，要在 sub 里再打开 END SUB</pre>
相关指令	INT_ENABLE , TIMER_START

INT_CYCLE -- 中断周期执行

类型	系统参数
描述	中断周期执行 BASIC 功能，每个 SERVO_PERIOD 执行一次。 4 系列以上，20170630 以上版本开放。
语法	命令语法: INT_CYCLE(function, taskid [, subname]) 参数描述: - function: 1-启动，2-停止 - taskid: 使用的 BASIC 任务号，BASIC 本身不能使用 - subname: 周期执行的 SUB 名称，程序必须足够精简 函数语法: var = INT_CYCLE(function, taskid) 参数描述: - function:

	3 -返回状态：1-使能，0-停止 4 -返回时间，上次的执行时间 us 5 -返回时间，执行最长时间 us 6 -返回时间最长限制 us 7 -返回错误情况的错误码，当 BASIC 中断函数执行错误的情况下，错误会设置 8 -返回错误行号 taskid: 使用的 BASIC 任务号
适用控制器	通用
例子	<pre> DIM times INT_CYCLE(1,1,intisr) END GLOBAL SUB intisr() times=times+1 MOVE_PT(1,1) '每周期运行 END SUB </pre>
相关指令	INT_ENABLE

16.2 定时器指令

TIMER_IFEND -- 定时器状态

类型	系统函数
描述	返回定时器是否结束。
语法	<pre>value = TIMER_IFEND (timernum)</pre> 返回 0: 定时器正在定时，即未执行对应中断程序。 返回 1: 定时器定时完成，开始执行对应中断程序。 支持 PLC 的固件在启动定时器前打印 0，不支持 PLC 的固件打印 1。
适用控制器	通用
例子	<pre> INT_ENABLE=1 '开启中断 ?TIMER_IFEND(0) '定时器未启动，打印结果 0 '不支持 PLC 的固件则打印 1 TIMER_START(0,2000) '定时器 0 启动，定时 2s ?TIMER_IFEND(0) '打印结果，0，定时器正在定时 DELAY(2000) ?TIMER_IFEND(0) '打印结果，1，定时器定时完成，执行中断 </pre>
相关指令	ONTIMERn , TIMER_START

TIMER_START -- 启动定时器

类型	系统指令
描述	启动系统定时器，定时器只执行 1 次。
语法	TIMER_START(timernum, time_ms) timernum: 定时器编号, 0-定时器个数减 1 time_ms: 定时器长度, 单位毫秒 time 100 及以上为累积性计时器。
适用控制器	通用
例子	参考 TIMER_IFEND 例程
相关指令	ONTIMERn , TIMER_IFEND

TIMER_COUNT -- 定时器累计时间

类型	系统函数
描述	读取定时器的累计时间，掉电保持，需手动清零。
语法	value = TIMER_COUNT (timernum) timernum: 定时器编号, 0-定时器个数减 1
适用控制器	通用
例子	<pre> INT_ENABLE=1 '开启中断 TIMER_COUNT(0)=0 '清零 TIMER_START(0,2000) '定时器 0 启动，定时 2s DELAY(2000) ?TIMER_COUNT(0) '打印累计时间：2000 TIMER_STOP(0) '停止计时器 0 END </pre>
相关指令	ONTIMERn , TIMER_START

TIMER_STOP -- 停止定时器

类型	系统指令
描述	强制停止系统定时器。
语法	TIMER_STOP (timernum) timernum: 编号, 0-定时器个数减 1
适用控制器	通用
例子	<pre> INT_ENABLE=1 '开启中断 TIMER_START(0,2000) '定时器 0 启动，周期 2s ?TIMER_IFEND(0) '打印结果，0，未运行 DELAY(2000) ?TIMER_IFEND(0) '打印定时器 0 状态，打印结果 1，已运行 </pre>

	TIMER_STOP(0)	'停止计时器 0
	?TIMER_IFEND(0)	'打印结果，0，未运行
相关指令	ONTIMERn , TIMER_IFEND	

第十七章 总线相关指令

17.1 编号释义

槽位号

槽位号是指控制器上接口的编号，总线接口缺省为 0。当控制器上有多个总线接口，?*SLOT 查看。
在指令说明中，用 SLOT 代表槽位号。
运动控制器支持单总线时，槽位号为 0；支持双总线时，EtherCAT 总线槽位号为 0，RTEX 总线槽位号为 1。

>>?*slot	>>?*slot
Slot:0-ETHERCAT.	Slot:0-ETHERCAT.
	Slot:1-RTEX.

设备号

设备号是指一个槽位上连接的所有设备的编号，从 0 开始，按连接顺序依次增加，可以通过 NODE_COUNT(slot)查看。
在指令说明中，用 node 代表设备号。

驱动器编号

控制器会自动识别出槽位上的驱动器，编号从 0 开始，按连接顺序依次增加。
驱动器编号与设备号不同，假设控制器连接了 3 个设备，驱动器前面连接了两个其他 IO 设备，那么驱动器此时的设备号 node=2，而驱动器编号为 0。

17.2 基础指令

SLOT_SCAN -- 总线扫描

类型	总线指令
描述	总线扫描 通过 RETURN 返回成功与否， 返回-1 扫描成功，0 扫描失败。 Rtex 扫描失败会直接报错 6209。 遇到不支持的设备类型，RETURN 会返回 0。 Rtex 控制器未连接设备时会报错，EtherCAT 控制器不会。

	扫描后可以使用 NODE 指令读取相关设备信息，并使用 DRIVE 相关指令配置。
语法	SLOT_SCAN (slot) slot: 控制器 EtherCAT 槽位号或 RTEX 槽位号，0-缺省
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> aa: '标记 aa SLOT_SCAN(0) '总线扫描 ? RETURN '打印返回值，-1 成功，0 失败 IF RETURN THEN ?NODE_COUNT(0) '扫描成功，返回当前连接的设备个数 ELSE ?"扫描失败" DELAY (1000) '等待 1s GOTO aa '扫描失败，跳转到 aa，重新扫描 ENDIF </pre>
相关指令	SLOT_START , SLOT_STOP

SLOT_START -- 总线开启

类型	总线指令
描述	总线启动。 通过 RETURN 返回成功与否。返回-1 启动成功，0 启动失败。 需在总线扫描 SLOT_SCAN 成功后再执行。 执行前，先设置好 AXIS_ADDRESS, ATYPE, DRIVE_PROFILE
语法	SLOT_START (slot [,opstate]) slot: 控制器 EtherCAT 槽位号或 RTEX 槽位号，0-缺省 opstate: 启动到的 EtherCAT 状态，4-SAFEOP, 8- OP(缺省) NODE_PDOBUFF 需要启动以后才能修改，可以先启动到 SAFEOP，然后设置初始化 PDO 状态，再启动到 OP。4 系列产品 20170515 以上版本增加。
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> aa: '标记 aa SLOT_SCAN(0) '总线扫描 IF RETURN THEN '扫描成功，进入轴设置 AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=65 '轴类型 65，位置控制 DRIVE_PROFILE(0)=0 'PDO 周期扫描设置 SLOT_START(0) '开启总线 ELSE ?"扫描失败" DELAY (1000) '等待 1s GOTO aa '扫描失败，跳转到 aa，重新扫描 ENDIF </pre>
相关指令	SLOT_STOP , SLOT_SCAN , NODE_PDOBUFF

SLOT_STOP -- 总线停止

类型	总线指令
描述	总线停止。 通过 RETURN 返回成功与否。返回-1 停止成功，0 停止失败。 停止总线，轴使能会掉。
语法	SLOT_STOP (slot) slot: 控制器 EtherCAT 槽位号或 RTEX 槽位号，0-缺省
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> aa: '标记 aa SLOT_SCAN(0) '总线扫描 IF RETURN THEN '扫描成功，进入轴设置 AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=65 '轴类型 65，位置控制 DRIVE_PROFILE(0)=0 'PDO 周期扫描设置 SLOT_START(0) '开启总线 WHILE 1 IF SCAN_EVENT(IN(0))>0 THEN '输入 IN 口上升沿触发 SLOT_STOP(0) '停止总线 ENDIF WEND ELSE ?"扫描失败" DELAY (1000) '等待 1s GOTO aa '扫描失败，跳转到 aa，重新扫描 ENDIF </pre>
相关指令	SLOT_START , SLOT_SCAN

SLOT_INFO -- 总线信息读取

类型	EtherCAT 总线指令						
描述	总线的信息读取。 必须总线扫描后才能读取。						
语法	只读: var = SLOT_INFO (slot, sel) slot: 槽位号，0-缺省 sel: 信息编号 <table border="1" data-bbox="504 1666 1286 2040"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td> 槽位类型: SLOT_TYPE_NULL = 0, // 无效 SLOT_TYPE_ECAT = 1, // ECAT SLOT_TYPE_RTEX = 4, // RTEX SLOT_TYPE_HLINK = 5, // 华为 HLINK </td></tr> <tr> <td>4</td><td> AL 状态读取 1--Init--初始化状态 2--pre-operational--预运行状态 </td></tr> </tbody> </table>	值	描述	0	槽位类型: SLOT_TYPE_NULL = 0, // 无效 SLOT_TYPE_ECAT = 1, // ECAT SLOT_TYPE_RTEX = 4, // RTEX SLOT_TYPE_HLINK = 5, // 华为 HLINK	4	AL 状态读取 1--Init--初始化状态 2--pre-operational--预运行状态
值	描述						
0	槽位类型: SLOT_TYPE_NULL = 0, // 无效 SLOT_TYPE_ECAT = 1, // ECAT SLOT_TYPE_RTEX = 4, // RTEX SLOT_TYPE_HLINK = 5, // 华为 HLINK						
4	AL 状态读取 1--Init--初始化状态 2--pre-operational--预运行状态						

			4--safe-operational--安全运行状态 8--operational--运行状态（使能）	
	5		读取是否检测到断线，冗余模式使用，0529 增加 0--线路正常 1--检测到短线	
	14		发送 PDO 的字节数	
	15		接收 PDO 的字节数	
	16		总设备数，必须扫描后读取	
	17		总电机个数，必须扫描后读取	
适用控制器	VERSION_BUILD: 240529 以上版本.			
相关指令	SLOT_SLAVE			

?*SLOT -- 打印总线接口

类型	EtherCAT 辅助指令
描述	查看控制器总线接口编号及类型。
语法	?*SLOT
适用控制器	带总线接口
例子	?*SLOT 打印结果 Slot:0-ETHERCAT '当前只有一个 EtherCAT 总线，编号为 0

?*ETHERCAT -- 打印 EtherCAT 总线状态

类型	EtherCAT 辅助指令
描述	调试时使用，可以显示每个 NODE 的重要状态。 总线扫描后才能显示设备状态。
语法	?*ETHERCAT
适用控制器	带 EtherCAT 接口
例子	?*ETHERCAT 打印结果： <pre>>>?*ethercat Slot:0 contain 2 nodes. dc:step-sensitive.Lostcount:2-0. Node:0 status:1 manid:66fh productid:60380005h axes:1 Alstate:8 Node_profile:0.</pre> Slot 0 contain 1 nodes: 0 槽位口共连接了 1 个设备 Lostcount 0-0: 没有应答次数-时钟冲突次数 Node: 设备连接 NODE 编号 Status: 设备连接状态，参考 NODE_STATUS Manid: 厂商 ID Productid: 设备 ID Axes: 设备总轴数 AL Status: 设备 OP 状态

	Node_profile: 设备 Profile 设置 Bindaxis: 映射到控制器轴号 Drive_profile: 设备收发 PDO 设置 Controlword: 控制字 Drive_status: 设备当前状态, 参考 DRIVE_STATUS Drive_mode: 设备控制模式 Target: 电机位置 Encoder: 编码器位置
相关指令	PRINT

ZTEST -- EtherCAT 总线信息查询

类型	EtherCAT 辅助指令
描述	调试时使用, 可以查看多种信息。
语法	<pre>ztest(30,10,nodeid) nodeid = 设备编号, 0--(n-1)</pre>
适用控制器	带 EtherCAT 接口
例子	例一 查询当前 PDO 与关键数据字典 <pre>ztest(30,10,nodeid) nodeid = 设备编号, 0--(n-1)</pre> <pre>>>ztest(30,10,0) TestDriver_ecat para1:10,para2:0! reg:1c12:0 value:0x1 reg:1c12:1 value:0x1600 reg:1600:0 value:0x3 reg:1600:1 value:0x60400010 reg:1600:2 value:0x607a0020 reg:1600:3 value:0x60600008 reg:1c13:0 value:0x1 reg:1c13:1 value:0x1a00 reg:1a00:0 value:0x2 reg:1a00:1 value:0x60410010 reg:1a00:2 value:0x60640020 reg:6040:0 value:0xf reg:6041:0 value:0x1237 reg:6060:0 value:0x8 reg:6061:0 value:0x8 reg:6064:0 value:0x10ac4 reg:607a:0 value:0x10ac4 reg:603f:0 value:0x0</pre> 例二 设备 AL 状态查询, 1-init, 2-preop, 4-safeop, 8-op 查询所有 ECAT 合并的 AL 状态。 <pre>>>ztest(30,1) TestDriver_ecat para1:1,para2:0! al:0x8 code:0x0. alctrl:0x8</pre> <pre>ztest(30,2,nodeid) nodeid = 设备编号, 0---(n-1)</pre>

单个的 AL 状态查询。

```
>>ztest(30,2,0)
TestDriver_ecat para1:2,para2:0!
al:0x8 code:0x0.
alctrl:0x8
```

例三 丢包查询

```
>>ztest(30,12)
TestDriver_ecat para1:12,para2:0!
Slot:0 contain 1 nodes.
Lostcount:0-0.
第一个数据: 没有应答次数
第二个数据: 时钟冲突次数
```

例四 查询是否支持指定设备:

ztest (30,20,厂商 ID, 产品 ID, 版本号)

```
>>ztest(30,20,$41b,0,11)
Id:0x41b ProductCode:0x0 version:0xb support.
>>ztest(30,20,$41b,145,11)
Id:0x41b ProductCode:0x91 version:0xb not support.
```

例五: PDO 长度查询

```
>>ztest(30,13)
打印所有设备的 pdo 字节数;
方式 2: 2310 以后版本支持.
NODE_INFO(islot, inode, 15) 查询发送 PDO 字节数
NODE_INFO(islot, inode, 16) 查询接收 PDO 字节数.
```

version_build:231124 以后支持:

```
>>ztest(30,17)
打印发送 PDO 的所有内容.
>>ztest(30,18)
打印接收 PDO 的所有内容.
```

例六: 多 slot 查询

```
>>ztest(32,13) ‘第 2 个 ECAT
>>ztest(33,13) ‘第 3 个 ECAT
>>ztest(34,13) ‘第 4 个 ECAT
```

例七: 连续丢包认为掉线次数

ztest(30,80, 0, times) 设置连续丢包个数
可以同时查看当前的设置。

20170612.以上固件支持;

例八: 驱动器从端丢包个数查看

旧版本可以通过这个指令查看:

```
ztest(30,0, nodeid, $300)
nodeid 0- 最大个数-1
```

例九: 从端寄存器查看

```
ztest (30, 0, nodeid, regnum)
```

	nodeid 0- 最大个数-1 打印 regnum 开始的连续 8 个 reg 例十：收发包测试 扫描设备 0 的时候使用判断链路是否正常 ZTEST(60,7,0,1) ZTEST(60,8,0,1) 以上两个都可以，每发一次网口数据灯闪烁一次 例十一：实时性检查 ZTEST(61,0)，监控前复位 ZTEST(61,1)，运行一段时间后打印 TestDriver_int para1:1,para2:0! inttest_fastenterticks:9307 inttest_slowestticks:34321 小于扫描周期 1/2 判断实时性较好 inttest_maxprocessticks:143584 小于扫描周期 1/2 判断实时性较好 inttest_curprocessticks:111792 例十二：中断检查（7 系可用） m_pciisrtimes、m_isrimes 值需要一致 >>ZTEST(60,97) m_pciisrtimes 0x:3956106 m_isrimes 0x:3956106 m_isrfirstclkmin -4987 m_isrfirstclkmax 4930 m_isrfirstclkminisr 72 m_isrfirstclkmaxisr 3125 m_isrfirstclkCur 186 m_isrLostIsres 0 m_isrLostIsres2 6413553 m_isrOverIsres 0 closeisrns 0 m_iTimerPeriod 10000 m_iCpuTimerPeriod 10000 m_iTimerPeriodns 1000000 m_iTimerPeriodAdjustMax 10000 m_iTimerPeriodAdjustMin 10000
相关指令	PRINT

?*RTEX -- 打印 Rtex 总线状态

类型	Rtex 辅助指令
描述	调试时使用，可以显示每个 NODE 的重要状态。 总线启动后才能显示设备状态。
语法	?*RTEX
适用控制器	带 RTEX 接口
例子	?*RTEX

	打印结果: <pre>>>?*RTEX Slot:1 contain 1 nodes. Lostcount:0-0. Node:0 status:3 man:Panasonic devicetype:3lh axes:1 Alstate:1. BindAxis:-1 Drive_profile:0 Controlword:0h drive_status:0h target:0h encode:0h.</pre> Slot 0 contain 1 nodes: 0 槽位口共连接了 1 个设备 Lostcount 0-0: 丢包数 Node: 设备连接 NODE 编号 Status: 设备连接状态, 参考 NODE_STATUS Mainid: 厂商 ID Productid: 设备 ID Axes: 设备总轴数 AL Status: 设备 OP 状态 Node_profile: 设备 Profile 设置 Bindaxis: 映射到控制器轴号 Drive_profile: 设备收发 PDO 设置 Controlword: 控制字 Drive_status: 设备当前状态, 参考 DRIVE_STATUS Drive_mode: 设备控制模式 Target: 电机位置 Encoder: 编码器位置
相关指令	PRINT

?*ZML -- 打印 ZML 信息

类型	EtherCAT 总线指令
描述	查看当前工程加入的 ZML 设备列表, 以及被使用的情况。 ZML 文件是 XML 的压缩文件, 用于扩展控制器对特定设备的支持。 20221021 以后的 ECAT 控制器固件支持。
语法	?*ZML
适用控制器	带 EtherCAT 接口
例子	<pre>>>?*zml</pre> 打印结果: <pre>Vender:41bh id:1ab0h ver:1lh used:1.</pre>
相关指令	PRINT , ZML_INFO

17.3 SDO 操作指令

SDO_WRITE -- 数据字典写入

类型	总线指令, 仅 EtherCAT 可用
描述	通过设备号和槽位号进行 SDO 写入。 通过 RETURN 返回成功与否, -1 写入成功, 0 写入失败。

	需连接好设备，扫描总线后才能执行。 只有可写的字典才能写入。 SDO 不要频繁读写。																		
语法	SDO_WRITE (slot, node, index, subindex ,type ,value) slot: 槽位号 0-缺省 node: 设备编号 从 0 开始 index: 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 subindex: 子编号，用一个两位十六进制数表示，如\$01，不同的子编号对应不同的功能 type: 数据类型 <table border="1"> <tr><td>1</td><td>boolean</td></tr> <tr><td>2</td><td>integer 8</td></tr> <tr><td>3</td><td>integer 16</td></tr> <tr><td>4</td><td>integer 32</td></tr> <tr><td>5</td><td>unsigned 8</td></tr> <tr><td>6</td><td>unsigned 16</td></tr> <tr><td>7</td><td>unsigned 32</td></tr> <tr><td>8</td><td>float</td></tr> <tr><td>9</td><td>string</td></tr> </table> value: 数据值，需在数据类型对应的范围内取值，如 type=2，范围为-128 到 127；type=5，范围为 0 到 255，value=255 时表示 8 位二进制 11111111；type=9，value 值为 table 寄存器地址，将所需要写入的值写在 table 寄存器中	1	boolean	2	integer 8	3	integer 16	4	integer 32	5	unsigned 8	6	unsigned 16	7	unsigned 32	8	float	9	string
1	boolean																		
2	integer 8																		
3	integer 16																		
4	integer 32																		
5	unsigned 8																		
6	unsigned 16																		
7	unsigned 32																		
8	float																		
9	string																		
适用控制器	带 EtherCAT 接口																		
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN SDO_WRITE(0,0,\$6060,0,2,8) '0 号设备设置控制模式为 8，位置控制 SDO_WRITE(0,0,\$6200,\$01,5,255) 'EIO 总线扩展模块 OUT0~OUT7 全打开 SDO_WRITE(0,0,\$6200,\$01,5,0) 'EIO 总线扩展模块 OUT0~OUT7 全关闭 SDO_WRITE(0,0,\$6200,\$02,5,255) 'EIO 总线扩展模块 OUT8~OUT15 全打开 ENDIF </pre>																		
相关指令	SDO_WRITE_AXIS , SDO_READ																		

SDO_WRITE_AXIS -- 数据字典写入

类型	总线指令，仅 EtherCAT 可用		
描述	通过轴号 SDO 写入。 通过 RETURN 返回成功与否，-1 写入成功，0 写入失败。 需连接好设备，扫描总线后才能执行。 只有可写的字典才能写入。		
语法	SDO_WRITE_AXIS(axis, index, subindex ,type ,value) axis: 轴号 index: 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 subindex: 子编号 type: 数据类型 <table border="1"> <tr><td>1</td><td>boolean</td></tr> </table>	1	boolean
1	boolean		

	<table> <tr><td>2</td><td>integer 8</td></tr> <tr><td>3</td><td>integer 16</td></tr> <tr><td>4</td><td>integer 32</td></tr> <tr><td>5</td><td>unsigned 8</td></tr> <tr><td>6</td><td>unsigned 16</td></tr> <tr><td>7</td><td>unsigned 32</td></tr> <tr><td>8</td><td>float</td></tr> <tr><td>9</td><td>string</td></tr> </table> value: 数据值	2	integer 8	3	integer 16	4	integer 32	5	unsigned 8	6	unsigned 16	7	unsigned 32	8	float	9	string
2	integer 8																
3	integer 16																
4	integer 32																
5	unsigned 8																
6	unsigned 16																
7	unsigned 32																
8	float																
9	string																
适用控制器	带 EtherCAT 接口																
例子	正确的连接了一个 EtherCAT 轴设备再使用例程 <pre> SLOT_SCAN(0) '总线扫描 IF NODE_COUNT(0)>0 THEN AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=65 '轴类型 65，位置控制 DRIVE_PROFILE(0)=0 'PDO 周期扫描设置 SDO_WRITE_AXIS(0,\$6060,0,2,8) '轴 0 设备设置控制模式为 8，位置控制 ENDIF </pre>																
相关指令	SDO_WRITE , SDO_READ_AXIS																

SDO_READ -- 数据字典读取

类型	总线指令，仅 EtherCAT 可用																		
描述	通过设备号和槽位号进行 SDO 读取。 通过 RETURN 返回成功与否，-1 写入成功，0 写入失败 需连接好设备，扫描总线后才能执行。 只有可读的数据字典才能读取。 SDO 不要频繁读写。																		
语法	SDO_READ (slot, node, index, subindex ,type, tablenum) slot: 槽位号 0-缺省 node: 设备编号，从 0 开始 index: 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 subindex: 子编号，用一个两位十六进制数表示，如\$01，不同的子编号对应不同的功能 type: 数据类型 <table> <tr><td>1</td><td>boolean</td></tr> <tr><td>2</td><td>integer 8</td></tr> <tr><td>3</td><td>integer 16</td></tr> <tr><td>4</td><td>integer 32</td></tr> <tr><td>5</td><td>unsigned 8</td></tr> <tr><td>6</td><td>unsigned 16</td></tr> <tr><td>7</td><td>unsigned 32</td></tr> <tr><td>8</td><td>float</td></tr> <tr><td>9</td><td>string</td></tr> </table> tablenum: 读取的数据存储的 TABLE 位置 在线命令直接输入该指令，tablenum 为-1，不存储，直接打印	1	boolean	2	integer 8	3	integer 16	4	integer 32	5	unsigned 8	6	unsigned 16	7	unsigned 32	8	float	9	string
1	boolean																		
2	integer 8																		
3	integer 16																		
4	integer 32																		
5	unsigned 8																		
6	unsigned 16																		
7	unsigned 32																		
8	float																		
9	string																		
适用控制器	带 EtherCAT 接口																		

例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN SDO_READ (0,0,\$6061,0,2,0) '读取 0 号设备的控制模式，数据存到 table(0) ?table(0) '打印出数据 SDO_READ (0,0,\$6000,\$01,5,0) '读取 EIO 总线扩展模块 IN0~IN7 的状态，结果存 到 table(0) SDO_READ (0,0,\$6000,\$02,5,-1) '读取 EIO 总线扩展模块 IN8~IN15 的状态，结果 自动打印出来 ENDIF </pre>
相关指令	SDO_READ_AXIS , SDO_WRITE

SDO_READ_AXIS -- 数据字典读取

类型	总线指令，仅 EtherCAT 可用																		
描述	通过轴号进行 SDO 读取。 通过 RETURN 返回成功与否，-1 写入成功，0 写入失败 需连接好设备，扫描总线后才能执行。 只有可读的数据字典才能读取。																		
语法	SDO_READ_AXIS(axis, index, subindex ,type, tablenum) axis: 轴号 index: 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 subindex: 子编号 type: 数据类型 <table border="1"> <tr><td>1</td><td>boolean</td></tr> <tr><td>2</td><td>integer 8</td></tr> <tr><td>3</td><td>integer 16</td></tr> <tr><td>4</td><td>integer 32</td></tr> <tr><td>5</td><td>unsigned 8</td></tr> <tr><td>6</td><td>unsigned 16</td></tr> <tr><td>7</td><td>unsigned 32</td></tr> <tr><td>8</td><td>float</td></tr> <tr><td>9</td><td>string</td></tr> </table> tablenum: 读取的数据存储的 TABLE 位置	1	boolean	2	integer 8	3	integer 16	4	integer 32	5	unsigned 8	6	unsigned 16	7	unsigned 32	8	float	9	string
1	boolean																		
2	integer 8																		
3	integer 16																		
4	integer 32																		
5	unsigned 8																		
6	unsigned 16																		
7	unsigned 32																		
8	float																		
9	string																		
适用控制器	带 EtherCAT 接口																		
例子	正确的连接了一个 EtherCAT 轴设备再使用例程 <pre> SLOT_SCAN(0) '总线扫描 IF NODE_COUNT(0)>0 THEN AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=65 '轴类型 65，位置控制 DRIVE_PROFILE(0)=0 'PDO 周期扫描设置 SDO_WRITE_AXIS(0,\$6060,0,2,8)'轴 0 设备设置控制模式为 8，位置控制 SDO_READ_AXIS (0,\$6061,0,2,0) '读取 0 号设备的控制模式，数据存到 table(0) ?table(0) '打印出数据 ENDIF </pre>																		
相关指令	SDO_READ , SDO_WRITE_AXIS																		

SDO_READ2 -- 数据字典读取至 MODBUS 寄存器

类型	总线指令，仅 EtherCAT 可用																		
描述	通过设备号和槽位号进行 SDO 读取。 通过 RETURN 返回成功与否，-1 写入成功，0 写入失败 需连接好设备，扫描总线后才能执行。 只有可读的数据字典才能读取。																		
语法	SDO_READ2 (slot, node, index, subindex, type, modbusnum) slot: 槽位号 0-缺省 node: 设备编号 0- index: 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 subindex: 子编号 type: 数据类型 <table border="1"> <tr><td>1</td><td>boolean</td></tr> <tr><td>2</td><td>integer 8</td></tr> <tr><td>3</td><td>integer 16</td></tr> <tr><td>4</td><td>integer 32</td></tr> <tr><td>5</td><td>unsigned 8</td></tr> <tr><td>6</td><td>unsigned 16</td></tr> <tr><td>7</td><td>unsigned 32</td></tr> <tr><td>8</td><td>float</td></tr> <tr><td>9</td><td>string</td></tr> </table> modbusnum: 读取的数据存储的 MODBUS 位置 在线命令直接输入该指令,modbusnum 为-1，不存储，直接打印 注：此语法也可以避免 MODBUS 读取方式转换浮点型后造成精度丢失问题。	1	boolean	2	integer 8	3	integer 16	4	integer 32	5	unsigned 8	6	unsigned 16	7	unsigned 32	8	float	9	string
1	boolean																		
2	integer 8																		
3	integer 16																		
4	integer 32																		
5	unsigned 8																		
6	unsigned 16																		
7	unsigned 32																		
8	float																		
9	string																		
适用控制器	带 EtherCAT 接口																		
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN SDO_READ (0,0,\$6060,0,2,0) '读取 0 号设备的控制模式，数据存到 modbus_long(0) 寄存器 ?modbus_long(0) '打印出数据 ENDIF </pre>																		
相关指令	SDO_READ_AXIS , SDO_WRITE																		

17.4 设备相关指令

NODE_COUNT -- 设备个数

类型	总线指令
描述	通过总线连接的设备总个数。 总线扫描后可用。
语法	只读: var = NODE_COUNT (slot) slot: 槽位号，0-缺省

	可以直接打印出来，见例一 可以直接作为数据，见例二
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	例一 SLOT_SCAN(0) ?NODE_COUNT(0) '打印出 0 槽口连接的设备数 例二 SLOT_SCAN(0) IF NODE_COUNT(0)=3 THEN '指定连接了多少设备，才进行下一步轴映射、轴类型设置等程序代码块 ENDIF
相关指令	NODE_INFO

NODE_STATUS -- 设备状态

类型	总线指令								
描述	设备状态，总线扫描后可用 <table border="1" data-bbox="448 943 1069 1093"> <thead> <tr> <th>BIT 位</th><th>意义</th></tr> </thead> <tbody> <tr> <td>0</td><td>指明 node 是否存在；1- 存在</td></tr> <tr> <td>1</td><td>通讯状态；1- 出错；</td></tr> <tr> <td>2</td><td>节点状态；1- 出错；</td></tr> </tbody> </table> 值为 1 时，bit0 为 1，bit1 和 bit2 为 0，设备通讯正常 值为 3 时，bit0 和 bit1 为 1，bit2 为 0，设备通讯出错	BIT 位	意义	0	指明 node 是否存在；1- 存在	1	通讯状态；1- 出错；	2	节点状态；1- 出错；
BIT 位	意义								
0	指明 node 是否存在；1- 存在								
1	通讯状态；1- 出错；								
2	节点状态；1- 出错；								
语法	只读：var = NODE_STATUS (slot, node) slot: 槽位号，0-缺省 node: 设备编号，编号从 0 开始								
适用控制器	带 EtherCAT 接口或 RTEX 接口								
例子	SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ?NODE_STATUS(0,0) '读取设备 0 的状态，打印值 1，通讯正常 ENDIF								
相关指令	NODE_INFO								

NODE_AXIS_COUNT -- 设备电机数

类型	总线指令
描述	每个设备带电机个数读取。 必须总线扫描后才能读取。
语法	只读：var = NODE_AXIS_COUNT (slot, node) slot: 槽位号，0-缺省 node: 设备编号，编号从 0 开始
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN

	? NODE_AXIS_COUNT (0,0)	'打印出设备 0 带电机个数
	ENDIF	
相关指令	NODE_INFO	

NODE_IO -- 设备 IO

类型	EtherCAT 总线指令	
描述	设备的 IO 起始编号设置，单个设备的输入输出的起始编号一样。 设置结果只会是 8 的倍数。 必须总线扫描后才能读取。 一般用于 EIO 扩展板的 IO 设置，其他设备含有 IO 口时也可使用。	
语法	可读：var = NODE_IO (slot, node) 可写：NODE_IO(slot, node)=iobase slot: 槽位号，0-缺省 node: 设备编号，编号从 0 开始	
适用控制器	带 EtherCAT 接口或 RTEX 接口	
例子	SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN NODE_IO(0,0)=32 '设置设备 0 的 IO 起始编号为 32 ?NODE_IO(0,0) '打印出设备 0 的 IO 起始编号 ENDIF SLOT_START(0)	
相关指令	NODE_AIO	

NODE_AIO -- 设备模拟量

类型	总线指令							
描述	设备的 AIO 起始编号设置，单个设备的输入输出的起始编号一样。 必须总线扫描后才能读取。 一般用于 ZMIO 扩展子模块的 AIO 设置，其他设备含有 AIO 口时也可使用。							
语法	可读：var = NODE_AIO (slot, node[,idir]) 可写：NODE_AIO(slot, node[,idir])=Aiobase slot: 槽位号，0-缺省 node: 设备编号，编号从 0 开始 idir: AD/DA 选择 <table><tr><td>0</td><td>缺省，同时设置 AIN、AOUT；读取时只读 AIN</td></tr><tr><td>3</td><td>AIN</td></tr><tr><td>4</td><td>AOUT</td></tr></table>		0	缺省，同时设置 AIN、AOUT；读取时只读 AIN	3	AIN	4	AOUT
0	缺省，同时设置 AIN、AOUT；读取时只读 AIN							
3	AIN							
4	AOUT							
适用控制器	带 EtherCAT 接口或 RTEX 接口							
例子	SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN NODE_AIO(0,0,3)=3 '设置设备 0 的 AIN 起始编号为 3 ?NODE_AIO(0,0,3) '打印出设备 0 的 AIN 起始编号 ENDIF							

	SLOT_START(0)
相关指令	NODE_IO

NODE_INFO -- 设备信息

类型	EtherCAT 总线指令																																																												
描述	总线设备的信息读取。 必须总线扫描后才能读取。																																																												
语法	读取: var = NODE_INFO (slot, node, sel, [, moduid]) slot: 槽位号, 0-缺省 node: 设备编号, 编号从 0 开始 sel: 信息编号 moduid: 可选参数, 子模块信息查询时使用, 参数值为 n-1, n 为子模块数量 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr><td>0</td><td>VENDER, 厂商编号</td></tr> <tr><td>1</td><td>DEVICE, 设备编号</td></tr> <tr><td>2</td><td>VERSION, 版本</td></tr> <tr><td>3</td><td>ALIAS, 别名, 一般用来识别驱动器</td></tr> <tr><td>4</td><td>预留</td></tr> <tr><td>5</td><td>断线可能, 按 BIT 位, 240531 增加</td></tr> <tr><td>6</td><td>网口状态, 按 BIT 位, 240531 增加</td></tr> <tr><td>10</td><td>IN 个数</td></tr> <tr><td>11</td><td>OP 个数</td></tr> <tr><td>12</td><td>AIN 个数</td></tr> <tr><td>13</td><td>AOUT 个数</td></tr> <tr><td>14</td><td>发送 PDO 字节数, 230823 增加</td></tr> <tr><td>15</td><td>接收 PDO 字节数, 230823 增加</td></tr> <tr><td>16</td><td>子模块个数</td></tr> <tr><td>17</td><td>子模块类型查询, 需要 moduid</td></tr> <tr><td>18</td><td>扫描的设备时钟滞后(ns)</td></tr> <tr><td>19</td><td>Ecat 的 sync 偏移(ns)</td></tr> <tr><td></td><td>20-27, 241203 以后增加:需要 moduid 20-23 只支持带 zml 设备. 参数错误返回-1.</td></tr> <tr><td>20</td><td>插拔子模块 IN 个数</td></tr> <tr><td>21</td><td>插拔子模块 OP 个数</td></tr> <tr><td>22</td><td>插拔子模块 AIN 个数</td></tr> <tr><td>23</td><td>插拔子模块 AOUT 个数</td></tr> <tr><td>24</td><td>子模块发送 PDO 字节数</td></tr> <tr><td>25</td><td>子模块接收 PDO 字节数</td></tr> <tr><td>26</td><td>子模块发送 PDO 起始偏移</td></tr> <tr><td>27</td><td>子模块接收 PDO 起始偏移</td></tr> <tr><td></td><td></td></tr> <tr><td>30</td><td>设备特殊参数</td></tr> <tr><td>0x300-0x307</td><td>Esc 的丢包信息</td></tr> </tbody> </table> 工作途中若通过 SDO 写配置量程时, 需要重新 SLOT_SCAN(0)扫描设备, 对应	值	描述	0	VENDER, 厂商编号	1	DEVICE, 设备编号	2	VERSION, 版本	3	ALIAS, 别名, 一般用来识别驱动器	4	预留	5	断线可能, 按 BIT 位, 240531 增加	6	网口状态, 按 BIT 位, 240531 增加	10	IN 个数	11	OP 个数	12	AIN 个数	13	AOUT 个数	14	发送 PDO 字节数, 230823 增加	15	接收 PDO 字节数, 230823 增加	16	子模块个数	17	子模块类型查询, 需要 moduid	18	扫描的设备时钟滞后(ns)	19	Ecat 的 sync 偏移(ns)		20-27, 241203 以后增加:需要 moduid 20-23 只支持带 zml 设备. 参数错误返回-1.	20	插拔子模块 IN 个数	21	插拔子模块 OP 个数	22	插拔子模块 AIN 个数	23	插拔子模块 AOUT 个数	24	子模块发送 PDO 字节数	25	子模块接收 PDO 字节数	26	子模块发送 PDO 起始偏移	27	子模块接收 PDO 起始偏移			30	设备特殊参数	0x300-0x307	Esc 的丢包信息
值	描述																																																												
0	VENDER, 厂商编号																																																												
1	DEVICE, 设备编号																																																												
2	VERSION, 版本																																																												
3	ALIAS, 别名, 一般用来识别驱动器																																																												
4	预留																																																												
5	断线可能, 按 BIT 位, 240531 增加																																																												
6	网口状态, 按 BIT 位, 240531 增加																																																												
10	IN 个数																																																												
11	OP 个数																																																												
12	AIN 个数																																																												
13	AOUT 个数																																																												
14	发送 PDO 字节数, 230823 增加																																																												
15	接收 PDO 字节数, 230823 增加																																																												
16	子模块个数																																																												
17	子模块类型查询, 需要 moduid																																																												
18	扫描的设备时钟滞后(ns)																																																												
19	Ecat 的 sync 偏移(ns)																																																												
	20-27, 241203 以后增加:需要 moduid 20-23 只支持带 zml 设备. 参数错误返回-1.																																																												
20	插拔子模块 IN 个数																																																												
21	插拔子模块 OP 个数																																																												
22	插拔子模块 AIN 个数																																																												
23	插拔子模块 AOUT 个数																																																												
24	子模块发送 PDO 字节数																																																												
25	子模块接收 PDO 字节数																																																												
26	子模块发送 PDO 起始偏移																																																												
27	子模块接收 PDO 起始偏移																																																												
30	设备特殊参数																																																												
0x300-0x307	Esc 的丢包信息																																																												

	NODE_INFO(缺省,缺省,17,模块地址) 命令值才会被同步更新,不重新 SLOT_SCAN(0) 也不影响正常工作,可使用 SDO 读来替换 NODE_INFO 命令实时监测量程数据。 4 系列 fast 固件 20190201 以后提供写入模式。 写入: NODE_INFO (slot, node, sel)=value slot: 槽位号, 0-缺省 node: 设备编号, 编号从 0 开始 sel: 信息编号 <table border="1"> <tr> <th>值</th><th>描述</th></tr> <tr> <td>3</td><td>ALIAS, 别名, 一般用来识别驱动器, 驱动器使用 eeprom 存储的, 可以使用这个来修改</td></tr> <tr> <td>19</td><td>Ecat 的 sync 偏移(ns), 总线扫描后总线开启前可以修改</td></tr> </table>	值	描述	3	ALIAS, 别名, 一般用来识别驱动器, 驱动器使用 eeprom 存储的, 可以使用这个来修改	19	Ecat 的 sync 偏移(ns), 总线扫描后总线开启前可以修改
值	描述						
3	ALIAS, 别名, 一般用来识别驱动器, 驱动器使用 eeprom 存储的, 可以使用这个来修改						
19	Ecat 的 sync 偏移(ns), 总线扫描后总线开启前可以修改						
适用控制器	带 EtherCAT 接口或 RTEXT 接口						
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ?NODE_INFO(0,0,10) '读取设备 0 的输入 IN 个数 ?NODE_INFO(0,0,0) '读取设备 0 的厂商编号 ?NODE_INFO(0,0,11) '读取设备 0 的输出 OP 个数 ?NODE_INFO(0,0,20,0) '读取插拔子模块 0 的输入 IN 个数 ENDIF </pre>						
相关指令	SLOT_SCAN						

NODE_PROFILE -- PDO 预设置

类型	总线指令
描述	总线设备的 profile 设置。 预留, 总线启动后不能再修改。 在 SLOT_START 前增加 NODE_PROFILE(slot, node)=-1 是开启控制器自定义 PDO 设置
语法	可读: var= NODE_PROFILE(slot,node) 可写: NODE_PROFILE(slot, node) = iprofile[, reserve]
适用控制器	带 EtherCAT 接口或 RTEXT 接口

NODE_PDOBUFF -- 特殊设备 PDO 设置

类型	总线指令
描述	特殊 EtherCAT 设备的 PDO 支持。 非轴和 IO 的设备, 通过这个指令来读写 PDO, 例如电源设备。 轴和 IO 类型的设备, 已经可以通过轴参数和 IO 指令来访问 PDO, 不能使用这个指令。 SLOT_START 会自动提前读取当前的 PDO 列表, 以及可写 PDO 的当前值。 可以在 SLOT_START 调用前通过 SDO 修改 PDO 列表, 或相关数据字典的当前值。

	需要启动以后才能修改，可以先用 SLOT_START 启动到 SAFEOP，然后设置初始化 PDO 状态，再启动到 OP。																		
语法	命令语法：NODE_PDOBUFF (slot, node, index, subindex ,type) 函数语法：Buff = NODE_PDOBUFF (slot, node, index, subindex ,type) slot: 槽位号 0-缺省 node: 设备编号 0- index: 数据字典编号，前面可加"\$"表示 16 进制，如\$6060 subindex: 子编号 type: 数据类型 <table border="1"> <tr><td>1</td><td>boolean</td></tr> <tr><td>2</td><td>integer 8</td></tr> <tr><td>3</td><td>integer 16</td></tr> <tr><td>4</td><td>integer 32</td></tr> <tr><td>5</td><td>unsigned 8</td></tr> <tr><td>6</td><td>unsigned 16</td></tr> <tr><td>7</td><td>unsigned 32</td></tr> <tr><td>8</td><td>float</td></tr> <tr><td>9</td><td>string</td></tr> </table>	1	boolean	2	integer 8	3	integer 16	4	integer 32	5	unsigned 8	6	unsigned 16	7	unsigned 32	8	float	9	string
1	boolean																		
2	integer 8																		
3	integer 16																		
4	integer 32																		
5	unsigned 8																		
6	unsigned 16																		
7	unsigned 32																		
8	float																		
9	string																		
适用控制器	带 EtherCAT 接口，4 系列产品，20170508 以上版本支持																		
例子	<pre>>>NODE_PDOBUFF(0,0,\$6040,0,3)=15</pre> <pre>>>?NODE_PDOBUFF(0,0,\$6041,0,3)</pre>																		
相关指令	SDO_WRITE , SDO_READ																		

NODE_PDO_WRBUFF -- 偏移修改 PDO

类型	总线指令
描述	按偏移修改 PDO 的内容。
语法	命令语法：NODE_PDO_WRBUFF(slot, node, offset, tableindex, size) slot: 槽位号 0-缺省 node: 设备编号 0- offset: PDO 字节偏移 tableindex: 数据存储的 TABLE 起始编号 size: 要写入的数据字节数
适用控制器	带 EtherCAT 接口，4 系列产品，20170515 以上版本支持
例子	>> NODE_PDO_WRBUFF(0, 0, offset, tableindex, size)
相关指令	NODE_PDO_RDBUFF

NODE_PDO_RDBUFF -- 偏移读取 PDO

类型	总线指令
描述	按偏移读取 PDO 的内容。
语法	命令语法：NODE_PDO_RDBUFF(slot, node, offset, tableindex, size) slot: 槽位号 0-缺省 node: 设备编号 0-

	offse: PDO 字节偏移 tableindex: 数据存储的 TABLE 起始编号 size: 要读取的数据字节数
适用控制器	带 EtherCAT 接口, 4 系列产品, 20170515 以上版本支持
例子	>> NODE_PDO_RDBUFF(0, 0, offset, tableindex, size)
相关指令	NODE_PDO_WRBUFF

NODE_REGWRITE -- ESC 寄存器写入

类型	总线指令, 仅 EtherCAT 可用
描述	通过设备号和槽位号进行 ESC 寄存器写入。 通过 RETURN 返回成功与否, -1 写入成功, 0 写入失败。 需连接好设备, 扫描总线后才能执行。 只有可写的地址才能写入。
语法	命令语法: NODE_REGWRITE (slot, node, address, bytes ,value) slot: 槽位号 0-缺省 node: 设备编号 0- address: 寄存器地址, 前面可加"\$"表示 16 进制, 如\$980 bytes: 数据长度, 1,2,4 value: 数据值
适用控制器	带 EtherCAT 接口, 220418 增加
例子	>> NODE_REGWRITE (0, 0, address, bytes ,value)
相关指令	NODE_REGREAD

NODE_REGREAD -- ESC 寄存器读取

类型	总线指令, 仅 EtherCAT 可用
描述	通过设备号和槽位号进行 ESC 寄存器读取。 通过 RETURN 返回成功与否, -1 读取成功, 0 读取失败。 需连接好设备, 扫描总线后才能执行。 只有可写的地址才能写入。
语法	命令语法: NODE_REGREAD (slot, node, address, bytes ,modbusindex) slot: 槽位号 0-缺省 node: 设备编号 0- address: 寄存器地址, 前面可加"\$"表示 16 进制, 如\$980 bytes: 数据长度, 1,2,4 modbusindex: 读取过来的数据存储的 MODBUS 寄存器编号,-1,不存储,直接打印
适用控制器	带 EtherCAT 接口, 220418 增加
例子	>> NODE_REGREAD (0, 0, address, bytes ,modbusindex)
相关指令	NODE_REGWRITE

NODE_PRESET -- 设备预配置

类型	EtherCAT 总线指令
描述	总线设备预先配置，配置以后没有挂上外设也提前启动总线。 总线启动后不能再修改。 预设置后，可以通过 NODE_STATUS 判断外设是否挂上。 预设值的类型与实际不一样，将不能启动总线。 中途连入的外设没有预设值，也没有扫描到，也不能启动总线。 固件 20160601 以上版本支持。
语法	命令语法 1: NODE_PRESET (slot, node, manuid, productid) 命令语法 2: NODE_PRESET (slot, -1) 清除所有预设置 函数语法 1: VALUE = NODE_PRESET (slot, node) 返回是否有预设置 函数语法 1: VALUE = NODE_PRESET (slot) 返回预设值的最大个数 slot: 槽位号, 0-缺省 node: 设备编号, 0- manuid: 厂商 ID 编号, 参考 NODE_INFO productid: 设备 ID 编号, 参考 NODE_INFO
适用控制器	带 EtherCAT 接口或 RTECH 接口
例子	NODE_PRESET(0,-1)清除原来的预设置 NODE_PRESET(0,0,\$83,5)设置第一个 NODE 为 OMRON 驱动器 SLOT_SCAN(0) ? "SCAN RESULT:",RETURN, "MAX", NODE_COUNT(0) '此时会自动显示总数为 1 FOR i= 0 TO NODE_COUNT(0) -1 ? "node", i ? "status",NODE_STATUS(0,i) ? "manu:",NODE_INFO(0,i,0) ? "dev:",NODE_INFO(0,i,1) ? "motor:", NODE_AXIS_COUNT(0,i) NEXT
相关指令	NODE_STATUS

ZML_INFO -- 设备 XML 查询

类型	EtherCAT 总线指令												
描述	EtherCAT 设备 XML 文件的部分内容查询。 ZML 文件是 XML 的压缩文件，用于扩展控制器对特定设备的支持。												
语法	函数语法: para = ZML_INFO(infosel, venderid, devid, [version,]) infosel: 操作编号 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>VENDER, 厂商编号</td></tr> <tr> <td>1</td><td>DEVICE, 设备编号</td></tr> <tr> <td>2</td><td>VERSION, 版本</td></tr> <tr> <td>10</td><td>设备固定 IN 个数</td></tr> <tr> <td>11</td><td>设备固定 OP 个数</td></tr> </tbody> </table>	值	描述	0	VENDER, 厂商编号	1	DEVICE, 设备编号	2	VERSION, 版本	10	设备固定 IN 个数	11	设备固定 OP 个数
值	描述												
0	VENDER, 厂商编号												
1	DEVICE, 设备编号												
2	VERSION, 版本												
10	设备固定 IN 个数												
11	设备固定 OP 个数												

	<table><tr><td>12</td><td>设备固定 AIN 个数</td></tr><tr><td>13</td><td>设备固定 AOUT 个数</td></tr><tr><td>19</td><td>Shift 时间设置, ns 单位</td></tr><tr><td>21</td><td>AL 状态切换 1-2 等待时间, ms 单位</td></tr><tr><td>22</td><td>AL 状态切换 2-4 等待时间, ms 单位</td></tr><tr><td>23</td><td>AL 状态切换 4-8 等待时间, ms 单位</td></tr></table> venderid: 厂商 ID devid: 设备 ID version: 版本号, 可选 21-22: 230927 以后版本增加支持. 函数语法: para = ZML_INFO(infosel, venderid, devid, [version,]) 命令语法: ZML_INFO(infosel, venderid, devid, [version,]) = para 暂时只支持 19 模式。请在总线扫描前修改。修改的是同步零点偏移量, 示意图如下: 注意: 内置 XML 可能多个设备共一个 XML 文件, 此时修改会多个设备同时变化, venderid、devid 可以查询 XML 文件, 也可以扫描后直接?*ETHERCAT 查询	12	设备固定 AIN 个数	13	设备固定 AOUT 个数	19	Shift 时间设置, ns 单位	21	AL 状态切换 1-2 等待时间, ms 单位	22	AL 状态切换 2-4 等待时间, ms 单位	23	AL 状态切换 4-8 等待时间, ms 单位
12	设备固定 AIN 个数												
13	设备固定 AOUT 个数												
19	Shift 时间设置, ns 单位												
21	AL 状态切换 1-2 等待时间, ms 单位												
22	AL 状态切换 2-4 等待时间, ms 单位												
23	AL 状态切换 4-8 等待时间, ms 单位												
适用控制器	带 EtherCAT 接口												
例子	<pre>>>?ZML_INFO(19, \$418108, \$9252) 打印: 0 Drive_Vender = NODE_INFO(Bus_Slot,i,0) '读取驱动器厂商 Drive_Device = NODE_INFO(Bus_Slot,i,1) '取设备编号 ZML_INFO(19, Drive_Vender, Drive_Device)=500000 '扫描前修改刷新周期 SYSTEM_ZSET = SYSTEM_ZSET OR 128 SLOT_SCAN(Bus_Slot) LOCAL LOCAL_zmlinfo, LOCAL_nodeinfo LOCAL_zmlinfo=ZML_INFO(19, Drive_Vender, Drive_Device) LOCAL_nodeinfo=NODE_INFO(0,0,19) IF LOCAL_zmlinfo = LOCAL_nodeinfo THEN ?"XML 写入成功" ELSE ?"XML 写入失败" ENDIF</pre>												
相关指令	SLOT_SCAN												

17.5 驱动器相关指令

DRIVE_MODE -- 驱动器模式

类型	轴参数
描述	驱动器的控制模式，对应数据字典 0x6060。 必须设置正确的 ATYPE（设置为 65/66/67）以后才能操作这个参数。
语法	可读：var=DRIVE_MODE (axis) 可写：DRIVE_MODE (axis)= value axis: 轴号
适用控制器	带 EtherCAT 接口或 RTECH 接口
例子	<pre> SLOT_SCAN(0) ... '轴使能过程，参考第十七章简易例程 IF NODE_COUNT(0)>0 THEN DRIVE_MODE(0)=8 '轴 0 设备设置为位置控制模式 ? DRIVE_MODE(0) '打印轴 0 的控制模式 ENDIF </pre>

DRIVE_PROFILE -- 驱动器 PDO 设置

类型	EtherCAT 轴参数
描述	每个轴的发送 pdo 接收 pdo 的配置选择。 必须设置正确的 ATYPE（设置为 65/66/67）以后才能操作这个参数。 详细配置请咨询厂家。 EtherCAT 总线 -1 表示使用驱动器的内置缺省 PDO 列表，20160601 以上版本支持，缺省 PDO 不带 0X6060 时，无法使用 datum(21)回零指令。 -1-驱动器默认设置，需要控制器版本 20160601 及以上 0-缺省配置，csp 位置模式 {0x60400010, 0x607a0020, 0x60600008}, //控制字 目标位置 模式 {0x60410010, 0x60640020}, //状态字 反馈位置 1-csp 位置模式+力矩反馈 {0x60400010, 0x607a0020, 0x60600008}, //控制字 目标位置 模式 {0x60410010, 0x60640020, 0x60770010}, //状态字 反馈位置 当前力矩 2-csp 位置模式+力矩反馈+锁存 lup {0x60400010, 0x607a0020, 0x60b80010, 0x60600008}, //控制字 目标位置 probe 设置 模式 {0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020}, //状态字 反馈位置 当前力矩 probe 状态 probe 位置

3-csp 位置模式+力矩限制+力矩反馈+锁存 1 上升沿

```
{0x60400010, 0x607a0020, 0x60b80010, 0x60720010, 0x60600008},
//控制字      目标位置      probe 设置      力矩限制      模式
{0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020},
//状态字      反馈位置      当前力矩      probe 状态      probe 位置
```

4-csp 位置模式+力矩反馈+驱动器 IO 输入

```
{0x60400010, 0x607a0020, 0x60600008},
//控制字      目标位置      模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020},
//状态字      反馈位置      当前力矩      驱动器 IO 输入
```

5-csp 位置模式+力矩反馈+驱动器 IO 输出+驱动器 IO 输入

```
{0x60400010, 0x607a0020, 0x60fe0120, 0x60600008},
//控制字      目标位置      IO 输出      模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020},
//状态字      反馈位置      当前力矩      驱动器 IO 输入
```

6-特殊驱动器专用

7-特殊驱动器专用

8-特殊驱动器专用

9-固件版本 160504 支持

```
{0x60400010, 0x607a0020, 0x60fe0120, 0x60b80010, 0x60720010, 0x60600008},
//控制字      目标位置      IO 输出(32 个)      probe 设置      力矩限制      模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020, 0x60b90010, 0x60ba0020},
//状态字      反馈位置      当前力矩      驱动器 IO 输入(32 个)      probe 状态      probe 位置
```

10-固件版本 160504 以上支持,加 drive_fe 部分

```
{0x60400010, 0x607a0020, 0x60fe0120, 0x60b80010, 0x60720010, 0x60600008},
//控制字      目标位置      IO 输出      probe 设置      力矩限制      模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020, 0x60b90010, 0x60ba0020, 0x60f40020},
//状态字      反馈位置      当前力矩      驱动器 IO 输入      probe 状态      probe 位置      drive_fe
```

11-固件版本 160504 以上支持,probe 专用测试

```
{0x60400010, 0x607a0020, 0x60b80010, 0x60600008},
//控制字      目标位置      probe 设置      模式
{0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020, 0x60bb0020, 0x60bc0020,
0x60bd0020},
//状态字      反馈位置      当前力矩      probe 状态      probe 位置 1/位置 2/位置 3/位置 4
```

12-固件版本 160504 以上支持,特殊驱动器专用

```
{0x60400010, 0x607a0020, 0x60600008},
//控制字      目标位置      模式
{0x60410010, 0x60640020, 0x60fd0020},
//状态字      反馈位置      驱动器 IO 输入
```

13-固件版本 160504 以上支持,带速度前馈与加速度前馈

```
{0x60400010, 0x60B20010, 0x607a0020, 0x60B10020, 0x60600008},
//控制字      加速度前馈      目标位置      速度前馈      模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020, 0x606c0020},
```

```

//状态字      反馈位置      当前力矩      驱动器 IO 输入      实际速度

17-固件版本 160504 以上支持, csp/csv/cst 三种模式可以切换
{0x60400010,0x60710010,0x60ff0020,0x607a0020,0x60b80010,0x60720010, x60600008},
//控制字      周期力矩      周期速度      目标位置      probe 设置      力矩限制      模式
{0x60410010,0x60770010,0x60640020,0x60fd0020,0x60b90010,0x60ba0020, x60bb0020},
//状态字      当前力矩      反馈位置      驱动器 IO 输入      probe 状态      probe 位置 1/位置 2/

18-固件版本 160504 以上支持,csp/csv/cst 三种模式可以切换+力矩反馈读取
{0x60400010,0x60710010,0x60ff0020,0x607a0020,0x60b80010,0x60720010,0x60600008},
//控制字      周期力矩      周期速度      目标位置      probe 设置      力矩限制      模式
{0x60410010,0x60770010,0x60640020,0x60fd0020,0x60b90010,0x60ba0020,0x60bb0020,
0x60bc0020, 0x60bd0020},
//状态字      当前力矩      反馈位置      驱动器 IO 输入      probe 状态      probe 位置 1/位置 2/
probe 位置 3/位置 4

20-固件版本 160504 以上支持,csp 位置+csv 速度
{0x60400010, 0x60ff0020, 0x607a0020, 0x60600008},
//控制字      目标速度      目标位置      模式
{0x60410010, 0x60640020},
//状态字      反馈位置

21-固件版本 160504 以上支持,csp 位置+csv 速度+力矩反馈
{0x60400010,0x60ff0020,0x607a0020,0x60600008},
//控制字      目标速度      目标位置      模式
{0x60410010,0x60640020,0x60770010},
//状态字      反馈位置      当前力矩

22-固件版本 160504 以上支持,csp 位置+csv 速度+力矩反馈+色标锁存 1 上升沿
{0x60400010, 0x60ff0020, 0x607a0020, 0x60b80010, 0x60600008},
//控制字      目标速度      目标位置      probe 设置      模式
{0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020},
//状态字      反馈位置      当前力矩      probe 状态      probe 位置

23-固件版本 160504 以上支持,csp 位置+csv 速度+力矩反馈+锁存 1 上升沿+力矩限制
{0x60400010,0x60ff0020,0x607a0020,0x60b80010,0x60720010,0x60600008},
//控制字      目标速度      目标位置      probe 设置      力矩限制      模式
{0x60410010,0x60640020, 0x60770010,0x60b90010,0x60ba0020},
//状态字      反馈位置      当前力矩      probe 状态      probe 位置

24-固件版本 160504 以上支持,csp 位置+csv 速度+IO 输入+位置+力矩反馈
{0x60400010, 0x60ff0020, 0x607a0020, 0x60600008},
//控制字      目标速度      目标位置      模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020},
//状态字      反馈位置      当前力矩      驱动器 IO 输入

25-固件版本 160504 以上支持,csp 位置+csv 速度+IO 输入+位置+力矩反馈
{0x60400010, 0x60ff0020, 0x607a0020, 0x60fe0120,0x60600008},
//控制字      目标速度      目标位置      驱动器 IO 输出      模式
{0x60410010, 0x60640020, 0x60770010, 0x60fd0020},
//状态字      反馈位置      当前力矩      驱动器 IO 输入

30-固件版本 160504 以上支持,csp 位置+cst 转矩
{0x60400010, 0x60710010, 0x607a0020, 0x60600008},

```

	<pre>//控制字 目标转矩 目标位置 模式 {0x60410010, 0x60640020}, //状态字 反馈位置 31-固件版本 160504 以上支持,csp 位置+cst 转矩+力矩反馈 {0x60400010,0x60710010,0x607a0020,0x60600008}, //控制字 目标力矩 目标位置 模式 {0x60410010,0x60640020,0x60770010}, //状态字 反馈位置 当前力矩 32-固件版本 160504 以上支持,csp 位置+cst 转矩+力矩反馈+锁存 1 上升沿 {0x60400010, 0x60710010, 0x607a0020, 0x60b80010 , 0x60600008}, //控制字 目标转矩 目标位置 probe 设置 模式 {0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020}, //状态字 反馈位置 当前力矩 probe 状态 probe 位置 33-固件版本 160504 以上支持,csp 位置+cst 转矩+力矩反馈+锁存 1 上升沿+ 力矩限制 {0x60400010,0x60710010,0x607a0020,0x60b80010,0x60720010,0x60600008}, //控制字 目标转矩 目标位置 probe 设置 力矩限制 模式 {0x60410010, 0x60640020, 0x60770010, 0x60b90010, 0x60ba0020}, //状态字 反馈位置 当前力矩 probe 状态 probe 位置 34-固件版本 160504 以上支持,csp 位置+cst 转矩+力矩反馈+驱动器 IO 输入 {0x60400010, 0x60710010, 0x607a0020, 0x60600008}, //控制字 目标转矩 目标位置 模式 {0x60410010, 0x60640020, 0x60770010, 0x60fd0020}, //状态字 反馈位置 当前力矩 驱动器 IO 输入 35-固件版本 160504 以上支持,csp 位置+cst 转矩+力矩反馈+IO 输入+IO 输出 {0x60400010, 0x60710010, 0x607a0020, 0x60fe0120,0x60600008}, //控制字 目标转矩 目标位置 驱动器 IO 输出 模式 {0x60410010, 0x60640020, 0x60770010, 0x60fd0020}, //状态字 反馈位置 当前力矩 驱动器 IO 输入 Rtex 总线 0- 不带 IO 映射 1- 带驱动器 IO 映射 带 IO 映射时, 按 DRIVE_IO 指令设置起始地址 不同的 drive_profile 模式, 映射驱动器的 IO 信息映射的位数不同, 有的模式只映射 16 位信息, 具体可以通过 DRIVE_IOCOUNT 指令去监控映射的 IO 位数。</pre>
语法	<pre>可读: var= DRIVE_PROFILE(axis) 可写: DRIVE_PROFILE(axis)= value axis: 轴号</pre>
适用控制器	带 EtherCAT 接口或 RTEXTX 接口
例子	<pre>SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN AXIS_ADDRESS(1)=1 ATYPE(1)=65 DRIVE_PROFILE(1)=-1 '轴 1 PDO 设置为-1, 设备默认值 ? DRIVE_PROFILE(1) '打印出轴 1 的 PDO 设置</pre>

	ENDIF
相关指令	ATYPE , NODE_PROFILE

DRIVE_CW_MODE -- 驱动器设置

类型	轴参数
描述	驱动器设置参数 必须设置正确的 ATYPE（设置为 65/66/67）以后才能操作这个参数。 部分版本为使用方便，可以直接操作 MODE。 RTEX 的控制字不要随便修改。 0-控制器自动调整 DRIVE_CONTROLWORD 参数，此时 DRIVE_CONTROLWORD 指令无效。 1- 可以手动调整，此时可以使用 DRIVE_CONTROLWORD 指令。
语法	可读：var= DRIVE_CW_MODE(axis) 可写： DRIVE_CW_MODE(axis)=value axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
相关指令	ATYPE , DRIVE_CONTROLWORD

DRIVE_CONTROLWORD -- 驱动器控制字

类型	轴参数																																																																									
描述	驱动控制字，按位操作 必须设置正确的 ATYPE（设置为 65/66/67）以后才能操作这个参数。 对于 EtherCAT 驱动器， 对应数据字典 0x6040 EtherCAT 控制器 ATYPE=65 时控制字会根据 WDOG/AXIS_ENABLE 自动切换以使用驱动器，主要位操作如下图。具体位意义请查看对应驱动器手册。 <table><tr><th rowspan="3">command</th><th colspan="5">bits of the controlword</th><th rowspan="3">PDS transitions</th></tr><tr><th>bit 7</th><th>bit 3</th><th>bit 2</th><th>bit 1</th><th>bit 0</th></tr><tr><th>fault reset</th><th>enable operation</th><th>quick stop</th><th>enable voltage</th><th>switch on</th></tr><tr><td>shut down</td><td>0</td><td>-</td><td>1</td><td>1</td><td>1</td><td>2,6,8</td></tr><tr><td>switch on</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>3</td></tr><tr><td>switch on+enable operation</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>3+4 (*1)</td></tr><tr><td>enable operation</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>4,16</td></tr><tr><td>disable voltage</td><td>0</td><td>-</td><td>-</td><td>0</td><td>-</td><td>7,9,10,12</td></tr><tr><td>quick stop</td><td>0</td><td>-</td><td>0(*2)</td><td>1</td><td>-</td><td>7,10,11</td></tr><tr><td>disable operation</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>5</td></tr><tr><td>fault reset</td><td></td><td>-</td><td>-</td><td>-</td><td>-</td><td>15</td></tr></table>	command	bits of the controlword					PDS transitions	bit 7	bit 3	bit 2	bit 1	bit 0	fault reset	enable operation	quick stop	enable voltage	switch on	shut down	0	-	1	1	1	2,6,8	switch on	0	0	1	1	1	3	switch on+enable operation	0	1	1	1	1	3+4 (*1)	enable operation	0	1	1	1	1	4,16	disable voltage	0	-	-	0	-	7,9,10,12	quick stop	0	-	0(*2)	1	-	7,10,11	disable operation	0	1	1	1	1	5	fault reset		-	-	-	-	15
command	bits of the controlword					PDS transitions																																																																				
	bit 7		bit 3	bit 2	bit 1		bit 0																																																																			
	fault reset	enable operation	quick stop	enable voltage	switch on																																																																					
shut down	0	-	1	1	1	2,6,8																																																																				
switch on	0	0	1	1	1	3																																																																				
switch on+enable operation	0	1	1	1	1	3+4 (*1)																																																																				
enable operation	0	1	1	1	1	4,16																																																																				
disable voltage	0	-	-	0	-	7,9,10,12																																																																				
quick stop	0	-	0(*2)	1	-	7,10,11																																																																				
disable operation	0	1	1	1	1	5																																																																				
fault reset		-	-	-	-	15																																																																				

	对于 Rtex 驱动器 Rtex 控制器控制字缺省自动设置，需要手动时先将 DRIVE_CW_MODE 设为 1，不能随便修改，位意义如下图，详细说明请看松下 Rtex 手册的 4-2-3 章节。 <table><tr><td>bit7</td><td>bit6</td><td>bit5</td><td>bit4</td><td>bit3</td><td>bit2</td><td>bit1</td><td>bit0</td></tr><tr><td>Servo_On</td><td>0</td><td>0</td><td>Gain_SW</td><td>TL_SW</td><td>HM_Ctrl</td><td>0</td><td>0</td></tr><tr><td>bit15</td><td>bit14</td><td>bit13</td><td>bit12</td><td>bit11</td><td>bit10</td><td>bit9</td><td>bit8</td></tr><tr><td>Hard_Stop</td><td>Smooth_Stop</td><td>Pause</td><td>0</td><td>SL_SW</td><td>0</td><td>EX-OUT2</td><td>EX-OUT1</td></tr></table>	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Servo_On	0	0	Gain_SW	TL_SW	HM_Ctrl	0	0	bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8	Hard_Stop	Smooth_Stop	Pause	0	SL_SW	0	EX-OUT2	EX-OUT1
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0																										
Servo_On	0	0	Gain_SW	TL_SW	HM_Ctrl	0	0																										
bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8																										
Hard_Stop	Smooth_Stop	Pause	0	SL_SW	0	EX-OUT2	EX-OUT1																										
语法	可读：var=DRIVE_CONTROLWORD(axis) 可写：DRIVE_CONTROLWORD(axis)=value axis: 轴号																																
适用控制器	带 EtherCAT 接口或 RTEX 接口																																
例子	例一 EtherCAT SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN AXIS_ADDRESS(0)=1 ATYPE(0)=65																																

DRIVE_STATUS -- 驱动器状态

类型	轴状态
描述	驱动器的当前状态，按位判断状态。 必须设置正确的 ATYPE (EtherCAT 总线设置为 65/66/67, RTEX 总线设置为 50/51/52) 以后才能读取。 对于 EtherCAT 驱动器，对应数据字典 6041 根据此 bit 可以确认 PDS 的状态，以下表示状态和对应的 bit

	<table><tr><th>状态字</th><th colspan="2">PDS state</th></tr><tr><td>xxxx xxxx x0xx 0000 b</td><td>not ready to switch on</td><td>初始化 未完成状态</td></tr><tr><td>xxxx xxxx x1xx 0000 b</td><td>switch on disabled</td><td>初始化 完成状态</td></tr><tr><td>xxxx xxxx x01x 0001 b</td><td>ready to switch on</td><td>主电路电源 off 状态</td></tr><tr><td>xxxx xxxx x01x 0011 b</td><td>switch on</td><td>伺服使能 off/伺服准备</td></tr><tr><td>xxxx xxxx x01x 0111 b</td><td>operation enabled</td><td>伺服使能 on</td></tr><tr><td>xxxx xxxx x00x 0111 b</td><td>quick stop active</td><td>快速停止</td></tr><tr><td>xxxx xxxx x0xx 1111 b</td><td>fault reaction active</td><td>异常（报警）判断</td></tr><tr><td>xxxx xxxx x0xx 1000 b</td><td>fault</td><td>异常（报警）状态</td></tr></table>	状态字	PDS state		xxxx xxxx x0xx 0000 b	not ready to switch on	初始化 未完成状态	xxxx xxxx x1xx 0000 b	switch on disabled	初始化 完成状态	xxxx xxxx x01x 0001 b	ready to switch on	主电路电源 off 状态	xxxx xxxx x01x 0011 b	switch on	伺服使能 off/伺服准备	xxxx xxxx x01x 0111 b	operation enabled	伺服使能 on	xxxx xxxx x00x 0111 b	quick stop active	快速停止	xxxx xxxx x0xx 1111 b	fault reaction active	异常（报警）判断	xxxx xxxx x0xx 1000 b	fault	异常（报警）状态						
	状态字	PDS state																																
	xxxx xxxx x0xx 0000 b	not ready to switch on	初始化 未完成状态																															
	xxxx xxxx x1xx 0000 b	switch on disabled	初始化 完成状态																															
	xxxx xxxx x01x 0001 b	ready to switch on	主电路电源 off 状态																															
	xxxx xxxx x01x 0011 b	switch on	伺服使能 off/伺服准备																															
	xxxx xxxx x01x 0111 b	operation enabled	伺服使能 on																															
	xxxx xxxx x00x 0111 b	quick stop active	快速停止																															
	xxxx xxxx x0xx 1111 b	fault reaction active	异常（报警）判断																															
	xxxx xxxx x0xx 1000 b	fault	异常（报警）状态																															
其他位意义请查看对应驱动器手册说明。																																		
对于 Rtex 驱动器																																		
Rtex 驱动器的状态字位意义如下图，详细含义请查看松下 Rtex 手册第 4-2-3 章节。																																		
<table><tr><td>bit7</td><td>bit6</td><td>bit5</td><td>bit4</td><td>bit3</td><td>bit2</td><td>bit1</td><td>bit0</td></tr><tr><td>Servo_Active</td><td>Servo_Ready</td><td>Alarm</td><td>Warning</td><td>Torque_Limited</td><td>Homing_Complete</td><td>In_Progress</td><td>In_Position</td></tr><tr><td>bit15</td><td>bit14</td><td>bit13</td><td>bit12</td><td>bit11</td><td>bit10</td><td>bit9</td><td>bit8</td></tr><tr><td>SI-MON5 /E-STOP</td><td>SI-MON4/ EX-SON</td><td>SI-MON3/E XT3/STOP</td><td>SI-MON2/E XT2/RET</td><td>SI-MON1/ EXT1</td><td>HOME</td><td>POT/NOT</td><td>NOT/POT</td></tr></table>			bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Servo_Active	Servo_Ready	Alarm	Warning	Torque_Limited	Homing_Complete	In_Progress	In_Position	bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8	SI-MON5 /E-STOP	SI-MON4/ EX-SON	SI-MON3/E XT3/STOP	SI-MON2/E XT2/RET	SI-MON1/ EXT1	HOME	POT/NOT	NOT/POT
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0																											
Servo_Active	Servo_Ready	Alarm	Warning	Torque_Limited	Homing_Complete	In_Progress	In_Position																											
bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8																											
SI-MON5 /E-STOP	SI-MON4/ EX-SON	SI-MON3/E XT3/STOP	SI-MON2/E XT2/RET	SI-MON1/ EXT1	HOME	POT/NOT	NOT/POT																											
语法	只读：toq = DRIVE_STATUS (axis) axis: 轴号																																	
适用控制器	带 EtherCAT 接口或 RTEX 接口																																	
例子	SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ATYPE(0)=65																																	

DRIVE_IO -- 驱动器 IO

类型	轴参数
描述	直接配置 DRIVE 的 IO 的起始编号，输入输出都是同样的编号。 DRIVE_PROFILE 支持读取驱动器 IO 时起作用。
语法	可读：var= DRIVE_IO (axis) 可写：DRIVE_IO (axis)=value axis: 轴号

	EtherCAT 总线伺服的输入输出点功能及地址参考数据字典 60FD, 60FE (某些厂家不是按照标准协议地址, 请查看对应驱动器手册) Rtex 总线伺服的输入输出点功能对应控制器的地址=DRIVE_IO+编号 <table border="1"> <thead> <tr> <th>DRIVE_IO+以下编号</th><th>伺服功能</th></tr> </thead> <tbody> <tr> <td colspan="2">输入点</td></tr> <tr> <td>0</td><td>NOT/POT</td></tr> <tr> <td>1</td><td>POT/NOT</td></tr> <tr> <td>2</td><td>HOME</td></tr> <tr> <td>3</td><td>SI-MON1/EXT1</td></tr> <tr> <td>4</td><td>SI-MON2/EXT2</td></tr> <tr> <td>5</td><td>SI-MON3/EXT3</td></tr> <tr> <td>6</td><td>SI-MON4/EX-SON</td></tr> <tr> <td>7</td><td>SI-MON5/E-STOP</td></tr> <tr> <td colspan="2">输出点</td></tr> <tr> <td>0</td><td>EX-OUT1</td></tr> <tr> <td>1</td><td>EX-OUT2</td></tr> </tbody> </table>	DRIVE_IO+以下编号	伺服功能	输入点		0	NOT/POT	1	POT/NOT	2	HOME	3	SI-MON1/EXT1	4	SI-MON2/EXT2	5	SI-MON3/EXT3	6	SI-MON4/EX-SON	7	SI-MON5/E-STOP	输出点		0	EX-OUT1	1	EX-OUT2
DRIVE_IO+以下编号	伺服功能																										
输入点																											
0	NOT/POT																										
1	POT/NOT																										
2	HOME																										
3	SI-MON1/EXT1																										
4	SI-MON2/EXT2																										
5	SI-MON3/EXT3																										
6	SI-MON4/EX-SON																										
7	SI-MON5/E-STOP																										
输出点																											
0	EX-OUT1																										
1	EX-OUT2																										
适用控制器	带 EtherCAT 接口或 RTEX 接口																										
例子	<pre> SLOT_SCAN(0) ... IF NODE_COUNT(0)>0 THEN DRIVE_IO(1)=32 DIM var var=DRIVE_IO(1) ?var ENDIF </pre> '轴使能过程, 参考第十七章简易例程 '轴 1 设备的 IO 起始编号设为 32 '定义变量 var 'var 赋值为轴 1 设备的 IO 起始编号 '直接打印出轴 1 设备的 IO 起始编号																										
相关指令	NODE_IO , DRIVE_PROFILE																										

DRIVE_TORQUE -- 驱动器力矩

类型	EtherCAT 轴状态
描述	驱动器的当前力矩。 必须设置正确的 ATYPE (设置为 65/66/67) 与 DRIVE_PROFILE 以后才能读取。
语法	只读: var = DRIVE_TORQUE(axis) axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ATYPE(0)=65 DRIVE_PROFILE(0)=1 ? DRIVE_TORQUE (0) ENDIF </pre> '总线位置控制 'pdo 设为 1, 含有返回力矩功能 '打印轴 0 的力矩
相关指令	ATYPE , DRIVE_PROFILE

DRIVE_TORQUEMAX -- 驱动器最大力矩限制

类型	EtherCAT 轴状态
描述	驱动器的当前力矩的最大限制。 必须设置正确的 ATYPE（设置为 65/66/67）与 DRIVE_PROFILE 以后才能读取。
语法	可读：var = DRIVE_TORQUEMAX(axis) 可写：DRIVE_TORQUEMAX(axis) = value axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ATYPE(0)=65 '总线位置控制 DRIVE_PROFILE(0)=1 'pdo 设为 1，含有返回力矩功能 ? DRIVE_TORQUEMAX (0) '打印轴 0 的力矩 ENDIF </pre>
相关指令	ATYPE , DRIVE_PROFILE , DRIVE_TORQUE , DRIVE_TORQUEMIN

DRIVE_TORQUEMIN -- 驱动器最小力矩限制

类型	EtherCAT 轴状态
描述	驱动器的当前力矩的最小限制。 必须设置正确的 ATYPE（设置为 65/66/67）与 DRIVE_PROFILE 以后才能读取。
语法	可读：var = DRIVE_TORQUEMIN(axis) 可写：DRIVE_TORQUEMIN(axis) = value axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN ATYPE(0)=65 '总线位置控制 DRIVE_PROFILE(0)=1 'pdo 设为 1，含有返回力矩功能 ? DRIVE_TORQUEMIN (0) '打印轴 0 的力矩 ENDIF </pre>
相关指令	ATYPE , DRIVE_PROFILE , DRIVE_TORQUE , DRIVE_TORQUEMAX

DRIVE_FE -- 驱动器误差

类型	轴状态
描述	驱动器的当前误差超限，对应数据字典 0x60F4。 必须设置正确的 ATYPE（设置为 65/66/67）以后才能设置。 ZMC408SCAN 的 ATYPE=22 带反馈位置时有效，此时返回驱动器的接收和反馈位置偏差。
语法	只读：var = DRIVE_FE (axis)

	axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
例子	<pre> SLOT_SCAN(0) '总线扫描 IF NODE_COUNT(0)>0 THEN AXIS_ADDRESS(0)=1 '第一个驱动器映射到轴 0 ATYPE(0)=65 '轴类型 65，位置控制 DRIVE_PROFILE(0)=10 'PDO 周期报文设置，根据 DRIVE_PROFILE ?DRIVE_FE (0) '读取轴 0 设备的跟踪误差值 ENDIF </pre>
相关指令	DRIVE_FE_LIMIT

DRIVE_FE_LIMIT -- 驱动器误差限制

类型	EtherCAT 轴参数
描述	驱动器的当前误差超限设置。 必须设置正确的 ATYPE（设置为 65/66/67）以后才能设置。预留。
语法	可读：var=DRIVE_FE_LIMIT(axis) 可写：DRIVE_FE_LIMIT(axis)= value axis: 轴号
适用控制器	带 EtherCAT 接口或 RTEX 接口
相关指令	DRIVE_FE

DRIVE_CLEAR -- 清除报警

类型	总线指令								
描述	操作当前 BASE 轴，清除驱动器报警。 通过 RETURN 返回成功与否。 驱动器没有错误时，使用指令控制器会警告 6015，不影响程序运行。								
语法	<pre> BASE(轴号) DRIVE_CLEAR(para) para: </pre> <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>0</td><td>清除当前告警</td></tr> <tr> <td>1</td><td>清除历史告警</td></tr> <tr> <td>2</td><td>清除外部输入告警</td></tr> </tbody> </table>	值	描述	0	清除当前告警	1	清除历史告警	2	清除外部输入告警
值	描述								
0	清除当前告警								
1	清除历史告警								
2	清除外部输入告警								
适用控制器	带 EtherCAT 接口或 RTEX 接口								
例子	<pre> SLOT_SCAN(0) IF NODE_COUNT(0)>0 THEN BASE(0) '选择轴 0 DRIVE_CLEAR(0) '清除当前告警 ENDIF </pre>								
相关指令	DRIVE_READ , DRIVE_WRITE								

DRIVE_READ -- 参数读取

类型	总线指令，仅 Rtex 控制器可用	
描述	操作当前 BASE 轴，读取驱动器参数。 通过 RETURN 返回成功与否。	
语法	DRIVE_READ(para [,vr_index])	
	para:	
	值	描述
	1	参数分类*256+参数编号（Pr7.20=7*256+20）
	2	参数=130,读取钳位的状态,BIT0,BIT1 表示两个通道的状态
	3	参数=\$10000+(ssid)读取 RTEX 驱动器系统信息,字符串存储在 VRSTRING
	4	参数=\$20000+(报警功能码)+(\$1000*索引)读取报警信息
	5	参数=\$30000+(监视功能码)+(\$1000*索引)读取监视器信息
	vr_index：读取数据存储到 vr 里面，若没有，直接在输出栏打印出来	
	以下为常用参数	
	1.伺服参数	
	参数	功能
Pr0.00	设置电机转向	0: CW 1: CCW
Pr0.01	控制模式设置	一般设置为 0: 半闭环控制
Pr0.08	电机一圈脉冲数	0-8388608（根据实际电机）
Pr0.09	齿轮比分子设定	0-1073741824
Pr0.10	齿轮比分母设定	1-1073741824
Pr4.01	正限位信号设置	常闭: 00818181h（8487297） 常开: 00010101h（65793）
Pr4.02	负限位信号设置	常闭: 00828282h（8553090） 常开: 00020202h（131586）
Pr4.03	Home 信号设置	常闭: 00A2A2A2h（10658466） 常开: 00222222h（2236962）
2.Rtex 通讯参数		
Pr7.20	Rtex 通讯周期	-1: 将 Pr7.91 的设定生效 3: 0.5ms 6: 1.0ms
Pr7.21	Rtex 指令更新周期比	1: 1 倍 2: 2 倍
Pr7.91	Rtex 通讯周期扩展	62500 ns 125000 ns 250000 ns 500000 ns 1000000 ns 2000000 ns
3.驱动器系统信息		
SSID	含义	
\$01	厂商名	
\$05	设备类型	
\$12	驱动器型号	

	\$13	驱动器序列号	
	\$14	驱动器软件版本	
	\$15	驱动器类型	
	\$22	电机型号	
	\$23	电机序列号	
	4.报警信息		
	功能码	功能	索引
	\$000	读取当前报警/报警履历	0: 本次的报警码 1: 上次的报警码 2: 前两次的报警码 ... 14: 前 14 次的报警码
	\$001	清除当前报警	0: 清除本次的报警
	\$011	清除所有报警	0: 清除报警履历
	\$021	外部位移传感器的错误清除	0: 通过串行通信类型的外部位移传感器清除箝位的错误。 进行外部位移传感器的错误清除后, 请断开控制电源后重启。
	5.监视器		
	功能码	功能	索引
	\$01	位置偏差, 指令单位	0: 滤波后的指令的偏差
	\$02	编码器分辨率, 脉冲/转	0: 电机编码器分辨率
	\$04	指令位置, 指令单位	0: 滤波后的内部指令位置
	\$05	实际速度, Pr7.25 单位	0: 电机实际速度
	\$06	内部指令转矩, 0.1%	0: 到电机的指令转矩
	\$07	实际位置, 指令单位	0: 电机实际的位置
	\$09	箝位位置 1, 指令单位	0: CH1 箝位的电机的实际位置
	\$0A	箝位位置 2, 指令单位	0: CH2 箝位的电机的实际位置
适用控制器	带 RTEX 接口		
例子	总线开启后才能使用 例一 <pre>IF NODE_COUNT(0)>0 THEN BASE(0) '选择轴 0 DRIVE_READ(7*256+11,0) '读取 Pr7.11 参数数值, 保存到 vr(0) ?vr(0) '打印 ENDIF</pre> 例二 <pre>IF NODE_COUNT(0)>0 THEN BASE(0) '选择轴 0 DRIVE_READ(\$10000+\$01) '读取厂商名, 直接打印出来 ENDIF</pre>		
相关指令	DRIVE_WRITE , DRIVE_CLEAR		

DRIVE_WRITE -- 参数写入

类型	总线指令，仅 Rtex 控制器可用						
描述	操作当前 BASE 轴，驱动器参数修改。 通过 RETURN 返回成功与否。						
语法	DRIVE_WRITE(para,value)						
	para:						
	值	描述					
	1	参数分类*256 + 参数编号（Pr7.20=7*256+20）					
	2	特殊参数=128，当前参数写入 EEPROM（此时 value=1）					
	3	特殊参数=\$40000 + typecode + (设置值*256)，使用驱动器的回零箝位功能					
	value: 参数值						
	1.伺服参数						
	伺服参数	功能			值		
	Pr0.00	设置电机转向			0: CW 1: CCW		
	Pr0.01	控制模式设置			一般设置为 0: 半闭环控制		
	Pr0.08	电机一圈脉冲数			0-8388608（根据实际电机）		
	Pr0.09	齿轮比分子设定			0-1073741824		
	Pr0.10	齿轮比分母设定			1-1073741824		
	Pr4.01	正限位信号设置			常闭：00818181h（8487297） 常开：00010101h（65793）		
	Pr4.02	负限位信号设置			常闭：00828282h（8553090） 常开：00020202h（131586）		
	Pr4.03	Home 信号设置			常闭：00A2A2A2h（10658466） 常开：00222222h（2236962）		
	转矩相关						
	Pr0.13	第一转矩限制			0~500%		
	Pr5.21	转矩限制选择 转矩控制时，固定为 Pr0.13（第 1 转矩限制）。			如下图所示		
			设定值	TL_SW=0		TL_SW=1	
				负方向	正方向	负方向	正方向
				Pr0.13			
2				Pr5.22	Pr0.13	Pr5.22	Pr0.13
3				Pr0.13		Pr5.22	
4		Pr5.22	Pr0.13	Pr5.22	Pr5.25		
Pr5.22	第二转矩限制			0~500%			
Pr5.25	正方向转矩限制			0~500%			
Pr5.26	负方向转矩限制			0~500%			
速度相关							
Pr3.12	加速时间设置			0~10000ms （达到 1000.r/min）			
Pr3.13	减速时间设置			0~10000ms （达到 1000.r/min）			
Pr3.14	S 加减速设置			0~1000ms			
Pr3.17	速度限制选择			设定值	SL_SW		

		转矩控制时的速度限制值的方式选择	<table><tr><td></td><td>0</td><td>1</td></tr><tr><td>[0]</td><td colspan="2">Pr3.21</td></tr><tr><td>1</td><td>Pr3.21</td><td>Pr3.22</td></tr></table> 设置为 1 时，根据 RTEX 通信指令 SL_SW 的数值进行选择		0	1	[0]	Pr3.21		1	Pr3.21	Pr3.22
		0	1									
	[0]	Pr3.21										
	1	Pr3.21	Pr3.22									
	Pr3.21	速度限制值 1	0~20000r/min									
	Pr3.22	速度限制值 2	0~20000r/min									
	2.Rtex 通讯参数											
	Pr7.20	Rtex 通讯周期	-1：将 Pr7.91 的设定生效 3：0.5ms 6：1.0ms									
	Pr7.21	Rtex 指令更新周期比	1：1 倍 2：2 倍									
	Pr7.91	Rtex 通讯周期扩展	62500 ns 125000 ns 250000 ns 500000 ns 1000000 ns 2000000 ns									
3.回零箝位模式												
typecode		描述										
\$50		位置箝位状态监视器										
\$51		位置箝位 1 启动										
\$52		位置箝位 2 启动										
\$53		位置箝位 1,2 启动										
\$54		位置箝位 1 解除										
\$58		位置箝位 2 解除										
\$5c		位置箝位 1,2 解除										
4.设置值												
设置值=钳位 1 设置 + (\$10* 钳位 2 设置)												
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0					
LATCH_SEL2				LATCH_SEL1								
箝位 1/2 设置		描述										
\$0		Z 相信号触发										
\$1		EXT1 的逻辑上升沿										
\$2		EXT2 的逻辑上升沿										
\$3		EXT3 的逻辑上升沿										
\$9		EXT1 的逻辑下降沿										
\$10		EXT2 的逻辑下降沿										
\$11		EXT3 的逻辑下降沿										
适用控制器	支持 RTEX 的控制器											
例子	总线开启后才能使用 例一 IF NODE COUNT(0)>0 THEN											

	<pre>BASE(0) '选择轴 0 DRIVE_WRITE(7*256+11,6) '写 Pr7.11 参数为 6 DRIVE_READ(7*256+11,0) '读取 Pr7.11 参数数值，保存到 vr(0) ?vr(0) '打印，打印值 6 ENDIF 例二 IF NODE_COUNT(0)>0 THEN BASE(0) '选择轴 0 DRIVE_WRITE(128,1) '将修改的参数写入 EEPROM ENDIF</pre>
相关指令	DRIVE_READ , DRIVE_CLEAR

第十八章 ZHD 示教盒参数

18.1 示教盒相关指令

LCD_CONNECT -- LCD 编号设置

类型	示教盒指令
描述	该指令建立在控制器与示教盒通讯成功的基础上，在示教盒端操作。 设置要链接的对方控制器的 LCD（HMI）编号，缺省 0，当控制器支持多个 HMI 时，可以设置为其它值。 设置后可能需要重启示教盒才能连上。
语法	VAR1=LCD_CONNECT LCD_CONNECT = VAR1
例子	>>LCD_CONNECT=1 >>LCD_CONNECT=0

IP_CONNECT -- IP 连接

类型	示教盒指令
描述	该指令建立在控制器与示教盒通讯成功的基础上，在示教盒端操作。 设置要链接的对方的控制器 IP 地址。 每次修改后会重新链接。
语法	IP_CONNECT = dot.dot.dot.dot
例子	>>IP_CONNECT=192.168.0.14

IP_ADDRESS -- IP 地址

类型	示教盒指令
描述	该指令建立在控制器与示教盒通讯成功的基础上，在示教盒端操作。 指定示教盒的 IP 地址，缺省 192.168.0.10
语法	IP_ADDRESS = dot.dot.dot.dot
例子	>>IP_ADDRESS=192.168.0.14
相关指令	IP_GATEWAY IP_NETMASK

IP_GATEWAY -- IP 网关

类型	示教盒指令
描述	该指令建立在控制器与示教盒通讯成功的基础上，在示教盒端操作。 指定示教盒的 IP 网关，缺省 192.168.0.1

语法	IP_GATEWAY = dot.dot.dot.dot
例子	>> IP_GATEWAY=192.168.0.1
相关指令	IP_ADDRESS IP_NETMASK

IP_NETMASK -- IP 掩码

类型	示教盒指令
描述	该指令建立在控制器与示教盒通讯成功的基础上，在示教盒端操作。 指定示教盒的 IP 掩码，缺省 255.255.0.0
语法	IP_NETMASK = dot.dot.dot.dot
例子	>>IP_NETMASK= 255.255.0.0
相关指令	IP_ADDRESS IP_GATEWAY

18.2 控制器相关指令

LCD_LEDSTATE -- LED 灯状态设置

类型	系统屏参数
描述	该指令建立在控制器与示教盒通讯成功的基础上，在控制器端操作。 设置示教盒上的 LED 灯状态，按 BIT 位来设置，缺省=1，亮运行灯。 需要控制器和示教盒固件同时支持。 VERSION_BUILD = 230801 以上版本支持
语法	VAR = LCD_LEDSTATE(lcdnum) LCD_LEDSTATE(lcdnum) = VAR lcdnum: 控制器的 LCD (HMI) 编号，缺省 0
适用控制器	支持 RTHmi
例子	LCD_LEDSTATE(0) = 1

LCD_WDOGTIME -- 屏掉线处理时间设置

类型	系统屏参数
描述	该指令建立在控制器与示教盒通讯成功的基础上，在控制器端操作。 设置屏掉线处理时间，ms 单位。 当屏操作这个时间没有通讯时，急停开关(物理按键编号=5)自动按下，等于 0 时，此功能不启用。 5 系列 20180404 以上固件版本支持，4 系列标准固件 20170721 以上版本支持
语法	LCD_WDOGTIME(lcdnum)=time lcdnum: 控制器的 LCD (HMI) 编号，缺省 0 time: 时间，ms 单位
适用控制器	支持 RTHmi

例子	LCD_WDOGTIME(0)=100
----	---------------------

第十九章 MotionRT 相关指令

下面指令为 RT7 增加指令，以前指令也都兼容支持。

CARD_INFO -- 读写控制卡信息

类型	控制卡指令																																																								
描述	读取/写入控制卡信息																																																								
语法	读取信息语法: <pre>var = CARD_INFO (cardnum, sel)</pre> cardnum: 子卡号, 0-N-1 (N: 控制卡数量。无子卡时, N 为-1) sel: 信息编号 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr><td>0</td><td>返回子卡的总数, 此时 cardnum 填 0</td></tr> <tr><td>1</td><td>DEVICE, 设备编号, hardid.</td></tr> <tr><td>2</td><td>VERSION, 版本</td></tr> <tr><td>3</td><td>子卡拨码, PCI 才有</td></tr> <tr><td>4</td><td>预留</td></tr> <tr><td>5</td><td>子卡唯一编号, 最后一个 PCI 子卡的唯一编号作为 RT 的唯一编号</td></tr> <tr><td>6</td><td>预留特殊编号</td></tr> <tr><td>7</td><td>预留</td></tr> <tr><td>8</td><td>IO 偏移, 8 对齐, 上电缺省会自动排列</td></tr> <tr><td>9</td><td>AIO 偏移</td></tr> <tr><td>10</td><td>IN 个数</td></tr> <tr><td>11</td><td>OP 个数</td></tr> <tr><td>12</td><td>AIN 个数</td></tr> <tr><td>13</td><td>AOUT 个数</td></tr> <tr><td>14</td><td>预留</td></tr> <tr><td>15</td><td>HW 的 OP 个数</td></tr> <tr><td>16</td><td>脉冲轴个数</td></tr> <tr><td>17</td><td>编码器轴个数</td></tr> <tr><td>18</td><td>Ecat 总线个数 0/1</td></tr> <tr><td>19</td><td>scan 个数 0/1</td></tr> <tr><td>20</td><td>3D 振镜个数</td></tr> <tr><td>21</td><td>掉电支持, VR 个数, XPCI 一般不带 VR</td></tr> <tr><td>22</td><td>VR 的 offset, 自动编号</td></tr> <tr><td>23</td><td>PWM 个数</td></tr> <tr><td>24</td><td>子卡上的 PWM 起始编号</td></tr> </tbody> </table> 写入信息语法: <pre>CARD_INFO (cardnum, sel)=value</pre> cardnum: 子卡号, 0-缺省 sel: 信息编号 <table border="1"> <thead> <tr> <th>值</th><th>描述</th></tr> </thead> <tbody> <tr> <td>8</td><td>IO 偏移, 8 对齐, 上电会自动顺序排列 注: value 须为 8 的倍数</td></tr> </tbody> </table>	值	描述	0	返回子卡的总数, 此时 cardnum 填 0	1	DEVICE, 设备编号, hardid.	2	VERSION, 版本	3	子卡拨码, PCI 才有	4	预留	5	子卡唯一编号, 最后一个 PCI 子卡的唯一编号作为 RT 的唯一编号	6	预留特殊编号	7	预留	8	IO 偏移, 8 对齐, 上电缺省会自动排列	9	AIO 偏移	10	IN 个数	11	OP 个数	12	AIN 个数	13	AOUT 个数	14	预留	15	HW 的 OP 个数	16	脉冲轴个数	17	编码器轴个数	18	Ecat 总线个数 0/1	19	scan 个数 0/1	20	3D 振镜个数	21	掉电支持, VR 个数, XPCI 一般不带 VR	22	VR 的 offset, 自动编号	23	PWM 个数	24	子卡上的 PWM 起始编号	值	描述	8	IO 偏移, 8 对齐, 上电会自动顺序排列 注: value 须为 8 的倍数
值	描述																																																								
0	返回子卡的总数, 此时 cardnum 填 0																																																								
1	DEVICE, 设备编号, hardid.																																																								
2	VERSION, 版本																																																								
3	子卡拨码, PCI 才有																																																								
4	预留																																																								
5	子卡唯一编号, 最后一个 PCI 子卡的唯一编号作为 RT 的唯一编号																																																								
6	预留特殊编号																																																								
7	预留																																																								
8	IO 偏移, 8 对齐, 上电缺省会自动排列																																																								
9	AIO 偏移																																																								
10	IN 个数																																																								
11	OP 个数																																																								
12	AIN 个数																																																								
13	AOUT 个数																																																								
14	预留																																																								
15	HW 的 OP 个数																																																								
16	脉冲轴个数																																																								
17	编码器轴个数																																																								
18	Ecat 总线个数 0/1																																																								
19	scan 个数 0/1																																																								
20	3D 振镜个数																																																								
21	掉电支持, VR 个数, XPCI 一般不带 VR																																																								
22	VR 的 offset, 自动编号																																																								
23	PWM 个数																																																								
24	子卡上的 PWM 起始编号																																																								
值	描述																																																								
8	IO 偏移, 8 对齐, 上电会自动顺序排列 注: value 须为 8 的倍数																																																								

	9	AIO 偏移, 上电会自动顺序排列
例子	GLOBAL value value=CARD_INFO(0,1) 读取控制卡设备编号 ?value end	

?*CARD -- 打印子卡信息

类型	控制卡指令
描述	打印子卡信息（可在 RTSys/RTSys 中在线命令发送）
语法	?*CARD HardId: 硬件版本 Pul: 脉冲数 In: 输入个数 Op: 输出个数 Ad: 模拟量输入个数 Da: 模拟量输出个数 Pwm: Pwm 个数 flash: flash 空间大小 size: ROM 大小 serial: 卡编号 license: 参数配置
例子	?*CARD >>*CARD Card0: is XPCI, HardId:7133-0 Pul:4 In:16 Op:16 Ad:0 Da:0 Pwm:4 flash:ef4016h size:4194304 serial:221090003 license:AX64 M08 ZV HW

REG_CARD -- 锁存选择

类型	控制卡指令
描述	锁存使用的输入选择。当多个子卡都支持锁存时，支持切换。 与 REGINPUTS, REGIST 配合使用，REGIST 调用后，支持立刻切换。 锁存位置: REG_POSE, REG_POSF, REG_POSG, REG_POSH 锁存通道: MAKRE, MAKRF, MAKRG, MAKRH（可扩展到最多 8 通道的锁存）
语法	REG_CARD=value 使用轴的专有锁存，设置 REG_CARD = -1 设置其余数值为指定操作锁存 IO 的卡号
适用控制器	7 系，多卡支持
例子	例一：单卡使用 reg_card(0) = -1 '设置子卡编号 0 的 in 锁轴 0，一般子卡编号从 0 开始 base(0)

```

dpos(0)=0
mpos(0)=0
reg_inputs=$0000      '将 R0-R3 都对应到输入口 0，最多支持拓展 8 个锁存通道
atype = 1
vmove(1)
regist(21)
wait until markd
? reg_posd(0)

例二：多卡使用，使用卡 1 的 IN 去锁存卡 2 的轴
reg_card(4) = 0      '设置子卡编号 0 的 in 锁卡 2 的轴 4，一般子卡编号从 0 开始
base(4)
dpos(4)=0
mpos(4)=0
reg_inputs=$0000      '将 R0-R3 都对应到输入口 0，最多支持拓展 8 个锁存通道
atype = 1
vmove(1)
regist(21)
wait until markd
? reg_posd(4)

例三：卡 1 使用新增通道锁卡 1 的轴 0（与单卡使用一致）
' 卡 1 使用自身的 in 去锁卡 1 的轴，使用新增通道
BASE(0)
REG_CARD = -1          '默认为-1，表示使用本卡的 in 锁本卡的轴
    REG_INPUTS = $32103210      '映射 R0-R3 对应卡 1 的 in0-3,R4-7 对应卡 1 的
in0-3
    ATYPE=1
    DPOS(0) = 0
    MPOS(0) = 0
    REGIST(25)                '单次锁存
    REGIST(27)
    REGIST(29)
    REGIST(31)
    VMOVE(1)
    WAIT UNTIL MARKE
    ? REG_POSE
    WAIT UNTIL MARKF
    ? REG_POSF
    WAIT UNTIL MARKG
    ? REG_POSG
    WAIT UNTIL MARKH
    ? REG_POSH

```

SLOT_SLAVE -- ETHERCAT 冗余配置

类型	特殊参数
描述	配置两个 ECAT 通道为一个通道,以支持热冗余备份 注意 1: 必须总线停止的时候设置; 注意 2: 设置后只能对 islot1 操作, 不能对 islot2 操作; 注意 3: 总线初始化时必须接线正常, 否则会初始化失败, 或扫描设备个数错误, 查看是否配置正确可以用?SLOT_SLAVE(islot1)。
语法	SLOT_SLAVE(islot1) = islot2 islot1: 主总线, 必须接第一个设备的 IN 口 islot2: 从总线, 必须接最后一个设备的 OUT 口
适用控制器	7 系 240520 版本以上, 多卡支持
例子	SLOT_SLAVE(0) = 1
相关指令	SLOT_INFO 、 NODE_READ

SLOT_PERIOD -- 多 ECAT 总线周期配置

类型	槽位参数
描述	多个 ECAT, 每个 ECAT 可以指定不同的周期 注意 1: 必须总线停止的时候设置, 否则不会立即生效, 总线重新扫描的时候才生效 注意 2: 周期必须是 SERVO_PERIOD 的整数倍 注意 3: 此参数不保存 flash, 上电缺省值为 SERVO_PERIOD。
语法	SLOT_PERIOD (islot) = value islot1: 总线槽位号 value: 总线周期
适用控制器	RT750 支持; 多槽位的 F 系列控制器支持
例子	SLOT_PERIOD (0) = 250
相关指令	SERVO_PERIOD

第二十章 本地后级接口指令

ZMIO_CONFIG -- 设置/获取模拟量量程和通道状态

类型	XPLC300/ ZMC432M 本地后级接口 RS485 总线指令																																																
描述	读取或配置扩展子模块 AD/DA 通道开关状态及量程类型；																																																
语法	可读：var = ZMIO_CONFIG (sel, moduleid) 可写：ZMIO_CONFIG (sel, moduleid, value) sel: 功能编号； moduleid: 扩展子模块地址，从 0 开始计数； value: 配置扩展子模块通道值或量程类型； sel 功能编号描述： <table><tr><td>sel</td><td>描述</td></tr><tr><td>1</td><td>配置扩展子模块 AD/DA 量程类型</td></tr><tr><td>2</td><td>配置扩展子模块模拟量 AD 通道开关状态</td></tr></table> value 量程数据值的含义： <table><tr><td>type</td><td>value</td><td>量程</td></tr><tr><td rowspan="6">AD</td><td>2</td><td>0~10V</td></tr><tr><td>3</td><td>-10~10V</td></tr><tr><td>4</td><td>4~20mA</td></tr><tr><td>5</td><td>0~20mA</td></tr><tr><td>6</td><td>0~5V</td></tr><tr><td>7</td><td>-5~5V</td></tr></table> <table><tr><td>type</td><td>value</td><td>量程</td></tr><tr><td rowspan="6">DA</td><td>10</td><td>0~10V</td></tr><tr><td>11</td><td>-10~10V</td></tr><tr><td>12</td><td>4~20mA</td></tr><tr><td>13</td><td>0~20mA</td></tr><tr><td>14</td><td>0~5V</td></tr><tr><td>15</td><td>-5~5V</td></tr></table> value 通道数据值模型，4 组 value 值分别对应 AD 模块的四个通道： <table><tr><td>AD 通道</td><td>CH3</td><td>CH2</td><td>CH1</td><td>CH0</td></tr><tr><td>value</td><td>8</td><td>4</td><td>2</td><td>1</td></tr></table> 同时配置多通道开启，value 大小为开启的通道值总和；若同时开启 CH1 和 CH2，即 value 大小为： value = CH2 + CH1 = 6	sel	描述	1	配置扩展子模块 AD/DA 量程类型	2	配置扩展子模块模拟量 AD 通道开关状态	type	value	量程	AD	2	0~10V	3	-10~10V	4	4~20mA	5	0~20mA	6	0~5V	7	-5~5V	type	value	量程	DA	10	0~10V	11	-10~10V	12	4~20mA	13	0~20mA	14	0~5V	15	-5~5V	AD 通道	CH3	CH2	CH1	CH0	value	8	4	2	1
sel	描述																																																
1	配置扩展子模块 AD/DA 量程类型																																																
2	配置扩展子模块模拟量 AD 通道开关状态																																																
type	value	量程																																															
AD	2	0~10V																																															
	3	-10~10V																																															
	4	4~20mA																																															
	5	0~20mA																																															
	6	0~5V																																															
	7	-5~5V																																															
type	value	量程																																															
DA	10	0~10V																																															
	11	-10~10V																																															
	12	4~20mA																																															
	13	0~20mA																																															
	14	0~5V																																															
	15	-5~5V																																															
AD 通道	CH3	CH2	CH1	CH0																																													
value	8	4	2	1																																													
适用控制器	XPLC300、ZMC432M																																																
例子	ZMIO_CONFIG (1, 0, 10) '配置地址为 0 的扩展子模块 DA 量程类型为 0~10V ZMIO_CONFIG (2, 0, 15) '配置地址为 0 的扩展子模块 AD 所有通道全打开 ?ZMIO_CONFIG (1, 0) '获取地址为 0 的扩展子模块 AD/DA 量程类型 ?ZMIO_CONFIG (2, 0) '获取地址为 0 的扩展子模块 AD 通道开关状态																																																
相关指令	ZMIO_INFO																																																

ZMIO_INFO -- 查询自带 ZMIO 扩展

类型	XPLC300/ ZMC432M 本地后级接口 RS485 总线指令															
描述	用于查询 XPLC300B 系列控制器自带 ZMIO 扩展的情况															
语法	语法1: var=ZMIO_INFO(sel) 语法 2: var=ZMIO_INFO(17,moduleid) sel: 功能编号; <table><tr><th>功能号</th><th>功能内容</th></tr><tr><td>10</td><td>最多输入数</td></tr><tr><td>11</td><td>最多输出数</td></tr><tr><td>12</td><td>最多 AIN 数</td></tr><tr><td>13</td><td>最多 AOUT 数</td></tr><tr><td>16</td><td>模块数</td></tr><tr><td>17</td><td>子模块的类型号, 必须后面带 moduleid</td></tr></table> moduleid: 扩展子模块地址, 按接入耦合器的顺序, 从 0 依次编号;		功能号	功能内容	10	最多输入数	11	最多输出数	12	最多 AIN 数	13	最多 AOUT 数	16	模块数	17	子模块的类型号, 必须后面带 moduleid
功能号	功能内容															
10	最多输入数															
11	最多输出数															
12	最多 AIN 数															
13	最多 AOUT 数															
16	模块数															
17	子模块的类型号, 必须后面带 moduleid															
适用控制器	XPLC300、ZMC432M															
例子	?ZMIO_INFO(10) '打印自带 ZMIO 扩展的最多输入数 ?ZMIO_INFO(11) '打印自带 ZMIO 扩展的最多输出数 ?ZMIO_INFO(12) '打印自带 ZMIO 扩展的最多 AIN 数 ?ZMIO_INFO(13) '打印自带 ZMIO 扩展的最多 AOUT 数 ?ZMIO_INFO(16) '打印自带 ZMIO 扩展的模块数 ?ZMIO_INFO(17,0) '打印扩展的第一块模块的类型编号															
相关指令	ZMIO_CONFIG															

ZMIO_OFFSET -- 自带 ZMIO 扩展 IO 偏移

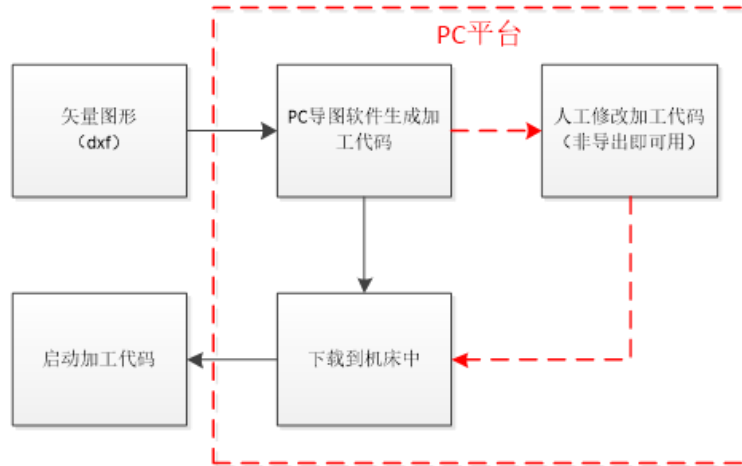
类型	XPLC300/ ZMC432M 本地后级接口 RS485 总线指令	
描述	此功能用于偏移控制器自带 ZMIO 扩展的 IO 地址。 ZMIO_OFFSET 修改之后要重启上电。 若本地扩展后级接口不需要接入 ZMIO310 子模块则要把该指令设置为负数或值超出输入输出起始编号范围(设置值须为 8 的倍数), 例如 ZMIO_OFFSET=-8 (XPLC300 不带扩展 IO)、ZMIO_OFFSET=560 (XPLC300 扩展 IO 的最大起始范围为 0-511), 否则会报 201 错误码。	
语法	ZMIO_OFFSET=value value: IO 起始地址, 缺省 32 注: 设置的 IO 起始地址只能是 8 的倍数。	
适用控制器	XPLC300、ZMC432M	
例子	ZMIO_OFFSET=48 '自带 ZMIO 扩展 IO 地址偏移到 48	
相关指令	ZMAIO_OFFSET	

ZMAIO_OFFSET -- 自带 ZMIO 扩展 AIO 偏移

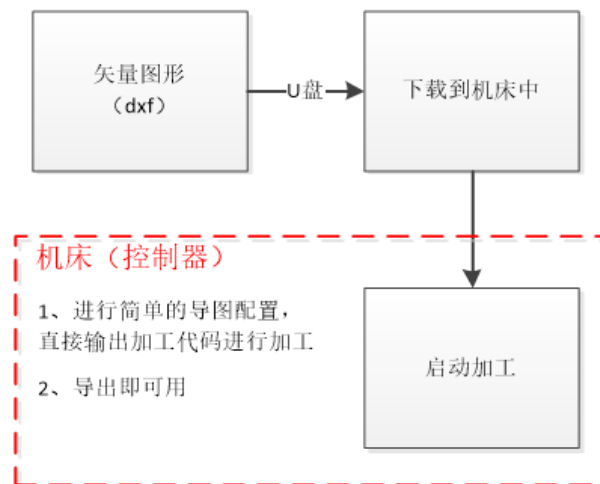
类型	XPLC300/ ZMC432M 本地后级接口 RS485 总线指令
描述	此功能用于偏移控制器自带 ZMIO 扩展的 AIO 地址。 ZMAIO_OFFSET 修改之后要重启上电。
语法	ZMAIO_OFFSET=value value: AIO 起始地址，缺省 32
适用控制器	XPLC300、ZMC432M
例子	ZMAIO_OFFSET=33 ‘自带 ZMIO 扩展 AIO 地址偏移到 33
相关指令	ZMIO_OFFSET

第二十一章 CAD 控件及指令

当前关于机床加工大部分的流程都是得到加工的矢量图形（如 dxf），通过 PC 的相关导图软件生成相应的加工代码，甚至还要在 PC 上对加工代码进行修改，最后才将加工代码下载到机床控制器中进行加工。该流程通常离不开 PC 机，用户在使用机床加工时，不可避免要使用 PC 去生成加工代码。



针对一些不需要 PC 的机床加工场景，使得机床加工彻底摆脱 PC 平台的依赖，降低成本，简化机床加工流程，我们需要将 CAD 导图功能集成在机床（控制器）中，用户只需要获得加工的矢量图形，直接在机床（控制器）中进行简单的操作就能直接导出加工代码进行加工，甚至不需要导出加工代码，直接加工矢量图形（dxf）。流程如下图：

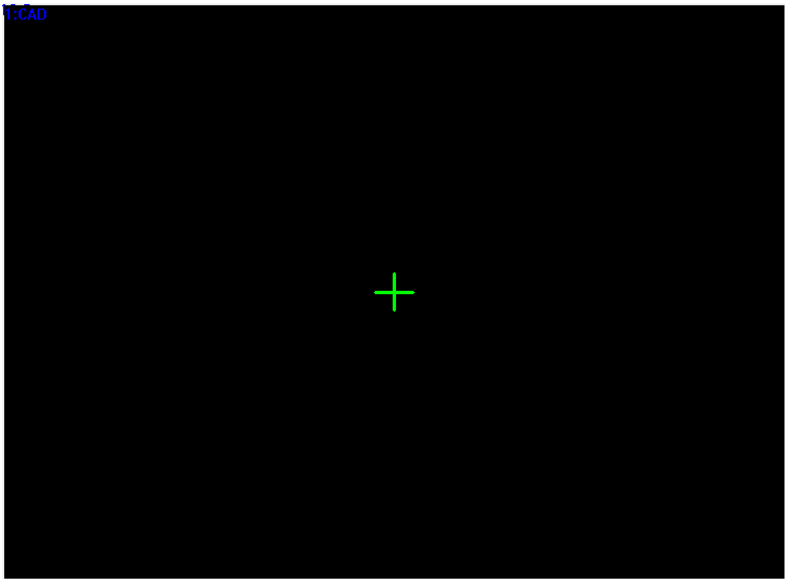


由于导图功能集成在机床（控制器）中，直接导出的加工代码能与机床有着高兼容性，导出即可用的特点，用户一般无需再对加工代码进行修改调整，十分方便。

由于操作平台一般在**控制器+触摸屏**上，因此需要提供**很简单的操作**就能够**实现一般的 CAD 导图功能**（尽量去除一些繁琐的不通用配置操作），同时还要预留出**可编辑接口**及**必要的参数配置接口**（高级功能），以供用户选择使用。

该功能当前仅支持 5 系列的控制器。

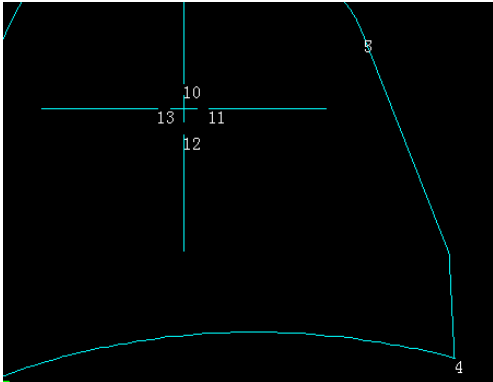
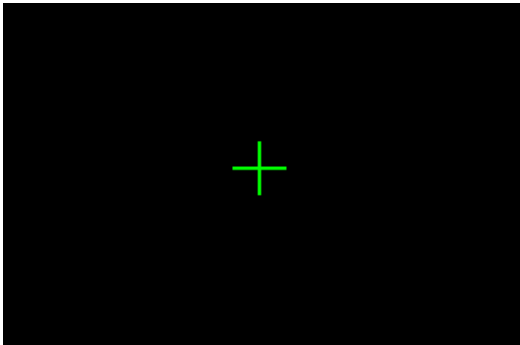
21.1 CAD 控件说明



基本属性：编号、名称、层次、开始位置 X、开始位置 Y、宽度、高度、有效显示、有效控制（为 TRUE 时有属性：设备编号、寄存器类型、寄存器编号）。

其他属性：

属性	描述
通道号	默认为 0。 第一版本只提供一个通道。
是否边框	为 TRUE 时有属性： 边框颜色 、 边框宽度 。
背景颜色	RGB 背景颜色尽量与其他颜色有高对比度。
是否使用图层颜色	0-1（false/true） 当=0（false）时，轨迹颜色属性生效； 当=1（true）时，使用系统内置的图层颜色，忽略轨迹颜色；
轨迹颜色	RGB，默认天蓝色 该属性生效时，所有图层都使用轨迹颜色
空移颜色	RGB，默认红色 两个连续轨迹段之间的连接线段使用空移颜色
选中颜色	RGB，默认黄色

格式文本 (标记)	格式文本（只有一个状态） 字体颜色作为连续曲线的起点、顺序编号的显示颜色，字体大小、字体作用于顺序编号的显示。 格式文本的对齐、背景颜色、样式等属性无效，只有字体颜色、字体大小、字体这些属性有效。 
辅助颜色	RGB，默认绿色 如原点坐标轴，框选框等的显示颜色 

21.2 CAD 指令说明 -- 基础指令

通过基础指令可实现 G 代码或 Basic 的一些基本 CAD 导图应用。

CAD_CHANNEL -- CAD 通道

当前版本只提供一个 CAD 通道，即用户使用多个 CAD 控件，最终也只作用于同一个 CAD 通道，即多个 CAD 控件也可作为同一个 CAD 通道的不同可视窗口。

类型	CAD 指令
描述	设置/获取当前 CAD 通道，切换通道后，所有 CAD 指令将只对切换后的通道进行操作。预留。
语法	<pre>CAD_CHANNEL = channel channel = CAD_CHANNEL</pre> channel: 当前 CAD 通道，缺省为 0
适用控制器	支持 ZHMI

例子	CAD_CHANNEL = 0 '切换通道 0 ?CAD_CHANNEL '==> 0
----	--

CAD_LOADFILE -- 从文件中载入图片

类型	CAD 指令
描述	设置/获取当前加载图形文件，在 CAD 控件内显示出来 支持格式：dxf、plt、dst、ai、nc、cnc 当前版本不支持加载大文件，限制 10M 以内，5M 以上的图形文件加载时间过长可能会出现断连现象，重新连接即可。建议都使用 5M 以下的图形文件。
语法	CAD_LOADFILE(dxfname [, reldis[, layers]]) dxfname = CAD_LOADFILE () dxfname: dxf 文件名，带路径 reldis: 导入复杂曲线分割精度，范围[0.01~0.5] 该分割精度决定导入复杂曲线时是否自动分割为线段，reldis 缺省或=0 时，不自动分割曲线，反之则根据 reldis 精度进行曲线分割 layers: 选择加载图层，bit0~bit31 位有效，支持最大 32 个图层 缺省加载所有图层（加载 G 代码文件时无效）
适用控制器	支持 ZHMI
例子	例子 1: CAD_LOADFILE ("zheng.dxf") '加载 "zheng.dxf" 文件 ? CAD_LOADFILE () ' => zheng.dxf 例子 2 CAD_LOADFILE ("") '加载文件不存在，[CAD]显示内容清空 ? CAD_LOADFILE () ' => 例子 3: CAD_LOADFILE ("zheng.dxf", 0.05, 1) '参考精度=0.05，仅加载图层 1 ? CAD_LOADFILE () ' => zheng.dxf

CAD_LOADTABLE -- 从 TABLE 中载入图形

类型	CAD 指令
描述	从 TABLE 中加载图形数据，可参考 TABLE 数据格式对 TABLE 数据进行调整再进行导入。
语法	CAD_LOADTABLE (tabid) tabid = CAD_LOADTABLE () tabid: table 下标
适用控制器	支持 ZHMI
例子	CAD_LOADTABLE (10000) '从 TABLE(10000)开始加载图形

	? CAD_LOADTABLE ()	' => 10000
--	--------------------	------------

CAD_LOADBIN -- 从 BIN 文件中载入图形

类型	CAD 指令
描述	从 BIN 文件中加载图形数据，数据格式可参考 TABLE 数据格式。
语法	CAD_LOADBIN(filename, type) filename = CAD_LOADBIN () filename: bin 文件名，带路径 type: BIN 文件数据格式 0 - float 类型，每一项 4 字节 1 - double 类型，每一项 8 字节
适用控制器	支持 ZHMI
例子	CAD_LOADBIN("ZHENG.BIN", 0) '载入 BIN 文件，float 类型。

CAD_CLOSE -- 关闭 CAD 加载图形

类型	CAD 指令
描述	关闭当前加载的 CAD 图形文件
语法	CAD_CLOSE()
适用控制器	支持 ZHMI
例子	CAD_LOADFILE ("zheng.dxf") '加载 "zheng.dxf" 文件 CAD_CLOSE() '关闭当前加载的 "zheng.dxf" 文件

CAD_TOCODE -- 导出代码文件（NC/Z3P）

类型	CAD 指令
描述	当前加载图形导出代码文件 支持导出 NC 文件和 Z3P 文件。
语法	CAD_TOCODE(filename, type, layers) filename: 导出代码文件名 xxx.Z3P 输出 basic 的 Z3P 文件 xxx.nc 输出 G 代码的 NC 文件 当 filename="" 时，直接输出到当前 3 次文件加载区 通过 FILE3_CHANNEL 指令指定输出到哪个三次文件通道 type: 导出代码类型，0-Z3P（Basic），1-NC（G 代码） layers: 导出图层，bit0~bit31 位有效，-1 表示导出所有图层 如 layers=1+2+4+8，表示导出图层 0、1、2、3
适用控制器	支持 ZHMI

例子	<pre> CAD_LOADFILE("zheng.dxf") '加载 "zheng.dxf" 文件 CAD_TOCODE("zheng.nc", 1,-1) '导出 zheng.nc 文件 ' 输出 NC 代码到 3 次文件（通道 1）加载区上，若加载区大小不够则报错 FILE3_CHANNEL = 1 CAD_TOCODE("",1,-1) ' </pre>
----	--

CAD_TABSIZE -- 占用 table 空间

类型	CAD 指令
描述	获取当前加载的矢量图形导出 table 时需要多大的 table 空间（table 个数），在导出 table 时可以判断 table 空间是否充足。
语法	<pre> tabsize = CAD_TABSIZE (layers) </pre> layers: 导出图层，bit0~bit31 位有效，-1 表示导出所有图层 tabsize: 返回矢量图形导出 table 时所需空间大小
适用控制器	支持 ZHMI
例子	<pre> CAD_LOADFILE("zheng.dxf") '加载 "zheng.dxf" 文件 Tabsize = CAD_TABSIZE (-1) '获取 zheng.dxf 图形导出 table 所需大小 </pre>

CAD_TOTABLE -- 导出 TABLE 数据

类型	CAD 指令
描述	将当前加载的图形文件按标准格式输出到 table 中，用户可自行取 table 中的数据进行运动。 确认 table 中有足够的空间存储 dxf 轨迹。
语法	<pre> CAD_TOTABLE(datatabid, layers) </pre> datatabid: table 地址，输出 table 数据 table 存储数据格式如下：详看 TABLE 数据格式设计 小节。 layers: 导出图层，bit0~bit31 位有效，-1 表示导出所有图层 如 layers=1+2+4+8，表示导出图层 0、1、2、3
适用控制器	支持 ZHMI
例子	<pre> CAD_LOADFILE("zheng.dxf") '加载 "zheng.dxf" 文件 tableSize = CAD_TABSIZE(-1) '获取导出 Table 需要多大空间 if tableSize < 300000 then '不超过 30 万的都支持导出 CAD_TOTABLE(20000, -1) '导出到 table(20000)之后 endif CAD_CLOSE() '关闭加载图形 </pre>

CAD_TOBIN -- 导出 BIN 文件

类型	CAD 指令
描述	将当前加载的图形文件按标准格式输出到 BIN 文件中
语法	CAD_TOBIN(filename, type, layers) filename: 输出的 BIN 文件名称 type: BIN 文件数据类型 0 - float 类型, 每一项 4 字节 1 - double 类型, 每一项 8 字节 layers: 导出图层, bit0~bit31 位有效, -1 表示导出所有图层 如 layers=1+2+4+8, 表示导出图层 0、1、2、3
适用控制器	支持 ZHMI
例子	CAD_LOADFILE("zheng.dxf") '加载 "zheng.dxf" 文件 CAD_TOBIN (zheng.BIN, 0, -1) '导出 BIN 文件 CAD_CLOSE() '关闭加载图形

CAD_TAB2CODE -- TABLE 数据转代码文件

类型	CAD 指令
描述	将 table 中的图形轨迹导出代码文件。 确认 table 中存储的 dxf 轨迹是否正常, 支持导出 NC 文件和 Z3P 文件。
语法	CAD_TAB2CODE(filename, type, tabid, layers) filename: 导出代码文件名 type: 导出代码类型, 0-Z3P (Basic), 1-NC (G 代码) tabid: table 索引 layers: 导出图层, bit0~bit31 位有效, -1 表示导出所有图层 如 layers=1+2+4+8, 表示导出图层 0、1、2、3
适用控制器	支持 ZHMI
例子	'将 table(3000)存储的图形导出 G 代码 CAD_TAB2CODE("xxx.nc", 1, 3000, -1)

CAD_PARA -- 参数设置

类型	CAD 指令
描述	CAD 工艺参数设置。 当 bIfUseSel 缺省或=0 时, 该指令作用于全局, 反之作用于选择[CAD]选中部分。 注: 导出 G 代码和导出 Basic 不一样, G 代码不需要设置轴号、空移速度、加速度、减速度等。
语法	CAD_PARA(para, setval [,bIfUseSel]) para: 设置参数编号 详见 CAD 参数表。

	setval: 参数设置值 bIfUseSel: 预留, 是否作用于当前选中部分, 缺省 0 则作用于全局 setval = CAD_PARA(para [,bIfUseSel]) para: 设置参数命令, 同上 bIfUseSel: 预留, 是否作用于当前选中部分, 缺省 0 则作用于全局 setval: 返回最新设置参数值
适用控制器	支持 ZHMI
例子	CAD_PARA(12, 1) '设置参数 12 (是否允许反向)=1 ?CAD_PARA(12) '==> 1

CAD_DISPLAY -- 显示方式设置

类型	CAD 指令
描述	CAD 显示方式设置。
语法	CAD_DISPLAY(mode) mode = CAD_DISPLAY() mode: 显示模式上电默认为 0 bit0: 1, 是否显示不封闭图形 bit1: 2, 是否显示空移 bit2: 4, 是否显示顺序 bit3: 8, 是否显示起点 bit4: 16, 是否显示方向 bit5: 32, 动作点是否高亮
适用控制器	支持 ZHMI
例子	CAD_DISPLAY(2+8) '显示空移+起点

CAD_MOUSE -- 鼠标模式设置

类型	CAD 指令
描述	CAD 鼠标控制模式
语法	CAD_MOUSE(mode) mode = CAD_MOUSE() mode: 鼠标控制模式, 上电默认为 0 0: 视图平移 1: 视图缩放, 向上滑动=>放大, 向下滑动=>缩小 2: 选择轨迹, 鼠标点击或划过区域的轨迹将处于选中状态 3: 原点设置, 鼠标点击位置设为原点 注: mode=2 选择轨迹可分两种框选方式, 向右边框选需要完全包含该图形才会选中, 向左边框选只需要包含一部分即可选中。
适用控制器	支持 ZHMI

例子	CAD_MOUSE(2) '设置鼠标控制选择轨迹模式
----	----------------------------

CAD_AUTOVIEW -- 自适应视图

类型	CAD 指令
描述	自动适应显示视图，用户可通过此指令自动居中显示、视野范围缩放显示。
语法	CAD_AUTOVIEW (winId, controlId [,offX, offY [, fZoom [,plane [,RotX, RotY, RotZ]]]]) winId, controlId: 要设置的控件所处窗口号以及及控件编号 offX,offY: 视图偏移，缺省(0,0)视图居中显示 fZoom: 缩放比例，缺省 1.0 ，范围内刚好显示 以下属性为预留三维 CAD 图形：（当前版本不支持） plane: 显示平面，缺省 0-XY, 1-YZ, 2-ZX RotX,RotY,RotZ: 绕 XYZ 轴旋转角度，0~360，缺省 0。
适用控制器	支持 ZHMI
例子	'自动视图范围内完整居中显示所有图形， CAD 控件在窗口 10，编号 1 CAD_AUTOVIEW(10, 1)

CAD_RANGE -- 获取轨迹范围

类型	CAD 指令												
描述	获取当前加载或指定 TABLE 的图形数据的范围												
语法	CAD_RANGE (tabout [,tabin]) tabout: 结果输出到 TABLE(tabout)~TABLE(tabout+5)中 <table border="1"> <tr> <td>TABLE(tabout)</td><td>图形轨迹最小 X 轴位置</td></tr> <tr> <td>TABLE(tabout+1)</td><td>图形轨迹最小 Y 轴位置</td></tr> <tr> <td>TABLE(tabout+2)</td><td>图形轨迹 X 轴宽度</td></tr> <tr> <td>TABLE(tabout+3)</td><td>图形轨迹 Y 轴宽度</td></tr> <tr> <td>TABLE(tabout+4)</td><td>图形轨迹最小 Z 轴位置，预留</td></tr> <tr> <td>TABLE(tabout+5)</td><td>图形轨迹 Z 轴高度，预留</td></tr> </table> tabin: TABLE 图形数据的起始位置，缺省为-1 注：若 tabin 为无效的 table 下标，则以当前 CAD 加载的图形数据作为参考，反之则以该 TABLE 图形数据作为参考	TABLE(tabout)	图形轨迹最小 X 轴位置	TABLE(tabout+1)	图形轨迹最小 Y 轴位置	TABLE(tabout+2)	图形轨迹 X 轴宽度	TABLE(tabout+3)	图形轨迹 Y 轴宽度	TABLE(tabout+4)	图形轨迹最小 Z 轴位置，预留	TABLE(tabout+5)	图形轨迹 Z 轴高度，预留
TABLE(tabout)	图形轨迹最小 X 轴位置												
TABLE(tabout+1)	图形轨迹最小 Y 轴位置												
TABLE(tabout+2)	图形轨迹 X 轴宽度												
TABLE(tabout+3)	图形轨迹 Y 轴宽度												
TABLE(tabout+4)	图形轨迹最小 Z 轴位置，预留												
TABLE(tabout+5)	图形轨迹 Z 轴高度，预留												
适用控制器	支持 ZHMI												
例子	CAD_RANGE(0) '获取当前加载图形的范围 ? "left = ", table(0) ? "bottom = ", table(1) ? "width = ", table(3) ? "height = ", table(4)												

21.3 CAD 指令说明 --自定义控件扩展指令

CAD_SETCOLOR -- 设置 CAD 绘制颜色

类型	CAD 指令
描述	自定义控件内绘制当前 CAD 加载的图形 只能在自定义控件的刷新/绘图函数上使用。
语法	CAD_SETCOLOR (type, color) type: 颜色类型 0 – 轨迹颜色, 默认=-1 使用内置图层颜色 1 – 空移颜色, 默认=红色 RGB(255, 0, 0) 2 – 选中颜色, 默认=黄色 RGB(255, 215, 0) 3 – 标记颜色 (起点/顺序), 默认=白色 RGB(255, 255, 255) color: 设置颜色 注: 只有轨迹颜色设置为-1 有效, 可以使用内置图层颜色。
适用控制器	支持 ZHMI
例子	CAD_SETCOLOR(1, RGB(255,0,0)) '空移轨迹设为红色

CAD_DRAW -- 绘制 CAD 图形

类型	CAD 指令
描述	自定义控件内绘制当前 CAD 加载的图形 只能在自定义控件的绘图函数上使用。 由指令 CAD_DISPLAY 控制显示模式。
语法	CAD_DRAW (offsetX, offsetY, fZoom [,plane, RotX, RotY, RotZ]) offset1, offset2: 显示图形偏移, =(0,0)时, 图形中心处于自定义控件中心位置 fZoom: 显示缩放比例, 1.0=自定义控件范围刚好完整显示加载图形 注: 缩放比例范围为 0.02~1000.0 以下属性为预留三维 CAD 图形: plane: 显示平面, 缺省 0-XY, 1-YZ, 2-ZX RotX, RotY, RotZ: 绕 XYZ 轴旋转角度, 0~360, 缺省 0。
适用控制器	支持 ZHMI
例子	CAD_DRAW(0, 0, 1.0) '自定义控件上居中显示加载 CAD 图形

CAD_DRAWTAB -- 从 TABLE 中绘制 CAD 图形

类型	CAD 指令
描述	自定义控件内绘制 TABLE 存储的图形 只能在自定义控件的绘图函数上使用。

	由指令 CAD_DISPLAY 控制显示模式，但不支持 bit0、bit5 模式。
语法	CAD_DRAWTAB (tabid , offsetX, offsetY, fZoom [,plane, RotX, RotY, RotZ]) tabid: table 下标, 绘制 table 下标处存储的图形。 offset1, offset2: 显示图形偏移, =(0,0)时, 图形中心处于自定义控件中心位置 fZoom: 显示缩放比例, 1.0=自定义控件范围刚好完整显示加载图形 以下属性为预留三维 CAD 图形: plane: 显示平面, 缺省 0-XY, 1-YZ, 2-ZX RotX, RotY, RotZ: 绕 XYZ 轴旋转角度, 0~360, 缺省 0。
适用控制器	支持 ZHMI
例子	'自定义控件上居中显示 TABLE 图形 CAD_DRAWTAB(3000, 0, 0, 1.0)

CAD_TRANSPOINT -- 转换坐标为控件位置

类型	CAD 指令
描述	将数据坐标转换为当前自定义控件的像素位置(X,Y) 只能在自定义控件的绘图函数上使用。
语法	CAD_TRANSPOINT (tabout, x, y, z, offsetX, offsetY, fZoom [,tabin] [,plane, RotX, RotY, RotZ]) tabout: 结果输出到 TABLE(tabout)=X, TABLE(tabout+1)=Y x,y,z: 需要转换的坐标, z 轴预留三维功能 offset1, offset2: 显示图形偏移, =(0,0)时, 图形中心处于自定义控件中心 fZoom: 显示缩放比例, 1.0=自定义控件范围刚好完整显示加载图形 tabin: 缺省以当前加载数据作为参考, 否则以该 TABLE 数据作为参考 以下属性为预留三维 CAD 图形: plane: 显示平面, 缺省 0-XY, 1-YZ, 2-ZX RotX, RotY, RotZ: 绕 XYZ 轴旋转角度, 0~360, 缺省 0。
适用控制器	支持 ZHMI
例子	例子 1: CAD_DRAW(g_offsetX, g_offsetY, g_fZoom) '绘制当前加载图形 '以当前加载图形参考, 获取坐标 (10,10, 0) 在自定义控件的位置 CAD_TRANSPOINT(0, 10, 10, 0, g_offsetX, g_offsetY, g_fZoom)

CAD_TRANSPOINT2 -- 转换控件位置为图形坐标

类型	CAD 指令
描述	将当前自定义控件的像素位置(px, py)转换为数据坐标 只能在自定义控件的刷新/绘图函数上使用。
语法	CAD_TRANSPOINT2 (tabout, px, py, centerX, centerY, fZoom [,tabin] [,plane, RotX, RotY, RotZ]) tabout: 结果输出到 TABLE(tabout)=X, TABLE(tabout+1)=Y

	px, py: 需要转换的像素位置 centerX, centerY: 视图中心偏移, =(0,0)时, 图形中心处于自定义控件中心位置 fZoom: 显示缩放比例, 1.0=自定义控件范围刚好完整显示加载图形 tabin: TABLE 图形数据的起始位置, 缺省为-1 注: 若 tabin 为无效的 table 下标, 则以当前 CAD 加载的图形数据作为参考, 反之则以该 TABLE 图形数据作为参考 以下属性为预留三维 CAD 图形: plane: 显示平面, 缺省 0-XY, 1-YZ, 2-ZX RotX, RotY, RotZ: 绕 XYZ 轴旋转角度, 0~360, 缺省 0。
适用控制器	支持 ZHMI
例子	例子 1: CAD_DRAW(g_offsetX, g_offsetY, g_fZoom) '绘制当前加载图形 '以当前加载图形参考, 获取坐标 (10,10, 0) 在自定义控件的位置 CAD_TRANSPOINT2(0, 10, 10, g_offsetX, g_offsetY, g_fZoom)

CAD_SELECT -- 选中图形

类型	CAD 指令
描述	在自定义控件内对当前 CAD 加载的图形进行点选、框选 只能在自定义控件的刷新函数上使用。
语法	CAD_SELECT (ix1, iy1, ix2, iy2, offsetX, offsetY, fZoom[,plane, RotX, RotY, RotZ]) ix1,iy1, ix2,iy2: 坐标的, 选择框两个对顶点坐标 (两点成矩形) offsetX, offsetY: 显示图形偏移, =(0,0)时, 图形中心处于自定义控件中心位置 fZoom: 显示缩放比例, 1.0=自定义控件范围刚好完整显示加载图形 以下属性为预留三维 CAD 图形: plane: 显示平面, 缺省 0-XY, 1-YZ, 2-ZX RotX, RotY, RotZ: 绕 XYZ 轴旋转角度, 0~360, 缺省 0。
适用控制器	支持 ZHMI
例子	CAD_SELECT(10, 10, 90, 90, 0, 0, 1.0) '框选(10,10)→(90,90)区域

21.4 CAD 指令说明 -- 高级指令

高级指令提供对 CAD 导图数据进行一定的修改功能。

提供智能处理指令, 快速根据不同行业工艺自动导出相应数据代码。

CAD_MACHTYPE -- 机器类型

针对不同的非标设备时, CAD 只需要根据用户其特殊结构需求, 底层增加一个机器类型即可。支持之后, 无需用户其他繁琐的配置, 使用该指令设置一下对应机器类型即可, 在导出生成代码时会自动根据机械类型生成相匹配的加工代码, 用户导出即可加工。

类型	CAD 指令
----	--------

描述	设置机器类型，CAD 会根据机器类型智能地输出相对应的加工代码。						
语法	CAD_MACHTYPE (type [, strAxes [,axis1[,axis2[,...]]]]) type: 机器类型 <table border="1"> <tr> <td>0</td><td>通用三轴机床 (XYZ)，默认</td></tr> <tr> <td>1</td><td>特殊三轴钻孔机床 (XYZ)，自动过滤曲线，只导出钻孔相关代码。</td></tr> <tr> <td>11</td><td>通用二轴机床 (XY)</td></tr> </table> 需要增加其他机器类型可联系我们技术研发。 strAxes: 重新指定导出轴名称，导出 G 代码时有效，可缺省。 如指定四轴机床类型时，指定 strAxes = "XYZC" 表示使用直线轴使用 XYZ，旋转轴使用 C 建议使用 XYZABCUVW 这九个轴名称，目前支持 A~Z 字母 Axis1~N: 重新指定 basic 轴号，导出 basic 时有效，缺省为 0,1,2,...	0	通用三轴机床 (XYZ)，默认	1	特殊三轴钻孔机床 (XYZ)，自动过滤曲线，只导出钻孔相关代码。	11	通用二轴机床 (XY)
0	通用三轴机床 (XYZ)，默认						
1	特殊三轴钻孔机床 (XYZ)，自动过滤曲线，只导出钻孔相关代码。						
11	通用二轴机床 (XY)						
适用控制器	支持 ZHMI						
例子	’设置为通用三轴类型，并制定坐标轴名称为 XYZ，绑定轴号 0,1,2 CAD_MACHTYPE(0, "XYZ", 0, 1, 2)						

CAD_OPTIMIZE -- 轨迹优化

类型	CAD 指令
描述	对 dxf 图形轨迹进行优化处理 当 bIfUseSel 缺省或=0 时，该指令作用于全局，反之作用于选择[CAD]选中部分。
语法	CAD_OPTIMIZE(option, optval [,bIfUseSel]) option: 优化选项 0: 去除极小图形最大尺寸 (a×a) 1: 是否去除重复线, 0 或 1 2: 合并相连线, 可合并最大距离 3: 曲线分割, 输入曲线分割精度 (0.01~0.5) 4: 优化空移, 输入参考精度 (0.01~0.5) 5: 路径排序 0-不排序 1-网格排序 2-局部最小空移排序 3-小图优先排序 4-从里到外排序 5-从左到右排序 6-从右到左排序 7-从上到下排序 8-从下到上排序 9-反向排序 6: 曲线平滑 (预留)，输入平滑精度 7: 路径反向, 输入参数无效

	optval: 优化值, 根据优化选项赋值 bIfUseSel: 是否指定选中部分, 缺省 0 忽略选择部分, 作用于全局
适用控制器	支持 ZHMI
例子	CAD_OPTIMIZE(3, 0.05) '曲线分割, 参考精度为 0.05

CAD_EDIT -- 自定义编辑

考虑自动导出代码时对于一些工艺考虑不到, 该指令可以给用户自己进行补充。

类型	CAD 指令
描述	CAD 自定义编辑输出文件, 向指定编辑位置插入动作字符串 当 CAD_PARAM(13)=1 时, 仅编辑选中部分的轨迹, 反之则作用于全局。
语法	CAD_EDIT (EditPos, action) EditPos: 编辑位置 0: 文件/选中区域起始位置 1: 文件/选中区域结束位置 2: 文件/选中区域的每一连续段起始位置 3: 文件/选中区域的每一连续段结束位置 action: 编辑插入动作 (编号, 需要先创建动作数据) 注: 动作编号从 1 开始, 0 可表示为空动作, 即插入动作 0 也可表示为删除动作 action = CAD_EDIT (EditPos [,bIfUseSel]) EditPos: 编辑位置, 同上 返回 action: 最新编辑插入动作 (编号)
适用控制器	支持 ZHMI
例子	CAD_PARAM(13, 0) CAD_EDIT(0, 10) '向整个图形的起始位置插入动作 10

CAD_MOVE -- 轨迹平移

类型	CAD 指令
描述	CAD 导入轨迹进行平移 当 CAD_PARAM(13)=1 时, 仅编辑选中部分的轨迹, 反之则作用于全局。
语法	CAD_MOVE(x, y [,z]) x,y: 相对平移坐标 z: 预留三维 CAD 图形
适用控制器	支持 ZHMI
例子	CAD_MOVE(100, 0) '向右平移 100

CAD_SCALE -- 轨迹缩放

类型	CAD 指令
描述	CAD 导入轨迹进行缩放 当 CAD_PARA(13)=1 时，仅编辑选中部分轨迹，反之则作用于全局。
语法	CAD_SCALE(fscaleX, fscaleY [, x, y[,z]]) fscaleX: X 方向缩放比例 fscaleY: Y 方向缩放比例 x,y: 缩放中心位置，缺省为图形中心 z: 预留三维 CAD 图形
适用控制器	支持 ZHMI
例子	CAD_SCALE(2.0, 2.0) '放大两倍

CAD_ROTATE -- 轨迹旋转

类型	CAD 指令
描述	CAD 导入轨迹进行缩放 当 CAD_PARA(13)=1 时，仅编辑选中部分轨迹，反之则作用于全局。
语法	CAD_ROTATE(fRotAngle [, center1, center2 [,plane]]) fRotAngle: 旋转角度，弧度值 center1, center2: 旋转中心位置，缺省为图形中心 以下预留三维 CAD 图形： plane: 旋转平面，缺省 0-XY, 1-YZ, 2-ZX
适用控制器	支持 ZHMI
例子	CAD_ROTATE(90*PI/180) '绕图形中心旋转 90 度

CAD_DELETE -- 轨迹删除

类型	CAD 指令
描述	选择图形轨迹进行删除 当 CAD_PARA(13)=1 时，仅编辑选中部分轨迹，反之则作用于全局。
语法	CAD_DELETE()
适用控制器	支持 ZHMI
例子	CAD_PARA(13, 1) CAD_DELETE() '删除选中轨迹

CAD_ACTION -- 获取/修改动作数据

类型	CAD 指令
----	--------

描述	获取/修改指定动作编号的动作数据 动作编号从 1 开始计数。
语法	CAD_ACTION(action, strAct) action: 指定动作编号 strAct: 写入修改的动作数据（字符串，可支持多行） strAct = CAD_ACTION(action) action: 指定动作编号 返回 strAct: 动作数据（字符串）
适用控制器	支持 ZHMI
例子	CAD_ACTION(10,“动作 10”) ’修改动作 10 内容 ? CAD_ACTION(10) ’打印动作 10 内容

CAD_ACTIONADD -- 添加动作

类型	CAD 指令
描述	添加动作数据，操作成功返回动作编号，反之返回-1
语法	action = CAD_ACTIONADD(strAct) strAct: 动作数据（字符串，可支持多行） 返回 action: 动作编号
适用控制器	支持 ZHMI
例子	action = CAD_ACTIONADD(“M7 ;开水”) ’添加开水动作，编号为 action

CAD_ACTIONDEL -- 删除动作数据

类型	CAD 指令
描述	删除指定动作编号的动作数据
语法	CAD_ACTIONDEL([action]) action: 指定动作编号，缺省为删除全部
适用控制器	支持 ZHMI
例子	CAD_ACTIONDEL(10) ’删除动作 10

CAD_UNDO -- 撤销上一步编辑操作

类型	CAD 指令
描述	撤销上一步编辑操作
语法	CAD_UNDO()
适用控制器	支持 ZHMI

例子	无
----	---

CAD_REDO -- 重做上一步撤销动作

类型	CAD 指令
描述	撤销上一步编辑操作
语法	CAD_REDO()
适用控制器	支持 ZHMI
例子	无

CAD_SORT -- 手动排序

类型	CAD 指令												
描述	对当前选中的图形进行手动排序 注：只能手动排序当前已选中的图形，该指令暂不支持撤销重做。												
语法	CAD_SORT(mode [,seqnum]) mode – 排序模式 <table border="1"> <tr> <td>0</td><td>手动设置选中图形顺序（序号） 注：仅支持单选，选中多个图形时，仅操作序号在最前面的图形曲线</td></tr> <tr> <td>1</td><td>选中图形曲线向前移动一个</td></tr> <tr> <td>2</td><td>选中图形曲线向后移动一个</td></tr> <tr> <td>3</td><td>选中图形曲线移到最前面</td></tr> <tr> <td>4</td><td>选中图形曲线移到最后面</td></tr> <tr> <td>5</td><td>选中图形反序 注：多选时有效</td></tr> </table> seqnum – 可选，mode=0 时有效，设置选中图形的顺序号	0	手动设置选中图形顺序（序号） 注：仅支持单选，选中多个图形时，仅操作序号在最前面的图形曲线	1	选中图形曲线向前移动一个	2	选中图形曲线向后移动一个	3	选中图形曲线移到最前面	4	选中图形曲线移到最后面	5	选中图形反序 注：多选时有效
0	手动设置选中图形顺序（序号） 注：仅支持单选，选中多个图形时，仅操作序号在最前面的图形曲线												
1	选中图形曲线向前移动一个												
2	选中图形曲线向后移动一个												
3	选中图形曲线移到最前面												
4	选中图形曲线移到最后面												
5	选中图形反序 注：多选时有效												
适用控制器	支持 ZHMI												
例子	CAD_SORT(3) 将选中图形移到最前面												

21.5 CAD 参数表

通用参数

参数编号	说明	范围限制	初始值
1	运动方式：0-绝对运动，1-相对运动	[0, 1]	0
2	机器类型 0- 通用三轴机床（XYZ）	-	0

	1- 特殊三轴机床，过滤曲线，只钻孔 11- 通用二轴机床（XY） 12- 通用二轴机床（ZX），预留 101- 特殊四轴机床，刀尖角度自动跟随切削方向，预留		
3	加工速度，切削图形时的速度 • 导出 G 代码时单位：mm/min • 导出 basic 时单位：units/s	[0,999999]	200.0
4	加工高度	[-999999,999999]	1.0
5	加工中是否上抬	[0,1]	0
6	上抬高度	[-999999,999999]	10.0
7	加工前后是否回机械原点	[0,1]	0
8	圆弧是否转小线段	[0,1]	0
9	曲线分割参考精度，0.01~0.5	[0.01,0.5]	0.05
10	加工平面，0-XY，1-YZ，2-ZX 预留	[0,2]	0
11	加工平面半径补偿值，计算到导出代码中 0 不补偿，>0 表示右补偿，<0 表示左补偿 预留	[-999999,999999]	0
12	排序优化是否允许反向	[0,1]	0
13	编辑功能是否仅作用于选中部分 限 相 关 指 令： CAD_MOVE、CAD_SCALE、 CAD_ROTATE、CAD_DELETE、CAD_EDIT	[0,1]	0
14	导出 TABLE、BIN 文件时是否仅保留轨迹部分	[0,1]	0
20	加工前动作编号	[0,999999]	0
21	加工后动作编号	[0,999999]	0
50	钻孔工艺，R 点高度（Z 坐标）	[-999999,999999]	5.0
51	钻孔工艺，钻孔深度（Z 坐标）	[-999999,999999]	0.0
52	钻孔速度（下钻）	[0,999999]	200.0
53	出孔速度（上钻），设置为 0 则以空移速度进行出孔	[0,999999]	0
54	孔底动作编号，下钻到孔底时执行的动作	[0,999999]	0

G 代码参数

参数编号	说明	限制	初始值
101	是否导出 N 行号	[0,1]	0
102	工作刀号，预留	[0,999999]	0
103	刀具半径补偿号，预留	[0,999999]	0
104	刀具长度补偿号，预留	[0,999999]	0
105	刀具半径补偿方向，1 右补偿（G42），-1 左补偿（G41）， 0 不补偿	[-1,1]	0

	预留		
106	刀具长度补偿方向, 1 正向补偿 (G43), -1 负向补偿 (G44), 0 不补偿 预留	[-1,1]	0
120~136	轴名称	['A'~'Z']	XYZ

Basic 参数

参数编号	说明	限制	初始值
201	加工加速度	[-999999,999999]	1000.0
202	加工减速度	[-999999,999999]	1000.0
203	空移速度	[-999999,999999]	1000.0
204	空移加速度	[-999999,999999]	1000.0
205	空移减速度	[-999999,999999]	1000.0
220~236	控制器轴号	[0~127]	0~2

21.6 TABLE 数据格式说明

将图形加工据按照一定格式导出存储在 TABLE 指定位置, 供 basic 直接读取加工。

数据格式设计

tabid 为存储导出 table 数据的 table 下标, 其导出数据存储格式如下:

tabid	tabid+1	tabid+2	tabid+3 ~ taid+3+N	...	...	...	...
TableSize	DataType	ParaSize	Para1~ParaN	...	DataType	ParaSize	Para1~ParaN

第一个 table 值为 TableSize, 即存储导出 Table 数据占用大小 (包含 TableSize 的储存)。

第二个 table 值开始存储着运动数据, 每一段运动数据的格式为: 数据类型 (DataType) + 参数个数 (ParaSize) + 所有参数值 (Para1~ParaN), 可根据当前的运动数据, 推断下一运动数据的地址, 直到全部读取完 (可通过 Tablesize 判断)。举例如下: (TableSize=10)

10	0	3	para1	para2	para3	1	2	para1	para2
----	---	---	-------	-------	-------	---	---	-------	-------

DateType 定义

DateType (整型) 包含标记类型、运动类型、参数设置类型等。

1. 标记类型

0~99 为标记类型，主要是用于辅助的一些数据，用户可以忽略。

用户也可根据标记类型指定的动作编号，在解析 table 数据时自己选择插入哪些特殊处理。

DateType	类型说明	参数说明
0	结束标记	ParaSize: 参数个数=0 表示 TABLE 数据结束。 TABLE 数据读取过程可以通过 TableSize 判断是否结束，也可以通过结束标记（0）去判断是否结束
1	标记图形开始位置	ParaSize: 参数个数 N（0~1） N=0 时表示无参数，不指定动作数据 N>0 时表示有指定动作数据（预留） para1: 指定动作编号（预留）
2	标记图形结束位置	ParaSize: 参数个数 N（0~1） para1: 指定动作编号（预留）
3	标记每一段连续线段的开始位置	ParaSize: 参数个数 N（0~1） para1: 指定动作编号（预留）
4	标记每一段连续线段的结束位置	ParaSize: 参数个数 N（0~1） para1: 指定动作编号（预留）
5	标记钻孔孔底位置	ParaSize: 参数个数 N（0~1） para1: 指定动作编号
20	图层编号	ParaSize: 参数个数 1 para1: 图形所处图层（1~32）
21	图形轨迹范围	ParaSize: 参数个数 N（4~6） para1: 最小 X 坐标 para2: 最小 Y 坐标 para3: X 方向宽度 para4: Y 方向宽度 para5: 最小 Z 坐标（预留） para6: Z 方向高度（预留）

其他标记类型根据需求扩展。

2. 运动类型

100~999 为运动类型，一般导出数据主要以线段、圆弧段为主，其他运动类型可能考虑特殊图形下预留。

DateType	类型说明	参数说明
100	空移运动	ParaSize: 参数个数 N para1~paraN: 直线运动数据 参考: MOVE(para1) AXIS(0) MOVE(para2) AXIS(1) MOVE(para3) AXIS(3)

101	直线运动	ParaSize: 参数个数 N para1~paraN: 直线运动数据 参考: MOVE(para1, para2, ..., paraN) MOVEABS(para1, para2, ..., paraN)
102	圆弧运动	ParaSize: 参数个数 N (5~6) para1, para2: 终点坐标 para3, para4: 圆心坐标 para5: 0-逆时针, 1-顺时针 para6: 可缺省, 第三个轴的运动距离, 需要螺旋时填入 参考: 根据参数个数选择不同指令 MOVECIRC (para1, para2, para3, para4, para5) MHELICAL (para1, para2, para3, para4, para5, para6, 1) MOVECIRCABS (para1, para2, para3, para4, para5) MHELICALABS (para1, para2, para3, para4, para5, para6, 1)
103	椭圆运动	ParaSize: 参数个数 N (7~8) para1, para2: 椭圆终点坐标 para3, para4: 椭圆圆心坐标 para5: 0-逆时针, 1-顺时针 para6, para7: 第一轴和第二轴的椭圆半径 para8: 可缺省, 第三个轴的运动距离, 需要螺旋时填入 参考: 根据参数个数适应指令参数 MECLIPSE (para1, para2, para3, para4, para5, para7) MECLIPSE(para1, para2, para3, para4, para5, para7, para8) MECLIPSEABS (para1, para2, para3, para4, para5, para7) MECLIPSEABS(para1, para2, para3, para4, para5, para7, para8)
104	运动延迟	ParaSize: 参数个数 N (1) para1: 运动延迟时间 ms 参考: MOVE_DELAY(para1)
200	回机械原点	ParaSize: 参数个数 N (0) 参考:
201	刀具高度定位	ParaSize: 参数个数 N (1) Para1: Z 轴高度 参考: MOVEABS(Para1) AXIS(2)

其他运动类型根据需求扩展。

如一些特殊机械类型的开关动作可在后面扩展。

当前运动类型不满足需求时，可以根据需求，再往后增加运动类型标准。

3. 运动参数类型

1000 以上为参数类型，主要控制运动数据的运行参数，一般需要用户以 MOVE_XXX 指令进行缓冲参数设置。

DateType	类型说明	参数说明
1000	设置相对/绝对模式	ParaSize: 参数个数 1 para1: 0-绝对模式，1-相对模式 参考：无
1001	设置速度	ParaSize: 参数个数 1~2 Para1: 速度值 Para2: 轴编号，缺省为所有轴或主轴 参考： MOVE_PARA(SPEED, para2, para1) 注：轴编号自动从 0 开始编，并不对应控制器轴号，需要外部应用自己做映射
1002	设置加速度	ParaSize: 参数个数 1~2 Para1: 速度值 Para2: 轴编号，缺省为所有轴或主轴 参考： MOVE_PARA(ACCEL, para2, para1)
1003	设置减速度	ParaSize: 参数个数 1~2 Para1: 速度值 Para2: 轴编号 参考： MOVE_PARA(DECCEL, para2, para1)
1004	...	...

其他参数设置类型根据需求扩展。

当前参数类型不满足需求时，可以根据需求，再往后增加参数类型标准。

21.7 机器类型工艺

通用三轴机床

上电默认的机床类型，一般设置为 XYZ 三轴。CAD 图形轨迹即运动路径，由 CAD 图形提供 XY 二轴的轨迹路径，Z 轴仅作抬刀和下刀，分别在抬刀高度和工作高度之间进行运动。用户可设置切削完当前连续曲线后，下一段空移前进行抬刀，空移到下一段切削轨迹处再进行下刀。

通用二轴机床

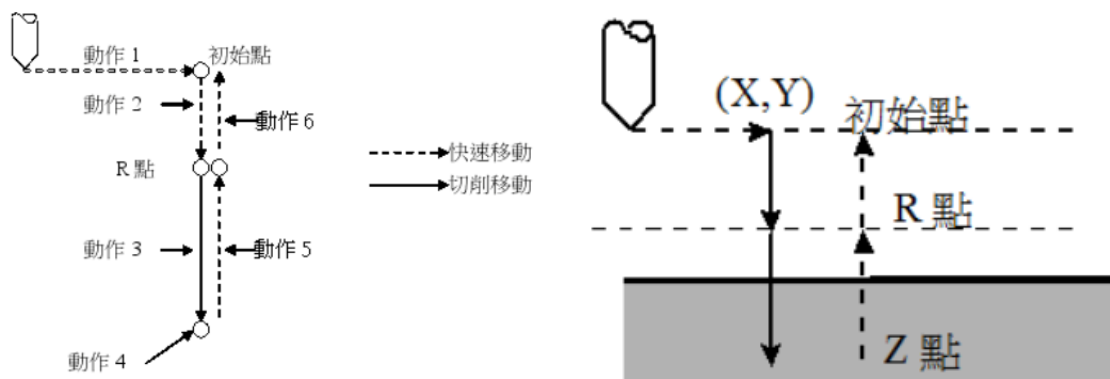
一般设置为 XY 两轴，CAD 图形轨迹即运动路径，无 Z 轴的抬刀高度和工作高度。

特殊三轴钻孔机床

特殊的三轴钻孔类型，设置该类型后，在进行代码工艺导出过程中，会自动过滤 CAD 图形中非圆形的轨迹，也就是仅保留圆形轨迹做为钻孔刀路，多个圆形轨迹间采取抬刀空移的方式进行运动。钻孔的位置一般取圆形轨迹的圆心处。

导出钻孔工艺流程一般由六个连续动作组成：

1. 快速移动至 X、Y 轴定位点（钻孔点）
2. 快速移动至 R 点（R 点由参数 Pr50 控制）
3. 钻孔加工（下钻，下钻速度由参数 Pr52 控制，深度由参数 Pr51 控制）
4. 在孔底位置上的动作（可选，由参数 Pr54 控制）
5. 离开到 R 点（出孔，出孔速度由参数 Pr53 控制）
6. 快速移动至初始点（可选，由参数 Pr5 控制）



第二十二章 DT 运动函数

为了支持 G 代码的变参数个数，增加 DT 运动函数，指令调用 TABLE 表的参数运动。

没有 ABS 后缀的为相对运动指令，带 ABS 后缀的为绝对运动指令。

MOVEDTSP/MOVEDTABSSP -- DT 直线运动

类型	DT 运动函数
描述	将轴号和运动距离分别存放到 TABLE 表，通过 TABLE 列表进行直线运动。使用位存在选择 TABLE 表里的轴号。
语法	MOVEDTSP (最大轴数, 位存在, 轴 dt 列表起始编号, 距离 dt 列表起始编号)
适用控制器	4 系以上支持
例子	<pre>TABLE(10,4,5,6) 'TABLE 表 10 存放 456 三个轴 TABLE(20,100,50,-10) 'TABLE 表 20 存放三个轴的运距离 WHILE 1 IF SCAN_EVENT(IN(0))>0 THEN MOVEDTSP(3,5,10,20) '每次运动 TABLE 表中的距离 'MOVEDTSPABS(3,5,10,20) '运动到 TABLE 表中的位置 '位存在为 5，转换为二进制 0101，只选择了轴 4 和轴 6 运动 ENDIF WEND</pre>

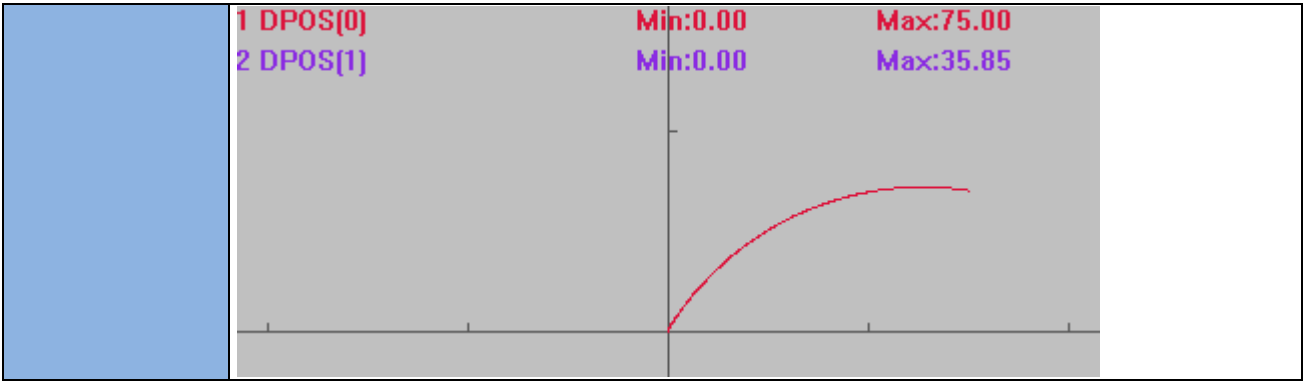
MOVECIRCDTSP/MOVECIRCDTABSSP -- DT 圆弧运动

类型	DT 运动函数
描述	将轴号和运动距离分别存放到 TABLE 表，通过 TABLE 列表进行圆弧运动。使用位存在选择 TABLE 表里的轴号。 根据当前点(起始点)和中点的位置，以及半径画圆，圆心自动计算。当终点与起始点的直线距离大于半径时，以这两点画半圆，半径为连线距离一半，圆心为连线中点。
语法	MOVECIRCDTSP (最大轴数, 位存在, 轴 dt 列表, 终点 dt 列表, 半径, 顺时针 0/逆时针 1)
适用控制器	4 系以上支持
例子	<pre>BASE(0,1,2) ATYPE=1,1,1 DPOS=0,0,0 TABLE(10,0,1) 'TABLE(10)存放轴 01 TABLE(20,0,50) '只需要放一个终点坐标 (X, Y) TRIGGER WHILE 1 IF SCAN_EVENT(IN(0))>0 THEN 'MOVECIRCDTABSSP(6,3,10,20,50,1)</pre>

	MOVECIRCDTSP(6,3,10,20,50,1) '从起始点经过（0,50），半径为 50，逆时针画圆弧 ENDIF WEND 1 DPOS[0] 2 DPOS[1] Min:0.00 Min:0.00 Max:6.70 Max:50.00
--	---

MOVECIRC2DTSP/MOVECIRC2DTABSSP -- DT 三点画圆弧运动

类型	DT 运动函数
描述	将轴号和运动距离分别存放到 TABLE 表，通过 TABLE 列表进行圆弧运动。使用位存在选择 TABLE 表里的轴号。 根据起始点、参考点、终点三点画圆弧。
语法	MOVECIRC2DTSP（最大轴数，位存在，轴 dt 列表，终点 dt 列表，参考点 dt 列表，mode） mode: <0 参考点在当前点的前面 =0 参考点在中间 >0 参考点在当前结束点的后面
适用控制器	4 系以上支持
例子	BASE(0,1) ATYPE=1,1 DPOS=0,0 TABLE(10,0,1) 'TABLE(10)存储轴 0,1 TABLE(20,50,10) 'TABLE(20)存储终点坐标 TABLE(30,25,25) 'TABLE(30)存储参考点坐标 TRIGGER WHILE 1 IF SCAN_EVENT(IN(0))>0 THEN MOVECIRC2DTSP(3,3,10,20,30,0) 'MODE=0 时，轨迹经过参考点，运行到参考点+终点的坐标位置 ENDIF WEND



MSPHERICALDTSP/MSPHERICALDTABSSP -- DT 空间圆弧运动

类型	DT 运动函数								
描述	将轴号和运动距离分别存放到 TABLE 表，通过 TABLE 列表进行空间圆弧运动。使用位存在选择 TABLE 表里的轴号。根据起始点、参考点、终点三点画圆弧。								
语法	MSPHERICALDTSP(轴数, 位存在, 轴 dt 列表, 终点 dt 列表, 参考点 dt 列表, mode) mode: 指定第二个点的位置 <table><tr><td>值</td><td>描述</td></tr><tr><td>-1</td><td>在当前点的前面，只是参考，不运行到参考点</td></tr><tr><td>0</td><td>中间，运行到参考点</td></tr><tr><td>1</td><td>在当前点的和结束点的后面</td></tr></table>	值	描述	-1	在当前点的前面，只是参考，不运行到参考点	0	中间，运行到参考点	1	在当前点的和结束点的后面
值	描述								
-1	在当前点的前面，只是参考，不运行到参考点								
0	中间，运行到参考点								
1	在当前点的和结束点的后面								
适用控制器	4 系以上支持								
例子	BASE(0,1,2) ATYPE=1,1,1 DPOS=0,0,0 TABLE(10,0,1,2) TABLE(10)轴号 0, 1,2 TABLE(20,0,0,100) 'TABLE(20)终点位置 TABLE(30,30,40,50) 'TABLE(30)参考点位置 WHILE 1 IF SCAN_EVENT(IN(0))>0 THEN MSPHERICALDTSP(3,7,10,20,30,0) 'MODE=0 时，轨迹从当前点开始，经过参考点，运行参考点+终点坐标位置 ENDIF WEND								

第二十三章 简易例程

23.1 常用操作

IO 操作

```

WHILE 1                                '循环检测输入信号
  IF IN(0) = ON THEN                   '输入口 0 有效
    OP(2, OFF)                         '关闭输出口 2
  ELSE
    OP(2, ON)                         '打开输出口 2
  ENDIF
WEND

```

SP 指令连续插补

```

ERRSWITCH = 3                          '全部信息输出
BASE(0,1,2,3)                          '选择 X Y Z U
RAPIDSTOP(2)
WAIT IDLE

DPOS = 0,0,0,0
ATYPE=1,1,1,1                          '脉冲方式步进或伺服
UNITS = 100,100,100,100                '脉冲当量, 每 mm100 脉冲
SPEED = 200,200,200,200                '此速度会对 FORCE_SPEED 进行限制
ACCEL = 2000,2000,2000,2000            '加速度设置
DECEL = 2000,2000,2000,2000            '减速度设置
MERGE = ON                             '启动连续插补
CORNER_MODE = 0                        '不启动自动拐角减速, 手动设置 STARTMOVE_SPEED, ENDMOVE_SPEED
'DECEL_ANGLE = 15 * (PI/180)            '开始减速的角度 15 度
'STOP_ANGLE = 45 * (PI/180)            '降到最低速度的角度 45 度

WHILE 1 '循环运动
  IF IN(0) = ON THEN                    '输入 0 有效启动运动
    TRACE "start movesp"
    '走一个方框, 每一段的速度和停止速度都不一样
    FORCE_SPEED = 100                    '第一段速度 100
    ENDMOVE_SPEED = 10                  '第一段结束的速度 10
    MOVESP(100,0)

    FORCE_SPEED = 150                    '第二段速度 150
    ENDMOVE_SPEED = 15
    STARTMOVE_SPEED = 15 '第二段的起始速度 15, 因为高于第一段的结束速度 10, 因此实际的
                                起始速度为 10
    MOVESP(0,100)

    FORCE_SPEED = 200                    '第三段速度 200

```

```

ENDMOVE_SPEED = 20
STARTMOVE_SPEED = 20 '第三段的起始速度 20， 因为高于第二段的结束速度 15，因此实际的
                        起始速度为 15
MOVESP(-100,0)

FORCE_SPEED = 300      '第四段速度 300，受 SPEED 限制其实为 200
ENDMOVE_SPEED = 30
STARTMOVE_SPEED = 30    '第三段的起始速度 30，因为高于第二段的结束速度 20，因此实际的
                        起始速度为 20
MOVESP(0,-100)
WAIT IDLE               '等待运动停止
DELAY(100)              '延时
ENDIF
WEND
END

```

字符串与数据相互转化

```

DIM val1,val2,array1(15)    '定义变量和数组
val1=1234                  '变量 1 赋值 1234
FOR i=0 TO 14
    array1(i)=0             '清空数组
NEXT
?val1,val2                 '打印两个变量确认值
?*array1                   '打印数组确认值

array1=TOSTR(val1)          '数据转成字符串，赋值给数组
?*array1                   '再次打印数组确认
array1=TOSTR(val1)+"1asf"  'tostr 也可与其他字符串合并
?*array1

val2=val(array1)            '字符串转化为数据赋值给变量 2
?val2                      '打印变量 2 确认
'只有数字字符可以来回转化，字母、符号字符不可以

```

手轮

手轮就是一个编码器，通常用来对点校准工件位置坐标。手轮运动就是类似与机械齿轮传动的运动，通过不同的比例线性运动。

```

ERRSWITCH = 3              '全部信息输出
CONST AXISHAND = 0
BASE(AXISHAND)             '选择第 0 轴接手轮
ATYPE=6                    '脉冲+方向的手轮，正交输入手轮使用 3
BASE(1)                    '轴 1 被手轮控制
ATYPE=1                    '步进
DPOS = 0
UNITS = 100                '脉冲当量，每 mm100 脉冲
SPEED = 200
ACCEL = 3000

```

```

DECEL = 3000
SRAMP = 20
CLUTCH_RATE = 0           '采用速度加速度来进行限制

DIM POSLAST                '记录上一个位置变量
POSLAST = DPOS
WHILE 1                    '手动选择手轮连接倍率
    IF IN(0) = ON AND IN(1) = OFF THEN
        CONNECT(1, AXISHAND) '链接到轴 0, 倍率 1
    ELSEIF IN(1) = ON AND IN(0) = OFF THEN
        CONNECT(10, AXISHAND) '链接到轴 0, 倍率 10
    ELSEIF IN(0) = ON AND IN(1) = ON THEN
        CONNECT(50, AXISHAND) '链接到轴 0, 倍率 50,对步进, 倍率太高会出现丢步或长时间才
                                能结束
    ELSEIF MTYPE = 21 THEN    '取消 CONNECT
        CANCEL
    ENDIF

    IF POSLAST <> DPOS THEN
        POSLAST = DPOS
        TRACE DPOS
    ENDIF
WEND
END

```

追剪应用

参见 [MOVELINK](#) 自动凸轮指令例二。

位置比较输出

参见第十一章[位置比较输出](#)小节，分为硬件位置比较输出和软件位置比较输出。

掉电保存

参见 [ONPOWEROFF](#) 掉电中断例程。

机械手应用

参见第四章机械手“[六自由度机械手](#)”的应用例程。

机械手使用 movesync 指令

该示例为一个 SCARA.zpj 项目文件，包含[建模.bas](#)文件和[运动程序.bas](#)文件。

建模.bas

DELAY (1000)

dim axis_j0,axis_j1,axis_jz,axis_jv,axis_x,axis_y,axis_z,axis_RZ

axis_j0 =0 '轴大臂

axis_j1 =1 '轴小臂

axis_jz =2 '轴伸缩轴

axis_jv =3 '轴旋转轴

axis_x =7'模拟轴大臂

axis_y =8 '模拟轴小臂

axis_z =9'模拟轴伸缩

axis_RZ =10 '模拟旋转轴

dim L1 '大臂长度

dim L2 '小臂长度

dim L3 'X 方向偏移

dim ZDis '旋转轴一圈，z 轴移动距离

L1=400

L2=400

L3=0

ZDis=0 '0 为 RZ 和 Z 解耦

dim u_m1 '电机 1 一圈脉冲数

dim u_m2 '电机 2 一圈脉冲数

dim u_mz '电机 z 一圈脉冲数

dim u_mv '电机 v 一圈脉冲数

u_m1=131072

u_m2=131072

u_mz=131072

u_mv=131072

dim i_1 '关节 1 传动比

dim i_2 '关节 2 传动比

dim i_z '关节 z 传动比

dim i_v '关节 v 传动比

i_1=80

i_2=50

i_z=2

i_v=6750/384

dim u_j1 '关节 1 实际一圈脉冲数

dim u_j2 '关节 2 实际一圈脉冲数

dim u_jz '关节 z 实际一圈脉冲数

dim u_jv '关节 v 实际一圈脉冲数

u_j1=u_m1*i_1


```

u_j2=u_m2*i_2
u_jz=u_mz*i_z
u_jv=u_mv*i_v

dim p_z      'z 轴螺距
p_z=20

""关节轴设置
BASE(axis_j0,axis_j1,axis_jv,axis_jz)      '选择关节轴号
DPOS= 0,0,0,0      '回零
atype=0,0,0,0      '轴类型设为脉冲轴
UNITS=u_j1/360,u_j2/360,u_jv/360,u_jz/p_z  '把 z 轴 units 设成 1mm 的脉冲数'其余轴设成 1° 的脉冲数
speed=100,100,100,100      '速度参数设置
accel=1000,1000,1000,1000
decel=1000,1000,1000,1000
CLUTCH_RATE=1000,1000,1000,1000      '使用关节轴的速度和加速度限制
merge=on      '开启连续插补
SRAMP = 100
FS_LIMIT = 1000,1000,1000,1000
RS_LIMIT = -1000,-1000,-1000,-1000

""虚拟轴设置
BASE(axis_x,axis_y,axis_RZ,axis_z)
ATYPE=0,0,0,0      '设置为虚拟轴
TABLE(1000,L1,L2,u_j1,u_j2,u_jv,L3,ZDis)      '根据手册说明填写参数
UNITS=1000,1000,u_jv/360,1000      '运动精度，要提前设置，中途不能变化
speed=100,100,100,100      '速度参数设置
accel=1000,1000,1000,1000
decel=1000,1000,1000,1000
FS_LIMIT = 1000,1000,1000,1000
RS_LIMIT = -1000,-1000,-1000,-1000

GLOBAL g_op
g_op= 0
while 1
  if g_op=1 then
    BASE(0,1,2,3)      '配置关节轴
    CONNFRAME(1,1000,7,8,9,10) '第 7,8,9,10 轴为虚拟的 XYZW 轴，启动逆解连接。
    g_op= 0      '复位
    ?"逆解模式"
  elseif g_op= 2 then
    base(7,8,9,10)      '选择虚拟轴号
    CONNREFRAME(1,1000,0,1,2,3)      '第 0/1/2/3 轴为关节轴，启动正解连接

```

```

    g_op= 0
    ?"正解模式"
endif
wend
end

```

运动程序.bas

```
GLOBAL SUB MOVESYNCTEST()
```

```
    ERRSWITCH = 4
```

```
    TRIGGER
```

```
    OP(1,ON)
```

```
    g_op =1
```

```
    DIM
```

```
WAIT_X, WAIT_Y, WAIT_Z, WAIT_A, CONVREG_Axis, CONVREG_POS, CONVSYNCGO_Time, CONVSYNC_
Time, CONVBack_Time
```

```
    WAIT_X = -26
```

```
    WAIT_Y = 362
```

```
    WAIT_Z = 0
```

```
    WAIT_A = 0
```

```
    '运行通用参数
```

```
    CONVREG_Axis = 11
```

```
    CONVREG_POS = 350
```

```
    CONVSYNCGO_Time = 300    '追赶时间
```

```
    CONVSYNC_Time = 2000    '同步时间
```

```
    CONVBack_Time = 500
```

```
    BASE(7,8,9,10)    '运动到等待位置
```

```
    MOVEABS(WAIT_X, WAIT_Y, WAIT_Z, WAIT_A)
```

```
    WAIT UNTIL IDLE(7)
```

```
    DIM convREG_X, convREG_Y, convREG_Z, convREG_A, convAngleX, convAngleY
```

```
    '//传送带在坐标系 225 度方向，皮带 dpos 正向与 X 轴夹角为 225，与 Y 轴夹角为 135
```

```
    convREG_X = 56    '触发位置锁存时候的坐标位置
```

```
    convREG_Y = 462
```

```
    convREG_Z = 0
```

```
    convREG_A = 0
```

```
    convAngleX = 225/180*PI '皮带正和机械手 X 轴的角度弧度
```

```
    convAngleY = 225/180*PI - PI/2 '皮带正和机械手 Y 轴的角度弧度，注意弧度值不能为负，如果得负数
需要+2*PI
```

```
    BASE(11)'传送带轴
```

```

UNITS = 1910738.4889
SPEED = 30
VMOVE(1)
MPOS=350
WAIT UNTIL DPOS(11) < 407'等待传送带到触发位置

```

- '注意分离轴写法，要保证每次都有所有轴，不用跟随就写跟随轴为-1，但是不要随便用-1 的跟随模式
- '这里只做了追赶，不做同步

```

base(7)
MOVESYNC(convAngleX,CONVSYNCGO_Time,CONVREG_POS,CONVREG_Axis,convREG_X)
BASE(8)
MOVESYNC(convAngleY,CONVSYNCGO_Time,CONVREG_POS,CONVREG_Axis,convREG_Y)
BASE(9)
MOVESYNC(0,CONVSYNCGO_Time,0,-1,convREG_Z)
BASE(10)
MOVESYNC(0,CONVSYNCGO_Time,0,-1,convREG_A)

```

'同步运动段

```

base(7)
MOVESYNC(convAngleX,CONVSYNCGO_Time,CONVREG_POS,CONVREG_Axis,convREG_X)
BASE(8)
MOVESYNC(convAngleY,CONVSYNCGO_Time,CONVREG_POS,CONVREG_Axis,convREG_Y)
BASE(9)
MOVESYNC(0,CONVSYNCGO_Time,0,-1,convREG_Z)
BASE(10)
MOVESYNC(0,CONVSYNCGO_Time,0,-1,convREG_A)

```

'这里停止同步，回到指定放料位置上方

```

BASE(7)
MOVESYNC(0,CONVBack_Time,0,-1,WAIT_X)
BASE(8)
MOVESYNC(0,CONVBack_Time,0,-1,WAIT_Y)
BASE(9)
MOVESYNC(0,CONVBack_Time,0,-1,WAIT_Z)
BASE(10)
MOVESYNC(0,CONVBack_Time,0,-1,WAIT_A)

```

ENDSUB

编码器读取

松下 A6 编码器读取例程

```

*****绝对值编码器部分*****
SETCOM(38400,8,1,0,1,0)      '设置 485 口 PORT1 为自定义协议

```

```

GLOBAL DIM tempchar          '接收的一个字节

GLOBAL DIM neqbuff(2)        '发送识别码 485 为 81H, 05H
neqbuff(0) = $81
neqbuff(1) = $05

GLOBAL DIM eotbuff(2)        '接收识别码 485 为 80H,04H
eotbuff(0) = $80
eotbuff(1) = $04

GLOBAL DIM ackbuff           '接收应答 06H
ackbuff = $06

GLOBAL DIM cmdbuff(20)       '发送命令数组
GLOBAL DIM getbuff(20)       '接收的字符串

GLOBAL DIM getnum            '接收的字节数
getnum = 0

GLOBAL DIM highdata          '编码器多圈数据
GLOBAL DIM lowdata           '单圈数据

runtask 4,get_char           '启动接收字符串任务

MODBUS_REG(0)=0
WHILE 1
    IF MODBUS_REG(0)=1 THEN '判断是否接收到了数据
        MODBUS_REG(0)=0
        getmpos(1,45)      '读取站号 1 的多圈与单圈值。
    ENDIF
WEND
END

'读坐标
GLOBAL SUB getmpos(sifunum,rcr) '读伺服编号为 1 的电机的绝对值位置
    cmdbuff(0) = $00
    cmdbuff(1) = sifunum
    cmdbuff(2) = $d2
    cmdbuff(3) = rcr

    neqbuff(0) = $80 + sifunum
    neqbuff(1) = $05

    eotbuff(0) = $80
    eotbuff(1) = $04

    getnum = 0
    PUTCHAR#1,neqbuff
    TICKS = 2000          '延时
    WAIT UNTIL (getnum = 2) OR TICKS < 0
    IF getnum = 2 THEN    '如果接到了 2 个字符
        IF(getbuff(0)=$80+sifunum) AND (getbuff(1)=$04) THEN'收到应答发送命令
            getnum = 0
            PUTCHAR#1,cmdbuff          '发送读取编码器命令
        
```

```

TICKS = 2000
WAIT UNTIL (getnum = 3) OR TICKS < 0
IF (getbuff(0) = $06) AND (getbuff(1) = $80) AND (getbuff(2) = $05) THEN
    '收到发送请求，给应答

    getnum = 0
    PUTCHAR#1,eotbuff          '发送应答，等待接收数据
    TICKS = 2000
    WAIT UNTIL (getnum = 15) OR TICKS < 0
    IF getnum = 15 THEN        '读到数据 11-10 为多圈数据 9-7 为单圈数据
        PUTCHAR#1,ackbuff
        highdata = getbuff(11) * $100 + getbuff(10)
        lowdata = getbuff(9) * $10000 + getbuff(8) * $100 + getbuff(7)
        PRINT getbuff(11),getbuff(10),getbuff(9),getbuff(8),getbuff(7),getnum
    ELSE
        PRINT getnum,getbuff(0),getbuff(1),"超时重新读取 1"
    ENDIF
ELSE
    PRINT getbuff(0),getbuff(1),"超时重新读取 2"
ENDIF
ELSE
    PRINT getbuff(0),getbuff(1),"驱动器无应答"
ENDIF
ELSE
    PRINT "驱动器无应答"
ENDIF
ENDIF
END SUB

'串口接收
GLOBAL SUB get_char()
    WHILE 1
        GET#1,tempchar
        getbuff(getnum) = tempchar
        getnum = getnum + 1
    WEND
END SUB

```

自定义 G 代码

```

ERRSWITCH = 3                '全部信息输出
BASE(0,1,2,3)                '选择 X Y Z U，G01 里面有指定，不能随意修改
RAPIDSTOP(2)
WAIT IDLE

DPOS = 0,0,0,0
ATYPE=1,1,1,1                '脉冲方式步进或伺服
UNITS = 100,100,100,100      '脉冲当量，每 MM100 脉冲
SPEED = 200,200,200,200
ACCEL = 2000,2000,2000,2000
DECEL = 2000,2000,2000,2000
MERGE = ON                    '启动连续插补
CORNER_MODE = 2               '启动拐角减速
DECEL_ANGLE = 15 * (PI/180)
STOP_ANGLE = 45 * (PI/180)

```

```

G_INIT()          'G 初始化
WHILE 1           '循环运动
  IF IN(0) = ON THEN '输入 0 有效启动运动
    '走一个方框
    G91 '相对位置
    G01 X100 Y0 '运动轨迹
    G01 X0 Y100
    G01 X-100 Y0
    G01 X0 Y-100
    WAIT IDLE '等待运动停止
    DELAY(100) '延时
  ENDIF
WEND
END '使得本文件自动运行时退出，避免重复执行后方的 SUB 文件

'相对位置模式
GLOBAL SUB G_INIT()
DIM coor_rel
coor_rel = 1 '相对位置模式
END SUB

GLOBAL GSUB G01(X Y Z U)
TRACE "G01 entered, distance:" sub_para(0),sub_para(1),sub_para(2),sub_para(3)'调试输出
IF coor_rel THEN
  MOVE(sub_para(0),sub_para(1),sub_para(2),sub_para(3)) '相对位置
ELSE
  LOCAL xdis, ydis, zdis, udis
  IF sub_ifpara(0) THEN '判断是否有参数传入 SUB
    xdis = sub_para(0)
  ELSE
    xdis = ENDMOVE_BUFFER(0)
  ENDIF
  IF sub_ifpara(1) THEN
    ydis = sub_para(1)
  ELSE
    ydis = ENDMOVE_BUFFER(1)
  ENDIF
  IF sub_ifpara(2) then
    zdis = sub_para(2)
  ELSE
    zdis = ENDMOVE_BUFFER(2)
  ENDIF
  IF sub_ifpara(3) then
    udis = sub_para(3)
  ELSE
    udis = ENDMOVE_BUFFER(3)
  ENDIF
  MOVEABS(xdis,ydis,zdis,udis) '绝对位置
ENDIF
END SUB

'绝对位置模式
GLOBAL GSUB G90()
TRACE "G90 entered"
coor_rel = 0

```

END SUB

'相对

```
GLOBAL GSUB G91()  
    TRACE "G91 entered"  
    coor_rel = 1  
END SUB
```

'延时

```
GLOBAL GSUB G04(P)  
    TRACE "G04 entered"  
    IF sub_ifpara(0) THEN  
        DELAY (sub_para(0))  
    ELSE  
        ENDIF  
END SUB
```

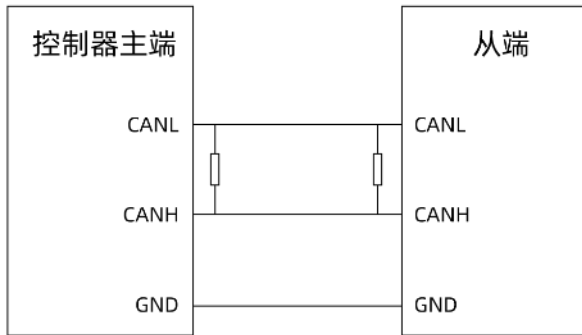
GLOBAL GSUB G00(X Y Z U)

```
    TRACE "G00 entered, distance:" sub_para(0),sub_para(1),sub_para(2),sub_para(3)'调试输出  
    IF coor_rel THEN  
        MOVE(sub_para(0),sub_para(1),sub_para(2),sub_para(3))  
    ELSE  
        LOCAL xdis, ydis, zdis, udis  
        IF sub_ifpara(0) THEN  
            xdis = sub_para(0)  
        ELSE  
            xdis = ENDMOVE_BUFFER(0)  
        ENDIF  
        IF sub_ifpara(1) then  
            ydis = sub_para(1)  
        ELSE  
            ydis = ENDMOVE_BUFFER(1)  
        ENDIF  
        IF sub_ifpara(2) THEN  
            zdis = sub_para(2)  
        ELSE  
            zdis = ENDMOVE_BUFFER(2)  
        ENDIF  
        IF sub_ifpara(3) THEN  
            udis = sub_para(3)  
        ELSE  
            udis = ENDMOVE_BUFFER(3)  
        ENDIF  
        MOVEABS(xdis,ydis,zdis,udis)  
    ENDIF  
END SUB
```

23.2 模块通讯

CAN 通讯

控制器间接线



CANL - CANL

CANH - CANH

在控制器主端 CANL 和 CANH 间各连接一个 120 欧电阻。

主端程序

```

CANIO_ADDRESS = 32          '主端
STOPTASK 1
RUNTASK 1,task_canget      '启动接受任务
GLOBAL if_send
if_send = 1

WHILE 1
  IF if_send = 1 THEN
    TABLE(0,0,8,1,2,3,4,5,6,7,8) '向 can cob id 为 0 的控制器发送 8 个字节依次为 1-8
    CAN(0,7,0)                  '0 - CAN 通道号、7 - 发送、0-数据 table 起始地址
    if_send = 0
  ENDIF
WEND
END

GLOBAL SUB task_canget()      '接受任务
  WHILE 1
    CAN(0,6,10)               '数据接受，存放在 table(10)之后，table(10)表示发送端 CANID，<0 时表示没有数据
                                'table(11)表示接受到数据个数，table(12)...表示数据
    IF(table(10) >= 0 ) THEN   '有接受到数据
      ?"数据发送端 ID:",table(10)
      ?"数据字节数:",table(11)
      ?"数据: "table(12),table(13),table(14),table(15),table(16),table(17),table(18)
    ENDIF
  WEND
END SUB

```

从端程序

```

CANIO_ADDRESS = 0          '从端
STOPTASK 1
RUNTASK 1,task_canget      '启动接受任务
GLOBAL if_send
if_send = 0

WHILE 1
  WAIT UNTIL if_send = 1     '等待有数据发送过来
  TABLE(0,32,1,$ff)        '向 can cob id 为 32 的控制器发送 1 个字节，依次为 FF
  CAN(0,7,0)                 '0 - CAN 通道号、7 - 发送、0-数据 table 起始地址

```



```

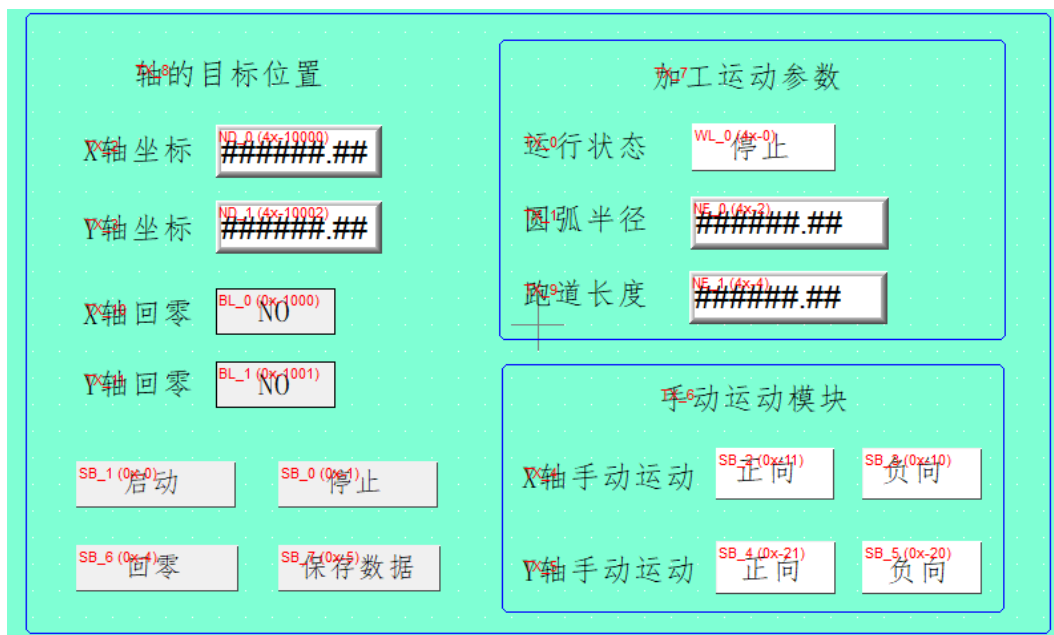
        if_send = 0
    WEND
END

GLOBAL SUB task_canget()      '接受任务
    WHILE 1
        CAN(0,6,10) '数据接受，存放在 table(10)之后，table(10)表示发送端 CANID，<0 时表示没有数据
                        'table(11)表示接受到数据个数，table(12)...表示数据
        IF(table(10) >= 0 ) THEN      '有接受到数据
            ?"数据发送端 ID:",table(10)
            ?"数据字节数:",table(11)
            ?"数据: "table(12),table(13),table(14),table(15),table(16),table(17),table(18)
            if_send = 1
        ENDIF
    WEND
END SUB

```

触摸屏通讯

该例程触摸屏地址从 0 开始，也有部分触摸屏地址是从 1 开始的，地址从 1 开始的触摸屏上的 1 对应程序中的 0。



*****初始化模块*****

```

ERRSWITCH = 3      '全部信息输出
RAPIDSTOP(2)
WAIT IDLE

```

```

BASE(0,1)          '选定 X Y 轴
DPOS=0,0
ATYPE=1,1
UNITS = 100,100
SPEED = 100,100
ACCEL = 1000,1000
DECEL = 1000,1000
SRAMP = 100,100

```

```

DIM run_state          '运行状态
run_state = 0          '0 停止, 1 运行, 2 回零
MODBUS_REG(0) = run_state  '显示运行状态

DIM radius,length      '半径,长度
radius = 100           '缺省半径大小
length = 300           '缺省长度
FLASH_READ 0,radius,length
MODBUS_IEEE(2) = radius  '显示半径大小
MODBUS_IEEE(4) = length  '显示长度

DIM home_done          '回零完成的标志位 0 未回零,1 已回零
home_done = 0          '上电进入未回零状态

MODBUS_BIT(0) = 0       '启动 按钮复归
MODBUS_BIT(1) = 0       '停止 按钮复归
MODBUS_BIT(4) = 0       '回零 按钮复归
MODBUS_BIT(5) = 0       '保存数据 按钮复归

MODBUS_BIT(1000)=0      'X 轴 回零标志为 0
MODBUS_BIT(1001)=0      'Y 轴 回零标志为 0

STOPTASK 2
RUNTASK 2, guidetask    '启动手动运行任务

*****按键扫描模块*****
WHILE 1                  '扫描触摸屏端按钮输入
  IF MODBUS_BIT(0)= 1 THEN  '启动按钮按下
    MODBUS_BIT(0) = 0      '按钮复归

    IF run_state = 0 THEN  '待机停止状态
      IF home_done = 0 THEN  '未回零时不启动运动
        TRACE "before move need home"
      ELSEIF home_done = 1 THEN  '已回零启动任务运行
        TRACE "move start"
        STOPTASK 1          '软件安全, 停止任务 0
        RUNTASK 1, movetask  '启动运行加工任务 1
      ENDIF
    ENDIF

  ELSEIF MODBUS_BIT(1) = 1 THEN  '停止按钮按下
    TRACE "move stop"
    MODBUS_BIT(1) = 0          '按钮复归

    RAPIDSTOP(2)
    STOPTASK 1
    RAPIDSTOP(2)

    run_state = 0              '停止标志
    MODBUS_REG(0) = run_state  '显示状态
  ENDIF

```

```

IF MODBUS_BIT(4) = 1 THEN      '回零按钮按下
    MODBUS_BIT(4) = 0          '回零复归

    IF run_state= 0 THEN
        stoptask 1
        runtask 1,home_task    '启动回零任务
    ENDIF
ENDIF

'"保存数据处理
IF MODBUS_BIT(5) = 1 THEN      '保存数据 按钮按下
    MODBUS_BIT(5) = 0          '保存数据 按钮复归

    print "写入数据到 FLASH"
    radius = MODBUS_IEEE(2)
    length = MODBUS_IEEE(4)
    FLASH_WRITE 0,radius,length '往扇区 0 写入数据
ENDIF
WEND
END

*****加工运动模块*****
movetask:                      '运行画圆弧+跑道的任务
    run_state = 1              '进入运行状态
    MODBUS_REG(0) = run_state

    radius = MODBUS_IEEE(2)     '读取半径
    length = MODBUS_IEEE(4)     '读取长度

    TRIGGER
    BASE(0,1)                  '选定 X Y 轴
    MOVEABS(0,0)

    MOVE(length,0)              '从原点开始走跑道轨迹
    MOVECIRC(0,radius*2,0,radius,0)
    MOVE(-length,0)
    MOVECIRC(0,-radius*2,0,-radius,0)
    WAIT IDLE(0)

    run_state = 0              '进入待机状态
    MODBUS_REG(0) = run_state
END

*****回零任务*****
home_task:
    TRACE "enter home task"

    run_state = 2              '回零标志
    MODBUS_REG(0) = run_state   '显示状态

    TRIGGER

    BASE(0,1)
    CANCEL(2) AXIS(0)           '先轴 0,轴 1 停止
    CANCEL(2) AXIS(1)

```

```

WAIT IDLE(0)
WAIT IDLE(1)

'DATUM(3) AXIS(0)      '实际设备轴 0 的归零
'DATUM(3) AXIS(1)      '实际设备轴 1 的归零
MOVEABS(0) AXIS(0)     '虚拟设备轴 0 的归零
MOVEABS(0) AXIS(1)     '虚拟设备轴 1 的归零

WAIT IDLE(0)
MODBUS_BIT(1000)=1      '设置轴 0 已归零的标志

WAIT IDLE(1)
MODBUS_BIT(1001)=1      '设置轴 1 已归零的标志

home_done = 1
TRACE "home task done"

run_state = 0            '回到待机状态
MODBUS_REG(0) = run_state
END

*****手动运动*****
guidetask:
WHILE 1
    IF run_state = 0 THEN      '判断是否处于停止状态
        BASE(0)
        IF MODBUS_BIT(10) = 1 THEN      '左
            MODBUS_BIT(10) = 0
            VMOVE(-1)
        ELSEIF MODBUS_BIT(11) = 1 THEN      '右
            MODBUS_BIT(11) = 0
            VMOVE(1)
        ELSEIF MTYPE = 10 OR MTYPE = 11 THEN      '非 VMOVE 运动
            CANCEL(2)
        ENDIF

        BASE(1)
        IF MODBUS_BIT(20) = 1 THEN      '左
            MODBUS_BIT(20) = 0
            VMOVE(-1)
        ELSEIF MODBUS_BIT(21) = 1 THEN      '右
            MODBUS_BIT(21) = 0
            VMOVE(1)
        ELSEIF MTYPE = 10 OR MTYPE = 11 THEN      '非 VMOVE 运动
            CANCEL(2)
        ENDIF
    ENDIF
    DELAY(100)
WEND
END

```

自定义网口通讯

```

OPEN#11, "TCP_SERVER", 10      '使用自定义网口通道 2，作为主端，端口号 10

```

```

GLOBAL DIM tempchar
GLOBAL CONST datamax=20
GLOBAL DIM datanum
        datanum=0
GLOBAL DIM DATA(datamax)

STOPTASK 1
RUNTASK 1,aaa
WHILE 1
    tempchar = 0                '清除之前的字符
    GET#11,tempchar            '获取单个字符到 tempchar
    PRINT tempchar              '打印出字符的 ASCII 码
    DATA(datanum) = tempchar  '保存到数组
    datanum = datanum + 1
    IF datanum = datamax THEN  '超过数组空间
        datanum = 0
        FOR i = 0 TO datamax-1 '清除数组空间内容
            DATA(i) = 0
        NEXT
    ENDIF
    IF tempchar = 59 THEN      '号终止位

        PRINT#11,"ok1245"
    ENDIF
    DELAY(10)
WEND
END

SUB aaa()
    WHILE 1
        tempchar = 0                '清除之前的字符
        GET#10,tempchar            '获取单个字符到 tempchar
        PRINT tempchar              '打印出字符的 ASCII 码
        DATA(datanum) = tempchar  '保存到数组
        datanum = datanum + 1
        IF datanum = datamax THEN  '超过数组空间
            datanum = 0
            FOR i = 0 TO datamax-1 '清除数组空间内容
                DATA(i) = 0
            NEXT
        ENDIF
        IF tempchar = 59 THEN      '号终止位
            PRINT#10,"aaaaaaaaaa"
        ENDIF
    WEND
END SUB

```

控制器之间通讯

最新固件支持此功能，将主端和从端程序分别下载到不同的控制器中，用网线连接控制器的网口（可通过交换机）。

*****主端控制器

```

DIM i,j,lasttick
MODBUS_REG(j)=0
MODBUSM_DES2($1, 20, "192.168.0.11") '控制器 IP 不能相同
WHILE 1
    lasttick=TICKS
    FOR i=0 TO 9999
        MODBUS_REG(0) = i
        MODBUSM_REGSET(0,1,0)
        MODBUS_REG(0) = 99
        MODBUSM_REGGET(0,1,0)
        IF MODBUS_REG(0) <> i THEN PRINT "MODBUS_REG(0)=" MODBUS_REG(0),
"MODBUSM_STATE=" MODBUSM_STATE
    NEXT
    ?lasttick-TICKS
WEND
END

```

*****从端控制器

```

DIM j
ADDRESS=1
MODBUS_REG(j)=0
WHILE 1
    IF MODBUS_REG(0) <> 0 then
        SPEAKOUT(100)
    ENDIF
WEND
END

```

自定义串口通讯和字符串使用

```

SETCOM(38400,8,1,0,0,0) '配置串口为 RAM 模式
DIM TEMPVAR '定义变量
DIM VALUE
DIM CHLIST(10) '定义数组
FOR i=0 TO 9
    GET#0, TEMPVAR '从通道 0 读取数据
    CHLIST(i)=TEMPVAR '读取的数据依次存储到数组
NEXT
TRACE CHLIST '调试
VALUE = VAL(CHLIST) '转成变量
PRINT#0, TOSTR(VALUE) '转成字符串输出

```

23.3 总线初始化

EtherCAT 初始化程序

- 1.使用要求：控制器带 EtherCAT 接口，伺服驱动器必须支持 EtherCAT 总线，RTSys 需要 2.5 以上版本。
- 2.此初始化程序只进行了总线使能操作，轴的脉冲当量、轴速度、运动轨迹需要在上位机进行设置。

```

*****ECAT 总线初始化
GLOBAL CONST BUS_TYPE = 0          '总线类型。可用于上位机区分当前总线类型
GLOBAL CONST MAX_AXISNUM = 16      '最大轴数
GLOBAL CONST Bus_Slot = 0          '槽位号 0（单总线控制器缺省 0）
GLOBAL CONST PUL_AxisStart = 0     '本地脉冲轴起始轴号
GLOBAL CONST PUL_AxisNum = 0       '本地脉冲轴轴数量
GLOBAL CONST Bus_AxisStart = 0     '总线轴起始轴号
GLOBAL CONST Bus_NodeNum = 1       '总线配置节点数量,用于判断实际检测到的从站数量是否一致

GLOBAL Bus_InitStatus              '总线初始化完成状态
Bus_InitStatus = -1
GLOBAL Bus_TotalAxisnum           '检查扫描的总轴数

delay(3000)                       '延时 3S 等待驱动器上电，不同驱动器自身上电时间不同，具体根据驱动器调整延时

?"总线通讯周期: ",SERVO_PERIOD,"us"
Ecat_Init()                       '初始化 ECAT 总线

WHILE (Bus_InitStatus = 0)
    Ecat_Init()
WEND

END

*****ECAT 总线初始*****
'初始流程: slot_scan（扫描总线）-> 从站节点映射轴/io -> SLOT_START（启动总线）-> 初始化成功
*****
GLOBAL SUB Ecat_Init()
    LOCAL Node_Num,Temp_Axis,Drive_Vender,Drive_Device,Drive_Alias
    RAPIDSTOP(2)
    FOR i=0 TO MAX_AXISNUM - 1
        AXIS_ENABLE(i) = 0          '初始化还原轴类型
        ATYPE(I)=0
        AXIS_ADDRESS(i)=0
        DELAY(10)                   '防止所有驱动器全部同时切换使能导致瞬间电流过大
    NEXT

    Bus_InitStatus = -1
    Bus_TotalAxisnum = 0
    SLOT_STOP(Bus_Slot)
    DELAY(200)
    SLOT_SCAN(Bus_Slot)             '扫描总线
    IF RETURN THEN
        ?"总线扫描成功","连接从站设备数: "NODE_COUNT(Bus_Slot)
        IF NODE_COUNT(Bus_Slot) <> Bus_NodeNum THEN '判断总线检测数量是否为实际接线数量
            ?""
            ?"扫描节点数量与程序配置数量不一致!", "配置数量:"Bus_NodeNum,"检测数量: "
            "NODE_COUNT(Bus_Slot)
            Bus_InitStatus = 0      '初始化失败。报警提示
            RETURN
        ENDIF

        ""开始映射轴号"
        FOR Node_Num=0 TO NODE_COUNT(Bus_Slot)-1
            '遍历扫描到的所有从站节点

```

```

Drive_Vender = NODE_INFO(Bus_Slot,Node_Num,0)      '读取驱动器厂商
Drive_Device = NODE_INFO(Bus_Slot,Node_Num,1)      '读取设备编号
Drive_Alias = NODE_INFO(Bus_Slot,Node_Num,3)       '读取设备拨码 ID

IF NODE_AXIS_COUNT(Bus_Slot,Node_Num) <> 0 THEN    '判断当前节点是否有电机
  FOR j=0 TO NODE_AXIS_COUNT(Bus_Slot,Node_Num)-1  '根据节点带的电机数量
    循环配置轴参数(针对一拖多驱动器)
    IF Drive_Vender = $83 THEN
      'Sub_SetPdo(Node_Num,Drive_Vender,Drive_Device) '设定 PDO 参数
    ELSE
      Temp_Axis = Bus_AxisStart + Bus_TotalAxisnum '轴号按 NODE 顺序分配
      'Temp_Axis = Drive_Alias                     '轴号按驱动器设定的拨
      码分配（一拖多需要特殊处理）
      BASE(Temp_Axis)
      AXIS_ADDRESS= Bus_TotalAxisnum+1             '映射轴号
      ATYPE=65                                     '设置控制模式 65-位置
    66-速度 67-转矩
    DRIVE_PROFILE = -1
    'Sub_SetPdo(Node_Num,Drive_Vender,Drive_Device) '设定 PDO 参数
    'Sub_SetDriverIo(Drive_Vender,Temp_Axis,32 + 32*Temp_Axis) '映射驱动器 IO
    IO 映射到控制器 IO32-以后每个驱动器间隔 32 点
    'Sub_SetNodePara(Node_Num,Drive_Vender,Drive_Device,j) '设置特殊总线
    参数
    DISABLE_GROUP(Temp_Axis)                       '每轴单独分组
    Bus_TotalAxisnum=Bus_TotalAxisnum+1            '总轴数+1
  ENDIF
NEXT
ELSE
  'IO 扩展模块
  'Sub_SetNodeIo(Node_Num,Drive_Vender,Drive_Device,1024 + 32*Node_Num) '映射
  扩展模块 IO
ENDIF
NEXT
?"轴号映射完成","连接总轴数: "Bus_TotalAxisnum

DELAY 200
SLOT_START(Bus_Slot) '启动总线
IF RETURN THEN

WDOG=1 '使能总开关

?"开始清除驱动器错误"
FOR i= Bus_AxisStart TO Bus_AxisStart + Bus_TotalAxisnum - 1
  BASE(i)
  DRIVE_CLEAR(0)
  DELAY 50

  ?"驱动器错误清除完成"
  DATUM(0) '清除控制器轴状态错误"
  DELAY 100

  ""轴使能"
  AXIS_ENABLE=1
NEXT

```



```

        Bus_InitStatus = 1
        ?"轴使能完成"

        '本地脉冲轴配置
        FOR i = 0 TO PUL_AxisNum - 1
            BASE(PUL_AxisStart + i)
            AXIS_ADDRESS = (-1<<16) + i
            ATYPE = 4
        NEXT
        ?"总线开启成功"
    ELSE
        ?"总线开启失败"
        Bus_InitStatus = 0
    ENDIF
ELSE
    ?"总线扫描失败"
    Bus_InitStatus = 0
ENDIF

END SUB

*****从站节点特殊参数配置*****
'通过 SDO 方式修改对对象字典的值修改从站参数(具体对象字典查看驱动器手册)
*****
GLOBAL SUB Sub_SetNodePara(iNode,iVender,iDevice,Iaxis)
    IF iVender = $41B AND iDevice = $1ab0 THEN          '正运动 24088 脉冲扩展轴
        SDO_WRITE(Bus_Slot,iNode,$6011,Iaxis*$800,5,4)    '设置扩展脉冲轴 ATYPE 类型
        SDO_WRITE(Bus_Slot,iNode,$6012,Iaxis*$800,6,0)    '设置扩展脉冲轴 INVERT_STEP 脉冲
        输出模式
        NODE_IO(Bus_Slot,iNode) = 32 + 32*iNode          '设置 240808 上 IO 的起始映射地址
    ELSEIF iVender = $66f THEN                          '松下驱动器
        SDO_WRITE(Bus_Slot,iNode,$3401,0,4,$10101)        '正限位电平 $818181
        SDO_WRITE(Bus_Slot,iNode,$3402,0,4,$20202)        '负限位电平 $828282

        SDO_WRITE(Bus_Slot,iNode,$6091,1,7,1)            '齿轮比
        SDO_WRITE(Bus_Slot,iNode,$6091,2,7,1)
        SDO_WRITE(Bus_Slot,iNode,$6092,1,7,10000)        '电机一圈脉冲数
        SDO_WRITE(Bus_Slot,iNode,$607E,0,5,224)          '电机方向 0 反转 224
        SDO_WRITE(Bus_Slot,iNode,$6085,0,7,4290000000)    '异常减速度
        'SDO_WRITE(Bus_Slot,iNode,$1010,1,7,$65766173)    '写 EPPROM(写 EPPROM 后驱动
        器需要重新上电)
    ELSEIF iVender = $100000 THEN                        '汇川驱动器
        SDO_WRITE(Bus_Slot,iNode,$6091,1,7,1)            '齿轮比
        SDO_WRITE(Bus_Slot,iNode,$6091,2,7,1)
    ENDIF
END SUB

*****总线驱动 IO 映射*****
'通过 DRIVE_IO 指令映射驱动器对象字典中 60FD,60FE 输入输出状态, 要设置正确的 DRIVE_PROFILEE
或者 POD 后才可以正常映射
'DRIVE_PROFILE 模式包含 60FD/60FE
'iAxis - 轴号 iVender - 驱动器类型 i_IoNum - 输入输出起始编号
*****
GLOBAL SUB Sub_SetDriverIo(iVender,Iaxis,i_IoNum)

```

```

IF iVender = $66f THEN      '松下驱动器
  DRIVE_PROFILE(iAxis) = 5    '设定对应的带 IO 映射的 PDO 模式
  DRIVE_IO(iAxis) = i_IoNum

  REV_IN(iAxis) = i_IoNum      '负限位应 60FD BIT0
  FWD_IN(iAxis) = i_IoNum + 1  '正限位先对应 60FD BIT1
  DATUM_IN(iAxis) = i_IoNum + 2 '原点先对应 60FD BIT2

  INVERT_IN(i_IoNum,ON)        '特殊信号有效电平反转
  INVERT_IN(i_IoNum + 1,ON)
  INVERT_IN(i_IoNum + 2,ON)
ELSE
  DRIVE_PROFILE(iAxis) = 12    '不带转矩获取的模式,总线步进驱动器 IO 可以使用
  DRIVE_IO(iAxis) = i_IoNum

  REV_IN(iAxis) = i_IoNum
  FWD_IN(iAxis) = i_IoNum + 1
  DATUM_IN(iAxis) = i_IoNum + 2

  INVERT_IN(i_IoNum,ON)
  INVERT_IN(i_IoNum + 1,ON)
  INVERT_IN(i_IoNum + 2,ON)
ENDIF

```

END SUB

```

***** 总线 IO 模块映射 *****
'通过 NODE_IO(Bus_Slot,Node_Num)分配模块 IO 起始地址
*****
GLOBAL SUB Sub_SetNodeIo(iNode,iVender,iDevice,i_IoNum)
  IF iVender = $41B AND iDevice = $130 THEN '正运动 EIO1616
    NODE_IO(Bus_Slot,iNode) = i_IoNum
  ENDIF

```

END SUB

```

***** 总线驱动器回零 *****
'驱动器回零
*****
GLOBAL SUB Sub_SetDriverHome(iAxis,Imode)

```

```

  TRIGGER
  LOCAL home_sp1,home_sp2,home_mode,home_offset,home_acc
  home_sp1 = 10000
  home_sp2 = 10000
  home_acc = 1000000
  home_mode = Imode
  home_offset = 0

  BASE(iAxis)
  UNITS = 1000
  AXIS_STOPREASON = 0
  DATUM(21,Imode)
  WAIT IDLE
  IF AXIS_STOPREASON = 0 THEN

```

```

        ?"回零成功"
    ELSE
        ?"回零失败" ,"停止原因: ",AXIS_STOPREASON,"状态字 0X",HEX(DRIVE_STATUS)
    ENDIF
END SUB

```

Rtex 初始化程序

使用要求：控制器带 RTEX 接口，采用松下 RTEX 伺服驱动器。

```

RAPIDSTOP(2)
WAIT IDLE
FOR i=0 TO 10                '取消原来的总线轴设置
    ATYPE(i)=0
NEXT

SLOT_SCAN(0)                '开始扫描
IF RETURN THEN
    ?"总线扫描成功","连接设备数: "NODE_COUNT(0)
    ?"开始映射轴号"
    AXIS_ADDRESS(0)=0+1      '映射轴号
    ATYPE(0)=50              'Rtex 类型, 50 位置控制, 51 速度控制, 52 力矩控制
    DRIVE_PROFILE(0)=1      '伺服带 IO 映射
    DISABLE_GROUP(0)        '每轴单独分组
    ?"轴号映射完成"
    DELAY (100)

    SLOT_START(0)            '总线开启
    IF RETURN THEN
        ?"总线开启成功"
        DRIVE_CLEAR(0)      '清除 RTEX 总线远程轴错误
        DATUM(0)            '清除控制器错误
        DELAY (100)
        ?"轴使能准备"
        AXIS_ENABLE(0)=1    '轴 0 使能
        WDOG=1              '使能总开关
        ?"轴使能完成"
    ELSE
        ?"总线开启失败"
    ENDIF
ELSE
    ?"总线扫描失败"
ENDIF
END

```

第二十四章 错误与调试

控制器在使用过程中，因为接线不对、程序逻辑问题、指令使用错误等各种原因，会出现电机没有按照设定轨迹运行、控制器报错等各种问题。

如何来排查原因和解决问题，第一原则是关闭其他软件，使用 RTSys 软件排查，需要熟练使用以下功能：手动运动调试、断点调试、示波器抓取、寄存器查看、在线命令发送、程序打印信息、IO 口快速检测。

24.1 常见问题列表

现象	排查方法
电机不动	手动运动调试
触摸屏寄存器数据值不对	寄存器查看
没有按照程序预期运行	断点调试+打印程序信息
输入输出不起作用	IO 口快速检测
机台抖动大	示波器抓取

问题排查

程序报错先排查程序问题：



当程序运动出错后，RTSys 软件会显示出错信息，如果出错信息没有看到，可以通过命令行输入 ?*task 再次查看出错信息，双击出错信息可以自动切换到程序出错位置。下表中未找到对应代码的，请查看“[错误码列表](#)”章节。

问题	可能原因
2043:Unknown function is met	不认识的标识符，控制器不支持的功能
stop of error:2049 Line not ended.	①部分命令必须占用一整行 ②GSUB 调用不需要括号()
stop of error:2033 Unknown label is met	①未定义的变量或数组 ②未定义的 SUB 过程 ③有定义数组，但是定义的语句没有执行到，可能是对应文件没有设置自动运行
2048:Function can only be used in expression	函数必须返回值，不用在一行的开头地方
2064:Param few	函数参数过少
2063:Param too many	函数参数过多
2072:Need = sign	未写“=”号
2060:Syntax format error	指令语法错误
error:1010	重复暂停
error:1011	没有运动，无法暂停

程序运行错误解决后，仍然不能正常运行，若电机不动，再检查以下设置。

1. 驱动器原因：

驱动器的出厂设置一般没有反转 IO 电平，会导致驱动器限位报警，要根据驱动器手册设置限位电平反

转。比如松下伺服要将 Pr4.01、Pr4.02 的参数分别设置为 010101h（65793）、020202h（131586）。其他品牌驱动器请根据相关驱动器手册操作。

对应参数	出厂设置值（10 进制）	位置控制/全闭环控制	
		信号名称	逻辑
Pr4.00	00323232h（3289650）	SI-MON5	常开（ON）
Pr4.01	00818181h（8487297）	POT	常闭（NC）
Pr4.02	00828282h（8553090）	NOT	常闭（NC）

信号名称	记号	设定值	
		常开（ON）	常闭（NC）
无效	-	00h	设定不可
正方向驱动禁止输入	POT	01h	81h
负方向驱动禁止输入	NOT	02h	82h

2. 程序可能原因：

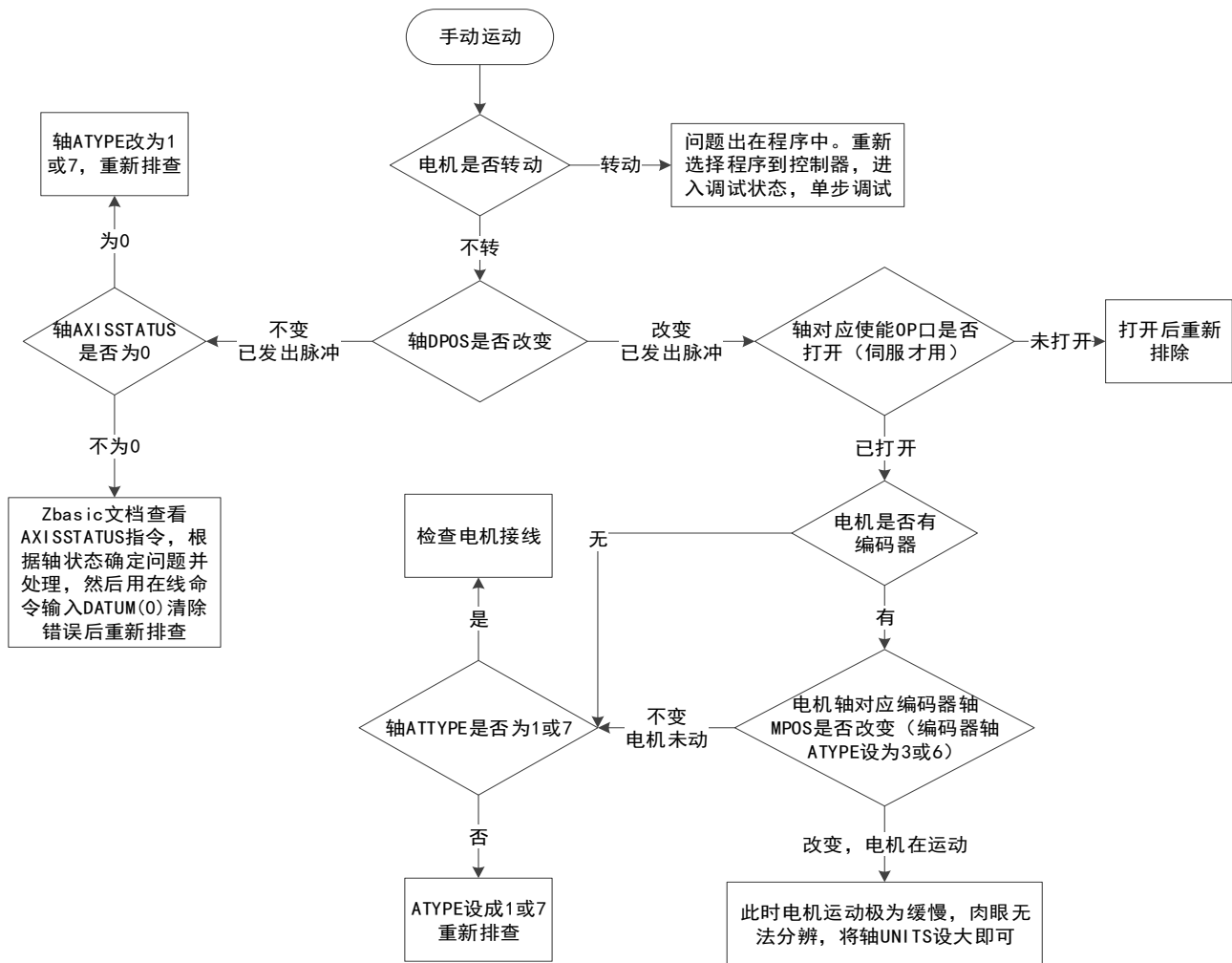
- 1) UNITS 设置过小，电机运动极为缓慢。肉眼无法分辨。
- 2) 电机处于异常状态（限位、报警…），无法运动，打印 AXISSTATUS 的值判断。
- 3) 电机接线错误，发出脉冲无法正确传递。
- 4) 轴使能 OP 口未打开（伺服电机才需要打开）。
- 5) 程序处理使得电机无法运动，下载空程序确认。
- 6) 驱动器报警。

以下原因为总线轴才会出现：

- 7) 总线扫描、开启失败，打印返回值确认。
- 8) 未打开 WDOG 总使能、AXIS_ENABLE 轴使能指令。
- 9) 驱动器状态设置错误，具体查看驱动器手册。

3. 问题排查步骤：

- 1) 使用 RTSys 软件进行问题排查。
- 2) 关闭除 RTSys 的其他连接控制器的软件、程序等，避免外部因素影响。
- 3) 使用 RTSys 下载一个空程序到控制器，避免内部因素影响。
- 4) 打开 RTSys 的“工具”-“手动运动”和“视图”-“轴参数”，便于操作和查看。
- 5) 脉冲轴按照下方步骤进行排查。



电机只能单向运动，可能原因：

1. 电机处于限位状态，查看 AXISSTATUS 确认。
2. 电机控制模式不对，INVERT_STEP 设置为相应模式（双脉冲或脉冲+方向）。
3. 电机接线问题，确认接线。

24.2 解决办法

手动运动调试

手动运动可以排查电机接线问题。

关闭所有除 RTSys 的软件，同时使用 RTSys 连接控制器，下载空程序，并在“视图”-“轴参数”中选择轴号，手动设置好轴类型 ATYPE、脉冲当量 UNITS、加速度 ACCEL、减速度 DECEL、速度 SPEED，然后打开“工具”-“手动运动”，手动操作电机。（下图为 RTSys V1.00.02 版本）

手动运动

轴	轴类型	脉冲当量	加速度	减速度	运动速度	指令位置	左转	右转	距离	绝对	反馈位置	运动状态	轴状态		
0	1	100.000	1000.00	1000.00	100.000	0.000	左	右		☐	运动	0.000	-1	0h	停止
1	1	100.000	1000.00	1000.00	100.000	0.000	左	右		☐	运动	0.000	-1	0h	停止
2	7	1.000	10000.0	0.000	1000.00	0.000	左	右		☐	运动	0.000	-1	0h	停止
3	7	1.000	10000.0	0.000	1000.00	0.000	左	右		☐	运动	0.000	-1	0h	停止
4	7	1.000	10000.0	0.000	1000.00	0.000	左	右		☐	运动	0.000	-1	0h	停止
5	7	1.000	10000.0	0.000	1000.00	0.000	左	右		☐	运动	0.000	-1	0h	停止

操作方法：按住“左”/“右”不放，电机持续运动，松开停止。“指令位置”显示当前发出的脉冲 DPOS（单位为 UNITS）。填写“距离”参数，点击“运动”，勾选“绝对”时，电机运动到距离参数位置；不勾选“绝对”时，电机按距离参数继续运动。

“左”“右”操作时可能会出现的问题及解决办法：

1. 电机不动，但 DPOS 改变。

此时控制器脉冲已发出，检查驱动器有无报警、电机接线。

也可能是 UNITS 设置过小，电机在转动，但是转动不明显。

2. 电机只往一个方向转动。

查看电机控制方式，目前控制器脉冲轴只能用双脉冲和脉冲+方向两轴控制模式，不可使用正交脉冲控制。

3. 只操作某一边时电机才转动。

检查电机接线。

电机当前控制模式与控制器当前控制模式不同，控制器默认为脉冲+方向控制，使用 INVERT_STEP 指令可以修改。

4. 电机不动，DPOS 也不改变。

检查轴参数 AXISSTATUS 是否报警。

5. 双电源供电的控制器，检查 IO 24V 是否正常接线和供电。

断点调试

断点调试可以查看程序运行的具体过程，主要用于判断程序逻辑错误。配合监视内容可以查看程序每执行一步对寄存器、变量、数组等的影响。调试程序 and 控制器程序必须一致。

断点快捷键 F9 添加，可以添加多个，调试完毕后将所有断点移除后再次下载程序到控制器运行。

对于使用 RTSys 软件开发的程序，连接好控制器，点击“调试”-“启动/停止调试”进入调试状态。详情参见“[程序调试](#)”章节。

Basic1.bas

```

10  DECEL = 2000
11
12  ' 计算TABLE的数据
13  DIM deg, rad, x, stepdeg
14  stepdeg = 2 ' 可以通过这个来修改段数, 段数
15  FOR deg=0 TO 360 STEP stepdeg
16      rad = deg * 2 * PI / 360
17      X = deg * 25 + 10000 * (1 - COS(rad))
18      TABLE (deg/stepdeg, X) ' 存
19      TRACE deg/stepdeg, X
20  NEXT deg
21
22  TRIGGER ' 触发示波器采样
23  WHILE 1 ' 循环运动
24      CAN(0, 360/stepdeg, 0.1, 300) ' 虚拟跟踪
25  WAIT UNTIL IDLE ' 等待运动停止
26  WEND
27  END
28
  
```

监视

监视内容	值
dpos	2040.73000
mpos	2040.73000

任务

任务	状态	文件和行号
0	Running	BASIC1.BAS, line: 25

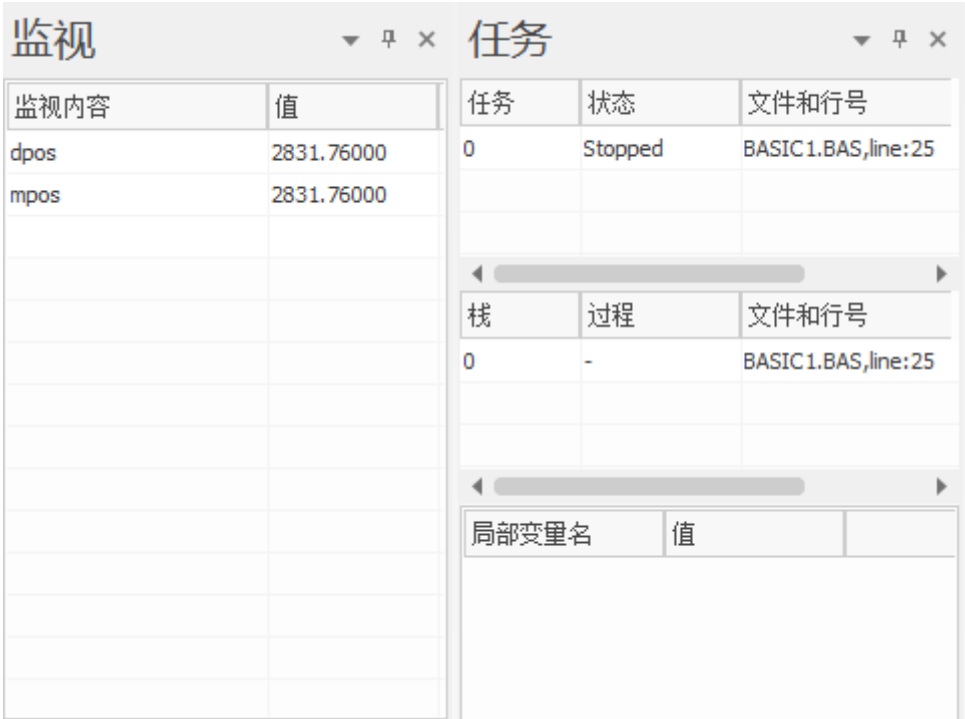
栈

过程	文件和行号
0	- BASIC1.BAS, line: 25

局部变量名

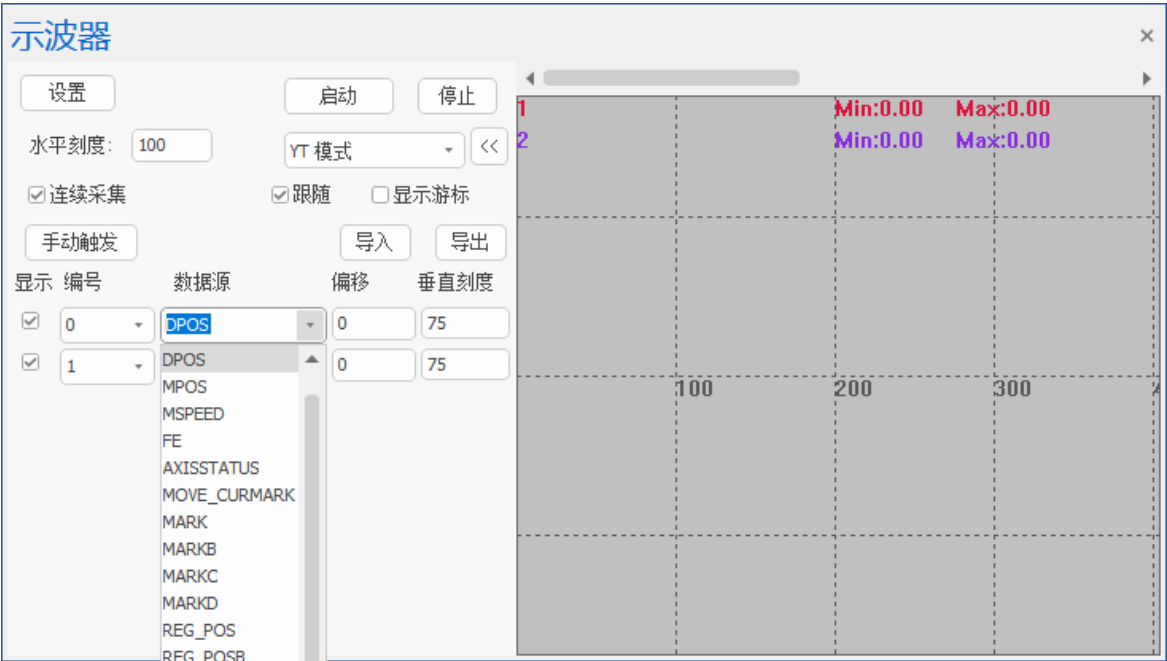
局部变量名	值
-------	---

此时可以查看个任务运行情况、监视内容、子函数调用过程、子函数局部变量值。



示波器抓取

示波器可以抓取各类数据，数据具体种类查看“工具”-“示波器”-“数据源”。
示波器抓取一般用来判断电机实际运行速度和实际位置。

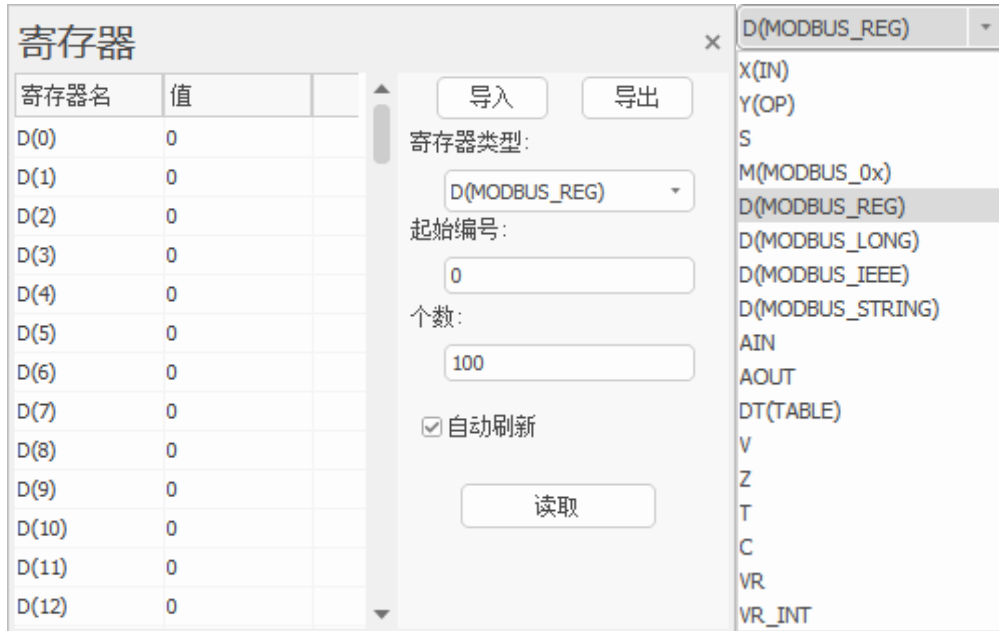


XY 模式可以查看二维的路径轨迹，需要将前两个通道的“数据源”设置为 DPOS 或 MPOS。具体使用方法查看“[示波器的使用](#)”章节。

机台实际运行抖动大时，可以用示波器抓取编码器 MSPEED，查看波形是否光滑；如果光滑说明脉冲发送平稳，此时查看速度曲线加减速过程是否过陡，匀速时速度值是否过大。

寄存器查看

使用 RTSys 软件可以方便的查看寄存器的数据，点击“工具”-“寄存器”查看。寄存器种类有 IN、OP、MODBUS_4X、MODBUS_0X、TABLE、VR、AIN、AOUT。



必须支持 PLC 功能的控制器才能使用。

比如出现触摸屏某个寄存器参数没有按照预期变化时，此时可直接查看触摸屏对应的寄存器类型及编号，确认控制器中此寄存器数值有无变化。如果变化，说明控制器与触摸屏通讯出现问题，检查接线和通讯参数设置；如果无变化，说明程序执行有问题，断点调试检查程序执行是否正确。

在线发送命令

在线命令可以实时执行发送的指令，一般用于直接判断控制器指令功能是否正常。

比如程序已经执行，但实际电机没有按照程序预设功能运行，无法确定是否是程序中其他任务或流程影响，还是控制器功能错误。此时就可以下载空程序到控制器，然后直接在线命令发送功能指令，观察现象即可判断问题原因。

打印程序信息

在各段程序间打印信息，可以方便的查看程序段是否执行，执行了几次，执行时对应的参数是什么。

打印指令为 PRINT，也可以使用简写“?”（注意是英文符号）。

```
1  DI ccv(20), aa, bb
2  ccv="ad"
3  aa=100
4  bb=300
5
6  BASE(0, 1, 2)
7  ATYPE=1, 1, 1
8  UNITS=100, 100, 100
9  SPEED=100, 100, 100
10 ACCEL=1000, , 1000, 1000
11
12 STOPTASK 1
13 RUNTASK 1, aaa

14
15 WHILE 1
16   ?bb
17   MOVE(bb)
18 WEND
19 END
20
21 SUB aaa(num)
22   WHILE 1
23     ?aa
24     MOVE(aa)
25   WEND
26 ENDSUB
```

IO 口快速检测

RTSys 连接控制器，点击“工具”-“输入口”、“输出口”，同时，将输出口和输入口一对一连接起来（输入口和输出口的 EGND 端子内部是通的），然后操作“输出口”，可以查看到“输入口”中对应状态变化。

某些控制器 IO 口需要另接 24V 电源给 IO 单独口供电，不接电源无法输出。

如果检测扩展板 IO，先确认扩展模块与控制器能成功通讯（CANL 和 CANH 接线时是否接入 120 欧电阻；然后确认拨码开关设置是否正确，不能覆盖控制器原本 IO 编号）；再确认主电源与 IO 电源是否接好；最后按照上面所述进行检测。

轴参数状态判断

轴的基本参数在轴参数窗口里随时查看，例如 ATYPE、UNITS、SPEED 等重要参数的设置情况。在此窗口可直接修改轴参数，修改完成立即生效，只读的参数不支持修改。

轴运行状态判断主要依据 IDLE、AXISSTATUS 和 AXIS_STOPREASON 这三个指令，在轴参数窗口对这三个指令参数实时监控。

轴参数						
轴选择		参数选择				
	轴0	轴1	轴2	轴3	轴4	轴5
COMMENT						
ATYPE	0	0	0	0	0	0
UNITS	1	1	1	1	1	1
ACCEL	1000	1000	10000	10000	10000	10000
DECEL	1000	1000	0	0	0	0
SPEED	100	100	1000	1000	1000	1000
CREEP	100	100	100	100	100	100
LSPEED	0	0	0	0	0	0
MERGE	0	0	0	0	0	0
SRAMP	0	0	0	0	0	0
DPOS	0	0	0	0	0	0
MPOS	0	0	0	0	0	0
ENDMOVE	0	0	0	0	0	0
FS_LIMIT	200000000	200000000	200000000	200000000	200000000	200000000
RS_LIMIT	-200000000	-200000000	-200000000	-200000000	-200000000	-200000000
DATUM_IN	-1	-1	-1	-1	-1	-1
FWD_IN	-1	-1	-1	-1	-1	-1
REV_IN	-1	-1	-1	-1	-1	-1
IDLE	-1	-1	-1	-1	-1	-1
LOADED	-1	-1	-1	-1	-1	-1
MSPEED	0	0	0	0	0	0
MTYPE	0	0	0	0	0	0
NTYPE	0	0	0	0	0	0
REMAIN	0	0	0	0	0	0
VECTOR_BUFFERED	0	0	0	0	0	0
VP_SPEED	0	0	0	0	0	0
AXISSTATUS	0h	0h	0h	0h	0h	0h
MOVE_MARK	0	0	0	0	0	0
MOVE_CURMARK	-1	-1	-1	-1	-1	-1
AXIS_STOPREASON	0h	0h	0h	0h	0h	0h
MOVES_BUFFERED	0	0	0	0	0	0
轴参数	帮助 属性					

1、IDLE 指令用于判断加在轴上的运动指令是否完成，运动中返回 0，运动结束返回-1，程序中一般使用 WAIT IDLE(轴号)语句判断轴状态。

2、AXISSTATUS 指令查看轴的各种状态。按十进制显示数值，按二进制对应位判断状态，可同时发生多个错误。

3、AXIS_STOPREASON 指令轴历史停止原因锁存，写 0 清除，按位锁存，锁存的是 AXISSTATUS 的信息。

附录一 错误码列表

错误码	错误码描述	可能原因	解决方法
各个模块错误码			
nand 模块(存储模块)			
101	路径错误		1.尽量使用带盘符的路径
102	读取错误		
103	写入错误		
104	芯片错误		
105	名字过长		
106	文件已经打开		
107	没有找到文件		
108	磁盘错误		
109	没有重启		
110	没有空间		
111	只读		
ecc 模块(存储数据纠错模块)			
112	ecc 错误, 1 个 bit 错误能修正		
113	ecc 没有设置		
114	ecc 错误, 多个 bit 错误, 无法修正		
bsp 模块(驱动模块)			
120	bsp 错误, 参数错误	控制器驱动错误	可联系技术工程师排查
121	bsp 错误, 检查错误	控制器驱动错误	可联系技术工程师排查
122	bsp 错误, 等待超时	控制器驱动错误	可联系技术工程师排查
123	bsp 错误, 状态错误	控制器驱动错误	可联系技术工程师排查
外部错误码			
201	子模块不存在		1.zmio_offset 设置为负数或超过输入输出起始编号范围(设置数值须为 8 的倍数), 例如 zmio_offset = -8
202	控制器与 zmio 模块通信失败		
210	文件过大		1.下载的 zpj 工程或 zar 文件过大, ?*max 对应检查控制器规格
211	文件大小错误		
212	状态错误	Resume 时为非暂停状态	1.Resume 时为非暂停状态, 检查 RT 运行状态是否切换过快, 一般在 MotionRT7 出现。
213	文件下载上传出错, 丢包	PC 函数调用返回此错误	1.PC 函数调用返回此错误
214	下载文件的长度校验错误		
215	缓冲长度不够	发送字符串命令过长时返回此错误	1.发送字符串命令过长时返回此错误。

216	EPCS 不存在	flash 芯片异常	1.联系技术支持，返修
217	控制器不支持或禁止的功能		
218	调用传递的参数错误		
219	下载冲突，同时启用了多个文件下载		1.下载并发，检查是否同时启动了多个文件下载
220	文件名错误，有特殊字符		1.检查文件名是否存在不支持的字符
221	文件名错误，超过长度		1.检查文件名是否超过长度
222	文件不存在		1.尝试打开了不存在的文件，检查工程目录是否存在报错提示的文件
223	受到密码保护，LOCK 后返回		1.输入正确密码
224	受到密码保护，密码尝试太快		1.短时间内频繁解锁控制器且密码错误
225	未知错误		
226	磁盘空间错误		
227	固件版本太老或者固件不匹配		1.更新最新固件 2.更换匹配该控制器的固件
228	文件打开失败		
229	创建网络连接失败		
230	bind 失败		
231	文件读取失败		
232	文件写入失败		
233	链接加密		
234	固件检查错误，dll 要更新		
235	文件删除错误		
236	路径错误		
237	文件关闭错误		
240	XPCI 子卡状态错误		
241	XPCI 子卡内存资源错误		
242	子卡配置错误		
243	不支持的子卡		1.检查 MotionRT7 的驱动版本是否过旧，新卡不能使用旧驱动
244	内存过大		
芯片自检错误			
260	硬件错误，LED 闪烁提示(错误码-260)次		
261	磁盘没有格式化		
262	RTC 错误		
263	NORFLASH 错误		1.强干扰可能出现，重启仍无法恢复或频繁出现联系技术支持，返修 2.XPCIe 和可能与转接线不良有关

264	RAM 错误		1.检查网络是否稳定 2.硬件器件损坏，联系技术支持，返修
265	NANDFLASH 错误		1.强干扰可能出现，重启仍无法恢复或频繁出现联系技术支持，返修 2.XPCie 和可能与转接线不良有关
266	U 盘错误		1.检查 U 盘是否插稳或不支持 2.接口损坏需要维修
267	FPGA 错误		1.联系技术支持，返修
268	以太网硬件错误		1.联系技术支持，返修
269	探针错误		
270	其他错误		
软件错误			
271	备份电源错误		1.联系技术支持，返修
272	子卡不存在		
273	ID 文件丢失		1.联系技术支持，返修
274	系统文件丢失		1.联系技术支持，返修
275	无主控，子卡上产生		
276	程序文件校验错误		1.0 系产品检查请是否有 ROM 文件 2.检查 ZAR 文件是否有错误
277	程序文件错误导致没有启动		1.检查程序文件是否有缺失
278	ZAR 文件校验 apppass 错误		
279	ZAR 文件校验 ID 错误		
280	BAS 文件数量超过控制器限制		1.检查 bas 文件数是否超过控制器规格，?*max 的 max_file 选项，每个控制器不一样
281	子卡 ID 冲突或者多主冲突		
282	不支持的功能	从卡 atype 不支持时闪烁	1.控制器不支持的功能，检查选型配置 2.检查是否控制器新的功能，需要更新固件
283	set 文件错误	参数文件丢失	可以通过强制修改需要 flash 保存的参数进行自动生成 set 文件
284	zar 文件与控制器不匹配		
285	图片文件错误		
286	字体文件错误		
287	.c 文件函数语法错误	一般为 C 语言程序问题	用户写的 c 语言问题则用户修改 c 语言程序，如果为底层 c 语言出问题则反馈技术更新固件
288	以上异常，导致下次开机报错		
289	复位太快		
290	驱动程序初始化出错		

291	fpga 错误		1.尝试重启 2.反馈售后返修或更换设备
292	mmap 资源不够		
293	MotionRT 试用到期		1. 检 查 是 否 授 权 卡 配 置 了 License 2.试用模式请重启 MotionRT
294	CAN 驱动 id 错误	CAN 设备的驱动 id 错误	
295	驱动丢失	驱动找不到	
296	动态模块接口缺失		
298	中断中不能执行长时间操作		
299	超时		
300	no mmap		
zmotion			
1000	运动模块返回错误偏移		
1001	必须是插补状态		
1002	无运动缓冲		
1003	不能位于插补状态		
1004	从轴运动中		
1005	不支持得运动功能		
1006	圆弧位置错误		
1007	椭圆 AB 参数错误		
1008	运动模块输入参数错误		
1009	运动中，无法操作		
1010	暂停等重复操作		
1011	IDLE 无法做暂停等操作		
1012	当前运动不支持暂停		
1013	找不到暂停点		
1014	ATYPE 不支持		
1015	ZCAN 得 ATYPE 冲突		
1016	轴不支持的功能		
1017	FRAME 校正数据错误		
1018	FRAME 校正数据过少		
1019	FRAME 校正数据满足条件的数据过少		
1020	FRAME 校正数据辅助参数过少		
1021	FRAME 校正数据间隔过小，小于关节轴数		
1022	FRAME 的输入坐标错误		
1023	FRAME 状态下坐标不能强制修改		
1024	FRAME 逆解异常		
1025	不是 FRAME 状态		
1026	FRAME HAND 错误		
1027	姿态在插补中不能切换		

1028	特殊关节轴与虚拟轴脉冲当量需一致		
1029	FRAME 调用的 Init 参数错误	距离参数或者角度参数超过范围，注意角度单位	
1030	CORNERMODE 7 位设置了但不支持此运动		
1031	CORNERMODE 7 位设置了但不是 FRAME 状态		
1032	AXIS_ADDRESS 错误		
1033	插补轴数过多		
1034	INTCYCLE 超时		
1035	迭代超过次数		
1036	迭代找出最优解但验证错误		
1037	机械手不可在运动状态执行	frame_rotate 指令执行发现 关节主轴和虚拟主轴 idle 都为 0	
1038	机械手 frame 不支持修改功能		
1040	叠加关系错误		
1041	叠加关系错误，过多		
1051	输入的圆弧 MOVER2_ARC 点位错误，如出现 3 点共线状态		
1052	当前点位于奇异区域里，不可执行轨迹运动指令		
1053	当前轨迹运动指令经过奇异区域，不可执行轨迹运动指令		
1054	轨迹运动的目标点位于奇异区域，不可执行轨迹运动指令		
1055	平滑系数 ZSMOOTH 参数设置超出[0,100]范围值		
1056	拱形运动参数设置出现错误		
2000	RTBasic 模块偏移，模块内参数错误		
2001-2020	RTBasic 模块内部错误		1.根据提示信息检查是否不同类型变量定义重名引起错误 2.根据提示信息检查是否在函数打印未赋值变量等
2000	参数错误		
2001	对象存在		
2002	对象不存在		
2003	没有位置		
2004	致命错误		
2006	类型不一致		

2007	期望没有出现		
2008	读取到的不是操作符		
2009	读取到的不是操作数		
2010	自动加载后退出		
2021	手动停止		
2022	因其他任务错误导致本任务停止		
2023	试图修改只读状态参数		1.只读参数不能被修改
2024	数组越界		1.超过了定义的数组个数，根据提示信息修正定义的数组个数；提示：数组索引从 0 开始
2025	变量数超过控制器规格		1.超过了控制器能够支持定义的变量数量，?*max 检查 max_sub 项，如需更多请重新选型
2026	数组数超过控制器规格		1.超过了控制器能够支持定义的数组数量，?*max 检查 max_sub 项，如需更多请重新选型
2027	数组空间超过控制器规格		1.超过了控制器能够支持定义的数组空间，?*max 检查 max_sub 项，如需更多请重新选型
2028	SUB 数超过控制器规格		1.超过了控制器能够支持定义的函数数量，?*max 检查 max_sub 项，如需更多请重新选型
2029	标识符命名错误	识别到指令书写错误或未加注释的中文	1.根据提示信息检查代码中的函数定义、变量定义等是否含有不支持的符号
2030	标识符命名过长		1.根据提示信息检查代码中的函数定义、变量定义等是否太长，控制器类型不一样限制不同
2031	没有右括号		1.根据提示信息检查代码中的括弧是否完整 2.根据提示信息检查括号格式书写或括号中间是否存在特殊字符
2032	不认识的字符	指令参数逗号为中文符号报此错误	1.根据提示信息检查代码中的是否存在不支持的字符
2033	表达式中碰到不认识的名称	未定义的变量或数组	1.根据提示信息检查代码是否有书写错误的指令，不支持的指令
2034	SUB 不能再表达式中使用		1.根据提示信息修改 SUB 使用方式，SUB 函数只能通过调用使用

2043	不认识的命令标识符。当前行第一个标识符名称	指令书写错误	1.根据提示信息检查代码是否有书写错误的指令，不支持的指令
2044	堆栈溢出	SUB 函数递归调用超过系统规定的次数	1.根据提示信息检查 SUB 函数递归调用是否超过次数"?*max"打印规格,查看 max_callstack 参数,不同控制器规格不一样 2.根据提示信息检查函数是否被相互调用
2045	数学表达式太复杂,不同控制器规格不一致		1.根据提示信息检查并修正数学表达式,不同控制器的规格不一样
2046	没有找到结束引用标号		1.根据提示信息检查并修正引用标号
2047	指令没有返回值,不能读取		1.根据提示信息把相关指令或函数取消在表达式中或在线命令中使用
2048	函数必须返回值,不能再一行开头		1.根据提示信息检查是否执行了需要返回值的函数但表达式没有输出返回值,如直接执行 SIN(1)
2049	特殊指令必须单独一行	部分指令必须占用一整行	1.根据提示信息把相关指令独立一行编写
2050	参数或数组需要索引		
2051	变量不能使用索引		
2052	数组重定义且长度不一致	相同数组定义多次	1.根据提示信息检查数组是否被重定义并且定义的长度不一致
2053	数组定义长度参数错误,负数或过大		
2054	标识符已经定义为 sub 过程	sub 过程标识符不可以再次定义	1.根据提示信息检查是否有函数名称被重复定义
2055	标识符已经定义为参数		1.根据提示信息检查是否有参数名称被重复定义
2056	标识符预留,不能使用		1.根据提示信息检查定义的标识符名称与已有参数指令冲突,如 MPOS 等
2057	出现不识别的字符	例如"&"字符系统无法识别	1.根据提示信息检查代码中的函数是否传入未定义的数值
2058	sub 调用重复出栈		1.根据提示信息检查 SUB 调用逻辑并更正
2060	语法格式错误		1.代码中存在语法错误的行,修正使用语法
2061	参数溢出	参数过多	
2062	函数参数范围错误		1.根据提示信息检查指令参数或自定义函数参数输入范围是否超出 2.根据提示信息检查任务号是

			否超出?*max, max_task 所定义 3.根据提示信息检查执行的一行命令是否输入过多内容
2063	函数参数过多		1.根据提示信息检查指令参数或自定义函数参数个数输入范围是否超出
2064	函数参数过少		1.根据提示信息检查指令必要参数或自定义函数参数个数输入缺少
2065	缺少操作数		1.根据提示信息检查运算表达式是否完整
2066	操作符后面缺少操作数		1.根据提示信息检查运算表达式是否完整
2067	操作符前面缺少操作数		1.根据提示信息检查运算表达式是否完整
2068	系统不认识的操作符		1.根据提示信息检查运算表达式是否完整
2069	缺少双目操作符	二个指令在同一行，中间需要操作符连接	1.根据提示信息检查运算表达式是否完整
2070	CALL 必须调用 SUB		1.根据提示信息更正 CALL 指令后调用的内容，必须为 SUB 类型
2071	没有 AUTO 不启动		
2072	需要赋值符号	数据之间不加逗号用空格表示	1.根据提示信息更正表达信息需要添加的赋值符号
2073	空文件		
2074	SUB 定义的标识符名称冲突		1.根据提示检查参数定义并纠正
2075	要启动的任务在运行中		1.根据提示信息检查执行的任务号是否有冲突
2076	多个参数要使用逗号隔开		1.根据提示检查参数使用格式并纠正
2077	括号不匹配，无左括号		1.根据提示信息检查括号是否完整
2078	IF 判断嵌套太多		1.根据提示信息裁剪 IF 判断嵌套数量，每个控制器不一样
2079	循环语句嵌套太多		1.根据提示信息裁剪循环嵌套数量，?*max 中的 max_loopnest 包含具体规格，每个控制器不一样
2080	插补轴数太少		1.根据提示信息纠正必要的轴数量
2081	CONST 常量不能修改	定义好的常量数据再次赋值报错	1.根据提示信息纠正 CONST 用法
2082	命令不能从 pc 在线发送		1.输入了不支持在线发送的命令

2083	SUB 定义的参数太多		1.根据提示信息裁剪 local 参数量小于?*max 中的 max_local of one sub 包含具体规格，每个控制器不一样
2084	SUB 带参数，不能用于 GOTO 语句		1.根据提示信息正确使用 GOTO 语句
2085	局部定义的参数太多		1.减少 LOCAL 定义数量，每个控制器可能不一样
2086	LOCAL 变量名跟文件变量名或其他标识符名称冲突		1.根据提示信息更正冲突内容
2087	LOCAL 不支持数组定义		1.根据提示信息更正 LOCAL 指令使用方式
2088	GSUB 定义的参数字母重复		1.根据提示信息更正重复内容
2089	GSUB 定义的参数只能为单字母		1.根据提示信息更正非单字母内容
2090	不能修改只读参数		1.只读参数不能被修改
2091	GSUB_IFPARA 函数使用场合错误		
2092	除数为 0		1.检查代码中的数学公式是否包含除数为 0 并更正
2093	超过缓冲		
2094	在线命令阻塞时间过长		1.检查网络是否稳定 2.检查是否短时间有大量数据在传输 保证通讯链路稳定传输带宽合理
2095	参数重名		1.根据提示信息更正重名的参数
2096	变量没有初始化就使用		1.根据提示信息更正需要初始化的值
2097	轴号冲突		1.根据提示信息更正轴映射
2098	数据类型错误		
2099	内部错误		1.一般是定义命名冲突，根据提示更正相关内容
2100	SCANEDGE 个数过多		
2101	ZINDEX 类型不匹配		
2102	ZVOBJ 个数超过规格		
2103	ZVOBJ 重复定义并且不一致		
2104	ZINDEX 值错误		
2105	local 不支持指针		
2110	回调指令没有使能		
2120	结构定义冲突，不能多地方同时定义		
2121	名称与系统指令冲突		
2122	结构定义不能递归		
2123	结构 item 与结构名称冲突		

2124	语法错误, 需要结构类型		
2125	结构 item 错误		
2126	需要结构元素		
2127	需要结构变量		
2128	结构个数超出规格		
2129	结构元素超出规格		
2130	结构类型没有定义		
2131	数据类型没有定义		
2132	结构定义要在使用代码前		
2133	静态定义的不能动态删除		
2134	需要数组类型		
2135	数组定义使用二个括号		
2136	ref 类型错误, 不支持特定读写函数		
2137	系统 ref 类型错误, 不支持特定读写函数		
2138	ref 类型不能再常量表达式中使用		
2139	CONST 顺序错误		
2140	SUB 调用传递的参数重复		1.SUB 调用支持了特殊的机械手语法, 检查语法是否有问题
2141	等号语法必须是参数名称		
2142	不能索引的内容		
2143	local 定义其他类型, 类型不一致		
2144	ref 类型错误, 定义与实际不匹配		
2145	数组重定义大小必须一致		
2146	参考类型为空		
2147	特殊类型必须是 ref 定义		
2148	ref 类型错误, 定义与实际不匹配		
2149	没有左括号		
mange 错误			
2150	函数返回没有立即完成		
2151	函数不能立即返回, 当前表达式不支持		
2152	动态堆栈溢出		
2160	{ } 初始化层次错误		
2161	初始化需要 {		
2162	结构初始化必须要有 item 名称		
2163	重复定义		
2164	参数重名		
2165	SO 文件平台错误		so 文件编译平台须和控制器一致
2166	文件过多		

2167	文件内容错误		
2168	so 文件重定位错误		
2169	so 文件不认识的符号		
2200	函数回调没有完成		
zbasic			
2900	指针参数错误		
2901	系统错误，定义的标识符过多 包括变量，数组，过程，过程参数等		
2902	指针为空		
3201	超过缓冲		
3202	文件非正常结束		
3203	程序结构指令不配对	IF 指令后缺少 THEN	1.兼容性错误，检查组合指令是否完整，如 IF 逻辑缺少 THEN
3204	内部状态错误		1.兼容性错误，
3205	不支持的功能		1.存在控制器不支持的功能，检查选型规格 2.固件版本较老，无法对新功能适配
3206	内部调用参数错误		1.使用了当前控制器不支持的功能，确认选型是否有此配置 2.EtherCAT 配置文件没有被加载
3207	需要其他模块才能运行		
3212	未知错误		
3230			
3231	资源不够		
3232			
3233	OS 返回错误		
3241	等待超时		
3242	os 错误		
3243	U 盘没有插入	u 盘错误	1.检查 U 盘是否插入，插稳 2.检查 U 盘是否控制器能够支持识别的版本
3244	文件重复打开	u 盘错误	
3245	文件过大	u 盘错误	
3246	未知错误	u 盘错误	
3247	文件太短	u 盘错误	
3248	文件名错误	文件名有特殊字符	1.检查文件名中是否存在特殊字符或不支持的字符
3249	文件名过长	文件名太长导致，超过控制器支持的最大文件名长度	1.检查工程相关文件名是否过长并更正，各控制器规格可能不一致，更正为小于提示信息的字符长度
3250	文件不存在	文件打开失败	
3251	文件已经打开	文件已经打开	
3252	重命名重复	重命名的文件已经存在	

3301	圆弧的三点在一条线上		1.根据提示信息更正指令运用方式
3302	二条直线平行，没有交点		1.根据提示信息更正指令运用方式
modbus 模块			
3401	软件不支持		1.检查 MODBUS 主端参数是否正确，如长度超过 2.用最新 zdevelop 或者 RTSYS 软件
3402	消息响应超时		1.检查通讯配置是否不正确 2.检查通讯程序是否有阻塞、网络不稳定
3403	消息长度超过最大缓冲		1.发送数据超过最大限制，每个通讯协议和控制器规格不一样。
3404	Modbus 消息字节数或 ID 错误		
3405	fator 错误		
3406	没有回应		
3407	MODBUS 返回参数错误		
3408	MODBUS 返回不支持		
3410	阻塞中再次受到数据		
3420	MODBUS 错误转换		
3421	MODBUS 从端返回不支持的功能码		
3422	MODBUS 从端返回地址空间错误		1.主从地址空间不匹配，尝试读写不存在的寄存器地址，根据报错提示信息更正正确的寄存器地址
3423	MODBUS 从端返回数据长度不对		
3424	MODBUS 从端返回长度过长		
3425	MODBUS 从端返回错误		
3430	MODBUS 通讯读取间隔太快		1.赠加延时
ZCAN 模块			
3500	ZCAN 返回无子卡		
3501	ZCAN 返回无子卡		
3502	ZCAN 返回子卡无对应轴		
3503	不支持的功能		
3510	超时没有发送		
PLC（控制器端）			
4001	plc 存储模块错误		
4002	参数错误		
4003	未知类型		

4004	未知函数	调用了未知函数，或者调用的函数没有定义为 GLOBAL 类型	修改错误语法用法
4005	压栈太多 STL		
4006	压栈太多		
4007	程序太复制，BLOCK 太多		
4008	没有压栈 BLOCK		
4009	没有压栈 STL		
4010	没有压栈	使用 MPS 跟 MPP 指令时出现 MPP 指令多余 MPS 指令导致	根据报错信息确定报错位置修改脚本
4011	MC 不能位于 STL 中间		
4012	MC 层次错		
4013	stl 只能主文件主任务		
4014	文件内容错误		
4015	RET 必须在 STL 后面	RET 指令在 STL 指令前面报错	修改脚本将 RET 指令修改到 STL 指令后面
4016	超过寄存器范围		
4017	低于寄存器范围		
4018	L 没有定义		
4019	不支持 G 代码函数		
4020	不能 GOTO 跨 PLC 与 BASIC	BASIC 脚本中使用 GOTO 跳转到 PLC 中	修改错误语法用法
4021	PLC 主任务只有一个		
4022	语法错误		
4023	FOR NEXT 错误，不匹配		修改错误语法用法
4024	FOR NEXT 错误，无 NEXT		修改错误语法用法
4026	FOR MC 混用		修改错误语法用法
4027	FOR STL 混用		修改错误语法用法
4030	必须 PLC 主任务中使用		
4031	必须中断中使用		
4032	寄存器个数少		
4033	寄存器个数多		
4034	要 8 的倍数		
4035	寄存器标识错误		
4036	寄存器类型错误	STL 指令等其他指令使用错误的寄存器类型导致	通过报错信息确定报错位置修改脚本
4037	LV 个数超过		
4038	只读		
PLC(PC 端)			
4501	参数错误		
4502	未知错误		
4503	内存不够	内存超过规格	减少内存使用
4504	回流到母线	软元件直接导线并联到母线	将错误的连线删除

4505	回流	无通过软元件直接导线并 联至其他软元件	将错误的连线删除
4506	AND 类型指令不能直接接 母线	AND 类型跟母线中间不 存在其他元件	AND 类型跟母线中间不存 在其他元件导致，要不增 加元件要不删除此条梯 形图
4510	右边悬空没有接输出指 令	ANB 等指令或者元件右 边没有接输出指令	根据报错信息确定报错 位置修改脚本或者梯形 图
4511	最右边不是输出类型指 令	最右侧的元件不是输出 类型的元件	检查最右侧是否有输出 类型的元件
4512	最右边不能连接在一 起	最右侧二个输出类型的 元件用导线相连	检查最右侧是否还存在 导线相连
4513	输出类型指令必须在最 右边	输出类型的元件不处于 梯形图的最右侧，而是 在中间或者左侧	检查梯形图中间或者左 侧是否存在输出类型的 元件
4514	不支持的指令类型	梯形图中存在不支持 类型的元件参数	通过报错信息排查不 支持的类型位于梯形图 的那个位置
4515	内部错误		
4516	内部错误		
4517	寄存器没有值	调用的寄存器为空没有 数值	对于需要调用的寄存器 进行赋值操作
4518	DOT 值超过范围		
4519	索引寄存器超过范围	调用的寄存器超过寄存 器的个数范围	通过报错信息修改调用 寄存器的使用范围在规 定的个数范围内
4520	字符数过多		
4521	寄存器类型错误	脚本使用的寄存器类型 不是控制器标准的寄存 器类型	通过报错信息确定报错 位置修改脚本
4522	寄存器值错误	元件输入的寄存器值错 误或者指令输入的寄存 器值错误	通过报错信息找到寄存 器值错误元件或者指令 修改，一般为寄存器数 量的值错误
4523	寄存器个数过多	脚本输入的寄存器个数 过多出现	通过报错信息确定报错 位置修改脚本
4524	寄存器个数过少	元件没有设置寄存器	对于没有设置寄存器的 元件或者指令增加寄存 器
4525	STL 使用错误	STL 指令使用错误导致	通过报错信息修改脚本 语法
4526	RET 使用错误	RET 指令使用错误导致 ，RET 指令需要再 STL 指令后使用	通过报错信息修改脚本 语法
4527	重复 RET	重复使用了 RET 指令 或 RET 元件	将重复使用的 RET 指令 或者元件删除
4528	END 或 LBL 的位置错 误	END 或者 LBL 使用在 了错误的位置	根据报错信息确定报错 位置修改脚本或者梯形 图
4529	函数不能直接接母线	调用函数的元件直接接 在母线上	通过报错信息确定报错 位置修改脚本或者梯形 图

4530	出栈没有压栈	使用 MPS 跟 MPP 指令时出现 MPP 指令多余 MPS 指令导致	根据报错信息确定报错位置修改脚本
4531	MPS 太多	MPS 连续使用次数超过 11 次	根据报错信息确定 MPS 指令的连续使用次数并将超过的进行修改
4532	寄存器类型使用错误	指令使用了不支持的寄存器类型	通过报错信息确定报错的指令或者元件，使用正确的寄存器类型
4533	ANB 错误，块数不够	ANB 后面没有接其他内容，实际使用的块数跟需要用到的块数对不上	修改错误语法用法
4534	ORB 错误，块数不够	ORB 后面没有接其他内容，实际使用的块数跟需要用到的块数对不上	修改错误语法用法
4535	ANB 错误，输出操作后不能合并	在输出指令后再调用 ANB 指令导致	修改错误语法用法
4536	ORB 错误，输出操作后不能合并	在输出指令后再调用 ORB 指令导致	修改错误语法用法
4537	AND 直接接母线	AND 指令或者元件直接接在母线上，前面没有其他指令或元件	根据报错信息确定报错位置修改脚本或者梯形图
4538	OR 直接接母线	OR 指令或者元件直接接在母线上，前面没有其他的指令或元件	根据报错信息确定报错位置修改脚本或者梯形图
4539	OR 不能再 OUT 指令后面	在 OUT 指令后面使用 OR 指令导致	修改错误语法用法
4540	STL 和 MC 不能共用	STL 指令跟 MC 指令同时使用导致	修改错误语法用法
4541	函数不能直接接母线	函数直接接在母线上	修改错误语法用法
4542	@寄存器要括号	@寄存器时寄存器没有用括号	修改错误用法
4543	注释错误		
4544	梯形图列数过多	超过控制器支持的梯形图列数	梯形图列数过多，删除至支持的梯形图列数即可
4545	输出类型不能直接接母线	输出类型元件直接接到了母线上	1.将接到母线上的输出类型元件的线删除即可
HMI			
5000	LCD 号错误	HMI 运行任务超过控制器规格	1.超过了控制器支持的 HMI 个数，?*max 检查 max_hmi 个数，如需更多请更换选型
5001	HMI 文件错误	内部错误	1.联系技术反馈
5002	LCD 号冲突	多个 HMI 任务使用相同 LCD 编号	1.检查多个 HMI 文件 LCD 编号是否冲突

5003	不支持的对象	内部错误	1.联系技术反馈
5004	内存不够	7 系 HMI 内存设置过小或者其他系列控制器规格不支持	1.7 系打开启启动台调整 config 参数的 hmisize 2.其他系列请联系技术
5005	控件层次错误	PC 软件设置层数传入一个异常数值	1.联系技术反馈
5006	窗口号超过	窗口基本属性中的窗口编号设置过大	1.通过调小窗口号进行调试解决 2.使用满则联系技术解决
5007	无效窗口号	1.基本窗口打开一个不存在的窗口 2.指令 HMI_SHOWWINDOW 打开一个不存在的窗口	1.检查打开的基本窗口是否存在已经指令打开的窗口是否存在
5008	HMI 文件内容错误	内部错误	1.联系技术反馈
5009	窗口号相同	HMI 文件二个或多个窗口使用重复的窗口号	1.排查报错 HMI 文件的窗口号是否重复
5010	对象属性丢失	内部错误	1.联系技术反馈
5011	输入窗口有多个显示元件		
5012	动作类型错误	PC 软件设置动作传值出现异常导致	1.联系技术反馈
5013	事件过多	预留	
5014	返回上一个窗口失败		
5015	不能关闭基本窗口	使用指令 HMI_CLOSEWINDOW 关闭基本窗口	1.检查 HMI 文件的基本窗口并检查脚本关闭逻辑
5016	字体中找不到对应字符	此错误码不会报错, 当遇到不识别的字体不会显示(暂留)	1.不会出现, 不需解决
5017	必须在 HMI 任务中使用		
5018	控件类型不对	使用 HMI_SHOWWINDOW 等指令操作控件但控件不支持此功能时报错(例如功能键进行弹软键盘操作)	通过对应的报错信息查看参数设置跟 HMI 对应窗口的是否相符
5019	控件 ID 不存在	使用 HMI_SHOWWINDOW 等指令操作的控件不存在时报错(例如功能键进行弹软键盘操作)	通过对应的报错信息查看参数设置跟 HMI 对应窗口的是否相符
5020	控件 ID 冲突	不同控件设置了相同的元件编号导致	根据报错提示找到对应的窗口以及控件进行修改
5021	LCD 号错误	PC 上位机错误	PC 上位机错误
5022	找不到可用 LCD	PC 上位机错误	PC 上位机错误
5023	LCD 没有打开	PC 上位机错误	PC 上位机错误
5024	LCD 无数据	PC 上位机错误	PC 上位机错误
5025	程序复位	PC 上位机错误	PC 上位机错误
5026	LCD 已经打开	PC 上位机错误	PC 上位机错误

5027	不是网络 LCD	PC 上位机错误(300 使用的是内部 LCD 编号, 带 x 的示教盒使用的是网络 LCD 编号)	PC 上位机错误(300 使用的是内部 LCD 编号, 带 x 的示教盒使用的是网络 LCD 编号)
5028	不支持的压缩方式	预留	预留
5029	颜色深度不支持	控制器不支持对应的颜色深度	联系技术反馈
5030	不支持的数据类型	内部错误	联系技术反馈
5031	设备号错误		
5032	LCD_SET 不能使用	预留	
5033	设置 REDRAW 不能再 DRAW 阶段	在绘图函数中使用了 set_redraw 指令	set_redraw 指令为刷新函数指令, 必须在刷新函数中使用
5034	DRAW 函数只能在 DRAW 阶段	绘图指令在刷新函数中使用	绘图指令必须在绘图函数中使用, 例如 draw 开头类型的指令
5035	操作不能再 DRAW 阶段调用	操作控件的指令在绘图函数中使用	操作控件显示, 控制状态等指令不能再绘图函数中使用
5036	内部 LCD 分辨率固定		内部错误
5037	LCD 分辨率超过	设置的分辨率超过控制器支持的最大分辨率	可以通过 ?*max 打印查看 max_hmi 的 x 跟 y 参数, 此参数对应为分辨率大小
5038	库文件名错误	在使用文本库时调用的库文件名出现错误	检查使用错误文本库的控件或者指令修改为正确的文本库名称
5039	字符过多		
5040	元件属性丢失	内部错误	联系技术反馈
5041	没有输入显示元件		
5042	状态数过多		
5043	不支持的绘图属性		
5044	远程通讯设备名称错误	预留	
5045	远程通讯数据没有更新	预留	
5101	无效的日期格式	使用 SYSTEM 指令打印日期时使用了无效的格式(例如: 不是%+字母的格式)	检查报错位置, 使用正确的日期格式(%+字母)
5102	控件不存在	使用指令对一个不存在的控件进行操作(例如在修改控件文本等)	使用错误的控件 ID 修改为正确的控件 ID
5103	多边形点数太少或太多	使用绘制多边形点数少于 2 个或者多余 32 个会报错	检查点数是否有超过 32 个的限制 并且 推荐使用 DRAW_POLYGON2 指令, 不会出现参数过多导致脚本死掉的情况
5104	无空闲可用滚动条	初始化滚动条时使用了自动分配 ID 语法出现	注意释放没有使用的 ID 号, 如果 ID 号实在不够用请跟技术反馈
5105	无效的滚动条 ID	使用滚动条指令时调用了无效的滚动条 ID(例	使用正确的初始化滚动条 ID

		如滚动条初始化时使用了超过 31 以上的 ID)	
5106	尚不支持的功能	控制器使用了尚不支持的 HMI 控件或指令导致	联系技术反馈是否有新的固件更新
5107	未加载图形	CAD 指令进行操作时没有导入图形出现	请先使用 CAD 控件导入对应图形在进行其他 CAD 操作
5108	文件已损坏	导入 bin 文件等强格式的损坏格式会报错, 矢量图形等明码文件一般不会报此错误	将损坏的 bin 文件进行重新导出使用
5109	菜单参数错误	预留	预留
5110	导出 table 空间不足, 溢出	调用的 table 空间不足导致	通过指令增大 table 空间或者已是 table 最大空间时更换控制器型号
5111	不支持的数据类型	数据类型不是指令支持的类型	通过报错信息确定出错的指令, 更换为支持的数据类型
5112	指令不支持的控件类型	使用指令操作控件时, 控件类型跟指令支持的类型不匹配	通过修改控件 ID 使用匹配的控件进行操作
5113	数组溢出	传入数组下标超过最大值	检查数组大小以及传入的数组最大值是否超过
5114	内部错误	HMI 文件内部出现错误	联系技术反馈
5115	通道溢出	传入通道号超过最大值	预留
5116	动态资源文件超过最大支持个数	设置动态内存后, 背景图片从 flash 中加载图片超过控制器规格	将图片添加到项目或者减少从 flash 中加载图片
5120	取样库太多	资料取样库设置超过控制器规格	查看控制器允许支持的规格并减少资料取样库
5121	操作对象不存在/下标溢出	hmi 指令操作的对象不存在或者下标错误	检查对应指令设置
5122	宽高太小	趋势图或者数据群组控件设置宽高错误	检查对应指令设置
5123	不支持的指令参数	hmi 指令使用了不支持的指令参数	检查对应指令设置
5124	错误的图表显示范围	趋势图或者数据群组控件设置宽高错误	检查对应指令设置
5125	无效的指令参数	hmi 指令使用了无效的指令参数	检查对应指令设置
5126	超过最大支持级别	菜单控件或者树形图控件设置级别超过限制	检查对应的控件设置
5127	开始时间和结束时间相同了	排程控件设置的开始结束时间相同	检查对应的控件设置
5128	动画切换点数太小	动画控件设置的点数太少	动画控件至少需要设置二个点

5129	条码非法输入字符	条码控件输入非法字符, 例如中文, 特殊字符等	检查对应的控件设置
5130	TTF 字体初始化失败	TTF 字体初始化加载失败	检查 TTF 字体是否损坏
5131	无效参数	hmi 指令使用了无效的指令参数	检查对应指令设置
5132	配方内存空间太小, 不够内存编辑	预留	
5133	找不到对应配方组	预留	
5134	编辑失败	预留	
5135	不支持的配方文件	预留	
5136	配方文件导入失败	预留	
5137	行列号不存在	预留	
5138	不支持访问的属性(读或者写)	使用指令 HMI_CONTROLATTR 和 HMI_CONTROLATTRS 时遇到该元件不支持的属性	
5139	不支持创建的控件		
EtherCAT 总线错误			
6000	ECAT 模块错误, SLOT 编号错误		1.槽位号错误, 更正即可
6001	内部错误, 功能不支持		
6002	没有压栈		
6003	未知		
6004	mbox 被其他抢占		
6005	参数错误		
6006	支持的设备类型数超过限制		
6007	超过系统的 PDO 规格		
6009	操作 NODE 超过个数		
6010	从状态错误		
6011	不支持的从站		
6012	资源不够		
6013	从设备反应超时		1.主站长时间(如超过 400ms)多次向从站写入本地时钟数据没有得到从站回应, 需结合驱动器错误、控制器表现、以及产生故障时刻分析具体原因
6014	超过 SDO 缓冲最大设置		
6015	应答包 WKC 错误		1.从站返回的 WKC 计数错误, 需结合具体错误数据分析错误原因
6016	SDO 应答超长		1.从站应答 SDO 长度超长, 检查发送 SDO 数据类型是否正确

6017	SDO 应答错误		1.SDO 读或写操作的传输被伺服主动拒绝终止, 需结合具体发送的 SDO 内容分析错误原因, 如读取了一个在数据字典中不存在的数据对象, 或者在运行过程写 PDO 数据
6018	SDO 应答数据长度错误		1.从站应答 SDO 长度错误, 检查发送 SDO 数据类型是否正确, 大多为发送的 SDO 数据类型不正确或不支持
6019	WKC 超时		1.从站返回的 WKC 超时, 需结合驱动器错误、控制器表现、以及产生故障时刻分析具体原因
6020	STATE 切换超时		1.SoE 状态机切换超时, 主站长时间多次请求切换状态机没有得到从站正确回应, 需结合驱动器错误、控制器表现、以及产生故障时刻分析具体原因
6021	SDO ABORT, 驱动器返回错误	数据字典读写错误或写入了驱动器不支持的功能	1.SDO 读或写操作的传输被伺服主动拒绝终止, 需结合具体发送的 SDO 内容分析错误原因, 大多为发送的 SDO 不正确或不支持
6022	初始化错误		
6023	NODE PROFILE 错误, 不能设置		1.SDO 读或写操作的传输被伺服主动拒绝终止, 需结合具体发送的 SDO 内容分析错误原因, 如读取了一个在数据字典中不存在的数据对象
6024	轴 PROFILE 错误, 不能设置		
6025	轴数超过		1.总线轴数超过最大规格, 检查并更正正确轴数
6026	超过自定义 PDO 的缓冲		
6027	自定义个数超过		
6028	启动后不能修改 PROFILE		
6029	PDO 包长度超过系统规格		1.检查 profile 设置, 不需要用到的功能尽量减少配置 PDO, 特别在设备数量过多的时候, 每个控制器限制规格不一样
6030	启动前必须扫描		
6031	设备个数超过		
6032	超过 buff 长度		
6035	preset 与 profile 冲突		
6036	pdo 个数过多		1.检查 profile 设置, 减少 PDO 个数。
6037	特殊的 profile, 驱动器不支持		
6038	preset scan 不符合		

6039	preset 中空		
6040	没有扫描		
6042	设备不支持		
6043	esi 信息冲突		
6044	写错误		
6045	邮箱超时		1.邮箱检查或邮箱长时间（如超过 600ms）多次读写没有回应，需结合驱动器错误、控制器表现、以及产生故障时刻分析具体原因
6046	数据丢失		
6047	数据类型错误		
6048	PDO 不支持		
6049	模块不支持的子模块		
6050	模块子模块数量超过		
6051	模块不认识的子模块		
6055	操作的 PDO 类型长度不匹配		
6056	PDO 读写内容没有找到		
6057	PDO 的关键内容错误，比如 DRIVE STATUS		
6058	AL 状态读取出错		
6059	AL 状态错误，不是 OP 状态		
6060	驱动器报错		
6061	XmlEsi 缓冲不够		
6062	接收丢包		
6063	发送失败		
6065	IO 的 Pdo 必须字节偏移不为 0		
6066	IO 的 Pdo 不连续		
6067	DA 的 PDO 类型冲突，只支持单一类型。		
6068	ZML 文件的 SM 信息丢失		1.更正 xml 文件重新转 zml，内置 xml 联系技术支持增加
6069	ZML 文件的关键段信息丢失		1.更正 xml 文件重新转 zml，内置 xml 联系技术支持增加
6070	ZML 文件需要的内存不够		
6071	ECAT 的模块个数不对		
6072	ZML 文件的段信息重复		
6073	模块 startup 不支持 CA 方式		
6074	别名重复但类型不一致		
6075	扫描到的类型号错误		
RTEX 模块			
6208	RTEX 驱动 ID 冲突		
6209	扫描超时	一般为网线接触问题	
6210	RTEX 初始化失败		
6211	RTEX 扫描结果错误		

6212	RTEX 设备类型错误		
6213	RTEX 消息超时		
6214	RTEX SDO 消息错误		
hlink 模块			
6500-6520	EIO 的错误信息		
6250	返回包的长度不对		
6251	接收参数错误		
6252	接收处理错误		
6253	接收不支持		
6254	接收掉线		
6501	PDO 长度设置错误		
6502	邮箱长度设置错误		
6503	试图修改只读数据字典		
6504	数据字典数组个数超过		
6510	PDO 写入内容错误,第一级索引内容错误		
6511	PDO 数据内容重复		
6512	PDO 内容错误,字典编号错误.		
6513	PDO 内容错误,字典子编号错误.		
6514	PDO 内容错误,字典长度错误.		
6515	PDO 个数太多.		
6516	从站 ALM 报警		
6517	从站 WDOG 报警, PDO 连续丢包		
NC 模块			
7001	文件执行结束		
7003	解析遇到不支持的语法		
7006	运动指令轴字缺失		
7008	ACOS 运算指令参数超出范围		
7009	ASIN 运算指令参数超出范围		
7010	除数不能为 0		
7011	乘方底数为负数时, 指数必须是为整数		
7012	出现不能解析的字符		
7014	错误的数字格式		
7032	内部语法错误		
7034	平面切换错误	传入了一个错误的平面值	
7037	不支持的运算指令		
7041	进给速度为 0, 不能执行 G1		

7043	进给速度为 0, 不能执行 G2、G3		
7057	出现未使用的轴参数		
7066	代码行长度溢出	一行代码最多 256 个字符, 超过溢出	
7069	半径方式圆弧指令的起点与终点重合		
7077	赋值语句确实等号"=		
7084	非法的 G 代码		
7096	ATAN 指令后缺失左括号"["		
7097	运算指令后确实左括号"["		
7098	非法的 N 码 >999999		
7100	非法的 M 码		
7101	圆弧指令参数 R 和 IJK 混用		
7102-7119	出现多个 A~F、H~L、P~Z 字 7102=A、7107=H、7112=P		
7121	负数不能开方		
7124	G 代码出现负值		
7127	M 代码出现负值		
7132	括号注释内容出现括号嵌套		
7133	语法错误, 无读取数值		
7134	数字丢失		
7135	读取数值非整数		
7136	内部语法错误		
7142	非法的参量地址		
7147	圆弧参数缺失		
7153	圆弧半径太小, 无法到达终点		
7156	ATAN 指令斜杠"/"缺失		
7161	内部语法错误		
7168	同 G 模态组指令出现 2 个以上		
7169	同 M 模态组指令出现 2 个以上		
7170	不能打开 NC 文件		
7171~7188	内部语法错误		
7196	LN 指令出现 0 或者负数		
7197	圆弧指令出现半径 R 为 0		
7200	解析代码为空		
7201	无符号整数不能带正负号	无符号整数参数出现了正负号	
7202	未读取到无符号整数		
7203	无符号实数不能带正负号		
7204	未读取到无符号实数		
7220~7246	代码行出现未使用的关键字 7220~7228: XYZABCUVW		

	7229~7231 : FST 7232~7241: EDHIJKLPQR		
7301	指令类型或指令代码不存在		
7302	注册指令已存在		
7303	宏程序号超出允许范围		
7304	指令组号超出范围		
7305	指令代码超出范围		
7306	扩展类型不存在		
7307	指令优先级超出范围		
7308	指令处理回调函数为空		
7309	指令不存在		
7311	优先级位 0 的指令与其他指令同时出现		
7312	同时出现多个运动指令或带坐标参数的指令		
7313	检查 codeline 参数出现错误	内部错误	
7314	清除 codeline 参数出现错误	内部错误	
7315	轴字缺失	内部错误	
7319	指令参数检查错误	检查参数过多	
7320	指令参数检查错误	检查缺少参数	
7350	参数检查不合法		
7351	未定义的 G 代码		
7352	未定义的长度单位		
7353	未定义的类型或模式		
7354	内部错误		
7355	未知的坐标轴	使用了未配置的轴	
7356	缩放指令 p、IJK 参数缺失		
7357	缩放指令 P 参数、IJK 参数混用		
7358	内部错误		
7359	刀补半径大于切削的圆弧半径		
7360	内部错误, 预处理的 GOTO 个数超过允许范围(256 个)		
7361	无效的通道号		
7362	目标通道正在运行其他任务		
7363	目标通道没有运行任务	暂停或停止了空闲通道任务	
7364	开启刀补或坐标旋转功能后, 不能切换工作平面、单位		
7365	在线指令不能执行 Gsub 扩展指令		
7366	G04.1 Q_ 传入错误的 Q 引数	Q 引数未包含自身通道	
		Q 引数传入的通道号越界	
7367	G04.1 Q_ 错误的等待 P 信号	相互等待的通道, 信号 P 不同	

7370	宏程序 WHILE 或 DO 缺失		
7371	WHILE 和 END 不成对出现		
7372	WHILE 超过最大嵌套层数 (5)		
7373	未发现 GOTO 目标地址		
7374	子程序调用超过可嵌套层数	最高可嵌套 8 层	
7375	子程序文件内容为空		
7376	不支持执行超大的子程序文件	限制在 10k 以内	
7379	不支持的宏程序号		
7380	主轴正在进行进给轴运动，不能修改主轴速度和主轴倍率	主轴与进给轴同轴模式下，需要做好运动、速度切换	
7400	参数错误		
7401	参数地址未开放		
7402	无效的参数地址		
7403	无效的参数值	参数值超出有效范围	
7404	无权限读写参数		
7405	参数不支持修改描述		
7407	参数表文件读取错误		
7408	参数表文件写入错误		
7410	G 代码文件扩展类型已满	最多支持 12 个扩展名	
7411	扩展 G 代码文件类型失败	传入无效的后缀名	
7412	打开参数表文件失败		
7413	动态扩展参数失败	超过可扩展的参数数量、参数地址越界	
7414	系统变量未使用	访问了无效的系统变量	
7421~7436	轴向 N 的第一行程软限位触发	超过设置的行程软限位 (轴向就是设置的 X,Y 等轴字)	
9912	下拉列表控件调用函数出错		
PC 端错误			
20000	PC 端产生错误的偏移		
20001	socket 错误		
20002	参数错误		
20003	超时	可能是 fifo 缓冲阻塞	
20004	控制器未响应		
20005	连接过多，超过控制器限制		
20006	操作系统错误		
20007	串口打开失败		
20008	网络打开失败	可能同一台工控机多个网口 ip 和控制器 ip 在同网段	1.检查连接 IP 输入是否正确，需要连接的 IP 的地址能否被扫描到，网络链路是否正常
20009	句柄错误		1.检查网络是否因接线问题或不稳定导致网络被中断

20010	发送错误		1.检查网络是否因接线问题或不稳定导致网络被中断
20011	文件错误:不支持头文件,无法识别		
20012	文件长度错误		
20013	文件名过长		1.检查工程相关文件名是否过长并更正,各控制器规格可能不一致,更正为小于提示信息的字符长度
20014	文件不存在		1.进入工程文件对应的文件夹检查是否缺少文件视图中的某个文件,一般没有在 IDE 操作删除文件下次打开会提示
20015	ZLB 库文件错误		
20016	文件没有编译	一般 PLC 文件没有编译	一般 PLC 文件没有编译
20018	固件文件错误	一般为固件文件损坏导致	
20020	固件文件不匹配		1.检查升级的固件是否与控制器型号匹配
20021	不支持的功能		1.控制器不支持的功能,检查选型配置 2.检查是否控制器新的功能,需要更新固件
PCI 模块			
20022	mmap 失败 RT LOCAL 本地接口打开失败	可能 RT 内存配置太大	
20023	xplcterm 没有正确运行或权限不够		
20024	PCI 卡链接时找不到卡或没有驱动		
20025	驱动枚举失败		
20026	接口枚举失败		
20027	未知的		
20028	PCI 卡不存在		
20029	PCI 卡连接太多		
20030	输入缓冲长度不够	若在线命令有反馈字符串,上位机没有去接收也会返回此错误码	1.检查定义的函数名,参数名等是否过长 2.检查输入指令名称是否固件版本过低不支持该长度
20031	受到密码保护, LOCK 后返回		
20032	受到密码保护, 密码尝试太快		
20033	打开文件错误	例如: 未开启 motionRT 控制台就使用指令打开	

20034	不支持的功能		
20035	消息长度超过		
20036	重新打开		
20037	上报参数过多		
20038	上报参数个数错误		
20039	指定参数不在上报参数中		
20040	MotionRT 连 接 失 败 , MotionRT 未开启		
20041	xpci 异常		
20100	应答缓冲长度不够		
30000	30000 以上是 ZAUX 辅助库 产生的错误码		

附录二 模块扩展

当控制器自身的轴资源、IO 资源不够用时，可采用扩展模块来扩展，可以扩展脉冲轴、数字量输入输出、模拟量输入输出这三种类型。只有带脉冲轴接口的扩展模块才支持扩展脉冲轴数，总线轴不可扩展。

IO 数字量扩展：4 系列及以上的 ZMC 控制器 IO 点数可扩展至 4096 点。

AIO 模拟量扩展：4 系列及以上的 ZMC 控制器 AIO 点数可扩展至 520 点。

ZCAN 总线轴扩展：只能扩展 4 个脉冲轴，不建议使用过多轴扩展板，可选用支持脉冲轴数较多的控制器型号。

控制器可扩展的最大 IO 点数可在硬件手册或“命令与输出”窗口输入?*max 打印查看。



扩展模块按连接方式可分为 ZCAN 总线扩展模块和 EtherCAT 总线扩展模块两类，两类总线的扩展接线与资源映射方法不同。

按产品系列划分，可分为 ZCAN 扩展模块，EtherCAT 扩展模块、ZMIO310 扩展模块三大类，ZMIO310 系列的通讯模块可选 CAN 通讯模块或 EtherCAT 通讯模块。

所有的控制器型号都包含 CAN 总线接口，EtherCAT 接口只有支持 EtherCAT 总线的控制器型号支持。

扩展模块与控制器接线完成后，扩展的 IO 和轴资源还需要操作映射才能使用，CAN 总线扩展与 EtherCAT 总线扩展的映射方法不同，映射时需要映射的编号在整个控制系统中不得重复，当控制器或扩展模块的 IO 编号范围重复时，只有一个有效。

ZCAN 扩展模块

扩展接线

CAN 总线上连接了多个 CAN 扩展模块时，全部 CAN 通讯模块的 CANL 和 CANH 端口分别接到一起，CAN 总线的左右两端分别接入一个 120 电阻。

扩展模块 CAN 端子：

针脚号	名称	说明
1	GND	内部电源地
2	CANL	CAN 差分数据-
3	EARTH/SHIELD	安规地/屏蔽层
4	CANH	CAN 差分数据+
5	+24V	内部电源 24V 输入

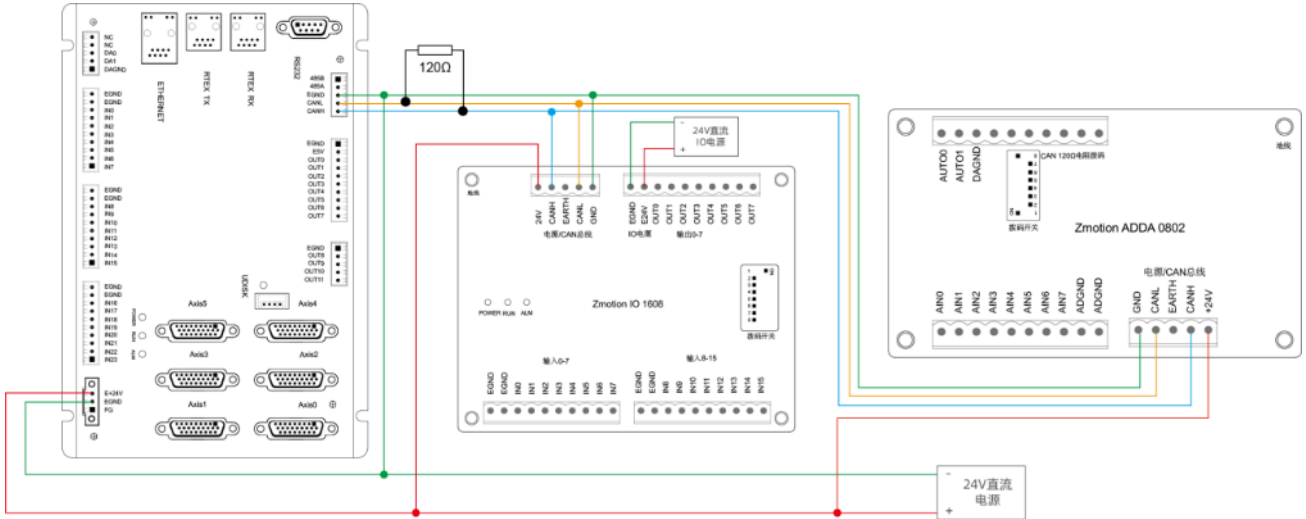
控制器连接 CAN 扩展模块的接线方法如下图所示，在控制器的 CANL 和 CANH 之间接入一个 120 欧姆电阻，并将最后一个 CAN 通讯模块的拨码开关第 8 位拨为 ON(表示 CANL 与 CANH 端口之间接入一个

120 欧姆的电阻), 其他模块的第 8 位拨码开关无需操作, 只需操作最末端的扩展模块。

CAN 通讯必须保证对应 GND 相连, 或是控制器主电源和扩展模块主电源用同一个电源, 防止扩展模块烧坏。

ZCAN 扩展接线参考如下: ZMC432+ZIO1608M+ZAIO0802M, CAN 扩展时建议使用双绞屏蔽线, 屏蔽层接地。

ZIO 扩展模块为双电源供电, 需要主电源和 IO 电源, 不接 IO 电源无法使用; ZAIO 扩展模块为单电源供电只需要主电源。



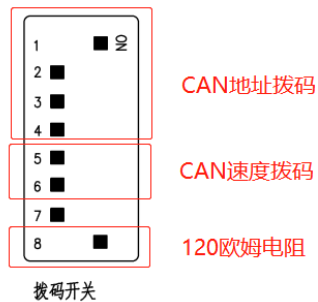
资源映射

ZCAN 扩展模块扩展的资源需要映射后才能使用, IO 映射采用扩展模块上自带的拨码开关设置, 轴映射采用 `AXIS_ADDRESS` 指令映射轴号。

数字量 IO 和模拟量的映射编号规则略有不同, 参见下文说明。

IO 映射

ZCAN 扩展板一般带 8 位拨码开关, 拨 ON 生效, 如下图所示, 拨码含义如下:



1-4: 4 位 CAN ID 用于 ZCAN 扩展模块 IO 地址映射, 对应值 0-15;

5-6: CAN 通讯速度, 对应值 0-3, 可选四种不同的速度;

7: 预留;

8: 120 欧姆电阻, 拨 ON 表示 CANL 和 CANH 之间接入了 120 欧电阻。

拨码 1-4 选择 CAN 地址, 控制器根据 CAN 拨码地址来设定对应扩展模块的 IO 编号范围, 拨码每位

OFF 时对应值 0，ON 时对应值 1，地址组合值=拨码 4×8+拨码 3×4+拨码 2×2+拨码 1。

拨码开关必须在上电之前拨好，上电后重新拨码无效，需再次上电才生效。

不同地址对应数字量 IO 编号分配情况如下表，数字量起始 IO 映射编号从 16 开始，按 16 的倍数递增。

拨码 1-4 组合值	起始 IO 编号	结束 IO 编号
0	16	31
1	32	47
2	48	63
3	64	79
4	80	95
5	96	111
6	112	127
7	128	143
8	144	159
9	160	175
10	176	191
11	192	207
12	208	223
13	224	239
14	240	255
15	256	271

不同地址对应模拟量编号分配情况如下表，模拟量 AD 起始 IO 映射编号从 8 开始，按 8 的倍数递增。

模拟量 DA 起始 IO 映射编号从 4 开始，按 4 的倍数递增。

拨码 1-4 组合值	起始 AD 编号	结束 AD 编号	起始 DA 编号	结束 DA 编号
0	8	15	4	7
1	16	23	8	11
2	24	31	12	15
3	32	39	16	19
4	40	47	20	23
5	48	55	24	27
6	56	63	28	31
7	64	71	32	35
8	72	79	36	39
9	80	87	40	43
10	88	95	44	47
11	96	103	48	51
12	104	111	52	55
13	112	119	56	59
14	120	127	60	63
15	128	135	64	67

拨码 5-6 选择 CAN 总线通讯速度，速度组合值=拨码 6×2+拨码 5×1，组合值范围 0-3，对应的速度如

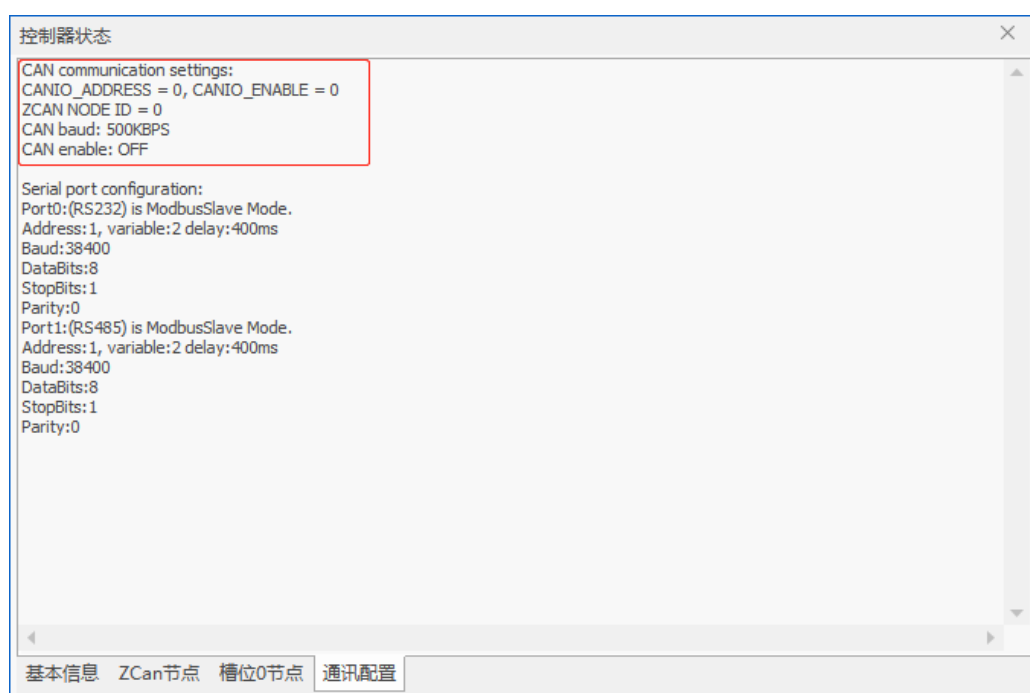
下表：

拨码 5-6 组合值	CANIO_ADDRESS 高 8 位值	CAN 通讯速度
0	0（对应十进制 128）	500kbps（缺省值）
1	1（对应十进制 256）	250kbps
2	2（对应十进制 512）	125kbps
3	3（对应十进制 768）	1Mbps

控制器端通过 CANIO_ADDRESS 指令设置 CAN 通讯速度，同样也是有四种速度参数可供选择，需要与组合值对应的扩展模块的通讯速度一致才可以互相通讯。

CANIO_ADDRESS 指令还可以设置 CAN 通讯的主从端，缺省值 32，做主端，设置为其他值便是做从端。

CAN 通讯配置情况可在“控制器状态”窗口查看通讯配置。



拨码开关设置注意事项：

扩展模块拨码开关根据当前已包含 IO 点数的 IN 和 OP 最大者（外部 IO 接口数+脉冲轴内的 IO 接口数）。

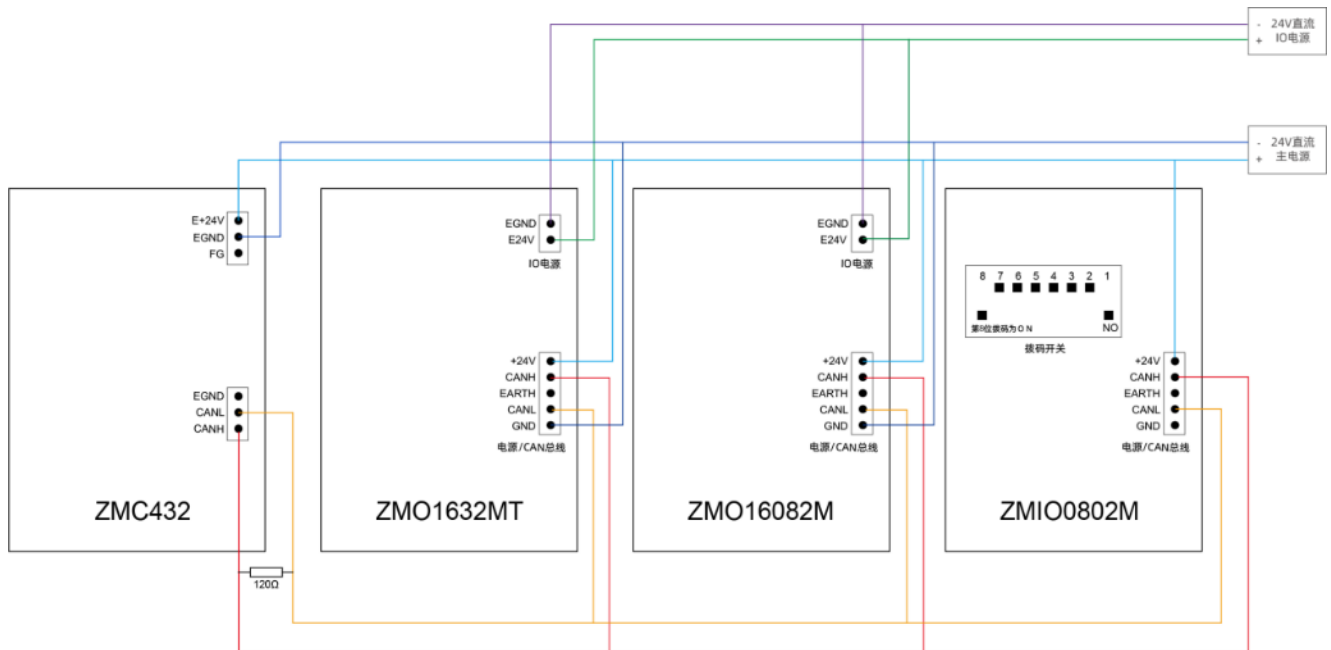
如控制器本身包含 28 个 IN，16 个 OP，那么第一个扩展模块设置的起始地址应超过最大值 28，按 IO 映射规则应将地址拨码设置为组合值 1（二进制组合值 0001，从右往左对应拨码 1-4，此时拨码 1 置 ON，其他置 OFF），此时扩展模块上的 IO 编号为 32-47，其中，29-31 空缺出来的 IO 编号舍去不用。

后续的扩展模块则依次按 IO 点数继续确认拨码设置。

当控制器或扩展模块的 IO 编号范围重复时，只有一个有效。建议重新设置拨码使整个控制系统的 IO 编号均不重复。

ZCAN 扩展模块 IO 映射配置示例：

控制模块配置：1 个 ZMC432+1 个 ZIO1632MT+1 个 ZIO16082M+1 个 ZAI00802M



CAN 接线方法参见上图，正确设置每个模块的拨码 ID，并将最后一个扩展模块的第八位拨码拨为 ON（表示 CANL 和 CANH 之间接入 120 欧姆电阻），使用 RTSys 软件连接上控制器，打开“控制器”-“控制器状态”窗口，查看 ZCAN 节点信息，可以看到 CAN 总线连接的全部设备的信息。

ZIO1632 的 CAN ID 设置为 1，扩展的数字量输入 IO 编号为 32-47 共 16 个，扩展的数字量输出 IO 编号为 32-63 共 32 个。

ZIO16082 的 CANID 设置为 3, 扩展的数字量输入 IO 编号为 64-79 共 16 个, 扩展的数字量输出 IO 编号为 64-71 共 8 个, 除此之外还带两个脉冲轴。

ZAIO0802 的 CAN ID 设置为 4，扩展的模拟量输入 AD 编号为 40-47 共 8 个，扩展的模拟量输出 DA 编号为 20-21 共 2 个。

[illegible]

轴映射

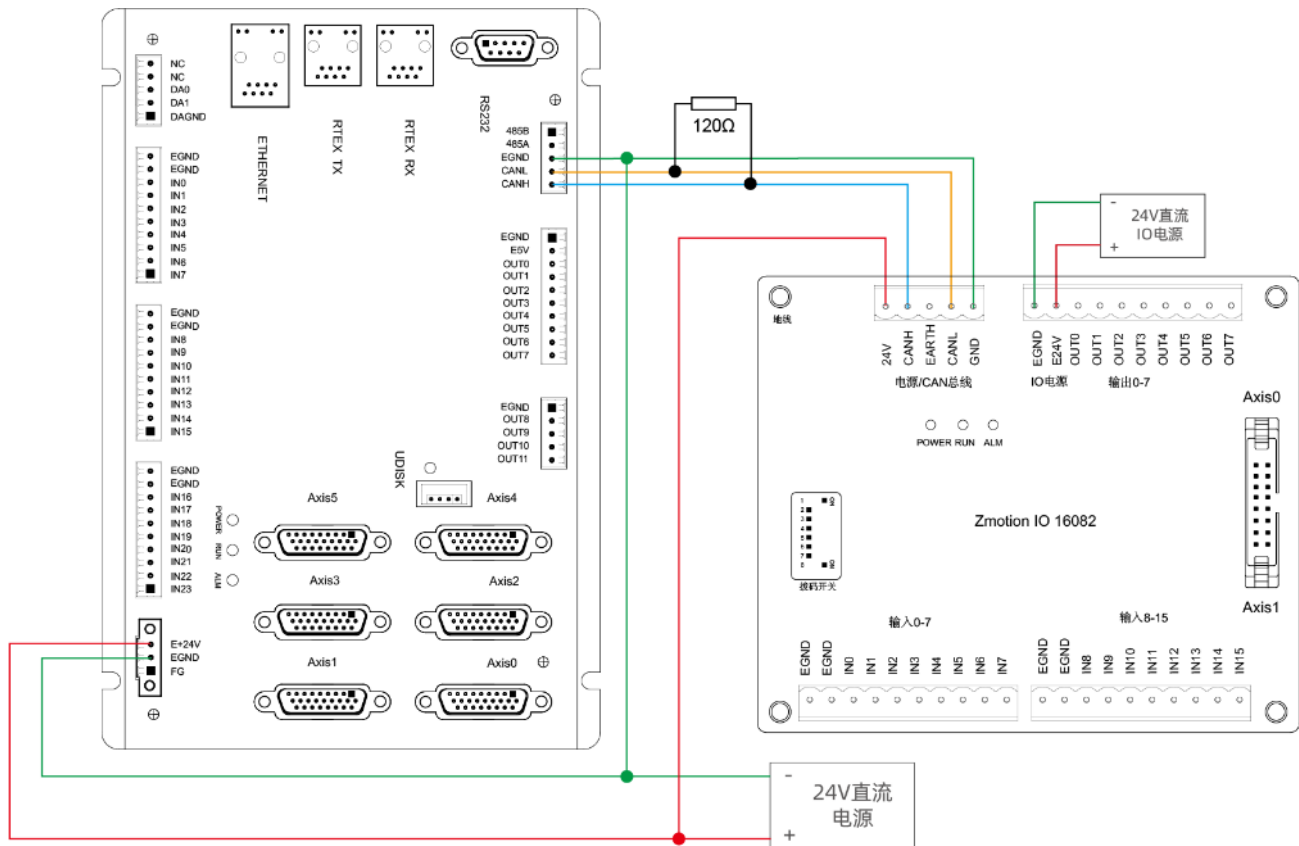
CAN 总线扩展方式扩展脉冲轴时，可选 ZIO16082M，扩展两个脉冲轴。

扩展轴需要进行轴映射操作，采用 AXIS_ADDRESS 指令映射，映射规则如下：

AXIS_ADDRESS(轴号)=(32*0)+ID '扩展模块的本地轴接口 0

AXIS_ADDRESS(轴号)=(32*1)+ID '扩展模块的本地轴接口 1

ID 为扩展模块 1-4 位地址拨码的组合值。



映射完成设置 ATYPE 等轴参数后就可以使用扩展轴，示例：

ATYPE(6)=0	'设为虚拟轴
AXIS_ADDRESS (6)=1+(32*0)	'ZCAN 扩展模块 ID 为 1 的轴号 0 映射到轴 6
ATYPE(6)=8	'ZCAN 扩展轴类型，脉冲方向方式步进或伺服
UNITS(6)=100	'脉冲当量 100
SPEED(6)=100	'速度 100units/s
ACCEL(6)=1000	'加速度 1000units/s^2
MOVE(100) AXIS(6)	'扩展轴运动 100units

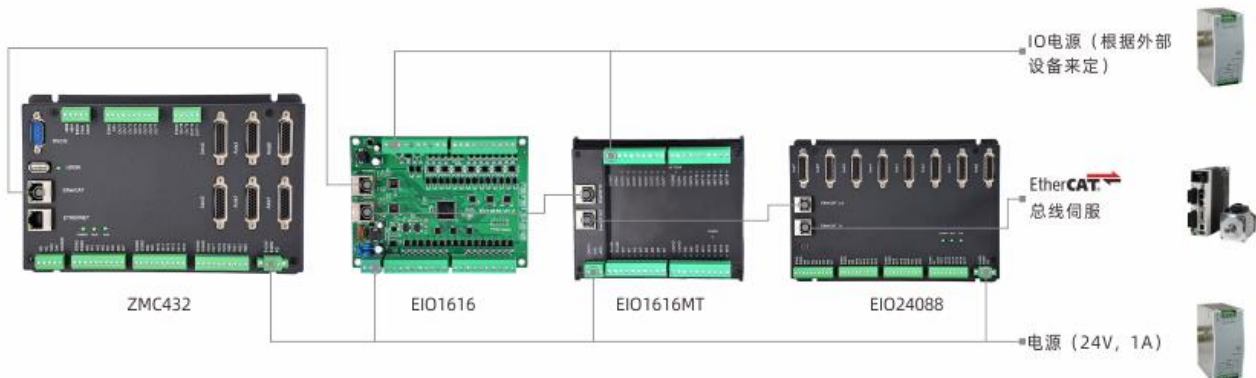
EtherCAT 扩展模块

扩展接线

EtherCAT 扩展模块接线只需将各个模块的 EtherCAT 口相互连接即可，EIO 系列扩展板带两个 EtherCAT

接口，EtherCAT 0 口接主控制器，EtherCAT 1 口接下级扩展板或驱动设备，不可混用。

EIO 扩展接线参考如下：ZMC432+EIO1616+EIO1616MT+EIO24088。



资源映射

EtherCAT 总线上的 IO 映射采用 NODE_IO 指令(数字量)、NODE_AIO 指令(模拟量)设置, 轴映射采用 AXIS ADDRESS 指令映射轴号。

槽位号和设备号按照与控制器的连接顺序,从0开始自行编号。

IO 映射

NODE_IO 指令设置设备的数字量 IO 起始编号，单个设备的输入输出的起始编号一样。必须总线扫描后才能设置，NODE_AIO 指令使用与 NODE_IO 指令基本相同。

语法:

NODE IO(slot, node)=iobase

slot: 槽位号, 0-缺省

node: 设备编号, 编号从 0 开始

ioBASE: 映射 IO 起始编号，设置结果只会是 8 的倍数

NODE AIO(slot, node[,idir])=aiobase

slot: 槽位号, 0-缺省

node: 设备编号, 编号从 0 开始

idir: AD/DA 选择。0-缺省, 同时设置 AIN、AOUT, 读取时只读 AIN; 3-AIN; 4-AOUT

IO 映射示例：ZMC432 控制器上依次连接两个 EtherCAT 扩展模块，配置：1 个 ZMC432+1 个 EIO1616MT+1 个 ZMIO-4AD。

SLOT_SCAN(0)	'扫描总线
IF NODE_COUNT(0)>0 THEN	'判断槽位 0 上是否有设备
NODE_IO(0,0)=32	'设置槽位 0 接口设备 0 的 IO 起始编号为 32
NODE_AIO(0,1,3)=8	'设置槽位 0 接口设备 1 的 AIN 起始编号为 8
ENDIF	

轴映射

总线轴需要进行轴映射操作，采用 `AXIS_ADDRESS` 指令映射，操作方法如下：

`AXIS_ADDRESS(轴号)=(槽位号<<16)+驱动器编号+1`

轴映射写在总线初始化程序中，扫描总线之后，开启总线之前。

示例：

`AXIS_ADDRESS (0)=(0<<16)+0+1` '第一个 ECAT 驱动器，驱动器编号 0，绑定为轴 0

`AXIS_ADDRESS (2)=(0<<16)+1+1` '第二个 ECAT 驱动器，驱动器编号 1，绑定为轴 2

`AXIS_ADDRESS (1)=(0<<16)+2+1` '第三个 ECAT 驱动器，驱动器编号 2，绑定为轴 1

`ATYPE(0)=65` '设置为 ECAT 轴类型， 65-位置 66-速度 67-转矩

`ATYPE(1)=65`

`ATYPE(2)=65`

附录三 触摸屏通讯

控制器与触摸屏通讯简介

控制器与触摸屏一般通过串口或网口连接，控制器的串口和网口采用 MODBUS 协议，只要支持 MODBUS 通讯协议的触摸屏都可以与正运动控制器连接使用，也可使用正运动自主研发的 ZHD 系列触摸屏，ZHD400X 触摸屏如下图。



控制器使用 MODBUS 协议与第三方触摸屏通讯时，此时需要将数据放在 MODBUS 寄存器内进行传递，控制器搭配 ZHD 系列触摸屏编程更为灵活、自由。

控制器的 MODBUS 地址与其他厂家的触摸屏地址映射关系有所不同，控制器与触摸屏 modbus 寄存器地址关系如下：

控制器的 MODBUS 地址从 0 开始，在与威纶触摸屏通讯时，地址都是从 0 开始，所以是一一对应。

控制器 MODBUS_BIT(0)对应威纶触摸屏 MODBUS_0X_0，布尔型。

控制器 MODBUS_REG(0)对应威纶触摸屏 MODBUS_4X_0，字寄存器。

在与昆仑通态触摸屏通讯时，昆仑通态地址从 1 开始，控制器地址从 0 开始，所以触摸屏地址加 1。

控制器 MODBUS_BIT(0)对应昆仑通态触摸屏 MODBUS_0X_1，布尔型。

控制器 MODBUS_REG(0)对应昆仑通态触摸屏 MODBUS_4X_1，字寄存器。

控制器端程序可使用 RTSys 软件支持的 Basic 语言或 PLC 梯形图编程，ZHD 系列触摸屏的程序则用 RTSys 软件的 HMI 语言编程。

MODBUS 寄存器的说明参见前面章节“寄存器”→“[MODBUS](#)”。

控制器与触摸屏连接

控制器连接触摸屏使用的一般流程：

1. 控制器端的程序使用 RTSys 软件编写完成下载到控制器内。
2. 触摸屏端的程序使用对应的编程软件编写完成后下载到触摸屏保存。
3. 程序下载完成之后，选择串口或网口连接触摸屏与控制器脱机运行

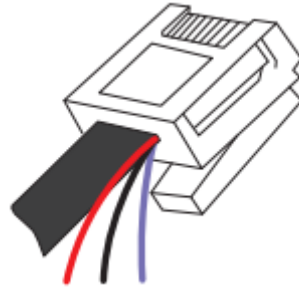
连接 ZHD 系列触摸屏

正运动控制器连接正运动 ZHD300X 和 ZHD400X 系列触摸屏使用十分便捷，触摸屏程序可下载到控制

器，下面是 ZHD400X 的使用方法，ZHD300X 和 ZHD400X 不同之处在于，前者 RS232 串口通讯，后者网口通讯，其他配置方法均相同。

ZHD400X 触摸屏配一根网线，如下图，将此网线连接到控制器的 EtherNET 网口，网线水晶头边上引出三根线，分别是示教盒电源线和急停信号线，红色为 24V 电源正极，黑色为 24V 电源负极，紫色为急停信号线。

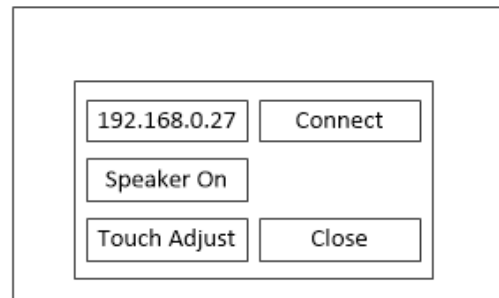
触摸屏和控制器的电源可共用一个。



触摸屏连接控制器使用步骤：

1. 先使用 RTSys 软件编辑好 HMI 程序，连接控制器，将程序下载到 ROM 掉电保存，就可以断开控制器和 RTSys 的连接。然后给触摸屏上电。

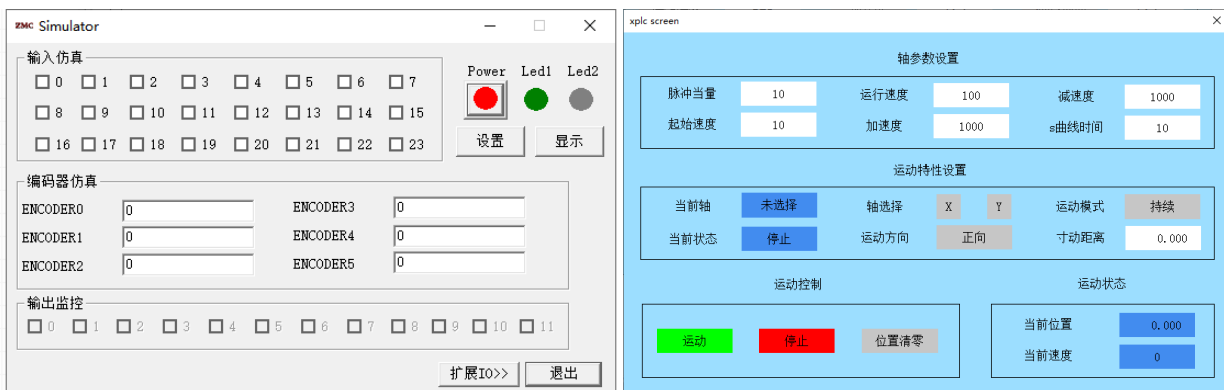
2. 直接使用配发的连接线将 ZHD400X 接到控制器的网口上，然后在屏上四个角，按画 Z 字顺序点击，连续 2 次，唤醒屏幕，弹出可以弹出设置窗口，可以进行触摸校正，控制器 IP 修改等。



3. 设置窗口如下，在弹出的窗口上自动获取到当前所连的控制器 IP 的地址，确认 IP 无误，点击 Connect 即可连接使用，此时触摸屏显示起始基本窗口的内容。

4. 若没有实物触摸屏，可将 HMI 程序下载到仿真器，在 XPLC screen 平台仿真。

连接仿真器下载程序之后，点击“显示”按钮即可弹出仿真界面。



连接第三方触摸屏

支持标准 MODBUS 协议的触摸屏都可以与正运动控制器通讯，将通讯数据放在 MODBUS 寄存器中传递，支持通过串口或网口连接到控制器。

触摸屏和控制器建立通讯连接的时候，主要在触摸屏端操作连接，连接时对应的串口或网口参数要匹配。

通讯时可用寄存器类型有如下几种：MODBUS_BIT（布尔型），MODBUS_REG(16 位整型，MODBUS_LONG(32 位整型)，MODBUS_IEEE(32 位浮点型)，MODBUS_STRING(8 位字节型)。

控制器和第三方触摸屏的通讯实例：以控制器和威纶屏通讯为例展开触摸屏的使用说明。

1. 下载控制器程序

控制器端的程序使用 RTSys 软件编写完成并下载到控制器内。详细步骤参见“新建项目运行程序”章节。

2. 下载触摸屏程序

威纶触摸屏端的程序使用 EasyBuilder 编程软件编写，程序编程完成后，打开“系统参数设置”窗口，如下图。



1) 添加要与触摸屏连接的设备

设备列表里会显示本机触摸屏和本机设备，若有本机设备双击该行，若没有本机设备，点击下图“新建设备/服务器...”，弹出设备属性窗口。



2) 设置设备属性

如上图所示，选择设备类型，先选 MODBUS IDA 通讯协议，再根据触摸屏与控制器的实际连接方式选择。

串口通讯和网口通讯所选的设备类型不同，详见后续说明。

若采用串口连接：

设备类型：选择模式 MODBUS RTU(Zero-BASEd Addressing)；

接口类型：选择串口类型（RS485 或 RS232）；

COM：通讯端口设置匹配的波特率等参数，如下图，此时参数必须与连接到控制器的端口参数一致，设置完成确认关闭系统参数设置窗口。

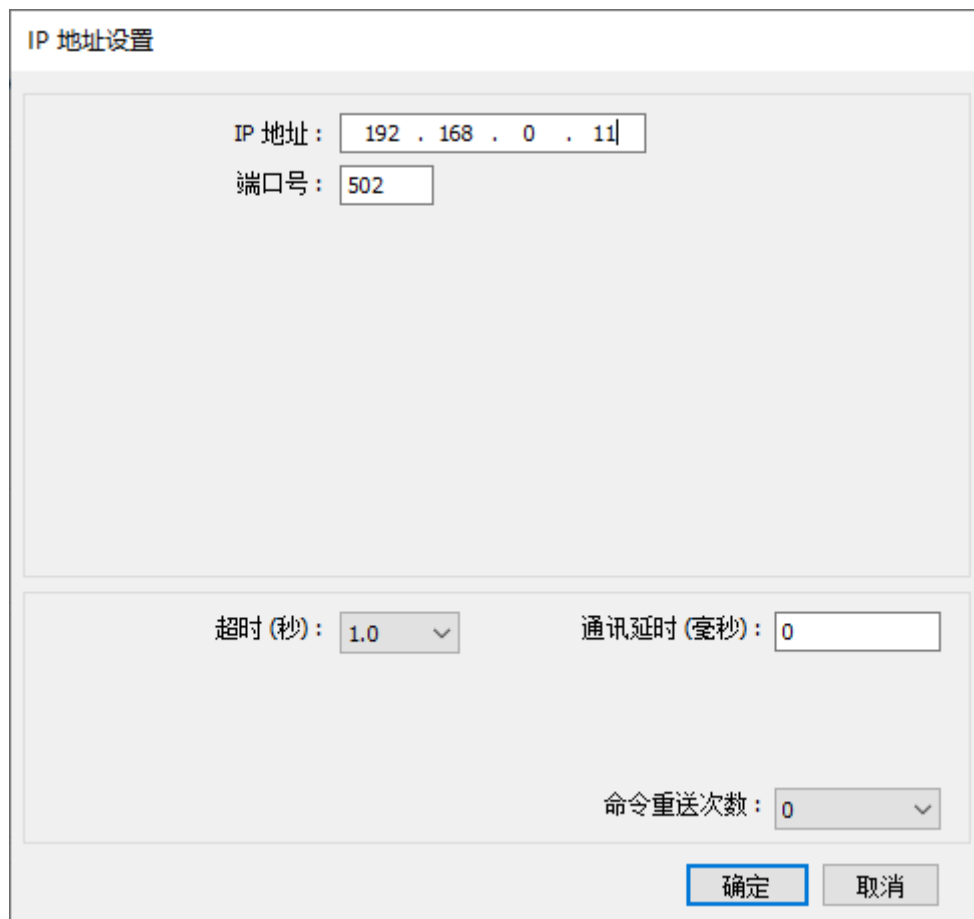


若采用网口连接：

设备类型：选择模式 MODBUS TCP/IP(Zero-BASEd Addressing)，接口类型自动改为以太网；

IP：填入当前要连接的控制器的 IP 地址和端口号，如下图；

设置完成确认关闭系统参数设置窗口。



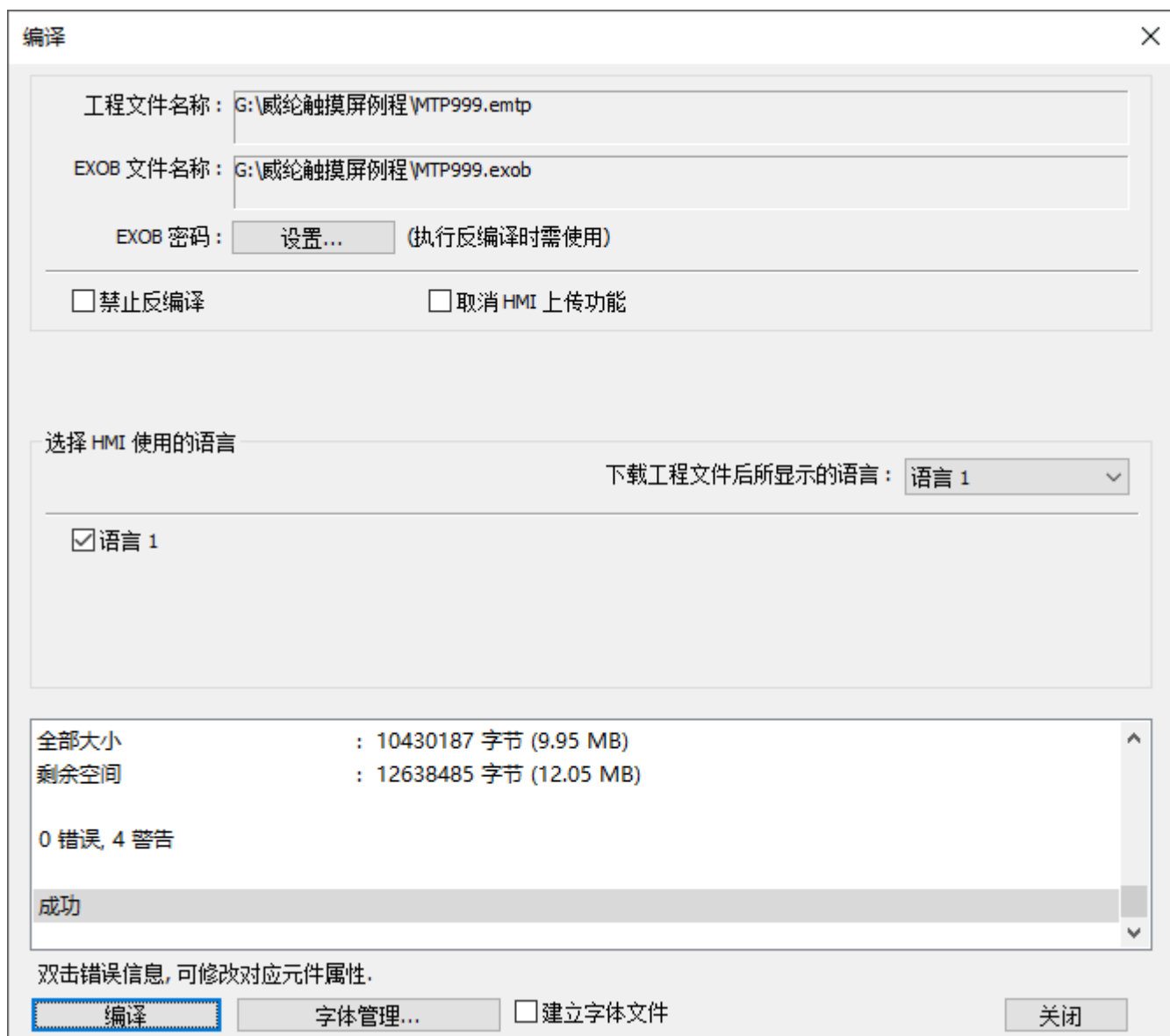
IP 地址设置对话框，包含以下配置项：

- IP 地址：192 . 168 . 0 . 11
- 端口号：502
- 超时 (秒)：1.0
- 通讯延时 (毫秒)：0
- 命令重送次数：0
- 底部按钮：确定、取消

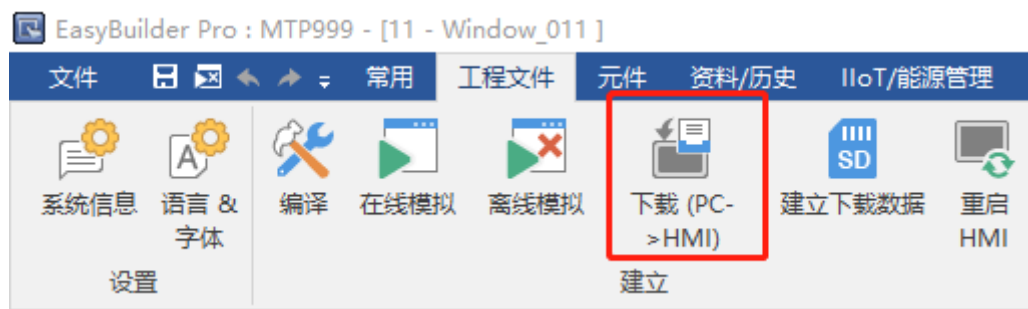
系统参数设置完成后，编译写好的组态程序，点击“编译”，打开如下编译窗口。



点击右下角“开始编译”按钮，编译成功打印信息提示，“开始编译”按钮变成“编译”按钮，程序不正确是编译窗口能打印出错误信息，修改程序知道编译成功。



程序编译成功，将触摸屏连接到 PC 下载程序。



点击下载按钮，弹出下方界面，通过以太网将程序下载到触摸屏，下载完成后，程序已写入触摸屏，可以断开触摸屏与 PC 的连接。

3. 触摸屏和控制器通讯

在控制器端的程序成功下载到控制器后，和触摸屏端的程序成功下载到触摸屏之后，可与 PC 断开连接，连接触摸屏与控制器，此时触摸屏与控制器就可以相互通信了。

4. 控制器与触摸屏脱机仿真

若没有控制器或触摸屏，可采用仿真器仿真，RTSys 程序下载到仿真器内，只支持网口连接仿真，按照上面的步骤，EasyBuilder 软件的系统参数设置时选择设备类型为 MODBUS IDA—MODBUS TCP/IP(Zero-BASEd Addressing)，IP 地址填入仿真器 IP：127.0.0.1，选择“在线模拟”即可连接控制器程序与组态程序进行仿真。



点击在线模拟之后自动开始编译，编译结果正确打开如下触摸屏仿真界面，此时可以操作。编译不成功会报错错误信息。

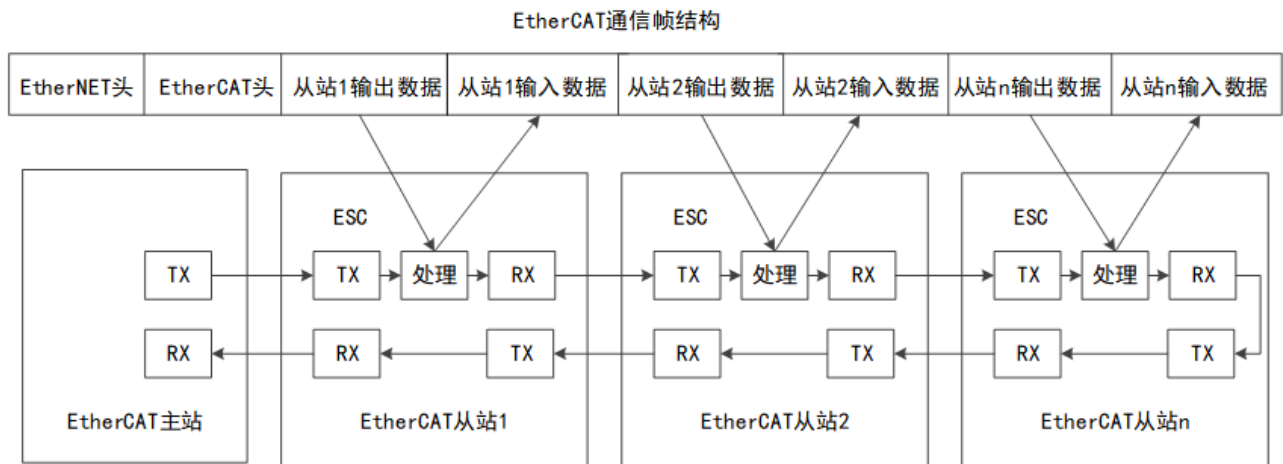
触摸屏仿真界面：



附录四 ETHERCAT 通讯

EtherCAT 总线是基于以太网开发架构的实时工业现场总线通讯协议，目前是最快的工业以太网技术之一，提供了纳秒级精确同步，具有高性能、拓扑结构灵活，低成本、高精度、应用简单等优点。

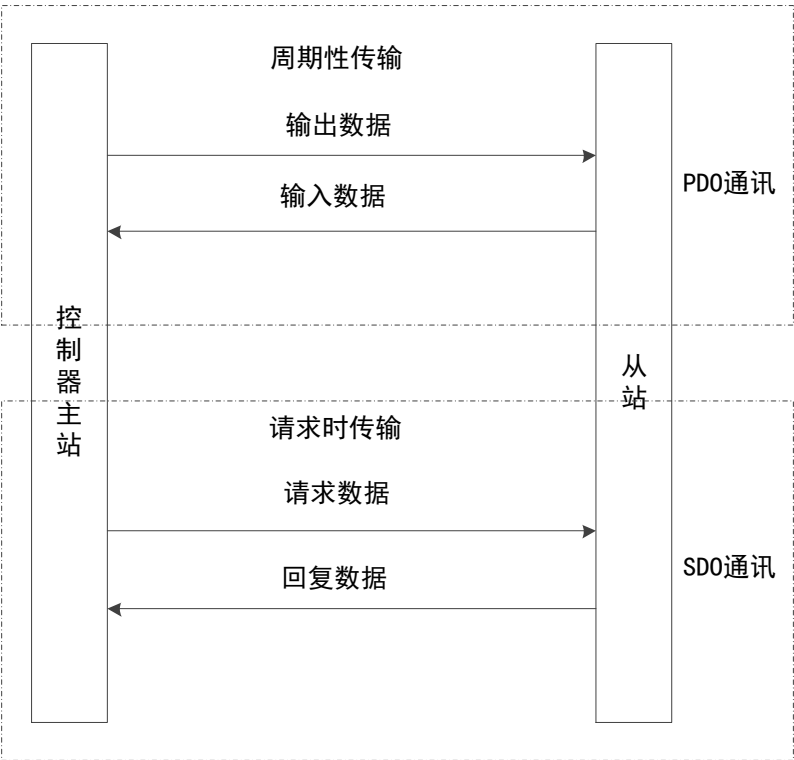
EtherCAT 充分利用了以太网的全双工特性，使用主从模式介质访问控制。EtherCAT 网络和普通以太网有明显不同，同一个 EtherCAT 网络内，只有一个 EtherCAT 主站，另外 EtherCAT 从站有专门处理 EtherCAT 通讯数据的芯片 ESC(EtherCAT Slave Controller)。ESC 芯片可以在 EtherCAT 数据帧通过时，取出主站发送给该从站的数据，并将该从站需要传送给主站的数据插入到 EtherCAT 数据帧中，网络中的最后一个 EtherCAT 从站 ESC 自动闭环，将处理过的报文依次返回给主站，数据传输示意图如下图所示。



控制器 EtherCAT 通讯口和 EtherCAT 从站之间通过 COE (CANopen over EtherCAT) 协议进行数据交换。

控制器和从站之间数据传输方式有两种，一种是按指定时间周期性交换数据，称之为 PDO(Process Data Object)，另外一种为请求应答式交换数据，称之为 SDO(Service Data Object)。

EtherCAT 总线通信过程如下：



过程数据对象(PDO)

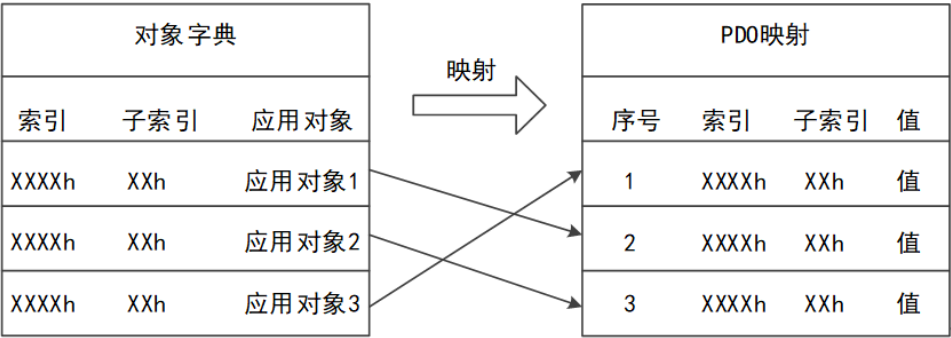
PDO 全名为 Process Data Object，指在 EtherCAT 总线网络中周期的进行主站与从站的数据交互的功能，PDO 数据用于周期性数据读取和控制，读写速度快。主站和从站通过 PDO 进行数据交换时，一方发送数据后，另一方不需要应答。控制器通过运动指令控制 EtherCAT 从站时，控制器和从站之间通过 PDO 方式进行数据交换。

EtherCAT 初始化过程中必须进行驱动器 PDO 配置，DRIVE_PROFILE 指令配置驱动器的 PDO 列表，目前提供约 20 几种配置选择，每种配置包含哪些数据字典查看该指令说明确认。DRIVE_PROFILE 设置不能满足的就自定义 PDO，采用 SDO 相关指令操作数据字典配置需要的 PDO。

PDO 列表可以看作一个数组空间，每个数组元素存放了不同的功能码，PDO 在一个周期中执行这些功能码对应的操作，这些功能码就叫做数据字典，数据字典用 4 位 16 进制数来表示，规划方式是透过对象字典中对应 PDO mapping 及 PDO 参数索引。

PDO 分为两种：传送用的 TxPDO 及接收用的 RxPDO。一个节点的 TxPDO 是将数据由此节点传输到其他节点，而 RxPDO 则是接收由其他节点传输的数据。一个节点分别有 4 个 TxPDO 及 4 个 RxPDO。PDO 报文数据域中每个字节都用作数据传输，因此报文利用率高。

PDO 中的所有传送数据必须由对象字典中映射进来：
配置完后 PDO 的传输顺序为：应用对象 3，应用对象 1，应用对象 2。



服务数据对象(SDO)

SDO 数据用于主站需要读或者写从站参数时才发送通讯数据。此种方式只能主站读或写从站的数据，主站发送数据后从站需要应答。

SDO 可用来存取远端节点的对象字典，读取或设定其中的数据。数据字典的读写可 PDO 列表的自定义配置通过指令 SDO_READ、SDO_READ_AXIS 和 SDO_WRITE、SDO_WRITE_AXIS 实现。

SDO 报文中包含索引和子索引信息，如此方便对象在对象字典中定位，而且对象字典中的复合数据结构易于通过 SDO 访问。SDO 的触发方式为命令响应型，即 SDO 客户发出读/写请求后，SDO 服务器须给予回应；客户端和服务端均可以主动终止 SDO 的传输；请求报文和响应报文通过不同的 COB-ID 进行区分。

SDO 可以传送任意长度的数据。如果传送的数据超过 4 个字节，则必须实行分段传送。最后一段数据包包含一个结束标志。

数据字典

EtherCAT 通讯操作对象字典，它是一个有序的对象组，每个对象采用一个 4 位 16 进制的索引值来寻址，为了允许访问数据结构中的单个元素，同时定义了一个 8 位的子索引，多个数据对象组合成一个数据字典，又称 PDO 列表。

每个节点都有一个对象字典，对象字典包含了描述这个设备和它的网络行为的所有参数。对象字典的结构参照下表，节点的对象字典的有关范围在 0x1000-0x9FFF 之间。

索引	内容
0x0001-0x0FFF	协议类型描述，数据类型，行规类型描述，配置表信息
0x1000-0x1FFF	通信区域
0x2000-0x5FFF	设备商自定义对象的功能属性，用于设置功能码，静态参数
0x6000-0x9FFF	行规定义的数据对象，用于设备控制与监视
0xA000-0xFFFF	保留

索引 1600h~17FFh 用以 RxPDO 映射的设定，设定完成分配到 1C12h，索引 1A00h~1BFFh 用于 TxPDO 映射的设定，设定完成分配到 1C13h。

常用数据字典参考：

索引	子索引	名称	数据范围	数据类型	读写	PDO	控制模式	EEPROM
6040h	00h	控制字	0-65535	U16	RW	RxPDO	全部	NO
6041h	00h	状态字	0-65535	U16	RO	RxPDO	全部	NO
6060h	00h	控制模式设置	-128~127	I8	RW	RxPDO	全部	YES
6061h	00h	控制模式查看	-128~127	I8	RO	TxPDO	全部	NO
6071h	00h	目标力矩	-32768~32767	I16	RW	RxPDO	tq, cst	YES
6072h	00h	最大力矩	0-65535	U16	RW	RxPDO	全部	YES
6077h	00h	实际力矩	-32768~32767	I16	RO	TxPDO	全部	NO
607Ah	00h	目标位置	-2147483648~ 2147483647	I32	RW	RxPDO	pp, csp	NO
607Eh	00h	电机极性	0-255	U8	RW	NO	全部	YES
6091h	01h	电子齿轮比分子	1~4294967295	U32	RW	NO	全部	YES
	02h	电子齿轮比分母	1~4294967295	U32	RW	NO	全部	YES
6098h	00h	回零模式	-128~127	I8	RW	RxPDO	hm	YES
60FDh	00h	数字输入	0~4294967295	U32	RO	TxPDO	全部	NO
60FDh	01h	数字输出	0~4294967295	U32	RW	RxPDO	全部	YES
60FFh	00h	目标速度	-2147483648~ 2147483647	I32	RW	RxPDO	pv, csv	NO

不同位的值表示的功能参见驱动器手册的描述去设置，部分参数设置完成需要写入到驱动器的掉电存储器中，并重启驱动器生效。

附录五 RTEX 总线

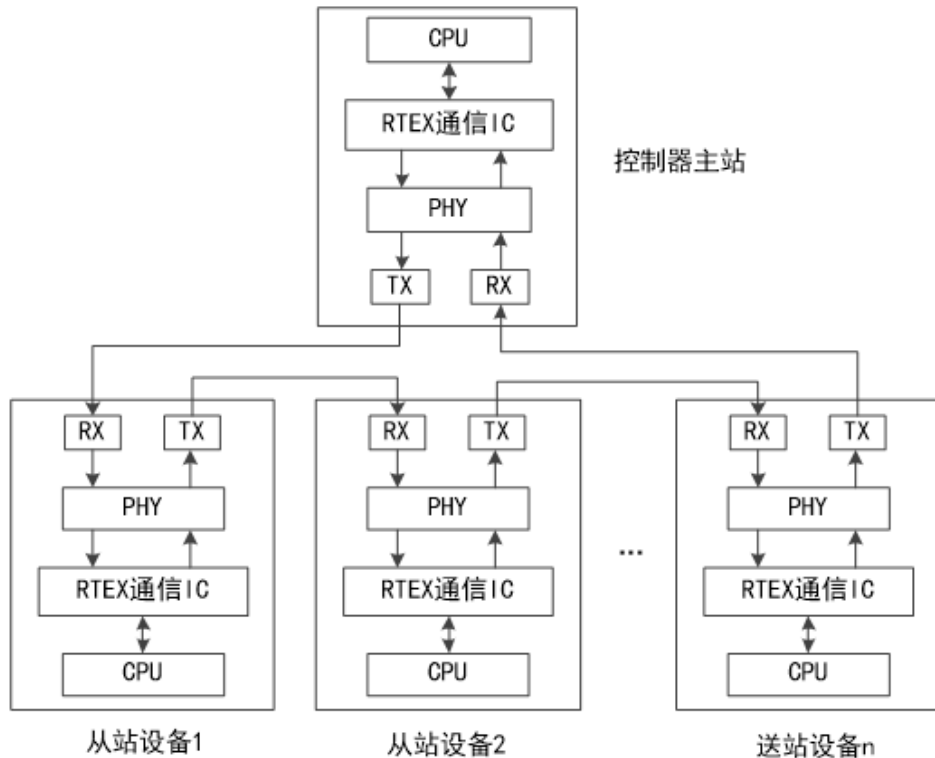
正运动控制器部分型号支持 RTEX 总线，支持 RTEX 总线轴、EtherCAT 总线轴、脉冲轴联合插补，RTEX 总线是松下自主开发出高速网络化的 RTEX 总线，适用于小系统的实时总线，小型设备组成的流水线更具有快速应变的优势。

目前，RTEX 总线支持 32 个节点，每个完整的数据包都包括了 32 个节点的输出信息与反馈信息，它共分为 64 个数据块。此外，RTEX 总线提供控制字寄存器与状态字寄存器。其中每个数据块大小为 16bytes，只包含有必要的位置、速度信息以及一些命令字和状态字。基于 RTEX 总线的主站为核心，主机侧都会一次性对所有节点发出控制命令，同时获取到所有节点的反馈信号，并完成对所有节点输出的控制。

搭载 RTEX 通信 IC 的主站上位装置和从站进行环形连接，构成多轴伺服通讯系统构成如下所示，其中 PHY 为物理层芯片。连接线应使用带屏蔽层的双绞线电缆。

通信和伺服的同步位确立状态下，指令收信、相应送信的时序是不定的，同步是否完成可以通过指令读取当前状态来判断。

RTEX 通信 IC 包括送信存储器、收信寄存器、控制寄存器和状态寄存器，送信存储器用于存储数据指令，收信寄存器用于存储响应数据。



RTEX 参数读写使用 DRIVE_READ 指令和 DRIVE_WRITE 指令操作下方驱动器参数进行相关设定。
相关设定参数：

分类	No.	属性	参数名称	设定范围	单位	描述
0	00	C	旋转方向设定	0~1	—	设定指令方向与电机旋转方向的关系 0-CW 为正方向, 1-CCW 为正方向
0	01	R	控制模式设定	0~6	—	设定伺服驱动器的控制模式 0: 半闭环控制 位置/速度/转矩控制模式可切换 6: 全闭环控制 只有位置控制 (轮廓/周期)
0	08	C	电机每旋转一圈 指令脉冲数	0~2 ²³	pulse	设定电机每旋转一圈指令脉冲数
0	09	C	电子齿轮分子	0~2 ³⁰	—	设定电子齿轮比的分子
0	10	C	电子齿轮分母	0~2 ³⁰	—	设定电子齿轮比分母
0	13	B	第一转矩限制	0~500	%	设定电机输出转矩的第 1 限制值, 参数值受 适用电机最大转矩限制
3	12	B	加速时间设定	0~10000	ms	设定加速的时间
3	12	B	减速时间设定	0~10000	ms	设定减速的时间
3	14	B	S 加减速设定	0~1000	—	对加减速进行 S 曲线处理
3	17	B	速度限制选择	0~1	—	选择速度限制; 0-速度限制 1, 1-速度限制 2
3	21	B	速度限制值 1	0~20000	r/min	设定速度限制值, 内部值被 Pr5.13「过速度 等级设定」、Pr6.15「第 2 过速度等级设定」、 以过速度保护水平的内部值的最小设定速度 进行限制
3	22	B	速度限制值 2	0~20000	r/min	设定 Pr3.17「速度限制选择」=1 设定时、 SL_SW 为 1 时的速度限制值。 内部值被 Pr5.13「过速度等级设定」、Pr6.15 「第 2 过速度等级设定」、以过速度保护水 平的内部值的最小设定速度进行限制
5	21	B	转矩限制选择	1~4	—	设定正方向/负方向的转矩限制选择方式 在设定为 0 时在内部设定为 1
5	22	B	第二转矩限制	0~500	%	设定电机的输出转矩的第 2 限制值, 电机的 最大转矩限制
5	25	B	正方向转矩限制	0~500	%	Pr5.21 (转矩限制选择)=4 设定时, TL_SW 为 1 时, 设定正方向转矩限制, 参数值被适 用电机的最大转矩限制
5	26	B	负方向转矩限制	0~500	%	Pr5.21 (转矩限制选择)=4 设定时, TL_SW 为 1 时, 设定正方向转矩限制, 参数值被适 用电机的最大转矩限制

7	10	A	软件限制功能	0~3	—	设定轮廓位置控制(PP)时的软件限位功能的有效/无效 有效时的软件限位值, 通过 Pr7.11(正侧软件限位值)与 Pr7.12(负侧软件限位值)设定 0-两侧软件限位有效 1-正侧软件限位无效、负有效 2-正侧软件限位有效、负无效 3-两侧软件限位无效 由于本设定值而无效的限位信号(PSL/NSL), RTEX 通信状态为 0, 原点复位未完成时也为 0
7	11	A	正向软件限位值	-1073741823~1073741823	指令单位	设定正方向以及负方向的软件限位超过限制时, RTEX 通信的状态 PSL/NSL 会 ON (=1) 必须正侧软件限位值>负侧软件限位值
7	12	A	负向软件限位值	-1073741823~1073741823	指令单位	
7	20	R	RTEX 通讯周期设定	-1~12	—	设定 RTEX 的通讯周期 -1: 将 Pr7.91 的设定设为有效 3: 0.5ms 6: 1.0ms
7	21	R	RTEX 指令更新周期比设定	1~2	—	设定 RTEX 通信的通信周期和指令更新周期的比 设定值=指令更新周期/通信周期 1: 1 倍 2: 2 倍
7	22	R	RTEX 功能扩展设定 1	-32768~32767	—	bit0 设定 RTEX 通信数据的大小 0: 16 字节模式 1: 32 字节模式 bit1 设定使用了 TMG_CNT 多个轴的同步模式, 未使用 TMG_CNT 时请设为 0 0: 轴间半同步模式 (部分非同步) 1: 轴间完全同步模式 bit4 半闭环控制时外部位移传感器位置信息监视器功能设定 0: 无效 1: 有效 (全闭环控制时跟此 bit 的设定无关, 可以监测外部位移传感器的位置信息)
7	91	R	RTEX 通信周期扩展设定	0~2000000	ns	设定 Pr7.20=1 时的 RTEX 通信的通信周期只可设定 62500、125000、250000、500000、1000000、2000000 这几个参数值, 否则会发生 Err93.5 “参数设定异常保护 4”

A: 一直有效。

B: 禁止电机动作中及指令发出中变更参数。

C: 在控制电源复位、RTEX 通信的复位指令的软件复位模式或属性 C 参数有效化模式执行后有效。

R: 控制电源重启后有效。

RTEX 的通信周期 (Pr7.20、Pr7.91) 和指令更新周期 (Pr7.21) 需要与上位装置的周期一致, 同时, RTEX 的扩展功能 (Pr7.22) 的设定也需要与上位装置一致, 设定若不同无法保证动作执行。

模式设定示例如下：通信周期 0.5ms，指令更新周期 1ms，半闭环控制，16 字节模式，轴间半同步模式的情况下的设置。

Pr0.01=0（半闭环控制）

Pr7.20=3（通信周期 0.5ms）

pr7.21=2（指令更新周期 1ms=0.5ms*2 倍）

pr7.22=0（16 字节模式、轴间半同步模式）

（在 Pr7.20 不等于-1 时，可以不设定 Pr7.91）

驱动器不动作原因排查：

序号	项目	描述
0	无原因	无法检出不旋转原因，通常是可旋转的状态
1	伺服未处于准备状态	驱动器的主电源未输入 报警发生 通信和伺服的同步未完成 重启指令下属性 C 参数有效化模式处理中等
2	伺服未使能	伺服 ON 指令未输入；指令的 Servo_On bit 为 0，EX_SON（外部伺服 ON 输入）进行了分配，信号为 OFF 等
3	驱动禁止输入有效	Pr5.05=0~1（驱动禁止时时序，立即停止除外）时 Pr5.04=0（驱动禁止输入有效），正方向驱动禁止输入（POT）为 ON 时，动作指令为正方向；负方向驱动禁止输入（NOT）为 ON，动作指令为负方向 Pr5.05=2（驱动禁止时时序：立即停止）时 Pr5.04=0（驱动禁止输入有效），与是否有动作指令输入无关，正方向驱动禁止输入（POT）或者负方向驱动禁止输入（NOT）为 ON 的状态下停止
4~5	转矩限制设定小	有效的转矩限制设定值，设定在额定的 5%以下
7	位置指令输入频率低	每个控制周期的位置指令为 1 指令单位以下
10	RTEX 通信的指令速度小	来自 RTEX 通信的指令速度设定为 30r/min 以下
11	厂家使用	—
12	RTEX 通信的指令转矩小	来自 RTEX 通信的指令转矩减小到额定转矩的 5%以下
13	速度限制小	Pr3.17=0 时，Pr3.21 速度限制值设定在 30r/min 以下 Pr3.17=1 时，指令的 SL_SW bit 指定的参数(Pr3.21 或者 Pr3.22)的速度限制值设定 30r/min 以下
14	其他原因	不是主要原因 1~13 的任一个，电机不运转（指令小、负载重、锁定、碰撞、驱动器、电机的故障等）